xtrackm

Source Code & Notes

367 files across 21 directories
Theme: tango

Table of Contents

. 10
Issues.txt 11
Notes.txt 12
TODO.txt 13
code2pdf.py 14
learning_python.py 19
prompts.txt 20
selected_topo_cmap.txt 22
vijay_ppt_TODO.txt 23
workflow.txt 24

./computed 25
computed/Issues.txt 26
computed/Notes.txt 27
computed/TODO.txt 28
computed/c.box_12_smooth.all.py 29
computed/c.cb.nearby_tracks_all.py 30
computed/c.coastal.nearby_tracks_all.py 31
computed/c.common_passes.l2pVSxtrack.py 32
computed/c.common_passes.l2pVSxtrack_selected.py 33
computed/c.gauss_smooth.all.py 34
computed/c.loess_0.05_smooth.all.py 35
computed/c.loess_0.2_smooth.all.py 36
computed/c.msla_uvVSguv_TP+J1+J2+J3+S6A.py 37
computed/c.msla_uvVSguv_TP+J1+J2+J3+S6A_reindex.py 39
computed/c.odo_smooth.and.guv.all.py 41
computed/c.odo_smooth.one.py 43
computed/c.odo_smooth.one_xarray.py 45
computed/c.optim_smooth.all.py 47
computed/current_working.txt 48
computed/f.combined_map_std_valid.S3A.py 49
computed/f.xtr.S3A.common_tracks.map.py 51
computed/odo_filter.py 52
computed/p.compare.gc.std_mean.l2pVSxtrack_map.py 53
computed/p.compare.std_mean.l2pVSxtrack.py 54
computed/p.compare.std_mean.l2pVSxtrack_map.py 55
computed/p.compare.std_meanANDvalid_count.mean.l2pVSxtrack_map.py 56
computed/p.compare.valid_points_mean.l2pVSxtrack_map.py 58
computed/p.msla_uv.guv.metrics.general.py 59
computed/p.std.S3A.common_tracks.with_mean.py 61
computed/p.std.S3A.common_tracks.with_mean_simple.py 63
computed/p.std.S3A.east_coast.py 64
computed/p.std.S3Avsl2p.common_tracks.with_mean.py 65
computed/p.std.S3Avsl2p.selected.pre_c.py 67
computed/p.std.gc.S3A.common_tracks.with_mean.py 68
computed/p.std.l2p.S3A.common_tracks.with_mean.py 70
computed/p.tracks.S3A.obtuse.py 71
computed/p.valid_points.S3A.selected.pre_c.py 72
computed/p.valid_points.S3Avsl2p.common_tracks.with_mean.py 73
computed/p.valid_points.S3Avsl2p.selected.pre_c.py 75
computed/p.valid_points.gshhg.S3A.py 76
computed/p.valid_points.gshhg.S3A.smoothVSraw.py 78
computed/p.valid_points.gshhg.S3A_simple.py 80
computed/prompts.txt 81
computed/workflow.txt 82

./computed/archive 83

computed/archive/extract_oscar_along_tracks_py.py 84

./computed/odo 85
computed/odo/Notes.txt 86
computed/odo/c.odo_smooth.one_unwrap.py 87
computed/odo/plot_odo_window_shapes.py 89
computed/odo/weights.py 90

./data 91
data/download_xtrack.py 92
data/link_raw_nonzero_files_orderbylon.py 93
data/link_raw_nonzero_files_orderbylon1.py 94
data/workflow.txt 95

./draft_figs02 96
draft_figs02/c.rossby_radius_L_R.py 97
draft_figs02/commands.txt 98
draft_figs02/fig01_f.combined_map_std_valid.S3A.py 99
draft_figs02/fig02_p.omni_uvVStrack_guv_c.ek.nearest.selected.py 101
draft_figs02/fig03_p.scatter_omniVSrama_uVSgu_ww_ek.py 105
draft_figs02/fig04_p.scatter_drifter_ve_vs_gu_ww_ek.py 107
draft_figs02/fig05_p.radar_uvVStrack_guv.at.intersections_single.py ... 109
draft_figs02/fig06_p.along_shore_at_155_116.intersection.S6A.py 112
draft_figs02/fig07_p.speed_at_coastal_points.eicc.trends.all_in_one.ERS.py 114
draft_figs02/make_figure_docx_new.py 117
draft_figs02/organise_scripts_pngs.py 118
draft_figs02/p.aviso_only_uvVStrack_guv.at.intersections_single.py 119
draft_figs02/p.gc.clim.cross_structure_x_arrows_map_ERS.py 122
draft_figs02/p.gc.clim.lat_wise.along_xwidth.py 124
draft_figs02/p.metrics.track_guvVSoscar_ww_ekman.py 126
draft_figs02/p.vskp.tseries_new_v2.2.py 127
draft_figs02/prompts.txt 130
draft_figs02/run_all.py 131
draft_figs02/sup01_satellite_mission_time_spans_new.py 132
draft_figs02/touch_captions_files.py 133
draft_figs02/workflow.txt 134

./draft_figs02/archive 135
draft_figs02/archive/fig01.py 136
draft_figs02/archive/fig01_p.compare.std_mean.l2pVSxtrack_map.py 137
draft_figs02/archive/fig01b.py 139
draft_figs02/archive/fig02.py 140
draft_figs02/archive/fig03.py 143
draft_figs02/archive/fig03a.py 145
draft_figs02/archive/fig04.py 148
draft_figs02/archive/fig04_old.py 150
draft_figs02/archive/fig05.py 151
draft_figs02/archive/fig05_p.radar_uvVStrack_guv.at.intersections_single.py 154
draft_figs02/archive/fig06.py 157
draft_figs02/archive/fig06_p.gc.clim.cross_structure_x_arrows_map_ERS.py 159
draft_figs02/archive/fig06_p.gc.clim.cross_structure_x_arrows_map_ERS_pre_gc.py 161
draft_figs02/archive/fig07.py 162
draft_figs02/archive/fig08.py 164
draft_figs02/archive/fig09.py 166
draft_figs02/archive/link_figs_png.py 168
draft_figs02/archive/make_figure_docx.py 169
draft_figs02/archive/p.along_shore_at_155_116.intersection.S6A.py 170
draft_figs02/archive/p.aviso_only_uvVStrack_guv.at.intersections_single.py 172
draft_figs02/archive/p.compare.std_mean.l2pVSxtrack_map.py 175
draft_figs02/archive/p.gc.clim.cross_structure_x_arrows_map_ERS.py 176
draft_figs02/archive/p.gc.clim.lat_wise.along_xwidth.py 178
draft_figs02/archive/p.metrics.track_guvVSoscar_ww_ekman.py 180

draft_figs02/archive/p.omni_uvVStrack_guv_c.ek.nearest.selected.py 181
draft_figs02/archive/p.radar_uvVStrack_guv.at.intersections_single.py .. 184
draft_figs02/archive/p.scatter.track_guVSdrifter_u_ww_ek.py 187
draft_figs02/archive/p.scatter_drifter_ve_vs_gu_ww_ek.py 189
draft_figs02/archive/p.scatter_omniVSrama_uVSgu_ww_ek.py 191
draft_figs02/archive/p.vskp.tseries_new_v2.2.py 193
draft_figs02/archive/rename_figs_py.py 196

./draft_figs02/captions 197
draft_figs02/captions/f.combined_map_std_valid.S3A.txt 198
draft_figs02/captions/p.along_shore_at_155_116.intersection.S6A.txt ... 199
draft_figs02/captions/p.compare.std_mean.l2pVSxtrack_map.txt 200
draft_figs02/captions/p.gc.clim.cross_structure_x_arrows_map_ERS.txt .. 201
draft_figs02/captions/p.gc.clim.lat_wise.along_xwidth.txt 202
draft_figs02/captions/p.omni_uvVStrack_guv_c.ek.nearest.selected.txt .. 203
draft_figs02/captions/p.radar_uvVStrack_guv.at.intersections_single.txt 204
draft_figs02/captions/p.scatter_drifter_ve_vs_gu_ww_ek.txt 205
draft_figs02/captions/p.scatter_omniVSrama_uVSgu_ww_ek.txt 206
draft_figs02/captions/p.speed_at_coastal_points.eicc.trends.all_in_one.ERS.txt 207

./drifter 208
drifter/Notes.txt 209
drifter/TODO.txt 210
drifter/c.close.drifters_at_intersection_point.py 211
drifter/c.close.drifters_at_intersection_point_efficient1.py 214
drifter/c.close.drifters_at_intersection_point_noplotting.py 216
drifter/c.is_drifter_within_any.py 218
drifter/c.metrics.track_guvVSnearby_drifters_at_intersection_points.py 220
drifter/c.metrics.track_guvVSnearby_drifters_at_intersection_points_v2.py 222
drifter/c.metrics.track_guvVSnearby_drifters_at_intersection_points_w_ekman.py 225
drifter/c.pool.drifterVStrack.gu_ek.allsat.within.isobath_rotate.py ... 228
drifter/debug1.py 231
drifter/drifter_meta_data.txt 233
drifter/l.check_chunk_sat_time_overlap.py 236
drifter/l.min_max_times.all.drifters.py 237
drifter/l.one_drifter_file.details.py 238
drifter/l.types_of_time.drifterVStrack.py 239
drifter/p.all.drifter_pathsANDintersection_points_on_one_map.py 240
drifter/p.drifters_passing_intersection_point_box.all.py 242
drifter/p.drifters_passing_intersection_point_box.all_simple.py 245
drifter/p.drifters_passing_intersection_point_box.one.py 248
drifter/p.scatter.track_guVSdrifter_u.py 251
drifter/p.scatter.track_guVSdrifter_u_2.py 252
drifter/p.scatter.track_guVSdrifter_u_ek.py 253
drifter/p.scatter.track_guVSdrifter_u_near_coast.py 254
drifter/prompts.txt 256
drifter/workflow.txt 257

./drifter_track_reg 258
drifter_track_reg/Plan.txt 259
drifter_track_reg/c.candidate_buoys.for.intersection_points.py 260
drifter_track_reg/c.extract.drifter_veANDgu_at_intersections.py 261
drifter_track_reg/c.extract.drifter_veANDgu_ek_at_intersections.py 263
drifter_track_reg/d.plot_candidate_drifters.for.intersections.py 265
drifter_track_reg/l.check_chunk_sat_time_overlap.py 267
drifter_track_reg/l.min_max_times.all.drifters.py 268
drifter_track_reg/p.scatter_drifter_ve_vs_gu.py 269
drifter_track_reg/p.scatter_drifter_ve_vs_gu_ek.py 271
drifter_track_reg/p.scatter_drifter_ve_vs_gu_ww_ek.py 273
drifter_track_reg/prompts.txt 275
drifter_track_reg/temporal_extents.of.chunks.sats.py 277
drifter_track_reg/workflow.txt 280

./eicc 281
eicc/Issues.txt 282
eicc/TODO.txt 283
eicc/c.eicc_path.as_transect.py 284
eicc/c.isobath_based_slopes.one_func.py 286
eicc/c.isobath_based_slopes.py 289
eicc/caption_speed_timeseries_allinone_ERS1+ERS2+ENV+SRL.txt 291
eicc/isobath_heading_at_point.py 292
eicc/p.alac_at_coastal_points.eicc.all_in_one.ERS.clim.py 294
eicc/p.alac_at_coastal_points.eicc.mimizing_angles.py 297
eicc/p.along_shore_at_155_116.intersection.S6A_c1.py 300
eicc/p.along_shore_at_155_116.intersection.S6A_periodogram.py 302
eicc/p.along_shore_at_coastal_points.eicc.periodogram.py 305
eicc/p.along_shore_at_coastal_points.eicc.py 308
eicc/p.coastline.gaussian.tangent.simple.py 310
eicc/p.gc.clim.cross_structure_x_arrows_map_ERS.py 311
eicc/p.gc.clim.cross_structure_x_arrows_map_ERS_pre_c.py 313
eicc/p.gc.clim.lat_wise.along_xwidth.py 315
eicc/p.guv_at_155_116.intersection.S6A.py 316
eicc/p.guv_at_coastal_points.eicc.all_in_one.ERS.clim.py 318
eicc/p.hovmoller_u_near_coast_bob.py 320
eicc/p.intersection_points.S6A.py 322
eicc/p.smoothed_coastline.gaussian.py 323
eicc/p.smoothed_coastline.gaussian.tangent.py 324
eicc/p.smoothed_coastline.others.py 326
eicc/p.speed_at_coastal_points.eicc.trends.all_in_one.ERS.py 328
eicc/p.speed_at_coastal_points.eicc.trends.all_in_one.S6A.py 332
eicc/p.speed_at_coastal_points.eicc.trends.all_in_one.TPN.py 336
eicc/p.speed_at_coastal_points.eicc.trends.py 340
eicc/prompts.txt 343
eicc/workflow.txt 345

./eicc/archive 346
eicc/archive/p.gc.clim.cross_structure_x_arrows_map_ERS1+ERS2+ENV+SRL_new_try.py 347

./ekman 349
ekman/Notes.txt 350
ekman/c.drag_coefficient_ccmp3_mean.py 351
ekman/c.ekman_uv_ccmp31_clim.py 352
ekman/c.extract.ekman.at.intersections.py 353
ekman/c.extract.ekman.at.intersections_cd.py 354
ekman/cp.ekman_uv_ccmp31_clim.py 355
ekman/p.compare.u_E.ww.cd.py 357
ekman/p.u_E.monsoon_panel_clim.py 358
ekman/prompts.txt 359

./guv 360
guv/Issues.txt 361
guv/Notes.txt 362
guv/TODO.txt 363
guv/azimuths.txt 364
guv/c.extract.msla_uv.at.intersections.py 384
guv/c.extract.oscar.at.all.intersections.py 385
guv/c.gc_at_intersection_points_loess_0.05.py 386
guv/c.gc_at_intersection_points_loess_0.05_diag.py 387
guv/c.guv_at_intersection_points_box_12.py 388
guv/c.guv_at_intersection_points_gauss_3.py 389
guv/c.guv_at_intersection_points_loess_0.02.py 390
guv/c.guv_at_intersection_points_loess_0.05.py 391
guv/c.guv_at_intersection_points_reindex_0.05.py 392
guv/c.guv_at_intersection_points_xsmooth_0.05.py 393

guv/c.guv_c.sat_wise.stats.py 394
guv/c.local_angleVSend_angle.py 395
guv/c.odo_smooth.and.guv.all.py 396
guv/demo_pandas_broadcast_rows.py 398
guv/f.point_wise_metrics.avisoVStrack_u.py 399
guv/f.point_wise_metrics.avisoVStrack_v.py 400
guv/f.point_wise_metrics.oscarVStrack_u.py 401
guv/f.point_wise_metrics.oscarVStrack_v.py 402
guv/l.temporal_distance_intersection_lines.py 403
guv/p.all_drifters.with.interseting_tracks.py 404
guv/p.aviso_only_uvVStrack_guv.at.intersections.py 405
guv/p.aviso_only_uvVStrack_guv.at.intersections_single.py 408
guv/p.aviso_only_uvVStrack_guv_c.at.intersections.py 411
guv/p.cb_buoys_uvVStrack_guv.nearest.all.py 414
guv/p.cb_uvVStrack_guv_c.nearest.all.py 417
guv/p.coastal_buoys_uvVStrack_guv.nearest.all.py 420
guv/p.gu_histogram_allsat.py 423
guv/p.metrics.track_guvVSoscar_ww_ekman.py 424
guv/p.omni_uvVStrack_guv_c.ek.nearest.all.py 425
guv/p.omni_uvVStrack_guv_c.ek.nearest.selected.py 428
guv/p.omni_uvVStrack_guv_c.nearest.all.py 431
guv/p.omni_uvVStrack_guv_c.nearest.selected.py 434
guv/p.omni_uvVStrack_sla.all.py 437
guv/p.omni_uvVStrack_sla.one.py 439
guv/p.oscar_ekmn_uvVStrack_guv.at.intersections.py 441
guv/p.oscar_only_uvVStrack_guv.at.intersections.py 444
guv/p.oscar_only_uvVStrack_guv.one.py 447
guv/p.oscar_only_uvVStrack_guv.one_simple.py 450
guv/p.plot_std_oscar_track_u_coastal.py 452
guv/p.rama_uvVStrack_guv_c.ek.nearest.all.py 453
guv/p.rama_uvVStrack_guv_c.ek.nearest.selected.py 456
guv/p.rama_uvVStrack_guv_c.nearest.all.py 459
guv/p.rama_uvVStrack_guv_c.nearest.selected.py 462
guv/p.rama_uvVStrack_guv_c.nearest_score.all.py 465
guv/p.scatter_omniVSrama_uVSgu_ww_ek.py 468
guv/p.scatter_omni_u_vs_gu.py 470
guv/p.scatter_omni_u_vs_gu_ek.py 471
guv/p.scatter_rama_u_vs_gu.py 472
guv/p.scatter_rama_u_vs_gu_ek.py 473
guv/p.std.at.intersection_points.oscarVStrack.py 474
guv/prompts.txt 476
guv/tools_drifter.py 477
guv/workflow.txt 479

./guv/archive 480

guv/archive/c.guv.sat_wise.stats.py 481
guv/archive/c.guv_at_intersection_points_0.05_single.py 482
guv/archive/c.guv_at_intersection_points_0.2.py 483
guv/archive/p.omni_uvVStrack_guv.nearest.all.py 484
guv/archive/p.omni_uvVStrack_guv.nearest.selected.py 487
guv/archive/p.omni_uvVStrack_guv.one.py 494
guv/archive/p.rama_uvVStrack_guv.nearest.all.py 497
guv/archive/p.rama_uvVStrack_sla.one.py 500

./lib 502

lib/dump.py 503
lib/namesm.py 505
lib/tools_shapely.py 510
lib/tools_xtrackm.py 512

./meta 532

meta/Notes.txt 533

meta/TODO.txt 534
meta/c.east_coast_track_intersections_xtrack.py 535
meta/c.intersection_points.all.py 536
meta/c.intersection_points.all_testing.py 537
meta/c.intersection_points.angles_map.py 538
meta/c.intersection_points.py 540
meta/c.outlier_detection1.py 541
meta/c.outlier_detection_all.py 542
meta/c.outlier_detection_all_beta.py 543
meta/c.valid_points.count.gshhs.loess.fits.py 545
meta/c.valid_points.count.gshhs.loess.fits_6km.py 546
meta/commands_htop.txt 547
meta/currently_working.txt 548
meta/data_coverage_basic.py 549
meta/data_coverage_df.py 550
meta/f.sla.TPN+J1N+J2N+J3N_all.py 551
meta/info.txt 552
meta/l.avrg.track_lens.py 553
meta/l.closest_descending_track_in_time.py 554
meta/l.fraction_of_distance_loess.py 555
meta/l.inter_track.dists_eq.py 556
meta/l.lat_index_at_equator.py 557
meta/l.lat_lon.dists2.all_sat.chunks.py 558
meta/l.lat_lon.dists2.all_sat.py 559
meta/l.lat_lon.dists2.chunks.py 560
meta/l.lat_lon.dists2.netCDF+prll.py 561
meta/l.lat_lon.dists2.netCDF+prll_chunks.py 562
meta/l.lat_lon.dists2.one_sat.chunks.py 563
meta/l.lat_lon.dists2.one_sat.py 564
meta/l.lat_lon.dists2.prll1.py 565
meta/l.lat_lon.dists2.prll2.py 566
meta/l.lat_lon.dists2.py 567
meta/l.lat_lon.max_min.netCDF4.py 568
meta/l.lat_lon.max_min.py 569
meta/l.lat_time_diff.py 570
meta/l.one_file.details.py 571
meta/l.one_file.test_asn.py 572
meta/l.reading_speeds.xrVSnetCDF4.py 573
meta/l.rossby_radius_of_deformation.py 574
meta/l.tan.tan2.angles.py 575
meta/l.timestamps_reindex.py 576
meta/l.zero_len.tracks.py 579
meta/meta_data.txt 580
meta/outliers_all.txt 582
meta/p.intersection_points.allsat.bob.py 605
meta/p.intersection_points.allsat.coast.py 606
meta/p.intersection_points.allsat.py 607
meta/p.only_tracks.with_track_numbers.ascending.py 608
meta/p.tracks.S3A.obtuse.py 610
meta/p.tracks.with.all_obs_locs_allsat_with_track_numbers.negative.py .. 611
meta/p.valid_points.count.gshhg.loess.4by2.py 613
meta/p.valid_points.count.gshhg.mean.4by2.py 614
meta/p.valid_points.count.gshhg.mean.S3A.py 615
meta/p.valid_points.count.gshhg_6km.loess.4by2.py 616
meta/p.valid_points.count.x.mean.4by2.py 617
meta/p.variable.histograms.py 618
meta/print_mission_names.py 619
meta/prompts.txt 620
meta/satellite_mission_time_spans_new.py 621
meta/selected_topo_maps.txt 622

./radar 623
radar/Notes.txt 624
radar/TODO.txt 625
radar/c.shp.polygon_around_AP.convex_hull.py 626
radar/c.shp.polygon_around_OR.convex_hull.py 627
radar/c.shp.polygon_around_TN.convex_hull.py 628
radar/f.ngp.APCodar_daily.py 629
radar/f.ngp.ORCodar_daily.py 630
radar/f.ngp.TNCodar_daily.py 631
radar/p.radar_uvVStrack_guv_c.at.intersections.py 632
radar/p.radar_uvVStrack_guv_c.ek.pool.at.intersections.py 635
radar/p.radar_uvVStrack_guv_c.pool.at.intersections.py 638
radar/p.rama_uvVStrack_guv_c.ek.nearest.all.py 641
radar/p.scatter_radar_uVSgu.py 644
radar/p.scatter_radar_uVSgu_ek.py 645
radar/prompts.txt 646
./radar/archive 647
radar/archive/p.radar_uvVStrack_guv.at.intersections.py 648
./schema 651
schema/p.schematic.coastal_slope.py 652
schema/p.schematic.matplotlib.lines.acute.py 653
schema/p.schematic.matplotlib.lines.g1g2.py 655
schema/p.schematic.matplotlib.lines.obtuse.py 657
schema/p.schematic.track_angles.py 659
schema/p.schematic.track_angles2.py 661
schema/prompts.txt 663
./selected 664
selected/f.ngp.APCodar_daily.py 665
selected/f.ngp.ORCodar_daily.py 666
selected/f.ngp.TNCodar_daily.py 667
selected/p.metrics.track_guvVSoscar_ww_ekman.py 668
selected/p.radar_uvVStrack_guv.at.intersections.py 669
selected/p.scatter.track_guVSdrifter_u_2.py 673
selected/p.scatter.track_guVSdrifter_u_ek.py 674
selected/p.scatter.track_guVSdrifter_u_ww_ek.py 675
selected/p.vskp.tseries_new_v2.2.py 677
selected/prompts.txt 680
selected/satellite_mission_time_spans_new.py 681

```
./
```

```
9 file(s)
```

Issues.txt

```
1
2  gc_other_at = gc_other_at.interp(time=gc_self_at.time)
3  in the above line should we shift the time axis of gc_other_at to
4  match the time axis of gc_self_at
5  Or interp is right?
6
7  In fig03.py, ideally there should be perfect match between our gu, gv with
8  aviso msla_uv. Because both are just geostrophic.
9
10 [x] meridional component sign issue
11 [x] medidional component in correct resolution
12
13 Minor:
14 While precomputing guv and comparing with omni, there is time mismatch issue
15
```

Notes.txt

```
1
2  correction - model dry tropospheric correction - wet tropospheric correction
3  - sea state bias correction - solid earth tide height - geocentric ocean tide
4  - geocentric pole tide height - Dynamic atmospheric correction - X-TRACK mean
5  sea surface - Global Mean Sea level Bias
6
7  old important Notes
8  ------
9
```

```
1
2  [] percentage valid points after smoothing near coast vs without smoothing
3  for ERS
4  [] guv latVStime plot
5
6  [✔] decadal eicc strength using TPS6A mission at intersection points
7  [✔] open ocean comparisons; atleast a scatter plot with few comparisons
8  although timeseries plots are not feasible.
9  [✔] standard deviation of aviso and track near the coast
10 [✔] we need to gain as much clarity on ekman currents as we did on guv
11 components
12
13 possbile plots:
14
15 ------ can wait -------
16 [] drifter comparison only withing 200km or 50km
17
18 [✔] Derive the formula for velocity components from the cross track geostrophic currents by thoroughly reading the papers
19 parke_1987_orbits_current_components.pdf and
20 rosemary_morrow_1994_velocity_variance_geosat_altimetry.pdf
21 and with help of AI apps.
22
```

```python
#!/usr/bin/env python3
"""
code2pdf.py — Gather all .py and .txt files and produce one combined,
syntax-highlighted PDF with a clickable Table of Contents.

Usage:
    python code2pdf.py                              # current dir, recursive
    python code2pdf.py -o xtrackm_code.pdf          # custom output name
    python code2pdf.py --dir computed/ guv/ lib/     # specific directories
    python code2pdf.py --theme github-dark --font-size 8
    python code2pdf.py --exclude archive/ --exclude draft_figs02/

The script walks the directory tree, collects every .py and .txt file,
groups them by directory, and writes them into a single PDF with:
  • Table of Contents (page numbers for each file)
  • Directory separator pages
  • Syntax colouring (Pygments) for .py files
  • Line numbers, dark background, filename headers, page numbers
"""
# copilot --resume=ae78b187-8512-43ec-af6b-5225001dbe76
import argparse
import os
import sys
import time
from collections import defaultdict
from pathlib import Path

from fpdf import FPDF
from fpdf.enums import XPos, YPos
from pygments import highlight
from pygments.lexers import PythonLexer, TextLexer, get_lexer_for_filename
from pygments.formatter import Formatter
from pygments.styles import get_style_by_name, get_all_styles
from pygments.token import Token


# ── Helpers ──────────────────────────────────────────────────────────────────

def hex_to_rgb(hex_color: str):
    """Convert '#rrggbb' to (r, g, b) ints."""
    h = hex_color.lstrip("#")
    return tuple(int(h[i : i + 2], 16) for i in (0, 2, 4))


def choose_lexer(filepath: str):
    """Return an appropriate Pygments lexer for *filepath*."""
    try:
        return get_lexer_for_filename(filepath)
    except Exception:
        if filepath.endswith(".py"):
            return PythonLexer()
        return TextLexer()


def collect_files(roots: list[str], exclude: list[str] | None = None) -> list[str]:
    """Recursively collect .py and .txt files under *roots*, skipping *exclude* dirs."""
    extensions = {".py", ".txt"}
    exclude = set(os.path.normpath(e) for e in (exclude or []))
    files = []
    for root in roots:
        p = Path(root)
        if p.is_file() and p.suffix in extensions:
            files.append(str(p))
            continue
        for dirpath, dirnames, filenames in os.walk(p):
            # Prune excluded and hidden directories
            rel = os.path.relpath(dirpath, ".")
            dirnames[:] = [
                d for d in dirnames
                if not d.startswith(".")
                and not d.startswith("__")
                and os.path.normpath(os.path.join(rel, d)) not in exclude
            ]
            dirnames.sort()
            for fn in sorted(filenames):
                if Path(fn).suffix in extensions:
                    files.append(os.path.join(dirpath, fn))
    return files


def group_by_directory(files: list[str]) -> dict[str, list[str]]:
    """Group file paths by their parent directory (relative)."""
    groups: dict[str, list[str]] = defaultdict(list)
    for f in files:
        d = os.path.dirname(f) or "."
        groups[d].append(f)
    return dict(groups)


# ── Custom Pygments → fpdf2 formatter ────────────────────────────────────────

class _TokenAccumulator(Formatter):
    """Collect (token_type, value) pairs from Pygments."""

    def __init__(self, **options):
        super().__init__(**options)
        self.tokens: list = []

    def format(self, tokensource, outfile):
        for ttype, value in tokensource:
            self.tokens.append((ttype, value))


# ── PDF builder ──────────────────────────────────────────────────────────────

class CodePDF(FPDF):
    """FPDF subclass that renders syntax-coloured source code."""

    MONO_FONT = "DejaVu"  # alias we register

    def __init__(self, doc_title: str, style_name: str = "monokai",
                 font_size: int = 9):
        super().__init__(orientation="P", unit="mm", format="A4")
        self.set_auto_page_break(auto=True, margin=15)
        self._doc_title = doc_title
        self._file_title = ""          # current file being rendered
        self._style_name = style_name
```

```
        self._font_size = font_size

        self._style = get_style_by_name(style_name)
        bg = self._style.background_color or "#272822"
        self._bg_rgb = hex_to_rgb(bg)

        self._color_cache: dict = {}

        # TOC data: list of (level, label, page_number)
        self._toc_entries: list[tuple[int, str, int]] = []

        # Register a Unicode-capable monospace TTF font
        self._register_mono_font()

    # ── header / footer ──────────────────────────────────────────

    def _register_mono_font(self):
        """Register DejaVu Sans Mono (Unicode TTF) with all 4 style variants."""
        base = "/usr/share/fonts/truetype/dejavu"
        self.add_font(self.MONO_FONT, "",  f"{base}/DejaVuSansMono.ttf")
        self.add_font(self.MONO_FONT, "B", f"{base}/DejaVuSansMono-Bold.ttf")
        self.add_font(self.MONO_FONT, "I", f"{base}/DejaVuSansMono-Oblique.ttf")
        self.add_font(self.MONO_FONT, "BI", f"{base}/DejaVuSansMono-BoldOblique.ttf")

    def header(self):
        if not self._file_title:
            return
        self.set_font(self.MONO_FONT, "B", 10)
        self.set_text_color(180, 180, 180)
        self.cell(0, 6, self._file_title,
                  new_x=XPos.LMARGIN, new_y=YPos.NEXT, align="L")
        self.set_draw_color(100, 100, 100)
        self.line(self.l_margin, self.get_y(), self.w - self.r_margin, self.get_y())
        self.ln(2)

    def footer(self):
        self.set_y(-12)
        self.set_font(self.MONO_FONT, "I", 7)
        self.set_text_color(140, 140, 140)
        self.cell(0, 10, f"Page {self.page_no()}/{{nb}}", align="C")

    # ── TOC helpers ──────────────────────────────────────────────

    def add_toc_entry(self, level: int, label: str):
        self._toc_entries.append((level, label, self.page_no()))

    def write_toc_pages(self):
        """Insert TOC pages at the beginning (call after all content is added).

       Since fpdf2 doesn't support inserting pages, we build the TOC as the
       first content, then render files.  So we use a two-pass approach:
       pass 1 gathers page numbers, pass 2 builds the final PDF.
       We handle this externally — see build_combined_pdf().
       """
        pass  # handled externally

    # ── directory separator ──────────────────────────────────────

    def add_directory_page(self, dirname: str, file_count: int):
        """Full-page divider for a new directory section."""
        self._file_title = ""  # suppress header
        self.add_page()
        self.add_toc_entry(0, dirname)

        # Centered directory name
        self.ln(60)
        self.set_font(self.MONO_FONT, "B", 22)
        self.set_text_color(*self._bg_rgb)
        # Dark box behind text
        self.set_fill_color(*self._bg_rgb)
        box_y = self.get_y() - 4
        self.rect(self.l_margin, box_y,
                  self.w - self.l_margin - self.r_margin, 20, "F")
        self.set_text_color(255, 200, 60)
        self.cell(0, 12, f"  {dirname}/", new_x=XPos.LMARGIN, new_y=YPos.NEXT)

        self.ln(6)
        self.set_font(self.MONO_FONT, "", 12)
        self.set_text_color(160, 160, 160)
        self.cell(0, 8, f"  {file_count} file(s)",
                  new_x=XPos.LMARGIN, new_y=YPos.NEXT)

    # ── colour resolution ────────────────────────────────────────

    def _get_color(self, ttype):
        if ttype in self._color_cache:
            return self._color_cache[ttype]
        t = ttype
        while t:
            se = self._style.style_for_token(t)
            c = se.get("color")
            bold = se.get("bold", False)
            italic = se.get("italic", False)
            if c:
                rgb = hex_to_rgb(f"#{c}")
                self._color_cache[ttype] = (rgb, bold, italic)
                return (rgb, bold, italic)
            t = t.parent
        default = self._style.style_for_token(Token)
        fg = default.get("color") or "f8f8f2"
        rgb = hex_to_rgb(f"#{fg}")
        self._color_cache[ttype] = (rgb, False, False)
        return (rgb, False, False)

    # ── code rendering ───────────────────────────────────────────

    def render_file(self, filepath: str):
        """Read, tokenize, and render a single file into the PDF."""
        source = Path(filepath).read_text(errors="replace")
        if not source.strip():
            return

        rel = os.path.relpath(filepath, ".")
        self._file_title = rel
        self.add_page()
        self.add_toc_entry(1, rel)

```

```
        lexer = choose_lexer(filepath)
        formatter = _TokenAccumulator()
        highlight(source, lexer, formatter)

        line_h = self._font_size * 0.45
        # Measure one space to get the true monospace glyph width from the font
        self.set_font(self.MONO_FONT, "", self._font_size)
        char_w = self.get_string_width(" ")
        tab_w = char_w * 4
        right_limit = self.w - self.r_margin

        self._draw_bg()
        x_start = self.l_margin + 12
        x = x_start
        line_no = 1
        self._draw_line_number(line_no, line_h)

        for ttype, value in formatter.tokens:
            (r, g, b), bold, italic = self._get_color(ttype)
            st = ("B" if bold else "") + ("I" if italic else "")
            self.set_font(self.MONO_FONT, st, self._font_size)
            self.set_text_color(r, g, b)

            # Split token on newlines, render each segment as a whole string
            parts = value.split("\n")
            for pi, part in enumerate(parts):
                # Handle newline between parts (all but the first piece)
                if pi > 0:
                    line_no += 1
                    x = x_start
                    self.ln(line_h)
                    if self.get_y() > self.h - 18:
                        self.add_page()
                        self._draw_bg()
                        x = x_start
                    self._draw_line_number(line_no, line_h)

                if not part:
                    continue

                # Expand tabs to spaces for correct width measurement
                part = part.replace("\t", "    ")

                # Soft-wrap long lines
                while part:
                    avail = right_limit - x
                    seg_w = self.get_string_width(part)

                    if seg_w <= avail:
                        # Fits on the current line — write it
                        self.set_xy(x, self.get_y())
                        self.cell(seg_w, line_h, part)
                        x += seg_w
                        break
                    else:
                        # Find how many chars fit in the available space
                        fit = int(avail / char_w) if char_w > 0 else len(part)
                        fit = max(fit, 1)
                        # Fine-tune: shrink if overflows
                        while fit > 1 and self.get_string_width(part[:fit]) > avail:
                            fit -= 1

                        chunk = part[:fit]
                        part = part[fit:]

                        self.set_xy(x, self.get_y())
                        self.cell(self.get_string_width(chunk), line_h, chunk)

                        # Wrap to next line (indented to show continuation)
                        x = x_start + tab_w
                        self.ln(line_h)
                        if self.get_y() > self.h - 18:
                            self.add_page()
                            self._draw_bg()
                            x = x_start + tab_w

    def _draw_bg(self):
        self.set_fill_color(*self._bg_rgb)
        self.rect(self.l_margin, self.get_y(),
                  self.w - self.l_margin - self.r_margin,
                  self.h - self.get_y() - 14, "F")

    def _draw_line_number(self, n: int, line_h: float):
        self.set_font(self.MONO_FONT, "", self._font_size - 1)
        self.set_text_color(100, 100, 100)
        num_str = str(n).rjust(4)
        num_w = self.get_string_width(num_str)
        self.set_x(self.l_margin + 10 - num_w)
        self.cell(num_w, line_h, num_str)


# ── Two-pass PDF builder ──────────────────────────────────────────────

def build_combined_pdf(files: list[str], output: str,
                       style: str = "monokai", font_size: int = 9):
    """
    Two-pass build:
      Pass 1 — render all files, collect TOC entries with page numbers.
      Pass 2 — build final PDF with TOC at front + all content.
    """

    groups = group_by_directory(files)
    dir_order = sorted(groups.keys())

    # ── Pass 1: render content, gather TOC data ──────────────────────
    print("  Pass 1/2: rendering content & gathering page numbers ...")
    t0 = time.time()

    content_pdf = CodePDF(doc_title="pass1", style_name=style, font_size=font_size)
    content_pdf.alias_nb_pages()

    for d in dir_order:
        flist = groups[d]
        content_pdf.add_directory_page(d, len(flist))
        for f in flist:
            print(f"    {f}")
            content_pdf.render_file(f)
```

```
    toc_entries = list(content_pdf._toc_entries)
    content_bytes = content_pdf.output()
    t1 = time.time()
    print(f"  Pass 1 done — {content_pdf.page_no()} pages in {t1-t0:.1f}s")

    # ── Estimate how many pages the TOC itself will take ──────────────
    lines_per_toc_page = 50
    toc_page_count = max(1, (len(toc_entries) + lines_per_toc_page - 1)
                         // lines_per_toc_page)

    # ── Pass 2: build final PDF = cover + TOC + content ───────────────
    print("  Pass 2/2: building final PDF with Table of Contents ...")

    final = CodePDF(doc_title=output, style_name=style, font_size=font_size)
    final.alias_nb_pages()

    # ── Cover page ────────────────────────────────────────────────────
    final._file_title = ""
    final.add_page()
    final.ln(50)
    final.set_font(final.MONO_FONT, "B", 28)
    final.set_text_color(255, 200, 60)
    final.cell(0, 14, "  xtrackm", new_x=XPos.LMARGIN, new_y=YPos.NEXT)
    final.ln(4)
    final.set_font(final.MONO_FONT, "", 14)
    final.set_text_color(180, 180, 180)
    final.cell(0, 8, "  Source Code & Notes",
               new_x=XPos.LMARGIN, new_y=YPos.NEXT)
    final.ln(8)
    final.set_font(final.MONO_FONT, "", 11)
    final.set_text_color(140, 140, 140)
    final.cell(0, 6, f"  {len(files)} files across {len(groups)} directories",
               new_x=XPos.LMARGIN, new_y=YPos.NEXT)
    final.cell(0, 6, f"  Theme: {style}",
               new_x=XPos.LMARGIN, new_y=YPos.NEXT)

    # ── Table of Contents ─────────────────────────────────────────────
    final._file_title = ""
    final.add_page()
    final.set_font(final.MONO_FONT, "B", 16)
    final.set_text_color(255, 200, 60)
    final.cell(0, 10, "Table of Contents",
               new_x=XPos.LMARGIN, new_y=YPos.NEXT)
    final.ln(4)

    # Page offset: cover(1) + toc pages + then content starts
    page_offset = 1 + toc_page_count

    for level, label, pg in toc_entries:
        adjusted_pg = pg + page_offset
        if level == 0:
            # Directory heading
            final.ln(2)
            final.set_font(final.MONO_FONT, "B", 10)
            final.set_text_color(255, 200, 60)
            indent = "  "
        else:
            final.set_font(final.MONO_FONT, "", 8)
            final.set_text_color(190, 190, 190)
            indent = "      "

        dots_w = 70 - len(label)
        dots = " " + "." * max(2, dots_w) + " " if dots_w > 0 else "  "
        line = f"{indent}{label}{dots}{adjusted_pg}"
        final.cell(0, 4.5, line, new_x=XPos.LMARGIN, new_y=YPos.NEXT)

        # New TOC page if needed
        if final.get_y() > final.h - 20:
            final.add_page()

    # ── Append content pages (re-render) ──────────────────────────────
    for d in dir_order:
        flist = groups[d]
        final.add_directory_page(d, len(flist))
        for f in flist:
            final.render_file(f)

    final.output(output)
    t2 = time.time()
    total_pages = final.page_no()
    size_mb = os.path.getsize(output) / (1024 * 1024)
    print(f"  Pass 2 done — {total_pages} pages, {size_mb:.1f} MB in {t2-t1:.1f}s")
    print(f"\n  ✓ {output}  ({total_pages} pages, {size_mb:.1f} MB)")


# ── CLI ──────────────────────────────────────────────────────────────

def main():
    parser = argparse.ArgumentParser(
        description="Gather all .py/.txt files and produce one combined, "
                    "syntax-highlighted PDF.",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="Available themes: " + ", ".join(sorted(get_all_styles())),
    )
    parser.add_argument("--dir", nargs="*", default=["."],
                        help="Root directories to scan (default: current dir)")
    parser.add_argument("-o", "--output", default="xtrackm_code.pdf",
                        help="Output PDF filename (default: xtrackm_code.pdf)")
    parser.add_argument("--exclude", action="append", default=[],
                        help="Directories to skip (repeatable)")
    parser.add_argument("--theme", default="monokai",
                        help="Pygments colour theme (default: monokai)")
    parser.add_argument("--font-size", type=int, default=9,
                        help="Font size in points (default: 9)")

    args = parser.parse_args()

    # Always exclude common non-source dirs
    exclude = args.exclude + ["archive", ".git", "__pycache__", ".ipynb_checkpoints"]

    files = collect_files(args.dir, exclude=exclude)
    if not files:
        print("No .py or .txt files found.", file=sys.stderr)
        sys.exit(1)

    groups = group_by_directory(files)
```

```
    print(f"Found {len(files)} files in {len(groups)} directories  "
          f"[theme: {args.theme}]\n")

    build_combined_pdf(files, args.output, style=args.theme,
                       font_size=args.font_size)
    print("\nDone.")


if __name__ == "__main__":
    main()
```

```
"""
We can load a shapefile using the cartopy.io.shapereader module instead of
downloading ne file.
"""
import cartopy.io.shapereader as shpreader
# Load Natural Earth 110m coastline
shpfilename = shpreader.natural_earth(
    resolution='50m',
    category='physical',
    name='coastline'
)

"""What is this"""
merged: pd.DataFrame | None = None
df[[key_col, metric]].rename(columns={metric: label})
merged = merged.merge(frame, on=key_col, how=join_mode)
from itertools import zip_longest
from numbers import Number
if df["track_self"].dtype == object:
    df["track_self"] = df["track_self"].astype(int)
lat_array = np.atleast_1d(latitude)
da = da.expand_dims(transect=[i])

def resolve_guv_file(guv_dir: Path, sat: str, track_self: int, track_other: int, method: str, frac: str) -> Path:
    t1 = str(int(track_self))
    t2 = str(int(track_other))
    candidates = [
        guv_dir / f"guv_at_intersection_{sat}_{t1}_{t2}_{method}_{frac}.nc",
        guv_dir / f"guv_at_intersection_{sat}_{t2}_{t1}_{method}_{frac}.nc",
    ]
    for p in candidates:
        if p.exists():
            return p

    raise FileNotFoundError(
        "No guv file found for tracks "
        f"({t1}, {t2}). Tried: " + ", ".join(str(x) for x in candidates)
    )

legend_handles = [
    Patch(facecolor="gold", edgecolor="none", alpha=0.25, label="Slope zone (200-2000 m)"),
    Line2D([0], [0], color="k", lw=1.2, label="200 m isobath"),
    Line2D([0], [0], color="darkorange", lw=1.0, label="1000 m isobath"),
    Line2D([0], [0], color="red", lw=1.0, label="2000 m isobath"),
]
ax1.legend(handles=legend_handles, loc="lower left", frameon=True, framealpha=0.85, fontsize=10)
return [tuple(xy) for xy in smooth_pts[::step]]
gs_interp = (
    pd.Series(gs_vals_raw).interpolate(limit_direction="both").values
)
x_scatter = np.broadcast_to(x_vals, std_all.shape).ravel()

```

```
1
2
3  u1, v1 are orthogonal vectors and u2, v2 are another set of othogonal vectors
4  sharing same origin. u1 and v1 are oriented along x-axis and y-axis
5  respectively. u2 makes an angle of theta with u1 clockwise. If u2, v2
6  frame is rotated counter clockwise by theta, then what are the new values of
7  vectors u2, v2. Give mathematical formulations and python function
8
9
10 Equation of a line passing through point P(x0, y0) and making an angle theta
11 with x-axis(counter clockwise). Call this line T1.
12 Equation of a line passing through P and perpedicular to T1. Call this Line P1.
13 Equation of a line passing through P and makes an acute angle alpha with T1 at
14 P. Call this Line N1.
15 Equation of line passing through P and parallel to y-axis
16 Equation of line passing through P and parallel to x-axis
17 write a python sympy program to solve the angle between x-axis and P1 and
18  to solve the acute angle between N1 and P1.
19
20 Write a python code that
21 1. Draw a parallel thin line to the x-axis of 10 units width and 5 units height
22 above x-axis.
23 2. Draw a line that traverse x-axis the parallel line, with an angle of
24 110 degrees (measured counter clock wise from x-axis).
25 3. Mark the point at which traversing line intersects the parallel line as P.
26 and mark the angle greek letter theta near the x-axis.
27 4. At the point of intersection draw a vertical arrow and horizontal arrow
28 pointing bothsides. Mark the vertical arrow as u bar (vector notation) and
29 horizontal arrow as v bar (vector notation).
30 5. At point p, draw a perpendicular line to traversing line as a thin dotted
31 arrow pointing bothsides.
32
33 Write a python code that
34 Draw y=2**x curve, draw a tangent to the curve at x=2.5 and y=2**x.
35 Mark the point of touching of tangent to the curve as P
36 At point P, draw a perpendicular line to tangent line as a thin dotted
37 arrow pointing bothsides.
38 Draw a line passing through the above point making an angle of
39 110 degrees (measured counter clock wise from x-axis). Mark this line as T1.
40 Mark the angle greek letter theta that T1 makes with the x-axis.
41 At the P, draw a vertical arrow and horizontal arrow
42 pointing bothsides. Mark the vertical arrow as u bar (vector notation) and
43 horizontal arrow as v bar (vector notation).
44
45 I am trying to compute cross-track geostrophic currents from along track
46 altimetry data which contains sea level anomaly(sla) and lon, lat data of the
47 ground track. For convenience, I have remapped the sla data as a function of
48 distance (s) from the coast. Now sla becomes a function s dimension. To compute
49 cross track geostrophic currents, I need to get the gradient of sla along
50 s which is d(sla)/ds and then multiply it with g/f where g is gravity as
51 function of lat and f is coriolis parameter as a function of lat. The satellite
52 ground track makes and angle of a with x-axis. At a given point on the tracks
53 del s = sqrt(del x ** 2 + del y ** 2). Now how do i compute zonal and
54 meridional components of currents from cross-track geostrophic currents. Give
55 mathematical formulation.
56
57 Write a python code that
58 Draw two lines one making an angle of 70 and the other making 110 with
59 x-axis(measured counter clock wise from x-axis). Both lines start at the
60 x-axis but from different points.
61 Mark the intersection of the lines as P.
62 At the point of intersection draw a vertical arrow and horizontal arrow
63 pointing bothsides. Mark the vertical arrow as v bar (vector notation) and
64 horizontal arrow as u bar (vector notation).
65 At point P, draw perpendicular lines to each of the two lines as a thin dotted
66 arrows extending both sides of the line. They have the same color as the
67 their perpedicular line.
68
69 There are two satellite altimetry tracks (T1, T2) that intersect at a point
70 P with an acute angle of theta. The cross track geostrophic current is computed
71 by differentiating the sea level anomaly (sla) along the track. The computed
72 geostrophic currents are normal to the track path. The geostrophic currents
73 computed at the point P are g1 and g2 for the tracks T1 and T2 respectively. g1
74 and g2 are normal to the tracks T1 and T2 at point P. Now how do we deduce the
75 zonal and meridional components of currents at point P from g1 and g2? Deduce
76 the relation mathematically.
77
78 If we have only one track T1 and geostrophic current at point P is g1,
79 can we not decude u and v as g1*sin(theta1) and g2*cos(theta1)?
80
81 There are two othogonal vectors g1, g2 originating at (0, 0). There is a third
82 vector r (originating from same point) inbetween g1 and g2 which makes an
83 angle of theta with g2. Now how do we express vector r with respect to g1 and
84 g2?
85
86 There are two orthogonal vectors u (along x-axis) and v(along y-axis).
87 If a vector r makes an angle of theta with x-axis, how do we express r
88 interms of u and v. Note that u and v are not unit vectors.
89
90 There are two vectors g1, g2 originating at (0, 0). g1 makes an angle of a
91 with x-axis counterclock wise and g2 makes an angle of b with x-axis clockwise
92 . So g1 and g2 are position on the either sides of x-axis. How is the resulting
93 vector along x-axis expressed in term of g1 and g2.
94
95 Write a python code with following requirements
96 I have a bunch of ascending and descending tracks each of which are defined
97 by two arrays of longitudes and latitudes contained in netcdf files.
98 The code starts as follows.
99 for f in sorted(
100     glob.glob(f"../data/ctoh.sla.ref.{sat}.nindian.*.nc")
101 ):
102     # print(f)
103     ds = xr.open_dataset(f, engine="h5netcdf", decode_times=False)
104     if len(ds.points_numbers) == 0:
105         continue
106     track_number = ds.pass_number
107     lons_track = ds.lon.values
108     lats_track = ds.lat.values
109     lon_equat = lons_track[0]
110     lat_equat = lats_track[0]
111     lon_coast = lons_track[-1]
112     lat_coast = lats_track[-1]
113 1. For each track define a shapely linestring, find the slope.
114 (assuming the track is linear and taking first and last points to find the
115  slope and angle it makes with with equator as atan(slope))
116 2. For each track, check intersection points with other tracks and the
```

```
117  acute angle between the tracks.
118  3. We have to keep saving the tracknumber of self, tracknumber of other track
119  , intersection point, acute angle between them
120
```

selected_topo_cmap.txt

```
1
2  cmaps twilight, twilight_shifted
3
```

vijay_ppt_TODO.txt

```
1
2  [] ekman corrected improvement plots for rama/omni/cb
3
4  [✓] drifter buoy scatter plots after ekman correction
5  [✓] extract ekman at all intersection points and make the
6  [✓] ekman corrected improvement plots for oscar
7  [✓] drifter buoy scatter plots for points very close to the coast
8  ✓ drifter .csv files have to save with lons_inter, lats_inter to filter to the points that are very close to the coast
9  [✓] some oscar versus track guv plots
10 ✓ stick with ekman computed with pickard for now
11
```

```
data
meta
schema
computed
guv

selected -- is usually last one

1. Downloading with data/download_xtrack.py
new version v2.2 data has a total of 8 satellite missions.
They have changed the path sligtly
These files are not lzma compressed, so no need to decompress
2. Lon wise ordered linking the data that also checks for zero size files
data/link_raw_nonzero_files_orderbylon.py
3. Basic meta data of all sats missions
meta/data_coverage_basic.py
4. data_coverage_df.py script does all meta data gathering in to a csv file, the following scripts functionally just parts of
meta/data_coverage_df.py script
l.lat_lon.dists2.py
l.lat_lon.max_min.py
5. satellite_mission_time_spans_new.py plots the schematic of temporal
durations of each satellite mission

```

./computed/

43 file(s)

```
1
2  Skipping TP+J1+J2+J3+S6A_231_142: missing file(s)
3  in c.msla_uvVSguv_TP+J1+J2+J3+S6A.py
4
```

```
1
2  very close to equator. Putting a tight limit on abs(lat) > 5 and excluding
3  short tracks near equator is fixing the issue.
4
5  rolling smooth along x dimension with 6 points is not giving any better
6  guv currrent.
7
8  Optimum difference operator:
9  1. The window shape (weights) are fixed for given p and q.
10 It is computed only once.
11 let n = abs(p) + abs(q)
12 a. if p, q are roughly equal the window shape is symmetric.
13 b. if p is say 7 and q is 1, then the window shape is very asymmetric.
14 c. In current implementation, the window config (-p, q) is fixed for the
15 whole track.
16 In a more sophisticated implementation, the window config has to change as we approach the coast.
17 2. Now we loop over all points of the track. At each point i, we find n  number of slopes, some on -p side and some on q side.
18 3. The weighted (previously computed weights) sum of these slopes becomes
19 the optimum slope at the point i.
20
21 Can odo filter be implemented using xarray methods like convolve, map, reduce etc instead of relying numpy math? Once the weights are calculated
22 use xarray methods to implement the filter.
23 @c.odo_smooth.one.py
24
25 STD:
26 There is no gshhg business in sla std scripts of l2p and xtrack
27 valid count:
28 There is definitely gshhg count business in the valid count script of l2p.
29 But what about xtrack count script
30
31
32 In both l2p and xtrack count variable, interpolation is done on the counter
33 variable after percentage is calculated.
34
```

computed/TODO.txt

```
1
2  [] Compare stds of along track gc of l2p and xtrack s3a and for common tracks
3
4  [] code optimal interpolater operator again
5
6  Minor:
7  [] save speed also in guv nc files
8
```

computed/c.box_12_smooth.all.py

```python
import xarray as xr
from tools_xtrackm import *

for sat in sats_new[:]:
    for f in sorted(
        glob.glob(
            f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False)
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) < 12:
            continue
        print(f"{sat} {track_number} ------------------")
        sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
        dac_track = track_dist_time_asn_midx(ds, var_str="dac", units_in="m")
        sla_smooth = track_smooth_box_x(sla_track, smooth=12, min_periods=6)
        dac_smooth = track_smooth_box_x(dac_track, smooth=12, min_periods=6)
        # sla_smooth = track_smooth_loess_statsmodels_x_parallel(sla_track, frac=0.2)
        # dac_smooth = track_smooth_loess_statsmodels_x_parallel(dac_track, frac=0.2)
        ds_out = xr.Dataset({"sla_smooth": sla_smooth, "dac_smooth": dac_smooth})
        ds_out.to_netcdf(
        f"sla_box_12/track_sla_along_{sat}_{track_number}_box_12.nc"
        )

```

```python
import pickle
from tools_xtrackm import *
import rich
from rich.console import Console

console = Console()

total_tuples = []

ids = list(cb_d.keys())
ln = len(ids)

for i in range(ln):
    id1 = ids[i]
    lon_cb, lat_cb = cb_d[id1]
    # if not is_within_region(lon_cb, lat_cb, *TRACKS_REG):
    #     continue
    for sat in sats_new[:]:
        list_tuples = cb_nearby_track_box(id1, sat, box_size=1.5)
        # print(list_tuples[0])
        if len(list_tuples) == 1 and all(
            element is None for element in list_tuples[0]
        ):
            continue
        console.print(f"{id1} {sat} ----------------------")
        console.print(list_tuples)
        total_tuples.extend(list_tuples)

sorted_list = sorted(total_tuples, key=lambda x: x[4])
print(sorted_list)

with open("closest_tracks_all_cb_new.pkl", "wb") as file:
    pickle.dump(sorted_list, file)

```

```python
import pickle
from tools_xtrackm import *
import rich
from rich.console import Console

console = Console()

ln=len(coastal_d.keys())
total_tuples = []
for id1 in coastal_d.keys():
    lon_coastal, lat_coastal = coastal_d[id1]
    if not is_within_region(lon_coastal, lat_coastal, *TRACKS_REG):
        continue
    for sat in sats_new[:]:
        list_tuples = coastal_nearby_track_box(id1, sat, box_size=1.5)
        print(list_tuples)
        if len(list_tuples) == 1 and all(
            element is None for element in list_tuples[0]
        ):
            continue
        console.print(f"{id1} {sat} ----------------------")
        console.print(list_tuples)
        total_tuples.extend(list_tuples)

sorted_list = sorted(total_tuples, key=lambda x: x[4])
print(sorted_list)

with open("closest_tracks_all_coastal.pkl", "wb") as file:
    pickle.dump(sorted_list, file)

```

computed/c.common_passes.l2pVSxtrack.py

```python
import pickle

# east_coast_tracks_l2p.pkl
# east_coast_tracks_xtrack.pkl
# read above files and compare

# read pkl files

with open('east_coast_tracks_l2p.pkl', 'rb') as f:
    l2p = pickle.load(f)

with open('east_coast_tracks_xtrack.pkl', 'rb') as f:
    xtrack = pickle.load(f)

l2p_passes = list(l2p.keys())
l2p_passes.sort()
xtrack_passes = [str(pass1).zfill(4) for pass1 in xtrack]
xtrack_passes.sort()

print(l2p_passes)
print(xtrack_passes)

# find common passes
# common = [l2p_passes[i] for i in range(len(l2p_passes)) if l2p_passes[i] in xtrack_passes]
common = list(set(l2p_passes).intersection(set(xtrack_passes)))
common.sort()

# save common passes
with open('common_passes_l2pVSxtrack.pkl', 'wb') as f:
    pickle.dump(common, f)

```

computed/c.common_passes.l2pVSxtrack_selected.py

```python
import pickle

with open('east_coast_intersections_a_110m_eq.pkl', 'rb') as f:
    l2p = pickle.load(f)

with open('east_coast_tracks_xtrack.pkl', 'rb') as f:
    xtrack = pickle.load(f)

# sorted based on lon_eq
l2p_passes = sorted(l2p, key=lambda k : l2p[k][0])
xtrack_passes = [str(pass1).zfill(4) for pass1 in xtrack]

# index of 0137
i1 = xtrack_passes.index('0137')
i2 = xtrack_passes.index('0621')
seleted = xtrack_passes[i1:i2+1]
print(seleted)

with open('common_passes_l2pVSxtrack_selected.pkl', 'wb') as f:
    pickle.dump(seleted, f)

# just to check
i1 = l2p_passes.index('0137')
i2 = l2p_passes.index('0621')
seleted = l2p_passes[i1:i2+1]
print(seleted)

```

```python
import xarray as xr
from tools_xtrackm import *

for sat in sats_new[:]:
    for f in sorted(
        glob.glob(
            f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False)
        track_number = ds.Pass
        print(f"{sat} {track_number} -------------------")
        if len(ds.lon.values) < 6:
            continue
        sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
        dac_track = track_dist_time_asn_midx(ds, var_str="dac", units_in="m")
        sla_smooth = track_smooth_gaussian_x(sla_track, sigma=3)
        dac_smooth = track_smooth_gaussian_x(dac_track, sigma=3)
        ds_out = xr.Dataset({"sla_smooth": sla_smooth, "dac_smooth": dac_smooth})
        ds_out.to_netcdf(
        f"sla_gauss/track_sla_along_{sat}_{track_number}_gauss3.nc"
        )
        sla_smooth = track_smooth_gaussian_x(sla_track, sigma=6)
        dac_smooth = track_smooth_gaussian_x(dac_track, sigma=6)
        ds_out = xr.Dataset({"sla_smooth": sla_smooth, "dac_smooth": dac_smooth})
        ds_out.to_netcdf(
        f"sla_gauss/track_sla_along_{sat}_{track_number}_gauss6.nc"
        )

```

computed/c.loess_0.05_smooth.all.py

```python
import xarray as xr
from tools_xtrackm import *
from namesm import sats_new

sats_new = ["ERS1+ERS2+ENV+SRL"]
for sat in sats_new[:]:
    for f in sorted(
        glob.glob(
            f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False)
        track_number = ds.Pass
        print(f"{sat} {track_number} -------------------")
        sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
        dac_track = track_dist_time_asn_midx(ds, var_str="dac", units_in="m")
        # sla_smooth = track_smooth_loess_statsmodels_x_parallel(sla_track, frac=0.2)
        # dac_smooth = track_smooth_loess_statsmodels_x_parallel(dac_track, frac=0.2)
        sla_smooth = track_smooth_loess_statsmodels_x(sla_track, frac=0.05)
        dac_smooth = track_smooth_loess_statsmodels_x(dac_track, frac=0.05)
        ds_out = xr.Dataset({"sla_smooth": sla_smooth, "dac_smooth": dac_smooth})
        ds_out.to_netcdf(
        f"sla_loess_0.05/track_sla_along_{sat}_{track_number}_loess_0.05.nc"
        )

```

```python
import xarray as xr
from tools_xtrackm import *
from namesm import sats_new
import os

method = "loess"
para = "0.2"
os.makedirs(f"sla_loess_0.2", exist_ok=True)

sat = "TP+J1+J2+J3+S6A"
sats_new = [sat]

for sat in sats_new[:]:
    for f in sorted(
        glob.glob(
            f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False)
        track_number = ds.Pass
        print(f"{sat} {track_number} ------------------")
        sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
        dac_track = track_dist_time_asn_midx(ds, var_str="dac", units_in="m")
        # sla_smooth = track_smooth_loess_statsmodels_x_parallel(sla_track, frac=0.2)
        # dac_smooth = track_smooth_loess_statsmodels_x_parallel(dac_track, frac=0.2)
        sla_smooth = track_smooth_loess_statsmodels_x(sla_track, frac=0.2)
        dac_smooth = track_smooth_loess_statsmodels_x(dac_track, frac=0.2)
        ds_out = xr.Dataset({"sla_smooth": sla_smooth, "dac_smooth": dac_smooth})
        ds_out.to_netcdf(
        f"sla_loess_0.2/track_sla_along_{sat}_{track_number}_{method}_{para}.nc"
        )

```

```python
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr

# codex resume 019ce71b-c364-7f20-9c6b-6e2a7108c17c

SAT = "TP+J1+J2+J3+S6A"
METHOD = "loess"
METHOD_OUT = "xsmooth"
PARA = "0.05"
GUV_ROOT = Path("/home/srinivasu/xtrackm/guv")
MSLA_DIR = GUV_ROOT / "msla_uv_at_intersections"
GUV_DIR = GUV_ROOT / f"guv_at_intersections_{METHOD_OUT}"
MONTHLY_RULE = "1ME"


def compute_metrics(track_da: xr.DataArray, ref_da: xr.DataArray) -> dict[str, float]:
    aligned = xr.Dataset({"track": track_da, "ref": ref_da}).dropna(dim="time", how="any")
    n_good = int(aligned.sizes.get("time", 0))
    if n_good == 0:
        return {
            "corr": np.nan,
            "rmse": np.nan,
            "bias": np.nan,
            "n_good_points": 0,
            "ref_std": np.nan,
        }

    track = aligned["track"].values
    ref = aligned["ref"].values
    diff = ref - track
    rmse = float(np.sqrt(np.mean(diff**2)))
    bias = float(np.mean(diff))
    ref_std = float(np.std(ref))
    corr = float(np.corrcoef(track, ref)[0, 1]) if n_good >= 2 else np.nan
    return {
        "corr": corr,
        "rmse": rmse,
        "bias": bias,
        "n_good_points": n_good,
        "ref_std": ref_std,
    }


def plot_metrics_bars_2x2(metrics_df: pd.DataFrame, output_path: Path) -> None:
    metrics = [
        ("corr_u", "Correlation (u)"),
        ("rmse_u", "RMSE (u)"),
        ("bias_u", "Bias (u)"),
        ("n_good_points_u", "Matching points (u)"),
    ]
    x = np.arange(len(metrics_df))
    # x_labels = (x + 1).astype(str)
    x_labels = metrics_df["lats_inter"].apply(lambda x: f"{x:.2f}").to_numpy()
    width = 0.38

    fig, axes = plt.subplots(2, 2, figsize=(24, 12), constrained_layout=True)
    axes_flat = axes.flatten()

    for ax, (column, title) in zip(axes_flat, metrics):
        values = metrics_df[column].to_numpy()
        if column == "rmse_u":
            ref_std = metrics_df["std_msla_u"].to_numpy()
            ax.bar(x - width / 2, values, width=width, label="RMSE (u)")
            ax.bar(x + width / 2, ref_std, width=width, label="std(msla_u)")
        else:
            ax.bar(x, values, width=0.7, label=column)
        ax.set_title(title)
        ax.set_xticks(x)
        ax.set_xticklabels(x_labels, rotation=90)
        ax.set_xlabel("Intersection index")
        ax.grid(axis="y", alpha=0.3)
        if column == "corr_u":
            ax.set_ylim(.0, 1.0)
        ax.legend()

    fig.savefig(output_path, dpi=200)
    plt.show()
    plt.close(fig)


def main() -> None:
    inter_csv = GUV_ROOT / f"tracks_intersections_{SAT}_1.csv"
    inter_df = pd.read_csv(inter_csv)
    # sort inter_df by lats_inter
    # inter_df = inter_df.sort_values(by="lats_inter")
    rows: list[dict[str, float | str]] = []

    for _, row in inter_df.iterrows():
        sat = row["sat"]
        track_self = str(row["track_self"])
        track_other = str(row["track_other"])
        lats_inter = float(row["lats_inter"])
        print(lats_inter)
        unique_str = f"{sat}_{track_self}_{track_other}"
        print(f"Processing {unique_str}")
        if lats_inter < 2:
            continue

        msla_path = MSLA_DIR / f"{unique_str}_msla_uv_at_intersections.nc"
        guv_path = GUV_DIR / f"guv_at_intersection_{sat}_{track_self}_{track_other}_{METHOD_OUT}_{PARA}.nc"

        if not guv_path.exists():
            print(f"Skipping guv {unique_str}: missing file(s)")
            continue

        if not msla_path.exists():
            print(f"Skipping msla uv {unique_str}: missing file(s)")
            continue

        with xr.open_dataset(msla_path) as ds_msla, xr.open_dataset(guv_path) as ds_guv:
            gu_monthly =  ds_guv["gu"].resample(time=MONTHLY_RULE).mean()
            gv_monthly =  ds_guv["gv"].resample(time=MONTHLY_RULE).mean()

```

```
            u_ref = ds_msla["u_at"].interp(time=gu_monthly.time)
            v_ref = ds_msla["v_at"].interp(time=gv_monthly.time)
            gu_monthly.plot(color="b", label="gu odo")
            u_ref.plot(color="r", label="msla u_ref")
            plt.legend()
            # plt.show()

            u_metrics = compute_metrics(gu_monthly, u_ref)
            v_metrics = compute_metrics(gv_monthly, v_ref)

        rows.append(
            {
                "unique_str": unique_str,
                "corr_u": u_metrics["corr"],
                "rmse_u": u_metrics["rmse"],
                "bias_u": u_metrics["bias"],
                "n_good_points_u": u_metrics["n_good_points"],
                "std_msla_u": u_metrics["ref_std"],
                "corr_v": v_metrics["corr"],
                "rmse_v": v_metrics["rmse"],
                "bias_v": v_metrics["bias"],
                "n_good_points_v": v_metrics["n_good_points"],
                "std_msla_v": v_metrics["ref_std"],
                "lats_inter": lats_inter,
            }
        )

    # metrics_df = pd.DataFrame(rows).sort_values("unique_str").reset_index(drop=True)
    metrics_df = pd.DataFrame(rows).sort_values("lats_inter", ascending=False).reset_index(drop=True)
    out_csv = Path(__file__).resolve().with_name(f"metrics_msla_uvVSguv_{SAT}_{METHOD_OUT}_{PARA}.csv")
    metrics_df.to_csv(out_csv, index=False)
    print(f"Saved metrics CSV: {out_csv}")

    out_plot = Path(__file__).resolve().with_name(f"metrics_msla_uvVSguv_{SAT}_{METHOD_OUT}_{PARA}_2x2.png")
    plot_metrics_bars_2x2(metrics_df, out_plot)
    print(f"Saved metrics plot: {out_plot}")


if __name__ == "__main__":
    main()

```

```python
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr

# codex resume 019ce71b-c364-7f20-9c6b-6e2a7108c17c

SAT = "TP+J1+J2+J3+S6A"
METHOD = "loess"
PARA = "0.05"
GUV_ROOT = Path("/home/srinivasu/xtrackm/guv")
MSLA_DIR = GUV_ROOT / "msla_uv_at_intersections"
GUV_DIR = GUV_ROOT / f"guv_at_intersections_reindex"
MONTHLY_RULE = "1ME"


def compute_metrics(track_da: xr.DataArray, ref_da: xr.DataArray) -> dict[str, float]:
    aligned = xr.Dataset({"track": track_da, "ref": ref_da}).dropna(dim="time", how="any")
    n_good = int(aligned.sizes.get("time", 0))
    if n_good == 0:
        return {
            "corr": np.nan,
            "rmse": np.nan,
            "bias": np.nan,
            "n_good_points": 0,
            "ref_std": np.nan,
        }

    track = aligned["track"].values
    ref = aligned["ref"].values
    diff = ref - track
    rmse = float(np.sqrt(np.mean(diff**2)))
    bias = float(np.mean(diff))
    ref_std = float(np.std(ref))
    corr = float(np.corrcoef(track, ref)[0, 1]) if n_good >= 2 else np.nan
    return {
        "corr": corr,
        "rmse": rmse,
        "bias": bias,
        "n_good_points": n_good,
        "ref_std": ref_std,
    }


def plot_metrics_bars_2x2(metrics_df: pd.DataFrame, output_path: Path) -> None:
    metrics = [
        ("corr_u", "Correlation (u)"),
        ("rmse_u", "RMSE (u)"),
        ("bias_u", "Bias (u)"),
        ("n_good_points_u", "Matching points (u)"),
    ]
    x = np.arange(len(metrics_df))
    # x_labels = (x + 1).astype(str)
    x_labels = metrics_df["lats_inter"].apply(lambda x: f"{x:.2f}").to_numpy()
    width = 0.38

    fig, axes = plt.subplots(2, 2, figsize=(24, 12), constrained_layout=True)
    axes_flat = axes.flatten()

    for ax, (column, title) in zip(axes_flat, metrics):
        values = metrics_df[column].to_numpy()
        if column == "rmse_u":
            ref_std = metrics_df["std_msla_u"].to_numpy()
            ax.bar(x - width / 2, values, width=width, label="RMSE (u)")
            ax.bar(x + width / 2, ref_std, width=width, label="std(msla_u)")
        else:
            ax.bar(x, values, width=0.7, label=column)
        ax.set_title(title)
        ax.set_xticks(x)
        ax.set_xticklabels(x_labels, rotation=90)
        ax.set_xlabel("Intersection index")
        ax.grid(axis="y", alpha=0.3)
        if column == "corr_u":
            ax.set_ylim(.0, 1.0)
        ax.legend()

    fig.savefig(output_path, dpi=200)
    plt.show()
    plt.close(fig)


def main() -> None:
    inter_csv = GUV_ROOT / f"tracks_intersections_{SAT}_1.csv"
    inter_df = pd.read_csv(inter_csv)
    # sort inter_df by lats_inter
    # inter_df = inter_df.sort_values(by="lats_inter")
    rows: list[dict[str, float | str]] = []

    for _, row in inter_df.iterrows():
        sat = row["sat"]
        track_self = str(row["track_self"])
        track_other = str(row["track_other"])
        lats_inter = float(row["lats_inter"])
        unique_str = f"{sat}_{track_self}_{track_other}"
        print(f"Processing {unique_str}")

        msla_path = MSLA_DIR / f"{unique_str}_msla_uv_at_intersections.nc"
        guv_path = GUV_DIR / f"guv_at_intersection_{sat}_{track_self}_{track_other}_{METHOD}_{PARA}.nc"

        if not msla_path.exists() or not guv_path.exists():
            print(f"Skipping {unique_str}: missing file(s)")
            print(guv_path)
            continue

        with xr.open_dataset(msla_path) as ds_msla, xr.open_dataset(guv_path) as ds_guv:
            gu_monthly = ds_guv["gu"].resample(time=MONTHLY_RULE).mean()
            gv_monthly = ds_guv["gv"].resample(time=MONTHLY_RULE).mean()

            u_ref = ds_msla["u_at"].interp(time=gu_monthly.time)
            v_ref = ds_msla["v_at"].interp(time=gv_monthly.time)

            u_metrics = compute_metrics(gu_monthly, u_ref)
            v_metrics = compute_metrics(gv_monthly, v_ref)

        rows.append(
```

```
            {
                "unique_str": unique_str,
                "corr_u": u_metrics["corr"],
                "rmse_u": u_metrics["rmse"],
                "bias_u": u_metrics["bias"],
                "n_good_points_u": u_metrics["n_good_points"],
                "std_msla_u": u_metrics["ref_std"],
                "corr_v": v_metrics["corr"],
                "rmse_v": v_metrics["rmse"],
                "bias_v": v_metrics["bias"],
                "n_good_points_v": v_metrics["n_good_points"],
                "std_msla_v": v_metrics["ref_std"],
                "lats_inter": lats_inter,
            }
        )

    # metrics_df = pd.DataFrame(rows).sort_values("unique_str").reset_index(drop=True)
    metrics_df = pd.DataFrame(rows).sort_values("lats_inter", ascending=False).reset_index(drop=True)
    out_csv = Path(__file__).resolve().with_name(f"metrics_msla_uvVSguv_{SAT}_reindex_{PARA}.csv")
    metrics_df.to_csv(out_csv, index=False)
    print(f"Saved metrics CSV: {out_csv}")

    out_plot = Path(__file__).resolve().with_name(f"metrics_msla_uvVSguv_{SAT}_reindex_{PARA}_2x2.png")
    plot_metrics_bars_2x2(metrics_df, out_plot)
    print(f"Saved metrics plot: {out_plot}")
    globals().update(locals())


if __name__ == "__main__":
    main()

```

```python
import math
import os

import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *

method = "odo"
p = -7
q = 8
para = f"{abs(p)}{abs(q)}"
ioff = 3
out_dir = f"/home/srinivasu/xtrackm/guv/guv_at_intersections_{method}/"
os.makedirs(out_dir, exist_ok=True)


def compute_geostrophy_gc_from_slope(ds, slope_smooth):
    lons_track_rev = ds.lon.values[::-1]
    lats_track_rev = ds.lat.values[::-1]
    if len(lons_track_rev) < 3:
        return None
    f1 = cor_p(lats_track_rev)
    f2 = f1.values
    f3 = np.where(np.abs(lats_track_rev) > 2, f2, np.nan)
    g = grav(lats_track_rev).values
    ratio = g / f3
    # to orient slope coast to equator as is the convention
    slope_smooth = -1 * slope_smooth
    gc = ratio[:, np.newaxis] * slope_smooth
    gc.attrs["units"] = "m/s"
    gc.attrs["long_name"] = "Cross-track geostrophic current"
    gc.attrs["description"] = "Geostrophic current perpendicular to satellite track"
    return gc


def track_dist_time_asn_midx_da(ds, da_smooth, units_in="m"):
    dists = compute_dist(ds, units=units_in)
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    dt0 = datetime.strptime("1950-01-01", "%Y-%m-%d")
    dists_rev = dists[::-1]
    ln = ds["sla"].sizes["points_numbers"]
    ln2 = ln // 2
    dvals = ds.time.isel(points_numbers=ln2).values
    try:
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    except ValueError:
        dvals = pd.Series(dvals).ffill().bfill()
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    var1_rev = da_smooth.load().isel(points_numbers=slice(None, None, -1)).values
    var2 = xr.DataArray(
        var1_rev,
        dims=["x", "time"],
        coords={
            "x": dists_rev,
            "time": dates,
            "lon": ("x", lons_track_rev),
            "lat": ("x", lats_track_rev),
        },
    )
    if units_in == "m":
        var2.coords["x"].attrs["units"] = "m"
    elif units_in == "km":
        var2.coords["x"].attrs["units"] = "km"
    return var2


def apply_optimal_operator(ds, p=-5, q=6, dt=1.0):
    def get_optimal_weights(p, q, dt):
        p_abs, q_abs = abs(p), abs(q)
        n = np.array([i for i in range(-p_abs, q_abs + 1) if i != 0])
        N = p_abs + q_abs
        T = N + 1
        A = np.zeros((T, T))
        A[N, :N] = 1.0
        for i in range(N):
            A[i, :N] = (1.0 / n) * (-n[i] / 2.0)
            A[i, N] = (n[i] ** 2 * dt**2) / 4.0
            A[i, i] = -1.0
        B = np.zeros(T)
        B[N] = 1.0
        results = np.linalg.solve(A, B)
        return results[:N], n

    cn, n_indices = get_optimal_weights(p, q, dt)
    sla = ds.sla.values
    points_dim = sla.shape[0]
    slopes = np.full(sla.shape, np.nan)
    p_abs = abs(p)
    q_abs = abs(q)
    # slopes here are oriented equator to coast
    for i in range(p_abs, points_dim - q_abs):
        h_i = sla[i, :]
        h_rel = sla[i + n_indices, :]
        window_slopes = (h_rel - h_i) / (n_indices[:, np.newaxis] * dt)
        valid = ~np.isnan(window_slopes)
        w = cn[:, np.newaxis] * valid
        w_sum = np.sum(np.abs(w), axis=0)
        w_sum[w_sum == 0] = np.nan
        cn_sum = np.sum(np.abs(cn))
        slopes[i, :] = np.nansum(cn[:, np.newaxis] * window_slopes, axis=0) * (cn_sum / w_sum)
    return xr.DataArray(slopes, coords=ds.sla.coords, dims=ds.sla.dims, name="slope")


def _dx_from_track(ds):
    dists_m = np.asarray(compute_dist(ds, units="m"), dtype=float)
    if dists_m.size < 2:
        return np.nan
    dx = np.diff(dists_m)
    dx = np.abs(dx[np.isfinite(dx)])
    dx = dx[dx > 0]
    if dx.size == 0:
        return np.nan
    return float(np.nanmedian(dx))
```

```python


def _compute_gc_odo(ds, p, q):
    dx_m = _dx_from_track(ds)
    print(dx_m, ds.Pass)
    if not np.isfinite(dx_m):
        return None
    slope_odo = apply_optimal_operator(ds, p=p, q=q, dt=dx_m)
    slope_odo_asn = track_dist_time_asn_midx_da(ds, slope_odo, units_in="m")
    return compute_geostrophy_gc_from_slope(ds, slope_odo_asn)


def _load_track_data_cached(sat, track_number, cache):
    key = (sat, str(track_number))
    if key in cache:
        return cache[key]
    f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    if not os.path.exists(f):
        cache[key] = None
        return None
    ds = xr.open_dataset(f, engine="netcdf4", decode_times=False)
    if len(ds.lon.values) < (abs(p) + abs(q) + 1):
        cache[key] = None
        return None
    gc = _compute_gc_odo(ds, p=p, q=q)
    if gc is None:
        cache[key] = None
        return None
    cache[key] = {"ds": ds, "gc": gc}
    return cache[key]


def main():
    sats_new = ["TP+J1+J2+J3+S6A"]
    print(sats_new)
    cache = {}
    for sat in sats_new[:]:
        inter_path = f"tracks_intersections_{sat}_1.csv"
        if not os.path.exists(inter_path):
            # print(f"Skipping {sat}: missing {inter_path}")
            continue
        df = pd.read_csv(inter_path)
        for _, r in df.iterrows():
            sat1 = r["sat"]
            track_number_self = str(r["track_self"])
            track_number_other = str(r["track_other"])
            lats_inter1 = r["lats_inter"]
            x_from_coast_self1 = r["x_from_coast_self"]
            x_from_coast_other1 = r["x_from_coast_other"]
            if abs(lats_inter1) < 2:
                continue

            self_data = _load_track_data_cached(sat, track_number_self, cache)
            other_data = _load_track_data_cached(sat, track_number_other, cache)
            if self_data is None or other_data is None:
                print(f"Skipping {sat} {track_number_self}-{track_number_other}: missing/invalid track data")
                continue

            ds_self = self_data["ds"]
            ds_other = other_data["ds"]
            gc_self = self_data["gc"]
            gc_other = other_data["gc"]

            ii_self = index_at_lat(ds_self, lats_inter1)
            ii_other = index_at_lat(ds_other, lats_inter1)
            a1 = local_angle(ds_self.lon.values, ds_self.lat.values, ii_self, ioff=ioff)
            a2 = local_angle(ds_other.lon.values, ds_other.lat.values, ii_other, ioff=ioff)
            # print(f"heading angle {sat}: a1 = {math.degrees(a1):.2f}, a2 = {math.degrees(a2):.2f}")

            gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
            gc_other_at = gc_other.sel(x=x_from_coast_other1, method="nearest", drop=True)
            gc_other_at = gc_other_at.interp(time=gc_self_at.time)

            gc_self_at_filt = mad_filter_1d_xr(gc_self_at, 3.5, dim="time")
            gc_other_at_filt = mad_filter_1d_xr(gc_other_at, 3.5, dim="time")
            try:
                gu, gv = geostrophic_components(gc_self_at_filt, gc_other_at_filt, a1, a2)
            except ValueError as err:
                print(f"Skipping {sat} {track_number_self}-{track_number_other}: {err}")
                continue

            gu_filt = gu.where((gu <= 3.5) & (gu >= -3.5))
            gv_filt = gv.where((gv <= 3.5) & (gv >= -3.5))
            gu_filtered = mad_filter_1d_xr(gu_filt, 3.5)
            gv_filtered = mad_filter_1d_xr(gv_filt, 3.5)

            ds_out = xr.Dataset({"gu": gu_filtered, "gv": gv_filtered})
            ds_out_self = xr.Dataset({"gc_self": gc_self_at_filt})
            ds_out_other = xr.Dataset({"gc_other": gc_other_at_filt})

            filename = f"{out_dir}/guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{method}_{para}.nc"
            # print(f"saving to {filename}")
            ds_out.to_netcdf(filename)
            ds_out_self.to_netcdf(f"{out_dir}/gc_at_{sat1}_{track_number_self}_{method}_{para}.nc")
            ds_out_other.to_netcdf(f"{out_dir}/gc_at_{sat1}_{track_number_other}_{method}_{para}.nc")


if __name__ == "__main__":
    main()

```

```python
import xarray as xr
import os
from tools_xtrackm import *
import numpy as np
import matplotlib.pyplot as plt
import sys

method = "odo"


def compute_geostrophy_gc_from_slope(ds, slope_smooth):
    lons_track_rev = ds.lon.values[::-1]
    lats_track_rev = ds.lat.values[::-1]
    if len(lons_track_rev) == 0:
        return None
    # Add minimum length check
    if len(lons_track_rev) < 3:
        print("Warning: Insufficient data points for gradient calculation")
        return None
    f1 = cor_p(lats_track_rev)
    f2 = f1.values
    f3 = np.where(np.abs(lats_track_rev) > 2, f2, np.nan)
    # 2s-2n band equatorial band is masked according to Sudre 2013
    g = grav(lats_track_rev)  # ms-1
    g1 = g.values
    ratio = g1 / f3
    ratio1 = ratio[:, np.newaxis]
    gc = ratio1 * slope_smooth
    # Add metadata
    gc.attrs["units"] = "m/s"
    gc.attrs["long_name"] = "Cross-track geostrophic current"
    gc.attrs["description"] = "Geostrophic current perpendicular to satellite track"
    return gc


def track_dist_time_asn_midx_da(ds, da_smooth, units_in="m"):
    # dists = compute_dist(ds, units="km")  # to convert to meters
    dists = compute_dist(ds, units=units_in)  # to convert to meters
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    dt0 = datetime.strptime("1950-01-01", "%Y-%m-%d")
    dists_rev = dists[::-1]
    ln = ds["sla"].sizes["points_numbers"]
    ln2 = ln // 2
    dvals = ds.time.isel(points_numbers=ln2).values
    try:
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    except ValueError:
        dvals = pd.Series(dvals).ffill().bfill()
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    # now sla values oriented from coast
    var1_rev = da_smooth.load().isel(points_numbers=slice(None, None, -1)).values
    var2 = xr.DataArray(
        var1_rev,
        dims=["x", "time"],
        coords={"x":dists_rev, "time":dates, "lon":("x", lons_track_rev), "lat":("x", lats_track_rev)}
    )
    if units_in == "m":
        var2.coords["x"].attrs["units"] = "m"
    elif units_in == "km":
        var2.coords["x"].attrs["units"] = "km"
    return var2


def apply_optimal_operator(ds, p=-5, q=6, dt=1.0):
    """
    Applies the Powell & Leben (2004) optimal difference operator to an
    X-TRACK dataset along the 'points_numbers' dimension.
    """
    def get_optimal_weights(p, q, dt):
        """Derives coefficients cn using Lagrange multipliers."""
        p_abs, q_abs = abs(p), abs(q)
        N = p_abs + q_abs
        T = N + 1
        # Relative indices excluding the current point (0)
        n = np.array([i for i in range(-p_abs, q_abs + 1) if i != 0])
        # Build Lagrange Matrix A [cite: 471-478]
        A = np.zeros((T, T))
        A[N, :N] = 1.0  # Constraint: sum(cn) = 1 [cite: 58, 693]
        for i in range(N):
            # c_n derivation from the Lagrange function [cite: 85, 477]
            # Match Powell/Leben MATLAB form:
            # A(i, 1:N) = (1./n) .* (-n(i)/2)
            A[i, :N] = (1.0 / n) * (-n[i] / 2.0)
            A[i, N] = (n[i]**2 * dt**2) / 4.0
            A[i, i] = -1.0
        B = np.zeros(T)
        B[N] = 1.0
        # Solve for coefficients [cite: 88, 483]
        results = np.linalg.solve(A, B)
        return results[:N], n
    # 1. Calculate the weights
    cn, n_indices = get_optimal_weights(p, q, dt)
    # 2. Extract the SLA variable [cite: 1422]
    # Dimensions: (points_numbers: 210, cycles_numbers: 186)
    sla = ds.sla.values
    points_dim, cycles_dim = sla.shape
    # Initialize output array with NaNs [cite: 1422]
    slopes = np.full(sla.shape, np.nan)
    # 3. Apply operator along the points_numbers dimension
    # Window constraints to avoid index errors at boundaries [cite: 161]
    p_abs = abs(p)
    q_abs = abs(q)
    for i in range(p_abs, points_dim - q_abs):
        # Calculate derivative: sum( (cn / (n*dt)) * (h_{i+n} - h_i) ) [cite: 55, 690]
        # This computes the across-track geostrophic velocity anomaly [cite: 700]
        h_i = sla[i, :]
        h_rel = sla[i + n_indices, :]  # Shape: (N, cycles_numbers)
        # Element-wise slope calculation for all cycles at once
        # Slope = sum_{n} [ cn * (h_{i+n} - h_i) / (n * dt) ]
        window_slopes = (h_rel - h_i) / (n_indices[:, np.newaxis] * dt)
        # Use nansum so that NaN in part of the window doesn't kill the whole estimate;
        # renormalize weights to account for missing points.
        valid = ~np.isnan(window_slopes)
        w = cn[:, np.newaxis] * valid  # zero out weights for NaN slots
```

```
        w_sum = np.sum(np.abs(w), axis=0)
        w_sum[w_sum == 0] = np.nan  # avoid division by zero
        cn_sum = np.sum(np.abs(cn))
        slopes[i, :] = np.nansum(cn[:, np.newaxis] * window_slopes, axis=0) * (cn_sum / w_sum)
    # Return as a new xarray DataArray to maintain metadata
    return xr.DataArray(
        slopes,
        coords=ds.sla.coords,
        dims=ds.sla.dims,
        name="slope"
    )


p = -5
q = 6
para = f"{abs(p)}{abs(q)}"

os.makedirs(f"sla_{method}_{para}", exist_ok=True)

sat = "GFO"
pass1 = "008"
f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass1}.nc"
ds = xr.open_dataset(f, decode_times=False)
track_number = ds.Pass
print(f"{sat} {track_number} -------------------")
# Use physical along-track spacing (meters) so ODO slope has SI-consistent units.
dists_m = np.asarray(compute_dist(ds, units="m"), dtype=float)
dx_m = float(np.nanmedian(np.abs(np.diff(dists_m))))
slope_odo = apply_optimal_operator(ds, p=p, q=q, dt=dx_m)

# making sla as a function of distance
sla_asn = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
# Regular slope using physical distance (same units as ODO)
slope_asn = sla_asn.differentiate("x")
slope_asn1 = slope_asn.isel(time=0)

slope_odo_asn = track_dist_time_asn_midx_da(ds, slope_odo, units_in="m")
slope_odo_asn1 = slope_odo_asn.isel(time=0)# + 0.05

slope_asn1.plot(label="slope regular sla")
slope_odo_asn1.plot(label="odo slope")
plt.legend()
plt.show()
plt.close("all")

gc1 = compute_geostrophy_gc(ds, sla_asn)
gc11 = gc1.isel(time=10)

gc2 = compute_geostrophy_gc_from_slope(ds, slope_odo_asn)
gc21 = gc2.isel(time=10)

gc11.plot(label="regular gc")
gc21.plot(label="odo gc")

plt.legend()
plt.show()
plt.close("all")
sys.exit()

```

```python
import xarray as xr
import os
from tools_xtrackm import *
import numpy as np
import matplotlib.pyplot as plt
import sys

method = "odo"


def compute_geostrophy_gc_from_slope(ds, slope_smooth):
    lons_track_rev = ds.lon.values[::-1]
    lats_track_rev = ds.lat.values[::-1]
    if len(lons_track_rev) == 0:
        return None
    # Add minimum length check
    if len(lons_track_rev) < 3:
        print("Warning: Insufficient data points for gradient calculation")
        return None
    f1 = cor_p(lats_track_rev)
    f2 = f1.values
    f3 = np.where(np.abs(lats_track_rev) > 2, f2, np.nan)
    # 2s-2n band equatorial band is masked according to Sudre 2013
    g = grav(lats_track_rev)  # ms-1
    g1 = g.values
    ratio = g1 / f3
    ratio1 = ratio[:, np.newaxis]
    gc = ratio1 * slope_smooth
    # Add metadata
    gc.attrs["units"] = "m/s"
    gc.attrs["long_name"] = "Cross-track geostrophic current"
    gc.attrs["description"] = "Geostrophic current perpendicular to satellite track"
    return gc


def track_dist_time_asn_midx_da(ds, da_smooth, units_in="m"):
    # dists = compute_dist(ds, units="km")  # to convert to meters
    dists = compute_dist(ds, units=units_in)  # to convert to meters
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    dt0 = datetime.strptime("1950-01-01", "%Y-%m-%d")
    dists_rev = dists[::-1]
    ln = ds["sla"].sizes["points_numbers"]
    ln2 = ln // 2
    dvals = ds.time.isel(points_numbers=ln2).values
    try:
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    except ValueError:
        dvals = pd.Series(dvals).ffill().bfill()
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    # now sla values oriented from coast
    var1_rev = da_smooth.load().isel(points_numbers=slice(None, None, -1)).values
    var2 = xr.DataArray(
        var1_rev,
        dims=["x", "time"],
        coords={"x":dists_rev, "time":dates, "lon":("x", lons_track_rev), "lat":("x", lats_track_rev)}
    )
    if units_in == "m":
        var2.coords["x"].attrs["units"] = "m"
    elif units_in == "km":
        var2.coords["x"].attrs["units"] = "km"
    return var2


def apply_optimal_operator(ds, p=-5, q=6, dt=1.0):
    """
    Applies the Powell & Leben (2004) optimal difference operator to an
    X-TRACK dataset along the 'points_numbers' dimension.
    """
    def get_optimal_weights(p, q, dt):
        """Derives coefficients cn using Lagrange multipliers."""
        p_abs, q_abs = abs(p), abs(q)
        N = p_abs + q_abs
        T = N + 1
        # Relative indices excluding the current point (0)
        n = np.array([i for i in range(-p_abs, q_abs + 1) if i != 0])
        # Build Lagrange Matrix A [cite: 471-478]
        A = np.zeros((T, T))
        A[N, :N] = 1.0  # Constraint: sum(cn) = 1 [cite: 58, 693]
        for i in range(N):
            # c_n derivation from the Lagrange function [cite: 85, 477]
            # Match Powell/Leben MATLAB form:
            # A(i, 1:N) = (1./n) .* (-n(i)/2)
            A[i, :N] = (1.0 / n) * (-n[i] / 2.0)
            A[i, N] = (n[i]**2 * dt**2) / 4.0
            A[i, i] = -1.0
        B = np.zeros(T)
        B[N] = 1.0
        # Solve for coefficients [cite: 88, 483]
        results = np.linalg.solve(A, B)
        return results[:N], n
    # 1. Calculate the weights
    cn, n_indices = get_optimal_weights(p, q, dt)
    # 2. Extract the SLA variable as xarray to keep coords/dims intact.
    sla = ds.sla
    points_dim = sla.sizes["points_numbers"]
    # 3. Apply operator along the points_numbers dimension with xarray ops.
    p_abs = abs(p)
    q_abs = abs(q)

    offset = xr.DataArray(n_indices, dims=("offset",), coords={"offset": n_indices})
    weights = xr.DataArray(cn, dims=("offset",), coords={"offset": n_indices})

    # Build h_{i+n} for all n as an xarray stack over "offset".
    h_rel = xr.concat(
        [sla.shift(points_numbers=-int(n)) for n in n_indices],
        dim=offset,
    )
    h_i = sla.expand_dims(offset=offset)
    window_slopes = (h_rel - h_i) / (offset * dt)

    # NaN-aware weighted reduction with renormalization.
    valid = window_slopes.notnull()
    w_abs_sum = weights.abs().where(valid, 0.0).sum("offset")
    cn_abs_sum = weights.abs().sum("offset")
```

```
    numer = (weights * window_slopes).sum("offset", skipna=True)
    slope = numer * (cn_abs_sum / w_abs_sum.where(w_abs_sum != 0))

    # Preserve explicit boundary masking used by the original implementation.
    p_idx = xr.DataArray(np.arange(points_dim), dims=("points_numbers",))
    interior = (p_idx >= p_abs) & (p_idx < (points_dim - q_abs))
    slope = slope.where(interior)

    return slope.rename("slope")


p = -5
q = 6
para = f"{abs(p)}{abs(q)}"

os.makedirs(f"sla_{method}_{para}", exist_ok=True)

sat = "GFO"
pass1 = "008"
f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass1}.nc"
ds = xr.open_dataset(f, decode_times=False)
track_number = ds.Pass
print(f"{sat} {track_number} -------------------")
# Use physical along-track spacing (meters) so ODO slope has SI-consistent units.
dists_m = np.asarray(compute_dist(ds, units="m"), dtype=float)
dx_m = float(np.nanmedian(np.abs(np.diff(dists_m))))
slope_odo = apply_optimal_operator(ds, p=p, q=q, dt=dx_m)

# making sla as a function of distance
sla_asn = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
# Regular slope using physical distance (same units as ODO)
slope_asn = sla_asn.differentiate("x")
slope_asn1 = slope_asn.isel(time=0)

slope_odo_asn = track_dist_time_asn_midx_da(ds, slope_odo, units_in="m")
slope_odo_asn1 = slope_odo_asn.isel(time=0)# + 0.05

slope_asn1.plot(label="slope regular sla")
slope_odo_asn1.plot(label="odo slope")
plt.legend()
plt.show()
plt.close("all")

gc1 = compute_geostrophy_gc(ds, sla_asn)
gc11 = gc1.isel(time=0)

gc2 = compute_geostrophy_gc_from_slope(ds, slope_odo_asn)
gc21 = gc2.isel(time=0)

gc11.plot(label="regular gc")
gc21.plot(label="odo gc")

plt.legend()
plt.show()
plt.close("all")
sys.exit()

```

computed/c.optim_smooth.all.py

```python
import xarray as xr
from tools_xtrackm import *

for sat in sats_new[:]:
    for f in sorted(
        glob.glob(
            f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False)
        track_number = ds.pass_number
        print(f"{sat} {track_number} --------------------")
        sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
        sla_slope = sla_track.differentiate("x")
        # What is slope for dac, following is wrong
        # dac_slope = dac_track.differentiate("x")
        sla_smooth = slope_smooth_optimal_filter_x(sla_slope, p=-7, q=8)
        # dac_smooth = slope_smooth_optimal_filter_x(dac_slope, p=-7, q=8)
        # sla_smooth.plot()
        # plt.show()
        ds_out = xr.Dataset({"sla_smooth": sla_smooth})
        ds_out.to_netcdf(
        f"sla_optim_78/track_sla_slope_along_{sat}_{track_number}_optim_78.nc"
        )

```

computed/current_working.txt

```
1
2  p.valid_points.gshhg.S3A_simple.py
3
```

```python
import math
import pickle

import matplotlib.gridspec as gridspec
import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *

SAT = "S3A"
L2P_PATH = "/home/srinivasu/allData/aviso_l2p/s3a/"
EAST_COAST_REG = (78.0, 95, 0, 25.4)
cmap = "twilight_shifted"

with open("common_passes_l2pVSxtrack_selected.pkl", "rb") as f:
    common = pickle.load(f)

trim_lim = 300
x_common = xr.DataArray(
    np.arange(trim_lim) * 6_675,
    dims=["x"],
    attrs={"units": "m"},
)

# ── figure layout: map left (2 rows), two line plots right ──
fig = plt.figure(figsize=(16, 8))
gs = gridspec.GridSpec(2, 2, width_ratios=[1, 1.3], wspace=0.05, hspace=0.3)

ax_map = fig.add_subplot(gs[:, 0], projection=ccrs.PlateCarree())
ax_std = fig.add_subplot(gs[0, 1])
ax_vp = fig.add_subplot(gs[1, 1])

# ═══════════════════════════════════════════════════════════
# LEFT PANEL — track map
# ═══════════════════════════════════════════════════════════
dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*EAST_COAST_REG[:2])).sel(
    ETOPO05_Y=slice(*EAST_COAST_REG[2:])
)
rose_cut.plot(ax=ax_map, add_colorbar=False, add_labels=False, cmap=cmap)
decorate_axis(ax_map, "", *EAST_COAST_REG, step=4)
ax_map.text(
    0.25,
    0.90,
    f"MISSION: {SAT}",
    transform=ax_map.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
for i, pass1 in enumerate(common):
    pass2 = pass1[1:]
    fn = f"../data/{SAT}/ctoh.sla.ref.{SAT}.nindian.{pass2}.nc"
    with xr.open_dataset(fn, decode_times=False) as ds:
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        ln = len(lons_track)
        mid_i = ln // 2
        lonm, latm = lons_track[mid_i], lats_track[mid_i]
        a1_d = math.degrees(
            local_angle(lons_track, lats_track, mid_i, ioff=3, degrees=False)
        )
        ax_map.scatter(lons_track, lats_track, s=48, color="w")
        if is_within_region(lonm, latm, *NIO):
            ax_map.text(
                lonm,
                latm,
                s=str(track_number),
                fontsize=12,
                rotation=a1_d,
                color="r",
                ha="center",
                va="center",
            )
dse.close()

# ═══════════════════════════════════════════════════════════
# RIGHT-TOP — SLA standard deviation vs distance from coast
# ═══════════════════════════════════════════════════════════
std_arrays_xt, std_arrays_l2p = [], []
track_numbers_xt, track_numbers_l2p = [], []

for pass1 in common:
    pass2 = pass1[1:]
    pass3 = pass1.lstrip("0")
    fn = f"../data/{SAT}/ctoh.sla.ref.{SAT}.nindian.{pass2}.nc"
    ds = xr.open_dataset(fn, decode_times=False)
    sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
    sla_track = sla_track.interp(x=x_common)
    std = sla_track.std(dim="time")
    ax_std.scatter(std.x.values, std.values, color="cyan", marker=".", s=1)
    std_arrays_xt.append(std)
    track_numbers_xt.append(pass3)

tc = xr.DataArray(track_numbers_xt, dims=["track"], name="track")
std_mean_xt = xr.concat(std_arrays_xt, dim=tc).mean(dim="track", skipna=True)
ax_std.plot(
    std_mean_xt.x.values,
    std_mean_xt.values,
    color="b",
    linewidth=3,
    label="XTRACK",
)

for pass1 in common:
    fn = f"{L2P_PATH}/aviso_l2p_s3a_dist_time_{pass1}_asn.nc"
    with xr.open_dataset(fn, decode_times=False) as ds:
        std = ds.sla.std(dim="time")
        ax_std.scatter(std.x.values, std.values, color="gray", marker=".", s=1)
        std_arrays_l2p.append(std)
        track_numbers_l2p.append(ds.attrs["pass"])

tc = xr.DataArray(track_numbers_l2p, dims=["track"], name="track")
```

```python
std_mean_l2p = xr.concat(std_arrays_l2p, dim=tc).mean(dim="track", skipna=True)
ax_std.plot(
    std_mean_l2p.x.values,
    std_mean_l2p.values,
    color="dimgray",
    linewidth=3,
    label="L2P",
)

ax_std.set_xlabel("Distance (km)")
ax_std.set_ylabel("STD (m)")
ax_std.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax_std.text(
    0.02,
    0.5,
    "coast",
    transform=ax_std.transAxes,
    fontsize=10,
    fontweight="bold",
    color="r",
    rotation=90,
)
ax_std.set_xlim(0,1000e3)
ax_std.set_ylim(0.05, 0.25)
ax_std.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax_std.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax_std.legend(ncol=2, fontsize=8)

# ═══════════════════════════════════════════════════
# RIGHT-BOTTOM — valid point count vs GSHHG distance from coast
# ═══════════════════════════════════════════════════
cnt_arrays_xt, cnt_arrays_l2p = [], []
track_numbers_xt2, track_numbers_l2p2 = [], []

for pass1 in common:
    pass2 = pass1[1:]
    pass3 = pass1.lstrip("0")
    fn = f"../data/{SAT}/ctoh.sla.ref.{SAT}.nindian.{pass2}.nc"
    ds = xr.open_dataset(fn, decode_times=False)
    sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
    sla_track = sla_track.interp(x=x_common)
    gshhg_track = dist_asn_gshhg(
        ds, var_str="dist_to_coast_gshhg", units_in="m"
    )
    gshhg_interp = gshhg_track.interp(x=x_common)
    cnt = sla_track.count(dim="time")
    ax_vp.scatter(
        gshhg_interp.values, cnt.values, color="cyan", marker=".", s=1
    )
    cnt_arrays_xt.append(cnt)
    track_numbers_xt2.append(pass3)

tc = xr.DataArray(track_numbers_xt2, dims=["track"], name="track")
cnt_mean_xt = xr.concat(cnt_arrays_xt, dim=tc).mean(dim="track", skipna=True)
ax_vp.plot(
    cnt_mean_xt.x.values,
    cnt_mean_xt.values,
    color="b",
    linewidth=3,
    label="XTRACK",
)

for pass1 in common:
    fn = f"{L2P_PATH}/aviso_l2p_s3a_dist_time_{pass1}_asn.nc"
    with xr.open_dataset(fn, decode_times=False) as ds:
        cnt = ds.sla.count(dim="time")
        ax_vp.scatter(
            ds.gshhg_da_trim.values, cnt.values, color="gray", marker=".", s=1
        )
        cnt_arrays_l2p.append(cnt)
        track_numbers_l2p2.append(ds.attrs["pass"])

tc = xr.DataArray(track_numbers_l2p2, dims=["track"], name="track")
cnt_mean_l2p = xr.concat(cnt_arrays_l2p, dim=tc).mean(dim="track", skipna=True)
ax_vp.plot(
    cnt_mean_l2p.x.values,
    cnt_mean_l2p.values,
    color="dimgray",
    linewidth=3,
    label="L2P",
)

ax_vp.set_xlabel("GSHHG Distance from coast (km)")
ax_vp.set_ylabel("Valid points (count)")
ax_vp.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax_vp.text(
    0.02,
    0.5,
    "coast",
    transform=ax_vp.transAxes,
    fontsize=10,
    fontweight="bold",
    color="r",
    rotation=90,
)
ax_vp.set_xlim(0, 1000e3)
ax_vp.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax_vp.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax_vp.legend(ncol=2, fontsize=8)

# ── panel labels ──
ax_map.set_title("(a)", loc="left", fontweight="bold")
ax_std.set_title("(b)", loc="left", fontweight="bold")
ax_vp.set_title("(c)", loc="left", fontweight="bold")

plt.tight_layout()
plt.savefig("f.combined_map_std_valid.S3A.png", dpi=300)
plt.show()
plt.close("all")

```

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle

SAT = "S3A"
cmap = "twilight_shifted"
EAST_COAST_REG = (78.0, 95, 0, 25.4)

dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
fig = plt.figure(figsize=(8, 8))
ax1 = fig.add_subplot(1, 1, 1, projection=ccrs.PlateCarree())

rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*EAST_COAST_REG[:2])).sel(
    ETOPO05_Y=slice(*EAST_COAST_REG[2:]))

rose_cut.plot(
    ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap
)

decorate_axis(ax1, "", *EAST_COAST_REG, step=8)
ax1.text(
    0.25,
    0.90,
    f"MISSION: {SAT}",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)

with open("common_passes_l2pVSxtrack_selected.pkl", "rb") as f:
    common = pickle.load(f)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
i=0
for pass1 in common:
    pass2 = pass1[1:]
    f = f"../data/{SAT}/ctoh.sla.ref.{SAT}.nindian.{pass2}.nc"
    with xr.open_dataset(f, decode_times=False) as ds:
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        ln = len(lons_track)
        middle_lon = lons_track[ln // 2]
        middle_lat = lats_track[ln // 2]
        mid_lon_i = np.argmin(np.abs(lons_track - middle_lon))
        mid_lat_i = np.argmin(np.abs(lats_track - middle_lat))
        lonm = middle_lon
        latm = middle_lat
        a1 = local_angle(lons_track, lats_track, mid_lat_i, ioff=3, degrees=False)
        a1_d = math.degrees(a1)
        color = colors[i]
        print(f"{SAT} {track_number} -------------------")
        ax1.scatter(lons_track, lats_track, s=38, color="w")
        if is_within_region(lonm, latm, *NIO):
            ax1.text(
                lonm,
                latm-0.5,
                s=str(track_number),
                fontsize=10,
                fontweight="bold",
                rotation=a1_d,
                color='r',
                ha='center',
                va='center',
            )
        i+=1

plt.tight_layout()
plt.savefig(f"f.l2p.S3A.common_tracks.map.png", dpi=300)
plt.show()
plt.close("all")

dse.close()

```

computed/odo_filter.py

```python
import numpy as np
import xarray as xr

def apply_optimal_operator(ds, p, q, dt=1.0):
    """
    Applies the Powell & Leben (2004) optimal difference operator to an
    X-TRACK dataset along the 'points_numbers' dimension.
    """

    def get_optimal_weights(p, q, dt):
        """Derives coefficients cn using Lagrange multipliers."""
        p_abs, q_abs = abs(p), abs(q)
        N = p_abs + q_abs
        T = N + 1

        # Relative indices excluding the current point (0)
        n = np.array([i for i in range(-p_abs, q_abs + 1) if i != 0])

        # Build Lagrange Matrix A [cite: 471-478]
        A = np.zeros((T, T))
        A[N, :N] = 1.0  # Constraint: sum(cn) = 1 [cite: 58, 693]

        for i in range(N):
            # c_n derivation from the Lagrange function [cite: 85, 477]
            A[i, :N] = (1.0 / n) * (-n[i] / 2.0)
            A[i, N] = (n[i]**2 * dt**2) / 4.0
            A[i, i] = -1.0

        B = np.zeros(T)
        B[N] = 1.0

        # Solve for coefficients [cite: 88, 483]
        results = np.linalg.solve(A, B)
        return results[:N], n

    # 1. Calculate the weights
    cn, n_indices = get_optimal_weights(p, q, dt)

    # 2. Extract the SLA variable [cite: 1422]
    # Dimensions: (points_numbers: 210, cycles_numbers: 186)
    sla = ds.sla.values
    points_dim, cycles_dim = sla.shape

    # Initialize output array with NaNs [cite: 1422]
    slopes = np.full(sla.shape, np.nan)

    # 3. Apply operator along the points_numbers dimension
    # Window constraints to avoid index errors at boundaries [cite: 161]
    p_abs = abs(p)
    q_abs = abs(q)

    for i in range(p_abs, points_dim - q_abs):
        # Calculate derivative: sum( (cn / (n*dt)) * (h_{i+n} - h_i) ) [cite: 55, 690]
        # This computes the across-track geostrophic velocity anomaly [cite: 700]
        h_i = sla[i, :]
        h_rel = sla[i + n_indices, :] # Shape: (N, cycles_numbers)

        # Element-wise slope calculation for all cycles at once
        # Slope = sum_{n} [ cn * (h_{i+n} - h_i) / (n * dt) ]
        window_slopes = (h_rel - h_i) / (n_indices[:, np.newaxis] * dt)
        slopes[i, :] = np.sum(cn[:, np.newaxis] * window_slopes, axis=0)

    # Return as a new xarray DataArray to maintain metadata
    return xr.DataArray(
        slopes,
        coords=ds.sla.coords,
        dims=ds.sla.dims,
        name="geostrophic_velocity_anomaly"
    )

# Example Usage:
# ds = xr.open_dataset("your_xtrack_file.nc")
# # A 12-point filter (p=-5, q=6) is common for WFS mesoscale
# v_geo = apply_optimal_operator(ds, p=-5, q=6)

```

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle

sat = "S3A"
cmap = "twilight_shifted"
EAST_COAST_REG = (78.0, 95, 0, 25.4)

ds_xtr = xr.open_dataset("std_gc_mean_S3A_xtrack.nc")
ds_l2p = xr.open_dataset("/home/srinivasu/allData/aviso_l2p/s3a/std_gc_mean_S3A_l2p.nc")

std_xtr = ds_xtr["std"]
std_l2p = ds_l2p["std"]

with open("common_passes_l2pVSxtrack.pkl", "rb") as f:
    common = pickle.load(f)

dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
fig = plt.figure(figsize=(10, 6))
ax1 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax2 = fig.add_subplot(2, 1, 2)

dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:])).plot(
    ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap
)
decorate_axis(ax1, "", *NIO, step=8)
ax1.text(
    0.45,
    0.80,
    f"MISSION: {sat}",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
i = 0
exclude_tracks = ["451", "565", "679", "23"]
for pass1 in common:
    pass2 = pass1[1:]
    pass3 = pass1.lstrip('0')
    if pass3 in exclude_tracks:
        continue
    f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass2}.nc"
    with xr.open_dataset(f, decode_times=False) as ds:
        lon_eq = ds.lon.values[0]
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        lon_equat = lons_track[0]
        lon_coast = lons_track[-1]
        lat_equat = lats_track[0]
        lat_coast = lats_track[-1]
        a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
        a1_d = math.degrees(a1)
        track_number = ds.Pass
        lonm = lons_track.mean()
        latm = lats_track.mean()
        color = colors[i]
        print(f"{sat} {track_number} -------------------")
        ax1.scatter(lons_track, lats_track, s=2, color=color)
        if is_within_region(lonm, latm, *NIO):
            ax1.text(
                lonm,
                latm,
                s=str(track_number),
                fontsize=7,
                rotation=a1_d,
                color='k',
                ha="center",
                va="center",
                transform=ccrs.PlateCarree(),
            )
        i += 1

# ax2.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))

ax2.plot(std_xtr, label="xtrack std", color="red")
ax2.plot(std_l2p, label="l2p std", color="blue")
ax2.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax2.set_xlabel("distance along track (km)")
ax2.set_ylabel("std")
ax2.set_title("std_mean_xtrack vs std_mean_l2p")
ax2.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax2.text(
    0.02,
    0.5,
    "coast",
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
    color="k",
    rotation=90,
)

plt.legend()
plt.tight_layout()
plt.show()
plt.close("all")

dse.close()

```

computed/p.compare.std_mean.l2pVSxtrack.py

```python
import matplotlib.pyplot as plt
import xarray as xr

ds_xtr = xr.open_dataset("std_mean_S3A_xtrack.nc")
ds_l2p = xr.open_dataset("/home/srinivasu/allData/aviso_l2p/s3a/std_mean_S3A_l2p.nc")

std_xtr = ds_xtr["std"]
std_l2p = ds_l2p["std"]

fig, ax1 = plt.subplots()
ax1.plot(std_xtr, label="xtrack", color="red")
ax1.plot(std_l2p, label="l2p", color="blue")
ax1.set_xlabel("std_mean_xtrack")
ax1.set_ylabel("std_mean_l2p")
ax1.set_title("std_mean_xtrack vs std_mean_l2p")
plt.legend()
plt.show()

```

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle

sat = "S3A"
cmap = "twilight_shifted"
EAST_COAST_REG = (78.0, 95, 0, 25.4)

ds_xtr = xr.open_dataset("std_mean_S3A_xtrack.nc")
ds_l2p = xr.open_dataset("/home/srinivasu/allData/aviso_l2p/s3a/std_mean_S3A_l2p.nc")

std_xtr = ds_xtr["std"]
std_l2p = ds_l2p["std"]

with open("common_passes_l2pVSxtrack.pkl", "rb") as f:
    common = pickle.load(f)

dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
fig = plt.figure(figsize=(10, 6))
ax1 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax2 = fig.add_subplot(2, 1, 2)

dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:])).plot(
    ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap
)
decorate_axis(ax1, "", *NIO, step=8)
ax1.text(
    0.45,
    0.80,
    f"MISSION: {sat}",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
i = 0
for pass1 in common:
    pass2 = pass1[1:]
    f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass2}.nc"
    with xr.open_dataset(f, decode_times=False) as ds:
        lon_eq = ds.lon.values[0]
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        lon_equat = lons_track[0]
        lon_coast = lons_track[-1]
        lat_equat = lats_track[0]
        lat_coast = lats_track[-1]
        a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
        a1_d = math.degrees(a1)
        track_number = ds.Pass
        lonm = lons_track.mean()
        latm = lats_track.mean()
        color = colors[i]
        print(f"{sat} {track_number} ------------------")
        ax1.scatter(lons_track, lats_track, s=2, color=color)
        if is_within_region(lonm, latm, *NIO):
            ax1.text(
                lonm,
                latm,
                s=str(track_number),
                fontsize=7,
                rotation=a1_d,
                color='k',
                ha="center",
                va="center",
                transform=ccrs.PlateCarree(),
            )
        i += 1

# ax2.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))

ax2.plot(std_xtr, label="xtrack std", color="red")
ax2.plot(std_l2p, label="l2p std", color="blue")
ax2.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax2.set_xlabel("distance along track (km)")
ax2.set_ylabel("std")
ax2.set_title("std_mean_xtrack vs std_mean_l2p")
ax2.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax2.text(
    0.02,
    0.5,
    "coast",
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
    color="k",
    rotation=90,
)

plt.legend()
plt.tight_layout()
plt.show()
plt.close("all")

dse.close()

```

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle
import os
import sys
from pathlib import Path

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")
print(png_file)

sat = "S3A"
cmap = "twilight_shifted"
EAST_COAST_REG = (78.0, 95, 0, 25.4)

ds_xtr = xr.open_dataset("../computed/std_mean_S3A_xtrack.nc")
ds_l2p = xr.open_dataset("/home/srinivasu/allData/aviso_l2p/s3a/std_mean_S3A_l2p.nc")

std_xtr = ds_xtr["std"]
std_l2p = ds_l2p["std"]

with open("common_passes_l2pVSxtrack.pkl", "rb") as f:
    common = pickle.load(f)

dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
fig = plt.figure(figsize=(10, 6))
ax1 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax2 = fig.add_subplot(2, 1, 2)

dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:])).plot(
    ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap
)
decorate_axis(ax1, "", *NIO, step=8)
ax1.text(
    0.45,
    0.80,
    f"MISSION: {sat}",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)
ax1.text(
    0.05,
    0.80,
    f"a)",
    transform=ax1.transAxes,
    fontsize=14,
    fontweight="bold",
    color="k",
)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
i = 0
for pass1 in common:
    pass2 = pass1[1:]
    f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass2}.nc"
    with xr.open_dataset(f, decode_times=False) as ds:
        lon_eq = ds.lon.values[0]
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        lon_equat = lons_track[0]
        lon_coast = lons_track[-1]
        lat_equat = lats_track[0]
        lat_coast = lats_track[-1]
        a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
        a1_d = math.degrees(a1)
        track_number = ds.Pass
        lonm = lons_track.mean()
        latm = lats_track.mean()
        color = colors[i]
        print(f"{sat} {track_number} -------------------")
        ax1.scatter(lons_track, lats_track, s=2, color="w")
        if is_within_region(lonm, latm, *NIO):
            ax1.text(
                lonm,
                latm,
                s=str(track_number),
                fontsize=7,
                rotation=a1_d,
                color='k',
                ha="center",
                va="center",
                transform=ccrs.PlateCarree(),
            )
        i += 1

# ax2.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))

ax2.plot(std_xtr, label="xtrack std", color="red")
ax2.plot(std_l2p, label="l2p std", color="blue")
ax2.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax2.set_xlabel("distance along track (km)")
ax2.set_ylabel("std (m)")
# ax2.set_title("std_mean_xtrack vs std_mean_l2p")
ax2.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax2.text(
    0.02,
    0.5,
    "coast",
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
    color="k",
    rotation=90,
)
ax2.text(
    0.02,
    0.80,
    f"b)",
    transform=ax2.transAxes,
```

```
    fontsize=14,
    fontweight="bold",
    color="k",
)

plt.legend()
plt.tight_layout()
plt.savefig(f"std_mean_xtrackVSstd_mean_l2p_{sat}.png")
ln_sf(f"std_mean_xtrackVSstd_mean_l2p_{sat}.png", png_file)
plt.show()
plt.close("all")

dse.close()

```

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle

sat = "S3A"
cmap = "twilight_shifted"
EAST_COAST_REG = (78.0, 95, 0, 25.4)

ds_xtr = xr.open_dataset("count1_mean_S3A_xtrack.nc")
ds_l2p = xr.open_dataset("/home/srinivasu/allData/aviso_l2p/s3a/count1_mean_S3A_l2p.nc")

count1_xtr = ds_xtr["count1"]
count1_l2p = ds_l2p["count1"]

dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
fig = plt.figure(figsize=(10, 6))
ax1 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax2 = fig.add_subplot(2, 1, 2)

dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:])).plot(
    ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap
)
decorate_axis(ax1, "", *NIO, step=8)
ax1.text(
    0.45,
    0.80,
    f"MISSION: {sat}",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)

with open("common_passes_l2pVSxtrack.pkl", "rb") as f:
    common = pickle.load(f)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
i = 0
exclude_tracks = ["451", "565", "679", "23", "621"]
for pass1 in common:
    pass2 = pass1[1:]
    pass3 = pass1.lstrip('0')
    if pass3 in exclude_tracks:
        continue
    f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass2}.nc"
    ds = xr.open_dataset(f, decode_times=False)
    lon_eq = ds.lon.values[0]
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lon_equat = lons_track[0]
    lon_coast = lons_track[-1]
    lat_equat = lats_track[0]
    lat_coast = lats_track[-1]
    a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
    a1_d = math.degrees(a1)

    track_number = int(ds.Pass.values if hasattr(ds.Pass, "values") else ds.Pass)
    lonm = lons_track.mean()
    latm = lats_track.mean()

    color = colors[i]
    print(f"{sat} {track_number} --------------------")
    ax1.scatter(lons_track, lats_track, s=2, color=color)
    if is_within_region(lonm, latm, *NIO):
        ax1.text(
            lonm,
            latm,
            s=str(track_number),
            fontsize=7,
            rotation=a1_d,
            color='k',
            ha='center',
            va='center',
            transform=ccrs.PlateCarree(),
        )
    i += 1

# ax2.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))

ax2.plot(count1_xtr, label="xtrack count1", color="red")
ax2.plot(count1_l2p, label="l2p count1", color="blue")
ax2.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax2.set_xlabel("distance along track (km)")
ax2.set_xlim(-10, 125)
ax2.set_ylabel("count1")
ax2.set_title("valid points Xtrack VS l2p")
ax2.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax2.text(
    0.02,
    0.5,
    "coast",
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
    color="k",
    rotation=90,
)

plt.legend()
plt.tight_layout()
plt.show()
plt.close("all")

dse.close()

```

```python
#!/usr/bin/env python3
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import sys

# codex resume 019ce71b-c364-7f20-9c6b-6e2a7108c17c

# -------------------- USER CONFIG --------------------
SAT = "TP+J1+J2+J3+S6A"

# Explicit (method, para) pairs to compare.
# Example: [("gauss", "3"), ("loess", "0.05"), ("odo", "56")]
METHOD_PARA_PAIRS: list[tuple[str, str]] = [
    ("loess", "0.05"),
    ("xsmooth", "0.05"),
]

# Column to compare across files.
METRIC = "corr_u"

# Column used to align rows across CSVs.
KEY_COL = "unique_str"

# "inner" keeps only matching keys in all files; "outer" keeps union.
JOIN_MODE = "inner"

# Use "index" to show 1..N on x-axis, or "key" to show KEY_COL values.
XLABEL_MODE = "index"
PLOT_MODE = "line"

FIGSIZE = (18, 6)
OUTPUT_PNG = f"compare_{METRIC}.png"
DRAW_GROUP_FRAMES = True
# -----------------------------------------------------


def csv_path_from_method_para(sat: str, method: str, para: str) -> Path:
    return Path(f"metrics_msla_uvVSguv_{sat}_{method}_{para}.csv")


def load_metric_frame(path: Path, key_col: str, metric: str, label: str) -> pd.DataFrame:
    df = pd.read_csv(path)
    if key_col not in df.columns:
        raise KeyError(f"{path}: missing key column '{key_col}'")
    if metric not in df.columns:
        raise KeyError(f"{path}: missing metric column '{metric}'")
    return df[[key_col, metric]].rename(columns={metric: label})


def build_aligned_frame(
    csv_paths: list[Path],
    labels: list[str],
    key_col: str,
    metric: str,
    join_mode: str,
) -> pd.DataFrame:
    merged: pd.DataFrame | None = None
    for path, label in zip(csv_paths, labels):
        frame = load_metric_frame(path, key_col, metric, label)
        if merged is None:
            merged = frame
        else:
            merged = merged.merge(frame, on=key_col, how=join_mode)
    if merged is None:
        raise RuntimeError("No data loaded.")
    return merged.sort_values(key_col).reset_index(drop=True)


def companion_msla_std_metric(metric: str) -> str | None:
    if metric == "rmse_u":
        return "std_msla_u"
    if metric == "rmse_v":
        return "std_msla_v"
    return None


def plot_line_common_only(
    aligned_df: pd.DataFrame,
    key_col: str,
    labels: list[str],
    metric: str,
    xlabel_mode: str,
    figsize: tuple[float, float],
    output_png: str,
    companion_df: pd.DataFrame | None = None,
    companion_metric: str | None = None,
) -> None:
    common_df = aligned_df.dropna(subset=labels).reset_index(drop=True)
    if companion_df is not None and companion_metric is not None:
        companion_common = companion_df[[key_col] + labels]
        common_df = common_df.merge(companion_common, on=key_col, suffixes=("", "_comp"))
        all_required_cols = labels + [f"{label}_comp" for label in labels]
        common_df = common_df.dropna(subset=all_required_cols).reset_index(drop=True)

    if common_df.empty:
        raise ValueError("No rows where all metric values exist across all files.")

    x = np.arange(len(common_df))
    fig, ax = plt.subplots(figsize=figsize, constrained_layout=True)

    if companion_df is not None and companion_metric is not None:
        show_companion_legend = True
        for label in labels:
            ax.plot(
                x,
                common_df[f"{label}_comp"].to_numpy(),
                color="gray",
                alpha=0.35,
                linewidth=4.0,
                zorder=1,
                label=companion_metric if show_companion_legend else None,
            )
            show_companion_legend = False

```

```
    for label in labels:
        ax.plot(x, common_df[label].to_numpy(), linewidth=1.5, label=label, zorder=2)

    ax.set_title(f"{metric} line comparison (common rows only)")
    ylabel = metric if companion_metric is None else f"{metric} (+ {companion_metric})"
    ax.set_ylabel(ylabel)
    ax.grid(alpha=0.3)
    ax.legend()

    ax.set_xlabel("Intersection index" if xlabel_mode == "index" else key_col)
    ax.set_xticks(x)
    if xlabel_mode == "index":
        ax.set_xticklabels((x + 1).astype(str))
    else:
        ax.set_xticklabels(common_df[key_col].astype(str), rotation=90)

    out_path = Path(output_png)
    fig.savefig(out_path, dpi=200)
    plt.show()
    plt.close(fig)
    print(f"Saved line plot: {out_path.resolve()}")
    print(f"Rows plotted (all files valid): {len(common_df)}")


def main() -> None:
    if not METHOD_PARA_PAIRS:
        raise ValueError("METHOD_PARA_PAIRS is empty. Add at least one (method, para) pair.")

    csv_paths = []
    labels = []
    for method, para in METHOD_PARA_PAIRS:
        path = csv_path_from_method_para(SAT, str(method), str(para))
        if not path.exists():
            raise FileNotFoundError(f"Missing CSV for ({method}, {para}): {path}")
        csv_paths.append(path)
        labels.append(f"{method}:{para}")

    if len(labels) != len(csv_paths):
        raise ValueError("LABELS length must match number of CSV files.")

    aligned_df = build_aligned_frame(
        csv_paths=csv_paths,
        labels=labels,
        key_col=KEY_COL,
        metric=METRIC,
        join_mode=JOIN_MODE,
    )

    companion_metric = companion_msla_std_metric(METRIC)
    companion_df = None
    if companion_metric is not None:
        companion_df = build_aligned_frame(
            csv_paths=csv_paths,
            labels=labels,
            key_col=KEY_COL,
            metric=companion_metric,
            join_mode=JOIN_MODE,
        )

    print(f"Using files: {[str(p) for p in csv_paths]}")
    line_output = str(Path(OUTPUT_PNG).with_name(f"{Path(OUTPUT_PNG).stem}_line.png"))
    plot_line_common_only(
        aligned_df=aligned_df,
        key_col=KEY_COL,
        labels=labels,
        metric=METRIC,
        xlabel_mode=XLABEL_MODE,
        figsize=FIGSIZE,
        output_png=line_output,
        companion_df=companion_df,
        companion_metric=companion_metric,
    )
    globals().update(locals())
    sys.exit()


if __name__ == "__main__":
    main()

```

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle

sat = "S3A"
cmap = "twilight_shifted"
EAST_COAST_REG = (78.0, 95, 0, 25.4)

dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
fig = plt.figure(figsize=(10, 6))
ax1 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax2 = fig.add_subplot(2, 1, 2)

rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:]))
rose_cut.plot(
    ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap
)
decorate_axis(ax1, "", *NIO, step=8)
ax1.text(
    0.45,
    0.80,
    f"MISSION: {sat}",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)

x_common = xr.DataArray(np.arange(0, 1_300_001, 6_000), dims=["x"],
  attrs={"units": "m"})  # 0..1300 km, 1 km step

with open("common_passes_l2pVSxtrack_selected.pkl", "rb") as f:
    common = pickle.load(f)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
i = 0
std_arrays = []
track_numbers = []
for pass1 in common:
    pass2 = pass1[1:]
    pass3 = pass1.lstrip("0")
    f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass2}.nc"
    ds = xr.open_dataset(f, decode_times=False)
    lon_eq = ds.lon.values[0]
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lon_equat = lons_track[0]
    lon_coast = lons_track[-1]
    lat_equat = lats_track[0]
    lat_coast = lats_track[-1]
    a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
    a1_d = math.degrees(a1)
    track_number = int(ds.Pass.values if hasattr(ds.Pass, "values") else ds.Pass)
    lonm = lons_track.mean()
    latm = lats_track.mean()
    sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
    sla_track = sla_track.interp(x=x_common)
    std = sla_track.std(dim="time")
    color = colors[i]
    print(f"{sat} {track_number} -------------------")
    ax1.scatter(lons_track, lats_track, s=2, color=color)
    if is_within_region(lonm, latm, *NIO):
        ax1.text(
            lonm,
            latm,
            s=str(track_number),
            fontsize=7,
            rotation=a1_d,
            color='k',
            ha='center',
            va='center',
            transform=ccrs.PlateCarree(),
        )
    ax2.plot(std.x.values, std.values, color=color, linewidth=1, label=str(track_number))
    std_arrays.append(std)
    track_numbers.append(pass3)
    i += 1

# std_mean = xr.concat(std_arrays, dim="track").mean(dim="track", skipna=True)
# std_all = xr.concat(std_arrays, dim="track")
track_coord = xr.DataArray(track_numbers, dims=["track"], name="track")
std_all = xr.concat(std_arrays, dim=track_coord)
std_mean = std_all.mean(dim="track", skipna=True)

ds_out2 = xr.Dataset({"std_all": std_all})
ds_out2.to_netcdf(f"std_all_S3A_xtrack.nc")
ds_out1 = xr.Dataset({"std": std_mean})
ds_out1.to_netcdf(f"std_mean_S3A_xtrack.nc")

ax2.plot(
    std_mean.x.values,
    std_mean.values,
    color="dimgray",
    linewidth=3,
    label="Mean",
)

ax2.set_xlabel("Distance (km)")
ax2.set_ylabel("STD (m)")
ax2.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax2.text(
    0.02,
    0.5,
    "coast",
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
    color="r",
    rotation=90,
)
ax2.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
```

```
ax2.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax2.legend(title="Track", ncol=4, fontsize=7, title_fontsize=8)

plt.tight_layout()
plt.show()
plt.close("all")

dse.close()

```

computed/p.std.S3A.common_tracks.with_mean_simple.py

```python
import numpy as np
import xarray as xr
import matplotlib.pyplot as plt

# Input file produced by your processing script
nc_file = "std_all_S3A_xtrack.nc"

ds = xr.open_dataset(nc_file)
std_all = ds["std_all"]

# Mean across tracks
std_mean = std_all.mean(dim="track", skipna=True)

# Build scatter inputs: every (track, x) point
x_vals = std_all["x"].values
x_scatter = np.broadcast_to(x_vals, std_all.shape).ravel()
y_scatter = std_all.values.ravel()

fig, ax = plt.subplots(figsize=(10, 5))

# All track STD points as cyan dots
ax.scatter(x_scatter, y_scatter, s=8, c="cyan", alpha=0.6, edgecolors="none")

# Mean STD as thick cyan solid line
ax.plot(std_mean["x"].values, std_mean.values, color="b", linewidth=3.5, linestyle="-")

ax.set_xlabel("Distance (m)")
ax.set_ylabel("STD (m)")
ax.set_title("All track STD points and mean STD")
ax.grid(True, linestyle="--", alpha=0.3)

plt.tight_layout()
plt.show()

ds.close()

```

```python
import xarray as xr
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
from namesm import sats_new
import cmaps
import cmocean as cmo
import math

sat = 'S3A'
cmap = 'twilight_shifted'
EAST_COAST_REG = (78.0, 99, 0, 25.4)
dse = xr.open_dataset(
    f"/home/srinivasu/allData/topo/etopo5.cdf"
)
files = sorted(
    glob.glob(
        f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc"
    )
)
tracks_n = len(files)
east_coast_tracks = []
for f in files:
    ds = xr.open_dataset(f, decode_times=False)
    lon_eq = ds.lon.values[0]
    if lon_eq < EAST_COAST_REG[0] or lon_eq > EAST_COAST_REG[1]:
        continue
    track_number = ds.Pass
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lon_equat = lons_track[0]
    lon_coast = lons_track[-1]
    lat_equat = lats_track[0]
    lat_coast = lats_track[-1]
    m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
    lonm = lons_track.mean()
    latm = lats_track.mean()
    dx = 1.5
    dy = m * dx
    lonm3 = lonm - dx
    latm3 = latm - dy
    a1 = math.atan2(lat_coast - lat_equat,
                    lon_coast - lon_equat)
    a1_d = math.degrees(a1)
    if a1_d > 90:
        # append 4 zero padded track number
        east_coast_tracks.append(track_number.zfill(4))
        fig = plt.figure(figsize=(10, 5))
        ax1 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
        dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:])).plot(
            ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap
        )
        decorate_axis(ax1, "", *NIO, step=8)
        info = f"MISSION: {sat}"
        plt.text(
            0.40,
            0.90,
            info,
            transform=ax1.transAxes,
            fontsize=10,
            fontweight="bold",
            color="b",
        )
        # need to put scatter point color to black
        ax1.scatter(
            lons_track, lats_track, s=1, c="r"
            )
        if is_within_region(lonm, latm, *NIO):
            plt.text(
                lonm,
                latm,
                s=track_number,
                fontsize=8,
                rotation=a1_d,
                color="k",
            )
        print(f"{sat} {track_number} ------------------")
        sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
        dac_track = track_dist_time_asn_midx(ds, var_str="dac", units_in="m")
        std = sla_track.std(dim="time")
        ax2 = fig.add_subplot(2, 1, 2)
        std.plot(ax=ax2)
        # Keep axis values in meters, format tick labels as kilometers.
        ax2.set_xlabel("Distance (km)")
        # draw vertical line at x=0 and name it "coast"
        ax2.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
        # plt.text vertical line at x=0 and name it "coast"
        plt.text(
            0.02,
            0.5,
            "coast",
            transform=ax2.transAxes,
            fontsize=10,
            fontweight="bold",
            color="r",
            rotation=90,
        )
        ax2.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
        plt.show()
        plt.close("all")
dse.close()
east_coast_tracks = sorted(east_coast_tracks)
print(east_coast_tracks)

```

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle

L2P_PATH = "/home/srinivasu/allData/aviso_l2p/s3a/"
SAT = "S3A"
EAST_COAST_REG = (78.0, 95, 0, 25.4)

trim_lim = 300
x_common = xr.DataArray(
    np.arange(trim_lim) * 6_675,
    dims=["x"],
    attrs={"units": "m"},
)  # 300 points, 6_675 m spacing

with open("common_passes_l2pVSxtrack_selected.pkl", "rb") as f:
    common = pickle.load(f)

fig, ax1 = plt.subplots(1, 1, figsize=(10, 6))
std_arrays = []
track_numbers = []
for pass1 in common:
    pass2 = pass1[1:]
    pass3 = pass1.lstrip("0")
    f = f"../data/{SAT}/ctoh.sla.ref.{SAT}.nindian.{pass2}.nc"
    ds = xr.open_dataset(f, decode_times=False)
    lon_eq = ds.lon.values[0]
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lon_equat = lons_track[0]
    lon_coast = lons_track[-1]
    lat_equat = lats_track[0]
    lat_coast = lats_track[-1]
    a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
    a1_d = math.degrees(a1)
    track_number = int(ds.Pass.values if hasattr(ds.Pass, "values") else ds.Pass)
    lonm = lons_track.mean()
    latm = lats_track.mean()
    sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
    sla_track = sla_track.interp(x=x_common)
    std = sla_track.std(dim="time")
    print(f"{SAT} {track_number} --------------------")
    ax1.scatter(std.x.values, std.values, color='cyan', marker='.', s=1)
    std_arrays.append(std)
    track_numbers.append(pass3)

track_coord = xr.DataArray(track_numbers, dims=["track"], name="track")
std_all = xr.concat(std_arrays, dim=track_coord)
std_mean = std_all.mean(dim="track", skipna=True)

ax1.plot(
    std_mean.x.values,
    std_mean.values,
    color="b",
    linewidth=3,
    label="XTRACK",
)

ax1.set_xlabel("Distance (km)")
ax1.set_ylabel("STD (m)")
ax1.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax1.text(
    0.02,
    0.5,
    "coast",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="r",
    rotation=90,
)
ax1.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax1.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax1.legend(title="Track", ncol=4, fontsize=7, title_fontsize=8)

std_arrays = []
track_numbers = []
for pass1 in common:
    f = f"{L2P_PATH}/aviso_l2p_s3a_dist_time_{pass1}_asn.nc"
    with xr.open_dataset(f, decode_times=False) as ds:
        track_number = ds.attrs["pass"]
        lons_track = ds.lons.values
        lats_track = ds.lats.values
        ln = len(lons_track)
        sla_track = ds.sla
        std = sla_track.std(dim="time")
        print(f"{SAT} {track_number} --------------------")
        ax1.scatter(std.x.values, std.values, color='gray', marker='.', s=1)
        std_arrays.append(std)
        track_numbers.append(pass3)

track_coord = xr.DataArray(track_numbers, dims=["track"], name="track")
std_all = xr.concat(std_arrays, dim=track_coord)
std_mean = std_all.mean(dim="track", skipna=True)
ax1.plot(
    std_mean.x.values,
    std_mean.values,
    color="dimgray",
    linewidth=3,
    label="L2P",
)

ax1.set_xlabel("Distance (km)")
ax1.set_ylabel("STD (m)")
ax1.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax1.text(
    0.02,
    0.5,
    "coast",
    transform=ax1.transAxes,
    fontsize=10,
```

```
    fontweight="bold",
    color="r",
    rotation=90,
)
ax1.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax1.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax1.legend(title="Track", ncol=4, fontsize=7, title_fontsize=8)

plt.tight_layout()
plt.show()
plt.close("all")

```

computed/p.std.S3Avsl2p.selected.pre_c.py

```python
import numpy as np
import xarray as xr
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter

# Input file produced by your processing script
nc_xtr = "std_all_S3A_xtrack.nc"
nc_l2p = "/home/srinivasu/allData/aviso_l2p/s3a/std_all_S3A_l2p.nc"

ds_xtr = xr.open_dataset(nc_xtr)
ds_l2p = xr.open_dataset(nc_l2p)
std_all_xtr = ds_xtr["std_all"]
std_all_l2p = ds_l2p["std_all"]

# Mean across tracks
std_mean_xtr = std_all_xtr.mean(dim="track", skipna=True)
std_mean_l2p = std_all_l2p.mean(dim="track", skipna=True)

# Build scatter inputs: every (track, x) point
x_vals_xtr = std_all_xtr["x"].values
x_scatter_xtr = np.broadcast_to(x_vals_xtr, std_all_xtr.shape).ravel()
y_scatter_xtr = std_all_xtr.values.ravel()

x_vals_l2p = std_all_l2p["x"].values
x_scatter_l2p = np.broadcast_to(x_vals_l2p, std_all_l2p.shape).ravel()
y_scatter_l2p = std_all_l2p.values.ravel()

fig, ax = plt.subplots(figsize=(10, 5))

# All track std points as cyan dots
ax.scatter(x_scatter_xtr, y_scatter_xtr, s=8, c="cyan", alpha=0.6, edgecolors="none")
ax.scatter(x_scatter_l2p, y_scatter_l2p, s=8, c="red", alpha=0.6, edgecolors="none")

# Mean std as thick cyan solid line
ax.plot(std_mean_xtr["x"].values, std_mean_xtr.values, color="b", linewidth=3.5, linestyle="-")
ax.plot(std_mean_l2p["x"].values, std_mean_l2p.values, color="r", linewidth=3.5, linestyle="-")

ax.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax.set_xlim(-10e3, 300e3)
ax.set_xlabel("Distance (m)")
ax.set_ylabel("std (m)")
ax.set_title("All track std points and mean std")
ax.grid(True, linestyle="--", alpha=0.3)

plt.tight_layout()
plt.show()

ds.close()

```

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle


def compute_geostrophy_gc(ds, sla_smooth):
    lons_track_rev = ds.lon.values[::-1]
    lats_track_rev = ds.lat.values[::-1]
    if len(lons_track_rev) == 0:
        return None
    # Add minimum length check
    if len(lons_track_rev) < 3:
        print("Warning: Insufficient data points for gradient calculation")
        return None
    f1 = cor_p(lats_track_rev)
    f2 = f1.values
    f3 = np.where(lats_track_rev > 5, f2, np.nan)
    # 2s-2n band equatorial band is masked according to Sudre 2013
    g = grav(lats_track_rev)  # ms-1
    g1 = g.values
    ratio = g1 / f3
    ratio1 = ratio[:, np.newaxis]
    dhbydx = sla_smooth.differentiate("x")
    gc = ratio1 * dhbydx
    # Add metadata
    gc.attrs["units"] = "m/s"
    gc.attrs["long_name"] = "Cross-track geostrophic current"
    gc.attrs["description"] = "Geostrophic current perpendicular to satellite track"
    return gc


sat = "S3A"
method = "loess"
cmap = "twilight_shifted"
EAST_COAST_REG = (78.0, 95, 0, 25.4)

dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
fig = plt.figure(figsize=(10, 6))
ax1 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax2 = fig.add_subplot(2, 1, 2)

rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:]))
rose_cut.plot(
    ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap
)
decorate_axis(ax1, "", *NIO, step=8)
ax1.text(
    0.45,
    0.80,
    f"MISSION: {sat}",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)

x_common = xr.DataArray(np.arange(0, 1_300_001, 6_000), dims=["x"],
  attrs={"units": "m"})  # 0..1300 km, 1 km step

with open("common_passes_l2pVSxtrack.pkl", "rb") as f:
    common = pickle.load(f)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
i = 0
frac1 = "0.05"
std_arrays = []
exclude_tracks = ["451", "565", "679", "23", "621"]
for pass1 in common:
    pass2 = pass1[1:]
    f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass2}.nc"
    pass3 = pass1.lstrip('0')
    if pass3 in exclude_tracks:
        continue
    f_smooth = f"sla_{method}_{frac1}a/track_sla_along_{sat}_{pass3}_{method}_{frac1}.nc"
    ds = xr.open_dataset(f, decode_times=False)
    lon_eq = ds.lon.values[0]
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lon_equat = lons_track[0]
    lon_coast = lons_track[-1]
    lat_equat = lats_track[0]
    lat_coast = lats_track[-1]
    a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
    a1_d = math.degrees(a1)
    track_number = ds.Pass
    lonm = lons_track.mean()
    latm = lats_track.mean()
    sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
    ds_smooth = xr.open_dataset(f_smooth)
    sla_smooth = ds_smooth.sla_smooth
    gc = compute_geostrophy_gc(ds, sla_smooth)
    gc = gc.interp(x=x_common)
    # gc_anom = gc - gc.mean(dim="x")
    # gc_anom = gc - gc.mean(dim="time")
    std = gc.std(dim="time")
    # std.plot()
    # plt.show()
    color = colors[i]
    print(f"{sat} {track_number} --------------------")
    ax1.scatter(lons_track, lats_track, s=2, color=color)
    if is_within_region(lonm, latm, *NIO):
        ax1.text(
            lonm,
            latm,
            s=str(track_number),
            fontsize=7,
            rotation=a1_d,
            color='k',
            ha='center',
            va='center',
            transform=ccrs.PlateCarree(),
```

```
        )
    ax2.plot(std.x.values, std.values, color=color, linewidth=1, label=str(track_number))
    std_arrays.append(std)
    i += 1

std_mean = xr.concat(std_arrays, dim="track").mean(dim="track", skipna=True)
ds_out = xr.Dataset({"std": std_mean})
ds_out.to_netcdf(f"std_gc_mean_S3A_xtrack.nc")
ax2.plot(
    std_mean.x.values,
    std_mean.values,
    color="dimgray",
    linewidth=3,
    label="Mean",
)

ax2.set_xlabel("Distance (km)")
ax2.set_ylabel("STD (m)")
ax2.set_xlim(-10e3, 800e3)
ax2.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax2.text(
    0.02,
    0.5,
    "coast",
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
    color="r",
    rotation=90,
)
ax2.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax2.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax2.legend(title="Track", ncol=4, fontsize=7, title_fontsize=8)

plt.tight_layout()
plt.show()
plt.close("all")

dse.close()

```

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle

sat = "S3A"
cmap = "twilight_shifted"
EAST_COAST_REG = (78.0, 95, 0, 25.4)

dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
fig = plt.figure(figsize=(10, 6))
ax1 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax2 = fig.add_subplot(2, 1, 2)

dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:])).plot(
    ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap
)
decorate_axis(ax1, "", *NIO, step=8)
ax1.text(
    0.45,
    0.80,
    f"MISSION: {sat}",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)

with open("common_passes_l2pVSxtrack_selected.pkl", "rb") as f:
    common = pickle.load(f)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
std_arrays = []
i=0
for pass1 in common:
    f = f"aviso_l2p_s3a_dist_time_{pass1}_asn.nc"
    with xr.open_dataset(f, decode_times=False) as ds:
        track_number = ds.attrs["pass"]
        lons_track = ds.lons.values
        lats_track = ds.lats.values
        ln = len(lons_track)
        sla_track = ds.sla
        middle_lon = lons_track[ln // 2]
        middle_lat = lats_track[ln // 2]
        mid_lon_i = np.argmin(np.abs(lons_track - middle_lon))
        mid_lat_i = np.argmin(np.abs(lats_track - middle_lat))
        lonm = middle_lon
        latm = middle_lat
        a1 = local_angle(lons_track, lats_track, mid_lat_i, ioff=3, degrees=False)
        a1_d = math.degrees(a1)
        std = sla_track.std(dim="time")
        color = colors[i]
        print(f"{sat} {track_number} -------------------")
        ax1.scatter(lons_track, lats_track, s=2, color=color)
        if is_within_region(lonm, latm, *NIO):
            ax1.text(
                lonm,
                latm,
                s=str(track_number),
                fontsize=7,
                rotation=a1_d,
                color='k',
            )
        ax2.plot(std.x.values, std.values, color=color, linewidth=1, label=str(track_number))
        std_arrays.append(std)
        i+=1

# std_mean = xr.concat(std_arrays, dim="track").mean(dim="track", skipna=True)
# save std mean
# ds_out = xr.Dataset({"std": std_mean})
# ds_out.to_netcdf(f"std_mean_{sat}_l2p.nc")
std_all = xr.concat(std_arrays, dim="track")
std_mean = std_all.mean(dim="track", skipna=True)

ds_out2 = xr.Dataset({"std_all": std_all})
ds_out2.to_netcdf(f"std_all_{sat}_l2p.nc")
ds_out1 = xr.Dataset({"std": std_mean})
ds_out1.to_netcdf(f"std_mean_{sat}_l2p.nc")
ax2.plot(
    std_mean.x.values,
    std_mean.values,
    color="dimgray",
    linewidth=3,
    label="Mean",
)

ax2.set_xlabel("Distance (km)")
ax2.set_ylabel("STD (m)")
ax2.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax2.text(
    0.02,
    0.5,
    "coast",
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
    color="r",
    rotation=90,
)
ax2.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax2.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax2.legend(title="Track", ncol=4, fontsize=7, title_fontsize=8)

plt.tight_layout()
plt.savefig(f"std_mean_{sat}_l2p.png", dpi=300)
plt.show()
plt.close("all")

dse.close()

```

```python
from glob import glob

import cartopy.crs as ccrs
import cmaps
import cmocean as cmo
import matplotlib.pyplot as plt
import xarray as xr
from cartopy.io.img_tiles import GoogleTiles
from tools_xtrackm import *

cmap = "Spectral"
cmap = "PuBuGn"
cmap = "YlGnBu"
cmap = "YlGn"
cmap_r = plt.cm.get_cmap(cmap)
cmap1 = cmo.cm.topo
cmap2 = cmo.cm.thermal
cmap1 = cmo.cm.topo
cmap1 = cmo.cm.diff
cmap3 = cmaps.BlAqGrYeOrReVi200
cmap4 = cmo.cm.phase

# cmap1="viridis"
zone = "nindian"

IO = (30.0, 120.0, -30.0, 30.0)
NIO = (40.0, 95.0, -5.0, 30.0)
NIO = (40.0, 110.0, -5.0, 30.0)
NIO = (60.0, 100.0, -5.0, 30.0)

height, width = 9, 14

xticks = [30, 45, 60, 75, 90, 105]
yticks = [-30, -15, -5, 0, 5, 15, 30]
xticks = [45, 60, 75, 90, 105]
yticks = [0, 5, 15, 30]
tiles = GoogleTiles(style="satellite")

sats_new = ["S3A"]
for sat in sats_new:
    tracks_n = get_total_tracks(sat)
    tsta, tend = get_time_limits(sat)
    dse = xr.open_dataset(
        "/home/srinivasu/allData/topo/etopo5.cdf"
    )  # open etopo dataset
    fig, ax1 = plt.subplots(
        1,
        1,
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(width, height),
        layout="constrained",
    )
    dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:])).plot(
        ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap1
    )
    decorate_axis(ax1, "", *NIO)
    ax1.grid()
    info = f"MISSION: {sat} from {tsta} to {tend} with {tracks_n} tracks"
    plt.text(
        0.15,
        0.95,
        info,
        transform=ax1.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    for f in sorted(
        glob.glob(f"/home/srinivasu/xtrackm/data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")
    ):
        ds = xr.open_dataset(f, decode_times=False)
        track_number = ds.Pass
        # print(ds)
        sla1 = ds.sla.values
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            continue
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        lon_equat = lons_track[0]  # this on equator
        lat_equat = lats_track[0]  # this on equator
        a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
        a1_d = math.degrees(a1)
        m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
        lonm = lons_track.mean()
        latm = lats_track.mean()
        dx = 1.5
        dy = m * dx
        lonm3 = lonm - dx
        latm3 = latm - dy
        print(a1_d)
        if a1_d > 90:
            ax1.scatter(
                lons_track, lats_track, s=10
            )
            if is_within_region(lonm, latm, *NIO):
                plt.text(
                    lonm,
                    latm,
                    s=track_number,
                    fontsize=10,
                    rotation=a1_d,
                    color="k",
                )
    plt.show()
    plt.close("all")

```

computed/p.valid_points.S3A.selected.pre_c.py

```python
import numpy as np
import xarray as xr
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter

# Input file produced by your processing script
nc_file = "counts_all_S3A_xtrack.nc"

ds = xr.open_dataset(nc_file)
counts_all = ds["counts_all"]

# Mean across tracks
counts_mean = counts_all.mean(dim="track", skipna=True)

# Build scatter inputs: every (track, x) point
x_vals = counts_all["x"].values
x_scatter = np.broadcast_to(x_vals, counts_all.shape).ravel()
y_scatter = counts_all.values.ravel()

fig, ax = plt.subplots(figsize=(10, 5))

# All track counts points as cyan dots
ax.scatter(x_scatter, y_scatter, s=8, c="cyan", alpha=0.6, edgecolors="none")

# Mean counts as thick cyan solid line
ax.plot(counts_mean["x"].values, counts_mean.values, color="b", linewidth=3.5, linestyle="-")

ax.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax.set_xlim(-10e3, 300e3)
ax.set_xlabel("Distance (m)")
ax.set_ylabel("counts (m)")
ax.set_title("All track counts points and mean counts")
ax.grid(True, linestyle="--", alpha=0.3)

plt.tight_layout()
plt.show()

ds.close()

```

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle

L2P_PATH = "/home/srinivasu/allData/aviso_l2p/s3a/"
SAT = "S3A"
EAST_COAST_REG = (78.0, 95, 0, 25.4)

x_common = xr.DataArray(np.arange(0, 1_300_001, 6_000), dims=["x"],
  attrs={"units": "m"})  # 0..1300 km, 1 km step

with open("common_passes_l2pVSxtrack_selected.pkl", "rb") as f:
    common = pickle.load(f)

fig, ax1 = plt.subplots(1, 1, figsize=(10, 6))
std_arrays = []
track_numbers = []
for pass1 in common:
    pass2 = pass1[1:]
    pass3 = pass1.lstrip("0")
    f = f"../data/{SAT}/ctoh.sla.ref.{SAT}.nindian.{pass2}.nc"
    ds = xr.open_dataset(f, decode_times=False)
    lon_eq = ds.lon.values[0]
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lon_equat = lons_track[0]
    lon_coast = lons_track[-1]
    lat_equat = lats_track[0]
    lat_coast = lats_track[-1]
    a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
    a1_d = math.degrees(a1)
    track_number = int(ds.Pass.values if hasattr(ds.Pass, "values") else ds.Pass)
    lonm = lons_track.mean()
    latm = lats_track.mean()
    sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
    sla_track = sla_track.interp(x=x_common)
    gshhg_track = dist_asn_gshhg(ds, var_str="dist_to_coast_gshhg", units_in="m")
    gshhg_track_interp = gshhg_track.interp(x=x_common)
    std = sla_track.count(dim="time")
    print(f"{SAT} {track_number} --------------------")
    ax1.scatter(gshhg_track_interp.values, std.values, color='cyan', marker='.', s=1)
    std_arrays.append(std)
    track_numbers.append(pass3)

track_coord = xr.DataArray(track_numbers, dims=["track"], name="track")
std_all = xr.concat(std_arrays, dim=track_coord)
std_mean = std_all.mean(dim="track", skipna=True)

ax1.plot(
    std_mean.x.values,
    std_mean.values,
    color="b",
    linewidth=3,
    label="XTRACK",
)

ax1.set_xlabel("Distance (km)")
ax1.set_ylabel("STD (m)")
ax1.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax1.text(
    0.02,
    0.5,
    "coast",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="r",
    rotation=90,
)
ax1.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax1.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax1.legend(title="Track", ncol=4, fontsize=7, title_fontsize=8)

std_arrays = []
track_numbers = []
for pass1 in common:
    f = f"{L2P_PATH}/aviso_l2p_s3a_dist_time_{pass1}_asn.nc"
    with xr.open_dataset(f, decode_times=False) as ds:
        track_number = ds.attrs["pass"]
        lons_track = ds.lons.values
        lats_track = ds.lats.values
        gshhg_da_trim = ds.gshhg_da_trim
        ln = len(lons_track)
        sla_track = ds.sla
        std = sla_track.count(dim="time")
        print(f"{SAT} {track_number} --------------------")
        ax1.scatter(gshhg_da_trim.values, std.values, color='gray', marker='.', s=1)
        std_arrays.append(std)
        track_numbers.append(pass3)

track_coord = xr.DataArray(track_numbers, dims=["track"], name="track")
std_all = xr.concat(std_arrays, dim=track_coord)
std_mean = std_all.mean(dim="track", skipna=True)
ax1.plot(
    std_mean.x.values,
    std_mean.values,
    color="dimgray",
    linewidth=3,
    label="L2P",
)

ax1.set_xlabel("Distance (km)")
ax1.set_ylabel("STD (m)")
ax1.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax1.text(
    0.02,
    0.5,
    "coast",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
```

```
    color="r",
    rotation=90,
)
ax1.set_xlim(0, 1000e3)
ax1.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax1.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax1.legend(title="Track", ncol=4, fontsize=7, title_fontsize=8)

plt.tight_layout()
plt.show()
plt.close("all")

```

computed/p.valid_points.S3Avsl2p.selected.pre_c.py

```python
import numpy as np
import xarray as xr
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter

# Input file produced by your processing script
nc_xtr = "counts_all_S3A_xtrack.nc"
nc_l2p = "/home/srinivasu/allData/aviso_l2p/s3a/counts_all_S3A_l2p.nc"

ds_xtr = xr.open_dataset(nc_xtr)
ds_l2p = xr.open_dataset(nc_l2p)
counts_all_xtr = ds_xtr["counts_all"]
counts_all_l2p = ds_l2p["counts_all"]

# Mean across tracks
counts_mean_xtr = counts_all_xtr.mean(dim="track", skipna=True)
counts_mean_l2p = counts_all_l2p.mean(dim="track", skipna=True)

# Build scatter inputs: every (track, x) point
x_vals_xtr = counts_all_xtr["x"].values
x_scatter_xtr = np.broadcast_to(x_vals_xtr, counts_all_xtr.shape).ravel()
y_scatter_xtr = counts_all_xtr.values.ravel()

x_vals_l2p = counts_all_l2p["x"].values
x_scatter_l2p = np.broadcast_to(x_vals_l2p, counts_all_l2p.shape).ravel()
y_scatter_l2p = counts_all_l2p.values.ravel()

fig, ax = plt.subplots(figsize=(10, 5))

# All track counts points as cyan dots
ax.scatter(x_scatter_xtr, y_scatter_xtr, s=8, c="cyan", alpha=0.6, edgecolors="none")
ax.scatter(x_scatter_l2p, y_scatter_l2p, s=8, c="red", alpha=0.6, edgecolors="none")

# Mean counts as thick cyan solid line
ax.plot(counts_mean_xtr["x"].values, counts_mean_xtr.values, color="b", linewidth=3.5, linestyle="-")
ax.plot(counts_mean_l2p["x"].values, counts_mean_l2p.values, color="r", linewidth=3.5, linestyle="-")

ax.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax.set_xlim(-10e3, 300e3)
ax.set_xlabel("Distance (m)")
ax.set_ylabel("counts (m)")
ax.set_title("All track counts points and mean counts")
ax.grid(True, linestyle="--", alpha=0.3)

plt.tight_layout()
plt.show()

ds.close()

```

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle

sat = "S3A"
cmap = "twilight_shifted"
EAST_COAST_REG = (78.0, 95, 0, 25.4)

dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
fig = plt.figure(figsize=(10, 6))
ax1 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax2 = fig.add_subplot(2, 1, 2)

dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:])).plot(
    ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap
)
decorate_axis(ax1, "", *NIO, step=8)
ax1.text(
    0.45,
    0.80,
    f"MISSION: {sat}",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)

x_common = xr.DataArray(np.arange(0, 1_300_001, 6_675), dims=["x"],
  attrs={"units": "m"})  # 0..1300 km, 1 km step

# open common pickle
with open("common_passes_l2pVSxtrack_selected.pkl", "rb") as f:
    common = pickle.load(f)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
counts_arrays = []
i=0
# exclude_tracks=["679"]
exclude_tracks = ["451", "565", "679", "23", "621"]
track_numbers = []
for pass1 in common:
    pass2 = pass1[1:]
    pass3 = pass1.lstrip('0')
    if pass3 in exclude_tracks:
        continue
    f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass2}.nc"
    ds = xr.open_dataset(f, decode_times=False)
    lon_eq = ds.lon.values[0]
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    ln = len(lons_track)
    lon_equat = lons_track[0]
    lon_coast = lons_track[-1]
    lat_equat = lats_track[0]
    lat_coast = lats_track[-1]
    a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
    a1_d = math.degrees(a1)
    track_number = ds.Pass
    lonm = lons_track.mean()
    latm = lats_track.mean()
    color = colors[i]
    ax1.scatter(lons_track, lats_track, s=2, color=colors[i])
    if is_within_region(lonm, latm, *NIO):
        ax1.text(
            lonm,
            latm,
            s=str(track_number),
            fontsize=7,
            rotation=a1_d,
            color='k',
            ha="center",
            va="center",
            transform=ccrs.PlateCarree(),
        )
    # sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
    sla_asn = track_dist_time_asn_gshhg(ds, var_str="sla", units_in="m")
    # gshhg = ds.dist_to_coast_gshhg.values
    gshhg_da = dist_asn_gshhg(ds, var_str="dist_to_coast_gshhg")
    gshhg_da_interp = gshhg_da.interp(x=x_common)
    counts = sla_asn.count(dim="time")
    count_perc = 100 * counts / len(sla_asn.time)
    count_perc1 = xr.where(count_perc == 0, np.nan, count_perc)
    count_perc_interp = count_perc1.interp(x=x_common)
    count_da = xr.DataArray(count_perc_interp.values, coords={"x": gshhg_da_interp.x.values}, dims=["x"])
    ax2.scatter(gshhg_da_interp.values, count_perc_interp.values, s=1, label=pass2, color=color)
    print(f"{sat} {track_number} --------------------")
    # ax2.scatter(gshhg, count2.values, color=color, s=1, label=str(track_number))
    # ax2.scatter(
    #     count_perc_interp.x.values,
    #     count_perc_interp.values,
    #     marker=".",
    #     s=1.0,
    # )
    counts_arrays.append(count_perc_interp)
    track_numbers.append(pass3)
    i+=1

# counts_mean = xr.concat(counts_arrays, dim="track").mean(dim="track", skipna=True)
# ds_out = xr.Dataset({"counts": counts_mean})
# ds_out.to_netcdf(f"counts_mean_S3A_xtrack.nc")
track_coord = xr.DataArray(track_numbers, dims=["track"], name="track")
counts_all = xr.concat(counts_arrays, dim=track_coord)
counts_mean = counts_all.mean(dim="track", skipna=True)

ds_out2 = xr.Dataset({"counts_all": counts_all})
ds_out2.to_netcdf(f"counts_all_S3A_xtrack.nc")
ds_out1 = xr.Dataset({"counts": counts_mean})
ds_out1.to_netcdf(f"counts_mean_S3A_xtrack.nc")
ax2.plot(
    counts_mean.x.values,
    counts_mean.values,
```

```
    color="dimgray",
    linewidth=3,
    label="Mean",
)

ax2.set_xlabel("Distance (km)")
ax2.set_ylabel("counts (m)")
ax2.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax2.text(
    0.02,
    0.5,
    "coast",
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
    color="r",
    rotation=90,
)
ax2.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax2.set_xlim(-10e3, 300e3)
ax2.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax2.legend(title="Track", ncol=4, fontsize=7, title_fontsize=8)

plt.tight_layout()
plt.show()
plt.close("all")

dse.close()

```

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle

sat = "S3A"
cmap = "twilight_shifted"
EAST_COAST_REG = (78.0, 95, 0, 25.4)
COAST_NEAR_M = 50_000

frac1 = "0.05"
# smooth_dir = f"sla_loess_{frac1}a/"
dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
fig = plt.figure(figsize=(10, 9))
ax1 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
ax2 = fig.add_subplot(3, 1, 2)
ax3 = fig.add_subplot(3, 1, 3)

dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:])).plot(
    ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap
)
decorate_axis(ax1, "", *NIO, step=8)
ax1.text(
    0.45,
    0.80,
    f"MISSION: {sat}",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)

x_common = xr.DataArray(np.arange(0, 1_300_001, 6_675), dims=["x"],
  attrs={"units": "m"})  # 0..1300 km, 1 km step


def valid_percent(da):
    counts = da.count(dim="time")
    count_perc = 100 * counts / da.sizes["time"]
    return xr.where(count_perc == 0, np.nan, count_perc)

# open common pickle
with open("common_passes_l2pVSxtrack.pkl", "rb") as f:
    common = pickle.load(f)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
counts_raw_arrays = []
counts_smooth_arrays = []
coast_raw_track = []
coast_smooth_track = []
i=0
for pass1 in common:
    pass2 = pass1[1:]
    f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass2}.nc"
    ds = xr.open_dataset(f, decode_times=False)
    lon_eq = ds.lon.values[0]
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    ln = len(lons_track)
    lon_equat = lons_track[0]
    lon_coast = lons_track[-1]
    lat_equat = lats_track[0]
    lat_coast = lats_track[-1]
    a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
    a1_d = math.degrees(a1)
    track_number = ds.Pass
    lonm = lons_track.mean()
    latm = lats_track.mean()
    color = colors[i]
    ax1.scatter(lons_track, lats_track, s=2, color=colors[i])
    if is_within_region(lonm, latm, *NIO):
        ax1.text(
            lonm,
            latm,
            s=str(track_number),
            fontsize=7,
            rotation=a1_d,
            color='k',
            ha="center",
            va="center",
            transform=ccrs.PlateCarree(),
        )
    sla_asn = track_dist_time_asn_gshhg(ds, var_str="sla", units_in="m")
    sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
    sla_smooth = track_smooth_loess_statsmodels_x(sla_track, frac=0.05)
    gshhg_da = dist_asn_gshhg(ds, var_str="dist_to_coast_gshhg")
    gshhg_da_interp = gshhg_da.interp(x=x_common)
    count_raw_interp = valid_percent(sla_asn).interp(x=x_common)
    count_smooth_interp = valid_percent(sla_smooth).interp(x=x_common)

    coast_mask = np.abs(gshhg_da_interp.values) <= COAST_NEAR_M
    raw_near = np.nanmean(count_raw_interp.values[coast_mask]) if coast_mask.any() else np.nan
    smooth_near = np.nanmean(count_smooth_interp.values[coast_mask]) if coast_mask.any() else np.nan

    ax2.scatter(
        gshhg_da_interp.values,
        count_raw_interp.values,
        s=3,
        color=color,
        alpha=0.5,
    )
    ax2.scatter(
        gshhg_da_interp.values,
        count_smooth_interp.values,
        s=3,
        color=color,
        alpha=0.5,
        marker="x",
    )
    print(
        f"{sat} {track_number}: near-coast valid% raw={raw_near:.2f}, "
        f"smooth={smooth_near:.2f}, delta={smooth_near - raw_near:.2f}"
```

```
    )

    counts_raw_arrays.append(count_raw_interp)
    counts_smooth_arrays.append(count_smooth_interp)
    coast_raw_track.append(raw_near)
    coast_smooth_track.append(smooth_near)
    i+=1

counts_raw_mean = xr.concat(counts_raw_arrays, dim="track").mean(dim="track", skipna=True)
counts_smooth_mean = xr.concat(counts_smooth_arrays, dim="track").mean(dim="track", skipna=True)
counts_delta_mean = counts_smooth_mean - counts_raw_mean

ds_out = xr.Dataset({
    "counts_raw": counts_raw_mean,
    "counts_smooth": counts_smooth_mean,
    "counts_delta": counts_delta_mean,
})
ds_out.to_netcdf(f"counts_mean_S3A_xtrack_smoothVSraw.nc")

ax2.plot(
    counts_raw_mean.x.values,
    counts_raw_mean.values,
    color="dimgray",
    linewidth=3,
    label="Mean raw",
)
ax2.plot(
    counts_smooth_mean.x.values,
    counts_smooth_mean.values,
    color="tab:blue",
    linewidth=3,
    label=f"Mean smooth (frac={frac1})",
)

ax2.set_xlabel("Distance (km)")
ax2.set_ylabel("% valid points")
ax2.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax2.axvspan(-COAST_NEAR_M, COAST_NEAR_M, color="tab:red", alpha=0.08)
ax2.text(
    0.02,
    0.5,
    "coast",
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
    color="r",
    rotation=90,
)
ax2.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax2.set_xlim(-10e3, 150e3)
ax2.set_ylim(0, 105)
ax2.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax2.legend(fontsize=8, ncol=2)

ax3.plot(
    counts_delta_mean.x.values,
    counts_delta_mean.values,
    color="tab:green",
    linewidth=2,
)
ax3.axhline(y=0, color="k", linewidth=0.8, linestyle="--")
ax3.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax3.axvspan(-COAST_NEAR_M, COAST_NEAR_M, color="tab:red", alpha=0.08)
ax3.set_xlabel("Distance (km)")
ax3.set_ylabel("Smooth - Raw\n(% points)")
ax3.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax3.set_xlim(-10e3, 150e3)
ax3.grid(alpha=0.3, linestyle="--", linewidth=0.5)

coast_raw_mean = float(np.nanmean(coast_raw_track))
coast_smooth_mean = float(np.nanmean(coast_smooth_track))
coast_delta_mean = coast_smooth_mean - coast_raw_mean
ax3.text(
    0.01,
    0.95,
    (
        f"|dist_to_coast| <= {COAST_NEAR_M/1000:.0f} km: "
        f"raw={coast_raw_mean:.2f}%, smooth={coast_smooth_mean:.2f}%, "
        f"delta={coast_delta_mean:.2f}%"
    ),
    transform=ax3.transAxes,
    fontsize=9,
    va="top",
)

print(
    f"ALL TRACKS near-coast (|dist|<={COAST_NEAR_M/1000:.0f} km): "
    f"raw={coast_raw_mean:.2f}%, smooth={coast_smooth_mean:.2f}%, "
    f"delta={coast_delta_mean:.2f}%"
)

plt.tight_layout()
plt.show()
plt.close("all")

dse.close()

```

```python
import glob

import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import sats_new, track_dist_time_asn_gshhg, dist_asn_gshhg
import pickle

fig, ax1 = plt.subplots(1, 1, figsize=(10, 6))

xlim1, xlim2 = (-10e3, 500e3)

x_common = xr.DataArray(np.arange(0, 1_300_001, 6_675), dims=["x"],
  attrs={"units": "m"})  # 0..1300 km, 1 km step

sat = "S3A"
counts_array = []
with open("common_passes_l2pVSxtrack.pkl", "rb") as f:
    common = pickle.load(f)

for pass1 in common:
    pass2 = pass1[1:]
    f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass2}.nc"
    ds = xr.open_dataset(f, decode_times=False)
    track_number = ds.Pass
    sla_asn = track_dist_time_asn_gshhg(ds, var_str="sla", units_in="m")
    gshhg_da = dist_asn_gshhg(ds, var_str="dist_to_coast_gshhg")
    gshhg_da_interp = gshhg_da.interp(x=x_common)
    # print(sla_asn)
    counts = sla_asn.count(dim="time")
    count_perc = 100 * counts / len(sla_asn.time)
    count_perc1 = xr.where(count_perc == 0, np.nan, count_perc)
    count_perc_interp = count_perc1.interp(x=x_common)
    count_da = xr.DataArray(count_perc_interp.values, coords={"x": gshhg_da_interp.x.values}, dims=["x"])
    ax1.scatter(gshhg_da_interp.values, count_perc_interp.values, s=1, label=pass2)
    # ax1.scatter(
    #     count_perc.x.values,
    #     count_perc.values,
    #     marker=".",
    #     s=1.0,
    # )
    # ax1.scatter(
    #     count_perc_interp.x.values,
    #     count_perc_interp.values,
    #     marker=".",
    #     s=1,
    # )
    # counts_array.append(count_perc_interp)
    counts_array.append(count_da)

counts_mean = xr.concat(counts_array, dim="track").mean(dim="track", skipna=True)
ds_out = xr.Dataset({"count1": counts_mean})
ds_out.to_netcdf(f"count1_mean_S3A_xtrack.nc")
counts_mean.plot(ax = ax1)
ax1.set_xlabel("distance from coast")
ax1.set_ylabel("% valid points")
ax1.set_xlim(xlim1, xlim2)
ax1.set_ylim(0, 110)
ax1.axvline(x=0, color='gray', linestyle='--')
ax1.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax1.text(
    0.5,
    0.8,
    f"{sat}",
    transform=ax1.transAxes,
    fontweight="bold",
    color="r",
)
plt.show()

```

```
1
2  (filled by f.xtr.S3A.common_tracks.map.py) and right side has two row subplots for lines plots. Upper subplots is filled by p.std.S3Avsl2p.common_tracks.with_mean.py a
       nd lower subplot filled by p.valid_points.S3Avsl2p.common_tracks.with_mean.py. Make combined script.
3
4  modify following script p.compare.std_meanANDvalid_count.mean.l2pVSxtrack_map.py to
5  1. The cartopy plot should be on the left side
6  2. Left side should include two subplots
7  a) top one plots sla std mean as it is currently in the bottom plot
8  b) bottom new subplot should include a new plot of valid points count comparison
9  I am trying to find the standard deviation of geostrophic currents as
10 a function of distance away from the coast with the script
11 p.std.gc.S3A.east_coast.common_tracks.with_mean.py
12 I am expecting that the std is more near coast. But it is not the case.
13 For many tracks the std is very high in the middle of the ocean.
14 Computational or scientific issues?
15
16 I am trying to code optimal difference operator discussed in
17  Powell_Leben_2004_optimal_difference_operator.pdf and
18 'Liu et al.-2012-Comparison of the X-TRACK altimetry estimated currents with moored ADCP and HF radar observations on the West Florida Shelf.pdf'
19 in the script c.odo_smooth.one.py
20 This returns smoothed slopes of sla from which we can compute geostrophic
21 currents.
22 What issues do you find the script c.odo_smooth.one.py?
23
24 Extend c.msla_uvVSguv_TP+J1+J2+J3+S6A.py script to find metrics like corr,
25 rmse, bias, number of good point(matching points) for msla_uv and guv at
26 each unique_str and save them to csv file.
27 Write a seperate function to do a 2x2 bar plot of metrics for all unique_str.
28 Following modifications are needed
29 1. Stick to only u metrics in the bar plot for all subplots
30 2. For rmse plot, plot std of msla_uv as adjacent bar
31 3. Don't put those monstrous unique_str as xtick labels, just index numbers good
32
33 I am comparing the performance of several combination of smoothing and
34 parameters. Write a dedicated bar plot script that can take any number of dfs
35 and plot adjacent bars of a metric from different dfs.
36
37 Based on the attached documents, code optimal difference operator in python
38 as a function. p, q parameters should be function inputs.
39 It should smooth the data along the points_numbers dimensions of xtrack
40 coastal altimetry data.
41 see meta_data
42
43 Smoothing validation:
44 We need to test different methods of smoothing (guassian, box smooth, loess, optim) with different parameters.
45 Objective is to test which smoothing method and what parameters is best
46 for reproduing components of geostrophic currents.
47 1. First we take a smoothing method like guassian
48 At each intersection point, we take track_self and track_other
49 2. track_dist_time_asn to x as coordinate
50 3. smooth them
51 4. compute gc_self and gc_other and then compute gu, gv components
52 save them at temp location
53 The above four steps will make one function
54 see ../guv/c.guv_at_intersection_points_0.05.py
55 smoothing functions are in
56 ../lib/tools_xtrackm.py
57 5. compare them with extracted msla_uv at intersection points
58 see the dir msla_uv_at_intersections
59 6. save metrics like corr, rmse, bias etc to a .csv file with smoothin method, paramters used etc.
60 7. We do this exercise for all smoothing methods and various paramters
61 The above four steps make one function
62 8. In the we bar plot and see combination is doing best.
63 Last plot is another function
64
65 modify p.std.S3A.east_coast.all_tracks.with_mean.py so that bottom subplot
66 now includes the mean of all std plots with thick gray line.
67
```

```
1
2  Comparing with msla uv extracted at TP+J1+J2+J3+S6A intersection points
3  ----------------------------
4
5  c.guv_at_intersection_points_box_12.py
6  compute guv at TP+J1+J2+J3+S6A intersection points for different combinations of smoothing method and parameters
7
8  c.odo_smooth.and.guv.all.py
9  This script is specially for optimal difference operator, since it both smoothes computes slope (and guv) at the same time.
10
11 c.msla_uvVSguv_TP+J1+J2+J3+S6A.py
12 compute metrics comparing with msla uv extracted at TP+J1+J2+J3+S6A intersection points and guv computed with specific combination of smoothing
13 method and parameters
14
15 p.msla_uv.guv.metrics.general.py
16 plot a comparison metric like corr_u for different combinations of smoothing method and parameters
17
```

```
./computed/archive/
```

```
1 file(s)
```

```python
import sys

import numpy as np
import xarray as xr
from tools_xtrackd import *
import os


def extract_oscar_xy(ds, da_oscar):
    dists = compute_dist(ds, units="m")  # to convert to meters
    dists_rev = dists[::-1]
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    times = da_oscar.time.values
    vals = np.full((len(lons_track_rev), len(times)), np.nan)
    vals[:, :] = da_oscar.sel(lon=ds.lon, lat=ds.lat, method="nearest").values.T
    var2 = xr.DataArray(
        vals,
        coords=[dists_rev, times],
        dims=["x", "time"],
    )
    return var2


ds_oscar = xr.open_dataset("/home/srinivasu/allData/oscar_0.25/oscar_0.25_u_full_xr_concat.nc")
print(ds_oscar)
oscar_u = ds_oscar.u

# sats = ["S3A"]
for sat in sats[:]:
    for f in sorted(
            glob.glob(
                f"../data/ctoh.sla.ref.{sat}.nindian.*.nc"
            )):
        ds = xr.open_dataset(f, decode_times=False)
        # print(ds)
        track_number = ds.pass_number
        fe = f"oscar_e_py/oscar_along_{sat}_{track_number}_u.nc"
        # check if fe exists
        if os.path.exists(fe):
            print(fe)
            continue
        u_oscar_e = extract_oscar_xy(ds, oscar_u)
        # u_oscar_e = oscar_u.sel(lon=ds.lon, lat=ds.lat, method="nearest")
        # v_oscar_e = extract_oscar_xy(ds, oscar_v)
        # s_oscar_e = extract_oscar_xy(ds, oscar_s)
        ds_out = xr.Dataset({
            "u_oscar_e": u_oscar_e,
        })
        ds_out.to_netcdf(
            f"oscar_e_py/oscar_along_{sat}_{track_number}_u.nc")

sys.exit()
# ds_out = xr.Dataset({
#     "u_oscar_e": u_oscar_e,
#     "v_oscar_e": v_oscar_e,
#     "s_oscar_e": s_oscar_e,
# })

```

```
./computed/odo/
```

4 file(s)

```
1
2  1. The window shape (weights) are fixed for given p and q.
3  It is computed only once.
4  2. if p, q are roughly equal the window shape is symmetric.
5  3. if p is say 7 and q is 1, then the window shape is very asymmetric.
6
```

```python
import xarray as xr
import os
from tools_xtrackm import *
import numpy as np
import matplotlib.pyplot as plt
import sys

method = "odo"


def compute_geostrophy_gc_from_slope(ds, slope_smooth):
    lons_track_rev = ds.lon.values[::-1]
    lats_track_rev = ds.lat.values[::-1]
    if len(lons_track_rev) == 0:
        return None
    # Add minimum length check
    if len(lons_track_rev) < 3:
        print("Warning: Insufficient data points for gradient calculation")
        return None
    f1 = cor_p(lats_track_rev)
    f2 = f1.values
    f3 = np.where(np.abs(lats_track_rev) > 2, f2, np.nan)
    # 2s-2n band equatorial band is masked according to Sudre 2013
    g = grav(lats_track_rev)  # ms-1
    g1 = g.values
    ratio = g1 / f3
    ratio1 = ratio[:, np.newaxis]
    gc = ratio1 * slope_smooth
    # Add metadata
    gc.attrs["units"] = "m/s"
    gc.attrs["long_name"] = "Cross-track geostrophic current"
    gc.attrs["description"] = "Geostrophic current perpendicular to satellite track"
    return gc


def track_dist_time_asn_midx_da(ds, da_smooth, units_in="m"):
    # dists = compute_dist(ds, units="km")  # to convert to meters
    dists = compute_dist(ds, units=units_in)  # to convert to meters
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    dt0 = datetime.strptime("1950-01-01", "%Y-%m-%d")
    dists_rev = dists[::-1]
    ln = ds["sla"].sizes["points_numbers"]
    ln2 = ln // 2
    dvals = ds.time.isel(points_numbers=ln2).values
    try:
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    except ValueError:
        dvals = pd.Series(dvals).ffill().bfill()
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    # now sla values oriented from coast
    var1_rev = da_smooth.load().isel(points_numbers=slice(None, None, -1)).values
    var2 = xr.DataArray(
        var1_rev,
        dims=["x", "time"],
        coords={"x":dists_rev, "time":dates, "lon":("x", lons_track_rev), "lat":("x", lats_track_rev)}
    )
    if units_in == "m":
        var2.coords["x"].attrs["units"] = "m"
    elif units_in == "km":
        var2.coords["x"].attrs["units"] = "km"
    return var2


"""
Applies the Powell & Leben (2004) optimal difference operator to an
X-TRACK dataset along the 'points_numbers' dimension.
"""
def get_optimal_weights(p, q, dt):
    """Derives coefficients cn using Lagrange multipliers."""
    p_abs, q_abs = abs(p), abs(q)
    N = p_abs + q_abs
    T = N + 1
    # Relative indices excluding the current point (0)
    n = np.array([i for i in range(-p_abs, q_abs + 1) if i != 0])
    # Build Lagrange Matrix A [cite: 471-478]
    A = np.zeros((T, T))
    A[N, :N] = 1.0  # Constraint: sum(cn) = 1 [cite: 58, 693]
    for i in range(N):
        # c_n derivation from the Lagrange function [cite: 85, 477]
        # Match Powell/Leben MATLAB form:
        # A(i, 1:N) = (1./n) .* (-n(i)/2)
        A[i, :N] = (1.0 / n) * (-n[i] / 2.0)
        A[i, N] = (n[i]**2 * dt**2) / 4.0
        A[i, i] = -1.0
    B = np.zeros(T)
    B[N] = 1.0
    # Solve for coefficients [cite: 88, 483]
    results = np.linalg.solve(A, B)
    return results[:N], n


p=-5
q=6
dt=1.0
para = f"{abs(p)}{abs(q)}"

os.makedirs(f"sla_{method}_{para}", exist_ok=True)

sat = "GFO"
pass1 = "008"
f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass1}.nc"
ds = xr.open_dataset(f, decode_times=False)
track_number = ds.Pass
print(f"{sat} {track_number} -------------------")
# Use physical along-track spacing (meters) so ODO slope has SI-consistent units.
dists_m = np.asarray(compute_dist(ds, units="m"), dtype=float)
dx_m = float(np.nanmedian(np.abs(np.diff(dists_m))))

# 1. Calculate the weights
cn, n_indices = get_optimal_weights(p, q, dt)
# plt.plot(n_indices, cn)
# plt.show()
# 2. Extract the SLA variable [cite: 1422]
# Dimensions: (points_numbers: 210, cycles_numbers: 186)
```

```
sla = ds.sla.values
points_dim, cycles_dim = sla.shape
# Initialize output array with NaNs [cite: 1422]
slopes = np.full(sla.shape, np.nan)
# 3. Apply operator along the points_numbers dimension
# Window constraints to avoid index errors at boundaries [cite: 161]
p_abs = abs(p)
q_abs = abs(q)
for i in range(p_abs, points_dim - q_abs):
    # Calculate derivative: sum( (cn / (n*dt)) * (h_{i+n} - h_i) ) [cite: 55, 690]
    # This computes the across-track geostrophic velocity anomaly [cite: 700]
    h_i = sla[i, :]
    h_rel = sla[i + n_indices, :]  # Shape: (N, cycles_numbers)
    # Element-wise slope calculation for all cycles at once
    # Slope = sum_{n} [ cn * (h_{i+n} - h_i) / (n * dt) ]
    window_slopes = (h_rel - h_i) / (n_indices[:, np.newaxis] * dt)
    print(window_slopes)
    # Use nansum so that NaN in part of the window doesn't kill the whole estimate;
    # renormalize weights to account for missing points.
    valid = ~np.isnan(window_slopes)
    w = cn[:, np.newaxis] * valid  # zero out weights for NaN slots
    w_sum = np.sum(np.abs(w), axis=0)
    w_sum[w_sum == 0] = np.nan  # avoid division by zero
    cn_sum = np.sum(np.abs(cn))
    slopes[i, :] = np.nansum(cn[:, np.newaxis] * window_slopes, axis=0) * (cn_sum / w_sum)
    break

```

computed/odo/plot_odo_window_shapes.py

```python
#!/usr/bin/env python3
"""Plot ODO filtering-window shapes for multiple (p, q) combinations."""

import argparse
import itertools
import numpy as np
import matplotlib.pyplot as plt


def get_optimal_weights(p: int, q: int, dt: float = 1.0):
    """Derive Powell & Leben ODO coefficients c_n."""
    p_abs, q_abs = abs(p), abs(q)
    N = p_abs + q_abs
    T = N + 1
    n = np.array([i for i in range(-p_abs, q_abs + 1) if i != 0], dtype=float)

    A = np.zeros((T, T), dtype=float)
    A[N, :N] = 1.0
    for i in range(N):
        A[i, :N] = (1.0 / n) * (-n[i] / 2.0)
        A[i, N] = (n[i] ** 2 * dt ** 2) / 4.0
        A[i, i] = -1.0

    B = np.zeros(T, dtype=float)
    B[N] = 1.0
    results = np.linalg.solve(A, B)
    return results[:N], n


def parse_int_list(values: str):
    return [int(v.strip()) for v in values.split(",") if v.strip()]


def main():
    parser = argparse.ArgumentParser(
        description="Plot ODO filtering-window shape for different (p, q) combinations.")
    parser.add_argument("--p", default="3,5", help="Comma-separated p values (example: 3,5,7)")
    parser.add_argument("--q", default="3,5", help="Comma-separated q values (example: 3,5,7)")
    parser.add_argument("--dt", type=float, default=1.0, help="Sampling interval dt")
    parser.add_argument("--output", default="odo_filter_windows.png", help="Output figure path")
    parser.add_argument("--show", action="store_true", help="Show figure interactively")
    args = parser.parse_args()

    p_values = parse_int_list(args.p)
    q_values = parse_int_list(args.q)

    if not p_values or not q_values:
        raise ValueError("Both --p and --q must include at least one integer.")

    combos = list(itertools.product(p_values, q_values))

    fig, ax = plt.subplots(figsize=(10, 6))
    for p, q in combos:
        cn, n = get_optimal_weights(-abs(p), abs(q), args.dt)
        ax.plot(n, cn, marker="o", linewidth=1.5, label=f"p={-abs(p)}, q={abs(q)}")

    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xlabel("Relative index n")
    ax.set_ylabel("Weight c_n")
    ax.set_title(f"ODO filtering-window shape (dt={args.dt})")
    ax.grid(True, alpha=0.3)
    ax.legend(ncol=2, fontsize=9)
    fig.tight_layout()
    fig.savefig(args.output, dpi=200)

    if args.show:
        plt.show()

    print(f"Saved plot: {args.output}")
    print("Combinations:", ", ".join([f"(p={-abs(p)},q={abs(q)})" for p, q in combos]))


if __name__ == "__main__":
    main()

```

```python
import numpy as np
import sys

p = -5
q = 6
dt: float = 1.0
p_abs, q_abs = abs(p), abs(q)
N = p_abs + q_abs
T = N + 1
# indices from -p to q excluding 0
# Relative indices excluding the current point (0)
n = np.array([i for i in range(-p_abs, q_abs + 1) if i != 0], dtype=float)
# Build Lagrange Matrix A [cite: 471-478]
A = np.zeros((T, T), dtype=float)
# to make bottom rows one; except the last element of bottom row
A[N, :N] = 1.0
# print(A)
for i in range(N):
    # c_n derivation from the Lagrange function [cite: 85, 477]
    # assinging all elements of each row (except last element)
    # the RHS is vector of size N where A's size is N+1 by N+1
    A[i, :N] = (1.0 / n) * (-n[i] / 2.0)
    # print(i, (1.0 / n)*(-n[i] / 2.0))
    # assinging last element of each row
    A[i, N] = (n[i] ** 2 * dt ** 2) / 4.0
    print((n[i] ** 2 * dt ** 2) / 4.0)
    A[i, i] = -1.0

B = np.zeros(T, dtype=float)
B[N] = 1.0
# print(B)
results = np.linalg.solve(A, B)
weights = results[:N]
print(weights)
sys.exit()

```

```
./data/
```

```
4 file(s)
```

data/download_xtrack.py

```python
import ftplib
import os

avuser = "srinivasu.u@incois.gov.in"
avpass = "B9FQgr"

ftpsite = "ftp-access.aviso.altimetry.fr"
print(dir(ftplib))

zone = "NINDIAN"
version = "v2.2"
path_old = f"/regional-xtrack-coastal/version_xtrack_l2p_2022"
path_new = f"/regional-xtrack-coastal/version_x_track_l2p_2022"
sats = ["GFO", "TP+J1+J2+J3", "S3A", "TPN+J1N+J2N", "ERS1+ERS2+ENV+SRL", "HY2"]

sats_new = ["GFO", "TP+J1+J2+J3+S6A", "S3A", "S3B", "TPN+J1N+J2N+J3N", "ERS1+ERS2+ENV+SRL", "HY2A", "HY2B"]
# sats_new = ["TP+J1+J2+J3+S6A", "S3A", "S3B", "TPN+J1N+J2N+J3N", "ERS1+ERS2+ENV+SRL", "HY2A", "HY2B"]

ftp = ftplib.FTP(ftpsite)
ftp.login(avuser, avpass)
print(ftp.getwelcome())
for sat in sats_new:
    # if sat directory does not exist create it
    if not os.path.exists(f"{sat}"):
        os.makedirs(f"{sat}")
    cddir = (
        f"{path_new}/{version}/{zone}/{sat}/SLA"
    )
    ftp.cwd(cddir)
    # ftp.retrlines('LIST')
    flist = ftp.nlst()
    for f in flist:
        print(f)
        f_out = f"{sat}/{f}"
        # if not os.path.exists(f_out) and f.endswith(".lzma"):
        if not os.path.exists(f_out):
            print(f_out, "is not here")
            with open(f_out, "wb") as fh:
                ftp.retrbinary("RETR " + f, fh.write)

ftp.quit()

# drwxr-xr-x   3 10845    100          1035 May 31 14:01 GFO
# drwxr-xr-x   3 10845    100          1035 May 31 14:02 TP+J1+J2+J3
# drwxr-xr-x   3 10845    100          1035 May 31 14:02 S3A
# drwxr-xr-x   3 10845    100          1035 May 31 14:02 TPN+J1N+J2N
# drwxr-xr-x   3 10845    100          1035 May 31 14:02 HY2
# drwxr-xr-x   3 10845    100          1035 Jul  3 12:30 ERS1+ERS2+ENV+SRL
#

```

data/link_raw_nonzero_files_orderbylon.py

```python
import glob
import os

import xarray as xr
from namesm import sats_new
from tools_xtrackm import *

for sat in sats_new[:]:
    os.makedirs(f"{sat}_lon_ordered", exist_ok=True)
    for f in sorted(
            glob.glob(
                f"/home/srinivasu/xtrackm/data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
            )):
        f_base = os.path.basename(f)
        ds = xr.open_dataset(f, decode_times=False)
        trackn = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            print(f, "zero length")
            continue
        dvals = ds.time.isel(points_numbers=1).values
        lon_coast = lons_track[-1]
        lat_coast = lats_track[-1]
        lon_equat = lons_track[0]
        lat_equat = lats_track[0]
        if not is_monotonic(dvals):
            print(f, "not monotonic")
            continue
        # lon_track_equat_str = str(int(lon_equat*1000))
        lon_equat_str = str(int(lon_equat * 10000))
        m = (lat_coast - lat_equat) / (lon_coast - lon_equat)
        if m > 0:
            f_new = f"{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.{lon_equat_str}.ascending.nc"
            print(f, f_new)
        else:
            f_new = f"{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.{lon_equat_str}.descending.nc"
            print(f, f_new)
        if not os.path.exists(f_new):
            # pass
            os.symlink(f, f_new)
        else:
            # pass
            os.remove(f_new)
            os.symlink(f, f_new)
            # print(f,f_new)
        # break
    # break

```

data/link_raw_nonzero_files_orderbylon1.py

```python
import glob
import os

import xarray as xr
from namesm import sats_new
from tools_xtrackm import *

for sat in sats_new[:]:
    os.makedirs(f"{sat}_lon_ordered", exist_ok=True)
    for f in sorted(
            glob.glob(
                f"/home/srinivasu/xtrackm/data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
            )):
        f_base = os.path.basename(f)
        ds = xr.open_dataset(f, decode_times=False)
        trackn = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            print(f, "zero length")
            continue
        dvals = ds.time.isel(points_numbers=1).values
        lon_coast = lons_track[-1]
        lat_coast = lats_track[-1]
        lon_equat = lons_track[0]
        lat_equat = lats_track[0]
        if not is_monotonic(dvals):
            print(f, "not monotonic")
            continue

```

data/workflow.txt

```
1
2
```

```
./draft_figs02/
```

```
21 file(s)
```

draft_figs02/c.rossby_radius_L_R.py

```python
import numpy as np

def rossby_radius_nio(latitude, c=2.7):
    """
    Computes the first baroclinic Rossby radius of deformation (in km)
    as a function of latitude for Northern Indian Ocean conditions.
    Parameters:
    -----------
    latitude : float or numpy.ndarray
        Latitude in degrees (Positive for North, e.g., 0 to 30 for NIO).
    c : float
        Phase speed of the first baroclinic mode in m/s.
        Default is 2.7 m/s, typical for the Arabian Sea / Bay of Bengal.
    Returns:
    --------
    L_R_km : float or numpy.ndarray
        Rossby radius of deformation in kilometers.
    """
    # Geophysical constants
    omega = 7.2921e-5     # Earth's angular velocity in rad/s
    R_earth = 6371000.0   # Earth's mean radius in meters
    # Convert latitude to radians and take absolute value
    # (dynamics are symmetric across the equator)
    lat_rad = np.radians(np.abs(latitude))
    # Coriolis parameter (f)
    f = 2 * omega * np.sin(lat_rad)
    # Equatorial beta parameter (change in Coriolis parameter with latitude at equator)
    beta = 2 * omega / R_earth
    # Calculate Equatorial Rossby Radius (for regions very close to the equator)
    # R_eq = sqrt(c / beta)
    R_eq = np.sqrt(c / beta)
    # Calculate Mid-latitude Rossby Radius
    # To avoid division by zero at exactly 0 degrees, we mask the array
    # If a scalar is passed, we convert it to an array for safe handling
    lat_array = np.atleast_1d(latitude)
    f_array = np.atleast_1d(f)
    L_R = np.zeros_like(lat_array, dtype=float)
    for i in range(len(lat_array)):
        if np.abs(lat_array[i]) < 2.0:
            # Within ~2 degrees of the equator, the standard f-plane approximation
            # breaks down. We use the equatorial deformation radius instead.
            L_R[i] = R_eq
        else:
            # Standard mid-latitude formula: L_R = c / |f|
            L_R[i] = c / np.abs(f_array[i])
    # Convert from meters to kilometers
    L_R_km = L_R / 1000.0
    # Return scalar if input was scalar
    return L_R_km[0] if np.isscalar(latitude) else L_R_km

# --- Example Usage ---
if __name__ == "__main__":
    # Example 1: Calculate at a specific latitude (e.g., 15°N, middle of Arabian Sea)
    lat_15 = 15.0
    radius_15 = rossby_radius_nio(lat_15)
    print(f"Rossby Radius at {lat_15}°N: {radius_15:.1f} km")
    # Example 2: Array of latitudes across the Northern Indian Ocean
    lats = np.array([5, 10, 15, 20, 25])
    radii = rossby_radius_nio(lats)
    print("Latitudes (°N):", lats)
    print("Radii (km):    ", np.round(radii, 1))

```

```
1
2
```

```python
import math
import pickle

import matplotlib.gridspec as gridspec
import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
from pathlib import Path
import sys

SAT = "S3A"
L2P_PATH = "/home/srinivasu/allData/aviso_l2p/s3a/"
EAST_COAST_REG = (78.0, 95, 0, 25.4)
cmap = "twilight_shifted"

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

with open("common_passes_l2pVSxtrack_selected.pkl", "rb") as f:
    common = pickle.load(f)

trim_lim = 300
x_common = xr.DataArray(
    np.arange(trim_lim) * 6_675,
    dims=["x"],
    attrs={"units": "m"},
)

# ── figure layout: map left (2 rows), two line plots right ──
fig = plt.figure(figsize=(16, 8))
gs = gridspec.GridSpec(2, 2, width_ratios=[1, 1.3], wspace=0.05, hspace=0.3)

ax_map = fig.add_subplot(gs[:, 0], projection=ccrs.PlateCarree())
ax_std = fig.add_subplot(gs[0, 1])
ax_vp = fig.add_subplot(gs[1, 1])

# ═══════════════════════════════════════════════════════
# LEFT PANEL — track map
# ═══════════════════════════════════════════════════════
dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*EAST_COAST_REG[:2])).sel(
    ETOPO05_Y=slice(*EAST_COAST_REG[2:])
)
rose_cut.plot(ax=ax_map, add_colorbar=False, add_labels=False, cmap=cmap)
decorate_axis(ax_map, "", *EAST_COAST_REG, step=4)
ax_map.text(
    0.25,
    0.90,
    f"MISSION: {SAT}",
    transform=ax_map.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
for i, pass1 in enumerate(common):
    pass2 = pass1[1:]
    fn = f"../data/{SAT}/ctoh.sla.ref.{SAT}.nindian.{pass2}.nc"
    with xr.open_dataset(fn, decode_times=False) as ds:
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        ln = len(lons_track)
        mid_i = ln // 2
        lonm, latm = lons_track[mid_i], lats_track[mid_i]
        a1_d = math.degrees(
            local_angle(lons_track, lats_track, mid_i, ioff=3, degrees=False)
        )
        ax_map.scatter(lons_track, lats_track, s=48, color="w")
        if is_within_region(lonm, latm, *NIO):
            ax_map.text(
                lonm,
                latm,
                s=str(track_number),
                fontsize=12,
                rotation=a1_d,
                color="r",
                ha="center",
                va="center",
            )
dse.close()

# ═══════════════════════════════════════════════════════
# RIGHT-TOP — SLA standard deviation vs distance from coast
# ═══════════════════════════════════════════════════════
std_arrays_xt, std_arrays_l2p = [], []
track_numbers_xt, track_numbers_l2p = [], []

for pass1 in common:
    pass2 = pass1[1:]
    pass3 = pass1.lstrip("0")
    fn = f"../data/{SAT}/ctoh.sla.ref.{SAT}.nindian.{pass2}.nc"
    ds = xr.open_dataset(fn, decode_times=False)
    sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
    sla_track = sla_track.interp(x=x_common)
    std = sla_track.std(dim="time")
    ax_std.scatter(std.x.values, std.values, color="cyan", marker=".", s=1)
    std_arrays_xt.append(std)
    track_numbers_xt.append(pass3)

tc = xr.DataArray(track_numbers_xt, dims=["track"], name="track")
std_mean_xt = xr.concat(std_arrays_xt, dim=tc).mean(dim="track", skipna=True)
ax_std.plot(
    std_mean_xt.x.values,
    std_mean_xt.values,
    color="b",
    linewidth=3,
    label="XTRACK",
)

for pass1 in common:
    fn = f"{L2P_PATH}/aviso_l2p_s3a_dist_time_{pass1}_asn.nc"
    with xr.open_dataset(fn, decode_times=False) as ds:
        std = ds.sla.std(dim="time")
```

```
        ax_std.scatter(std.x.values, std.values, color="gray", marker=".", s=1)
        std_arrays_l2p.append(std)
        track_numbers_l2p.append(ds.attrs["pass"])

tc = xr.DataArray(track_numbers_l2p, dims=["track"], name="track")
std_mean_l2p = xr.concat(std_arrays_l2p, dim=tc).mean(dim="track", skipna=True)
ax_std.plot(
    std_mean_l2p.x.values,
    std_mean_l2p.values,
    color="dimgray",
    linewidth=3,
    label="L2P",
)

ax_std.set_xlabel("Distance (km)")
ax_std.set_ylabel("STD (m)")
ax_std.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax_std.text(
    0.02,
    0.5,
    "coast",
    transform=ax_std.transAxes,
    fontsize=10,
    fontweight="bold",
    color="r",
    rotation=90,
)
ax_std.set_xlim(0,1000e3)
ax_std.set_ylim(0.05, 0.25)
ax_std.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax_std.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax_std.legend(ncol=2, fontsize=8)

# ═══════════════════════════════════════════════════════
# RIGHT-BOTTOM — valid point count vs GSHHG distance from coast
# ═══════════════════════════════════════════════════════
cnt_arrays_xt, cnt_arrays_l2p = [], []
track_numbers_xt2, track_numbers_l2p2 = [], []

for pass1 in common:
    pass2 = pass1[1:]
    pass3 = pass1.lstrip("0")
    fn = f"../data/{SAT}/ctoh.sla.ref.{SAT}.nindian.{pass2}.nc"
    ds = xr.open_dataset(fn, decode_times=False)
    sla_track = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")
    sla_track = sla_track.interp(x=x_common)
    gshhg_track = dist_asn_gshhg(
        ds, var_str="dist_to_coast_gshhg", units_in="m"
    )
    gshhg_interp = gshhg_track.interp(x=x_common)
    cnt = sla_track.count(dim="time")
    ax_vp.scatter(
        gshhg_interp.values, cnt.values, color="cyan", marker=".", s=1
    )
    cnt_arrays_xt.append(cnt)
    track_numbers_xt2.append(pass3)

tc = xr.DataArray(track_numbers_xt2, dims=["track"], name="track")
cnt_mean_xt = xr.concat(cnt_arrays_xt, dim=tc).mean(dim="track", skipna=True)
ax_vp.plot(
    cnt_mean_xt.x.values,
    cnt_mean_xt.values,
    color="b",
    linewidth=3,
    label="XTRACK",
)

for pass1 in common:
    fn = f"{L2P_PATH}/aviso_l2p_s3a_dist_time_{pass1}_asn.nc"
    with xr.open_dataset(fn, decode_times=False) as ds:
        cnt = ds.sla.count(dim="time")
        ax_vp.scatter(
            ds.gshhg_da_trim.values, cnt.values, color="gray", marker=".", s=1
        )
        cnt_arrays_l2p.append(cnt)
        track_numbers_l2p2.append(ds.attrs["pass"])

tc = xr.DataArray(track_numbers_l2p2, dims=["track"], name="track")
cnt_mean_l2p = xr.concat(cnt_arrays_l2p, dim=tc).mean(dim="track", skipna=True)
ax_vp.plot(
    cnt_mean_l2p.x.values,
    cnt_mean_l2p.values,
    color="dimgray",
    linewidth=3,
    label="L2P",
)

ax_vp.set_xlabel("GSHHG Distance from coast (km)")
ax_vp.set_ylabel("Valid points (count)")
ax_vp.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax_vp.text(
    0.02,
    0.5,
    "coast",
    transform=ax_vp.transAxes,
    fontsize=10,
    fontweight="bold",
    color="r",
    rotation=90,
)
ax_vp.set_xlim(0, 1000e3)
ax_vp.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax_vp.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax_vp.legend(ncol=2, fontsize=8)

# ── panel labels ──
ax_map.set_title("(a)", loc="left", fontweight="bold")
ax_std.set_title("(b)", loc="left", fontweight="bold")
ax_vp.set_title("(c)", loc="left", fontweight="bold")

plt.tight_layout()
# plt.savefig("f.combined_map_std_valid.S3A.png", dpi=300)
plt.savefig(f"{png_file}", dpi=300)
plt.show()
plt.close("all")

```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
import xskillscore as xs
from geopy import distance
from tools_xtrackm import *
from pathlib import Path
import sys

# codex resume 019cc947-b006-7d82-8941-d68e330c388a

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
aviso_msla_uv_dir = "/home/srinivasu/sealevel/data/msla_uv/"
aviso_msla_uv_dir = "/home/srinivasu/allData/aviso/msla_uv/"
ds_aviso_u = xr.open_dataset(f"{aviso_msla_uv_dir}/allyear_dt_global_allsat_msla_u.nc")
ds_aviso_v = xr.open_dataset(f"{aviso_msla_uv_dir}/allyear_dt_global_allsat_msla_v.nc")
print(ds_aviso_u)
aviso_u = ds_aviso_u.u
aviso_v = ds_aviso_v.v
script = Path(sys.argv[0])
png_file = script.with_suffix(".png")
method = "loess"

@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(
        f_track, engine="netcdf4", decode_times=False
    ) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


def build_unique_str(sat, row):
    track_self = track_to_str(row["track_self"])
    track_other = track_to_str(row["track_other"])
    return f"{sat}_{track_self}_{track_other}"


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)


cmap1 = cmo.cm.diff
d = 1.5
dpth = 15
height, width = 9, 9
frac1 = "0.05"


dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
omni_data_dir = "/home/srinivasu/allData/MB_cleaned/raju_omni_clean"
guv_dir = f"/home/srinivasu/xtrackm/guv/guv_at_intersections_{method}"
out_match_csv = "omni_guv_c_nearest_global_matches.csv"
matched_rows_all = []
# SELECT_SAT = "TP+J1+J2+J3+S6A"
# SELECT_OMNI_ID = "BD09"
# SELECT_TRACK_SELF = "53"
# SELECT_TRACK_OTHER = "14"
SELECT_SAT = 'S3A'
SELECT_OMNI_ID = 'BD10'
SELECT_TRACK_SELF = '522'
SELECT_TRACK_OTHER = '279'
SELECT_SAT = 'S3A'
SELECT_OMNI_ID = 'BD10'
SELECT_TRACK_SELF = '636'
SELECT_TRACK_OTHER = '393'

EKMAN_DIR = "/home/srinivasu/xtrackm/ekman/ekman_at_intersections_cd"
EKMAN_DEPTH = "10"
# TP+J1+J2+J3+S6A BD09 53 14
# S3A BD10 522 279

df_all_list = []
for sat in sats_new:
    df_all_list.append(pd.read_csv(f"tracks_intersections_{sat}_1.csv"))
df_all = pd.concat(df_all_list, ignore_index=True)


lon_omni, lat_omni = omni_d[SELECT_OMNI_ID]
omni_file = f"{omni_data_dir}/{SELECT_OMNI_ID}_current_sst.nc"
print(omni_file)
if not os.path.exists(omni_file):
    print(f"File not found: {omni_file}")
    raise(f"no omni file")
if abs(lat_omni) < 2:
    raise(f"lat less than 2")

ds_omni = xr.open_dataset(omni_file)
print(ds_omni)
# print(ds_omni)
ds_omni = xr.open_dataset(omni_file)
print(ds_omni)
omni_u = ds_omni.u_current.interp(current_depth=dpth)
```

```python
# omni_u = omni_u.rolling(time=10, center=True).mean()
omni_u = omni_u
omni_v = ds_omni.v_current.interp(current_depth=dpth)
# omni_v = omni_v.rolling(time=10, center=True).mean()
omni_v = omni_v
N = omni_v.count().item()

df_selected = df_all[
    (df_all["sat"] == SELECT_SAT)
    & (df_all["track_self"].astype(str) == SELECT_TRACK_SELF)
    & (df_all["track_other"].astype(str) == SELECT_TRACK_OTHER)
].copy()
if df_selected.empty:
    raise("selected sat/track combination not found in intersections file")
lons = df_selected["lons_inter"].to_numpy(dtype=float)
lats = df_selected["lats_inter"].to_numpy(dtype=float)
distances = [distance.distance((lat, lon), (lat_omni, lon_omni)).km for lat, lon in zip(lats, lons)]
df_selected["distance_km"] = distances
row = df_selected.nsmallest(1, "distance_km").iloc[0]

sat = row["sat"]
track_number_self = str(row["track_self"])
track_number_other = str(row["track_other"])
lon1 = row["lons_inter"]
lat1 = row["lats_inter"]
msla_u_at = aviso_u.interp(longitude=lon1, latitude=lat1)
distance_km = row["distance_km"]
angle_acute = row["angle_acute"]
angle_obtuse = row["angle_obtuse"]
unique_str = build_unique_str(sat, row)
f_ek = f"{EKMAN_DIR}/{unique_str}_ekman_{EKMAN_DEPTH}m_at_intersections_cd.nc"
ds_ek = xr.open_dataset(f_ek)
u_ek = ds_ek['u_at']
v_ek = ds_ek['v_at']

print(
    sat,
    SELECT_OMNI_ID,
    track_number_self,
    track_number_other,
    lon1,
    lat1,
    angle_acute,
    angle_obtuse,
)

f_guv = f"{guv_dir}/guv_at_intersection_{sat}_{track_number_self}_{track_number_other}_{method}_{frac1}.nc"
if not os.path.exists(f_guv):
    print(f"missing file: {f_guv}")
    raise("missing file")

with xr.open_dataset(f_guv) as ds_guv:
    gu = ds_guv.gu
    gv = ds_guv.gv

(
    lons_track_self,
    lats_track_self,
    lon_equat_self,
    lat_equat_self,
    lon_coast_self,
    lat_coast_self,
    angle_self,
) = get_track_geometry(sat, track_number_self)
(
    lons_track_other,
    lats_track_other,
    lon_equat_other,
    lat_equat_other,
    lon_coast_other,
    lat_coast_other,
    angle_other,
) = get_track_geometry(sat, track_number_other)

gu_cut = gu
gv_cut = gv

# Collocate omni at track-gu timestamps; avoid sat-dependent smoothing.
omni_u_match = omni_u.reindex(
    time=gu_cut.time,
    method="nearest",
    tolerance=np.timedelta64(24, "h"),
)
ek_u_match = u_ek.reindex(
    time=gu_cut.time,
    method="nearest",
    tolerance=np.timedelta64(24, "h"),
)
valid = np.isfinite(gu_cut) & np.isfinite(omni_u_match)
gu_pair = gu_cut.where(valid)
omni_u_pair = omni_u_match.where(valid)
ek_u_pair = ek_u_match.where(valid)

gu_ek_pair = gu_pair + ek_u_pair


df_pairs = xr.Dataset({"gu": gu_pair, "omni_u": omni_u_pair}).to_dataframe().reset_index()
df_pairs = df_pairs.dropna(subset=["gu", "omni_u"])

# check valid points
fig = plt.figure(figsize=(12, 8))
ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax1 = fig.add_subplot(2, 1, 2)
omni_u_pair.plot(ax=ax1, color="r", linewidth=2, label="omni")
gu_pair.plot(ax=ax1, color="k", linewidth=2, label="track")
gu_ek_pair.plot(ax=ax1, color="b", linewidth=2, label="track+ekman")
msla_u_at.plot(ax=ax1, color="c", linewidth=2, label="msla u")
ax1.set_xlim(omni_u_pair.time.min(), omni_u_pair.time.max())
ax1.axhline(0, color="k", linestyle="--", linewidth=0.5)

corr = xs.pearson_r(gu_pair, omni_u_pair, dim="time", skipna=True)
corr1 = corr.item()

info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
print(f"Correlation between gu and omni u: {corr1}")
plt.text(
    0.25,
```

```
    0.90,
    info,
    transform=ax1.transAxes,
    fontsize=12,
    fontweight="bold",
)
ax1.text(
    0.02,
    0.80,
    f"b)",
    transform=ax1.transAxes,
    fontsize=14,
    fontweight="bold",
    color="k",
)
ax1.set_ylabel("zonal current (m/s)")
plt.legend()

rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=10)
ax2.grid()
ax2.plot(
    lon_omni,
    lat_omni,
    c="r",
    marker="o",
    markersize=4,
    markerfacecolor="red",
    markeredgecolor="red",
    markeredgewidth=2,
)

ax2.plot(
    lon1,
    lat1,
    c="k",
    marker=".",
    markersize=8,
    markerfacecolor="black",
    markeredgecolor="black",
    markeredgewidth=2,
)
ax2.text(
    0.05,
    0.80,
    f"a)",
    transform=ax2.transAxes,
    fontsize=14,
    fontweight="bold",
    color="k",
)
ax2.text(
    lon_omni-0.5,
    lat_omni,
    f"{SELECT_OMNI_ID}",
    transform=ccrs.PlateCarree(),
    fontsize=10,
    color="r",
    ha="right",
)
ax2.scatter(
    lons_track_self, lats_track_self, marker=".", color="c", s=4
)
ax2.scatter(
    lons_track_other, lats_track_other, marker=".", color="c", s=4
)

lonm, latm = get_point_at_distance(
    lon_equat_self,
    lat_equat_self,
    lon_coast_self,
    lat_coast_self,
    d,
)
info = f"{sat} {track_number_self} {track_number_other} {distance_km:.2f} km"
plt.text(
    0.49,
    0.92,
    info,
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=angle_self,
        color="w",
    )

lonm, latm = get_point_at_distance(
    lon_equat_other,
    lat_equat_other,
    lon_coast_other,
    lat_coast_other,
    d,
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=angle_other,
        color="w",
    )

# plt.savefig("p.omni_uvVStrack_guv_c.ek.nearest.selected.png")
# ln_sf(f"p.omni_uvVStrack_guv_c.ek.nearest.selected.png", png_file)
plt.savefig(f"{png_file}", dpi=300)
plt.show()
plt.close("all")

dse.close()
```

```
352
```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from pathlib import Path
import sys
import os
from tools_xtrackm import ln_sf

# codex resume 019cd8ab-eee9-72e0-9952-b9e4a8bf1b11

RAMA_NO_EK_CSV = "rama_guv_c_nearest_global_matches.csv"
RAMA_EK_CSV = "rama_guv_c_nearest_global_matches.ek.csv"
OMNI_NO_EK_CSV = "omni_guv_c_nearest_global_matches.csv"
OMNI_EK_CSV = "omni_guv_c_nearest_global_matches.ek.csv"

OUT_PNG = "scatter_omniVSrama_uVSgu_ww_ek.png"
# get file name

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

MIN_DISTANCE_KM = None
MAX_DISTANCE_KM = 250.0


def load_xy(csv_path, y_col, with_ekman=False):
    df = pd.read_csv(csv_path)

    dist = df["distance_km"].to_numpy(dtype=float)
    keep = np.isfinite(dist)
    if MIN_DISTANCE_KM is not None:
        keep &= dist >= float(MIN_DISTANCE_KM)
    if MAX_DISTANCE_KM is not None:
        keep &= dist <= float(MAX_DISTANCE_KM)
    df = df.loc[keep].copy()

    x = df["gu"].to_numpy(dtype=float)
    if with_ekman:
        x = x + df["u_ek"].to_numpy(dtype=float)

    y = df[y_col].to_numpy(dtype=float)

    valid = np.isfinite(x) & np.isfinite(y)
    x = x[valid]
    y = y[valid]

    return x, y, len(df)


def add_panel(ax, x, y, title, ylabel, xlabel, extent=None, panel_label=None):
    if x.size < 3:
        ax.text(
            0.5,
            0.98,
            f"{title} (insufficient points)",
            transform=ax.transAxes,
            va="top",
            ha="center",
            fontsize=10,
            bbox=dict(boxstyle="round,pad=0.25", facecolor="black", alpha=0.5, edgecolor="gray"),
            color="white",
        )
        ax.set_xlabel(xlabel)
        ax.set_ylabel(ylabel)
        ax.grid(alpha=0.25)
        if panel_label:
            ax.text(
                0.03,
                0.03,
                panel_label,
                transform=ax.transAxes,
                va="bottom",
                ha="left",
                fontsize=14,
                fontweight="bold",
                color="black",
            )
        return None

    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))

    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1, extent=extent)

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=10,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="white", alpha=0.55, edgecolor="gray"),
        color="black",
    )

    ax.text(
        0.6,
        0.98,
        title,
        transform=ax.transAxes,
        va="top",
        ha="center",
        fontsize=10,
        bbox=dict(boxstyle="round,pad=0.25", facecolor="white", alpha=0.5, edgecolor="white"),
```

```
        color="black",
    )
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right", fontsize=9)
    ax.set_ylim(-1, 1)
    ax.set_xlim(-1, 1)
    if panel_label:
        ax.text(
            0.03,
            0.03,
            panel_label,
            transform=ax.transAxes,
            va="bottom",
            ha="left",
            fontsize=14,
            fontweight="bold",
            color="black",
        )
    return hb


def main():
    min_lbl = MIN_DISTANCE_KM if MIN_DISTANCE_KM is not None else "-inf"
    max_lbl = MAX_DISTANCE_KM if MAX_DISTANCE_KM is not None else "+inf"
    x_label_no_ek = "Altimetry gu (m/s)"
    x_label_ek = "Altimetry gu + u_ek (m/s)"
    none = ""

    x_rama_no, y_rama_no, n_rama_no = load_xy(RAMA_NO_EK_CSV, "rama_u", with_ekman=False)
    x_rama_ek, y_rama_ek, n_rama_ek = load_xy(RAMA_EK_CSV, "rama_u", with_ekman=True)
    x_omni_no, y_omni_no, n_omni_no = load_xy(OMNI_NO_EK_CSV, "omni_u", with_ekman=False)
    x_omni_ek, y_omni_ek, n_omni_ek = load_xy(OMNI_EK_CSV, "omni_u", with_ekman=True)
    all_x = np.concatenate([x_rama_no, x_rama_ek, x_omni_no, x_omni_ek])
    all_y = np.concatenate([y_rama_no, y_rama_ek, y_omni_no, y_omni_ek])
    global_extent = (
        float(np.nanmin(all_x)),
        float(np.nanmax(all_x)),
        float(np.nanmin(all_y)),
        float(np.nanmax(all_y)),
    )

    fig, axs = plt.subplots(2, 2, figsize=(16, 12), sharex=True, sharey=True, constrained_layout=True)

    hb00 = add_panel(
        axs[0, 0],
        x_rama_no,
        y_rama_no,
        f"RAMA u vs gu (no Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        "RAMA u (m/s)",
        none,
        extent=global_extent,
        panel_label="a)",
    )
    hb01 = add_panel(
        axs[0, 1],
        x_rama_ek,
        y_rama_ek,
        f"RAMA u vs gu+u_ek (with Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        none,
        none,
        extent=global_extent,
        panel_label="b)",
    )
    hb10 = add_panel(
        axs[1, 0],
        x_omni_no,
        y_omni_no,
        f"OMNI u vs gu (no Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        "OMNI u (m/s)",
        x_label_no_ek,
        extent=global_extent,
        panel_label="c)",
    )
    hb11 = add_panel(
        axs[1, 1],
        x_omni_ek,
        y_omni_ek,
        f"OMNI u vs gu+u_ek (with Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        none,
        x_label_ek,
        extent=global_extent,
        panel_label="d)",
    )

    top_hbs = [hb for hb in [hb00, hb01] if hb is not None]
    if top_hbs:
        top_vmax = max(float(np.nanmax(hb.get_array())) for hb in top_hbs)
        for hb in top_hbs:
            hb.set_clim(1.0, top_vmax)
        cbar_top = fig.colorbar(top_hbs[0], ax=axs[0, :], location="right", fraction=0.03, pad=0.02)
        cbar_top.set_label("Count")

    bottom_hbs = [hb for hb in [hb10, hb11] if hb is not None]
    if bottom_hbs:
        bottom_vmax = max(float(np.nanmax(hb.get_array())) for hb in bottom_hbs)
        for hb in bottom_hbs:
            hb.set_clim(1.0, bottom_vmax)
        cbar_bottom = fig.colorbar(bottom_hbs[0], ax=axs[1, :], location="right", fraction=0.03, pad=0.02)
        cbar_bottom.set_label("Count")

    # fig.savefig(OUT_PNG, dpi=180)
    plt.savefig(f"{png_file}", dpi=300)
    plt.show()
    plt.close(fig)

    print(f"Rows after distance filter | RAMA no-ek: {n_rama_no}, RAMA ek: {n_rama_ek}")
    print(f"Rows after distance filter | OMNI no-ek: {n_omni_no}, OMNI ek: {n_omni_ek}")
    # print(f"Saved {OUT_PNG}")
    # link OUT_PNG to png_file
    # ln_sf(OUT_PNG, png_file)


if __name__ == "__main__":
    main()

```

```python
import os
import sys

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from pathlib import Path
from tools_xtrackm import ln_sf

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

SAT = "TP+J1+J2+J3+S6A"
CSV_NO_EK = f"drifter_ve_vs_gu_{SAT}.csv"
CSV_W_EK = f"drifter_ve_vs_gu_ek_{SAT}.csv"
OUT_PNG = f"scatter_drifter_ve_vs_gu_ww_ek_{SAT}.png"
COAST_DIST_PATH = "~/allData/topo/GMT_intermediate_coast_distance_01d_track_reg.nc"
PROXIMITY_DIST_M = 200_000.0
MAX_ABS_TIME_DIFF_DAYS = None


def get_coast_distance_km(lons, lats):
    with xr.open_dataset(os.path.expanduser(COAST_DIST_PATH)) as ds_coast:
        lon_da = xr.DataArray(np.asarray(lons, dtype=float), dims="point")
        lat_da = xr.DataArray(np.asarray(lats, dtype=float), dims="point")
        coast_km = ds_coast["coast_distance"].sel(lon=lon_da, lat=lat_da, method="nearest")
    return coast_km.values.astype(float)


def compute_time_diff_days(df):
    drifter_t = pd.to_datetime(df["drifter_time"], errors="coerce")
    gu_t = pd.to_datetime(df["gu_time"], errors="coerce")
    return (drifter_t - gu_t).dt.total_seconds() / 86400.0


def summarize_time_diff_days(dt_days, label):
    dt_days = dt_days[np.isfinite(dt_days.to_numpy(dtype=float))]
    if dt_days.size == 0:
        print(f"[{label}] No valid drifter_time/gu_time pairs for temporal-difference stats.")
        return
    dt_abs = np.abs(dt_days)
    mode_signed = pd.Series(np.round(dt_days, 6)).mode().to_numpy()
    mode_abs = pd.Series(np.round(dt_abs, 6)).mode().to_numpy()
    mode_signed_str = ", ".join([f"{v:.6f}" for v in mode_signed[:5]])
    mode_abs_str = ", ".join([f"{v:.6f}" for v in mode_abs[:5]])
    print(
        f"[{label}] Time diff days (drifter_time - gu_time): "
        f"min={dt_days.min():.6f}, max={dt_days.max():.6f}, "
        f"mean={dt_days.mean():.6f}, median={dt_days.median():.6f}, mode={mode_signed_str}"
    )
    print(
        f"[{label}] Abs time diff days: "
        f"min={dt_abs.min():.6f}, max={dt_abs.max():.6f}, "
        f"mean={dt_abs.mean():.6f}, median={dt_abs.median():.6f}, mode={mode_abs_str}"
    )


def load_filtered_df(csv_path, label):
    df = pd.read_csv(csv_path)
    coast_km = get_coast_distance_km(df["lon_inter"].values, df["lat_inter"].values)
    proximity_km = PROXIMITY_DIST_M / 1000.0
    keep = np.isfinite(coast_km) & (np.abs(coast_km) <= proximity_km)
    df = df.loc[keep].copy()

    dt_days = compute_time_diff_days(df)
    if MAX_ABS_TIME_DIFF_DAYS is not None:
        keep_time = np.isfinite(dt_days.to_numpy(dtype=float)) & (
            np.abs(dt_days.to_numpy(dtype=float)) <= float(MAX_ABS_TIME_DIFF_DAYS)
        )
        df = df.loc[keep_time].copy()
        dt_days = dt_days.loc[df.index]

    summarize_time_diff_days(dt_days, label)
    return df


def fit_and_stats(x, y):
    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))
    return slope, intercept, r, stderr


def prepare_xy(df, x_col):
    x = df[x_col].to_numpy(dtype=float)
    y = df["drifter_ve"].to_numpy(dtype=float)
    valid = np.isfinite(x) & np.isfinite(y)
    return x[valid], y[valid]


def prepare_xy_with_ek(df):
    x = df["gu"].to_numpy(dtype=float) + df["u_ek"].to_numpy(dtype=float)
    y = df["drifter_ve"].to_numpy(dtype=float)
    valid = np.isfinite(x) & np.isfinite(y)
    return x[valid], y[valid]


def plot_panel(ax, x, y, panel_title, x_label, y_label, panel_label=None):
    slope, intercept, r, stderr = fit_and_stats(x, y)
    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1)

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
```

```
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=10,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="white", alpha=0.55, edgecolor="gray"),
        color="k",
    )

    ax.text(
        0.5,
        0.97,
        panel_title,
        transform=ax.transAxes,
        va="top",
        ha="center",
        fontsize=11,
        bbox=dict(boxstyle="round,pad=0.25", facecolor="White", alpha=0.35, edgecolor="gray"),
        color="black",
    )
    if panel_label:
        ax.text(
            0.03,
            0.03,
            panel_label,
            transform=ax.transAxes,
            va="bottom",
            ha="left",
            fontsize=14,
            fontweight="bold",
            color="black",
        )
    ax.set_xlabel(x_label)
    # ax.set_ylabel("drifter ve (m/s)")
    ax.set_ylabel(y_label)
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right")
    return hb


def main():
    none = ""
    df_no_ek = load_filtered_df(CSV_NO_EK, "without_ek")
    df_w_ek = load_filtered_df(CSV_W_EK, "with_ek")

    x_no_ek, y_no_ek = prepare_xy(df_no_ek, "gu")
    x_w_ek, y_w_ek = prepare_xy_with_ek(df_w_ek)

    if x_no_ek.size < 3 or x_w_ek.size < 3:
        print(
            "Insufficient valid points after filtering.\n"
            f"without_ek: {x_no_ek.size} points, with_ek: {x_w_ek.size} points.\n"
            f"Current filters: PROXIMITY_DIST_M={PROXIMITY_DIST_M}, "
            f"MAX_ABS_TIME_DIFF_DAYS={MAX_ABS_TIME_DIFF_DAYS}"
        )
        return

    all_vals = np.concatenate([x_no_ek, y_no_ek, x_w_ek, y_w_ek])
    vmin = np.nanmin(all_vals)
    vmax = np.nanmax(all_vals)

    fig, axes = plt.subplots(1, 2, figsize=(16, 7), sharex=True, sharey=True)

    hb0 = plot_panel(
        axes[0],
        x_no_ek,
        y_no_ek,
        "Without Ekman",
        "Altimetry gu (m/s)",
        "Drifter ve (m/s)",
        panel_label="a)"
    )
    hb1 = plot_panel(
        axes[1],
        x_w_ek,
        y_w_ek,
        "With Ekman",
        "Altimetry gu + u_ek (m/s)",
        none,
        panel_label="b)"
    )

    for ax in axes:
        # ax.set_xlim(vmin, vmax)
        # ax.set_ylim(vmin, vmax)
        ax.set_xlim(vmin, 1.5)
        ax.set_ylim(vmin, 1.5)

    cmax = max(hb0.get_array().max(), hb1.get_array().max())
    hb0.set_clim(1, cmax)
    hb1.set_clim(1, cmax)

    # fig.suptitle(
    #     f"{SAT}: Drifter ve comparison | coast <= {PROXIMITY_DIST_M:.0f} m, "
    #     f"|dt| <= {MAX_ABS_TIME_DIFF_DAYS if MAX_ABS_TIME_DIFF_DAYS is not None else 'all'} d"
    # )
    fig.tight_layout(rect=[0.0, 0.0, 0.89, 0.95])
    cax = fig.add_axes([0.91, 0.13, 0.015, 0.75])
    cbar = fig.colorbar(hb1, cax=cax)
    cbar.set_label("Count")
    # fig.savefig(OUT_PNG, dpi=180)
    plt.savefig(f"{png_file}", dpi=300)
    plt.show()
    plt.close(fig)

    print(f"Rows after filters (without_ek): {len(df_no_ek)}")
    print(f"Rows after filters (with_ek): {len(df_w_ek)}")
    # print(f"Saved {OUT_PNG}")

    # ln_sf(OUT_PNG, png_file)

if __name__ == "__main__":
    main()
```

```python
import os
import sys

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs
from matplotlib.colors import ListedColormap
from pathlib import Path

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

radar_str = "TN"
track_self_here = 608
track_other_here = 137
sat_here = "S3A"
width, height = 13, 6
frac1 = "0.05"
method = "loess"
TRACKS_REG = (73, 99, 0, 26)
d = 1.5

cmap1 = cmo.cm.diff
cmap2 = ListedColormap(['white'])
oscar_dir = "/home/srinivasu/allData/oscar_0.25/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
oscar_extracted_dir = f"/home/srinivasu/xtrackm/guv/oscar_at_intersections/"
guv_dir = f"/home/srinivasu/xtrackm/guv/guv_at_intersections_{method}"

REG = radar_extents[radar_str + "_BLOCK"]
gdf = gpd.read_file(f"polygon_shapefile_around_{radar_str}.shp")
polygon = gdf.geometry.iloc[0]
radar_tsta_o, radar_tend_o = radar_times[radar_str]

OUT_PNG = f"guv_timeseries_{sat_here}_{track_self_here}_{track_other_here}_{radar_str}.png"


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)

dsr = xr.open_dataset(
    f"/home/srinivasu/allData/radar/{radar_str}Codar_daily.nc")
# print(dsr)
dsr = dsr.rename({
    "XAXS": "longitude",
    "YAXS": "latitude",
    "ZAXS": "lev",
    "TAXIS1D": "time"
})
# print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
s_radar = 0.01 * dsr.S_RADAR.isel(lev=0, drop=True)
sizes = u_radar.sizes
ln = sizes["time"]

df = pd.read_csv(f"../guv/tracks_intersections_{sat_here}_1.csv")

df_filter = df[(df["track_self"] == track_self_here) & (df["track_other"] == track_other_here)]

ln = len(df)
for idx, row in df_filter.iterrows():
    sat1 = row["sat"]
    track_number_self = track_to_str(row["track_self"])
    track_number_other = track_to_str(row["track_other"])
    lon1 = row["lons_inter"]
    lat1 = row["lats_inter"]
    lons_inter1 = row["lons_inter"]
    lats_inter1 = row["lats_inter"]
    x_from_coast_self1 = row["x_from_coast_self"]
    x_from_coast_other1 = row["x_from_coast_other"]
    angle_acute = row["angle_acute"]
    angle_obtuse = row["angle_obtuse"]
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    overlap_tsta, overlap_tend = overlap_dates(track_tsta_o, track_tend_o,
                                               radar_tsta_o, radar_tend_o)
    ru_at = u_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    rv_at = v_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    f_oscar = f"{oscar_extracted_dir}/oscar_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc"
    ds_oscar = xr.open_dataset(f_oscar)
    oscar_u = ds_oscar.u
    oscar_v = ds_oscar.v
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    ds_self = xr.open_dataset(f_self, decode_times=False)
    ds_other = xr.open_dataset(f_other, decode_times=False)
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    f_guv = (
        f"{guv_dir}/guv_at_intersection_{sat1}_{track_number_self}_"
        f"{track_number_other}_{method}_{frac1}.nc"
    )
```

```
    if not os.path.exists(f_guv):
        raise FileNotFoundError(f"missing file: {f_guv}")
    with xr.open_dataset(f_guv) as ds_guv:
        gu = ds_guv.gu.load()
        gv = ds_guv.gv.load()
    fig = plt.figure(figsize=(12, 10))
    ax0 = fig.add_subplot(3, 1, 2)
    ax1 = fig.add_subplot(3, 1, 3)

    gu_cut = gu
    gv_cut = gv
    tfreq = sats_tfreq[sat1]

    oscar_u = oscar_u.resample(time=f"{tfreq}D").mean()
    ru_at = ru_at.resample(time=f"{tfreq}D").mean()
    oscar_u = oscar_u.interp(time=gu_cut.time)
    ru_at = ru_at.interp(time=gu_cut.time)
    oscar_max = oscar_u.max().item()
    gu_cut = gu_cut.where(gu_cut < oscar_max, np.nan)
    gu_cut = gu_cut.where(gu_cut > -oscar_max, np.nan)

    oscar_v = oscar_v.resample(time=f"{tfreq}D").mean()
    rv_at = rv_at.resample(time=f"{tfreq}D").mean()
    oscar_v = oscar_v.interp(time=gv_cut.time)
    rv_at = rv_at.interp(time=gu_cut.time)
    oscar_max = oscar_v.max().item()
    gv_cut = gv_cut.where(gv_cut < oscar_max, np.nan)
    gv_cut = gv_cut.where(gv_cut > -oscar_max, np.nan)

    oscar_u.plot(ax=ax0, color="b", linewidth=2, label="oscar u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="k", linewidth=2)
    ru_at.plot(ax=ax0, label=f"radar zonal", color="r", linewidth=2)
    ax0.legend()
    ax0.set_ylabel("oscar u m/s")
    oscar_v.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="k", linewidth=2)
    rv_at.plot(ax=ax1, label=f"radar meridional", color="r", linewidth=2)
    corr_u = xs.pearson_r(gu_cut, ru_at, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, ru_at, dim="time", skipna=True)
    bias_var_u = ru_at - gu_cut
    bias_u = bias_var_u.mean(dim="time")
    corr_v = xs.pearson_r(gv_cut, rv_at, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, rv_at, dim="time", skipna=True)
    bias_var_v = rv_at - gv_cut
    bias_v = bias_var_v.mean(dim="time")
    print(f"Correlation between gu and u_radar: {corr_u.item()}")
    print(f"Correlation between gv and v_radar: {corr_v.item()}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax0.text(
        0.01,
        0.80,
        f"b)",
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
        color="k",
    )
    ax1.text(
        0.01,
        0.80,
        f"c)",
        transform=ax1.transAxes,
        fontsize=14,
        fontweight="bold",
        color="k",
    )
    plt.text(
        0.05,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("oscar v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap2)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.add_geometries(
        [polygon],
        ccrs.PlateCarree(),
        facecolor="blue",
        edgecolor="red",
        alpha=0.2,
    )
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=4,
        markerfacecolor="red",
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    # plot.text radar_str
    plt.text(
        0.15,
```

```
        0.90,
        f"radar region: {radar_str}",
        transform=ax2.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax2.text(
        0.05,
        0.80,
        f"a)",
        transform=ax2.transAxes,
        fontsize=14,
        fontweight="bold",
        color="k",
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    # plt.savefig(
    #     f"guv_timeseries_{sat1}_{track_number_self}_{track_number_other}_{radar_str}.png",
    # )
    plt.savefig(f"{png_file}", dpi=300)
    # ln_sf(OUT_PNG, png_file)
    plt.show()
    plt.close("all")

dse.close()

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
from tools_shapely import *
import sys
from pathlib import Path
import sys

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")


def along_across_shore_components(gu, gv, coast_heading_deg):
    """
    Resolve vector (gu, gv) into along-shore and across-shore components.
    Sign convention:
    - coast_heading_deg is measured CCW from +x (East).
    - along_shore is positive in the heading direction.
    - across_shore is positive 90 deg CCW from heading (left-hand side of coast).
      If you need right-hand positive, negate across_shore.
    """
    theta = np.deg2rad(coast_heading_deg)
    c = np.cos(theta)
    s = np.sin(theta)
    # Elementwise projection keeps scalar or array/xarray shape.
    along_shore = gu * c + gv * s
    across_shore = -gu * s + gv * c  # +90 deg CCW
    return along_shore, across_shore


cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

sat_val = "TP+J1+J2+J3+S6A"
track_self_val = 155
track_other_val = 116

df = pd.read_csv(f"tracks_intersections_{sat_val}_1.csv")
df_out = df.copy()

rows = df_out.loc[
    (df_out["sat"] == sat_val)
    & (df_out["track_self"] == track_self_val)
    & (df_out["track_other"] == track_other_val)
]
# Optional input: Gaussian smoothing sigma
sigma = 3
# Optional input: clipping bbox tuple
# Default used by the function is (70, 5, 95, 30)
region_bbox_tuple = (70, 5, 95, 30)

frac1 = "0.05"
i = 0
sat1 = rows["sat"].values[i]
# track_number_self = rows["track_self"].apply(lambda x: str(x)).values[i]
# track_number_other = rows["track_other"].apply(lambda x: str(x)).values[i]
track_number_self = str(rows["track_self"].values[i])
track_number_other = str(rows["track_other"].values[i])
lons_inter1 = rows["lons_inter"].values[i]
lats_inter1 = rows["lats_inter"].values[i]
lon1 = lons_inter1
lat1 = lats_inter1
query_point = (lon1, lat1)
result = coastline_gaussian_tangent(
    query_point=query_point,
    sigma=sigma,
    region_bbox_tuple=region_bbox_tuple,
)
print("heading_angle_deg:", result["heading_angle_deg"])
print("tangent_vector:", result["tangent_vector"])
print("nearest_point:", result["nearest_point"])
print("distance:", result["distance"])
coast_heading_deg = result["heading_angle_deg"]
x_from_coast_self1 = rows["x_from_coast_self"].values[i]
x_from_coast_other1 = rows["x_from_coast_other"].values[i]
angle_acute = rows["angle_acute"].values[i]
angle_obtuse = rows["angle_obtuse"].values[i]
print(
    sat1,
    track_number_self,
    track_number_other,
    lons_inter1,
    lats_inter1,
    x_from_coast_self1,
    x_from_coast_other1,
    angle_acute,
    angle_obtuse,
)
method = "loess"
# sys.exit()
f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
f_self_smooth = f"../computed/sla_{method}_{frac1}a/track_sla_along_{sat1}_{track_number_self}_{method}_{frac1}.nc"
f_other_smooth = f"../computed/sla_{method}_{frac1}a/track_sla_along_{sat1}_{track_number_other}_{method}_{frac1}.nc"
ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
ds_self_smooth = xr.open_dataset(f_self_smooth)
sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
sla_self_smooth = ds_self_smooth.sla_smooth
ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
ds_other_smooth = xr.open_dataset(f_other_smooth)
sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
sla_other_smooth = ds_other_smooth.sla_smooth
lons_track_self = ds_self.lon.values
lats_track_self = ds_self.lat.values
lon_coast_self = lons_track_self[-1]  # this on coast
lat_coast_self = lats_track_self[-1]  # this on coast
lon_equat_self = lons_track_self[0]  # this on equator
lat_equat_self = lats_track_self[0]  # this on equator
iself = index_at_lat(ds_self, lat1)
a1 = local_angle(
    lons_track_self, lats_track_self, iself, ioff=3, degrees=False
```

```python
)
a1_d = math.degrees(a1)
lons_track_other = ds_other.lon.values
lats_track_other = ds_other.lat.values
iothr = index_at_lat(ds_other, lat1)
a2 = local_angle(
    lons_track_other, lats_track_other, iothr, ioff=3, degrees=False
)
a2_d = math.degrees(a2)
lon_equat_other = lons_track_other[0]
lat_equat_other = lats_track_other[0]
lon_coast_other = lons_track_other[-1]
lat_coast_other = lats_track_other[-1]
gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
gc_other_at = gc_other.sel(
    x=x_from_coast_other1, method="nearest", drop=True
)
gc_other_at = gc_other_at.interp(time=gc_self_at.time)
print(f"heading angle {a1_d:.2f} {a2_d:.2f}")
gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
al, ac = along_across_shore_components(gu, gv, coast_heading_deg)
fig = plt.figure(figsize=(12, 10))
ax1 = fig.add_subplot(3, 1, 3)
ax0 = fig.add_subplot(3, 1, 2)
gu_cut = gu.resample(time="1ME").mean()
gv_cut = gv.resample(time="1ME").mean()
gc_self_at_cut = gc_self_at.resample(time="1ME").mean()
gc_other_at_cut = gc_other_at.resample(time="1ME").mean()
# gu_cut.plot(ax=ax0, label="track zonal", color="r", linewidth=2)
al.plot(ax=ax0, label="alongshore", color="r", linewidth=2)
# gc_self_at_cut.plot(ax=ax0, label=f"gc at {track_number_self}", color="r", linewidth=2)
ax0.legend()
ax0.set_ylabel("along current m/s")
# gv_cut.plot(ax=ax1, label="track meridional", color="r", linewidth=2)
ac.plot(ax=ax1, label="acrossshore", color="r", linewidth=2)
# gc_other_at_cut.plot(ax=ax1, label=f"gc at {track_number_other}", color="r", linewidth=2)
info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
plt.text(
    0.15,
    0.90,
    info,
    transform=ax0.transAxes,
    fontsize=14,
    fontweight="bold",
)
ax1.legend()
ax1.set_ylabel("across current m/s")
ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=5)
ax2.grid()
ax2.plot(
    lon1,
    lat1,
    c="r",
    marker="o",
    markersize=6,
    markerfacecolor="red",
    markeredgecolor="black",
    markeredgewidth=2,
)
ax2.scatter(
    lons_track_self,
    lats_track_self,
    marker=".",
    color="c",
    s=4,
)
ax2.scatter(
    lons_track_other,
    lats_track_other,
    marker=".",
    color="c",
    s=4,
)
lonm, latm = get_point_at_distance(
    lon_equat_self, lat_equat_self, lon_coast_self, lat_coast_self, d
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=a1_d,
        color="k",
    )
lonm, latm = get_point_at_distance(
    lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=a2_d,
        color="k",
    )
# plt.savefig(
#     f"along_shore_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
# )
# ln_sf(
#     f"along_shore_timeseries_{sat1}_{track_number_self}_{track_number_other}.png", png_file)
plt.savefig(f"{png_file}", dpi=300)
plt.show()
plt.close("all")

```

```python
import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.gridspec as gridspec
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from matplotlib.lines import Line2D
from matplotlib.patches import Patch
from scipy.signal import butter, filtfilt
from scipy.stats import linregress
from tools_shapely import *
from tools_xtrackm import *
from pathlib import Path
import sys

# copilot --resume=d419a1e5-878f-477e-813b-3b06f3d9a50b

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")
print(png_file)

method = "loess"
frac1 = "0.05"

sat_val = "TP+J1+J2+J3+S6A"
sat_val = "S3A"
sat_val = "ERS1+ERS2+ENV+SRL"

df = pd.read_csv(f"intersections_near_east_coast_{sat_val}.csv")

# Skip the first point (lowest latitude)
df_out = df.sort_values("lats_inter").iloc[1:].reset_index(drop=True)

N_PTS = 7
assert len(df_out) == N_PTS, (
    f"Expected {N_PTS} points after skipping first, got {len(df_out)}"
)

POINT_COLORS = [plt.cm.tab10(i) for i in range(N_PTS)]
POINT_LABELS = list("ABCDEFG")

# Map extent: 10N–22N, 78E–88E
MAP_EXTENT = (78.0, 88.0, 10.0, 22.0)

# --- Pre-collect all timeseries data ---
point_data = []
for i, row in df_out.iterrows():
    sat1 = row["sat"]
    tno_s = str(row["track_self"])
    tno_o = str(row["track_other"])
    lon1 = float(row["lons_inter"])
    lat1 = float(row["lats_inter"])

    f_guv = (
        f"../guv/guv_at_intersections_{method}/"
        f"guv_at_intersection_{sat1}_{tno_s}_{tno_o}_{method}_{frac1}.nc"
    )
    ds_guv = xr.open_dataset(f_guv)
    gu = ds_guv.gu
    gv = ds_guv.gv
    gs = np.sqrt(gu**2 + gv**2)
    times = pd.to_datetime(gs.time.values)
    gs_vals_raw = gs.values.astype(float)

    # 500-day low-pass filter
    dt_days = np.median(
        np.diff(times).astype("timedelta64[s]").astype(float) / 86400.0
    )
    print(dt_days)
    nyq = 0.5 / dt_days
    cutoff = 1.0 / 500.0
    b_filt, a_filt = butter(4, cutoff / nyq, btype="low")
    nan_mask = np.isnan(gs_vals_raw)
    gs_interp = (
        pd.Series(gs_vals_raw).interpolate(limit_direction="both").values
    )
    if len(gs_interp) > 3 * max(len(a_filt), len(b_filt)):
        gs_lp_vals = filtfilt(b_filt, a_filt, gs_interp)
    else:
        samples_500d = max(3, int(round(500.0 / dt_days)))
        gs_lp_vals = (
            pd.Series(gs_interp)
            .rolling(window=samples_500d, center=True, min_periods=1)
            .mean()
            .values
        )
    gs_lp_vals[nan_mask] = np.nan
    gs_lowpass = xr.DataArray(
        gs_lp_vals, coords=gs.coords, dims=gs.dims, name="gs_lowpass"
    )

    # Linear trend on low-pass series
    t_days = (times - times[0]).days.astype(float)
    valid = np.isfinite(gs_lp_vals)
    trend_res = linregress(t_days[valid], gs_lp_vals[valid])
    fit_vals = trend_res.intercept + trend_res.slope * t_days
    trend_fit = xr.DataArray(
        fit_vals, coords=gs.coords, dims=gs.dims, name="trend_fit"
    )

    point_data.append(
        dict(
            sat1=sat1,
            tno_s=tno_s,
            tno_o=tno_o,
            lon1=lon1,
            lat1=lat1,
            gs=gs,
            gs_lowpass=gs_lowpass,
            trend_fit=trend_fit,
            times=times,
            slope_yr=trend_res.slope * 365.25,
            stderr_yr=trend_res.stderr * 365.25,
            pvalue=trend_res.pvalue,
            significant=trend_res.pvalue < 0.05,
        )
```

```
    )

# --- Build figure: left = tall map, right = 7 timeseries rows ---
fig = plt.figure(figsize=(18, 14))
gs_layout = gridspec.GridSpec(
    N_PTS,
    2,
    figure=fig,
    width_ratios=[1, 2.5],
    hspace=0.0,
    wspace=0.3,
)

# --- Left: vertically elongated cartopy map spanning all rows ---
ax_map = fig.add_subplot(gs_layout[:, 0], projection=ccrs.PlateCarree())
ax_map.set_extent(MAP_EXTENT, crs=ccrs.PlateCarree())
ax_map.add_feature(cfeature.LAND, facecolor="0.93", zorder=0)
ax_map.add_feature(cfeature.OCEAN, facecolor="aliceblue", zorder=0)
ax_map.coastlines(resolution="50m", linewidth=1.0, zorder=2)
gl = ax_map.gridlines(
    draw_labels=True,
    linewidth=0.5,
    color="gray",
    alpha=0.6,
    linestyle="--",
)
gl.top_labels = False
gl.right_labels = False
# ax_map.set_title(f"{sat_val}\nIntersection points", fontsize=9)

# Connect all 7 points with a dashed gray line
lons_pts = [e["lon1"] for e in point_data]
lats_pts = [e["lat1"] for e in point_data]
ax_map.plot(
    lons_pts,
    lats_pts,
    color="gray",
    linewidth=0.8,
    linestyle="--",
    transform=ccrs.PlateCarree(),
    zorder=3,
)

# Plot each point with its assigned color and letter label
for idx, entry in enumerate(point_data):
    color = POINT_COLORS[idx]
    label = POINT_LABELS[idx]
    ax_map.plot(
        entry["lon1"],
        entry["lat1"],
        marker="o",
        markersize=9,
        color=color,
        markeredgecolor="black",
        markeredgewidth=0.8,
        transform=ccrs.PlateCarree(),
        zorder=5,
    )
    ax_map.text(
        entry["lon1"] + 0.18,
        entry["lat1"] + 0.1,
        label,
        fontsize=10,
        fontweight="bold",
        color=color,
        transform=ccrs.PlateCarree(),
        zorder=6,
    )
    # Slope annotation: gray if not significant, colored if significant
    slope_color = color if entry["significant"] else "0.6"
    ax_map.text(
        entry["lon1"] + 0.18,
        entry["lat1"] - 0.45,
        f"{entry['slope_yr']:+.4f} m/s/yr",
        fontsize=7,
        color=slope_color,
        transform=ccrs.PlateCarree(),
        zorder=6,
    )

# --- Right: 7 timeseries subplots, A at bottom (row 6) → G at top (row 0) ---
for idx, entry in enumerate(point_data):
    row = N_PTS - 1 - idx  # A → row 6, G → row 0
    ax_ts = fig.add_subplot(gs_layout[row, 1])
    color = POINT_COLORS[idx]
    label = POINT_LABELS[idx]
    times = entry["times"]
    gs_da = entry["gs"]
    gs_lowpass = entry["gs_lowpass"]
    trend_fit = entry["trend_fit"]

    # 500-day low-pass (same color, bold)
    ax_ts.plot(
        times,
        gs_lowpass.values,
        color=color,
        linewidth=2.0,
        label="500-day LP",
    )
    # Linear trend (black dashed)
    ax_ts.plot(
        times,
        trend_fit.values,
        color="k",
        linewidth=1.5,
        linestyle="--",
        label="trend",
    )

    # Seasonal background shading
    year_min, year_max = times.year.min(), times.year.max()
    for yr in range(year_min, year_max + 1):
        ax_ts.axvspan(
            pd.Timestamp(yr, 2, 1),
            pd.Timestamp(yr, 5, 31),
            color="yellow",
            alpha=0.15,
```

```
            zorder=0,
        )
        ax_ts.axvspan(
            pd.Timestamp(yr, 6, 1),
            pd.Timestamp(yr, 9, 30),
            color="cyan",
            alpha=0.15,
            zorder=0,
        )
        ax_ts.axvspan(
            pd.Timestamp(yr, 10, 1),
            pd.Timestamp(yr, 12, 31),
            color="gray",
            alpha=0.15,
            zorder=0,
        )
        ax_ts.axvline(
            pd.Timestamp(yr, 1, 1),
            color="k",
            linewidth=0.3,
            linestyle="--",
            alpha=0.4,
        )

    # Point label inside left edge of plot
    ax_ts.text(
        0.005, 0.5, label,
        transform=ax_ts.transAxes, fontsize=12,
        fontweight="bold", color=color,
        va="center", ha="left",
    )
    # Tighten y-axis to the filtered (low-pass) series range
    all_vals = gs_lowpass.values[np.isfinite(gs_lowpass.values)]
    ypad = (all_vals.max() - all_vals.min()) * 0.08
    ax_ts.set_ylim(all_vals.min() - ypad, all_vals.max() + ypad)
    ax_ts.set_ylabel("m/s", fontsize=14)
    ax_ts.tick_params(labelsize=12)
    # Only bottom subplot (A, idx=0) shows x-tick labels
    if idx > 0:
        ax_ts.set_xticklabels([])

    # Legend only on the top panel (G, idx=N_PTS-1)
    if idx == N_PTS - 1:
        handles = [
            Line2D([0], [0], color=color, lw=2.0, label="500-day low-pass"),
            Line2D(
                [0],
                [0],
                color="k",
                lw=1.5,
                linestyle="--",
                label="linear trend",
            ),
            Patch(
                facecolor="yellow",
                alpha=0.4,
                edgecolor="none",
                label="Feb–May",
            ),
            Patch(
                facecolor="cyan", alpha=0.4, edgecolor="none", label="Jun–Sep"
            ),
            Patch(
                facecolor="gray", alpha=0.4, edgecolor="none", label="Oct–Dec"
            ),
        ]
        ax_ts.legend(handles=handles, loc="upper right", fontsize=12, ncol=3)

# fig.suptitle(
#     f"Current speed at EICC coastal intersection points — {sat_val}",
#     fontsize=13,
#     fontweight="bold",
#     y=1.002,
# )
# out_png = f"speed_timeseries_allinone_{sat_val}.png"
# plt.savefig(out_png, dpi=150, bbox_inches="tight")
# ln_sf(out_png, png_file)
plt.savefig(f"{png_file}", dpi=300)
plt.show()
plt.close("all")
# print(f"Saved: {out_png}")

```

```python
import os
import sys

from docx import Document
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.shared import Inches
from docx.shared import Pt
from pathlib import Path

base_names = {
    "f.combined_map_std_valid.S3A.py": 1,
    "p.omni_uvVStrack_guv_c.ek.nearest.selected.py": 1,
    "p.scatter_omniVSrama_uVSgu_ww_ek.py": 1,
    "p.scatter_drifter_ve_vs_gu_ww_ek.py": 1,
    "p.radar_uvVStrack_guv.at.intersections_single.py": 1,
    "p.along_shore_at_155_116.intersection.S6A.py": 1,
    "p.speed_at_coastal_points.eicc.trends.all_in_one.ERS.py": 1,
    "satellite_mission_time_spans_new.py": 2,
    "p.compare.std_mean.l2pVSxtrack_map.py": 0,
    "p.gc.clim.cross_structure_x_arrows_map_ERS.py": 0,
    "p.gc.clim.lat_wise.along_xwidth.py": 0,
    "p.aviso_only_uvVStrack_guv.at.intersections_single.py": 0,
    "p.metrics.track_guvVSoscar_ww_ekman.py": 0,
    "p.vskp.tseries_new_v2.2.py": 0,
}

image_list = []
caption_list = []
fig_i = 1
sup_j = 1
for original_key, status in base_names.items():
    if status == 0:
        continue
    if status == 1:
        prefix = f"fig{str(fig_i).zfill(2)}"
        image_name = Path(original_key).with_suffix(".png")
        image_name = f"{prefix}_{image_name}"
        if os.path.exists(image_name):
            # print(f"{image_name} exists")
            image_list.append(image_name)
            fig_i += 1
    elif status == 2:
        prefix = f"sup{str(sup_j).zfill(2)}"
        image_name = Path(original_key).with_suffix(".png")
        image_name = f"{prefix}_{image_name}"
        if os.path.exists(image_name):
            # print(f"{image_name} exists")
            image_list.append(image_name)
            sup_j += 1
    if status != 0:
        prefix = "captions/"
        caption_name = Path(original_key).with_suffix(".txt")
        caption_name = f"{prefix}{caption_name}"
        if os.path.exists(caption_name):
            print(f"{caption_name} exists")
            caption_list.append(caption_name)

# Caption styling options (edit these as needed).
CAPTION_FONT_NAME = "Times New Roman"
CAPTION_FONT_SIZE_PT = 12
CAPTION_ALIGNMENT = "center"  # one of: left, center, right, justify

ALIGNMENT_MAP = {
    "left": WD_ALIGN_PARAGRAPH.LEFT,
    "center": WD_ALIGN_PARAGRAPH.CENTER,
    "right": WD_ALIGN_PARAGRAPH.RIGHT,
    "justify": WD_ALIGN_PARAGRAPH.JUSTIFY,
}

doc_name = "figures_v2.1.docx"
if os.path.exists(doc_name):
    os.remove(doc_name)

doc = Document()
inserted_count = 0

for i, img in enumerate(image_list):
    if not os.path.exists(img):
        print(f"Skipping missing image: {img}")
        continue

    doc.add_picture(img, width=Inches(8))
    inserted_count += 1

    caption_text = ""
    if i < len(caption_list):
        caption_path = caption_list[i]
        if os.path.exists(caption_path):
            with open(caption_path, "r", encoding="utf-8") as f:
                caption_text = f.read().strip()

    prefix = f"Fig{inserted_count}"
    full_caption = f"{prefix}: {caption_text}" if caption_text else prefix

    paragraph = doc.add_paragraph()
    paragraph.alignment = ALIGNMENT_MAP.get(
        CAPTION_ALIGNMENT.lower(), WD_ALIGN_PARAGRAPH.CENTER
    )
    run = paragraph.add_run(full_caption)
    run.font.name = CAPTION_FONT_NAME
    run.font.size = Pt(CAPTION_FONT_SIZE_PT)

    if i < len(image_list) - 1:
        doc.add_page_break()

doc.save(doc_name)
print(f"Inserted {inserted_count} figure(s) into {doc_name}")
# convert docx to pdf
cmd = "libreoffice --headless --convert-to pdf " + doc_name
os.system(cmd)

```

```python
import os
import re
from glob import glob

base_names = {
    "f.combined_map_std_valid.S3A.py": 1,
    "p.omni_uvVStrack_guv_c.ek.nearest.selected.py": 1,
    "p.scatter_omniVSrama_uVSgu_ww_ek.py": 1,
    "p.scatter_drifter_ve_vs_gu_ww_ek.py": 1,
    "p.radar_uvVStrack_guv.at.intersections_single.py": 1,
    "p.along_shore_at_155_116.intersection.S6A.py": 1,
    "p.speed_at_coastal_points.eicc.trends.all_in_one.ERS.py": 1,
    "satellite_mission_time_spans_new.py": 2,
    "p.compare.std_mean.l2pVSxtrack_map.py": 0,
    "p.gc.clim.cross_structure_x_arrows_map_ERS.py": 0,
    "p.gc.clim.lat_wise.along_xwidth.py": 0,
    "p.aviso_only_uvVStrack_guv.at.intersections_single.py": 0,
    "p.metrics.track_guvVSoscar_ww_ekman.py": 0,
    "p.vskp.tseries_new_v2.2.py": 0,
}


def get_clean_name(filename):
    """Strips any existing 'figNN_' or 'supNN_' prefix."""
    return re.sub(r'^(fig|sup)\d{2}_', '', filename)


def strip_all_files(base_names):
    files = sorted(glob(f"fig??_*.py"))
    if not files:
        print("No figNN files found.")
        return
    for f in files:
        f_clean = get_clean_name(f)
        print(f"Striping {f} -> {f_clean}")
        os.rename(f, f_clean)
    files = sorted(glob(f"sup??_*.py"))
    if not files:
        print("No supNN files found.")
        return
    for f in files:
        f_clean = get_clean_name(f)
        print(f"Striping {f} -> {f_clean}")
        os.rename(f, f_clean)


def new_order_rename(base_names):
    strip_all_files(base_names)
    fig_i = 1
    sup_j = 1
    print(f"{'Current File':<45} | {'New Name':<45} | {'Caption'}")
    print("-" * 60)
    for original_key, status in base_names.items():
        if status == 1:
            new_name = f"fig{str(fig_i).zfill(2)}_{original_key}"
            fig_i += 1
        elif status == 2:
            new_name = f"sup{str(sup_j).zfill(2)}_{original_key}"
            sup_j += 1
        else:
            new_name = original_key

        # Check Caption
        ktxt = f"{os.path.splitext(original_key)[0]}.txt"
        kpng = f"{os.path.splitext(original_key)[0]}.png"
        # cap_path = os.path.join(cap_dir, ktxt)
        # cap_status = "" if os.path.exists(cap_path) else ""
        if os.path.exists(original_key):
            os.rename(original_key, new_name)
            print(f"{original_key:<45} -> {new_name:<45}")

new_order_rename(base_names)
# sync_filenames_and_captions(base_names)

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs
import sys
# from rich.traceback import install
# install(show_locals=True, suppress=["site-packages"])

cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
aviso_msla_uv_dir = "/home/srinivasu/sealevel/data/msla_uv/"
aviso_msla_uv_dir = "/home/srinivasu/allData/aviso/msla_uv/"
ds_aviso_u = xr.open_dataset(f"{aviso_msla_uv_dir}/allyear_dt_global_allsat_msla_u.nc")
ds_aviso_v = xr.open_dataset(f"{aviso_msla_uv_dir}/allyear_dt_global_allsat_msla_v.nc")
print(ds_aviso_u)
aviso_u = ds_aviso_u.u
aviso_v = ds_aviso_v.v
# sys.exit()

sat_val = "TP+J1+J2+J3+S6A"
track_self_val = 29
track_other_val = 92
track_self_val = 53
track_other_val = 14
df = pd.read_csv(f"tracks_intersections_{sat_val}_1.csv")
df_out = df.copy()

rows = df_out.loc[
    (df_out["sat"] == sat_val) &
    (df_out["track_self"] == track_self_val) &
    (df_out["track_other"] == track_other_val)
]
# sys.exit()
corrs_u = []
biass_u = []
rmses_u = []
ngp_u = []

corrs_v = []
biass_v = []
rmses_v = []
ngp_v = []

ln = len(rows)
count = 0
frac1 = "0.05"
for i in range(ln):
    sat1 = rows["sat"].values[i]
    track_number_self = rows["track_self"].apply(lambda x: str(x)).values[i]
    track_number_other = rows["track_other"].apply(lambda x: str(x)).values[i]
    lons_inter1 = rows["lons_inter"].values[i]
    lats_inter1 = rows["lats_inter"].values[i]
    lon1 = lons_inter1
    lat1 = lats_inter1
    x_from_coast_self1 = rows["x_from_coast_self"].values[i]
    x_from_coast_other1 = rows["x_from_coast_other"].values[i]
    angle_acute = rows["angle_acute"].values[i]
    angle_obtuse = rows["angle_obtuse"].values[i]
    if abs(lat1) < 2:
        corrs_u.append(np.nan)
        rmses_u.append(np.nan)
        biass_u.append(np.nan)
        ngp_u.append(np.nan)
        corrs_v.append(np.nan)
        rmses_v.append(np.nan)
        biass_v.append(np.nan)
        ngp_v.append(np.nan)
        continue
    print(
        sat1,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat1}_{track_number_self}_loess_{frac1}.nc"
    f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat1}_{track_number_other}_loess_{frac1}.nc"
    ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
```

```
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)
    print(f"azimuths {a1:.2f} {a2:.2f}")
    gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
    fig = plt.figure(figsize=(12, 10))
    ax1 = fig.add_subplot(3, 1, 3)
    ax0 = fig.add_subplot(3, 1, 2)

    gu_cut = gu.resample(time="1ME").mean()
    gu_cut_ngp = 100 * gu_cut.count(dim="time")/len(gu_cut.time)
    gv_cut = gv.resample(time="1ME").mean()
    gv_cut_ngp = 100 * gv_cut.count(dim="time")/len(gv_cut.time)
    tfreq = sats_tfreq[sat1]

    aviso_u1 = aviso_u.sel(longitude=lon1, latitude=lat1, method="nearest")
    aviso_u2 = aviso_u1.interp(time=gu_cut.time)
    aviso_max_u = aviso_u2.max().item()
    gu_cut = gu_cut.where(gu_cut < aviso_max_u, np.nan)
    gu_cut = gu_cut.where(gu_cut > -aviso_max_u, np.nan)

    aviso_v1 = aviso_v.sel(longitude=lon1, latitude=lat1, method="nearest")
    aviso_v2 = aviso_v1.interp(time=gv_cut.time)

    aviso_max_v = aviso_v2.max().item()
    gv_cut = gv_cut.where(gv_cut < aviso_max_v, np.nan)
    gv_cut = gv_cut.where(gv_cut > -aviso_max_v, np.nan)

    aviso_u2.plot(ax=ax0, color="b", linewidth=2, label="aviso u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="r", linewidth=2)
    # ax0.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax0.legend()
    ax0.set_ylabel("aviso u m/s")

    aviso_v2.plot(ax=ax1, color="b", linewidth=2, label="aviso v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="r", linewidth=2)

    # percentage of good points of gu_cut
    corr_u = xs.pearson_r(gu_cut, aviso_u2, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, aviso_u2, dim="time", skipna=True)
    bias_var_u = aviso_u2 - gu_cut
    bias_u = bias_var_u.mean(dim="time")

    corr_v = xs.pearson_r(gv_cut, aviso_v2, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, aviso_v2, dim="time", skipna=True)
    bias_var_v = aviso_v2 - gv_cut
    bias_v = bias_var_v.mean(dim="time")

    corrs_u.append(corr_u.item())
    rmses_u.append(rmse_u.item())
    biass_u.append(bias_u.item())
    ngp_u.append(gu_cut_ngp.item())

    corrs_v.append(corr_v.item())
    rmses_v.append(rmse_v.item())
    biass_v.append(bias_v.item())
    ngp_v.append(gv_cut_ngp.item())

    print(f"Correlation between gu and u_aviso_at: {corr_u.item():.2f} {gu_cut_ngp.item():.2f}")
    print(f"Correlation between gv and v_aviso_at: {corr_v.item():.2f} {gv_cut_ngp.item():.2f}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.15,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("aviso v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor="red",
        markeredgecolor="black",
        markeredgewidth=2,
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
```

```
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    ln_sf(
        f"guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
        "fig03.png",
    )
    # plt.show()
    plt.close("all")
    # ds_aviso_u.close()
dse.close()

```

```python
import xarray as xr
from tools_xtrackm import *
from namesm import sats_new
import glob
import os
from matplotlib.collections import LineCollection
from pathlib import Path
import sys

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

method = "loess"
frac1 = "0.05"
sat = "ERS1+ERS2+ENV+SRL"

REG = (80.0, 92.0, 11.0, 21.0)
WBOB = (78.0, 87.0, 8.0, 21.0)
WBOB2 = (78.0, 97.0, 0.0, 25.0)
proj = ccrs.PlateCarree()

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 10),
                               subplot_kw={'projection': proj})
# adjust subplots to have minimum space between them
month_num = 2
month_name = months[month_num+1]
info =f"{sat} {months[month_num+1]}"
fig.subplots_adjust(wspace=0.1)
ax1.set_extent([*WBOB2], crs=proj)
ax1.coastlines()
# ax1.legend()
gl = ax1.gridlines(draw_labels=True, linewidth=0.5, color='gray', alpha=0.7, linestyle='--')
gl.top_labels = gl.right_labels = False  # Hide top and right labels
plt.text(0.4, 0.95, f"a) {info}",
         horizontalalignment='center',
         verticalalignment='center',
         fontsize=15,
         fontweight="bold",
         transform=ax1.transAxes)
for f_sla in sorted(
    glob.glob(
        f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
    )
):
    basename = os.path.basename(f_sla)
    ds_sla = xr.open_dataset(f_sla, decode_times=False)
    trackn = ds_sla.Pass
    f = f"../computed/sla_{method}_{frac1}a/track_sla_along_{sat}_{trackn}_{method}_{frac1}.nc"
    lons_track = ds_sla.lon.values
    lats_track = ds_sla.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    lon_equat = lons_track[0]
    lat_equat = lats_track[0]
    # compute tangent vectors or slope of track at each point
    dlon = np.gradient(lons_track_rev)
    dlat = np.gradient(lats_track_rev)
    tangent_mag = np.hypot(dlon, dlat)
    tangent_u = dlon / tangent_mag
    tangent_v = dlat / tangent_mag
    # Convert latitude to radians
    phi = np.deg2rad(lats_track_rev)
    # Length of one degree of latitude and longitude in meters
    meters_per_degree_lat = 111_320  # Approximate value
    meters_per_degree_lon = 111_320 * np.cos(phi)
    # Handle division by zero at the poles
    meters_per_degree_lon = np.where(meters_per_degree_lon == 0,
                                     np.nan, meters_per_degree_lon)
    # Compute normal vectors
    normal_u = -tangent_v
    normal_v = tangent_u
    theta = np.arctan2(lat_coast-lat_equat, lon_coast-lon_equat)
    print(theta, "-----------------------------------------------------")
    # if is_within_region(lon_coast, lat_coast, *REG):
    if True:
        ds_smooth = xr.open_dataset(f)
        sla_smooth = ds_smooth.sla_smooth
        gc = compute_geostrophy_gc(ds_sla, sla_smooth)
        # gc = gc.where(gc > -2.5)
        # gc = gc.where(gc < 2.5)
        # gc = gc.where((gc <= 3.5) & (gc >= -3.5))
        # gc = mad_outlier_detection_fixed(gc, threshold=3.5, dim="time")
        gc_clim = gc.groupby("time.month").mean("time")
        gc_clim_m = gc_clim.sel(month=month_num)
        # Convert gc from m/s to degrees (assuming cross-track direction)
        gc_deg = gc_clim_m.values / meters_per_degree_lat
        # Calculate displacements
        delta_lon = normal_u * gc_deg
        delta_lat = normal_v * gc_deg
        # Apply scaling factor for visualization
        scaling_factor = 1e5  # Adjust as needed
        delta_lon *= scaling_factor
        delta_lat *= scaling_factor
        # Compute endpoints
        lon_start = lons_track_rev
        lat_start = lats_track_rev
        lon_end = lon_start + delta_lon
        lat_end = lat_start + delta_lat
        # Create line segments
        segments = [[[lon_start[i], lat_start[i]], [lon_end[i], lat_end[i]]]
                    for i in range(len(lon_start))]
        # Create a LineCollection
        lc = LineCollection(segments, colors='red', linewidths=1,
                            transform=proj)
        ax1.plot(lons_track_rev, lats_track_rev, 'k-',
                 transform=proj, label='Ground Track')
        ax1.add_collection(lc)

month_num = 10
month_name = months[month_num+1]
info =f"{sat} {months[month_num+1]}"
ax2.set_extent([*WBOB2], crs=proj)
ax2.coastlines()
# ax1.legend()
gl = ax2.gridlines(draw_labels=True, linewidth=0.5, color='gray', alpha=0.7, linestyle='--')
```

```
gl.top_labels = gl.right_labels = False  # Hide top and right labels
plt.text(0.4, 0.95, f"b) {info}",
         horizontalalignment='center',
         verticalalignment='center',
         fontsize=15,
         fontweight="bold",
         transform=ax2.transAxes)
for f_sla in sorted(
    glob.glob(
        f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
    )
):
    basename = os.path.basename(f_sla)
    ds_sla = xr.open_dataset(f_sla, decode_times=False)
    trackn = ds_sla.Pass
    # f = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{trackn}_loess_0.2.nc"
    f = f"../computed/sla_{method}_{frac1}a/track_sla_along_{sat}_{trackn}_{method}_{frac1}.nc"
    lons_track = ds_sla.lon.values
    lats_track = ds_sla.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    lon_equat = lons_track[0]
    lat_equat = lats_track[0]
    # compute tangent vectors or slope of track at each point
    dlon = np.gradient(lons_track_rev)
    dlat = np.gradient(lats_track_rev)
    tangent_mag = np.hypot(dlon, dlat)
    tangent_u = dlon / tangent_mag
    tangent_v = dlat / tangent_mag
    # Convert latitude to radians
    phi = np.deg2rad(lats_track_rev)
    # Length of one degree of latitude and longitude in meters
    meters_per_degree_lat = 111_320  # Approximate value
    meters_per_degree_lon = 111_320 * np.cos(phi)
    # Handle division by zero at the poles
    meters_per_degree_lon = np.where(meters_per_degree_lon == 0,
                                     np.nan, meters_per_degree_lon)
    # Compute normal vectors
    normal_u = -tangent_v
    normal_v = tangent_u
    theta = np.arctan2(lat_coast-lat_equat, lon_coast-lon_equat)
    print(theta, "--------------------------------------------------")
    # if is_within_region(lon_coast, lat_coast, *REG):
    if True:
        ds_smooth = xr.open_dataset(f)
        sla_smooth = ds_smooth.sla_smooth
        gc = compute_geostrophy_gc(ds_sla, sla_smooth)
        # gc = gc.where((gc <= 3.5) & (gc >= -3.5))
        # gc = mad_outlier_detection_fixed(gc, threshold=3.5, dim="time")
        gc_clim = gc.groupby("time.month").mean("time")
        gc_clim_m = gc_clim.sel(month=month_num)
        # Convert gc from m/s to degrees (assuming cross-track direction)
        gc_deg = gc_clim_m.values / meters_per_degree_lat
        # Calculate displacements
        delta_lon = normal_u * gc_deg
        delta_lat = normal_v * gc_deg
        # Apply scaling factor for visualization
        scaling_factor = 1e5  # Adjust as needed
        delta_lon *= scaling_factor
        delta_lat *= scaling_factor
        # Compute endpoints
        lon_start = lons_track_rev
        lat_start = lats_track_rev
        lon_end = lon_start + delta_lon
        lat_end = lat_start + delta_lat
        # Create line segments
        segments = [[[lon_start[i], lat_start[i]], [lon_end[i], lat_end[i]]]
                    for i in range(len(lon_start))]
        # Create a LineCollection
        lc = LineCollection(segments, colors='red', linewidths=1,
                            transform=proj)
        ax2.plot(lons_track_rev, lats_track_rev, 'k-',
                 transform=proj, label='Ground Track')
        ax2.add_collection(lc)
plt.savefig(f"cross_structure_x_arrows_map_{sat}_1by2.png",
            bbox_inches='tight', dpi=300)
ln_sf(f"cross_structure_x_arrows_map_{sat}_1by2.png", png_file)
plt.show()
plt.close("all")

```

```python
import xarray as xr
from tools_xtrackm import *
import glob
import os

import matplotlib.cm as cm
from pathlib import Path
import sys

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

# sats = ["S3A"]
REG = (80.0, 89.0, 11.0, 21.0)
REG = (75.0, 92.0, 10.0, 23.0)
sats = ["S3A"]
sats = ["ERS1+ERS2+ENV+SRL"]
xwidth = 290e3
# Load the YlOrRd colormap
ncolors = 21
cmap1 = cm.get_cmap('YlOrRd', ncolors)  # Split into 10 colors
colors1 = cmap1(np.linspace(0, 1, ncolors))
cmap2 = cm.get_cmap('BuGn', ncolors)  # Split into 10 colors
colors2 = cmap2(np.linspace(0, 1, ncolors))

norm2 = plt.Normalize(10.18, 22.00)
sm2 = plt.cm.ScalarMappable(cmap=cmap2, norm=norm2)
sm2.set_array([])

norm1 = plt.Normalize(10.18, 22.00)
sm1 = plt.cm.ScalarMappable(cmap=cmap1, norm=norm1)
sm1.set_array([])

# fig, ax = plt.subplots(1, 1, figsize=(9, 4))
frac1 = "0.05"
method = "loess"
for sat in sats[:]:
    fig, ax = plt.subplots(1, 1, figsize=(12, 6))
    # adjust subplots increase x spacing
    plt.subplots_adjust(left=0.1, bottom=0.1, right=0.8, top=0.8, wspace=0.3, hspace=0.3)
    i = 0
    lat_values = []
    for f_sla in sorted(
        glob.glob(
            f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
        )
    ):
        basename = os.path.basename(f_sla)
        ds_sla = xr.open_dataset(f_sla, decode_times=False)
        trackn = ds_sla.Pass
        f = f"../computed/sla_{method}_{frac1}a/track_sla_along_{sat}_{trackn}_{method}_{frac1}.nc"
        lons_track = ds_sla.lon.values
        lats_track = ds_sla.lat.values
        lons_track_rev = lons_track[::-1]
        lats_track_rev = lats_track[::-1]
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        print(f, lon_coast, lat_coast)
        if is_within_region(lon_coast, lat_coast, *REG):
            lat_values.append(lat_coast)
            ds_smooth = xr.open_dataset(f)
            # print(ds_smooth)
            sla_smooth = ds_smooth.sla_smooth
            print(f"stats of sla_smooth {sat} {trackn} --------------------")
            print(f"stats of gu {sat} {trackn} --------------------")
            gc = compute_geostrophy_gc(ds_sla, sla_smooth)
            gc_clim = gc.groupby("time.month").mean("time")
            gc_clim_xwidth = gc_clim.sel(x=slice(0, xwidth))
            gc_clim_xwidth_m = gc_clim_xwidth.isel(month=2, drop=True) # march
            gc_clim_xwidth_n = gc_clim_xwidth.isel(month=10, drop=True) # november
            gc_clim_xwidth_m.plot(ax=ax, label=f"{lat_coast}",
                                  color=colors1[i])
            x0, y0 = -15, gc_clim_xwidth_m.sel(x=0).values
            plt.text(x0, y0, f"{lat_coast:.2f}", fontsize=10, color=colors1[i])
            gc_clim_xwidth_n.plot(ax=ax, label=f"{lat_coast}",
                                  color=colors2[i])
            x0, y0 = -15, gc_clim_xwidth_n.sel(x=0).values
            plt.text(x0, y0, f"{lat_coast:.2f}", fontsize=10, color=colors2[i])
            # set xticks between 0 to xwidth
            ax.set_xlim(-15, xwidth)
            ax.set_ylim(-0.7, 0.7)
            xticks1 = ax.get_xticks()
            xticks_km = np.round((xticks1 / 1000), 0)
            ax.set_xticks(xticks1, xticks_km)
            ax.set_xlabel("distance along track (km)")
            ax.set_ylabel("geostrophic velocity (m/s)")
            # ax.set_ylim(-0.3, 0.3)
            ax.axhline(0, color="k", linestyle="--")
            i += 1
            # print_stats(gc)
    # plt.legend()
    print(f"number of tracks {i}")
    minlat = min(lat_values)
    maxlat = max(lat_values)
    print(f"min lat {min(lat_values)} max lat {max(lat_values)}")
    pos = ax.get_position()
    # Adjust and add the first colorbar closer to the plot
    cbar_ax1 = fig.add_axes([pos.x1 + 0.01, pos.y0, 0.02, pos.height])  # Adjust 0.01 to control horizontal gap
    cbar_ax1.text(0.5, 0.5, "MARCH", fontsize=10, horizontalalignment="center", verticalalignment="center", transform=cbar_ax1.transAxes, rotation = "vertical")
    cbar = plt.colorbar(sm1, cax=cbar_ax1)
    # cbar.set_label('latitude')
    cbar.set_ticks(list(range(int(minlat), int(maxlat)+1, 2)))
    # Adjust and add the second colorbar even closer
    cbar_ax2 = fig.add_axes([pos.x1 + 0.06, pos.y0, 0.02, pos.height])  # Adjust 0.03 as needed for spacing
    cbar_ax2.text(0.5, 0.5, "NOVEMBER", fontsize=10, horizontalalignment="center", verticalalignment="center", transform=cbar_ax2.transAxes, rotation = "vertical")
    cbar = plt.colorbar(sm2, cax=cbar_ax2)
    cbar.set_label('latitude')
    cbar.set_ticks(list(range(int(minlat), int(maxlat)+1, 2)))

    plt.savefig(
        f"p.gc.clim.lat_wise.along_xwidth_{sat}a.png",
        bbox_inches="tight",
        dpi=300,
    )
    ln_sf(
        f"p.gc.clim.lat_wise.along_xwidth_{sat}a.png",
        png_file,
```

```
    )
    plt.show()
    plt.close("all")

```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from tools_xtrackm import ln_sf

only_df = pd.read_csv(
    "oscar_onlyVStrack_corrs_rmses_biass_TP+J1+J2+J3+S6A_u.csv")
ekmn_df = pd.read_csv(
    "oscar_ekmnVStrack_corrs_rmses_biass_TP+J1+J2+J3+S6A_u.csv")

# Create unique column combining zero-padded sat and track_self, track_other
only_df["unique_id"] = (only_df["sat"].apply(lambda s: s.zfill(20)) + "_" +
                        only_df["track_self"].astype(str) + "_" +
                        only_df["track_other"].astype(str))
only_df = only_df.set_index("unique_id")

ekmn_df["unique_id"] = (ekmn_df["sat"].apply(lambda s: s.zfill(20)) + "_" +
                        ekmn_df["track_self"].astype(str) + "_" +
                        ekmn_df["track_other"].astype(str))
ekmn_df = ekmn_df.set_index("unique_id")

common_ids = only_df.index.intersection(ekmn_df.index)
only_df = only_df.loc[common_ids]
ekmn_df = ekmn_df.loc[common_ids]

# Extract metrics for plotting
metrics = ["biass", "corrs", "rmses"]
titles = ["Bias", "Correlation", "Rmse"]

# Prepare figure with 2x2 grid
fig, axs = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle("comparison metrics with oscar u: track gu only VS track gu + ekman", fontsize=14)

summary_lines = []

for i, metric in enumerate(metrics):
    ax = axs[i // 2, i % 2]
    valid_mask = only_df[metric].notna() & ekmn_df[metric].notna()
    only_df1 = only_df[valid_mask]
    ekmn_df1 = ekmn_df[valid_mask]
    # Get data
    only_df_data = only_df1[metric].values
    ekmn_df_data = ekmn_df1[metric].values
    indices = np.arange(len(only_df_data))
    ax.plot(indices, only_df_data, label="track gu only")
    ax.plot(indices, ekmn_df_data, label="track gu+ekman")
    # draw zero line
    ax.axhline(0, color="gray", linestyle="--")
    ax.set_xlabel("Data Point Index")
    ax.set_ylabel(titles[i])
    ax.set_title(titles[i])
    ax.legend()
    # collect summary text for this metric
    if len(only_df_data) > 0:
        mean_o = np.nanmean(only_df_data)
        mean_e = np.nanmean(ekmn_df_data)
        diff = mean_e - mean_o
        cnt = len(only_df_data)
        summary_lines.append(f"\n{titles[i]}:")
        summary_lines.append(f" count = {cnt}")
        summary_lines.append(f" mean track u = {mean_o:.4f}")
        summary_lines.append(f" mean track u + ekman = {mean_e:.4f}")
        summary_lines.append(f" mean improvement = {diff:.4f}")
    else:
        summary_lines.append(f"\n{titles[i]}: no valid paired data")

ax_sum = axs[1, 1]
ax_sum.axis("off")
text = " ".join(summary_lines)

ax_sum.annotate(
    "\n".join(summary_lines),
    xy=(0, 1),
    xycoords="axes fraction",
    va="top",
    ha="left",
    fontsize=10,
    family="monospace",
    bbox=dict(boxstyle="round,pad=0.5", facecolor="white", alpha=0.7),
    annotation_clip=False,
)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(f"track_guvVSoscar_ww_ekman_u.png")
ln_sf("track_guvVSoscar_ww_ekman_u.png", "fig04.png")
# plt.show()

```

```python
import math
import pickle
from datetime import datetime
import sys
import os

import cmocean as cmo
import matplotlib.gridspec as gridspec
import matplotlib.pyplot as plt
import numpy as np
import shapely
import shapely as shp
import xarray as xr
from tools_xtrackm import *


# function to get percentage valid point in an xarray dataarray
def perc_valid_points(da):
    return 100 * (da.count().item() / len(da.time))


xr.set_options(display_expand_data=False)
gs = gridspec.GridSpec(1,
                       2,
                       width_ratios=[3, 1],
                       height_ratios=[2],
                       wspace=0.1,
                       hspace=0.05)
PSMSL_ID = 414  # id for visakhaptnam
width, height = 13, 6
data_dir = f"/home/srinivasu/xtrackm/data/"

cmap1 = cmo.cm.topo
cmap1 = cmo.cm.diff
cmap1 = cmo.cm.balance
cmap2 = cmo.cm.thermal

# tide gauge data
df = read_tide_meta()
df_nio = region_selected(df, *NIO)
lon_psmsl, lat_psmsl, name_psmsl = get_lonlat_psmsl(PSMSL_ID, df_nio)
print(lon_psmsl, lat_psmsl)
name_psmsl = name_psmsl.strip()

delx2 = 5
lonlat_box2 = (
    lon_psmsl - delx2,
    lon_psmsl + delx2,
    lat_psmsl - delx2,
    lat_psmsl + delx2,
)
poly_box = shp.geometry.box(lonlat_box2[0], lonlat_box2[2], lonlat_box2[1],
                            lonlat_box2[3])
# get the perimeter of the box
ds_id = read_id(PSMSL_ID)
sla_tide = ds_id["height"]
sla_tide = 0.001 * sla_tide
tide_tsta_o = ds_id.time[0].values
tide_tend_o = ds_id.time[-1].values
tide_tsta_o = n64todatetime(tide_tsta_o)
tide_tend_o = n64todatetime(tide_tend_o)

tlim1 = datetime.strptime("2013-01-01", "%Y-%m-%d")
tlim2 = datetime.strptime("2021-01-01", "%Y-%m-%d")


def point_on_line_at_distance(m, c, x0, y0, d):
    """
    Find a point on the line y = mx + c at a distance d from (x0, y0).
    Parameters:
        m (float): Slope of the line.
        c (float): y-intercept of the line.
        x0 (float): x-coordinate of the starting point.
        y0 (float): y-coordinate of the starting point.
        d (float): Distance from the starting point.
    Returns:
        tuple: Coordinates (x, y) of the new point.
    """
    # Normalize the direction vector of the line
    dx = 1 / math.sqrt(1 + m**2)  # x-component of the unit direction vector
    dy = m / math.sqrt(1 + m**2)  # y-component of the unit direction vector
    # Compute the new point
    if m < 0:
        d = -d
    x_new = x0 + d * dx
    y_new = y0 + d * dy
    return x_new, y_new


ds_dt1 = xr.open_dataset(
    "/home/srinivasu/allData/aviso/allyear_dt_global_allsat_msla_h.nc",
    decode_times=False,
)
# ds_dt1 = xr.open_dataset(
#     "/media/srinivasu/allData/sealevel/data/allyear_dt_global_allsat_msla_h.nc",
#     decode_times=False,
# )
print(ds_dt1)
ds_dt = change_time(ds_dt1, "time")
sla_aviso = ds_dt.sla  # .sel(time=slice(tsta_o, tend_o))
sla_aviso_at = sla_aviso.interp(longitude=lon_psmsl,
                                method="nearest").interp(latitude=lat_psmsl,
                                                         method="nearest")

sla_tide_interp = sla_tide.interp(time=sla_aviso.time)
sla_tide_interp = sla_tide_interp - sla_tide_interp.mean(dim="time")
# sla_tide_adj = sla_tide_interp  - dac_p_smooth

chosen_index = 0
dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")

with open("closest_tracks_all_psmsl_new.pkl", "rb") as file:
    sorted_list = pickle.load(file)


def filter_list(sorted_list, sat=None, track_no=None, id=None):
    result = sorted_list
```

```
    if sat is not None:
        result = [tup for tup in result if tup[0] == sat]
    if track_no is not None:
        result = [tup for tup in result if tup[1] == track_no]
    if id is not None:
        result = [tup for tup in result if tup[5] == id]
    return result


filtered_list = sorted_list[:]
sat_here = "ERS1+ERS2+ENV+SRL"
track_no = "309"
filtered_list = filter_list(sorted_list, sat=sat_here, track_no=track_no, id=PSMSL_ID)
print(filtered_list)

# sys.exit(0)
for sat, trackn, x1, y1, mindist, id in filtered_list:
    if mindist > 50:
        continue
    track_tsta_o, track_tend_o = get_time_limits_o(sat)
    overlap_tsta, overlap_tend = overlap_dates(track_tsta_o, track_tend_o,
                                               tide_tsta_o, tide_tend_o)
    tsta_o, tend_o = get_time_limits_o(sat)
    print(sat, trackn, x1, y1, mindist)
    if trackn is None:
        continue
    # zero pad trackn with three digits
    trackn1 = trackn.zfill(3)
    f = f"{data_dir}/{sat}/ctoh.sla.ref.{sat}.nindian.{trackn1}.nc"
    ds = xr.open_dataset(f, decode_times=False)
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
    c = (lats_track[0]) - m * (lons_track[0])
    x_from_coast = distance.distance((lat_coast, lon_coast), (y1, x1)).km
    # shapely path of the track
    track_path = shapely.geometry.LineString(list(zip(lons_track, lats_track)))
    if len(lons_track) == 0:
        continue
    m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
    angle = np.rad2deg(np.arctan(m))
    c = (lats_track[0]) - m * (lons_track[0])
    ds_smooth = xr.open_dataset(
        f"/home/srinivasu/xtrackm/computed/sla_loess_0.2a/track_sla_along_{sat}_{trackn}_loess_0.2.nc"
    )
    sla_track_smooth = ds_smooth.sla_smooth
    dac_track_smooth = ds_smooth.dac_smooth
    sla_track_p_smooth = sla_track_smooth.sel(x=x_from_coast,
                                              method="nearest",
                                              drop=True)
    dac_track_p_smooth = dac_track_smooth.sel(x=x_from_coast,
                                              method="nearest",
                                              drop=True)
    sla_plus = sla_track_p_smooth + dac_track_p_smooth
    fig = plt.figure(figsize=(width, height))
    ax1 = fig.add_subplot(gs[0, 0])
    # sla_aviso_at.plot(ax=ax1,
    #                   label="extracted aviso grid data",
    #                   linestyle="--")
    sla_tide_interp.plot(ax=ax1, label="tide gauge data", linewidth=4)
    # sla_track_p_smooth.plot(ax=ax1, label=f"track {trackn} {sat}")
    sla_plus.plot(ax=ax1, label=f"track {trackn} {sat} + dac")
    # ax1.set_xlim(tlim1, tlim2)
    ax1.set_xlim(overlap_tsta, overlap_tend)
    ax1.set_ylim(-0.6, 0.6)
    info1 = f"{sat} {trackn} {mindist:.2f}km"
    ax1.text(
        0.5,
        0.9,
        info1,
        horizontalalignment="center",
        verticalalignment="center",
        transform=ax1.transAxes,
        fontweight="bold",
        fontsize=14,
    )
    ax1.legend(loc="lower right")

    ax2 = fig.add_subplot(gs[0, 1], projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*lonlat_box2[:2])).sel(ETOPO05_Y=slice(
        *lonlat_box2[2:])).plot(ax=ax2,
                                add_colorbar=False,
                                add_labels=False,
                                cmap=cmap1)
    decorate_axis(ax2, "", *lonlat_box2, step=2)
    # get intersection of track and box
    intersection = track_path.intersection(poly_box)
    # get the intersection points
    intersection_points = intersection.coords
    # sort intesections points by latitude
    intersection_points = sorted(intersection_points, key=lambda x: x[1])
    # take the first intersection points
    if len(intersection_points) == 0:
        continue
    lonl = intersection_points[0][0]
    latl = intersection_points[0][1]
    print(lonl, latl)
    info = f"{sat} {trackn}"
    lonm, latm = point_on_line_at_distance(m, c, lonl, latl, 2)
    print(lonm, latm)
    ax2.scatter(lons_track, lats_track, linewidths=0.0, s=8, color="r")
    ax2.text(
        lonm,
        latm,
        s=f"{trackn}",
        fontsize=10,
        rotation=angle,
        color="white",
        horizontalalignment="center",
        verticalalignment="center",
        fontweight="bold",
    )
    ax2.add_geometries(
        [poly_box],
        ccrs.PlateCarree(),
```

```
        facecolor="none",
        edgecolor="lightblue",
        alpha=0.5,
    )
    # PSMSL tide gauge location
    ax2.scatter(
        lon_psmsl,
        lat_psmsl,
        marker="*",
        c="yellow",
        s=150,
        edgecolor="black",
    )
    ax2.scatter(
        x1,
        y1,
        marker="x",
        c="cyan",
        s=10,
    )
    # PSMSL tide gauge name
    ax2.text(
        lon_psmsl,
        lat_psmsl,
        s=name_psmsl,
        fontsize=10,
        color="k",
        horizontalalignment="left",
    )
    ax2.set_xlim([*lonlat_box2[:2]])
    ax2.set_ylim([*lonlat_box2[2:]])
    # sla_tide_adj.plot(ax=ax1, label="tide gauge data adjusted")
    ax2.legend(loc="lower right")
    info = f"{sat}"
    ax2.text(0.1, 0.95, info, transform=ax2.transAxes, fontweight="bold")
    plt.savefig(
        f"tseries_loess_adj_allsat_map_box_separate_{sat}_{trackn}.png",
        bbox_inches="tight",
    )
    ln_sf(
        f"tseries_loess_adj_allsat_map_box_separate_{sat}_{trackn}.png", "fig02.png")
    # plt.show()
    plt.close("all")

```

```
1
2  figures in order along with captions(captions/*.txt) with the script make_figure_docx.py.
3  I am changing the figure numbers and sometimes removing some figures.
4
```

draft_figs02/run_all.py

```python
import os

for f in sorted(os.listdir(".")):
    if f.startswith("fig") and f.endswith(".py"):
        cmd = f"MPLBACKEND=Agg python3 {f}"
        print(cmd)
        try:
            os.system(cmd)
        except Exception as e:
            print(e)
            pass

```

```python
from datetime import datetime

import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import pandas as pd
from dateutil.relativedelta import relativedelta
import os
import sys
from pathlib import Path
from tools_xtrackm import ln_sf

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

missions = pd.read_csv("data_coverage_df.csv")
print(missions)

colors = ["red", "green", "blue", "purple"]
date_format = "%Y-%m-%d"

min_start_date = min(pd.to_datetime(missions["first_date"]))
max_end_date = max(pd.to_datetime(missions["last_date"]))
min_start_date1 = min_start_date - relativedelta(years=1)
max_end_date1 = max_end_date + relativedelta(years=1)
# print(min_start_date1, max_end_date1)

fig, ax = plt.subplots(figsize=(10, 6), layout="constrained")

for index, row in missions.iterrows():
    sat, track_nos, start_date, end_date, tfreq, xfreq = row.loc[
        ["sat", "trackn", "first_date", "last_date", "tfreq", "xfreq"]
    ]
    sln = len(sat)
    words = [sat, str(track_nos), str(tfreq), str(round(xfreq, 1))]
    start_date_o = datetime.strptime(start_date, date_format)
    end_date_o = datetime.strptime(end_date, date_format)
    duration = end_date_o - start_date_o
    ax.barh(
        index,
        duration,
        left=start_date_o,
        color="skyblue",
        edgecolor="black",
    )
    # current_start_time = start_date_o + relativedelta(years=4)
    current_start_time = start_date_o + duration / 4
    for k, word in enumerate(words):
        sln = len(word)
        text_obj = ax.text(
            current_start_time,
            index,
            word,
            ha="center",
            va="center",
            fontweight="bold",
            fontsize=12,
            color=colors[k],
        )
        if k == 0:
            yrs = int(0.21 * sln) + 2
            mns = int(2.53 * sln) + 10
        else:
            yrs = int(0.21 * sln) + 1.5
            mns = int(2.53 * sln) + 8
        # current_start_time = current_start_time + relativedelta(years=yrs)
        current_start_time = current_start_time + relativedelta(months=mns)

ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
ax.xaxis.set_major_locator(mdates.YearLocator(2))
# ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
# ax.xaxis.set_major_locator(mdates.YearLocator(1))
ax.set_yticks([])
ax.set_yticklabels([])
ax.set_xlabel("years")
ax.set_xlim(min_start_date1, max_end_date1)
ax.set_title("Satellite Missions Timeline")
ax.invert_yaxis()

satellite = mpatches.Patch(color="red", label="satellite")
tracks = mpatches.Patch(color="green", label="no. of tracks")
tfreq = mpatches.Patch(color="blue", label="temporal frequency")
xfreq = mpatches.Patch(color="purple", label="spatial resolution")
plt.legend(handles=[satellite, tracks, tfreq, xfreq])

plt.savefig(f"satellite_mission_time_spans_new_fig01.png")
ln_sf("satellite_mission_time_spans_new_fig01.png", png_file)
# plt.tight_layout()
# plt.show()

```

draft_figs02/touch_captions_files.py

```python
from pathlib import Path

base_names = {
    "p.aviso_only_uvVStrack_guv.at.intersections_single.py": 0,
    "p.metrics.track_guvVSoscar_ww_ekman.py": 0,
    "p.vskp.tseries_new_v2.2.py": 0,
    "f.combined_map_std_valid.S3A.py": 1,
    "p.compare.std_mean.l2pVSxtrack_map.py": 1,
    "p.omni_uvVStrack_guv_c.ek.nearest.selected.py": 1,
    "p.scatter_omniVSrama_uVSgu_ww_ek.py": 1,
    "p.scatter_drifter_ve_vs_gu_ww_ek.py": 1,
    "p.radar_uvVStrack_guv.at.intersections_single.py": 1,
    "p.gc.clim.cross_structure_x_arrows_map_ERS.py": 0,
    "p.gc.clim.lat_wise.along_xwidth.py": 0,
    "p.along_shore_at_155_116.intersection.S6A.py": 1,
    "p.speed_at_coastal_points.eicc.trends.all_in_one.ERS.py": 1,
    "satellite_mission_time_spans_new.py": 2
}

for k, v in base_names.items():
    # add captions dir before each file
    k1 = "captions/" + k
    p = Path(k1)
    # check if p exists
    if p.with_suffix(".txt").exists():
        print(f"File already exists: {p}")
    else:
        print(f"touching file: {p.with_suffix('.txt')}")
        # p.with_suffix(".txt").touch()

```

```
1
2  run_all.py
3
4  2. make_figure_docx.py
5  To make docx with on fig??.png file insert in each page
6
```

```
./draft_figs02/archive/
```

31 file(s)

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle
import os
import sys
from pathlib import Path

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")
print(png_file)

sat = "S3A"
cmap = "twilight_shifted"
EAST_COAST_REG = (78.0, 95, 0, 25.4)

ds_xtr = xr.open_dataset("../computed/std_mean_S3A_xtrack.nc")
ds_l2p = xr.open_dataset("/home/srinivasu/allData/aviso_l2p/s3a/std_mean_S3A_l2p.nc")

std_xtr = ds_xtr["std"]
std_l2p = ds_l2p["std"]

with open("common_passes_l2pVSxtrack.pkl", "rb") as f:
    common = pickle.load(f)

dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
fig = plt.figure(figsize=(10, 6))
ax1 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax2 = fig.add_subplot(2, 1, 2)

dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:])).plot(
    ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap
)
decorate_axis(ax1, "", *NIO, step=8)
ax1.text(
    0.45,
    0.80,
    f"MISSION: {sat}",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
i = 0
for pass1 in common:
    pass2 = pass1[1:]
    f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass2}.nc"
    with xr.open_dataset(f, decode_times=False) as ds:
        lon_eq = ds.lon.values[0]
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        lon_equat = lons_track[0]
        lon_coast = lons_track[-1]
        lat_equat = lats_track[0]
        lat_coast = lats_track[-1]
        a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
        a1_d = math.degrees(a1)
        track_number = ds.Pass
        lonm = lons_track.mean()
        latm = lats_track.mean()
        color = colors[i]
        print(f"{sat} {track_number} -------------------")
        ax1.scatter(lons_track, lats_track, s=2, color=color)
        if is_within_region(lonm, latm, *NIO):
            ax1.text(
                lonm,
                latm,
                s=str(track_number),
                fontsize=7,
                rotation=a1_d,
                color='k',
                ha="center",
                va="center",
                transform=ccrs.PlateCarree(),
            )
        i += 1

# ax2.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))

ax2.plot(std_xtr, label="xtrack std", color="red")
ax2.plot(std_l2p, label="l2p std", color="blue")
ax2.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax2.set_xlabel("distance along track (km)")
ax2.set_ylabel("std")
ax2.set_title("std_mean_xtrack vs std_mean_l2p")
ax2.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax2.text(
    0.02,
    0.5,
    "coast",
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
    color="k",
    rotation=90,
)

plt.legend()
plt.tight_layout()
plt.savefig(f"std_mean_xtrackVSstd_mean_l2p_{sat}.png")
ln_sf(f"std_mean_xtrackVSstd_mean_l2p_{sat}.png", png_file)
# plt.show()
plt.close("all")

dse.close()

```

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle
import os
import sys
from pathlib import Path

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")
print(png_file)

sat = "S3A"
cmap = "twilight_shifted"
EAST_COAST_REG = (78.0, 95, 0, 25.4)

ds_xtr = xr.open_dataset("../computed/std_mean_S3A_xtrack.nc")
ds_l2p = xr.open_dataset("/home/srinivasu/allData/aviso_l2p/s3a/std_mean_S3A_l2p.nc")

std_xtr = ds_xtr["std"]
std_l2p = ds_l2p["std"]

with open("common_passes_l2pVSxtrack.pkl", "rb") as f:
    common = pickle.load(f)

dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
fig = plt.figure(figsize=(10, 6))
ax1 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax2 = fig.add_subplot(2, 1, 2)

dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:])).plot(
    ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap
)
decorate_axis(ax1, "", *NIO, step=8)
ax1.text(
    0.45,
    0.80,
    f"MISSION: {sat}",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)
ax1.text(
    0.05,
    0.80,
    f"a)",
    transform=ax1.transAxes,
    fontsize=14,
    fontweight="bold",
    color="k",
)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
i = 0
for pass1 in common:
    pass2 = pass1[1:]
    f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass2}.nc"
    with xr.open_dataset(f, decode_times=False) as ds:
        lon_eq = ds.lon.values[0]
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        lon_equat = lons_track[0]
        lon_coast = lons_track[-1]
        lat_equat = lats_track[0]
        lat_coast = lats_track[-1]
        a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
        a1_d = math.degrees(a1)
        track_number = ds.Pass
        lonm = lons_track.mean()
        latm = lats_track.mean()
        color = colors[i]
        print(f"{sat} {track_number} -------------------")
        ax1.scatter(lons_track, lats_track, s=2, color="w")
        if is_within_region(lonm, latm, *NIO):
            ax1.text(
                lonm,
                latm,
                s=str(track_number),
                fontsize=7,
                rotation=a1_d,
                color='k',
                ha="center",
                va="center",
                transform=ccrs.PlateCarree(),
            )
        i += 1

# ax2.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))

ax2.plot(std_xtr, label="xtrack std", color="red")
ax2.plot(std_l2p, label="l2p std", color="blue")
ax2.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax2.set_xlabel("distance along track (km)")
ax2.set_ylabel("std (m)")
# ax2.set_title("std_mean_xtrack vs std_mean_l2p")
ax2.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax2.text(
    0.02,
    0.5,
    "coast",
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
    color="k",
    rotation=90,
)
ax2.text(
    0.02,
    0.80,
    f"b)",
    transform=ax2.transAxes,
```

```
    fontsize=14,
    fontweight="bold",
    color="k",
)

plt.legend()
plt.tight_layout()
plt.savefig(f"std_mean_xtrackVSstd_mean_l2p_{sat}.png")
ln_sf(f"std_mean_xtrackVSstd_mean_l2p_{sat}.png", png_file)
plt.show()
plt.close("all")

dse.close()

```

```python
from datetime import datetime

import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import pandas as pd
from dateutil.relativedelta import relativedelta
import os
import sys
from pathlib import Path
from tools_xtrackm import ln_sf

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

missions = pd.read_csv("data_coverage_df.csv")
print(missions)

colors = ["red", "green", "blue", "purple"]
date_format = "%Y-%m-%d"

min_start_date = min(pd.to_datetime(missions["first_date"]))
max_end_date = max(pd.to_datetime(missions["last_date"]))
min_start_date1 = min_start_date - relativedelta(years=1)
max_end_date1 = max_end_date + relativedelta(years=1)
# print(min_start_date1, max_end_date1)

fig, ax = plt.subplots(figsize=(10, 6), layout="constrained")

for index, row in missions.iterrows():
    sat, track_nos, start_date, end_date, tfreq, xfreq = row.loc[
        ["sat", "trackn", "first_date", "last_date", "tfreq", "xfreq"]
    ]
    sln = len(sat)
    words = [sat, str(track_nos), str(tfreq), str(round(xfreq, 1))]
    start_date_o = datetime.strptime(start_date, date_format)
    end_date_o = datetime.strptime(end_date, date_format)
    duration = end_date_o - start_date_o
    ax.barh(
        index,
        duration,
        left=start_date_o,
        color="skyblue",
        edgecolor="black",
    )
    # current_start_time = start_date_o + relativedelta(years=4)
    current_start_time = start_date_o + duration / 4
    for k, word in enumerate(words):
        sln = len(word)
        text_obj = ax.text(
            current_start_time,
            index,
            word,
            ha="center",
            va="center",
            fontweight="bold",
            fontsize=12,
            color=colors[k],
        )
        if k == 0:
            yrs = int(0.21 * sln) + 2
            mns = int(2.53 * sln) + 10
        else:
            yrs = int(0.21 * sln) + 1.5
            mns = int(2.53 * sln) + 8
        # current_start_time = current_start_time + relativedelta(years=yrs)
        current_start_time = current_start_time + relativedelta(months=mns)

ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
ax.xaxis.set_major_locator(mdates.YearLocator(2))
# ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
# ax.xaxis.set_major_locator(mdates.YearLocator(1))
ax.set_yticks([])
ax.set_yticklabels([])
ax.set_xlabel("years")
ax.set_xlim(min_start_date1, max_end_date1)
ax.set_title("Satellite Missions Timeline")
ax.invert_yaxis()

satellite = mpatches.Patch(color="red", label="satellite")
tracks = mpatches.Patch(color="green", label="no. of tracks")
tfreq = mpatches.Patch(color="blue", label="temporal frequency")
xfreq = mpatches.Patch(color="purple", label="spatial resolution")
plt.legend(handles=[satellite, tracks, tfreq, xfreq])

plt.savefig(f"satellite_mission_time_spans_new_fig01.png")
ln_sf("satellite_mission_time_spans_new_fig01.png", png_file)
# plt.tight_layout()
# plt.show()

```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
import xskillscore as xs
from geopy import distance
from tools_xtrackm import *
from pathlib import Path
import sys

# codex resume 019cc947-b006-7d82-8941-d68e330c388a

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(
        f_track, engine="netcdf4", decode_times=False
    ) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


def build_unique_str(sat, row):
    track_self = track_to_str(row["track_self"])
    track_other = track_to_str(row["track_other"])
    return f"{sat}_{track_self}_{track_other}"


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)


cmap1 = cmo.cm.diff
d = 1.5
dpth = 15
height, width = 9, 9
frac1 = "0.05"


dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
omni_data_dir = "/home/srinivasu/allData/MB_cleaned/raju_omni_clean"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
out_match_csv = "omni_guv_c_nearest_global_matches.csv"
matched_rows_all = []
# SELECT_SAT = "TP+J1+J2+J3+S6A"
# SELECT_OMNI_ID = "BD09"
# SELECT_TRACK_SELF = "53"
# SELECT_TRACK_OTHER = "14"
SELECT_SAT = 'S3A'
SELECT_OMNI_ID = 'BD10'
SELECT_TRACK_SELF = '522'
SELECT_TRACK_OTHER = '279'
SELECT_SAT = 'S3A'
SELECT_OMNI_ID = 'BD10'
SELECT_TRACK_SELF = '636'
SELECT_TRACK_OTHER = '393'

EKMAN_DIR = "/home/srinivasu/xtrackm/ekman/ekman_at_intersections"
# TP+J1+J2+J3+S6A BD09 53 14
# S3A BD10 522 279

df_all_list = []
for sat in sats_new:
    df_all_list.append(pd.read_csv(f"tracks_intersections_{sat}_1.csv"))
df_all = pd.concat(df_all_list, ignore_index=True)


lon_omni, lat_omni = omni_d[SELECT_OMNI_ID]
omni_file = f"{omni_data_dir}/{SELECT_OMNI_ID}_current_sst.nc"
print(omni_file)
if not os.path.exists(omni_file):
    print(f"File not found: {omni_file}")
    raise(f"no omni file")
if abs(lat_omni) < 2:
    raise(f"lat less than 2")

ds_omni = xr.open_dataset(omni_file)
print(ds_omni)
# print(ds_omni)
ds_omni = xr.open_dataset(omni_file)
print(ds_omni)
omni_u = ds_omni.u_current.interp(current_depth=dpth)
# omni_u = omni_u.rolling(time=10, center=True).mean()
omni_u = omni_u
omni_v = ds_omni.v_current.interp(current_depth=dpth)
# omni_v = omni_v.rolling(time=10, center=True).mean()
omni_v = omni_v
N = omni_v.count().item()

df_selected = df_all[
    (df_all["sat"] == SELECT_SAT)
    & (df_all["track_self"].astype(str) == SELECT_TRACK_SELF)
    & (df_all["track_other"].astype(str) == SELECT_TRACK_OTHER)
].copy()
if df_selected.empty:
    raise("selected sat/track combination not found in intersections file")
lons = df_selected["lons_inter"].to_numpy(dtype=float)
```

```
lats = df_selected["lats_inter"].to_numpy(dtype=float)
distances = [distance.distance((lat, lon), (lat_omni, lon_omni)).km for lat, lon in zip(lats, lons)]
df_selected["distance_km"] = distances
row = df_selected.nsmallest(1, "distance_km").iloc[0]

sat = row["sat"]
track_number_self = str(row["track_self"])
track_number_other = str(row["track_other"])
lon1 = row["lons_inter"]
lat1 = row["lats_inter"]
distance_km = row["distance_km"]
angle_acute = row["angle_acute"]
angle_obtuse = row["angle_obtuse"]
unique_str = build_unique_str(sat, row)
f_ek = f"{EKMAN_DIR}/{unique_str}_ekman_at_intersections.nc"
ds_ek = xr.open_dataset(f_ek)
u_ek = ds_ek['u_at']
v_ek = ds_ek['v_at']

print(
    sat,
    SELECT_OMNI_ID,
    track_number_self,
    track_number_other,
    lon1,
    lat1,
    angle_acute,
    angle_obtuse,
)

f_guv = f"{guv_dir}/guv_at_intersection_{sat}_{track_number_self}_{track_number_other}_{frac1}.nc"
if not os.path.exists(f_guv):
    print(f"missing file: {f_guv}")
    raise("missing file")

with xr.open_dataset(f_guv) as ds_guv:
    gu = ds_guv.gu
    gv = ds_guv.gv

(
    lons_track_self,
    lats_track_self,
    lon_equat_self,
    lat_equat_self,
    lon_coast_self,
    lat_coast_self,
    angle_self,
) = get_track_geometry(sat, track_number_self)
(
    lons_track_other,
    lats_track_other,
    lon_equat_other,
    lat_equat_other,
    lon_coast_other,
    lat_coast_other,
    angle_other,
) = get_track_geometry(sat, track_number_other)

gu_cut = gu
gv_cut = gv

# Collocate omni at track-gu timestamps; avoid sat-dependent smoothing.
omni_u_match = omni_u.reindex(
    time=gu_cut.time,
    method="nearest",
    tolerance=np.timedelta64(24, "h"),
)
ek_u_match = u_ek.reindex(
    time=gu_cut.time,
    method="nearest",
    tolerance=np.timedelta64(24, "h"),
)
valid = np.isfinite(gu_cut) & np.isfinite(omni_u_match)
gu_pair = gu_cut.where(valid)
omni_u_pair = omni_u_match.where(valid)
ek_u_pair = ek_u_match.where(valid)

gu_ek_pair = gu_pair + ek_u_pair


df_pairs = xr.Dataset({"gu": gu_pair, "omni_u": omni_u_pair}).to_dataframe().reset_index()
df_pairs = df_pairs.dropna(subset=["gu", "omni_u"])

# check valid points
fig = plt.figure(figsize=(12, 8))
ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax1 = fig.add_subplot(2, 1, 2)
omni_u_pair.plot(ax=ax1, color="r", linewidth=2, label="omni")
gu_pair.plot(ax=ax1, color="k", linewidth=2, label="track")
gu_ek_pair.plot(ax=ax1, color="b", linewidth=2, label="track+ekman")

corr = xs.pearson_r(gu_pair, omni_u_pair, dim="time", skipna=True)
corr1 = corr.item()

info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
print(f"Correlation between gu and omni u: {corr1}")
plt.text(
    0.25,
    0.90,
    info,
    transform=ax1.transAxes,
    fontsize=12,
    fontweight="bold",
)
plt.legend()

rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=10)
ax2.grid()
ax2.plot(
    lon_omni,
    lat_omni,
    c="r",
    marker="o",
    markersize=4,
    markerfacecolor="red",
    markeredgecolor="red",
```

```
    markeredgewidth=2,
)

ax2.plot(
    lon1,
    lat1,
    c="k",
    marker=".",
    markersize=8,
    markerfacecolor="black",
    markeredgecolor="black",
    markeredgewidth=2,
)
plt.text(lon_omni, lat_omni, f"{SELECT_OMNI_ID}", fontsize=10, color="k")
ax2.scatter(
    lons_track_self, lats_track_self, marker=".", color="c", s=4
)
ax2.scatter(
    lons_track_other, lats_track_other, marker=".", color="c", s=4
)

lonm, latm = get_point_at_distance(
    lon_equat_self,
    lat_equat_self,
    lon_coast_self,
    lat_coast_self,
    d,
)
info = f"{sat} {track_number_self} {track_number_other} {distance_km:.2f} km"
plt.text(
    0.55,
    0.90,
    info,
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=angle_self,
        color="w",
    )

lonm, latm = get_point_at_distance(
    lon_equat_other,
    lat_equat_other,
    lon_coast_other,
    lat_coast_other,
    d,
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=angle_other,
        color="w",
    )

plt.savefig("p.omni_uvVStrack_guv_c.ek.nearest.selected.png")
plt.show()
ln_sf(f"p.omni_uvVStrack_guv_c.ek.nearest.selected.png", png_file)
plt.close("all")

dse.close()

```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from pathlib import Path
import sys
import os
from tools_xtrackm import ln_sf

# codex resume 019cd8ab-eee9-72e0-9952-b9e4a8bf1b11

RAMA_NO_EK_CSV = "rama_guv_c_nearest_global_matches.csv"
RAMA_EK_CSV = "rama_guv_c_nearest_global_matches.ek.csv"
OMNI_NO_EK_CSV = "omni_guv_c_nearest_global_matches.csv"
OMNI_EK_CSV = "omni_guv_c_nearest_global_matches.ek.csv"

OUT_PNG = "scatter_omniVSrama_uVSgu_ww_ek.png"
# get file name

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

MIN_DISTANCE_KM = None
MAX_DISTANCE_KM = 250.0


def load_xy(csv_path, y_col, with_ekman=False):
    df = pd.read_csv(csv_path)

    dist = df["distance_km"].to_numpy(dtype=float)
    keep = np.isfinite(dist)
    if MIN_DISTANCE_KM is not None:
        keep &= dist >= float(MIN_DISTANCE_KM)
    if MAX_DISTANCE_KM is not None:
        keep &= dist <= float(MAX_DISTANCE_KM)
    df = df.loc[keep].copy()

    x = df["gu"].to_numpy(dtype=float)
    if with_ekman:
        x = x + df["u_ek"].to_numpy(dtype=float)

    y = df[y_col].to_numpy(dtype=float)

    valid = np.isfinite(x) & np.isfinite(y)
    x = x[valid]
    y = y[valid]

    return x, y, len(df)


def add_panel(ax, x, y, title, ylabel, xlabel, extent=None):
    if x.size < 3:
        ax.text(
            0.5,
            0.98,
            f"{title} (insufficient points)",
            transform=ax.transAxes,
            va="top",
            ha="center",
            fontsize=10,
            bbox=dict(boxstyle="round,pad=0.25", facecolor="black", alpha=0.5, edgecolor="white"),
            color="white",
        )
        ax.set_xlabel(xlabel)
        ax.set_ylabel(ylabel)
        ax.grid(alpha=0.25)
        return None

    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))

    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1, extent=extent)

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=10,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="black", alpha=0.55, edgecolor="white"),
        color="white",
    )

    ax.text(
        0.6,
        0.98,
        title,
        transform=ax.transAxes,
        va="top",
        ha="center",
        fontsize=10,
        bbox=dict(boxstyle="round,pad=0.25", facecolor="white", alpha=0.5, edgecolor="white"),
        color="black",
    )
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right", fontsize=9)
    ax.set_ylim(-1, 1)
    ax.set_xlim(-1, 1)
    return hb


def main():
```

```python
    min_lbl = MIN_DISTANCE_KM if MIN_DISTANCE_KM is not None else "-inf"
    max_lbl = MAX_DISTANCE_KM if MAX_DISTANCE_KM is not None else "+inf"
    x_label_no_ek = "Altimetry gu (m/s)"
    x_label_ek = "Altimetry gu + u_ek (m/s)"
    none = ""

    x_rama_no, y_rama_no, n_rama_no = load_xy(RAMA_NO_EK_CSV, "rama_u", with_ekman=False)
    x_rama_ek, y_rama_ek, n_rama_ek = load_xy(RAMA_EK_CSV, "rama_u", with_ekman=True)
    x_omni_no, y_omni_no, n_omni_no = load_xy(OMNI_NO_EK_CSV, "omni_u", with_ekman=False)
    x_omni_ek, y_omni_ek, n_omni_ek = load_xy(OMNI_EK_CSV, "omni_u", with_ekman=True)
    all_x = np.concatenate([x_rama_no, x_rama_ek, x_omni_no, x_omni_ek])
    all_y = np.concatenate([y_rama_no, y_rama_ek, y_omni_no, y_omni_ek])
    global_extent = (
        float(np.nanmin(all_x)),
        float(np.nanmax(all_x)),
        float(np.nanmin(all_y)),
        float(np.nanmax(all_y)),
    )

    fig, axs = plt.subplots(2, 2, figsize=(16, 12), sharex=True, sharey=True, constrained_layout=True)

    hb00 = add_panel(
        axs[0, 0],
        x_rama_no,
        y_rama_no,
        f"RAMA u vs gu (no Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        "RAMA u (m/s)",
        none,
        extent=global_extent,
    )
    hb01 = add_panel(
        axs[0, 1],
        x_rama_ek,
        y_rama_ek,
        f"RAMA u vs gu+u_ek (with Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        none,
        none,
        extent=global_extent,
    )
    hb10 = add_panel(
        axs[1, 0],
        x_omni_no,
        y_omni_no,
        f"OMNI u vs gu (no Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        "OMNI u (m/s)",
        x_label_no_ek,
        extent=global_extent,
    )
    hb11 = add_panel(
        axs[1, 1],
        x_omni_ek,
        y_omni_ek,
        f"OMNI u vs gu+u_ek (with Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        none,
        x_label_ek,
        extent=global_extent,
    )

    top_hbs = [hb for hb in [hb00, hb01] if hb is not None]
    if top_hbs:
        top_vmax = max(float(np.nanmax(hb.get_array())) for hb in top_hbs)
        for hb in top_hbs:
            hb.set_clim(1.0, top_vmax)
        cbar_top = fig.colorbar(top_hbs[0], ax=axs[0, :], location="right", fraction=0.03, pad=0.02)
        cbar_top.set_label("Count")

    bottom_hbs = [hb for hb in [hb10, hb11] if hb is not None]
    if bottom_hbs:
        bottom_vmax = max(float(np.nanmax(hb.get_array())) for hb in bottom_hbs)
        for hb in bottom_hbs:
            hb.set_clim(1.0, bottom_vmax)
        cbar_bottom = fig.colorbar(bottom_hbs[0], ax=axs[1, :], location="right", fraction=0.03, pad=0.02)
        cbar_bottom.set_label("Count")

    fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)

    print(f"Rows after distance filter | RAMA no-ek: {n_rama_no}, RAMA ek: {n_rama_ek}")
    print(f"Rows after distance filter | OMNI no-ek: {n_omni_no}, OMNI ek: {n_omni_ek}")
    print(f"Saved {OUT_PNG}")
    # link OUT_PNG to png_file
    ln_sf(OUT_PNG, png_file)


if __name__ == "__main__":
    main()

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs
import sys
# from rich.traceback import install
# install(show_locals=True, suppress=["site-packages"])

cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
aviso_msla_uv_dir = "/home/srinivasu/sealevel/data/msla_uv/"
aviso_msla_uv_dir = "/home/srinivasu/allData/aviso/msla_uv/"
ds_aviso_u = xr.open_dataset(f"{aviso_msla_uv_dir}/allyear_dt_global_allsat_msla_u.nc")
ds_aviso_v = xr.open_dataset(f"{aviso_msla_uv_dir}/allyear_dt_global_allsat_msla_v.nc")
print(ds_aviso_u)
aviso_u = ds_aviso_u.u
aviso_v = ds_aviso_v.v
# sys.exit()

sat_val = "TP+J1+J2+J3+S6A"
track_self_val = 29
track_other_val = 92
track_self_val = 53
track_other_val = 14
df = pd.read_csv(f"tracks_intersections_{sat_val}_1.csv")
df_out = df.copy()

rows = df_out.loc[
    (df_out["sat"] == sat_val) &
    (df_out["track_self"] == track_self_val) &
    (df_out["track_other"] == track_other_val)
]
# sys.exit()
corrs_u = []
biass_u = []
rmses_u = []
ngp_u = []

corrs_v = []
biass_v = []
rmses_v = []
ngp_v = []

ln = len(rows)
count = 0
frac1 = "0.05"
for i in range(ln):
    sat1 = rows["sat"].values[i]
    track_number_self = rows["track_self"].apply(lambda x: str(x)).values[i]
    track_number_other = rows["track_other"].apply(lambda x: str(x)).values[i]
    lons_inter1 = rows["lons_inter"].values[i]
    lats_inter1 = rows["lats_inter"].values[i]
    lon1 = lons_inter1
    lat1 = lats_inter1
    x_from_coast_self1 = rows["x_from_coast_self"].values[i]
    x_from_coast_other1 = rows["x_from_coast_other"].values[i]
    angle_acute = rows["angle_acute"].values[i]
    angle_obtuse = rows["angle_obtuse"].values[i]
    if abs(lat1) < 2:
        corrs_u.append(np.nan)
        rmses_u.append(np.nan)
        biass_u.append(np.nan)
        ngp_u.append(np.nan)
        corrs_v.append(np.nan)
        rmses_v.append(np.nan)
        biass_v.append(np.nan)
        ngp_v.append(np.nan)
        continue
    print(
        sat1,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat1}_{track_number_self}_loess_{frac1}.nc"
    f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat1}_{track_number_other}_loess_{frac1}.nc"
    ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
```

```
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)
    print(f"azimuths {a1:.2f} {a2:.2f}")
    gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
    fig = plt.figure(figsize=(12, 10))
    ax1 = fig.add_subplot(3, 1, 3)
    ax0 = fig.add_subplot(3, 1, 2)

    gu_cut = gu.resample(time="1ME").mean()
    gu_cut_ngp = 100 * gu_cut.count(dim="time")/len(gu_cut.time)
    gv_cut = gv.resample(time="1ME").mean()
    gv_cut_ngp = 100 * gv_cut.count(dim="time")/len(gv_cut.time)
    tfreq = sats_tfreq[sat1]

    aviso_u1 = aviso_u.sel(longitude=lon1, latitude=lat1, method="nearest")
    aviso_u2 = aviso_u1.interp(time=gu_cut.time)
    aviso_max_u = aviso_u2.max().item()
    gu_cut = gu_cut.where(gu_cut < aviso_max_u, np.nan)
    gu_cut = gu_cut.where(gu_cut > -aviso_max_u, np.nan)

    aviso_v1 = aviso_v.sel(longitude=lon1, latitude=lat1, method="nearest")
    aviso_v2 = aviso_v1.interp(time=gv_cut.time)

    aviso_max_v = aviso_v2.max().item()
    gv_cut = gv_cut.where(gv_cut < aviso_max_v, np.nan)
    gv_cut = gv_cut.where(gv_cut > -aviso_max_v, np.nan)

    aviso_u2.plot(ax=ax0, color="b", linewidth=2, label="aviso u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="r", linewidth=2)
    # ax0.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax0.legend()
    ax0.set_ylabel("aviso u m/s")

    aviso_v2.plot(ax=ax1, color="b", linewidth=2, label="aviso v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="r", linewidth=2)

    # percentage of good points of gu_cut
    corr_u = xs.pearson_r(gu_cut, aviso_u2, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, aviso_u2, dim="time", skipna=True)
    bias_var_u = aviso_u2 - gu_cut
    bias_u = bias_var_u.mean(dim="time")

    corr_v = xs.pearson_r(gv_cut, aviso_v2, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, aviso_v2, dim="time", skipna=True)
    bias_var_v = aviso_v2 - gv_cut
    bias_v = bias_var_v.mean(dim="time")

    corrs_u.append(corr_u.item())
    rmses_u.append(rmse_u.item())
    biass_u.append(bias_u.item())
    ngp_u.append(gu_cut_ngp.item())

    corrs_v.append(corr_v.item())
    rmses_v.append(rmse_v.item())
    biass_v.append(bias_v.item())
    ngp_v.append(gv_cut_ngp.item())

    print(f"Correlation between gu and u_aviso_at: {corr_u.item():.2f} {gu_cut_ngp.item():.2f}")
    print(f"Correlation between gv and v_aviso_at: {corr_v.item():.2f} {gv_cut_ngp.item():.2f}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.15,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("aviso v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor="red",
        markeredgecolor="black",
        markeredgewidth=2,
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
```

```
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    ln_sf(
        f"guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
        "fig03.png",
    )
    # plt.show()
    plt.close("all")
    # ds_aviso_u.close()
dse.close()

```

```python
import os
import sys

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from pathlib import Path
from tools_xtrackm import ln_sf

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

SAT = "TP+J1+J2+J3+S6A"
CSV_NO_EK = f"drifter_ve_vs_gu_{SAT}.csv"
CSV_W_EK = f"drifter_ve_vs_gu_ek_{SAT}.csv"
OUT_PNG = f"scatter_drifter_ve_vs_gu_ww_ek_{SAT}.png"
COAST_DIST_PATH = "~/allData/topo/GMT_intermediate_coast_distance_01d_track_reg.nc"
PROXIMITY_DIST_M = 1000000.0
MAX_ABS_TIME_DIFF_DAYS = None


def get_coast_distance_km(lons, lats):
    with xr.open_dataset(os.path.expanduser(COAST_DIST_PATH)) as ds_coast:
        lon_da = xr.DataArray(np.asarray(lons, dtype=float), dims="point")
        lat_da = xr.DataArray(np.asarray(lats, dtype=float), dims="point")
        coast_km = ds_coast["coast_distance"].sel(lon=lon_da, lat=lat_da, method="nearest")
    return coast_km.values.astype(float)


def compute_time_diff_days(df):
    drifter_t = pd.to_datetime(df["drifter_time"], errors="coerce")
    gu_t = pd.to_datetime(df["gu_time"], errors="coerce")
    return (drifter_t - gu_t).dt.total_seconds() / 86400.0


def summarize_time_diff_days(dt_days, label):
    dt_days = dt_days[np.isfinite(dt_days.to_numpy(dtype=float))]
    if dt_days.size == 0:
        print(f"[{label}] No valid drifter_time/gu_time pairs for temporal-difference stats.")
        return
    dt_abs = np.abs(dt_days)
    mode_signed = pd.Series(np.round(dt_days, 6)).mode().to_numpy()
    mode_abs = pd.Series(np.round(dt_abs, 6)).mode().to_numpy()
    mode_signed_str = ", ".join([f"{v:.6f}" for v in mode_signed[:5]])
    mode_abs_str = ", ".join([f"{v:.6f}" for v in mode_abs[:5]])
    print(
        f"[{label}] Time diff days (drifter_time - gu_time): "
        f"min={dt_days.min():.6f}, max={dt_days.max():.6f}, "
        f"mean={dt_days.mean():.6f}, median={dt_days.median():.6f}, mode={mode_signed_str}"
    )
    print(
        f"[{label}] Abs time diff days: "
        f"min={dt_abs.min():.6f}, max={dt_abs.max():.6f}, "
        f"mean={dt_abs.mean():.6f}, median={dt_abs.median():.6f}, mode={mode_abs_str}"
    )


def load_filtered_df(csv_path, label):
    df = pd.read_csv(csv_path)
    coast_km = get_coast_distance_km(df["lon_inter"].values, df["lat_inter"].values)
    proximity_km = PROXIMITY_DIST_M / 1000.0
    keep = np.isfinite(coast_km) & (np.abs(coast_km) <= proximity_km)
    df = df.loc[keep].copy()

    dt_days = compute_time_diff_days(df)
    if MAX_ABS_TIME_DIFF_DAYS is not None:
        keep_time = np.isfinite(dt_days.to_numpy(dtype=float)) & (
            np.abs(dt_days.to_numpy(dtype=float)) <= float(MAX_ABS_TIME_DIFF_DAYS)
        )
        df = df.loc[keep_time].copy()
        dt_days = dt_days.loc[df.index]

    summarize_time_diff_days(dt_days, label)
    return df


def fit_and_stats(x, y):
    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))
    return slope, intercept, r, stderr


def prepare_xy(df, x_col):
    x = df[x_col].to_numpy(dtype=float)
    y = df["drifter_ve"].to_numpy(dtype=float)
    valid = np.isfinite(x) & np.isfinite(y)
    return x[valid], y[valid]


def prepare_xy_with_ek(df):
    x = df["gu"].to_numpy(dtype=float) + df["u_ek"].to_numpy(dtype=float)
    y = df["drifter_ve"].to_numpy(dtype=float)
    valid = np.isfinite(x) & np.isfinite(y)
    return x[valid], y[valid]


def plot_panel(ax, x, y, panel_title, x_label, y_label):
    slope, intercept, r, stderr = fit_and_stats(x, y)
    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1)

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
```

```
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=10,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="white", alpha=0.55, edgecolor="gray"),
        color="k",
    )

    ax.text(
        0.5,
        0.97,
        panel_title,
        transform=ax.transAxes,
        va="top",
        ha="center",
        fontsize=11,
        bbox=dict(boxstyle="round,pad=0.25", facecolor="White", alpha=0.35, edgecolor="gray"),
        color="black",
    )
    ax.set_xlabel(x_label)
    # ax.set_ylabel("drifter ve (m/s)")
    ax.set_ylabel(y_label)
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right")
    return hb


def main():
    none = ""
    df_no_ek = load_filtered_df(CSV_NO_EK, "without_ek")
    df_w_ek = load_filtered_df(CSV_W_EK, "with_ek")

    x_no_ek, y_no_ek = prepare_xy(df_no_ek, "gu")
    x_w_ek, y_w_ek = prepare_xy_with_ek(df_w_ek)

    if x_no_ek.size < 3 or x_w_ek.size < 3:
        print(
            "Insufficient valid points after filtering.\n"
            f"without_ek: {x_no_ek.size} points, with_ek: {x_w_ek.size} points.\n"
            f"Current filters: PROXIMITY_DIST_M={PROXIMITY_DIST_M}, "
            f"MAX_ABS_TIME_DIFF_DAYS={MAX_ABS_TIME_DIFF_DAYS}"
        )
        return

    all_vals = np.concatenate([x_no_ek, y_no_ek, x_w_ek, y_w_ek])
    vmin = np.nanmin(all_vals)
    vmax = np.nanmax(all_vals)

    fig, axes = plt.subplots(1, 2, figsize=(16, 7), sharex=True, sharey=True)

    hb0 = plot_panel(
        axes[0],
        x_no_ek,
        y_no_ek,
        "Without Ekman",
        "Altimetry gu (m/s)",
        "Drifter ve (m/s)"
    )
    hb1 = plot_panel(
        axes[1],
        x_w_ek,
        y_w_ek,
        "With Ekman",
        "Altimetry gu + u_ek (m/s)",
        none
    )

    for ax in axes:
        # ax.set_xlim(vmin, vmax)
        # ax.set_ylim(vmin, vmax)
        ax.set_xlim(vmin, 1.5)
        ax.set_ylim(vmin, 1.5)

    cmax = max(hb0.get_array().max(), hb1.get_array().max())
    hb0.set_clim(1, cmax)
    hb1.set_clim(1, cmax)

    # fig.suptitle(
    #     f"{SAT}: Drifter ve comparison | coast <= {PROXIMITY_DIST_M:.0f} m, "
    #     f"|dt| <= {MAX_ABS_TIME_DIFF_DAYS if MAX_ABS_TIME_DIFF_DAYS is not None else 'all'} d"
    # )
    fig.tight_layout(rect=[0.0, 0.0, 0.89, 0.95])
    cax = fig.add_axes([0.91, 0.13, 0.015, 0.75])
    cbar = fig.colorbar(hb1, cax=cax)
    cbar.set_label("Count")
    fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)

    print(f"Rows after filters (without_ek): {len(df_no_ek)}")
    print(f"Rows after filters (with_ek): {len(df_w_ek)}")
    print(f"Saved {OUT_PNG}")

    ln_sf(OUT_PNG, png_file)

if __name__ == "__main__":
    main()
```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from tools_xtrackm import ln_sf

only_df = pd.read_csv(
    "oscar_onlyVStrack_corrs_rmses_biass_TP+J1+J2+J3+S6A_u.csv")
ekmn_df = pd.read_csv(
    "oscar_ekmnVStrack_corrs_rmses_biass_TP+J1+J2+J3+S6A_u.csv")

# Create unique column combining zero-padded sat and track_self, track_other
only_df["unique_id"] = (only_df["sat"].apply(lambda s: s.zfill(20)) + "_" +
                        only_df["track_self"].astype(str) + "_" +
                        only_df["track_other"].astype(str))
only_df = only_df.set_index("unique_id")

ekmn_df["unique_id"] = (ekmn_df["sat"].apply(lambda s: s.zfill(20)) + "_" +
                        ekmn_df["track_self"].astype(str) + "_" +
                        ekmn_df["track_other"].astype(str))
ekmn_df = ekmn_df.set_index("unique_id")

common_ids = only_df.index.intersection(ekmn_df.index)
only_df = only_df.loc[common_ids]
ekmn_df = ekmn_df.loc[common_ids]

# Extract metrics for plotting
metrics = ["biass", "corrs", "rmses"]
titles = ["Bias", "Correlation", "Rmse"]

# Prepare figure with 2x2 grid
fig, axs = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle("comparison metrics with oscar u: track gu only VS track gu + ekman", fontsize=14)

summary_lines = []

for i, metric in enumerate(metrics):
    ax = axs[i // 2, i % 2]
    valid_mask = only_df[metric].notna() & ekmn_df[metric].notna()
    only_df1 = only_df[valid_mask]
    ekmn_df1 = ekmn_df[valid_mask]
    # Get data
    only_df_data = only_df1[metric].values
    ekmn_df_data = ekmn_df1[metric].values
    indices = np.arange(len(only_df_data))
    ax.plot(indices, only_df_data, label="track gu only")
    ax.plot(indices, ekmn_df_data, label="track gu+ekman")
    # draw zero line
    ax.axhline(0, color="gray", linestyle="--")
    ax.set_xlabel("Data Point Index")
    ax.set_ylabel(titles[i])
    ax.set_title(titles[i])
    ax.legend()
    # collect summary text for this metric
    if len(only_df_data) > 0:
        mean_o = np.nanmean(only_df_data)
        mean_e = np.nanmean(ekmn_df_data)
        diff = mean_e - mean_o
        cnt = len(only_df_data)
        summary_lines.append(f"\n{titles[i]}:")
        summary_lines.append(f" count = {cnt}")
        summary_lines.append(f" mean track u = {mean_o:.4f}")
        summary_lines.append(f" mean track u + ekman = {mean_e:.4f}")
        summary_lines.append(f" mean improvement = {diff:.4f}")
    else:
        summary_lines.append(f"\n{titles[i]}: no valid paired data")

ax_sum = axs[1, 1]
ax_sum.axis("off")
text = " ".join(summary_lines)

ax_sum.annotate(
    "\n".join(summary_lines),
    xy=(0, 1),
    xycoords="axes fraction",
    va="top",
    ha="left",
    fontsize=10,
    family="monospace",
    bbox=dict(boxstyle="round,pad=0.5", facecolor="white", alpha=0.7),
    annotation_clip=False,
)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(f"track_guvVSoscar_ww_ekman_u.png")
ln_sf("track_guvVSoscar_ww_ekman_u.png", "fig04.png")
# plt.show()

```

```python
import math
import sys

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs
from matplotlib.colors import ListedColormap
from pathlib import Path

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

radar_str = "TN"
track_self_here = 608
track_other_here = 137
sat_here = "S3A"
width, height = 13, 6
TRACKS_REG = (73, 99, 0, 26)
d = 1.5

cmap1 = cmo.cm.diff
cmap2 = ListedColormap(['white'])
oscar_dir = "/home/srinivasu/allData/oscar_0.25/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
oscar_extracted_dir = f"/home/srinivasu/xtrackm/guv/oscar_at_intersections/"

REG = radar_extents[radar_str + "_BLOCK"]
gdf = gpd.read_file(f"polygon_shapefile_around_{radar_str}.shp")
polygon = gdf.geometry.iloc[0]
radar_tsta_o, radar_tend_o = radar_times[radar_str]

OUT_PNG = f"guv_timeseries_{sat_here}_{track_self_here}_{track_other_here}_{radar_str}.png"

dsr = xr.open_dataset(
    f"/home/srinivasu/allData/radar/{radar_str}Codar_daily.nc")
# print(dsr)
dsr = dsr.rename({
    "XAXS": "longitude",
    "YAXS": "latitude",
    "ZAXS": "lev",
    "TAXIS1D": "time"
})
# print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
s_radar = 0.01 * dsr.S_RADAR.isel(lev=0, drop=True)
sizes = u_radar.sizes
ln = sizes["time"]

df = pd.read_csv(f"../guv/tracks_intersections_{sat_here}_1.csv")

df_filter = df[(df["track_self"] == track_self_here) & (df["track_other"] == track_other_here)]

ln = len(df)
for idx, row in df_filter.iterrows():
    sat1 = row["sat"]
    track_number_self = str(row["track_self"])
    track_number_other = str(row["track_other"])
    lon1 = row["lons_inter"]
    lat1 = row["lats_inter"]
    lons_inter1 = row["lons_inter"]
    lats_inter1 = row["lats_inter"]
    x_from_coast_self1 = row["x_from_coast_self"]
    x_from_coast_other1 = row["x_from_coast_other"]
    angle_acute = row["angle_acute"]
    angle_obtuse = row["angle_obtuse"]
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    overlap_tsta, overlap_tend = overlap_dates(track_tsta_o, track_tend_o,
                                               radar_tsta_o, radar_tend_o)
    ru_at = u_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    rv_at = v_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    f_oscar = f"{oscar_extracted_dir}/oscar_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc"
    ds_oscar = xr.open_dataset(f_oscar)
    oscar_u = ds_oscar.u
    oscar_v = ds_oscar.v
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_self}_loess_0.2.nc"
    f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_other}_loess_0.2.nc"
    ds_self = xr.open_dataset(f_self, decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
```

```
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)
    gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
    fig = plt.figure(figsize=(12, 10))
    ax0 = fig.add_subplot(3, 1, 2)
    ax1 = fig.add_subplot(3, 1, 3)

    gu_cut = gu
    gv_cut = gv
    tfreq = sats_tfreq[sat1]

    oscar_u = oscar_u.resample(time=f"{tfreq}D").mean()
    ru_at = ru_at.resample(time=f"{tfreq}D").mean()
    oscar_u = oscar_u.interp(time=gu_cut.time)
    ru_at = ru_at.interp(time=gu_cut.time)
    oscar_max = oscar_u.max().item()
    gu_cut = gu_cut.where(gu_cut < oscar_max, np.nan)
    gu_cut = gu_cut.where(gu_cut > -oscar_max, np.nan)

    oscar_v = oscar_v.resample(time=f"{tfreq}D").mean()
    rv_at = rv_at.resample(time=f"{tfreq}D").mean()
    oscar_v = oscar_v.interp(time=gv_cut.time)
    rv_at = rv_at.interp(time=gu_cut.time)
    oscar_max = oscar_v.max().item()
    gv_cut = gv_cut.where(gv_cut < oscar_max, np.nan)
    gv_cut = gv_cut.where(gv_cut > -oscar_max, np.nan)

    oscar_u.plot(ax=ax0, color="b", linewidth=2, label="oscar u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="k", linewidth=2)
    ru_at.plot(ax=ax0, label=f"radar zonal", color="r", linewidth=2)
    ax0.legend()
    ax0.set_ylabel("oscar u m/s")
    oscar_v.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="k", linewidth=2)
    rv_at.plot(ax=ax1, label=f"radar meridional", color="r", linewidth=2)
    corr_u = xs.pearson_r(gu_cut, ru_at, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, ru_at, dim="time", skipna=True)
    bias_var_u = ru_at - gu_cut
    bias_u = bias_var_u.mean(dim="time")
    corr_v = xs.pearson_r(gv_cut, rv_at, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, rv_at, dim="time", skipna=True)
    bias_var_v = rv_at - gv_cut
    bias_v = bias_var_v.mean(dim="time")
    print(f"Correlation between gu and u_radar: {corr_u.item()}")
    print(f"Correlation between gv and v_radar: {corr_v.item()}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.05,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("oscar v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap2)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.add_geometries(
        [polygon],
        ccrs.PlateCarree(),
        facecolor="blue",
        edgecolor="red",
        alpha=0.2,
    )
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=4,
        markerfacecolor="red",
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    # plot.text radar_str
    plt.text(
        0.15,
        0.90,
        f"radar region: {radar_str}",
        transform=ax2.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
```

```
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"guv_timeseries_{sat1}_{track_number_self}_{track_number_other}_{radar_str}.png",
    )
    ln_sf(
        f"guv_timeseries_{sat1}_{track_number_self}_{track_number_other}_{radar_str}.png", "fig06.png"
    )
    ln_sf(OUT_PNG, png_file)
    plt.show()
    plt.close("all")

dse.close()

```

```python
import math
import sys

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs
from matplotlib.colors import ListedColormap
from pathlib import Path

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

radar_str = "TN"
track_self_here = 608
track_other_here = 137
sat_here = "S3A"
width, height = 13, 6
frac1 = 0.05
TRACKS_REG = (73, 99, 0, 26)
d = 1.5

cmap1 = cmo.cm.diff
cmap2 = ListedColormap(['white'])
oscar_dir = "/home/srinivasu/allData/oscar_0.25/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
oscar_extracted_dir = f"/home/srinivasu/xtrackm/guv/oscar_at_intersections/"

REG = radar_extents[radar_str + "_BLOCK"]
gdf = gpd.read_file(f"polygon_shapefile_around_{radar_str}.shp")
polygon = gdf.geometry.iloc[0]
radar_tsta_o, radar_tend_o = radar_times[radar_str]

OUT_PNG = f"guv_timeseries_{sat_here}_{track_self_here}_{track_other_here}_{radar_str}.png"

dsr = xr.open_dataset(
    f"/home/srinivasu/allData/radar/{radar_str}Codar_daily.nc")
# print(dsr)
dsr = dsr.rename({
    "XAXS": "longitude",
    "YAXS": "latitude",
    "ZAXS": "lev",
    "TAXIS1D": "time"
})
# print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
s_radar = 0.01 * dsr.S_RADAR.isel(lev=0, drop=True)
sizes = u_radar.sizes
ln = sizes["time"]

df = pd.read_csv(f"../guv/tracks_intersections_{sat_here}_1.csv")

df_filter = df[(df["track_self"] == track_self_here) & (df["track_other"] == track_other_here)]

ln = len(df)
for idx, row in df_filter.iterrows():
    sat1 = row["sat"]
    track_number_self = str(row["track_self"])
    track_number_other = str(row["track_other"])
    lon1 = row["lons_inter"]
    lat1 = row["lats_inter"]
    lons_inter1 = row["lons_inter"]
    lats_inter1 = row["lats_inter"]
    x_from_coast_self1 = row["x_from_coast_self"]
    x_from_coast_other1 = row["x_from_coast_other"]
    angle_acute = row["angle_acute"]
    angle_obtuse = row["angle_obtuse"]
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    overlap_tsta, overlap_tend = overlap_dates(track_tsta_o, track_tend_o,
                                               radar_tsta_o, radar_tend_o)
    ru_at = u_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    rv_at = v_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    f_oscar = f"{oscar_extracted_dir}/oscar_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc"
    ds_oscar = xr.open_dataset(f_oscar)
    oscar_u = ds_oscar.u
    oscar_v = ds_oscar.v
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_{frac1}/track_sla_along_{sat1}_{track_number_self}_loess_{frac1}.nc"
    f_other_smooth = f"../computed/sla_loess_{frac1}/track_sla_along_{sat1}_{track_number_other}_loess_{frac1}.nc"
    ds_self = xr.open_dataset(f_self, decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
```

```
                                drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)
    gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
    fig = plt.figure(figsize=(12, 10))
    ax0 = fig.add_subplot(3, 1, 2)
    ax1 = fig.add_subplot(3, 1, 3)

    gu_cut = gu
    gv_cut = gv
    tfreq = sats_tfreq[sat1]

    oscar_u = oscar_u.resample(time=f"{tfreq}D").mean()
    ru_at = ru_at.resample(time=f"{tfreq}D").mean()
    oscar_u = oscar_u.interp(time=gu_cut.time)
    ru_at = ru_at.interp(time=gu_cut.time)
    oscar_max = oscar_u.max().item()
    gu_cut = gu_cut.where(gu_cut < oscar_max, np.nan)
    gu_cut = gu_cut.where(gu_cut > -oscar_max, np.nan)

    oscar_v = oscar_v.resample(time=f"{tfreq}D").mean()
    rv_at = rv_at.resample(time=f"{tfreq}D").mean()
    oscar_v = oscar_v.interp(time=gv_cut.time)
    rv_at = rv_at.interp(time=gu_cut.time)
    oscar_max = oscar_v.max().item()
    gv_cut = gv_cut.where(gv_cut < oscar_max, np.nan)
    gv_cut = gv_cut.where(gv_cut > -oscar_max, np.nan)

    oscar_u.plot(ax=ax0, color="b", linewidth=2, label="oscar u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="k", linewidth=2)
    ru_at.plot(ax=ax0, label=f"radar zonal", color="r", linewidth=2)
    ax0.legend()
    ax0.set_ylabel("oscar u m/s")
    oscar_v.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="k", linewidth=2)
    rv_at.plot(ax=ax1, label=f"radar meridional", color="r", linewidth=2)
    corr_u = xs.pearson_r(gu_cut, ru_at, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, ru_at, dim="time", skipna=True)
    bias_var_u = ru_at - gu_cut
    bias_u = bias_var_u.mean(dim="time")
    corr_v = xs.pearson_r(gv_cut, rv_at, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, rv_at, dim="time", skipna=True)
    bias_var_v = rv_at - gv_cut
    bias_v = bias_var_v.mean(dim="time")
    print(f"Correlation between gu and u_radar: {corr_u.item()}")
    print(f"Correlation between gv and v_radar: {corr_v.item()}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax1.text(
        0.05,
        0.80,
        f"a)",
        transform=ax1.transAxes,
        fontsize=14,
        fontweight="bold",
        color="k",
    )
    ax0.text(
        0.05,
        0.80,
        f"b)",
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
        color="k",
    )
    plt.text(
        0.05,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("oscar v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap2)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.add_geometries(
        [polygon],
        ccrs.PlateCarree(),
        facecolor="blue",
        edgecolor="red",
        alpha=0.2,
    )
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=4,
        markerfacecolor="red",
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    # plot.text radar_str
```

```
    plt.text(
        0.15,
        0.90,
        f"radar region: {radar_str}",
        transform=ax2.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"guv_timeseries_{sat1}_{track_number_self}_{track_number_other}_{radar_str}.png",
    )
    ln_sf(
        f"guv_timeseries_{sat1}_{track_number_self}_{track_number_other}_{radar_str}.png", "fig06.png"
    )
    ln_sf(OUT_PNG, png_file)
    plt.show()
    plt.close("all")

dse.close()

```

```python
import xarray as xr
from tools_xtrackm import *
from namesm import sats_new
import glob
import os
from matplotlib.collections import LineCollection
from pathlib import Path
import sys

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

sat = "ERS1+ERS2+ENV+SRL"

REG = (80.0, 92.0, 11.0, 21.0)
WBOB = (78.0, 87.0, 8.0, 21.0)
WBOB2 = (78.0, 97.0, 0.0, 25.0)
proj = ccrs.PlateCarree()

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 10),
                       subplot_kw={'projection': proj})
# adjust subplots to have minimum space between them
month_num = 2
month_name = months[month_num+1]
info =f"{sat} {months[month_num+1]}"
fig.subplots_adjust(wspace=0.1)
ax1.set_extent([*WBOB2], crs=proj)
ax1.coastlines()
# ax1.legend()
gl = ax1.gridlines(draw_labels=True, linewidth=0.5, color='gray', alpha=0.7, linestyle='--')
gl.top_labels = gl.right_labels = False  # Hide top and right labels
plt.text(0.3, 0.95, info,
         horizontalalignment='center',
         verticalalignment='center',
         fontsize=15,
         fontweight="bold",
         transform=ax1.transAxes)
for f_sla in sorted(
    glob.glob(
        f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
    )
):
    basename = os.path.basename(f_sla)
    ds_sla = xr.open_dataset(f_sla, decode_times=False)
    trackn = ds_sla.Pass
    # f = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{trackn}_loess_0.2.nc"
    f = f"../computed/sla_loess_0.05a/track_sla_along_{sat}_{trackn}_loess_0.05.nc"
    lons_track = ds_sla.lon.values
    lats_track = ds_sla.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    lon_equat = lons_track[0]
    lat_equat = lats_track[0]
    # compute tangent vectors or slope of track at each point
    dlon = np.gradient(lons_track_rev)
    dlat = np.gradient(lats_track_rev)
    tangent_mag = np.hypot(dlon, dlat)
    tangent_u = dlon / tangent_mag
    tangent_v = dlat / tangent_mag
    # Convert latitude to radians
    phi = np.deg2rad(lats_track_rev)
    # Length of one degree of latitude and longitude in meters
    meters_per_degree_lat = 111_320  # Approximate value
    meters_per_degree_lon = 111_320 * np.cos(phi)
    # Handle division by zero at the poles
    meters_per_degree_lon = np.where(meters_per_degree_lon == 0,
                                     np.nan, meters_per_degree_lon)
    # Compute normal vectors
    normal_u = -tangent_v
    normal_v = tangent_u
    theta = np.arctan2(lat_coast-lat_equat, lon_coast-lon_equat)
    print(theta, "--------------------------------------------------")
    # if is_within_region(lon_coast, lat_coast, *REG):
    if True:
        ds_smooth = xr.open_dataset(f)
        sla_smooth = ds_smooth.sla_smooth
        gc = compute_geostrophy_gc(ds_sla, sla_smooth)
        gc = gc.where(gc > -2.5)
        gc = gc.where(gc < 2.5)
        gc_clim = gc.groupby("time.month").mean("time")
        gc_clim_m = gc_clim.sel(month=month_num)
        # Convert gc from m/s to degrees (assuming cross-track direction)
        gc_deg = gc_clim_m.values / meters_per_degree_lat
        # Calculate displacements
        delta_lon = normal_u * gc_deg
        delta_lat = normal_v * gc_deg
        # Apply scaling factor for visualization
        scaling_factor = 1e5  # Adjust as needed
        delta_lon *= scaling_factor
        delta_lat *= scaling_factor
        # Compute endpoints
        lon_start = lons_track_rev
        lat_start = lats_track_rev
        lon_end = lon_start + delta_lon
        lat_end = lat_start + delta_lat
        # Create line segments
        segments = [[[lon_start[i], lat_start[i]], [lon_end[i], lat_end[i]]]
                    for i in range(len(lon_start))]
        # Create a LineCollection
        lc = LineCollection(segments, colors='red', linewidths=1,
                            transform=proj)
        ax1.plot(lons_track_rev, lats_track_rev, 'k-',
                 transform=proj, label='Ground Track')
        ax1.add_collection(lc)

month_num = 10
month_name = months[month_num+1]
info =f"{sat} {months[month_num+1]}"
ax2.set_extent([*WBOB2], crs=proj)
ax2.coastlines()
# ax1.legend()
gl = ax2.gridlines(draw_labels=True, linewidth=0.5, color='gray', alpha=0.7, linestyle='--')
gl.top_labels = gl.right_labels = False  # Hide top and right labels
plt.text(0.3, 0.95, info,
         horizontalalignment='center',
```

```
         verticalalignment='center',
         fontsize=15,
         fontweight="bold",
         transform=ax2.transAxes)
for f_sla in sorted(
    glob.glob(
        f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
    )
):
    basename = os.path.basename(f_sla)
    ds_sla = xr.open_dataset(f_sla, decode_times=False)
    trackn = ds_sla.Pass
    # f = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{trackn}_loess_0.2.nc"
    f = f"../computed/sla_loess_0.05a/track_sla_along_{sat}_{trackn}_loess_0.05.nc"
    lons_track = ds_sla.lon.values
    lats_track = ds_sla.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    lon_equat = lons_track[0]
    lat_equat = lats_track[0]
    # compute tangent vectors or slope of track at each point
    dlon = np.gradient(lons_track_rev)
    dlat = np.gradient(lats_track_rev)
    tangent_mag = np.hypot(dlon, dlat)
    tangent_u = dlon / tangent_mag
    tangent_v = dlat / tangent_mag
    # Convert latitude to radians
    phi = np.deg2rad(lats_track_rev)
    # Length of one degree of latitude and longitude in meters
    meters_per_degree_lat = 111_320  # Approximate value
    meters_per_degree_lon = 111_320 * np.cos(phi)
    # Handle division by zero at the poles
    meters_per_degree_lon = np.where(meters_per_degree_lon == 0,
                                     np.nan, meters_per_degree_lon)
    # Compute normal vectors
    normal_u = -tangent_v
    normal_v = tangent_u
    theta = np.arctan2(lat_coast-lat_equat, lon_coast-lon_equat)
    print(theta, "----------------------------------------------------")
    # if is_within_region(lon_coast, lat_coast, *REG):
    if True:
        ds_smooth = xr.open_dataset(f)
        sla_smooth = ds_smooth.sla_smooth
        gc = compute_geostrophy_gc(ds_sla, sla_smooth)
        gc = gc.where(gc > -2.5)
        gc = gc.where(gc < 2.5)
        gc_clim = gc.groupby("time.month").mean("time")
        gc_clim_m = gc_clim.sel(month=month_num)
        # Convert gc from m/s to degrees (assuming cross-track direction)
        gc_deg = gc_clim_m.values / meters_per_degree_lat
        # Calculate displacements
        delta_lon = normal_u * gc_deg
        delta_lat = normal_v * gc_deg
        # Apply scaling factor for visualization
        scaling_factor = 1e5  # Adjust as needed
        delta_lon *= scaling_factor
        delta_lat *= scaling_factor
        # Compute endpoints
        lon_start = lons_track_rev
        lat_start = lats_track_rev
        lon_end = lon_start + delta_lon
        lat_end = lat_start + delta_lat
        # Create line segments
        segments = [[[lon_start[i], lat_start[i]], [lon_end[i], lat_end[i]]]
                    for i in range(len(lon_start))]
        # Create a LineCollection
        lc = LineCollection(segments, colors='red', linewidths=1,
                            transform=proj)
        ax2.plot(lons_track_rev, lats_track_rev, 'k-',
                 transform=proj, label='Ground Track')
        ax2.add_collection(lc)
plt.savefig(f"cross_structure_x_arrows_map_{sat}_1by2.png",
            bbox_inches='tight', dpi=300)
ln_sf(f"cross_structure_x_arrows_map_{sat}_1by2.png", png_file)
# plt.show()
plt.close("all")

```

```python
import xarray as xr
from tools_xtrackm import *
from namesm import sats_new
import glob
import os
from matplotlib.collections import LineCollection
from pathlib import Path
import sys

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

method = "loess"
frac1 = "0.05"
sat = "ERS1+ERS2+ENV+SRL"

REG = (80.0, 92.0, 11.0, 21.0)
WBOB = (78.0, 87.0, 8.0, 21.0)
WBOB2 = (78.0, 97.0, 0.0, 25.0)
proj = ccrs.PlateCarree()

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 10),
                               subplot_kw={'projection': proj})
# adjust subplots to have minimum space between them
month_num = 2
month_name = months[month_num+1]
info =f"{sat} {months[month_num+1]}"
fig.subplots_adjust(wspace=0.1)
ax1.set_extent([*WBOB2], crs=proj)
ax1.coastlines()
# ax1.legend()
gl = ax1.gridlines(draw_labels=True, linewidth=0.5, color='gray', alpha=0.7, linestyle='--')
gl.top_labels = gl.right_labels = False  # Hide top and right labels
plt.text(0.3, 0.95, info,
         horizontalalignment='center',
         verticalalignment='center',
         fontsize=15,
         fontweight="bold",
         transform=ax1.transAxes)
for f_sla in sorted(
    glob.glob(
        f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
    )
):
    basename = os.path.basename(f_sla)
    ds_sla = xr.open_dataset(f_sla, decode_times=False)
    trackn = ds_sla.Pass
    # f = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{trackn}_loess_0.2.nc"
    f = f"../computed/sla_{method}_{frac1}a/track_sla_along_{sat}_{trackn}_{method}_{frac1}.nc"
    lons_track = ds_sla.lon.values
    lats_track = ds_sla.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    lon_equat = lons_track[0]
    lat_equat = lats_track[0]
    # compute tangent vectors or slope of track at each point
    dlon = np.gradient(lons_track_rev)
    dlat = np.gradient(lats_track_rev)
    tangent_mag = np.hypot(dlon, dlat)
    tangent_u = dlon / tangent_mag
    tangent_v = dlat / tangent_mag
    # Convert latitude to radians
    phi = np.deg2rad(lats_track_rev)
    # Length of one degree of latitude and longitude in meters
    meters_per_degree_lat = 111_320  # Approximate value
    meters_per_degree_lon = 111_320 * np.cos(phi)
    # Handle division by zero at the poles
    meters_per_degree_lon = np.where(meters_per_degree_lon == 0,
                                     np.nan, meters_per_degree_lon)
    # Compute normal vectors
    normal_u = -tangent_v
    normal_v = tangent_u
    theta = np.arctan2(lat_coast-lat_equat, lon_coast-lon_equat)
    print(theta, "-------------------------------------------------")
    # if is_within_region(lon_coast, lat_coast, *REG):
    if True:
        ds_smooth = xr.open_dataset(f)
        sla_smooth = ds_smooth.sla_smooth
        gc = compute_geostrophy_gc(ds_sla, sla_smooth)
        gc = gc.where(gc > -2.5)
        gc = gc.where(gc < 2.5)
        gc_clim = gc.groupby("time.month").mean("time")
        gc_clim_m = gc_clim.sel(month=month_num)
        # Convert gc from m/s to degrees (assuming cross-track direction)
        gc_deg = gc_clim_m.values / meters_per_degree_lat
        # Calculate displacements
        delta_lon = normal_u * gc_deg
        delta_lat = normal_v * gc_deg
        # Apply scaling factor for visualization
        scaling_factor = 1e5  # Adjust as needed
        delta_lon *= scaling_factor
        delta_lat *= scaling_factor
        # Compute endpoints
        lon_start = lons_track_rev
        lat_start = lats_track_rev
        lon_end = lon_start + delta_lon
        lat_end = lat_start + delta_lat
        # Create line segments
        segments = [[[lon_start[i], lat_start[i]], [lon_end[i], lat_end[i]]]
                    for i in range(len(lon_start))]
        # Create a LineCollection
        lc = LineCollection(segments, colors='red', linewidths=1,
                            transform=proj)
        ax1.plot(lons_track_rev, lats_track_rev, 'k-',
                 transform=proj, label='Ground Track')
        ax1.add_collection(lc)

month_num = 10
month_name = months[month_num+1]
info =f"{sat} {months[month_num+1]}"
ax2.set_extent([*WBOB2], crs=proj)
ax2.coastlines()
# ax1.legend()
gl = ax2.gridlines(draw_labels=True, linewidth=0.5, color='gray', alpha=0.7, linestyle='--')
gl.top_labels = gl.right_labels = False  # Hide top and right labels
```

```
plt.text(0.3, 0.95, info,
         horizontalalignment='center',
         verticalalignment='center',
         fontsize=15,
         fontweight="bold",
         transform=ax2.transAxes)
for f_sla in sorted(
    glob.glob(
        f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
    )
):
    basename = os.path.basename(f_sla)
    ds_sla = xr.open_dataset(f_sla, decode_times=False)
    trackn = ds_sla.Pass
    # f = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{trackn}_loess_0.2.nc"
    f = f"../computed/sla_{method}_{frac1}a/track_sla_along_{sat}_{trackn}_{method}_{frac1}.nc"
    lons_track = ds_sla.lon.values
    lats_track = ds_sla.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    lon_equat = lons_track[0]
    lat_equat = lats_track[0]
    # compute tangent vectors or slope of track at each point
    dlon = np.gradient(lons_track_rev)
    dlat = np.gradient(lats_track_rev)
    tangent_mag = np.hypot(dlon, dlat)
    tangent_u = dlon / tangent_mag
    tangent_v = dlat / tangent_mag
    # Convert latitude to radians
    phi = np.deg2rad(lats_track_rev)
    # Length of one degree of latitude and longitude in meters
    meters_per_degree_lat = 111_320  # Approximate value
    meters_per_degree_lon = 111_320 * np.cos(phi)
    # Handle division by zero at the poles
    meters_per_degree_lon = np.where(meters_per_degree_lon == 0,
                                     np.nan, meters_per_degree_lon)
    # Compute normal vectors
    normal_u = -tangent_v
    normal_v = tangent_u
    theta = np.arctan2(lat_coast-lat_equat, lon_coast-lon_equat)
    print(theta, "--------------------------------------------------")
    # if is_within_region(lon_coast, lat_coast, *REG):
    if True:
        ds_smooth = xr.open_dataset(f)
        sla_smooth = ds_smooth.sla_smooth
        gc = compute_geostrophy_gc(ds_sla, sla_smooth)
        gc = gc.where(gc > -2.5)
        gc = gc.where(gc < 2.5)
        gc_clim = gc.groupby("time.month").mean("time")
        gc_clim_m = gc_clim.sel(month=month_num)
        # Convert gc from m/s to degrees (assuming cross-track direction)
        gc_deg = gc_clim_m.values / meters_per_degree_lat
        # Calculate displacements
        delta_lon = normal_u * gc_deg
        delta_lat = normal_v * gc_deg
        # Apply scaling factor for visualization
        scaling_factor = 1e5  # Adjust as needed
        delta_lon *= scaling_factor
        delta_lat *= scaling_factor
        # Compute endpoints
        lon_start = lons_track_rev
        lat_start = lats_track_rev
        lon_end = lon_start + delta_lon
        lat_end = lat_start + delta_lat
        # Create line segments
        segments = [[[lon_start[i], lat_start[i]], [lon_end[i], lat_end[i]]]
                    for i in range(len(lon_start))]
        # Create a LineCollection
        lc = LineCollection(segments, colors='red', linewidths=1,
                            transform=proj)
        ax2.plot(lons_track_rev, lats_track_rev, 'k-',
                 transform=proj, label='Ground Track')
        ax2.add_collection(lc)
plt.savefig(f"cross_structure_x_arrows_map_{sat}_1by2.png",
            bbox_inches='tight', dpi=300)
ln_sf(f"cross_structure_x_arrows_map_{sat}_1by2.png", png_file)
# plt.show()
plt.close("all")

```

```python
import glob
import os
import sys
from pathlib import Path

import xarray as xr
from matplotlib.collections import LineCollection
from tools_xtrackm import *

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

frac1 = "0.05"
sat = "ERS1+ERS2+ENV+SRL"
gc_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"

WBOB2 = (78.0, 97.0, 0.0, 25.0)
proj = ccrs.PlateCarree()
scaling_factor = 1e5
meters_per_degree_lat = 111_320


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)


def load_gc_clim_month(sat_name, track_number, month_num):
    f_gc = f"{gc_dir}/gc_at_{sat_name}_{track_number}_{frac1}.nc"
    if not os.path.exists(f_gc):
        return None
    with xr.open_dataset(f_gc) as ds_gc:
        gc = ds_gc[next(iter(ds_gc.data_vars))].load()
    gc = gc.where((gc > -2.5) & (gc < 2.5))
    gc_clim = gc.groupby("time.month").mean("time")
    return gc_clim.sel(month=month_num)


def add_month_panel(ax, month_num):
    info = f"{sat} {months[month_num + 1]}"
    ax.set_extent([*WBOB2], crs=proj)
    ax.coastlines()
    gl = ax.gridlines(draw_labels=True, linewidth=0.5, color="gray", alpha=0.7, linestyle="--")
    gl.top_labels = gl.right_labels = False
    ax.text(
        0.3,
        0.95,
        info,
        horizontalalignment="center",
        verticalalignment="center",
        fontsize=15,
        fontweight="bold",
        transform=ax.transAxes,
    )

    for f_sla in sorted(
        glob.glob(
            f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
        )
    ):
        with xr.open_dataset(f_sla, decode_times=False) as ds_sla:
            # track_number = track_to_str(ds_sla.Pass.item())
            track_number = ds_sla.Pass
            gc_clim_m = load_gc_clim_month(sat, track_number, month_num)
            if gc_clim_m is None:
                continue
            gc_val = float(gc_clim_m.values)
            if not np.isfinite(gc_val):
                continue

            lons_track_rev = ds_sla.lon.values[::-1]
            lats_track_rev = ds_sla.lat.values[::-1]

        dlon = np.gradient(lons_track_rev)
        dlat = np.gradient(lats_track_rev)
        tangent_mag = np.hypot(dlon, dlat)
        tangent_mag = np.where(tangent_mag == 0, np.nan, tangent_mag)
        tangent_u = dlon / tangent_mag
        tangent_v = dlat / tangent_mag
        normal_u = -tangent_v
        normal_v = tangent_u

        gc_deg = gc_val / meters_per_degree_lat
        delta_lon = normal_u * gc_deg * scaling_factor
        delta_lat = normal_v * gc_deg * scaling_factor

        lon_end = lons_track_rev + delta_lon
        lat_end = lats_track_rev + delta_lat
        segments = [
            [[lons_track_rev[i], lats_track_rev[i]], [lon_end[i], lat_end[i]]]
            for i in range(len(lons_track_rev))
            if np.isfinite(lon_end[i]) and np.isfinite(lat_end[i])
        ]
        if not segments:
            continue

        ax.plot(lons_track_rev, lats_track_rev, "k-", transform=proj)
        ax.add_collection(LineCollection(segments, colors="red", linewidths=1, transform=proj))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 10),
                       subplot_kw={'projection': proj})
fig.subplots_adjust(wspace=0.1)
add_month_panel(ax1, month_num=2)
add_month_panel(ax2, month_num=10)
plt.savefig(f"cross_structure_x_arrows_map_{sat}_1by2.png",
            bbox_inches='tight', dpi=300)
ln_sf(f"cross_structure_x_arrows_map_{sat}_1by2.png", png_file)
plt.show()
plt.close("all")

```

```python
import xarray as xr
from tools_xtrackm import *
import glob
import os

import matplotlib.cm as cm
from pathlib import Path
import sys

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

# sats = ["S3A"]
REG = (80.0, 89.0, 11.0, 21.0)
REG = (75.0, 92.0, 10.0, 23.0)
sats = ["S3A"]
sats = ["ERS1+ERS2+ENV+SRL"]
xwidth = 290e3
# Load the YlOrRd colormap
ncolors = 21
cmap1 = cm.get_cmap('YlOrRd', ncolors)  # Split into 10 colors
colors1 = cmap1(np.linspace(0, 1, ncolors))
cmap2 = cm.get_cmap('BuGn', ncolors)  # Split into 10 colors
colors2 = cmap2(np.linspace(0, 1, ncolors))

norm2 = plt.Normalize(10.18, 22.00)
sm2 = plt.cm.ScalarMappable(cmap=cmap2, norm=norm2)
sm2.set_array([])

norm1 = plt.Normalize(10.18, 22.00)
sm1 = plt.cm.ScalarMappable(cmap=cmap1, norm=norm1)
sm1.set_array([])

# fig, ax = plt.subplots(1, 1, figsize=(9, 4))
frac1 = "0.2"
frac1 = "0.05"
for sat in sats[:]:
    fig, ax = plt.subplots(1, 1, figsize=(12, 6))
    # adjust subplots increase x spacing
    plt.subplots_adjust(left=0.1, bottom=0.1, right=0.8, top=0.8, wspace=0.3, hspace=0.3)
    i = 0
    lat_values = []
    for f_sla in sorted(
        glob.glob(
            f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
        )
    ):
        basename = os.path.basename(f_sla)
        ds_sla = xr.open_dataset(f_sla, decode_times=False)
        trackn = ds_sla.Pass
        f = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{trackn}_loess_{frac1}.nc"
        lons_track = ds_sla.lon.values
        lats_track = ds_sla.lat.values
        lons_track_rev = lons_track[::-1]
        lats_track_rev = lats_track[::-1]
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        print(f, lon_coast, lat_coast)
        if is_within_region(lon_coast, lat_coast, *REG):
            lat_values.append(lat_coast)
            ds_smooth = xr.open_dataset(f)
            # print(ds_smooth)
            sla_smooth = ds_smooth.sla_smooth
            print(f"stats of sla_smooth {sat} {trackn} --------------------")
            print(f"stats of gu {sat} {trackn} --------------------")
            gc = compute_geostrophy_gc(ds_sla, sla_smooth)
            gc_clim = gc.groupby("time.month").mean("time")
            gc_clim_xwidth = gc_clim.sel(x=slice(0, xwidth))
            gc_clim_xwidth_m = gc_clim_xwidth.isel(month=2, drop=True) # march
            gc_clim_xwidth_n = gc_clim_xwidth.isel(month=10, drop=True) # november
            gc_clim_xwidth_m.plot(ax=ax, label=f"{lat_coast}",
                                  color=colors1[i])
            x0, y0 = -15, gc_clim_xwidth_m.sel(x=0).values
            plt.text(x0, y0, f"{lat_coast:.2f}", fontsize=10, color=colors1[i])
            gc_clim_xwidth_n.plot(ax=ax, label=f"{lat_coast}",
                                  color=colors2[i])
            x0, y0 = -15, gc_clim_xwidth_n.sel(x=0).values
            plt.text(x0, y0, f"{lat_coast:.2f}", fontsize=10, color=colors2[i])
            # set xticks between 0 to xwidth
            ax.set_xlim(-15, xwidth)
            ax.set_ylim(-0.7, 0.7)
            xticks1 = ax.get_xticks()
            xticks_km = np.round((xticks1 / 1000), 0)
            ax.set_xticks(xticks1, xticks_km)
            ax.set_xlabel("distance along track (km)")
            ax.set_ylabel("geostrophic velocity (m/s)")
            # ax.set_ylim(-0.3, 0.3)
            ax.axhline(0, color="k", linestyle="--")
            i += 1
            # print_stats(gc)
    # plt.legend()
    print(f"number of tracks {i}")
    minlat = min(lat_values)
    maxlat = max(lat_values)
    print(f"min lat {min(lat_values)} max lat {max(lat_values)}")
    pos = ax.get_position()
    # Adjust and add the first colorbar closer to the plot
    cbar_ax1 = fig.add_axes([pos.x1 + 0.01, pos.y0, 0.02, pos.height])  # Adjust 0.01 to control horizontal gap
    cbar_ax1.text(0.5, 0.5, "MARCH", fontsize=10, horizontalalignment="center", verticalalignment="center", transform=cbar_ax1.transAxes, rotation = "vertical")
    cbar = plt.colorbar(sm1, cax=cbar_ax1)
    # cbar.set_label('latitude')
    cbar.set_ticks(list(range(int(minlat), int(maxlat)+1, 2)))
    # Adjust and add the second colorbar even closer
    cbar_ax2 = fig.add_axes([pos.x1 + 0.06, pos.y0, 0.02, pos.height])  # Adjust 0.03 as needed for spacing
    cbar_ax2.text(0.5, 0.5, "NOVEMBER", fontsize=10, horizontalalignment="center", verticalalignment="center", transform=cbar_ax2.transAxes, rotation = "vertical")
    cbar = plt.colorbar(sm2, cax=cbar_ax2)
    cbar.set_label('latitude')
    cbar.set_ticks(list(range(int(minlat), int(maxlat)+1, 2)))

    plt.savefig(
        f"p.gc.clim.lat_wise.along_xwidth_{sat}a_fig09.png",
        bbox_inches="tight",
        dpi=300,
    )
    ln_sf(
        f"p.gc.clim.lat_wise.along_xwidth_{sat}a_fig09.png",
        png_file,
```

```
    )
    # plt.show()
    plt.close("all")

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
from tools_shapely import *
import sys
from pathlib import Path
import sys

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")


def along_across_shore_components(gu, gv, coast_heading_deg):
    """
    Resolve vector (gu, gv) into along-shore and across-shore components.
    Sign convention:
    - coast_heading_deg is measured CCW from +x (East).
    - along_shore is positive in the heading direction.
    - across_shore is positive 90 deg CCW from heading (left-hand side of coast).
      If you need right-hand positive, negate across_shore.
    """
    theta = np.deg2rad(coast_heading_deg)
    c = np.cos(theta)
    s = np.sin(theta)
    # Elementwise projection keeps scalar or array/xarray shape.
    along_shore = gu * c + gv * s
    across_shore = -gu * s + gv * c  # +90 deg CCW
    return along_shore, across_shore


cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

sat_val = "TP+J1+J2+J3+S6A"
track_self_val = 155
track_other_val = 116

df = pd.read_csv(f"tracks_intersections_{sat_val}_1.csv")
df_out = df.copy()

rows = df_out.loc[
    (df_out["sat"] == sat_val)
    & (df_out["track_self"] == track_self_val)
    & (df_out["track_other"] == track_other_val)
]
# Optional input: Gaussian smoothing sigma
sigma = 3
# Optional input: clipping bbox tuple
# Default used by the function is (70, 5, 95, 30)
region_bbox_tuple = (70, 5, 95, 30)

frac1 = "0.05"
i = 0
sat1 = rows["sat"].values[i]
# track_number_self = rows["track_self"].apply(lambda x: str(x)).values[i]
# track_number_other = rows["track_other"].apply(lambda x: str(x)).values[i]
track_number_self = str(rows["track_self"].values[i])
track_number_other = str(rows["track_other"].values[i])
lons_inter1 = rows["lons_inter"].values[i]
lats_inter1 = rows["lats_inter"].values[i]
lon1 = lons_inter1
lat1 = lats_inter1
query_point = (lon1, lat1)
result = coastline_gaussian_tangent(
    query_point=query_point,
    sigma=sigma,
    region_bbox_tuple=region_bbox_tuple,
)
print("heading_angle_deg:", result["heading_angle_deg"])
print("tangent_vector:", result["tangent_vector"])
print("nearest_point:", result["nearest_point"])
print("distance:", result["distance"])
coast_heading_deg = result["heading_angle_deg"]
x_from_coast_self1 = rows["x_from_coast_self"].values[i]
x_from_coast_other1 = rows["x_from_coast_other"].values[i]
angle_acute = rows["angle_acute"].values[i]
angle_obtuse = rows["angle_obtuse"].values[i]
print(
    sat1,
    track_number_self,
    track_number_other,
    lons_inter1,
    lats_inter1,
    x_from_coast_self1,
    x_from_coast_other1,
    angle_acute,
    angle_obtuse,
)
# sys.exit()
f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat1}_{track_number_self}_loess_{frac1}.nc"
f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat1}_{track_number_other}_loess_{frac1}.nc"
ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
ds_self_smooth = xr.open_dataset(f_self_smooth)
sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
sla_self_smooth = ds_self_smooth.sla_smooth
ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
ds_other_smooth = xr.open_dataset(f_other_smooth)
sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
sla_other_smooth = ds_other_smooth.sla_smooth
lons_track_self = ds_self.lon.values
lats_track_self = ds_self.lat.values
lon_coast_self = lons_track_self[-1]  # this on coast
lat_coast_self = lats_track_self[-1]  # this on coast
lon_equat_self = lons_track_self[0]  # this on equator
lat_equat_self = lats_track_self[0]  # this on equator
iself = index_at_lat(ds_self, lat1)
a1 = local_angle(
    lons_track_self, lats_track_self, iself, ioff=3, degrees=False
)
```

```python
a1_d = math.degrees(a1)
lons_track_other = ds_other.lon.values
lats_track_other = ds_other.lat.values
iothr = index_at_lat(ds_other, lat1)
a2 = local_angle(
    lons_track_other, lats_track_other, iothr, ioff=3, degrees=False
)
a2_d = math.degrees(a2)
lon_equat_other = lons_track_other[0]
lat_equat_other = lats_track_other[0]
lon_coast_other = lons_track_other[-1]
lat_coast_other = lats_track_other[-1]
gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
gc_other_at = gc_other.sel(
    x=x_from_coast_other1, method="nearest", drop=True
)
gc_other_at = gc_other_at.interp(time=gc_self_at.time)
print(f"heading angle {a1_d:.2f} {a2_d:.2f}")
gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
al, ac = along_across_shore_components(gu, gv, coast_heading_deg)
fig = plt.figure(figsize=(12, 10))
ax1 = fig.add_subplot(3, 1, 3)
ax0 = fig.add_subplot(3, 1, 2)
gu_cut = gu.resample(time="1ME").mean()
gv_cut = gv.resample(time="1ME").mean()
gc_self_at_cut = gc_self_at.resample(time="1ME").mean()
gc_other_at_cut = gc_other_at.resample(time="1ME").mean()
# gu_cut.plot(ax=ax0, label="track zonal", color="r", linewidth=2)
al.plot(ax=ax0, label="alongshore", color="r", linewidth=2)
# gc_self_at_cut.plot(ax=ax0, label=f"gc at {track_number_self}", color="r", linewidth=2)
ax0.legend()
ax0.set_ylabel("along current m/s")
# gv_cut.plot(ax=ax1, label="track meridional", color="r", linewidth=2)
ac.plot(ax=ax1, label="acrossshore", color="r", linewidth=2)
# gc_other_at_cut.plot(ax=ax1, label=f"gc at {track_number_other}", color="r", linewidth=2)
info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
plt.text(
    0.15,
    0.90,
    info,
    transform=ax0.transAxes,
    fontsize=14,
    fontweight="bold",
)
ax1.legend()
ax1.set_ylabel("across current m/s")
ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=5)
ax2.grid()
ax2.plot(
    lon1,
    lat1,
    c="r",
    marker="o",
    markersize=6,
    markerfacecolor="red",
    markeredgecolor="black",
    markeredgewidth=2,
)
ax2.scatter(
    lons_track_self,
    lats_track_self,
    marker=".",
    color="c",
    s=4,
)
ax2.scatter(
    lons_track_other,
    lats_track_other,
    marker=".",
    color="c",
    s=4,
)
lonm, latm = get_point_at_distance(
    lon_equat_self, lat_equat_self, lon_coast_self, lat_coast_self, d
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=a1_d,
        color="k",
    )
lonm, latm = get_point_at_distance(
    lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=a2_d,
        color="k",
    )
plt.savefig(
    f"along_shore_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
)
ln_sf(
    f"along_shore_timeseries_{sat1}_{track_number_self}_{track_number_other}.png", png_file)
# plt.show()
plt.close("all")

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
from tools_shapely import *
import sys
from pathlib import Path
import sys

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")


def along_across_shore_components(gu, gv, coast_heading_deg):
    """
    Resolve vector (gu, gv) into along-shore and across-shore components.
    Sign convention:
    - coast_heading_deg is measured CCW from +x (East).
    - along_shore is positive in the heading direction.
    - across_shore is positive 90 deg CCW from heading (left-hand side of coast).
      If you need right-hand positive, negate across_shore.
    """
    theta = np.deg2rad(coast_heading_deg)
    c = np.cos(theta)
    s = np.sin(theta)
    # Elementwise projection keeps scalar or array/xarray shape.
    along_shore = gu * c + gv * s
    across_shore = -gu * s + gv * c  # +90 deg CCW
    return along_shore, across_shore


cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

sat_val = "TP+J1+J2+J3+S6A"
track_self_val = 155
track_other_val = 116

df = pd.read_csv(f"tracks_intersections_{sat_val}_1.csv")
df_out = df.copy()

rows = df_out.loc[
    (df_out["sat"] == sat_val)
    & (df_out["track_self"] == track_self_val)
    & (df_out["track_other"] == track_other_val)
]
# Optional input: Gaussian smoothing sigma
sigma = 3
# Optional input: clipping bbox tuple
# Default used by the function is (70, 5, 95, 30)
region_bbox_tuple = (70, 5, 95, 30)

frac1 = "0.05"
i = 0
sat1 = rows["sat"].values[i]
# track_number_self = rows["track_self"].apply(lambda x: str(x)).values[i]
# track_number_other = rows["track_other"].apply(lambda x: str(x)).values[i]
track_number_self = str(rows["track_self"].values[i])
track_number_other = str(rows["track_other"].values[i])
lons_inter1 = rows["lons_inter"].values[i]
lats_inter1 = rows["lats_inter"].values[i]
lon1 = lons_inter1
lat1 = lats_inter1
query_point = (lon1, lat1)
result = coastline_gaussian_tangent(
    query_point=query_point,
    sigma=sigma,
    region_bbox_tuple=region_bbox_tuple,
)
print("heading_angle_deg:", result["heading_angle_deg"])
print("tangent_vector:", result["tangent_vector"])
print("nearest_point:", result["nearest_point"])
print("distance:", result["distance"])
coast_heading_deg = result["heading_angle_deg"]
x_from_coast_self1 = rows["x_from_coast_self"].values[i]
x_from_coast_other1 = rows["x_from_coast_other"].values[i]
angle_acute = rows["angle_acute"].values[i]
angle_obtuse = rows["angle_obtuse"].values[i]
print(
    sat1,
    track_number_self,
    track_number_other,
    lons_inter1,
    lats_inter1,
    x_from_coast_self1,
    x_from_coast_other1,
    angle_acute,
    angle_obtuse,
)
# sys.exit()
f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat1}_{track_number_self}_loess_{frac1}.nc"
f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat1}_{track_number_other}_loess_{frac1}.nc"
ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
ds_self_smooth = xr.open_dataset(f_self_smooth)
sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
sla_self_smooth = ds_self_smooth.sla_smooth
ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
ds_other_smooth = xr.open_dataset(f_other_smooth)
sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
sla_other_smooth = ds_other_smooth.sla_smooth
lons_track_self = ds_self.lon.values
lats_track_self = ds_self.lat.values
lon_coast_self = lons_track_self[-1]  # this on coast
lat_coast_self = lats_track_self[-1]  # this on coast
lon_equat_self = lons_track_self[0]  # this on equator
lat_equat_self = lats_track_self[0]  # this on equator
iself = index_at_lat(ds_self, lat1)
a1 = local_angle(
    lons_track_self, lats_track_self, iself, ioff=3, degrees=False
)
```

```python
a1_d = math.degrees(a1)
lons_track_other = ds_other.lon.values
lats_track_other = ds_other.lat.values
iothr = index_at_lat(ds_other, lat1)
a2 = local_angle(
    lons_track_other, lats_track_other, iothr, ioff=3, degrees=False
)
a2_d = math.degrees(a2)
lon_equat_other = lons_track_other[0]
lat_equat_other = lats_track_other[0]
lon_coast_other = lons_track_other[-1]
lat_coast_other = lats_track_other[-1]
gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
gc_other_at = gc_other.sel(
    x=x_from_coast_other1, method="nearest", drop=True
)
gc_other_at = gc_other_at.interp(time=gc_self_at.time)
print(f"heading angle {a1_d:.2f} {a2_d:.2f}")
gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
al, ac = along_across_shore_components(gu, gv, coast_heading_deg)
fig = plt.figure(figsize=(12, 10))
ax1 = fig.add_subplot(3, 1, 3)
ax0 = fig.add_subplot(3, 1, 2)
gu_cut = gu.resample(time="1ME").mean()
gv_cut = gv.resample(time="1ME").mean()
gc_self_at_cut = gc_self_at.resample(time="1ME").mean()
gc_other_at_cut = gc_other_at.resample(time="1ME").mean()
# gu_cut.plot(ax=ax0, label="track zonal", color="r", linewidth=2)
al.plot(ax=ax0, label="alongshore", color="r", linewidth=2)
# gc_self_at_cut.plot(ax=ax0, label=f"gc at {track_number_self}", color="r", linewidth=2)
ax0.legend()
ax0.set_ylabel("along current m/s")
# gv_cut.plot(ax=ax1, label="track meridional", color="r", linewidth=2)
ac.plot(ax=ax1, label="acrossshore", color="r", linewidth=2)
# gc_other_at_cut.plot(ax=ax1, label=f"gc at {track_number_other}", color="r", linewidth=2)
info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
plt.text(
    0.15,
    0.90,
    info,
    transform=ax0.transAxes,
    fontsize=14,
    fontweight="bold",
)
ax1.legend()
ax1.set_ylabel("across current m/s")
ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=5)
ax2.grid()
ax2.plot(
    lon1,
    lat1,
    c="r",
    marker="o",
    markersize=6,
    markerfacecolor="red",
    markeredgecolor="black",
    markeredgewidth=2,
)
ax2.scatter(
    lons_track_self,
    lats_track_self,
    marker=".",
    color="c",
    s=4,
)
ax2.scatter(
    lons_track_other,
    lats_track_other,
    marker=".",
    color="c",
    s=4,
)
lonm, latm = get_point_at_distance(
    lon_equat_self, lat_equat_self, lon_coast_self, lat_coast_self, d
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=a1_d,
        color="k",
    )
lonm, latm = get_point_at_distance(
    lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=a2_d,
        color="k",
    )
plt.savefig(
    f"along_shore_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
)
ln_sf(
    f"along_shore_timeseries_{sat1}_{track_number_self}_{track_number_other}.png", png_file)
# plt.show()
plt.close("all")

```

draft_figs02/archive/link_figs_png.py

```python
import shutil
from pathlib import Path

links_dict = {
    "p.compare.std_mean.l2pVSxtrack_map.py":"fig01.py",
    "p.omni_uvVStrack_guv_c.ek.nearest.selected.py": "fig02.py",
    "p.scatter_omniVSrama_uVSgu_ww_ek.py": "fig03.py",
    "p.scatter_drifter_ve_vs_gu_ww_ek.py": "fig04.py",
    "p.radar_uvVStrack_guv.at.intersections_single.py": "fig05.py",
    "p.gc.clim.cross_structure_x_arrows_map_ERS.py": "fig06.py",
    "p.gc.clim.lat_wise.along_xwidth.py": "fig07.py",
    "p.along_shore_at_155_116.intersection.S6A.py": "fig08.py",
}

archive_dir = Path("archive")
for full_path, fig_path in links_dict.items():
    src = Path(full_path)
    dst = Path(fig_path)

    if dst.exists() or dst.is_symlink():
        if dst.is_symlink():
            dst.unlink()
        elif dst.is_file():
            archive_path = archive_dir / dst.name
            if archive_path.exists():
                archive_path.unlink()
            shutil.move(str(dst), str(archive_path))
        else:
            continue
    print(f"{src} -> {dst}")
    dst.hardlink_to(src)

```

```python
import os
import sys

from docx import Document
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.shared import Inches
from docx.shared import Pt

# images_list = [
#     "fig01_f.combined_map_std_valid.S3A.png",
#     "fig02_p.omni_uvVStrack_guv_c.ek.nearest.selected.png",
#     "fig03_p.scatter_omniVSrama_uVSgu_ww_ek.png",
#     "fig04_p.scatter_drifter_ve_vs_gu_ww_ek.png",
#     "fig05_p.radar_uvVStrack_guv.at.intersections_single.png",
#     "fig06_p.gc.clim.cross_structure_x_arrows_map_ERS.png",
#     "fig07_p.gc.clim.lat_wise.along_xwidth.png",
#     "fig08_p.along_shore_at_155_116.intersection.S6A.png",
#     "fig09_p.speed_at_coastal_points.eicc.trends.all_in_one.ERS.png",
# ]

base_names = {
    "p.aviso_only_uvVStrack_guv.at.intersections_single.py": 0,
    "p.metrics.track_guvVSoscar_ww_ekman.py": 0,
    "p.vskp.tseries_new_v2.2.py": 0,
    "f.combined_map_std_valid.S3A.py": 1,
    "p.compare.std_mean.l2pVSxtrack_map.py": 0,
    "p.omni_uvVStrack_guv_c.ek.nearest.selected.py": 1,
    "p.scatter_omniVSrama_uVSgu_ww_ek.py": 1,
    "p.scatter_drifter_ve_vs_gu_ww_ek.py": 1,
    "p.radar_uvVStrack_guv.at.intersections_single.py": 1,
    "p.gc.clim.cross_structure_x_arrows_map_ERS.py": 0,
    "p.gc.clim.lat_wise.along_xwidth.py": 0,
    "p.along_shore_at_155_116.intersection.S6A.py": 1,
    "p.speed_at_coastal_points.eicc.trends.all_in_one.ERS.py": 1,
    "satellite_mission_time_spans_new.py": 2
}

for image in images_list:
    if not os.path.exists(image):
        raise FileNotFoundError(f"Image {image} not found.")
print("all good")

captions_list = [
    "captions/f.combined_map_std_valid.S3A.txt",
    "captions/p.omni_uvVStrack_guv_c.ek.nearest.selected.txt",
    "captions/p.scatter_omniVSrama_uVSgu_ww_ek.txt",
    "captions/p.scatter_drifter_ve_vs_gu_ww_ek.txt",
    "captions/p.radar_uvVStrack_guv.at.intersections_single.txt",
    "captions/p.gc.clim.cross_structure_x_arrows_map_ERS.txt",
    "captions/p.gc.clim.lat_wise.along_xwidth.txt",
    "captions/p.along_shore_at_155_116.intersection.S6A.txt",
    "captions/p.speed_at_coastal_points.eicc.trends.all_in_one.ERS.txt",
]

# Caption styling options (edit these as needed).
CAPTION_FONT_NAME = "Times New Roman"
CAPTION_FONT_SIZE_PT = 12
CAPTION_ALIGNMENT = "center"  # one of: left, center, right, justify

ALIGNMENT_MAP = {
    "left": WD_ALIGN_PARAGRAPH.LEFT,
    "center": WD_ALIGN_PARAGRAPH.CENTER,
    "right": WD_ALIGN_PARAGRAPH.RIGHT,
    "justify": WD_ALIGN_PARAGRAPH.JUSTIFY,
}

doc_name = "figures_v2.1.docx"
if os.path.exists(doc_name):
    os.remove(doc_name)

doc = Document()
inserted_count = 0

for i, img in enumerate(images_list):
    if not os.path.exists(img):
        print(f"Skipping missing image: {img}")
        continue

    doc.add_picture(img, width=Inches(6))
    inserted_count += 1

    caption_text = ""
    if i < len(captions_list):
        caption_path = captions_list[i]
        if os.path.exists(caption_path):
            with open(caption_path, "r", encoding="utf-8") as f:
                caption_text = f.read().strip()

    prefix = f"Fig{inserted_count}"
    full_caption = f"{prefix}: {caption_text}" if caption_text else prefix

    paragraph = doc.add_paragraph()
    paragraph.alignment = ALIGNMENT_MAP.get(
        CAPTION_ALIGNMENT.lower(), WD_ALIGN_PARAGRAPH.CENTER
    )
    run = paragraph.add_run(full_caption)
    run.font.name = CAPTION_FONT_NAME
    run.font.size = Pt(CAPTION_FONT_SIZE_PT)

    if i < len(images_list) - 1:
        doc.add_page_break()

doc.save(doc_name)
print(f"Inserted {inserted_count} figure(s) into {doc_name}")

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
from tools_shapely import *
import sys
from pathlib import Path
import sys

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")


def along_across_shore_components(gu, gv, coast_heading_deg):
    """
    Resolve vector (gu, gv) into along-shore and across-shore components.
    Sign convention:
    - coast_heading_deg is measured CCW from +x (East).
    - along_shore is positive in the heading direction.
    - across_shore is positive 90 deg CCW from heading (left-hand side of coast).
      If you need right-hand positive, negate across_shore.
    """
    theta = np.deg2rad(coast_heading_deg)
    c = np.cos(theta)
    s = np.sin(theta)
    # Elementwise projection keeps scalar or array/xarray shape.
    along_shore = gu * c + gv * s
    across_shore = -gu * s + gv * c  # +90 deg CCW
    return along_shore, across_shore


cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

sat_val = "TP+J1+J2+J3+S6A"
track_self_val = 155
track_other_val = 116

df = pd.read_csv(f"tracks_intersections_{sat_val}_1.csv")
df_out = df.copy()

rows = df_out.loc[
    (df_out["sat"] == sat_val)
    & (df_out["track_self"] == track_self_val)
    & (df_out["track_other"] == track_other_val)
]
# Optional input: Gaussian smoothing sigma
sigma = 3
# Optional input: clipping bbox tuple
# Default used by the function is (70, 5, 95, 30)
region_bbox_tuple = (70, 5, 95, 30)

frac1 = "0.05"
i = 0
sat1 = rows["sat"].values[i]
# track_number_self = rows["track_self"].apply(lambda x: str(x)).values[i]
# track_number_other = rows["track_other"].apply(lambda x: str(x)).values[i]
track_number_self = str(rows["track_self"].values[i])
track_number_other = str(rows["track_other"].values[i])
lons_inter1 = rows["lons_inter"].values[i]
lats_inter1 = rows["lats_inter"].values[i]
lon1 = lons_inter1
lat1 = lats_inter1
query_point = (lon1, lat1)
result = coastline_gaussian_tangent(
    query_point=query_point,
    sigma=sigma,
    region_bbox_tuple=region_bbox_tuple,
)
print("heading_angle_deg:", result["heading_angle_deg"])
print("tangent_vector:", result["tangent_vector"])
print("nearest_point:", result["nearest_point"])
print("distance:", result["distance"])
coast_heading_deg = result["heading_angle_deg"]
x_from_coast_self1 = rows["x_from_coast_self"].values[i]
x_from_coast_other1 = rows["x_from_coast_other"].values[i]
angle_acute = rows["angle_acute"].values[i]
angle_obtuse = rows["angle_obtuse"].values[i]
print(
    sat1,
    track_number_self,
    track_number_other,
    lons_inter1,
    lats_inter1,
    x_from_coast_self1,
    x_from_coast_other1,
    angle_acute,
    angle_obtuse,
)
# sys.exit()
f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat1}_{track_number_self}_loess_{frac1}.nc"
f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat1}_{track_number_other}_loess_{frac1}.nc"
ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
ds_self_smooth = xr.open_dataset(f_self_smooth)
sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
sla_self_smooth = ds_self_smooth.sla_smooth
ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
ds_other_smooth = xr.open_dataset(f_other_smooth)
sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
sla_other_smooth = ds_other_smooth.sla_smooth
lons_track_self = ds_self.lon.values
lats_track_self = ds_self.lat.values
lon_coast_self = lons_track_self[-1]  # this on coast
lat_coast_self = lats_track_self[-1]  # this on coast
lon_equat_self = lons_track_self[0]  # this on equator
lat_equat_self = lats_track_self[0]  # this on equator
iself = index_at_lat(ds_self, lat1)
a1 = local_angle(
    lons_track_self, lats_track_self, iself, ioff=3, degrees=False
)
```

```python
a1_d = math.degrees(a1)
lons_track_other = ds_other.lon.values
lats_track_other = ds_other.lat.values
iothr = index_at_lat(ds_other, lat1)
a2 = local_angle(
    lons_track_other, lats_track_other, iothr, ioff=3, degrees=False
)
a2_d = math.degrees(a2)
lon_equat_other = lons_track_other[0]
lat_equat_other = lats_track_other[0]
lon_coast_other = lons_track_other[-1]
lat_coast_other = lats_track_other[-1]
gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
gc_other_at = gc_other.sel(
    x=x_from_coast_other1, method="nearest", drop=True
)
gc_other_at = gc_other_at.interp(time=gc_self_at.time)
print(f"heading angle {a1_d:.2f} {a2_d:.2f}")
gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
al, ac = along_across_shore_components(gu, gv, coast_heading_deg)
fig = plt.figure(figsize=(12, 10))
ax1 = fig.add_subplot(3, 1, 3)
ax0 = fig.add_subplot(3, 1, 2)
gu_cut = gu.resample(time="1ME").mean()
gv_cut = gv.resample(time="1ME").mean()
gc_self_at_cut = gc_self_at.resample(time="1ME").mean()
gc_other_at_cut = gc_other_at.resample(time="1ME").mean()
# gu_cut.plot(ax=ax0, label="track zonal", color="r", linewidth=2)
al.plot(ax=ax0, label="alongshore", color="r", linewidth=2)
# gc_self_at_cut.plot(ax=ax0, label=f"gc at {track_number_self}", color="r", linewidth=2)
ax0.legend()
ax0.set_ylabel("along current m/s")
# gv_cut.plot(ax=ax1, label="track meridional", color="r", linewidth=2)
ac.plot(ax=ax1, label="acrossshore", color="r", linewidth=2)
# gc_other_at_cut.plot(ax=ax1, label=f"gc at {track_number_other}", color="r", linewidth=2)
info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
plt.text(
    0.15,
    0.90,
    info,
    transform=ax0.transAxes,
    fontsize=14,
    fontweight="bold",
)
ax1.legend()
ax1.set_ylabel("across current m/s")
ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=5)
ax2.grid()
ax2.plot(
    lon1,
    lat1,
    c="r",
    marker="o",
    markersize=6,
    markerfacecolor="red",
    markeredgecolor="black",
    markeredgewidth=2,
)
ax2.scatter(
    lons_track_self,
    lats_track_self,
    marker=".",
    color="c",
    s=4,
)
ax2.scatter(
    lons_track_other,
    lats_track_other,
    marker=".",
    color="c",
    s=4,
)
lonm, latm = get_point_at_distance(
    lon_equat_self, lat_equat_self, lon_coast_self, lat_coast_self, d
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=a1_d,
        color="k",
    )
lonm, latm = get_point_at_distance(
    lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=a2_d,
        color="k",
    )
plt.savefig(
    f"along_shore_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
)
ln_sf(
    f"along_shore_timeseries_{sat1}_{track_number_self}_{track_number_other}.png", png_file)
# plt.show()
plt.close("all")

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs
import sys
# from rich.traceback import install
# install(show_locals=True, suppress=["site-packages"])

cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
aviso_msla_uv_dir = "/home/srinivasu/sealevel/data/msla_uv/"
aviso_msla_uv_dir = "/home/srinivasu/allData/aviso/msla_uv/"
ds_aviso_u = xr.open_dataset(f"{aviso_msla_uv_dir}/allyear_dt_global_allsat_msla_u.nc")
ds_aviso_v = xr.open_dataset(f"{aviso_msla_uv_dir}/allyear_dt_global_allsat_msla_v.nc")
print(ds_aviso_u)
aviso_u = ds_aviso_u.u
aviso_v = ds_aviso_v.v
# sys.exit()

sat_val = "TP+J1+J2+J3+S6A"
track_self_val = 29
track_other_val = 92
track_self_val = 53
track_other_val = 14
df = pd.read_csv(f"tracks_intersections_{sat_val}_1.csv")
df_out = df.copy()

rows = df_out.loc[
    (df_out["sat"] == sat_val) &
    (df_out["track_self"] == track_self_val) &
    (df_out["track_other"] == track_other_val)
]
# sys.exit()
corrs_u = []
biass_u = []
rmses_u = []
ngp_u = []

corrs_v = []
biass_v = []
rmses_v = []
ngp_v = []

ln = len(rows)
count = 0
frac1 = "0.05"
for i in range(ln):
    sat1 = rows["sat"].values[i]
    track_number_self = rows["track_self"].apply(lambda x: str(x)).values[i]
    track_number_other = rows["track_other"].apply(lambda x: str(x)).values[i]
    lons_inter1 = rows["lons_inter"].values[i]
    lats_inter1 = rows["lats_inter"].values[i]
    lon1 = lons_inter1
    lat1 = lats_inter1
    x_from_coast_self1 = rows["x_from_coast_self"].values[i]
    x_from_coast_other1 = rows["x_from_coast_other"].values[i]
    angle_acute = rows["angle_acute"].values[i]
    angle_obtuse = rows["angle_obtuse"].values[i]
    if abs(lat1) < 2:
        corrs_u.append(np.nan)
        rmses_u.append(np.nan)
        biass_u.append(np.nan)
        ngp_u.append(np.nan)
        corrs_v.append(np.nan)
        rmses_v.append(np.nan)
        biass_v.append(np.nan)
        ngp_v.append(np.nan)
        continue
    print(
        sat1,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat1}_{track_number_self}_loess_{frac1}.nc"
    f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat1}_{track_number_other}_loess_{frac1}.nc"
    ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
```

```
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)
    print(f"azimuths {a1:.2f} {a2:.2f}")
    gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
    fig = plt.figure(figsize=(12, 10))
    ax1 = fig.add_subplot(3, 1, 3)
    ax0 = fig.add_subplot(3, 1, 2)

    gu_cut = gu.resample(time="1ME").mean()
    gu_cut_ngp = 100 * gu_cut.count(dim="time")/len(gu_cut.time)
    gv_cut = gv.resample(time="1ME").mean()
    gv_cut_ngp = 100 * gv_cut.count(dim="time")/len(gv_cut.time)
    tfreq = sats_tfreq[sat1]

    aviso_u1 = aviso_u.sel(longitude=lon1, latitude=lat1, method="nearest")
    aviso_u2 = aviso_u1.interp(time=gu_cut.time)
    aviso_max_u = aviso_u2.max().item()
    gu_cut = gu_cut.where(gu_cut < aviso_max_u, np.nan)
    gu_cut = gu_cut.where(gu_cut > -aviso_max_u, np.nan)

    aviso_v1 = aviso_v.sel(longitude=lon1, latitude=lat1, method="nearest")
    aviso_v2 = aviso_v1.interp(time=gv_cut.time)

    aviso_max_v = aviso_v2.max().item()
    gv_cut = gv_cut.where(gv_cut < aviso_max_v, np.nan)
    gv_cut = gv_cut.where(gv_cut > -aviso_max_v, np.nan)

    aviso_u2.plot(ax=ax0, color="b", linewidth=2, label="aviso u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="r", linewidth=2)
    # ax0.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax0.legend()
    ax0.set_ylabel("aviso u m/s")

    aviso_v2.plot(ax=ax1, color="b", linewidth=2, label="aviso v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="r", linewidth=2)

    # percentage of good points of gu_cut
    corr_u = xs.pearson_r(gu_cut, aviso_u2, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, aviso_u2, dim="time", skipna=True)
    bias_var_u = aviso_u2 - gu_cut
    bias_u = bias_var_u.mean(dim="time")

    corr_v = xs.pearson_r(gv_cut, aviso_v2, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, aviso_v2, dim="time", skipna=True)
    bias_var_v = aviso_v2 - gv_cut
    bias_v = bias_var_v.mean(dim="time")

    corrs_u.append(corr_u.item())
    rmses_u.append(rmse_u.item())
    biass_u.append(bias_u.item())
    ngp_u.append(gu_cut_ngp.item())

    corrs_v.append(corr_v.item())
    rmses_v.append(rmse_v.item())
    biass_v.append(bias_v.item())
    ngp_v.append(gv_cut_ngp.item())

    print(f"Correlation between gu and u_aviso_at: {corr_u.item():.2f} {gu_cut_ngp.item():.2f}")
    print(f"Correlation between gv and v_aviso_at: {corr_v.item():.2f} {gv_cut_ngp.item():.2f}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.15,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("aviso v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor="red",
        markeredgecolor="black",
        markeredgewidth=2,
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
```

```
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    ln_sf(
        f"guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
        "fig03.png",
    )
    # plt.show()
    plt.close("all")
    # ds_aviso_u.close()
dse.close()

```

```python
import glob
import math

import numpy as np
import xarray as xr
from matplotlib import colormaps
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import *
import pickle
import os
import sys
from pathlib import Path

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")
print(png_file)

sat = "S3A"
cmap = "twilight_shifted"
EAST_COAST_REG = (78.0, 95, 0, 25.4)

ds_xtr = xr.open_dataset("../computed/std_mean_S3A_xtrack.nc")
ds_l2p = xr.open_dataset("/home/srinivasu/allData/aviso_l2p/s3a/std_mean_S3A_l2p.nc")

std_xtr = ds_xtr["std"]
std_l2p = ds_l2p["std"]

with open("common_passes_l2pVSxtrack.pkl", "rb") as f:
    common = pickle.load(f)

dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")
fig = plt.figure(figsize=(10, 6))
ax1 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax2 = fig.add_subplot(2, 1, 2)

dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:])).plot(
    ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap
)
decorate_axis(ax1, "", *NIO, step=8)
ax1.text(
    0.45,
    0.80,
    f"MISSION: {sat}",
    transform=ax1.transAxes,
    fontsize=10,
    fontweight="bold",
    color="b",
)

colors = colormaps["tab20"](np.linspace(0, 1, len(common)))
i = 0
for pass1 in common:
    pass2 = pass1[1:]
    f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{pass2}.nc"
    with xr.open_dataset(f, decode_times=False) as ds:
        lon_eq = ds.lon.values[0]
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        lon_equat = lons_track[0]
        lon_coast = lons_track[-1]
        lat_equat = lats_track[0]
        lat_coast = lats_track[-1]
        a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
        a1_d = math.degrees(a1)
        track_number = ds.Pass
        lonm = lons_track.mean()
        latm = lats_track.mean()
        color = colors[i]
        print(f"{sat} {track_number} --------------------")
        ax1.scatter(lons_track, lats_track, s=2, color=color)
        if is_within_region(lonm, latm, *NIO):
            ax1.text(
                lonm,
                latm,
                s=str(track_number),
                fontsize=7,
                rotation=a1_d,
                color='k',
                ha="center",
                va="center",
                transform=ccrs.PlateCarree(),
            )
        i += 1

# ax2.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))

ax2.plot(std_xtr, label="xtrack std", color="red")
ax2.plot(std_l2p, label="l2p std", color="blue")
ax2.grid(alpha=0.3, linestyle="--", linewidth=0.5)
ax2.set_xlabel("distance along track (km)")
ax2.set_ylabel("std")
ax2.set_title("std_mean_xtrack vs std_mean_l2p")
ax2.axvline(x=0, color="k", linewidth=0.5, linestyle="--")
ax2.text(
    0.02,
    0.5,
    "coast",
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
    color="k",
    rotation=90,
)

plt.legend()
plt.tight_layout()
plt.savefig(f"std_mean_xtrackVSstd_mean_l2p_{sat}.png")
ln_sf(f"std_mean_xtrackVSstd_mean_l2p_{sat}.png", png_file)
# plt.show()
plt.close("all")

dse.close()

```

```python
import xarray as xr
from tools_xtrackm import *
from namesm import sats_new
import glob
import os
from matplotlib.collections import LineCollection
from pathlib import Path
import sys

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

sat = "ERS1+ERS2+ENV+SRL"

REG = (80.0, 92.0, 11.0, 21.0)
WBOB = (78.0, 87.0, 8.0, 21.0)
WBOB2 = (78.0, 97.0, 0.0, 25.0)
proj = ccrs.PlateCarree()

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 10),
                       subplot_kw={'projection': proj})
# adjust subplots to have minimum space between them
month_num = 2
month_name = months[month_num+1]
info =f"{sat} {months[month_num+1]}"
fig.subplots_adjust(wspace=0.1)
ax1.set_extent([*WBOB2], crs=proj)
ax1.coastlines()
# ax1.legend()
gl = ax1.gridlines(draw_labels=True, linewidth=0.5, color='gray', alpha=0.7, linestyle='--')
gl.top_labels = gl.right_labels = False  # Hide top and right labels
plt.text(0.3, 0.95, info,
         horizontalalignment='center',
         verticalalignment='center',
         fontsize=15,
         fontweight="bold",
         transform=ax1.transAxes)
for f_sla in sorted(
    glob.glob(
        f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
    )
):
    basename = os.path.basename(f_sla)
    ds_sla = xr.open_dataset(f_sla, decode_times=False)
    trackn = ds_sla.Pass
    # f = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{trackn}_loess_0.2.nc"
    f = f"../computed/sla_loess_0.05a/track_sla_along_{sat}_{trackn}_loess_0.05.nc"
    lons_track = ds_sla.lon.values
    lats_track = ds_sla.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    lon_equat = lons_track[0]
    lat_equat = lats_track[0]
    # compute tangent vectors or slope of track at each point
    dlon = np.gradient(lons_track_rev)
    dlat = np.gradient(lats_track_rev)
    tangent_mag = np.hypot(dlon, dlat)
    tangent_u = dlon / tangent_mag
    tangent_v = dlat / tangent_mag
    # Convert latitude to radians
    phi = np.deg2rad(lats_track_rev)
    # Length of one degree of latitude and longitude in meters
    meters_per_degree_lat = 111_320  # Approximate value
    meters_per_degree_lon = 111_320 * np.cos(phi)
    # Handle division by zero at the poles
    meters_per_degree_lon = np.where(meters_per_degree_lon == 0,
                                     np.nan, meters_per_degree_lon)
    # Compute normal vectors
    normal_u = -tangent_v
    normal_v = tangent_u
    theta = np.arctan2(lat_coast-lat_equat, lon_coast-lon_equat)
    print(theta, "--------------------------------------------------")
    # if is_within_region(lon_coast, lat_coast, *REG):
    if True:
        ds_smooth = xr.open_dataset(f)
        sla_smooth = ds_smooth.sla_smooth
        gc = compute_geostrophy_gc(ds_sla, sla_smooth)
        gc = gc.where(gc > -2.5)
        gc = gc.where(gc < 2.5)
        gc_clim = gc.groupby("time.month").mean("time")
        gc_clim_m = gc_clim.sel(month=month_num)
        # Convert gc from m/s to degrees (assuming cross-track direction)
        gc_deg = gc_clim_m.values / meters_per_degree_lat
        # Calculate displacements
        delta_lon = normal_u * gc_deg
        delta_lat = normal_v * gc_deg
        # Apply scaling factor for visualization
        scaling_factor = 1e5  # Adjust as needed
        delta_lon *= scaling_factor
        delta_lat *= scaling_factor
        # Compute endpoints
        lon_start = lons_track_rev
        lat_start = lats_track_rev
        lon_end = lon_start + delta_lon
        lat_end = lat_start + delta_lat
        # Create line segments
        segments = [[[lon_start[i], lat_start[i]], [lon_end[i], lat_end[i]]]
                    for i in range(len(lon_start))]
        # Create a LineCollection
        lc = LineCollection(segments, colors='red', linewidths=1,
                            transform=proj)
        ax1.plot(lons_track_rev, lats_track_rev, 'k-',
                 transform=proj, label='Ground Track')
        ax1.add_collection(lc)

month_num = 10
month_name = months[month_num+1]
info =f"{sat} {months[month_num+1]}"
ax2.set_extent([*WBOB2], crs=proj)
ax2.coastlines()
# ax1.legend()
gl = ax2.gridlines(draw_labels=True, linewidth=0.5, color='gray', alpha=0.7, linestyle='--')
gl.top_labels = gl.right_labels = False  # Hide top and right labels
plt.text(0.3, 0.95, info,
         horizontalalignment='center',
```

```
         verticalalignment='center',
         fontsize=15,
         fontweight="bold",
         transform=ax2.transAxes)
for f_sla in sorted(
    glob.glob(
        f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
    )
):
    basename = os.path.basename(f_sla)
    ds_sla = xr.open_dataset(f_sla, decode_times=False)
    trackn = ds_sla.Pass
    # f = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{trackn}_loess_0.2.nc"
    f = f"../computed/sla_loess_0.05a/track_sla_along_{sat}_{trackn}_loess_0.05.nc"
    lons_track = ds_sla.lon.values
    lats_track = ds_sla.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    lon_equat = lons_track[0]
    lat_equat = lats_track[0]
    # compute tangent vectors or slope of track at each point
    dlon = np.gradient(lons_track_rev)
    dlat = np.gradient(lats_track_rev)
    tangent_mag = np.hypot(dlon, dlat)
    tangent_u = dlon / tangent_mag
    tangent_v = dlat / tangent_mag
    # Convert latitude to radians
    phi = np.deg2rad(lats_track_rev)
    # Length of one degree of latitude and longitude in meters
    meters_per_degree_lat = 111_320  # Approximate value
    meters_per_degree_lon = 111_320 * np.cos(phi)
    # Handle division by zero at the poles
    meters_per_degree_lon = np.where(meters_per_degree_lon == 0,
                                     np.nan, meters_per_degree_lon)
    # Compute normal vectors
    normal_u = -tangent_v
    normal_v = tangent_u
    theta = np.arctan2(lat_coast-lat_equat, lon_coast-lon_equat)
    print(theta, "----------------------------------------------------")
    # if is_within_region(lon_coast, lat_coast, *REG):
    if True:
        ds_smooth = xr.open_dataset(f)
        sla_smooth = ds_smooth.sla_smooth
        gc = compute_geostrophy_gc(ds_sla, sla_smooth)
        gc = gc.where(gc > -2.5)
        gc = gc.where(gc < 2.5)
        gc_clim = gc.groupby("time.month").mean("time")
        gc_clim_m = gc_clim.sel(month=month_num)
        # Convert gc from m/s to degrees (assuming cross-track direction)
        gc_deg = gc_clim_m.values / meters_per_degree_lat
        # Calculate displacements
        delta_lon = normal_u * gc_deg
        delta_lat = normal_v * gc_deg
        # Apply scaling factor for visualization
        scaling_factor = 1e5  # Adjust as needed
        delta_lon *= scaling_factor
        delta_lat *= scaling_factor
        # Compute endpoints
        lon_start = lons_track_rev
        lat_start = lats_track_rev
        lon_end = lon_start + delta_lon
        lat_end = lat_start + delta_lat
        # Create line segments
        segments = [[[lon_start[i], lat_start[i]], [lon_end[i], lat_end[i]]]
                    for i in range(len(lon_start))]
        # Create a LineCollection
        lc = LineCollection(segments, colors='red', linewidths=1,
                            transform=proj)
        ax2.plot(lons_track_rev, lats_track_rev, 'k-',
                 transform=proj, label='Ground Track')
        ax2.add_collection(lc)
plt.savefig(f"cross_structure_x_arrows_map_{sat}_1by2.png",
            bbox_inches='tight', dpi=300)
ln_sf(f"cross_structure_x_arrows_map_{sat}_1by2.png", png_file)
# plt.show()
plt.close("all")

```

```python
import xarray as xr
from tools_xtrackm import *
import glob
import os

import matplotlib.cm as cm
from pathlib import Path
import sys

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

# sats = ["S3A"]
REG = (80.0, 89.0, 11.0, 21.0)
REG = (75.0, 92.0, 10.0, 23.0)
sats = ["S3A"]
sats = ["ERS1+ERS2+ENV+SRL"]
xwidth = 290e3
# Load the YlOrRd colormap
ncolors = 21
cmap1 = cm.get_cmap('YlOrRd', ncolors)  # Split into 10 colors
colors1 = cmap1(np.linspace(0, 1, ncolors))
cmap2 = cm.get_cmap('BuGn', ncolors)  # Split into 10 colors
colors2 = cmap2(np.linspace(0, 1, ncolors))

norm2 = plt.Normalize(10.18, 22.00)
sm2 = plt.cm.ScalarMappable(cmap=cmap2, norm=norm2)
sm2.set_array([])

norm1 = plt.Normalize(10.18, 22.00)
sm1 = plt.cm.ScalarMappable(cmap=cmap1, norm=norm1)
sm1.set_array([])

# fig, ax = plt.subplots(1, 1, figsize=(9, 4))
frac1 = "0.2"
frac1 = "0.05"
for sat in sats[:]:
    fig, ax = plt.subplots(1, 1, figsize=(12, 6))
    # adjust subplots increase x spacing
    plt.subplots_adjust(left=0.1, bottom=0.1, right=0.8, top=0.8, wspace=0.3, hspace=0.3)
    i = 0
    lat_values = []
    for f_sla in sorted(
        glob.glob(
            f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
        )
    ):
        basename = os.path.basename(f_sla)
        ds_sla = xr.open_dataset(f_sla, decode_times=False)
        trackn = ds_sla.Pass
        f = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{trackn}_loess_{frac1}.nc"
        lons_track = ds_sla.lon.values
        lats_track = ds_sla.lat.values
        lons_track_rev = lons_track[::-1]
        lats_track_rev = lats_track[::-1]
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        print(f, lon_coast, lat_coast)
        if is_within_region(lon_coast, lat_coast, *REG):
            lat_values.append(lat_coast)
            ds_smooth = xr.open_dataset(f)
            # print(ds_smooth)
            sla_smooth = ds_smooth.sla_smooth
            print(f"stats of sla_smooth {sat} {trackn} --------------------")
            print(f"stats of gu {sat} {trackn} --------------------")
            gc = compute_geostrophy_gc(ds_sla, sla_smooth)
            gc_clim = gc.groupby("time.month").mean("time")
            gc_clim_xwidth = gc_clim.sel(x=slice(0, xwidth))
            gc_clim_xwidth_m = gc_clim_xwidth.isel(month=2, drop=True) # march
            gc_clim_xwidth_n = gc_clim_xwidth.isel(month=10, drop=True) # november
            gc_clim_xwidth_m.plot(ax=ax, label=f"{lat_coast}",
                                  color=colors1[i])
            x0, y0 = -15, gc_clim_xwidth_m.sel(x=0).values
            plt.text(x0, y0, f"{lat_coast:.2f}", fontsize=10, color=colors1[i])
            gc_clim_xwidth_n.plot(ax=ax, label=f"{lat_coast}",
                                  color=colors2[i])
            x0, y0 = -15, gc_clim_xwidth_n.sel(x=0).values
            plt.text(x0, y0, f"{lat_coast:.2f}", fontsize=10, color=colors2[i])
            # set xticks between 0 to xwidth
            ax.set_xlim(-15, xwidth)
            ax.set_ylim(-0.7, 0.7)
            xticks1 = ax.get_xticks()
            xticks_km = np.round((xticks1 / 1000), 0)
            ax.set_xticks(xticks1, xticks_km)
            ax.set_xlabel("distance along track (km)")
            ax.set_ylabel("geostrophic velocity (m/s)")
            # ax.set_ylim(-0.3, 0.3)
            ax.axhline(0, color="k", linestyle="--")
            i += 1
            # print_stats(gc)
    # plt.legend()
    print(f"number of tracks {i}")
    minlat = min(lat_values)
    maxlat = max(lat_values)
    print(f"min lat {min(lat_values)} max lat {max(lat_values)}")
    pos = ax.get_position()
    # Adjust and add the first colorbar closer to the plot
    cbar_ax1 = fig.add_axes([pos.x1 + 0.01, pos.y0, 0.02, pos.height])  # Adjust 0.01 to control horizontal gap
    cbar_ax1.text(0.5, 0.5, "MARCH", fontsize=10, horizontalalignment="center", verticalalignment="center", transform=cbar_ax1.transAxes, rotation = "vertical")
    cbar = plt.colorbar(sm1, cax=cbar_ax1)
    # cbar.set_label('latitude')
    cbar.set_ticks(list(range(int(minlat), int(maxlat)+1, 2)))
    # Adjust and add the second colorbar even closer
    cbar_ax2 = fig.add_axes([pos.x1 + 0.06, pos.y0, 0.02, pos.height])  # Adjust 0.03 as needed for spacing
    cbar_ax2.text(0.5, 0.5, "NOVEMBER", fontsize=10, horizontalalignment="center", verticalalignment="center", transform=cbar_ax2.transAxes, rotation = "vertical")
    cbar = plt.colorbar(sm2, cax=cbar_ax2)
    cbar.set_label('latitude')
    cbar.set_ticks(list(range(int(minlat), int(maxlat)+1, 2)))

    plt.savefig(
        f"p.gc.clim.lat_wise.along_xwidth_{sat}a_fig09.png",
        bbox_inches="tight",
        dpi=300,
    )
    ln_sf(
        f"p.gc.clim.lat_wise.along_xwidth_{sat}a_fig09.png",
        png_file,
```

```
    )
    # plt.show()
    plt.close("all")

```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from tools_xtrackm import ln_sf

only_df = pd.read_csv(
    "oscar_onlyVStrack_corrs_rmses_biass_TP+J1+J2+J3+S6A_u.csv")
ekmn_df = pd.read_csv(
    "oscar_ekmnVStrack_corrs_rmses_biass_TP+J1+J2+J3+S6A_u.csv")

# Create unique column combining zero-padded sat and track_self, track_other
only_df["unique_id"] = (only_df["sat"].apply(lambda s: s.zfill(20)) + "_" +
                        only_df["track_self"].astype(str) + "_" +
                        only_df["track_other"].astype(str))
only_df = only_df.set_index("unique_id")

ekmn_df["unique_id"] = (ekmn_df["sat"].apply(lambda s: s.zfill(20)) + "_" +
                        ekmn_df["track_self"].astype(str) + "_" +
                        ekmn_df["track_other"].astype(str))
ekmn_df = ekmn_df.set_index("unique_id")

common_ids = only_df.index.intersection(ekmn_df.index)
only_df = only_df.loc[common_ids]
ekmn_df = ekmn_df.loc[common_ids]

# Extract metrics for plotting
metrics = ["biass", "corrs", "rmses"]
titles = ["Bias", "Correlation", "Rmse"]

# Prepare figure with 2x2 grid
fig, axs = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle("comparison metrics with oscar u: track gu only VS track gu + ekman", fontsize=14)

summary_lines = []

for i, metric in enumerate(metrics):
    ax = axs[i // 2, i % 2]
    valid_mask = only_df[metric].notna() & ekmn_df[metric].notna()
    only_df1 = only_df[valid_mask]
    ekmn_df1 = ekmn_df[valid_mask]
    # Get data
    only_df_data = only_df1[metric].values
    ekmn_df_data = ekmn_df1[metric].values
    indices = np.arange(len(only_df_data))
    ax.plot(indices, only_df_data, label="track gu only")
    ax.plot(indices, ekmn_df_data, label="track gu+ekman")
    # draw zero line
    ax.axhline(0, color="gray", linestyle="--")
    ax.set_xlabel("Data Point Index")
    ax.set_ylabel(titles[i])
    ax.set_title(titles[i])
    ax.legend()
    # collect summary text for this metric
    if len(only_df_data) > 0:
        mean_o = np.nanmean(only_df_data)
        mean_e = np.nanmean(ekmn_df_data)
        diff = mean_e - mean_o
        cnt = len(only_df_data)
        summary_lines.append(f"\n{titles[i]}:")
        summary_lines.append(f" count = {cnt}")
        summary_lines.append(f" mean track u = {mean_o:.4f}")
        summary_lines.append(f" mean track u + ekman = {mean_e:.4f}")
        summary_lines.append(f" mean improvement = {diff:.4f}")
    else:
        summary_lines.append(f"\n{titles[i]}: no valid paired data")

ax_sum = axs[1, 1]
ax_sum.axis("off")
text = " ".join(summary_lines)

ax_sum.annotate(
    "\n".join(summary_lines),
    xy=(0, 1),
    xycoords="axes fraction",
    va="top",
    ha="left",
    fontsize=10,
    family="monospace",
    bbox=dict(boxstyle="round,pad=0.5", facecolor="white", alpha=0.7),
    annotation_clip=False,
)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(f"track_guvVSoscar_ww_ekman_u.png")
ln_sf("track_guvVSoscar_ww_ekman_u.png", "fig04.png")
# plt.show()

```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
import xskillscore as xs
from geopy import distance
from tools_xtrackm import *
from pathlib import Path
import sys

# codex resume 019cc947-b006-7d82-8941-d68e330c388a

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(
        f_track, engine="netcdf4", decode_times=False
    ) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


def build_unique_str(sat, row):
    track_self = track_to_str(row["track_self"])
    track_other = track_to_str(row["track_other"])
    return f"{sat}_{track_self}_{track_other}"


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)


cmap1 = cmo.cm.diff
d = 1.5
dpth = 15
height, width = 9, 9
frac1 = "0.05"


dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
omni_data_dir = "/home/srinivasu/allData/MB_cleaned/raju_omni_clean"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
out_match_csv = "omni_guv_c_nearest_global_matches.csv"
matched_rows_all = []
# SELECT_SAT = "TP+J1+J2+J3+S6A"
# SELECT_OMNI_ID = "BD09"
# SELECT_TRACK_SELF = "53"
# SELECT_TRACK_OTHER = "14"
SELECT_SAT = 'S3A'
SELECT_OMNI_ID = 'BD10'
SELECT_TRACK_SELF = '522'
SELECT_TRACK_OTHER = '279'
SELECT_SAT = 'S3A'
SELECT_OMNI_ID = 'BD10'
SELECT_TRACK_SELF = '636'
SELECT_TRACK_OTHER = '393'

EKMAN_DIR = "/home/srinivasu/xtrackm/ekman/ekman_at_intersections"
# TP+J1+J2+J3+S6A BD09 53 14
# S3A BD10 522 279

df_all_list = []
for sat in sats_new:
    df_all_list.append(pd.read_csv(f"tracks_intersections_{sat}_1.csv"))
df_all = pd.concat(df_all_list, ignore_index=True)


lon_omni, lat_omni = omni_d[SELECT_OMNI_ID]
omni_file = f"{omni_data_dir}/{SELECT_OMNI_ID}_current_sst.nc"
print(omni_file)
if not os.path.exists(omni_file):
    print(f"File not found: {omni_file}")
    raise(f"no omni file")
if abs(lat_omni) < 2:
    raise(f"lat less than 2")

ds_omni = xr.open_dataset(omni_file)
print(ds_omni)
# print(ds_omni)
ds_omni = xr.open_dataset(omni_file)
print(ds_omni)
omni_u = ds_omni.u_current.interp(current_depth=dpth)
# omni_u = omni_u.rolling(time=10, center=True).mean()
omni_u = omni_u
omni_v = ds_omni.v_current.interp(current_depth=dpth)
# omni_v = omni_v.rolling(time=10, center=True).mean()
omni_v = omni_v
N = omni_v.count().item()

df_selected = df_all[
    (df_all["sat"] == SELECT_SAT)
    & (df_all["track_self"].astype(str) == SELECT_TRACK_SELF)
    & (df_all["track_other"].astype(str) == SELECT_TRACK_OTHER)
].copy()
if df_selected.empty:
    raise("selected sat/track combination not found in intersections file")
lons = df_selected["lons_inter"].to_numpy(dtype=float)
```

```
lats = df_selected["lats_inter"].to_numpy(dtype=float)
distances = [distance.distance((lat, lon), (lat_omni, lon_omni)).km for lat, lon in zip(lats, lons)]
df_selected["distance_km"] = distances
row = df_selected.nsmallest(1, "distance_km").iloc[0]

sat = row["sat"]
track_number_self = str(row["track_self"])
track_number_other = str(row["track_other"])
lon1 = row["lons_inter"]
lat1 = row["lats_inter"]
distance_km = row["distance_km"]
angle_acute = row["angle_acute"]
angle_obtuse = row["angle_obtuse"]
unique_str = build_unique_str(sat, row)
f_ek = f"{EKMAN_DIR}/{unique_str}_ekman_at_intersections.nc"
ds_ek = xr.open_dataset(f_ek)
u_ek = ds_ek['u_at']
v_ek = ds_ek['v_at']

print(
    sat,
    SELECT_OMNI_ID,
    track_number_self,
    track_number_other,
    lon1,
    lat1,
    angle_acute,
    angle_obtuse,
)

f_guv = f"{guv_dir}/guv_at_intersection_{sat}_{track_number_self}_{track_number_other}_{frac1}.nc"
if not os.path.exists(f_guv):
    print(f"missing file: {f_guv}")
    raise("missing file")

with xr.open_dataset(f_guv) as ds_guv:
    gu = ds_guv.gu
    gv = ds_guv.gv

(
    lons_track_self,
    lats_track_self,
    lon_equat_self,
    lat_equat_self,
    lon_coast_self,
    lat_coast_self,
    angle_self,
) = get_track_geometry(sat, track_number_self)
(
    lons_track_other,
    lats_track_other,
    lon_equat_other,
    lat_equat_other,
    lon_coast_other,
    lat_coast_other,
    angle_other,
) = get_track_geometry(sat, track_number_other)

gu_cut = gu
gv_cut = gv

# Collocate omni at track-gu timestamps; avoid sat-dependent smoothing.
omni_u_match = omni_u.reindex(
    time=gu_cut.time,
    method="nearest",
    tolerance=np.timedelta64(24, "h"),
)
ek_u_match = u_ek.reindex(
    time=gu_cut.time,
    method="nearest",
    tolerance=np.timedelta64(24, "h"),
)
valid = np.isfinite(gu_cut) & np.isfinite(omni_u_match)
gu_pair = gu_cut.where(valid)
omni_u_pair = omni_u_match.where(valid)
ek_u_pair = ek_u_match.where(valid)

gu_ek_pair = gu_pair + ek_u_pair


df_pairs = xr.Dataset({"gu": gu_pair, "omni_u": omni_u_pair}).to_dataframe().reset_index()
df_pairs = df_pairs.dropna(subset=["gu", "omni_u"])

# check valid points
fig = plt.figure(figsize=(12, 8))
ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax1 = fig.add_subplot(2, 1, 2)
omni_u_pair.plot(ax=ax1, color="r", linewidth=2, label="omni")
gu_pair.plot(ax=ax1, color="k", linewidth=2, label="track")
gu_ek_pair.plot(ax=ax1, color="b", linewidth=2, label="track+ekman")

corr = xs.pearson_r(gu_pair, omni_u_pair, dim="time", skipna=True)
corr1 = corr.item()

info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
print(f"Correlation between gu and omni u: {corr1}")
plt.text(
    0.25,
    0.90,
    info,
    transform=ax1.transAxes,
    fontsize=12,
    fontweight="bold",
)
plt.legend()

rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=10)
ax2.grid()
ax2.plot(
    lon_omni,
    lat_omni,
    c="r",
    marker="o",
    markersize=4,
    markerfacecolor="red",
    markeredgecolor="red",
```

```
    markeredgewidth=2,
)

ax2.plot(
    lon1,
    lat1,
    c="k",
    marker=".",
    markersize=8,
    markerfacecolor="black",
    markeredgecolor="black",
    markeredgewidth=2,
)
plt.text(lon_omni, lat_omni, f"{SELECT_OMNI_ID}", fontsize=10, color="k")
ax2.scatter(
    lons_track_self, lats_track_self, marker=".", color="c", s=4
)
ax2.scatter(
    lons_track_other, lats_track_other, marker=".", color="c", s=4
)

lonm, latm = get_point_at_distance(
    lon_equat_self,
    lat_equat_self,
    lon_coast_self,
    lat_coast_self,
    d,
)
info = f"{sat} {track_number_self} {track_number_other} {distance_km:.2f} km"
plt.text(
    0.55,
    0.90,
    info,
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=angle_self,
        color="w",
    )

lonm, latm = get_point_at_distance(
    lon_equat_other,
    lat_equat_other,
    lon_coast_other,
    lat_coast_other,
    d,
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=angle_other,
        color="w",
    )

plt.savefig("p.omni_uvVStrack_guv_c.ek.nearest.selected.png")
plt.show()
ln_sf(f"p.omni_uvVStrack_guv_c.ek.nearest.selected.png", png_file)
plt.close("all")

dse.close()

```

```python
import math
import sys

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs
from matplotlib.colors import ListedColormap
from pathlib import Path

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

radar_str = "TN"
track_self_here = 608
track_other_here = 137
sat_here = "S3A"
width, height = 13, 6
TRACKS_REG = (73, 99, 0, 26)
d = 1.5

cmap1 = cmo.cm.diff
cmap2 = ListedColormap(['white'])
oscar_dir = "/home/srinivasu/allData/oscar_0.25/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
oscar_extracted_dir = f"/home/srinivasu/xtrackm/guv/oscar_at_intersections/"

REG = radar_extents[radar_str + "_BLOCK"]
gdf = gpd.read_file(f"polygon_shapefile_around_{radar_str}.shp")
polygon = gdf.geometry.iloc[0]
radar_tsta_o, radar_tend_o = radar_times[radar_str]

OUT_PNG = f"guv_timeseries_{sat_here}_{track_self_here}_{track_other_here}_{radar_str}.png"

dsr = xr.open_dataset(
    f"/home/srinivasu/allData/radar/{radar_str}Codar_daily.nc")
# print(dsr)
dsr = dsr.rename({
    "XAXS": "longitude",
    "YAXS": "latitude",
    "ZAXS": "lev",
    "TAXIS1D": "time"
})
# print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
s_radar = 0.01 * dsr.S_RADAR.isel(lev=0, drop=True)
sizes = u_radar.sizes
ln = sizes["time"]

df = pd.read_csv(f"../guv/tracks_intersections_{sat_here}_1.csv")

df_filter = df[(df["track_self"] == track_self_here) & (df["track_other"] == track_other_here)]

ln = len(df)
for idx, row in df_filter.iterrows():
    sat1 = row["sat"]
    track_number_self = str(row["track_self"])
    track_number_other = str(row["track_other"])
    lon1 = row["lons_inter"]
    lat1 = row["lats_inter"]
    lons_inter1 = row["lons_inter"]
    lats_inter1 = row["lats_inter"]
    x_from_coast_self1 = row["x_from_coast_self"]
    x_from_coast_other1 = row["x_from_coast_other"]
    angle_acute = row["angle_acute"]
    angle_obtuse = row["angle_obtuse"]
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    overlap_tsta, overlap_tend = overlap_dates(track_tsta_o, track_tend_o,
                                               radar_tsta_o, radar_tend_o)
    ru_at = u_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    rv_at = v_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    f_oscar = f"{oscar_extracted_dir}/oscar_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc"
    ds_oscar = xr.open_dataset(f_oscar)
    oscar_u = ds_oscar.u
    oscar_v = ds_oscar.v
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_self}_loess_0.2.nc"
    f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_other}_loess_0.2.nc"
    ds_self = xr.open_dataset(f_self, decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
```

```
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)
    gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
    fig = plt.figure(figsize=(12, 10))
    ax0 = fig.add_subplot(3, 1, 2)
    ax1 = fig.add_subplot(3, 1, 3)

    gu_cut = gu
    gv_cut = gv
    tfreq = sats_tfreq[sat1]

    oscar_u = oscar_u.resample(time=f"{tfreq}D").mean()
    ru_at = ru_at.resample(time=f"{tfreq}D").mean()
    oscar_u = oscar_u.interp(time=gu_cut.time)
    ru_at = ru_at.interp(time=gu_cut.time)
    oscar_max = oscar_u.max().item()
    gu_cut = gu_cut.where(gu_cut < oscar_max, np.nan)
    gu_cut = gu_cut.where(gu_cut > -oscar_max, np.nan)

    oscar_v = oscar_v.resample(time=f"{tfreq}D").mean()
    rv_at = rv_at.resample(time=f"{tfreq}D").mean()
    oscar_v = oscar_v.interp(time=gv_cut.time)
    rv_at = rv_at.interp(time=gu_cut.time)
    oscar_max = oscar_v.max().item()
    gv_cut = gv_cut.where(gv_cut < oscar_max, np.nan)
    gv_cut = gv_cut.where(gv_cut > -oscar_max, np.nan)

    oscar_u.plot(ax=ax0, color="b", linewidth=2, label="oscar u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="k", linewidth=2)
    ru_at.plot(ax=ax0, label=f"radar zonal", color="r", linewidth=2)
    ax0.legend()
    ax0.set_ylabel("oscar u m/s")
    oscar_v.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="k", linewidth=2)
    rv_at.plot(ax=ax1, label=f"radar meridional", color="r", linewidth=2)
    corr_u = xs.pearson_r(gu_cut, ru_at, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, ru_at, dim="time", skipna=True)
    bias_var_u = ru_at - gu_cut
    bias_u = bias_var_u.mean(dim="time")
    corr_v = xs.pearson_r(gv_cut, rv_at, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, rv_at, dim="time", skipna=True)
    bias_var_v = rv_at - gv_cut
    bias_v = bias_var_v.mean(dim="time")
    print(f"Correlation between gu and u_radar: {corr_u.item()}")
    print(f"Correlation between gv and v_radar: {corr_v.item()}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.05,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("oscar v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap2)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.add_geometries(
        [polygon],
        ccrs.PlateCarree(),
        facecolor="blue",
        edgecolor="red",
        alpha=0.2,
    )
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=4,
        markerfacecolor="red",
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    # plot.text radar_str
    plt.text(
        0.15,
        0.90,
        f"radar region: {radar_str}",
        transform=ax2.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
```

```
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"guv_timeseries_{sat1}_{track_number_self}_{track_number_other}_{radar_str}.png",
    )
    ln_sf(
        f"guv_timeseries_{sat1}_{track_number_self}_{track_number_other}_{radar_str}.png", "fig06.png"
    )
    ln_sf(OUT_PNG, png_file)
    plt.show()
    plt.close("all")

dse.close()
```

```python
import sys
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tools_xtrackm import ln_sf


# ---------------------------------------------------------------------
# Outlier cleaning
# ---------------------------------------------------------------------
def clean_outliers(gu, ve, ek=None, max_speed=3.0, mad_thresh=3.5):
    gu = np.asarray(gu)
    ve = np.asarray(ve)

    mask = np.isfinite(gu) & np.isfinite(ve)

    if ek is not None:
        ek = np.asarray(ek)
        mask &= np.isfinite(ek)

    # Physical bounds
    mask &= (np.abs(gu) <= max_speed) & (np.abs(ve) <= max_speed)
    if ek is not None:
        mask &= (np.abs(ek) <= max_speed)

    def mad_outlier(x, thresh):
        med = np.median(x)
        mad = np.median(np.abs(x - med))
        if mad == 0:
            return np.zeros_like(x, dtype=bool)
        z = 0.6745 * (x - med) / mad
        return np.abs(z) > thresh

    mask &= ~mad_outlier(gu, mad_thresh)
    mask &= ~mad_outlier(ve, mad_thresh)
    if ek is not None:
        mask &= ~mad_outlier(ek, mad_thresh)

    if ek is None:
        return gu[mask], ve[mask]
    else:
        return gu[mask], ve[mask], ek[mask]


# ---------------------------------------------------------------------
# Plot helper
# ---------------------------------------------------------------------
def scatter_with_fit(ax, x, y, xlabel, title):
    ax.scatter(x, y, alpha=0.5)
    ax.grid(True)

    # Correlation
    corr = np.corrcoef(x, y)[0, 1]

    ax.text(
        0.05, 0.95, f"Correlation: {corr:.2f}",
        transform=ax.transAxes,
        va="top",
        bbox=dict(boxstyle="round,pad=0.4", fc="yellow", alpha=0.5),
    )

    # Trend line
    m, b = np.polyfit(x, y, 1)
    xs = np.sort(x)
    ax.plot(xs, m * xs + b, color="red", lw=2, label="Trend line")

    ax.set_xlabel(xlabel)
    ax.set_title(title)
    ax.legend()


# ---------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------
# sat = sys.argv[1]
sat = "TP+J1+J2+J3+S6A"

# Read data
df_no_ek = pd.read_csv(
    f"track_guvVSnearby_drifters_at_intersection_points_{sat}_2.csv"
)

df_ek = pd.read_csv(
    f"track_guvVSnearby_drifters_at_intersection_points_{sat}_w_ekman.csv"
)

# Clean data
gu1, ve1 = clean_outliers(df_no_ek["gu"], df_no_ek["ve"])
gu2, ve2, ek2 = clean_outliers(df_ek["gu"], df_ek["ve"], df_ek["ek"])
gu_ek = gu2 + ek2

# ---------------------------------------------------------------------
# Figure: side-by-side, shared y-axis
# ---------------------------------------------------------------------
fig, axes = plt.subplots(
    ncols=2,
    nrows=1,
    figsize=(14, 6),
    sharey=True
)

# Left: gu vs ve
scatter_with_fit(
    axes[0],
    gu1,
    ve1,
    xlabel="Track gu (m/s)",
    title="gu vs drifter ve"
)
axes[0].set_ylabel("Drifter ve (m/s)")

# Right: gu+ek vs ve
scatter_with_fit(
    axes[1],
    gu_ek,
    ve2,
```

```
    xlabel="Track (gu + ek) (m/s)",
    title="gu + ek vs drifter ve"
)

plt.tight_layout()
plt.savefig(f"scatter_{sat}_gu_vs_ve_and_guEk_vs_ve.png", dpi=300)
ln_sf(f"scatter_{sat}_gu_vs_ve_and_guEk_vs_ve.png", "fig05.png")
plt.show()

```

```python
import os
import sys

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from pathlib import Path
from tools_xtrackm import ln_sf

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

SAT = "TP+J1+J2+J3+S6A"
CSV_NO_EK = f"drifter_ve_vs_gu_{SAT}.csv"
CSV_W_EK = f"drifter_ve_vs_gu_ek_{SAT}.csv"
OUT_PNG = f"scatter_drifter_ve_vs_gu_ww_ek_{SAT}.png"
COAST_DIST_PATH = "~/allData/topo/GMT_intermediate_coast_distance_01d_track_reg.nc"
PROXIMITY_DIST_M = 1000000.0
MAX_ABS_TIME_DIFF_DAYS = None


def get_coast_distance_km(lons, lats):
    with xr.open_dataset(os.path.expanduser(COAST_DIST_PATH)) as ds_coast:
        lon_da = xr.DataArray(np.asarray(lons, dtype=float), dims="point")
        lat_da = xr.DataArray(np.asarray(lats, dtype=float), dims="point")
        coast_km = ds_coast["coast_distance"].sel(lon=lon_da, lat=lat_da, method="nearest")
    return coast_km.values.astype(float)


def compute_time_diff_days(df):
    drifter_t = pd.to_datetime(df["drifter_time"], errors="coerce")
    gu_t = pd.to_datetime(df["gu_time"], errors="coerce")
    return (drifter_t - gu_t).dt.total_seconds() / 86400.0


def summarize_time_diff_days(dt_days, label):
    dt_days = dt_days[np.isfinite(dt_days.to_numpy(dtype=float))]
    if dt_days.size == 0:
        print(f"[{label}] No valid drifter_time/gu_time pairs for temporal-difference stats.")
        return
    dt_abs = np.abs(dt_days)
    mode_signed = pd.Series(np.round(dt_days, 6)).mode().to_numpy()
    mode_abs = pd.Series(np.round(dt_abs, 6)).mode().to_numpy()
    mode_signed_str = ", ".join([f"{v:.6f}" for v in mode_signed[:5]])
    mode_abs_str = ", ".join([f"{v:.6f}" for v in mode_abs[:5]])
    print(
        f"[{label}] Time diff days (drifter_time - gu_time): "
        f"min={dt_days.min():.6f}, max={dt_days.max():.6f}, "
        f"mean={dt_days.mean():.6f}, median={dt_days.median():.6f}, mode={mode_signed_str}"
    )
    print(
        f"[{label}] Abs time diff days: "
        f"min={dt_abs.min():.6f}, max={dt_abs.max():.6f}, "
        f"mean={dt_abs.mean():.6f}, median={dt_abs.median():.6f}, mode={mode_abs_str}"
    )


def load_filtered_df(csv_path, label):
    df = pd.read_csv(csv_path)
    coast_km = get_coast_distance_km(df["lon_inter"].values, df["lat_inter"].values)
    proximity_km = PROXIMITY_DIST_M / 1000.0
    keep = np.isfinite(coast_km) & (np.abs(coast_km) <= proximity_km)
    df = df.loc[keep].copy()

    dt_days = compute_time_diff_days(df)
    if MAX_ABS_TIME_DIFF_DAYS is not None:
        keep_time = np.isfinite(dt_days.to_numpy(dtype=float)) & (
            np.abs(dt_days.to_numpy(dtype=float)) <= float(MAX_ABS_TIME_DIFF_DAYS)
        )
        df = df.loc[keep_time].copy()
        dt_days = dt_days.loc[df.index]

    summarize_time_diff_days(dt_days, label)
    return df


def fit_and_stats(x, y):
    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))
    return slope, intercept, r, stderr


def prepare_xy(df, x_col):
    x = df[x_col].to_numpy(dtype=float)
    y = df["drifter_ve"].to_numpy(dtype=float)
    valid = np.isfinite(x) & np.isfinite(y)
    return x[valid], y[valid]


def prepare_xy_with_ek(df):
    x = df["gu"].to_numpy(dtype=float) + df["u_ek"].to_numpy(dtype=float)
    y = df["drifter_ve"].to_numpy(dtype=float)
    valid = np.isfinite(x) & np.isfinite(y)
    return x[valid], y[valid]


def plot_panel(ax, x, y, panel_title, x_label, y_label):
    slope, intercept, r, stderr = fit_and_stats(x, y)
    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1)

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
```

```
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=10,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="white", alpha=0.55, edgecolor="gray"),
        color="k",
    )

    ax.text(
        0.5,
        0.97,
        panel_title,
        transform=ax.transAxes,
        va="top",
        ha="center",
        fontsize=11,
        bbox=dict(boxstyle="round,pad=0.25", facecolor="White", alpha=0.35, edgecolor="gray"),
        color="black",
    )
    ax.set_xlabel(x_label)
    # ax.set_ylabel("drifter ve (m/s)")
    ax.set_ylabel(y_label)
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right")
    return hb


def main():
    none = ""
    df_no_ek = load_filtered_df(CSV_NO_EK, "without_ek")
    df_w_ek = load_filtered_df(CSV_W_EK, "with_ek")

    x_no_ek, y_no_ek = prepare_xy(df_no_ek, "gu")
    x_w_ek, y_w_ek = prepare_xy_with_ek(df_w_ek)

    if x_no_ek.size < 3 or x_w_ek.size < 3:
        print(
            "Insufficient valid points after filtering.\n"
            f"without_ek: {x_no_ek.size} points, with_ek: {x_w_ek.size} points.\n"
            f"Current filters: PROXIMITY_DIST_M={PROXIMITY_DIST_M}, "
            f"MAX_ABS_TIME_DIFF_DAYS={MAX_ABS_TIME_DIFF_DAYS}"
        )
        return

    all_vals = np.concatenate([x_no_ek, y_no_ek, x_w_ek, y_w_ek])
    vmin = np.nanmin(all_vals)
    vmax = np.nanmax(all_vals)

    fig, axes = plt.subplots(1, 2, figsize=(16, 7), sharex=True, sharey=True)

    hb0 = plot_panel(
        axes[0],
        x_no_ek,
        y_no_ek,
        "Without Ekman",
        "Altimetry gu (m/s)",
        "Drifter ve (m/s)"
    )
    hb1 = plot_panel(
        axes[1],
        x_w_ek,
        y_w_ek,
        "With Ekman",
        "Altimetry gu + u_ek (m/s)",
        none
    )

    for ax in axes:
        # ax.set_xlim(vmin, vmax)
        # ax.set_ylim(vmin, vmax)
        ax.set_xlim(vmin, 1.5)
        ax.set_ylim(vmin, 1.5)

    cmax = max(hb0.get_array().max(), hb1.get_array().max())
    hb0.set_clim(1, cmax)
    hb1.set_clim(1, cmax)

    # fig.suptitle(
    #     f"{SAT}: Drifter ve comparison | coast <= {PROXIMITY_DIST_M:.0f} m, "
    #     f"|dt| <= {MAX_ABS_TIME_DIFF_DAYS if MAX_ABS_TIME_DIFF_DAYS is not None else 'all'} d"
    # )
    fig.tight_layout(rect=[0.0, 0.0, 0.89, 0.95])
    cax = fig.add_axes([0.91, 0.13, 0.015, 0.75])
    cbar = fig.colorbar(hb1, cax=cax)
    cbar.set_label("Count")
    fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)

    print(f"Rows after filters (without_ek): {len(df_no_ek)}")
    print(f"Rows after filters (with_ek): {len(df_w_ek)}")
    print(f"Saved {OUT_PNG}")

    ln_sf(OUT_PNG, png_file)

if __name__ == "__main__":
    main()
```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from pathlib import Path
import sys
import os
from tools_xtrackm import ln_sf

# codex resume 019cd8ab-eee9-72e0-9952-b9e4a8bf1b11

RAMA_NO_EK_CSV = "rama_guv_c_nearest_global_matches.csv"
RAMA_EK_CSV = "rama_guv_c_nearest_global_matches.ek.csv"
OMNI_NO_EK_CSV = "omni_guv_c_nearest_global_matches.csv"
OMNI_EK_CSV = "omni_guv_c_nearest_global_matches.ek.csv"

OUT_PNG = "scatter_omniVSrama_uVSgu_ww_ek.png"
# get file name

script = Path(sys.argv[0])
png_file = script.with_suffix(".png")

MIN_DISTANCE_KM = None
MAX_DISTANCE_KM = 250.0


def load_xy(csv_path, y_col, with_ekman=False):
    df = pd.read_csv(csv_path)

    dist = df["distance_km"].to_numpy(dtype=float)
    keep = np.isfinite(dist)
    if MIN_DISTANCE_KM is not None:
        keep &= dist >= float(MIN_DISTANCE_KM)
    if MAX_DISTANCE_KM is not None:
        keep &= dist <= float(MAX_DISTANCE_KM)
    df = df.loc[keep].copy()

    x = df["gu"].to_numpy(dtype=float)
    if with_ekman:
        x = x + df["u_ek"].to_numpy(dtype=float)

    y = df[y_col].to_numpy(dtype=float)

    valid = np.isfinite(x) & np.isfinite(y)
    x = x[valid]
    y = y[valid]

    return x, y, len(df)


def add_panel(ax, x, y, title, ylabel, xlabel, extent=None):
    if x.size < 3:
        ax.text(
            0.5,
            0.98,
            f"{title} (insufficient points)",
            transform=ax.transAxes,
            va="top",
            ha="center",
            fontsize=10,
            bbox=dict(boxstyle="round,pad=0.25", facecolor="black", alpha=0.5, edgecolor="white"),
            color="white",
        )
        ax.set_xlabel(xlabel)
        ax.set_ylabel(ylabel)
        ax.grid(alpha=0.25)
        return None

    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))

    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1, extent=extent)

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=10,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="black", alpha=0.55, edgecolor="white"),
        color="white",
    )

    ax.text(
        0.6,
        0.98,
        title,
        transform=ax.transAxes,
        va="top",
        ha="center",
        fontsize=10,
        bbox=dict(boxstyle="round,pad=0.25", facecolor="white", alpha=0.5, edgecolor="white"),
        color="black",
    )
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right", fontsize=9)
    ax.set_ylim(-1, 1)
    ax.set_xlim(-1, 1)
    return hb


def main():
```

```python
    min_lbl = MIN_DISTANCE_KM if MIN_DISTANCE_KM is not None else "-inf"
    max_lbl = MAX_DISTANCE_KM if MAX_DISTANCE_KM is not None else "+inf"
    x_label_no_ek = "Altimetry gu (m/s)"
    x_label_ek = "Altimetry gu + u_ek (m/s)"
    none = ""

    x_rama_no, y_rama_no, n_rama_no = load_xy(RAMA_NO_EK_CSV, "rama_u", with_ekman=False)
    x_rama_ek, y_rama_ek, n_rama_ek = load_xy(RAMA_EK_CSV, "rama_u", with_ekman=True)
    x_omni_no, y_omni_no, n_omni_no = load_xy(OMNI_NO_EK_CSV, "omni_u", with_ekman=False)
    x_omni_ek, y_omni_ek, n_omni_ek = load_xy(OMNI_EK_CSV, "omni_u", with_ekman=True)
    all_x = np.concatenate([x_rama_no, x_rama_ek, x_omni_no, x_omni_ek])
    all_y = np.concatenate([y_rama_no, y_rama_ek, y_omni_no, y_omni_ek])
    global_extent = (
        float(np.nanmin(all_x)),
        float(np.nanmax(all_x)),
        float(np.nanmin(all_y)),
        float(np.nanmax(all_y)),
    )

    fig, axs = plt.subplots(2, 2, figsize=(16, 12), sharex=True, sharey=True, constrained_layout=True)

    hb00 = add_panel(
        axs[0, 0],
        x_rama_no,
        y_rama_no,
        f"RAMA u vs gu (no Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        "RAMA u (m/s)",
        none,
        extent=global_extent,
    )
    hb01 = add_panel(
        axs[0, 1],
        x_rama_ek,
        y_rama_ek,
        f"RAMA u vs gu+u_ek (with Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        none,
        none,
        extent=global_extent,
    )
    hb10 = add_panel(
        axs[1, 0],
        x_omni_no,
        y_omni_no,
        f"OMNI u vs gu (no Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        "OMNI u (m/s)",
        x_label_no_ek,
        extent=global_extent,
    )
    hb11 = add_panel(
        axs[1, 1],
        x_omni_ek,
        y_omni_ek,
        f"OMNI u vs gu+u_ek (with Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        none,
        x_label_ek,
        extent=global_extent,
    )

    top_hbs = [hb for hb in [hb00, hb01] if hb is not None]
    if top_hbs:
        top_vmax = max(float(np.nanmax(hb.get_array())) for hb in top_hbs)
        for hb in top_hbs:
            hb.set_clim(1.0, top_vmax)
        cbar_top = fig.colorbar(top_hbs[0], ax=axs[0, :], location="right", fraction=0.03, pad=0.02)
        cbar_top.set_label("Count")

    bottom_hbs = [hb for hb in [hb10, hb11] if hb is not None]
    if bottom_hbs:
        bottom_vmax = max(float(np.nanmax(hb.get_array())) for hb in bottom_hbs)
        for hb in bottom_hbs:
            hb.set_clim(1.0, bottom_vmax)
        cbar_bottom = fig.colorbar(bottom_hbs[0], ax=axs[1, :], location="right", fraction=0.03, pad=0.02)
        cbar_bottom.set_label("Count")

    fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)

    print(f"Rows after distance filter | RAMA no-ek: {n_rama_no}, RAMA ek: {n_rama_ek}")
    print(f"Rows after distance filter | OMNI no-ek: {n_omni_no}, OMNI ek: {n_omni_ek}")
    print(f"Saved {OUT_PNG}")
    # link OUT_PNG to png_file
    ln_sf(OUT_PNG, png_file)


if __name__ == "__main__":
    main()

```

```python
import math
import pickle
from datetime import datetime
import sys
import os

import cmocean as cmo
import matplotlib.gridspec as gridspec
import matplotlib.pyplot as plt
import numpy as np
import shapely
import shapely as shp
import xarray as xr
from tools_xtrackm import *


# function to get percentage valid point in an xarray dataarray
def perc_valid_points(da):
    return 100 * (da.count().item() / len(da.time))


xr.set_options(display_expand_data=False)
gs = gridspec.GridSpec(1,
                       2,
                       width_ratios=[3, 1],
                       height_ratios=[2],
                       wspace=0.1,
                       hspace=0.05)
PSMSL_ID = 414  # id for visakhaptnam
width, height = 13, 6
data_dir = f"/home/srinivasu/xtrackm/data/"

cmap1 = cmo.cm.topo
cmap1 = cmo.cm.diff
cmap1 = cmo.cm.balance
cmap2 = cmo.cm.thermal

# tide gauge data
df = read_tide_meta()
df_nio = region_selected(df, *NIO)
lon_psmsl, lat_psmsl, name_psmsl = get_lonlat_psmsl(PSMSL_ID, df_nio)
print(lon_psmsl, lat_psmsl)
name_psmsl = name_psmsl.strip()

delx2 = 5
lonlat_box2 = (
    lon_psmsl - delx2,
    lon_psmsl + delx2,
    lat_psmsl - delx2,
    lat_psmsl + delx2,
)
poly_box = shp.geometry.box(lonlat_box2[0], lonlat_box2[2], lonlat_box2[1],
                            lonlat_box2[3])
# get the perimeter of the box
ds_id = read_id(PSMSL_ID)
sla_tide = ds_id["height"]
sla_tide = 0.001 * sla_tide
tide_tsta_o = ds_id.time[0].values
tide_tend_o = ds_id.time[-1].values
tide_tsta_o = n64todatetime(tide_tsta_o)
tide_tend_o = n64todatetime(tide_tend_o)

tlim1 = datetime.strptime("2013-01-01", "%Y-%m-%d")
tlim2 = datetime.strptime("2021-01-01", "%Y-%m-%d")


def point_on_line_at_distance(m, c, x0, y0, d):
    """
    Find a point on the line y = mx + c at a distance d from (x0, y0).
    Parameters:
        m (float): Slope of the line.
        c (float): y-intercept of the line.
        x0 (float): x-coordinate of the starting point.
        y0 (float): y-coordinate of the starting point.
        d (float): Distance from the starting point.
    Returns:
        tuple: Coordinates (x, y) of the new point.
    """
    # Normalize the direction vector of the line
    dx = 1 / math.sqrt(1 + m**2)  # x-component of the unit direction vector
    dy = m / math.sqrt(1 + m**2)  # y-component of the unit direction vector
    # Compute the new point
    if m < 0:
        d = -d
    x_new = x0 + d * dx
    y_new = y0 + d * dy
    return x_new, y_new


ds_dt1 = xr.open_dataset(
    "/home/srinivasu/allData/aviso/allyear_dt_global_allsat_msla_h.nc",
    decode_times=False,
)
# ds_dt1 = xr.open_dataset(
#     "/media/srinivasu/allData/sealevel/data/allyear_dt_global_allsat_msla_h.nc",
#     decode_times=False,
# )
print(ds_dt1)
ds_dt = change_time(ds_dt1, "time")
sla_aviso = ds_dt.sla  # .sel(time=slice(tsta_o, tend_o))
sla_aviso_at = sla_aviso.interp(longitude=lon_psmsl,
                                method="nearest").interp(latitude=lat_psmsl,
                                                         method="nearest")

sla_tide_interp = sla_tide.interp(time=sla_aviso.time)
sla_tide_interp = sla_tide_interp - sla_tide_interp.mean(dim="time")
# sla_tide_adj = sla_tide_interp  - dac_p_smooth

chosen_index = 0
dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")

with open("closest_tracks_all_psmsl_new.pkl", "rb") as file:
    sorted_list = pickle.load(file)


def filter_list(sorted_list, sat=None, track_no=None, id=None):
    result = sorted_list
```

```
    if sat is not None:
        result = [tup for tup in result if tup[0] == sat]
    if track_no is not None:
        result = [tup for tup in result if tup[1] == track_no]
    if id is not None:
        result = [tup for tup in result if tup[5] == id]
    return result


filtered_list = sorted_list[:]
sat_here = "ERS1+ERS2+ENV+SRL"
track_no = "309"
filtered_list = filter_list(sorted_list, sat=sat_here, track_no=track_no, id=PSMSL_ID)
print(filtered_list)

# sys.exit(0)
for sat, trackn, x1, y1, mindist, id in filtered_list:
    if mindist > 50:
        continue
    track_tsta_o, track_tend_o = get_time_limits_o(sat)
    overlap_tsta, overlap_tend = overlap_dates(track_tsta_o, track_tend_o,
                                               tide_tsta_o, tide_tend_o)
    tsta_o, tend_o = get_time_limits_o(sat)
    print(sat, trackn, x1, y1, mindist)
    if trackn is None:
        continue
    # zero pad trackn with three digits
    trackn1 = trackn.zfill(3)
    f = f"{data_dir}/{sat}/ctoh.sla.ref.{sat}.nindian.{trackn1}.nc"
    ds = xr.open_dataset(f, decode_times=False)
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
    c = (lats_track[0]) - m * (lons_track[0])
    x_from_coast = distance.distance((lat_coast, lon_coast), (y1, x1)).km
    # shapely path of the track
    track_path = shapely.geometry.LineString(list(zip(lons_track, lats_track)))
    if len(lons_track) == 0:
        continue
    m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
    angle = np.rad2deg(np.arctan(m))
    c = (lats_track[0]) - m * (lons_track[0])
    ds_smooth = xr.open_dataset(
        f"/home/srinivasu/xtrackm/computed/sla_loess_0.2a/track_sla_along_{sat}_{trackn}_loess_0.2.nc"
    )
    sla_track_smooth = ds_smooth.sla_smooth
    dac_track_smooth = ds_smooth.dac_smooth
    sla_track_p_smooth = sla_track_smooth.sel(x=x_from_coast,
                                              method="nearest",
                                              drop=True)
    dac_track_p_smooth = dac_track_smooth.sel(x=x_from_coast,
                                              method="nearest",
                                              drop=True)
    sla_plus = sla_track_p_smooth + dac_track_p_smooth
    fig = plt.figure(figsize=(width, height))
    ax1 = fig.add_subplot(gs[0, 0])
    # sla_aviso_at.plot(ax=ax1,
    #                   label="extracted aviso grid data",
    #                   linestyle="--")
    sla_tide_interp.plot(ax=ax1, label="tide gauge data", linewidth=4)
    # sla_track_p_smooth.plot(ax=ax1, label=f"track {trackn} {sat}")
    sla_plus.plot(ax=ax1, label=f"track {trackn} {sat} + dac")
    # ax1.set_xlim(tlim1, tlim2)
    ax1.set_xlim(overlap_tsta, overlap_tend)
    ax1.set_ylim(-0.6, 0.6)
    info1 = f"{sat} {trackn} {mindist:.2f}km"
    ax1.text(
        0.5,
        0.9,
        info1,
        horizontalalignment="center",
        verticalalignment="center",
        transform=ax1.transAxes,
        fontweight="bold",
        fontsize=14,
    )
    ax1.legend(loc="lower right")

    ax2 = fig.add_subplot(gs[0, 1], projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*lonlat_box2[:2])).sel(ETOPO05_Y=slice(
        *lonlat_box2[2:])).plot(ax=ax2,
                                add_colorbar=False,
                                add_labels=False,
                                cmap=cmap1)
    decorate_axis(ax2, "", *lonlat_box2, step=2)
    # get intersection of track and box
    intersection = track_path.intersection(poly_box)
    # get the intersection points
    intersection_points = intersection.coords
    # sort intesections points by latitude
    intersection_points = sorted(intersection_points, key=lambda x: x[1])
    # take the first intersection points
    if len(intersection_points) == 0:
        continue
    lonl = intersection_points[0][0]
    latl = intersection_points[0][1]
    print(lonl, latl)
    info = f"{sat} {trackn}"
    lonm, latm = point_on_line_at_distance(m, c, lonl, latl, 2)
    print(lonm, latm)
    ax2.scatter(lons_track, lats_track, linewidths=0.0, s=8, color="r")
    ax2.text(
        lonm,
        latm,
        s=f"{trackn}",
        fontsize=10,
        rotation=angle,
        color="white",
        horizontalalignment="center",
        verticalalignment="center",
        fontweight="bold",
    )
    ax2.add_geometries(
        [poly_box],
        ccrs.PlateCarree(),
```

```
        facecolor="none",
        edgecolor="lightblue",
        alpha=0.5,
    )
    # PSMSL tide gauge location
    ax2.scatter(
        lon_psmsl,
        lat_psmsl,
        marker="*",
        c="yellow",
        s=150,
        edgecolor="black",
    )
    ax2.scatter(
        x1,
        y1,
        marker="x",
        c="cyan",
        s=10,
    )
    # PSMSL tide gauge name
    ax2.text(
        lon_psmsl,
        lat_psmsl,
        s=name_psmsl,
        fontsize=10,
        color="k",
        horizontalalignment="left",
    )
    ax2.set_xlim([*lonlat_box2[:2]])
    ax2.set_ylim([*lonlat_box2[2:]])
    # sla_tide_adj.plot(ax=ax1, label="tide gauge data adjusted")
    ax2.legend(loc="lower right")
    info = f"{sat}"
    ax2.text(0.1, 0.95, info, transform=ax2.transAxes, fontweight="bold")
    plt.savefig(
        f"tseries_loess_adj_allsat_map_box_separate_{sat}_{trackn}.png",
        bbox_inches="tight",
    )
    ln_sf(
        f"tseries_loess_adj_allsat_map_box_separate_{sat}_{trackn}.png", "fig02.png")
    # plt.show()
    plt.close("all")

```

```python
import shutil
from pathlib import Path
import os
from glob import glob

links_dict = {
    "p.compare.std_mean.l2pVSxtrack_map.py":"fig01.py",
    "p.omni_uvVStrack_guv_c.ek.nearest.selected.py": "fig02.py",
    "p.scatter_omniVSrama_uVSgu_ww_ek.py": "fig03.py",
    "p.scatter_drifter_ve_vs_gu_ww_ek.py": "fig04.py",
    "p.radar_uvVStrack_guv.at.intersections_single.py": "fig05.py",
    "p.gc.clim.cross_structure_x_arrows_map_ERS.py": "fig06.py",
    "p.gc.clim.lat_wise.along_xwidth.py": "fig07.py",
    "p.along_shore_at_155_116.intersection.S6A.py": "fig08.py",
    "p.speed_at_coastal_points.eicc.trends.all_in_one.ERS.py": "fig09.py",
    "satellite_mission_time_spans_new.py": "fig_sup_01.py"
}

# archive_dir = Path("archive")
# for full_path, fig_path in links_dict.items():
#     src = Path(full_path)
#     dst = Path(fig_path)
#     stem = dst.stem
#     new_name = f"{stem}_{src.name}.py"
#     # check if src exists
#     if src.exists():
#         print(f"{src.name} --> {new_name}")
#         os.rename(src, new_name)
#     else:
#         print(f"File {src} does not exist ---------------------------")

def get_fig_glob(f):
    GLOB = f"fig*{f}"
    files = glob(GLOB)
    return files[0]


for f in links_dict.keys():
    figf = get_fig_glob(f)
    if Path(figf).exists():
        print(f"{figf} exists")
    else:
        print(f"{figf} does not exist ---------------------------")

```

```
./draft_figs02/captions/
```

```
10 file(s)
```

```
1
2  between X-TRACK and along-track Level-2+ (L2P) products for common ground
3  tracks in the North Indian Ocean. (a) Spatial distribution of selected common
4  S3A passes over ETOPO5 bathymetry, with track numbers annotated in red. (b)
5  Temporal standard deviation of SLA as a function of distance from coast for
6  individual tracks (dots) and their multi-track mean (solid lines) from
7  X-TRACK (blue) and L2P (gray). (c) Number of valid SLA observations per
8  along-track point plotted against GSHHG coastline distance, showing X-TRACK
9  (blue) and L2P (gray) individual track values (dots) and multi-track means
10 (solid lines). The dashed vertical line in (b) and (c) marks the coastline
11
```

```
1
2  intersection of two satellite tracks (155, 116).
3
```

1

2

1 Left: Radar data from Doppler radar scanning vertically through the center of a wake vortex pair generated by an aircraft during approach and landing. Right: Vertical profiles of the circulation derived from the radar data.

2

```
1
2  zonal currents(x-axis) at nearby intersection points. b) Shows scatter plot with altimetry derived currents corrected with ekman zonal currents. c) and d) show scatter
        plots of OMNI currents with and without Ekman correction against altimetry zonal currents. The colorbar shows density of points in the scatter plot in terms of co
       unt. A redline passing through the points indicates linear fit.
3
```

```
./drifter/
```

26 file(s)

```
1
2
3  Our track_dist_time_asn function outputs numpy.datetime64 not list of datetime objects after opening file with decode_times=False
4  drifter files are opened without decode_times=False and defalut time datatype is numpy.datetime64, even after drifter_time_asn, it remains
5  numpy.datetime64
6  following seems redunt here.
7  drift_tsta_o, drift_tend_o = n64todatetime1(drift_tsta_o1), n64todatetime1(drift_tend_o1)
8
9  The datatypes of times in the drifter or track depend on option like
10 decode_times=False or decode_times=True while reading the netcdf files.
11
12 datatypes of times
13 drift_tsta_o1, drift_tend_o1 = (
14     ds_d.start_date.values[0],
15     ds_d.end_date.values[0],
16 )
17 <class 'numpy.datetime64'>
18
19
20 The total number of drifters in track_reg is only 852.
21 Total buoy over whole glove is 2401.
22
```

```
1
2
3  1. At the intersection points, the two passing tracks can be measuring
4  the same point at different times (as much seperated as the orbital period of the satellite mission)
5
```

```python
import os

import numpy as np
import xarray as xr

# import Polygon
from shapely.geometry.polygon import Polygon
from tools_xtrackm import *


def closest_index(lons_drift, lats_drift, lon1, lat1):
    dist = np.sqrt((lons_drift - lon1)**2 + (lats_drift - lat1)**2)
    return np.argmin(dist)


def is_within_region_any(lons,
                         lats,
                         lon_min,
                         lon_max,
                         lat_min,
                         lat_max,
                         include_boundary=True):
    """
    Return True if any (lon, lat) point falls inside the rectangular region.
    include_boundary=True counts points on the edges/corners as inside.
    """
    lons = np.asarray(lons, dtype=float)
    lats = np.asarray(lats, dtype=float)
    if lons.shape != lats.shape:
        raise ValueError("lons and lats must have the same shape")
    if include_boundary:
        lons_in = (lons >= lon_min) & (lons <= lon_max)
        lats_in = (lats >= lat_min) & (lats <= lat_max)
    else:
        lons_in = (lons > lon_min) & (lons < lon_max)
        lats_in = (lats > lat_min) & (lats < lat_max)
    inside_mask = lons_in & lats_in
    hit = np.any(inside_mask)
    if hit:
        print("At least one point is inside the region.")
    else:
        print("No points are inside the region.")
    return bool(hit)


chunk_time_dict = {
    "1_5000": (
        np.datetime64("1985-10-30T00:00:00.000000000"),
        np.datetime64("1999-11-01T00:00:00.000000000"),
    ),
    "5001_10000": (
        np.datetime64("1997-03-20T00:00:00.000000000"),
        np.datetime64("2006-09-16T00:00:00.000000000"),
    ),
    "10001_15000": (
        np.datetime64("2006-05-24T00:00:00.000000000"),
        np.datetime64("2011-01-04T00:00:00.000000000"),
    ),
    "15001_current": (
        np.datetime64("2010-10-30T00:00:00.000000000"),
        np.datetime64("2024-12-31T00:00:00.000000000"),
    ),
}

# df_all_list = []
# for sat in sats_new:
#     df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
#     df_all_list.append(df)
# df_all = pd.concat(df_all_list, ignore_index=True)

d = 1.5
data_loc = f"/home/srinivasu/allData/drifter1/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
rose = dse.ROSE
cmap1 = "Greys_r"
dist_threshold = 0.5  # in degrees
# chunk = "15001_current"
# chunks = ["1_5000", "5001_10000", "10001_15000", "15001_current"]
# chunks = ["15001_current"]
# chunks = ["1_5000"]
chunks = ["5001_10000"]
df_all = pd.read_csv(f"tracks_intersections_TP+J1+J2+J3+S6A_test.csv")
# df_close = df_all.copy()
# creat df with same colums as df_all
# df_close = pd.DataFrame(columns=df_all.columns)
df_close = df_all.copy()
close_drifters_column = []

found = False
for i, r in df_all.iterrows():
    close_ones = []
    sat1 = r["sat"]
    # if sat1 != "S3A":
    #     continue
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    track_number_self = str(r["track_self"])
    track_number_other = str(r["track_other"])
    lons_inter1 = r["lons_inter"]
    lats_inter1 = r["lats_inter"]
    x_from_coast_self1 = r["x_from_coast_self"]
    x_from_coast_other1 = r["x_from_coast_other"]
    angle_acute = r["angle_acute"]
    angle_obtuse = r["angle_obtuse"]
    lon1 = lons_inter1
    lat1 = lats_inter1
    if abs(lat1) < 2:
        close_drifters_column.append([])
        continue
    print(
        sat1,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
```

```
        angle_obtuse,
    )
    lon_inter_box_min = lon1 - dist_threshold
    lon_inter_box_max = lon1 + dist_threshold
    lat_inter_box_min = lat1 - dist_threshold
    lat_inter_box_max = lat1 + dist_threshold
    box = Polygon([
        (lon_inter_box_min, lat_inter_box_min),
        (lon_inter_box_max, lat_inter_box_min),
        (lon_inter_box_max, lat_inter_box_max),
        (lon_inter_box_min, lat_inter_box_max),
    ])
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    ds_self = xr.open_dataset(f_self, engine="h5netcdf", decode_times=False)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    print(f"{angle_self} {angle_other} --------------------")
    for chunk in chunks:
        chunk_tsta_o1, chunk_tend_o1 = chunk_time_dict[chunk]
        chunk_tsta_o, chunk_tend_o = n64todatetime1(
            chunk_tsta_o1), n64todatetime1(chunk_tend_o1)
        chunk_overlap_tsta_o, chunk_overlap_tend_o = overlap_dates(
            track_tsta_o, track_tend_o, chunk_tsta_o, chunk_tend_o)
        if chunk_overlap_tsta_o is None or chunk_overlap_tend_o is None:
            continue
        for fd in sorted(
                glob.glob(
                    f"{data_loc}/netcdf_{chunk}/track_reg/drifter_6h_*.nc")):
            print(fd)
            basename1 = os.path.basename(fd)
            ds_d = xr.open_dataset(fd, drop_variables=["WMO"])

            byte_id = ds_d.ID.values[0]
            str_id = byte_id.decode("utf-8").strip()
            drifter_id = str_id

            drift_tsta_o1, drift_tend_o1 = (
                ds_d.start_date.values[0],
                ds_d.end_date.values[0],
            )
            drift_tsta_o, drift_tend_o = n64todatetime1(
                drift_tsta_o1), n64todatetime1(drift_tend_o1)
            overlap_tsta_o, overlap_tend_o = overlap_dates(
                track_tsta_o, track_tend_o, drift_tsta_o, drift_tend_o)
            print(f"overlap period {overlap_tsta_o} {overlap_tend_o}")
            if overlap_tsta_o is None or overlap_tend_o is None:
                continue
            lons_da = drifter_time_asn(ds_d, var_str="longitude")
            lats_da = drifter_time_asn(ds_d, var_str="latitude")
            lons_da2 = lons_da.rolling(time=3).mean()
            lats_da2 = lats_da.rolling(time=3).mean()
            lons_da3 = lons_da2.ffill(dim="time").bfill(dim="time")
            lats_da3 = lats_da2.ffill(dim="time").bfill(dim="time")

            lons_drift = lons_da3.values
            lats_drift = lats_da3.values
            if is_within_region_any(
                    lons_drift,
                    lats_drift,
                    lon_inter_box_min,
                    lon_inter_box_max,
                    lat_inter_box_min,
                    lat_inter_box_max,
            ):
                close_ones.append(drifter_id)
                nidx = closest_index(lons_drift, lats_drift, lon1, lat1)
                close_drift_lon = lons_drift[nidx]
                close_drift_lat = lats_drift[nidx]
                fig = plt.figure(figsize=(12, 10), layout="constrained")
                ax2 = fig.add_subplot(1, 1, 1, projection=ccrs.PlateCarree())
                rose.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
                    *TRACKS_REG[2:])).plot(ax=ax2,
                                           add_colorbar=False,
                                           add_labels=False,
                                           cmap=cmap1)
                decorate_axis(ax2, "", *TRACKS_REG, step=5)
                ax2.grid()
                ax2.plot(
                    lon1,
                    lat1,
                    c="r",
                    marker="o",
                    markersize=4,
                    markerfacecolor="red",
                )
                ax2.scatter(
                    lons_track_self,
                    lats_track_self,
                    marker=".",
                    color="c",
                    s=4,
                )
                ax2.scatter(
                    lons_track_other,
                    lats_track_other,
                    marker=".",
                    color="c",
```

```
                    s=4,
                )
                ax2.add_geometries([box],
                                   ccrs.PlateCarree(),
                                   edgecolor="g",
                                   facecolor="none",
                                   linewidth=2)
                lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                                   lon_coast_self, lat_coast_self, d)
                if is_within_region(lonm, latm, *TRACKS_REG):
                    plt.text(
                    lonm,
                    latm,
                    s=track_number_self,
                    fontsize=14,
                    rotation=angle_self,
                    color="w",
                    )
                lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                                   lon_coast_other, lat_coast_other, d)
                if is_within_region(lonm, latm, *TRACKS_REG):
                    plt.text(
                        lonm,
                        latm,
                        s=track_number_other,
                        fontsize=14,
                        rotation=angle_other,
                        color="w",
                    )
                ax2.scatter(
                    lons_drift,
                    lats_drift,
                    marker=".",
                    color="y",
                    s=4,
                )
                ax2.plot(
                    close_drift_lon,
                    close_drift_lat,
                    marker="x",
                    markersize=8,
                    color="k",
                )
                plt.savefig(f"close_drifters/close_drifters_at_intersection_point_{drifter_id}.png")
                plt.show()
                plt.close("all")
                found = True
    close_drifters_column.append(close_ones)
df_close["close_drifters_column"] = close_drifters_column
df_close.to_csv(
    f"close_drifters_at_intersection_point.csv"
)
dse.close()

```

```python
import os

import numpy as np
import xarray as xr
from tools_xtrackm import *
import sys


def is_within_region_any(lons,
                         lats,
                         lon_min,
                         lon_max,
                         lat_min,
                         lat_max,
                         include_boundary=True):
    """
    Return True if any (lon, lat) point falls inside the rectangular region.
    include_boundary=True counts points on the edges/corners as inside.
    """
    lons = np.asarray(lons, dtype=float)
    lats = np.asarray(lats, dtype=float)
    if lons.shape != lats.shape:
        raise ValueError("lons and lats must have the same shape")
    if include_boundary:
        lons_in = (lons >= lon_min) & (lons <= lon_max)
        lats_in = (lats >= lat_min) & (lats <= lat_max)
    else:
        lons_in = (lons > lon_min) & (lons < lon_max)
        lats_in = (lats > lat_min) & (lats < lat_max)
    inside_mask = lons_in & lats_in
    hit = np.any(inside_mask)
    if hit:
        print("At least one point is inside the region.")
    else:
        print("No points are inside the region.")
    return bool(hit)


chunk_time_dict = {
    "1_5000": (
        np.datetime64("1985-10-30T00:00:00.000000000"),
        np.datetime64("1999-11-01T00:00:00.000000000"),
    ),
    "5001_10000": (
        np.datetime64("1997-03-20T00:00:00.000000000"),
        np.datetime64("2006-09-16T00:00:00.000000000"),
    ),
    "10001_15000": (
        np.datetime64("2006-05-24T00:00:00.000000000"),
        np.datetime64("2011-01-04T00:00:00.000000000"),
    ),
    "15001_current": (
        np.datetime64("2010-10-30T00:00:00.000000000"),
        np.datetime64("2024-12-31T00:00:00.000000000"),
    ),
}

d = 1.5
data_loc = f"/home/srinivasu/allData/drifter1/"
dist_threshold = 0.5  # in degrees
# chunks = ["5001_10000", "10001_15000"]
# take satellite from as first argument
sat_here = sys.argv[1]
df_all = pd.read_csv(f"tracks_intersections_{sat_here}_1.csv")
df_close = df_all.copy()

close_drifters_column = []

found = False
for i, r in df_all.iterrows():
    close_ones = []
    sat1 = r["sat"]
    # if sat1 != "S3A":
    #     continue
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    track_number_self = str(r["track_self"])
    track_number_other = str(r["track_other"])
    lons_inter1 = r["lons_inter"]
    lats_inter1 = r["lats_inter"]
    lon1 = lons_inter1
    lat1 = lats_inter1
    if abs(lat1) < 2:
        close_drifters_column.append([])
        continue
    print(
        sat1,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
    )
    lon_inter_box_min = lon1 - dist_threshold
    lon_inter_box_max = lon1 + dist_threshold
    lat_inter_box_min = lat1 - dist_threshold
    lat_inter_box_max = lat1 + dist_threshold
    for chunk in chunks:
        chunk_tsta_o1, chunk_tend_o1 = chunk_time_dict[chunk]
        chunk_tsta_o, chunk_tend_o = n64todatetime1(
            chunk_tsta_o1), n64todatetime1(chunk_tend_o1)
        chunk_overlap_tsta_o, chunk_overlap_tend_o = overlap_dates(
            track_tsta_o, track_tend_o, chunk_tsta_o, chunk_tend_o)
        if chunk_overlap_tsta_o is None or chunk_overlap_tend_o is None:
            continue
        for fd in sorted(
                glob.glob(
                    f"{data_loc}/netcdf_{chunk}/track_reg/drifter_6h_*.nc")):
            print(fd)
            basename1 = os.path.basename(fd)
            ds_d = xr.open_dataset(fd, drop_variables=["WMO"])

            byte_id = ds_d.ID.values[0]
            str_id = byte_id.decode("utf-8").strip()
            drifter_id = str_id

            drift_tsta_o1, drift_tend_o1 = (
                ds_d.start_date.values[0],
                ds_d.end_date.values[0],
```

```
            )
            drift_tsta_o, drift_tend_o = n64todatetime1(
                drift_tsta_o1), n64todatetime1(drift_tend_o1)
            overlap_tsta_o, overlap_tend_o = overlap_dates(
                track_tsta_o, track_tend_o, drift_tsta_o, drift_tend_o)
            print(f"overlap period {overlap_tsta_o} {overlap_tend_o}")
            if overlap_tsta_o is None or overlap_tend_o is None:
                continue
            lons_drift = ds_d.longitude.values
            lats_drift = ds_d.latitude.values
            if is_within_region_any(
                    lons_drift,
                    lats_drift,
                    lon_inter_box_min,
                    lon_inter_box_max,
                    lat_inter_box_min,
                    lat_inter_box_max,
            ):
                close_ones.append(drifter_id)
                found = True
    close_drifters_column.append(close_ones)
df_close["close_drifters_column"] = close_drifters_column
df_close.to_csv(f"close_drifters_at_intersection_point_{sat_here}.csv")

```

```python
import os

import numpy as np
import xarray as xr
from tools_xtrackm import *


def closest_index(lons_drift, lats_drift, lon1, lat1):
    dist = np.sqrt((lons_drift - lon1)**2 + (lats_drift - lat1)**2)
    return np.argmin(dist)


def is_within_region_any(lons,
                         lats,
                         lon_min,
                         lon_max,
                         lat_min,
                         lat_max,
                         include_boundary=True):
    """
    Return True if any (lon, lat) point falls inside the rectangular region.
    include_boundary=True counts points on the edges/corners as inside.
    """
    lons = np.asarray(lons, dtype=float)
    lats = np.asarray(lats, dtype=float)
    if lons.shape != lats.shape:
        raise ValueError("lons and lats must have the same shape")
    if include_boundary:
        lons_in = (lons >= lon_min) & (lons <= lon_max)
        lats_in = (lats >= lat_min) & (lats <= lat_max)
    else:
        lons_in = (lons > lon_min) & (lons < lon_max)
        lats_in = (lats > lat_min) & (lats < lat_max)
    inside_mask = lons_in & lats_in
    hit = np.any(inside_mask)
    if hit:
        print("At least one point is inside the region.")
    else:
        print("No points are inside the region.")
    return bool(hit)


chunk_time_dict = {
    "1_5000": (
        np.datetime64("1985-10-30T00:00:00.000000000"),
        np.datetime64("1999-11-01T00:00:00.000000000"),
    ),
    "5001_10000": (
        np.datetime64("1997-03-20T00:00:00.000000000"),
        np.datetime64("2006-09-16T00:00:00.000000000"),
    ),
    "10001_15000": (
        np.datetime64("2006-05-24T00:00:00.000000000"),
        np.datetime64("2011-01-04T00:00:00.000000000"),
    ),
    "15001_current": (
        np.datetime64("2010-10-30T00:00:00.000000000"),
        np.datetime64("2024-12-31T00:00:00.000000000"),
    ),
}

d = 1.5
data_loc = f"/home/srinivasu/allData/drifter1/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
rose = dse.ROSE
cmap1 = "Greys_r"
dist_threshold = 0.5  # in degrees
chunks = ["5001_10000"]
df_all = pd.read_csv(f"tracks_intersections_TP+J1+J2+J3+S6A_test.csv")
df_close = df_all.copy()

close_drifters_column = []

found = False
for i, r in df_all.iterrows():
    close_ones = []
    sat1 = r["sat"]
    # if sat1 != "S3A":
    #     continue
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    track_number_self = str(r["track_self"])
    track_number_other = str(r["track_other"])
    lons_inter1 = r["lons_inter"]
    lats_inter1 = r["lats_inter"]
    x_from_coast_self1 = r["x_from_coast_self"]
    x_from_coast_other1 = r["x_from_coast_other"]
    angle_acute = r["angle_acute"]
    angle_obtuse = r["angle_obtuse"]
    lon1 = lons_inter1
    lat1 = lats_inter1
    if abs(lat1) < 2:
        close_drifters_column.append([])
        continue
    print(
        sat1,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    lon_inter_box_min = lon1 - dist_threshold
    lon_inter_box_max = lon1 + dist_threshold
    lat_inter_box_min = lat1 - dist_threshold
    lat_inter_box_max = lat1 + dist_threshold
    for chunk in chunks:
        chunk_tsta_o1, chunk_tend_o1 = chunk_time_dict[chunk]
        chunk_tsta_o, chunk_tend_o = n64todatetime1(
            chunk_tsta_o1), n64todatetime1(chunk_tend_o1)
        chunk_overlap_tsta_o, chunk_overlap_tend_o = overlap_dates(
            track_tsta_o, track_tend_o, chunk_tsta_o, chunk_tend_o)
        if chunk_overlap_tsta_o is None or chunk_overlap_tend_o is None:
            continue
        for fd in sorted(
```

```
                glob.glob(
                    f"{data_loc}/netcdf_{chunk}/track_reg/drifter_6h_*.nc")):
            print(fd)
            basename1 = os.path.basename(fd)
            ds_d = xr.open_dataset(fd, drop_variables=["WMO"])

            byte_id = ds_d.ID.values[0]
            str_id = byte_id.decode("utf-8").strip()
            drifter_id = str_id

            drift_tsta_o1, drift_tend_o1 = (
                ds_d.start_date.values[0],
                ds_d.end_date.values[0],
            )
            drift_tsta_o, drift_tend_o = n64todatetime1(
                drift_tsta_o1), n64todatetime1(drift_tend_o1)
            overlap_tsta_o, overlap_tend_o = overlap_dates(
                track_tsta_o, track_tend_o, drift_tsta_o, drift_tend_o)
            print(f"overlap period {overlap_tsta_o} {overlap_tend_o}")
            if overlap_tsta_o is None or overlap_tend_o is None:
                continue
            lons_da = drifter_time_asn(ds_d, var_str="longitude")
            lats_da = drifter_time_asn(ds_d, var_str="latitude")
            lons_da2 = lons_da.rolling(time=3).mean()
            lats_da2 = lats_da.rolling(time=3).mean()
            lons_da3 = lons_da2.ffill(dim="time").bfill(dim="time")
            lats_da3 = lats_da2.ffill(dim="time").bfill(dim="time")

            lons_drift = lons_da3.values
            lats_drift = lats_da3.values
            if is_within_region_any(
                    lons_drift,
                    lats_drift,
                    lon_inter_box_min,
                    lon_inter_box_max,
                    lat_inter_box_min,
                    lat_inter_box_max,
            ):
                close_ones.append(drifter_id)
                nidx = closest_index(lons_drift, lats_drift, lon1, lat1)
                close_drift_lon = lons_drift[nidx]
                close_drift_lat = lats_drift[nidx]
                found = True
    close_drifters_column.append(close_ones)
df_close["close_drifters_column"] = close_drifters_column
df_close.to_csv(f"close_drifters_at_intersection_point.csv")
dse.close()
```

```python
import os

import xarray as xr
from rich.console import Console
from tools_xtrackm import *

console = Console()

dist_threshold = 2
computed_loc = f"/home/srinivasu/slnew/xtrackm/computed/"
data_loc = f"/home/srinivasu/allData/drifter1/"


def is_within_region_all(lons_drifter, lats_drifter, lon_min, lon_max, lat_min,
                         lat_max):
    lons_within_range = np.logical_and(lons_drifter >= lon_min, lons_drifter
                                       <= lon_max)
    # Check if all latitudes are within the range
    lats_within_range = np.logical_and(lats_drifter >= lat_min, lats_drifter
                                       <= lat_max)
    # Combine both conditions
    all_within_region = np.logical_and(lons_within_range, lats_within_range)
    # Check if all points are within the region
    if np.all(all_within_region):
        print("All points are within the region.")
        return True
    else:
        # print("Not all points are within the region.")
        return False


def is_within_region_any(lons,
                         lats,
                         lon_min,
                         lon_max,
                         lat_min,
                         lat_max,
                         include_boundary=True):
    """
    Return True if any (lon, lat) point falls inside the rectangular region.
    include_boundary=True counts points on the edges/corners as inside.
    """
    lons = np.asarray(lons, dtype=float)
    lats = np.asarray(lats, dtype=float)
    if lons.shape != lats.shape:
        raise ValueError("lons and lats must have the same shape")
    if include_boundary:
        lons_in = (lons >= lon_min) & (lons <= lon_max)
        lats_in = (lats >= lat_min) & (lats <= lat_max)
    else:
        lons_in = (lons > lon_min) & (lons < lon_max)
        lats_in = (lats > lat_min) & (lats < lat_max)
    inside_mask = lons_in & lats_in
    hit = np.any(inside_mask)
    if hit:
        print("At least one point is inside the region.")
    else:
        print("No points are inside the region.")
    return bool(hit)


sat, track_number_self, track_number_other, lons_inter, lats_inter = (
    "S3A",
    "8",
    "79",
    94.14047103085358,
    8.36326782978126,
)
# a square polygon with lons_inter, lats_inter as the center
box = create_box_patch(lons_inter, lats_inter, dist_threshold)
lon_max, lat_max = lons_inter + dist_threshold, lats_inter + dist_threshold
lat_min, lon_min = lats_inter - dist_threshold, lons_inter - dist_threshold
track_tsta_o, track_tend_o = get_time_limits_o(sat)

chunks = ["1_5000", "5001_10000", "10001_15000", "15001_current"]
found = False
for chunk in chunks:
    for fd in sorted(
            glob.glob(f"{data_loc}/netcdf_{chunk}/"
                      f"track_reg/drifter_6h_*.nc")):
        # print(fd)
        basename1 = os.path.basename(fd)
        ds_d = xr.open_dataset(fd, drop_variables=["WMO"])
        byte_id = ds_d.ID.values[0]
        str_id = byte_id.decode("utf-8")
        drifter_id = str_id
        drift_tsta_o1, drift_tend_o1 = (
            ds_d.start_date.values[0],
            ds_d.end_date.values[0],
        )
        drift_tsta_o, drift_tend_o = n64todatetime(
            drift_tsta_o1), n64todatetime(drift_tend_o1)
        overlap_tsta_o, overlap_tend_o = overlap_dates(track_tsta_o,
                                                       track_tend_o,
                                                       drift_tsta_o,
                                                       drift_tend_o)
        times_drift = ds_d.time.isel(traj=0).values
        lons_da = drifter_time_asn(ds_d, var_str="longitude")
        lats_da = drifter_time_asn(ds_d, var_str="latitude")
        lons_da2 = lons_da.rolling(time=3).mean()
        lats_da2 = lats_da.rolling(time=3).mean()
        lons_da3 = lons_da2.ffill(dim="time").bfill(dim="time")
        lats_da3 = lats_da2.ffill(dim="time").bfill(dim="time")

        lons_drift = lons_da3.values
        lats_drift = lats_da3.values

        lon_start = ds_d.start_lon.isel(traj=0).values
        lon_end = ds_d.end_lon.isel(traj=0).values
        lat_start = ds_d.start_lat.isel(traj=0).values
        lat_end = ds_d.end_lat.isel(traj=0).values
        if overlap_tsta_o is None or overlap_tend_o is None:
            # console.print(f"{overlap_tsta_o}, {overlap_tend_o}", style="red")
            continue
        else:
            print(f"{overlap_tsta_o}, {overlap_tend_o}")
        if is_within_region_any(lons_drift, lats_drift, lon_min, lon_max,
```

drifter/c.is_drifter_within_any.py

```
                                lat_min, lat_max):
            print(f"{fd}")
            print(f"Drifter {drifter_id} is within the region.")
            found = True
            break
    if found:
        break

```

```python
import ast
import os
import sys

import numpy as np
import xarray as xr
from geopy import distance

# import Polygon
from tools_xtrackm import *


def convert_to_list(x):
    try:
        return ast.literal_eval(x) if isinstance(x, str) else x
    except (ValueError, SyntaxError):
        return x  # Return as-is if conversion fails


def closest_index(lons_drift, lats_drift, lon1, lat1):
    dist = [
        distance.distance((y, x), (lat1, lon1)).m
        for x, y in zip(lons_drift, lats_drift)
    ]
    return np.argmin(dist)


def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    u = -1 * u
    return u, v


data_loc = f"/home/srinivasu/allData/drifter1/"
sat_here = sys.argv[1]
df_all = pd.read_csv(f"close_drifters_at_intersection_point_{sat_here}.csv")
df_all["close_drifters_column"] = df_all["close_drifters_column"].apply(
    convert_to_list)
dist_limit = 15  # km
gu_ats = []
ve_ats = []
df_out = pd.DataFrame()

for i, r in df_all.iterrows():
    # if i > 10:
    #     break
    sat1 = r["sat"]
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    print(type(track_tsta_o))
    # break
    track_number_self = str(r["track_self"])
    track_number_other = str(r["track_other"])
    lons_inter1 = r["lons_inter"]
    lats_inter1 = r["lats_inter"]
    x_from_coast_self1 = r["x_from_coast_self"]
    x_from_coast_other1 = r["x_from_coast_other"]
    angle_acute = r["angle_acute"]
    angle_obtuse = r["angle_obtuse"]
    lon1 = lons_inter1
    lat1 = lats_inter1
    close_ones = r["close_drifters_column"]
    print(close_ones)
    # break
    if abs(lat1) < 2:
        continue
    print(
        sat1,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
```

```
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_self}_loess_0.2.nc"
    f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_other}_loess_0.2.nc"
    # opened with decode_times=False
    ds_self = xr.open_dataset(f_self, engine="h5netcdf", decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other - lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)
    a2 = math.atan2(
        lat_coast_other - lat_equat_other,
        lon_coast_other - lon_equat_other,
    )
    print(f"azimuths {a1:.2f} {a2:.2f}")
    gu, gv = geostrophic_components_from_a(gc_self_at, gc_other_at, a1, a2)
    print(f"{angle_self} {angle_other} --------------------")
    print(type(gu.time.values[0]), type(gv.time.values[0]))
    # sys.exit(0)
    for id_drifter in close_ones:
        print(id_drifter)
        fd = f"{data_loc}/netcdf_all/track_reg/drifter_6h_{id_drifter}.nc"
        print(fd)
        basename1 = os.path.basename(fd)
        # opened without decode_times=False
        ds_d = xr.open_dataset(fd, drop_variables=["WMO"])

        byte_id = ds_d.ID.values[0]
        str_id = byte_id.decode("utf-8").strip()
        drifter_id = str_id

        drift_tsta_o1, drift_tend_o1 = (
            ds_d.start_date.values[0],
            ds_d.end_date.values[0],
        )
        drift_tsta_o, drift_tend_o = n64todatetime1(
            drift_tsta_o1), n64todatetime1(drift_tend_o1)
        # drift_tsta_o, drift_tend_o = drift_tsta_o1, drift_tend_o1
        print(type(track_tsta_o), type(track_tend_o))
        overlap_tsta_o, overlap_tend_o = overlap_dates(track_tsta_o,
                                                       track_tend_o,
                                                       drift_tsta_o,
                                                       drift_tend_o)
        print(f"satellite period {track_tsta_o} {track_tend_o}")
        print(f"drifter period {drift_tsta_o} {drift_tend_o}")
        print(f"overlap period {overlap_tsta_o} {overlap_tend_o}")
        if overlap_tsta_o is None or overlap_tend_o is None:
            continue
        ve_da = drifter_time_asn(ds_d, var_str="ve")
        vn_da = drifter_time_asn(ds_d, var_str="vn")
        lons_da = drifter_time_asn(ds_d, var_str="longitude")
        lats_da = drifter_time_asn(ds_d, var_str="latitude")
        lons_da2 = lons_da.rolling(time=3).mean()
        lats_da2 = lats_da.rolling(time=3).mean()
        lons_da3 = lons_da2.ffill(dim="time").bfill(dim="time")
        lats_da3 = lats_da2.ffill(dim="time").bfill(dim="time")

        lons_drift = lons_da3.values
        lats_drift = lats_da3.values
        driter_times = ve_da.time.values
        print(type(driter_times[0]))
        nidx = closest_index(lons_drift, lats_drift, lon1, lat1)
        match_time = driter_times[nidx]
        gu_at = gu.interp(time=match_time).item()
        ve_at = ve_da.isel(time=nidx).item()
        close_drift_lon = lons_drift[nidx]
        close_drift_lat = lats_drift[nidx]
        close_dist = distance.distance((lat1, lon1),
                                       (close_drift_lat, close_drift_lon)).km
        if close_dist < dist_limit:
            gu_ats.append(gu_at)
            ve_ats.append(ve_at)

df_out["gu"] = gu_ats
df_out["ve"] = ve_ats
df_out.to_csv(f"track_guvVSnearby_drifters_at_intersection_points_{sat1}.csv")

```

```python
import ast
import os
import sys

import numpy as np
import xarray as xr
from geopy import distance

# import Polygon
from tools_xtrackm import *

# Three changes
# Index based selection
# local azimuths
# smoothing of ve and gc


def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    u = -1 * u
    return u, v


def index_at_lat(ds, lat1):
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    i1 = np.abs(lats_track_rev - lat1).argmin()
    return i1


def convert_to_list(x):
    try:
        return ast.literal_eval(x) if isinstance(x, str) else x
    except (ValueError, SyntaxError):
        return x  # Return as-is if conversion fails


def closest_index(lons_drift, lats_drift, lon1, lat1):
    dist = [
        distance.distance((y, x), (lat1, lon1)).m
        for x, y in zip(lons_drift, lats_drift)
    ]
    return np.argmin(dist)


sat_here = sys.argv[1]
dist_limit = 15  # km
idx_delta = 5
tolerance = 0.05
smoothing = "0.2"
gu_ats = []
ve_ats = []
close_dists = []
lons_inters = []
lats_inters = []
df_out = pd.DataFrame()
data_loc = f"/home/srinivasu/allData/drifter1/"
df_all = pd.read_csv(f"close_drifters_at_intersection_point_{sat_here}.csv")
df_all["close_drifters_column"] = df_all["close_drifters_column"].apply(
    convert_to_list)

for i, r in df_all.iterrows():
    sat1 = r["sat"]
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    # print(type(track_tsta_o))
    track_number_self = str(r["track_self"])
    track_number_other = str(r["track_other"])
    lons_inter1 = r["lons_inter"]
    lats_inter1 = r["lats_inter"]
    x_from_coast_self1 = r["x_from_coast_self"]
    x_from_coast_other1 = r["x_from_coast_other"]
    angle_acute = r["angle_acute"]
    angle_obtuse = r["angle_obtuse"]
    lon1 = lons_inter1
    lat1 = lats_inter1
```

```
    close_ones = r["close_drifters_column"]
    if abs(lat1) < 2:
        continue
    # print(
    #     sat1,
    #     track_number_self,
    #     track_number_other,
    #     lons_inter1,
    #     lats_inter1,
    #     x_from_coast_self1,
    #     x_from_coast_other1,
    #     angle_acute,
    #     angle_obtuse,
    # )
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_{smoothing}a/track_sla_along_{sat1}_{track_number_self}_loess_{smoothing}.nc"
    f_other_smooth = f"../computed/sla_loess_{smoothing}a/track_sla_along_{sat1}_{track_number_other}_loess_{smoothing}.nc"
    # opened with decode_times=False
    ds_self = xr.open_dataset(f_self, engine="h5netcdf", decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lons_track_self_rev = ds_self.lon.values[::-1]
    lats_track_self_rev = ds_self.lat.values[::-1]
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lons_track_other_rev = ds_other.lon.values[::-1]
    lats_track_other_rev = ds_other.lat.values[::-1]
    idx_self = index_at_lat(ds_self, lat1)
    idx_other = index_at_lat(ds_other, lat1)
    lat1_test = lats_track_self_rev[idx_self]
    lon1_test = lons_track_self_rev[idx_self]
    lat2_test = lats_track_other_rev[idx_other]
    lon2_test = lons_track_other_rev[idx_other]
    print(f"idx_self {idx_self}, idx_other {idx_other}")
    print(f"lats_self {lat1_test}, lats_other {lat2_test}")
    print(f"lons_self {lon1_test}, lons_other {lon2_test}")
    try:
        assert math.isclose(lat1_test, lat2_test, abs_tol=tolerance), "intersection lats not close"
    except AssertionError:
        continue
    try:
        assert math.isclose(lon1_test, lon2_test, abs_tol=tolerance), "intersection lons not close"
    except AssertionError:
        continue
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.isel(x=idx_self)
    gc_other_at = gc_other.isel(x=idx_other)
    # gc_self_at1 = gc_self.interp(x=x_from_coast_self1)
    # gc_other_at1 = gc_other.interp(x=x_from_coast_other1)
    # gc_self_at.plot(label="self isel idx")
    # gc_self_at1.plot(label="self sel x")
    # plt.legend()
    # plt.show()
    # sys.exit(0)
    if idx_self-2 < 0 or idx_self+2 > len(lons_track_self_rev) or idx_other-2 < 0 or idx_other+2 > len(lons_track_other_rev):
        continue
    a1 = math.atan2(
        lats_track_self_rev[idx_self-2] - lats_track_self_rev[idx_self+2],
        lons_track_self_rev[idx_self-2] - lons_track_self_rev[idx_self+2],
    )
    a2 = math.atan2( lats_track_other_rev[idx_other-2] - lats_track_other_rev[idx_other+2], lons_track_other_rev[idx_other-2] - lons_track_other_rev[idx_other+2],
    )
    print(f"azimuths {a1:.2f} {a2:.2f}")
    for id_drifter in close_ones:
        # print(id_drifter)
        fd = f"{data_loc}/netcdf_all/track_reg/drifter_6h_{id_drifter}.nc"
        # print(fd)
        basename1 = os.path.basename(fd)
        # opened without decode_times=False
        ds_d = xr.open_dataset(fd, drop_variables=["WMO"])

        byte_id = ds_d.ID.values[0]
        str_id = byte_id.decode("utf-8").strip()
        drifter_id = str_id

        drift_tsta_o1, drift_tend_o1 = (
            ds_d.start_date.values[0],
            ds_d.end_date.values[0],
        )
        drift_tsta_o, drift_tend_o = n64todatetime1(
            drift_tsta_o1), n64todatetime1(drift_tend_o1)
        # drift_tsta_o, drift_tend_o = drift_tsta_o1, drift_tend_o1
        print(type(track_tsta_o), type(track_tend_o))
        overlap_tsta_o, overlap_tend_o = overlap_dates(track_tsta_o,
                                                       track_tend_o,
                                                       drift_tsta_o,
                                                       drift_tend_o)
        print(f"satellite period {track_tsta_o} {track_tend_o}")
        print(f"drifter period {drift_tsta_o} {drift_tend_o}")
        print(f"overlap period {overlap_tsta_o} {overlap_tend_o}")
        if overlap_tsta_o is None or overlap_tend_o is None:
            continue
        ve_da = drifter_time_asn(ds_d, var_str="ve")
        vn_da = drifter_time_asn(ds_d, var_str="vn")
        lons_da = drifter_time_asn(ds_d, var_str="longitude")
        lats_da = drifter_time_asn(ds_d, var_str="latitude")
        lons_da2 = lons_da.rolling(time=3).mean()
        lats_da2 = lats_da.rolling(time=3).mean()
        lons_da3 = lons_da2.ffill(dim="time").bfill(dim="time")
        lats_da3 = lats_da2.ffill(dim="time").bfill(dim="time")
        ve_da = ve_da.rolling(time=3).mean()
        vn_da = vn_da.rolling(time=3).mean()

        lons_drift = lons_da3.values
        lats_drift = lats_da3.values
        driter_times = ve_da.time.values
        # print(type(driter_times[0]))
        nidx = closest_index(lons_drift, lats_drift, lon1, lat1)
```

```
        match_time = driter_times[nidx]
        # ve_at = ve_da.isel(time=slice(nidx-1, nidx+1)).mean(dim="time").item()
        ve_at = ve_da.isel(time=nidx)
        # try:
        #     gc_self_at1 = gc_self_at.interp(time=match_time)
        # except:
        #     continue
        gc_self_at1 = gc_self_at.interp(time=match_time)
        gc_other_at1 = gc_other_at.interp(time=match_time)
        gu_at, gv_at = geostrophic_components_from_a(gc_self_at1, gc_other_at1, a1, a2)
        print(type(gu_at))
        gu_at = gu_at.item()
        ve_at = ve_at.item()
        close_drift_lon = lons_drift[nidx]
        close_drift_lat = lats_drift[nidx]
        close_dist = distance.distance((lat1, lon1),
                                       (close_drift_lat, close_drift_lon)).km
        if close_dist < dist_limit:
            gu_ats.append(gu_at)
            ve_ats.append(ve_at)
            lons_inters.append(lon1)
            lats_inters.append(lat1)
            close_dists.append(close_dist)

df_out["gu"] = gu_ats
df_out["ve"] = ve_ats
df_out["close_dist"] = close_dists
df_out["lon_inters"] = lons_inters
df_out["lat_inters"] = lats_inters
df_out.to_csv(f"track_guvVSnearby_drifters_at_intersection_points_{sat1}_2.csv")

```

```python
import ast
import os

import numpy as np
import xarray as xr
from geopy import distance

# import Polygon
from tools_xtrackm import *

# Three changes
# Index based selection
# local azimuths
# smoothing of ve and gc


def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    u = -1 * u
    return u, v


def index_at_lat(ds, lat1):
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    i1 = np.abs(lats_track_rev - lat1).argmin()
    return i1


def convert_to_list(x):
    try:
        return ast.literal_eval(x) if isinstance(x, str) else x
    except (ValueError, SyntaxError):
        return x  # Return as-is if conversion fails


def closest_index(lons_drift, lats_drift, lon1, lat1):
    dist = [
        distance.distance((y, x), (lat1, lon1)).m
        for x, y in zip(lons_drift, lats_drift)
    ]
    return np.argmin(dist)


# sat_here = sys.argv[1]
dist_limit = 15  # km
idx_delta = 5
tolerance = 0.05
smoothing = "0.2"
gu_ats = []
ve_ats = []
ek_u_ats = []
close_dists = []
lons_inters = []
lats_inters = []
df_out = pd.DataFrame()
data_loc = f"/home/srinivasu/allData/drifter1/"
ekman_extracted_dir = "/home/srinivasu/xtrackm/ekman/ekman_at_intersections/"
sat_here = "TP+J1+J2+J3+S6A"
df_all = pd.read_csv(f"close_drifters_at_intersection_point_{sat_here}.csv")
df_all["close_drifters_column"] = df_all["close_drifters_column"].apply(
    convert_to_list)

for i, r in df_all.iterrows():
    sat1 = r["sat"]
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    # print(type(track_tsta_o))
    track_number_self = str(r["track_self"])
    track_number_other = str(r["track_other"])
    lons_inter1 = r["lons_inter"]
    lats_inter1 = r["lats_inter"]
    x_from_coast_self1 = r["x_from_coast_self"]
    x_from_coast_other1 = r["x_from_coast_other"]
    angle_acute = r["angle_acute"]
    angle_obtuse = r["angle_obtuse"]
```

```
    lon1 = lons_inter1
    lat1 = lats_inter1
    close_ones = r["close_drifters_column"]
    if abs(lat1) < 2:
        continue
    # print(
    #     sat1,
    #     track_number_self,
    #     track_number_other,
    #     lons_inter1,
    #     lats_inter1,
    #     x_from_coast_self1,
    #     x_from_coast_other1,
    #     angle_acute,
    #     angle_obtuse,
    # )
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_{smoothing}a/track_sla_along_{sat1}_{track_number_self}_loess_{smoothing}.nc"
    f_other_smooth = f"../computed/sla_loess_{smoothing}a/track_sla_along_{sat1}_{track_number_other}_loess_{smoothing}.nc"
    ek_fname = f"{ekman_extracted_dir}/ekman_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc"
    ds_ek = xr.open_dataset(ek_fname)
    ek_u = ds_ek.u
    ek_v = ds_ek.v
    # opened with decode_times=False
    ds_self = xr.open_dataset(f_self, engine="h5netcdf", decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lons_track_self_rev = ds_self.lon.values[::-1]
    lats_track_self_rev = ds_self.lat.values[::-1]
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lons_track_other_rev = ds_other.lon.values[::-1]
    lats_track_other_rev = ds_other.lat.values[::-1]
    idx_self = index_at_lat(ds_self, lat1)
    idx_other = index_at_lat(ds_other, lat1)
    lat1_test = lats_track_self_rev[idx_self]
    lon1_test = lons_track_self_rev[idx_self]
    lat2_test = lats_track_other_rev[idx_other]
    lon2_test = lons_track_other_rev[idx_other]
    print(f"idx_self {idx_self}, idx_other {idx_other}")
    print(f"lats_self {lat1_test}, lats_other {lat2_test}")
    print(f"lons_self {lon1_test}, lons_other {lon2_test}")
    try:
        assert math.isclose(lat1_test, lat2_test,
                            abs_tol=tolerance), "intersection lats not close"
    except AssertionError:
        continue
    try:
        assert math.isclose(lon1_test, lon2_test,
                            abs_tol=tolerance), "intersection lons not close"
    except AssertionError:
        continue
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.isel(x=idx_self)
    gc_other_at = gc_other.isel(x=idx_other)
    # gc_self_at1 = gc_self.interp(x=x_from_coast_self1)
    # gc_other_at1 = gc_other.interp(x=x_from_coast_other1)
    # gc_self_at.plot(label="self isel idx")
    # gc_self_at1.plot(label="self sel x")
    # plt.legend()
    # plt.show()
    # sys.exit(0)
    if (idx_self - 2 < 0 or idx_self + 2 > len(lons_track_self_rev)
            or idx_other - 2 < 0 or idx_other + 2 > len(lons_track_other_rev)):
        continue
    a1 = math.atan2(
        lats_track_self_rev[idx_self - 2] - lats_track_self_rev[idx_self + 2],
        lons_track_self_rev[idx_self - 2] - lons_track_self_rev[idx_self + 2],
    )
    a2 = math.atan2(
        lats_track_other_rev[idx_other - 2] -
        lats_track_other_rev[idx_other + 2],
        lons_track_other_rev[idx_other - 2] -
        lons_track_other_rev[idx_other + 2],
    )
    print(f"azimuths {a1:.2f} {a2:.2f}")
    for id_drifter in close_ones:
        # print(id_drifter)
        fd = f"{data_loc}/netcdf_all/track_reg/drifter_6h_{id_drifter}.nc"
        # print(fd)
        basename1 = os.path.basename(fd)
        # opened without decode_times=False
        ds_d = xr.open_dataset(fd, drop_variables=["WMO"])

        byte_id = ds_d.ID.values[0]
        str_id = byte_id.decode("utf-8").strip()
        drifter_id = str_id

        drift_tsta_o1, drift_tend_o1 = (
            ds_d.start_date.values[0],
            ds_d.end_date.values[0],
        )
        drift_tsta_o, drift_tend_o = n64todatetime1(
            drift_tsta_o1), n64todatetime1(drift_tend_o1)
        # drift_tsta_o, drift_tend_o = drift_tsta_o1, drift_tend_o1
        print(type(track_tsta_o), type(track_tend_o))
        overlap_tsta_o, overlap_tend_o = overlap_dates(track_tsta_o,
                                                       track_tend_o,
                                                       drift_tsta_o,
                                                       drift_tend_o)
        print(f"satellite period {track_tsta_o} {track_tend_o}")
        print(f"drifter period {drift_tsta_o} {drift_tend_o}")
        print(f"overlap period {overlap_tsta_o} {overlap_tend_o}")
        if overlap_tsta_o is None or overlap_tend_o is None:
            continue
        ve_da = drifter_time_asn(ds_d, var_str="ve")
        vn_da = drifter_time_asn(ds_d, var_str="vn")
        lons_da = drifter_time_asn(ds_d, var_str="longitude")
```

```
        lats_da = drifter_time_asn(ds_d, var_str="latitude")
        lons_da2 = lons_da.rolling(time=3).mean()
        lats_da2 = lats_da.rolling(time=3).mean()
        lons_da3 = lons_da2.ffill(dim="time").bfill(dim="time")
        lats_da3 = lats_da2.ffill(dim="time").bfill(dim="time")
        ve_da = ve_da.rolling(time=3).mean()
        vn_da = vn_da.rolling(time=3).mean()

        lons_drift = lons_da3.values
        lats_drift = lats_da3.values
        driter_times = ve_da.time.values
        # print(type(driter_times[0]))
        nidx = closest_index(lons_drift, lats_drift, lon1, lat1)
        match_time = driter_times[nidx]
        # ve_at = ve_da.isel(time=slice(nidx-1, nidx+1)).mean(dim="time").item()
        ve_at = ve_da.isel(time=nidx)
        # try:
        #     gc_self_at1 = gc_self_at.interp(time=match_time)
        # except:
        #     continue
        gc_self_at1 = gc_self_at.interp(time=match_time)
        gc_other_at1 = gc_other_at.interp(time=match_time)
        ek_u_at = ek_u.interp(time=match_time)
        gu_at, gv_at = geostrophic_components_from_a(gc_self_at1, gc_other_at1,
                                                     a1, a2)

        print(type(gu_at))
        gu_at = gu_at.item()
        ve_at = ve_at.item()
        ek_u_at = ek_u_at.item()
        close_drift_lon = lons_drift[nidx]
        close_drift_lat = lats_drift[nidx]
        close_dist = distance.distance((lat1, lon1),
                                       (close_drift_lat, close_drift_lon)).km
        if close_dist < dist_limit:
            gu_ats.append(gu_at)
            ve_ats.append(ve_at)
            ek_u_ats.append(ek_u_at)
            lons_inters.append(lon1)
            lats_inters.append(lat1)
            close_dists.append(close_dist)

df_out["gu"] = gu_ats
df_out["ve"] = ve_ats
df_out["ek"] = ek_u_ats
df_out["close_dist"] = close_dists
df_out["lon_inters"] = lons_inters
df_out["lat_inters"] = lats_inters
df_out.to_csv(
    f"track_guvVSnearby_drifters_at_intersection_points_{sat1}_w_ekman.csv")

```

```python
import os

import xarray as xr
from rich.console import Console
from shapely.geometry import LineString, Point
from tools_xtrackd import *

console = Console()

topo_dir = "/home/srinivasu/allData/topo/"
dsc = xr.open_dataset(f"{topo_dir}"
                      f"GMT_intermediate_coast_distance_01d_track_reg.nc")
print(dsc)
dist_to_coast = dsc.coast_dist


def get_closest_point_index(drifter_path: LineString,
                            intersection_point: Point) -> int:
    distances = [
        Point(p).distance(intersection_point) for p in drifter_path.coords
    ]
    nearest_index = np.argmin(distances)
    return nearest_index


def rotate_to_gc(u, v, angle_r):
    u1 = u * np.sin(angle_r) - v * np.cos(angle_r)
    return u1


chunk = "15001_current"
dist_threshold = 200
computed_loc = f"/home/srinivasu/slnew/xtrackd/computed/"
data_loc = f"/home/srinivasu/allData/drifter1/"

ds_ekman = xr.open_dataset(
    f"{computed_loc}/full_series_ekman_uv_ccmp3.0_cdo.nc"
)
ekman_u = ds_ekman.ekman_u
ekman_v = ds_ekman.ekman_v

count = 0
for sat in sats[:]:
    track_tsta_o, track_tend_o = get_time_limits_o(sat)
    ekman_u_cut = ekman_u.sel(time=slice(track_tsta_o, track_tend_o))
    ekman_v_cut = ekman_v.sel(time=slice(track_tsta_o, track_tend_o))
    df_list = []
    df_e_list = []
    for fd in sorted(
            glob.glob(f"{data_loc}/netcdf_*/"
                      f"track_reg/drifter_6h_*.nc")):
        basename1 = os.path.basename(fd)
        ds_d = xr.open_dataset(fd, drop_variables=["WMO"])
        byte_id = ds_d.ID.values[0]
        str_id = byte_id.decode("utf-8")
        drifter_id = str_id
        drift_tsta_o1, drift_tend_o1 = (
            ds_d.start_date.values[0],
            ds_d.end_date.values[0],
        )
        # drift_tsta_o, drift_tend_o = pd.to_datetime(drift_tsta_o1), pd.to_datetime(
        #     drift_tend_o1)
        drift_tsta_o, drift_tend_o = n64todatetime(drift_tsta_o1), n64todatetime(
            drift_tend_o1)
        overlap_tsta_o, overlap_tend_o = overlap_dates(track_tsta_o, track_tend_o,
                                                       drift_tsta_o, drift_tend_o)
        if overlap_tsta_o is None or overlap_tend_o is None:
            console.print(f"{overlap_tsta_o}, {overlap_tend_o}", style="red")
            continue
        # lons_drift = ds_d.longitude.isel(traj=0).values
        # lats_drift = ds_d.latitude.isel(traj=0).values
        times_drift = ds_d.time.isel(traj=0).values
        # times_drift_o = [pd.to_datetime(t) for t in times_drift]
        # times_drift_o1 = [n64todatetime(t) for t in times_drift]
        ve_da = drifter_time_asn(ds_d, var_str="ve")
        vn_da = drifter_time_asn(ds_d, var_str="vn")
        lons_da = drifter_time_asn(ds_d, var_str="longitude")
        lats_da = drifter_time_asn(ds_d, var_str="latitude")
        lons_da1 = lons_da.resample(time="1D").mean()
        lats_da1 = lats_da.resample(time="1D").mean()
        lons_da2 = lons_da1.rolling(time=3).mean()
        lats_da2 = lats_da1.rolling(time=3).mean()
        lons_da3 = lons_da2.ffill(dim="time").bfill(dim="time")
        lats_da3 = lats_da2.ffill(dim="time").bfill(dim="time")

        ve_da1 = ve_da.resample(time="1D").mean()
        vn_da1 = vn_da.resample(time="1D").mean()
        ve_da2 = ve_da1.rolling(time=3).mean()
        vn_da2 = vn_da1.rolling(time=3).mean()
        times_drift_o2 = ve_da2.time
        lons_drift = lons_da3.values
        lats_drift = lats_da3.values
        # times_drift_o = times_drift_o1[::4]
        times_drift_o = [n64todatetime(t) for t in times_drift_o2]

        # ve_drift = ds_d.ve.isel(traj=0).values
        # vn_drift = ds_d.vn.isel(traj=0).values
        # ve_drift = ve_da2.values
        # vn_drift = vn_da2.values

        # k = 0
        # for lon, lat, time in zip(lons_drift, lats_drift, times_drift):
        #     print(k, lon, lat, time)
        #     k = k + 1

        lon_start = ds_d.start_lon.isel(traj=0).values
        lon_end = ds_d.end_lon.isel(traj=0).values
        lat_start = ds_d.start_lat.isel(traj=0).values
        lat_end = ds_d.end_lat.isel(traj=0).values
        drifter_path = sg.LineString([
            (lon, lat) for lon, lat in zip(lons_drift, lats_drift)
        ])
        for ft in sorted(glob.glob(f"../data/ctoh.sla.ref.{sat}.nindian.*.nc")):
            basename_t = os.path.basename(ft)
            trackn = basename_t.split(".")[5]
            print(sat, drifter_id, trackn)
            ds_t = xr.open_dataset(ft, decode_times=False, engine="h5netcdf")
```

```
            track_number = ds_t.pass_number
            fs = (f"{computed_loc}/sla_loess_0.2a/"
                  f"track_sla_along_{sat}_{track_number}_loess_0.2.nc")
            ds_smooth = xr.open_dataset(fs, engine="h5netcdf")
            sla_smooth = ds_smooth.sla_smooth
            gc = compute_geostrophy_gc(ds_t, sla_smooth)
            lons_track = ds_t.lon.values
            lats_track = ds_t.lat.values
            lon_coast = lons_track[-1]
            lat_coast = lats_track[-1]
            lon_equat = lons_track[0]
            lat_equat = lats_track[0]
            angle_r2 = np.arctan2(lat_coast - lat_equat, lon_coast - lon_equat)
            ve_da_r = rotate_to_gc(ve_da2, vn_da2, angle_r2)
            ve_drift = ve_da_r.values
            # gc = gc.sel(time=slice(overlap_tsta, overlap_tend))
            track_path = sg.LineString([
                (lon, lat) for lon, lat in zip(lons_track, lats_track)
            ])
            if track_path.intersects(drifter_path):
                intersection1 = track_path.intersection(drifter_path)
                if intersection1.geom_type == "Point":
                    pointa = intersection1
                    ii = get_closest_point_index(drifter_path, pointa)
                    time11 = times_drift_o[ii]
                    # time11 = times_drift_o.isel(time=ii).values
                    console.print(
                        f"intersection point: {pointa}, closestindex = {ii}",
                        style="blue",
                    )
                    console.print(
                        f"lon_at_index = {lons_drift[ii]}, lat_at_index = "
                        f"{lats_drift[ii]}, time_at_index = {time11}",
                        style="blue",
                    )
                    is_between = overlap_tsta_o <= time11 <= overlap_tend_o
                    if is_between:
                        ve11 = ve_drift[ii]
                        jj = get_closest_point_index(track_path, pointa)
                        x1, y1 = lons_track[jj], lats_track[jj]
                        dist_to_coast_xy = dist_to_coast.sel(
                            lon=x1, lat=y1, method="nearest").item()
                        print(f"dist_to_coast = {dist_to_coast_xy} -------------------------")
                        if dist_to_coast_xy < dist_threshold:
                            x_from_coast = distance.distance(
                                (y1, x1), (lat_coast, lon_coast)).m
                            gc1 = gc.interp(x=x_from_coast)
                            gc11 = gc1.interp(time=time11).item()
                            ekman_at_u = ekman_u_cut.interp(lon=x1).interp(lat=y1)
                            ekman_at_v = ekman_v_cut.interp(lon=x1).interp(lat=y1)
                            ekman_at_u_r = rotate_to_gc(ekman_at_u, ekman_at_v, angle_r2)
                            ek11 = ekman_at_u_r.interp(time=time11).item()
                            gc_ek = gc11 + ek11
                            console.print(f"ve at {time11} = {ve11}",
                                          style="yellow")
                            console.print(f"gc at {time11} = {gc11}",
                                          style="yellow")
                            df = pd.DataFrame({
                                "ve": [ve11],
                                "gc": [gc11],
                                "time": [time11],
                                "lon": x1,
                                "lat": y1,
                            })
                            df_e = pd.DataFrame({
                                "ve": [ve11],
                                "gc": [gc_ek],
                                "time": [time11],
                                "lon": x1,
                                "lat": y1,
                            })
                            # df = pd.concat([df, new_row], ignore_index=True)
                            df_list.append(df)
                            df_e_list.append(df_e)
                            ii = ii + 1
                elif intersection1.geom_type == "MultiPoint":
                    for pointa in (intersection1.geoms
                                   ):  # Use .geoms to iterate over MultiPoint
                        ii = get_closest_point_index(drifter_path, pointa)
                        time11 = times_drift_o[ii]
                        console.print(
                            f"intersection point: {pointa}, closestindex = {ii}",
                            style="blue",
                        )
                        console.print(
                            f"lon_at_index = {lons_drift[ii]}, lat_at_index = "
                            f"{lats_drift[ii]}, time_at_index = {time11}",
                            style="blue",
                        )
                        is_between = overlap_tsta_o <= time11 <= overlap_tend_o
                        if is_between:
                            ve11 = ve_drift[ii]
                            jj = get_closest_point_index(track_path, pointa)
                            x1, y1 = lons_track[jj], lats_track[jj]
                            dist_to_coast_xy = dist_to_coast.sel(
                                lon=x1, lat=y1, method="nearest").item()
                            print(f"dist_to_coast = {dist_to_coast_xy} -------------------------")
                            if dist_to_coast_xy < dist_threshold:
                                x_from_coast = distance.distance(
                                    (y1, x1), (lat_coast, lon_coast)).m
                                gc1 = gc.interp(x=x_from_coast)
                                gc11 = gc1.interp(time=time11).item()
                                ekman_at_u = ekman_u_cut.interp(lon=x1).interp(lat=y1)
                                ekman_at_v = ekman_v_cut.interp(lon=x1).interp(lat=y1)
                                ekman_at_u_r = rotate_to_gc(ekman_at_u, ekman_at_v, angle_r2)
                                ek11 = ekman_at_u_r.interp(time=time11).item()
                                gc_ek = gc11 + ek11
                                console.print(f"ve at {time11} = {ve11}",
                                              style="yellow")
                                console.print(f"gc at {time11} = {gc11}",
                                              style="yellow")
                                df = pd.DataFrame({
                                    "ve": [ve11],
                                    "gc": [gc11],
                                    "time": [time11],
                                    "lon": x1,
                                    "lat": y1,
```

```
                                })
                                df_e = pd.DataFrame({
                                    "ve": [ve11],
                                    "gc": [gc_ek],
                                    "time": [time11],
                                    "lon": x1,
                                    "lat": y1,
                                })
                                    # df = pd.concat([df, new_row], ignore_index=True)
                                df_list.append(df)
                                df_e_list.append(df_e)
                                count = count + 1

    df_out = pd.concat(df_list, ignore_index=True)
    df_e_out = pd.concat(df_e_list, ignore_index=True)
    df_out.to_csv(f"ve_gc_all_{sat}.within.isobath.{dist_threshold}km.csv")
    df_e_out.to_csv(f"ve_gc_ek_all_{sat}.within.isobath.{dist_threshold}km.csv")

```

```python
import ast
import os
import sys

import numpy as np
import xarray as xr
from geopy import distance

# import Polygon
from tools_xtrackm import *

# Three changes
# Index based selection
# local azimuths
# smoothing of ve and gc


def index_at_lat(ds, lat1):
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    i1 = np.abs(lats_track_rev - lat1).argmin()
    return i1


def convert_to_list(x):
    try:
        return ast.literal_eval(x) if isinstance(x, str) else x
    except (ValueError, SyntaxError):
        return x  # Return as-is if conversion fails


def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    u = -1 * u
    return u, v


data_loc = f"/home/srinivasu/allData/drifter1/"
sat_here = sys.argv[1]
df_all = pd.read_csv(f"close_drifters_at_intersection_point_{sat_here}.csv")
df_all["close_drifters_column"] = df_all["close_drifters_column"].apply(
    convert_to_list)
dist_limit = 15  # km
gu_ats = []
ve_ats = []
df_out = pd.DataFrame()
idx_delta = 5
close_dists = []
tolerance = 0.05

for i, r in df_all.iterrows():
    sat1 = r["sat"]
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    # print(type(track_tsta_o))
    track_number_self = str(r["track_self"])
    track_number_other = str(r["track_other"])
    lons_inter1 = r["lons_inter"]
    lats_inter1 = r["lats_inter"]
    x_from_coast_self1 = r["x_from_coast_self"]
    x_from_coast_other1 = r["x_from_coast_other"]
    angle_acute = r["angle_acute"]
    angle_obtuse = r["angle_obtuse"]
    lon1 = lons_inter1
    lat1 = lats_inter1
    close_ones = r["close_drifters_column"]
    # print(close_ones)
    # break
    if abs(lat1) < 2:
        continue
    print("----------------------------------------")
    print(
        sat1,
        track_number_self,
        track_number_other,
        lons_inter1,
```

```
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_self}_loess_0.2.nc"
    f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_other}_loess_0.2.nc"
    # opened with decode_times=False
    ds_self = xr.open_dataset(f_self, engine="h5netcdf", decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lons_track_self_rev = ds_self.lon.values[::-1]
    lats_track_self_rev = ds_self.lat.values[::-1]
    # lons_track_self = sla_self.lon.values
    # lats_track_self = sla_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lat_coast_self - lat_equat_self) / (lon_coast_self - lon_equat_self)
    angle_self = np.rad2deg(np.arctan(slope_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lons_track_other_rev = ds_other.lon.values[::-1]
    lats_track_other_rev = ds_other.lat.values[::-1]
    # lons_track_other = sla_other.lon.values
    # lats_track_other = sla_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other - lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    idx_self = index_at_lat(ds_self, lat1)
    idx_other = index_at_lat(ds_other, lat1)
    lat1_test = lats_track_self_rev[idx_self]
    lon1_test = lons_track_self_rev[idx_self]
    lat2_test = lats_track_other_rev[idx_other]
    lon2_test = lons_track_other_rev[idx_other]
    print(f"idx_self {idx_self}, idx_other {idx_other}")
    print(f"lats_self {lat1_test}, lats_other {lat2_test}")
    assert math.isclose(lat1_test, lat2_test, abs_tol=tolerance), "intersection lats not close"
    print(f"lons_self {lon1_test}, lons_other {lon2_test}")
    assert math.isclose(lon1_test, lon2_test, abs_tol=tolerance), "intersection lons not close"
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.isel(x=slice(idx_self-1, idx_self+1)).mean(dim="x")
    gc_other_at = gc_other.isel(x=slice(idx_other-1, idx_other+1)).mean(dim="x")
    # sys.exit(0)
    a1 = math.atan2(
        lats_track_self_rev[idx_self-2] - lats_track_self_rev[idx_self+2],
        lons_track_self_rev[idx_self-2] - lons_track_self_rev[idx_self+2],
    )
    a2 = math.atan2(
        lats_track_other_rev[idx_other-2] - lats_track_other_rev[idx_other+2],
        lons_track_other_rev[idx_other-2] - lons_track_other_rev[idx_other+2],
    )
    print(f"azimuths {a1:.2f} {a2:.2f}")
    a1a = math.atan2(lat_coast_self - lat_equat_self,
                     lon_coast_self - lon_equat_self)
    a2a = math.atan2(
        lat_coast_other - lat_equat_other,
        lon_coast_other - lon_equat_other,
    )
    print(f"azimuths {a1a:.2f} {a2a:.2f}")
    if i > 10:
        break

```

```

dimensions:
    string_15 = 15 ;
    traj = 1 ;
    string_10 = 10 ;
    obs = 76 ;
variables:
    char ID(traj, string_15) ;
        ID:long_name = "Global Drifter Program Buoy ID" ;
        ID:cf_role = "trajectory_id" ;
        ID:history = "From buoydata_15001_dec23.dat" ;
    int rowsize(traj) ;
        rowsize:long_name = "number of obs for this trajectory" ;
        rowsize:sample_dimension = "obs" ;
        rowsize:history = "From buoydata_15001_dec23.dat" ;
    int WMO(traj) ;
        WMO:long_name = "World Meteorological Organization buoy identification number" ;
        WMO:missing_value = "-999999" ;
        WMO:history = "From dirall.dat" ;
        WMO:_FillValue = -999999 ;
    int expno(traj) ;
        expno:long_name = "Experiment number" ;
        expno:missing_value = -999999 ;
        expno:history = "From dirall.dat" ;
        expno:units = "count" ;
        expno:_FillValue = -999999 ;
    double deploy_date(traj) ;
        deploy_date:long_name = "Deployment date and time" ;
        deploy_date:units = "seconds since 1970-01-01 00:00:00 UTC" ;
        deploy_date:missing_value = -999999 ;
        deploy_date:history = "From deployment log deplog.dat" ;
        deploy_date:_FillValue = -999999 ;
    double deploy_lat(traj) ;
        deploy_lat:long_name = "Deployment latitude" ;
        deploy_lat:units = "degrees_north" ;
        deploy_lat:missing_value = -999999 ;
        deploy_lat:history = "From deployment log deplog.dat" ;
        deploy_lat:_FillValue = -999999 ;
    double deploy_lon(traj) ;
        deploy_lon:long_name = "Deployment longitude" ;
        deploy_lon:units = "degrees_east" ;
        deploy_lon:missing_value = -999999 ;
        deploy_lon:history = "From deployment log deplog.dat" ;
        deploy_lon:_FillValue = -999999 ;
    double start_date(traj) ;
        start_date:long_name = "First good date and time derived by DAC quality control" ;
        start_date:units = "seconds since 1970-01-01 00:00:00 UTC" ;
        start_date:missing_value = -999999 ;
        start_date:history = "From directory file dirall.dat" ;
        start_date:_FillValue = -999999 ;
    double start_lat(traj) ;
        start_lat:long_name = "First good latitude derived by DAC quality control" ;
        start_lat:units = "degrees_north" ;
        start_lat:missing_value = -999999 ;
        start_lat:history = "From directory file dirall.dat" ;
        start_lat:_FillValue = -999999 ;
    double start_lon(traj) ;
        start_lon:long_name = "First good longitude derived by DAC quality control" ;
        start_lon:units = "degrees_east" ;
        start_lon:missing_value = -999999 ;
        start_lon:history = "From directory file dirall.dat" ;
        start_lon:_FillValue = -999999 ;
    double end_date(traj) ;
        end_date:long_name = "Last good date and time derived by DAC quality control" ;
        end_date:units = "seconds since 1970-01-01 00:00:00 UTC" ;
        end_date:missing_value = -999999 ;
        end_date:history = "From directory file dirall.dat" ;
        end_date:_FillValue = -999999 ;
    double end_lat(traj) ;
        end_lat:long_name = "Last good latitude derived by DAC quality control" ;
        end_lat:units = "degrees_north" ;
        end_lat:missing_value = -999999 ;
        end_lat:history = "From directory file dirall.dat" ;
        end_lat:_FillValue = -999999 ;
    double end_lon(traj) ;
        end_lon:long_name = "Last good longitude derived by DAC quality control" ;
        end_lon:units = "degrees_east" ;
        end_lon:missing_value = -999999 ;
        end_lon:history = "From directory file dirall.dat" ;
        end_lon:_FillValue = -999999 ;
    double drogue_lost_date(traj) ;
        drogue_lost_date:long_name = "Date of drogue loss (_FillValue=drogue still attached; 0=drogue status uncertain from beginning)" ;
        drogue_lost_date:units = "seconds since 1970-01-01 00:00:00 UTC" ;
        drogue_lost_date:missing_value = -999999 ;
        drogue_lost_date:history = "From directory file dirall.dat" ;
        drogue_lost_date:_FillValue = -999999 ;
    int typedeath(traj) ;
        typedeath:long_name = "Type of death (0=buoy still alive, 1=buoy ran aground, 2=picked up by vessel, 3=stopped transmitting, 4=sporadic transmissions, 5=bad batteries, 6=inactive status)" ;
        typedeath:missing_value = -999999 ;
        typedeath:history = "From directory file dirall.dat" ;
        typedeath:_FillValue = -999999 ;
    char typebuoy(traj, string_10) ;
        typebuoy:long_name = "Buoy type (see https://www.aoml.noaa.gov/phod/dac/dirall.html)" ;
        typebuoy:history = "From directory file dirall.dat" ;
    double longitude(traj, obs) ;
        longitude:long_name = "Longitude" ;
        longitude:units = "degrees_east" ;
        longitude:axis = "X" ;
        longitude:valid_min = -180. ;
        longitude:valid_max = 180. ;
        longitude:data_min = 106.321 ;
        longitude:data_max = 106.64 ;
        longitude:standard_name = "longitude" ;
        longitude:missing_value = -999999. ;
        longitude:history = "From buoydata_15001_dec23.dat" ;
        longitude:_FillValue = -999999. ;
    double latitude(traj, obs) ;
        latitude:long_name = "Latitude" ;
        latitude:units = "degrees_north" ;
        latitude:axis = "Y" ;
        latitude:valid_min = -90. ;
        latitude:valid_max = 90. ;
        latitude:data_min = 8.016 ;
        latitude:data_max = 8.681 ;
        latitude:standard_name = "latitude" ;
        latitude:missing_value = -999999. ;
        latitude:history = "From buoydata_15001_dec23.dat" ;
```

```
        latitude:_FillValue = -999999. ;
    double lon360(traj, obs) ;
        lon360:long_name = "Longitude" ;
        lon360:units = "degrees_east" ;
        lon360:axis = "X" ;
        lon360:valid_min = 0. ;
        lon360:valid_max = 360. ;
        lon360:data_min = 106.321 ;
        lon360:data_max = 106.64 ;
        lon360:standard_name = "longitude" ;
        lon360:missing_value = -999999. ;
        lon360:history = "Computed from buoydata_15001_dec23.dat" ;
        lon360:_FillValue = -999999. ;
    double time(traj, obs) ;
        time:long_name = "Time" ;
        time:units = "seconds since 1970-01-01 00:00:00 UTC" ;
        time:axis = "T" ;
        time:valid_min = 200000000. ;
        time:valid_max = 30000000000. ;
        time:data_min = 1378123200. ;
        time:data_max = 1379743200. ;
        time:standard_name = "time" ;
        time:missing_value = -999999. ;
        time:history = "From buoydata_15001_dec23.dat" ;
        time:_FillValue = -999999. ;
    double temp(traj, obs) ;
        temp:long_name = "Sea Surface Bulk Temperature" ;
        temp:units = "degree_Celsius" ;
        temp:valid_min = -15. ;
        temp:valid_max = 40. ;
        temp:data_min = 25.085 ;
        temp:data_max = 30.452 ;
        temp:standard_name = "sea_surface_subskin_temperature" ;
        temp:missing_value = -999999. ;
        temp:history = "From buoydata_15001_dec23.dat" ;
        temp:_FillValue = -999999. ;
    double ve(traj, obs) ;
        ve:long_name = "Eastward velocity (12 hr centered difference)" ;
        ve:units = "m/s" ;
        ve:data_min = -0.42221 ;
        ve:data_max = 0.40372 ;
        ve:standard_name = "surface_eastward_sea_water_velocity" ;
        ve:missing_value = -999999. ;
        ve:history = "From buoydata_15001_dec23.dat" ;
        ve:_FillValue = -999999. ;
    double vn(traj, obs) ;
        vn:long_name = "Northward velocity (12 hr centered difference)" ;
        vn:units = "m/s" ;
        vn:data_min = -0.98642 ;
        vn:data_max = 1.17994 ;
        vn:standard_name = "surface_northward_sea_water_velocity" ;
        vn:missing_value = -999999. ;
        vn:history = "From buoydata_15001_dec23.dat" ;
        vn:_FillValue = -999999. ;
    double err_temp(traj, obs) ;
        err_temp:long_name = "Standard error in temperature" ;
        err_temp:units = "degree_Celsius" ;
        err_temp:missing_value = -999999. ;
        err_temp:history = "From buoydata_15001_dec23.dat" ;
        err_temp:_FillValue = -999999. ;
    double err_lat(traj, obs) ;
        err_lat:long_name = "Standard error in latitude" ;
        err_lat:units = "degrees_north" ;
        err_lat:missing_value = -999999. ;
        err_lat:history = "From buoydata_15001_dec23.dat" ;
        err_lat:_FillValue = -999999. ;
    double err_lon(traj, obs) ;
        err_lon:long_name = "Standard error in longitude" ;
        err_lon:units = "degrees_east" ;
        err_lon:missing_value = -999999. ;
        err_lon:history = "From buoydata_15001_dec23.dat" ;
        err_lon:_FillValue = -999999. ;

// global attributes:
        :title = "Global Drifter Program drifting buoy collection" ;
        :id = "Global Drifter Program ID 116117" ;
        :wmo_platform_code = "2200706" ;
        :ncei_template_version = "NCEI_NetCDF_Trajectory_Template_v2" ;
        :cdm_data_type = "Trajectory" ;
        :history = "From buoydata_15001_dec23.dat with metadata from dirall.dat and deplog.dat" ;
        :featureType = "trajectory" ;
        :Conventions = "CF-1.6, ACDD-1.3" ;
        :time_coverage_start = "2013-09-02T12:00:00Z" ;
        :time_coverage_end = "2013-09-21T06:00:00Z" ;
        :geospatial_lat_min = "8.016f" ;
        :geospatial_lat_max = "8.681f" ;
        :geospatial_lat_units = "degrees_north" ;
        :geospatial_lat_resolution = "tbd" ;
        :geospatial_lon_min = "106.321f" ;
        :geospatial_lon_max = "106.640f" ;
        :geospatial_lon_units = "degrees_east" ;
        :geospatial_lon_resolution = "tbd" ;
        :geospatial_bounds = "polygon (  180.000  8.016,  106.640  8.016,  106.640  8.681, 180.000  8.681, 180.000  8.016" ;
        :date_created = "2024-05-01T12:25:59Z" ;
        :date_modified = "2024-05-01T12:25:59Z" ;
        :date_issued = "2024-05-01T12:25:59Z" ;
        :publisher_name = "AOML Drifter DAC" ;
        :publisher_email = "aoml.dftr@noaa.gov" ;
        :publisher_url = "https://www.aoml.noaa.gov/phod/gdp" ;
        :license = "Creative Commons Attribution 4.0 (https://creative/commons.org/licenses/by/4.0/) This study used data collected and made freely available by the NO
     AA Global Drifter Program (https://www.aoml.noaa.gov/phod/gdp/) accessed from https://erddap.aoml.noaa.gov/gdp/erddap/index.html" ;
        :processing_level = "Level 2 QC by AOML drifter DAC" ;
        :metadata_link = "https://www.aoml.noaa.gov/phod/dac/dirall.html" ;
        :source = "source description" ;
        :uuid = "n/a" ;
        :naming_authority = "n/a" ;
        :contributor_name = "NOAA Global Drifter Program" ;
        :contributor_role = "Data Acquisition Center" ;
        :institution = "NOAA Atlantic Oceanographic and Meteorological Laboratory" ;
        :creator_name = "AOML drifter DAC" ;
        :creator_url = "https://www.aoml.noaa.gov/phod/gdp" ;
        :creator_email = "aoml.dftr@noaa.gov" ;
        :acknowledgement = "Lumpkin, Rick; Centurioni, Luca (2019). Global Drifter Program quality-controlled 6-hour interpolated data from ocean surface drifting buoy
     s. [indicate subset used].NOAA National Centers for Environmental Information. Dataset. https://doi.org/10.25921/7ntx-z961. Accessed [date]." ;
        :doi = "10.25921/7ntx-z961" ;
        :summary = "Global Drifter Program 6h data" ;
        :comment = "Global Drifter Program 6h data" ;
```

```
        :standard_name_vocabulary = "CF Standard Name Table v30" ;
        :instrument_vocabulary = "n/a" ;
        :keywords_vocabulary = "n/a" ;
        :keywords = "drifter, surface current, sea surface temperature, interpolated" ;
        :DeployingShip = "UNKNOWN" ;
        :DeploymentStatus = "good" ;
        :BuoyTypeManufacturer = "SIO" ;
        :BuoyTypeSensorArray = "SVP3A3" ;
        :CurrentProgram = "5325" ;
        :PurchaserFunding = "NOAA" ;
        :SensorUpgrade = "" ;
        :Transmissions = "NOAA" ;
        :DeployingCountry = "" ;
        :DeploymentComments = "" ;
        :ManufactureYear = "2013" ;
        :ManufactureMonth = "5" ;
        :ManufactureSensorType = "SVP3A3" ;
        :ManufactureVoltage = "SVP3A3 volts" ;
        :FloatDiameter = "38.1 cm" ;
        :SubsfcFloatPresence = "0" ;
        :DrogueType = "HOLY225" ;
        :DrogueLength = "6.1 m" ;
        :DrogueBallast = "1.8 kg" ;
        :DragAreaAboveDrogue = "11.45 m^2" ;
        :DragAreaOfDrogue = "512.51 m^2" ;
        :DragAreaRatio = "44.7607" ;
        :DrogueCenterDepth = "15 m" ;
        :DrogueDetectSensor = "tether strain" ;
}
```

drifter/l.check_chunk_sat_time_overlap.py

```python
import numpy
from tools_xtrackm import *

chunk_time_dict = {
    "1_5000": (
        np.datetime64("1985-10-30T00:00:00.000000000"),
        np.datetime64("2001-09-26T00:00:00.000000000"),
    ),
    "5001_10000": (
        np.datetime64("1997-03-20T00:00:00.000000000"),
        np.datetime64("2009-04-25T00:00:00.000000000"),
    ),
    "10001_15000": (
        np.datetime64("2006-03-05T00:00:00.000000000"),
        np.datetime64("2012-11-04T00:00:00.000000000"),
    ),
    "15001_current": (
        np.datetime64("2010-09-30T00:00:00.000000000"),
        np.datetime64("NaT", "ns"),
    ),
}

chunks = ["1_5000", "5001_10000", "10001_15000", "15001_current"]

for chunk in chunks:
    drift_tsta_o1, drift_tend_o1 = chunk_time_dict[chunk]
    drift_tsta_o, drift_tend_o = (
        n64todatetime1(drift_tsta_o1),
        n64todatetime1(drift_tend_o1),
    )
    # print(type(track_tsta_o), type(track_tend_o))
    for sat in sats_new:
        track_tsta_o, track_tend_o = get_time_limits_o(sat)
        # print(type(drift_tsta_o), type(drift_tend_o))
        overlap_tsta_o, overlap_tend_o = overlap_dates(
            track_tsta_o, track_tend_o, drift_tsta_o, drift_tend_o
        )
        if overlap_tsta_o is None or overlap_tend_o is None:
            continue
        print(
            f"{chunk} overlap for {sat} is {overlap_tsta_o} to {overlap_tend_o}"
        )

```

drifter/l.min_max_times.all.drifters.py

```python
import os
import glob
import xarray as xr
import numpy as np

data_loc = f"/home/srinivasu/allData/drifter_track_reg/"
chunks = ["1_5000", "5001_10000", "10001_15000", "15001_current"]

chunk_time_dict = {}
for chunk in chunks:
    start_times = []
    end_times = []
    for fd in sorted(
            glob.glob(f"{data_loc}/netcdf_{chunk}/"
                      f"drifter_6h_*.nc")):
        basename1 = os.path.basename(fd)
        ds_d = xr.open_dataset(fd, drop_variables=["WMO"])
        drift_tsta_o1, drift_tend_o1 = (
            ds_d.start_date.values[0],
            ds_d.end_date.values[0],
        )
        # print(type(drift_tsta_o1), type(drift_tend_o1))
        start_times.append(drift_tsta_o1)
        end_times.append(drift_tend_o1)
    chunk_time_dict[chunk] = (np.min(start_times), np.max(end_times))
    print(f"min of start times for {chunk} is {np.min(start_times)}")
    print(f"max of end times for {chunk} is {np.max(end_times)}")

print(chunk_time_dict)

```

```python
import os
import sys

import numpy as np
import xarray as xr
from tools_xtrackm import *

TRACKS_REG = (73.0, 99.0, 0.0, 25.4)
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
rose = dse.ROSE
cmap1 = "Greys_r"
dist_threshold = 2
data_loc = f"/home/srinivasu/allData/drifter_track_reg/"
fd = f"{data_loc}/netcdf_15001_current/drifter_6h_133666.nc"

basename1 = os.path.basename(fd)
ds_d = xr.open_dataset(fd, drop_variables=["WMO"])
print(ds_d)
# print attributes
for k, v in ds_d.attrs.items():
    print(f"{k}: {v}")
# byte_id = ds_d.ID.values[0]
# str_id = byte_id.decode("utf-8")
# drifter_id = str_id
byte_id = ds_d["ID"].isel(traj=0).values  # numpy bytes scalar like b'133666'
drifter_id = byte_id.item().decode("utf-8").strip()
drift_tsta_o1, drift_tend_o1 = (
    ds_d.start_date.values[0],
    ds_d.end_date.values[0],
)
print(f"{drifter_id} {drift_tsta_o1} {drift_tend_o1}")
drift_tsta_o, drift_tend_o = n64todatetime(drift_tsta_o1), n64todatetime(drift_tend_o1)

times_drift = ds_d.time.isel(traj=0).values
ve_da = drifter_time_asn(ds_d, var_str="ve")
vn_da = drifter_time_asn(ds_d, var_str="vn")
lons_da = drifter_time_asn(ds_d, var_str="longitude")
lats_da = drifter_time_asn(ds_d, var_str="latitude")
lons_da2 = lons_da.rolling(time=3).mean()
lats_da2 = lats_da.rolling(time=3).mean()
lons_da3 = lons_da2.ffill(dim="time").bfill(dim="time")
lats_da3 = lats_da2.ffill(dim="time").bfill(dim="time")

lons_drift = lons_da3.values
lats_drift = lats_da3.values

fig = plt.figure(figsize=(12, 10), layout="constrained")
ax2 = fig.add_subplot(1, 1, 1, projection=ccrs.PlateCarree())
rose_cut = rose.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:]))
rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=5)
ax2.grid()
ax2.scatter(
    lons_drift,
    lats_drift,
    marker=".",
    color="b",
    s=4,
)
plt.show()

```

```python
import os
import sys

import numpy as np
import xarray as xr
from tools_xtrackm import *
import sys

import xarray as xr

def grep_netcdf_meta(file: str, query: str):
    """
   Search for a string in variable names, variable attributes,
   coordinate names, and global attributes of an xarray dataset.
   Parameters
   ----------
   ds : xr.Dataset
       The opened xarray dataset.
   query : str
       The search string (case-insensitive).
   """
    ds = xr.open_dataset(file, decode_cf=False)
    q = query.lower()
    # Search variable names
    print("variable names --------------")
    for var in ds.data_vars:
        if q in var.lower():
            print(var)
    # Search variable attributes
    print("variable attributes --------------")
    for var in ds.data_vars:
        for attr, value in ds[var].attrs.items():
            if q in attr.lower() or q in str(value).lower():
                print(var, attr, value)
    # Search coordinate names
    print("coordinate names and attributes --------------")
    for coord in ds.coords:
        if q in coord.lower():
            print(coord)
        for attr, value in ds[coord].attrs.items():
            if q in attr.lower() or q in str(value).lower():
                print(coord, attr, value)
    # Search global attributes
    print("global attributes --------------")
    for attr, value in ds.attrs.items():
        if q in attr.lower() or q in str(value).lower():
            print(attr, value)

# drifter_data_loc = f"/home/srinivasu/allData/drifter1/"
# track_data_loc = f"/home/srinivasu/xtrackm/data/"
# sat = "GFO"
# fd = f"{drifter_data_loc}/netcdf_15001_current/track_reg/drifter_6h_133666.nc"
# ft = f"{track_data_loc}/{sat}/ctoh.sla.ref.GFO.nindian.238.nc"

fd = "drifter_6h_133666.nc"
ft = "ctoh.sla.ref.GFO.nindian.238.nc"

# decode_cd=False is the key here as xarray interfers with raw meta data of the file
# ds_d = xr.open_dataset(fd, drop_variables=["WMO"], decode_cf=False)
# ds_t = xr.open_dataset(ft, decode_cf=False)
ds_d = xr.open_dataset(fd, drop_variables=["WMO"])
ds_t = xr.open_dataset(ft, decode_times=False)
print(ds_d)
print(ds_t)

type_d = type(ds_d.time.values[0,0])
type_t = type(ds_t.time.values[0,0])

print(f"type of drifter time is {type_d}")
print(f"type of track time is {type_t}")

print(f"time units of drifter ******************")
grep_netcdf_meta(fd, "since")
print(f"time units of track ******************")
grep_netcdf_meta(ft, "since")

drift_tsta_o1, drift_tend_o1 = (
    ds_d.start_date.values[0],
    ds_d.end_date.values[0],
)
print(f"{type(drift_tsta_o1)}")
drift_tsta_o, drift_tend_o = n64todatetime1(drift_tsta_o1), n64todatetime1(drift_tend_o1)
# print(type(drift_tsta_o))

sla = track_dist_time_asn(ds_t, var_str="sla", units_in="m")
print(type(sla.time.values[0]))

times_drift = ds_d.time.isel(traj=0).values
ve_da = drifter_time_asn(ds_d, var_str="ve")
vn_da = drifter_time_asn(ds_d, var_str="vn")
lons_da = drifter_time_asn(ds_d, var_str="longitude")
lats_da = drifter_time_asn(ds_d, var_str="latitude")
sys.exit(0)

```

```python
import glob

import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.pyplot as plt
import pandas as pd
import xarray as xr
from matplotlib.colors import ListedColormap

# ------------------------
# User/configurable inputs
# ------------------------
# Region: (lon_min, lon_max, lat_min, lat_max)
REG = (43.0, 99.0, 0.0, 25.4)
cmap1 = ListedColormap(["white"])

# ETOPO/topo file
ETOPO_PATH = "/home/srinivasu/allData/topo/etopo_60s_io.nc"

# Drifter files
CHUNK = "15001_current"
DRIFTER_GLOB = f"/home/srinivasu/allData/drifter1/netcdf_{CHUNK}/track_reg/drifter_6h_*.nc"

sats_new = [
    "GFO",
    "TP+J1+J2+J3+S6A",
    "S3A",
    "S3B",
    "TPN+J1N+J2N+J3N",
    "ERS1+ERS2+ENV+SRL",
    "HY2A",
    "HY2B",
]
# Marker per satellite (you can adjust)
sat_marker_dict = {
    "GFO": "*",
    "TP+J1+J2+J3+S6A": "o",
    "S3A": "^",
    "S3B": "^",
    "TPN+J1N+J2N+J3N": "o",
    "ERS1+ERS2+ENV+SRL": "x",
    "HY2A": "s",
    "HY2B": "s",
}


# ------------------------
# Helper to pick topo var
# ------------------------
def get_topo_and_coords(dset):
    # Try common variable names for etopo/topo height
    for v in ["ROSE", "rose", "z", "elevation", "topo", "ETOPO", "Band1"]:
        if v in dset.variables:
            topo = dset[v]
            break
    else:
        topo = None

    # Try coordinate names
    lon_names = ["ETOPO05_X", "x", "lon", "longitude"]
    lat_names = ["ETOPO05_Y", "y", "lat", "latitude"]

    lon_name = next((n for n in lon_names if n in dset.coords), None)
    lat_name = next((n for n in lat_names if n in dset.coords), None)

    return topo, lon_name, lat_name


# ------------------------
# Load topo
# ------------------------
dse = xr.open_dataset(ETOPO_PATH)
topo, lon_name, lat_name = get_topo_and_coords(dse)

# ------------------------
# Create figure
# ------------------------
fig, ax = plt.subplots(1,
                       1,
                       figsize=(12, 9),
                       subplot_kw=dict(projection=ccrs.PlateCarree()))

# Add simple land/coatlines if desired
ax.add_feature(cfeature.LAND, facecolor="0.9")
ax.add_feature(cfeature.COASTLINE, linewidth=0.6)
ax.gridlines(draw_labels=True,
             linewidth=0.3,
             color="gray",
             alpha=0.5,
             linestyle="--")

# Plot topo if available
topo_sel = topo.sel({
    lon_name: slice(REG[0], REG[1]),
    lat_name: slice(REG[2], REG[3])
})
# A neutral background colormap to keep points visible
topo_sel.plot(ax=ax, add_colorbar=False, cmap="Greys_r")
# Set extent
ax.set_extent(REG, crs=ccrs.PlateCarree())

# ------------------------
# Plot drifter tracks
# ------------------------
# Expecting variables: longitude, latitude, ve in each drifter file
# and potentially traj dimension; adjust as needed
# white cmap for driter paths
# cmap = cm.get_cmap("white")
lon_var = "longitude"
lat_var = "latitude"
ve_var = "ve"
for fd in sorted(glob.glob(DRIFTER_GLOB)):
    ds_d = xr.open_dataset(fd, drop_variables=["WMO"])
    lons = ds_d[lon_var].isel(traj=0).values
    lats = ds_d[lat_var].isel(traj=0).values
    ve = ds_d[ve_var].isel(traj=0).values
    sc = ax.scatter(
```

```
        lons,
        lats,
        c=ve,
        cmap=cmap1,
        s=4,
        transform=ccrs.PlateCarree(),
        alpha=0.9,
    )

# Optional: add a colorbar for ve if the last scatter produced it
# Comment this block if you have heterogeneous presence of ve
try:
    cbar = plt.colorbar(sc,
                        ax=ax,
                        orientation="vertical",
                        fraction=0.04,
                        pad=0.03)
    cbar.set_label("Drifter velocity (ve)")
except Exception:
    pass

# ------------------------
# Plot intersection points from all satellites
# ------------------------
colors = plt.cm.get_cmap("tab10", len(sats_new))

any_points = False
for i, sat in enumerate(sats_new):
    csv_path = f"tracks_intersections_{sat}_1.csv"
    df = pd.read_csv(csv_path)
    ax.scatter(
        df["lons_inter"],
        df["lats_inter"],
        color=colors(i),
        s=18,
        marker=sat_marker_dict.get(sat, "o"),
        transform=ccrs.PlateCarree(),
        label=sat,
    )
    any_points = True

# Legend for satellites
if any_points:
    ax.legend(title="Satellite", loc="upper right", frameon=True)

plt.title("Drifter Tracks with All Satellite Intersection Points")
plt.tight_layout()
plt.show()

```

```python
import os

import numpy as np
import xarray as xr

# import Polygon
from shapely.geometry.polygon import Polygon
from tools_xtrackm import *


def closest_index(lons_drift, lats_drift, lon1, lat1):
    dist = np.sqrt((lons_drift - lon1)**2 + (lats_drift - lat1)**2)
    return np.argmin(dist)


def is_within_region_any(lons,
                         lats,
                         lon_min,
                         lon_max,
                         lat_min,
                         lat_max,
                         include_boundary=True):
    """
    Return True if any (lon, lat) point falls inside the rectangular region.
    include_boundary=True counts points on the edges/corners as inside.
    """
    lons = np.asarray(lons, dtype=float)
    lats = np.asarray(lats, dtype=float)
    if lons.shape != lats.shape:
        raise ValueError("lons and lats must have the same shape")
    if include_boundary:
        lons_in = (lons >= lon_min) & (lons <= lon_max)
        lats_in = (lats >= lat_min) & (lats <= lat_max)
    else:
        lons_in = (lons > lon_min) & (lons < lon_max)
        lats_in = (lats > lat_min) & (lats < lat_max)
    inside_mask = lons_in & lats_in
    hit = np.any(inside_mask)
    if hit:
        print("At least one point is inside the region.")
    else:
        print("No points are inside the region.")
    return bool(hit)


chunk_time_dict = {
    "1_5000": (
        np.datetime64("1985-10-30T00:00:00.000000000"),
        np.datetime64("1999-11-01T00:00:00.000000000"),
    ),
    "5001_10000": (
        np.datetime64("1997-03-20T00:00:00.000000000"),
        np.datetime64("2006-09-16T00:00:00.000000000"),
    ),
    "10001_15000": (
        np.datetime64("2006-05-24T00:00:00.000000000"),
        np.datetime64("2011-01-04T00:00:00.000000000"),
    ),
    "15001_current": (
        np.datetime64("2010-10-30T00:00:00.000000000"),
        np.datetime64("2024-12-31T00:00:00.000000000"),
    ),
}

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

d = 1.5
data_loc = f"/home/srinivasu/allData/drifter1/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
rose = dse.ROSE
cmap1 = "Greys_r"
dist_threshold = 1  # in degrees
computed_loc = f"/home/srinivasu/slnew/xtrackm/computed/"
chunk = "15001_current"
chunks = ["1_5000", "5001_10000", "10001_15000", "15001_current"]

found = False
for i, r in df_all.iterrows():
    sat1 = r["sat"]
    # if sat1 != "S3A":
    #     continue
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    track_number_self = str(r["track_self"])
    track_number_other = str(r["track_other"])
    lons_inter1 = r["lons_inter"]
    lats_inter1 = r["lats_inter"]
    x_from_coast_self1 = r["x_from_coast_self"]
    x_from_coast_other1 = r["x_from_coast_other"]
    angle_acute = r["angle_acute"]
    angle_obtuse = r["angle_obtuse"]
    lon1 = lons_inter1
    lat1 = lats_inter1
    if abs(lat1) < 2:
        continue
    print(
        sat1,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    lon_inter_box_min = lon1 - dist_threshold
    lon_inter_box_max = lon1 + dist_threshold
    lat_inter_box_min = lat1 - dist_threshold
    lat_inter_box_max = lat1 + dist_threshold
    box = Polygon([
        (lon_inter_box_min, lat_inter_box_min),
        (lon_inter_box_max, lat_inter_box_min),
        (lon_inter_box_max, lat_inter_box_max),
```

```
        (lon_inter_box_min, lat_inter_box_max),
    ])
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_self}_loess_0.2.nc"
    f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_other}_loess_0.2.nc"
    ds_self = xr.open_dataset(f_self, engine="h5netcdf", decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    print(f"{angle_self} {angle_other} --------------------")
    fig = plt.figure(figsize=(12, 10), layout="constrained")
    ax2 = fig.add_subplot(1, 1, 1, projection=ccrs.PlateCarree())
    rose.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=4,
        markerfacecolor="red",
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    ax2.add_geometries([box],
                       ccrs.PlateCarree(),
                       edgecolor="g",
                       facecolor="none",
                       linewidth=2)
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="w",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="w",
        )
    for chunk in chunks:
        chunk_tsta_o1, chunk_tend_o1 = chunk_time_dict[chunk]
        chunk_tsta_o, chunk_tend_o = n64todatetime1(
            chunk_tsta_o1), n64todatetime1(chunk_tend_o1)
        chunk_overlap_tsta_o, chunk_overlap_tend_o = overlap_dates(
            track_tsta_o, track_tend_o, chunk_tsta_o, chunk_tend_o)
        if chunk_overlap_tsta_o is None or chunk_overlap_tend_o is None:
            continue
        for fd in sorted(
                glob.glob(
                    f"{data_loc}/netcdf_{chunk}/track_reg/drifter_6h_*.nc")):
            print(fd)
            basename1 = os.path.basename(fd)
            ds_d = xr.open_dataset(fd, drop_variables=["WMO"])
            byte_id = ds_d.ID.values[0]
            str_id = byte_id.decode("utf-8")
            drifter_id = str_id
            drift_tsta_o1, drift_tend_o1 = (
                ds_d.start_date.values[0],
                ds_d.end_date.values[0],
            )
            drift_tsta_o, drift_tend_o = n64todatetime1(
                drift_tsta_o1), n64todatetime1(drift_tend_o1)
            overlap_tsta_o, overlap_tend_o = overlap_dates(
                track_tsta_o, track_tend_o, drift_tsta_o, drift_tend_o)
            print(f"overlap period {overlap_tsta_o} {overlap_tend_o}")
```

```
            if overlap_tsta_o is None or overlap_tend_o is None:
                continue
            times_drift = ds_d.time.isel(traj=0).values
            ve_da = drifter_time_asn(ds_d, var_str="ve")
            vn_da = drifter_time_asn(ds_d, var_str="vn")
            lons_da = drifter_time_asn(ds_d, var_str="longitude")
            lats_da = drifter_time_asn(ds_d, var_str="latitude")
            lons_da2 = lons_da.rolling(time=3).mean()
            lats_da2 = lats_da.rolling(time=3).mean()
            lons_da3 = lons_da2.ffill(dim="time").bfill(dim="time")
            lats_da3 = lats_da2.ffill(dim="time").bfill(dim="time")

            lons_drift = lons_da3.values
            lats_drift = lats_da3.values
            if is_within_region_any(
                    lons_drift,
                    lats_drift,
                    lon_inter_box_min,
                    lon_inter_box_max,
                    lat_inter_box_min,
                    lat_inter_box_max,
            ):

                nidx = closest_index(lons_drift, lats_drift, lon1, lat1)
                close_drift_lon = lons_drift[nidx]
                close_drift_lat = lats_drift[nidx]
                ax2.scatter(
                    lons_drift,
                    lats_drift,
                    marker=".",
                    color="y",
                    s=4,
                )
                ax2.plot(
                    close_drift_lon,
                    close_drift_lat,
                    marker="x",
                    markersize=8,
                    color="k",
                )
                plt.show()
                plt.close()
                found = True
                # break

```

```python
import os

import numpy as np
import xarray as xr

# import Polygon
from shapely.geometry.polygon import Polygon
from tools_xtrackm import *
from geopy import distance
import sys
import ast


def convert_to_list(x):
    try:
        return ast.literal_eval(x) if isinstance(x, str) else x
    except (ValueError, SyntaxError):
        return x  # Return as-is if conversion fails


def closest_index(lons_drift, lats_drift, lon1, lat1):
    dist = [distance.distance((y, x), (lat1, lon1)).m for x, y in zip(lons_drift, lats_drift)]
    return np.argmin(dist)


def is_within_region_any(lons,
                         lats,
                         lon_min,
                         lon_max,
                         lat_min,
                         lat_max,
                         include_boundary=True):
    """
    Return True if any (lon, lat) point falls inside the rectangular region.
    include_boundary=True counts points on the edges/corners as inside.
    """
    lons = np.asarray(lons, dtype=float)
    lats = np.asarray(lats, dtype=float)
    if lons.shape != lats.shape:
        raise ValueError("lons and lats must have the same shape")
    if include_boundary:
        lons_in = (lons >= lon_min) & (lons <= lon_max)
        lats_in = (lats >= lat_min) & (lats <= lat_max)
    else:
        lons_in = (lons > lon_min) & (lons < lon_max)
        lats_in = (lats > lat_min) & (lats < lat_max)
    inside_mask = lons_in & lats_in
    hit = np.any(inside_mask)
    if hit:
        print("At least one point is inside the region.")
    else:
        print("No points are inside the region.")
    return bool(hit)


chunk_time_dict = {
    "1_5000": (
        np.datetime64("1985-10-30T00:00:00.000000000"),
        np.datetime64("1999-11-01T00:00:00.000000000"),
    ),
    "5001_10000": (
        np.datetime64("1997-03-20T00:00:00.000000000"),
        np.datetime64("2006-09-16T00:00:00.000000000"),
    ),
    "10001_15000": (
        np.datetime64("2006-05-24T00:00:00.000000000"),
        np.datetime64("2011-01-04T00:00:00.000000000"),
    ),
    "15001_current": (
        np.datetime64("2010-10-30T00:00:00.000000000"),
        np.datetime64("2024-12-31T00:00:00.000000000"),
    ),
}

sat_here = "TP+J1+J2+J3+S6A"
df_all = pd.read_csv(f"close_drifters_at_intersection_point_{sat_here}.csv")
d = 1.5
data_loc = f"/home/srinivasu/allData/drifter_track_reg/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
rose = dse.ROSE
cmap1 = "Greys_r"
dist_threshold = 1  # in degrees
# Apply the conversion to the column with lists
df_all['close_drifters_column'] = df_all['close_drifters_column'].apply(convert_to_list)

found = False
for i, r in df_all.iterrows():
    sat1 = r["sat"]
    # if sat1 != "S3A":
    #     continue
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    track_number_self = str(r["track_self"])
    track_number_other = str(r["track_other"])
    lons_inter1 = r["lons_inter"]
    lats_inter1 = r["lats_inter"]
    x_from_coast_self1 = r["x_from_coast_self"]
    x_from_coast_other1 = r["x_from_coast_other"]
    angle_acute = r["angle_acute"]
    angle_obtuse = r["angle_obtuse"]
    lon1 = lons_inter1
    lat1 = lats_inter1
    close_ones = r["close_drifters_column"]
    print(close_ones)
    # break
    if abs(lat1) < 2:
        continue
    print(
        sat1,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
```

```
    lon_inter_box_min = lon1 - dist_threshold
    lon_inter_box_max = lon1 + dist_threshold
    lat_inter_box_min = lat1 - dist_threshold
    lat_inter_box_max = lat1 + dist_threshold
    box = Polygon([
        (lon_inter_box_min, lat_inter_box_min),
        (lon_inter_box_max, lat_inter_box_min),
        (lon_inter_box_max, lat_inter_box_max),
        (lon_inter_box_min, lat_inter_box_max),
    ])
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    print(f"{angle_self} {angle_other} -------------------")
    for id_drifter in close_ones:
        print(id_drifter)
        fd = f"{data_loc}/netcdf_all/drifter_6h_{id_drifter}.nc"
        print(fd)
        basename1 = os.path.basename(fd)
        ds_d = xr.open_dataset(fd, drop_variables=["WMO"])
        byte_id = ds_d.ID.values[0]
        str_id = byte_id.decode("utf-8")
        drifter_id = str_id
        drift_tsta_o1, drift_tend_o1 = (
            ds_d.start_date.values[0],
            ds_d.end_date.values[0],
        )
        drift_tsta_o, drift_tend_o = n64todatetime1(
            drift_tsta_o1), n64todatetime1(drift_tend_o1)
        overlap_tsta_o, overlap_tend_o = overlap_dates(
            track_tsta_o, track_tend_o, drift_tsta_o, drift_tend_o)
        print(f"satellite period {track_tsta_o} {track_tend_o}")
        print(f"drifter period {drift_tsta_o} {drift_tend_o}")
        print(f"overlap period {overlap_tsta_o} {overlap_tend_o}")
        if overlap_tsta_o is None or overlap_tend_o is None:
            continue
        lons_da = drifter_time_asn(ds_d, var_str="longitude")
        lats_da = drifter_time_asn(ds_d, var_str="latitude")
        lons_da2 = lons_da.rolling(time=3).mean()
        lats_da2 = lats_da.rolling(time=3).mean()
        lons_da3 = lons_da2.ffill(dim="time").bfill(dim="time")
        lats_da3 = lats_da2.ffill(dim="time").bfill(dim="time")

        lons_drift = lons_da3.values
        lats_drift = lats_da3.values
        nidx = closest_index(lons_drift, lats_drift, lon1, lat1)
        close_drift_lon = lons_drift[nidx]
        close_drift_lat = lats_drift[nidx]
        close_dist = distance.distance((lat1, lon1), (close_drift_lat, close_drift_lon)).km
        info = f"{sat1} {track_number_self.zfill(3)} {track_number_other.zfill(3)} {drifter_id} {close_drift_lon:.2f} {close_drift_lat:.2f}"
        info1 = f"{close_dist:.2f}"
        fig = plt.figure(figsize=(12, 10), layout="constrained")
        ax2 = fig.add_subplot(1, 1, 1, projection=ccrs.PlateCarree())
        rose.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
            *TRACKS_REG[2:])).plot(ax=ax2,
                                   add_colorbar=False,
                                   add_labels=False,
                                   cmap=cmap1)
        decorate_axis(ax2, "", *TRACKS_REG, step=5)
        ax2.grid()
        # add info to ax2 to the figure
        ax2.text(
            0.3,
            0.9,
            info,
            transform=ax2.transAxes,
            fontsize=18,
            color="b",
        )
        ax2.text(
            0.6,
            0.7,
            info1,
            transform=ax2.transAxes,
            fontsize=18,
            color="b",
        )
        ax2.plot(
            lon1,
            lat1,
            c="r",
            marker="o",
            markersize=4,
            markerfacecolor="red",
        )
        ax2.scatter(
            lons_track_self,
            lats_track_self,
            marker=".",
            color="c",
            s=4,
        )
        ax2.scatter(
            lons_track_other,
            lats_track_other,
            marker=".",
```

```
            color="c",
            s=4,
        )
        ax2.add_geometries([box],
                           ccrs.PlateCarree(),
                           edgecolor="g",
                           facecolor="none",
                           linewidth=2)
        lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                           lon_coast_self, lat_coast_self, d)
        if is_within_region(lonm, latm, *TRACKS_REG):
            plt.text(
                lonm,
                latm,
                s=track_number_self,
                fontsize=14,
                rotation=angle_self,
                color="w",
            )
        lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                           lon_coast_other, lat_coast_other, d)
        if is_within_region(lonm, latm, *TRACKS_REG):
            plt.text(
                lonm,
                latm,
                s=track_number_other,
                fontsize=14,
                rotation=angle_other,
                color="w",
            )
        ax2.scatter(
            lons_drift,
            lats_drift,
            marker=".",
            color="y",
            s=4,
        )
        ax2.plot(
            close_drift_lon,
            close_drift_lat,
            marker="x",
            markersize=8,
            color="k",
        )
        plt.show()
        plt.close("all")
        found = True
            # break

```

```python
import os

import numpy as np
import xarray as xr
from tools_xtrackm import *


def closest_index(lons_drift, lats_drift, lon1, lat1):
    dist = np.sqrt((lons_drift - lon1) ** 2 + (lats_drift - lat1) ** 2)
    return np.argmin(dist)


def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    u = -1 * u
    return u, v


sat, track_number_self, track_number_other, lons_inter, lats_inter = (
    "S3A",
    "8",
    "79",
    94.14047103085358,
    8.36326782978126,
)
x_from_coast_self1, x_from_coast_other1, angle_acute, angle_obtuse = 885051.8986275726, 1543629.620555419, 0.44727413372659675, 2.6943185198631965

d = 1.5
TRACKS_REG = (73.0, 99.0, 0.0, 25.4)
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
rose = dse.ROSE
cmap1 = "Greys_r"
lon1, lat1 = lons_inter, lats_inter
dist_threshold = 2
computed_loc = f"/home/srinivasu/slnew/xtrackm/computed/"
data_loc = f"/home/srinivasu/allData/drifter1/"
fd = f"{data_loc}/netcdf_15001_current/track_reg/drifter_6h_133666.nc"
box = create_box_patch(lons_inter, lats_inter, dist_threshold)
lon_max, lat_max = lons_inter + dist_threshold, lats_inter + dist_threshold
lat_min, lon_min = lats_inter - dist_threshold, lons_inter - dist_threshold
track_tsta_o, track_tend_o = get_time_limits_o(sat)
print(f"{sat} {track_tsta_o} {track_tend_o}")

basename1 = os.path.basename(fd)
ds_d = xr.open_dataset(fd, drop_variables=["WMO"])
byte_id = ds_d.ID.values[0]
str_id = byte_id.decode("utf-8")
drifter_id = str_id
drift_tsta_o1, drift_tend_o1 = (
    ds_d.start_date.values[0],
    ds_d.end_date.values[0],
)
print(f"{drifter_id} {drift_tsta_o1} {drift_tend_o1}")
drift_tsta_o, drift_tend_o = n64todatetime(drift_tsta_o1), n64todatetime(
    drift_tend_o1)
overlap_tsta_o, overlap_tend_o = overlap_dates(track_tsta_o, track_tend_o,
                                               drift_tsta_o, drift_tend_o)
print(f"overlap period {overlap_tsta_o} {overlap_tend_o}")
times_drift = ds_d.time.isel(traj=0).values
ve_da = drifter_time_asn(ds_d, var_str="ve")
vn_da = drifter_time_asn(ds_d, var_str="vn")
lons_da = drifter_time_asn(ds_d, var_str="longitude")
lats_da = drifter_time_asn(ds_d, var_str="latitude")
lons_da1 = lons_da.resample(time="1D").mean()
lats_da1 = lats_da.resample(time="1D").mean()
lons_da2 = lons_da1.rolling(time=3).mean()
lats_da2 = lats_da1.rolling(time=3).mean()
lons_da3 = lons_da2.ffill(dim="time").bfill(dim="time")
lats_da3 = lats_da2.ffill(dim="time").bfill(dim="time")

lons_drift = lons_da3.values
lats_drift = lats_da3.values

nidx = closest_index(lons_drift, lats_drift, lon1, lat1)
close_drift_lon = lons_drift[nidx]
close_drift_lat = lats_drift[nidx]

```

```
f_self = (
    f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
)
f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_self}_loess_0.2.nc"
f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_other}_loess_0.2.nc"
ds_self = xr.open_dataset(f_self, engine="h5netcdf", decode_times=False)
ds_self_smooth = xr.open_dataset(f_self_smooth)
sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
sla_self_smooth = ds_self_smooth.sla_smooth
ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
ds_other_smooth = xr.open_dataset(f_other_smooth)
sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
sla_other_smooth = ds_other_smooth.sla_smooth
lons_track_self = ds_self.lon.values
lats_track_self = ds_self.lat.values
lon_coast_self = lons_track_self[-1]  # this on coast
lat_coast_self = lats_track_self[-1]  # this on coast
lon_equat_self = lons_track_self[0]  # this on equator
lat_equat_self = lats_track_self[0]  # this on equator
slope_self = (lats_track_self[-1] -
              lats_track_self[0]) / (lons_track_self[-1] - lons_track_self[0])
angle_self = np.rad2deg(np.arctan(slope_self))
track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
lons_track_other = ds_other.lon.values
lats_track_other = ds_other.lat.values
lon_equat_other = lons_track_other[0]
lat_equat_other = lats_track_other[0]
lon_coast_other = lons_track_other[-1]
lat_coast_other = lats_track_other[-1]
slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                     lon_equat_other)
angle_other = np.rad2deg(np.arctan(slope_other))
track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))

gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
gc_self_at = gc_self.sel(x=x_from_coast_self1,
                         method="nearest",
                         drop=True)
gc_other_at = gc_other.sel(x=x_from_coast_other1,
                           method="nearest",
                           drop=True)
gc_other_at = gc_other_at.interp(time=gc_self_at.time)

a2 = math.atan2(
    lat_coast_other - lat_equat_other,
    lon_coast_other - lon_equat_other,
)
a1 = math.atan2(lat_coast_self - lat_equat_self,
                lon_coast_self - lon_equat_self)
print(f"azimuths {a1:.2f} {a2:.2f}")
gu, gv = geostrophic_components_from_a(gc_self_at, gc_other_at, a1, a2)

fig = plt.figure(figsize=(12, 10), layout="constrained")
ax2 = fig.add_subplot(1, 1, 1, projection=ccrs.PlateCarree())
rose.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
    *TRACKS_REG[2:])).plot(ax=ax2,
                           add_colorbar=False,
                           add_labels=False,
                           cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=5)
ax2.grid()
ax2.plot(
    lon1,
    lat1,
    c="r",
    marker="o",
    markersize=4,
    markerfacecolor="red",
)
ax2.scatter(
    lons_track_self,
    lats_track_self,
    marker=".",
    color="c",
    s=4,
)
ax2.scatter(
    lons_track_other,
    lats_track_other,
    marker=".",
    color="c",
    s=4,
)
ax2.scatter(
    lons_drift,
    lats_drift,
    marker=".",
    color="b",
    s=4,
)
ax2.plot(
    close_drift_lon,
    close_drift_lat,
    marker="x",
    markersize=8,
    color="k",
)
# add polygon box as geometry
ax2.add_geometries([box],
                   ccrs.PlateCarree(),
                   edgecolor="g",
                   facecolor="none",
                   linewidth=2)
lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                   lon_coast_self, lat_coast_self, d)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=angle_self,
        color="k",
    )
lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
```

```
                                    lon_coast_other, lat_coast_other, d)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=angle_other,
        color="k",
    )
plt.show()

```

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import sys

import numpy as np
import pandas as pd


def clean_outliers(gu, ve, max_speed=3.0, mad_thresh=3.5):
    """
    Remove outliers from paired arrays (gu, ve).
    Parameters
    ----------
    gu, ve : array-like
        Arrays of satellite geostrophic u and drifter ve.
    max_speed : float
        Max absolute speed (m/s) allowed before removing.
    mad_thresh : float
        Threshold (in MAD units) for robust outlier removal.
    Returns
    -------
    gu_clean, ve_clean : ndarray
        Filtered arrays with outliers removed.
    mask : ndarray (bool)
        Boolean mask of valid points.
    """
    # gu = pd.to_numeric(gu, errors='coerce')
    # ve = pd.to_numeric(ve, errors='coerce')
    gu = np.asarray(gu)
    ve = np.asarray(ve)
    # Step 1: remove NaNs and infinities
    mask = np.isfinite(gu) & np.isfinite(ve)
    # Step 2: physical limit
    mask &= (np.abs(gu) <= max_speed) & (np.abs(ve) <= max_speed)
    # Step 3: robust MAD filter (applied separately to gu and ve)
    def mad_based_outlier(points, thresh=mad_thresh):
        median = np.median(points)
        mad = np.median(np.abs(points - median))
        if mad == 0:
            return np.zeros_like(points, dtype=bool)
        z = 0.6745 * (points - median) / mad
        return np.abs(z) > thresh
    mask &= ~mad_based_outlier(gu, mad_thresh)
    mask &= ~mad_based_outlier(ve, mad_thresh)
    return gu[mask], ve[mask], mask

sat_here = sys.argv[1]
df1 = pd.read_csv(f"track_guvVSnearby_drifters_at_intersection_points_{sat_here}.csv")

gu_clean, ve_clean, mask = clean_outliers(df1["gu"], df1["ve"])

df = pd.DataFrame({
    "gu_mask": gu_clean,
    "ve_mask": ve_clean
})
# Create a scatter plot
plt.figure(figsize=(8, 6))
plt.scatter(df["gu_mask"], df["ve_mask"], alpha=0.5)
plt.title("Scatter Plot of gu vs ve")
plt.xlabel("track gu")
plt.ylabel("drifter ve")
plt.grid(True)

# Calculate the correlation coefficient
correlation = df["gu_mask"].corr(df["ve_mask"])

# Place the correlation coefficient value on the plot
plt.text(0.05, 0.95, f'Correlation: {correlation:.2f}', transform=plt.gca().transAxes, fontsize=12,
         verticalalignment='top', bbox=dict(boxstyle='round,pad=0.5', fc='yellow', alpha=0.5))

# Calculate the trend line (line of best fit)
m, b = np.polyfit(df['gu_mask'], df['ve_mask'], 1)
x_sorted = np.sort(df['gu_mask'])

# import statsmodels.api as sm
# X = sm.add_constant(df['gu_mask'])
# model = sm.RLM(df['ve_mask'], X, M=sm.robust.norms.HuberT()).fit()
# b, m = model.params
# x_sorted = np.sort(df['gu_mask'])

# Add the trend line to the plot
plt.plot(x_sorted, m*x_sorted + b, color='red', linewidth=2, label='Trend Line')
plt.legend()
# Save the plot
plt.savefig(f'scatter_plot_{sat_here}_track_uVSnearby_drifter_u.png')
plt.show()

print(f"Correlation coefficient: {correlation}")

```

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import sys

import numpy as np
import pandas as pd


def clean_outliers(gu, ve, max_speed=3.0, mad_thresh=3.5):
    """
    Remove outliers from paired arrays (gu, ve).
    Parameters
    ----------
    gu, ve : array-like
        Arrays of satellite geostrophic u and drifter ve.
    max_speed : float
        Max absolute speed (m/s) allowed before removing.
    mad_thresh : float
        Threshold (in MAD units) for robust outlier removal.
    Returns
    -------
    gu_clean, ve_clean : ndarray
        Filtered arrays with outliers removed.
    mask : ndarray (bool)
        Boolean mask of valid points.
    """
     gu = np.asarray(gu)
     ve = np.asarray(ve)
     # Step 1: remove NaNs and infinities
     mask = np.isfinite(gu) & np.isfinite(ve)
     # Step 2: physical limit
     mask &= (np.abs(gu) <= max_speed) & (np.abs(ve) <= max_speed)
     # Step 3: robust MAD filter (applied separately to gu and ve)
     def mad_based_outlier(points, thresh=mad_thresh):
         median = np.median(points)
         mad = np.median(np.abs(points - median))
         if mad == 0:
             return np.zeros_like(points, dtype=bool)
         z = 0.6745 * (points - median) / mad
         return np.abs(z) > thresh

     mask &= ~mad_based_outlier(gu, mad_thresh)
     mask &= ~mad_based_outlier(ve, mad_thresh)

     return gu[mask], ve[mask], mask

sat_here = sys.argv[1]
df1 = pd.read_csv(f"track_guvVSnearby_drifters_at_intersection_points_{sat_here}_2.csv")

gu_clean, ve_clean, mask = clean_outliers(df1["gu"], df1["ve"])

df = pd.DataFrame({
    "gu_mask": gu_clean,
    "ve_mask": ve_clean
})
# Create a scatter plot
plt.figure(figsize=(8, 6))
plt.scatter(df["gu_mask"], df["ve_mask"], alpha=0.5)
plt.title("Scatter Plot of gu vs ve")
plt.xlabel("track gu")
plt.ylabel("drifter ve")
plt.grid(True)

# Calculate the correlation coefficient
correlation = df["gu_mask"].corr(df["ve_mask"])

# Place the correlation coefficient value on the plot
plt.text(0.05, 0.95, f'Correlation: {correlation:.2f}', transform=plt.gca().transAxes, fontsize=12,
         verticalalignment='top', bbox=dict(boxstyle='round,pad=0.5', fc='yellow', alpha=0.5))

# Calculate the trend line (line of best fit)
m, b = np.polyfit(df['gu_mask'], df['ve_mask'], 1)
x_sorted = np.sort(df['gu_mask'])

# import statsmodels.api as sm
# X = sm.add_constant(df['gu_mask'])
# model = sm.RLM(df['ve_mask'], X, M=sm.robust.norms.HuberT()).fit()
# b, m = model.params
# x_sorted = np.sort(df['gu_mask'])

# Add the trend line to the plot
plt.plot(x_sorted, m*x_sorted + b, color='red', linewidth=2, label='Trend Line')
plt.legend()
# Save the plot
plt.savefig(f'scatter_plot_{sat_here}_track_uVSnearby_drifter_u.png')
plt.show()

print(f"Correlation coefficient: {correlation}")

```

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import sys

import numpy as np
import pandas as pd


def clean_outliers(gu, ve, ek, max_speed=3.0, mad_thresh=3.5):
    """
    Remove outliers from paired arrays (gu, ve).
    Parameters
    ----------
    gu, ve : array-like
        Arrays of satellite geostrophic u and drifter ve.
    max_speed : float
        Max absolute speed (m/s) allowed before removing.
    mad_thresh : float
        Threshold (in MAD units) for robust outlier removal.
    Returns
    -------
    gu_clean, ve_clean : ndarray
        Filtered arrays with outliers removed.
    mask : ndarray (bool)
        Boolean mask of valid points.
    """
     gu = np.asarray(gu)
     ve = np.asarray(ve)
     ek = np.asarray(ek)
     # Step 1: remove NaNs and infinities
     mask = np.isfinite(gu) & np.isfinite(ve) & np.isfinite(ek)
     # Step 2: physical limit
     mask &= (np.abs(gu) <= max_speed) & (np.abs(ve) <= max_speed) & (np.abs(ek) <= max_speed)
     # Step 3: robust MAD filter (applied separately to gu and ve)
     def mad_based_outlier(points, thresh=mad_thresh):
         median = np.median(points)
         mad = np.median(np.abs(points - median))
         if mad == 0:
             return np.zeros_like(points, dtype=bool)
         z = 0.6745 * (points - median) / mad
         return np.abs(z) > thresh

     mask &= ~mad_based_outlier(gu, mad_thresh)
     mask &= ~mad_based_outlier(ve, mad_thresh)
     mask &= ~mad_based_outlier(ek, mad_thresh)

     return gu[mask], ve[mask], ek[mask], mask

sat_here = sys.argv[1]
df1 = pd.read_csv(f"track_guvVSnearby_drifters_at_intersection_points_{sat_here}_w_ekman.csv")

gu_clean, ve_clean, ek_clean, mask = clean_outliers(df1["gu"], df1["ve"], df1["ek"])

df = pd.DataFrame({
    "gu_mask": gu_clean,
    "ve_mask": ve_clean,
    "ek_mask": ek_clean,
    "gu_ek": gu_clean + ek_clean
})
# Create a scatter plot
plt.figure(figsize=(8, 6))
plt.scatter(df["gu_ek"], df["ve_mask"], alpha=0.5)
plt.title("Scatter Plot of gu+ek vs ve")
plt.xlabel("track gu+ek")
plt.ylabel("drifter ve")
plt.grid(True)

# Calculate the correlation coefficient
correlation = df["gu_ek"].corr(df["ve_mask"])

# Place the correlation coefficient value on the plot
plt.text(0.05, 0.95, f'Correlation: {correlation:.2f}', transform=plt.gca().transAxes, fontsize=12,
         verticalalignment='top', bbox=dict(boxstyle='round,pad=0.5', fc='yellow', alpha=0.5))

# Calculate the trend line (line of best fit)
m, b = np.polyfit(df['gu_mask'], df['ve_mask'], 1)
x_sorted = np.sort(df['gu_mask'])

# import statsmodels.api as sm
# X = sm.add_constant(df['gu_mask'])
# model = sm.RLM(df['ve_mask'], X, M=sm.robust.norms.HuberT()).fit()
# b, m = model.params
# x_sorted = np.sort(df['gu_mask'])

# Add the trend line to the plot
plt.plot(x_sorted, m*x_sorted + b, color='red', linewidth=2, label='Trend Line')
plt.legend()
# Save the plot
plt.savefig(f'scatter_plot_{sat_here}_track_uVSnearby_drifter_u_ek.png')
plt.show()

print(f"Correlation coefficient: {correlation}")

```

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import sys

import numpy as np
import pandas as pd
import xarray as xr


def clean_outliers(gu, ve, max_speed=3.0, mad_thresh=3.5):
    """
    Remove outliers from paired arrays (gu, ve).
    Parameters
    ----------
    gu, ve : array-like
        Arrays of satellite geostrophic u and drifter ve.
    max_speed : float
        Max absolute speed (m/s) allowed before removing.
    mad_thresh : float
        Threshold (in MAD units) for robust outlier removal.
    Returns
    -------
    gu_clean, ve_clean : ndarray
        Filtered arrays with outliers removed.
    mask : ndarray (bool)
        Boolean mask of valid points.
    """
    gu = np.asarray(gu)
    ve = np.asarray(ve)
    # Step 1: remove NaNs and infinities
    mask = np.isfinite(gu) & np.isfinite(ve)
    # Step 2: physical limit
    mask &= (np.abs(gu) <= max_speed) & (np.abs(ve) <= max_speed)
    # Step 3: robust MAD filter (applied separately to gu and ve)
    def mad_based_outlier(points, thresh=mad_thresh):
        median = np.median(points)
        mad = np.median(np.abs(points - median))
        if mad == 0:
            return np.zeros_like(points, dtype=bool)
        z = 0.6745 * (points - median) / mad
        return np.abs(z) > thresh

    mask &= ~mad_based_outlier(gu, mad_thresh)
    mask &= ~mad_based_outlier(ve, mad_thresh)

    return gu[mask], ve[mask], mask

sat_here = sys.argv[1]
df1 = pd.read_csv(f"track_guvVSnearby_drifters_at_intersection_points_{sat_here}_2.csv")

topo_dir = "/home/srinivasu/allData/topo/"
dsc = xr.open_dataset(f"{topo_dir}"
                      f"GMT_intermediate_coast_distance_01d_track_reg.nc")
print(dsc)
dist_to_coast = dsc.coast_dist


def is_within_coast(dist_to_coast, lon_inter, lat_inter, threshold=100):
    """
    Checks if a coordinate pair is within a certain distance from the coast.
    """
    # Using try-except to handle coordinates that may be outside the grid bounds
    try:
        dist1 = dist_to_coast.sel(lon=lon_inter, lat=lat_inter, method='nearest').item()
        if dist1 < threshold:
            return True
    except (KeyError, ValueError):
        # This will catch errors if lon/lat are not in the 'dist_to_coast' dataset
        return False
    return False


print(f"Original number of rows: {len(df1)}")

# Create a boolean mask by applying the function to each row
coastal_mask = df1.apply(
    lambda row: is_within_coast(dist_to_coast, row['lon_inters'], row['lat_inters'], threshold=1000),
    axis=1
)

# Use the mask to create a new DataFrame containing only coastal points
df_coastal = df1[coastal_mask].copy()

print(f"Number of rows after filtering for coast proximity: {len(df_coastal)}")

# --- Continue with your analysis on the filtered DataFrame ---
# Now use df_coastal instead of df1
gu_clean, ve_clean, mask = clean_outliers(df_coastal["gu"], df_coastal["ve"])

df = pd.DataFrame({
    "gu_mask": gu_clean,
    "ve_mask": ve_clean
})
# Create a scatter plot
plt.figure(figsize=(8, 6))
plt.scatter(df["gu_mask"], df["ve_mask"], alpha=0.5)
plt.title(f"Scatter Plot of gu vs ve for {sat_here}")
plt.xlabel("track gu")
plt.ylabel("drifter ve")
plt.grid(True)

# Calculate the correlation coefficient
correlation = df["gu_mask"].corr(df["ve_mask"])

# Place the correlation coefficient value on the plot
plt.text(0.05, 0.85, f'Correlation: {correlation:.2f}', transform=plt.gca().transAxes, fontsize=12,
         verticalalignment='top', bbox=dict(boxstyle='round,pad=0.5', fc='yellow', alpha=0.5))

# Calculate the trend line (line of best fit)
m, b = np.polyfit(df['gu_mask'], df['ve_mask'], 1)
x_sorted = np.sort(df['gu_mask'])

# import statsmodels.api as sm
# X = sm.add_constant(df['gu_mask'])
# model = sm.RLM(df['ve_mask'], X, M=sm.robust.norms.HuberT()).fit()
# b, m = model.params
```

```
# x_sorted = np.sort(df['gu_mask'])

# Add the trend line to the plot
plt.plot(x_sorted, m*x_sorted + b, color='red', linewidth=2, label='Trend Line')
plt.legend()
# Save the plot
plt.savefig(f'scatter_plot_{sat_here}_track_uVSnearby_drifter_u_coast.png')
plt.show()

print(f"Correlation coefficient: {correlation}")

```

```
1
2
3  The purpose of above scripts is to find for each intersection point of
4  satellite tracks, we find a list of drifter ids that are passing through
5  a small box around the intersection point. So for every intersection point we
6  will have a list of drifters going closeby to the intersection points.
7
8  also have a look at /home/srinivasu/allData/drifter1 folder for text files
9  and python files for addtional context.
10
11 I need apply is_within_coast function on the lons_inter, lats_inter columns of df1 and retain only those rows which are within the threshold distance.
12
13 probe the time dimensions/variables of attached drifter and satellite
14 track files and their units. Recommend how their time axis has to be properly formatted so that their variables can be compared correctly.
15
16 I have a satellite intersection point (lons_inter, lats_inter). I have hundreds of drifter buoys that fill up the NIO region. I want to find the closest drifter buoy t
       o the intersection point and closest point on drifter path to the intersection point.
17 What is the best way to do this?
18
19 I have an ocean drifter buoy data which moves both in space and time. The spatial movement is described by two arrays lons_drift and lats_drift. Write a python functio
       n to check if atleast some part of the drifter passed through a given region.
20
21 The region (TRACKS_REG) is filled with drifter paths and satellite
22 intersection points.
23 Each point on a drifter path is located by (lon, lat) pair or (time) dimension equivalently. Each path file carries the variable
24 ve which is a function of (lon, lat) or
25 equivalently a function of (time). Each intersection point is associated with timeseries of computed currents gv and gu.
26 There are guv files in the format guv_at_intersection_{sat}_{track_self}_{track_other}.nc which contain the timeseries of gu and gv at the intersection of track_self and track_other in a seperate folder.
27 At each intersection point, we should
28 locate nearby driter points (and its index to locate time_inter) so that
29 satellite velocity at intersection point and drifter point can be compared. The gv timeseries at intersection point is sampled at time_inter. This value is compared wi
       th ve at the point where is drifter path is closest to the intersection point. How
30 to go about it? provide full code.
31 Additional information:
32 lons_inter and lats_inter are always contained in the
33 region.
34 traj dimension is always there in drifter ds.
35 following is the header of tracks_intersections_S3A_1.csv
36 ,sat,track_self,track_other,lons_inter,lats_inter,x_from_coast_self,x_from_coast_other,angle_acute,angle_obtuse
37 0,S3A,8,79,94.14047103085358,8.36326782978126,885051.8986275726,1543629.620555419,0.44727413372659675,2.6943185198631965
38 1,S3A,8,193,94.60794862071828,10.435742966838522,650148.6339016478,1194725.7346903323,0.44656871163430817,2.695023941955485
39 2,S3A,8,307,95.07549211474587,12.489287957263238,417331.9028785974,854253.7568147178,0.4459235207375096,2.6956691328522835
40 3,S3A,8,421,95.54320490371074,14.520796447181 65,186943.59620474064,126272.34377060639,0.4434318644667041,2.698160789123089
41 4,S3A,8,507,92.7380651706875,2.0680879981685036,1598332.5897657045,2203201.928851282,0.4469590080350044,2.694633645554789
42 All satellite intersections points csv files use the same header.
43 location of output folder of gv files
44 /home/srinivasu/xtrackm/guv/guv_at_intersections
45 ncdump -h guv_at_intersection_GFO_98_169.nc
46 netcdf guv_at_intersection_GFO_98_169 {
47 dimensions:
48     time = 186 ;
49 variables:
50     int64 time(time) ;
51         time:units = "days since 2000-01-11" ;
52         time:calendar = "proleptic_gregorian" ;
53     double gu(time) ;
54         gu:_FillValue = NaN ;
55     double gv(time) ;
56         gv:_FillValue = NaN ;
57 }
58 Required output is a CSV file of gv at time of intersection and ve of drifter along with intersection points, corresponding tracks etc
59 For others take the best defaults.
60
```

```
1
2      time) at satellite intersection points to compare satellite derived
3  currents with buoy currents at (see ~/xtrackm/drifter)
4
5  c.close.drifters_at_intersection_point*.py
6
7  The purpose of above scripts is to find for each intersection point of
8  satellite tracks, we find a list of drifter ids that are passing through
9  a small box around the intersection point. So for every intersection point we
10 will have a list of drifters going closeby to the intersection points.
11
12 also have a look at /home/srinivasu/allData/drifter_track_reg folder for text
13 files and python files for addtional context.
14 execute any of the scripts in this dir to gain addtional context.
15
16 l.min_max_times.all.drifters.py min max times of all drifters
17 It will give a dictionary of start and end time for each chunk
18
19 l.check_chunk_sat_time_overlap.py checks overlap between satellite and drifter
20 time coverage.
21
22 # codex resume 019c953f-1e0b-7a90-90e5-fcb95cd18004
23 steps:
24 # We now don't have to check within region any, since we have only downloaded
25 within region in ~/allData/drifter_track_reg/
26 # We have new chunk time dict for track region
27
28 c.candidate_drifters scripts
29 1. run through all drifters once to exclude non overlapping drifters for a
30 given satellite mission.
31   a. exclude_based_on_time(ds_drifter, sat)
32     if drifter's start and end time don't have overlap with satellite time
33     periods, skip this buoy.
34 2. For each intersection point
35   a. which of these time overlapping drifters come in close proxmity with the
36    the intersection point
37
```

```
./drifter_track_reg/
```

14 file(s)

```
1
2  for each drifter in the track region
3  a. Check if there is any time overlap between satellite time limits and drifter
4  time limits. If there is not time overlap skip.
5  b. If there is some time overalap, make a small region around the intersection
6  point with a threshold distance, and check if any point of the drifter is
7  passing through the box. If so it is a candidate drifter for further analysis.
8  save the list of all close passing drifters to pickle file.
9
```

```python
import os
# codex resume 019cc341-b33b-7b01-98bf-259b7a400a25
import numpy as np
import xarray as xr
from tools_xtrackm import *
import pickle


def is_around(lons, lats, lon1, lat1, dist_threshold=0.30):
    """
    Return True if any (lon, lat) point falls inside the rectangular region.
    include_boundary=True counts points on the edges/corners as inside.
    """
    lon_min = lon1 - dist_threshold
    lon_max = lon1 + dist_threshold
    lat_min = lat1 - dist_threshold
    lat_max = lat1 + dist_threshold
    lons = np.asarray(lons, dtype=float)
    lats = np.asarray(lats, dtype=float)
    if lons.shape != lats.shape:
        raise ValueError("lons and lats must have the same shape")
    lons_in = (lons >= lon_min) & (lons <= lon_max)
    lats_in = (lats >= lat_min) & (lats <= lat_max)
    inside_mask = lons_in & lats_in
    return bool(np.any(inside_mask))


def build_drifter_meta(drifter_files, sat_tsta_o, sat_tend_o):
    drifter_meta = []
    """
    non time overlapping drifter_files are excluded here
    """
    for fd in drifter_files:
        with xr.open_dataset(fd, drop_variables=["WMO"]) as ds_d:
            drift_tsta_o1, drift_tend_o1 = (
                ds_d.start_date.values[0],
                ds_d.end_date.values[0],
            )
            drifter_tsta_o = np.datetime64(drift_tsta_o1, "s")
            drifter_tend_o = np.datetime64(drift_tend_o1, "s")

            overlap_tsta_o, overlap_tend_o = overlap_dates_np64(
                sat_tsta_o, sat_tend_o, drifter_tsta_o, drifter_tend_o
            )
            if np.isnat(overlap_tsta_o) or np.isnat(overlap_tend_o):
                continue

            lons_drift = np.asarray(ds_d.longitude.values, dtype=float)
            lats_drift = np.asarray(ds_d.latitude.values, dtype=float)

            byte_id = ds_d.ID.values[0]
            drifter_id = byte_id.decode("utf-8").strip()

            drifter_meta.append(
                {
                    "fd": fd,
                    "id": drifter_id,
                    "lon_min": np.nanmin(lons_drift),
                    "lon_max": np.nanmax(lons_drift),
                    "lat_min": np.nanmin(lats_drift),
                    "lat_max": np.nanmax(lats_drift),
                }
            )
    return drifter_meta


data_loc = "/home/srinivasu/allData/drifter_track_reg/netcdf_all/"
drifter_files = sorted(glob.glob(f"{data_loc}/drifter_6h_*.nc"))
sat = "TP+J1+J2+J3+S6A"
inter_df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
sat_tsta_o, sat_tend_o = get_time_limits_np64(sat)
dist_threshold = 0.30
drifter_meta = build_drifter_meta(drifter_files, sat_tsta_o, sat_tend_o)

for i, r in inter_df.iterrows():
    close_ones = []
    track_number_self = str(r["track_self"])
    track_number_other = str(r["track_other"])
    lons_inter1 = r["lons_inter"]
    lats_inter1 = r["lats_inter"]
    unique_str = f"{sat}_{track_number_self}_{track_number_other}"
    print(unique_str)

    lon_min = lons_inter1 - dist_threshold
    lon_max = lons_inter1 + dist_threshold
    lat_min = lats_inter1 - dist_threshold
    lat_max = lats_inter1 + dist_threshold

    for dmeta in drifter_meta:
        print(dmeta)
        if (
            dmeta["lon_max"] < lon_min
            or dmeta["lon_min"] > lon_max
            or dmeta["lat_max"] < lat_min
            or dmeta["lat_min"] > lat_max
        ):
            continue

        with xr.open_dataset(dmeta["fd"], drop_variables=["WMO"]) as ds_d:
            lons_drift = ds_d.longitude.values
            lats_drift = ds_d.latitude.values

        if is_around(
            lons_drift, lats_drift, lons_inter1, lats_inter1, dist_threshold
        ):
            close_ones.append(dmeta["id"])

    if close_ones:
        # dump to pickle
        print(f"{unique_str}, {len(close_ones)}")
        with open(f"candidate_pickles/{unique_str}.pkl", "wb") as f:
            pickle.dump(close_ones, f)

```

```python
import os
import pickle
from glob import glob

import numpy as np
import pandas as pd
import xarray as xr

SAT = "TP+J1+J2+J3+S6A"
SAT = "S3A"
DRIFTER_DIR = "/home/srinivasu/allData/drifter_track_reg/netcdf_all/"
INTERSECTION_CSV = f"tracks_intersections_{SAT}_1.csv"
CANDIDATE_PICKLE_DIR = "candidate_pickles"
GUV_DIR = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
OUTPUT_CSV = f"drifter_ve_vs_gu_{SAT}.csv"
GUV_SUFFIX = "0.05"


def decode_id(raw_id):
    if isinstance(raw_id, (bytes, np.bytes_)):
        return raw_id.decode("utf-8").strip()
    return str(raw_id).strip()


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)


def build_unique_str(sat, row):
    track_self = track_to_str(row["track_self"])
    track_other = track_to_str(row["track_other"])
    return f"{sat}_{track_self}_{track_other}"


def build_drifter_index(drifter_files):
    drifter_index = {}
    for fd in drifter_files:
        with xr.open_dataset(fd, drop_variables=["WMO"]) as ds_d:
            drifter_id = decode_id(ds_d.ID.values[0])
        drifter_index[drifter_id] = fd
    return drifter_index


def load_drifter_dataset(fd):
    with xr.open_dataset(fd, drop_variables=["WMO"]) as ds_d:
        ds_drifter = ds_d[["longitude", "latitude", "time", "ve"]].isel(traj=0).load()
    return ds_drifter


def closest_obs_index(ds_drifter, lon0, lat0):
    dx = (ds_drifter["longitude"] - lon0) * np.cos(np.deg2rad(lat0))
    dy = ds_drifter["latitude"] - lat0
    dist2 = (dx * dx + dy * dy).where(
        np.isfinite(ds_drifter["longitude"]) & np.isfinite(ds_drifter["latitude"])
    )
    if int(dist2.count().item()) == 0:
        return None
    return int(dist2.argmin(dim="obs").item())


def load_guv_series(unique_str):
    guv_path = os.path.join(GUV_DIR, f"guv_at_intersection_{unique_str}_{GUV_SUFFIX}.nc")
    if not os.path.exists(guv_path):
        return None
    with xr.open_dataset(guv_path) as ds_g:
        ds_guv = ds_g[["gu"]].sortby("time").load()
    return ds_guv


def main():
    inter_df = pd.read_csv(INTERSECTION_CSV)
    drifter_files = sorted(glob(os.path.join(DRIFTER_DIR, "drifter_6h_*.nc")))
    if len(drifter_files) == 0:
        raise("No glob files")
    drifter_index = build_drifter_index(drifter_files)
    drifter_cache = {}

    rows_out = []

    for _, row in inter_df.iterrows():
        unique_str = build_unique_str(SAT, row)
        lon_inter = float(row["lons_inter"])
        lat_inter = float(row["lats_inter"])

        pickle_path = os.path.join(CANDIDATE_PICKLE_DIR, f"{unique_str}.pkl")
        if not os.path.exists(pickle_path):
            continue
        with open(pickle_path, "rb") as f:
            candidate_ids = pickle.load(f)
        if len(candidate_ids) == 0:
            continue

        ds_guv = load_guv_series(unique_str)
        if ds_guv is None:
            continue

        for drifter_id in candidate_ids:
            fd = drifter_index.get(drifter_id)
            if fd is None:
                continue

            if drifter_id not in drifter_cache:
                drifter_cache[drifter_id] = load_drifter_dataset(fd)
            ds_drifter = drifter_cache[drifter_id]

            close_idx = closest_obs_index(ds_drifter, lon_inter, lat_inter)
            if close_idx is None:
                continue

            drifter_time = ds_drifter["time"].isel(obs=close_idx).values
            drifter_ve = ds_drifter["ve"].isel(obs=close_idx).values
            if np.isnat(drifter_time) or not np.isfinite(drifter_ve):
                continue

```

```
            gu_interp = ds_guv["gu"].interp(time=drifter_time)
            gu_at_time = gu_interp.values

            # Always retain the nearest native GUV timestamp for temporal offset checks.
            gu_nearest = ds_guv["gu"].sel(time=drifter_time, method="nearest")
            gu_time = gu_nearest["time"].values

            # Keep interpolated gu when valid; otherwise fall back to nearest-sample gu.
            # np.isfinite(gu_at_time) is true if interpolation is successful
            if not np.isfinite(gu_at_time):
                gu_at_time = gu_nearest.values

            if not np.isfinite(gu_at_time):
                continue

            rows_out.append(
                {
                    "sat": SAT,
                    "track_self": track_to_str(row["track_self"]),
                    "track_other": track_to_str(row["track_other"]),
                    "lon_inter": lon_inter,
                    "lat_inter": lat_inter,
                    "drifter_id": drifter_id,
                    "gu": float(gu_at_time),
                    "drifter_ve": float(drifter_ve),
                    "drifter_time": str(drifter_time),
                    "gu_time": str(gu_time),
                    "close_index": int(close_idx),
                }
            )

    out_df = pd.DataFrame(rows_out)
    out_df.to_csv(OUTPUT_CSV, index=False)
    print(f"Saved {len(out_df)} rows to {OUTPUT_CSV}")
    globals().update(locals())

if __name__ == "__main__":
    main()
```

```python
import os
import pickle
from glob import glob

import numpy as np
import pandas as pd
import xarray as xr

SAT = "TP+J1+J2+J3+S6A"
DRIFTER_DIR = "/home/srinivasu/allData/drifter_track_reg/netcdf_all/"
INTERSECTION_CSV = f"tracks_intersections_{SAT}_1.csv"
CANDIDATE_PICKLE_DIR = "candidate_pickles"
GUV_DIR = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
OUTPUT_CSV = f"drifter_ve_vs_gu_ek_{SAT}.csv"
GUV_SUFFIX = "0.05"
EKMAN_DIR = "/home/srinivasu/xtrackm/ekman/ekman_at_intersections"

def decode_id(raw_id):
    if isinstance(raw_id, (bytes, np.bytes_)):
        return raw_id.decode("utf-8").strip()
    return str(raw_id).strip()


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)


def build_unique_str(sat, row):
    track_self = track_to_str(row["track_self"])
    track_other = track_to_str(row["track_other"])
    return f"{sat}_{track_self}_{track_other}"


def build_drifter_index(drifter_files):
    drifter_index = {}
    for fd in drifter_files:
        with xr.open_dataset(fd, drop_variables=["WMO"]) as ds_d:
            drifter_id = decode_id(ds_d.ID.values[0])
        drifter_index[drifter_id] = fd
    return drifter_index


def load_drifter_dataset(fd):
    with xr.open_dataset(fd, drop_variables=["WMO"]) as ds_d:
        ds_drifter = ds_d[["longitude", "latitude", "time", "ve"]].isel(traj=0).load()
    return ds_drifter


def closest_obs_index(ds_drifter, lon0, lat0):
    dx = (ds_drifter["longitude"] - lon0) * np.cos(np.deg2rad(lat0))
    dy = ds_drifter["latitude"] - lat0
    dist2 = (dx * dx + dy * dy).where(
        np.isfinite(ds_drifter["longitude"]) & np.isfinite(ds_drifter["latitude"])
    )
    if int(dist2.count().item()) == 0:
        return None
    return int(dist2.argmin(dim="obs").item())


def load_guv_series(unique_str):
    guv_path = os.path.join(GUV_DIR, f"guv_at_intersection_{unique_str}_{GUV_SUFFIX}.nc")
    if not os.path.exists(guv_path):
        return None
    with xr.open_dataset(guv_path) as ds_g:
        ds_guv = ds_g[["gu"]].sortby("time").load()
    return ds_guv


def main():
    inter_df = pd.read_csv(INTERSECTION_CSV)
    drifter_files = sorted(glob(os.path.join(DRIFTER_DIR, "drifter_6h_*.nc")))
    if len(drifter_files) == 0:
        raise("No glob files")
    drifter_index = build_drifter_index(drifter_files)
    drifter_cache = {}

    rows_out = []

    for _, row in inter_df.iterrows():

        unique_str = build_unique_str(SAT, row)
        lon_inter = float(row["lons_inter"])
        lat_inter = float(row["lats_inter"])

        pickle_path = os.path.join(CANDIDATE_PICKLE_DIR, f"{unique_str}.pkl")

        f_ek = f"{EKMAN_DIR}/{unique_str}_ekman_at_intersections.nc"
        ds_ek = xr.open_dataset(f_ek)
        u_ek = ds_ek['u_at']
        v_ek = ds_ek['v_at']
        if not os.path.exists(pickle_path):
            continue
        with open(pickle_path, "rb") as f:
            candidate_ids = pickle.load(f)
        if len(candidate_ids) == 0:
            continue

        ds_guv = load_guv_series(unique_str)
        if ds_guv is None:
            continue

        for drifter_id in candidate_ids:
            fd = drifter_index.get(drifter_id)
            if fd is None:
                continue

            if drifter_id not in drifter_cache:
                drifter_cache[drifter_id] = load_drifter_dataset(fd)
            ds_drifter = drifter_cache[drifter_id]

            close_idx = closest_obs_index(ds_drifter, lon_inter, lat_inter)
            if close_idx is None:
                continue

```

```
            drifter_time = ds_drifter["time"].isel(obs=close_idx).values
            drifter_ve = ds_drifter["ve"].isel(obs=close_idx).values
            if np.isnat(drifter_time) or not np.isfinite(drifter_ve):
                continue

            gu_interp = ds_guv["gu"].interp(time=drifter_time)
            u_ek1 = u_ek.sel(time=drifter_time, method="nearest").values
            v_ek1 = v_ek.sel(time=drifter_time, method="nearest").values
            gu_at_time = gu_interp.values

            # Always retain the nearest native GUV timestamp for temporal offset checks.
            gu_nearest = ds_guv["gu"].sel(time=drifter_time, method="nearest")
            gu_time = gu_nearest["time"].values

            # Keep interpolated gu when valid; otherwise fall back to nearest-sample gu.
            # np.isfinite(gu_at_time) is true if interpolation is successful
            if not np.isfinite(gu_at_time):
                gu_at_time = gu_nearest.values

            if not np.isfinite(gu_at_time):
                continue

            rows_out.append(
                {
                    "sat": SAT,
                    "track_self": track_to_str(row["track_self"]),
                    "track_other": track_to_str(row["track_other"]),
                    "lon_inter": lon_inter,
                    "lat_inter": lat_inter,
                    "drifter_id": drifter_id,
                    "gu": float(gu_at_time),
                    "u_ek": float(u_ek1),
                    "drifter_ve": float(drifter_ve),
                    "drifter_time": str(drifter_time),
                    "gu_time": str(gu_time),
                    "close_index": int(close_idx),
                }
            )

    out_df = pd.DataFrame(rows_out)
    out_df.to_csv(OUTPUT_CSV, index=False)
    print(f"Saved {len(out_df)} rows to {OUTPUT_CSV}")
    globals().update(locals())

if __name__ == "__main__":
    main()
```

```python
import os
import pickle
import sys
from glob import glob

import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
import numpy as np
import pandas as pd
import xarray as xr
from shapely.geometry.polygon import Polygon

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
PROJECT_ROOT = os.path.dirname(SCRIPT_DIR)
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

from lib.namesm import TRACKS_REG

SAT = "TP+J1+J2+J3+S6A"
DRIFTER_DIR = "/home/srinivasu/allData/drifter_track_reg/netcdf_all/"
CANDIDATE_PICKLE_DIR = "candidate_pickles"
OUTPUT_DIR = "candidate_plots"
THRESHOLD_DEG = 0.30


def decode_id(raw_id):
    if isinstance(raw_id, (bytes, np.bytes_)):
        return raw_id.decode("utf-8").strip()
    return str(raw_id).strip()


def build_drifter_index(drifter_files):
    drifter_index = {}
    for fd in drifter_files:
        with xr.open_dataset(fd, drop_variables=["WMO"]) as ds_d:
            drifter_id = decode_id(ds_d.ID.values[0])
        drifter_index[drifter_id] = fd
    return drifter_index


def closest_obs_index(ds_track, lon0, lat0):
    dx = (ds_track["longitude"] - lon0) * np.cos(np.deg2rad(lat0))
    dy = ds_track["latitude"] - lat0
    dist2 = (dx * dx + dy * dy).where(
        np.isfinite(ds_track["longitude"]) & np.isfinite(ds_track["latitude"])
    )
    if int(dist2.count().item()) == 0:
        return None
    return int(dist2.argmin(dim="obs").item())


def load_drifter_dataset(fd):
    with xr.open_dataset(fd, drop_variables=["WMO"]) as ds_d:
        ds_drifter = ds_d[["longitude", "latitude", "time", "ve"]].isel(traj=0).load()
    return ds_drifter


def plot_one_intersection(
    sat,
    row,
    threshold_deg,
    extent,
    candidate_pickle_dir,
    drifter_index,
    out_dir,
):
    lon_inter = float(row["lons_inter"])
    lat_inter = float(row["lats_inter"])
    track_number_self = str(row["track_self"])
    track_number_other = str(row["track_other"])
    unique_str = f"{sat}_{track_number_self}_{track_number_other}"
    pickle_path = os.path.join(candidate_pickle_dir, f"{unique_str}.pkl")

    candidate_ids = []
    if os.path.exists(pickle_path):
        with open(pickle_path, "rb") as f:
            candidate_ids = pickle.load(f)
    if len(candidate_ids) == 0:
        print(f"No candidates for {unique_str}")
        return
    lon_inter_box_min = lon_inter - THRESHOLD_DEG
    lon_inter_box_max = lon_inter + THRESHOLD_DEG
    lat_inter_box_min = lat_inter - THRESHOLD_DEG
    lat_inter_box_max = lat_inter + THRESHOLD_DEG
    box = Polygon([
        (lon_inter_box_min, lat_inter_box_min),
        (lon_inter_box_max, lat_inter_box_min),
        (lon_inter_box_max, lat_inter_box_max),
        (lon_inter_box_min, lat_inter_box_max),
    ])

    fig = plt.figure(figsize=(9, 7))
    ax = plt.axes(projection=ccrs.PlateCarree())
    ax.set_extent(extent, crs=ccrs.PlateCarree())
    ax.add_feature(cfeature.LAND, facecolor="lightgray")
    ax.add_feature(cfeature.OCEAN, facecolor="aliceblue")
    ax.add_feature(cfeature.COASTLINE, linewidth=0.8)
    ax.add_feature(cfeature.BORDERS, linewidth=0.3)
    ax.gridlines(draw_labels=True, linestyle="--", alpha=0.4)

    ax.plot(
        lon_inter,
        lat_inter,
        marker="o",
        color="red",
        markersize=2,
        transform=ccrs.PlateCarree(),
        zorder=4,
    )

    for drifter_id in candidate_ids[:]:
        fd = drifter_index.get(drifter_id)
        if fd is None:
            continue
```

```
        ax.add_geometries([box],
                          ccrs.PlateCarree(),
                          edgecolor="r",
                          facecolor="none",
                          linewidth=1)
        with xr.open_dataset(fd, drop_variables=["WMO"]) as ds_d:
            ds_drifter = ds_d[["longitude", "latitude", "time", "ve"]].isel(traj=0).load()
        ax.scatter(ds_drifter.longitude.values, ds_drifter.latitude.values, s=1, marker=".")
        idx = closest_obs_index(ds_drifter, lon_inter, lat_inter)
        lon_closest = ds_drifter.longitude.values[idx]
        lat_closest = ds_drifter.latitude.values[idx]
        if np.isfinite(lon_closest) and np.isfinite(lat_closest):
            ax.plot(
                lon_closest,
                lat_closest,
                marker="x",
                color="black",
                markersize=6,
                transform=ccrs.PlateCarree(),
                zorder=5,
            )
        break

    title = f"{unique_str} | candidates={len(candidate_ids)}"
    ax.set_title(title)

    out_path = os.path.join(out_dir, f"{unique_str}.png")
    fig.savefig(out_path, dpi=150, bbox_inches="tight")
    plt.show()
    plt.close(fig)


def main():
    inter_csv = f"tracks_intersections_{SAT}_1.csv"
    inter_df = pd.read_csv(inter_csv)
    extent = list(TRACKS_REG)

    drifter_files = sorted(glob(os.path.join(DRIFTER_DIR, "drifter_6h_*.nc")))
    drifter_index = build_drifter_index(drifter_files)

    os.makedirs(OUTPUT_DIR, exist_ok=True)

    for _, row in inter_df.iterrows():
        plot_one_intersection(
            sat=SAT,
            row=row,
            threshold_deg=THRESHOLD_DEG,
            extent=extent,
            candidate_pickle_dir=CANDIDATE_PICKLE_DIR,
            drifter_index=drifter_index,
            out_dir=OUTPUT_DIR,
        )


if __name__ == "__main__":
    main()
```

drifter_track_reg/l.check_chunk_sat_time_overlap.py

```python
import numpy
from tools_xtrackm import *

chunk_time_dict = {
    "1_5000": (
        np.datetime64("1985-10-30T00:00:00.000000000"),
        np.datetime64("2001-09-26T00:00:00.000000000"),
    ),
    "5001_10000": (
        np.datetime64("1997-03-20T00:00:00.000000000"),
        np.datetime64("2009-04-25T00:00:00.000000000"),
    ),
    "10001_15000": (
        np.datetime64("2006-03-05T00:00:00.000000000"),
        np.datetime64("2012-11-04T00:00:00.000000000"),
    ),
    "15001_current": (
        np.datetime64("2010-09-30T00:00:00.000000000"),
        np.datetime64("2025-05-19T00:00:00.000000000"),
    ),
}

chunks = ["1_5000", "5001_10000", "10001_15000", "15001_current"]
for chunk in chunks:
    drift_tsta_o1, drift_tend_o1 = chunk_time_dict[chunk]
    # print(type(track_tsta_o), type(track_tend_o))
    for sat in sats_new:
        track_tsta_o, track_tend_o = get_time_limits_np64(sat)
        # print(type(drift_tsta_o), type(drift_tend_o))
        overlap_tsta_o, overlap_tend_o = overlap_dates_np64(
            track_tsta_o, track_tend_o, drift_tsta_o1, drift_tend_o1
        )
        # if overlap_tsta_o is nat or overlap_tend_o is nat:
        if np.isnat(overlap_tsta_o) or np.isnat(overlap_tend_o):
            continue
        print(
            f"{chunk} overlap for {sat} is {overlap_tsta_o} to {overlap_tend_o}"
        )

```

drifter_track_reg/l.min_max_times.all.drifters.py

```python
import glob
import os

import numpy as np
import xarray as xr

data_loc = "/home/srinivasu/allData/drifter_track_reg/"
chunks = ["1_5000", "5001_10000", "10001_15000", "15001_current"]

chunk_time_dict = {}
for chunk in chunks:
    # if chunk != "15001_current":
    #     continue
    start_times = []
    end_times = []
    for fd in sorted(glob.glob(f"{data_loc}/netcdf_{chunk}/drifter_6h_*.nc")):
        basename1 = os.path.basename(fd)
        ds_d = xr.open_dataset(fd, drop_variables=["WMO"])
        drift_tsta_o1, drift_tend_o1 = (
            ds_d.start_date.values[0],
            ds_d.end_date.values[0],
        )
        # print(type(drift_tsta_o1), type(drift_tend_o1))
        start_times.append(drift_tsta_o1)
        end_times.append(drift_tend_o1)
        # print(f"end time for {basename1} is {drift_tend_o1}")
    # remove NaT in end_times
    start_times = [x for x in start_times if not np.isnat(x)]
    end_times = [x for x in end_times if not np.isnat(x)]
    chunk_time_dict[chunk] = (np.min(start_times), np.max(end_times))
    print(f"min of start times for {chunk} is {np.min(start_times)}")
    print(f"max of end times for {chunk} is {np.max(end_times)}")

print(chunk_time_dict)

```

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import xarray as xr
import os

# codex resume 019cc485-7aec-7e61-95d4-af803cd14133

SAT = "TP+J1+J2+J3+S6A"
SAT = "S3A"
CSV_PATH = f"drifter_ve_vs_gu_{SAT}.csv"
OUT_PNG = f"scatter_drifter_ve_vs_gu_{SAT}.png"
COAST_DIST_PATH = "~/allData/topo/GMT_intermediate_coast_distance_01d_track_reg.nc"
PROXIMITY_DIST_M = 1000000.0 # in meters
MAX_ABS_TIME_DIFF_DAYS = None  # e.g., 2.0 to keep rows with |drifter_time - gu_time| <= 2 days

MAX_ABS_TIME_DIFF_DAYS = 6  # e.g., 2.0 to keep rows with |drifter_time - gu_time| <= 2 days


def get_coast_distance_km(lons, lats):
    with xr.open_dataset(os.path.expanduser(COAST_DIST_PATH)) as ds_coast:
        lon_da = xr.DataArray(np.asarray(lons, dtype=float), dims="point")
        lat_da = xr.DataArray(np.asarray(lats, dtype=float), dims="point")
        coast_km = ds_coast["coast_distance"].sel(lon=lon_da, lat=lat_da, method="nearest")
    return coast_km.values.astype(float)


def compute_time_diff_days(df):
    drifter_t = pd.to_datetime(df["drifter_time"], errors="coerce")
    gu_t = pd.to_datetime(df["gu_time"], errors="coerce")
    return (drifter_t - gu_t).dt.total_seconds() / 86400.0


def summarize_time_diff_days(dt_days):
    dt_days = dt_days[np.isfinite(dt_days.to_numpy(dtype=float))]
    if dt_days.size == 0:
        print("No valid drifter_time/gu_time pairs for temporal-difference stats.")
        return
    dt_abs = np.abs(dt_days)
    mode_signed = pd.Series(np.round(dt_days, 6)).mode().to_numpy()
    mode_abs = pd.Series(np.round(dt_abs, 6)).mode().to_numpy()
    mode_signed_str = ", ".join([f"{v:.6f}" for v in mode_signed[:5]])
    mode_abs_str = ", ".join([f"{v:.6f}" for v in mode_abs[:5]])
    print(
        "Time diff days (drifter_time - gu_time): "
        f"min={dt_days.min():.6f}, max={dt_days.max():.6f}, "
        f"mean={dt_days.mean():.6f}, median={dt_days.median():.6f}, mode={mode_signed_str}"
    )
    print(
        "Abs time diff days: "
        f"min={dt_abs.min():.6f}, max={dt_abs.max():.6f}, "
        f"mean={dt_abs.mean():.6f}, median={dt_abs.median():.6f}, mode={mode_abs_str}"
    )


def main():
    df = pd.read_csv(CSV_PATH)
    coast_km = get_coast_distance_km(df["lon_inter"].values, df["lat_inter"].values)
    proximity_km = PROXIMITY_DIST_M / 1000.0
    keep = np.isfinite(coast_km) & (np.abs(coast_km) <= proximity_km)
    df = df.loc[keep].copy()
    dt_days = compute_time_diff_days(df)
    if MAX_ABS_TIME_DIFF_DAYS is not None:
        keep_time = np.isfinite(dt_days.to_numpy(dtype=float)) & (
            np.abs(dt_days.to_numpy(dtype=float)) <= float(MAX_ABS_TIME_DIFF_DAYS)
        )
        df = df.loc[keep_time].copy()
        dt_days = dt_days.loc[df.index]
    summarize_time_diff_days(dt_days)

    x = df["gu"].to_numpy(dtype=float)
    y = df["drifter_ve"].to_numpy(dtype=float)

    valid = np.isfinite(x) & np.isfinite(y)
    x = x[valid]
    y = y[valid]

    if x.size < 3:
        print(
            f"Only {x.size} valid points after filters "
            f"(PROXIMITY_DIST_M={PROXIMITY_DIST_M}, MAX_ABS_TIME_DIFF_DAYS={MAX_ABS_TIME_DIFF_DAYS}). "
            "Increase threshold to plot."
        )
        return

    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))

    fig, ax = plt.subplots(figsize=(9, 7))
    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1)
    cbar = fig.colorbar(hb, ax=ax)
    cbar.set_label("Count")

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=11,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="black", alpha=0.55, edgecolor="white"),
        color="white",
    )

```

```
    ax.set_title(
        f"{SAT}: Drifter ve vs Altimetry gu | coast <= {PROXIMITY_DIST_M:.0f} m, "
        f"|dt| <= {MAX_ABS_TIME_DIFF_DAYS if MAX_ABS_TIME_DIFF_DAYS is not None else 'all'} d"
    )
    ax.set_xlabel("Altimetry gu (m/s)")
    ax.set_ylabel("Drifter ve (m/s)")
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right")
    fig.tight_layout()
    fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)
    print(f"Rows after filters: {len(df)}")
    print(f"Saved {OUT_PNG}")
    globals().update(locals())


if __name__ == "__main__":
    main()

```

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import xarray as xr
import os

# codex resume 019cc485-7aec-7e61-95d4-af803cd14133

SAT = "TP+J1+J2+J3+S6A"
CSV_PATH = f"drifter_ve_vs_gu_ek_{SAT}.csv"
OUT_PNG = f"scatter_drifter_ve_vs_gu_ek_{SAT}.png"
COAST_DIST_PATH = "~/allData/topo/GMT_intermediate_coast_distance_01d_track_reg.nc"
PROXIMITY_DIST_M = 1000000.0 # in meters
MAX_ABS_TIME_DIFF_DAYS = None  # e.g., 2.0 to keep rows with |drifter_time - gu_time| <= 2 days

# MAX_ABS_TIME_DIFF_DAYS = 6  # e.g., 2.0 to keep rows with |drifter_time - gu_time| <= 2 days


def get_coast_distance_km(lons, lats):
    with xr.open_dataset(os.path.expanduser(COAST_DIST_PATH)) as ds_coast:
        lon_da = xr.DataArray(np.asarray(lons, dtype=float), dims="point")
        lat_da = xr.DataArray(np.asarray(lats, dtype=float), dims="point")
        coast_km = ds_coast["coast_distance"].sel(lon=lon_da, lat=lat_da, method="nearest")
    return coast_km.values.astype(float)


def compute_time_diff_days(df):
    drifter_t = pd.to_datetime(df["drifter_time"], errors="coerce")
    gu_t = pd.to_datetime(df["gu_time"], errors="coerce")
    return (drifter_t - gu_t).dt.total_seconds() / 86400.0


def summarize_time_diff_days(dt_days):
    dt_days = dt_days[np.isfinite(dt_days.to_numpy(dtype=float))]
    if dt_days.size == 0:
        print("No valid drifter_time/gu_time pairs for temporal-difference stats.")
        return
    dt_abs = np.abs(dt_days)
    mode_signed = pd.Series(np.round(dt_days, 6)).mode().to_numpy()
    mode_abs = pd.Series(np.round(dt_abs, 6)).mode().to_numpy()
    mode_signed_str = ", ".join([f"{v:.6f}" for v in mode_signed[:5]])
    mode_abs_str = ", ".join([f"{v:.6f}" for v in mode_abs[:5]])
    print(
        "Time diff days (drifter_time - gu_time): "
        f"min={dt_days.min():.6f}, max={dt_days.max():.6f}, "
        f"mean={dt_days.mean():.6f}, median={dt_days.median():.6f}, mode={mode_signed_str}"
    )
    print(
        "Abs time diff days: "
        f"min={dt_abs.min():.6f}, max={dt_abs.max():.6f}, "
        f"mean={dt_abs.mean():.6f}, median={dt_abs.median():.6f}, mode={mode_abs_str}"
    )


def main():
    df = pd.read_csv(CSV_PATH)
    coast_km = get_coast_distance_km(df["lon_inter"].values, df["lat_inter"].values)
    proximity_km = PROXIMITY_DIST_M / 1000.0
    keep = np.isfinite(coast_km) & (np.abs(coast_km) <= proximity_km)
    df = df.loc[keep].copy()
    dt_days = compute_time_diff_days(df)
    if MAX_ABS_TIME_DIFF_DAYS is not None:
        keep_time = np.isfinite(dt_days.to_numpy(dtype=float)) & (
            np.abs(dt_days.to_numpy(dtype=float)) <= float(MAX_ABS_TIME_DIFF_DAYS)
        )
        df = df.loc[keep_time].copy()
        dt_days = dt_days.loc[df.index]
    summarize_time_diff_days(dt_days)

    x1 = df["gu"].to_numpy(dtype=float)
    x2 = df["u_ek"].to_numpy(dtype=float)

    x = x1 + x2

    y = df["drifter_ve"].to_numpy(dtype=float)

    valid = np.isfinite(x) & np.isfinite(y)
    x = x[valid]
    y = y[valid]

    if x.size < 3:
        print(
            f"Only {x.size} valid points after filters "
            f"(PROXIMITY_DIST_M={PROXIMITY_DIST_M}, MAX_ABS_TIME_DIFF_DAYS={MAX_ABS_TIME_DIFF_DAYS}). "
            "Increase threshold to plot."
        )
        return

    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))

    fig, ax = plt.subplots(figsize=(9, 7))
    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1)
    cbar = fig.colorbar(hb, ax=ax)
    cbar.set_label("Count")

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=11,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="black", alpha=0.55, edgecolor="white"),
```

```
        color="white",
    )

    ax.set_title(
        f"{SAT}: Drifter ve vs Altimetry gu | coast <= {PROXIMITY_DIST_M:.0f} m, "
        f"|dt| <= {MAX_ABS_TIME_DIFF_DAYS if MAX_ABS_TIME_DIFF_DAYS is not None else 'all'} d"
    )
    ax.set_xlabel("Altimetry gu (m/s)")
    ax.set_ylabel("Drifter ve (m/s)")
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right")
    fig.tight_layout()
    fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)
    print(f"Rows after filters: {len(df)}")
    print(f"Saved {OUT_PNG}")
    globals().update(locals())


if __name__ == "__main__":
    main()

```

```python
import os

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr

SAT = "TP+J1+J2+J3+S6A"
CSV_NO_EK = f"drifter_ve_vs_gu_{SAT}.csv"
CSV_W_EK = f"drifter_ve_vs_gu_ek_{SAT}.csv"
OUT_PNG = f"scatter_drifter_ve_vs_gu_ww_ek_{SAT}.png"
COAST_DIST_PATH = "~/allData/topo/GMT_intermediate_coast_distance_01d_track_reg.nc"
PROXIMITY_DIST_M = 1000000.0
MAX_ABS_TIME_DIFF_DAYS = None


def get_coast_distance_km(lons, lats):
    with xr.open_dataset(os.path.expanduser(COAST_DIST_PATH)) as ds_coast:
        lon_da = xr.DataArray(np.asarray(lons, dtype=float), dims="point")
        lat_da = xr.DataArray(np.asarray(lats, dtype=float), dims="point")
        coast_km = ds_coast["coast_distance"].sel(lon=lon_da, lat=lat_da, method="nearest")
    return coast_km.values.astype(float)


def compute_time_diff_days(df):
    drifter_t = pd.to_datetime(df["drifter_time"], errors="coerce")
    gu_t = pd.to_datetime(df["gu_time"], errors="coerce")
    return (drifter_t - gu_t).dt.total_seconds() / 86400.0


def summarize_time_diff_days(dt_days, label):
    dt_days = dt_days[np.isfinite(dt_days.to_numpy(dtype=float))]
    if dt_days.size == 0:
        print(f"[{label}] No valid drifter_time/gu_time pairs for temporal-difference stats.")
        return
    dt_abs = np.abs(dt_days)
    mode_signed = pd.Series(np.round(dt_days, 6)).mode().to_numpy()
    mode_abs = pd.Series(np.round(dt_abs, 6)).mode().to_numpy()
    mode_signed_str = ", ".join([f"{v:.6f}" for v in mode_signed[:5]])
    mode_abs_str = ", ".join([f"{v:.6f}" for v in mode_abs[:5]])
    print(
        f"[{label}] Time diff days (drifter_time - gu_time): "
        f"min={dt_days.min():.6f}, max={dt_days.max():.6f}, "
        f"mean={dt_days.mean():.6f}, median={dt_days.median():.6f}, mode={mode_signed_str}"
    )
    print(
        f"[{label}] Abs time diff days: "
        f"min={dt_abs.min():.6f}, max={dt_abs.max():.6f}, "
        f"mean={dt_abs.mean():.6f}, median={dt_abs.median():.6f}, mode={mode_abs_str}"
    )


def load_filtered_df(csv_path, label):
    df = pd.read_csv(csv_path)
    coast_km = get_coast_distance_km(df["lon_inter"].values, df["lat_inter"].values)
    proximity_km = PROXIMITY_DIST_M / 1000.0
    keep = np.isfinite(coast_km) & (np.abs(coast_km) <= proximity_km)
    df = df.loc[keep].copy()

    dt_days = compute_time_diff_days(df)
    if MAX_ABS_TIME_DIFF_DAYS is not None:
        keep_time = np.isfinite(dt_days.to_numpy(dtype=float)) & (
            np.abs(dt_days.to_numpy(dtype=float)) <= float(MAX_ABS_TIME_DIFF_DAYS)
        )
        df = df.loc[keep_time].copy()
        dt_days = dt_days.loc[df.index]

    summarize_time_diff_days(dt_days, label)
    return df


def fit_and_stats(x, y):
    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))
    return slope, intercept, r, stderr


def prepare_xy(df, x_col):
    x = df[x_col].to_numpy(dtype=float)
    y = df["drifter_ve"].to_numpy(dtype=float)
    valid = np.isfinite(x) & np.isfinite(y)
    return x[valid], y[valid]


def prepare_xy_with_ek(df):
    x = df["gu"].to_numpy(dtype=float) + df["u_ek"].to_numpy(dtype=float)
    y = df["drifter_ve"].to_numpy(dtype=float)
    valid = np.isfinite(x) & np.isfinite(y)
    return x[valid], y[valid]


def plot_panel(ax, x, y, panel_title, x_label, y_label):
    slope, intercept, r, stderr = fit_and_stats(x, y)
    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1)

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=10,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="white", alpha=0.55, edgecolor="gray"),
        color="k",
```

```
    )

    ax.text(
        0.5,
        0.97,
        panel_title,
        transform=ax.transAxes,
        va="top",
        ha="center",
        fontsize=11,
        bbox=dict(boxstyle="round,pad=0.25", facecolor="White", alpha=0.35, edgecolor="gray"),
        color="black",
    )
    ax.set_xlabel(x_label)
    # ax.set_ylabel("drifter ve (m/s)")
    ax.set_ylabel(y_label)
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right")
    return hb


def main():
    none = ""
    df_no_ek = load_filtered_df(CSV_NO_EK, "without_ek")
    df_w_ek = load_filtered_df(CSV_W_EK, "with_ek")

    x_no_ek, y_no_ek = prepare_xy(df_no_ek, "gu")
    x_w_ek, y_w_ek = prepare_xy_with_ek(df_w_ek)

    if x_no_ek.size < 3 or x_w_ek.size < 3:
        print(
            "Insufficient valid points after filtering.\n"
            f"without_ek: {x_no_ek.size} points, with_ek: {x_w_ek.size} points.\n"
            f"Current filters: PROXIMITY_DIST_M={PROXIMITY_DIST_M}, "
            f"MAX_ABS_TIME_DIFF_DAYS={MAX_ABS_TIME_DIFF_DAYS}"
        )
        return

    all_vals = np.concatenate([x_no_ek, y_no_ek, x_w_ek, y_w_ek])
    vmin = np.nanmin(all_vals)
    vmax = np.nanmax(all_vals)

    fig, axes = plt.subplots(1, 2, figsize=(16, 7), sharex=True, sharey=True)

    hb0 = plot_panel(
        axes[0],
        x_no_ek,
        y_no_ek,
        "Without Ekman",
        "Altimetry gu (m/s)",
        "Drifter ve (m/s)"
    )
    hb1 = plot_panel(
        axes[1],
        x_w_ek,
        y_w_ek,
        "With Ekman",
        "Altimetry gu + u_ek (m/s)",
        none
    )

    for ax in axes:
        # ax.set_xlim(vmin, vmax)
        # ax.set_ylim(vmin, vmax)
        ax.set_xlim(vmin, 1.5)
        ax.set_ylim(vmin, 1.5)

    cmax = max(hb0.get_array().max(), hb1.get_array().max())
    hb0.set_clim(1, cmax)
    hb1.set_clim(1, cmax)

    # fig.suptitle(
    #     f"{SAT}: Drifter ve comparison | coast <= {PROXIMITY_DIST_M:.0f} m, "
    #     f"|dt| <= {MAX_ABS_TIME_DIFF_DAYS if MAX_ABS_TIME_DIFF_DAYS is not None else 'all'} d"
    # )
    fig.tight_layout(rect=[0.0, 0.0, 0.89, 0.95])
    cax = fig.add_axes([0.91, 0.13, 0.015, 0.75])
    cbar = fig.colorbar(hb1, cax=cax)
    cbar.set_label("Count")
    fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)

    print(f"Rows after filters (without_ek): {len(df_no_ek)}")
    print(f"Rows after filters (with_ek): {len(df_w_ek)}")
    print(f"Saved {OUT_PNG}")


if __name__ == "__main__":
    main()

```

```
1
2  1. While saving .csv files, instead of unique_str, save, sat, track_self, track_other as individual columns
3  2. Also add lon_inter, lat_inter as seperate columns
4
5  Also gu_time that is saved should always be nearest one. Then we will able to see how far way temporally the interpreted gu from drifter_time
6
7  We can now add following functionality to p.scatter_drifter_ve_vs_gu.py
8  1. We should be able to filter the data points based on their proximity
9  coast using ~/allData/topo/GMT_intermediate_coast_distance_01d_track_reg.nc
10 We should be able to set PROXIMITY_DIST=50m, then we get scatter points only from points within the distance.
11 We can use lon_inter, lat_inter in .csv file to get coastdist
12
13 2. First extract the tracks numbers of tracks from unique_str
14 3. Next We can extract lon, lat of intersection points from tracks_intersections_S3A_1.csv by selecting respective points
15
16 load drifter_ve_vs_gu_{SAT}.csv and for given unique string
17 I am trying to do the following with c.candidate_buoys.for.intersection_points_copy.py
18 1. For each satellite and for each intersection point
19 for each drifter in the track region
20 a. Check if there is any time overlap between satellite time limits and drifter
21 time limits. If there is not time overlap skip.
22 b. If there is some time overalap, make a small region around the intersection
23 point with a threshold distance, and check if any point of the drifter is
24 passing through the box. If so it is a candidate drifter for further analysis.
25 save the list of all close passing drifters to pickle file.
26
27 Any general recommendations on how to do it better? Speed it up?
28 Don't edit any files.
29
30 In continuation
31 Write a script that plots for each intersection point
32 1. Plot a track region cartopy plot
33 2. Intersection point with the little box around it
34 3. Check candidate pickle file (if it exists there are close drifters)
35 4. For each close by drifter bouy plot the drifter track
36 5. Also mark the closest point of the drifter track to the intersection point
37 with x
38 TRACK_REG is a fixed variable in ../lib/namesm.py
39 For each intersection point, on closeby drifter track point is enough.
40 plot it as a scatter plot.
41
42 No need to compute it.
43
44 Refactor the code c.extract.drifter_veANDgu_at_intersections.py
45 to do the following:
46 for each intersection point
47 1. Read the corresponding satellite altimetry computed geostrophic currents
48 at /home/srinivasu/xtrackm/guv/guv_at_intersections
49 Now for each closeby drifter
50 2. Using the close index, extract the time stamp of time variable of driter
51 data
52 3. Extract zonal velocity from timeseries of guv data and driter data at
53 that time stamp
54 4. Save unique str, drifter_id, float values of  extracted guv,
55   extracted drifter velocity to a global csv for future scatter plot.
56
57 Write a python script to plot horizontal bars of temporal extent of
58 I seem to have used several data types for timesteps in these scripts. Suggest
59 simplifying the datatypes of timesteps. (general suggestions)
60 c.candidate_buoys.for.intersection_points_time_exclusion.py
61 l.check_chunk_sat_time_overlap.py
62 l.min_max_times.all.drifters.py
63 tools_xtrackm is ../lib
64 #codex resume 019c98ae-2158-7230-9757-7f2cb8a4920d
65
66 The purpose of above scripts is to find for each intersection point of
67 satellite tracks, we find a list of drifter ids that are passing through
68 a small box around the intersection point. So for every intersection point we
69 will have a list of drifters going closeby to the intersection points.
70
71 also have a look at /home/srinivasu/allData/drifter1 folder for text files
72 and python files for addtional context.
73
74 I need apply is_within_coast function on the lons_inter, lats_inter columns of df1 and retain only those rows which are within the threshold distance.
75
76 probe the time dimensions/variables of attached drifter and satellite
77 track files and their units. Recommend how their time axis has to be properly formatted so that their variables can be compared correctly.
78
79 I have a satellite intersection point (lons_inter, lats_inter). I have hundreds of drifter buoys that fill up the NIO region. I want to find the closest drifter buoy t
       o the intersection point and closest point on drifter path to the intersection point.
80 What is the best way to do this?
81
82 I have an ocean drifter buoy data which moves both in space and time. The spatial movement is described by two arrays lons_drift and lats_drift. Write a python functio
       n to check if atleast some part of the drifter passed through a given region.
83
84 The region (TRACKS_REG) is filled with drifter paths and satellite
85 intersection points.
86 Each point on a drifter path is located by (lon, lat) pair or (time) dimension equivalently. Each path file carries the variable
87 ve which is a function of (lon, lat) or
88 equivalently a function of (time). Each intersection point is associated with timeseries of computed currents gv and gu.
89 There are guv files in the format guv_at_intersection_{sat}_{track_self}_{track_other}.nc which contain the timeseries of gu and gv at the intersection of track_self and track_other in a seperate folder.
90 At each intersection point, we should
91 locate nearby driter points (and its index to locate time_inter) so that
92 satellite velocity at intersection point and drifter point can be compared. The gv timeseries at intersection point is sampled at time_inter. This value is compared wi
       th ve at the point where is drifter path is closest to the intersection point. How
93 to go about it? provide full code.
94 Additional information:
95 lons_inter and lats_inter are always contained in the
96 region.
97 traj dimension is always there in drifter ds.
98 following is the header of tracks_intersections_S3A_1.csv
99 ,sat,track_self,track_other,lons_inter,lats_inter,x_from_coast_self,x_from_coast_other,angle_acute,angle_obtuse
100 0,S3A,8,79,94.14047103085358,8.36326782978126,885051.8986275726,1543629.620555419,0.44727413372659675,2.6943185198631965
101 1,S3A,8,193,94.60794862071828,10.435742966838522,650148.6339016478,1194725.7346903323,0.44656871163430817,2.695023941955485
102 2,S3A,8,307,95.07549211474587,12.489287957263238,417331.9028785974,854253.7568147178,0.4459235207375096,2.695669132852283 5
103 3,S3A,8,421,95.54320490371074,14.520796447181 65,186943.59620474064,126272.34377060639,0.4434318644667041,2.698160789123089
104 4,S3A,8,507,92.7380651706875,2.0680879981685036,1598332.5897657045,2203201.928851282,0.4469590080350044,2.694633645554789
105 All satellite intersections points csv files use the same header.
106 location of output folder of gv files
107 /home/srinivasu/xtrackm/guv/guv_at_intersections
108 ncdump -h guv_at_intersection_GFO_98_169.nc
109 netcdf guv_at_intersection_GFO_98_169 {
110 dimensions:
111     time = 186 ;
112 variables:
113     int64 time(time) ;
114         time:units = "days since 2000-01-11" ;
```

```
        time:calendar = "proleptic_gregorian" ;
    double gu(time) ;
        gu:_FillValue = NaN ;
    double gv(time) ;
        gv:_FillValue = NaN ;
}
Required output is a CSV file of gv at time of intersection and ve of drifter along with intersection points, corresponding tracks etc
For others take the best defaults.

```

```python
from datetime import datetime
from pathlib import Path

import matplotlib.dates as mdates
import matplotlib.patches as mpatches
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from dateutil.relativedelta import relativedelta
from namesm import sats_new


# all buoys' time range is within following ranges
chunk_time_dict = {
    "1_5000": (
        np.datetime64("1985-10-30T00:00:00.000000000"),
        np.datetime64("2001-09-26T00:00:00.000000000"),
    ),
    "5001_10000": (
        np.datetime64("1997-03-20T00:00:00.000000000"),
        np.datetime64("2009-04-25T00:00:00.000000000"),
    ),
    "10001_15000": (
        np.datetime64("2006-03-05T00:00:00.000000000"),
        np.datetime64("2012-11-04T00:00:00.000000000"),
    ),
    "15001_current": (
        np.datetime64("2010-09-30T00:00:00.000000000"),
        np.datetime64("2025-05-19T00:00:00.000000000"),
    ),
}

DATE_FORMAT = "%Y-%m-%d"


def load_missions(csv_path="data_coverage_df.csv"):
    return pd.read_csv(csv_path)


def _get_satellite_extent(missions, sat_name):
    sat_rows = missions[missions["sat"] == sat_name]
    if sat_rows.empty:
        available = ", ".join(sorted(missions["sat"].unique()))
        raise ValueError(f"Satellite '{sat_name}' not found. Available: {available}")

    row = sat_rows.iloc[0]
    start_date = datetime.strptime(row["first_date"], DATE_FORMAT)
    end_date = datetime.strptime(row["last_date"], DATE_FORMAT)
    return start_date, end_date


def plot_all_temporal_extents(
    missions,
    chunk_ranges,
    output_path="temporal_extents_chunks_sats.png",
    show=True,
):
    mission_starts = pd.to_datetime(missions["first_date"])
    mission_ends = pd.to_datetime(missions["last_date"])
    chunk_starts = pd.to_datetime([v[0] for v in chunk_ranges.values()])
    chunk_ends = pd.to_datetime([v[1] for v in chunk_ranges.values()])

    min_start_date = min(mission_starts.min(), chunk_starts.min())
    max_end_date = max(mission_ends.max(), chunk_ends.max())
    min_start_date1 = min_start_date - relativedelta(years=1)
    max_end_date1 = max_end_date + relativedelta(years=1)

    fig, ax = plt.subplots(figsize=(12, 8), layout="constrained")

    y_positions = []
    y_labels = []

    # Plot satellites first
    for sat_idx, row in missions.iterrows():
        sat = row["sat"]
        start_date = datetime.strptime(row["first_date"], DATE_FORMAT)
        end_date = datetime.strptime(row["last_date"], DATE_FORMAT)
        duration = end_date - start_date

        ax.barh(
            sat_idx,
            duration,
            left=start_date,
            color="skyblue",
            edgecolor="black",
            alpha=0.9,
        )
        ax.text(
            start_date + duration / 2,
            sat_idx,
            sat,
            ha="center",
            va="center",
            fontweight="bold",
            fontsize=10,
            color="black",
        )
        y_positions.append(sat_idx)
        y_labels.append(f"SAT: {sat}")

    # Plot chunks on new rows below satellites
    chunk_offset = len(missions)
    for chunk_idx, (chunk, (start_date64, end_date64)) in enumerate(chunk_ranges.items()):
        y = chunk_offset + chunk_idx
        start_date = pd.to_datetime(start_date64).to_pydatetime()
        end_date = pd.to_datetime(end_date64).to_pydatetime()
        duration = end_date - start_date

        ax.barh(
            y,
            duration,
            left=start_date,
            color="salmon",
            edgecolor="black",
            alpha=0.9,
        )
        ax.text(
```

```
            start_date + duration / 2,
            y,
            chunk,
            ha="center",
            va="center",
            fontweight="bold",
            fontsize=10,
            color="black",
        )
        y_positions.append(y)
        y_labels.append(f"CHUNK: {chunk}")

    ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
    ax.xaxis.set_major_locator(mdates.YearLocator(2))
    ax.set_yticks(y_positions)
    ax.set_yticklabels(y_labels)
    ax.set_xlabel("Year")
    ax.set_xlim(min_start_date1, max_end_date1)
    ax.set_title("Temporal Extents of Satellite Missions and Chunks")
    ax.invert_yaxis()

    satellite_patch = mpatches.Patch(color="skyblue", label="Satellite")
    chunk_patch = mpatches.Patch(color="salmon", label="Chunk")
    ax.legend(handles=[satellite_patch, chunk_patch])

    plt.savefig(output_path, dpi=200)
    if show:
        plt.show()


def plot_single_chunk_sat_extent(
    chunk_name,
    sat_name,
    missions,
    chunk_ranges,
    output_path="temporal_extent_single_chunk_single_sat.png",
    show=True,
):
    if chunk_name not in chunk_ranges:
        available = ", ".join(chunk_ranges.keys())
        raise ValueError(f"Chunk '{chunk_name}' not found. Available: {available}")

    sat_start, sat_end = _get_satellite_extent(missions, sat_name)
    chunk_start64, chunk_end64 = chunk_ranges[chunk_name]
    chunk_start = pd.to_datetime(chunk_start64).to_pydatetime()
    chunk_end = pd.to_datetime(chunk_end64).to_pydatetime()

    min_start = min(sat_start, chunk_start) - relativedelta(years=1)
    max_end = max(sat_end, chunk_end) + relativedelta(years=1)

    fig, ax = plt.subplots(figsize=(10, 4), layout="constrained")

    sat_duration = sat_end - sat_start
    ax.barh(
        0,
        sat_duration,
        left=sat_start,
        color="skyblue",
        edgecolor="black",
        alpha=0.9,
    )
    ax.text(
        sat_start + sat_duration / 2,
        0,
        sat_name,
        ha="center",
        va="center",
        fontweight="bold",
        fontsize=10,
        color="black",
    )

    chunk_duration = chunk_end - chunk_start
    ax.barh(
        1,
        chunk_duration,
        left=chunk_start,
        color="salmon",
        edgecolor="black",
        alpha=0.9,
    )
    ax.text(
        chunk_start + chunk_duration / 2,
        1,
        chunk_name,
        ha="center",
        va="center",
        fontweight="bold",
        fontsize=10,
        color="black",
    )

    ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
    ax.xaxis.set_major_locator(mdates.YearLocator(2))
    ax.set_yticks([0, 1])
    ax.set_yticklabels([f"SAT: {sat_name}", f"CHUNK: {chunk_name}"])
    ax.set_xlabel("Year")
    ax.set_xlim(min_start, max_end)
    ax.set_title(f"Temporal Extents: {sat_name} and {chunk_name}")
    ax.invert_yaxis()

    satellite_patch = mpatches.Patch(color="skyblue", label="Satellite")
    chunk_patch = mpatches.Patch(color="salmon", label="Chunk")
    ax.legend(handles=[satellite_patch, chunk_patch])

    plt.savefig(output_path, dpi=200)
    if show:
        plt.show()
    plt.close(fig)


def plot_pairwise_chunk_sat_extents(
    missions,
    chunk_ranges,
    sat_names=None,
    output_dir="pairwise_temporal_extents",
    show=False,
```

```
):
    if sat_names is None:
        sat_names = list(missions["sat"].unique())

    output_path = Path(output_dir)
    output_path.mkdir(parents=True, exist_ok=True)

    for chunk_name in chunk_ranges.keys():
        for sat_name in sat_names:
            plot_single_chunk_sat_extent(
                chunk_name,
                sat_name,
                missions,
                chunk_ranges,
                output_path=str(output_path / f"{chunk_name}__{sat_name}.png"),
                show=show,
            )


if __name__ == "__main__":
    missions_df = load_missions()
    plot_pairwise_chunk_sat_extents(
        missions_df,
        chunk_time_dict,
        sat_names=sats_new,
        output_dir="pairwise_temporal_extents",
        show=True,
    )

```

```
1
2  Find candidate closeby drifter tracks for each intersection point
3  excluding drifters with no overlapping time periods
4  save the list of close drifter ids to pickle files
5
6  2. Check visually d.plot_candidate_drifters.for.intersections.py
7  closeby drifter for each intersection point, box of close region,
8  closest point on drifter track
9
10 3.
11 c.extract.drifter_veANDgu_at_intersections.py
12 Read the corresponding satellite altimetry computed geostrophic currents
13 Using the close index, extract zonal velocity from timeseries of guv data and driter data at time stamp
14 Save unique str, drifter_id, float values of  extracted guv,
15 extracted drifter velocity to a global csv for future scatter plot.
16
17 4. p.scatter_drifter_ve_vs_gu_S3A.py
18 To plot scatter plot
19
20
21
22 My broader objective is to find nearby buoy trajectory points (in space and
23     time) at satellite intersection points to compare satellite derived
24 currents with buoy currents at (see ~/xtrackm/drifter)
25
26 l.min_max_times.all.drifters.py min max times of all drifters
27 It will give a dictionary of start and end time for each chunk
28
29 l.check_chunk_sat_time_overlap.py checks overlap between satellite and drifter
30 time coverage.
31
32 # codex resume 019c953f-1e0b-7a90-90e5-fcb95cd18004
33 steps:
34 # We now don't have to check within region any, since we have only downloaded
35 within region in ~/allData/drifter_track_reg/
36 # We have new chunk time dict for track region
37
38 c.candidate_drifters scripts
39 1. run through all drifters once to exclude non overlapping drifters for a
40 given satellite mission.
41   a. exclude_based_on_time(ds_drifter, sat)
42      if drifter's start and end time don't have overlap with satellite time
43      periods, skip this buoy.
44 2. For each intersection point
45   a. which of these time overlapping drifters come in close proxmity with the
46   the intersection point
47
48 The purpose of above script is to find for each intersection point of
49 satellite tracks, we find a list of drifter ids that are passing through
50 a small box around the intersection point. So for every intersection point we
51 will have a list of drifters going closeby to the intersection points.
52
53 also have a look at /home/srinivasu/allData/drifter_track_reg folder for text
54 files and python files for addtional context.
55 execute any of the scripts in this dir to gain addtional context.
56
```

```
./eicc/
```

30 file(s)

```
1
2  At what isobath should we take slope on
3  The shapely based slopes are not matching with that of isobath contour extraction scripts
4  These are inturn not matching verymuch with mukherji 2014 paper slopes
5
```

```
1
2 [] slope at points on 200m isobath
3
```

```python
#!/usr/bin/env python3
import os

import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr

# codex resume 019d11d3-4f11-75a3-9cd3-897d94b9bb77

SAT = "TP+J1+J2+J3+S6A"
SAT = "HY2A"
SAT = "ERS1+ERS2+ENV+SRL"
SAT = "S3A"
SAT = "TPN+J1N+J2N+J3N"
INPUT_CSV = f"tracks_intersections_{SAT}_1.csv"
COAST_DIST_PATH = (
    "~/allData/topo/GMT_intermediate_coast_distance_01d_track_reg.nc"
)
THRESHOLD_KM = 100.0
BOB_EXTENT = (78.0, 100.0, 4.0, 24.0)  # lon_min, lon_max, lat_min, lat_max
OUT_PNG = f"intersections_near_east_coast_{SAT}.png"
OUT_CSV = f"intersections_near_east_coast_{SAT}.csv"


def get_coast_distance_km(lons, lats):
    with xr.open_dataset(os.path.expanduser(COAST_DIST_PATH)) as ds_coast:
        lon_da = xr.DataArray(np.asarray(lons, dtype=float), dims="point")
        lat_da = xr.DataArray(np.asarray(lats, dtype=float), dims="point")
        coast_km = ds_coast["coast_distance"].sel(
            lon=lon_da, lat=lat_da, method="nearest"
        )
    return coast_km.values.astype(float)


def plot_bob(df_all, df_sel, THRESHOLD_KM, OUT_PNG):
    proj = ccrs.PlateCarree()
    fig = plt.figure(figsize=(11, 9))
    ax = plt.axes(projection=proj)

    ax.set_extent(BOB_EXTENT, crs=proj)
    ax.add_feature(cfeature.LAND, facecolor="0.93", zorder=0)
    ax.add_feature(cfeature.OCEAN, facecolor="white", zorder=0)
    ax.coastlines(resolution="50m", linewidth=1.0, zorder=2)
    gl = ax.gridlines(
        draw_labels=True,
        linewidth=0.5,
        color="gray",
        alpha=0.6,
        linestyle="--",
    )
    gl.top_labels = False
    gl.right_labels = False

    ax.scatter(
        df_all["lons_inter"],
        df_all["lats_inter"],
        s=22,
        c="0.65",
        alpha=0.6,
        transform=proj,
        zorder=3,
        label="All intersections",
    )

    ax.scatter(
        df_sel["lons_inter"],
        df_sel["lats_inter"],
        s=36,
        c="crimson",
        edgecolors="black",
        linewidths=0.4,
        transform=proj,
        zorder=4,
        label=f"|coast distance| <= {THRESHOLD_KM:.1f} km",
    )

    ax.plot(
        df_sel["lons_inter"],
        df_sel["lats_inter"],
        color="tab:blue",
        linewidth=1.5,
        marker="o",
        markersize=4,
        transform=proj,
        zorder=3,
    )

    for i, row in df_sel.reset_index(drop=True).iterrows():
        lon = float(row["lons_inter"])
        lat = float(row["lats_inter"])
        txt = f"({lon:.2f}, {lat:.2f})"
        dx = 0.12 if i % 2 == 0 else -0.12
        dy = 0.10 if i % 3 == 0 else -0.10
        # ax.text(
        #     lon + dx,
        #     lat + dy,
        #     txt,
        #     fontsize=8,
        #     transform=proj,
        #     zorder=5,
        #     bbox=dict(facecolor="white", alpha=0.7, edgecolor="none", pad=1.5),
        # )

    ax.set_title(
        f"Bay of Bengal intersections near coast\n"
        f"Selected points: {len(df_sel)} / {len(df_all)} (threshold = {THRESHOLD_KM:.1f} km)"
    )
    ax.legend(loc="lower left")
    fig.tight_layout()
    fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)


```

```
df = pd.read_csv(INPUT_CSV)
coast_km = get_coast_distance_km(
    df["lons_inter"].values, df["lats_inter"].values
)
df["coast_distance_km_nearest"] = coast_km

# Distance filtering
keep_dist = np.isfinite(coast_km) & (
    np.abs(coast_km) <= float(THRESHOLD_KM)
)
lon_min, lon_max, lat_min, lat_max = BOB_EXTENT
# BOB extent filtering
keep_bob = (
    (df["lons_inter"] >= lon_min)
    & (df["lons_inter"] <= lon_max)
    & (df["lats_inter"] >= lat_min)
    & (df["lats_inter"] <= lat_max)
)
df_selected = df.loc[keep_dist & keep_bob].copy()

# Discard points with lat < 9N and lon > 90E
drop_mask = (df_selected["lats_inter"] < 9.0) | (
    df_selected["lons_inter"] > 90.0
)
df_selected = df_selected.loc[~drop_mask].copy()

# Sort by latitude
df_selected = df_selected.sort_values("lats_inter").copy()

# If multiple points share the same latitude at 2 decimal places, keep the one
# whose absolute coast distance is closest to half the threshold distance.
target_half = float(THRESHOLD_KM) / 2.0
df_selected["lat_2dp"] = df_selected["lats_inter"].round(2)
df_selected["dist_to_half"] = np.abs(
    np.abs(df_selected["coast_distance_km_nearest"]) - target_half
)
keep_idx = df_selected.groupby("lat_2dp")["dist_to_half"].idxmin()
df_selected = df_selected.loc[keep_idx].sort_values("lats_inter").copy()
df_selected = df_selected.drop(columns=["lat_2dp", "dist_to_half"])

selected_points = [
    (
        float(r["lons_inter"]),
        float(r["lats_inter"]),
        float(r["coast_distance_km_nearest"]),
    )
    for _, r in df_selected.iterrows()
]

df_selected.to_csv(OUT_CSV, index=False)
plot_bob(df, df_selected, THRESHOLD_KM, OUT_PNG)

print(f"Total points: {len(df)}")
print(
    f"Near-coast points in full domain (|distance| <= {THRESHOLD_KM:.1f} km): {int(np.sum(keep_dist))}"
)
print(f"Selected points in BOB extent after extra rules: {len(df_selected)}")
print(f"Saved selected points CSV: {OUT_CSV}")
print(f"Saved plot: {OUT_PNG}")
print("Selected points list as (lon, lat, coast_distance_km):")
for p in selected_points:
    print(p)

```

```python
import numpy as np
import xarray as xr
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_filter1d
from scipy.signal import savgol_filter
from tools_xtrackm import *
from geopy import distance

_VALID_METHODS = ("gaussian", "savgol")

# copilot --resume=1108ce5c-68e7-40f0-95f7-b8e8708e8e56

def isobath_heading_at_point(
    topo,
    level,
    query_point,
    method="gaussian",
    sigma=5.0,
    window_length=21,
    polyorder=3,
    step=10,
    lon_name="lon",
    lat_name="lat",
):
    """
    Compute the isobath tangent heading angle at a query point.
    Combines three steps in one call:
      1. Extract all (lon, lat) points along the requested isobath using
         matplotlib contour tracing.
      2. Smooth the raw polyline with the chosen filter and subsample every
         *step* points.
      3. Find the nearest smoothed point to *query_point* and return the local
         tangent direction as an angle measured counter-clockwise from the
         positive x-axis (east), in degrees, range [-180, 180].
    Smoothing methods
    -----------------
    ``'gaussian'``  (default)
        1-D Gaussian filter via ``scipy.ndimage.gaussian_filter1d``.
        Controlled by *sigma* (std dev in grid points).  Better frequency
        response than a box filter; reflect-padded edges.
    ``'savgol'``
        Savitzky-Golay filter via ``scipy.signal.savgol_filter``.
        Controlled by *window_length* (must be odd) and *polyorder*.
        Fits a local polynomial, so it preserves shape features better
        than Gaussian while also allowing analytical derivative extraction
        (``deriv=1``), making the tangent estimate more accurate.
    Parameters
    ----------
    topo : xarray.DataArray
        2-D bathymetry / topography field with lon and lat coordinates.
    level : float
        Isobath depth level (e.g. -200 for the 200 m isobath).
    query_point : tuple[float, float]
        (lon, lat) of the location of interest.
    method : {'gaussian', 'savgol'}, optional
        Smoothing method (default ``'gaussian'``).
    sigma : float, optional
        Std dev of the Gaussian kernel in grid points (default 5.0).
        Used only when ``method='gaussian'``.
    window_length : int, optional
        Length of the Savitzky-Golay filter window; must be odd and
        greater than *polyorder* (default 21).
        Used only when ``method='savgol'``.
    polyorder : int, optional
        Polynomial order for the Savitzky-Golay fit (default 3).
        Used only when ``method='savgol'``.
    step : int, optional
        Keep every *step*-th point after smoothing (default 10).
    lon_name : str, optional
        Name of the longitude coordinate in *topo* (default ``"lon"``).
    lat_name : str, optional
        Name of the latitude coordinate in *topo* (default ``"lat"``).
    Returns
    -------
    dict with keys:
        heading_deg : float
            Heading angle in degrees, measured counter-clockwise from east
            (positive x-axis).  Range [-180, 180].
        isobath_pts : list[tuple[float, float]]
            Smoothed and subsampled (lon, lat) isobath points, ready for
            plotting.
        nearest_point : tuple[float, float]
            The smoothed isobath point closest to *query_point*.
        nearest_index : int
            Index of *nearest_point* within *isobath_pts*.
        method : str
            The smoothing method used.
    Raises
    ------
    ValueError
        If *method* is not recognised, no isobath points are found, or
        the smoothed polyline has fewer than two points.
    """
    if method not in _VALID_METHODS:
        raise ValueError(
            f"method={method!r} is not recognised. "
            f"Choose one of {_VALID_METHODS}."
        )
    # ------------------------------------------------------------------ #
    # Step 1 – extract raw isobath points via matplotlib contour          #
    # ------------------------------------------------------------------ #
    lon = topo[lon_name].values
    lat = topo[lat_name].values
    z = topo.values
    lon2d, lat2d = np.meshgrid(lon, lat)
    fig, ax = plt.subplots()
    cs = ax.contour(lon2d, lat2d, z, levels=[level])
    plt.close(fig)

    raw_pts = []
    segments = cs.allsegs[0] if cs.allsegs else []
    for seg in segments:
        raw_pts.extend(map(tuple, seg))

    if not raw_pts:
        raise ValueError(
            f"No isobath points found for level={level}. "
```

```
            "Check that the level exists within the topo domain."
        )

    arr = np.asarray(raw_pts, dtype=float)

    # ------------------------------------------------------------------ #
    # Step 2 — smooth and subsample                                       #
    # ------------------------------------------------------------------ #
    if method == "gaussian":
        lon_smooth = gaussian_filter1d(arr[:, 0], sigma=sigma)
        lat_smooth = gaussian_filter1d(arr[:, 1], sigma=sigma)
        # Tangent via finite differences on the smoothed positions.
        arr_smooth = np.column_stack((lon_smooth, lat_smooth))
        pts = arr_smooth[::step]
        dlon_arr = None  # computed on demand in step 3
        dlat_arr = None

    else:  # savgol
        # Ensure window_length is odd and feasible.
        wl = window_length if window_length % 2 == 1 else window_length + 1
        wl = max(wl, polyorder + 2 if (polyorder + 2) % 2 == 1 else polyorder + 3)
        wl = min(wl, len(arr))
        if wl % 2 == 0:
            wl -= 1

        lon_smooth = savgol_filter(arr[:, 0], window_length=wl, polyorder=polyorder)
        lat_smooth = savgol_filter(arr[:, 1], window_length=wl, polyorder=polyorder)
        # Analytical first derivative directly from the polynomial fit.
        dlon_smooth = savgol_filter(
            arr[:, 0], window_length=wl, polyorder=polyorder, deriv=1
        )
        dlat_smooth = savgol_filter(
            arr[:, 1], window_length=wl, polyorder=polyorder, deriv=1
        )
        arr_smooth = np.column_stack((lon_smooth, lat_smooth))
        pts = arr_smooth[::step]
        dlon_arr = dlon_smooth[::step]
        dlat_arr = dlat_smooth[::step]

    if len(pts) < 2:
        raise ValueError(
            "Fewer than two points remain after smoothing/subsampling; "
            "reduce smoothing parameters or *step*."
        )

    # ------------------------------------------------------------------ #
    # Step 3 — nearest-point lookup and tangent angle                     #
    # ------------------------------------------------------------------ #
    # q = np.asarray(query_point, dtype=float)
    # d2 = np.sum((pts - q) ** 2, axis=1)
    d2 = [distance.distance((query_point[1], query_point[0]), (pt[1], pt[0])).km ** 2 for pt in pts]
    idx = int(np.argmin(d2))

    if dlon_arr is not None:
        # savgol: use the analytical polynomial derivative directly —
        # more accurate than a finite-difference approximation.
        dx = float(dlon_arr[idx])
        dy = float(dlat_arr[idx])
    else:
        # gaussian: finite differences on the smoothed positions.
        if idx == 0:
            v = pts[1] - pts[0]
        elif idx == len(pts) - 1:
            v = pts[-1] - pts[-2]
        else:
            v = pts[idx + 1] - pts[idx - 1]
        dx, dy = float(v[0]), float(v[1])

    heading_deg = float(np.degrees(np.arctan2(dy, dx)))

    return {
        "heading_deg": heading_deg,
        "isobath_pts": [tuple(xy) for xy in pts],
        "nearest_point": (float(pts[idx, 0]), float(pts[idx, 1])),
        "nearest_index": idx,
        "method": method,
    }


# ====================================================================== #
# Example usage                                                          #
# ====================================================================== #
if __name__ == "__main__":
    REG = (62, 92, 0, 30)
    dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo_60s_io.nc")
    topo = dse.rose.sel(lon=slice(*REG[:2])).sel(lat=slice(*REG[2:]))

    # Example query points along the east coast of India / Bay of Bengal
    query_points = (
        (83.62, 17.17),    # Kakinada area
        (88.92, 19.00),    # Odisha shelf
        (80.203, 12.015),  # Chennai shelf
        (82.982, 16.837),  # Visakhapatnam area
        (85.442, 18.856),  # Northern Andhra shelf
    )

    query_points_names = {
            "Paradip  ":(88.92, 19.00),    # Odisha shelf
            "Cuddalore":(80.203, 12.015),  # Chennai shelf
            "Kakinada ":(82.492, 16.387),  # Visakhapatnam area
            "Gopalpur ":(85.442, 18.856),  # Northern Andhra shelf
    }

    common_kw = dict(topo=topo, level=-1000, step=15)

    configs = [
        dict(method="gaussian", sigma=10.0),
        dict(method="savgol",   window_length=41, polyorder=3),
    ]

    for cfg in configs:
        label = (
            f"gaussian σ={cfg['sigma']}"
            if cfg["method"] == "gaussian"
            else f"savgol wl={cfg['window_length']} po={cfg['polyorder']}"
        )
        print(f"\n{'='*56}")
```

```
        print(f"  Method: {label}")
        print(f"{'='*56}")
        print(f"  {'Query point':<22} {'Heading (° CCW from east)':>26}")
        print(f"  {'-'*48}")

        results = []
        for k, q in query_points_names.items():
            res = isobath_heading_at_point(query_point=q, **common_kw, **cfg)
            results.append(res)
            print(f"  ({q[0]:.3f}, {q[1]:.3f}) {k}       {res['heading_deg']:>10.1f}°")

    # ---------------------------------------------------------------- #
    # Plot both methods side by side for visual comparison              #
    # ---------------------------------------------------------------- #
    fig, axes = plt.subplots(
        1, 2,
        figsize=(18, 8),
        subplot_kw={"projection": ccrs.PlateCarree()},
    )

    for ax, cfg in zip(axes, configs):
        label = (
            f"Gaussian σ={cfg['sigma']}"
            if cfg["method"] == "gaussian"
            else f"Savitzky-Golay wl={cfg['window_length']} po={cfg['polyorder']}"
        )

        topo.plot.contourf(
            ax=ax, add_colorbar=False, add_labels=False, cmap="twilight_shifted"
        )
        topo.plot.contour(
            ax=ax, levels=[-200], add_colorbar=False, add_labels=False,
            colors=["k"], linewidths=[0.6], linestyles=["--"],
        )

        results = [
            isobath_heading_at_point(query_point=q, **common_kw, **cfg)
            for q in query_points
        ]

        # Smoothed isobath from first result (same for all query points)
        iso_lons, iso_lats = zip(*results[0]["isobath_pts"])
        ax.scatter(iso_lons, iso_lats, color="b", s=2, zorder=3,
                   label=f"~200 m ({label})")

        for q, res in zip(query_points, results):
            np_lon, np_lat = res["nearest_point"]
            ax.plot(q[0], q[1], "r*", ms=5, zorder=5)
            # ax.plot(np_lon, np_lat, "wo", ms=6, zorder=5)
            # ax.plot([q[0], np_lon], [q[1], np_lat], "w-", lw=0.8, zorder=4)
            ax.annotate(
                f"{res['heading_deg']:.1f}°",
                xy=(np_lon, np_lat), xytext=(4, 4),
                textcoords="offset points", fontsize=12, color="k",
            )

        xsta, xend, ysta, yend = REG
        ax.add_feature(cfeature.COASTLINE, linewidth=1.5)
        ax.set_xticks(range(int(xsta), int(xend), 8), crs=ccrs.PlateCarree())
        ax.set_yticks(range(int(ysta), int(yend), 8), crs=ccrs.PlateCarree())
        ax.xaxis.set_major_formatter(LongitudeFormatter())
        ax.yaxis.set_major_formatter(LatitudeFormatter())
        ax.set_extent([xsta, xend, ysta, yend], crs=ccrs.PlateCarree())
        ax.legend(loc="lower left", fontsize=9, framealpha=0.85)
        ax.set_title(label, fontsize=12)

    plt.tight_layout()
    plt.savefig("topo.isobath.png", dpi=150)
    plt.show()
    plt.close(fig)
```

```python
import numpy as np
import xarray as xr
from matplotlib.lines import Line2D
from matplotlib.patches import Patch
from tools_xtrackm import *
# codex resume 019d1654-1b08-70c0-adfb-1fd3acdb5f72

def extract_isobath_all_pts(topo, level, lon_name="lon", lat_name="lat"):
    """
    Extract all (lon, lat) points along an isobath.
    Parameters
    ----------
    topo : xarray.DataArray
        2D bathymetry/topography field.
    level : float
        Isobath level (e.g., -200 for 200 m isobath).
    lon_name, lat_name : str
        Coordinate names in topo.
    Returns
    -------
    list[tuple[float, float]]
        Flat list of (lon, lat) points from all isobath segments.
    """
    lon = topo[lon_name].values
    lat = topo[lat_name].values
    z = topo.values
    lon2d, lat2d = np.meshgrid(lon, lat)
    fig, ax = plt.subplots()
    cs = ax.contour(lon2d, lat2d, z, levels=[level])
    plt.close(fig)
    all_pts = []
    # Matplotlib versions differ in contour internals; allsegs is stable.
    # For a single requested level, cs.allsegs[0] holds all disconnected segments.
    segments = cs.allsegs[0] if cs.allsegs else []
    for seg in segments:
        all_pts.extend(map(tuple, seg))
    return all_pts


def smooth_and_subsample_pts(points, window=5, step=10):
    """
    Smooth (lon, lat) points with a moving average and subsample.
    """
    if not points:
        return []
    arr = np.asarray(points, dtype=float)
    if arr.shape[0] < window:
        return [tuple(xy) for xy in arr[::step]]
    kernel = np.ones(window, dtype=float) / window
    lon_smooth = np.convolve(arr[:, 0], kernel, mode="same")
    lat_smooth = np.convolve(arr[:, 1], kernel, mode="same")
    smooth_pts = np.column_stack((lon_smooth, lat_smooth))
    return [tuple(xy) for xy in smooth_pts[::step]]


def nearest_point_and_tangent(
    points, query_point, return_degrees_from_east=False
):
    """
    Find nearest point on a polyline and local tangent slope at that point.
    Parameters
    ----------
    points : list[tuple[float, float]]
        Polyline points as (lon, lat), ideally smoothed/subsampled.
    query_point : tuple[float, float]
        Query (lon, lat).
    return_degrees_from_east : bool
        If True, include tangent angle in degrees measured CCW from east.
    Returns
    -------
    dict
        nearest_point: (lon, lat)
        index: index in points
        distance: euclidean distance in lon/lat units
        slope_dy_dx: tangent slope dy/dx (np.inf for vertical tangent)
        degrees_from_east: optional, range [-180, 180]
    """
    if not points:
        raise ValueError("points is empty")
    if len(points) == 1:
        raise ValueError(
            "at least two points are required to estimate tangent"
        )
    arr = np.asarray(points, dtype=float)
    q = np.asarray(query_point, dtype=float)
    d2 = np.sum((arr - q) ** 2, axis=1)
    idx = int(np.argmin(d2))
    nearest = arr[idx]
    if idx == 0:
        v = arr[1] - arr[0]
    elif idx == len(arr) - 1:
        v = arr[-1] - arr[-2]
    else:
        v = arr[idx + 1] - arr[idx - 1]
    dx, dy = float(v[0]), float(v[1])
    slope = np.inf if np.isclose(dx, 0.0) else (dy / dx)
    out = {
        "nearest_point": (float(nearest[0]), float(nearest[1])),
        "index": idx,
        "distance": float(np.sqrt(d2[idx])),
        "slope_dy_dx": float(slope) if np.isfinite(slope) else np.inf,
    }
    if return_degrees_from_east:
        out["degrees_from_east"] = float(np.degrees(np.arctan2(dy, dx)))
    return out


REG = (62, 92, 0, 30)
dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo_60s_io.nc")
topo = dse.rose.sel(lon=slice(*REG[:2])).sel(lat=slice(*REG[2:]))

cmap = "twilight_shifted"
isobath_levels = [-200, -1000, -2000]
fig = plt.figure(figsize=(16, 10))
ax1 = fig.add_subplot(1, 2, 1, projection=ccrs.PlateCarree())
ax2 = fig.add_subplot(1, 2, 2, projection=ccrs.PlateCarree())
topo.plot.contourf(ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap)
```

```
118
119 # Continental slope zone: 200 m to 2000 m depth
120 slope_band = topo.where((topo <= -200) & (topo >= -2000))
121 slope_band.plot.contourf(
122     ax=ax1,
123     levels=[-2000, -200],
124     colors=["gold"],
125     alpha=0.25,
126     add_colorbar=False,
127     add_labels=False,
128 )
129 # Plot each isobath separately to avoid level-color reordering issues.
130 isobath_style = {
131     -200: ("k", 1.2),
132     -1000: ("darkorange", 1.0),
133     -2000: ("red", 1.0),
134 }
135 for level in isobath_levels:
136     color, lw = isobath_style[level]
137     topo.plot.contour(
138         ax=ax1,
139         levels=[level],
140         add_colorbar=False,
141         add_labels=False,
142         colors=[color],
143         linewidths=[lw],
144     )
145 xsta, xend, ysta, yend = REG
146 step = 8
147 ax1.add_feature(cfeature.COASTLINE, linewidth=2.0)
148 xticks = list(range(int(xsta), int(xend), step))
149 yticks = list(range(int(ysta), int(yend), step))
150 ax1.axhline(y=0.0, linestyle="--", color="k")
151 ax1.set_xticks(xticks, crs=ccrs.PlateCarree())
152 ax1.set_yticks(yticks, crs=ccrs.PlateCarree())
153 ax1.xaxis.set_major_formatter(LongitudeFormatter())
154 ax1.yaxis.set_major_formatter(LatitudeFormatter())
155 ax1.set_extent([xsta, xend, ysta, yend], crs=ccrs.PlateCarree())
156 info = f"name of cmap: {cmap}, slope: 200-2000 m"
157 plt.text(
158     0.15,
159     0.90,
160     info,
161     transform=ax1.transAxes,
162     fontsize=14,
163     fontweight="bold",
164     color="w",
165 )
166
167 legend_handles = [
168     Patch(
169         facecolor="gold",
170         edgecolor="none",
171         alpha=0.25,
172         label="Slope zone (200-2000 m)",
173     ),
174     Line2D([0], [0], color="k", lw=1.2, label="200 m isobath"),
175     Line2D([0], [0], color="darkorange", lw=1.0, label="1000 m isobath"),
176     Line2D([0], [0], color="red", lw=1.0, label="2000 m isobath"),
177 ]
178 ax1.legend(
179     handles=legend_handles,
180     loc="lower left",
181     frameon=True,
182     framealpha=0.85,
183     fontsize=10,
184 )
185
186 topo.plot.contourf(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap)
187 all_pts = extract_isobath_all_pts(topo, level=-1)
188 all_pts = smooth_and_subsample_pts(all_pts, window=10, step=15)
189 if all_pts:
190     lons, lats = zip(*all_pts)
191     ax2.scatter(
192         lons, lats, color="k", marker=".", s=1, label="200 m isobath points"
193     )
194 else:
195     print("No points found for the requested isobath level.")
196 xsta, xend, ysta, yend = REG
197 step = 8
198 xticks = list(range(int(xsta), int(xend), step))
199 yticks = list(range(int(ysta), int(yend), step))
200 ax2.axhline(y=0.0, linestyle="--", color="k")
201 ax2.set_xticks(xticks, crs=ccrs.PlateCarree())
202 ax2.set_yticks(yticks, crs=ccrs.PlateCarree())
203 ax2.xaxis.set_major_formatter(LongitudeFormatter())
204 ax2.yaxis.set_major_formatter(LatitudeFormatter())
205 ax2.set_extent([xsta, xend, ysta, yend], crs=ccrs.PlateCarree())
206
207 query_points = (
208     (83.62, 17.17),
209     (88.92, 19.00),
210     (80.203, 12.015),
211     (82.982, 16.837),
212     (85.442, 18.856),
213 )
214 for q in query_points:
215     tangent_info = nearest_point_and_tangent(
216         all_pts, q, return_degrees_from_east=True
217     )
218     nearest_point = tangent_info["nearest_point"]
219     degrees_from_east = tangent_info["degrees_from_east"]
220     print(f"{q}, {degrees_from_east:.1f}°")
221
222 plt.savefig("topo.isobath.png", dpi=300)
223 plt.show()
224 plt.close(fig)
225
```

```python
import numpy as np
import xarray as xr
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_filter1d
from scipy.signal import savgol_filter
from tools_xtrackm import *
from geopy import distance

_VALID_METHODS = ("gaussian", "savgol")

# copilot --resume=1108ce5c-68e7-40f0-95f7-b8e8708e8e56

def isobath_heading_at_point(
    topo,
    level,
    query_point,
    method="gaussian",
    sigma=5.0,
    window_length=21,
    polyorder=3,
    step=10,
    lon_name="lon",
    lat_name="lat",
):
    """
    Compute the isobath tangent heading angle at a query point.
    Combines three steps in one call:
      1. Extract all (lon, lat) points along the requested isobath using
         matplotlib contour tracing.
      2. Smooth the raw polyline with the chosen filter and subsample every
         *step* points.
      3. Find the nearest smoothed point to *query_point* and return the local
         tangent direction as an angle measured counter-clockwise from the
         positive x-axis (east), in degrees, range [-180, 180].
    Smoothing methods
    -----------------
    ``'gaussian'``  (default)
        1-D Gaussian filter via ``scipy.ndimage.gaussian_filter1d``.
        Controlled by *sigma* (std dev in grid points).  Better frequency
        response than a box filter; reflect-padded edges.
    ``'savgol'``
        Savitzky-Golay filter via ``scipy.signal.savgol_filter``.
        Controlled by *window_length* (must be odd) and *polyorder*.
        Fits a local polynomial, so it preserves shape features better
        than Gaussian while also allowing analytical derivative extraction
        (``deriv=1``), making the tangent estimate more accurate.
    Parameters
    ----------
    topo : xarray.DataArray
        2-D bathymetry / topography field with lon and lat coordinates.
    level : float
        Isobath depth level (e.g. -200 for the 200 m isobath).
    query_point : tuple[float, float]
        (lon, lat) of the location of interest.
    method : {'gaussian', 'savgol'}, optional
        Smoothing method (default ``'gaussian'``).
    sigma : float, optional
        Std dev of the Gaussian kernel in grid points (default 5.0).
        Used only when ``method='gaussian'``.
    window_length : int, optional
        Length of the Savitzky-Golay filter window; must be odd and
        greater than *polyorder* (default 21).
        Used only when ``method='savgol'``.
    polyorder : int, optional
        Polynomial order for the Savitzky-Golay fit (default 3).
        Used only when ``method='savgol'``.
    step : int, optional
        Keep every *step*-th point after smoothing (default 10).
    lon_name : str, optional
        Name of the longitude coordinate in *topo* (default ``"lon"``).
    lat_name : str, optional
        Name of the latitude coordinate in *topo* (default ``"lat"``).
    Returns
    -------
    dict with keys:
        heading_deg : float
            Heading angle in degrees, measured counter-clockwise from east
            (positive x-axis).  Range [-180, 180].
        isobath_pts : list[tuple[float, float]]
            Smoothed and subsampled (lon, lat) isobath points, ready for
            plotting.
        nearest_point : tuple[float, float]
            The smoothed isobath point closest to *query_point*.
        nearest_index : int
            Index of *nearest_point* within *isobath_pts*.
        method : str
            The smoothing method used.
    Raises
    ------
    ValueError
        If *method* is not recognised, no isobath points are found, or
        the smoothed polyline has fewer than two points.
    """
    if method not in _VALID_METHODS:
        raise ValueError(
            f"method={method!r} is not recognised. "
            f"Choose one of {_VALID_METHODS}."
        )
    # ------------------------------------------------------------------ #
    # Step 1 – extract raw isobath points via matplotlib contour          #
    # ------------------------------------------------------------------ #
    lon = topo[lon_name].values
    lat = topo[lat_name].values
    z = topo.values
    lon2d, lat2d = np.meshgrid(lon, lat)
    fig, ax = plt.subplots()
    cs = ax.contour(lon2d, lat2d, z, levels=[level])
    plt.close(fig)

    raw_pts = []
    segments = cs.allsegs[0] if cs.allsegs else []
    for seg in segments:
        raw_pts.extend(map(tuple, seg))

    if not raw_pts:
        raise ValueError(
            f"No isobath points found for level={level}. "
```

```
            "Check that the level exists within the topo domain."
        )

    arr = np.asarray(raw_pts, dtype=float)

    # ------------------------------------------------------------------ #
    # Step 2 — smooth and subsample                                       #
    # ------------------------------------------------------------------ #
    if method == "gaussian":
        lon_smooth = gaussian_filter1d(arr[:, 0], sigma=sigma)
        lat_smooth = gaussian_filter1d(arr[:, 1], sigma=sigma)
        # Tangent via finite differences on the smoothed positions.
        arr_smooth = np.column_stack((lon_smooth, lat_smooth))
        pts = arr_smooth[::step]
        dlon_arr = None  # computed on demand in step 3
        dlat_arr = None

    else:  # savgol
        # Ensure window_length is odd and feasible.
        wl = window_length if window_length % 2 == 1 else window_length + 1
        wl = max(wl, polyorder + 2 if (polyorder + 2) % 2 == 1 else polyorder + 3)
        wl = min(wl, len(arr))
        if wl % 2 == 0:
            wl -= 1

        lon_smooth = savgol_filter(arr[:, 0], window_length=wl, polyorder=polyorder)
        lat_smooth = savgol_filter(arr[:, 1], window_length=wl, polyorder=polyorder)
        # Analytical first derivative directly from the polynomial fit.
        dlon_smooth = savgol_filter(
            arr[:, 0], window_length=wl, polyorder=polyorder, deriv=1
        )
        dlat_smooth = savgol_filter(
            arr[:, 1], window_length=wl, polyorder=polyorder, deriv=1
        )
        arr_smooth = np.column_stack((lon_smooth, lat_smooth))
        pts = arr_smooth[::step]
        dlon_arr = dlon_smooth[::step]
        dlat_arr = dlat_smooth[::step]

    if len(pts) < 2:
        raise ValueError(
            "Fewer than two points remain after smoothing/subsampling; "
            "reduce smoothing parameters or *step*."
        )

    # ------------------------------------------------------------------ #
    # Step 3 — nearest-point lookup and tangent angle                     #
    # ------------------------------------------------------------------ #
    # q = np.asarray(query_point, dtype=float)
    # d2 = np.sum((pts - q) ** 2, axis=1)
    d2 = [distance.distance((query_point[1], query_point[0]), (pt[1], pt[0])).km ** 2 for pt in pts]
    idx = int(np.argmin(d2))

    if dlon_arr is not None:
        # savgol: use the analytical polynomial derivative directly —
        # more accurate than a finite-difference approximation.
        dx = float(dlon_arr[idx])
        dy = float(dlat_arr[idx])
    else:
        # gaussian: finite differences on the smoothed positions.
        if idx == 0:
            v = pts[1] - pts[0]
        elif idx == len(pts) - 1:
            v = pts[-1] - pts[-2]
        else:
            v = pts[idx + 1] - pts[idx - 1]
        dx, dy = float(v[0]), float(v[1])

    heading_deg = float(np.degrees(np.arctan2(dy, dx)))

    return {
        "heading_deg": heading_deg,
        "isobath_pts": [tuple(xy) for xy in pts],
        "nearest_point": (float(pts[idx, 0]), float(pts[idx, 1])),
        "nearest_index": idx,
        "method": method,
    }

```

```python
import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.gridspec as gridspec
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from matplotlib.patches import Patch
from tools_shapely import *
from tools_xtrackm import *
from isobath_heading_at_point import isobath_heading_at_point
# copilot --resume=d419a1e5-878f-477e-813b-3b06f3d9a50b


REG = (62, 92, 0, 30)
dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo_60s_io.nc")
topo = dse.rose.sel(lon=slice(*REG[:2])).sel(lat=slice(*REG[2:]))
common_kw = dict(topo=topo, level=-1000, step=15)
configs = dict(method="savgol", window_length=41, polyorder=3)


def along_across_shore_components(gu, gv, coast_heading_deg):
    """
    Resolve vector (gu, gv) into along-shore and across-shore components.
    Sign convention:
    - coast_heading_deg is measured CCW from +x (East).
    - along_shore is positive in the heading direction.
    - across_shore is positive 90 deg CCW from heading (left-hand side of coast).
      If you need right-hand positive, negate across_shore.
    """
    theta = np.deg2rad(coast_heading_deg)
    c = np.cos(theta)
    s = np.sin(theta)
    # Elementwise projection keeps scalar or array/xarray shape.
    along_shore = gu * c + gv * s
    across_shore = -gu * s + gv * c  # +90 deg CCW
    return along_shore, across_shore


method = "loess"
frac1 = "0.05"

sat_val = "TP+J1+J2+J3+S6A"
sat_val = "S3A"
sat_val = "ERS1+ERS2+ENV+SRL"

df = pd.read_csv(f"intersections_near_east_coast_{sat_val}.csv")

# Skip the first point (lowest latitude)
df_out = df.sort_values("lats_inter").iloc[1:].reset_index(drop=True)

N_PTS = 7
assert len(df_out) == N_PTS, (
    f"Expected {N_PTS} points after skipping first, got {len(df_out)}"
)

POINT_COLORS = [plt.cm.tab10(i) for i in range(N_PTS)]
POINT_LABELS = list("ABCDEFG")

# Map extent: 10N–22N, 78E–88E
MAP_EXTENT = (78.0, 88.0, 10.0, 22.0)

# --- Pre-collect all timeseries data ---
point_data = []
sigma = 3
for i, row in df_out.iterrows():
    sat1 = row["sat"]
    tno_s = str(row["track_self"])
    tno_o = str(row["track_other"])
    lon1 = float(row["lons_inter"])
    lat1 = float(row["lats_inter"])
    query_point = (lon1, lat1)

    result = isobath_heading_at_point(query_point=query_point, **common_kw, **configs)
    coast_heading_deg = result["heading_deg"]
    f_guv = (
        f"../guv/guv_at_intersections_{method}/"
        f"guv_at_intersection_{sat1}_{tno_s}_{tno_o}_{method}_{frac1}.nc"
    )
    ds_guv = xr.open_dataset(f_guv)
    gu = ds_guv.gu
    gv = ds_guv.gv
    gs = np.sqrt(gu**2 + gv**2)
    print(f"coast_heading_deg={coast_heading_deg:.1f}° at point {i} ({lon1:.2f}E, {lat1:.2f}N)")
    al, ac = along_across_shore_components(gu, gv, coast_heading_deg)
    times = pd.to_datetime(gs.time.values)
    # Monthly climatology (mean of each calendar month across all years)
    gs_clim = gs.groupby("time.month").mean("time")
    gu_clim = gu.groupby("time.month").mean("time")
    al_clim = al.groupby("time.month").mean("time")

    point_data.append(
        dict(
            sat1=sat1,
            tno_s=tno_s,
            tno_o=tno_o,
            lon1=lon1,
            lat1=lat1,
            al=al,
            al_clim=al_clim,
        )
    )

# --- Build figure: left = tall map, right = 7 timeseries rows ---
fig = plt.figure(figsize=(18, 14))
gs_layout = gridspec.GridSpec(
    N_PTS,
    2,
    figure=fig,
    width_ratios=[1, 2.5],
    hspace=0.0,
    wspace=0.3,
)

# --- Left: vertically elongated cartopy map spanning all rows ---
ax_map = fig.add_subplot(gs_layout[:, 0], projection=ccrs.PlateCarree())
ax_map.set_extent(MAP_EXTENT, crs=ccrs.PlateCarree())
```

```
ax_map.add_feature(cfeature.LAND, facecolor="0.93", zorder=0)
ax_map.add_feature(cfeature.OCEAN, facecolor="aliceblue", zorder=0)
ax_map.coastlines(resolution="50m", linewidth=1.0, zorder=2)
gl = ax_map.gridlines(
    draw_labels=True,
    linewidth=0.5,
    color="gray",
    alpha=0.6,
    linestyle="--",
)
gl.top_labels = False
gl.right_labels = False
# ax_map.set_title(f"{sat_val}\nIntersection points", fontsize=9)

# Connect all 7 points with a dashed gray line
lons_pts = [e["lon1"] for e in point_data]
lats_pts = [e["lat1"] for e in point_data]
ax_map.plot(
    lons_pts,
    lats_pts,
    color="gray",
    linewidth=0.8,
    linestyle="--",
    transform=ccrs.PlateCarree(),
    zorder=3,
)

# Plot each point with its assigned color and letter label
for idx, entry in enumerate(point_data):
    color = POINT_COLORS[idx]
    label = POINT_LABELS[idx]
    ax_map.plot(
        entry["lon1"],
        entry["lat1"],
        marker="o",
        markersize=9,
        color=color,
        markeredgecolor="black",
        markeredgewidth=0.8,
        transform=ccrs.PlateCarree(),
        zorder=5,
    )
    ax_map.text(
        entry["lon1"] + 0.18,
        entry["lat1"] + 0.1,
        label,
        fontsize=10,
        fontweight="bold",
        color=color,
        transform=ccrs.PlateCarree(),
        zorder=6,
    )

# --- Right: 7 monthly climatology subplots, A at bottom (row 6) → G at top (row 0) ---
MONTHS = np.arange(1, 13)
MONTH_NAMES = ["J", "F", "M", "A", "M", "J", "J", "A", "S", "O", "N", "D"]

for idx, entry in enumerate(point_data):
    row = N_PTS - 1 - idx  # A → row 6, G → row 0
    ax_ts = fig.add_subplot(gs_layout[row, 1])
    color = POINT_COLORS[idx]
    label = POINT_LABELS[idx]
    al_clim = entry["al_clim"]

    # Seasonal background shading (month-based x-axis)
    ax_ts.axvspan(2 - 0.5, 5 + 0.5, color="yellow", alpha=0.15, zorder=0)   # Feb–May
    ax_ts.axvspan(6 - 0.5, 9 + 0.5, color="cyan",   alpha=0.15, zorder=0)   # Jun–Sep
    ax_ts.axvspan(10 - 0.5, 12 + 0.5, color="gray", alpha=0.15, zorder=0)   # Oct–Dec

    # Monthly climatology bar plot
    # ax_ts.bar(
    #     MONTHS,
    #     gs_clim.values,
    #     color=color,
    #     alpha=0.8,
    #     edgecolor="k",
    #     linewidth=0.5,
    #     zorder=2,
    # )

    ax_ts.plot(
        MONTHS,
        al_clim.values,
        color=color,
        linewidth=3,
        zorder=2,
    )

    # Point label inside left edge of plot
    ax_ts.text(
        0.005, 0.5, label,
        transform=ax_ts.transAxes, fontsize=12,
        fontweight="bold", color=color,
        va="center", ha="left",
    )

    ax_ts.set_xlim(0.5, 12.5)
    all_vals = gs_clim.values[np.isfinite(gs_clim.values)]
    ypad = (all_vals.max() - all_vals.min()) * 0.08
    # ax_ts.set_ylim(-all_vals.max() - ypad, all_vals.max() + ypad)
    # ax_ts.set_ylim(-all_vals.max() - ypad, all_vals.max() + ypad)
    ax_ts.set_ylabel("m/s", fontsize=14)
    ax_ts.set_xticks(MONTHS)
    ax_ts.axhline(0, color="gray", linewidth=0.8, linestyle="--", zorder=1)
    ax_ts.tick_params(labelsize=12)
    # Only bottom subplot (A, idx=0) shows month labels
    if idx == 0:
        ax_ts.set_xticklabels(MONTH_NAMES)
    else:
        ax_ts.set_xticklabels([])

    # Legend only on the top panel (G, idx=N_PTS-1)
    if idx == N_PTS - 1:
        handles = [
            Patch(facecolor="yellow", alpha=0.4, edgecolor="none", label="Feb–May"),
            Patch(facecolor="cyan",   alpha=0.4, edgecolor="none", label="Jun–Sep"),
            Patch(facecolor="gray",   alpha=0.4, edgecolor="none", label="Oct–Dec"),
```

```
        ]
        ax_ts.legend(handles=handles, loc="upper right", fontsize=12, ncol=3)
        ax_ts.set_title("Monthly climatology of |u| (gs)", fontsize=11)

# fig.suptitle(
#     f"Current speed at EICC coastal intersection points — {sat_val}",
#     fontsize=13,
#     fontweight="bold",
#     y=1.002,
# )
out_png = f"speed_timeseries_allinone_{sat_val}.png"
plt.savefig(out_png, dpi=150, bbox_inches="tight")
plt.show()
plt.close("all")
print(f"Saved: {out_png}")

```

```python
import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.gridspec as gridspec
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from matplotlib.patches import Patch
from tools_shapely import *
from tools_xtrackm import *

# copilot --resume=d419a1e5-878f-477e-813b-3b06f3d9a50b


def solve_theta_cartesian(gu, gv, dim='time', decode_degrees=True):
    """
    Calculates the angle theta (CCW from East/X-axis) that minimizes
    the perpendicular variance S(theta) = sum((v*cos(theta) - u*sin(theta))**2).
    This identifies the orientation of the Major Axis (Principal Axis).
    Parameters:
    -----------
    u : xr.DataArray
        Zonal velocity component (East-West).
    v : xr.DataArray
        Meridional velocity component (North-South).
    dim : str
        The dimension to sum over (e.g., 'time').
    decode_degrees : bool
        If True, returns the result in degrees. Otherwise, radians.
    Returns:
    --------
    theta : float or xr.DataArray
        The calculated angle.
    """
    # It is standard practice to remove the mean if calculating
    # the principal axis of fluctuations (variance ellipse)
    u_prime = gu - gu.mean(dim=dim)
    v_prime = gv - gv.mean(dim=dim)
    # Calculate terms for the tan(2*theta) formula
    sum_uv = (u_prime * v_prime).sum(dim=dim)
    sum_u2 = (u_prime**2).sum(dim=dim)
    sum_v2 = (v_prime**2).sum(dim=dim)
    # Using np.arctan2(y, x) handles the quadrant correctly and
    # prevents division by zero if sum_u2 == sum_v2.
    # atan2(2 * sum_uv, sum_u2 - sum_v2)
    two_theta = np.arctan2(2 * sum_uv, sum_u2 - sum_v2)
    theta = 0.5 * two_theta
    if decode_degrees:
        return np.degrees(theta)
    return theta


REG = (62, 92, 0, 30)
dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo_60s_io.nc")
topo = dse.rose.sel(lon=slice(*REG[:2])).sel(lat=slice(*REG[2:]))


def along_across_shore_components(gu, gv, coast_heading_deg):
    """
    Resolve vector (gu, gv) into along-shore and across-shore components.
    Sign convention:
    - coast_heading_deg is measured CCW from +x (East).
    - along_shore is positive in the heading direction.
    - across_shore is positive 90 deg CCW from heading (left-hand side of coast).
      If you need right-hand positive, negate across_shore.
    """
    theta = np.deg2rad(coast_heading_deg)
    c = np.cos(theta)
    s = np.sin(theta)
    # Elementwise projection keeps scalar or array/xarray shape.
    along_shore = gu * c + gv * s
    across_shore = -gu * s + gv * c  # +90 deg CCW
    return along_shore, across_shore


method = "loess"
frac1 = "0.05"

sat_val = "TP+J1+J2+J3+S6A"
sat_val = "S3A"
sat_val = "ERS1+ERS2+ENV+SRL"

df = pd.read_csv(f"intersections_near_east_coast_{sat_val}.csv")

# Skip the first point (lowest latitude)
df_out = df.sort_values("lats_inter").iloc[1:].reset_index(drop=True)

N_PTS = 7
assert len(df_out) == N_PTS, (
    f"Expected {N_PTS} points after skipping first, got {len(df_out)}"
)

POINT_COLORS = [plt.cm.tab10(i) for i in range(N_PTS)]
POINT_LABELS = list("ABCDEFG")

# Map extent: 10N–22N, 78E–88E
MAP_EXTENT = (78.0, 88.0, 10.0, 22.0)

# --- Pre-collect all timeseries data ---
point_data = []
sigma = 3
for i, row in df_out.iterrows():
    sat1 = row["sat"]
    tno_s = str(row["track_self"])
    tno_o = str(row["track_other"])
    lon1 = float(row["lons_inter"])
    lat1 = float(row["lats_inter"])
    query_point = (lon1, lat1)

    f_guv = (
        f"../guv/guv_at_intersections_{method}/"
        f"guv_at_intersection_{sat1}_{tno_s}_{tno_o}_{method}_{frac1}.nc"
    )
    ds_guv = xr.open_dataset(f_guv)
    gu = ds_guv.gu
    gv = ds_guv.gv
    theta = solve_theta_cartesian(gu, gv, dim="time", decode_degrees=True)
```

```
    gs = np.sqrt(gu**2 + gv**2)
    print(f"coast_heading_deg={theta:.1f}° at point {i} ({lon1:.2f}E, {lat1:.2f}N)")
    al, ac = along_across_shore_components(gu, gv, theta)
    times = pd.to_datetime(gs.time.values)
    # Monthly climatology (mean of each calendar month across all years)
    gs_clim = gs.groupby("time.month").mean("time")
    gu_clim = gu.groupby("time.month").mean("time")
    al_clim = al.groupby("time.month").mean("time")

    point_data.append(
        dict(
            sat1=sat1,
            tno_s=tno_s,
            tno_o=tno_o,
            lon1=lon1,
            lat1=lat1,
            al=al,
            al_clim=al_clim,
        )
    )

# --- Build figure: left = tall map, right = 7 timeseries rows ---
fig = plt.figure(figsize=(18, 14))
gs_layout = gridspec.GridSpec(
    N_PTS,
    2,
    figure=fig,
    width_ratios=[1, 2.5],
    hspace=0.0,
    wspace=0.3,
)

# --- Left: vertically elongated cartopy map spanning all rows ---
ax_map = fig.add_subplot(gs_layout[:, 0], projection=ccrs.PlateCarree())
ax_map.set_extent(MAP_EXTENT, crs=ccrs.PlateCarree())
ax_map.add_feature(cfeature.LAND, facecolor="0.93", zorder=0)
ax_map.add_feature(cfeature.OCEAN, facecolor="aliceblue", zorder=0)
ax_map.coastlines(resolution="50m", linewidth=1.0, zorder=2)
gl = ax_map.gridlines(
    draw_labels=True,
    linewidth=0.5,
    color="gray",
    alpha=0.6,
    linestyle="--",
)
gl.top_labels = False
gl.right_labels = False
# ax_map.set_title(f"{sat_val}\nIntersection points", fontsize=9)

# Connect all 7 points with a dashed gray line
lons_pts = [e["lon1"] for e in point_data]
lats_pts = [e["lat1"] for e in point_data]
ax_map.plot(
    lons_pts,
    lats_pts,
    color="gray",
    linewidth=0.8,
    linestyle="--",
    transform=ccrs.PlateCarree(),
    zorder=3,
)

# Plot each point with its assigned color and letter label
for idx, entry in enumerate(point_data):
    color = POINT_COLORS[idx]
    label = POINT_LABELS[idx]
    ax_map.plot(
        entry["lon1"],
        entry["lat1"],
        marker="o",
        markersize=9,
        color=color,
        markeredgecolor="black",
        markeredgewidth=0.8,
        transform=ccrs.PlateCarree(),
        zorder=5,
    )
    ax_map.text(
        entry["lon1"] + 0.18,
        entry["lat1"] + 0.1,
        label,
        fontsize=10,
        fontweight="bold",
        color=color,
        transform=ccrs.PlateCarree(),
        zorder=6,
    )

# --- Right: 7 monthly climatology subplots, A at bottom (row 6) → G at top (row 0) ---
MONTHS = np.arange(1, 13)
MONTH_NAMES = ["J", "F", "M", "A", "M", "J", "J", "A", "S", "O", "N", "D"]

for idx, entry in enumerate(point_data):
    row = N_PTS - 1 - idx  # A → row 6, G → row 0
    ax_ts = fig.add_subplot(gs_layout[row, 1])
    color = POINT_COLORS[idx]
    label = POINT_LABELS[idx]
    al_clim = entry["al_clim"]

    # Seasonal background shading (month-based x-axis)
    ax_ts.axvspan(2 - 0.5, 5 + 0.5, color="yellow", alpha=0.15, zorder=0)   # Feb–May
    ax_ts.axvspan(6 - 0.5, 9 + 0.5, color="cyan",   alpha=0.15, zorder=0)   # Jun–Sep
    ax_ts.axvspan(10 - 0.5, 12 + 0.5, color="gray", alpha=0.15, zorder=0)   # Oct–Dec

    # Monthly climatology bar plot
    # ax_ts.bar(
    #     MONTHS,
    #     gs_clim.values,
    #     color=color,
    #     alpha=0.8,
    #     edgecolor="k",
    #     linewidth=0.5,
    #     zorder=2,
    # )

    ax_ts.plot(
        MONTHS,
```

```
        al_clim.values,
        color=color,
        linewidth=3,
        zorder=2,
    )

    # Point label inside left edge of plot
    ax_ts.text(
        0.005, 0.5, label,
        transform=ax_ts.transAxes, fontsize=12,
        fontweight="bold", color=color,
        va="center", ha="left",
    )

    ax_ts.set_xlim(0.5, 12.5)
    all_vals = gs_clim.values[np.isfinite(gs_clim.values)]
    ypad = (all_vals.max() - all_vals.min()) * 0.08
    # ax_ts.set_ylim(-all_vals.max() - ypad, all_vals.max() + ypad)
    # ax_ts.set_ylim(-all_vals.max() - ypad, all_vals.max() + ypad)
    ax_ts.set_ylabel("m/s", fontsize=14)
    ax_ts.set_xticks(MONTHS)
    ax_ts.axhline(0, color="gray", linewidth=0.8, linestyle="--", zorder=1)
    ax_ts.tick_params(labelsize=12)
    # Only bottom subplot (A, idx=0) shows month labels
    if idx == 0:
        ax_ts.set_xticklabels(MONTH_NAMES)
    else:
        ax_ts.set_xticklabels([])

    # Legend only on the top panel (G, idx=N_PTS-1)
    if idx == N_PTS - 1:
        handles = [
            Patch(facecolor="yellow", alpha=0.4, edgecolor="none", label="Feb–May"),
            Patch(facecolor="cyan",   alpha=0.4, edgecolor="none", label="Jun–Sep"),
            Patch(facecolor="gray",   alpha=0.4, edgecolor="none", label="Oct–Dec"),
        ]
        ax_ts.legend(handles=handles, loc="upper right", fontsize=12, ncol=3)
        ax_ts.set_title("Monthly climatology of |u| (gs)", fontsize=11)

# fig.suptitle(
#     f"Current speed at EICC coastal intersection points — {sat_val}",
#     fontsize=13,
#     fontweight="bold",
#     y=1.002,
# )
out_png = f"speed_timeseries_allinone_{sat_val}.png"
plt.savefig(out_png, dpi=150, bbox_inches="tight")
plt.show()
plt.close("all")
print(f"Saved: {out_png}")

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
from tools_shapely import *


def along_across_shore_components(gu, gv, coast_heading_deg):
    """
    Resolve vector (gu, gv) into along-shore and across-shore components.
    Sign convention:
    - coast_heading_deg is measured CCW from +x (East).
    - along_shore is positive in the heading direction.
    - across_shore is positive 90 deg CCW from heading (left-hand side of coast).
      If you need right-hand positive, negate across_shore.
    """
    theta = np.deg2rad(coast_heading_deg)
    c = np.cos(theta)
    s = np.sin(theta)
    # Elementwise projection keeps scalar or array/xarray shape.
    along_shore = gu * c + gv * s
    across_shore = -gu * s + gv * c  # +90 deg CCW
    return along_shore, across_shore


method = "loess"
cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

sat_val = "TP+J1+J2+J3+S6A"
track_self_val = 155
track_other_val = 116

df = pd.read_csv(f"tracks_intersections_{sat_val}_1.csv")
df_out = df.copy()

rows = df_out.loc[
    (df_out["sat"] == sat_val)
    & (df_out["track_self"] == track_self_val)
    & (df_out["track_other"] == track_other_val)
]
# Optional input: Gaussian smoothing sigma
sigma = 3
# Optional input: clipping bbox tuple
# Default used by the function is (70, 5, 95, 30)
region_bbox_tuple = (70, 5, 95, 30)

frac1 = "0.05"
i = 0
sat1 = rows["sat"].values[i]
# track_number_self = rows["track_self"].apply(lambda x: str(x)).values[i]
# track_number_other = rows["track_other"].apply(lambda x: str(x)).values[i]
track_number_self = str(rows["track_self"].values[i])
track_number_other = str(rows["track_other"].values[i])
lons_inter1 = rows["lons_inter"].values[i]
lats_inter1 = rows["lats_inter"].values[i]
lon1 = lons_inter1
lat1 = lats_inter1
query_point = (lon1, lat1)
result = coastline_gaussian_tangent(
    query_point=query_point,
    sigma=sigma,
    region_bbox_tuple=region_bbox_tuple,
)
print("heading_angle_deg:", result["heading_angle_deg"])
print("tangent_vector:", result["tangent_vector"])
print("nearest_point:", result["nearest_point"])
print("distance:", result["distance"])
coast_heading_deg = result["heading_angle_deg"]
print(
    sat1,
    track_number_self,
    track_number_other,
    lons_inter1,
    lats_inter1,
)
f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
f_guv = (
    f"../guv/guv_at_intersections_{method}/"
    f"guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{method}_{frac1}.nc"
)
ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
ds_guv = xr.open_dataset(f_guv)
lons_track_self = ds_self.lon.values
lats_track_self = ds_self.lat.values
lon_coast_self = lons_track_self[-1]  # this on coast
lat_coast_self = lats_track_self[-1]  # this on coast
lon_equat_self = lons_track_self[0]  # this on equator
lat_equat_self = lats_track_self[0]  # this on equator
lons_track_other = ds_other.lon.values
lats_track_other = ds_other.lat.values
lon_equat_other = lons_track_other[0]
lat_equat_other = lats_track_other[0]
lon_coast_other = lons_track_other[-1]
lat_coast_other = lats_track_other[-1]
iself = index_at_lat(ds_self, lat1)
a1_d = local_angle(
    lons_track_self, lats_track_self, iself, ioff=3, degrees=True
)
iothr = index_at_lat(ds_other, lat1)
a2_d = local_angle(
    lons_track_other, lats_track_other, iothr, ioff=3, degrees=True
)
print(f"heading angle {a1_d:.2f} {a2_d:.2f}")
gu = ds_guv.gu
gv = ds_guv.gv
al, ac = along_across_shore_components(gu, gv, coast_heading_deg)
fig = plt.figure(figsize=(12, 10))
ax1 = fig.add_subplot(3, 1, 3)
ax0 = fig.add_subplot(3, 1, 2)
al.plot(ax=ax0, label="alongshore", color="r", linewidth=2)
ax0.legend()
```

```
ax0.set_ylabel("along current m/s")
ac.plot(ax=ax1, label="acrossshore", color="r", linewidth=2)
info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
plt.text(
    0.15,
    0.90,
    info,
    transform=ax0.transAxes,
    fontsize=14,
    fontweight="bold",
)
ax1.legend()
ax1.set_ylabel("across current m/s")
ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=5)
ax2.grid()
ax2.plot(
    lon1,
    lat1,
    c="r",
    marker="o",
    markersize=6,
    markerfacecolor="red",
    markeredgecolor="black",
    markeredgewidth=2,
)
ax2.scatter(
    lons_track_self,
    lats_track_self,
    marker=".",
    color="c",
    s=4,
)
ax2.scatter(
    lons_track_other,
    lats_track_other,
    marker=".",
    color="c",
    s=4,
)
lonm, latm = get_point_at_distance(
    lon_equat_self, lat_equat_self, lon_coast_self, lat_coast_self, d
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=a1_d,
        color="k",
    )
lonm, latm = get_point_at_distance(
    lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=a2_d,
        color="k",
    )
plt.savefig(
    f"along_shore_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
)
plt.show()
plt.close("all")

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
from tools_shapely import *
# copilot --resume=6b563fd7-e036-45d8-95d0-bd5af130d84f


def along_across_shore_components(gu, gv, coast_heading_deg):
    """
   Resolve vector (gu, gv) into along-shore and across-shore components.
   Sign convention:
    - coast_heading_deg is measured CCW from +x (East).
    - along_shore is positive in the heading direction.
    - across_shore is positive 90 deg CCW from heading (left-hand side of coast).
      If you need right-hand positive, negate across_shore.
    """
    theta = np.deg2rad(coast_heading_deg)
    c = np.cos(theta)
    s = np.sin(theta)
    # Elementwise projection keeps scalar or array/xarray shape.
    along_shore = gu * c + gv * s
    across_shore = -gu * s + gv * c  # +90 deg CCW
    return along_shore, across_shore


method = "loess"
cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

sat_val = "TP+J1+J2+J3+S6A"
track_self_val = 155
track_other_val = 116

df = pd.read_csv(f"tracks_intersections_{sat_val}_1.csv")
df_out = df.copy()

rows = df_out.loc[
    (df_out["sat"] == sat_val)
    & (df_out["track_self"] == track_self_val)
    & (df_out["track_other"] == track_other_val)
]
# Optional input: Gaussian smoothing sigma
sigma = 3
# Optional input: clipping bbox tuple
# Default used by the function is (70, 5, 95, 30)
region_bbox_tuple = (70, 5, 95, 30)

frac1 = "0.05"
i = 0
sat1 = rows["sat"].values[i]
# track_number_self = rows["track_self"].apply(lambda x: str(x)).values[i]
# track_number_other = rows["track_other"].apply(lambda x: str(x)).values[i]
track_number_self = str(rows["track_self"].values[i])
track_number_other = str(rows["track_other"].values[i])
lons_inter1 = rows["lons_inter"].values[i]
lats_inter1 = rows["lats_inter"].values[i]
lon1 = lons_inter1
lat1 = lats_inter1
query_point = (lon1, lat1)
result = coastline_gaussian_tangent(
    query_point=query_point,
    sigma=sigma,
    region_bbox_tuple=region_bbox_tuple,
)
print("heading_angle_deg:", result["heading_angle_deg"])
print("tangent_vector:", result["tangent_vector"])
print("nearest_point:", result["nearest_point"])
print("distance:", result["distance"])
coast_heading_deg = result["heading_angle_deg"]
print(
    sat1,
    track_number_self,
    track_number_other,
    lons_inter1,
    lats_inter1,
)
f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
f_guv = (
    f"../guv/guv_at_intersections_{method}/"
    f"guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{method}_{frac1}.nc"
)
ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
ds_guv = xr.open_dataset(f_guv)
lons_track_self = ds_self.lon.values
lats_track_self = ds_self.lat.values
lon_coast_self = lons_track_self[-1]  # this on coast
lat_coast_self = lats_track_self[-1]  # this on coast
lon_equat_self = lons_track_self[0]  # this on equator
lat_equat_self = lats_track_self[0]  # this on equator
lons_track_other = ds_other.lon.values
lats_track_other = ds_other.lat.values
lon_equat_other = lons_track_other[0]
lat_equat_other = lats_track_other[0]
lon_coast_other = lons_track_other[-1]
lat_coast_other = lats_track_other[-1]
iself = index_at_lat(ds_self, lat1)
a1_d = local_angle(
    lons_track_self, lats_track_self, iself, ioff=3, degrees=True
)
iothr = index_at_lat(ds_other, lat1)
a2_d = local_angle(
    lons_track_other, lats_track_other, iothr, ioff=3, degrees=True
)
print(f"heading angle {a1_d:.2f} {a2_d:.2f}")
gu = ds_guv.gu
gv = ds_guv.gv
al, ac = along_across_shore_components(gu, gv, coast_heading_deg)

# Bandpass filter: remove periods shorter than 45 days (high-freq noise)
from scipy.signal import butter, filtfilt

```

```
al_vals_raw = al.values.copy()
dt_days_bp = np.median(np.diff(pd.to_datetime(al.time.values)).astype("timedelta64[D]").astype(float))
fs_bp = 1.0 / dt_days_bp  # cycles per day
# Low-pass cutoff: 1/45 cycles per day (keep periods > 45 days)
cutoff = 1.0 / 60.0
# Nyquist frequency
nyq = fs_bp / 2.0
b_filt, a_filt = butter(4, cutoff / nyq, btype="low")
# Interpolate NaNs for filtering, then restore NaN positions
nan_mask = np.isnan(al_vals_raw)
al_interp = pd.Series(al_vals_raw).interpolate(limit_direction="both").values
al_filtered = filtfilt(b_filt, a_filt, al_interp)
al_filtered[nan_mask] = np.nan
al = al.copy(data=al_filtered)

fig = plt.figure(figsize=(12, 10))
ax1 = fig.add_subplot(3, 1, 3)
ax0 = fig.add_subplot(3, 1, 2)
al.plot(ax=ax0, label="alongshore", color="r", linewidth=2)
ax0.legend()
ax0.set_ylabel("along current m/s")
info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
plt.text(
    0.15,
    0.90,
    info,
    transform=ax0.transAxes,
    fontsize=14,
    fontweight="bold",
)

# --- Seasonal shading and year markers on along-shore plot (ax0) ---
times = pd.to_datetime(al.time.values)
year_min, year_max = times.year.min(), times.year.max()
for yr in range(year_min, year_max + 1):
    # Feb-May: yellow
    ax0.axvspan(pd.Timestamp(yr, 2, 1), pd.Timestamp(yr, 5, 31), color="yellow", alpha=0.15, zorder=0)
    # Jun-Sep: cyan
    ax0.axvspan(pd.Timestamp(yr, 6, 1), pd.Timestamp(yr, 9, 30), color="cyan", alpha=0.15, zorder=0)
    # Oct-Dec: gray
    ax0.axvspan(pd.Timestamp(yr, 10, 1), pd.Timestamp(yr, 12, 31), color="gray", alpha=0.15, zorder=0)
    # Year boundary vertical line on Jan 1
    ax0.axvline(pd.Timestamp(yr, 1, 1), color="k", linewidth=0.5, linestyle="--", alpha=0.5)

# --- Periodogram of along-shore current ---
from scipy.signal import periodogram as scipy_periodogram

al_vals = al.values
al_times = pd.to_datetime(al.time.values)
# Compute median sampling interval in days
dt_days = np.median(np.diff(al_times).astype("timedelta64[D]").astype(float))
fs = 1.0 / dt_days  # sampling frequency in cycles/day

# Remove NaNs for periodogram
mask = np.isfinite(al_vals)
al_clean = al_vals[mask]
al_clean = al_clean - np.nanmean(al_clean)

freqs, psd = scipy_periodogram(al_clean, fs=fs, scaling="density")
# Convert frequency to period in days (skip freq=0)
freqs = freqs[1:]
psd = psd[1:]
periods = 1.0 / freqs

ax1.semilogy(periods, psd, color="b", linewidth=1)
ax1.set_xlabel("Period (days)")
ax1.set_ylabel("PSD (m²/s²/cpd)")
ax1.set_title("Periodogram of along-shore current")
ax1.set_xlim(0, 600)

# Mark important periodicities
important_periods = {"365 d": 365, "180 d": 180, "120 d": 120, "90 d": 90, "60 d": 60}
for label, p in important_periods.items():
    ax1.axvline(p, color="r", linewidth=0.8, linestyle="--", alpha=0.7)
    ax1.text(p, ax1.get_ylim()[1] * 0.5, label, rotation=90, va="top", ha="right",
             fontsize=9, color="r", alpha=0.9)
ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=5)
ax2.grid()
ax2.plot(
    lon1,
    lat1,
    c="r",
    marker="o",
    markersize=6,
    markerfacecolor="red",
    markeredgecolor="black",
    markeredgewidth=2,
)
ax2.scatter(
    lons_track_self,
    lats_track_self,
    marker=".",
    color="c",
    s=4,
)
ax2.scatter(
    lons_track_other,
    lats_track_other,
    marker=".",
    color="c",
    s=4,
)
lonm, latm = get_point_at_distance(
    lon_equat_self, lat_equat_self, lon_coast_self, lat_coast_self, d
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=al_d,
```

```
        color="k",
    )
lonm, latm = get_point_at_distance(
    lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=a2_d,
        color="k",
    )
plt.savefig(
    f"along_shore_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
)
plt.show()
plt.close("all")

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
from tools_shapely import *
from scipy.signal import butter, filtfilt
from scipy.signal import periodogram as scipy_periodogram
# copilot --resume=6b563fd7-e036-45d8-95d0-bd5af130d84f


def along_across_shore_components(gu, gv, coast_heading_deg):
    """
    Resolve vector (gu, gv) into along-shore and across-shore components.
    Sign convention:
    - coast_heading_deg is measured CCW from +x (East).
    - along_shore is positive in the heading direction.
    - across_shore is positive 90 deg CCW from heading (left-hand side of coast).
      If you need right-hand positive, negate across_shore.
    """
    theta = np.deg2rad(coast_heading_deg)
    c = np.cos(theta)
    s = np.sin(theta)
    # Elementwise projection keeps scalar or array/xarray shape.
    along_shore = gu * c + gv * s
    across_shore = -gu * s + gv * c  # +90 deg CCW
    return along_shore, across_shore


method = "loess"
cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

sat_val = "ERS1+ERS2+ENV+SRL"
sat_val = "TP+J1+J2+J3+S6A"
sat_val = "S3A"

df = pd.read_csv(f"intersections_near_east_coast_{sat_val}.csv")
df_out = df.copy()

# Optional input: Gaussian smoothing sigma
sigma = 3
# Optional input: clipping bbox tuple
# Default used by the function is (70, 5, 95, 30)
region_bbox_tuple = (70, 5, 95, 30)

frac1 = "0.05"
for i, rows in df_out.iterrows():
    sat1 = rows["sat"]
    track_number_self = str(rows["track_self"])
    track_number_other = str(rows["track_other"])
    lons_inter1 = rows["lons_inter"]
    lats_inter1 = rows["lats_inter"]
    lon1 = lons_inter1
    lat1 = lats_inter1
    query_point = (lon1, lat1)
    result = coastline_gaussian_tangent(
        query_point=query_point,
        sigma=sigma,
        region_bbox_tuple=region_bbox_tuple,
    )
    print("heading_angle_deg:", result["heading_angle_deg"])
    print("tangent_vector:", result["tangent_vector"])
    print("nearest_point:", result["nearest_point"])
    print("distance:", result["distance"])
    coast_heading_deg = result["heading_angle_deg"]
    print(
        sat1,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
    )
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_guv = (
        f"../guv/guv_at_intersections_{method}/"
        f"guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{method}_{frac1}.nc"
    )
    ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
    ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
    ds_guv = xr.open_dataset(f_guv)
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    iself = index_at_lat(ds_self, lat1)
    a1_d = local_angle(
        lons_track_self, lats_track_self, iself, ioff=3, degrees=True
    )
    iothr = index_at_lat(ds_other, lat1)
    a2_d = local_angle(
        lons_track_other, lats_track_other, iothr, ioff=3, degrees=True
    )
    print(f"heading angle {a1_d:.2f} {a2_d:.2f}")
    gu = ds_guv.gu
    gv = ds_guv.gv
    al, ac = along_across_shore_components(gu, gv, coast_heading_deg)
    # Bandpass filter: remove periods shorter than 45 days (high-freq noise)
    # al_vals_raw = al.values.copy()
    # dt_days_bp = np.median(np.diff(pd.to_datetime(al.time.values)).astype("timedelta64[D]").astype(float))
    # fs_bp = 1.0 / dt_days_bp  # cycles per day
    # # Low-pass cutoff: 1/45 cycles per day (keep periods > 45 days)
    # cutoff = 1.0 / 60.0
    # # Nyquist frequency
    # nyq = fs_bp / 2.0
    # b_filt, a_filt = butter(4, cutoff / nyq, btype="low")
```

```
    # # Interpolate NaNs for filtering, then restore NaN positions
    # nan_mask = np.isnan(al_vals_raw)
    # al_interp = pd.Series(al_vals_raw).interpolate(limit_direction="both").values
    # al_filtered = filtfilt(b_filt, a_filt, al_interp)
    # al_filtered[nan_mask] = np.nan
    # al = al.copy(data=al_filtered)

    fig = plt.figure(figsize=(12, 10))
    ax1 = fig.add_subplot(3, 1, 3)
    ax0 = fig.add_subplot(3, 1, 2)
    al.plot(ax=ax0, label="alongshore", color="r", linewidth=2)
    ax0.legend()
    ax0.set_ylabel("along current m/s")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
    plt.text(
        0.15,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )

    # --- Seasonal shading and year markers on along-shore plot (ax0) ---
    times = pd.to_datetime(al.time.values)
    year_min, year_max = times.year.min(), times.year.max()
    for yr in range(year_min, year_max + 1):
        # Feb-May: yellow
        ax0.axvspan(pd.Timestamp(yr, 2, 1), pd.Timestamp(yr, 5, 31), color="yellow", alpha=0.15, zorder=0)
        # Jun-Sep: cyan
        ax0.axvspan(pd.Timestamp(yr, 6, 1), pd.Timestamp(yr, 9, 30), color="cyan", alpha=0.15, zorder=0)
        # Oct-Dec: gray
        ax0.axvspan(pd.Timestamp(yr, 10, 1), pd.Timestamp(yr, 12, 31), color="gray", alpha=0.15, zorder=0)
        # Year boundary vertical line on Jan 1
        ax0.axvline(pd.Timestamp(yr, 1, 1), color="k", linewidth=0.5, linestyle="--", alpha=0.5)

    # --- Periodogram of along-shore current ---
    al_vals = al.values
    al_times = pd.to_datetime(al.time.values)
    # Compute median sampling interval in days
    dt_days = np.median(np.diff(al_times).astype("timedelta64[D]").astype(float))
    fs = 1.0 / dt_days  # sampling frequency in cycles/day

    # Remove NaNs for periodogram
    mask = np.isfinite(al_vals)
    al_clean = al_vals[mask]
    al_clean = al_clean - np.nanmean(al_clean)

    freqs, psd = scipy_periodogram(al_clean, fs=fs, scaling="density")
    # Convert frequency to period in days (skip freq=0)
    freqs = freqs[1:]
    psd = psd[1:]
    periods = 1.0 / freqs

    ax1.semilogy(periods, psd, color="b", linewidth=1)
    ax1.set_xlabel("Period (days)")
    ax1.set_ylabel("PSD (m²/s²/cpd)")
    ax1.set_title("Periodogram of along-shore current")
    ax1.set_xlim(0, 600)

    # Mark important periodicities
    important_periods = {"365 d": 365, "180 d": 180, "120 d": 120, "90 d": 90, "60 d": 60}
    for label, p in important_periods.items():
        ax1.axvline(p, color="r", linewidth=0.8, linestyle="--", alpha=0.7)
        ax1.text(p, ax1.get_ylim()[1] * 0.5, label, rotation=90, va="top", ha="right",
                 fontsize=9, color="r", alpha=0.9)
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
        ETOPO05_Y=slice(*TRACKS_REG[2:])
    )
    rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor="red",
        markeredgecolor="black",
        markeredgewidth=2,
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(
        lon_equat_self, lat_equat_self, lon_coast_self, lat_coast_self, d
    )
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=a1_d,
            color="k",
        )
    lonm, latm = get_point_at_distance(
        lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d
    )
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
```

```
            fontsize=14,
            rotation=a2_d,
            color="k",
        )
    plt.savefig(
        f"along_shore_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    plt.show()
    plt.close("all")

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
from tools_shapely import *
from scipy.signal import butter, filtfilt
from scipy.signal import periodogram as scipy_periodogram
# copilot --resume=6b563fd7-e036-45d8-95d0-bd5af130d84f


def along_across_shore_components(gu, gv, coast_heading_deg):
    """
    Resolve vector (gu, gv) into along-shore and across-shore components.
    Sign convention:
    - coast_heading_deg is measured CCW from +x (East).
    - along_shore is positive in the heading direction.
    - across_shore is positive 90 deg CCW from heading (left-hand side of coast).
      If you need right-hand positive, negate across_shore.
    """
    theta = np.deg2rad(coast_heading_deg)
    c = np.cos(theta)
    s = np.sin(theta)
    # Elementwise projection keeps scalar or array/xarray shape.
    along_shore = gu * c + gv * s
    across_shore = -gu * s + gv * c  # +90 deg CCW
    return along_shore, across_shore


method = "loess"
cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

sat_val = "ERS1+ERS2+ENV+SRL"
sat_val = "TP+J1+J2+J3+S6A"
sat_val = "S3A"

df = pd.read_csv(f"intersections_near_east_coast_{sat_val}.csv")
df_out = df.copy()

# Optional input: Gaussian smoothing sigma
sigma = 3
# Optional input: clipping bbox tuple
# Default used by the function is (70, 5, 95, 30)
region_bbox_tuple = (70, 5, 95, 30)

frac1 = "0.05"
for i, rows in df_out.iterrows():
    sat1 = rows["sat"]
    track_number_self = str(rows["track_self"])
    track_number_other = str(rows["track_other"])
    lons_inter1 = rows["lons_inter"]
    lats_inter1 = rows["lats_inter"]
    lon1 = lons_inter1
    lat1 = lats_inter1
    query_point = (lon1, lat1)
    result = coastline_gaussian_tangent(
        query_point=query_point,
        sigma=sigma,
        region_bbox_tuple=region_bbox_tuple,
    )
    print("heading_angle_deg:", result["heading_angle_deg"])
    print("tangent_vector:", result["tangent_vector"])
    print("nearest_point:", result["nearest_point"])
    print("distance:", result["distance"])
    coast_heading_deg = result["heading_angle_deg"]
    print(
        sat1,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
    )
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_guv = (
        f"../guv/guv_at_intersections_{method}/"
        f"guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{method}_{frac1}.nc"
    )
    ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
    ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
    ds_guv = xr.open_dataset(f_guv)
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    iself = index_at_lat(ds_self, lat1)
    a1_d = local_angle(
        lons_track_self, lats_track_self, iself, ioff=3, degrees=True
    )
    iothr = index_at_lat(ds_other, lat1)
    a2_d = local_angle(
        lons_track_other, lats_track_other, iothr, ioff=3, degrees=True
    )
    print(f"heading angle {a1_d:.2f} {a2_d:.2f}")
    gu = ds_guv.gu
    gv = ds_guv.gv
    al, ac = along_across_shore_components(gu, gv, coast_heading_deg)
    # Bandpass filter: remove periods shorter than 45 days (high-freq noise)
    # al_vals_raw = al.values.copy()
    # dt_days_bp = np.median(np.diff(pd.to_datetime(al.time.values)).astype("timedelta64[D]").astype(float))
    # fs_bp = 1.0 / dt_days_bp  # cycles per day
    # # Low-pass cutoff: 1/45 cycles per day (keep periods > 45 days)
    # cutoff = 1.0 / 60.0
    # # Nyquist frequency
    # nyq = fs_bp / 2.0
    # b_filt, a_filt = butter(4, cutoff / nyq, btype="low")
```

```
    # # Interpolate NaNs for filtering, then restore NaN positions
    # nan_mask = np.isnan(al_vals_raw)
    # al_interp = pd.Series(al_vals_raw).interpolate(limit_direction="both").values
    # al_filtered = filtfilt(b_filt, a_filt, al_interp)
    # al_filtered[nan_mask] = np.nan
    # al = al.copy(data=al_filtered)

    fig = plt.figure(figsize=(12, 10))
    ax0 = fig.add_subplot(3, 1, 2)
    al.plot(ax=ax0, label="alongshore", color="r", linewidth=2)
    ax0.legend()
    ax0.set_ylabel("along current m/s")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
    plt.text(
        0.15,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )

    # --- Seasonal shading and year markers on along-shore plot (ax0) ---
    times = pd.to_datetime(al.time.values)
    year_min, year_max = times.year.min(), times.year.max()
    for yr in range(year_min, year_max + 1):
        # Feb-May: yellow
        ax0.axvspan(pd.Timestamp(yr, 2, 1), pd.Timestamp(yr, 5, 31), color="yellow", alpha=0.15, zorder=0)
        # Jun-Sep: cyan
        ax0.axvspan(pd.Timestamp(yr, 6, 1), pd.Timestamp(yr, 9, 30), color="cyan", alpha=0.15, zorder=0)
        # Oct-Dec: gray
        ax0.axvspan(pd.Timestamp(yr, 10, 1), pd.Timestamp(yr, 12, 31), color="gray", alpha=0.15, zorder=0)
        # Year boundary vertical line on Jan 1
        ax0.axvline(pd.Timestamp(yr, 1, 1), color="k", linewidth=0.5, linestyle="--", alpha=0.5)

    # --- Periodogram of along-shore current ---
    al_vals = al.values
    al_times = pd.to_datetime(al.time.values)
    # Compute median sampling interval in days
    dt_days = np.median(np.diff(al_times).astype("timedelta64[D]").astype(float))
    fs = 1.0 / dt_days  # sampling frequency in cycles/day

    # Remove NaNs for periodogram
    mask = np.isfinite(al_vals)
    al_clean = al_vals[mask]
    al_clean = al_clean - np.nanmean(al_clean)

    freqs, psd = scipy_periodogram(al_clean, fs=fs, scaling="density")
    # Convert frequency to period in days (skip freq=0)
    freqs = freqs[1:]
    psd = psd[1:]
    periods = 1.0 / freqs

    # Mark important periodicities
    important_periods = {"365 d": 365, "180 d": 180, "120 d": 120, "90 d": 90, "60 d": 60}
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
        ETOPO05_Y=slice(*TRACKS_REG[2:])
    )
    rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor="red",
        markeredgecolor="black",
        markeredgewidth=2,
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(
        lon_equat_self, lat_equat_self, lon_coast_self, lat_coast_self, d
    )
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=a1_d,
            color="k",
        )
    lonm, latm = get_point_at_distance(
        lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d
    )
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=a2_d,
            color="k",
        )
    plt.savefig(
        f"along_shore_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    plt.show()
    plt.close("all")

```

```python
"""Simple standalone example for coastline_gaussian_tangent().

Inputs:
- query_point
- sigma (Gaussian smoothing)
- optional region_bbox_tuple = (lon_min, lat_min, lon_max, lat_max)

Outputs:
- heading angle in degrees
- tangent_vector
- nearest_point
- distance
"""

from tools_shapely import coastline_gaussian_tangent


# Required input: (lon, lat)
query_point = (83.62, 17.17)

# Optional input: Gaussian smoothing sigma
sigma = 3

# Optional input: clipping bbox tuple
# Default used by the function is (70, 5, 95, 30)
region_bbox_tuple = (70, 5, 95, 30)


result = coastline_gaussian_tangent(
    query_point=query_point,
    sigma=sigma,
    region_bbox_tuple=region_bbox_tuple,
)

print("heading_angle_deg:", result["heading_angle_deg"])
print("tangent_vector:", result["tangent_vector"])
print("nearest_point:", result["nearest_point"])
print("distance:", result["distance"])

```

```python
import xarray as xr
from tools_xtrackm import *
from namesm import sats_new
import glob
import os
from matplotlib.collections import LineCollection

sat = "ERS1+ERS2+ENV+SRL"

REG = (80.0, 92.0, 11.0, 21.0)
WBOB = (78.0, 87.0, 8.0, 21.0)
WBOB2 = (78.0, 97.0, 0.0, 25.0)
proj = ccrs.PlateCarree()

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 10),
                       subplot_kw={'projection': proj})
# adjust subplots to have minimum space between them
month_num = 2
month_name = months[month_num+1]
info =f"{sat} {months[month_num+1]}"
fig.subplots_adjust(wspace=0.1)
ax1.set_extent([*WBOB2], crs=proj)
ax1.coastlines()
# ax1.legend()
gl = ax1.gridlines(draw_labels=True, linewidth=0.5, color='gray', alpha=0.7, linestyle='--')
gl.top_labels = gl.right_labels = False  # Hide top and right labels
plt.text(0.3, 0.95, info,
         horizontalalignment='center',
         verticalalignment='center',
         fontsize=15,
         fontweight="bold",
         transform=ax1.transAxes)
for f_sla in sorted(
    glob.glob(
        f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
    )
):
    basename = os.path.basename(f_sla)
    ds_sla = xr.open_dataset(f_sla, decode_times=False)
    trackn = ds_sla.Pass
    # f = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{trackn}_loess_0.2.nc"
    f = f"../computed/sla_loess_0.05a/track_sla_along_{sat}_{trackn}_loess_0.05.nc"
    lons_track = ds_sla.lon.values
    lats_track = ds_sla.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    lon_equat = lons_track[0]
    lat_equat = lats_track[0]
    # compute tangent vectors or slope of track at each point
    dlon = np.gradient(lons_track_rev)
    dlat = np.gradient(lats_track_rev)
    tangent_mag = np.hypot(dlon, dlat)
    tangent_u = dlon / tangent_mag
    tangent_v = dlat / tangent_mag
    # Convert latitude to radians
    phi = np.deg2rad(lats_track_rev)
    # Length of one degree of latitude and longitude in meters
    meters_per_degree_lat = 111_320  # Approximate value
    meters_per_degree_lon = 111_320 * np.cos(phi)
    # Handle division by zero at the poles
    meters_per_degree_lon = np.where(meters_per_degree_lon == 0,
                                     np.nan, meters_per_degree_lon)
    # Compute normal vectors
    normal_u = -tangent_v
    normal_v = tangent_u
    theta = np.arctan2(lat_coast-lat_equat, lon_coast-lon_equat)
    print(theta, "--------------------------------------------------")
    # if is_within_region(lon_coast, lat_coast, *REG):
    if True:
        ds_smooth = xr.open_dataset(f)
        sla_smooth = ds_smooth.sla_smooth
        gc = compute_geostrophy_gc(ds_sla, sla_smooth)
        gc = gc.where(gc > -2.5)
        gc = gc.where(gc < 2.5)
        gc_clim = gc.groupby("time.month").mean("time")
        gc_clim_m = gc_clim.sel(month=month_num)
        # Convert gc from m/s to degrees (assuming cross-track direction)
        gc_deg = gc_clim_m.values / meters_per_degree_lat
        # Calculate displacements
        delta_lon = normal_u * gc_deg
        delta_lat = normal_v * gc_deg
        # Apply scaling factor for visualization
        scaling_factor = 1e5  # Adjust as needed
        delta_lon *= scaling_factor
        delta_lat *= scaling_factor
        # Compute endpoints
        lon_start = lons_track_rev
        lat_start = lats_track_rev
        lon_end = lon_start + delta_lon
        lat_end = lat_start + delta_lat
        # Create line segments
        segments = [[[lon_start[i], lat_start[i]], [lon_end[i], lat_end[i]]]
                    for i in range(len(lon_start))]
        # Create a LineCollection
        lc = LineCollection(segments, colors='red', linewidths=1,
                            transform=proj)
        ax1.plot(lons_track_rev, lats_track_rev, 'k-',
                 transform=proj, label='Ground Track')
        ax1.add_collection(lc)

month_num = 10
month_name = months[month_num+1]
info =f"{sat} {months[month_num+1]}"
ax2.set_extent([*WBOB2], crs=proj)
ax2.coastlines()
# ax1.legend()
gl = ax2.gridlines(draw_labels=True, linewidth=0.5, color='gray', alpha=0.7, linestyle='--')
gl.top_labels = gl.right_labels = False  # Hide top and right labels
plt.text(0.3, 0.95, info,
         horizontalalignment='center',
         verticalalignment='center',
         fontsize=15,
         fontweight="bold",
         transform=ax2.transAxes)
for f_sla in sorted(
```

```
    glob.glob(
        f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
    )
):
    basename = os.path.basename(f_sla)
    ds_sla = xr.open_dataset(f_sla, decode_times=False)
    trackn = ds_sla.Pass
    # f = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{trackn}_loess_0.2.nc"
    f = f"../computed/sla_loess_0.05a/track_sla_along_{sat}_{trackn}_loess_0.05.nc"
    lons_track = ds_sla.lon.values
    lats_track = ds_sla.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    lon_equat = lons_track[0]
    lat_equat = lats_track[0]
    # compute tangent vectors or slope of track at each point
    dlon = np.gradient(lons_track_rev)
    dlat = np.gradient(lats_track_rev)
    tangent_mag = np.hypot(dlon, dlat)
    tangent_u = dlon / tangent_mag
    tangent_v = dlat / tangent_mag
    # Convert latitude to radians
    phi = np.deg2rad(lats_track_rev)
    # Length of one degree of latitude and longitude in meters
    meters_per_degree_lat = 111_320  # Approximate value
    meters_per_degree_lon = 111_320 * np.cos(phi)
    # Handle division by zero at the poles
    meters_per_degree_lon = np.where(meters_per_degree_lon == 0,
                                     np.nan, meters_per_degree_lon)
    # Compute normal vectors
    normal_u = -tangent_v
    normal_v = tangent_u
    theta = np.arctan2(lat_coast-lat_equat, lon_coast-lon_equat)
    print(theta, "-----------------------------------------------------")
    # if is_within_region(lon_coast, lat_coast, *REG):
    if True:
        ds_smooth = xr.open_dataset(f)
        sla_smooth = ds_smooth.sla_smooth
        gc = compute_geostrophy_gc(ds_sla, sla_smooth)
        gc = gc.where(gc > -2.5)
        gc = gc.where(gc < 2.5)
        gc_clim = gc.groupby("time.month").mean("time")
        gc_clim_m = gc_clim.sel(month=month_num)
        # Convert gc from m/s to degrees (assuming cross-track direction)
        gc_deg = gc_clim_m.values / meters_per_degree_lat
        # Calculate displacements
        delta_lon = normal_u * gc_deg
        delta_lat = normal_v * gc_deg
        # Apply scaling factor for visualization
        scaling_factor = 1e5  # Adjust as needed
        delta_lon *= scaling_factor
        delta_lat *= scaling_factor
        # Compute endpoints
        lon_start = lons_track_rev
        lat_start = lats_track_rev
        lon_end = lon_start + delta_lon
        lat_end = lat_start + delta_lat
        # Create line segments
        segments = [[[lon_start[i], lat_start[i]], [lon_end[i], lat_end[i]]]
                    for i in range(len(lon_start))]
        # Create a LineCollection
        lc = LineCollection(segments, colors='red', linewidths=1,
                            transform=proj)
        ax2.plot(lons_track_rev, lats_track_rev, 'k-',
                 transform=proj, label='Ground Track')
        ax2.add_collection(lc)
plt.savefig(f"cross_structure_x_arrows_map_{sat}_1by2.png",
            bbox_inches='tight', dpi=300)
ln_sf(f"cross_structure_x_arrows_map_{sat}_1by2.png", "fig07.png")
plt.show()
plt.close("all")

```

```python
import glob
import os

import xarray as xr
from matplotlib.collections import LineCollection
from tools_xtrackm import *

sat = "ERS1+ERS2+ENV+SRL"

REG = (80.0, 92.0, 11.0, 21.0)
WBOB = (78.0, 87.0, 8.0, 21.0)
WBOB2 = (78.0, 97.0, 0.0, 25.0)
proj = ccrs.PlateCarree()

fig, (ax1, ax2) = plt.subplots(
    1, 2, figsize=(14, 10), subplot_kw={"projection": proj}
)
# adjust subplots to have minimum space between them
month_num = 2
month_name = months[month_num + 1]
info = f"a) {sat} {months[month_num + 1]}"
fig.subplots_adjust(wspace=0.1)
ax1.set_extent([*WBOB2], crs=proj)
ax1.coastlines()
gl = ax1.gridlines(
    draw_labels=True, linewidth=0.5, color="gray", alpha=0.7, linestyle="--"
)
gl.top_labels = gl.right_labels = False  # Hide top and right labels
plt.text(
    0.3,
    0.95,
    info,
    horizontalalignment="center",
    verticalalignment="center",
    fontsize=15,
    fontweight="bold",
    transform=ax1.transAxes,
)

for f_sla in sorted(
    glob.glob(
        f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
    )
):
    basename = os.path.basename(f_sla)
    ds_sla = xr.open_dataset(f_sla, decode_times=False)
    trackn = ds_sla.Pass
    f = f"../guv/guv_at_intersections_loess/gc_at_ERS1+ERS2+ENV+SRL_{trackn}_loess_0.05.nc"
    ds_gc = xr.open_dataset(f)
    gc = ds_gc.gc
    lons_track = ds_sla.lon.values
    lats_track = ds_sla.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    lon_equat = lons_track[0]
    lat_equat = lats_track[0]
    # compute tangent vectors or slope of track at each point
    dlon = np.gradient(lons_track_rev)
    dlat = np.gradient(lats_track_rev)
    tangent_mag = np.hypot(dlon, dlat)
    tangent_u = dlon / tangent_mag
    tangent_v = dlat / tangent_mag
    # Convert latitude to radians
    phi = np.deg2rad(lats_track_rev)
    # Length of one degree of latitude and longitude in meters
    meters_per_degree_lat = 111_320  # Approximate value
    meters_per_degree_lon = 111_320 * np.cos(phi)
    # Handle division by zero at the poles
    meters_per_degree_lon = np.where(
        meters_per_degree_lon == 0, np.nan, meters_per_degree_lon
    )
    # Compute normal vectors
    normal_u = -tangent_v
    normal_v = tangent_u
    theta = np.arctan2(lat_coast - lat_equat, lon_coast - lon_equat)
    print(theta, "-----------------------------------------------------")
    # if is_within_region(lon_coast, lat_coast, *REG):
    if True:
        gc_clim = gc.groupby("time.month").mean("time")
        gc_clim_m = gc_clim.sel(month=month_num)
        # Convert gc from m/s to degrees (assuming cross-track direction)
        gc_deg = gc_clim_m.values / meters_per_degree_lat
        # Calculate displacements
        delta_lon = normal_u * gc_deg
        delta_lat = normal_v * gc_deg
        # Apply scaling factor for visualization
        scaling_factor = 1e5  # Adjust as needed
        delta_lon *= scaling_factor
        delta_lat *= scaling_factor
        # Compute endpoints
        lon_start = lons_track_rev
        lat_start = lats_track_rev
        lon_end = lon_start + delta_lon
        lat_end = lat_start + delta_lat
        # Create line segments
        segments = [
            [[lon_start[i], lat_start[i]], [lon_end[i], lat_end[i]]]
            for i in range(len(lon_start))
        ]
        # Create a LineCollection
        lc = LineCollection(
            segments, colors="red", linewidths=1, transform=proj
        )
        ax1.plot(
            lons_track_rev,
            lats_track_rev,
            "k-",
            transform=proj,
            label="Ground Track",
        )
        ax1.add_collection(lc)

month_num = 10
month_name = months[month_num + 1]
info = f"{sat} {months[month_num + 1]}"
```

```
ax2.set_extent([*WBOB2], crs=proj)
ax2.coastlines()
# ax1.legend()
gl = ax2.gridlines(
    draw_labels=True, linewidth=0.5, color="gray", alpha=0.7, linestyle="--"
)
gl.top_labels = gl.right_labels = False  # Hide top and right labels
plt.text(
    0.3,
    0.95,
    info,
    horizontalalignment="center",
    verticalalignment="center",
    fontsize=15,
    fontweight="bold",
    transform=ax2.transAxes,
)

for f_sla in sorted(
    glob.glob(
        f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
    )
):
    basename = os.path.basename(f_sla)
    ds_sla = xr.open_dataset(f_sla, decode_times=False)
    trackn = ds_sla.Pass
    f = f"../guv/guv_at_intersections_loess/gc_at_ERS1+ERS2+ENV+SRL_{trackn}_loess_0.05.nc"
    ds_gc = xr.open_dataset(f)
    gc = ds_gc.gc
    lons_track = ds_sla.lon.values
    lats_track = ds_sla.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    lon_equat = lons_track[0]
    lat_equat = lats_track[0]
    # compute tangent vectors or slope of track at each point
    dlon = np.gradient(lons_track_rev)
    dlat = np.gradient(lats_track_rev)
    tangent_mag = np.hypot(dlon, dlat)
    tangent_u = dlon / tangent_mag
    tangent_v = dlat / tangent_mag
    # Convert latitude to radians
    phi = np.deg2rad(lats_track_rev)
    # Length of one degree of latitude and longitude in meters
    meters_per_degree_lat = 111_320  # Approximate value
    meters_per_degree_lon = 111_320 * np.cos(phi)
    # Handle division by zero at the poles
    meters_per_degree_lon = np.where(
        meters_per_degree_lon == 0, np.nan, meters_per_degree_lon
    )
    # Compute normal vectors
    normal_u = -tangent_v
    normal_v = tangent_u
    theta = np.arctan2(lat_coast - lat_equat, lon_coast - lon_equat)
    print(theta, "-------------------------------------------------")
    # if is_within_region(lon_coast, lat_coast, *REG):
    if True:
        gc_clim = gc.groupby("time.month").mean("time")
        gc_clim_m = gc_clim.sel(month=month_num)
        # Convert gc from m/s to degrees (assuming cross-track direction)
        gc_deg = gc_clim_m.values / meters_per_degree_lat
        # Calculate displacements
        delta_lon = normal_u * gc_deg
        delta_lat = normal_v * gc_deg
        # Apply scaling factor for visualization
        scaling_factor = 1e5  # Adjust as needed
        delta_lon *= scaling_factor
        delta_lat *= scaling_factor
        # Compute endpoints
        lon_start = lons_track_rev
        lat_start = lats_track_rev
        lon_end = lon_start + delta_lon
        lat_end = lat_start + delta_lat
        # Create line segments
        segments = [
            [[lon_start[i], lat_start[i]], [lon_end[i], lat_end[i]]]
            for i in range(len(lon_start))
        ]
        # Create a LineCollection
        lc = LineCollection(
            segments, colors="red", linewidths=1, transform=proj
        )
        ax2.plot(
            lons_track_rev,
            lats_track_rev,
            "k-",
            transform=proj,
            label="Ground Track",
        )
        ax2.add_collection(lc)
plt.savefig(
    f"cross_structure_x_arrows_map_{sat}_1by2.png",
    bbox_inches="tight",
    dpi=300,
)
ln_sf(f"cross_structure_x_arrows_map_{sat}_1by2.png", "fig06.png")
plt.show()
plt.close("all")

```

```python
import xarray as xr
from tools_xtrackm import *
from namesm import sats_new
import glob
import os

import matplotlib.cm as cm

# sats = ["S3A"]
REG = (80.0, 89.0, 11.0, 21.0)
REG = (75.0, 92.0, 10.0, 23.0)
sats = ["S3A"]
sats = ["ERS1+ERS2+ENV+SRL"]
xwidth = 290e3
# Load the YlOrRd colormap
ncolors = 21
cmap1 = cm.get_cmap('YlOrRd', ncolors)  # Split into 10 colors
colors1 = cmap1(np.linspace(0, 1, ncolors))
cmap2 = cm.get_cmap('BuGn', ncolors)  # Split into 10 colors
colors2 = cmap2(np.linspace(0, 1, ncolors))

norm2 = plt.Normalize(10.18, 22.00)
sm2 = plt.cm.ScalarMappable(cmap=cmap2, norm=norm2)
sm2.set_array([])

norm1 = plt.Normalize(10.18, 22.00)
sm1 = plt.cm.ScalarMappable(cmap=cmap1, norm=norm1)
sm1.set_array([])

# fig, ax = plt.subplots(1, 1, figsize=(9, 4))
frac1 = "0.2"
frac1 = "0.05"
for sat in sats[:]:
    fig, ax = plt.subplots(1, 1, figsize=(12, 6))
    # adjust subplots increase x spacing
    plt.subplots_adjust(left=0.1, bottom=0.1, right=0.8, top=0.8, wspace=0.3, hspace=0.3)
    i = 0
    lat_values = []
    for f_sla in sorted(
        glob.glob(
            f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.descending.nc"
        )
    ):
        basename = os.path.basename(f_sla)
        ds_sla = xr.open_dataset(f_sla, decode_times=False)
        trackn = ds_sla.Pass
        f = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{trackn}_loess_{frac1}.nc"
        lons_track = ds_sla.lon.values
        lats_track = ds_sla.lat.values
        lons_track_rev = lons_track[::-1]
        lats_track_rev = lats_track[::-1]
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        print(f, lon_coast, lat_coast)
        if is_within_region(lon_coast, lat_coast, *REG):
            lat_values.append(lat_coast)
            ds_smooth = xr.open_dataset(f)
            # print(ds_smooth)
            sla_smooth = ds_smooth.sla_smooth
            print(f"stats of sla_smooth {sat} {trackn} --------------------")
            print(f"stats of gu {sat} {trackn} --------------------")
            gc = compute_geostrophy_gc(ds_sla, sla_smooth)
            gc_clim = gc.groupby("time.month").mean("time")
            gc_clim_xwidth = gc_clim.sel(x=slice(0, xwidth))
            gc_clim_xwidth_m = gc_clim_xwidth.isel(month=2, drop=True) # march
            gc_clim_xwidth_n = gc_clim_xwidth.isel(month=10, drop=True) # november
            gc_clim_xwidth_m.plot(ax=ax, label=f"{lat_coast}",
                                  color=colors1[i])
            x0, y0 = -15, gc_clim_xwidth_m.sel(x=0).values
            plt.text(x0, y0, f"{lat_coast:.2f}", fontsize=10, color=colors1[i])
            gc_clim_xwidth_n.plot(ax=ax, label=f"{lat_coast}",
                                  color=colors2[i])
            x0, y0 = -15, gc_clim_xwidth_n.sel(x=0).values
            plt.text(x0, y0, f"{lat_coast:.2f}", fontsize=10, color=colors2[i])
            # set xticks between 0 to xwidth
            ax.set_xlim(-15, xwidth)
            ax.set_ylim(-0.7, 0.7)
            xticks1 = ax.get_xticks()
            xticks_km = np.round((xticks1 / 1000), 0)
            ax.set_xticks(xticks1, xticks_km)
            ax.set_xlabel("distance along track (km)")
            ax.set_ylabel("geostrophic velocity (m/s)")
            # ax.set_ylim(-0.3, 0.3)
            ax.axhline(0, color="k", linestyle="--")
            i += 1
            # print_stats(gc)
    # plt.legend()
    print(f"number of tracks {i}")
    minlat = min(lat_values)
    maxlat = max(lat_values)
    print(f"min lat {min(lat_values)} max lat {max(lat_values)}")
    pos = ax.get_position()
    # Adjust and add the first colorbar closer to the plot
    cbar_ax1 = fig.add_axes([pos.x1 + 0.01, pos.y0, 0.02, pos.height])  # Adjust 0.01 to control horizontal gap
    cbar_ax1.text(0.5, 0.5, "MARCH", fontsize=10, horizontalalignment="center", verticalalignment="center", transform=cbar_ax1.transAxes, rotation = "vertical")
    cbar = plt.colorbar(sm1, cax=cbar_ax1)
    # cbar.set_label('latitude')
    cbar.set_ticks(list(range(int(minlat), int(maxlat)+1, 2)))
    # Adjust and add the second colorbar even closer
    cbar_ax2 = fig.add_axes([pos.x1 + 0.06, pos.y0, 0.02, pos.height])  # Adjust 0.03 as needed for spacing
    cbar_ax2.text(0.5, 0.5, "NOVEMBER", fontsize=10, horizontalalignment="center", verticalalignment="center", transform=cbar_ax2.transAxes, rotation = "vertical")
    cbar = plt.colorbar(sm2, cax=cbar_ax2)
    cbar.set_label('latitude')
    cbar.set_ticks(list(range(int(minlat), int(maxlat)+1, 2)))

    plt.savefig(
        f"p.gc.clim.lat_wise.along_xwidth_{sat}a_fig09.png",
        bbox_inches="tight",
        dpi=300,
    )
    plt.show()
    plt.close("all")

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import pandas as pd
import xarray as xr
from tools_xtrackm import *


def decode_id(raw_id):
    if isinstance(raw_id, (bytes, np.bytes_)):
        return raw_id.decode("utf-8").strip()
    return str(raw_id).strip()


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)


def build_unique_str(sat, row):
    track_self = track_to_str(row["track_self"])
    track_other = track_to_str(row["track_other"])
    return f"{sat}_{track_self}_{track_other}"


cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

sat_val = "TP+J1+J2+J3+S6A"
SAT = "TP+J1+J2+J3+S6A"
track_self_val = 155
track_other_val = 116
EKMAN_DIR = "/home/srinivasu/xtrackm/ekman/ekman_at_intersections"

df = pd.read_csv(f"tracks_intersections_{sat_val}_1.csv")
df_out = df.copy()

rows = df_out.loc[
    (df_out["sat"] == sat_val)
    & (df_out["track_self"] == track_self_val)
    & (df_out["track_other"] == track_other_val)
]

frac1 = "0.05"
i = 0
row = rows.iloc[i]
sat1 = row["sat"]
# track_number_self = rows["track_self"].apply(lambda x: str(x)).values[i]
# track_number_other = rows["track_other"].apply(lambda x: str(x)).values[i]
track_number_self = str(row["track_self"])
track_number_other = str(row["track_other"])
unique_str = build_unique_str(SAT, row)
lons_inter1 = row["lons_inter"]
lats_inter1 = row["lats_inter"]
lon1 = lons_inter1
lat1 = lats_inter1
x_from_coast_self1 = row["x_from_coast_self"]
x_from_coast_other1 = row["x_from_coast_other"]
angle_acute = row["angle_acute"]
angle_obtuse = row["angle_obtuse"]
print(
    sat1,
    track_number_self,
    track_number_other,
    lons_inter1,
    lats_inter1,
    x_from_coast_self1,
    x_from_coast_other1,
    angle_acute,
    angle_obtuse,
)
f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat1}_{track_number_self}_loess_{frac1}.nc"
f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat1}_{track_number_other}_loess_{frac1}.nc"
ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
ds_self_smooth = xr.open_dataset(f_self_smooth)
sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
sla_self_smooth = ds_self_smooth.sla_smooth
ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
ds_other_smooth = xr.open_dataset(f_other_smooth)
sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
sla_other_smooth = ds_other_smooth.sla_smooth
lons_track_self = ds_self.lon.values
lats_track_self = ds_self.lat.values
lon_coast_self = lons_track_self[-1]  # this on coast
lat_coast_self = lats_track_self[-1]  # this on coast
lon_equat_self = lons_track_self[0]  # this on equator
lat_equat_self = lats_track_self[0]  # this on equator
iself = index_at_lat(ds_self, lat1)
a1 = local_angle(
    lons_track_self, lats_track_self, iself, ioff=3, degrees=False
)
a1_d = math.degrees(a1)
lons_track_other = ds_other.lon.values
lats_track_other = ds_other.lat.values
iothr = index_at_lat(ds_other, lat1)
a2 = local_angle(
    lons_track_other, lats_track_other, iothr, ioff=3, degrees=False
)
a2_d = math.degrees(a2)
lon_equat_other = lons_track_other[0]
lat_equat_other = lats_track_other[0]
lon_coast_other = lons_track_other[-1]
lat_coast_other = lats_track_other[-1]
gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
gc_other_at = gc_other.sel(
    x=x_from_coast_other1, method="nearest", drop=True
)
gc_other_at = gc_other_at.interp(time=gc_self_at.time)
print(f"heading angle {a1_d:.2f} {a2_d:.2f}")
```

```
gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
fig = plt.figure(figsize=(12, 10))
ax1 = fig.add_subplot(3, 1, 3)
ax0 = fig.add_subplot(3, 1, 2)
gu_cut = gu.resample(time="1ME").mean()
gv_cut = gv.resample(time="1ME").mean()
f_ek = f"{EKMAN_DIR}/{unique_str}_ekman_at_intersections.nc"
ds_ek = xr.open_dataset(f_ek)
u_ek = ds_ek['u_at']
v_ek = ds_ek['v_at']
gu_ek = gu_cut + u_ek.interp(time=gu_cut.time)
gc_self_at_cut = gc_self_at.resample(time="1ME").mean()
gc_other_at_cut = gc_other_at.resample(time="1ME").mean()
gu_cut.plot(ax=ax0, label="track zonal", color="r", linewidth=2)
gu_ek.plot(ax=ax0, label="track+ek zonal", color="b", linewidth=2)
# gc_self_at_cut.plot(ax=ax0, label=f"gc at {track_number_self}", color="r", linewidth=2)
ax0.legend()
ax0.set_ylabel("zonal current u m/s")
gv_cut.plot(ax=ax1, label="track meridional", color="r", linewidth=2)
# gc_other_at_cut.plot(ax=ax1, label=f"gc at {track_number_other}", color="r", linewidth=2)
info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
plt.text(
    0.15,
    0.90,
    info,
    transform=ax0.transAxes,
    fontsize=14,
    fontweight="bold",
)
ax1.legend()
ax1.set_ylabel("meridional current v m/s")
ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=5)
ax2.grid()
ax2.plot(
    lon1,
    lat1,
    c="r",
    marker="o",
    markersize=6,
    markerfacecolor="red",
    markeredgecolor="black",
    markeredgewidth=2,
)
ax2.scatter(
    lons_track_self,
    lats_track_self,
    marker=".",
    color="c",
    s=4,
)
ax2.scatter(
    lons_track_other,
    lats_track_other,
    marker=".",
    color="c",
    s=4,
)
lonm, latm = get_point_at_distance(
    lon_equat_self, lat_equat_self, lon_coast_self, lat_coast_self, d
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=a1_d,
        color="k",
    )
lonm, latm = get_point_at_distance(
    lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=a2_d,
        color="k",
    )
plt.savefig(
    f"guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
)
plt.show()
plt.close("all")

```

```python
import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.gridspec as gridspec
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from matplotlib.patches import Patch
from tools_shapely import *
from tools_xtrackm import *

# copilot --resume=d419a1e5-878f-477e-813b-3b06f3d9a50b

method = "loess"
frac1 = "0.05"

sat_val = "TP+J1+J2+J3+S6A"
sat_val = "S3A"
sat_val = "ERS1+ERS2+ENV+SRL"

df = pd.read_csv(f"intersections_near_east_coast_{sat_val}.csv")

# Skip the first point (lowest latitude)
df_out = df.sort_values("lats_inter").iloc[1:].reset_index(drop=True)

N_PTS = 7
assert len(df_out) == N_PTS, (
    f"Expected {N_PTS} points after skipping first, got {len(df_out)}"
)

POINT_COLORS = [plt.cm.tab10(i) for i in range(N_PTS)]
POINT_LABELS = list("ABCDEFG")

# Map extent: 10N–22N, 78E–88E
MAP_EXTENT = (78.0, 88.0, 10.0, 22.0)

# --- Pre-collect all timeseries data ---
point_data = []
for i, row in df_out.iterrows():
    sat1 = row["sat"]
    tno_s = str(row["track_self"])
    tno_o = str(row["track_other"])
    lon1 = float(row["lons_inter"])
    lat1 = float(row["lats_inter"])

    f_guv = (
        f"../guv/guv_at_intersections_{method}/"
        f"guv_at_intersection_{sat1}_{tno_s}_{tno_o}_{method}_{frac1}.nc"
    )
    ds_guv = xr.open_dataset(f_guv)
    gu = ds_guv.gu
    gv = ds_guv.gv
    gs = np.sqrt(gu**2 + gv**2)
    times = pd.to_datetime(gs.time.values)
    # Monthly climatology (mean of each calendar month across all years)
    gs_clim = gs.groupby("time.month").mean("time")
    gu_clim = gu.groupby("time.month").mean("time")

    point_data.append(
        dict(
            sat1=sat1,
            tno_s=tno_s,
            tno_o=tno_o,
            lon1=lon1,
            lat1=lat1,
            gs=gs,
            gu_clim=gu_clim,
        )
    )

# --- Build figure: left = tall map, right = 7 timeseries rows ---
fig = plt.figure(figsize=(18, 14))
gs_layout = gridspec.GridSpec(
    N_PTS,
    2,
    figure=fig,
    width_ratios=[1, 2.5],
    hspace=0.0,
    wspace=0.3,
)

# --- Left: vertically elongated cartopy map spanning all rows ---
ax_map = fig.add_subplot(gs_layout[:, 0], projection=ccrs.PlateCarree())
ax_map.set_extent(MAP_EXTENT, crs=ccrs.PlateCarree())
ax_map.add_feature(cfeature.LAND, facecolor="0.93", zorder=0)
ax_map.add_feature(cfeature.OCEAN, facecolor="aliceblue", zorder=0)
ax_map.coastlines(resolution="50m", linewidth=1.0, zorder=2)
gl = ax_map.gridlines(
    draw_labels=True,
    linewidth=0.5,
    color="gray",
    alpha=0.6,
    linestyle="--",
)
gl.top_labels = False
gl.right_labels = False
# ax_map.set_title(f"{sat_val}\nIntersection points", fontsize=9)

# Connect all 7 points with a dashed gray line
lons_pts = [e["lon1"] for e in point_data]
lats_pts = [e["lat1"] for e in point_data]
ax_map.plot(
    lons_pts,
    lats_pts,
    color="gray",
    linewidth=0.8,
    linestyle="--",
    transform=ccrs.PlateCarree(),
    zorder=3,
)

# Plot each point with its assigned color and letter label
for idx, entry in enumerate(point_data):
    color = POINT_COLORS[idx]
    label = POINT_LABELS[idx]
    ax_map.plot(
        entry["lon1"],
```

```
        entry["lat1"],
        marker="o",
        markersize=9,
        color=color,
        markeredgecolor="black",
        markeredgewidth=0.8,
        transform=ccrs.PlateCarree(),
        zorder=5,
    )
    ax_map.text(
        entry["lon1"] + 0.18,
        entry["lat1"] + 0.1,
        label,
        fontsize=10,
        fontweight="bold",
        color=color,
        transform=ccrs.PlateCarree(),
        zorder=6,
    )

# --- Right: 7 monthly climatology subplots, A at bottom (row 6) → G at top (row 0) ---
MONTHS = np.arange(1, 13)
MONTH_NAMES = ["J", "F", "M", "A", "M", "J", "J", "A", "S", "O", "N", "D"]

for idx, entry in enumerate(point_data):
    row = N_PTS - 1 - idx  # A → row 6, G → row 0
    ax_ts = fig.add_subplot(gs_layout[row, 1])
    color = POINT_COLORS[idx]
    label = POINT_LABELS[idx]
    gu_clim = entry["gu_clim"]

    # Seasonal background shading (month-based x-axis)
    ax_ts.axvspan(2 - 0.5, 5 + 0.5, color="yellow", alpha=0.15, zorder=0)   # Feb–May
    ax_ts.axvspan(6 - 0.5, 9 + 0.5, color="cyan",   alpha=0.15, zorder=0)   # Jun–Sep
    ax_ts.axvspan(10 - 0.5, 12 + 0.5, color="gray", alpha=0.15, zorder=0)   # Oct–Dec

    # Monthly climatology bar plot
    # ax_ts.bar(
    #     MONTHS,
    #     gs_clim.values,
    #     color=color,
    #     alpha=0.8,
    #     edgecolor="k",
    #     linewidth=0.5,
    #     zorder=2,
    # )

    ax_ts.plot(
        MONTHS,
        gu_clim.values,
        color=color,
        linewidth=3,
        zorder=2,
    )

    # Point label inside left edge of plot
    ax_ts.text(
        0.005, 0.5, label,
        transform=ax_ts.transAxes, fontsize=12,
        fontweight="bold", color=color,
        va="center", ha="left",
    )

    ax_ts.set_xlim(0.5, 12.5)
    all_vals = gs_clim.values[np.isfinite(gs_clim.values)]
    ypad = (all_vals.max() - all_vals.min()) * 0.08
    # ax_ts.set_ylim(-all_vals.max() - ypad, all_vals.max() + ypad)
    # ax_ts.set_ylim(-all_vals.max() - ypad, all_vals.max() + ypad)
    ax_ts.set_ylabel("m/s", fontsize=14)
    ax_ts.set_xticks(MONTHS)
    ax_ts.axhline(0, color="gray", linewidth=0.8, linestyle="--", zorder=1)
    ax_ts.tick_params(labelsize=12)
    # Only bottom subplot (A, idx=0) shows month labels
    if idx == 0:
        ax_ts.set_xticklabels(MONTH_NAMES)
    else:
        ax_ts.set_xticklabels([])

    # Legend only on the top panel (G, idx=N_PTS-1)
    if idx == N_PTS - 1:
        handles = [
            Patch(facecolor="yellow", alpha=0.4, edgecolor="none", label="Feb–May"),
            Patch(facecolor="cyan",   alpha=0.4, edgecolor="none", label="Jun–Sep"),
            Patch(facecolor="gray",   alpha=0.4, edgecolor="none", label="Oct–Dec"),
        ]
        ax_ts.legend(handles=handles, loc="upper right", fontsize=12, ncol=3)
        ax_ts.set_title("Monthly climatology of |u| (gs)", fontsize=11)

# fig.suptitle(
#     f"Current speed at EICC coastal intersection points — {sat_val}",
#     fontsize=13,
#     fontweight="bold",
#     y=1.002,
# )
out_png = f"speed_timeseries_allinone_{sat_val}.png"
plt.savefig(out_png, dpi=150, bbox_inches="tight")
plt.show()
plt.close("all")
print(f"Saved: {out_png}")

```

```python
#!/usr/bin/env python3
import argparse
from pathlib import Path

import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr

# copilot --resume=c0c59aeb-32fd-409b-a69f-a03a902a3b60

DEFAULT_SAT = "ERS1+ERS2+ENV+SRL"
DEFAULT_METHOD = "loess"
DEFAULT_FRAC = "0.05"
DEFAULT_VAR = "gu"  # requested "u" maps to gu for this dataset family
DEFAULT_INPUT_CSV = "intersections_near_coast_BOB_ERS1+ERS2+ENV+SRL.csv"
DEFAULT_GUV_DIR = "../guv/guv_at_intersections_loess"
DEFAULT_OUT_NC = "hovmoller_u_near_coast_BOB_ERS1+ERS2+ENV+SRL.nc"
DEFAULT_OUT_PNG = "hovmoller_u_near_coast_BOB_ERS1+ERS2+ENV+SRL.png"
BOB_EXTENT = (78.0, 100.0, 4.0, 24.0)  # lon_min, lon_max, lat_min, lat_max


def resolve_guv_file(guv_dir: Path, sat: str, track_self: int, track_other: int, method: str, frac: str) -> Path:
    t1 = str(int(track_self))
    t2 = str(int(track_other))
    candidates = [
        guv_dir / f"guv_at_intersection_{sat}_{t1}_{t2}_{method}_{frac}.nc",
        guv_dir / f"guv_at_intersection_{sat}_{t2}_{t1}_{method}_{frac}.nc",
    ]
    for p in candidates:
        if p.exists():
            return p
    raise FileNotFoundError(
        "No guv file found for tracks "
        f"({t1}, {t2}). Tried: " + ", ".join(str(x) for x in candidates)
    )

# create pandas date range between 1992-11-11,2016-03-21 with timeperiod 35 days
date_range = pd.date_range(start="1992-11-11", end="2016-03-21", freq="35D")
def build_transect_hovmoller(input_csv: Path, guv_dir: Path, sat: str, method: str, frac: str, var: str):
    df = pd.read_csv(input_csv).copy()
    if "lats_inter" not in df or "lons_inter" not in df:
        raise ValueError("Input CSV must contain lats_inter and lons_inter columns.")
    # Step 4: selected points sorted by latitude -> transect order
    df = df.sort_values("lats_inter").reset_index(drop=True)
    series = []
    kept_rows = []
    for i, row in df.iterrows():
        track_self = int(row["track_self"])
        track_other = int(row["track_other"])
        p = resolve_guv_file(guv_dir, sat, track_self, track_other, method, frac)
        with xr.open_dataset(p) as ds:
            if var not in ds.data_vars:
                raise KeyError(f"Variable '{var}' not found in {p}. Available: {list(ds.data_vars)}")
            # Step 3: append requested u-like variable from each guv file
            da = ds[var].load().rename("u")
        da = da.interp(time=date_range, method="nearest", kwargs={"fill_value": "extrapolate"})
        da = da.expand_dims(transect=[i])
        da = da.assign_coords(
            latitude=("transect", [float(row["lats_inter"])]),
            longitude=("transect", [float(row["lons_inter"])]),
            track_self=("transect", [track_self]),
            track_other=("transect", [track_other]),
        )
        series.append(da)
        kept_rows.append(row)
    if not series:
        raise ValueError("No intersection points were loaded.")
    # Step 5: concat array of u dataarrays with transect dimension
    u_transect = xr.concat(series, dim="transect", join="outer").sortby("latitude")
    return u_transect, pd.DataFrame(kept_rows)


def plot_bob_transect_map(ax, df_used: pd.DataFrame):
    proj = ccrs.PlateCarree()
    df_plot = df_used.sort_values("lats_inter").reset_index(drop=True)
    ax.set_extent(BOB_EXTENT, crs=proj)
    ax.add_feature(cfeature.LAND, facecolor="0.92", zorder=0)
    ax.add_feature(cfeature.OCEAN, facecolor="white", zorder=0)
    ax.coastlines(resolution="50m", linewidth=1.0, zorder=2)
    gl = ax.gridlines(draw_labels=True, linewidth=0.5, color="gray", alpha=0.6, linestyle="--")
    gl.top_labels = False
    gl.right_labels = False
    ax.plot(
        df_plot["lons_inter"],
        df_plot["lats_inter"],
        color="tab:blue",
        linewidth=1.5,
        marker="o",
        markersize=4,
        transform=proj,
        zorder=3,
    )
    for i, row in df_plot.iterrows():
        lon = float(row["lons_inter"])
        lat = float(row["lats_inter"])
        dx = 0.18 if i % 2 == 0 else -0.95
        dy = 0.18 if i % 3 == 0 else -0.18
        ax.text(
            lon + dx,
            lat + dy,
            f"({lon:.2f}, {lat:.2f})",
            fontsize=8,
            transform=proj,
            zorder=4,
            bbox=dict(facecolor="white", alpha=0.75, edgecolor="none", pad=1.0),
        )
    ax.set_title("BOB transect intersections")


def plot_hovmoller(u_transect: xr.DataArray, df_used: pd.DataFrame, out_png: Path, title: str):
    hov_plot = u_transect.copy()
    vmax = float(np.nanpercentile(np.abs(hov_plot.values), 98))
    if not np.isfinite(vmax) or vmax == 0:
        vmax = 1.8
```

```
    fig = plt.figure(figsize=(16, 7), constrained_layout=True)
    gs = fig.add_gridspec(1, 2, width_ratios=[1.05, 2.4], wspace=0.05)
    ax_map = fig.add_subplot(gs[0, 0], projection=ccrs.PlateCarree())
    ax_hov = fig.add_subplot(gs[0, 1])
    plot_bob_transect_map(ax_map, df_used)
    vmax = 1.2
    hov_plot.plot.contourf(
        ax=ax_hov,
        x="time",
        y="latitude",
        cmap="RdBu_r",
        vmin=-vmax,
        vmax=vmax,
        cbar_kwargs={"label": "u (m/s)"},
    )
    # add some contour lines for better visibility
    hov_plot.plot.contour(
        ax=ax_hov,
        x="time",
        y="latitude",
        colors="k",
        linewidths=0.5,
        levels=10,
        alpha=0.7,
        )
    ax_hov.set_xlabel("Time")
    ax_hov.set_ylabel("Latitude")
    ax_hov.set_title(title + " (monthly interpolated)")
    fig.savefig(out_png, dpi=180)
    plt.show()
    plt.close(fig)


parser = argparse.ArgumentParser(
    description=(
        "Build transect-time array from selected intersections and plot Hovmoller diagram "
        "(latitude vs time)."
    )
)
parser.add_argument("--input-csv", default=DEFAULT_INPUT_CSV, help="Selected intersections CSV")
parser.add_argument("--sat", default=DEFAULT_SAT, help="Satellite family string in guv filenames")
parser.add_argument("--method", default=DEFAULT_METHOD, help="Method token in guv filenames (e.g., loess)")
parser.add_argument("--frac", default=DEFAULT_FRAC, help="Frac token in guv filenames (e.g., 0.05)")
parser.add_argument("--var", default=DEFAULT_VAR, help="Variable name inside guv files (default: gu)")
parser.add_argument("--guv-dir", default=DEFAULT_GUV_DIR, help="Directory containing guv intersection netCDF files")
parser.add_argument("--out-nc", default=DEFAULT_OUT_NC, help="Output netCDF for concatenated transect array")
parser.add_argument("--out-png", default=DEFAULT_OUT_PNG, help="Output PNG for Hovmoller diagram")
args = parser.parse_args()

input_csv = Path(args.input_csv)
guv_dir = Path(args.guv_dir)
out_nc = Path(args.out_nc)
out_png = Path(args.out_png)

u_transect, df_used = build_transect_hovmoller(
    input_csv=input_csv,
    guv_dir=guv_dir,
    sat=args.sat,
    method=args.method,
    frac=args.frac,
    var=args.var,
)

ds_out = u_transect.to_dataset(name="u")
ds_out.to_netcdf(out_nc)

title = f"Hovmoller: {args.var}->u at intersections ({args.sat})"
plot_hovmoller(u_transect, df_used, out_png, title)

print(f"Loaded points: {len(df_used)}")
print(f"Transect dimension: {u_transect.sizes.get('transect', 0)}")
print(f"Time dimension: {u_transect.sizes.get('time', 0)}")
print(f"Saved concatenated array: {out_nc}")
print(f"Saved Hovmoller diagram: {out_png}")

```

eicc/p.intersection_points.S6A.py

```python
import pandas as pd
import xarray as xr
import cartopy.crs as ccrs
from tools_xtrackm import *
import matplotlib.pyplot as plt

cmap1 = "twilight_shifted"

d = 1.5
height, width = 9, 12

sat = "TP+J1+J2+J3+S6A"
df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")

dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
rose = dse.ROSE
rose_cut = rose.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:]))

fig2, ax2 = plt.subplots(
    1,
    1,
    subplot_kw={"projection": ccrs.PlateCarree()},
    figsize=(width, height),
    layout="constrained",
)

rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=5)
ax2.set_extent([*TRACKS_REG], crs=ccrs.PlateCarree())
ax2.scatter(
    df["lons_inter"],
    df["lats_inter"],
    color="w",
    label=f"Satellite {sat}",
    s=18,
    marker="o",
    edgecolors='black',
    transform=ccrs.PlateCarree(),
)
for i, row in df.iterrows():
    # plot self and other tracks as text
    tself = row["track_self"]
    tothr = row["track_other"]
    txt = f"({tself}, {tothr})"
    ax2.text(
        row["lons_inter"],
        row["lats_inter"],
        txt,
        color="k",
        transform=ccrs.PlateCarree(),
        fontsize=8,
        rotation=-45,
    )

# edgecolors='black',
ax2.legend(loc="upper right")
# plt.title("Intersection Points of Tracks in North Indian Ocean")
plt.show()
dse.close()

```

```python
import geopandas as gpd
import cartopy.io.shapereader as shpreader
import matplotlib.pyplot as plt
import numpy as np
from shapely.geometry import LineString, MultiLineString, box
from scipy.ndimage import gaussian_filter1d

lon_min, lon_max = 70, 95
lat_min, lat_max = 5, 30
region_bbox = box(lon_min, lat_min, lon_max, lat_max)

# Load Natural Earth 110m coastline

shpfilename = shpreader.natural_earth(
    resolution='110m',
    category='physical',
    name='coastline'
)
coast = gpd.read_file(shpfilename)

# Clip to Indian Ocean region

coast_clip = gpd.clip(coast, region_bbox)

# Reproject to metric CRS for smoothing

coast_proj = coast_clip.to_crs("EPSG:3857")

# Gaussian smoothing

def smooth_linestring(line, sigma=20):
    coords = np.array(line.coords)
    if len(coords) < 5:
        return line
    x = coords[:, 0]
    y = coords[:, 1]
    x_s = gaussian_filter1d(x, sigma=sigma, mode='nearest')
    y_s = gaussian_filter1d(y, sigma=sigma, mode='nearest')
    return LineString(np.column_stack([x_s, y_s]))


def smooth_geometry(geom, sigma=20):
    if isinstance(geom, LineString):
        return smooth_linestring(geom, sigma)
    elif isinstance(geom, MultiLineString):
        return MultiLineString([smooth_linestring(g, sigma) for g in geom.geoms])
    else:
        return geom


sigma_value = 1   # Increase for stronger smoothing

coast_smooth = coast_proj.copy()
coast_smooth["geometry"] = coast_proj.geometry.apply(
    lambda g: smooth_geometry(g, sigma=sigma_value)
)

# Reproject back to lat/lon

coast_orig = coast_clip
coast_smooth = coast_smooth.to_crs("EPSG:4326")

fig, ax = plt.subplots(figsize=(10, 8))

coast_orig.plot(ax=ax, linewidth=0.8, color="black", label="Original 110m")
coast_smooth.plot(ax=ax, linewidth=2.0, color="red", label="Smoothed")

ax.set_xlim(lon_min, lon_max)
ax.set_ylim(lat_min, lat_max)

ax.set_title("Indian Ocean Coastline\nOriginal vs Gaussian Smoothed")
ax.set_xlabel("Longitude")
ax.set_ylabel("Latitude")
ax.legend()

plt.tight_layout()
plt.show()

```

```python
import cartopy.io.shapereader as shpreader
import geopandas as gpd
import matplotlib.pyplot as plt
import numpy as np
from shapely.geometry import LineString, MultiLineString, Point, box
from tools_shapely import *

lon_min, lon_max = 70, 95
lat_min, lat_max = 5, 30
region_bbox = box(lon_min, lat_min, lon_max, lat_max)

# Load Natural Earth 110m coastline
shpfilename = shpreader.natural_earth(
    resolution="50m", category="physical", name="coastline"
)

coast = gpd.read_file(shpfilename)
coast_clip = gpd.clip(coast, region_bbox)

# Reproject to metric CRS
coast_proj = coast_clip.to_crs("EPSG:3857")


def tangent_slope_on_line_near_point(line, point_xy, neighborhood=2):
    """
    Compute local tangent slope dy/dx on a LineString near a query point.
    Parameters
    ----------
    line : shapely.geometry.LineString
        Smoothed coastline line.
    point_xy : tuple(float, float)
        Query point (x, y) in the same CRS as `line`.
    neighborhood : int
        Number of points on each side used for a local finite-difference tangent.
    Returns
    -------
    dict with keys:
      slope : float (np.inf for vertical tangent)
      tangent_vector : tuple(float, float)
      nearest_point : tuple(float, float)
      distance : float
    """
    coords = np.asarray(line.coords, dtype=float)
    if coords.shape[0] < 2:
        raise ValueError("LineString must have at least 2 coordinates.")
    px, py = point_xy
    p = np.array([px, py], dtype=float)
    best = {
        "dist2": np.inf,
        "seg_idx": 0,
        "proj": coords[0].copy(),
        "t": 0.0,
    }
    for i in range(len(coords) - 1):
        a = coords[i]
        b = coords[i + 1]
        ab = b - a
        denom = np.dot(ab, ab)
        if denom == 0:
            t = 0.0
            q = a
        else:
            t = np.dot(p - a, ab) / denom
            t = np.clip(t, 0.0, 1.0)
            q = a + t * ab
        d2 = np.dot(p - q, p - q)
        if d2 < best["dist2"]:
            best.update({"dist2": d2, "seg_idx": i, "proj": q, "t": t})
    i_center = best["seg_idx"] + (1 if best["t"] >= 0.5 else 0)
    i0 = max(0, i_center - neighborhood)
    i1 = min(len(coords) - 1, i_center + neighborhood)
    tangent = coords[i1] - coords[i0]
    if np.allclose(tangent, 0.0):
        tangent = coords[best["seg_idx"] + 1] - coords[best["seg_idx"]]
    dx, dy = tangent[0], tangent[1]
    slope = np.inf if np.isclose(dx, 0.0) else dy / dx
    return {
        "slope": slope,
        "tangent_vector": (float(dx), float(dy)),
        "nearest_point": (float(best["proj"][0]), float(best["proj"][1])),
        "distance": float(np.sqrt(best["dist2"])),
    }


def tangent_slope_near_point(geom, point_xy, neighborhood=2):
    """
    Compute tangent slope near a point for LineString or MultiLineString geometry.
    """
    if isinstance(geom, LineString):
        return tangent_slope_on_line_near_point(
            geom, point_xy, neighborhood=neighborhood
        )
    if isinstance(geom, MultiLineString):
        pt = Point(point_xy)
        line = min(geom.geoms, key=lambda g: g.distance(pt))
        return tangent_slope_on_line_near_point(
            line, point_xy, neighborhood=neighborhood
        )
    raise TypeError("Geometry must be LineString or MultiLineString.")


def tangent_slope_near_point_in_geoseries(geoms, point_xy, neighborhood=2):
    """
    Find nearest geometry in a GeoSeries and compute tangent slope near point.
    """
    pt = Point(point_xy)
    best_geom = min(geoms, key=lambda g: g.distance(pt))
    return tangent_slope_near_point(
        best_geom, point_xy, neighborhood=neighborhood
    )


coast_gauss = coast_proj.copy()
coast_gauss["geometry"] = coast_proj.geometry.apply(
    lambda g: apply_to_geom(g, lambda line: gaussian_smooth(line, sigma=2))
)

```

```python
# Reproject back to lat/lon

def angle_in_degrees(tangent_info):
    tx, ty = tangent_info["tangent_vector"]
    if tx < 0:
        tx, ty = -tx, -ty
    theta_east = (np.degrees(np.arctan2(ty, tx)) + 360) % 360
    # print(theta_east, "--------------------------------------------------")
    return theta_east


coast_orig = coast_clip
coast_gauss = coast_gauss.to_crs("EPSG:4326")

# Query point in lon/lat for tangent visualization
query_point = (83.62, 17.17)
query_point = (88.92, 19.00) # Paradip
query_point = (80.203, 12.015) # Cuddalore
query_point = (82.982, 16.837) ## Kakinada
query_point = (85.442, 18.856) # Gopalpur
tangent_info = tangent_slope_near_point_in_geoseries(
    coast_gauss.geometry, query_point, neighborhood=2
)
nearest_point = tangent_info["nearest_point"]
query_points = ((83.62, 17.17),(88.92, 19.00),(80.203, 12.015),(82.982, 16.837),(85.442, 18.856))
for q in query_points:
    tangent_info = tangent_slope_near_point_in_geoseries(
        coast_gauss.geometry, q, neighborhood=2
    )
    nearest_point = tangent_info["nearest_point"]
    theta_east = angle_in_degrees(tangent_info)
    # print(f"Query: {q}, Nearest: {nearest_point}, Angle from east: {degrees_from_east:.1f}°")
    print(f"{q}, {theta_east:.1f}°")

tx, ty = tangent_info["tangent_vector"]
tangent_norm = np.hypot(tx, ty)
if tangent_norm > 0:
    ux, uy = tx / tangent_norm, ty / tangent_norm
else:
    ux, uy = 1.0, 0.0

# Tangent segment length in degree units (lon/lat plot)
tangent_half_len = 0.35
tangent_x = [
    nearest_point[0] - ux * tangent_half_len,
    nearest_point[0] + ux * tangent_half_len,
]
tangent_y = [
    nearest_point[1] - uy * tangent_half_len,
    nearest_point[1] + uy * tangent_half_len,
]


theta_east = angle_in_degrees(tangent_info)

# Plot

fig, ax = plt.subplots(figsize=(10, 8))

coast_orig.plot(ax=ax, linewidth=0.8, color="black", label="Original 110m")
coast_gauss.plot(ax=ax, linewidth=2, color="red", label="Gaussian")
ax.scatter(
    *query_point, color="dodgerblue", s=60, zorder=6, label="Input point"
)
ax.scatter(
    *nearest_point, color="orange", s=60, zorder=7, label="Nearest point"
)
ax.plot(
    tangent_x,
    tangent_y,
    color="green",
    linewidth=5,
    zorder=8,
    label="Tangent segment",
)

ax.set_xlim(lon_min, lon_max)
ax.set_ylim(lat_min, lat_max)

ax.set_title("Coastline Smoothing Comparison (70–95E, 5–30N)")
ax.set_xlabel("Longitude")
ax.set_ylabel("Latitude")
ax.legend()

plt.tight_layout()
plt.show()

```

```python
import geopandas as gpd
import cartopy.io.shapereader as shpreader
import matplotlib.pyplot as plt
import numpy as np
from shapely.geometry import LineString, MultiLineString, Point, box
from scipy.ndimage import gaussian_filter1d
from scipy.signal import savgol_filter
from shapely_smooth_funcs import *

lon_min, lon_max = 70, 95
lat_min, lat_max = 5, 30
region_bbox = box(lon_min, lat_min, lon_max, lat_max)

# Load Natural Earth 110m coastline
shpfilename = shpreader.natural_earth(
    resolution='50m',
    category='physical',
    name='coastline'
)

coast = gpd.read_file(shpfilename)
coast_clip = gpd.clip(coast, region_bbox)

# Reproject to metric CRS
coast_proj = coast_clip.to_crs("EPSG:3857")


def tangent_slope_on_line_near_point(line, point_xy, neighborhood=2):
    """
    Compute local tangent slope dy/dx on a LineString near a query point.
    Parameters
    ----------
    line : shapely.geometry.LineString
        Smoothed coastline line.
    point_xy : tuple(float, float)
        Query point (x, y) in the same CRS as `line`.
    neighborhood : int
        Number of points on each side used for a local finite-difference tangent.
    Returns
    -------
    dict with keys:
      slope : float (np.inf for vertical tangent)
      tangent_vector : tuple(float, float)
      nearest_point : tuple(float, float)
      distance : float
    """
    coords = np.asarray(line.coords, dtype=float)
    if coords.shape[0] < 2:
        raise ValueError("LineString must have at least 2 coordinates.")
    px, py = point_xy
    p = np.array([px, py], dtype=float)
    best = {
        "dist2": np.inf,
        "seg_idx": 0,
        "proj": coords[0].copy(),
        "t": 0.0,
    }
    for i in range(len(coords) - 1):
        a = coords[i]
        b = coords[i + 1]
        ab = b - a
        denom = np.dot(ab, ab)
        if denom == 0:
            t = 0.0
            q = a
        else:
            t = np.dot(p - a, ab) / denom
            t = np.clip(t, 0.0, 1.0)
            q = a + t * ab
        d2 = np.dot(p - q, p - q)
        if d2 < best["dist2"]:
            best.update({"dist2": d2, "seg_idx": i, "proj": q, "t": t})
    i_center = best["seg_idx"] + (1 if best["t"] >= 0.5 else 0)
    i0 = max(0, i_center - neighborhood)
    i1 = min(len(coords) - 1, i_center + neighborhood)
    tangent = coords[i1] - coords[i0]
    if np.allclose(tangent, 0.0):
        tangent = coords[best["seg_idx"] + 1] - coords[best["seg_idx"]]
    dx, dy = tangent[0], tangent[1]
    slope = np.inf if np.isclose(dx, 0.0) else dy / dx
    return {
        "slope": slope,
        "tangent_vector": (float(dx), float(dy)),
        "nearest_point": (float(best["proj"][0]), float(best["proj"][1])),
        "distance": float(np.sqrt(best["dist2"])),
    }


def tangent_slope_near_point(geom, point_xy, neighborhood=2):
    """
    Compute tangent slope near a point for LineString or MultiLineString geometry.
    """
    if isinstance(geom, LineString):
        return tangent_slope_on_line_near_point(geom, point_xy, neighborhood=neighborhood)
    if isinstance(geom, MultiLineString):
        pt = Point(point_xy)
        line = min(geom.geoms, key=lambda g: g.distance(pt))
        return tangent_slope_on_line_near_point(line, point_xy, neighborhood=neighborhood)
    raise TypeError("Geometry must be LineString or MultiLineString.")


def tangent_slope_near_point_in_geoseries(geoms, point_xy, neighborhood=2):
    """
    Find nearest geometry in a GeoSeries and compute tangent slope near point.
    """
    pt = Point(point_xy)
    best_geom = min(geoms, key=lambda g: g.distance(pt))
    return tangent_slope_near_point(best_geom, point_xy, neighborhood=neighborhood)


def along_across_shore_components(gu, gv, coast_heading_deg):
    """
    Resolve vector (gu, gv) into along-shore and across-shore components.

    Sign convention:
    - coast_heading_deg is measured CCW from +x (East).
    - along_shore is positive in the heading direction.
```

```
    - across_shore is positive 90 deg CCW from heading (left-hand side of coast).
      If you need right-hand positive, negate across_shore.
    """
    theta = np.deg2rad(coast_heading_deg)

    # Unit vectors
    along_hat = np.array([np.cos(theta), np.sin(theta)])
    across_hat = np.array([-np.sin(theta), np.cos(theta)])  # +90 deg CCW

    v = np.array([gu, gv], dtype=float)
    along_shore = float(np.dot(v, along_hat))
    across_shore = float(np.dot(v, across_hat))

    return along_shore, across_shore


coast_gauss = coast_proj.copy()
coast_gauss["geometry"] = coast_proj.geometry.apply(
    lambda g: apply_to_geom(g, lambda line: gaussian_smooth(line, sigma=1))
)

coast_simplify = coast_proj.copy()
coast_simplify["geometry"] = coast_proj.geometry.simplify(
    tolerance=30000, preserve_topology=True
)

coast_movavg = coast_proj.copy()
coast_movavg["geometry"] = coast_proj.geometry.apply(
    lambda g: apply_to_geom(g, lambda line: moving_average_smooth(line, window=15))
)

coast_savgol = coast_proj.copy()
coast_savgol["geometry"] = coast_proj.geometry.apply(
    lambda g: apply_to_geom(g, lambda line: savgol_smooth(line, window=21, poly=2))
)

coast_poly = coast_proj.copy()
coast_poly["geometry"] = coast_proj.geometry.apply(
    lambda g: apply_to_geom(g, lambda line: polyfit_smooth(line, window_km=300, degree=2))
)

# Reproject back to lat/lon

coast_orig = coast_clip
coast_gauss = coast_gauss.to_crs("EPSG:4326")
coast_simplify = coast_simplify.to_crs("EPSG:4326")
coast_movavg = coast_movavg.to_crs("EPSG:4326")
coast_savgol = coast_savgol.to_crs("EPSG:4326")
coast_poly = coast_poly.to_crs("EPSG:4326")

# Query point in lon/lat for tangent visualization
query_point = (83.62, 17.17)

tangent_info = tangent_slope_near_point_in_geoseries(
    coast_gauss.geometry, query_point, neighborhood=2
)
nearest_point = tangent_info["nearest_point"]
tx, ty = tangent_info["tangent_vector"]
tangent_norm = np.hypot(tx, ty)
if tangent_norm > 0:
    ux, uy = tx / tangent_norm, ty / tangent_norm
else:
    ux, uy = 1.0, 0.0

# Tangent segment length in degree units (lon/lat plot)
tangent_half_len = 0.35
tangent_x = [
    nearest_point[0] - ux * tangent_half_len,
    nearest_point[0] + ux * tangent_half_len,
]
tangent_y = [
    nearest_point[1] - uy * tangent_half_len,
    nearest_point[1] + uy * tangent_half_len,
]

def angle_in_degrees(tangent_info):
    tx, ty = tangent_info["tangent_vector"]
    if tx < 0:
        tx, ty = -tx, -ty
    theta_east = (np.degrees(np.arctan2(ty, tx)) + 360) % 360
    print(theta_east, "--------------------------------------------------")
    return theta_east

theta_east = angle_in_degrees(tangent_info)

# Plot

fig, ax = plt.subplots(figsize=(10, 8))

coast_orig.plot(ax=ax, linewidth=0.8, color="black", label="Original 110m")
coast_gauss.plot(ax=ax, linewidth=2, color="red", label="Gaussian")
# coast_simplify.plot(ax=ax, linewidth=2, color="blue", label="Simplify")
# coast_movavg.plot(ax=ax, linewidth=2, color="green", label="Moving Avg")
# coast_savgol.plot(ax=ax, linewidth=2, color="purple", label="Savitzky-Golay")
# coast_poly.plot(ax=ax, linewidth=2, color="purple", label="Polynomial")
ax.scatter(*query_point, color="dodgerblue", s=60, zorder=6, label="Input point")
ax.scatter(*nearest_point, color="orange", s=60, zorder=7, label="Nearest point")
ax.plot(tangent_x, tangent_y, color="green", linewidth=5, zorder=8, label="Tangent segment")

ax.set_xlim(lon_min, lon_max)
ax.set_ylim(lat_min, lat_max)

ax.set_title("Coastline Smoothing Comparison (70–95E, 5–30N)")
ax.set_xlabel("Longitude")
ax.set_ylabel("Latitude")
ax.legend()

plt.tight_layout()
plt.show()

```

```python
import os

import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.gridspec as gridspec
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from matplotlib.lines import Line2D
from matplotlib.patches import Patch
from scipy.signal import butter, filtfilt
from scipy.stats import linregress
from tools_shapely import *
from tools_xtrackm import *

# copilot --resume=d419a1e5-878f-477e-813b-3b06f3d9a50b

method = "loess"
frac1 = "0.05"

sat_val = "TP+J1+J2+J3+S6A"
sat_val = "S3A"
sat_val = "ERS1+ERS2+ENV+SRL"

df = pd.read_csv(f"intersections_near_east_coast_{sat_val}.csv")

# Skip the first point (lowest latitude)
df_out = df.sort_values("lats_inter").iloc[1:].reset_index(drop=True)

N_PTS = 7
assert len(df_out) == N_PTS, (
    f"Expected {N_PTS} points after skipping first, got {len(df_out)}"
)

POINT_COLORS = [plt.cm.tab10(i) for i in range(N_PTS)]
POINT_LABELS = list("ABCDEFG")

# Map extent: 10N–22N, 78E–88E
MAP_EXTENT = (78.5, 88.0, 10.0, 22.0)

# --- Pre-collect all timeseries data ---
point_data = []
for i, row in df_out.iterrows():
    sat1 = row["sat"]
    tno_s = str(row["track_self"])
    tno_o = str(row["track_other"])
    lon1 = float(row["lons_inter"])
    lat1 = float(row["lats_inter"])

    f_guv = (
        f"../guv/guv_at_intersections_{method}/"
        f"guv_at_intersection_{sat1}_{tno_s}_{tno_o}_{method}_{frac1}.nc"
    )
    ds_guv = xr.open_dataset(f_guv)
    gu = ds_guv.gu
    gv = ds_guv.gv
    gs = np.sqrt(gu**2 + gv**2)
    times = pd.to_datetime(gs.time.values)
    gs_vals_raw = gs.values.astype(float)

    # 500-day low-pass filter
    dt_days = np.median(
        np.diff(times).astype("timedelta64[s]").astype(float) / 86400.0
    )
    print(dt_days)
    nyq = 0.5 / dt_days
    cutoff = 1.0 / 500.0
    b_filt, a_filt = butter(4, cutoff / nyq, btype="low")
    nan_mask = np.isnan(gs_vals_raw)
    gs_interp = (
        pd.Series(gs_vals_raw).interpolate(limit_direction="both").values
    )
    if len(gs_interp) > 3 * max(len(a_filt), len(b_filt)):
        gs_lp_vals = filtfilt(b_filt, a_filt, gs_interp)
    else:
        samples_500d = max(3, int(round(500.0 / dt_days)))
        gs_lp_vals = (
            pd.Series(gs_interp)
            .rolling(window=samples_500d, center=True, min_periods=1)
            .mean()
            .values
        )
    gs_lp_vals[nan_mask] = np.nan
    gs_lowpass = xr.DataArray(
        gs_lp_vals, coords=gs.coords, dims=gs.dims, name="gs_lowpass"
    )

    # Linear trend on low-pass series
    t_days = (times - times[0]).days.astype(float)
    valid = np.isfinite(gs_lp_vals)
    trend_res = linregress(t_days[valid], gs_lp_vals[valid])
    fit_vals = trend_res.intercept + trend_res.slope * t_days
    trend_fit = xr.DataArray(
        fit_vals, coords=gs.coords, dims=gs.dims, name="trend_fit"
    )

    point_data.append(
        dict(
            sat1=sat1,
            tno_s=tno_s,
            tno_o=tno_o,
            lon1=lon1,
            lat1=lat1,
            gs=gs,
            gs_lowpass=gs_lowpass,
            trend_fit=trend_fit,
            times=times,
            slope_yr=trend_res.slope * 365.25,
            stderr_yr=trend_res.stderr * 365.25,
            pvalue=trend_res.pvalue,
            significant=trend_res.pvalue < 0.05,
        )
    )

# --- Build figure: left = tall map, right = 7 timeseries rows ---
fig = plt.figure(figsize=(18, 14))
```

```
gs_layout = gridspec.GridSpec(
    N_PTS,
    2,
    figure=fig,
    width_ratios=[1, 2.5],
    hspace=0.0,
    wspace=0.15,
)

# --- Left: vertically elongated cartopy map spanning all rows ---
ax_map = fig.add_subplot(gs_layout[1:, 0], projection=ccrs.PlateCarree())
ax_map.set_extent(MAP_EXTENT, crs=ccrs.PlateCarree())
ax_map.add_feature(cfeature.LAND, facecolor="0.93", zorder=0)
ax_map.add_feature(cfeature.OCEAN, facecolor="aliceblue", zorder=0)
ax_map.coastlines(resolution="50m", linewidth=1.0, zorder=2)
gl = ax_map.gridlines(
    draw_labels=True,
    linewidth=0.5,
    color="gray",
    alpha=0.6,
    linestyle="--",
)
gl.top_labels = False
gl.right_labels = False

# --- Isobaths: 200, 1000, 2000 m ---
dse = xr.open_dataset(os.path.expanduser("~/allData/topo/etopo5.cdf"))
rose_slice = dse.ROSE.sel(
    ETOPO05_X=slice(MAP_EXTENT[0], MAP_EXTENT[1]),
    ETOPO05_Y=slice(MAP_EXTENT[2], MAP_EXTENT[3]),
)
lons_topo = rose_slice.ETOPO05_X.values
lats_topo = rose_slice.ETOPO05_Y.values
topo_vals = rose_slice.values
iso_levels = [-2000, -1000, -200]
iso_level1 = [-200]
cs_iso = ax_map.contour(
    lons_topo, lats_topo, topo_vals,
    levels=iso_level1,
    colors="steelblue",
    linewidths=0.7,
    linestyles="--",
    transform=ccrs.PlateCarree(),
    zorder=4,
)
iso_level2 = [-1000]
cs_iso = ax_map.contour(
    lons_topo, lats_topo, topo_vals,
    levels=iso_level2,
    colors="r",
    linewidths=0.7,
    linestyles="--",
    transform=ccrs.PlateCarree(),
    zorder=4,
)
# ax_map.clabel(
#     cs_iso,
#     fmt={-200: "200 m", -1000: "1000 m", -2000: "2000 m"},
#     fontsize=7,
#     inline=True,
# )

# Connect all 7 points with a dashed gray line
lons_pts = [e["lon1"] for e in point_data]
lats_pts = [e["lat1"] for e in point_data]
ax_map.plot(
    lons_pts,
    lats_pts,
    color="gray",
    linewidth=0.8,
    linestyle="--",
    transform=ccrs.PlateCarree(),
    zorder=3,
)

# Plot each point with its assigned color and letter label
for idx, entry in enumerate(point_data):
    color = POINT_COLORS[idx]
    label = POINT_LABELS[idx]
    ax_map.plot(
        entry["lon1"],
        entry["lat1"],
        marker="o",
        markersize=5,
        color=color,
        markeredgecolor="black",
        markeredgewidth=0.8,
        transform=ccrs.PlateCarree(),
        zorder=5,
    )
    ax_map.text(
        entry["lon1"] + 0.18,
        entry["lat1"] + 0.1,
        label,
        fontsize=10,
        fontweight="bold",
        color=color,
        transform=ccrs.PlateCarree(),
        zorder=6,
    )
    # Slope annotation: gray if not significant, colored if significant
    slope_color = color if entry["significant"] else "0.6"
    ax_map.text(
        entry["lon1"] + 0.18,
        entry["lat1"] - 0.45,
        f"{entry['slope_yr']:+.4f} m/s/yr",
        fontsize=7,
        color=slope_color,
        transform=ccrs.PlateCarree(),
        zorder=6,
    )

# --- Description box above the map ---
desc_lines = ["Pt   Longitude   Latitude    Tracks"]
desc_lines.append("—" * 38)
for idx, entry in enumerate(point_data):
    desc_lines.append(
```

```
        f" {POINT_LABELS[idx]}   {entry['lon1']:6.2f}°E  "
        f"{entry['lat1']:5.2f}°N   T{entry['tno_s']}×T{entry['tno_o']}"
    )
ax_map.text(
    0.15, 1.05,
    "\n".join(desc_lines),
    transform=ax_map.transAxes,
    fontsize=7.5,
    fontfamily="monospace",
    va="bottom", ha="left",
    clip_on=False,
    bbox=dict(
        boxstyle="round,pad=0.45",
        facecolor="white",
        edgecolor="0.35",
        alpha=0.95,
    ),
    zorder=10,
)

# --- Right: 7 timeseries subplots, A at bottom (row 6) → G at top (row 0) ---
for idx, entry in enumerate(point_data):
    row = N_PTS - 1 - idx  # A → row 6, G → row 0
    ax_ts = fig.add_subplot(gs_layout[row, 1])
    color = POINT_COLORS[idx]
    label = POINT_LABELS[idx]
    times = entry["times"]
    gs_da = entry["gs"]
    gs_lowpass = entry["gs_lowpass"]
    trend_fit = entry["trend_fit"]

    # 500-day low-pass (same color, bold)
    ax_ts.plot(
        times,
        gs_lowpass.values,
        color=color,
        linewidth=2.0,
        label="500-day LP",
    )
    # Linear trend (black dashed)
    ax_ts.plot(
        times,
        trend_fit.values,
        color="k",
        linewidth=1.5,
        linestyle="--",
        label="trend",
    )

    # Seasonal background shading
    year_min, year_max = times.year.min(), times.year.max()
    for yr in range(year_min, year_max + 1):
        ax_ts.axvspan(
            pd.Timestamp(yr, 2, 1),
            pd.Timestamp(yr, 5, 31),
            color="yellow",
            alpha=0.15,
            zorder=0,
        )
        ax_ts.axvspan(
            pd.Timestamp(yr, 6, 1),
            pd.Timestamp(yr, 9, 30),
            color="cyan",
            alpha=0.15,
            zorder=0,
        )
        ax_ts.axvspan(
            pd.Timestamp(yr, 10, 1),
            pd.Timestamp(yr, 12, 31),
            color="gray",
            alpha=0.15,
            zorder=0,
        )
        ax_ts.axvline(
            pd.Timestamp(yr, 1, 1),
            color="k",
            linewidth=0.3,
            linestyle="--",
            alpha=0.4,
        )

    # Point label inside left edge of plot
    ax_ts.text(
        0.005, 0.5, label,
        transform=ax_ts.transAxes, fontsize=12,
        fontweight="bold", color=color,
        va="center", ha="left",
    )
    # Tighten y-axis to the filtered (low-pass) series range
    all_vals = gs_lowpass.values[np.isfinite(gs_lowpass.values)]
    ypad = (all_vals.max() - all_vals.min()) * 0.08
    ax_ts.set_ylim(all_vals.min() - ypad, all_vals.max() + ypad)
    ax_ts.set_ylabel("m/s", fontsize=14)
    ax_ts.tick_params(labelsize=12)
    # Only bottom subplot (A, idx=0) shows x-tick labels
    if idx > 0:
        ax_ts.set_xticklabels([])

    # Legend only on the top panel (G, idx=N_PTS-1)
    if idx == N_PTS - 1:
        handles = [
            Line2D([0], [0], color=color, lw=2.0, label="500-day low-pass"),
            Line2D(
                [0],
                [0],
                color="k",
                lw=1.5,
                linestyle="--",
                label="linear trend",
            ),
            Patch(
                facecolor="yellow",
                alpha=0.4,
                edgecolor="none",
                label="Feb–May",
            ),
            Patch(
```

```
                facecolor="cyan", alpha=0.4, edgecolor="none", label="Jun–Sep"
            ),
            Patch(
                facecolor="gray", alpha=0.4, edgecolor="none", label="Oct–Dec"
            ),
        ]
        ax_ts.legend(handles=handles, loc="upper right", fontsize=12, ncol=3)

# fig.suptitle(
#     f"Current speed at EICC coastal intersection points — {sat_val}",
#     fontsize=13,
#     fontweight="bold",
#     y=1.002,
# )
out_png = f"speed_timeseries_allinone_{sat_val}.png"
plt.savefig(out_png, dpi=150, bbox_inches="tight")
plt.show()
plt.close("all")
print(f"Saved: {out_png}")

```

```python
import os

import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.gridspec as gridspec
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from matplotlib.lines import Line2D
from matplotlib.patches import Patch
from scipy.signal import butter, filtfilt
from scipy.stats import linregress
from tools_shapely import *
from tools_xtrackm import *

# copilot --resume=d419a1e5-878f-477e-813b-3b06f3d9a50b

method = "loess"
frac1 = "0.05"

sat_val = "S3A"
sat_val = "ERS1+ERS2+ENV+SRL"
sat_val = "TP+J1+J2+J3+S6A"

df = pd.read_csv(f"intersections_near_east_coast_{sat_val}.csv")

# Skip the first point (lowest latitude)
df_out = df.sort_values("lats_inter").reset_index(drop=True)

N_PTS = 3
assert len(df_out) == N_PTS, (
    f"Expected {N_PTS} points after skipping first, got {len(df_out)}"
)

POINT_COLORS = [plt.cm.tab10(i) for i in range(N_PTS)]
POINT_LABELS = list("ABC")

# Map extent: 10N–22N, 78E–88E
MAP_EXTENT = (78., 89.0, 9.5, 22.0)

# --- Pre-collect all timeseries data ---
point_data = []
for i, row in df_out.iterrows():
    sat1 = row["sat"]
    tno_s = str(row["track_self"])
    tno_o = str(row["track_other"])
    lon1 = float(row["lons_inter"])
    lat1 = float(row["lats_inter"])

    f_guv = (
        f"../guv/guv_at_intersections_{method}/"
        f"guv_at_intersection_{sat1}_{tno_s}_{tno_o}_{method}_{frac1}.nc"
    )
    ds_guv = xr.open_dataset(f_guv)
    gu = ds_guv.gu
    gv = ds_guv.gv
    gs = np.sqrt(gu**2 + gv**2)
    times = pd.to_datetime(gs.time.values)
    gs_vals_raw = gs.values.astype(float)

    # 500-day low-pass filter
    dt_days = np.median(
        np.diff(times).astype("timedelta64[s]").astype(float) / 86400.0
    )
    print(dt_days)
    nyq = 0.5 / dt_days
    cutoff = 1.0 / 500.0
    b_filt, a_filt = butter(4, cutoff / nyq, btype="low")
    nan_mask = np.isnan(gs_vals_raw)
    gs_interp = (
        pd.Series(gs_vals_raw).interpolate(limit_direction="both").values
    )
    if len(gs_interp) > 3 * max(len(a_filt), len(b_filt)):
        gs_lp_vals = filtfilt(b_filt, a_filt, gs_interp)
    else:
        samples_500d = max(3, int(round(500.0 / dt_days)))
        gs_lp_vals = (
            pd.Series(gs_interp)
            .rolling(window=samples_500d, center=True, min_periods=1)
            .mean()
            .values
        )
    gs_lp_vals[nan_mask] = np.nan
    gs_lowpass = xr.DataArray(
        gs_lp_vals, coords=gs.coords, dims=gs.dims, name="gs_lowpass"
    )

    # Linear trend on low-pass series
    t_days = (times - times[0]).days.astype(float)
    valid = np.isfinite(gs_lp_vals)
    trend_res = linregress(t_days[valid], gs_lp_vals[valid])
    fit_vals = trend_res.intercept + trend_res.slope * t_days
    trend_fit = xr.DataArray(
        fit_vals, coords=gs.coords, dims=gs.dims, name="trend_fit"
    )

    point_data.append(
        dict(
            sat1=sat1,
            tno_s=tno_s,
            tno_o=tno_o,
            lon1=lon1,
            lat1=lat1,
            gs=gs,
            gs_lowpass=gs_lowpass,
            trend_fit=trend_fit,
            times=times,
            slope_yr=trend_res.slope * 365.25,
            stderr_yr=trend_res.stderr * 365.25,
            pvalue=trend_res.pvalue,
            significant=trend_res.pvalue < 0.05,
        )
    )

# --- Build figure: left = tall map, right = 7 timeseries rows ---
fig = plt.figure(figsize=(18, 14))
```

```
gs_layout = gridspec.GridSpec(
    N_PTS,
    2,
    figure=fig,
    width_ratios=[1, 2.5],
    hspace=0.0,
    wspace=0.15,
)

# --- Left: vertically elongated cartopy map spanning all rows ---
ax_map = fig.add_subplot(gs_layout[:, 0], projection=ccrs.PlateCarree())
ax_map.set_extent(MAP_EXTENT, crs=ccrs.PlateCarree())
ax_map.add_feature(cfeature.LAND, facecolor="0.93", zorder=0)
ax_map.add_feature(cfeature.OCEAN, facecolor="aliceblue", zorder=0)
ax_map.coastlines(resolution="50m", linewidth=1.0, zorder=2)
gl = ax_map.gridlines(
    draw_labels=True,
    linewidth=0.5,
    color="gray",
    alpha=0.6,
    linestyle="--",
)
gl.top_labels = False
gl.right_labels = False

# --- Isobaths: 200, 1000, 2000 m ---
dse = xr.open_dataset(os.path.expanduser("~/allData/topo/etopo5.cdf"))
rose_slice = dse.ROSE.sel(
    ETOPO05_X=slice(MAP_EXTENT[0], MAP_EXTENT[1]),
    ETOPO05_Y=slice(MAP_EXTENT[2], MAP_EXTENT[3]),
)
lons_topo = rose_slice.ETOPO05_X.values
lats_topo = rose_slice.ETOPO05_Y.values
topo_vals = rose_slice.values
iso_levels = [-2000, -1000, -200]
iso_level1 = [-200]
cs_iso = ax_map.contour(
    lons_topo, lats_topo, topo_vals,
    levels=iso_level1,
    colors="steelblue",
    linewidths=0.7,
    linestyles="--",
    transform=ccrs.PlateCarree(),
    zorder=4,
)
iso_level2 = [-1000]
cs_iso = ax_map.contour(
    lons_topo, lats_topo, topo_vals,
    levels=iso_level2,
    colors="r",
    linewidths=0.7,
    linestyles="--",
    transform=ccrs.PlateCarree(),
    zorder=4,
)
# ax_map.clabel(
#     cs_iso,
#     fmt={-200: "200 m", -1000: "1000 m", -2000: "2000 m"},
#     fontsize=7,
#     inline=True,
# )

# Connect all 7 points with a dashed gray line
lons_pts = [e["lon1"] for e in point_data]
lats_pts = [e["lat1"] for e in point_data]
ax_map.plot(
    lons_pts,
    lats_pts,
    color="gray",
    linewidth=0.8,
    linestyle="--",
    transform=ccrs.PlateCarree(),
    zorder=3,
)

# Plot each point with its assigned color and letter label
for idx, entry in enumerate(point_data):
    color = POINT_COLORS[idx]
    label = POINT_LABELS[idx]
    ax_map.plot(
        entry["lon1"],
        entry["lat1"],
        marker="o",
        markersize=5,
        color=color,
        markeredgecolor="black",
        markeredgewidth=0.8,
        transform=ccrs.PlateCarree(),
        zorder=5,
    )
    ax_map.text(
        entry["lon1"] + 0.18,
        entry["lat1"] + 0.1,
        label,
        fontsize=10,
        fontweight="bold",
        color=color,
        transform=ccrs.PlateCarree(),
        zorder=6,
    )
    # Slope annotation: gray if not significant, colored if significant
    slope_color = color if entry["significant"] else "0.6"
    ax_map.text(
        entry["lon1"] + 0.18,
        entry["lat1"] - 0.45,
        f"{entry['slope_yr']:+.4f} m/s/yr",
        fontsize=7,
        color=slope_color,
        transform=ccrs.PlateCarree(),
        zorder=6,
    )

# --- Description box above the map ---
desc_lines = ["Pt   Longitude   Latitude    Tracks"]
desc_lines.append("—" * 38)
for idx, entry in enumerate(point_data):
    desc_lines.append(
```

```
        f" {POINT_LABELS[idx]}   {entry['lon1']:6.2f}°E  "
        f"{entry['lat1']:5.2f}°N   T{entry['tno_s']}×T{entry['tno_o']}"
    )
ax_map.text(
    0.15, 1.05,
    "\n".join(desc_lines),
    transform=ax_map.transAxes,
    fontsize=7.5,
    fontfamily="monospace",
    va="bottom", ha="left",
    clip_on=False,
    bbox=dict(
        boxstyle="round,pad=0.45",
        facecolor="white",
        edgecolor="0.35",
        alpha=0.95,
    ),
    zorder=10,
)

# --- Right: 7 timeseries subplots, A at bottom (row 6) → G at top (row 0) ---
for idx, entry in enumerate(point_data):
    row = N_PTS - 1 - idx  # A → row 6, G → row 0
    ax_ts = fig.add_subplot(gs_layout[row, 1])
    color = POINT_COLORS[idx]
    label = POINT_LABELS[idx]
    times = entry["times"]
    gs_da = entry["gs"]
    gs_lowpass = entry["gs_lowpass"]
    trend_fit = entry["trend_fit"]

    # 500-day low-pass (same color, bold)
    ax_ts.plot(
        times,
        gs_lowpass.values,
        color=color,
        linewidth=2.0,
        label="500-day LP",
    )
    # Linear trend (black dashed)
    ax_ts.plot(
        times,
        trend_fit.values,
        color="k",
        linewidth=1.5,
        linestyle="--",
        label="trend",
    )

    # Seasonal background shading
    year_min, year_max = times.year.min(), times.year.max()
    for yr in range(year_min, year_max + 1):
        ax_ts.axvspan(
            pd.Timestamp(yr, 2, 1),
            pd.Timestamp(yr, 5, 31),
            color="yellow",
            alpha=0.15,
            zorder=0,
        )
        ax_ts.axvspan(
            pd.Timestamp(yr, 6, 1),
            pd.Timestamp(yr, 9, 30),
            color="cyan",
            alpha=0.15,
            zorder=0,
        )
        ax_ts.axvspan(
            pd.Timestamp(yr, 10, 1),
            pd.Timestamp(yr, 12, 31),
            color="gray",
            alpha=0.15,
            zorder=0,
        )
        ax_ts.axvline(
            pd.Timestamp(yr, 1, 1),
            color="k",
            linewidth=0.3,
            linestyle="--",
            alpha=0.4,
        )

    # Point label inside left edge of plot
    ax_ts.text(
        0.005, 0.5, label,
        transform=ax_ts.transAxes, fontsize=12,
        fontweight="bold", color=color,
        va="center", ha="left",
    )
    # Tighten y-axis to the filtered (low-pass) series range
    all_vals = gs_lowpass.values[np.isfinite(gs_lowpass.values)]
    ypad = (all_vals.max() - all_vals.min()) * 0.08
    ax_ts.set_ylim(all_vals.min() - ypad, all_vals.max() + ypad)
    ax_ts.set_ylabel("m/s", fontsize=14)
    ax_ts.tick_params(labelsize=12)
    # Only bottom subplot (A, idx=0) shows x-tick labels
    if idx > 0:
        ax_ts.set_xticklabels([])

    # Legend only on the top panel (G, idx=N_PTS-1)
    if idx == N_PTS - 1:
        handles = [
            Line2D([0], [0], color=color, lw=2.0, label="500-day low-pass"),
            Line2D(
                [0],
                [0],
                color="k",
                lw=1.5,
                linestyle="--",
                label="linear trend",
            ),
            Patch(
                facecolor="yellow",
                alpha=0.4,
                edgecolor="none",
                label="Feb–May",
            ),
            Patch(
```

```
                facecolor="cyan", alpha=0.4, edgecolor="none", label="Jun–Sep"
            ),
            Patch(
                facecolor="gray", alpha=0.4, edgecolor="none", label="Oct–Dec"
            ),
        ]
        ax_ts.legend(handles=handles, loc="upper right", fontsize=12, ncol=3)

# fig.suptitle(
#     f"Current speed at EICC coastal intersection points — {sat_val}",
#     fontsize=13,
#     fontweight="bold",
#     y=1.002,
# )
out_png = f"speed_timeseries_allinone_{sat_val}.png"
plt.savefig(out_png, dpi=150, bbox_inches="tight")
plt.show()
plt.close("all")
print(f"Saved: {out_png}")

```

```python
import os

import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.gridspec as gridspec
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from matplotlib.lines import Line2D
from matplotlib.patches import Patch
from scipy.signal import butter, filtfilt
from scipy.stats import linregress
from tools_shapely import *
from tools_xtrackm import *

# copilot --resume=d419a1e5-878f-477e-813b-3b06f3d9a50b

method = "loess"
frac1 = "0.05"

sat_val = "TPN+J1N+J2N+J3N"

df = pd.read_csv(f"intersections_near_east_coast_{sat_val}.csv")

# Skip the first point (lowest latitude)
df_out = df.sort_values("lats_inter").reset_index(drop=True)

N_PTS = 2
assert len(df_out) == N_PTS, (
    f"Expected {N_PTS} points after skipping first, got {len(df_out)}"
)

POINT_COLORS = [plt.cm.tab10(i) for i in range(N_PTS)]
POINT_LABELS = list("ABCDEFG")

# Map extent: 10N–22N, 78E–88E
MAP_EXTENT = (78.5, 88.0, 9.0, 22.0)

# --- Pre-collect all timeseries data ---
point_data = []
for i, row in df_out.iterrows():
    sat1 = row["sat"]
    tno_s = str(row["track_self"])
    tno_o = str(row["track_other"])
    lon1 = float(row["lons_inter"])
    lat1 = float(row["lats_inter"])

    f_guv = (
        f"../guv/guv_at_intersections_{method}/"
        f"guv_at_intersection_{sat1}_{tno_s}_{tno_o}_{method}_{frac1}.nc"
    )
    ds_guv = xr.open_dataset(f_guv)
    gu = ds_guv.gu
    gv = ds_guv.gv
    gs = np.sqrt(gu**2 + gv**2)
    times = pd.to_datetime(gs.time.values)
    gs_vals_raw = gs.values.astype(float)

    # 500-day low-pass filter
    dt_days = np.median(
        np.diff(times).astype("timedelta64[s]").astype(float) / 86400.0
    )
    print(dt_days)
    nyq = 0.5 / dt_days
    cutoff = 1.0 / 500.0
    b_filt, a_filt = butter(4, cutoff / nyq, btype="low")
    nan_mask = np.isnan(gs_vals_raw)
    gs_interp = (
        pd.Series(gs_vals_raw).interpolate(limit_direction="both").values
    )
    if len(gs_interp) > 3 * max(len(a_filt), len(b_filt)):
        gs_lp_vals = filtfilt(b_filt, a_filt, gs_interp)
    else:
        samples_500d = max(3, int(round(500.0 / dt_days)))
        gs_lp_vals = (
            pd.Series(gs_interp)
            .rolling(window=samples_500d, center=True, min_periods=1)
            .mean()
            .values
        )
    gs_lp_vals[nan_mask] = np.nan
    gs_lowpass = xr.DataArray(
        gs_lp_vals, coords=gs.coords, dims=gs.dims, name="gs_lowpass"
    )

    # Linear trend on low-pass series
    t_days = (times - times[0]).days.astype(float)
    valid = np.isfinite(gs_lp_vals)
    trend_res = linregress(t_days[valid], gs_lp_vals[valid])
    fit_vals = trend_res.intercept + trend_res.slope * t_days
    trend_fit = xr.DataArray(
        fit_vals, coords=gs.coords, dims=gs.dims, name="trend_fit"
    )

    point_data.append(
        dict(
            sat1=sat1,
            tno_s=tno_s,
            tno_o=tno_o,
            lon1=lon1,
            lat1=lat1,
            gs=gs,
            gs_lowpass=gs_lowpass,
            trend_fit=trend_fit,
            times=times,
            slope_yr=trend_res.slope * 365.25,
            stderr_yr=trend_res.stderr * 365.25,
            pvalue=trend_res.pvalue,
            significant=trend_res.pvalue < 0.05,
        )
    )

# --- Build figure: left = tall map, right = 7 timeseries rows ---
fig = plt.figure(figsize=(18, 14))
gs_layout = gridspec.GridSpec(
    N_PTS,
```

```
    2,
    figure=fig,
    width_ratios=[1, 2.5],
    hspace=0.0,
    wspace=0.15,
)

# --- Left: vertically elongated cartopy map spanning all rows ---
ax_map = fig.add_subplot(gs_layout[:, 0], projection=ccrs.PlateCarree())
ax_map.set_extent(MAP_EXTENT, crs=ccrs.PlateCarree())
ax_map.add_feature(cfeature.LAND, facecolor="0.93", zorder=0)
ax_map.add_feature(cfeature.OCEAN, facecolor="aliceblue", zorder=0)
ax_map.coastlines(resolution="50m", linewidth=1.0, zorder=2)
gl = ax_map.gridlines(
    draw_labels=True,
    linewidth=0.5,
    color="gray",
    alpha=0.6,
    linestyle="--",
)
gl.top_labels = False
gl.right_labels = False

# --- Isobaths: 200, 1000, 2000 m ---
dse = xr.open_dataset(os.path.expanduser("~/allData/topo/etopo5.cdf"))
rose_slice = dse.ROSE.sel(
    ETOPO05_X=slice(MAP_EXTENT[0], MAP_EXTENT[1]),
    ETOPO05_Y=slice(MAP_EXTENT[2], MAP_EXTENT[3]),
)
lons_topo = rose_slice.ETOPO05_X.values
lats_topo = rose_slice.ETOPO05_Y.values
topo_vals = rose_slice.values
iso_levels = [-2000, -1000, -200]
iso_level1 = [-200]
cs_iso = ax_map.contour(
    lons_topo, lats_topo, topo_vals,
    levels=iso_level1,
    colors="steelblue",
    linewidths=0.7,
    linestyles="--",
    transform=ccrs.PlateCarree(),
    zorder=4,
)
iso_level2 = [-1000]
cs_iso = ax_map.contour(
    lons_topo, lats_topo, topo_vals,
    levels=iso_level2,
    colors="r",
    linewidths=0.7,
    linestyles="--",
    transform=ccrs.PlateCarree(),
    zorder=4,
)
# ax_map.clabel(
#     cs_iso,
#     fmt={-200: "200 m", -1000: "1000 m", -2000: "2000 m"},
#     fontsize=7,
#     inline=True,
# )

# Connect all 7 points with a dashed gray line
lons_pts = [e["lon1"] for e in point_data]
lats_pts = [e["lat1"] for e in point_data]
ax_map.plot(
    lons_pts,
    lats_pts,
    color="gray",
    linewidth=0.8,
    linestyle="--",
    transform=ccrs.PlateCarree(),
    zorder=3,
)

# Plot each point with its assigned color and letter label
for idx, entry in enumerate(point_data):
    color = POINT_COLORS[idx]
    label = POINT_LABELS[idx]
    ax_map.plot(
        entry["lon1"],
        entry["lat1"],
        marker="o",
        markersize=5,
        color=color,
        markeredgecolor="black",
        markeredgewidth=0.8,
        transform=ccrs.PlateCarree(),
        zorder=5,
    )
    ax_map.text(
        entry["lon1"] + 0.18,
        entry["lat1"] + 0.1,
        label,
        fontsize=10,
        fontweight="bold",
        color=color,
        transform=ccrs.PlateCarree(),
        zorder=6,
    )
    # Slope annotation: gray if not significant, colored if significant
    slope_color = color if entry["significant"] else "0.6"
    ax_map.text(
        entry["lon1"] + 0.18,
        entry["lat1"] - 0.45,
        f"{entry['slope_yr']:+.4f} m/s/yr",
        fontsize=7,
        color=slope_color,
        transform=ccrs.PlateCarree(),
        zorder=6,
    )

# --- Description box above the map ---
desc_lines = ["Pt   Longitude   Latitude    Tracks"]
desc_lines.append("—" * 38)
for idx, entry in enumerate(point_data):
    desc_lines.append(
        f" {POINT_LABELS[idx]}   {entry['lon1']:6.2f}°E  "
        f"{entry['lat1']:5.2f}°N   T{entry['tno_s']}×T{entry['tno_o']}"
```

```
    )
ax_map.text(
    0.15, 1.05,
    "\n".join(desc_lines),
    transform=ax_map.transAxes,
    fontsize=7.5,
    fontfamily="monospace",
    va="bottom", ha="left",
    clip_on=False,
    bbox=dict(
        boxstyle="round,pad=0.45",
        facecolor="white",
        edgecolor="0.35",
        alpha=0.95,
    ),
    zorder=10,
)

# --- Right: 7 timeseries subplots, A at bottom (row 6) → G at top (row 0) ---
for idx, entry in enumerate(point_data):
    row = N_PTS - 1 - idx  # A → row 6, G → row 0
    ax_ts = fig.add_subplot(gs_layout[row, 1])
    color = POINT_COLORS[idx]
    label = POINT_LABELS[idx]
    times = entry["times"]
    gs_da = entry["gs"]
    gs_lowpass = entry["gs_lowpass"]
    trend_fit = entry["trend_fit"]

    # 500-day low-pass (same color, bold)
    ax_ts.plot(
        times,
        gs_lowpass.values,
        color=color,
        linewidth=2.0,
        label="500-day LP",
    )
    # Linear trend (black dashed)
    ax_ts.plot(
        times,
        trend_fit.values,
        color="k",
        linewidth=1.5,
        linestyle="--",
        label="trend",
    )

    # Seasonal background shading
    year_min, year_max = times.year.min(), times.year.max()
    for yr in range(year_min, year_max + 1):
        ax_ts.axvspan(
            pd.Timestamp(yr, 2, 1),
            pd.Timestamp(yr, 5, 31),
            color="yellow",
            alpha=0.15,
            zorder=0,
        )
        ax_ts.axvspan(
            pd.Timestamp(yr, 6, 1),
            pd.Timestamp(yr, 9, 30),
            color="cyan",
            alpha=0.15,
            zorder=0,
        )
        ax_ts.axvspan(
            pd.Timestamp(yr, 10, 1),
            pd.Timestamp(yr, 12, 31),
            color="gray",
            alpha=0.15,
            zorder=0,
        )
        ax_ts.axvline(
            pd.Timestamp(yr, 1, 1),
            color="k",
            linewidth=0.3,
            linestyle="--",
            alpha=0.4,
        )

    # Point label inside left edge of plot
    ax_ts.text(
        0.005, 0.5, label,
        transform=ax_ts.transAxes, fontsize=12,
        fontweight="bold", color=color,
        va="center", ha="left",
    )
    # Tighten y-axis to the filtered (low-pass) series range
    all_vals = gs_lowpass.values[np.isfinite(gs_lowpass.values)]
    ypad = (all_vals.max() - all_vals.min()) * 0.08
    ax_ts.set_ylim(all_vals.min() - ypad, all_vals.max() + ypad)
    ax_ts.set_ylabel("m/s", fontsize=14)
    ax_ts.tick_params(labelsize=12)
    # Only bottom subplot (A, idx=0) shows x-tick labels
    if idx > 0:
        ax_ts.set_xticklabels([])

    # Legend only on the top panel (G, idx=N_PTS-1)
    if idx == N_PTS - 1:
        handles = [
            Line2D([0], [0], color=color, lw=2.0, label="500-day low-pass"),
            Line2D(
                [0],
                [0],
                color="k",
                lw=1.5,
                linestyle="--",
                label="linear trend",
            ),
            Patch(
                facecolor="yellow",
                alpha=0.4,
                edgecolor="none",
                label="Feb–May",
            ),
            Patch(
                facecolor="cyan", alpha=0.4, edgecolor="none", label="Jun–Sep"
            ),
```

```
            Patch(
                facecolor="gray", alpha=0.4, edgecolor="none", label="Oct–Dec"
            ),
        ]
        ax_ts.legend(handles=handles, loc="upper right", fontsize=12, ncol=3)

# fig.suptitle(
#     f"Current speed at EICC coastal intersection points — {sat_val}",
#     fontsize=13,
#     fontweight="bold",
#     y=1.002,
# )
out_png = f"speed_timeseries_allinone_{sat_val}.png"
plt.savefig(out_png, dpi=150, bbox_inches="tight")
plt.show()
plt.close("all")
print(f"Saved: {out_png}")

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from matplotlib.patches import Patch
from matplotlib.lines import Line2D
from tools_xtrackm import *
from tools_shapely import *
from scipy.signal import butter, filtfilt
from scipy.signal import periodogram as scipy_periodogram
from scipy.stats import linregress
# copilot --resume=6b563fd7-e036-45d8-95d0-bd5af130d84f

method = "loess"
cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

sat_val = "TP+J1+J2+J3+S6A"
sat_val = "S3A"
sat_val = "ERS1+ERS2+ENV+SRL"

df = pd.read_csv(f"intersections_near_east_coast_{sat_val}.csv")
df_out = df.copy()

# Optional input: Gaussian smoothing sigma
sigma = 3
# Optional input: clipping bbox tuple
# Default used by the function is (70, 5, 95, 30)
region_bbox_tuple = (70, 5, 95, 30)

frac1 = "0.05"
for i, rows in df_out.iterrows():
    sat1 = rows["sat"]
    track_number_self = str(rows["track_self"])
    track_number_other = str(rows["track_other"])
    lons_inter1 = rows["lons_inter"]
    lats_inter1 = rows["lats_inter"]
    lon1 = lons_inter1
    lat1 = lats_inter1
    query_point = (lon1, lat1)
    result = coastline_gaussian_tangent(
        query_point=query_point,
        sigma=sigma,
        region_bbox_tuple=region_bbox_tuple,
    )
    print("heading_angle_deg:", result["heading_angle_deg"])
    print("tangent_vector:", result["tangent_vector"])
    print("nearest_point:", result["nearest_point"])
    print("distance:", result["distance"])
    coast_heading_deg = result["heading_angle_deg"]
    print(
        sat1,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
    )
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_guv = (
        f"../guv/guv_at_intersections_{method}/"
        f"guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{method}_{frac1}.nc"
    )
    ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
    ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
    ds_guv = xr.open_dataset(f_guv)
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    iself = index_at_lat(ds_self, lat1)
    a1_d = local_angle(
        lons_track_self, lats_track_self, iself, ioff=3, degrees=True
    )
    iothr = index_at_lat(ds_other, lat1)
    a2_d = local_angle(
        lons_track_other, lats_track_other, iothr, ioff=3, degrees=True
    )
    print(f"heading angle {a1_d:.2f} {a2_d:.2f}")
    gu = ds_guv.gu
    gv = ds_guv.gv
    gs = np.sqrt(gu**2 + gv**2)
    times = pd.to_datetime(gs.time.values)
    gs_vals_raw = gs.values.astype(float)

    # 500-day low-pass filter for long-term variability
    dt_days = np.median(
        np.diff(times).astype("timedelta64[s]").astype(float) / 86400.0
    )
    fs = 1.0 / dt_days  # cycles/day
    nyq = fs / 2.0
    cutoff = 1.0 / 500.0  # cycles/day
    b_filt, a_filt = butter(4, cutoff / nyq, btype="low")
    nan_mask = np.isnan(gs_vals_raw)
    gs_interp = pd.Series(gs_vals_raw).interpolate(limit_direction="both").values
    if len(gs_interp) > 3 * max(len(a_filt), len(b_filt)):
        gs_lowpass_vals = filtfilt(b_filt, a_filt, gs_interp)
    else:
        # Fallback if the record is too short for filtfilt padding constraints
        samples_500d = max(3, int(round(500.0 / dt_days)))
        gs_lowpass_vals = (
            pd.Series(gs_interp)
            .rolling(window=samples_500d, center=True, min_periods=1)
            .mean()
            .values
        )
    gs_lowpass_vals[nan_mask] = np.nan
```

```
    gs_lowpass = xr.DataArray(
        gs_lowpass_vals, coords=gs.coords, dims=gs.dims, name="gs_lowpass"
    )

    # Linear trend on low-pass series with standard error and significance
    t_days = (times - times[0]).days.astype(float)
    valid = np.isfinite(gs_lowpass_vals)
    trend_res = linregress(t_days[valid], gs_lowpass_vals[valid])
    fit_vals = trend_res.intercept + trend_res.slope * t_days
    trend_fit = xr.DataArray(fit_vals, coords=gs.coords, dims=gs.dims, name="trend_fit")
    slope_per_year = trend_res.slope * 365.25
    stderr_per_year = trend_res.stderr * 365.25
    significant = trend_res.pvalue < 0.05

    fig = plt.figure(figsize=(12, 10))
    ax0 = fig.add_subplot(2, 1, 2)
    gs.plot(ax=ax0, label="current speed", color="lightcoral", linewidth=1.2, alpha=0.7)
    gs_lowpass.plot(ax=ax0, label="500-day low-pass", color="r", linewidth=2.0)
    trend_fit.plot(ax=ax0, label="linear trend fit", color="k", linewidth=2.0, linestyle="--")
    ax0.set_ylabel("current speed m/s")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
    plt.text(
        0.15,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )

    # --- Seasonal shading and year markers on (ax0) ---
    year_min, year_max = times.year.min(), times.year.max()
    for yr in range(year_min, year_max + 1):
        # Feb-May: yellow
        ax0.axvspan(
            pd.Timestamp(yr, 2, 1),
            pd.Timestamp(yr, 5, 31),
            color="yellow",
            alpha=0.15,
            zorder=0,
        )
        # Jun-Sep: cyan
        ax0.axvspan(
            pd.Timestamp(yr, 6, 1),
            pd.Timestamp(yr, 9, 30),
            color="cyan",
            alpha=0.15,
            zorder=0,
        )
        # Oct-Dec: gray
        ax0.axvspan(
            pd.Timestamp(yr, 10, 1),
            pd.Timestamp(yr, 12, 31),
            color="gray",
            alpha=0.15,
            zorder=0,
        )
        # Year boundary vertical line on Jan 1
        ax0.axvline(
            pd.Timestamp(yr, 1, 1),
            color="k",
            linewidth=0.5,
            linestyle="--",
            alpha=0.5,
        )

    sig_txt = "significant (p<0.05)" if significant else "not significant (p>=0.05)"
    trend_txt = (
        f"Trend={slope_per_year:.4f} m/s/yr ± {stderr_per_year:.4f} (SE), "
        f"p={trend_res.pvalue:.3g}, {sig_txt}"
    )
    ax0.text(
        0.02,
        0.03,
        trend_txt,
        transform=ax0.transAxes,
        fontsize=11,
        bbox=dict(boxstyle="round", facecolor="white", alpha=0.8),
    )

    seasonal_handles = [
        Patch(facecolor="yellow", alpha=0.15, edgecolor="none", label="Feb-May"),
        Patch(facecolor="cyan", alpha=0.15, edgecolor="none", label="Jun-Sep"),
        Patch(facecolor="gray", alpha=0.15, edgecolor="none", label="Oct-Dec"),
    ]
    line_handles = [
        Line2D([0], [0], color="lightcoral", lw=1.2, label="current speed"),
        Line2D([0], [0], color="r", lw=2.0, label="500-day low-pass"),
        Line2D([0], [0], color="k", lw=2.0, linestyle="--", label="linear trend fit"),
    ]
    ax0.legend(handles=line_handles + seasonal_handles, loc="upper right")

    ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
    rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
        ETOPO05_Y=slice(*TRACKS_REG[2:])
    )
    rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor="red",
        markeredgecolor="black",
        markeredgewidth=2,
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
```

```
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(
        lon_equat_self, lat_equat_self, lon_coast_self, lat_coast_self, d
    )
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=a1_d,
            color="k",
        )
    lonm, latm = get_point_at_distance(
        lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d
    )
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=a2_d,
            color="k",
        )
    plt.savefig(
        f"speed_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    plt.show()
    plt.close("all")

```

```
1
2  to add
3  1. 200, 1000, 2000 isobaths on left cartopy map as dashed contours with labels
4  2. discription of points A to G on the top space of left cartopy map
5  With coordinates of intersection, the track numbers corresponding to the intersection points; all enclosed in box
6
7  Write one huge function that combines all functions in the script
8  c.isobath_based_slopes.py
9  which return heading angle (angle in counter-clockwise from positive x axis east). It can take following arguments
10 topo ds to use
11 isobath level
12 smoothing window width
13 sampling step
14 query point
15 implement this in a new script c.isobath_based_slopes.one_func.py
16
17 modify the script
18 p.speed_at_coastal_points.eicc.trends.all_in_one.ERS.clim.py
19 to
20 1. plot monthly climatology of gs on the right viewports instead of long filtered timeseries
21 2. No need of trend plots, slope annotations on the left etc
22 3. Seasonal shading to be intact.
23
24 update the script
25 p.speed_at_coastal_points.eicc.trends.all_in_one.py
26 to
27 1. Ignore the first intersection point (lowest latitude) of the .csv file
28 2. There will be seven intersection points on the coast. Make seven rows of subplots on the right side to plot each point's timeseries, filtered timeseries, linear fit, other info
29 3. On the left side, plot one big vertically elongated cartopy plot with all seven points with boundaries (10N-22N and 78E-88E).
30 The way it is done in
31 c.eicc_path.as_transect.py
32 4. If a perticular intersection point's color is blue, its corresponding timeseries point on the right should be blue. Label each point from A (lowest lat) to G.
33 Three things:
34 1. Annotate slope values on the left plot. (don't print p value)
35 If the slope is insigficant, gray it out, we will explain in the caption
36 2. Remove trend annotations on the trend plots, tighten up ylimits so that whole space is occupied by plot
37 3. Remove information about the points and their intersection on top of horizonal subplot. There should not be any vertical space between the right side subplots. Also
       the A plot should be bottom and G plot should be at the top.
38
39 In the script p.speed_at_coastal_points.eicc.trends.py,
40 1. smooth each speed (gs) through a 500 day lowpass filter, plot it over
41 2. estimate linear trend and std error and plot the fit
42 3. determine whether the linear trend is significant or not
43 4. add legend for seasonal shading done
44
45 I have 30 years of along shore velocity data from several intersection points of (TP+J1+J2+J3+S6A mission) which are close to east coast (within 50km) of India at 10da
     y resolution. What analysis can I done on these time series to get more information about EICC which are not discussed earlier in the studies
46
47 1. Read intersections_near_coast_BOB_ERS1+ERS2+ENV+SRL.csv
48 2. For each intersection point read corresponding guv files using track_self and track_other
49 see p.along_shore_at_155_116.intersection.S6A_c1.py
50 3. append u variables of guv files
51 4. Form a transect dimension with selected intersection points sorted by latitude.
52 see p.intersections_near_coast_bob.py once
53 5. concat arrray of u dataarrays with above dimension
54 6. Make a hovmullor diagram with latitude as y axis and time as x axis
55
56 Add a left side subplot and
57 Plot a cartopy plot, selected to the region BOB (Bay of Bengal)
58 plot the selected intersection points along the coast along with lon, lat pairs as annotations
59 connect these points with lines to show the transect along the coast.
60 Currently the hovmoller plot looks blank.
61
62 Similar to the workflow of p.scatter_drifter_ve_vs_gu_ww_ek.py
63 1. Read tracks_intersections_TP+J1+J2+J3+S6A_1.csv, get nearest distances to the coast for each intersection point.
64 2. Gather points that are within a threshold distance, save them to a list
65 3. Plot a cartopy plot, selected to the region BOB (Bay of Bengal)
66 4. plot the listed points along the coast along with lon, lat pairs as annotations
67
68 We need to furthur select the points
69 On selected points discard points with lat < 9N or lon > 90E
70 sort based on latitude
71 if more than one point is on the same latitude (same with 2decimal points),
72  select the point that is close to the half of threshold distance
73
74  points > 90E are still present,
75
76 extend the script p.along_shore_at_155_116.intersection.S6A_periodogram.py to
77 shade periods of Feb-May in Yellow, Jun-Sep in cyan and Oct-Dec in transparent gray for all years. Mark years with vertical bars.
78 Include periodogram of along shore current with periodicities in days
79 inplace of cross current plot (bottom subplot)
80 Mark important periodicities like 365, 180, 120 etc.
81
82 Write a python script to plot climatological cross track gc as arrows along
83 the plotted tracks on a cartopy map set to a Bay Of Bengal region. Use
84 pre computed gc fields at
85 ../guv/guv_at_intersections_loess/gc_at_ERS1+ERS2+ENV+SRL_*_loess_0.05.nc
86 1. Plot a cartopy map for Bay of Bengal region.
87 WBOB2 = (78.0, 97.0, 0.0, 25.0)
88 2. for the tracknumber open sla file at ../data/ERS1+ERS2+ENV+SRL
89 3. Check if lat[-1] and lon[-1] fall in the the region and slope is negative
90 4. Plot tracks
91 5. Compute the monthly climatology of gc
92 6. The length of arrows on the tracks should be proportional to the magnitude
93 of gc at the point (Mar on left subplot) and Nov on right subplot.
94 also see general scripts at
95 ../meta
96 ../lib/tools_xtrackm.py
97 use xarray for all operations.
98
99
100 Write an all-encompassing function that take only the following arguments:
101 query points
102 smoothing sigma for guassion
103 following default box for cliping
104 lon_min, lon_max = 70, 95
105 lat_min, lat_max = 5, 30
106 region_bbox = box(lon_min, lat_min, lon_max, lat_max)
107 ability to add box cordinates in a tuple
108 outputs required are
109 heading angle in degrees
110 tangent_vector
111 nearest_point
112 distance
113
114 Ideally It should be used by a person not familiar with shapely library and coardinate systems. @p.smoothed_coastline.gaussian.tangent.py @shapely_smooth_funcs.py
115
```

```
116  If we give gu, gv components and the heading angle (an acute positve angle in degrees) of the coast at a point, write a funtion to find the along-shore and across-shore components.
117
118  In the script p.smoothed_coastline.others.py,
119  given gaussian smoothed coastline and a point near the coastline, write a function to find the slope of tangent of the gaussian curve near to the point.
120
```

```
1
2  p.smoothed_coastline.gaussian.py
3  p.smoothed_coastline.gaussian.tangent.py
4  p.smoothed_coastline.others.py
5  p.coastline.gaussian.tangent.simple.py
6
7  2. Isobaths are extracted for a given depth, smoothed and slope is found
8  c.isobath_based_slopes.py
9
10 3. Select points of SAT that are close to coast at distance of about 50km to trace the EICC path.
11 c.eicc_path.as_transect.py
12
13 4. Append the gu timeseries along the selected points (on eastcoast) as a trasect dimension and plot hovmoller diagram.
14 p.hovmoller_u_near_coast_bob.py
15
16 5. plots the along shore and cross shore components at a give intersection point.
17 p.along_shore_at_155_116.intersection.S6A_c1.py
18
19 6. plots periodogram of alongshore currents
20 p.along_shore_at_155_116.intersection.S6A_periodogram.py
21
```

```
./eicc/archive/
```

```
1 file(s)
```

```python
from pathlib import Path
import sys

import cartopy.crs as ccrs
import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
from geopy import distance

# copilot --resume=62ff5540-5ab6-44c0-b895-489d72a8c8cb

LIB_DIR = Path(__file__).resolve().parents[1] / "lib"
if str(LIB_DIR) not in sys.path:
    sys.path.insert(0, str(LIB_DIR))

from tools_xtrackm import decorate_axis, is_within_region

SAT = "ERS1+ERS2+ENV+SRL"
WBOB2 = (78.0, 97.0, 0.0, 25.0)
GC_GLOB = "../guv/guv_at_intersections_loess/gc_at_ERS1+ERS2+ENV+SRL_*_loess_0.05.nc"
SLA_TEMPLATE = "../data/ERS1+ERS2+ENV+SRL/ctoh.sla.ref.ERS1+ERS2+ENV+SRL.nindian.{track:03d}.nc"
MONTHS_TO_PLOT = ((3, "Mar"), (11, "Nov"))
ARROW_POINT_STEP = 4
ARROW_METERS_PER_MS = 150_000.0
TRACK_COLOR = "k"
ARROW_COLOR = "crimson"


def track_number_from_gc_path(path):
    return int(Path(path).stem.split("_")[-3])


def build_track_geometry(ds_sla):
    lon_rev = ds_sla["lon"].load().isel(points_numbers=slice(None, None, -1))
    lat_rev = ds_sla["lat"].load().isel(points_numbers=slice(None, None, -1))

    segment_lengths = [0.0]
    lon_vals = lon_rev.values
    lat_vals = lat_rev.values
    for idx in range(1, lon_rev.sizes["points_numbers"]):
        seg_len = distance.distance(
            (lat_vals[idx - 1], lon_vals[idx - 1]),
            (lat_vals[idx], lon_vals[idx]),
        ).m
        segment_lengths.append(seg_len)

    x_vals = np.cumsum(segment_lengths)
    geom = xr.Dataset(
        coords={"x": ("x", x_vals)},
        data_vars={
            "lon": ("x", lon_vals),
            "lat": ("x", lat_vals),
        },
    )
    return geom


def build_track_normals(track_geom):
    meters_per_degree_lat = xr.full_like(track_geom["lat"], 111_320.0)
    meters_per_degree_lon = 111_320.0 * np.cos(np.deg2rad(track_geom["lat"]))

    tangent_east = track_geom["lon"].differentiate("x") * meters_per_degree_lon
    tangent_north = track_geom["lat"].differentiate("x") * meters_per_degree_lat
    tangent_mag = np.hypot(tangent_east, tangent_north)

    normals = xr.Dataset(
        data_vars={
            "east": -tangent_north / tangent_mag,
            "north": tangent_east / tangent_mag,
        },
        coords={"x": track_geom["x"]},
    )
    return normals


def track_matches_filter(ds_sla):
    lon_track = ds_sla["lon"].values
    lat_track = ds_sla["lat"].values
    if lon_track.size == 0:
        return False
    lon_coast = float(lon_track[-1])
    lat_coast = float(lat_track[-1])
    if not is_within_region(lon_coast, lat_coast, *WBOB2):
        return False
    delta_lon = float(lon_track[-1] - lon_track[0])
    if np.isclose(delta_lon, 0.0):
        return False
    slope = float((lat_track[-1] - lat_track[0]) / delta_lon)
    return slope < 0.0


def build_plot_dataset(ds_sla, ds_gc, month_number):
    track_geom = build_track_geometry(ds_sla)
    track_normals = build_track_normals(track_geom)
    gc_clim = ds_gc["gc"].groupby("time.month").mean("time")
    gc_month = gc_clim.sel(month=month_number).interp(x=track_geom["x"])

    plot_ds = xr.Dataset(
        data_vars={
            "lon": track_geom["lon"],
            "lat": track_geom["lat"],
            "normal_east": track_normals["east"],
            "normal_north": track_normals["north"],
            "gc": gc_month,
        },
        coords={"x": track_geom["x"]},
    )
    plot_ds = plot_ds.where(np.isfinite(plot_ds["gc"]), drop=True)
    plot_ds = plot_ds.isel(x=slice(0, None, ARROW_POINT_STEP))
    return plot_ds


def add_track_and_arrows(ax, ds_sla, ds_gc, month_number):
    track_geom = build_track_geometry(ds_sla)
    plot_ds = build_plot_dataset(ds_sla, ds_gc, month_number)

    ax.plot(
```

```
        track_geom["lon"].values,
        track_geom["lat"].values,
        color=TRACK_COLOR,
        linewidth=0.8,
        transform=ccrs.PlateCarree(),
        zorder=2,
    )

    meters_per_degree_lon = 111_320.0 * np.cos(np.deg2rad(plot_ds["lat"]))
    arrow_length_m = plot_ds["gc"] * ARROW_METERS_PER_MS
    u = plot_ds["normal_east"] * arrow_length_m / meters_per_degree_lon
    v = plot_ds["normal_north"] * arrow_length_m / 111_320.0

    ax.quiver(
        plot_ds["lon"].values,
        plot_ds["lat"].values,
        u.values,
        v.values,
        angles="xy",
        scale_units="xy",
        scale=1.0,
        width=0.0018,
        headwidth=3.5,
        headlength=4.5,
        headaxislength=4.0,
        pivot="middle",
        color=ARROW_COLOR,
        transform=ccrs.PlateCarree(),
        zorder=3,
    )


def decorate_map_axis(ax, title):
    decorate_axis(ax, title, *WBOB2, step=5)
    gridlines = ax.gridlines(
        draw_labels=True,
        linewidth=0.5,
        color="gray",
        alpha=0.7,
        linestyle="--",
    )
    gridlines.top_labels = False
    gridlines.right_labels = False
    return gridlines


def main():
    fig, axes = plt.subplots(
        1,
        2,
        figsize=(14, 8),
        subplot_kw={"projection": ccrs.PlateCarree()},
        layout="constrained",
    )

    gc_paths = sorted(Path(".").glob(GC_GLOB))
    plotted_tracks = 0

    for ax, (month_number, month_label) in zip(axes, MONTHS_TO_PLOT):
        decorate_map_axis(ax, f"{SAT} {month_label}")

        for gc_path in gc_paths:
            track_number = track_number_from_gc_path(gc_path)
            sla_path = Path(SLA_TEMPLATE.format(track=track_number))
            if not sla_path.exists():
                continue

            with xr.open_dataset(sla_path, decode_times=False) as ds_sla:
                if not track_matches_filter(ds_sla):
                    continue
                with xr.open_dataset(gc_path) as ds_gc:
                    add_track_and_arrows(ax, ds_sla, ds_gc, month_number)
                    if month_number == MONTHS_TO_PLOT[0][0]:
                        plotted_tracks += 1

    axes[1].gridlines(draw_labels=False)
    output_path = Path(__file__).with_suffix(".png")
    fig.savefig(output_path, dpi=300, bbox_inches="tight")
    print(f"saved {output_path}")
    print(f"tracks plotted: {plotted_tracks}")
    plt.close(fig)


if __name__ == "__main__":
    main()
```

./ekman/

9 file(s)

```
1
2
```

```python
import xarray as xr
import matplotlib.pyplot as plt
import numpy as np

dsw = xr.open_dataset("/home/srinivasu/allData/ncep2/wspd.10m.mon.mean.nc")
print(dsw)
# speed1 = dsw.wspd.sel(time=slice("01-01-1992", "2024-01-01"))
speed1m = dsw.wspd.mean(dim="time")
speed2m = speed1m.sel(lon=slice(30, 120))
speed3m = speed2m.sel(lat=slice(20, -20))

ds = xr.open_dataset(
    f"/home/srinivasu/allData/ccmp3.0_lt/2003_CCMP_Wind_Analysis_io_daily.nc",
)
print(ds)
ds = ds.rename({"longitude": "lon", "latitude": "lat"})
print(ds)
uwnd1 = ds.uwnd
vwnd1 = ds.vwnd
speed1 = (uwnd1**2.0 + vwnd1**2.0) ** 0.5

# def compute_drag_coefficient(U10):
#     """
#     Compute the drag coefficient CD based on the wind speed U10 using the given piecewise formula.
#
#     Parameters:
#     U10 (xarray.DataArray): 2D DataArray of wind speed at 10 meters (m/s)
#
#     Returns:
#     xarray.DataArray: 2D DataArray of drag coefficient CD
#     """
#     # Initialize CD array with the same shape as U10
#     CD = xr.full_like(U10, np.nan)
#
#     # Apply the first formula for 3 <= U10 <= 6 m/s
#     mask_3_to_6 = (U10 >= 3) & (U10 <= 6)
#     CD = 0.001 * CD.where(~mask_3_to_6, 0.29 + (3.1 / U10) + (7.7 / U10**2))
#
#     # Apply the second formula for 6 < U10 <= 26 m/s
#     mask_6_to_26 = (U10 > 6) & (U10 <= 26)
#     CD = 0.001 * CD.where(~mask_6_to_26, 0.60 + (0.071 * U10))
#
#     return CD

def compute_drag_coefficient_ekmanlib(w):
    """
    https://github.com/Konfersi-Indonesia/ekmanlib/blob/main/ekmanlib/ekmanlib.py
    Computes non linear drag coefficient from Trenberth et al. (1990) as a function of wind speed
    Formula and a non-linear drag coefficient (cd) based on
    Large and Pond (1981)[2], modified for low wind speeds (Trenberth et
    al., 1990)[3]

    [1] A.E. Gill, 1982, Atmosphere-Ocean Dynamics, Academy Press.
    [2] W.G. Large & S. Pond., 1981,Open Ocean Measurements in Moderate
    to Strong Winds, J. Physical Oceanography, v11, p324-336.
    [3] K.E. Trenberth, W.G. Large & J.G. Olson, 1990, The Mean Annual
    Cycle in Global Ocean Wind Stress, J. Physical Oceanography, v20,
    p1742-1760.
    """
     C_N = xr.full_like(w, np.nan)
     C_N = xr.where(w > 10, 0.49 + 0.065 * w, C_N)
     C_N = xr.where((w > 3) & (w <= 10), 1.14, C_N)
     C_N = xr.where((w > 1) & (w <= 3), 0.62 + (1.56/w), C_N)
     C_N = xr.where(w <= 1, 2.18e-3, C_N)
     C_N = 0.001 * C_N
     return C_N

# def compute_drag_coefficient_ekmanlib(w):
#     """Convert wind speed (u,v) to wind stress (Tx,Ty).
#
#     It uses either wind-dependent or constant drag.
#
#     Args:
#         u, v: Wind vector components (m/s), 2d or 3d (for time series).
#         rho_air: Density of air (1.22 kg/m^3).
#         cd: Non-dimensional drag (wind-speed dependent).
#             For constant drag use cd=1.5e-3.
#     Notes:
#         Function to compute wind stress from wind field data is based on Gill,
#         (1982)[1]. Formula and a non-linear drag coefficient (cd) based on
#         Large and Pond (1981)[2], modified for low wind speeds (Trenberth et
#         al., 1990)[3]
#
#         [1] A.E. Gill, 1982, Atmosphere-Ocean Dynamics, Academy Press.
#         [2] W.G. Large & S. Pond., 1981,Open Ocean Measurements in Moderate
#         to Strong Winds, J. Physical Oceanography, v11, p324-336.
#         [3] K.E. Trenberth, W.G. Large & J.G. Olson, 1990, The Mean Annual
#         Cycle in Global Ocean Wind Stress, J. Physical Oceanography, v20,
#         p1742-1760.
#     """
#     # wind-dependent drag
#     cond1 = (w<=1)
#     cond2 = (w>1) & (w<=3)
#     cond3 = (w>3) & (w<10)
#     cond4 = (w>=10)
#     cd = np.zeros_like(w)
#     cd[cond1] = 2.18e-3
#     cd[cond2] = (0.62 + 1.56/w[cond2]) * 1e-3
#     cd[cond3] = 1.14e-3
#     cd[cond4] = (0.49 + 0.065*w[cond4]) * 1e-3
#     return cd

# CD = compute_drag_coefficient(speed1)
fig, ax = plt.subplots(figsize=(12, 8))
CD = compute_drag_coefficient_ekmanlib(speed1)
print(CD)
# CD.isel(time=1).plot(vmax=0.002, cmap="tab20c")
# CD.mean(dim="time").plot(vmin=0.00111, vmax=0.00116, cmap="tab20c", ax=ax)
CD.mean(dim="time").plot(cmap="tab20c", ax=ax)
CS = speed1.mean(dim="time").plot.contour(ax=ax, colors="k")
ax.clabel(CS, inline=1, fontsize=10)
plt.show()

```

```python
import numpy as np
import xarray as xr

dsu = xr.open_dataset(f"ccmp_v31_daily_iod_u_mon_clim.nc")
dsv = xr.open_dataset(f"ccmp_v31_daily_iod_v_mon_clim.nc")


def calculate_ekman_velocities(dsu, dsv, z, A=0.1, cd=1.5e-3, rho_air=1.225, rho_water=1025.0):
    """
    Calculates the upper ocean Ekman velocities from CCMP 10m wind fields.
    Parameters:
    ds : xarray.Dataset
        Dataset containing CCMP variables 'uwnd', 'vwnd', and 'latitude'.
    z_levels : array-like
        Depths (in meters, should be negative as z=0 is the surface and z -> -inf).
    A : float
        Eddy viscosity in m^2/s. Default is 0.1.
    cd : float
        Drag coefficient for bulk wind stress formula.
    rho_air : float
        Air density in kg/m^3.
    rho_water : float
        Ocean reference density in kg/m^3.
    Returns:
    xarray.Dataset
        Dataset containing the Ekman velocity components 'u_ekman' and 'v_ekman'.
    """
     # 1. Extract 10m wind components from CCMP data
     u10 = dsu['uwnd']
     v10 = dsv['vwnd']
     lat = dsu['latitude']
     # 2. Calculate kinematic wind stress (tau_tilde)
     # NOTE: The bulk aerodynamic formula to convert 10m winds to surface stress
     # uses information from outside the provided sources and should be independently verified.
     wind_speed = np.sqrt(u10**2 + v10**2)
     tau_x = rho_air * cd * wind_speed * u10
     tau_y = rho_air * cd * wind_speed * v10
     # Kinematic wind stress is stress divided by reference density
     tau_tilde_x = tau_x / rho_water
     tau_tilde_y = tau_y / rho_water
     # 3. Calculate Coriolis parameter (f) and mask the equator to avoid division by zero
     omega = 7.2921e-5 # Earth's rotation rate in rad/s
     f = 2 * omega * np.sin(np.deg2rad(lat))
     f = f.where(np.abs(lat) > 2.0) # Masking out 2 degrees near the equator
     # 4. Calculate Ekman layer depth (d)
     # The absolute value of f is used to ensure the depth is real and positive.
     d = np.sqrt(2 * A / np.abs(f))
     # 5. Calculate Ekman velocities for each z level
     decay = np.exp(z / d)
     phase_x = np.cos(z / d - np.pi / 4)
     phase_y = np.sin(z / d - np.pi / 4)
         # Ekman velocity equations
     u_E = (np.sqrt(2) / (f * d)) * decay * (tau_tilde_x * phase_x - tau_tilde_y * phase_y)
     v_E = (np.sqrt(2) / (f * d)) * decay * (tau_tilde_x * phase_y + tau_tilde_y * phase_x)
     return xr.Dataset({'u_ekman': u_E, 'v_ekman': v_E})

z = -15

ekman_velocities = calculate_ekman_velocities(dsu, dsv, z)
ekman_velocities.to_netcdf("ekman_velocities_15m_clim.nc", format="NETCDF4")
# u_ekman1 = ekman_velocities.u_ekman.isel(month=7)
# u_ekman1.plot()
# ekman_velocities.to_netcdf("ekman_velocities.nc")

```

```python
import pandas as pd
import xarray as xr
from pathlib import Path
from namesm import sats_new
import matplotlib.pyplot as plt

ekman_dir = "/home/srinivasu/allData/ccmp31/"
out_dir = Path("/home/srinivasu/xtrackm/ekman/ekman_at_intersections/")
out_dir.mkdir(parents=True, exist_ok=True)

df_all_list = [pd.read_csv(f"tracks_intersections_{sat}_1.csv") for sat in sats_new]
inter_df = pd.concat(df_all_list, ignore_index=True)

for z_in in ["0", "10", "15"]:
    ds_ekman = xr.open_dataset(
        f"{ekman_dir}/ekman_velocities_{z_in}m_all_years_daily.nc",
        chunks={"time": 365, "lat": 50, "lon": 50}
    )
    u_ekman = ds_ekman["u_ekman"]
    v_ekman = ds_ekman["v_ekman"]
    for i, r in inter_df.iterrows():
        sat = r["sat"]
        track_number_self = str(r["track_self"])
        track_number_other = str(r["track_other"])
        lons_inter1 = r["lons_inter"]
        lats_inter1 = r["lats_inter"]
        unique_str = f"{sat}_{track_number_self}_{track_number_other}"
        print(unique_str, z_in)
        u_at = u_ekman.sel(
            latitude=lats_inter1, longitude=lons_inter1, method="nearest"
        )
        v_at = v_ekman.sel(
            latitude=lats_inter1, longitude=lons_inter1, method="nearest"
        )
        # u_at.plot()
        # plt.show()
        ds_out = xr.Dataset(
            {
                "u_at": u_at,
                "v_at": v_at,
            }
        )
        ds_out.to_netcdf(
            f"{out_dir}/{unique_str}_ekman_{z_in}m_at_intersections.nc", format="NETCDF4"
        )

```

```python
import pandas as pd
import xarray as xr
from pathlib import Path
from namesm import sats_new
import matplotlib.pyplot as plt

ekman_dir = "/home/srinivasu/allData/ccmp31/"
out_dir = Path("/home/srinivasu/xtrackm/ekman/ekman_at_intersections_cd/")
out_dir.mkdir(parents=True, exist_ok=True)

df_all_list = [pd.read_csv(f"tracks_intersections_{sat}_1.csv") for sat in sats_new]
inter_df = pd.concat(df_all_list, ignore_index=True)

for z_in in ["0", "10", "15"]:
    ds_ekman = xr.open_dataset(
        f"{ekman_dir}/ekman_velocities_{z_in}m_all_years_daily_cd.nc",
        chunks={"time": 365, "lat": 50, "lon": 50}
    )
    u_ekman = ds_ekman["u_ekman"]
    v_ekman = ds_ekman["v_ekman"]
    for i, r in inter_df.iterrows():
        sat = r["sat"]
        track_number_self = str(r["track_self"])
        track_number_other = str(r["track_other"])
        lons_inter1 = r["lons_inter"]
        lats_inter1 = r["lats_inter"]
        unique_str = f"{sat}_{track_number_self}_{track_number_other}"
        print(unique_str, z_in)
        u_at = u_ekman.sel(
            latitude=lats_inter1, longitude=lons_inter1, method="nearest"
        )
        v_at = v_ekman.sel(
            latitude=lats_inter1, longitude=lons_inter1, method="nearest"
        )
        # u_at.plot()
        # plt.show()
        ds_out = xr.Dataset(
            {
                "u_at": u_at,
                "v_at": v_at,
            }
        )
        ds_out.to_netcdf(
            f"{out_dir}/{unique_str}_ekman_{z_in}m_at_intersections_cd.nc", format="NETCDF4"
        )

```

```python
import matplotlib.pyplot as plt
import numpy as np
import xarray as xr

import cartopy.crs as ccrs
import cartopy.feature as cfeature

dsu = xr.open_dataset(f"ccmp_v31_daily_iod_u_mon_clim.nc")
dsv = xr.open_dataset(f"ccmp_v31_daily_iod_v_mon_clim.nc")


def calculate_ekman_velocities(dsu, dsv, z, A=0.1, cd=1.5e-3, rho_air=1.225, rho_water=1025.0):
    """
    Calculates the upper ocean Ekman velocities from CCMP 10m wind fields.
    Parameters:
    ds : xarray.Dataset
        Dataset containing CCMP variables 'uwnd', 'vwnd', and 'latitude'.
    z_levels : array-like
        Depths (in meters, should be negative as z=0 is the surface and z -> -inf).
    A : float
        Eddy viscosity in m^2/s. Default is 0.1.
    cd : float
        Drag coefficient for bulk wind stress formula.
    rho_air : float
        Air density in kg/m^3.
    rho_water : float
        Ocean reference density in kg/m^3.
    Returns:
    xarray.Dataset
        Dataset containing the Ekman velocity components 'u_ekman' and 'v_ekman'.
    """
    # 1. Extract 10m wind components from CCMP data
    u10 = dsu['uwnd']
    v10 = dsv['vwnd']
    lat = dsu['latitude']
    # 2. Calculate kinematic wind stress (tau_tilde)
    # NOTE: The bulk aerodynamic formula to convert 10m winds to surface stress
    # uses information from outside the provided sources and should be independently verified.
    wind_speed = np.sqrt(u10**2 + v10**2)
    tau_x = rho_air * cd * wind_speed * u10
    tau_y = rho_air * cd * wind_speed * v10
    # Kinematic wind stress is stress divided by reference density
    tau_tilde_x = tau_x / rho_water
    tau_tilde_y = tau_y / rho_water
    # 3. Calculate Coriolis parameter (f) and mask the equator to avoid division by zero
    omega = 7.2921e-5 # Earth's rotation rate in rad/s
    f = 2 * omega * np.sin(np.deg2rad(lat))
    f = f.where(np.abs(lat) > 2.0) # Masking out 2 degrees near the equator
    # 4. Calculate Ekman layer depth (d)
    # The absolute value of f is used to ensure the depth is real and positive.
    d = np.sqrt(2 * A / np.abs(f))
    # 5. Calculate Ekman velocities for each z level
    decay = np.exp(z / d)
    phase_x = np.cos(z / d - np.pi / 4)
    phase_y = np.sin(z / d - np.pi / 4)
        # Ekman velocity equations
    u_E = (np.sqrt(2) / (f * d)) * decay * (tau_tilde_x * phase_x - tau_tilde_y * phase_y)
    v_E = (np.sqrt(2) / (f * d)) * decay * (tau_tilde_x * phase_y + tau_tilde_y * phase_x)
    return xr.Dataset({'u_ekman': u_E, 'v_ekman': v_E})

z = -15
# z = 0
# z = -30

ekman_velocities = calculate_ekman_velocities(dsu, dsv, z)
z_out = str(abs(z))
f_out = f"ekman_velocities_{z_out}m_clim.nc"
ekman_velocities.to_netcdf(f_out, format="NETCDF4")

INFILE = f_out
OUTFIG = (
    f"/home/srinivasu/allData/ccmp31/p.u_E.{z_out}m_monsoon_panel.png"
)
VAR = "u_ekman"
# Seasonal definitions for Indian Ocean monsoon cycle.
SEASONS = {
    "Summer Monsoon (JJAS)": [6, 7, 8, 9],
    "Winter Monsoon (DJF)": [12, 1, 2],
    "Intermonsoon-1 (MAM)": [3, 4, 5],
    "Intermonsoon-2 (ON)": [10, 11],
}

MONTH_DAYS = {
    1: 31,
    2: 28,
    3: 31,
    4: 30,
    5: 31,
    6: 30,
    7: 31,
    8: 31,
    9: 30,
    10: 31,
    11: 30,
    12: 31,
}


def weighted_season_mean(da_mon_clim: xr.DataArray, months: list[int]) -> xr.DataArray:
    subset = da_mon_clim.sel(month=months)
    weights = xr.DataArray(
        [MONTH_DAYS[m] for m in months],
        coords={"month": months},
        dims=["month"],
    )
    return (subset * weights).sum("month") / weights.sum()


def main() -> None:
    ds = xr.open_dataset(INFILE)
    if VAR not in ds:
        raise KeyError(f"Variable '{VAR}' not found in {INFILE}")
    da = ds[VAR]
    if "month" not in da.dims:
        raise ValueError("Input file must be monthly climatology with 'month' dimension")
    seasonal = {
        name: weighted_season_mean(da, months) for name, months in SEASONS.items()
```

```
    }
    all_vals = np.concatenate([seasonal[s].values.ravel() for s in seasonal])
    finite = all_vals[np.isfinite(all_vals)]
    vmax = np.nanpercentile(np.abs(finite), 98) if finite.size else 0.2
    if vmax <= 0:
        vmax = 0.2
    levels = np.linspace(-vmax, vmax, 21)
    proj = ccrs.PlateCarree()
    fig, axes = plt.subplots(
        2,
        2,
        figsize=(11, 7.5),
        subplot_kw={"projection": proj},
        sharex=True,
        sharey=True,
    )
    fig.subplots_adjust(left=0.01, right=0.98, top=0.98, bottom=0.07, wspace=0.01, hspace=0.01)
    mappable = None
    for ax, title in zip(axes.ravel(), SEASONS.keys()):
        fld = seasonal[title]
        mappable = ax.contourf(
            fld["longitude"],
            fld["latitude"],
            fld,
            levels=levels,
            cmap="RdBu_r",
            extend="both",
            transform=proj,
        )
        ax.coastlines(linewidth=0.8)
        ax.add_feature(cfeature.LAND, facecolor="0.9", zorder=10)
        ax.set_title(title)
        gl = ax.gridlines(draw_labels=True, linewidth=0.3, alpha=0.5)
        gl.top_labels = False
        gl.right_labels = False
    cbar = fig.colorbar(
        mappable,
        ax=axes.ravel().tolist(),
        orientation="horizontal",
        pad=0.04,
        fraction=0.045,
        shrink=0.82,
        aspect=35,
    )
    cbar.set_label("u_E (m s$^{-1}$)")
    # fig.suptitle("OSCAR - MSLA Ageostrophic U: Seasonal Means", y=0.98)
    fig.savefig(OUTFIG, dpi=200, bbox_inches="tight")
    plt.show()
    print(f"Wrote figure: {OUTFIG}")

if __name__ == "__main__":
    main()

```

```python
import pandas as pd
import xarray as xr
from pathlib import Path
from namesm import sats_new
import matplotlib.pyplot as plt

ekman_dir = "/home/srinivasu/allData/ccmp31/"
out_dir = Path("/home/srinivasu/xtrackm/ekman/ekman_at_intersections/")
out_dir_cd = Path("/home/srinivasu/xtrackm/ekman/ekman_at_intersections_cd/")

SAT = "TP+J1+J2+J3+S6A"
SAT = "S3A"
DEPTH = "0"
inter_df = pd.read_csv(f"tracks_intersections_{SAT}_1.csv")

for i, r in inter_df.iterrows():
    sat = r["sat"]
    track_number_self = str(r["track_self"])
    track_number_other = str(r["track_other"])
    lons_inter1 = r["lons_inter"]
    lats_inter1 = r["lats_inter"]
    unique_str = f"{sat}_{track_number_self}_{track_number_other}"
    f_ek = f"{out_dir}/{unique_str}_ekman_{DEPTH}m_at_intersections.nc"
    f_ek_cd = f"{out_dir_cd}/{unique_str}_ekman_{DEPTH}m_at_intersections_cd.nc"
    ds_ek = xr.open_dataset(f_ek)
    print(ds_ek)
    ds_ek_cd = xr.open_dataset(f_ek_cd)
    u_ek = ds_ek["u_at"].resample(time="ME").mean()
    u_ek_cd = ds_ek_cd["u_at"].resample(time="ME").mean()
    u_ek.plot(color="b", label="ekman")
    u_ek_cd.plot(color="r", label="ekman cd")
    plt.legend()
    plt.show()


```

ekman/p.u_E.monsoon_panel_clim.py

```python
import matplotlib.pyplot as plt
import numpy as np
import xarray as xr

import cartopy.crs as ccrs
import cartopy.feature as cfeature

INFILE = "/home/srinivasu/allData/ccmp31/ekman_velocities_15m_clim.nc"
OUTFIG = (
    "/home/srinivasu/allData/ccmp31/p.u_E.monsoon_panel.png"
)
VAR = "u_ekman"
# Seasonal definitions for Indian Ocean monsoon cycle.
SEASONS = {
    "Summer Monsoon (JJAS)": [6, 7, 8, 9],
    "Winter Monsoon (DJF)": [12, 1, 2],
    "Intermonsoon-1 (MAM)": [3, 4, 5],
    "Intermonsoon-2 (ON)": [10, 11],
}

MONTH_DAYS = {
    1: 31,
    2: 28,
    3: 31,
    4: 30,
    5: 31,
    6: 30,
    7: 31,
    8: 31,
    9: 30,
    10: 31,
    11: 30,
    12: 31,
}


def weighted_season_mean(da_mon_clim: xr.DataArray, months: list[int]) -> xr.DataArray:
    subset = da_mon_clim.sel(month=months)
    weights = xr.DataArray(
        [MONTH_DAYS[m] for m in months],
        coords={"month": months},
        dims=["month"],
    )
    return (subset * weights).sum("month") / weights.sum()


def main() -> None:
    ds = xr.open_dataset(INFILE)
    if VAR not in ds:
        raise KeyError(f"Variable '{VAR}' not found in {INFILE}")
    da = ds[VAR]
    if "month" not in da.dims:
        raise ValueError("Input file must be monthly climatology with 'month' dimension")
    seasonal = {
        name: weighted_season_mean(da, months) for name, months in SEASONS.items()
    }
    all_vals = np.concatenate([seasonal[s].values.ravel() for s in seasonal])
    finite = all_vals[np.isfinite(all_vals)]
    vmax = np.nanpercentile(np.abs(finite), 98) if finite.size else 0.2
    if vmax <= 0:
        vmax = 0.2
    levels = np.linspace(-vmax, vmax, 21)
    proj = ccrs.PlateCarree()
    fig, axes = plt.subplots(
        2,
        2,
        figsize=(11, 7.5),
        subplot_kw={"projection": proj},
        sharex=True,
        sharey=True,
    )
    fig.subplots_adjust(left=0.01, right=0.98, top=0.98, bottom=0.07, wspace=0.01, hspace=0.01)
    mappable = None
    for ax, title in zip(axes.ravel(), SEASONS.keys()):
        fld = seasonal[title]
        mappable = ax.contourf(
            fld["longitude"],
            fld["latitude"],
            fld,
            levels=levels,
            cmap="RdBu_r",
            extend="both",
            transform=proj,
        )
        ax.coastlines(linewidth=0.8)
        ax.add_feature(cfeature.LAND, facecolor="0.9", zorder=10)
        ax.set_title(title)
        gl = ax.gridlines(draw_labels=True, linewidth=0.3, alpha=0.5)
        gl.top_labels = False
        gl.right_labels = False
    cbar = fig.colorbar(
        mappable,
        ax=axes.ravel().tolist(),
        orientation="horizontal",
        pad=0.04,
        fraction=0.045,
        shrink=0.82,
        aspect=35,
    )
    cbar.set_label("u_ageostrophic (m s$^{-1}$)")
    # fig.suptitle("OSCAR - MSLA Ageostrophic U: Seasonal Means", y=0.98)
    fig.savefig(OUTFIG, dpi=200, bbox_inches="tight")
    plt.show()
    print(f"Wrote figure: {OUTFIG}")

if __name__ == "__main__":
    main()

```

```
wide?
```

```
./guv/
```

57 file(s)

```
1
2  c.extract.msla_uv.at.intersections.py
3
4  OMNI selected location has distance 8000km which is an issue
5
```

```
1
2  variable pool version for scatter plot.
3
4  We might have to use .reindex between gc_self and gc_other
5
6  refactor the code p.rama_uvVStrack_guv_c.nearest.selected.py to plot only for a selected combination of ERS1+ERS2+ENV+SRL sat 15n90e rama_id 810 track_self 423 track_other
7
8  ERS1+ERS2+ENV+SRL 15n90e 352 967
9
10 lots of difference between track gv and aviso gv at points very close to coast
11 track meridional velocities are usually poorly correlated with other products
12
13 I expected correlations of above 0.9 with aviso guv, because they are purely
14 geostrophic, but it was not the case. correlations are similar to that of with
15 oscar
16 Is it possible that the actuall correct sign of gu depends on satellite or
17 whethere the tracks is ascending or descending?
18
```

```
1
2  [] We might have to use .reindex between gc_self and gc_other
3  [] Compare stds of along track gc of l2p and xtrack s3a and for common tracks
4
5  [] code optimal interpolater operator again
6
7  Minor:
8  [] save speed also in guv nc files
9
```

```
1
2  azimuths GFO: a1 = 68.05, a2 = 112.50
3  azimuths GFO: a1 = 68.05, a2 = 112.52
4  azimuths GFO: a1 = 68.05, a2 = 112.77
5  azimuths GFO: a1 = 68.05, a2 = 112.63
6  azimuths GFO: a1 = 67.97, a2 = 112.70
7  azimuths GFO: a1 = 67.97, a2 = 112.84
8  azimuths GFO: a1 = 67.97, a2 = 112.43
9  azimuths GFO: a1 = 67.97, a2 = 112.83
10 azimuths GFO: a1 = 67.97, a2 = 112.85
11 azimuths GFO: a1 = 67.97, a2 = 112.85
12 azimuths GFO: a1 = 67.97, a2 = 112.85
13 azimuths GFO: a1 = 68.00, a2 = 111.98
14 azimuths GFO: a1 = 68.00, a2 = 112.00
15 azimuths GFO: a1 = 67.69, a2 = 111.92
16 azimuths GFO: a1 = 67.69, a2 = 112.08
17 azimuths GFO: a1 = 67.69, a2 = 112.43
18 azimuths GFO: a1 = 67.69, a2 = 112.13
19 azimuths GFO: a1 = 67.69, a2 = 112.46
20 azimuths GFO: a1 = 67.69, a2 = 112.18
21 azimuths GFO: a1 = 67.69, a2 = 111.79
22 azimuths GFO: a1 = 67.69, a2 = 112.23
23 azimuths GFO: a1 = 67.69, a2 = 112.30
24 azimuths GFO: a1 = 67.16, a2 = 112.15
25 azimuths GFO: a1 = 67.16, a2 = 112.75
26 azimuths GFO: a1 = 67.16, a2 = 112.17
27 azimuths GFO: a1 = 67.16, a2 = 112.84
28 azimuths GFO: a1 = 67.16, a2 = 112.00
29 azimuths GFO: a1 = 67.16, a2 = 112.22
30 azimuths GFO: a1 = 67.16, a2 = 112.83
31 azimuths GFO: a1 = 67.16, a2 = 112.04
32 azimuths GFO: a1 = 67.16, a2 = 112.28
33 azimuths GFO: a1 = 67.16, a2 = 112.06
34 azimuths GFO: a1 = 67.16, a2 = 112.37
35 azimuths GFO: a1 = 67.16, a2 = 112.08
36 azimuths GFO: a1 = 67.16, a2 = 112.42
37 azimuths GFO: a1 = 67.73, a2 = 112.36
38 azimuths GFO: a1 = 67.73, a2 = 112.43
39 azimuths GFO: a1 = 67.73, a2 = 112.13
40 azimuths GFO: a1 = 67.73, a2 = 112.50
41 azimuths GFO: a1 = 67.73, a2 = 112.18
42 azimuths GFO: a1 = 67.73, a2 = 112.52
43 azimuths GFO: a1 = 67.73, a2 = 112.23
44 azimuths GFO: a1 = 67.73, a2 = 112.77
45 azimuths GFO: a1 = 67.73, a2 = 112.30
46 azimuths GFO: a1 = 67.50, a2 = 112.75
47 azimuths GFO: a1 = 67.50, a2 = 112.70
48 azimuths GFO: a1 = 67.50, a2 = 112.84
49 azimuths GFO: a1 = 67.50, a2 = 112.22
50 azimuths GFO: a1 = 67.50, a2 = 112.83
51 azimuths GFO: a1 = 67.50, a2 = 112.28
52 azimuths GFO: a1 = 67.50, a2 = 112.85
53 azimuths GFO: a1 = 67.50, a2 = 112.37
54 azimuths GFO: a1 = 67.50, a2 = 112.85
55 azimuths GFO: a1 = 67.50, a2 = 112.42
56 azimuths GFO: a1 = 67.50, a2 = 112.85
57 azimuths GFO: a1 = 68.22, a2 = 112.77
58 azimuths GFO: a1 = 68.22, a2 = 112.63
59 azimuths GFO: a1 = 68.06, a2 = 112.70
60 azimuths GFO: a1 = 68.06, a2 = 112.43
61 azimuths GFO: a1 = 68.06, a2 = 112.83
62 azimuths GFO: a1 = 68.06, a2 = 112.85
63 azimuths GFO: a1 = 68.06, a2 = 112.85
64 azimuths GFO: a1 = 68.06, a2 = 112.85
65 azimuths GFO: a1 = 67.95, a2 = 112.00
66 azimuths GFO: a1 = 67.95, a2 = 112.04
67 azimuths GFO: a1 = 67.51, a2 = 111.92
68 azimuths GFO: a1 = 67.51, a2 = 112.36
69 azimuths GFO: a1 = 67.51, a2 = 112.08
70 azimuths GFO: a1 = 67.51, a2 = 112.43
71 azimuths GFO: a1 = 67.51, a2 = 112.13
72 azimuths GFO: a1 = 67.51, a2 = 112.46
73 azimuths GFO: a1 = 67.51, a2 = 112.18
74 azimuths GFO: a1 = 67.51, a2 = 111.79
75 azimuths GFO: a1 = 67.51, a2 = 112.23
76 azimuths GFO: a1 = 67.51, a2 = 112.30
77 azimuths GFO: a1 = 67.16, a2 = 112.15
78 azimuths GFO: a1 = 67.16, a2 = 112.75
79 azimuths GFO: a1 = 67.16, a2 = 112.17
80 azimuths GFO: a1 = 67.16, a2 = 112.84
81 azimuths GFO: a1 = 67.16, a2 = 112.22
82 azimuths GFO: a1 = 67.16, a2 = 112.83
83 azimuths GFO: a1 = 67.16, a2 = 112.04
84 azimuths GFO: a1 = 67.16, a2 = 112.28
85 azimuths GFO: a1 = 67.16, a2 = 112.85
86 azimuths GFO: a1 = 67.16, a2 = 112.06
87 azimuths GFO: a1 = 67.16, a2 = 112.37
88 azimuths GFO: a1 = 67.16, a2 = 112.08
89 azimuths GFO: a1 = 67.16, a2 = 112.42
90 azimuths GFO: a1 = 67.80, a2 = 112.36
91 azimuths GFO: a1 = 67.80, a2 = 112.43
92 azimuths GFO: a1 = 67.80, a2 = 112.50
93 azimuths GFO: a1 = 67.80, a2 = 112.18
94 azimuths GFO: a1 = 67.80, a2 = 112.52
95 azimuths GFO: a1 = 67.80, a2 = 112.23
96 azimuths GFO: a1 = 67.80, a2 = 112.77
97 azimuths GFO: a1 = 67.80, a2 = 112.30
98 azimuths GFO: a1 = 67.80, a2 = 112.63
99 azimuths GFO: a1 = 67.58, a2 = 112.75
100 azimuths GFO: a1 = 67.58, a2 = 112.70
101 azimuths GFO: a1 = 67.58, a2 = 112.84
102 azimuths GFO: a1 = 67.58, a2 = 112.43
103 azimuths GFO: a1 = 67.58, a2 = 112.83
104 azimuths GFO: a1 = 67.58, a2 = 112.28
105 azimuths GFO: a1 = 67.58, a2 = 112.85
106 azimuths GFO: a1 = 67.58, a2 = 112.37
107 azimuths GFO: a1 = 67.58, a2 = 112.85
108 azimuths GFO: a1 = 67.58, a2 = 112.42
109 azimuths GFO: a1 = 67.58, a2 = 112.85
110 azimuths GFO: a1 = 68.27, a2 = 112.52
111 azimuths GFO: a1 = 68.27, a2 = 112.63
112 azimuths GFO: a1 = 68.13, a2 = 112.70
113 azimuths GFO: a1 = 68.13, a2 = 112.43
114 azimuths GFO: a1 = 68.13, a2 = 112.85
115 azimuths GFO: a1 = 68.13, a2 = 112.85
116 azimuths GFO: a1 = 68.13, a2 = 112.85
117 azimuths GFO: a1 = 67.87, a2 = 111.90
```

```
azimuths GFO: a1 = 67.87, a2 = 112.06
azimuths GFO: a1 = 67.87, a2 = 112.08
azimuths GFO: a1 = 67.51, a2 = 111.92
azimuths GFO: a1 = 67.51, a2 = 112.36
azimuths GFO: a1 = 67.51, a2 = 112.08
azimuths GFO: a1 = 67.51, a2 = 112.43
azimuths GFO: a1 = 67.51, a2 = 112.13
azimuths GFO: a1 = 67.51, a2 = 112.50
azimuths GFO: a1 = 67.51, a2 = 112.18
azimuths GFO: a1 = 67.51, a2 = 111.79
azimuths GFO: a1 = 67.51, a2 = 112.23
azimuths GFO: a1 = 67.51, a2 = 111.83
azimuths GFO: a1 = 67.51, a2 = 112.30
azimuths GFO: a1 = 67.15, a2 = 112.15
azimuths GFO: a1 = 67.15, a2 = 112.75
azimuths GFO: a1 = 67.15, a2 = 112.17
azimuths GFO: a1 = 67.15, a2 = 112.84
azimuths GFO: a1 = 67.15, a2 = 112.22
azimuths GFO: a1 = 67.15, a2 = 112.83
azimuths GFO: a1 = 67.15, a2 = 112.28
azimuths GFO: a1 = 67.15, a2 = 112.85
azimuths GFO: a1 = 67.15, a2 = 112.06
azimuths GFO: a1 = 67.15, a2 = 112.37
azimuths GFO: a1 = 67.15, a2 = 112.85
azimuths GFO: a1 = 67.15, a2 = 112.08
azimuths GFO: a1 = 67.15, a2 = 112.42
azimuths GFO: a1 = 67.92, a2 = 112.36
azimuths GFO: a1 = 67.92, a2 = 112.43
azimuths GFO: a1 = 67.92, a2 = 112.50
azimuths GFO: a1 = 67.92, a2 = 112.52
azimuths GFO: a1 = 67.92, a2 = 112.23
azimuths GFO: a1 = 67.92, a2 = 112.77
azimuths GFO: a1 = 67.92, a2 = 112.30
azimuths GFO: a1 = 67.92, a2 = 112.63
azimuths GFO: a1 = 67.54, a2 = 112.75
azimuths GFO: a1 = 67.54, a2 = 112.70
azimuths GFO: a1 = 67.54, a2 = 112.84
azimuths GFO: a1 = 67.54, a2 = 112.43
azimuths GFO: a1 = 67.54, a2 = 112.83
azimuths GFO: a1 = 67.54, a2 = 112.46
azimuths GFO: a1 = 67.54, a2 = 112.85
azimuths GFO: a1 = 67.54, a2 = 112.37
azimuths GFO: a1 = 67.54, a2 = 112.85
azimuths GFO: a1 = 67.54, a2 = 112.42
azimuths GFO: a1 = 67.54, a2 = 112.85
azimuths GFO: a1 = 67.59, a2 = 111.90
azimuths GFO: a1 = 68.20, a2 = 112.70
azimuths GFO: a1 = 68.20, a2 = 112.43
azimuths GFO: a1 = 68.20, a2 = 112.85
azimuths GFO: a1 = 68.20, a2 = 112.85
azimuths GFO: a1 = 67.83, a2 = 112.15
azimuths GFO: a1 = 67.83, a2 = 111.98
azimuths GFO: a1 = 67.83, a2 = 112.00
azimuths GFO: a1 = 67.83, a2 = 112.04
azimuths GFO: a1 = 67.83, a2 = 112.06
azimuths GFO: a1 = 67.83, a2 = 111.94
azimuths GFO: a1 = 67.83, a2 = 112.08
azimuths GFO: a1 = 67.83, a2 = 111.97
azimuths GFO: a1 = 67.49, a2 = 111.92
azimuths GFO: a1 = 67.49, a2 = 112.36
azimuths GFO: a1 = 67.49, a2 = 112.08
azimuths GFO: a1 = 67.49, a2 = 112.43
azimuths GFO: a1 = 67.49, a2 = 112.13
azimuths GFO: a1 = 67.49, a2 = 112.50
azimuths GFO: a1 = 67.49, a2 = 112.18
azimuths GFO: a1 = 67.49, a2 = 112.52
azimuths GFO: a1 = 67.49, a2 = 112.23
azimuths GFO: a1 = 67.49, a2 = 111.83
azimuths GFO: a1 = 67.49, a2 = 112.30
azimuths GFO: a1 = 67.21, a2 = 112.15
azimuths GFO: a1 = 67.21, a2 = 112.75
azimuths GFO: a1 = 67.21, a2 = 112.17
azimuths GFO: a1 = 67.21, a2 = 112.84
azimuths GFO: a1 = 67.21, a2 = 112.22
azimuths GFO: a1 = 67.21, a2 = 112.83
azimuths GFO: a1 = 67.21, a2 = 112.28
azimuths GFO: a1 = 67.21, a2 = 112.85
azimuths GFO: a1 = 67.21, a2 = 112.37
azimuths GFO: a1 = 67.21, a2 = 112.85
azimuths GFO: a1 = 67.21, a2 = 112.08
azimuths GFO: a1 = 67.21, a2 = 112.42
azimuths GFO: a1 = 67.21, a2 = 112.85
azimuths GFO: a1 = 67.86, a2 = 112.36
azimuths GFO: a1 = 67.86, a2 = 112.75
azimuths GFO: a1 = 67.86, a2 = 112.43
azimuths GFO: a1 = 67.86, a2 = 112.50
azimuths GFO: a1 = 67.86, a2 = 112.52
azimuths GFO: a1 = 67.86, a2 = 112.77
azimuths GFO: a1 = 67.86, a2 = 112.30
azimuths GFO: a1 = 67.86, a2 = 112.63
azimuths GFO: a1 = 67.74, a2 = 112.75
azimuths GFO: a1 = 67.74, a2 = 112.70
azimuths GFO: a1 = 67.74, a2 = 112.84
azimuths GFO: a1 = 67.74, a2 = 112.43
azimuths GFO: a1 = 67.74, a2 = 112.83
azimuths GFO: a1 = 67.74, a2 = 112.85
azimuths GFO: a1 = 67.74, a2 = 112.85
azimuths GFO: a1 = 67.74, a2 = 112.42
azimuths GFO: a1 = 67.74, a2 = 112.85
azimuths GFO: a1 = 67.61, a2 = 111.94
azimuths GFO: a1 = 67.91, a2 = 112.70
azimuths GFO: a1 = 67.91, a2 = 112.08
azimuths GFO: a1 = 67.91, a2 = 112.43
azimuths GFO: a1 = 67.91, a2 = 112.85
azimuths GFO: a1 = 67.18, a2 = 112.15
azimuths GFO: a1 = 67.18, a2 = 112.75
azimuths GFO: a1 = 67.18, a2 = 111.98
azimuths GFO: a1 = 67.18, a2 = 112.17
azimuths GFO: a1 = 67.18, a2 = 112.00
azimuths GFO: a1 = 67.18, a2 = 112.22
azimuths GFO: a1 = 67.18, a2 = 112.04
azimuths GFO: a1 = 67.18, a2 = 112.06
azimuths GFO: a1 = 67.18, a2 = 112.08
azimuths GFO: a1 = 67.18, a2 = 111.97
azimuths GFO: a1 = 67.51, a2 = 111.92
azimuths GFO: a1 = 67.51, a2 = 112.36
azimuths GFO: a1 = 67.51, a2 = 112.08
```

```
azimuths GFO: a1 = 67.51, a2 = 112.43
azimuths GFO: a1 = 67.51, a2 = 112.13
azimuths GFO: a1 = 67.51, a2 = 112.50
azimuths GFO: a1 = 67.51, a2 = 112.18
azimuths GFO: a1 = 67.51, a2 = 112.52
azimuths GFO: a1 = 67.51, a2 = 112.23
azimuths GFO: a1 = 67.51, a2 = 112.77
azimuths GFO: a1 = 67.51, a2 = 112.30
azimuths GFO: a1 = 67.32, a2 = 112.15
azimuths GFO: a1 = 67.32, a2 = 112.75
azimuths GFO: a1 = 67.32, a2 = 112.17
azimuths GFO: a1 = 67.32, a2 = 112.84
azimuths GFO: a1 = 67.32, a2 = 112.22
azimuths GFO: a1 = 67.32, a2 = 112.83
azimuths GFO: a1 = 67.32, a2 = 112.28
azimuths GFO: a1 = 67.32, a2 = 112.85
azimuths GFO: a1 = 67.32, a2 = 112.37
azimuths GFO: a1 = 67.32, a2 = 112.85
azimuths GFO: a1 = 67.32, a2 = 112.42
azimuths GFO: a1 = 67.32, a2 = 112.85
azimuths GFO: a1 = 67.96, a2 = 112.36
azimuths GFO: a1 = 67.96, a2 = 112.75
azimuths GFO: a1 = 67.96, a2 = 112.43
azimuths GFO: a1 = 67.96, a2 = 112.50
azimuths GFO: a1 = 67.96, a2 = 112.52
azimuths GFO: a1 = 67.96, a2 = 112.77
azimuths GFO: a1 = 67.96, a2 = 112.63
azimuths GFO: a1 = 67.88, a2 = 112.75
azimuths GFO: a1 = 67.88, a2 = 112.70
azimuths GFO: a1 = 67.88, a2 = 112.84
azimuths GFO: a1 = 67.88, a2 = 112.43
azimuths GFO: a1 = 67.88, a2 = 112.83
azimuths GFO: a1 = 67.88, a2 = 112.85
azimuths GFO: a1 = 67.88, a2 = 112.85
azimuths GFO: a1 = 67.88, a2 = 112.85
azimuths GFO: a1 = 67.56, a2 = 111.97
azimuths GFO: a1 = 67.82, a2 = 111.92
azimuths GFO: a1 = 67.82, a2 = 112.70
azimuths GFO: a1 = 67.82, a2 = 112.08
azimuths GFO: a1 = 67.82, a2 = 112.43
azimuths GFO: a1 = 67.82, a2 = 112.13
azimuths GFO: a1 = 67.82, a2 = 112.46
azimuths GFO: a1 = 67.82, a2 = 111.79
azimuths GFO: a1 = 67.82, a2 = 111.83
azimuths GFO: a1 = 67.18, a2 = 112.15
azimuths GFO: a1 = 67.18, a2 = 111.98
azimuths GFO: a1 = 67.18, a2 = 112.17
azimuths GFO: a1 = 67.18, a2 = 112.84
azimuths GFO: a1 = 67.18, a2 = 112.00
azimuths GFO: a1 = 67.18, a2 = 112.22
azimuths GFO: a1 = 67.18, a2 = 112.04
azimuths GFO: a1 = 67.18, a2 = 112.28
azimuths GFO: a1 = 67.18, a2 = 112.06
azimuths GFO: a1 = 67.18, a2 = 112.37
azimuths GFO: a1 = 67.18, a2 = 112.08
azimuths GFO: a1 = 64.65, a2 = 112.36
azimuths GFO: a1 = 64.65, a2 = 112.08
azimuths GFO: a1 = 64.65, a2 = 112.43
azimuths GFO: a1 = 64.65, a2 = 112.13
azimuths GFO: a1 = 64.65, a2 = 112.50
azimuths GFO: a1 = 64.65, a2 = 112.18
azimuths GFO: a1 = 64.65, a2 = 112.52
azimuths GFO: a1 = 64.65, a2 = 112.23
azimuths GFO: a1 = 64.65, a2 = 112.77
azimuths GFO: a1 = 64.65, a2 = 112.30
azimuths GFO: a1 = 65.80, a2 = 112.75
azimuths GFO: a1 = 65.80, a2 = 112.70
azimuths GFO: a1 = 65.80, a2 = 112.17
azimuths GFO: a1 = 65.80, a2 = 112.84
azimuths GFO: a1 = 65.80, a2 = 112.22
azimuths GFO: a1 = 65.80, a2 = 112.83
azimuths GFO: a1 = 65.80, a2 = 112.28
azimuths GFO: a1 = 65.80, a2 = 112.85
azimuths GFO: a1 = 65.80, a2 = 112.37
azimuths GFO: a1 = 65.80, a2 = 112.85
azimuths GFO: a1 = 65.80, a2 = 112.42
azimuths GFO: a1 = 65.80, a2 = 112.85
azimuths TP+J1+J2+J3+S6A: a1 = 70.06, a2 = 110.99
azimuths TP+J1+J2+J3+S6A: a1 = 70.06, a2 = 111.34
azimuths TP+J1+J2+J3+S6A: a1 = 69.89, a2 = 110.11
azimuths TP+J1+J2+J3+S6A: a1 = 69.71, a2 = 110.82
azimuths TP+J1+J2+J3+S6A: a1 = 69.71, a2 = 110.85
azimuths TP+J1+J2+J3+S6A: a1 = 69.71, a2 = 110.61
azimuths TP+J1+J2+J3+S6A: a1 = 69.27, a2 = 111.13
azimuths TP+J1+J2+J3+S6A: a1 = 69.27, a2 = 110.99
azimuths TP+J1+J2+J3+S6A: a1 = 69.27, a2 = 111.33
azimuths TP+J1+J2+J3+S6A: a1 = 69.27, a2 = 111.32
azimuths TP+J1+J2+J3+S6A: a1 = 69.27, a2 = 111.34
azimuths TP+J1+J2+J3+S6A: a1 = 69.08, a2 = 110.82
azimuths TP+J1+J2+J3+S6A: a1 = 69.08, a2 = 110.28
azimuths TP+J1+J2+J3+S6A: a1 = 69.08, a2 = 110.85
azimuths TP+J1+J2+J3+S6A: a1 = 69.08, a2 = 110.42
azimuths TP+J1+J2+J3+S6A: a1 = 69.08, a2 = 110.61
azimuths TP+J1+J2+J3+S6A: a1 = 68.68, a2 = 111.13
azimuths TP+J1+J2+J3+S6A: a1 = 68.68, a2 = 110.31
azimuths TP+J1+J2+J3+S6A: a1 = 68.68, a2 = 111.33
azimuths TP+J1+J2+J3+S6A: a1 = 68.68, a2 = 110.37
azimuths TP+J1+J2+J3+S6A: a1 = 68.68, a2 = 111.32
azimuths TP+J1+J2+J3+S6A: a1 = 68.68, a2 = 110.54
azimuths TP+J1+J2+J3+S6A: a1 = 70.22, a2 = 110.99
azimuths TP+J1+J2+J3+S6A: a1 = 69.78, a2 = 110.18
azimuths TP+J1+J2+J3+S6A: a1 = 70.05, a2 = 110.85
azimuths TP+J1+J2+J3+S6A: a1 = 70.05, a2 = 110.61
azimuths TP+J1+J2+J3+S6A: a1 = 69.50, a2 = 110.99
azimuths TP+J1+J2+J3+S6A: a1 = 69.50, a2 = 111.33
azimuths TP+J1+J2+J3+S6A: a1 = 69.50, a2 = 111.32
azimuths TP+J1+J2+J3+S6A: a1 = 69.50, a2 = 111.34
azimuths TP+J1+J2+J3+S6A: a1 = 69.37, a2 = 110.82
azimuths TP+J1+J2+J3+S6A: a1 = 69.37, a2 = 110.85
azimuths TP+J1+J2+J3+S6A: a1 = 69.37, a2 = 110.42
azimuths TP+J1+J2+J3+S6A: a1 = 69.37, a2 = 110.61
azimuths TP+J1+J2+J3+S6A: a1 = 68.75, a2 = 111.13
azimuths TP+J1+J2+J3+S6A: a1 = 68.75, a2 = 111.33
azimuths TP+J1+J2+J3+S6A: a1 = 68.75, a2 = 110.37
azimuths TP+J1+J2+J3+S6A: a1 = 68.75, a2 = 111.32
azimuths TP+J1+J2+J3+S6A: a1 = 68.75, a2 = 110.54
azimuths TP+J1+J2+J3+S6A: a1 = 68.75, a2 = 111.34
```

```
azimuths TP+J1+J2+J3+S6A: a1 = 69.53, a2 = 110.28
azimuths TP+J1+J2+J3+S6A: a1 = 69.53, a2 = 110.42
azimuths TP+J1+J2+J3+S6A: a1 = 69.53, a2 = 109.82
azimuths TP+J1+J2+J3+S6A: a1 = 69.53, a2 = 109.99
azimuths TP+J1+J2+J3+S6A: a1 = 68.86, a2 = 111.13
azimuths TP+J1+J2+J3+S6A: a1 = 68.86, a2 = 110.31
azimuths TP+J1+J2+J3+S6A: a1 = 68.86, a2 = 110.11
azimuths TP+J1+J2+J3+S6A: a1 = 68.86, a2 = 110.18
azimuths TP+J1+J2+J3+S6A: a1 = 69.83, a2 = 110.99
azimuths TP+J1+J2+J3+S6A: a1 = 69.83, a2 = 111.32
azimuths TP+J1+J2+J3+S6A: a1 = 69.83, a2 = 111.34
azimuths TP+J1+J2+J3+S6A: a1 = 69.75, a2 = 110.82
azimuths TP+J1+J2+J3+S6A: a1 = 69.75, a2 = 110.85
azimuths TP+J1+J2+J3+S6A: a1 = 69.75, a2 = 110.61
azimuths TP+J1+J2+J3+S6A: a1 = 69.00, a2 = 111.13
azimuths TP+J1+J2+J3+S6A: a1 = 69.00, a2 = 111.33
azimuths TP+J1+J2+J3+S6A: a1 = 69.00, a2 = 111.32
azimuths TP+J1+J2+J3+S6A: a1 = 69.00, a2 = 110.54
azimuths TP+J1+J2+J3+S6A: a1 = 69.00, a2 = 111.34
azimuths TP+J1+J2+J3+S6A: a1 = 69.17, a2 = 110.82
azimuths TP+J1+J2+J3+S6A: a1 = 69.17, a2 = 110.28
azimuths TP+J1+J2+J3+S6A: a1 = 69.17, a2 = 110.42
azimuths TP+J1+J2+J3+S6A: a1 = 69.17, a2 = 110.61
azimuths TP+J1+J2+J3+S6A: a1 = 69.17, a2 = 109.99
azimuths TP+J1+J2+J3+S6A: a1 = 68.70, a2 = 111.13
azimuths TP+J1+J2+J3+S6A: a1 = 68.70, a2 = 110.31
azimuths TP+J1+J2+J3+S6A: a1 = 68.70, a2 = 111.33
azimuths TP+J1+J2+J3+S6A: a1 = 68.70, a2 = 110.37
azimuths TP+J1+J2+J3+S6A: a1 = 68.70, a2 = 110.54
azimuths TP+J1+J2+J3+S6A: a1 = 68.70, a2 = 110.18
azimuths S3A: a1 = 77.29, a2 = 102.92
azimuths S3A: a1 = 77.29, a2 = 102.88
azimuths S3A: a1 = 77.29, a2 = 102.84
azimuths S3A: a1 = 77.29, a2 = 102.70
azimuths S3A: a1 = 77.29, a2 = 102.90
azimuths S3A: a1 = 77.29, a2 = 102.91
azimuths S3A: a1 = 77.29, a2 = 102.91
azimuths S3A: a1 = 77.12, a2 = 103.07
azimuths S3A: a1 = 77.12, a2 = 103.08
azimuths S3A: a1 = 77.12, a2 = 103.05
azimuths S3A: a1 = 77.12, a2 = 102.99
azimuths S3A: a1 = 77.12, a2 = 103.06
azimuths S3A: a1 = 77.12, a2 = 102.95
azimuths S3A: a1 = 77.12, a2 = 103.06
azimuths S3A: a1 = 77.12, a2 = 102.87
azimuths S3A: a1 = 77.12, a2 = 103.07
azimuths S3A: a1 = 77.12, a2 = 102.88
azimuths S3A: a1 = 77.16, a2 = 102.62
azimuths S3A: a1 = 77.42, a2 = 102.90
azimuths S3A: a1 = 77.42, a2 = 102.59
azimuths S3A: a1 = 77.42, a2 = 102.95
azimuths S3A: a1 = 77.42, a2 = 102.87
azimuths S3A: a1 = 77.42, a2 = 102.88
azimuths S3A: a1 = 77.31, a2 = 102.66
azimuths S3A: a1 = 77.31, a2 = 102.68
azimuths S3A: a1 = 77.31, a2 = 102.68
azimuths S3A: a1 = 77.31, a2 = 102.64
azimuths S3A: a1 = 77.31, a2 = 102.64
azimuths S3A: a1 = 77.31, a2 = 102.65
azimuths S3A: a1 = 77.22, a2 = 102.61
azimuths S3A: a1 = 77.22, a2 = 102.70
azimuths S3A: a1 = 77.22, a2 = 102.71
azimuths S3A: a1 = 77.22, a2 = 102.72
azimuths S3A: a1 = 77.22, a2 = 102.75
azimuths S3A: a1 = 77.22, a2 = 102.56
azimuths S3A: a1 = 77.22, a2 = 102.77
azimuths S3A: a1 = 77.22, a2 = 102.57
azimuths S3A: a1 = 76.93, a2 = 102.73
azimuths S3A: a1 = 76.93, a2 = 103.02
azimuths S3A: a1 = 76.93, a2 = 102.74
azimuths S3A: a1 = 76.93, a2 = 103.05
azimuths S3A: a1 = 76.93, a2 = 102.76
azimuths S3A: a1 = 76.93, a2 = 103.07
azimuths S3A: a1 = 76.93, a2 = 102.77
azimuths S3A: a1 = 76.93, a2 = 102.80
azimuths S3A: a1 = 76.93, a2 = 102.70
azimuths S3A: a1 = 76.93, a2 = 102.83
azimuths S3A: a1 = 76.93, a2 = 102.72
azimuths S3A: a1 = 76.93, a2 = 102.86
azimuths S3A: a1 = 77.08, a2 = 102.81
azimuths S3A: a1 = 77.08, a2 = 102.92
azimuths S3A: a1 = 77.08, a2 = 102.83
azimuths S3A: a1 = 77.08, a2 = 102.85
azimuths S3A: a1 = 77.08, a2 = 102.90
azimuths S3A: a1 = 77.08, a2 = 102.90
azimuths S3A: a1 = 77.08, a2 = 102.77
azimuths S3A: a1 = 77.08, a2 = 102.91
azimuths S3A: a1 = 77.08, a2 = 102.78
azimuths S3A: a1 = 77.08, a2 = 102.91
azimuths S3A: a1 = 76.96, a2 = 103.02
azimuths S3A: a1 = 76.96, a2 = 103.07
azimuths S3A: a1 = 76.96, a2 = 103.05
azimuths S3A: a1 = 76.96, a2 = 103.08
azimuths S3A: a1 = 76.96, a2 = 103.07
azimuths S3A: a1 = 76.96, a2 = 103.05
azimuths S3A: a1 = 76.96, a2 = 103.06
azimuths S3A: a1 = 76.96, a2 = 103.06
azimuths S3A: a1 = 76.96, a2 = 102.83
azimuths S3A: a1 = 76.96, a2 = 103.06
azimuths S3A: a1 = 76.96, a2 = 102.86
azimuths S3A: a1 = 76.96, a2 = 103.07
azimuths S3A: a1 = 77.28, a2 = 102.92
azimuths S3A: a1 = 77.28, a2 = 102.88
azimuths S3A: a1 = 77.28, a2 = 102.84
azimuths S3A: a1 = 77.28, a2 = 102.70
azimuths S3A: a1 = 77.28, a2 = 102.78
azimuths S3A: a1 = 77.28, a2 = 102.91
azimuths S3A: a1 = 77.28, a2 = 102.91
azimuths S3A: a1 = 77.13, a2 = 103.07
azimuths S3A: a1 = 77.13, a2 = 102.90
azimuths S3A: a1 = 77.13, a2 = 103.08
azimuths S3A: a1 = 77.13, a2 = 103.05
azimuths S3A: a1 = 77.13, a2 = 102.99
azimuths S3A: a1 = 77.13, a2 = 102.95
azimuths S3A: a1 = 77.13, a2 = 103.06
azimuths S3A: a1 = 77.13, a2 = 102.87
azimuths S3A: a1 = 77.13, a2 = 103.07
```

```
azimuths S3A: a1 = 77.13, a2 = 102.88
azimuths S3A: a1 = 77.14, a2 = 102.63
azimuths S3A: a1 = 77.45, a2 = 102.90
azimuths S3A: a1 = 77.45, a2 = 102.59
azimuths S3A: a1 = 77.45, a2 = 102.87
azimuths S3A: a1 = 77.45, a2 = 102.88
azimuths S3A: a1 = 77.28, a2 = 102.66
azimuths S3A: a1 = 77.28, a2 = 102.67
azimuths S3A: a1 = 77.28, a2 = 102.68
azimuths S3A: a1 = 77.28, a2 = 102.68
azimuths S3A: a1 = 77.28, a2 = 102.70
azimuths S3A: a1 = 77.28, a2 = 102.64
azimuths S3A: a1 = 77.28, a2 = 102.72
azimuths S3A: a1 = 77.28, a2 = 102.65
azimuths S3A: a1 = 77.17, a2 = 102.61
azimuths S3A: a1 = 77.17, a2 = 102.81
azimuths S3A: a1 = 77.17, a2 = 102.70
azimuths S3A: a1 = 77.17, a2 = 102.71
azimuths S3A: a1 = 77.17, a2 = 102.72
azimuths S3A: a1 = 77.17, a2 = 102.75
azimuths S3A: a1 = 77.17, a2 = 102.77
azimuths S3A: a1 = 77.17, a2 = 102.57
azimuths S3A: a1 = 77.17, a2 = 102.78
azimuths S3A: a1 = 76.93, a2 = 102.73
azimuths S3A: a1 = 76.93, a2 = 103.02
azimuths S3A: a1 = 76.93, a2 = 102.74
azimuths S3A: a1 = 76.93, a2 = 103.05
azimuths S3A: a1 = 76.93, a2 = 102.76
azimuths S3A: a1 = 76.93, a2 = 103.07
azimuths S3A: a1 = 76.93, a2 = 102.77
azimuths S3A: a1 = 76.93, a2 = 103.06
azimuths S3A: a1 = 76.93, a2 = 102.80
azimuths S3A: a1 = 76.93, a2 = 102.83
azimuths S3A: a1 = 76.93, a2 = 102.72
azimuths S3A: a1 = 76.93, a2 = 102.86
azimuths S3A: a1 = 77.13, a2 = 102.81
azimuths S3A: a1 = 77.13, a2 = 102.92
azimuths S3A: a1 = 77.13, a2 = 102.83
azimuths S3A: a1 = 77.13, a2 = 102.88
azimuths S3A: a1 = 77.13, a2 = 102.85
azimuths S3A: a1 = 77.13, a2 = 102.90
azimuths S3A: a1 = 77.13, a2 = 102.90
azimuths S3A: a1 = 77.13, a2 = 102.91
azimuths S3A: a1 = 77.13, a2 = 102.78
azimuths S3A: a1 = 77.13, a2 = 102.91
azimuths S3A: a1 = 77.00, a2 = 103.02
azimuths S3A: a1 = 77.00, a2 = 103.07
azimuths S3A: a1 = 77.00, a2 = 103.05
azimuths S3A: a1 = 77.00, a2 = 103.08
azimuths S3A: a1 = 77.00, a2 = 103.07
azimuths S3A: a1 = 77.00, a2 = 103.05
azimuths S3A: a1 = 77.00, a2 = 103.06
azimuths S3A: a1 = 77.00, a2 = 103.06
azimuths S3A: a1 = 77.00, a2 = 103.06
azimuths S3A: a1 = 77.00, a2 = 102.86
azimuths S3A: a1 = 77.00, a2 = 103.07
azimuths S3A: a1 = 77.31, a2 = 102.92
azimuths S3A: a1 = 77.31, a2 = 102.88
azimuths S3A: a1 = 77.31, a2 = 102.84
azimuths S3A: a1 = 77.31, a2 = 102.70
azimuths S3A: a1 = 77.31, a2 = 102.78
azimuths S3A: a1 = 77.31, a2 = 102.89
azimuths S3A: a1 = 77.31, a2 = 102.91
azimuths S3A: a1 = 77.23, a2 = 103.07
azimuths S3A: a1 = 77.23, a2 = 103.08
azimuths S3A: a1 = 77.23, a2 = 103.05
azimuths S3A: a1 = 77.23, a2 = 102.99
azimuths S3A: a1 = 77.23, a2 = 102.95
azimuths S3A: a1 = 77.23, a2 = 102.87
azimuths S3A: a1 = 77.23, a2 = 103.07
azimuths S3A: a1 = 77.23, a2 = 102.88
azimuths S3A: a1 = 77.36, a2 = 102.64
azimuths S3A: a1 = 77.46, a2 = 102.90
azimuths S3A: a1 = 77.46, a2 = 102.59
azimuths S3A: a1 = 77.46, a2 = 102.88
azimuths S3A: a1 = 77.27, a2 = 102.73
azimuths S3A: a1 = 77.27, a2 = 102.66
azimuths S3A: a1 = 77.27, a2 = 102.67
azimuths S3A: a1 = 77.27, a2 = 102.68
azimuths S3A: a1 = 77.27, a2 = 102.68
azimuths S3A: a1 = 77.27, a2 = 102.70
azimuths S3A: a1 = 77.27, a2 = 102.72
azimuths S3A: a1 = 77.27, a2 = 102.65
azimuths S3A: a1 = 77.15, a2 = 102.61
azimuths S3A: a1 = 77.15, a2 = 102.81
azimuths S3A: a1 = 77.15, a2 = 102.70
azimuths S3A: a1 = 77.15, a2 = 102.83
azimuths S3A: a1 = 77.15, a2 = 102.71
azimuths S3A: a1 = 77.15, a2 = 102.72
azimuths S3A: a1 = 77.15, a2 = 102.75
azimuths S3A: a1 = 77.15, a2 = 102.77
azimuths S3A: a1 = 77.15, a2 = 102.78
azimuths S3A: a1 = 76.93, a2 = 102.73
azimuths S3A: a1 = 76.93, a2 = 103.02
azimuths S3A: a1 = 76.93, a2 = 102.74
azimuths S3A: a1 = 76.93, a2 = 103.05
azimuths S3A: a1 = 76.93, a2 = 102.76
azimuths S3A: a1 = 76.93, a2 = 103.07
azimuths S3A: a1 = 76.93, a2 = 102.77
azimuths S3A: a1 = 76.93, a2 = 103.06
azimuths S3A: a1 = 76.93, a2 = 102.80
azimuths S3A: a1 = 76.93, a2 = 103.06
azimuths S3A: a1 = 76.93, a2 = 102.83
azimuths S3A: a1 = 76.93, a2 = 102.86
azimuths S3A: a1 = 77.17, a2 = 102.81
azimuths S3A: a1 = 77.17, a2 = 102.92
azimuths S3A: a1 = 77.17, a2 = 102.83
azimuths S3A: a1 = 77.17, a2 = 102.88
azimuths S3A: a1 = 77.17, a2 = 102.85
azimuths S3A: a1 = 77.17, a2 = 102.90
azimuths S3A: a1 = 77.17, a2 = 102.90
azimuths S3A: a1 = 77.17, a2 = 102.91
azimuths S3A: a1 = 77.17, a2 = 102.91
azimuths S3A: a1 = 77.03, a2 = 103.02
azimuths S3A: a1 = 77.03, a2 = 103.07
azimuths S3A: a1 = 77.03, a2 = 103.05
azimuths S3A: a1 = 77.03, a2 = 103.08
```

```
azimuths S3A: a1 = 77.03, a2 = 103.07
azimuths S3A: a1 = 77.03, a2 = 103.05
azimuths S3A: a1 = 77.03, a2 = 103.06
azimuths S3A: a1 = 77.03, a2 = 102.99
azimuths S3A: a1 = 77.03, a2 = 103.06
azimuths S3A: a1 = 77.03, a2 = 103.06
azimuths S3A: a1 = 77.03, a2 = 103.07
azimuths S3A: a1 = 77.36, a2 = 102.92
azimuths S3A: a1 = 77.36, a2 = 102.88
azimuths S3A: a1 = 77.36, a2 = 102.84
azimuths S3A: a1 = 77.36, a2 = 102.70
azimuths S3A: a1 = 77.36, a2 = 102.78
azimuths S3A: a1 = 77.36, a2 = 102.89
azimuths S3A: a1 = 77.29, a2 = 103.07
azimuths S3A: a1 = 77.29, a2 = 103.08
azimuths S3A: a1 = 77.29, a2 = 103.05
azimuths S3A: a1 = 77.29, a2 = 102.99
azimuths S3A: a1 = 77.29, a2 = 102.95
azimuths S3A: a1 = 77.29, a2 = 102.87
azimuths S3A: a1 = 77.29, a2 = 102.88
azimuths S3A: a1 = 77.35, a2 = 102.64
azimuths S3A: a1 = 77.45, a2 = 102.90
azimuths S3A: a1 = 77.45, a2 = 102.59
azimuths S3A: a1 = 77.45, a2 = 102.54
azimuths S3A: a1 = 77.45, a2 = 102.54
azimuths S3A: a1 = 77.27, a2 = 102.73
azimuths S3A: a1 = 77.27, a2 = 102.66
azimuths S3A: a1 = 77.27, a2 = 102.74
azimuths S3A: a1 = 77.27, a2 = 102.67
azimuths S3A: a1 = 77.27, a2 = 102.68
azimuths S3A: a1 = 77.27, a2 = 102.68
azimuths S3A: a1 = 77.27, a2 = 102.70
azimuths S3A: a1 = 77.09, a2 = 102.81
azimuths S3A: a1 = 77.09, a2 = 102.70
azimuths S3A: a1 = 77.09, a2 = 102.83
azimuths S3A: a1 = 77.09, a2 = 102.71
azimuths S3A: a1 = 77.09, a2 = 102.85
azimuths S3A: a1 = 77.09, a2 = 102.72
azimuths S3A: a1 = 77.09, a2 = 102.90
azimuths S3A: a1 = 77.09, a2 = 102.75
azimuths S3A: a1 = 77.09, a2 = 102.77
azimuths S3A: a1 = 77.09, a2 = 102.78
azimuths S3A: a1 = 76.92, a2 = 103.02
azimuths S3A: a1 = 76.92, a2 = 102.74
azimuths S3A: a1 = 76.92, a2 = 103.05
azimuths S3A: a1 = 76.92, a2 = 102.76
azimuths S3A: a1 = 76.92, a2 = 103.07
azimuths S3A: a1 = 76.92, a2 = 102.77
azimuths S3A: a1 = 76.92, a2 = 103.06
azimuths S3A: a1 = 76.92, a2 = 102.80
azimuths S3A: a1 = 76.92, a2 = 103.06
azimuths S3A: a1 = 76.92, a2 = 102.83
azimuths S3A: a1 = 76.92, a2 = 103.06
azimuths S3A: a1 = 76.92, a2 = 102.86
azimuths S3A: a1 = 77.20, a2 = 102.92
azimuths S3A: a1 = 77.20, a2 = 102.83
azimuths S3A: a1 = 77.20, a2 = 102.88
azimuths S3A: a1 = 77.20, a2 = 102.85
azimuths S3A: a1 = 77.20, a2 = 102.84
azimuths S3A: a1 = 77.20, a2 = 102.90
azimuths S3A: a1 = 77.20, a2 = 102.90
azimuths S3A: a1 = 77.20, a2 = 102.91
azimuths S3A: a1 = 77.20, a2 = 102.91
azimuths S3A: a1 = 77.05, a2 = 103.07
azimuths S3A: a1 = 77.05, a2 = 103.05
azimuths S3A: a1 = 77.05, a2 = 103.08
azimuths S3A: a1 = 77.05, a2 = 103.07
azimuths S3A: a1 = 77.05, a2 = 103.05
azimuths S3A: a1 = 77.05, a2 = 103.06
azimuths S3A: a1 = 77.05, a2 = 102.99
azimuths S3A: a1 = 77.05, a2 = 103.06
azimuths S3A: a1 = 77.05, a2 = 102.95
azimuths S3A: a1 = 77.05, a2 = 103.06
azimuths S3A: a1 = 77.05, a2 = 103.07
azimuths S3A: a1 = 77.40, a2 = 102.88
azimuths S3A: a1 = 77.40, a2 = 102.70
azimuths S3A: a1 = 77.40, a2 = 102.78
azimuths S3A: a1 = 77.40, a2 = 102.89
azimuths S3A: a1 = 77.33, a2 = 102.90
azimuths S3A: a1 = 77.33, a2 = 103.08
azimuths S3A: a1 = 77.33, a2 = 103.05
azimuths S3A: a1 = 77.33, a2 = 102.99
azimuths S3A: a1 = 77.33, a2 = 102.95
azimuths S3A: a1 = 77.33, a2 = 102.87
azimuths S3A: a1 = 77.33, a2 = 102.88
azimuths S3A: a1 = 77.35, a2 = 102.62
azimuths S3A: a1 = 77.35, a2 = 102.65
azimuths S3A: a1 = 77.30, a2 = 102.70
azimuths S3A: a1 = 77.30, a2 = 102.59
azimuths S3A: a1 = 77.30, a2 = 102.54
azimuths S3A: a1 = 77.30, a2 = 102.54
azimuths S3A: a1 = 77.30, a2 = 102.56
azimuths S3A: a1 = 77.23, a2 = 102.73
azimuths S3A: a1 = 77.23, a2 = 102.74
azimuths S3A: a1 = 77.23, a2 = 102.67
azimuths S3A: a1 = 77.23, a2 = 102.76
azimuths S3A: a1 = 77.23, a2 = 102.68
azimuths S3A: a1 = 77.23, a2 = 102.68
azimuths S3A: a1 = 77.23, a2 = 102.70
azimuths S3A: a1 = 77.23, a2 = 102.72
azimuths S3A: a1 = 77.10, a2 = 102.81
azimuths S3A: a1 = 77.10, a2 = 102.83
azimuths S3A: a1 = 77.10, a2 = 102.71
azimuths S3A: a1 = 77.10, a2 = 102.85
azimuths S3A: a1 = 77.10, a2 = 102.72
azimuths S3A: a1 = 77.10, a2 = 102.90
azimuths S3A: a1 = 77.10, a2 = 102.75
azimuths S3A: a1 = 77.10, a2 = 102.90
azimuths S3A: a1 = 77.10, a2 = 102.77
azimuths S3A: a1 = 77.10, a2 = 102.78
azimuths S3A: a1 = 76.93, a2 = 103.02
azimuths S3A: a1 = 76.93, a2 = 103.05
azimuths S3A: a1 = 76.93, a2 = 102.76
azimuths S3A: a1 = 76.93, a2 = 103.07
azimuths S3A: a1 = 76.93, a2 = 102.77
azimuths S3A: a1 = 76.93, a2 = 103.06
azimuths S3A: a1 = 76.93, a2 = 102.80
```

```
azimuths S3A: a1 = 76.93, a2 = 103.06
azimuths S3A: a1 = 76.93, a2 = 102.83
azimuths S3A: a1 = 76.93, a2 = 103.06
azimuths S3A: a1 = 76.93, a2 = 102.86
azimuths S3A: a1 = 76.93, a2 = 103.07
azimuths S3A: a1 = 77.24, a2 = 102.92
azimuths S3A: a1 = 77.24, a2 = 102.88
azimuths S3A: a1 = 77.24, a2 = 102.85
azimuths S3A: a1 = 77.24, a2 = 102.84
azimuths S3A: a1 = 77.24, a2 = 102.90
azimuths S3A: a1 = 77.24, a2 = 102.90
azimuths S3A: a1 = 77.24, a2 = 102.91
azimuths S3A: a1 = 77.24, a2 = 102.91
azimuths S3A: a1 = 77.12, a2 = 103.07
azimuths S3A: a1 = 77.12, a2 = 103.08
azimuths S3A: a1 = 77.12, a2 = 103.07
azimuths S3A: a1 = 77.12, a2 = 103.05
azimuths S3A: a1 = 77.12, a2 = 103.06
azimuths S3A: a1 = 77.12, a2 = 102.99
azimuths S3A: a1 = 77.12, a2 = 103.06
azimuths S3A: a1 = 77.12, a2 = 102.95
azimuths S3A: a1 = 77.12, a2 = 103.06
azimuths S3A: a1 = 77.12, a2 = 103.07
azimuths S3A: a1 = 77.47, a2 = 102.84
azimuths S3A: a1 = 77.47, a2 = 102.78
azimuths S3A: a1 = 77.36, a2 = 102.90
azimuths S3A: a1 = 77.36, a2 = 103.05
azimuths S3A: a1 = 77.36, a2 = 102.99
azimuths S3A: a1 = 77.36, a2 = 102.95
azimuths S3A: a1 = 77.36, a2 = 102.87
azimuths S3A: a1 = 77.36, a2 = 102.88
azimuths S3A: a1 = 77.32, a2 = 102.66
azimuths S3A: a1 = 77.32, a2 = 102.67
azimuths S3A: a1 = 77.32, a2 = 102.62
azimuths S3A: a1 = 77.28, a2 = 102.61
azimuths S3A: a1 = 77.28, a2 = 102.70
azimuths S3A: a1 = 77.28, a2 = 102.71
azimuths S3A: a1 = 77.28, a2 = 102.54
azimuths S3A: a1 = 77.28, a2 = 102.54
azimuths S3A: a1 = 77.28, a2 = 102.56
azimuths S3A: a1 = 76.98, a2 = 102.73
azimuths S3A: a1 = 76.98, a2 = 103.02
azimuths S3A: a1 = 76.98, a2 = 102.74
azimuths S3A: a1 = 76.98, a2 = 102.76
azimuths S3A: a1 = 76.98, a2 = 102.68
azimuths S3A: a1 = 76.98, a2 = 102.77
azimuths S3A: a1 = 76.98, a2 = 102.68
azimuths S3A: a1 = 76.98, a2 = 102.80
azimuths S3A: a1 = 76.98, a2 = 102.70
azimuths S3A: a1 = 76.98, a2 = 102.72
azimuths S3A: a1 = 77.09, a2 = 102.81
azimuths S3A: a1 = 77.09, a2 = 102.83
azimuths S3A: a1 = 77.09, a2 = 102.85
azimuths S3A: a1 = 77.09, a2 = 102.72
azimuths S3A: a1 = 77.09, a2 = 102.90
azimuths S3A: a1 = 77.09, a2 = 102.75
azimuths S3A: a1 = 77.09, a2 = 102.90
azimuths S3A: a1 = 77.09, a2 = 102.77
azimuths S3A: a1 = 77.09, a2 = 102.91
azimuths S3A: a1 = 77.09, a2 = 102.78
azimuths S3A: a1 = 76.92, a2 = 103.02
azimuths S3A: a1 = 76.92, a2 = 103.07
azimuths S3A: a1 = 76.92, a2 = 103.05
azimuths S3A: a1 = 76.92, a2 = 103.07
azimuths S3A: a1 = 76.92, a2 = 102.77
azimuths S3A: a1 = 76.92, a2 = 103.06
azimuths S3A: a1 = 76.92, a2 = 102.80
azimuths S3A: a1 = 76.92, a2 = 103.06
azimuths S3A: a1 = 76.92, a2 = 102.83
azimuths S3A: a1 = 76.92, a2 = 103.06
azimuths S3A: a1 = 76.92, a2 = 102.86
azimuths S3A: a1 = 76.92, a2 = 103.07
azimuths S3A: a1 = 77.30, a2 = 102.92
azimuths S3A: a1 = 77.30, a2 = 102.88
azimuths S3A: a1 = 77.30, a2 = 102.84
azimuths S3A: a1 = 77.30, a2 = 102.90
azimuths S3A: a1 = 77.30, a2 = 102.90
azimuths S3A: a1 = 77.30, a2 = 102.91
azimuths S3A: a1 = 77.30, a2 = 102.91
azimuths S3A: a1 = 77.13, a2 = 103.07
azimuths S3A: a1 = 77.13, a2 = 103.08
azimuths S3A: a1 = 77.13, a2 = 103.05
azimuths S3A: a1 = 77.13, a2 = 103.06
azimuths S3A: a1 = 77.13, a2 = 102.99
azimuths S3A: a1 = 77.13, a2 = 103.06
azimuths S3A: a1 = 77.13, a2 = 102.95
azimuths S3A: a1 = 77.13, a2 = 103.06
azimuths S3A: a1 = 77.13, a2 = 102.87
azimuths S3A: a1 = 77.13, a2 = 103.07
azimuths S3A: a1 = 77.14, a2 = 102.62
azimuths S3A: a1 = 77.49, a2 = 102.78
azimuths S3A: a1 = 77.40, a2 = 102.90
azimuths S3A: a1 = 77.40, a2 = 102.99
azimuths S3A: a1 = 77.40, a2 = 102.95
azimuths S3A: a1 = 77.40, a2 = 102.87
azimuths S3A: a1 = 77.40, a2 = 102.88
azimuths S3A: a1 = 77.32, a2 = 102.67
azimuths S3A: a1 = 77.32, a2 = 102.68
azimuths S3A: a1 = 77.32, a2 = 102.63
azimuths S3A: a1 = 77.32, a2 = 102.64
azimuths S3A: a1 = 77.25, a2 = 102.70
azimuths S3A: a1 = 77.25, a2 = 102.71
azimuths S3A: a1 = 77.25, a2 = 102.72
azimuths S3A: a1 = 77.25, a2 = 102.54
azimuths S3A: a1 = 77.25, a2 = 102.75
azimuths S3A: a1 = 77.25, a2 = 102.56
azimuths S3A: a1 = 76.93, a2 = 102.73
azimuths S3A: a1 = 76.93, a2 = 102.74
azimuths S3A: a1 = 76.93, a2 = 103.05
azimuths S3A: a1 = 76.93, a2 = 102.76
azimuths S3A: a1 = 76.93, a2 = 102.77
azimuths S3A: a1 = 76.93, a2 = 102.68
azimuths S3A: a1 = 76.93, a2 = 102.80
azimuths S3A: a1 = 76.93, a2 = 102.70
azimuths S3A: a1 = 76.93, a2 = 102.83
azimuths S3A: a1 = 76.93, a2 = 102.72
azimuths S3A: a1 = 76.93, a2 = 102.86
```

```
azimuths S3A: a1 = 77.09, a2 = 102.81
azimuths S3A: a1 = 77.09, a2 = 102.83
azimuths S3A: a1 = 77.09, a2 = 102.85
azimuths S3A: a1 = 77.09, a2 = 102.90
azimuths S3A: a1 = 77.09, a2 = 102.75
azimuths S3A: a1 = 77.09, a2 = 102.90
azimuths S3A: a1 = 77.09, a2 = 102.77
azimuths S3A: a1 = 77.09, a2 = 102.91
azimuths S3A: a1 = 77.09, a2 = 102.78
azimuths S3A: a1 = 77.09, a2 = 102.91
azimuths S3A: a1 = 76.92, a2 = 103.02
azimuths S3A: a1 = 76.92, a2 = 103.07
azimuths S3A: a1 = 76.92, a2 = 103.05
azimuths S3A: a1 = 76.92, a2 = 103.08
azimuths S3A: a1 = 76.92, a2 = 103.07
azimuths S3A: a1 = 76.92, a2 = 103.06
azimuths S3A: a1 = 76.92, a2 = 102.80
azimuths S3A: a1 = 76.92, a2 = 103.06
azimuths S3A: a1 = 76.92, a2 = 102.83
azimuths S3A: a1 = 76.92, a2 = 103.06
azimuths S3A: a1 = 76.92, a2 = 102.86
azimuths S3A: a1 = 76.92, a2 = 103.07
azimuths S3B: a1 = 77.30, a2 = 102.91
azimuths S3B: a1 = 77.30, a2 = 102.93
azimuths S3B: a1 = 77.30, a2 = 102.84
azimuths S3B: a1 = 77.30, a2 = 102.80
azimuths S3B: a1 = 77.30, a2 = 102.91
azimuths S3B: a1 = 77.30, a2 = 102.90
azimuths S3B: a1 = 77.30, a2 = 102.91
azimuths S3B: a1 = 77.13, a2 = 103.07
azimuths S3B: a1 = 77.13, a2 = 103.08
azimuths S3B: a1 = 77.13, a2 = 103.08
azimuths S3B: a1 = 77.13, a2 = 103.00
azimuths S3B: a1 = 77.13, a2 = 103.07
azimuths S3B: a1 = 77.13, a2 = 102.97
azimuths S3B: a1 = 77.13, a2 = 103.07
azimuths S3B: a1 = 77.13, a2 = 102.95
azimuths S3B: a1 = 77.13, a2 = 103.07
azimuths S3B: a1 = 77.13, a2 = 102.87
azimuths S3B: a1 = 77.15, a2 = 102.62
azimuths S3B: a1 = 77.41, a2 = 102.88
azimuths S3B: a1 = 77.41, a2 = 102.62
azimuths S3B: a1 = 77.41, a2 = 102.97
azimuths S3B: a1 = 77.41, a2 = 102.95
azimuths S3B: a1 = 77.41, a2 = 102.87
azimuths S3B: a1 = 77.32, a2 = 102.67
azimuths S3B: a1 = 77.32, a2 = 102.68
azimuths S3B: a1 = 77.32, a2 = 102.64
azimuths S3B: a1 = 77.32, a2 = 102.64
azimuths S3B: a1 = 77.32, a2 = 102.65
azimuths S3B: a1 = 77.23, a2 = 102.66
azimuths S3B: a1 = 77.23, a2 = 102.70
azimuths S3B: a1 = 77.23, a2 = 102.72
azimuths S3B: a1 = 77.23, a2 = 102.73
azimuths S3B: a1 = 77.23, a2 = 102.55
azimuths S3B: a1 = 77.23, a2 = 102.75
azimuths S3B: a1 = 77.23, a2 = 102.57
azimuths S3B: a1 = 76.92, a2 = 102.73
azimuths S3B: a1 = 76.92, a2 = 103.00
azimuths S3B: a1 = 76.92, a2 = 102.74
azimuths S3B: a1 = 76.92, a2 = 102.74
azimuths S3B: a1 = 76.92, a2 = 103.06
azimuths S3B: a1 = 76.92, a2 = 102.77
azimuths S3B: a1 = 76.92, a2 = 102.79
azimuths S3B: a1 = 76.92, a2 = 102.69
azimuths S3B: a1 = 76.92, a2 = 102.80
azimuths S3B: a1 = 76.92, a2 = 102.72
azimuths S3B: a1 = 76.92, a2 = 102.86
azimuths S3B: a1 = 77.07, a2 = 102.80
azimuths S3B: a1 = 77.07, a2 = 102.91
azimuths S3B: a1 = 77.07, a2 = 102.82
azimuths S3B: a1 = 77.07, a2 = 102.84
azimuths S3B: a1 = 77.07, a2 = 102.89
azimuths S3B: a1 = 77.07, a2 = 102.91
azimuths S3B: a1 = 77.07, a2 = 102.75
azimuths S3B: a1 = 77.07, a2 = 102.90
azimuths S3B: a1 = 77.07, a2 = 102.78
azimuths S3B: a1 = 77.07, a2 = 102.91
azimuths S3B: a1 = 76.95, a2 = 103.00
azimuths S3B: a1 = 76.95, a2 = 103.07
azimuths S3B: a1 = 76.95, a2 = 103.04
azimuths S3B: a1 = 76.95, a2 = 103.08
azimuths S3B: a1 = 76.95, a2 = 103.06
azimuths S3B: a1 = 76.95, a2 = 103.08
azimuths S3B: a1 = 76.95, a2 = 103.07
azimuths S3B: a1 = 76.95, a2 = 103.07
azimuths S3B: a1 = 76.95, a2 = 102.80
azimuths S3B: a1 = 76.95, a2 = 103.07
azimuths S3B: a1 = 76.95, a2 = 102.86
azimuths S3B: a1 = 76.95, a2 = 103.07
azimuths S3B: a1 = 77.28, a2 = 102.91
azimuths S3B: a1 = 77.28, a2 = 102.93
azimuths S3B: a1 = 77.28, a2 = 102.84
azimuths S3B: a1 = 77.28, a2 = 102.80
azimuths S3B: a1 = 77.28, a2 = 102.78
azimuths S3B: a1 = 77.28, a2 = 102.90
azimuths S3B: a1 = 77.28, a2 = 102.91
azimuths S3B: a1 = 77.09, a2 = 103.07
azimuths S3B: a1 = 77.09, a2 = 102.88
azimuths S3B: a1 = 77.09, a2 = 103.08
azimuths S3B: a1 = 77.09, a2 = 103.08
azimuths S3B: a1 = 77.09, a2 = 103.00
azimuths S3B: a1 = 77.09, a2 = 102.97
azimuths S3B: a1 = 77.09, a2 = 103.07
azimuths S3B: a1 = 77.09, a2 = 102.95
azimuths S3B: a1 = 77.09, a2 = 103.07
azimuths S3B: a1 = 77.09, a2 = 102.87
azimuths S3B: a1 = 77.16, a2 = 102.62
azimuths S3B: a1 = 77.16, a2 = 102.62
azimuths S3B: a1 = 77.43, a2 = 102.88
azimuths S3B: a1 = 77.43, a2 = 102.62
azimuths S3B: a1 = 77.43, a2 = 102.95
azimuths S3B: a1 = 77.43, a2 = 102.87
azimuths S3B: a1 = 77.28, a2 = 102.66
azimuths S3B: a1 = 77.28, a2 = 102.67
azimuths S3B: a1 = 77.28, a2 = 102.68
azimuths S3B: a1 = 77.28, a2 = 102.69
```

```
azimuths S3B: a1 = 77.28, a2 = 102.64
azimuths S3B: a1 = 77.28, a2 = 102.65
azimuths S3B: a1 = 77.19, a2 = 102.60
azimuths S3B: a1 = 77.19, a2 = 102.80
azimuths S3B: a1 = 77.19, a2 = 102.66
azimuths S3B: a1 = 77.19, a2 = 102.70
azimuths S3B: a1 = 77.19, a2 = 102.72
azimuths S3B: a1 = 77.19, a2 = 102.73
azimuths S3B: a1 = 77.19, a2 = 102.75
azimuths S3B: a1 = 77.19, a2 = 102.57
azimuths S3B: a1 = 77.19, a2 = 102.78
azimuths S3B: a1 = 76.94, a2 = 102.73
azimuths S3B: a1 = 76.94, a2 = 103.00
azimuths S3B: a1 = 76.94, a2 = 102.74
azimuths S3B: a1 = 76.94, a2 = 103.04
azimuths S3B: a1 = 76.94, a2 = 102.74
azimuths S3B: a1 = 76.94, a2 = 103.06
azimuths S3B: a1 = 76.94, a2 = 102.77
azimuths S3B: a1 = 76.94, a2 = 103.07
azimuths S3B: a1 = 76.94, a2 = 102.79
azimuths S3B: a1 = 76.94, a2 = 102.80
azimuths S3B: a1 = 76.94, a2 = 102.72
azimuths S3B: a1 = 76.94, a2 = 102.86
azimuths S3B: a1 = 77.11, a2 = 102.80
azimuths S3B: a1 = 77.11, a2 = 102.91
azimuths S3B: a1 = 77.11, a2 = 102.82
azimuths S3B: a1 = 77.11, a2 = 102.93
azimuths S3B: a1 = 77.11, a2 = 102.84
azimuths S3B: a1 = 77.11, a2 = 102.89
azimuths S3B: a1 = 77.11, a2 = 102.91
azimuths S3B: a1 = 77.11, a2 = 102.90
azimuths S3B: a1 = 77.11, a2 = 102.78
azimuths S3B: a1 = 77.11, a2 = 102.91
azimuths S3B: a1 = 77.00, a2 = 103.00
azimuths S3B: a1 = 77.00, a2 = 103.07
azimuths S3B: a1 = 77.00, a2 = 103.04
azimuths S3B: a1 = 77.00, a2 = 103.08
azimuths S3B: a1 = 77.00, a2 = 103.06
azimuths S3B: a1 = 77.00, a2 = 103.08
azimuths S3B: a1 = 77.00, a2 = 103.07
azimuths S3B: a1 = 77.00, a2 = 103.07
azimuths S3B: a1 = 77.00, a2 = 103.07
azimuths S3B: a1 = 77.00, a2 = 102.86
azimuths S3B: a1 = 77.00, a2 = 103.07
azimuths S3B: a1 = 77.29, a2 = 102.91
azimuths S3B: a1 = 77.29, a2 = 102.93
azimuths S3B: a1 = 77.29, a2 = 102.84
azimuths S3B: a1 = 77.29, a2 = 102.80
azimuths S3B: a1 = 77.29, a2 = 102.78
azimuths S3B: a1 = 77.29, a2 = 102.83
azimuths S3B: a1 = 77.29, a2 = 102.91
azimuths S3B: a1 = 77.20, a2 = 103.07
azimuths S3B: a1 = 77.20, a2 = 102.88
azimuths S3B: a1 = 77.20, a2 = 103.08
azimuths S3B: a1 = 77.20, a2 = 103.08
azimuths S3B: a1 = 77.20, a2 = 103.00
azimuths S3B: a1 = 77.20, a2 = 102.97
azimuths S3B: a1 = 77.20, a2 = 102.95
azimuths S3B: a1 = 77.20, a2 = 103.07
azimuths S3B: a1 = 77.20, a2 = 102.87
azimuths S3B: a1 = 77.36, a2 = 102.64
azimuths S3B: a1 = 77.46, a2 = 102.88
azimuths S3B: a1 = 77.46, a2 = 102.62
azimuths S3B: a1 = 77.46, a2 = 102.54
azimuths S3B: a1 = 77.46, a2 = 102.87
azimuths S3B: a1 = 77.27, a2 = 102.73
azimuths S3B: a1 = 77.27, a2 = 102.66
azimuths S3B: a1 = 77.27, a2 = 102.67
azimuths S3B: a1 = 77.27, a2 = 102.67
azimuths S3B: a1 = 77.27, a2 = 102.68
azimuths S3B: a1 = 77.27, a2 = 102.69
azimuths S3B: a1 = 77.27, a2 = 102.72
azimuths S3B: a1 = 77.27, a2 = 102.65
azimuths S3B: a1 = 77.17, a2 = 102.60
azimuths S3B: a1 = 77.17, a2 = 102.80
azimuths S3B: a1 = 77.17, a2 = 102.66
azimuths S3B: a1 = 77.17, a2 = 102.82
azimuths S3B: a1 = 77.17, a2 = 102.70
azimuths S3B: a1 = 77.17, a2 = 102.72
azimuths S3B: a1 = 77.17, a2 = 102.73
azimuths S3B: a1 = 77.17, a2 = 102.75
azimuths S3B: a1 = 77.17, a2 = 102.78
azimuths S3B: a1 = 76.93, a2 = 102.73
azimuths S3B: a1 = 76.93, a2 = 103.00
azimuths S3B: a1 = 76.93, a2 = 102.74
azimuths S3B: a1 = 76.93, a2 = 103.04
azimuths S3B: a1 = 76.93, a2 = 102.74
azimuths S3B: a1 = 76.93, a2 = 103.06
azimuths S3B: a1 = 76.93, a2 = 102.77
azimuths S3B: a1 = 76.93, a2 = 103.07
azimuths S3B: a1 = 76.93, a2 = 102.79
azimuths S3B: a1 = 76.93, a2 = 103.07
azimuths S3B: a1 = 76.93, a2 = 102.80
azimuths S3B: a1 = 76.93, a2 = 102.86
azimuths S3B: a1 = 77.15, a2 = 102.80
azimuths S3B: a1 = 77.15, a2 = 102.91
azimuths S3B: a1 = 77.15, a2 = 102.82
azimuths S3B: a1 = 77.15, a2 = 102.93
azimuths S3B: a1 = 77.15, a2 = 102.84
azimuths S3B: a1 = 77.15, a2 = 102.89
azimuths S3B: a1 = 77.15, a2 = 102.91
azimuths S3B: a1 = 77.15, a2 = 102.90
azimuths S3B: a1 = 77.15, a2 = 102.91
azimuths S3B: a1 = 77.01, a2 = 103.00
azimuths S3B: a1 = 77.01, a2 = 103.07
azimuths S3B: a1 = 77.01, a2 = 103.04
azimuths S3B: a1 = 77.01, a2 = 103.08
azimuths S3B: a1 = 77.01, a2 = 103.06
azimuths S3B: a1 = 77.01, a2 = 103.08
azimuths S3B: a1 = 77.01, a2 = 103.07
azimuths S3B: a1 = 77.01, a2 = 103.00
azimuths S3B: a1 = 77.01, a2 = 103.07
azimuths S3B: a1 = 77.01, a2 = 103.07
azimuths S3B: a1 = 77.01, a2 = 103.07
azimuths S3B: a1 = 77.33, a2 = 102.91
azimuths S3B: a1 = 77.33, a2 = 102.93
azimuths S3B: a1 = 77.33, a2 = 102.84
```

```
azimuths S3B: a1 = 77.33, a2 = 102.80
azimuths S3B: a1 = 77.33, a2 = 102.78
azimuths S3B: a1 = 77.33, a2 = 102.83
azimuths S3B: a1 = 77.26, a2 = 103.07
azimuths S3B: a1 = 77.26, a2 = 102.88
azimuths S3B: a1 = 77.26, a2 = 103.08
azimuths S3B: a1 = 77.26, a2 = 103.08
azimuths S3B: a1 = 77.26, a2 = 103.00
azimuths S3B: a1 = 77.26, a2 = 102.97
azimuths S3B: a1 = 77.26, a2 = 102.95
azimuths S3B: a1 = 77.26, a2 = 102.87
azimuths S3B: a1 = 77.36, a2 = 102.64
azimuths S3B: a1 = 77.46, a2 = 102.88
azimuths S3B: a1 = 77.46, a2 = 102.62
azimuths S3B: a1 = 77.46, a2 = 102.54
azimuths S3B: a1 = 77.26, a2 = 102.73
azimuths S3B: a1 = 77.26, a2 = 102.66
azimuths S3B: a1 = 77.26, a2 = 102.74
azimuths S3B: a1 = 77.26, a2 = 102.67
azimuths S3B: a1 = 77.26, a2 = 102.67
azimuths S3B: a1 = 77.26, a2 = 102.68
azimuths S3B: a1 = 77.26, a2 = 102.69
azimuths S3B: a1 = 77.10, a2 = 102.80
azimuths S3B: a1 = 77.10, a2 = 102.66
azimuths S3B: a1 = 77.10, a2 = 102.82
azimuths S3B: a1 = 77.10, a2 = 102.70
azimuths S3B: a1 = 77.10, a2 = 102.84
azimuths S3B: a1 = 77.10, a2 = 102.72
azimuths S3B: a1 = 77.10, a2 = 102.89
azimuths S3B: a1 = 77.10, a2 = 102.73
azimuths S3B: a1 = 77.10, a2 = 102.75
azimuths S3B: a1 = 77.10, a2 = 102.78
azimuths S3B: a1 = 76.93, a2 = 103.00
azimuths S3B: a1 = 76.93, a2 = 102.74
azimuths S3B: a1 = 76.93, a2 = 103.04
azimuths S3B: a1 = 76.93, a2 = 102.74
azimuths S3B: a1 = 76.93, a2 = 103.06
azimuths S3B: a1 = 76.93, a2 = 102.77
azimuths S3B: a1 = 76.93, a2 = 103.07
azimuths S3B: a1 = 76.93, a2 = 102.79
azimuths S3B: a1 = 76.93, a2 = 103.07
azimuths S3B: a1 = 76.93, a2 = 102.80
azimuths S3B: a1 = 76.93, a2 = 103.07
azimuths S3B: a1 = 76.93, a2 = 102.86
azimuths S3B: a1 = 77.17, a2 = 102.91
azimuths S3B: a1 = 77.17, a2 = 102.82
azimuths S3B: a1 = 77.17, a2 = 102.93
azimuths S3B: a1 = 77.17, a2 = 102.84
azimuths S3B: a1 = 77.17, a2 = 102.84
azimuths S3B: a1 = 77.17, a2 = 102.89
azimuths S3B: a1 = 77.17, a2 = 102.91
azimuths S3B: a1 = 77.17, a2 = 102.90
azimuths S3B: a1 = 77.17, a2 = 102.91
azimuths S3B: a1 = 77.04, a2 = 103.07
azimuths S3B: a1 = 77.04, a2 = 103.04
azimuths S3B: a1 = 77.04, a2 = 103.08
azimuths S3B: a1 = 77.04, a2 = 103.06
azimuths S3B: a1 = 77.04, a2 = 103.08
azimuths S3B: a1 = 77.04, a2 = 103.07
azimuths S3B: a1 = 77.04, a2 = 103.00
azimuths S3B: a1 = 77.04, a2 = 103.07
azimuths S3B: a1 = 77.04, a2 = 102.97
azimuths S3B: a1 = 77.04, a2 = 103.07
azimuths S3B: a1 = 77.04, a2 = 103.07
azimuths S3B: a1 = 77.37, a2 = 102.93
azimuths S3B: a1 = 77.37, a2 = 102.80
azimuths S3B: a1 = 77.37, a2 = 102.78
azimuths S3B: a1 = 77.37, a2 = 102.83
azimuths S3B: a1 = 77.31, a2 = 102.88
azimuths S3B: a1 = 77.31, a2 = 103.08
azimuths S3B: a1 = 77.31, a2 = 103.08
azimuths S3B: a1 = 77.31, a2 = 103.00
azimuths S3B: a1 = 77.31, a2 = 102.97
azimuths S3B: a1 = 77.31, a2 = 102.95
azimuths S3B: a1 = 77.31, a2 = 102.87
azimuths S3B: a1 = 77.35, a2 = 102.61
azimuths S3B: a1 = 77.35, a2 = 102.65
azimuths S3B: a1 = 77.30, a2 = 102.62
azimuths S3B: a1 = 77.30, a2 = 102.54
azimuths S3B: a1 = 77.30, a2 = 102.54
azimuths S3B: a1 = 77.30, a2 = 102.55
azimuths S3B: a1 = 77.23, a2 = 102.74
azimuths S3B: a1 = 77.23, a2 = 102.67
azimuths S3B: a1 = 77.23, a2 = 102.74
azimuths S3B: a1 = 77.23, a2 = 102.67
azimuths S3B: a1 = 77.23, a2 = 102.68
azimuths S3B: a1 = 77.23, a2 = 102.69
azimuths S3B: a1 = 77.23, a2 = 102.72
azimuths S3B: a1 = 77.10, a2 = 102.80
azimuths S3B: a1 = 77.10, a2 = 102.82
azimuths S3B: a1 = 77.10, a2 = 102.70
azimuths S3B: a1 = 77.10, a2 = 102.84
azimuths S3B: a1 = 77.10, a2 = 102.72
azimuths S3B: a1 = 77.10, a2 = 102.89
azimuths S3B: a1 = 77.10, a2 = 102.73
azimuths S3B: a1 = 77.10, a2 = 102.91
azimuths S3B: a1 = 77.10, a2 = 102.75
azimuths S3B: a1 = 77.10, a2 = 102.78
azimuths S3B: a1 = 76.93, a2 = 103.00
azimuths S3B: a1 = 76.93, a2 = 103.04
azimuths S3B: a1 = 76.93, a2 = 102.74
azimuths S3B: a1 = 76.93, a2 = 103.06
azimuths S3B: a1 = 76.93, a2 = 102.77
azimuths S3B: a1 = 76.93, a2 = 103.07
azimuths S3B: a1 = 76.93, a2 = 102.79
azimuths S3B: a1 = 76.93, a2 = 103.07
azimuths S3B: a1 = 76.93, a2 = 102.80
azimuths S3B: a1 = 76.93, a2 = 103.07
azimuths S3B: a1 = 76.93, a2 = 102.86
azimuths S3B: a1 = 76.93, a2 = 103.07
azimuths S3B: a1 = 77.21, a2 = 102.93
azimuths S3B: a1 = 77.21, a2 = 102.84
azimuths S3B: a1 = 77.21, a2 = 102.84
azimuths S3B: a1 = 77.21, a2 = 102.89
azimuths S3B: a1 = 77.21, a2 = 102.80
azimuths S3B: a1 = 77.21, a2 = 102.91
azimuths S3B: a1 = 77.21, a2 = 102.90
```

```
azimuths S3B: a1 = 77.21, a2 = 102.91
azimuths S3B: a1 = 77.10, a2 = 103.07
azimuths S3B: a1 = 77.10, a2 = 103.08
azimuths S3B: a1 = 77.10, a2 = 103.06
azimuths S3B: a1 = 77.10, a2 = 103.08
azimuths S3B: a1 = 77.10, a2 = 103.07
azimuths S3B: a1 = 77.10, a2 = 103.00
azimuths S3B: a1 = 77.10, a2 = 103.07
azimuths S3B: a1 = 77.10, a2 = 102.97
azimuths S3B: a1 = 77.10, a2 = 103.07
azimuths S3B: a1 = 77.10, a2 = 103.07
azimuths S3B: a1 = 77.46, a2 = 102.84
azimuths S3B: a1 = 77.46, a2 = 102.78
azimuths S3B: a1 = 77.34, a2 = 102.88
azimuths S3B: a1 = 77.34, a2 = 103.08
azimuths S3B: a1 = 77.34, a2 = 103.00
azimuths S3B: a1 = 77.34, a2 = 102.97
azimuths S3B: a1 = 77.34, a2 = 102.95
azimuths S3B: a1 = 77.34, a2 = 102.87
azimuths S3B: a1 = 77.33, a2 = 102.66
azimuths S3B: a1 = 77.33, a2 = 102.67
azimuths S3B: a1 = 77.33, a2 = 102.62
azimuths S3B: a1 = 77.28, a2 = 102.60
azimuths S3B: a1 = 77.28, a2 = 102.66
azimuths S3B: a1 = 77.28, a2 = 102.70
azimuths S3B: a1 = 77.28, a2 = 102.54
azimuths S3B: a1 = 77.28, a2 = 102.54
azimuths S3B: a1 = 77.28, a2 = 102.55
azimuths S3B: a1 = 77.28, a2 = 102.57
azimuths S3B: a1 = 77.00, a2 = 102.73
azimuths S3B: a1 = 77.00, a2 = 102.74
azimuths S3B: a1 = 77.00, a2 = 102.74
azimuths S3B: a1 = 77.00, a2 = 102.67
azimuths S3B: a1 = 77.00, a2 = 102.77
azimuths S3B: a1 = 77.00, a2 = 102.68
azimuths S3B: a1 = 77.00, a2 = 102.79
azimuths S3B: a1 = 77.00, a2 = 102.69
azimuths S3B: a1 = 77.00, a2 = 102.72
azimuths S3B: a1 = 77.10, a2 = 102.80
azimuths S3B: a1 = 77.10, a2 = 102.82
azimuths S3B: a1 = 77.10, a2 = 102.84
azimuths S3B: a1 = 77.10, a2 = 102.72
azimuths S3B: a1 = 77.10, a2 = 102.89
azimuths S3B: a1 = 77.10, a2 = 102.73
azimuths S3B: a1 = 77.10, a2 = 102.91
azimuths S3B: a1 = 77.10, a2 = 102.75
azimuths S3B: a1 = 77.10, a2 = 102.90
azimuths S3B: a1 = 77.10, a2 = 102.78
azimuths S3B: a1 = 76.93, a2 = 103.00
azimuths S3B: a1 = 76.93, a2 = 103.07
azimuths S3B: a1 = 76.93, a2 = 103.04
azimuths S3B: a1 = 76.93, a2 = 103.06
azimuths S3B: a1 = 76.93, a2 = 102.77
azimuths S3B: a1 = 76.93, a2 = 103.07
azimuths S3B: a1 = 76.93, a2 = 102.79
azimuths S3B: a1 = 76.93, a2 = 103.07
azimuths S3B: a1 = 76.93, a2 = 102.80
azimuths S3B: a1 = 76.93, a2 = 103.07
azimuths S3B: a1 = 76.93, a2 = 102.86
azimuths S3B: a1 = 76.93, a2 = 103.07
azimuths S3B: a1 = 77.30, a2 = 102.91
azimuths S3B: a1 = 77.30, a2 = 102.93
azimuths S3B: a1 = 77.30, a2 = 102.84
azimuths S3B: a1 = 77.30, a2 = 102.89
azimuths S3B: a1 = 77.30, a2 = 102.91
azimuths S3B: a1 = 77.30, a2 = 102.90
azimuths S3B: a1 = 77.30, a2 = 102.91
azimuths S3B: a1 = 77.13, a2 = 103.07
azimuths S3B: a1 = 77.13, a2 = 103.08
azimuths S3B: a1 = 77.13, a2 = 103.08
azimuths S3B: a1 = 77.13, a2 = 103.07
azimuths S3B: a1 = 77.13, a2 = 103.00
azimuths S3B: a1 = 77.13, a2 = 103.07
azimuths S3B: a1 = 77.13, a2 = 102.97
azimuths S3B: a1 = 77.13, a2 = 103.07
azimuths S3B: a1 = 77.13, a2 = 102.95
azimuths S3B: a1 = 77.13, a2 = 103.07
azimuths S3B: a1 = 77.49, a2 = 102.80
azimuths S3B: a1 = 77.49, a2 = 102.78
azimuths S3B: a1 = 77.49, a2 = 102.83
azimuths S3B: a1 = 77.39, a2 = 102.88
azimuths S3B: a1 = 77.39, a2 = 103.00
azimuths S3B: a1 = 77.39, a2 = 102.97
azimuths S3B: a1 = 77.39, a2 = 102.95
azimuths S3B: a1 = 77.39, a2 = 102.87
azimuths S3B: a1 = 77.32, a2 = 102.67
azimuths S3B: a1 = 77.32, a2 = 102.67
azimuths S3B: a1 = 77.32, a2 = 102.62
azimuths S3B: a1 = 77.32, a2 = 102.64
azimuths S3B: a1 = 77.26, a2 = 102.66
azimuths S3B: a1 = 77.26, a2 = 102.70
azimuths S3B: a1 = 77.26, a2 = 102.72
azimuths S3B: a1 = 77.26, a2 = 102.54
azimuths S3B: a1 = 77.26, a2 = 102.73
azimuths S3B: a1 = 77.26, a2 = 102.55
azimuths S3B: a1 = 76.95, a2 = 102.73
azimuths S3B: a1 = 76.95, a2 = 102.74
azimuths S3B: a1 = 76.95, a2 = 103.04
azimuths S3B: a1 = 76.95, a2 = 102.74
azimuths S3B: a1 = 76.95, a2 = 102.77
azimuths S3B: a1 = 76.95, a2 = 102.68
azimuths S3B: a1 = 76.95, a2 = 102.79
azimuths S3B: a1 = 76.95, a2 = 102.69
azimuths S3B: a1 = 76.95, a2 = 102.80
azimuths S3B: a1 = 76.95, a2 = 102.72
azimuths S3B: a1 = 76.95, a2 = 102.86
azimuths S3B: a1 = 77.09, a2 = 102.80
azimuths S3B: a1 = 77.09, a2 = 102.82
azimuths S3B: a1 = 77.09, a2 = 102.84
azimuths S3B: a1 = 77.09, a2 = 102.89
azimuths S3B: a1 = 77.09, a2 = 102.73
azimuths S3B: a1 = 77.09, a2 = 102.91
azimuths S3B: a1 = 77.09, a2 = 102.75
azimuths S3B: a1 = 77.09, a2 = 102.90
azimuths S3B: a1 = 77.09, a2 = 102.78
azimuths S3B: a1 = 77.09, a2 = 102.91
azimuths S3B: a1 = 76.92, a2 = 103.00
```

```
azimuths S3B: a1 = 76.92, a2 = 103.07
azimuths S3B: a1 = 76.92, a2 = 103.04
azimuths S3B: a1 = 76.92, a2 = 103.08
azimuths S3B: a1 = 76.92, a2 = 103.06
azimuths S3B: a1 = 76.92, a2 = 103.07
azimuths S3B: a1 = 76.92, a2 = 102.79
azimuths S3B: a1 = 76.92, a2 = 103.07
azimuths S3B: a1 = 76.92, a2 = 102.80
azimuths S3B: a1 = 76.92, a2 = 103.07
azimuths S3B: a1 = 76.92, a2 = 102.86
azimuths S3B: a1 = 76.92, a2 = 103.07
azimuths TPN+J1N+J2N+J3N: a1 = 69.97, a2 = 111.08
azimuths TPN+J1N+J2N+J3N: a1 = 69.97, a2 = 111.31
azimuths TPN+J1N+J2N+J3N: a1 = 69.72, a2 = 110.67
azimuths TPN+J1N+J2N+J3N: a1 = 69.72, a2 = 110.83
azimuths TPN+J1N+J2N+J3N: a1 = 69.72, a2 = 111.18
azimuths TPN+J1N+J2N+J3N: a1 = 69.19, a2 = 110.70
azimuths TPN+J1N+J2N+J3N: a1 = 69.19, a2 = 111.08
azimuths TPN+J1N+J2N+J3N: a1 = 69.19, a2 = 111.25
azimuths TPN+J1N+J2N+J3N: a1 = 69.19, a2 = 111.28
azimuths TPN+J1N+J2N+J3N: a1 = 69.19, a2 = 111.31
azimuths TPN+J1N+J2N+J3N: a1 = 69.15, a2 = 110.67
azimuths TPN+J1N+J2N+J3N: a1 = 69.15, a2 = 110.21
azimuths TPN+J1N+J2N+J3N: a1 = 69.15, a2 = 110.83
azimuths TPN+J1N+J2N+J3N: a1 = 69.15, a2 = 110.33
azimuths TPN+J1N+J2N+J3N: a1 = 69.15, a2 = 110.51
azimuths TPN+J1N+J2N+J3N: a1 = 68.69, a2 = 110.70
azimuths TPN+J1N+J2N+J3N: a1 = 68.69, a2 = 110.22
azimuths TPN+J1N+J2N+J3N: a1 = 68.69, a2 = 111.25
azimuths TPN+J1N+J2N+J3N: a1 = 68.69, a2 = 110.34
azimuths TPN+J1N+J2N+J3N: a1 = 68.69, a2 = 111.28
azimuths TPN+J1N+J2N+J3N: a1 = 68.69, a2 = 110.44
azimuths TPN+J1N+J2N+J3N: a1 = 70.17, a2 = 111.08
azimuths TPN+J1N+J2N+J3N: a1 = 69.83, a2 = 110.15
azimuths TPN+J1N+J2N+J3N: a1 = 69.88, a2 = 110.83
azimuths TPN+J1N+J2N+J3N: a1 = 69.88, a2 = 111.18
azimuths TPN+J1N+J2N+J3N: a1 = 69.88, a2 = 111.12
azimuths TPN+J1N+J2N+J3N: a1 = 69.21, a2 = 111.08
azimuths TPN+J1N+J2N+J3N: a1 = 69.21, a2 = 111.25
azimuths TPN+J1N+J2N+J3N: a1 = 69.21, a2 = 110.78
azimuths TPN+J1N+J2N+J3N: a1 = 69.21, a2 = 111.28
azimuths TPN+J1N+J2N+J3N: a1 = 69.21, a2 = 111.31
azimuths TPN+J1N+J2N+J3N: a1 = 69.25, a2 = 110.67
azimuths TPN+J1N+J2N+J3N: a1 = 69.25, a2 = 110.83
azimuths TPN+J1N+J2N+J3N: a1 = 69.25, a2 = 110.33
azimuths TPN+J1N+J2N+J3N: a1 = 69.25, a2 = 111.18
azimuths TPN+J1N+J2N+J3N: a1 = 69.25, a2 = 110.51
azimuths TPN+J1N+J2N+J3N: a1 = 68.66, a2 = 110.70
azimuths TPN+J1N+J2N+J3N: a1 = 68.66, a2 = 111.25
azimuths TPN+J1N+J2N+J3N: a1 = 68.66, a2 = 110.34
azimuths TPN+J1N+J2N+J3N: a1 = 68.66, a2 = 111.28
azimuths TPN+J1N+J2N+J3N: a1 = 68.66, a2 = 110.44
azimuths TPN+J1N+J2N+J3N: a1 = 68.66, a2 = 111.31
azimuths TPN+J1N+J2N+J3N: a1 = 69.70, a2 = 110.21
azimuths TPN+J1N+J2N+J3N: a1 = 69.70, a2 = 110.78
azimuths TPN+J1N+J2N+J3N: a1 = 69.70, a2 = 109.87
azimuths TPN+J1N+J2N+J3N: a1 = 69.66, a2 = 110.22
azimuths TPN+J1N+J2N+J3N: a1 = 69.66, a2 = 110.07
azimuths TPN+J1N+J2N+J3N: a1 = 69.66, a2 = 110.15
azimuths TPN+J1N+J2N+J3N: a1 = 70.22, a2 = 111.18
azimuths TPN+J1N+J2N+J3N: a1 = 69.71, a2 = 111.08
azimuths TPN+J1N+J2N+J3N: a1 = 69.71, a2 = 111.28
azimuths TPN+J1N+J2N+J3N: a1 = 69.71, a2 = 111.31
azimuths TPN+J1N+J2N+J3N: a1 = 69.49, a2 = 110.67
azimuths TPN+J1N+J2N+J3N: a1 = 69.49, a2 = 110.83
azimuths TPN+J1N+J2N+J3N: a1 = 69.49, a2 = 111.18
azimuths TPN+J1N+J2N+J3N: a1 = 69.49, a2 = 110.51
azimuths TPN+J1N+J2N+J3N: a1 = 68.90, a2 = 110.70
azimuths TPN+J1N+J2N+J3N: a1 = 68.90, a2 = 111.25
azimuths TPN+J1N+J2N+J3N: a1 = 68.90, a2 = 111.28
azimuths TPN+J1N+J2N+J3N: a1 = 68.90, a2 = 110.44
azimuths TPN+J1N+J2N+J3N: a1 = 68.90, a2 = 111.31
azimuths TPN+J1N+J2N+J3N: a1 = 69.37, a2 = 110.21
azimuths TPN+J1N+J2N+J3N: a1 = 69.37, a2 = 110.33
azimuths TPN+J1N+J2N+J3N: a1 = 69.37, a2 = 110.51
azimuths TPN+J1N+J2N+J3N: a1 = 69.37, a2 = 109.87
azimuths TPN+J1N+J2N+J3N: a1 = 68.68, a2 = 110.22
azimuths TPN+J1N+J2N+J3N: a1 = 68.68, a2 = 111.25
azimuths TPN+J1N+J2N+J3N: a1 = 68.68, a2 = 110.34
azimuths TPN+J1N+J2N+J3N: a1 = 68.68, a2 = 110.44
azimuths TPN+J1N+J2N+J3N: a1 = 68.68, a2 = 110.15
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.74
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.83
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.82
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.81
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.86
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.95
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.98
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.78
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.89
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.52
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.55, a2 = 102.53
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.55, a2 = 102.78
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.55, a2 = 102.48
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.55, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.56
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.59
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.55
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.73
```

```
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.69
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.65
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.71
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.62
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.94
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.92
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.83
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.82
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.81
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.86
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.95
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.98
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.89
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.22, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.52, a2 = 102.53
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.52, a2 = 102.78
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.52, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.52, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.52, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.56
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.54
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.59
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.55
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.56
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.27, a2 = 102.59
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.27, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.27, a2 = 102.69
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.27, a2 = 102.65
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.27, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.27, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.27, a2 = 102.52
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.27, a2 = 102.71
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.27, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.27, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.27, a2 = 102.62
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.94
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.92
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.74
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.83
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.82
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.81
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.14, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.14, a2 = 102.86
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.14, a2 = 102.95
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.14, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.14, a2 = 102.98
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.14, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.14, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.14, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.14, a2 = 102.89
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.14, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.14, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.14, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.14, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.57, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.57, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.48, a2 = 102.86
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.48, a2 = 102.78
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.48, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.48, a2 = 102.89
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.48, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.48, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.43, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.43, a2 = 102.54
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.43, a2 = 102.58
```

```
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.32, a2 = 102.59
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.32, a2 = 102.47
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.32, a2 = 102.65
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.32, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.32, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.32, a2 = 102.48
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.32, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.32, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.32, a2 = 102.62
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.94
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.92
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.24, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.24, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.24, a2 = 102.83
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.24, a2 = 102.82
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.24, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.24, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.24, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.24, a2 = 102.71
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.24, a2 = 102.81
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.24, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.24, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.11, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.11, a2 = 102.94
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.11, a2 = 102.95
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.11, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.11, a2 = 102.98
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.11, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.11, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.11, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.11, a2 = 102.89
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.11, a2 = 102.92
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.11, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.11, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.11, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.11, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.47, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.47, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.47, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.47, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.47, a2 = 102.86
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.42, a2 = 102.86
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.42, a2 = 102.95
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.42, a2 = 102.78
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.42, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.42, a2 = 102.89
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.42, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.42, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.42, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.44, a2 = 102.56
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.44, a2 = 102.56
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.59
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.46
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.52
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.48
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.47
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.62
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.06, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.06, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.06, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.06, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.06, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.06, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.06, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.06, a2 = 102.59
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.06, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.06, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.17, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.17, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.17, a2 = 102.69
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.17, a2 = 102.83
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.17, a2 = 102.82
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.17, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.17, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.17, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.17, a2 = 102.71
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.17, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.17, a2 = 102.81
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.17, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.17, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.05, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.05, a2 = 102.94
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.05, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.05, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.05, a2 = 102.98
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.05, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.05, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.05, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.05, a2 = 102.92
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.05, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.05, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.05, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.05, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.05, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.74
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.83
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.81
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.86
```

```
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.86
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.95
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.78
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.89
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.45, a2 = 102.55
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.45, a2 = 102.54
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.59
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.46
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.44
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.48
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.47
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.48
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.33, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.33, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.33, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.33, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.33, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.33, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.33, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.33, a2 = 102.59
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.33, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.33, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.18, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.18, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.18, a2 = 102.69
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.18, a2 = 102.65
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.18, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.18, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.18, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.18, a2 = 102.71
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.18, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.18, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.18, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.18, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.94
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.98
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.92
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.74
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.83
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.82
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.81
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.86
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.95
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.98
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.78
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.89
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.24, a2 = 102.53
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.24, a2 = 102.54
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.57, a2 = 102.53
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.57, a2 = 102.48
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.57, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.59
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.55
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.69
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.65
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.71
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.62
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.94
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.98
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.92
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.96
```

```
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.40, a2 = 102.74
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.40, a2 = 102.83
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.40, a2 = 102.82
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.40, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.40, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.40, a2 = 102.81
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.40, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.40, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.23, a2 = 102.86
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.23, a2 = 102.95
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.23, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.23, a2 = 102.98
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.23, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.23, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.23, a2 = 102.89
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.23, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.23, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.23, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.23, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.23, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.27, a2 = 102.53
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.54, a2 = 102.53
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.54, a2 = 102.78
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.54, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.54, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.56
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.59
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.55
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.56
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.59
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.69
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.65
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.71
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.62
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.94
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.92
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.74
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.83
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.82
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.30, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.86
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.95
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.98
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.89
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.58, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.50, a2 = 102.86
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.50, a2 = 102.53
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.50, a2 = 102.78
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.50, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.50, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.50, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.55
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.54
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.41, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.28, a2 = 102.59
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.28, a2 = 102.47
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.28, a2 = 102.69
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.28, a2 = 102.65
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.28, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.28, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.28, a2 = 102.52
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.28, a2 = 102.71
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.28, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.28, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.28, a2 = 102.62
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.94
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.92
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.70
```

```
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.83
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.82
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.81
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.26, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.13, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.13, a2 = 102.94
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.13, a2 = 102.95
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.13, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.13, a2 = 102.98
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.13, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.13, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.13, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.13, a2 = 102.89
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.13, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.13, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.13, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.13, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.56, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.56, a2 = 102.74
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.56, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.44, a2 = 102.86
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.44, a2 = 102.95
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.44, a2 = 102.78
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.44, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.44, a2 = 102.89
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.44, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.44, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.42, a2 = 102.53
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.42, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.42, a2 = 102.55
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.59
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.46
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.48
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.35, a2 = 102.62
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.94
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.03, a2 = 102.67
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.69
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.83
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.82
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.73
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.71
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.81
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.21, a2 = 102.80
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.10, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.10, a2 = 102.94
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.10, a2 = 102.95
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.10, a2 = 102.77
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.10, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.10, a2 = 102.98
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.10, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.10, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.10, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.10, a2 = 102.92
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.10, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.10, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.10, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.10, a2 = 102.96
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.45, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.45, a2 = 102.74
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.45, a2 = 102.83
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.45, a2 = 102.70
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.45, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.45, a2 = 102.86
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.86
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.95
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.78
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.79
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.89
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.97
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.39, a2 = 102.76
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.45, a2 = 102.54
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.45, a2 = 102.56
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.59
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.47
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.46
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.44
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.52
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.48
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.47
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.38, a2 = 102.62
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.09, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.09, a2 = 102.63
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.09, a2 = 102.60
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.09, a2 = 102.58
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.09, a2 = 102.68
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.09, a2 = 102.64
azimuths ERS1+ERS2+ENV+SRL: a1 = 77.09, a2 = 102.64
```

```
1990  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.09, a2 = 102.63
1991  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.09, a2 = 102.59
1992  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.09, a2 = 102.67
1993  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.76
1994  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.73
1995  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.69
1996  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.65
1997  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.82
1998  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.80
1999  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.80
2000  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.73
2001  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.71
2002  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.67
2003  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.80
2004  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.19, a2 = 102.80
2005  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.97
2006  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.94
2007  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.77
2008  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.97
2009  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.70
2010  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.98
2011  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.97
2012  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.96
2013  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.96
2014  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.92
2015  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.97
2016  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.76
2017  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.97
2018  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.97
2019  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.96
2020  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.68
2021  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.74
2022  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.83
2023  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.82
2024  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.70
2025  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.60
2026  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.77
2027  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.81
2028  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.37, a2 = 102.86
2029  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.29, a2 = 102.86
2030  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.29, a2 = 102.95
2031  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.29, a2 = 102.97
2032  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.29, a2 = 102.98
2033  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.29, a2 = 102.78
2034  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.29, a2 = 102.79
2035  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.29, a2 = 102.89
2036  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.29, a2 = 102.97
2037  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.29, a2 = 102.97
2038  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.29, a2 = 102.79
2039  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.29, a2 = 102.76
2040  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.45, a2 = 102.55
2041  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.45, a2 = 102.54
2042  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.54, a2 = 102.44
2043  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.54, a2 = 102.53
2044  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.54, a2 = 102.47
2045  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.54, a2 = 102.48
2046  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.63
2047  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.63
2048  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.60
2049  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.58
2050  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.58
2051  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.64
2052  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.63
2053  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.59
2054  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.36, a2 = 102.58
2055  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.76
2056  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.73
2057  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.69
2058  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.65
2059  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.63
2060  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.80
2061  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.80
2062  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.73
2063  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.71
2064  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.67
2065  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.64
2066  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.20, a2 = 102.80
2067  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.97
2068  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.94
2069  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.77
2070  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.70
2071  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.98
2072  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.68
2073  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.97
2074  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.96
2075  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.96
2076  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.92
2077  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.76
2078  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.97
2079  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.70
2080  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.97
2081  azimuths ERS1+ERS2+ENV+SRL: a1 = 77.02, a2 = 102.96
2082  azimuths HY2A: a1 = 76.43, a2 = 103.79
2083  azimuths HY2A: a1 = 76.43, a2 = 103.77
2084  azimuths HY2A: a1 = 76.43, a2 = 103.81
2085  azimuths HY2A: a1 = 76.17, a2 = 103.83
2086  azimuths HY2A: a1 = 76.17, a2 = 103.95
2087  azimuths HY2A: a1 = 76.17, a2 = 103.93
2088  azimuths HY2A: a1 = 76.17, a2 = 103.95
2089  azimuths HY2A: a1 = 76.17, a2 = 103.96
2090  azimuths HY2A: a1 = 76.21, a2 = 103.79
2091  azimuths HY2A: a1 = 76.21, a2 = 103.68
2092  azimuths HY2A: a1 = 76.21, a2 = 103.77
2093  azimuths HY2A: a1 = 76.21, a2 = 103.63
2094  azimuths HY2A: a1 = 76.21, a2 = 103.71
2095  azimuths HY2A: a1 = 76.04, a2 = 103.95
2096  azimuths HY2A: a1 = 76.04, a2 = 103.72
2097  azimuths HY2A: a1 = 76.04, a2 = 103.95
2098  azimuths HY2A: a1 = 76.04, a2 = 103.67
2099  azimuths HY2A: a1 = 76.04, a2 = 103.96
2100  azimuths HY2A: a1 = 76.04, a2 = 103.94
2101  azimuths HY2A: a1 = 76.36, a2 = 103.55
2102  azimuths HY2A: a1 = 76.36, a2 = 103.63
2103  azimuths HY2A: a1 = 76.36, a2 = 103.42
2104  azimuths HY2A: a1 = 76.36, a2 = 103.58
2105  azimuths HY2A: a1 = 76.05, a2 = 103.60
2106  azimuths HY2A: a1 = 76.05, a2 = 103.67
```

```
2107  azimuths HY2A: a1 = 76.05, a2 = 103.57
2108  azimuths HY2A: a1 = 76.05, a2 = 103.94
2109  azimuths HY2A: a1 = 76.05, a2 = 103.63
2110  azimuths HY2A: a1 = 76.59, a2 = 103.43
2111  azimuths HY2A: a1 = 76.59, a2 = 103.75
2112  azimuths HY2A: a1 = 76.46, a2 = 103.54
2113  azimuths HY2A: a1 = 76.46, a2 = 103.50
2114  azimuths HY2A: a1 = 76.48, a2 = 103.84
2115  azimuths HY2A: a1 = 76.48, a2 = 103.56
2116  azimuths HY2A: a1 = 76.48, a2 = 103.81
2117  azimuths HY2A: a1 = 76.34, a2 = 103.83
2118  azimuths HY2A: a1 = 76.34, a2 = 103.93
2119  azimuths HY2A: a1 = 76.34, a2 = 103.75
2120  azimuths HY2A: a1 = 76.34, a2 = 103.96
2121  azimuths HY2A: a1 = 76.32, a2 = 103.79
2122  azimuths HY2A: a1 = 76.32, a2 = 103.77
2123  azimuths HY2A: a1 = 76.32, a2 = 103.81
2124  azimuths HY2A: a1 = 76.32, a2 = 103.71
2125  azimuths HY2A: a1 = 76.14, a2 = 103.95
2126  azimuths HY2A: a1 = 76.14, a2 = 103.93
2127  azimuths HY2A: a1 = 76.14, a2 = 103.95
2128  azimuths HY2A: a1 = 76.14, a2 = 103.96
2129  azimuths HY2A: a1 = 76.14, a2 = 103.94
2130  azimuths HY2A: a1 = 76.23, a2 = 103.68
2131  azimuths HY2A: a1 = 76.23, a2 = 103.77
2132  azimuths HY2A: a1 = 76.23, a2 = 103.63
2133  azimuths HY2A: a1 = 76.23, a2 = 103.71
2134  azimuths HY2A: a1 = 76.23, a2 = 103.58
2135  azimuths HY2A: a1 = 76.04, a2 = 103.95
2136  azimuths HY2A: a1 = 76.04, a2 = 103.72
2137  azimuths HY2A: a1 = 76.04, a2 = 103.95
2138  azimuths HY2A: a1 = 76.04, a2 = 103.67
2139  azimuths HY2A: a1 = 76.04, a2 = 103.94
2140  azimuths HY2A: a1 = 76.04, a2 = 103.63
2141  azimuths HY2A: a1 = 76.42, a2 = 103.39
2142  azimuths HY2A: a1 = 76.42, a2 = 103.55
2143  azimuths HY2A: a1 = 76.42, a2 = 103.42
2144  azimuths HY2A: a1 = 76.38, a2 = 103.54
2145  azimuths HY2A: a1 = 76.38, a2 = 103.60
2146  azimuths HY2A: a1 = 76.38, a2 = 103.57
2147  azimuths HY2A: a1 = 76.38, a2 = 103.63
2148  azimuths HY2A: a1 = 76.53, a2 = 103.83
2149  azimuths HY2A: a1 = 76.53, a2 = 103.75
2150  azimuths HY2A: a1 = 76.47, a2 = 103.51
2151  azimuths HY2A: a1 = 76.42, a2 = 103.79
2152  azimuths HY2A: a1 = 76.42, a2 = 103.56
2153  azimuths HY2A: a1 = 76.42, a2 = 103.81
2154  azimuths HY2A: a1 = 76.25, a2 = 103.83
2155  azimuths HY2A: a1 = 76.25, a2 = 103.95
2156  azimuths HY2A: a1 = 76.25, a2 = 103.93
2157  azimuths HY2A: a1 = 76.25, a2 = 103.96
2158  azimuths HY2A: a1 = 76.24, a2 = 103.79
2159  azimuths HY2A: a1 = 76.24, a2 = 103.68
2160  azimuths HY2A: a1 = 76.24, a2 = 103.77
2161  azimuths HY2A: a1 = 76.24, a2 = 103.81
2162  azimuths HY2A: a1 = 76.24, a2 = 103.71
2163  azimuths HY2A: a1 = 76.08, a2 = 103.95
2164  azimuths HY2A: a1 = 76.08, a2 = 103.72
2165  azimuths HY2A: a1 = 76.08, a2 = 103.93
2166  azimuths HY2A: a1 = 76.08, a2 = 103.95
2167  azimuths HY2A: a1 = 76.08, a2 = 103.96
2168  azimuths HY2A: a1 = 76.08, a2 = 103.94
2169  azimuths HY2A: a1 = 76.29, a2 = 103.68
2170  azimuths HY2A: a1 = 76.29, a2 = 103.55
2171  azimuths HY2A: a1 = 76.29, a2 = 103.63
2172  azimuths HY2A: a1 = 76.29, a2 = 103.58
2173  azimuths HY2A: a1 = 76.06, a2 = 103.72
2174  azimuths HY2A: a1 = 76.06, a2 = 103.60
2175  azimuths HY2A: a1 = 76.06, a2 = 103.95
2176  azimuths HY2A: a1 = 76.06, a2 = 103.67
2177  azimuths HY2A: a1 = 76.06, a2 = 103.94
2178  azimuths HY2A: a1 = 76.06, a2 = 103.63
2179  azimuths HY2A: a1 = 76.62, a2 = 103.43
2180  azimuths HY2A: a1 = 76.41, a2 = 103.54
2181  azimuths HY2A: a1 = 76.41, a2 = 103.51
2182  azimuths HY2A: a1 = 76.41, a2 = 103.57
2183  azimuths HY2A: a1 = 76.62, a2 = 103.56
2184  azimuths HY2A: a1 = 76.46, a2 = 103.83
2185  azimuths HY2A: a1 = 76.46, a2 = 103.93
2186  azimuths HY2A: a1 = 76.46, a2 = 103.75
2187  azimuths HY2B: a1 = 76.43, a2 = 103.42
2188  azimuths HY2B: a1 = 76.43, a2 = 103.40
2189  azimuths HY2B: a1 = 76.43, a2 = 103.56
2190  azimuths HY2B: a1 = 76.38, a2 = 103.57
2191  azimuths HY2B: a1 = 76.38, a2 = 103.63
2192  azimuths HY2B: a1 = 76.38, a2 = 103.54
2193  azimuths HY2B: a1 = 76.38, a2 = 103.59
2194  azimuths HY2B: a1 = 76.53, a2 = 103.74
2195  azimuths HY2B: a1 = 76.53, a2 = 103.83
2196  azimuths HY2B: a1 = 76.48, a2 = 103.51
2197  azimuths HY2B: a1 = 76.42, a2 = 103.81
2198  azimuths HY2B: a1 = 76.42, a2 = 103.79
2199  azimuths HY2B: a1 = 76.42, a2 = 103.55
2200  azimuths HY2B: a1 = 76.26, a2 = 103.96
2201  azimuths HY2B: a1 = 76.26, a2 = 103.83
2202  azimuths HY2B: a1 = 76.26, a2 = 103.95
2203  azimuths HY2B: a1 = 76.26, a2 = 103.96
2204  azimuths HY2B: a1 = 76.25, a2 = 103.81
2205  azimuths HY2B: a1 = 76.25, a2 = 103.72
2206  azimuths HY2B: a1 = 76.25, a2 = 103.79
2207  azimuths HY2B: a1 = 76.25, a2 = 103.68
2208  azimuths HY2B: a1 = 76.25, a2 = 103.77
2209  azimuths HY2B: a1 = 76.07, a2 = 103.96
2210  azimuths HY2B: a1 = 76.07, a2 = 103.93
2211  azimuths HY2B: a1 = 76.07, a2 = 103.95
2212  azimuths HY2B: a1 = 76.07, a2 = 103.71
2213  azimuths HY2B: a1 = 76.07, a2 = 103.96
2214  azimuths HY2B: a1 = 76.07, a2 = 103.95
2215  azimuths HY2B: a1 = 76.30, a2 = 103.63
2216  azimuths HY2B: a1 = 76.30, a2 = 103.57
2217  azimuths HY2B: a1 = 76.30, a2 = 103.68
2218  azimuths HY2B: a1 = 76.30, a2 = 103.56
2219  azimuths HY2B: a1 = 76.06, a2 = 103.66
2220  azimuths HY2B: a1 = 76.06, a2 = 103.93
2221  azimuths HY2B: a1 = 76.06, a2 = 103.63
2222  azimuths HY2B: a1 = 76.06, a2 = 103.71
2223  azimuths HY2B: a1 = 76.06, a2 = 103.59
```

```
azimuths HY2B: a1 = 76.06, a2 = 103.95
azimuths HY2B: a1 = 76.62, a2 = 103.42
azimuths HY2B: a1 = 76.41, a2 = 103.57
azimuths HY2B: a1 = 76.41, a2 = 103.54
azimuths HY2B: a1 = 76.41, a2 = 103.51
azimuths HY2B: a1 = 76.62, a2 = 103.55
azimuths HY2B: a1 = 76.46, a2 = 103.74
azimuths HY2B: a1 = 76.46, a2 = 103.83
azimuths HY2B: a1 = 76.46, a2 = 103.96
azimuths HY2B: a1 = 76.44, a2 = 103.81
azimuths HY2B: a1 = 76.44, a2 = 103.79
azimuths HY2B: a1 = 76.44, a2 = 103.77
azimuths HY2B: a1 = 76.17, a2 = 103.96
azimuths HY2B: a1 = 76.17, a2 = 103.83
azimuths HY2B: a1 = 76.17, a2 = 103.95
azimuths HY2B: a1 = 76.17, a2 = 103.96
azimuths HY2B: a1 = 76.17, a2 = 103.95
azimuths HY2B: a1 = 76.22, a2 = 103.63
azimuths HY2B: a1 = 76.22, a2 = 103.72
azimuths HY2B: a1 = 76.22, a2 = 103.79
azimuths HY2B: a1 = 76.22, a2 = 103.68
azimuths HY2B: a1 = 76.22, a2 = 103.77
azimuths HY2B: a1 = 76.04, a2 = 103.66
azimuths HY2B: a1 = 76.04, a2 = 103.96
azimuths HY2B: a1 = 76.04, a2 = 103.93
azimuths HY2B: a1 = 76.04, a2 = 103.95
azimuths HY2B: a1 = 76.04, a2 = 103.71
azimuths HY2B: a1 = 76.04, a2 = 103.95
azimuths HY2B: a1 = 76.36, a2 = 103.63
azimuths HY2B: a1 = 76.36, a2 = 103.42
azimuths HY2B: a1 = 76.36, a2 = 103.57
azimuths HY2B: a1 = 76.36, a2 = 103.56
azimuths HY2B: a1 = 76.05, a2 = 103.66
azimuths HY2B: a1 = 76.05, a2 = 103.57
azimuths HY2B: a1 = 76.05, a2 = 103.93
azimuths HY2B: a1 = 76.05, a2 = 103.63
azimuths HY2B: a1 = 76.05, a2 = 103.59
azimuths HY2B: a1 = 76.59, a2 = 103.74
azimuths HY2B: a1 = 76.59, a2 = 103.42
azimuths HY2B: a1 = 76.46, a2 = 103.50
azimuths HY2B: a1 = 76.46, a2 = 103.54
azimuths HY2B: a1 = 76.48, a2 = 103.81
azimuths HY2B: a1 = 76.48, a2 = 103.85
azimuths HY2B: a1 = 76.48, a2 = 103.55
azimuths HY2B: a1 = 76.35, a2 = 103.74
azimuths HY2B: a1 = 76.35, a2 = 103.96
azimuths HY2B: a1 = 76.35, a2 = 103.83
azimuths HY2B: a1 = 76.35, a2 = 103.96
azimuths HY2B: a1 = 76.32, a2 = 103.81
azimuths HY2B: a1 = 76.32, a2 = 103.72
azimuths HY2B: a1 = 76.32, a2 = 103.79
azimuths HY2B: a1 = 76.32, a2 = 103.77
azimuths HY2B: a1 = 76.15, a2 = 103.96
azimuths HY2B: a1 = 76.15, a2 = 103.93
azimuths HY2B: a1 = 76.15, a2 = 103.95
azimuths HY2B: a1 = 76.15, a2 = 103.96
azimuths HY2B: a1 = 76.15, a2 = 103.95
azimuths HY2B: a1 = 76.23, a2 = 103.63
azimuths HY2B: a1 = 76.23, a2 = 103.72
azimuths HY2B: a1 = 76.23, a2 = 103.57
azimuths HY2B: a1 = 76.23, a2 = 103.68
azimuths HY2B: a1 = 76.23, a2 = 103.77
azimuths HY2B: a1 = 76.05, a2 = 103.66
azimuths HY2B: a1 = 76.05, a2 = 103.93
azimuths HY2B: a1 = 76.05, a2 = 103.63
azimuths HY2B: a1 = 76.05, a2 = 103.95
azimuths HY2B: a1 = 76.05, a2 = 103.71
azimuths HY2B: a1 = 76.05, a2 = 103.95

```

```python
import pandas as pd
import xarray as xr
from pathlib import Path
from namesm import sats_new
import matplotlib.pyplot as plt

msla_uv_dir = "/home/srinivasu/allData/aviso/msla_uv/"
out_dir = Path("/home/srinivasu/xtrackm/guv/msla_uv_at_intersections/")
out_dir.mkdir(parents=True, exist_ok=True)

ds_msla_u = xr.open_dataset(
    f"{msla_uv_dir}/allyear_dt_global_allsat_msla_u.nc",
    chunks={"time": 365, "lat": 50, "lon": 50}
)
ds_msla_v = xr.open_dataset(
    f"{msla_uv_dir}/allyear_dt_global_allsat_msla_v.nc",
    chunks={"time": 365, "lat": 50, "lon": 50}
)

u_msla = ds_msla_u["u"]
v_msla = ds_msla_v["v"]

df_all_list = [pd.read_csv(f"tracks_intersections_{sat}_1.csv") for sat in sats_new]
inter_df = pd.concat(df_all_list, ignore_index=True)

for i, r in inter_df.iterrows():
    sat = r["sat"]
    if sat != "TP+J1+J2+J3+S6A":
        continue
    track_number_self = str(r["track_self"])
    track_number_other = str(r["track_other"])
    lons_inter1 = r["lons_inter"]
    lats_inter1 = r["lats_inter"]
    unique_str = f"{sat}_{track_number_self}_{track_number_other}"
    print(unique_str)
    u_at = u_msla.sel(
        latitude=lats_inter1, longitude=lons_inter1, method="nearest"
    )
    v_at = v_msla.sel(
        latitude=lats_inter1, longitude=lons_inter1, method="nearest"
    )
    # u_at.plot()
    # plt.show()
    ds_out = xr.Dataset(
        {
            "u_at": u_at,
            "v_at": v_at,
        }
    )
    ds_out.to_netcdf(
        f"{out_dir}/{unique_str}_msla_uv_at_intersections.nc", format="NETCDF4"
    )

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
from tools_xtrackm import *

oscar_dir = "/home/srinivasu/allData/oscar_0.25/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
out_dir = f"/home/srinivasu/xtrackm/guv/oscar_at_intersections/"

ds_oscar_u = xr.open_dataset(f"{oscar_dir}/oscar_0.25_u_full_xr_concat.nc")
print(ds_oscar_u)
u_oscar = ds_oscar_u.u

ds_oscar_v = xr.open_dataset(f"{oscar_dir}/oscar_0.25_v_full_xr_concat.nc")
print(ds_oscar_v)
v_oscar = ds_oscar_v.v

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

track_self = df_all["track_self"].apply(lambda x: str(x)).values
track_other = df_all["track_other"].apply(lambda x: str(x)).values
lons_inter = df_all["lons_inter"].values
lats_inter = df_all["lats_inter"].values
x_from_coast_self = df_all["x_from_coast_self"].values
x_from_coast_other = df_all["x_from_coast_other"].values
angle_acute = df_all["angle_acute"].values
angle_obtuse = df_all["angle_obtuse"].values
sat = df_all["sat"].values

ln = len(track_self)

k = 0
for i in range(ln):
    track_number_self = track_self[i]
    track_number_other = track_other[i]
    lon1 = lons_inter[i]
    lat1 = lats_inter[i]
    sat1 = sat[i]
    if sat1 != "HY2B":
    # if sat1 != "S3B":
        continue
    k = k + 1
    print(f"{k} {track_number_self} {track_number_other}")
    # extrack oscar at lon1, lat1
    u_at = u_oscar.sel(lon=lon1, lat=lat1, method="nearest")
    v_at = v_oscar.sel(lon=lon1, lat=lat1, method="nearest")
    ds_out = xr.Dataset({"u": u_at, "v": v_at})
    filename = f"{out_dir}/oscar_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc"
    print(filename)
    ds_out.to_netcdf(filename)

```

guv/c.gc_at_intersection_points_loess_0.05.py

```python
import os

import xarray as xr
from tools_xtrackm import *

frac1 = "0.05"
method = "loess"
out_dir = f"/home/srinivasu/xtrackm/guv/guv_at_intersections_{method}/"
os.makedirs(out_dir, exist_ok=True)
print(sats_new)
sat = "ERS1+ERS2+ENV+SRL"
files = sorted(glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"))
for f in files:
    ds = xr.open_dataset(f, engine="netcdf4", decode_times=False)
    track_number = ds.Pass
    if len(ds.lat.values) < 3:
        continue
    f_smooth = f"../computed/sla_{method}_{frac1}/track_sla_along_{sat}_{track_number}_{method}_{frac1}.nc"
    ds_smooth = xr.open_dataset(f_smooth)
    sla = track_dist_time_asn(ds, var_str="sla", units_in="m")
    sla_smooth = ds_smooth.sla_smooth
    gc = compute_geostrophy_gc(ds, sla_smooth)
    gc = mad_filter_2d_xr(gc, 3.5, dims=("time", "x"))
    ds_out = xr.Dataset({"gc": gc})
    print(
        f"saving to {out_dir}/gc_at_{sat}_{track_number}_{method}_{frac1}.nc")
    ds_out.to_netcdf(
        f"{out_dir}/gc_at_{sat}_{track_number}_{method}_{frac1}.nc"
    )

```

```python
import math
import os

import pandas as pd
import xarray as xr
from tools_xtrackm import *

frac1 = "0.05"
method = "loess"
out_dir = f"/home/srinivasu/xtrackm/guv/guv_at_intersections_{method}/"
os.makedirs(out_dir, exist_ok=True)
print(sats_new)
sat = "ERS1+ERS2+ENV+SRL"
files = sorted(glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"))
for f in files:
    ds = xr.open_dataset(f, engine="netcdf4", decode_times=False)
    track_number = ds.Pass
    if len(ds.lat.values) == 0:
        continue
    f_smooth = f"../computed/sla_{method}_{frac1}/track_sla_along_{sat}_{track_number}_{method}_{frac1}.nc"
    ds_smooth = xr.open_dataset(f_smooth)
    sla = track_dist_time_asn(ds, var_str="sla", units_in="m")
    sla_smooth = ds_smooth.sla_smooth
    gc = compute_geostrophy_gc(ds, sla_smooth)
    # print nan stats before and after smoothing
    print(f"Track {track_number} - {sat}:")
    print(f"  gc (raw) - NaN count: {np.isnan(gc.values).sum()}, Total count: {gc.size}")
    # print nammin, nanmax, and mean before and after smoothing
    print(f"  gc (raw) - min: {np.nanmin(gc.values)}, max: {np.nanmax(gc.values)}, mean: {np.nanmean(gc.values)}")
    # print nanmedian and nanmad before and after smoothing
    print(f"  gc (raw) - nanmedian: {np.nanmedian(gc.values)}, nanmad: {np.nanmedian(np.abs(gc.values - np.nanmedian(gc.values)))}")
    gc_filt = mad_filter_2d_xr(gc, 3.5, dims=("time", "x"))
    print(f"  gc (filt) - NaN count: {np.isnan(gc_filt.values).sum()}, Total count: {gc_filt.size}")
    # print nammin, nanmax, and mean before and after smoothing
    print(f"  gc (filt) - min: {np.nanmin(gc_filt.values)}, max: {np.nanmax(gc_filt.values)}, mean: {np.nanmean(gc_filt.values)}")
    # print nanmedian and nanmad before and after smoothing
    print(f"  gc (filt) - nanmedian: {np.nanmedian(gc_filt.values)}, nanmad: {np.nanmedian(np.abs(gc_filt.values - np.nanmedian(gc_filt.values)))}")
    # plot gc before and after smoothing
    fig, ax = plt.subplots(1, 2, figsize=(12, 6))
    # gc.plot(ax=ax[0], label="gc (raw)", vmin=-3.5, vmax=3.5)
    # gc_filt.plot(ax=ax[1], label="gc (mad-filtered)", vmin=-3.5, vmax=3.5)
    gc.plot(ax=ax[0], label="gc (raw)", robust = True)
    gc_filt.plot(ax=ax[1], label="gc (mad-filtered)", robust = True)
    ax[0].set_title(f"Geostrophic Current (raw) - {sat} Track {track_number}")
    ax[1].set_title(f"Geostrophic Current (mad-filtered) - {sat} Track {track_number}")
    ax[0].set_xlabel("Time")
    ax[0].set_ylabel("Geostrophic Current (m/s)")
    ax[1].set_xlabel("Time")
    ax[1].set_ylabel("Geostrophic Current (m/s)")
    ax[0].legend()
    ax[1].legend()
    plt.tight_layout()
    plt.show()
    ds_out = xr.Dataset({"gc": gc_filt, "gc_raw": gc})
    print(f"saving to {out_dir}/gc_at_{sat}_{track_number}_{method}_{frac1}.nc")
    ds_out.to_netcdf(
        f"{out_dir}/gc_at_{sat}_{track_number}_{method}_{frac1}.nc"
    )
    break

```

```python
import math
import os

import pandas as pd
import xarray as xr
from tools_xtrackm import *

ioff = 3
frac1 = "12"
method = "box"
out_dir = f"/home/srinivasu/xtrackm/guv/guv_at_intersections_{method}/"
os.makedirs(out_dir, exist_ok=True)
print(sats_new)
sat = "TP+J1+J2+J3+S6A"
sats_new = [sat]
for sat in sats_new[:]:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    for i, r in df.iterrows():
        sat1 = r["sat"]
        track_number_self = str(r["track_self"])
        track_number_other = str(r["track_other"])
        lons_inter1 = r["lons_inter"]
        lats_inter1 = r["lats_inter"]
        x_from_coast_self1 = r["x_from_coast_self"]
        x_from_coast_other1 = r["x_from_coast_other"]
        angle_acute = r["angle_acute"]
        angle_obtuse = r["angle_obtuse"]
        lon1 = lons_inter1
        lat1 = lats_inter1
        if abs(lat1) < 2:
            continue
        # print(
        #     sat,
        #     track_number_self,
        #     track_number_other,
        #     lons_inter1,
        #     lats_inter1,
        #     x_from_coast_self1,
        #     x_from_coast_other1,
        #     angle_acute,
        #     angle_obtuse,
        # )
        f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
        f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
        f_self_smooth = f"../computed/sla_{method}_{frac1}/track_sla_along_{sat}_{track_number_self}_{method}_{frac1}.nc"
        f_other_smooth = f"../computed/sla_{method}_{frac1}/track_sla_along_{sat}_{track_number_other}_{method}_{frac1}.nc"
        ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
        ds_self_smooth = xr.open_dataset(f_self_smooth)
        sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
        sla_self_smooth = ds_self_smooth.sla_smooth
        ds_other = xr.open_dataset(
            f_other, engine="netcdf4", decode_times=False
        )
        ds_other_smooth = xr.open_dataset(f_other_smooth)
        sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
        sla_other_smooth = ds_other_smooth.sla_smooth
        lons_track_self = ds_self.lon.values
        lats_track_self = ds_self.lat.values
        lon_coast_self = lons_track_self[-1]  # this on coast
        lat_coast_self = lats_track_self[-1]  # this on coast
        lon_equat_self = lons_track_self[0]  # this on equator
        lat_equat_self = lats_track_self[0]  # this on equator
        lons_track_other = ds_other.lon.values
        lats_track_other = ds_other.lat.values
        lon_equat_other = lons_track_other[0]
        lat_equat_other = lats_track_other[0]
        lon_coast_other = lons_track_other[-1]
        lat_coast_other = lats_track_other[-1]
        ii_self = index_at_lat(ds_self, lats_inter1)
        a1 = local_angle(lons_track_self, lats_track_self, ii_self, ioff=ioff)
        a1_d = math.degrees(a1)
        ii_other = index_at_lat(ds_other, lats_inter1)
        a2 = local_angle(
            lons_track_other, lats_track_other, ii_other, ioff=ioff
        )
        a2_d = math.degrees(a2)
        print(f"heading angle {sat}: a1 = {a1_d:.2f}, a2 = {a2_d:.2f}")
        gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
        gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
        gc_self_at = gc_self.sel(
            x=x_from_coast_self1, method="nearest", drop=True
        )
        gc_other_at = gc_other.sel(
            x=x_from_coast_other1, method="nearest", drop=True
        )
        gc_other_at = gc_other_at.interp(time=gc_self_at.time)
        gc_self_at_filt = mad_filter_1d_xr(gc_self_at, 3.5, dim="time")
        gc_other_at_filt = mad_filter_1d_xr(gc_other_at, 3.5, dim="time")
        gu, gv = geostrophic_components(
            gc_self_at_filt, gc_other_at_filt, a1, a2
        )
        # filter xarray dataarray gu if greater than 3.5 or less than -3.5
        gu_filt = gu.where((gu <= 3.5) & (gu >= -3.5))
        gu_filtered = mad_filter_1d_xr(gu_filt, 3.5)
        gv_filt = gv.where((gv <= 3.5) & (gv >= -3.5))
        gv_filtered = mad_filter_1d_xr(gv_filt, 3.5)
        # gu.plot(label="gu", marker=".")
        # gu_filtered.plot(label="gu_filtered", marker="x")
        # plt.legend()
        # plt.show()
        # create a ds_out with gu and gv
        ds_out = xr.Dataset({"gu": gu_filtered, "gv": gv_filtered})
        ds_out_self = xr.Dataset({"gc_self": gc_self_at_filt})
        ds_out_other = xr.Dataset({"gc_other": gc_other_at_filt})
        filename = f"{out_dir}/guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{method}_{frac1}.nc"
        print(f"saving to {filename}")
        ds_out.to_netcdf(filename)
        ds_out_self.to_netcdf(
            f"{out_dir}/gc_at_{sat1}_{track_number_self}_{method}_{frac1}.nc"
        )
        ds_out_other.to_netcdf(
            f"{out_dir}/gc_at_{sat1}_{track_number_other}_{method}_{frac1}.nc"
        )

```

```python
import math
import os

import pandas as pd
import xarray as xr
from tools_xtrackm import *

ioff = 3
frac1 = "6"
method = "gauss"
out_dir = f"/home/srinivasu/xtrackm/guv/guv_at_intersections_{method}/"
os.makedirs(out_dir, exist_ok=True)
print(sats_new)
sat = "TP+J1+J2+J3+S6A"
sats_new = [sat]
for sat in sats_new[:]:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    for i, r in df.iterrows():
        sat1 = r["sat"]
        track_number_self = str(r["track_self"])
        track_number_other = str(r["track_other"])
        lons_inter1 = r["lons_inter"]
        lats_inter1 = r["lats_inter"]
        x_from_coast_self1 = r["x_from_coast_self"]
        x_from_coast_other1 = r["x_from_coast_other"]
        angle_acute = r["angle_acute"]
        angle_obtuse = r["angle_obtuse"]
        lon1 = lons_inter1
        lat1 = lats_inter1
        if abs(lat1) < 2:
            continue
        # print(
        #     sat,
        #     track_number_self,
        #     track_number_other,
        #     lons_inter1,
        #     lats_inter1,
        #     x_from_coast_self1,
        #     x_from_coast_other1,
        #     angle_acute,
        #     angle_obtuse,
        # )
        f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
        f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
        f_self_smooth = f"../computed/sla_{method}/track_sla_along_{sat}_{track_number_self}_{method}{frac1}.nc"
        f_other_smooth = f"../computed/sla_{method}/track_sla_along_{sat}_{track_number_other}_{method}{frac1}.nc"
        ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
        ds_self_smooth = xr.open_dataset(f_self_smooth)
        sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
        sla_self_smooth = ds_self_smooth.sla_smooth
        ds_other = xr.open_dataset(
            f_other, engine="netcdf4", decode_times=False
        )
        ds_other_smooth = xr.open_dataset(f_other_smooth)
        sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
        sla_other_smooth = ds_other_smooth.sla_smooth
        lons_track_self = ds_self.lon.values
        lats_track_self = ds_self.lat.values
        lon_coast_self = lons_track_self[-1]  # this on coast
        lat_coast_self = lats_track_self[-1]  # this on coast
        lon_equat_self = lons_track_self[0]  # this on equator
        lat_equat_self = lats_track_self[0]  # this on equator
        lons_track_other = ds_other.lon.values
        lats_track_other = ds_other.lat.values
        lon_equat_other = lons_track_other[0]
        lat_equat_other = lats_track_other[0]
        lon_coast_other = lons_track_other[-1]
        lat_coast_other = lats_track_other[-1]
        ii_self = index_at_lat(ds_self, lats_inter1)
        a1 = local_angle(lons_track_self, lats_track_self, ii_self, ioff=ioff)
        a1_d = math.degrees(a1)
        ii_other = index_at_lat(ds_other, lats_inter1)
        a2 = local_angle(
            lons_track_other, lats_track_other, ii_other, ioff=ioff
        )
        a2_d = math.degrees(a2)
        print(f"heading angle {sat}: a1 = {a1_d:.2f}, a2 = {a2_d:.2f}")
        gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
        gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
        gc_self_at = gc_self.sel(
            x=x_from_coast_self1, method="nearest", drop=True
        )
        gc_other_at = gc_other.sel(
            x=x_from_coast_other1, method="nearest", drop=True
        )
        gc_other_at = gc_other_at.interp(time=gc_self_at.time)
        gc_self_at_filt = mad_filter_1d_xr(gc_self_at, 3.5, dim="time")
        gc_other_at_filt = mad_filter_1d_xr(gc_other_at, 3.5, dim="time")
        gu, gv = geostrophic_components(
            gc_self_at_filt, gc_other_at_filt, a1, a2
        )
        # filter xarray dataarray gu if greater than 3.5 or less than -3.5
        gu_filt = gu.where((gu <= 3.5) & (gu >= -3.5))
        gu_filtered = mad_filter_1d_xr(gu_filt, 3.5)
        gv_filt = gv.where((gv <= 3.5) & (gv >= -3.5))
        gv_filtered = mad_filter_1d_xr(gv_filt, 3.5)
        # gu.plot(label="gu", marker=".")
        # gu_filtered.plot(label="gu_filtered", marker="x")
        # plt.legend()
        # plt.show()
        # create a ds_out with gu and gv
        ds_out = xr.Dataset({"gu": gu_filtered, "gv": gv_filtered})
        ds_out_self = xr.Dataset({"gc_self": gc_self_at_filt})
        ds_out_other = xr.Dataset({"gc_other": gc_other_at_filt})
        filename = f"{out_dir}/guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{method}_{frac1}.nc"
        print(f"saving to {filename}")
        ds_out.to_netcdf(filename)
        ds_out_self.to_netcdf(
            f"{out_dir}/gc_at_{sat1}_{track_number_self}_{method}_{frac1}.nc"
        )
        ds_out_other.to_netcdf(
            f"{out_dir}/gc_at_{sat1}_{track_number_other}_{method}_{frac1}.nc"
        )

```

```python
import math
import os

import pandas as pd
import xarray as xr
from tools_xtrackm import *

ioff = 3
frac1 = "0.2"
method = "loess"
out_dir = f"/home/srinivasu/xtrackm/guv/guv_at_intersections_{method}/"
os.makedirs(out_dir, exist_ok=True)
print(sats_new)
sat = "TP+J1+J2+J3+S6A"
sats_new = [sat]
for sat in sats_new[:]:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    for i, r in df.iterrows():
        sat1 = r["sat"]
        track_number_self = str(r["track_self"])
        track_number_other = str(r["track_other"])
        lons_inter1 = r["lons_inter"]
        lats_inter1 = r["lats_inter"]
        x_from_coast_self1 = r["x_from_coast_self"]
        x_from_coast_other1 = r["x_from_coast_other"]
        angle_acute = r["angle_acute"]
        angle_obtuse = r["angle_obtuse"]
        lon1 = lons_inter1
        lat1 = lats_inter1
        if abs(lat1) < 2:
            continue
        # print(
        #     sat,
        #     track_number_self,
        #     track_number_other,
        #     lons_inter1,
        #     lats_inter1,
        #     x_from_coast_self1,
        #     x_from_coast_other1,
        #     angle_acute,
        #     angle_obtuse,
        # )
        f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
        f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
        f_self_smooth = f"../computed/sla_{method}_{frac1}/track_sla_along_{sat}_{track_number_self}_{method}_{frac1}.nc"
        f_other_smooth = f"../computed/sla_{method}_{frac1}/track_sla_along_{sat}_{track_number_other}_{method}_{frac1}.nc"
        ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
        ds_self_smooth = xr.open_dataset(f_self_smooth)
        sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
        sla_self_smooth = ds_self_smooth.sla_smooth
        ds_other = xr.open_dataset(
            f_other, engine="netcdf4", decode_times=False
        )
        ds_other_smooth = xr.open_dataset(f_other_smooth)
        sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
        sla_other_smooth = ds_other_smooth.sla_smooth
        lons_track_self = ds_self.lon.values
        lats_track_self = ds_self.lat.values
        lon_coast_self = lons_track_self[-1]  # this on coast
        lat_coast_self = lats_track_self[-1]  # this on coast
        lon_equat_self = lons_track_self[0]  # this on equator
        lat_equat_self = lats_track_self[0]  # this on equator
        lons_track_other = ds_other.lon.values
        lats_track_other = ds_other.lat.values
        lon_equat_other = lons_track_other[0]
        lat_equat_other = lats_track_other[0]
        lon_coast_other = lons_track_other[-1]
        lat_coast_other = lats_track_other[-1]
        ii_self = index_at_lat(ds_self, lats_inter1)
        a1 = local_angle(lons_track_self, lats_track_self, ii_self, ioff=ioff)
        a1_d = math.degrees(a1)
        ii_other = index_at_lat(ds_other, lats_inter1)
        a2 = local_angle(
            lons_track_other, lats_track_other, ii_other, ioff=ioff
        )
        a2_d = math.degrees(a2)
        print(f"heading angle {sat}: a1 = {a1_d:.2f}, a2 = {a2_d:.2f}")
        gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
        gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
        gc_self_at = gc_self.sel(
            x=x_from_coast_self1, method="nearest", drop=True
        )
        gc_other_at = gc_other.sel(
            x=x_from_coast_other1, method="nearest", drop=True
        )
        gc_other_at = gc_other_at.interp(time=gc_self_at.time)
        gc_self_at_filt = mad_filter_1d_xr(gc_self_at, 3.5, dim="time")
        gc_other_at_filt = mad_filter_1d_xr(gc_other_at, 3.5, dim="time")
        gu, gv = geostrophic_components(
            gc_self_at_filt, gc_other_at_filt, a1, a2
        )
        # filter xarray dataarray gu if greater than 3.5 or less than -3.5
        gu_filt = gu.where((gu <= 3.5) & (gu >= -3.5))
        gu_filtered = mad_filter_1d_xr(gu_filt, 3.5)
        gv_filt = gv.where((gv <= 3.5) & (gv >= -3.5))
        gv_filtered = mad_filter_1d_xr(gv_filt, 3.5)
        # gu.plot(label="gu", marker=".")
        # gu_filtered.plot(label="gu_filtered", marker="x")
        # plt.legend()
        # plt.show()
        # create a ds_out with gu and gv
        ds_out = xr.Dataset({"gu": gu_filtered, "gv": gv_filtered})
        ds_out_self = xr.Dataset({"gc_self": gc_self_at_filt})
        ds_out_other = xr.Dataset({"gc_other": gc_other_at_filt})
        filename = f"{out_dir}/guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{method}_{frac1}.nc"
        print(f"saving to {filename}")
        ds_out.to_netcdf(filename)
        ds_out_self.to_netcdf(
            f"{out_dir}/gc_at_{sat1}_{track_number_self}_{method}_{frac1}.nc"
        )
        ds_out_other.to_netcdf(
            f"{out_dir}/gc_at_{sat1}_{track_number_other}_{method}_{frac1}.nc"
        )

```

```python
import math
import os

import pandas as pd
import xarray as xr
from tools_xtrackm import *

ioff = 3
frac1 = "0.05"
method = "loess"
out_dir = f"/home/srinivasu/xtrackm/guv/guv_at_intersections_{method}/"
os.makedirs(out_dir, exist_ok=True)
print(sats_new)
sat = "TPN+J1N+J2N+J3N"
sats_new = [sat]
for sat in sats_new[:]:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    for i, r in df.iterrows():
        sat1 = r["sat"]
        track_number_self = str(r["track_self"])
        track_number_other = str(r["track_other"])
        lons_inter1 = r["lons_inter"]
        lats_inter1 = r["lats_inter"]
        x_from_coast_self1 = r["x_from_coast_self"]
        x_from_coast_other1 = r["x_from_coast_other"]
        angle_acute = r["angle_acute"]
        angle_obtuse = r["angle_obtuse"]
        lon1 = lons_inter1
        lat1 = lats_inter1
        if abs(lat1) < 2:
            continue
        # print(
        #     sat,
        #     track_number_self,
        #     track_number_other,
        #     lons_inter1,
        #     lats_inter1,
        #     x_from_coast_self1,
        #     x_from_coast_other1,
        #     angle_acute,
        #     angle_obtuse,
        # )
        f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
        f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
        f_self_smooth = f"../computed/sla_{method}_{frac1}a/track_sla_along_{sat}_{track_number_self}_{method}_{frac1}.nc"
        f_other_smooth = f"../computed/sla_{method}_{frac1}a/track_sla_along_{sat}_{track_number_other}_{method}_{frac1}.nc"
        ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
        ds_self_smooth = xr.open_dataset(f_self_smooth)
        sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
        sla_self_smooth = ds_self_smooth.sla_smooth
        ds_other = xr.open_dataset(
            f_other, engine="netcdf4", decode_times=False
        )
        ds_other_smooth = xr.open_dataset(f_other_smooth)
        sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
        sla_other_smooth = ds_other_smooth.sla_smooth
        lons_track_self = ds_self.lon.values
        lats_track_self = ds_self.lat.values
        lon_coast_self = lons_track_self[-1]  # this on coast
        lat_coast_self = lats_track_self[-1]  # this on coast
        lon_equat_self = lons_track_self[0]  # this on equator
        lat_equat_self = lats_track_self[0]  # this on equator
        lons_track_other = ds_other.lon.values
        lats_track_other = ds_other.lat.values
        lon_equat_other = lons_track_other[0]
        lat_equat_other = lats_track_other[0]
        lon_coast_other = lons_track_other[-1]
        lat_coast_other = lats_track_other[-1]
        ii_self = index_at_lat(ds_self, lats_inter1)
        a1 = local_angle(lons_track_self, lats_track_self, ii_self, ioff=ioff)
        a1_d = math.degrees(a1)
        ii_other = index_at_lat(ds_other, lats_inter1)
        a2 = local_angle(
            lons_track_other, lats_track_other, ii_other, ioff=ioff
        )
        a2_d = math.degrees(a2)
        print(f"heading angle {sat}: a1 = {a1_d:.2f}, a2 = {a2_d:.2f}")
        gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
        gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
        gc_self_at = gc_self.sel(
            x=x_from_coast_self1, method="nearest", drop=True
        )
        gc_other_at = gc_other.sel(
            x=x_from_coast_other1, method="nearest", drop=True
        )
        gc_other_at = gc_other_at.interp(time=gc_self_at.time)
        gc_self_at_filt = mad_filter_1d_xr(gc_self_at, 3.5, dim="time")
        gc_other_at_filt = mad_filter_1d_xr(gc_other_at, 3.5, dim="time")
        gu, gv = geostrophic_components(
            gc_self_at_filt, gc_other_at_filt, a1, a2
        )
        # filter xarray dataarray gu if greater than 3.5 or less than -3.5
        gu_filt = gu.where((gu <= 3.5) & (gu >= -3.5))
        gu_filtered = mad_filter_1d_xr(gu_filt, 3.5)
        gv_filt = gv.where((gv <= 3.5) & (gv >= -3.5))
        gv_filtered = mad_filter_1d_xr(gv_filt, 3.5)
        # gu.plot(label="gu", marker=".")
        # gu_filtered.plot(label="gu_filtered", marker="x")
        # plt.legend()
        # plt.show()
        # create a ds_out with gu and gv
        ds_out = xr.Dataset({"gu": gu_filtered, "gv": gv_filtered})
        ds_out_self = xr.Dataset({"gc_self": gc_self_at_filt})
        ds_out_other = xr.Dataset({"gc_other": gc_other_at_filt})
        filename = f"{out_dir}/guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{method}_{frac1}.nc"
        print(f"saving to {filename}")
        ds_out.to_netcdf(filename)
        ds_out_self.to_netcdf(
            f"{out_dir}/gc_at_{sat1}_{track_number_self}_{method}_{frac1}.nc"
        )
        ds_out_other.to_netcdf(
            f"{out_dir}/gc_at_{sat1}_{track_number_other}_{method}_{frac1}.nc"
        )

```

```python
import math
import os

import pandas as pd
import xarray as xr
from tools_xtrackm import *

ioff = 3
frac1 = "0.05"
method = "loess"
method_out = "reindex"
out_dir = f"/home/srinivasu/xtrackm/guv/guv_at_intersections_{method_out}/"
os.makedirs(out_dir, exist_ok=True)
print(sats_new)
sat = "TP+J1+J2+J3+S6A"
sats_new = [sat]
for sat in sats_new[:]:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    for i, r in df.iterrows():
        sat1 = r["sat"]
        track_number_self = str(r["track_self"])
        track_number_other = str(r["track_other"])
        lons_inter1 = r["lons_inter"]
        lats_inter1 = r["lats_inter"]
        x_from_coast_self1 = r["x_from_coast_self"]
        x_from_coast_other1 = r["x_from_coast_other"]
        angle_acute = r["angle_acute"]
        angle_obtuse = r["angle_obtuse"]
        lon1 = lons_inter1
        lat1 = lats_inter1
        if abs(lat1) < 2:
            continue
        # print(
        #     sat,
        #     track_number_self,
        #     track_number_other,
        #     lons_inter1,
        #     lats_inter1,
        #     x_from_coast_self1,
        #     x_from_coast_other1,
        #     angle_acute,
        #     angle_obtuse,
        # )
        f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
        f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
        f_self_smooth = f"../computed/sla_{method}_{frac1}a/track_sla_along_{sat}_{track_number_self}_{method}_{frac1}.nc"
        f_other_smooth = f"../computed/sla_{method}_{frac1}a/track_sla_along_{sat}_{track_number_other}_{method}_{frac1}.nc"
        ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
        ds_self_smooth = xr.open_dataset(f_self_smooth)
        sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
        sla_self_smooth = ds_self_smooth.sla_smooth
        ds_other = xr.open_dataset(
            f_other, engine="netcdf4", decode_times=False
        )
        ds_other_smooth = xr.open_dataset(f_other_smooth)
        sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
        sla_other_smooth = ds_other_smooth.sla_smooth
        lons_track_self = ds_self.lon.values
        lats_track_self = ds_self.lat.values
        lon_coast_self = lons_track_self[-1]  # this on coast
        lat_coast_self = lats_track_self[-1]  # this on coast
        lon_equat_self = lons_track_self[0]  # this on equator
        lat_equat_self = lats_track_self[0]  # this on equator
        lons_track_other = ds_other.lon.values
        lats_track_other = ds_other.lat.values
        lon_equat_other = lons_track_other[0]
        lat_equat_other = lats_track_other[0]
        lon_coast_other = lons_track_other[-1]
        lat_coast_other = lats_track_other[-1]
        ii_self = index_at_lat(ds_self, lats_inter1)
        a1 = local_angle(lons_track_self, lats_track_self, ii_self, ioff=ioff)
        a1_d = math.degrees(a1)
        ii_other = index_at_lat(ds_other, lats_inter1)
        a2 = local_angle(
            lons_track_other, lats_track_other, ii_other, ioff=ioff
        )
        a2_d = math.degrees(a2)
        print(f"heading angle {sat}: a1 = {a1_d:.2f}, a2 = {a2_d:.2f}")
        gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
        gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
        gc_self_at = gc_self.sel(
            x=x_from_coast_self1, method="nearest", drop=True
        )
        gc_other_at = gc_other.sel(
            x=x_from_coast_other1, method="nearest", drop=True
        )
        # gc_other_at = gc_other_at.interp(time=gc_self_at.time)
        gc_other_at = gc_other_at.reindex(time=gc_self_at.time, method="nearest", tolerance="1D")
        gc_self_at_filt = mad_filter_1d_xr(gc_self_at, 3.5, dim="time")
        gc_other_at_filt = mad_filter_1d_xr(gc_other_at, 3.5, dim="time")
        gu, gv = geostrophic_components(
            gc_self_at_filt, gc_other_at_filt, a1, a2
        )
        # filter xarray dataarray gu if greater than 3.5 or less than -3.5
        gu_filt = gu.where((gu <= 3.5) & (gu >= -3.5))
        gu_filtered = mad_filter_1d_xr(gu_filt, 3.5)
        gv_filt = gv.where((gv <= 3.5) & (gv >= -3.5))
        gv_filtered = mad_filter_1d_xr(gv_filt, 3.5)
        # gu.plot(label="gu", marker=".")
        # gu_filtered.plot(label="gu_filtered", marker="x")
        # plt.legend()
        # plt.show()
        # create a ds_out with gu and gv
        ds_out = xr.Dataset({"gu": gu_filtered, "gv": gv_filtered})
        ds_out_self = xr.Dataset({"gc_self": gc_self_at_filt})
        ds_out_other = xr.Dataset({"gc_other": gc_other_at_filt})
        filename = f"{out_dir}/guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{method}_{frac1}.nc"
        print(f"saving to {filename}")
        ds_out.to_netcdf(filename)
        ds_out_self.to_netcdf(
            f"{out_dir}/gc_at_{sat1}_{track_number_self}_{method}_{frac1}.nc"
        )
        ds_out_other.to_netcdf(
            f"{out_dir}/gc_at_{sat1}_{track_number_other}_{method}_{frac1}.nc"
        )

```

```python
import math
import os

import pandas as pd
import xarray as xr
from tools_xtrackm import *

ioff = 3
frac1 = "0.05"
method = "loess"
method_out = "xsmooth"
out_dir = f"/home/srinivasu/xtrackm/guv/guv_at_intersections_{method_out}/"
os.makedirs(out_dir, exist_ok=True)
print(sats_new)
# sat = "TP+J1+J2+J3+S6A"
# sats_new = [sat]
for sat in sats_new[:]:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    for i, r in df.iterrows():
        sat1 = r["sat"]
        track_number_self = str(r["track_self"])
        track_number_other = str(r["track_other"])
        lons_inter1 = r["lons_inter"]
        lats_inter1 = r["lats_inter"]
        x_from_coast_self1 = r["x_from_coast_self"]
        x_from_coast_other1 = r["x_from_coast_other"]
        angle_acute = r["angle_acute"]
        angle_obtuse = r["angle_obtuse"]
        lon1 = lons_inter1
        lat1 = lats_inter1
        if abs(lat1) < 2:
            continue
        # print(
        #     sat,
        #     track_number_self,
        #     track_number_other,
        #     lons_inter1,
        #     lats_inter1,
        #     x_from_coast_self1,
        #     x_from_coast_other1,
        #     angle_acute,
        #     angle_obtuse,
        # )
        f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
        f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
        f_self_smooth = f"../computed/sla_{method}_{frac1}a/track_sla_along_{sat}_{track_number_self}_{method}_{frac1}.nc"
        f_other_smooth = f"../computed/sla_{method}_{frac1}a/track_sla_along_{sat}_{track_number_other}_{method}_{frac1}.nc"
        ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
        ds_self_smooth = xr.open_dataset(f_self_smooth)
        sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
        sla_self_smooth = ds_self_smooth.sla_smooth
        ds_other = xr.open_dataset(
            f_other, engine="netcdf4", decode_times=False
        )
        ds_other_smooth = xr.open_dataset(f_other_smooth)
        sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
        sla_other_smooth = ds_other_smooth.sla_smooth
        lons_track_self = ds_self.lon.values
        lats_track_self = ds_self.lat.values
        lon_coast_self = lons_track_self[-1]  # this on coast
        lat_coast_self = lats_track_self[-1]  # this on coast
        lon_equat_self = lons_track_self[0]  # this on equator
        lat_equat_self = lats_track_self[0]  # this on equator
        lons_track_other = ds_other.lon.values
        lats_track_other = ds_other.lat.values
        lon_equat_other = lons_track_other[0]
        lat_equat_other = lats_track_other[0]
        lon_coast_other = lons_track_other[-1]
        lat_coast_other = lats_track_other[-1]
        ii_self = index_at_lat(ds_self, lats_inter1)
        a1 = local_angle(lons_track_self, lats_track_self, ii_self, ioff=ioff)
        a1_d = math.degrees(a1)
        ii_other = index_at_lat(ds_other, lats_inter1)
        a2 = local_angle(
            lons_track_other, lats_track_other, ii_other, ioff=ioff
        )
        a2_d = math.degrees(a2)
        print(f"heading angle {sat}: a1 = {a1_d:.2f}, a2 = {a2_d:.2f}")
        gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
        gc_self_smooth = gc_self.rolling(x=6, center=True).mean()
        gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
        gc_other_smooth = gc_other.rolling(x=6, center=True).mean()
        gc_self_at = gc_self_smooth.sel(
            x=x_from_coast_self1, method="nearest", drop=True
        )
        gc_other_at = gc_other_smooth.sel(
            x=x_from_coast_other1, method="nearest", drop=True
        )
        gc_other_at = gc_other_at.interp(time=gc_self_at.time)
        gc_self_at_filt = mad_filter_1d_xr(gc_self_at, 3.5, dim="time")
        gc_other_at_filt = mad_filter_1d_xr(gc_other_at, 3.5, dim="time")
        gu, gv = geostrophic_components(
            gc_self_at_filt, gc_other_at_filt, a1, a2
        )
        # filter xarray dataarray gu if greater than 3.5 or less than -3.5
        gu_filt = gu.where((gu <= 3.5) & (gu >= -3.5))
        gu_filtered = mad_filter_1d_xr(gu_filt, 3.5)
        gv_filt = gv.where((gv <= 3.5) & (gv >= -3.5))
        gv_filtered = mad_filter_1d_xr(gv_filt, 3.5)
        # gu.plot(label="gu", marker=".")
        # gu_filtered.plot(label="gu_filtered", marker="x")
        # plt.legend()
        # plt.show()
        # create a ds_out with gu and gv
        ds_out = xr.Dataset({"gu": gu_filtered, "gv": gv_filtered})
        ds_out_self = xr.Dataset({"gc_self": gc_self_at_filt})
        ds_out_other = xr.Dataset({"gc_other": gc_other_at_filt})
        filename = f"{out_dir}/guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{method_out}_{frac1}.nc"
        print(f"saving to {filename}")
        ds_out.to_netcdf(filename)
        ds_out_self.to_netcdf(
            f"{out_dir}/gc_at_{sat1}_{track_number_self}_{method_out}_{frac1}.nc"
        )
        ds_out_other.to_netcdf(
            f"{out_dir}/gc_at_{sat1}_{track_number_other}_{method_out}_{frac1}.nc"
        )

```

guv/c.guv_c.sat_wise.stats.py

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import os


# sats_new = ("TP+J1+J2+J3+S6A", "S3A")

frac1 = "0.05"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
for sat in sats_new:
    gu_all = []
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    for _, row in df.iterrows():
        sat1 = row["sat"]
        track_number_self = str(row["track_self"])
        track_number_other = str(row["track_other"])
        lons_inter1 = row["lons_inter"]
        lat1 = row["lats_inter"]
        if abs(lat1) < 2:
            continue
        f_guv = (
            f"{guv_dir}/guv_at_intersection_{sat1}_{track_number_self}_"
            f"{track_number_other}_{frac1}.nc"
        )
        if not os.path.exists(f_guv):
            print(f"missing file: {f_guv}")
            continue
        with xr.open_dataset(f_guv) as ds_guv:
            gu = ds_guv["gu"].values
        gu = gu[np.isfinite(gu)]
        if gu.size == 0:
            continue
        print(
            sat,
            track_number_self,
            track_number_other,
            lons_inter1,
            lat1,
            gu.size,
        )
        gu_all.extend(gu)

    # plot histogram of gu_all
    plt.hist(gu_all, bins=100, label=f"{sat}")
    np.savez(f"guv_at_intersections_{sat}_{frac1}.npz", gu_all=gu_all)
    plt.show()

```

```python
import math

import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import index_at_lat, local_angle

import math
from namesm import sats_new


ioff = 3
out_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections/"
print(sats_new)
for sat in sats_new[:]:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    for i, r in df.iterrows():
        sat1 = r["sat"]
        track_number_self = str(r["track_self"])
        track_number_other = str(r["track_other"])
        lons_inter1 = r["lons_inter"]
        lats_inter1 = r["lats_inter"]
        x_from_coast_self1 = r["x_from_coast_self"]
        x_from_coast_other1 = r["x_from_coast_other"]
        angle_acute = r["angle_acute"]
        angle_obtuse = r["angle_obtuse"]
        lon1 = lons_inter1
        lat1 = lats_inter1
        if abs(lat1) < 2:
            continue
        f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
        f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
        ds_self = xr.open_dataset(f_self,
                                  engine="netcdf4",
                                  decode_times=False)
        ds_other = xr.open_dataset(f_other,
                                   engine="netcdf4",
                                   decode_times=False)
        lons_track_self = ds_self.lon.values
        lats_track_self = ds_self.lat.values
        lon_coast_self = lons_track_self[-1]  # this on coast
        lat_coast_self = lats_track_self[-1]  # this on coast
        lon_equat_self = lons_track_self[0]  # this on equator
        lat_equat_self = lats_track_self[0]  # this on equator
        slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
            lons_track_self[-1] - lons_track_self[0])
        lons_track_other = ds_other.lon.values
        lats_track_other = ds_other.lat.values
        lon_equat_other = lons_track_other[0]
        lat_equat_other = lats_track_other[0]
        lon_coast_other = lons_track_other[-1]
        lat_coast_other = lats_track_other[-1]
        slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                             lon_equat_other)
        a1l = math.atan2(lat_coast_self - lat_equat_self,
                         lon_coast_self - lon_equat_self)
        a1l_d = math.degrees(a1l)
        ii_self = index_at_lat(ds_self, lats_inter1)
        a1 = local_angle(lons_track_self, lats_track_self, ii_self, ioff=ioff)
        a1_d = math.degrees(a1)
        ii_other = index_at_lat(ds_other, lats_inter1)
        a2l = math.atan2(
            lat_coast_other - lat_equat_other,
            lon_coast_other - lon_equat_other,
        )
        a2l_d = math.degrees(a2l)
        a2 = local_angle(lons_track_other, lats_track_other, ii_other, ioff=ioff)
        a2_d = math.degrees(a2)
        # print(f"heading angle {sat}: a1 = {a1_d:.2f}, a2 = {a2_d:.2f}")
        diff1 = abs(a1 - a1l)
        diff2 = abs(a2 - a2l)
        if diff1 > 0.05 or diff2 > 0.05:
            print(f"{sat} | {track_number_self} diff1 = {diff1:.2f}, {track_number_other} diff2 = {diff2:.2f} Error xxxxxxxxxxxxx")
        else:
            print(f"{sat} | {track_number_self} diff1 = {diff1:.2f}, {track_number_other} diff2 = {diff2:.2f}")

```

```python
import math
import os

import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *

method = "odo"
p = -7
q = 8
para = f"{abs(p)}{abs(q)}"
ioff = 3
out_dir = f"/home/srinivasu/xtrackm/guv/guv_at_intersections_{method}/"
os.makedirs(out_dir, exist_ok=True)


def compute_geostrophy_gc_from_slope(ds, slope_smooth):
    lons_track_rev = ds.lon.values[::-1]
    lats_track_rev = ds.lat.values[::-1]
    if len(lons_track_rev) < 3:
        return None
    f1 = cor_p(lats_track_rev)
    f2 = f1.values
    f3 = np.where(np.abs(lats_track_rev) > 2, f2, np.nan)
    g = grav(lats_track_rev).values
    ratio = g / f3
    # to orient slope coast to equator as is the convention
    slope_smooth = -1 * slope_smooth
    gc = ratio[:, np.newaxis] * slope_smooth
    gc.attrs["units"] = "m/s"
    gc.attrs["long_name"] = "Cross-track geostrophic current"
    gc.attrs["description"] = "Geostrophic current perpendicular to satellite track"
    return gc


def track_dist_time_asn_midx_da(ds, da_smooth, units_in="m"):
    dists = compute_dist(ds, units=units_in)
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    dt0 = datetime.strptime("1950-01-01", "%Y-%m-%d")
    dists_rev = dists[::-1]
    ln = ds["sla"].sizes["points_numbers"]
    ln2 = ln // 2
    dvals = ds.time.isel(points_numbers=ln2).values
    try:
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    except ValueError:
        dvals = pd.Series(dvals).ffill().bfill()
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    var1_rev = da_smooth.load().isel(points_numbers=slice(None, None, -1)).values
    var2 = xr.DataArray(
        var1_rev,
        dims=["x", "time"],
        coords={
            "x": dists_rev,
            "time": dates,
            "lon": ("x", lons_track_rev),
            "lat": ("x", lats_track_rev),
        },
    )
    if units_in == "m":
        var2.coords["x"].attrs["units"] = "m"
    elif units_in == "km":
        var2.coords["x"].attrs["units"] = "km"
    return var2


def apply_optimal_operator(ds, p=-5, q=6, dt=1.0):
    def get_optimal_weights(p, q, dt):
        p_abs, q_abs = abs(p), abs(q)
        n = np.array([i for i in range(-p_abs, q_abs + 1) if i != 0])
        N = p_abs + q_abs
        T = N + 1
        A = np.zeros((T, T))
        A[N, :N] = 1.0
        for i in range(N):
            A[i, :N] = (1.0 / n) * (-n[i] / 2.0)
            A[i, N] = (n[i] ** 2 * dt**2) / 4.0
            A[i, i] = -1.0
        B = np.zeros(T)
        B[N] = 1.0
        results = np.linalg.solve(A, B)
        return results[:N], n

    cn, n_indices = get_optimal_weights(p, q, dt)
    sla = ds.sla.values
    points_dim = sla.shape[0]
    slopes = np.full(sla.shape, np.nan)
    p_abs = abs(p)
    q_abs = abs(q)
    # slopes here are oriented equator to coast
    for i in range(p_abs, points_dim - q_abs):
        h_i = sla[i, :]
        h_rel = sla[i + n_indices, :]
        window_slopes = (h_rel - h_i) / (n_indices[:, np.newaxis] * dt)
        valid = ~np.isnan(window_slopes)
        w = cn[:, np.newaxis] * valid
        w_sum = np.sum(np.abs(w), axis=0)
        w_sum[w_sum == 0] = np.nan
        cn_sum = np.sum(np.abs(cn))
        slopes[i, :] = np.nansum(cn[:, np.newaxis] * window_slopes, axis=0) * (cn_sum / w_sum)
    return xr.DataArray(slopes, coords=ds.sla.coords, dims=ds.sla.dims, name="slope")


def _dx_from_track(ds):
    dists_m = np.asarray(compute_dist(ds, units="m"), dtype=float)
    if dists_m.size < 2:
        return np.nan
    dx = np.diff(dists_m)
    dx = np.abs(dx[np.isfinite(dx)])
    dx = dx[dx > 0]
    if dx.size == 0:
        return np.nan
    return float(np.nanmedian(dx))
```

```python


def _compute_gc_odo(ds, p, q):
    dx_m = _dx_from_track(ds)
    print(dx_m, ds.Pass)
    if not np.isfinite(dx_m):
        return None
    slope_odo = apply_optimal_operator(ds, p=p, q=q, dt=dx_m)
    slope_odo_asn = track_dist_time_asn_midx_da(ds, slope_odo, units_in="m")
    return compute_geostrophy_gc_from_slope(ds, slope_odo_asn)


def _load_track_data_cached(sat, track_number, cache):
    key = (sat, str(track_number))
    if key in cache:
        return cache[key]
    f = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    if not os.path.exists(f):
        cache[key] = None
        return None
    ds = xr.open_dataset(f, engine="netcdf4", decode_times=False)
    if len(ds.lon.values) < (abs(p) + abs(q) + 1):
        cache[key] = None
        return None
    gc = _compute_gc_odo(ds, p=p, q=q)
    if gc is None:
        cache[key] = None
        return None
    cache[key] = {"ds": ds, "gc": gc}
    return cache[key]


def main():
    sats_new = ["TP+J1+J2+J3+S6A"]
    print(sats_new)
    cache = {}
    for sat in sats_new[:]:
        inter_path = f"tracks_intersections_{sat}_1.csv"
        if not os.path.exists(inter_path):
            # print(f"Skipping {sat}: missing {inter_path}")
            continue
        df = pd.read_csv(inter_path)
        for _, r in df.iterrows():
            sat1 = r["sat"]
            track_number_self = str(r["track_self"])
            track_number_other = str(r["track_other"])
            lats_inter1 = r["lats_inter"]
            x_from_coast_self1 = r["x_from_coast_self"]
            x_from_coast_other1 = r["x_from_coast_other"]
            if abs(lats_inter1) < 2:
                continue

            self_data = _load_track_data_cached(sat, track_number_self, cache)
            other_data = _load_track_data_cached(sat, track_number_other, cache)
            if self_data is None or other_data is None:
                print(f"Skipping {sat} {track_number_self}-{track_number_other}: missing/invalid track data")
                continue

            ds_self = self_data["ds"]
            ds_other = other_data["ds"]
            gc_self = self_data["gc"]
            gc_other = other_data["gc"]

            ii_self = index_at_lat(ds_self, lats_inter1)
            ii_other = index_at_lat(ds_other, lats_inter1)
            a1 = local_angle(ds_self.lon.values, ds_self.lat.values, ii_self, ioff=ioff)
            a2 = local_angle(ds_other.lon.values, ds_other.lat.values, ii_other, ioff=ioff)
            # print(f"heading angle {sat}: a1 = {math.degrees(a1):.2f}, a2 = {math.degrees(a2):.2f}")

            gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
            gc_other_at = gc_other.sel(x=x_from_coast_other1, method="nearest", drop=True)
            gc_other_at = gc_other_at.interp(time=gc_self_at.time)

            gc_self_at_filt = mad_filter_1d_xr(gc_self_at, 3.5, dim="time")
            gc_other_at_filt = mad_filter_1d_xr(gc_other_at, 3.5, dim="time")
            try:
                gu, gv = geostrophic_components(gc_self_at_filt, gc_other_at_filt, a1, a2)
            except ValueError as err:
                print(f"Skipping {sat} {track_number_self}-{track_number_other}: {err}")
                continue

            gu_filt = gu.where((gu <= 3.5) & (gu >= -3.5))
            gv_filt = gv.where((gv <= 3.5) & (gv >= -3.5))
            gu_filtered = mad_filter_1d_xr(gu_filt, 3.5)
            gv_filtered = mad_filter_1d_xr(gv_filt, 3.5)

            ds_out = xr.Dataset({"gu": gu_filtered, "gv": gv_filtered})
            ds_out_self = xr.Dataset({"gc_self": gc_self_at_filt})
            ds_out_other = xr.Dataset({"gc_other": gc_other_at_filt})

            filename = f"{out_dir}/guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{method}_{para}.nc"
            # print(f"saving to {filename}")
            ds_out.to_netcdf(filename)
            ds_out_self.to_netcdf(f"{out_dir}/gc_at_{sat1}_{track_number_self}_{method}_{para}.nc")
            ds_out_other.to_netcdf(f"{out_dir}/gc_at_{sat1}_{track_number_other}_{method}_{para}.nc")


if __name__ == "__main__":
    main()

```

guv/demo_pandas_broadcast_rows.py

```python
#!/usr/bin/env python3
"""Small demo of row-wise broadcasting behavior in pandas column assignment."""

import pandas as pd


# Base table: row-varying columns
df = pd.DataFrame(
    {
        "time": pd.to_datetime(
            ["2026-01-01 00:00", "2026-01-01 06:00", "2026-01-01 12:00"]
        ),
        "gu": [0.15, 0.31, 0.22],
        "rama_u": [0.12, 0.29, 0.20],
    }
)

print("Original DataFrame:")
print(df, end="\n\n")

# Scalar assignment: pandas broadcasts one value to all rows.
sat = "S3A"
matched_pairs = len(df)
df["sat"] = sat
df["matched_pairs"] = matched_pairs

print("After scalar assignment (broadcast to every row):")
print(df[["time", "gu", "rama_u", "sat", "matched_pairs"]], end="\n\n")

# Sequence assignment: one value per row (no broadcast).
df["quality_flag"] = ["good", "ok", "good"]
print("After sequence assignment (row-by-row values):")
print(df[["time", "quality_flag"]], end="\n\n")

# Wrong-length sequence assignment raises an error.
try:
    df["bad_col"] = [1, 2]  # len=2 for 3-row DataFrame
except ValueError as exc:
    print("Wrong-length assignment error:")
    print(f"{type(exc).__name__}: {exc}")

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import pandas as pd
import xarray as xr
from tools_xtrackm import plot_etopo_subplot, TRACKS_REG
from matplotlib.colors import ListedColormap
import math

cmap1 = cmo.cm.diff
sat = "S3B"
sat = "TP+J1+J2+J3+S6A"
cmap1 = ListedColormap(["white"])
cmap2 = "bwr"
fig = plt.figure(figsize=(10, 10), layout="constrained")
long_part = "correlations between aviso zonal current and track zonal current"
details = "at intersection points of tracks TP+J1+J2+J3+S6A"
title = f"{long_part}\n{details}"
ax = plot_etopo_subplot(fig, 111, TRACKS_REG, cmap1, title=title, step=5,
                        data_path="~/allData/topo/etopo5.cdf")
df = pd.read_csv(f"tracksVSaviso_corrs_rmses_biass_{sat}_u.csv")

ln = len(df)
for i in range(ln):
    sat1 = df["sat"].values[i]
    lons_inter1 = df["lons_inter"].values[i]
    lats_inter1 = df["lats_inter"].values[i]
    lon1 = lons_inter1
    lat1 = lats_inter1
    corr = df["corrs"].values[i]
    rmse = df["rmses"].values[i]
    bias = df["biass"].values[i]
    # size of marker proportional to correlation
    # check if corr is a valid number
    if not math.isnan(corr):
        ax.scatter(
            lon1,
            lat1,
            s=100 * abs(corr),
            c=[corr],
            vmin=-1,
            vmax=1,
            cmap=cmap2,
        )
# sm.set_array([])
# plt.colorbar(sm, ax=ax, orientation="horizontal", label="Correlation")

# Choose a few example correlation values to represent
legend_corrs = [0.2, 0.5, 0.8]  # example correlation magnitudes
legend_sizes = [100 * abs(c) for c in legend_corrs]
# Create legend handles manually
legend_handles = [
    plt.scatter([], [], s=size, edgecolors='k', facecolors='red', label=f"{corr:.1f}")
    for corr, size in zip(legend_corrs, legend_sizes)
]
ax.legend(
    handles=legend_handles,
    title="|Correlation|",
    loc="lower right",  # or another suitable position
    frameon=True,
)

import matplotlib.cm as cm
sm = cm.ScalarMappable(cmap=cmap2, norm=plt.Normalize(vmin=-1, vmax=1))
plt.subplots_adjust(bottom=0.2)
# Add horizontal colorbar below
cbar_ax = fig.add_axes([0.15, 0.08, 0.7, 0.03])  # [left, bottom, width, height]
cbar = fig.colorbar(sm, cax=cbar_ax, orientation='horizontal')
cbar.set_label('Correlation')
plt.savefig(f"tracksVSaviso_corrs_rmses_biass_{sat}_u.png")
plt.show()

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import pandas as pd
import xarray as xr
from tools_xtrackm import plot_etopo_subplot, TRACKS_REG
from matplotlib.colors import ListedColormap
import math

cmap1 = cmo.cm.diff
sat = "S3B"
sat = "TP+J1+J2+J3+S6A"
cmap1 = ListedColormap(["white"])
cmap2 = "bwr"
fig = plt.figure(figsize=(10, 10), layout="constrained")
long_part = "correlations between aviso zonal current and track zonal current"
details = "at intersection points of tracks TP+J1+J2+J3+S6A"
title = f"{long_part}\n{details}"
ax = plot_etopo_subplot(fig, 111, TRACKS_REG, cmap1, title=title, step=5,
                        data_path="~/allData/topo/etopo5.cdf")
df = pd.read_csv(f"tracksVSaviso_corrs_rmses_biass_{sat}_v.csv")

ln = len(df)
for i in range(ln):
    sat1 = df["sat"].values[i]
    lons_inter1 = df["lons_inter"].values[i]
    lats_inter1 = df["lats_inter"].values[i]
    lon1 = lons_inter1
    lat1 = lats_inter1
    corr = df["corrs"].values[i]
    rmse = df["rmses"].values[i]
    bias = df["biass"].values[i]
    # size of marker proportional to correlation
    # check if corr is a valid number
    if not math.isnan(corr):
        ax.scatter(
            lon1,
            lat1,
            s=100 * abs(corr),
            c=[corr],
            vmin=-1,
            vmax=1,
            cmap=cmap2,
        )
# sm.set_array([])
# plt.colorbar(sm, ax=ax, orientation="horizontal", label="Correlation")

# Choose a few example correlation values to represent
legend_corrs = [0.2, 0.5, 0.8]  # example correlation magnitudes
legend_sizes = [100 * abs(c) for c in legend_corrs]
# Create legend handles manually
legend_handles = [
    plt.scatter([], [], s=size, edgecolors='k', facecolors='red', label=f"{corr:.1f}")
    for corr, size in zip(legend_corrs, legend_sizes)
]
ax.legend(
    handles=legend_handles,
    title="|Correlation|",
    loc="lower right",  # or another suitable position
    frameon=True,
)

import matplotlib.cm as cm
sm = cm.ScalarMappable(cmap=cmap2, norm=plt.Normalize(vmin=-1, vmax=1))
plt.subplots_adjust(bottom=0.2)
# Add horizontal colorbar below
cbar_ax = fig.add_axes([0.15, 0.08, 0.7, 0.03])  # [left, bottom, width, height]
cbar = fig.colorbar(sm, cax=cbar_ax, orientation='horizontal')
cbar.set_label('Correlation')
plt.savefig(f"tracksVSaviso_corrs_rmses_biass_{sat}_v.png")
plt.show()

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import pandas as pd
import xarray as xr
from tools_xtrackm import plot_etopo_subplot, TRACKS_REG
from matplotlib.colors import ListedColormap
import math

cmap1 = cmo.cm.diff
sat = "TP+J1+J2+J3+S6A"
sat = "S3B"
cmap1 = ListedColormap(["white"])
cmap2 = "bwr"
fig = plt.figure(figsize=(10, 10), layout="constrained")
long_part = "correlations between oscar zonal current and track zonal current"
details = "at intersection points of tracks TP+J1+J2+J3+S6A"
title = f"{long_part}\n{details}"
ax = plot_etopo_subplot(fig, 111, TRACKS_REG, cmap1, title=title, step=5,
                        data_path="~/allData/topo/etopo5.cdf")
df = pd.read_csv(f"tracks_corrs_rmses_biass_{sat}_u.csv")

ln = len(df)
for i in range(ln):
    sat1 = df["sat"].values[i]
    lons_inter1 = df["lons_inter"].values[i]
    lats_inter1 = df["lats_inter"].values[i]
    lon1 = lons_inter1
    lat1 = lats_inter1
    corr = df["corrs"].values[i]
    rmse = df["rmses"].values[i]
    bias = df["biass"].values[i]
    # size of marker proportional to correlation
    # check if corr is a valid number
    if not math.isnan(corr):
        ax.scatter(
            lon1,
            lat1,
            s=100 * abs(corr),
            c=[corr],
            vmin=-1,
            vmax=1,
            cmap=cmap2,
        )
# sm.set_array([])
# plt.colorbar(sm, ax=ax, orientation="horizontal", label="Correlation")

# Choose a few example correlation values to represent
legend_corrs = [0.2, 0.5, 0.8]  # example correlation magnitudes
legend_sizes = [100 * abs(c) for c in legend_corrs]
# Create legend handles manually
legend_handles = [
    plt.scatter([], [], s=size, edgecolors='k', facecolors='red', label=f"{corr:.1f}")
    for corr, size in zip(legend_corrs, legend_sizes)
]
ax.legend(
    handles=legend_handles,
    title="|Correlation|",
    loc="lower right",  # or another suitable position
    frameon=True,
)

import matplotlib.cm as cm
sm = cm.ScalarMappable(cmap=cmap2, norm=plt.Normalize(vmin=-1, vmax=1))
plt.subplots_adjust(bottom=0.2)
# Add horizontal colorbar below
cbar_ax = fig.add_axes([0.15, 0.08, 0.7, 0.03])  # [left, bottom, width, height]
cbar = fig.colorbar(sm, cax=cbar_ax, orientation='horizontal')
cbar.set_label('Correlation')
plt.savefig(f"tracks_corrs_rmses_biass_{sat}_u.png")
plt.show()

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import pandas as pd
import xarray as xr
from tools_xtrackm import plot_etopo_subplot, TRACKS_REG
from matplotlib.colors import ListedColormap
import matplotlib.cm as cm
import math

cmap1 = cmo.cm.diff
sat = "S3B"
sat = "TP+J1+J2+J3+S6A"
cmap1 = ListedColormap(["white"])
cmap2 = "bwr"
fig = plt.figure(figsize=(10, 10), layout="constrained")
long_part = "correlations between oscar zonal current and track zonal current"
details = "at intersection points of tracks TP+J1+J2+J3+S6A"
title = f"{long_part}\n{details}"
ax = plot_etopo_subplot(fig, 111, TRACKS_REG, cmap1, title=title, step=5,
                        data_path="~/allData/topo/etopo5.cdf")
df = pd.read_csv(f"tracks_corrs_rmses_biass_{sat}_v.csv")

ln = len(df)
for i in range(ln):
    sat1 = df["sat"].values[i]
    lons_inter1 = df["lons_inter"].values[i]
    lats_inter1 = df["lats_inter"].values[i]
    lon1 = lons_inter1
    lat1 = lats_inter1
    corr = df["corrs"].values[i]
    rmse = df["rmses"].values[i]
    bias = df["biass"].values[i]
    # size of marker proportional to correlation
    # check if corr is a valid number
    if not math.isnan(corr):
        ax.scatter(
            lon1,
            lat1,
            s=100 * abs(corr),
            c=[corr],
            vmin=-1,
            vmax=1,
            cmap=cmap2,
        )

# Choose a few example correlation values to represent
legend_corrs = [0.2, 0.5, 0.8]  # example correlation magnitudes
legend_sizes = [100 * abs(c) for c in legend_corrs]
# Create legend handles manually
legend_handles = [
    plt.scatter([], [], s=size, edgecolors='k', facecolors='red', label=f"{corr:.1f}")
    for corr, size in zip(legend_corrs, legend_sizes)
]
ax.legend(
    handles=legend_handles,
    title="|Correlation|",
    loc="lower right",  # or another suitable position
    frameon=True,
)

sm = cm.ScalarMappable(cmap=cmap2, norm=plt.Normalize(vmin=-1, vmax=1))
plt.subplots_adjust(bottom=0.2)
# Add horizontal colorbar below
cbar_ax = fig.add_axes([0.15, 0.08, 0.7, 0.03])  # [left, bottom, width, height]
cbar = fig.colorbar(sm, cax=cbar_ax, orientation='horizontal')
cbar.set_label('Correlation')
plt.savefig(f"tracks_corrs_rmses_biass_{sat}_v.png")
plt.show()

```

```python
import math

import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import sys


def get_time_limits_o(f):
    ds = xr.open_dataset(f, engine="h5netcdf")
    # ds = xr.open_dataset(f)
    time = ds.time.isel(points_numbers=0).values
    # these are cftime objects which are not very useful
    first_time = time[0]
    last_time = time[-1]
    # print(type(first_time), type(last_time))
    # Convert the first and last time stamps to string format
    first_time_str = first_time.strftime("%Y-%m-%d")
    last_time_str = last_time.strftime("%Y-%m-%d")
    # first_time_str = datetime.strptime(first_time, "%Y-%m-%d")
    # last_time_str = datetime.strptime(last_time, "%Y-%m-%d")
    # print(type(first_time_str), type(last_time_str))
    track_tsta_o = datetime.strptime(first_time_str, "%Y-%m-%d")
    track_tend_o = datetime.strptime(last_time_str, "%Y-%m-%d")
    return track_tsta_o, track_tend_o


for sat in sats_new:
    tfreq = sats_tfreq[sat]
    print(f"satellite name {sat} -------------- {tfreq} days")
    time_diff_list = []
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    for i, r in df.iterrows():
        sat1 = r["sat"]
        track_number_self = str(r["track_self"])
        track_number_other = str(r["track_other"])
        lons_inter1 = r["lons_inter"]
        lats_inter1 = r["lats_inter"]
        x_from_coast_self1 = r["x_from_coast_self"]
        x_from_coast_other1 = r["x_from_coast_other"]
        angle_acute = r["angle_acute"]
        angle_obtuse = r["angle_obtuse"]
        lon1 = lons_inter1
        lat1 = lats_inter1
        if abs(lat1) < 2:
            continue
        # print(
        #     sat,
        #     track_number_self,
        #     track_number_other,
        #     lons_inter1,
        #     lats_inter1,
        #     x_from_coast_self1,
        #     x_from_coast_other1,
        #     angle_acute,
        #     angle_obtuse,
        # )
        f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
        f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
        tsta_o_self, tend_o_self = get_time_limits_o(f_self)
        tsta_o_other, tend_o_other = get_time_limits_o(f_other)
        time_diff = (tsta_o_self - tsta_o_other).days
        time_diff = (tend_o_self - tend_o_other).days
        time_diff_list.append(time_diff)
        # print(f"{track_number_self} {tsta_o_self} {tend_o_self}")
        # print(f"{track_number_other} {tsta_o_other} {tend_o_other}")
        # print(f"{track_number_other} {time_diff} in days")
    print(f"max inter track time diff {max(time_diff_list)} in days")
    print(f"min inter track time diff {min(time_diff_list)} in days")

```

```python
import glob
import os

import xarray as xr
from tools_xtrackm import *

from tools_drifter import *

REG = (43.0, 99, 0, 25.4)
chunk = "15001_current"
sat = "S3A"
track_tsta_o, track_tend_o = get_time_limits_o(sat)
polygon_freq = 1
dse = xr.open_dataset(
    "/home/srinivasu/allData/topo/etopo_60s_io.nc"
)  # open etopo dataset
print(dse)

for fd in sorted(
    glob.glob(
        f"/home/srinivasu/allData/drifter_track_reg/netcdf_{chunk}/drifter_6h_*.nc"
    )
):
    print(fd)
    basename1 = os.path.basename(fd)
    ds_d = xr.open_dataset(fd, drop_variables=["WMO"])
    byte_id = ds_d.ID.values[0]
    str_id = byte_id.decode("utf-8")
    drift_tsta_o1, drift_tend_o1 = (
        ds_d.start_date.values[0],
        ds_d.end_date.values[0],
    )
    drift_tsta_o, drift_tend_o = pd.to_datetime(drift_tsta_o1), pd.to_datetime(
        drift_tend_o1
    )
    overlap_tsta, overlap_tend = overlap_dates(
        track_tsta_o, track_tend_o, drift_tsta_o, drift_tend_o
    )
    print(drift_tsta_o, drift_tend_o)
    if overlap_tsta is None or overlap_tend is None:
        print(f"no time overlap for track {sat} and drifter {str_id}")
        continue
    # Combine the sampled lon and lat into a polygon
    lons_drift = ds_d.longitude.isel(traj=0).values
    lats_drift = ds_d.latitude.isel(traj=0).values
    lon_sampled = lons_drift[::polygon_freq]
    lat_sampled = lats_drift[::polygon_freq]
    if len(lon_sampled) < 5:
        continue
    polygon_points = [(lon, lat) for lon, lat in zip(lon_sampled, lat_sampled)]
    polygon = Polygon(polygon_points)
    ve = ds_d.ve.isel(traj=0).values
    fig, ax = create_topo_map(dse, xsta=REG[0], xend=REG[1], ysta=REG[2], yend=REG[3], title1="", step=5)
    sc = ax.scatter(
        lons_drift,
        lats_drift,
        c=ve,
        cmap="viridis",
        s=12,
        transform=ccrs.PlateCarree(),
    )
    # add polygon geometry to the plot
    ax.add_geometries(
        [polygon], ccrs.PlateCarree(), facecolor="none", edgecolor="none"
    )
    for ft in sorted(
        glob.glob(
            f"../data/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        basename_t = os.path.basename(ft)
        trackn = basename_t.split(".")[5]
        ds_t = xr.open_dataset(ft, decode_times=False, engine="netcdf4")
        track_number = ds_t.pass_number
        lons_track = ds_t.lon.values
        lats_track = ds_t.lat.values
        if len(lons_track) == 0:
            continue
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        line_start = (lons_track[0], lats_track[0])
        line_end = (lons_track[-1], lats_track[-1])
        line = sg.LineString([line_start, line_end])
        if line.intersects(polygon):
            ax.scatter(
                lons_track,
                lats_track,
                c="k",
                s=1,
                transform=ccrs.PlateCarree(),
            )
        else:
            print(f"track {trackn} not intersecting")

    plt.savefig(f"pngs_dump/drifter_{str_id}_with_intersecting_tracks.png")
    # close all windows
    plt.close("all")
    plt.show()

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs
import sys
# from rich.traceback import install
# install(show_locals=True, suppress=["site-packages"])

cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
aviso_msla_uv_dir = "/home/srinivasu/sealevel/data/msla_uv/"
aviso_msla_uv_dir = "/home/srinivasu/allData/aviso/msla_uv/"
ds_aviso_u = xr.open_dataset(f"{aviso_msla_uv_dir}/allyear_dt_global_allsat_msla_u.nc")
ds_aviso_v = xr.open_dataset(f"{aviso_msla_uv_dir}/allyear_dt_global_allsat_msla_v.nc")
print(ds_aviso_u)
aviso_u = ds_aviso_u.u
aviso_v = ds_aviso_v.v
# sys.exit()

sat = "S3B"
sat = "TP+J1+J2+J3+S6A"
df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
df_out = df.copy()

corrs_u = []
biass_u = []
rmses_u = []
ngp_u = []

corrs_v = []
biass_v = []
rmses_v = []
ngp_v = []

ln = len(df)
count = 0
frac1 = "0.05"
for i in range(ln):
    sat1 = df["sat"].values[i]
    track_number_self = df["track_self"].apply(lambda x: str(x)).values[i]
    track_number_other = df["track_other"].apply(lambda x: str(x)).values[i]
    lons_inter1 = df["lons_inter"].values[i]
    lats_inter1 = df["lats_inter"].values[i]
    lon1 = lons_inter1
    lat1 = lats_inter1
    x_from_coast_self1 = df["x_from_coast_self"].values[i]
    x_from_coast_other1 = df["x_from_coast_other"].values[i]
    angle_acute = df["angle_acute"].values[i]
    angle_obtuse = df["angle_obtuse"].values[i]
    if abs(lat1) < 2:
        corrs_u.append(np.nan)
        rmses_u.append(np.nan)
        biass_u.append(np.nan)
        ngp_u.append(np.nan)
        corrs_v.append(np.nan)
        rmses_v.append(np.nan)
        biass_v.append(np.nan)
        ngp_v.append(np.nan)
        continue
    print(
        sat,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_self}_loess_{frac1}.nc"
    f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_other}_loess_{frac1}.nc"
    ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
```

```
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)
    print(f"azimuths {a1:.2f} {a2:.2f}")
    gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
    fig = plt.figure(figsize=(12, 10))
    ax1 = fig.add_subplot(3, 1, 3)
    ax0 = fig.add_subplot(3, 1, 2)

    gu_cut = gu.resample(time="1ME").mean()
    gu_cut_ngp = 100 * gu_cut.count(dim="time")/len(gu_cut.time)
    gv_cut = gv.resample(time="1ME").mean()
    gv_cut_ngp = 100 * gv_cut.count(dim="time")/len(gv_cut.time)
    tfreq = sats_tfreq[sat]

    aviso_u1 = aviso_u.sel(longitude=lon1, latitude=lat1, method="nearest")
    aviso_u2 = aviso_u1.interp(time=gu_cut.time)
    aviso_max_u = aviso_u2.max().item()
    gu_cut = gu_cut.where(gu_cut < aviso_max_u, np.nan)
    gu_cut = gu_cut.where(gu_cut > -aviso_max_u, np.nan)

    aviso_v1 = aviso_v.sel(longitude=lon1, latitude=lat1, method="nearest")
    aviso_v2 = aviso_v1.interp(time=gv_cut.time)
    aviso_max_v = aviso_v2.max().item()
    gv_cut = gv_cut.where(gv_cut < aviso_max_v, np.nan)
    gv_cut = gv_cut.where(gv_cut > -aviso_max_v, np.nan)

    aviso_u2.plot(ax=ax0, color="b", linewidth=2, label="aviso u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="r", linewidth=2)
    # ax0.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax0.legend()
    ax0.set_ylabel("aviso u m/s")

    aviso_v2.plot(ax=ax1, color="b", linewidth=2, label="aviso v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="r", linewidth=2)

    # percentage of good points of gu_cut
    corr_u = xs.pearson_r(gu_cut, aviso_u2, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, aviso_u2, dim="time", skipna=True)
    bias_var_u = aviso_u2 - gu_cut
    bias_u = bias_var_u.mean(dim="time")

    corr_v = xs.pearson_r(gv_cut, aviso_v2, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, aviso_v2, dim="time", skipna=True)
    bias_var_v = aviso_v2 - gv_cut
    bias_v = bias_var_v.mean(dim="time")

    corrs_u.append(corr_u.item())
    rmses_u.append(rmse_u.item())
    biass_u.append(bias_u.item())
    ngp_u.append(gu_cut_ngp.item())

    corrs_v.append(corr_v.item())
    rmses_v.append(rmse_v.item())
    biass_v.append(bias_v.item())
    ngp_v.append(gv_cut_ngp.item())

    print(f"Correlation between gu and u_aviso_at: {corr_u.item():.2f} {gu_cut_ngp.item():.2f}")
    print(f"Correlation between gv and v_aviso_at: {corr_v.item():.2f} {gv_cut_ngp.item():.2f}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.15,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("aviso v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor="red",
        markeredgecolor="black",
        markeredgewidth=2,
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
```

```
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"track_guvVSaviso_at_intersection_points/guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    plt.show()
    plt.close("all")
    # ds_aviso_u.close()

# add corrs, rmses, biass to dataframe df_out
df_out["corrs"] = corrs_u
df_out["rmses"] = rmses_u
df_out["biass"] = biass_u
df_out["ngp"] = ngp_u

df_out.to_csv(f"tracksVSaviso_corrs_rmses_biass_{sat1}_u.csv", index=False)

df_out["corrs"] = corrs_v
df_out["rmses"] = rmses_v
df_out["biass"] = biass_v
df_out["ngp"] = ngp_v

df_out.to_csv(f"tracksVSaviso_corrs_rmses_biass_{sat1}_v.csv", index=False)
dse.close()

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs
import sys
# from rich.traceback import install
# install(show_locals=True, suppress=["site-packages"])


def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    u = u
    return u, v


cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
aviso_msla_uv_dir = "/home/srinivasu/sealevel/data/msla_uv/"
ds_aviso_u = xr.open_dataset(f"{aviso_msla_uv_dir}/allyear_dt_global_allsat_msla_u_nio.nc")
ds_aviso_v = xr.open_dataset(f"{aviso_msla_uv_dir}/allyear_dt_global_allsat_msla_v_nio.nc")
print(ds_aviso_u)
aviso_u = ds_aviso_u.ugosa
aviso_v = ds_aviso_v.vgosa
# sys.exit()

sat_val = "TP+J1+J2+J3+S6A"
track_self_val = 29
track_other_val = 92
df = pd.read_csv(f"tracks_intersections_{sat_val}_1.csv")
df_out = df.copy()

rows = df_out.loc[
    (df_out["sat"] == sat_val) &
    (df_out["track_self"] == track_self_val) &
    (df_out["track_other"] == track_other_val)
]
# sys.exit()
corrs_u = []
biass_u = []
rmses_u = []
ngp_u = []

corrs_v = []
biass_v = []
rmses_v = []
ngp_v = []

ln = len(rows)
count = 0
for i in range(ln):
    sat1 = rows["sat"].values[i]
    track_number_self = rows["track_self"].apply(lambda x: str(x)).values[i]
    track_number_other = rows["track_other"].apply(lambda x: str(x)).values[i]
    lons_inter1 = rows["lons_inter"].values[i]
    lats_inter1 = rows["lats_inter"].values[i]
    lon1 = lons_inter1
    lat1 = lats_inter1
    x_from_coast_self1 = rows["x_from_coast_self"].values[i]
    x_from_coast_other1 = rows["x_from_coast_other"].values[i]
    angle_acute = rows["angle_acute"].values[i]
    angle_obtuse = rows["angle_obtuse"].values[i]
    if abs(lat1) < 2:
        corrs_u.append(np.nan)
        rmses_u.append(np.nan)
        biass_u.append(np.nan)
        ngp_u.append(np.nan)
        corrs_v.append(np.nan)
        rmses_v.append(np.nan)
        biass_v.append(np.nan)
        ngp_v.append(np.nan)
        continue
```

```
    print(
        sat1,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_self}_loess_0.2.nc"
    f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_other}_loess_0.2.nc"
    ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)
    print(f"azimuths {a1:.2f} {a2:.2f}")
    gu, gv = geostrophic_components_from_a(gc_self_at, gc_other_at, a1, a2)
    fig = plt.figure(figsize=(12, 10))
    ax1 = fig.add_subplot(3, 1, 3)
    ax0 = fig.add_subplot(3, 1, 2)

    gu_cut = -1 * gu.resample(time="1ME").mean()
    gu_cut_ngp = 100 * gu_cut.count(dim="time")/len(gu_cut.time)
    gv_cut = gv.resample(time="1ME").mean()
    gv_cut_ngp = 100 * gv_cut.count(dim="time")/len(gv_cut.time)
    tfreq = sats_tfreq[sat1]

    aviso_u1 = aviso_u.sel(longitude=lon1, latitude=lat1, method="nearest")
    aviso_u2 = aviso_u1.interp(time=gu_cut.time)
    aviso_max_u = aviso_u2.max().item()
    gu_cut = gu_cut.where(gu_cut < aviso_max_u, np.nan)
    gu_cut = gu_cut.where(gu_cut > -aviso_max_u, np.nan)

    aviso_v1 = aviso_v.sel(longitude=lon1, latitude=lat1, method="nearest")
    aviso_v2 = aviso_v1.interp(time=gv_cut.time)
    aviso_max_v = aviso_v2.max().item()
    gv_cut = gv_cut.where(gv_cut < aviso_max_v, np.nan)
    gv_cut = gv_cut.where(gv_cut > -aviso_max_v, np.nan)

    aviso_u2.plot(ax=ax0, color="b", linewidth=2, label="aviso u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="r", linewidth=2)
    # ax0.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax0.legend()
    ax0.set_ylabel("aviso u m/s")

    aviso_v2.plot(ax=ax1, color="b", linewidth=2, label="aviso v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="r", linewidth=2)

    # percentage of good points of gu_cut
    corr_u = xs.pearson_r(gu_cut, aviso_u2, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, aviso_u2, dim="time", skipna=True)
    bias_var_u = aviso_u2 - gu_cut
    bias_u = bias_var_u.mean(dim="time")

    corr_v = xs.pearson_r(gv_cut, aviso_v2, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, aviso_v2, dim="time", skipna=True)
    bias_var_v = aviso_v2 - gv_cut
    bias_v = bias_var_v.mean(dim="time")

    corrs_u.append(corr_u.item())
    rmses_u.append(rmse_u.item())
    biass_u.append(bias_u.item())
    ngp_u.append(gu_cut_ngp.item())

    corrs_v.append(corr_v.item())
    rmses_v.append(rmse_v.item())
    biass_v.append(bias_v.item())
    ngp_v.append(gv_cut_ngp.item())

    print(f"Correlation between gu and u_aviso_at: {corr_u.item():.2f} {gu_cut_ngp.item():.2f}")
    print(f"Correlation between gv and v_aviso_at: {corr_v.item():.2f} {gv_cut_ngp.item():.2f}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.15,
        0.90,
        info,
        transform=ax0.transAxes,
```

```
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("aviso v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor="red",
        markeredgecolor="black",
        markeredgewidth=2,
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"track_guvVSaviso_at_intersection_points/guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    plt.show()
    plt.close("all")
    # ds_aviso_u.close()
dse.close()

```

```python
import cmocean as cmo
from functools import lru_cache
import matplotlib.pyplot as plt
import numpy as np
import os
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs
# from rich.traceback import install
# install(show_locals=True, suppress=["site-packages"])

cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
aviso_msla_uv_dir = "/home/srinivasu/sealevel/data/msla_uv/"
aviso_msla_uv_dir = "/home/srinivasu/allData/aviso/msla_uv/"
ds_aviso_u = xr.open_dataset(f"{aviso_msla_uv_dir}/allyear_dt_global_allsat_msla_u.nc")
ds_aviso_v = xr.open_dataset(f"{aviso_msla_uv_dir}/allyear_dt_global_allsat_msla_v.nc")
print(ds_aviso_u)
aviso_u = ds_aviso_u.u
aviso_v = ds_aviso_v.v

rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(*TRACKS_REG[2:]))


@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(f_track, engine="netcdf4", decode_times=False) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values

    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


def get_track_end_to_end_angle(sat, track_number):
    return get_track_geometry(sat, track_number)[-1]


def append_nan_metrics(corrs_u, rmses_u, biass_u, ngp_u, corrs_v, rmses_v, biass_v, ngp_v):
    corrs_u.append(np.nan)
    rmses_u.append(np.nan)
    biass_u.append(np.nan)
    ngp_u.append(np.nan)
    corrs_v.append(np.nan)
    rmses_v.append(np.nan)
    biass_v.append(np.nan)
    ngp_v.append(np.nan)


sat = "S3B"
sat = "TP+J1+J2+J3+S6A"
df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
df_out = df.copy()

corrs_u = []
biass_u = []
rmses_u = []
ngp_u = []

corrs_v = []
biass_v = []
rmses_v = []
ngp_v = []

ln = len(df)
frac1 = "0.05"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
for i in range(ln):
    row = df.iloc[i]
    sat1 = row["sat"]
    track_number_self = str(row["track_self"])
    track_number_other = str(row["track_other"])
    lons_inter1 = row["lons_inter"]
    lats_inter1 = row["lats_inter"]
    lon1 = lons_inter1
    lat1 = lats_inter1
    angle_acute = row["angle_acute"]
    angle_obtuse = row["angle_obtuse"]
    if abs(lat1) < 2:
        append_nan_metrics(corrs_u, rmses_u, biass_u, ngp_u, corrs_v, rmses_v, biass_v, ngp_v)
        continue
    print(
        sat,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        angle_acute,
        angle_obtuse,
    )
    f_guv = f"{guv_dir}/guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{frac1}.nc"
    ds_guv = xr.open_dataset(f_guv)
    lons_track_self, lats_track_self, lon_equat_self, lat_equat_self, lon_coast_self, lat_coast_self, _ = get_track_geometry(
        sat, track_number_self
    )
    lons_track_other, lats_track_other, lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, _ = get_track_geometry(
        sat, track_number_other
    )
    angle_self = get_track_end_to_end_angle(sat, track_number_self)
    angle_other = get_track_end_to_end_angle(sat, track_number_other)

    gu = ds_guv.gu
    gv = ds_guv.gv

    fig = plt.figure(figsize=(12, 10))
    ax1 = fig.add_subplot(3, 1, 3)
    ax0 = fig.add_subplot(3, 1, 2)

    gu_cut = gu.resample(time="1ME").mean()
```

```
    gu_cut_ngp = 100 * gu_cut.count(dim="time")/len(gu_cut.time)
    gv_cut = gv.resample(time="1ME").mean()
    gv_cut_ngp = 100 * gv_cut.count(dim="time")/len(gv_cut.time)

    aviso_u1 = aviso_u.sel(longitude=lon1, latitude=lat1, method="nearest")
    aviso_u2 = aviso_u1.interp(time=gu_cut.time)
    aviso_max_u = aviso_u2.max().item()
    gu_cut = gu_cut.where(gu_cut < aviso_max_u, np.nan)
    gu_cut = gu_cut.where(gu_cut > -aviso_max_u, np.nan)

    aviso_v1 = aviso_v.sel(longitude=lon1, latitude=lat1, method="nearest")
    aviso_v2 = aviso_v1.interp(time=gv_cut.time)
    aviso_max_v = aviso_v2.max().item()
    gv_cut = gv_cut.where(gv_cut < aviso_max_v, np.nan)
    gv_cut = gv_cut.where(gv_cut > -aviso_max_v, np.nan)

    aviso_u2.plot(ax=ax0, color="b", linewidth=2, label="aviso u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="r", linewidth=2)
    # ax0.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax0.legend()
    ax0.set_ylabel("aviso u m/s")

    aviso_v2.plot(ax=ax1, color="b", linewidth=2, label="aviso v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="r", linewidth=2)

    # percentage of good points of gu_cut
    corr_u = xs.pearson_r(gu_cut, aviso_u2, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, aviso_u2, dim="time", skipna=True)
    bias_var_u = aviso_u2 - gu_cut
    bias_u = bias_var_u.mean(dim="time")

    corr_v = xs.pearson_r(gv_cut, aviso_v2, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, aviso_v2, dim="time", skipna=True)
    bias_var_v = aviso_v2 - gv_cut
    bias_v = bias_var_v.mean(dim="time")

    corrs_u.append(corr_u.item())
    rmses_u.append(rmse_u.item())
    biass_u.append(bias_u.item())
    ngp_u.append(gu_cut_ngp.item())

    corrs_v.append(corr_v.item())
    rmses_v.append(rmse_v.item())
    biass_v.append(bias_v.item())
    ngp_v.append(gv_cut_ngp.item())

    print(f"Correlation between gu and u_aviso_at: {corr_u.item():.2f} {gu_cut_ngp.item():.2f}")
    print(f"Correlation between gv and v_aviso_at: {corr_v.item():.2f} {gv_cut_ngp.item():.2f}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.15,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("aviso v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor="red",
        markeredgecolor="black",
        markeredgewidth=2,
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"track_guvVSaviso_at_intersection_points/guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    plt.show()
    plt.close("all")
    ds_guv.close()
    # ds_aviso_u.close()
```

```
235
236 # add corrs, rmses, biass to dataframe df_out
237 df_out["corrs"] = corrs_u
238 df_out["rmses"] = rmses_u
239 df_out["biass"] = biass_u
240 df_out["ngp"] = ngp_u
241
242 df_out.to_csv(f"tracksVSaviso_corrs_rmses_biass_{sat1}_u.csv", index=False)
243
244 df_out["corrs"] = corrs_v
245 df_out["rmses"] = rmses_v
246 df_out["biass"] = biass_v
247 df_out["ngp"] = ngp_v
248
249 df_out.to_csv(f"tracksVSaviso_corrs_rmses_biass_{sat1}_v.csv", index=False)
250 dse.close()
251
```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *


def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    return u, v


def haversine_distance(lon1, lat1, lon2, lat2):
    """Calculate great-circle distance between (lon1, lat1)
    and arrays (lon2, lat2) in kilometers."""
    R = 6371  # Radius of Earth in km
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlon = lon2 - lon1
    dlat = lat2 - lat1
    a = (np.sin(dlat / 2)**2 +
         np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2)**2)
    return 2 * R * np.arcsin(np.sqrt(a))


def find_n_closest_points(df, lon0, lat0, n=4):
    """Return the n rows in df closest to (lon0, lat0) based on
    lons_inter/lats_inter."""
    distances = haversine_distance(lon0, lat0, df["lons_inter"].values,
                                   df["lats_inter"].values)
    df = df.copy()
    df["distance_km"] = distances
    return df.nsmallest(n, "distance_km")


cmap1 = cmo.cm.diff
d = 1.5
dpth = 15.0
height, width = 9, 9
oscar_dir = "/home/srinivasu/allData/oscar_0.25/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
cb_data_dir = "/home/srinivasu/slnew/xtrackd/coastal/"
nth = 0

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

for cb_id in cb_d.keys():
    lon_cb, lat_cb = cb_d[cb_id]
    ds_cb = xr.open_dataset(f"{cb_data_dir}/{cb_id.upper()}_uvs.nc")
    print(ds_cb)

    cb_u = ds_cb.U
    cb_v = ds_cb.V
    cb_s = ds_cb.speed
    cb_dir = ds_cb.direction

    # plt.xlim(datetime(2016, 6, 1), datetime(2018, 6, 30))
    cb_u = cb_u.resample(time="1D").mean(dim="time")
    cb_v = cb_v.resample(time="1D").mean(dim="time")
    cb_s = cb_s.resample(time="1D").mean(dim="time")
    # print(N)
    df_4 = find_n_closest_points(df_all, lon_cb, lat_cb, n=1)
    # print(cb_id, lon_cb, lat_cb, "\n", df_4.head(1))
    sat = df_4["sat"].values[0]
    track_number_self = df_4["track_self"].apply(lambda x: str(x)).values[nth]
    track_number_other = df_4["track_other"].apply(lambda x: str(x)).values[nth]
    lons_inter1 = df_4["lons_inter"].values[nth]
    lats_inter1 = df_4["lats_inter"].values[nth]
    lon1 = lons_inter1
    lat1 = lats_inter1
    x_from_coast_self1 = df_4["x_from_coast_self"].values[nth]
```

```
    x_from_coast_other1 = df_4["x_from_coast_other"].values[nth]
    angle_acute = df_4["angle_acute"].values[nth]
    angle_obtuse = df_4["angle_obtuse"].values[nth]
    print(
        sat,
        cb_id,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_self}_loess_0.2.nc"
    f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_other}_loess_0.2.nc"
    ds_self = xr.open_dataset(f_self, decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    # u_oscar_at = u_oscar.sel(lon=lon1, lat=lat1, method="nearest")
    # v_oscar_at = v_oscar.sel(lon=lon1, lat=lat1, method="nearest")
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    # if track_path_self.intersects(track_path_other):
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    # Better approach:
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)

    gu, gv = geostrophic_components_from_a(gc_self_at, gc_other_at, a1, a2)
    # other_times.append((track_number_other, abs(dist_time)))
    fig1, ax1 = plt.subplots(1, 1, figsize=(12, 8), layout="constrained")
    gu_cut = -1 * gu.interp(time=cb_u.time)
    gv_cut = gv.interp(time=cb_v.time)
    # gu_cut.plot(ax=ax1, label=f"track zonal", color="r", linewidth=2)
    # cb_u.plot(ax=ax1, color="b", linewidth=2, label="cb")
    # u_oscar_at.plot(ax=ax1, color="b", linewidth=2, label="oscar u")
    # corr = xs.pearson_r(gu_cut, cb_u, dim="time", skipna=True)
    gv_cut.plot(ax=ax1, label=f"track zonal", color="r", linewidth=2)
    cb_v.plot(ax=ax1, color="b", linewidth=2, label="cb")
    # v_oscar_at.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
    corr = xs.pearson_r(gv_cut, cb_v, dim="time", skipna=True)
    corr1 = corr.item()
    # print(f"Correlation between gu and u_oscar_at: {corr1}")
    print(f"Correlation between gv and v_oscar_at: {corr1}")
    plt.text(
        0.15,
        0.95,
        f"intersection point ({lon1:.2f}, {lat1:.2f})",
        transform=ax1.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    plt.legend()
    fig2, ax2 = plt.subplots(
        1,
        1,
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(width, height),
        layout="constrained",
    )
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG)
    ax2.grid()
    ax2.plot(
        lon_cb,
        lat_cb,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor="red",
        markeredgecolor="black",
        markeredgewidth=2,
    )
    # plot plat form code of cb at lon_cb, lat_cb
    plt.text(lon_cb, lat_cb, f"{cb_id}", fontsize=10, color="k")
    ax2.scatter(
        lons_track_self,
        lats_track_self,
```

```
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="w",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="w",
        )
    plt.show()
    plt.close("all")

dse.close()
```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
import xskillscore as xs
from geopy import distance
from tools_xtrackm import *


@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(
        f_track, engine="netcdf4", decode_times=False
    ) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


cmap1 = cmo.cm.diff
d = 1.5
height, width = 9, 9
frac1 = "0.05"


dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
cb_data_dir = "/home/srinivasu/slnew/xtrackd/coastal/"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
out_match_csv = "cb_guv_c_nearest_global_matches.csv"
matched_rows_all = []


df_all_list = []
for sat in sats_new:
    df_all_list.append(pd.read_csv(f"tracks_intersections_{sat}_1.csv"))
df_all = pd.concat(df_all_list, ignore_index=True)

for cb_id in cb_d.keys():
    lon_cb, lat_cb = cb_d[cb_id]
    if abs(lat_cb) < 2:
        continue

    cb_file = f"{cb_data_dir}/{cb_id.upper()}_uvs.nc"
    if not os.path.exists(cb_file):
        continue
    print(cb_id, lon_cb, lat_cb)

    with xr.open_dataset(cb_file) as ds_cb:
        # print(ds_cb)
        # Coastal buoy currents are stored in cm/s; convert to m/s to match gu/gv.
        cb_u = 0.01 * ds_cb.U
        cb_v = 0.01 * ds_cb.V
        cb_s = 0.01 * ds_cb.speed
        cb_dir = ds_cb.direction

    # print(cb_v.count().item())
    df_4 = find_n_closest_points(df_all, lon_cb, lat_cb, n=4)
    for nth in range(4):
        row = df_4.iloc[nth]

        sat = row["sat"]
        track_number_self = str(row["track_self"])
        track_number_other = str(row["track_other"])
        lon1 = row["lons_inter"]
        lat1 = row["lats_inter"]
        distance_km = row["distance_km"]
        angle_acute = row["angle_acute"]
        angle_obtuse = row["angle_obtuse"]

        if abs(lat1) < 2:
            continue

        print(
            sat,
            cb_id,
            track_number_self,
            track_number_other,
            lon1,
            lat1,
            angle_acute,
            angle_obtuse,
        )

        f_guv = f"{guv_dir}/guv_at_intersection_{sat}_{track_number_self}_{track_number_other}_{frac1}.nc"
        if not os.path.exists(f_guv):
            print(f"missing file: {f_guv}")
            continue

        with xr.open_dataset(f_guv) as ds_guv:
            gu = ds_guv.gu
            gv = ds_guv.gv

        (
            lons_track_self,
            lats_track_self,
            lon_equat_self,
            lat_equat_self,
            lon_coast_self,
            lat_coast_self,
            angle_self,
        ) = get_track_geometry(sat, track_number_self)
        (
            lons_track_other,
```

```
            lats_track_other,
            lon_equat_other,
            lat_equat_other,
            lon_coast_other,
            lat_coast_other,
            angle_other,
        ) = get_track_geometry(sat, track_number_other)

        # In precomputed guv files, exact zeros often represent filtered/missing
        # points from upstream processing; exclude them before collocation.
        gu_cut = gu.where(np.abs(gu) > 1e-12)
        gv_cut = gv.where(np.abs(gv) > 1e-12)

        # Collocate cb at track-gu timestamps; avoid sat-dependent smoothing.
        cb_u_match = cb_u.reindex(
            time=gu_cut.time,
            method="nearest",
            tolerance=np.timedelta64(24, "h"),
        )
        valid = np.isfinite(gu_cut) & np.isfinite(cb_u_match)
        gu_pair = gu_cut.where(valid)
        cb_u_pair = cb_u_match.where(valid)

        df_pairs = xr.Dataset({"gu": gu_pair, "cb_u": cb_u_pair}).to_dataframe().reset_index()
        df_pairs = df_pairs.dropna(subset=["gu", "cb_u"])
        print(df_pairs.columns)
        matched_pairs = len(df_pairs)
        if matched_pairs > 0:
            """All these will broadcasted along length of gu, cb_u"""
            df_pairs["sat"] = sat
            df_pairs["cb_id"] = cb_id
            df_pairs["distance_km"] = distance_km
            df_pairs["track_self"] = track_number_self
            df_pairs["track_other"] = track_number_other
            df_pairs["matched_pairs"] = matched_pairs
            df_pairs["timestamp"] = pd.to_datetime(df_pairs["time"]).astype(str)
            print(df_pairs.columns)
            matched_rows_all.append(
                df_pairs[
                    [
                        "sat",
                        "cb_id",
                        "distance_km",
                        "track_self",
                        "track_other",
                        "matched_pairs",
                        "timestamp",
                        "gu",
                        "cb_u",
                    ]
                ]
            )

        # check valid points
        ngp = np.sum(valid)
        if ngp < 2:
            continue
        fig = plt.figure(figsize=(12, 8))
        ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
        ax1 = fig.add_subplot(2, 1, 2)
        cb_u_pair.plot(ax=ax1, color="r", linewidth=2, label="cb")
        gu_pair.plot(ax=ax1, color="k", linewidth=2, label="track")

        corr = xs.pearson_r(gu_pair, cb_u_pair, dim="time", skipna=True)
        corr1 = corr.item()

        info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
        print(f"Correlation between gu and cb u: {corr1}")
        plt.text(
            0.25,
            0.90,
            info,
            transform=ax1.transAxes,
            fontsize=12,
            fontweight="bold",
        )
        plt.legend()

        rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
        decorate_axis(ax2, "", *TRACKS_REG, step=10)
        ax2.grid()
        ax2.plot(
            lon_cb,
            lat_cb,
            c="r",
            marker="o",
            markersize=4,
            markerfacecolor="red",
            markeredgecolor="red",
            markeredgewidth=2,
        )

        ax2.plot(
            lon1,
            lat1,
            c="k",
            marker=".",
            markersize=8,
            markerfacecolor="black",
            markeredgecolor="black",
            markeredgewidth=2,
        )
        plt.text(lon_cb, lat_cb, f"{cb_id}", fontsize=10, color="k")
        ax2.scatter(
            lons_track_self, lats_track_self, marker=".", color="c", s=4
        )
        ax2.scatter(
            lons_track_other, lats_track_other, marker=".", color="c", s=4
        )

        lonm, latm = get_point_at_distance(
            lon_equat_self,
            lat_equat_self,
            lon_coast_self,
            lat_coast_self,
            d,
        )
```

```
        info = f"{sat} {track_number_self} {track_number_other} {distance_km:.2f} km"
        plt.text(
            0.55,
            0.90,
            info,
            transform=ax2.transAxes,
            fontsize=10,
            fontweight="bold",
        )
        if is_within_region(lonm, latm, *TRACKS_REG):
            plt.text(
                lonm,
                latm,
                s=track_number_self,
                fontsize=14,
                rotation=angle_self,
                color="w",
            )

        lonm, latm = get_point_at_distance(
            lon_equat_other,
            lat_equat_other,
            lon_coast_other,
            lat_coast_other,
            d,
        )
        if is_within_region(lonm, latm, *TRACKS_REG):
            plt.text(
                lonm,
                latm,
                s=track_number_other,
                fontsize=14,
                rotation=angle_other,
                color="w",
            )

        plt.show()
        plt.close("all")

if len(matched_rows_all) > 0:
    df_matched_all = pd.concat(matched_rows_all, ignore_index=True)
    df_matched_all.to_csv(out_match_csv, index=False)
    print(f"saved global match csv: {out_match_csv}, rows={len(df_matched_all)}")
#
dse.close()

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *


def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    return u, v


def haversine_distance(lon1, lat1, lon2, lat2):
    """Calculate great-circle distance between (lon1, lat1)
    and arrays (lon2, lat2) in kilometers."""
    R = 6371  # Radius of Earth in km
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlon = lon2 - lon1
    dlat = lat2 - lat1
    a = (np.sin(dlat / 2)**2 +
         np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2)**2)
    return 2 * R * np.arcsin(np.sqrt(a))


def find_n_closest_points(df, lon0, lat0, n=4):
    """Return the n rows in df closest to (lon0, lat0) based on
    lons_inter/lats_inter."""
    distances = haversine_distance(lon0, lat0, df["lons_inter"].values,
                                   df["lats_inter"].values)
    df = df.copy()
    df["distance_km"] = distances
    return df.nsmallest(n, "distance_km")


cmap1 = cmo.cm.diff
d = 1.5
dpth = 15.0
height, width = 9, 9
oscar_dir = "/home/srinivasu/allData/oscar_0.25/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
cb_data_dir = "/home/srinivasu/slnew/xtrackd/coastal/"
nth = 0

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

for cb_id in cb_d.keys():
    lon_cb, lat_cb = cb_d[cb_id]
    ds_cb = xr.open_dataset(f"{cb_data_dir}/{cb_id.upper()}_uvs.nc")
    print(ds_cb)

    cb_u = ds_cb.U
    cb_v = ds_cb.V
    cb_s = ds_cb.speed
    cb_dir = ds_cb.direction

    # plt.xlim(datetime(2016, 6, 1), datetime(2018, 6, 30))
    cb_u = cb_u.resample(time="1D").mean(dim="time")
    cb_v = cb_v.resample(time="1D").mean(dim="time")
    cb_s = cb_s.resample(time="1D").mean(dim="time")
    # print(N)
    df_4 = find_n_closest_points(df_all, lon_cb, lat_cb, n=1)
    # print(cb_id, lon_cb, lat_cb, "\n", df_4.head(1))
    sat = df_4["sat"].values[0]
    track_number_self = df_4["track_self"].apply(lambda x: str(x)).values[nth]
    track_number_other = df_4["track_other"].apply(lambda x: str(x)).values[nth]
    lons_inter1 = df_4["lons_inter"].values[nth]
    lats_inter1 = df_4["lats_inter"].values[nth]
    lon1 = lons_inter1
    lat1 = lats_inter1
    x_from_coast_self1 = df_4["x_from_coast_self"].values[nth]
```

```
    x_from_coast_other1 = df_4["x_from_coast_other"].values[nth]
    angle_acute = df_4["angle_acute"].values[nth]
    angle_obtuse = df_4["angle_obtuse"].values[nth]
    print(
        sat,
        cb_id,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_self}_loess_0.2.nc"
    f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_other}_loess_0.2.nc"
    ds_self = xr.open_dataset(f_self, decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    # u_oscar_at = u_oscar.sel(lon=lon1, lat=lat1, method="nearest")
    # v_oscar_at = v_oscar.sel(lon=lon1, lat=lat1, method="nearest")
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    # if track_path_self.intersects(track_path_other):
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    # Better approach:
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)

    gu, gv = geostrophic_components_from_a(gc_self_at, gc_other_at, a1, a2)
    # other_times.append((track_number_other, abs(dist_time)))
    fig1, ax1 = plt.subplots(1, 1, figsize=(12, 8), layout="constrained")
    gu_cut = -1 * gu.interp(time=cb_u.time)
    gv_cut = gv.interp(time=cb_v.time)
    # gu_cut.plot(ax=ax1, label=f"track zonal", color="r", linewidth=2)
    # cb_u.plot(ax=ax1, color="b", linewidth=2, label="cb")
    # u_oscar_at.plot(ax=ax1, color="b", linewidth=2, label="oscar u")
    # corr = xs.pearson_r(gu_cut, cb_u, dim="time", skipna=True)
    gv_cut.plot(ax=ax1, label=f"track zonal", color="r", linewidth=2)
    cb_v.plot(ax=ax1, color="b", linewidth=2, label="cb")
    # v_oscar_at.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
    corr = xs.pearson_r(gv_cut, cb_v, dim="time", skipna=True)
    corr1 = corr.item()
    # print(f"Correlation between gu and u_oscar_at: {corr1}")
    print(f"Correlation between gv and v_oscar_at: {corr1}")
    plt.text(
        0.15,
        0.95,
        f"intersection point ({lon1:.2f}, {lat1:.2f})",
        transform=ax1.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    plt.legend()
    fig2, ax2 = plt.subplots(
        1,
        1,
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(width, height),
        layout="constrained",
    )
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG)
    ax2.grid()
    ax2.plot(
        lon_cb,
        lat_cb,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor="red",
        markeredgecolor="black",
        markeredgewidth=2,
    )
    # plot plat form code of cb at lon_cb, lat_cb
    plt.text(lon_cb, lat_cb, f"{cb_id}", fontsize=10, color="k")
    ax2.scatter(
        lons_track_self,
        lats_track_self,
```

```
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="w",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="w",
        )
    plt.show()
    plt.close("all")

dse.close()
```

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import median_abs_deviation
import pandas as pd
import math
import os
from tools_xtrackm import *


def mad_filter_1d(x, thresh=3.5):
    if len(x) < 3 or np.all(np.isnan(x)):
        return np.zeros_like(x, dtype=bool)
    valid_mask = ~np.isnan(x)
    if np.sum(valid_mask) < 3:
        return np.zeros_like(x, dtype=bool)
    # size of valid data might be less than x depending on the
    # number of NaNs
    valid_data = x[valid_mask]
    median = np.median(valid_data)
    mad = median_abs_deviation(valid_data, scale='normal')
    if mad == 0:
        mad = np.median(np.abs(valid_data - median)) * 1.4826
        if mad == 0:
            return np.zeros_like(x, dtype=bool)
    modified_z_scores = 0.6745 * (x - median) / mad
    outliers = np.abs(modified_z_scores) > thresh
    # size of outliers is same as the size of x
    return outliers


for sat in sats_new:
    # read guv_at_intersections_HY2B_0.2.npz file
    arr = np.load(f"guv_at_intersections_{sat}_0.2.npz")
    arr1 = arr['gu_all']
    # remove nans for arr
    arr_nonan = arr1[~np.isnan(arr1)]
    # filter if arr is greater than 3 or less than -3
    arr_filter1 = arr_nonan[(arr_nonan > -3.5) & (arr_nonan < 3.5)]
    outliers = mad_filter_1d(arr_filter1, thresh=3.5)
    arr_filter = arr_filter1[~outliers]
    mean = np.nanmean(arr_filter)
    mx = np.nanmax(arr_filter)
    mn = np.nanmin(arr_filter)
    md = np.nanmedian(arr_filter)
    # plot histograms
    plt.hist(arr_filter, bins=100)
    # add text of mean, mx, min on plot
    plt.text(0.9, 0.9, f"sat={sat}\nmedian={md:.3f}\nmean={mean:.3f}\nmax={mx:.3f}\nmin={mn:.3f}", transform=plt.gca().transAxes, ha='right', va='top', fontsize=10)
    plt.show()

```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

only_df = pd.read_csv(
    "oscar_onlyVStrack_corrs_rmses_biass_TP+J1+J2+J3+S6A_u.csv")
ekmn_df = pd.read_csv(
    "oscar_ekmnVStrack_corrs_rmses_biass_TP+J1+J2+J3+S6A_u.csv")

# Create unique column combining zero-padded sat and track_self, track_other
only_df["unique_id"] = (only_df["sat"].apply(lambda s: s.zfill(20)) + "_" +
                        only_df["track_self"].astype(str) + "_" +
                        only_df["track_other"].astype(str))
only_df = only_df.set_index("unique_id")

ekmn_df["unique_id"] = (ekmn_df["sat"].apply(lambda s: s.zfill(20)) + "_" +
                        ekmn_df["track_self"].astype(str) + "_" +
                        ekmn_df["track_other"].astype(str))
ekmn_df = ekmn_df.set_index("unique_id")

common_ids = only_df.index.intersection(ekmn_df.index)
only_df = only_df.loc[common_ids]
ekmn_df = ekmn_df.loc[common_ids]

# Extract metrics for plotting
metrics = ["biass", "corrs", "rmses"]
titles = ["Bias", "Correlation", "Rmse"]

# Prepare figure with 2x2 grid
fig, axs = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle("comparison metrics with oscar u: track gu only VS track gu + ekman", fontsize=14)

summary_lines = []

for i, metric in enumerate(metrics):
    ax = axs[i // 2, i % 2]
    valid_mask = only_df[metric].notna() & ekmn_df[metric].notna()
    only_df1 = only_df[valid_mask]
    ekmn_df1 = ekmn_df[valid_mask]
    # Get data
    only_df_data = only_df1[metric].values
    ekmn_df_data = ekmn_df1[metric].values
    indices = np.arange(len(only_df_data))
    ax.plot(indices, only_df_data, label="track gu only")
    ax.plot(indices, ekmn_df_data, label="track gu+ekman")
    # draw zero line
    ax.axhline(0, color="gray", linestyle="--")
    ax.set_xlabel("Data Point Index")
    ax.set_ylabel(titles[i])
    ax.set_title(titles[i])
    ax.legend()
    # collect summary text for this metric
    if len(only_df_data) > 0:
        mean_o = np.nanmean(only_df_data)
        mean_e = np.nanmean(ekmn_df_data)
        diff = mean_e - mean_o
        cnt = len(only_df_data)
        summary_lines.append(f"\n{titles[i]}:")
        summary_lines.append(f" count = {cnt}")
        summary_lines.append(f" mean track u = {mean_o:.4f}")
        summary_lines.append(f" mean track u + ekman = {mean_e:.4f}")
        summary_lines.append(f" mean improvement = {diff:.4f}")
    else:
        summary_lines.append(f"\n{titles[i]}: no valid paired data")

ax_sum = axs[1, 1]
ax_sum.axis("off")
text = " ".join(summary_lines)

ax_sum.annotate(
    "\n".join(summary_lines),
    xy=(0, 1),
    xycoords="axes fraction",
    va="top",
    ha="left",
    fontsize=10,
    family="monospace",
    bbox=dict(boxstyle="round,pad=0.5", facecolor="white", alpha=0.7),
    annotation_clip=False,
)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(f"track_guvVSoscar_ww_ekman_u.png")
plt.show()

```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
import xskillscore as xs
from geopy import distance
from tools_xtrackm import *


@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(
        f_track, engine="netcdf4", decode_times=False
    ) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


def build_unique_str(sat, row):
    track_self = track_to_str(row["track_self"])
    track_other = track_to_str(row["track_other"])
    return f"{sat}_{track_self}_{track_other}"


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)

cmap1 = cmo.cm.diff
d = 1.5
height, width = 9, 9
frac1 = "0.05"
dpth = 15.0

dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
omni_data_dir = "/home/srinivasu/allData/MB_cleaned/"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
out_match_csv = "omni_guv_c_nearest_global_matches.ek.csv"
matched_rows_all = []
omni_data_dir = "/home/srinivasu/allData/MB_cleaned/raju_omni_clean"
EKMAN_DIR = "/home/srinivasu/xtrackm/ekman/ekman_at_intersections"

# BD08_current_sst.nc

df_all_list = []
for sat in sats_new:
    df_all_list.append(pd.read_csv(f"tracks_intersections_{sat}_1.csv"))
df_all = pd.concat(df_all_list, ignore_index=True)

for omni_id in omni_d.keys():
    lon_omni, lat_omni = omni_d[omni_id]
    print(omni_id, lon_omni, lat_omni)
    omni_file = f"{omni_data_dir}/{omni_id}_current_sst.nc"
    print(omni_file)
    if not os.path.exists(omni_file):
        print(f"File not found: {omni_file}")
        continue
    if abs(lat_omni) < 2:
        continue

    ds_omni = xr.open_dataset(omni_file)
    print(ds_omni)
    omni_u = ds_omni.u_current.interp(current_depth=dpth)
    # omni_u = omni_u.rolling(time=10, center=True).mean()
    omni_u = omni_u
    omni_v = ds_omni.v_current.interp(current_depth=dpth)
    # omni_v = omni_v.rolling(time=10, center=True).mean()
    omni_v = omni_v
    N = omni_v.count().item()

    # print(omni_v.count().item())
    df_4 = find_n_closest_points(df_all, lon_omni, lat_omni, n=4)
    for nth in range(4):
        row = df_4.iloc[nth]

        sat = row["sat"]
        track_number_self = str(row["track_self"])
        track_number_other = str(row["track_other"])
        lon1 = row["lons_inter"]
        lat1 = row["lats_inter"]
        distance_km = row["distance_km"]
        angle_acute = row["angle_acute"]
        angle_obtuse = row["angle_obtuse"]

        unique_str = build_unique_str(sat, row)
        f_ek = f"{EKMAN_DIR}/{unique_str}_ekman_at_intersections.nc"
        ds_ek = xr.open_dataset(f_ek)
        u_ek = ds_ek['u_at']
        v_ek = ds_ek['v_at']

        if abs(lat1) < 2:
            continue

        print(
            sat,
            omni_id,
            track_number_self,
            track_number_other,
            lon1,
            lat1,
```

```
            angle_acute,
            angle_obtuse,
        )
        print((
        f"\nSELECT_SAT = '{sat}'"
        f"\nSELECT_OMNI_ID = '{omni_id}'"
        f"\nSELECT_TRACK_SELF = '{track_number_self}'"
        f"\nSELECT_TRACK_OTHER = '{track_number_other}'"
        ))

        f_guv = f"{guv_dir}/guv_at_intersection_{sat}_{track_number_self}_{track_number_other}_{frac1}.nc"
        if not os.path.exists(f_guv):
            print(f"missing file: {f_guv}")
            continue

        with xr.open_dataset(f_guv) as ds_guv:
            gu = ds_guv.gu
            gv = ds_guv.gv

        (
            lons_track_self,
            lats_track_self,
            lon_equat_self,
            lat_equat_self,
            lon_coast_self,
            lat_coast_self,
            angle_self,
        ) = get_track_geometry(sat, track_number_self)
        (
            lons_track_other,
            lats_track_other,
            lon_equat_other,
            lat_equat_other,
            lon_coast_other,
            lat_coast_other,
            angle_other,
        ) = get_track_geometry(sat, track_number_other)

        gu_cut = gu
        gv_cut = gv

        # Collocate omni at track-gu timestamps; avoid sat-dependent smoothing.
        omni_u_match = omni_u.reindex(
            time=gu_cut.time,
            method="nearest",
            tolerance=np.timedelta64(24, "h"),
        )
        ek_u_match = u_ek.reindex(
            time=gu_cut.time,
            method="nearest",
            tolerance=np.timedelta64(24, "h"),
        )
        flat = (omni_u_match.diff("time") == 0).reindex(time=omni_u_match.time, fill_value=False)
        omni_u_match = omni_u_match.where(~flat)
        valid = np.isfinite(gu_cut) & np.isfinite(omni_u_match)
        gu_pair = gu_cut.where(valid)
        omni_u_pair = omni_u_match.where(valid)
        ek_u_pair = ek_u_match.where(valid)
        gu_ek_pair = gu_pair + ek_u_pair

        df_pairs = xr.Dataset({"gu": gu_pair, "omni_u": omni_u_pair, "u_ek": ek_u_pair}).to_dataframe().reset_index()
        df_pairs = df_pairs.dropna(subset=["gu", "omni_u"])
        matched_pairs = len(df_pairs)
        if matched_pairs > 0:
            df_pairs["sat"] = sat
            df_pairs["omni_id"] = omni_id
            df_pairs["distance_km"] = distance_km
            df_pairs["track_self"] = track_number_self
            df_pairs["track_other"] = track_number_other
            df_pairs["matched_pairs"] = matched_pairs
            df_pairs["timestamp"] = pd.to_datetime(df_pairs["time"]).astype(str)
            matched_rows_all.append(
                df_pairs[
                    [
                        "sat",
                        "omni_id",
                        "distance_km",
                        "track_self",
                        "track_other",
                        "matched_pairs",
                        "timestamp",
                        "gu",
                        "omni_u",
                        "u_ek",
                    ]
                ]
            )

        # check valid points
        ngp = np.sum(valid)
        if ngp < 2:
            continue
        fig = plt.figure(figsize=(12, 8))
        ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
        ax1 = fig.add_subplot(2, 1, 2)
        omni_u_pair.plot(ax=ax1, color="r", linewidth=2, label="omni")
        gu_pair.plot(ax=ax1, color="k", linewidth=2, label="track")
        gu_ek_pair.plot(ax=ax1, color="b", linewidth=2, label="track+ekman")

        corr = xs.pearson_r(gu_pair, omni_u_pair, dim="time", skipna=True)
        corr1 = corr.item()

        info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
        print(f"Correlation between gu and omni u: {corr1}")
        plt.text(
            0.25,
            0.95,
            info,
            transform=ax1.transAxes,
            fontsize=12,
            fontweight="bold",
        )
        plt.legend()

        rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
        decorate_axis(ax2, "", *TRACKS_REG, step=10)
        ax2.grid()
```

```
        ax2.plot(
            lon_omni,
            lat_omni,
            c="r",
            marker="o",
            markersize=4,
            markerfacecolor="red",
            markeredgecolor="red",
            markeredgewidth=2,
        )

        ax2.plot(
            lon1,
            lat1,
            c="k",
            marker=".",
            markersize=8,
            markerfacecolor="black",
            markeredgecolor="black",
            markeredgewidth=2,
        )
        plt.text(lon_omni, lat_omni, f"{omni_id}", fontsize=10, color="k")
        ax2.scatter(
            lons_track_self, lats_track_self, marker=".", color="c", s=4
        )
        ax2.scatter(
            lons_track_other, lats_track_other, marker=".", color="c", s=4
        )

        lonm, latm = get_point_at_distance(
            lon_equat_self,
            lat_equat_self,
            lon_coast_self,
            lat_coast_self,
            d,
        )
        info = f"{sat} {track_number_self} {track_number_other} {distance_km:.2f} km"
        plt.text(
            0.50,
            0.92,
            info,
            transform=ax2.transAxes,
            fontsize=10,
            fontweight="bold",
        )
        if is_within_region(lonm, latm, *TRACKS_REG):
            plt.text(
                lonm,
                latm,
                s=track_number_self,
                fontsize=14,
                rotation=angle_self,
                color="w",
            )

        lonm, latm = get_point_at_distance(
            lon_equat_other,
            lat_equat_other,
            lon_coast_other,
            lat_coast_other,
            d,
        )
        if is_within_region(lonm, latm, *TRACKS_REG):
            plt.text(
                lonm,
                latm,
                s=track_number_other,
                fontsize=14,
                rotation=angle_other,
                color="w",
            )

        # plt.show()
        plt.close("all")
    ds_omni.close()

if len(matched_rows_all) > 0:
    df_matched_all = pd.concat(matched_rows_all, ignore_index=True)
    df_matched_all.to_csv(out_match_csv, index=False)
    print(f"saved global match csv: {out_match_csv}, rows={len(df_matched_all)}")

dse.close()

```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
import xskillscore as xs
from geopy import distance
from tools_xtrackm import *
# codex resume 019cc947-b006-7d82-8941-d68e330c388a

@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(
        f_track, engine="netcdf4", decode_times=False
    ) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg



def build_unique_str(sat, row):
    track_self = track_to_str(row["track_self"])
    track_other = track_to_str(row["track_other"])
    return f"{sat}_{track_self}_{track_other}"


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)



cmap1 = cmo.cm.diff
d = 1.5
dpth = 15
height, width = 9, 9
frac1 = "0.05"
method = "loess"


dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
omni_data_dir = "/home/srinivasu/allData/MB_cleaned/raju_omni_clean"
guv_dir = f"/home/srinivasu/xtrackm/guv/guv_at_intersections_{method}"
out_match_csv = "omni_guv_c_nearest_global_matches.csv"
matched_rows_all = []
# SELECT_SAT = "TP+J1+J2+J3+S6A"
# SELECT_OMNI_ID = "BD09"
# SELECT_TRACK_SELF = "53"
# SELECT_TRACK_OTHER = "14"
SELECT_SAT = 'S3A'
SELECT_OMNI_ID = 'BD10'
SELECT_TRACK_SELF = '522'
SELECT_TRACK_OTHER = '279'
SELECT_SAT = 'S3A'
SELECT_OMNI_ID = 'BD10'
SELECT_TRACK_SELF = '636'
SELECT_TRACK_OTHER = '393'

EKMAN_DIR = "/home/srinivasu/xtrackm/ekman/ekman_at_intersections"
# TP+J1+J2+J3+S6A BD09 53 14
# S3A BD10 522 279

df_all_list = []
for sat in sats_new:
    df_all_list.append(pd.read_csv(f"tracks_intersections_{sat}_1.csv"))
df_all = pd.concat(df_all_list, ignore_index=True)


lat_omni, lon_omni = omni_d[SELECT_OMNI_ID]
omni_file = f"{omni_data_dir}/{SELECT_OMNI_ID}_current_sst.nc"
print(omni_file)
if not os.path.exists(omni_file):
    print(f"File not found: {omni_file}")
    raise(f"no omni file")
if abs(lat_omni) < 2:
    raise(f"lat less than 2")

ds_omni = xr.open_dataset(omni_file)
print(ds_omni)
# print(ds_omni)
ds_omni = xr.open_dataset(omni_file)
print(ds_omni)
omni_u = ds_omni.u_current.interp(current_depth=dpth)
# omni_u = omni_u.rolling(time=10, center=True).mean()
omni_u = omni_u
omni_v = ds_omni.v_current.interp(current_depth=dpth)
# omni_v = omni_v.rolling(time=10, center=True).mean()
omni_v = omni_v
N = omni_v.count().item()

df_selected = df_all[
    (df_all["sat"] == SELECT_SAT)
    & (df_all["track_self"].astype(str) == SELECT_TRACK_SELF)
    & (df_all["track_other"].astype(str) == SELECT_TRACK_OTHER)
].copy()
if df_selected.empty:
    raise("selected sat/track combination not found in intersections file")
lons = df_selected["lons_inter"].to_numpy(dtype=float)
lats = df_selected["lats_inter"].to_numpy(dtype=float)
distances = [distance.distance((lat, lon), (lat_omni, lon_omni)).km for lon, lat in zip(lons, lats)]
df_selected["distance_km"] = distances
```

```
row = df_selected.nsmallest(1, "distance_km").iloc[0]

sat = row["sat"]
track_number_self = str(row["track_self"])
track_number_other = str(row["track_other"])
lon1 = row["lons_inter"]
lat1 = row["lats_inter"]
distance_km = row["distance_km"]
angle_acute = row["angle_acute"]
angle_obtuse = row["angle_obtuse"]
unique_str = build_unique_str(sat, row)
f_ek = f"{EKMAN_DIR}/{unique_str}_ekman_at_intersections.nc"
ds_ek = xr.open_dataset(f_ek)
u_ek = ds_ek['u_at']
v_ek = ds_ek['v_at']

print(
    sat,
    SELECT_OMNI_ID,
    track_number_self,
    track_number_other,
    lon1,
    lat1,
    angle_acute,
    angle_obtuse,
)

f_guv = f"{guv_dir}/guv_at_intersection_{sat}_{track_number_self}_{track_number_other}_{method}_{frac1}.nc"
if not os.path.exists(f_guv):
    print(f"missing file: {f_guv}")
    raise("missing file")

with xr.open_dataset(f_guv) as ds_guv:
    gu = ds_guv.gu
    gv = ds_guv.gv

(
    lons_track_self,
    lats_track_self,
    lon_equat_self,
    lat_equat_self,
    lon_coast_self,
    lat_coast_self,
    angle_self,
) = get_track_geometry(sat, track_number_self)
(
    lons_track_other,
    lats_track_other,
    lon_equat_other,
    lat_equat_other,
    lon_coast_other,
    lat_coast_other,
    angle_other,
) = get_track_geometry(sat, track_number_other)

gu_cut = gu
gv_cut = gv

# gu_cut_6 = gu.rolling(x=6, center=True).mean()
# Collocate omni at track-gu timestamps; avoid sat-dependent smoothing.
omni_u_match = omni_u.reindex(
    time=gu_cut.time,
    method="nearest",
    tolerance=np.timedelta64(24, "h"),
)
ek_u_match = u_ek.reindex(
    time=gu_cut.time,
    method="nearest",
    tolerance=np.timedelta64(24, "h"),
)
valid = np.isfinite(gu_cut) & np.isfinite(omni_u_match)
gu_pair = gu_cut.where(valid)
# gu_6_pair = gu_cut_6.where(valid)
omni_u_pair = omni_u_match.where(valid)
ek_u_pair = ek_u_match.where(valid)

gu_ek_pair = gu_pair + ek_u_pair


df_pairs = xr.Dataset({"gu": gu_pair, "omni_u": omni_u_pair}).to_dataframe().reset_index()
df_pairs = df_pairs.dropna(subset=["gu", "omni_u"])

# check valid points
fig = plt.figure(figsize=(12, 8))
ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax1 = fig.add_subplot(2, 1, 2)
omni_u_pair.plot(ax=ax1, color="r", linewidth=2, label="omni")
gu_pair.plot(ax=ax1, color="k", linewidth=2, label="track")
# gu_6_pair.plot(ax=ax1, color="g", linewidth=2, label="track x 6 smooth")
gu_ek_pair.plot(ax=ax1, color="b", linewidth=2, label="track+ekman")

corr = xs.pearson_r(gu_pair, omni_u_pair, dim="time", skipna=True)
corr1 = corr.item()

info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
print(f"Correlation between gu and omni u: {corr1}")
plt.text(
    0.25,
    0.90,
    info,
    transform=ax1.transAxes,
    fontsize=12,
    fontweight="bold",
)
plt.legend()

rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=10)
ax2.grid()
ax2.plot(
    lon_omni,
    lat_omni,
    c="r",
    marker="o",
    markersize=4,
    markerfacecolor="red",
    markeredgecolor="red",
```

```
    markeredgewidth=2,
)

ax2.plot(
    lon1,
    lat1,
    c="k",
    marker=".",
    markersize=8,
    markerfacecolor="black",
    markeredgecolor="black",
    markeredgewidth=2,
)
plt.text(lon_omni, lat_omni, f"{SELECT_OMNI_ID}", fontsize=10, color="k")
ax2.scatter(
    lons_track_self, lats_track_self, marker=".", color="c", s=4
)
ax2.scatter(
    lons_track_other, lats_track_other, marker=".", color="c", s=4
)

lonm, latm = get_point_at_distance(
    lon_equat_self,
    lat_equat_self,
    lon_coast_self,
    lat_coast_self,
    d,
)
info = f"{sat} {track_number_self} {track_number_other} {distance_km:.2f} km"
plt.text(
    0.55,
    0.90,
    info,
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=angle_self,
        color="w",
    )

lonm, latm = get_point_at_distance(
    lon_equat_other,
    lat_equat_other,
    lon_coast_other,
    lat_coast_other,
    d,
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=angle_other,
        color="w",
    )

plt.show()
plt.close("all")

dse.close()

```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
import xskillscore as xs
from geopy import distance
from tools_xtrackm import *


@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(
        f_track, engine="netcdf4", decode_times=False
    ) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


cmap1 = cmo.cm.diff
d = 1.5
height, width = 9, 9
frac1 = "0.05"
dpth = 15.0

dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
omni_data_dir = "/home/srinivasu/allData/MB_cleaned/"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
out_match_csv = "omni_guv_c_nearest_global_matches.csv"
matched_rows_all = []
omni_data_dir = "/home/srinivasu/allData/MB_cleaned/raju_omni_clean"

# BD08_current_sst.nc

df_all_list = []
for sat in sats_new:
    df_all_list.append(pd.read_csv(f"tracks_intersections_{sat}_1.csv"))
df_all = pd.concat(df_all_list, ignore_index=True)


for omni_id in omni_d.keys():
    lon_omni, lat_omni = omni_d[omni_id]
    print(omni_id, lon_omni, lat_omni)
    omni_file = f"{omni_data_dir}/{omni_id}_current_sst.nc"
    print(omni_file)
    if not os.path.exists(omni_file):
        print(f"File not found: {omni_file}")
        continue
    if abs(lat_omni) < 2:
        continue

    ds_omni = xr.open_dataset(omni_file)
    print(ds_omni)
    omni_u = ds_omni.u_current.interp(current_depth=dpth)
    # omni_u = omni_u.rolling(time=10, center=True).mean()
    omni_u = omni_u
    omni_v = ds_omni.v_current.interp(current_depth=dpth)
    # omni_v = omni_v.rolling(time=10, center=True).mean()
    omni_v = omni_v
    N = omni_v.count().item()

    # print(omni_v.count().item())
    df_4 = find_n_closest_points(df_all, lon_omni, lat_omni, n=4)
    for nth in range(4):
        row = df_4.iloc[nth]

        sat = row["sat"]
        track_number_self = str(row["track_self"])
        track_number_other = str(row["track_other"])
        lon1 = row["lons_inter"]
        lat1 = row["lats_inter"]
        distance_km = row["distance_km"]
        angle_acute = row["angle_acute"]
        angle_obtuse = row["angle_obtuse"]

        if abs(lat1) < 2:
            continue

        print(
            sat,
            omni_id,
            track_number_self,
            track_number_other,
            lon1,
            lat1,
            angle_acute,
            angle_obtuse,
        )

        f_guv = f"{guv_dir}/guv_at_intersection_{sat}_{track_number_self}_{track_number_other}_{frac1}.nc"
        if not os.path.exists(f_guv):
            print(f"missing file: {f_guv}")
            continue

        with xr.open_dataset(f_guv) as ds_guv:
            gu = ds_guv.gu
            gv = ds_guv.gv

        (
            lons_track_self,
            lats_track_self,
            lon_equat_self,
            lat_equat_self,
```

```
            lon_coast_self,
            lat_coast_self,
            angle_self,
        ) = get_track_geometry(sat, track_number_self)
        (
            lons_track_other,
            lats_track_other,
            lon_equat_other,
            lat_equat_other,
            lon_coast_other,
            lat_coast_other,
            angle_other,
        ) = get_track_geometry(sat, track_number_other)

        gu_cut = gu
        gv_cut = gv

        # Collocate omni at track-gu timestamps; avoid sat-dependent smoothing.
        omni_u_match = omni_u.reindex(
            time=gu_cut.time,
            method="nearest",
            tolerance=np.timedelta64(24, "h"),
        )
        flat = (omni_u_match.diff("time") == 0).reindex(time=omni_u_match.time, fill_value=False)
        omni_u_match = omni_u_match.where(~flat)
        valid = np.isfinite(gu_cut) & np.isfinite(omni_u_match)
        gu_pair = gu_cut.where(valid)
        omni_u_pair = omni_u_match.where(valid)

        df_pairs = xr.Dataset({"gu": gu_pair, "omni_u": omni_u_pair}).to_dataframe().reset_index()
        df_pairs = df_pairs.dropna(subset=["gu", "omni_u"])
        matched_pairs = len(df_pairs)
        if matched_pairs > 0:
            df_pairs["sat"] = sat
            df_pairs["omni_id"] = omni_id
            df_pairs["distance_km"] = distance_km
            df_pairs["track_self"] = track_number_self
            df_pairs["track_other"] = track_number_other
            df_pairs["matched_pairs"] = matched_pairs
            df_pairs["timestamp"] = pd.to_datetime(df_pairs["time"]).astype(str)
            matched_rows_all.append(
                df_pairs[
                    [
                        "sat",
                        "omni_id",
                        "distance_km",
                        "track_self",
                        "track_other",
                        "matched_pairs",
                        "timestamp",
                        "gu",
                        "omni_u",
                    ]
                ]
            )

        # check valid points
        ngp = np.sum(valid)
        if ngp < 2:
            continue
        fig = plt.figure(figsize=(12, 8))
        ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
        ax1 = fig.add_subplot(2, 1, 2)
        omni_u_pair.plot(ax=ax1, color="r", linewidth=2, label="omni")
        gu_pair.plot(ax=ax1, color="k", linewidth=2, label="track")

        corr = xs.pearson_r(gu_pair, omni_u_pair, dim="time", skipna=True)
        corr1 = corr.item()

        info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
        print(f"Correlation between gu and omni u: {corr1}")
        plt.text(
            0.25,
            0.95,
            info,
            transform=ax1.transAxes,
            fontsize=12,
            fontweight="bold",
        )
        plt.legend()

        rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
        decorate_axis(ax2, "", *TRACKS_REG, step=10)
        ax2.grid()
        ax2.plot(
            lon_omni,
            lat_omni,
            c="r",
            marker="o",
            markersize=4,
            markerfacecolor="red",
            markeredgecolor="red",
            markeredgewidth=2,
        )

        ax2.plot(
            lon1,
            lat1,
            c="k",
            marker=".",
            markersize=8,
            markerfacecolor="black",
            markeredgecolor="black",
            markeredgewidth=2,
        )
        plt.text(lon_omni, lat_omni, f"{omni_id}", fontsize=10, color="k")
        ax2.scatter(
            lons_track_self, lats_track_self, marker=".", color="c", s=4
        )
        ax2.scatter(
            lons_track_other, lats_track_other, marker=".", color="c", s=4
        )

        lonm, latm = get_point_at_distance(
            lon_equat_self,
            lat_equat_self,
            lon_coast_self,
```

```
            lat_coast_self,
            d,
        )
        info = f"{sat} {track_number_self} {track_number_other} {distance_km:.2f} km"
        plt.text(
            0.50,
            0.92,
            info,
            transform=ax2.transAxes,
            fontsize=10,
            fontweight="bold",
        )
        if is_within_region(lonm, latm, *TRACKS_REG):
            plt.text(
                lonm,
                latm,
                s=track_number_self,
                fontsize=14,
                rotation=angle_self,
                color="w",
            )

        lonm, latm = get_point_at_distance(
            lon_equat_other,
            lat_equat_other,
            lon_coast_other,
            lat_coast_other,
            d,
        )
        if is_within_region(lonm, latm, *TRACKS_REG):
            plt.text(
                lonm,
                latm,
                s=track_number_other,
                fontsize=14,
                rotation=angle_other,
                color="w",
            )

        plt.show()
        plt.close("all")
    ds_omni.close()

if len(matched_rows_all) > 0:
    df_matched_all = pd.concat(matched_rows_all, ignore_index=True)
    df_matched_all.to_csv(out_match_csv, index=False)
    print(f"saved global match csv: {out_match_csv}, rows={len(df_matched_all)}")

dse.close()
```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
import xskillscore as xs
from geopy import distance
from tools_xtrackm import *
# codex resume 019cc947-b006-7d82-8941-d68e330c388a

@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(
        f_track, engine="netcdf4", decode_times=False
    ) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


cmap1 = cmo.cm.diff
d = 1.5
dpth = 15
height, width = 9, 9
frac1 = "0.05"


dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
omni_data_dir = "/home/srinivasu/allData/MB_cleaned/raju_omni_clean"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
out_match_csv = "omni_guv_c_nearest_global_matches.csv"
matched_rows_all = []
SELECT_SAT = "TP+J1+J2+J3+S6A"
SELECT_OMNI_ID = "BD09"
SELECT_TRACK_SELF = "53"
SELECT_TRACK_OTHER = "14"
# TP+J1+J2+J3+S6A BD09 53 14

df_all_list = []
for sat in sats_new:
    df_all_list.append(pd.read_csv(f"tracks_intersections_{sat}_1.csv"))
df_all = pd.concat(df_all_list, ignore_index=True)


lat_omni, lon_omni = omni_d[SELECT_OMNI_ID]
omni_file = f"{omni_data_dir}/{SELECT_OMNI_ID}_current_sst.nc"
print(omni_file)
if not os.path.exists(omni_file):
    print(f"File not found: {omni_file}")
    raise(f"no omni file")
if abs(lat_omni) < 2:
    raise(f"lat less than 2")

ds_omni = xr.open_dataset(omni_file)
print(ds_omni)
# print(ds_omni)
ds_omni = xr.open_dataset(omni_file)
print(ds_omni)
omni_u = ds_omni.u_current.interp(current_depth=dpth)
# omni_u = omni_u.rolling(time=10, center=True).mean()
omni_u = omni_u
omni_v = ds_omni.v_current.interp(current_depth=dpth)
# omni_v = omni_v.rolling(time=10, center=True).mean()
omni_v = omni_v
N = omni_v.count().item()

df_selected = df_all[
    (df_all["sat"] == SELECT_SAT)
    & (df_all["track_self"].astype(str) == SELECT_TRACK_SELF)
    & (df_all["track_other"].astype(str) == SELECT_TRACK_OTHER)
].copy()
if df_selected.empty:
    raise("selected sat/track combination not found in intersections file")
lons = df_selected["lons_inter"].to_numpy(dtype=float)
lats = df_selected["lats_inter"].to_numpy(dtype=float)
distances = [distance.distance((lat, lon), (lat_omni, lon_omni)).km for lon, lat in zip(lons, lats)]
df_selected["distance_km"] = distances
row = df_selected.nsmallest(1, "distance_km").iloc[0]

sat = row["sat"]
track_number_self = str(row["track_self"])
track_number_other = str(row["track_other"])
lon1 = row["lons_inter"]
lat1 = row["lats_inter"]
distance_km = row["distance_km"]
angle_acute = row["angle_acute"]
angle_obtuse = row["angle_obtuse"]

print(
    sat,
    SELECT_OMNI_ID,
    track_number_self,
    track_number_other,
    lon1,
    lat1,
    angle_acute,
    angle_obtuse,
)

f_guv = f"{guv_dir}/guv_at_intersection_{sat}_{track_number_self}_{track_number_other}_{frac1}.nc"
if not os.path.exists(f_guv):
    print(f"missing file: {f_guv}")
    raise("missing file")

```

```
with xr.open_dataset(f_guv) as ds_guv:
    gu = ds_guv.gu
    gv = ds_guv.gv

(
    lons_track_self,
    lats_track_self,
    lon_equat_self,
    lat_equat_self,
    lon_coast_self,
    lat_coast_self,
    angle_self,
) = get_track_geometry(sat, track_number_self)
(
    lons_track_other,
    lats_track_other,
    lon_equat_other,
    lat_equat_other,
    lon_coast_other,
    lat_coast_other,
    angle_other,
) = get_track_geometry(sat, track_number_other)

gu_cut = gu
gv_cut = gv

# Collocate omni at track-gu timestamps; avoid sat-dependent smoothing.
omni_u_match = omni_u.reindex(
    time=gu_cut.time,
    method="nearest",
    tolerance=np.timedelta64(24, "h"),
)
valid = np.isfinite(gu_cut) & np.isfinite(omni_u_match)
gu_pair = gu_cut.where(valid)
omni_u_pair = omni_u_match.where(valid)

df_pairs = xr.Dataset({"gu": gu_pair, "omni_u": omni_u_pair}).to_dataframe().reset_index()
df_pairs = df_pairs.dropna(subset=["gu", "omni_u"])

# check valid points
fig = plt.figure(figsize=(12, 8))
ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax1 = fig.add_subplot(2, 1, 2)
omni_u_pair.plot(ax=ax1, color="r", linewidth=2, label="omni")
gu_pair.plot(ax=ax1, color="k", linewidth=2, label="track")

corr = xs.pearson_r(gu_pair, omni_u_pair, dim="time", skipna=True)
corr1 = corr.item()

info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
print(f"Correlation between gu and omni u: {corr1}")
plt.text(
    0.25,
    0.90,
    info,
    transform=ax1.transAxes,
    fontsize=12,
    fontweight="bold",
)
plt.legend()

rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=10)
ax2.grid()
ax2.plot(
    lon_omni,
    lat_omni,
    c="r",
    marker="o",
    markersize=4,
    markerfacecolor="red",
    markeredgecolor="red",
    markeredgewidth=2,
)

ax2.plot(
    lon1,
    lat1,
    c="k",
    marker=".",
    markersize=8,
    markerfacecolor="black",
    markeredgecolor="black",
    markeredgewidth=2,
)
plt.text(lon_omni, lat_omni, f"{SELECT_OMNI_ID}", fontsize=10, color="k")
ax2.scatter(
    lons_track_self, lats_track_self, marker=".", color="c", s=4
)
ax2.scatter(
    lons_track_other, lats_track_other, marker=".", color="c", s=4
)

lonm, latm = get_point_at_distance(
    lon_equat_self,
    lat_equat_self,
    lon_coast_self,
    lat_coast_self,
    d,
)
info = f"{sat} {track_number_self} {track_number_other} {distance_km:.2f} km"
plt.text(
    0.55,
    0.90,
    info,
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=angle_self,
        color="w",
```

```
    )

lonm, latm = get_point_at_distance(
    lon_equat_other,
    lat_equat_other,
    lon_coast_other,
    lat_coast_other,
    d,
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=angle_other,
        color="w",
    )

plt.show()
plt.close("all")

dse.close()

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import sys

cmap1 = cmo.cm.diff
sat = "S3A"
sat = "ERS1+ERS2+ENV+SRL"
sat = "TP+J1+J2+J3+S6A"
d = 1.5
height, width = 9, 9


def haversine_distance(lon1, lat1, lon2, lat2):
    """Calculate great-circle distance between (lon1, lat1)
   and arrays (lon2, lat2) in kilometers."""
    R = 6371  # Radius of Earth in km
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlon = lon2 - lon1
    dlat = lat2 - lat1
    a = (np.sin(dlat / 2)**2 +
         np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2)**2)
    return 2 * R * np.arcsin(np.sqrt(a))


def find_n_closest_points(df, lon0, lat0, n=4):
    """Return the n rows in df closest to (lon0, lat0) based on
   lons_inter/lats_inter."""
    distances = haversine_distance(lon0, lat0, df["lons_inter"].values,
                                   df["lats_inter"].values)
    df = df.copy()
    df["distance_km"] = distances
    return df.nsmallest(n, "distance_km")


omni_id = "BD09"
for omni_id in omni_d.keys():
    lon_omni, lat_omni = omni_d[omni_id]
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    df_4 = find_n_closest_points(df, lon_omni, lat_omni, n=4)
    print(omni_id)
    print(df_4)

sys.exit()
track_self = df_4["track_self"].apply(lambda x: str(x)).values
track_other = df_4["track_other"].apply(lambda x: str(x)).values
lons_inter = df_4["lons_inter"].values
lats_inter = df_4["lats_inter"].values
x_from_coast_self = df_4["x_from_coast_self"].values
x_from_coast_other = df_4["x_from_coast_other"].values
angle_acute = df_4["angle_acute"].values
angle_obtuse = df_4["angle_obtuse"].values
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

for i in range(len(df_4)):
    track_number_self = track_self[i]
    track_number_other = track_other[i]
    f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
    lon1 = lons_inter[i]
    lat1 = lats_inter[i]
    x_from_coast_self1 = x_from_coast_self[i]
    x_from_coast_other1 = x_from_coast_other[i]
    angle_acute1 = angle_acute[i]
    angle_obtuse1 = angle_obtuse[i]
    ds_self = xr.open_dataset(f_self, decode_times=False)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other - lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    # if track_path_self.intersects(track_path_other):
    sla_self_at = sla_self.sel(x=x_from_coast_self1,
                               method="nearest",
                               drop=True)
    sla_other_at = sla_other.sel(x=x_from_coast_other1,
                                 method="nearest",
                                 drop=True)
    sla_other_at = sla_other_at.interp(time=sla_self_at.time)
    # other_times.append((track_number_other, abs(dist_time)))
    fig1, ax1 = plt.subplots(1,
                             1,
                             figsize=(width, height),
                             layout="constrained")
    sla_self_at.plot(ax=ax1, label=f"pass {track_number_self}")
    sla_other_at.plot(ax=ax1, label=f"pass {track_number_other}")
    # text coordinate of intersection point
    plt.text(
        0.15,
        0.95,
        f"intersection point ({lon1:.2f}, {lat1:.2f})",
        transform=ax1.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    plt.legend()
    fig2, ax2 = plt.subplots(
        1,
```

```
        1,
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(width, height),
        layout="constrained",
    )
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG)
    ax2.grid()
    ax2.plot(
        lon_omni,
        lat_omni,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor='red',
        markeredgecolor='black',
        markeredgewidth=2
    )
    # plot plat form code of omni at lon_omni, lat_omni
    plt.text(lon_omni, lat_omni, f"{omni_id}", fontsize=10, color="k")
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="w",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="w",
        )
    plt.show()
    plt.close("all")
    break

dse.close()

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import sys

cmap1 = cmo.cm.diff
sat = "S3A"
sat = "ERS1+ERS2+ENV+SRL"
sat = "TP+J1+J2+J3+S6A"
d = 1.5
height, width = 9, 9


def haversine_distance(lon1, lat1, lon2, lat2):
    """Calculate great-circle distance between (lon1, lat1)
   and arrays (lon2, lat2) in kilometers."""
    R = 6371  # Radius of Earth in km
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlon = lon2 - lon1
    dlat = lat2 - lat1
    a = (np.sin(dlat / 2)**2 +
         np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2)**2)
    return 2 * R * np.arcsin(np.sqrt(a))


def find_n_closest_points(df, lon0, lat0, n=4):
    """Return the n rows in df closest to (lon0, lat0) based on
   lons_inter/lats_inter."""
    distances = haversine_distance(lon0, lat0, df["lons_inter"].values,
                                   df["lats_inter"].values)
    df = df.copy()
    df["distance_km"] = distances
    return df.nsmallest(n, "distance_km")


omni_id = "BD09"
lon_omni, lat_omni = omni_d[omni_id]
df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
df_4 = find_n_closest_points(df, lon_omni, lat_omni, n=4)
print(omni_id)
print(df_4)

# sys.exit()
track_self = df_4["track_self"].apply(lambda x: str(x)).values
track_other = df_4["track_other"].apply(lambda x: str(x)).values
lons_inter = df_4["lons_inter"].values
lats_inter = df_4["lats_inter"].values
x_from_coast_self = df_4["x_from_coast_self"].values
x_from_coast_other = df_4["x_from_coast_other"].values
angle_acute = df_4["angle_acute"].values
angle_obtuse = df_4["angle_obtuse"].values
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

for i in range(len(df_4)):
    track_number_self = track_self[i]
    track_number_other = track_other[i]
    f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
    lon1 = lons_inter[i]
    lat1 = lats_inter[i]
    x_from_coast_self1 = x_from_coast_self[i]
    x_from_coast_other1 = x_from_coast_other[i]
    angle_acute1 = angle_acute[i]
    angle_obtuse1 = angle_obtuse[i]
    ds_self = xr.open_dataset(f_self, decode_times=False)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other - lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    # if track_path_self.intersects(track_path_other):
    sla_self_at = sla_self.sel(x=x_from_coast_self1,
                               method="nearest",
                               drop=True)
    sla_other_at = sla_other.sel(x=x_from_coast_other1,
                                 method="nearest",
                                 drop=True)
    sla_other_at = sla_other_at.interp(time=sla_self_at.time)
    # other_times.append((track_number_other, abs(dist_time)))
    fig1, ax1 = plt.subplots(1,
                             1,
                             figsize=(width, height),
                             layout="constrained")
    sla_self_at.plot(ax=ax1, label=f"pass {track_number_self}")
    sla_other_at.plot(ax=ax1, label=f"pass {track_number_other}")
    # text coordinate of intersection point
    plt.text(
        0.15,
        0.95,
        f"intersection point ({lon1:.2f}, {lat1:.2f})",
        transform=ax1.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    plt.legend()
    fig2, ax2 = plt.subplots(
        1,
        1,
```

```
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(width, height),
        layout="constrained",
    )
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG)
    ax2.grid()
    ax2.plot(
        lon_omni,
        lat_omni,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor='red',
        markeredgecolor='black',
        markeredgewidth=2
    )
    # plot plat form code of omni at lon_omni, lat_omni
    plt.text(lon_omni, lat_omni, f"{omni_id}", fontsize=10, color="k")
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="w",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="w",
        )
    plt.show()
    plt.close("all")
    break

dse.close()

```

```python
import math
import os
import warnings

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs


def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float or array-like
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components:
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    u = u * -1
    return u, v


cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
oscar_extracted_dir = f"/home/srinivasu/xtrackm/guv/oscar_at_intersections/"
ekman_extracted_dir = "/home/srinivasu/xtrackm/ekman/ekman_at_intersections/"

sat = "TP+J1+J2+J3+S6A"
df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
df_out = df.copy()

corrs_u = []
biass_u = []
rmses_u = []

corrs_v = []
biass_v = []
rmses_v = []

ln = len(df)
for i in range(ln):
    sat1 = df["sat"].values[i]
    track_number_self = df["track_self"].apply(lambda x: str(x)).values[i]
    track_number_other = df["track_other"].apply(lambda x: str(x)).values[i]
    lons_inter1 = df["lons_inter"].values[i]
    lats_inter1 = df["lats_inter"].values[i]
    lon1 = lons_inter1
    lat1 = lats_inter1
    x_from_coast_self1 = df["x_from_coast_self"].values[i]
    x_from_coast_other1 = df["x_from_coast_other"].values[i]
    angle_acute = df["angle_acute"].values[i]
    angle_obtuse = df["angle_obtuse"].values[i]
    if abs(lat1) < 2:
        corrs_u.append(np.nan)
        rmses_u.append(np.nan)
        biass_u.append(np.nan)
        corrs_v.append(np.nan)
        rmses_v.append(np.nan)
        biass_v.append(np.nan)
        continue
    print(
        sat,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    f_oscar = f"{oscar_extracted_dir}/oscar_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc"
    ds_oscar = xr.open_dataset(f_oscar)
    oscar_u = ds_oscar.u
    oscar_v = ds_oscar.v
    f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_self}_loess_0.2.nc"
    f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_other}_loess_0.2.nc"
    ds_self = xr.open_dataset(f_self, engine="h5netcdf", decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
```

```
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)
    gu, gv = geostrophic_components_from_a(gc_self_at, gc_other_at, a1, a2)

    # -------------------- NEW: add Ekman u,v to gu, gv --------------------
    # Ekman files are expected at:
    # /home/srinivasu/xtrackm/ekman/ekman_at_intersections/
    # named: ekman_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc
    ek_fname = os.path.join(
        ekman_extracted_dir,
        f"ekman_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc",
    )
    if os.path.exists(ek_fname):
        try:
            ds_ek = xr.open_dataset(ek_fname)
            ek_u = ds_ek.u
            ek_v = ds_ek.v

            # align Ekman times with gu/gv times similarly to OSCAR handling:
            tfreq = sats_tfreq[sat] if sat in sats_tfreq else None
            if tfreq is not None:
                # resample to same temporal frequency then interp to gu.time
                try:
                    ek_u = ek_u.resample(time=f"{tfreq}D").mean()
                    ek_v = ek_v.resample(time=f"{tfreq}D").mean()
                except Exception:
                    # if resample fails (e.g. non-datetime), skip resampling
                    pass
            # interpolate to gu/gv time (gu has same time as gv)
            try:
                ek_u = ek_u.interp(time=gu.time)
                ek_v = ek_v.interp(time=gv.time)
            except Exception:
                # fallback: try aligning by nearest join
                ek_u = ek_u.reindex_like(gu, method="nearest", tolerance=None)
                ek_v = ek_v.reindex_like(gv, method="nearest", tolerance=None)

            # add to geostrophic components
            gu = gu + ek_u
            gv = gv + ek_v

            ds_ek.close()
        except Exception as e:
            warnings.warn(f"Error reading/processing Ekman file {ek_fname}: {e}")
    else:
        warnings.warn(f"No Ekman file found at {ek_fname}, proceeding without Ekman add.")
    # -------------------- END NEW CODE --------------------

    fig = plt.figure(figsize=(12, 10))
    ax0 = fig.add_subplot(3, 1, 3)
    ax1 = fig.add_subplot(3, 1, 2)

    gu_cut = gu
    gv_cut = gv
    tfreq = sats_tfreq[sat]

    oscar_u = oscar_u.resample(time=f"{tfreq}D").mean()
    oscar_u = oscar_u.interp(time=gu_cut.time)
    oscar_max = oscar_u.max().item()
    gu_cut = gu_cut.where(gu_cut < oscar_max, np.nan)
    gu_cut = gu_cut.where(gu_cut > -oscar_max, np.nan)

    oscar_v = oscar_v.resample(time=f"{tfreq}D").mean()
    oscar_v = oscar_v.interp(time=gv_cut.time)
    oscar_max = oscar_v.max().item()
    gv_cut = gv_cut.where(gv_cut < oscar_max, np.nan)
    gv_cut = gv_cut.where(gv_cut > -oscar_max, np.nan)

    oscar_u.plot(ax=ax0, color="b", linewidth=2, label="oscar u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="r", linewidth=2)
    ax0.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax0.legend()
    ax0.set_ylabel("oscar u m/s")

    oscar_v.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="r", linewidth=2)

    corr_u = xs.pearson_r(gu_cut, oscar_u, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, oscar_u, dim="time", skipna=True)
    bias_var_u = oscar_u - gu_cut
    bias_u = bias_var_u.mean(dim="time")

```

```
    corr_v = xs.pearson_r(gv_cut, oscar_v, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, oscar_v, dim="time", skipna=True)
    bias_var_v = oscar_v - gv_cut
    bias_v = bias_var_v.mean(dim="time")

    corrs_u.append(corr_u.item())
    rmses_u.append(rmse_u.item())
    biass_u.append(bias_u.item())

    corrs_v.append(corr_v.item())
    rmses_v.append(rmse_v.item())
    biass_v.append(bias_v.item())

    print(f"Correlation between gu and u_oscar_at: {corr_u.item()}")
    print(f"Correlation between gv and v_oscar_at: {corr_v.item()}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1}"
    ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.15,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("oscar v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor="red",
        markeredgecolor="black",
        markeredgewidth=2,
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"track_guvVSoscar_at_intersection_points/guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    # plt.show()
    plt.close("all")

# add corrs, rmses, biass to dataframe df_out
df_out["corrs"] = corrs_u
df_out["rmses"] = rmses_u
df_out["biass"] = biass_u

df_out.to_csv(f"oscar_ekmnVStrack_corrs_rmses_biass_{sat1}_u.csv", index=False)

df_out["corrs"] = corrs_v
df_out["rmses"] = rmses_v
df_out["biass"] = biass_v

df_out.to_csv(f"oscar_ekmnVStrack_corrs_rmses_biass_{sat1}_v.csv", index=False)
dse.close()

```

```python
import math
import os
import warnings

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs


cmap1 = cmo.cm.diff
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
oscar_extracted_dir = f"/home/srinivasu/xtrackm/guv/oscar_at_intersections/"

sat = "TP+J1+J2+J3+S6A"
df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
df_out = df.copy()

corrs_u = []
biass_u = []
rmses_u = []

corrs_v = []
biass_v = []
rmses_v = []

frac1 = "0.05"
ln = len(df)
for i in range(ln):
    sat1 = df["sat"].values[i]
    track_number_self = df["track_self"].apply(lambda x: str(x)).values[i]
    track_number_other = df["track_other"].apply(lambda x: str(x)).values[i]
    lons_inter1 = df["lons_inter"].values[i]
    lats_inter1 = df["lats_inter"].values[i]
    lon1 = lons_inter1
    lat1 = lats_inter1
    x_from_coast_self1 = df["x_from_coast_self"].values[i]
    x_from_coast_other1 = df["x_from_coast_other"].values[i]
    angle_acute = df["angle_acute"].values[i]
    angle_obtuse = df["angle_obtuse"].values[i]
    if abs(lat1) < 2:
        corrs_u.append(np.nan)
        rmses_u.append(np.nan)
        biass_u.append(np.nan)
        corrs_v.append(np.nan)
        rmses_v.append(np.nan)
        biass_v.append(np.nan)
        continue
    print(
        sat,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    f_oscar = f"{oscar_extracted_dir}/oscar_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc"
    ds_oscar = xr.open_dataset(f_oscar)
    oscar_u = ds_oscar.u
    oscar_v = ds_oscar.v
    f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_self}_loess_{frac1}.nc"
    f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_other}_loess_{frac1}.nc"
    ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)
    gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)

    gv = -1 * gv

    gu_filt = gu.where((gu <= 3.5) & (gu >= -3.5))
    gu_filtered = mad_filter_1d_xr(gu_filt, 3.5)
    gv_filt = gv.where((gv <= 3.5) & (gv >= -3.5))
```

```
    gv_filtered = mad_filter_1d_xr(gv_filt, 3.5)

    fig = plt.figure(figsize=(12, 10))
    ax0 = fig.add_subplot(3, 1, 3)
    ax1 = fig.add_subplot(3, 1, 2)

    gu_cut = gu_filtered
    gv_cut = gv_filtered
    tfreq = sats_tfreq[sat]

    oscar_u = oscar_u.resample(time=f"{tfreq}D").mean()
    oscar_u = oscar_u.interp(time=gu_cut.time)
    oscar_max = oscar_u.max().item()
    gu_cut = gu_cut.where(gu_cut < oscar_max, np.nan)
    gu_cut = gu_cut.where(gu_cut > -oscar_max, np.nan)

    oscar_v = oscar_v.resample(time=f"{tfreq}D").mean()
    oscar_v = oscar_v.interp(time=gv_cut.time)
    oscar_max = oscar_v.max().item()
    gv_cut = gv_cut.where(gv_cut < oscar_max, np.nan)
    gv_cut = gv_cut.where(gv_cut > -oscar_max, np.nan)

    oscar_u.plot(ax=ax0, color="b", linewidth=2, label="oscar u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="r", linewidth=2)
    ax0.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax0.legend()
    ax0.set_ylabel("oscar u m/s")

    oscar_v.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="r", linewidth=2)

    corr_u = xs.pearson_r(gu_cut, oscar_u, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, oscar_u, dim="time", skipna=True)
    bias_var_u = oscar_u - gu_cut
    bias_u = bias_var_u.mean(dim="time")

    corr_v = xs.pearson_r(gv_cut, oscar_v, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, oscar_v, dim="time", skipna=True)
    bias_var_v = oscar_v - gv_cut
    bias_v = bias_var_v.mean(dim="time")

    corrs_u.append(corr_u.item())
    rmses_u.append(rmse_u.item())
    biass_u.append(bias_u.item())

    corrs_v.append(corr_v.item())
    rmses_v.append(rmse_v.item())
    biass_v.append(bias_v.item())

    print(f"Correlation between gu and u_oscar_at: {corr_u.item()}")
    print(f"Correlation between gv and v_oscar_at: {corr_v.item()}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1}"
    ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.15,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("oscar v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor="red",
        markeredgecolor="black",
        markeredgewidth=2,
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
```

```
    plt.savefig(
        f"track_guvVSoscar_at_intersection_points/guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    plt.show()
    plt.close("all")

# add corrs, rmses, biass to dataframe df_out
df_out["corrs"] = corrs_u
df_out["rmses"] = rmses_u
df_out["biass"] = biass_u

df_out.to_csv(f"oscar_onlyVStrack_corrs_rmses_biass_{sat1}_u.csv", index=False)

df_out["corrs"] = corrs_v
df_out["rmses"] = rmses_v
df_out["biass"] = biass_v

df_out.to_csv(f"oscar_onlyVStrack_corrs_rmses_biass_{sat1}_v.csv", index=False)
dse.close()

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
from tools_xtrackm import *


def geostrophic_components_from_slopes(gc1, gc2, slope1, slope2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Convert slopes to azimuths (angles from east axis)
    a1 = math.atan(slope1)  # azimuth of track 1 in radians
    a2 = math.atan(slope2)  # azimuth of track 2 in radians
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Check for parallel tracks (same slope)
    if abs(slope1 - slope2) < 1e-10:
        raise ValueError(
            f"Tracks are parallel (slopes: {slope1:.6f}, {slope2:.6f}). "
            "Cannot resolve both velocity components.")
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    return u, v


def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    return u, v


cmap1 = cmo.cm.diff
sat = "TP+J1+J2+J3+S6A"
track_number_self, track_number_other = "53", "14"
d = 1.5
height, width = 9, 9
# oscar_dir = "/home/srinivasu/allData/oscar_0.25/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

# ds_oscar_u = xr.open_dataset(f"{oscar_dir}/oscar_0.25_u_full_xr_concat.nc")
# print(ds_oscar_u)
```

```python
# u_oscar = ds_oscar_u.u

# ds_oscar_v = xr.open_dataset(f"{oscar_dir}/oscar_0.25_v_full_xr_concat.nc")
# print(ds_oscar_v)
# v_oscar = ds_oscar_v.v

f_self = (
    f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
)
ds_self = xr.open_dataset(f_self, decode_times=False)
lons_track_self = ds_self.lon.values
lats_track_self = ds_self.lat.values
lon_equat_self = lons_track_self[0]  # this on coast
lat_equat_self = lats_track_self[0]  # this on coast
lon_coast_self = lons_track_self[-1]  # this on coast
lat_coast_self = lats_track_self[-1]  # this on coast

slope_self = (lats_track_self[-1] -
              lats_track_self[0]) / (lons_track_self[-1] - lons_track_self[0])
angle_self = np.rad2deg(np.arctan(slope_self))

f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
lons_track_other = ds_other.lon.values
lats_track_other = ds_other.lat.values
lon_equat_other = lons_track_other[0]
lat_equat_other = lats_track_other[0]
lon_coast_other = lons_track_other[-1]
lat_coast_other = lats_track_other[-1]

slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                     lon_equat_other)
angle_other = np.rad2deg(np.arctan(slope_other))

track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))

point = track_path_self.intersection(track_path_other)
lon1 = point.x
lat1 = point.y

sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")

x_from_coast_self1 = distance.distance((lat_coast_self, lon_coast_self),
                                       (point.y, point.x)).m
x_from_coast_other1 = distance.distance((lat_coast_other, lon_coast_other),
                                        (point.y, point.x)).m
print(
    sat,
    track_number_self,
    track_number_other,
    point.x,
    point.y,
    x_from_coast_self1,
    x_from_coast_other1,
)

f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_self}_loess_0.2.nc"
f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_other}_loess_0.2.nc"

ds_self_smooth = xr.open_dataset(f_self_smooth)
ds_other_smooth = xr.open_dataset(f_other_smooth)
sla_self_smooth = ds_self_smooth.sla_smooth
sla_other_smooth = ds_other_smooth.sla_smooth

track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))

# u_oscar_at = u_oscar.sel(lon=lon1, lat=lat1, method="nearest")
# v_oscar_at = v_oscar.sel(lon=lon1, lat=lat1, method="nearest")
# ds_out = xr.Dataset({
#         "u_oscar_at": u_oscar_at,
#         "v_oscar_at": v_oscar_at,
#         })
#
# ds_out.to_netcdf(f"oscar_uv_at_{sat}_{track_number_self}_{track_number_other}.nc")

ds_oscar = xr.open_dataset(f"oscar_uv_at_{sat}_{track_number_self}_{track_number_other}.nc")
u_oscar_at = ds_oscar.u_oscar_at
v_oscar_at = ds_oscar.v_oscar_at

gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)

gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
gc_other_at = gc_other.sel(x=x_from_coast_other1, method="nearest", drop=True)
gc_other_at = gc_other_at.interp(time=gc_self_at.time)

# Better approach:
a2 = math.atan2(lat_coast_other - lat_equat_other,
                lon_coast_other - lon_equat_other)
a1 = math.atan2(lat_coast_self - lat_equat_self,
                lon_coast_self - lon_equat_self)

gu, gv = geostrophic_components_from_a(gc_self_at, gc_other_at, a1, a2)

fig1, ax1 = plt.subplots(1, 1, figsize=(width, height), layout="constrained")
gu.plot(ax=ax1, label=f"pass {track_number_other}", color="r", linewidth=2)
u_oscar_at.plot(ax=ax1, color="k", linewidth=2, label="oscar")

# gu.plot(ax=ax1, label=f"pass {track_number_other}", color="r", linewidth=2)
# u_oscar_at.plot(ax=ax1, color="k", linewidth=2, label="oscar")

# text coordinate of intersection point
plt.text(
    0.15,
    0.95,
    f"intersection point ({lon1:.2f}, {lat1:.2f})",
    transform=ax1.transAxes,
    fontsize=14,
    fontweight="bold",
)
plt.legend()
fig2, ax2 = plt.subplots(
    1,
    1,
```

```
    subplot_kw={"projection": ccrs.PlateCarree()},
    figsize=(width, height),
    layout="constrained",
)
dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
    *TRACKS_REG[2:])).plot(ax=ax2,
                           add_colorbar=False,
                           add_labels=False,
                           cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG)
ax2.grid()
ax2.scatter(
    lons_track_self,
    lats_track_self,
    marker=".",
    color="c",
    s=4,
)
ax2.scatter(
    lons_track_other,
    lats_track_other,
    marker=".",
    color="c",
    s=4,
)
lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                   lon_coast_self, lat_coast_self, d)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=angle_self,
        color="w",
    )
lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                   lon_coast_other, lat_coast_other, d)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=angle_other,
        color="w",
    )
plt.show()
plt.close("all")

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
from tools_xtrackm import *


def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    return u, v


sat = "TP+J1+J2+J3+S6A"
track_number_self, track_number_other = "53", "14"
ds_oscar = xr.open_dataset(f"oscar_uv_at_{sat}_{track_number_self}_{track_number_other}.nc")
u_oscar_at = ds_oscar.u_oscar_at
v_oscar_at = ds_oscar.v_oscar_at


f_self = (
    f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
)
ds_self = xr.open_dataset(f_self, decode_times=False)
lons_track_self = ds_self.lon.values
lats_track_self = ds_self.lat.values
lon_equat_self = lons_track_self[0]  # this on coast
lat_equat_self = lats_track_self[0]  # this on coast
lon_coast_self = lons_track_self[-1]  # this on coast
lat_coast_self = lats_track_self[-1]  # this on coast

slope_self = (lat_coast_self - lat_equat_self) / (lon_coast_self - lon_equat_self)

f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
lons_track_other = ds_other.lon.values
lats_track_other = ds_other.lat.values
lon_equat_other = lons_track_other[0]
lat_equat_other = lats_track_other[0]
lon_coast_other = lons_track_other[-1]
lat_coast_other = lats_track_other[-1]

slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                     lon_equat_other)
track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))

point = track_path_self.intersection(track_path_other)
lon1 = point.x
lat1 = point.y

sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")

x_from_coast_self1 = distance.distance((lat_coast_self, lon_coast_self),
                                       (point.y, point.x)).m
x_from_coast_other1 = distance.distance((lat_coast_other, lon_coast_other),
                                        (point.y, point.x)).m
print(
    sat,
    track_number_self,
    track_number_other,
    point.x,
    point.y,
    x_from_coast_self1,
    x_from_coast_other1,
)

f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_self}_loess_0.2.nc"
f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_other}_loess_0.2.nc"

ds_self_smooth = xr.open_dataset(f_self_smooth)
ds_other_smooth = xr.open_dataset(f_other_smooth)
sla_self_smooth = ds_self_smooth.sla_smooth
sla_other_smooth = ds_other_smooth.sla_smooth

```

```python
track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))

gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)

gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
gc_other_at = gc_other.sel(x=x_from_coast_other1, method="nearest", drop=True)
gc_other_at = gc_other_at.interp(time=gc_self_at.time)

# Better approach:
a1 = math.atan2(lat_coast_self - lat_equat_self,
                lon_coast_self - lon_equat_self)
a2 = math.atan2(lat_coast_other - lat_equat_other,
                lon_coast_other - lon_equat_other)

# gc_self_at = -1 * gc_self_at
# gc_other_at = -1 * gc_other_at
gu, gv = geostrophic_components_from_a(gc_self_at, gc_other_at, a1, a2)

# print correlation between gv and v_oscar_at

fig1, ax1 = plt.subplots(1, 1, figsize=(12, 8), layout="constrained")

gu_cut = -1 * gu.interp(time=u_oscar_at.time)
gu_cut.plot(ax=ax1, label=f"track zonal", color="r", linewidth=2)
u_oscar_at.plot(ax=ax1, color="b", linewidth=2, label="oscar u")
corr = xs.pearson_r(gu_cut, u_oscar_at, dim="time", skipna=True)
corr1 = corr.item()
print(f"Correlation between gu and u_oscar_at: {corr1}")

# gv_cut = gv.interp(time=v_oscar_at.time)
# gv_cut.plot(ax=ax1, label=f"track meridional", color="r", linewidth=2)
# v_oscar_at.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
# corr = xs.pearson_r(gv_cut, v_oscar_at, dim="time", skipna=True)
# corr1 = corr.item()
# print(f"Correlation between gv and v_oscar_at: {corr1}")

# text coordinate of intersection point
plt.text(
    0.15,
    0.95,
    f"intersection point ({lon1:.2f}, {lat1:.2f})",
    transform=ax1.transAxes,
    fontsize=14,
    fontweight="bold",
)
# set horizontal time limits in datetime objects
plt.xlim(datetime(2000, 1, 1), datetime(2010, 1, 1))
# plt.ylim(-1, 1)
plt.ylim(-0.8, 0.8)
plt.legend()
plt.show()
plt.close("all")

```

guv/p.plot_std_oscar_track_u_coastal.py

```python
import argparse

import matplotlib.pyplot as plt
import pandas as pd


default="std_oscar_track_u_coastal.csv"
df = pd.read_csv(default)
x = df.index
fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(x, df["std_oscar"], label="std_oscar", color="tab:blue", linewidth=1.8)
ax.plot(x, df["std_track"], label="std_track", color="tab:red", linewidth=1.8)
ax.set_xlabel("Index")
ax.set_ylabel("Std (m/s)")
ax.set_title("std_oscar and std_track vs index")
ax.grid(True, alpha=0.3)
ax.legend()
fig.tight_layout()
plt.savefig("std_oscar_track_u_coastal.png")
plt.show()
# fig.savefig(args.out, dpi=180)

```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
import xskillscore as xs
from geopy import distance
from tools_xtrackm import *


@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(
        f_track, engine="netcdf4", decode_times=False
    ) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


def build_unique_str(sat, row):
    track_self = track_to_str(row["track_self"])
    track_other = track_to_str(row["track_other"])
    return f"{sat}_{track_self}_{track_other}"


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)


cmap1 = cmo.cm.diff
d = 1.5
height, width = 9, 9
frac1 = "0.05"


dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rama_data_dir = "/home/srinivasu/allData/rama/cur/"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
out_match_csv = "rama_guv_c_nearest_global_matches.ek.csv"
EKMAN_DIR = "/home/srinivasu/xtrackm/ekman/ekman_at_intersections"
matched_rows_all = []

df_all_list = []
for sat in sats_new:
    df_all_list.append(pd.read_csv(f"tracks_intersections_{sat}_1.csv"))
df_all = pd.concat(df_all_list, ignore_index=True)

for rama_id, (lon_rama, lat_rama) in rama_d.items():
    if abs(lat_rama) < 2:
        continue
    rama_file = f"{rama_data_dir}/cur{rama_id}_dy.cdf"
    if not os.path.exists(rama_file):
        continue
    print(rama_id, lon_rama, lat_rama)
    with xr.open_dataset(f"{rama_data_dir}/cur{rama_id}_dy.cdf") as ds_rama:
        # print(ds_rama)
        rama_u = 0.01 * ds_rama.U_320.isel(depth=0, drop=True)
        rama_v = 0.01 * ds_rama.V_321.isel(depth=0, drop=True)

    # print(rama_v.count().item())
    df_4 = find_n_closest_points(df_all, lon_rama, lat_rama, n=4)
    for nth in range(4):
        row = df_4.iloc[nth]

        sat = row["sat"]
        track_number_self = str(row["track_self"])
        track_number_other = str(row["track_other"])
        lon1 = row["lons_inter"]
        lat1 = row["lats_inter"]
        distance_km = row["distance_km"]
        angle_acute = row["angle_acute"]
        angle_obtuse = row["angle_obtuse"]

        unique_str = build_unique_str(sat, row)
        f_ek = f"{EKMAN_DIR}/{unique_str}_ekman_at_intersections.nc"
        ds_ek = xr.open_dataset(f_ek)
        u_ek = ds_ek['u_at']
        v_ek = ds_ek['v_at']

        if abs(lat1) < 2:
            continue

        print(
            sat,
            rama_id,
            track_number_self,
            track_number_other,
            lon1,
            lat1,
            angle_acute,
            angle_obtuse,
        )
        print((
        f"\nSELECT_SAT = '{sat}'"
        f"\nSELECT_RAMA_ID = '{rama_id}'"
        f"\nSELECT_TRACK_SELF = '{track_number_self}'"
        f"\nSELECT_TRACK_OTHER = '{track_number_other}'"
        ))

        f_guv = f"{guv_dir}/guv_at_intersection_{sat}_{track_number_self}_{track_number_other}_{frac1}.nc"
```

```
        if not os.path.exists(f_guv):
            print(f"missing file: {f_guv}")
            continue

        with xr.open_dataset(f_guv) as ds_guv:
            gu = ds_guv.gu
            gv = ds_guv.gv

        (
            lons_track_self,
            lats_track_self,
            lon_equat_self,
            lat_equat_self,
            lon_coast_self,
            lat_coast_self,
            angle_self,
        ) = get_track_geometry(sat, track_number_self)
        (
            lons_track_other,
            lats_track_other,
            lon_equat_other,
            lat_equat_other,
            lon_coast_other,
            lat_coast_other,
            angle_other,
        ) = get_track_geometry(sat, track_number_other)

        gu_cut = gu
        gv_cut = gv

        # Collocate RAMA at track-gu timestamps; avoid sat-dependent smoothing.
        rama_u_match = rama_u.reindex(
            time=gu_cut.time,
            method="nearest",
            tolerance=np.timedelta64(24, "h"),
        )
        ek_u_match = u_ek.reindex(
            time=gu_cut.time,
            method="nearest",
            tolerance=np.timedelta64(24, "h"),
        )
        valid = np.isfinite(gu_cut) & np.isfinite(rama_u_match)
        gu_pair = gu_cut.where(valid)
        rama_u_pair = rama_u_match.where(valid)
        ek_u_pair = ek_u_match.where(valid)
        gu_ek_pair = gu_pair + ek_u_pair

        df_pairs = xr.Dataset({"gu": gu_pair, "rama_u": rama_u_pair, "u_ek": ek_u_pair}).to_dataframe().reset_index()
        df_pairs = df_pairs.dropna(subset=["gu", "rama_u", "u_ek"])
        print(df_pairs.columns)
        matched_pairs = len(df_pairs)
        if matched_pairs > 0:
            """All these will broadcasted along length of gu, rama_u"""
            df_pairs["sat"] = sat
            df_pairs["rama_id"] = rama_id
            df_pairs["distance_km"] = distance_km
            df_pairs["track_self"] = track_number_self
            df_pairs["track_other"] = track_number_other
            df_pairs["matched_pairs"] = matched_pairs
            df_pairs["timestamp"] = pd.to_datetime(df_pairs["time"]).astype(str)
            print(df_pairs.columns)
            matched_rows_all.append(
                df_pairs[
                    [
                        "sat",
                        "rama_id",
                        "distance_km",
                        "track_self",
                        "track_other",
                        "matched_pairs",
                        "timestamp",
                        "gu",
                        "rama_u",
                        "u_ek",
                    ]
                ]
            )

        # check valid points
        ngp = np.sum(valid)
        if ngp < 2:
            continue
        fig = plt.figure(figsize=(12, 8))
        ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
        ax1 = fig.add_subplot(2, 1, 2)
        rama_u_pair.plot(ax=ax1, color="r", linewidth=2, label="rama")
        gu_pair.plot(ax=ax1, color="k", linewidth=2, label="track")
        gu_ek_pair.plot(ax=ax1, color="b", linewidth=2, label="track+ekman")

        corr = xs.pearson_r(gu_pair, rama_u_pair, dim="time", skipna=True)
        corr1 = corr.item()

        info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
        print(f"Correlation between gu and rama u: {corr1}")
        plt.text(
            0.25,
            0.90,
            info,
            transform=ax1.transAxes,
            fontsize=12,
            fontweight="bold",
        )
        plt.legend()

        rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
        decorate_axis(ax2, "", *TRACKS_REG, step=10)
        ax2.grid()
        ax2.plot(
            lon_rama,
            lat_rama,
            c="r",
            marker="o",
            markersize=4,
            markerfacecolor="red",
            markeredgecolor="red",
            markeredgewidth=2,
        )
```

```
235
236         ax2.plot(
237             lon1,
238             lat1,
239             c="k",
240             marker=".",
241             markersize=8,
242             markerfacecolor="black",
243             markeredgecolor="black",
244             markeredgewidth=2,
245         )
246         plt.text(lon_rama, lat_rama, f"{rama_id}", fontsize=10, color="k")
247         ax2.scatter(
248             lons_track_self, lats_track_self, marker=".", color="c", s=4
249         )
250         ax2.scatter(
251             lons_track_other, lats_track_other, marker=".", color="c", s=4
252         )
253
254         lonm, latm = get_point_at_distance(
255             lon_equat_self,
256             lat_equat_self,
257             lon_coast_self,
258             lat_coast_self,
259             d,
260         )
261         info = f"{sat} {track_number_self} {track_number_other} {distance_km:.2f} km"
262         plt.text(
263             0.55,
264             0.90,
265             info,
266             transform=ax2.transAxes,
267             fontsize=10,
268             fontweight="bold",
269         )
270         if is_within_region(lonm, latm, *TRACKS_REG):
271             plt.text(
272                 lonm,
273                 latm,
274                 s=track_number_self,
275                 fontsize=14,
276                 rotation=angle_self,
277                 color="w",
278             )
279
280         lonm, latm = get_point_at_distance(
281             lon_equat_other,
282             lat_equat_other,
283             lon_coast_other,
284             lat_coast_other,
285             d,
286         )
287         if is_within_region(lonm, latm, *TRACKS_REG):
288             plt.text(
289                 lonm,
290                 latm,
291                 s=track_number_other,
292                 fontsize=14,
293                 rotation=angle_other,
294                 color="w",
295             )
296
297         # plt.show()
298         plt.close("all")
299
300 if len(matched_rows_all) > 0:
301     df_matched_all = pd.concat(matched_rows_all, ignore_index=True)
302     df_matched_all.to_csv(out_match_csv, index=False)
303     print(f"saved global match csv: {out_match_csv}, rows={len(df_matched_all)}")
304 #
305 dse.close()
306
```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
import xskillscore as xs
from geopy import distance
from tools_xtrackm import *
# codex resume 019cc947-b006-7d82-8941-d68e330c388a

@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(
        f_track, engine="netcdf4", decode_times=False
    ) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


def build_unique_str(sat, row):
    track_self = track_to_str(row["track_self"])
    track_other = track_to_str(row["track_other"])
    return f"{sat}_{track_self}_{track_other}"


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)


cmap1 = cmo.cm.diff
d = 1.5
height, width = 9, 9
frac1 = "0.05"


dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rama_data_dir = "/home/srinivasu/allData/rama/cur/"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
out_match_csv = "rama_guv_c_nearest_global_matches.csv"
matched_rows_all = []
SELECT_SAT = "ERS1+ERS2+ENV+SRL"
SELECT_RAMA_ID = "15n90e"
SELECT_TRACK_SELF = "810"
SELECT_TRACK_OTHER = "423"
EKMAN_DIR = "/home/srinivasu/xtrackm/ekman/ekman_at_intersections"


df_all_list = []
for sat in sats_new:
    df_all_list.append(pd.read_csv(f"tracks_intersections_{sat}_1.csv"))
df_all = pd.concat(df_all_list, ignore_index=True)


lon_rama, lat_rama = rama_d[SELECT_RAMA_ID]
rama_file = f"{rama_data_dir}/cur{SELECT_RAMA_ID}_dy.cdf"
if not os.path.exists(rama_file):
    raise("No rama file")
print(SELECT_RAMA_ID, lon_rama, lat_rama)

with xr.open_dataset(f"{rama_data_dir}/cur{SELECT_RAMA_ID}_dy.cdf") as ds_rama:
    # print(ds_rama)
    rama_u = 0.01 * ds_rama.U_320.isel(depth=0, drop=True)
    rama_v = 0.01 * ds_rama.V_321.isel(depth=0, drop=True)

df_selected = df_all[
    (df_all["sat"] == SELECT_SAT)
    & (df_all["track_self"].astype(str) == SELECT_TRACK_SELF)
    & (df_all["track_other"].astype(str) == SELECT_TRACK_OTHER)
].copy()
if df_selected.empty:
    raise("selected sat/track combination not found in intersections file")
lons = df_selected["lons_inter"].to_numpy(dtype=float)
lats = df_selected["lats_inter"].to_numpy(dtype=float)
distances = [distance.distance((lat, lon), (lat_rama, lon_rama)).km for lon, lat in zip(lons, lats)]
df_selected["distance_km"] = distances
row = df_selected.nsmallest(1, "distance_km").iloc[0]

sat = row["sat"]
track_number_self = str(row["track_self"])
track_number_other = str(row["track_other"])
lon1 = row["lons_inter"]
lat1 = row["lats_inter"]
distance_km = row["distance_km"]
angle_acute = row["angle_acute"]
angle_obtuse = row["angle_obtuse"]
unique_str = build_unique_str(sat, row)
f_ek = f"{EKMAN_DIR}/{unique_str}_ekman_at_intersections.nc"
ds_ek = xr.open_dataset(f_ek)
u_ek = ds_ek['u_at']
v_ek = ds_ek['v_at']

print(
    sat,
    SELECT_RAMA_ID,
    track_number_self,
    track_number_other,
    lon1,
    lat1,
    angle_acute,
    angle_obtuse,
```

```
)

f_guv = f"{guv_dir}/guv_at_intersection_{sat}_{track_number_self}_{track_number_other}_{frac1}.nc"
if not os.path.exists(f_guv):
    print(f"missing file: {f_guv}")
    raise("missing file")

with xr.open_dataset(f_guv) as ds_guv:
    gu = ds_guv.gu
    gv = ds_guv.gv

(
    lons_track_self,
    lats_track_self,
    lon_equat_self,
    lat_equat_self,
    lon_coast_self,
    lat_coast_self,
    angle_self,
) = get_track_geometry(sat, track_number_self)
(
    lons_track_other,
    lats_track_other,
    lon_equat_other,
    lat_equat_other,
    lon_coast_other,
    lat_coast_other,
    angle_other,
) = get_track_geometry(sat, track_number_other)

gu_cut = gu
gv_cut = gv

# Collocate RAMA at track-gu timestamps; avoid sat-dependent smoothing.
rama_u_match = rama_u.reindex(
    time=gu_cut.time,
    method="nearest",
    tolerance=np.timedelta64(24, "h"),
)
ek_u_match = u_ek.reindex(
    time=gu_cut.time,
    method="nearest",
    tolerance=np.timedelta64(24, "h"),
)
valid = np.isfinite(gu_cut) & np.isfinite(rama_u_match)
gu_pair = gu_cut.where(valid)
rama_u_pair = rama_u_match.where(valid)
ek_u_pair = ek_u_match.where(valid)

gu_ek_pair = gu_pair + ek_u_pair

df_pairs = xr.Dataset({"gu": gu_pair, "rama_u": rama_u_pair}).to_dataframe().reset_index()
df_pairs = df_pairs.dropna(subset=["gu", "rama_u"])

# check valid points
fig = plt.figure(figsize=(12, 8))
ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax1 = fig.add_subplot(2, 1, 2)
rama_u_pair.plot(ax=ax1, color="r", linewidth=2, label="rama")
gu_pair.plot(ax=ax1, color="k", linewidth=2, label="track")
gu_ek_pair.plot(ax=ax1, color="b", linewidth=2, label="track+ekman")

corr = xs.pearson_r(gu_pair, rama_u_pair, dim="time", skipna=True)
corr1 = corr.item()

info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
print(f"Correlation between gu and rama u: {corr1}")
plt.text(
    0.25,
    0.90,
    info,
    transform=ax1.transAxes,
    fontsize=12,
    fontweight="bold",
)
plt.legend()

rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=10)
ax2.grid()
ax2.plot(
    lon_rama,
    lat_rama,
    c="r",
    marker="o",
    markersize=4,
    markerfacecolor="red",
    markeredgecolor="red",
    markeredgewidth=2,
)

ax2.plot(
    lon1,
    lat1,
    c="k",
    marker=".",
    markersize=8,
    markerfacecolor="black",
    markeredgecolor="black",
    markeredgewidth=2,
)
plt.text(lon_rama, lat_rama, f"{SELECT_RAMA_ID}", fontsize=10, color="k")
ax2.scatter(
    lons_track_self, lats_track_self, marker=".", color="c", s=4
)
ax2.scatter(
    lons_track_other, lats_track_other, marker=".", color="c", s=4
)

lonm, latm = get_point_at_distance(
    lon_equat_self,
    lat_equat_self,
    lon_coast_self,
    lat_coast_self,
    d,
)
info = f"{sat} {track_number_self} {track_number_other} {distance_km:.2f} km"
```

```
plt.text(
    0.55,
    0.90,
    info,
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=angle_self,
        color="w",
    )

lonm, latm = get_point_at_distance(
    lon_equat_other,
    lat_equat_other,
    lon_coast_other,
    lat_coast_other,
    d,
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=angle_other,
        color="w",
    )

plt.show()
plt.close("all")

dse.close()

```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
import xskillscore as xs
from geopy import distance
from tools_xtrackm import *


@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(
        f_track, engine="netcdf4", decode_times=False
    ) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


cmap1 = cmo.cm.diff
d = 1.5
height, width = 9, 9
frac1 = "0.05"


dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rama_data_dir = "/home/srinivasu/allData/rama/cur/"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
out_match_csv = "rama_guv_c_nearest_global_matches.csv"
matched_rows_all = []


df_all_list = []
for sat in sats_new:
    df_all_list.append(pd.read_csv(f"tracks_intersections_{sat}_1.csv"))
df_all = pd.concat(df_all_list, ignore_index=True)


for rama_id, (lon_rama, lat_rama) in rama_d.items():
    if abs(lat_rama) < 2:
        continue

    rama_file = f"{rama_data_dir}/cur{rama_id}_dy.cdf"
    if not os.path.exists(rama_file):
        continue
    print(rama_id, lon_rama, lat_rama)

    with xr.open_dataset(f"{rama_data_dir}/cur{rama_id}_dy.cdf") as ds_rama:
        # print(ds_rama)
        rama_u = 0.01 * ds_rama.U_320.isel(depth=0, drop=True)
        rama_v = 0.01 * ds_rama.V_321.isel(depth=0, drop=True)

    # print(rama_v.count().item())
    df_4 = find_n_closest_points(df_all, lon_rama, lat_rama, n=4)
    for nth in range(4):
        row = df_4.iloc[nth]

        sat = row["sat"]
        track_number_self = str(row["track_self"])
        track_number_other = str(row["track_other"])
        lon1 = row["lons_inter"]
        lat1 = row["lats_inter"]
        distance_km = row["distance_km"]
        angle_acute = row["angle_acute"]
        angle_obtuse = row["angle_obtuse"]

        if abs(lat1) < 2:
            continue

        print(
            sat,
            rama_id,
            track_number_self,
            track_number_other,
            lon1,
            lat1,
            angle_acute,
            angle_obtuse,
        )

        f_guv = f"{guv_dir}/guv_at_intersection_{sat}_{track_number_self}_{track_number_other}_{frac1}.nc"
        if not os.path.exists(f_guv):
            print(f"missing file: {f_guv}")
            continue

        with xr.open_dataset(f_guv) as ds_guv:
            gu = ds_guv.gu
            gv = ds_guv.gv

        (
            lons_track_self,
            lats_track_self,
            lon_equat_self,
            lat_equat_self,
            lon_coast_self,
            lat_coast_self,
            angle_self,
        ) = get_track_geometry(sat, track_number_self)
        (
            lons_track_other,
            lats_track_other,
            lon_equat_other,
            lat_equat_other,
```

```
            lon_coast_other,
            lat_coast_other,
            angle_other,
        ) = get_track_geometry(sat, track_number_other)

        gu_cut = gu
        gv_cut = gv

        # Collocate RAMA at track-gu timestamps; avoid sat-dependent smoothing.
        rama_u_match = rama_u.reindex(
            time=gu_cut.time,
            method="nearest",
            tolerance=np.timedelta64(24, "h"),
        )
        valid = np.isfinite(gu_cut) & np.isfinite(rama_u_match)
        gu_pair = gu_cut.where(valid)
        rama_u_pair = rama_u_match.where(valid)

        df_pairs = xr.Dataset({"gu": gu_pair, "rama_u": rama_u_pair}).to_dataframe().reset_index()
        df_pairs = df_pairs.dropna(subset=["gu", "rama_u"])
        print(df_pairs.columns)
        matched_pairs = len(df_pairs)
        if matched_pairs > 0:
            """All these will broadcasted along length of gu, rama_u"""
            df_pairs["sat"] = sat
            df_pairs["rama_id"] = rama_id
            df_pairs["distance_km"] = distance_km
            df_pairs["track_self"] = track_number_self
            df_pairs["track_other"] = track_number_other
            df_pairs["matched_pairs"] = matched_pairs
            df_pairs["timestamp"] = pd.to_datetime(df_pairs["time"]).astype(str)
            print(df_pairs.columns)
            matched_rows_all.append(
                df_pairs[
                    [
                        "sat",
                        "rama_id",
                        "distance_km",
                        "track_self",
                        "track_other",
                        "matched_pairs",
                        "timestamp",
                        "gu",
                        "rama_u",
                    ]
                ]
            )

        # check valid points
        ngp = np.sum(valid)
        if ngp < 2:
            continue
        fig = plt.figure(figsize=(12, 8))
        ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
        ax1 = fig.add_subplot(2, 1, 2)
        rama_u_pair.plot(ax=ax1, color="r", linewidth=2, label="rama")
        gu_pair.plot(ax=ax1, color="k", linewidth=2, label="track")

        corr = xs.pearson_r(gu_pair, rama_u_pair, dim="time", skipna=True)
        corr1 = corr.item()

        info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
        print(f"Correlation between gu and rama u: {corr1}")
        plt.text(
            0.25,
            0.90,
            info,
            transform=ax1.transAxes,
            fontsize=12,
            fontweight="bold",
        )
        plt.legend()

        rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
        decorate_axis(ax2, "", *TRACKS_REG, step=10)
        ax2.grid()
        ax2.plot(
            lon_rama,
            lat_rama,
            c="r",
            marker="o",
            markersize=4,
            markerfacecolor="red",
            markeredgecolor="red",
            markeredgewidth=2,
        )

        ax2.plot(
            lon1,
            lat1,
            c="k",
            marker=".",
            markersize=8,
            markerfacecolor="black",
            markeredgecolor="black",
            markeredgewidth=2,
        )
        plt.text(lon_rama, lat_rama, f"{rama_id}", fontsize=10, color="k")
        ax2.scatter(
            lons_track_self, lats_track_self, marker=".", color="c", s=4
        )
        ax2.scatter(
            lons_track_other, lats_track_other, marker=".", color="c", s=4
        )

        lonm, latm = get_point_at_distance(
            lon_equat_self,
            lat_equat_self,
            lon_coast_self,
            lat_coast_self,
            d,
        )
        info = f"{sat} {track_number_self} {track_number_other} {distance_km:.2f} km"
        plt.text(
            0.55,
            0.90,
            info,
```

```
            transform=ax2.transAxes,
            fontsize=10,
            fontweight="bold",
        )
        if is_within_region(lonm, latm, *TRACKS_REG):
            plt.text(
                lonm,
                latm,
                s=track_number_self,
                fontsize=14,
                rotation=angle_self,
                color="w",
            )

        lonm, latm = get_point_at_distance(
            lon_equat_other,
            lat_equat_other,
            lon_coast_other,
            lat_coast_other,
            d,
        )
        if is_within_region(lonm, latm, *TRACKS_REG):
            plt.text(
                lonm,
                latm,
                s=track_number_other,
                fontsize=14,
                rotation=angle_other,
                color="w",
            )

        plt.show()
        plt.close("all")
        break
    break

# if len(matched_rows_all) > 0:
#     df_matched_all = pd.concat(matched_rows_all, ignore_index=True)
#     df_matched_all.to_csv(out_match_csv, index=False)
#     print(f"saved global match csv: {out_match_csv}, rows={len(df_matched_all)}")
#
dse.close()

```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
import xskillscore as xs
from geopy import distance
from tools_xtrackm import *
# codex resume 019cc947-b006-7d82-8941-d68e330c388a

@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(
        f_track, engine="netcdf4", decode_times=False
    ) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


cmap1 = cmo.cm.diff
d = 1.5
height, width = 9, 9
frac1 = "0.05"


dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rama_data_dir = "/home/srinivasu/allData/rama/cur/"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
out_match_csv = "rama_guv_c_nearest_global_matches.csv"
matched_rows_all = []
SELECT_SAT = "ERS1+ERS2+ENV+SRL"
SELECT_RAMA_ID = "15n90e"
SELECT_TRACK_SELF = "810"
SELECT_TRACK_OTHER = "423"


df_all_list = []
for sat in sats_new:
    df_all_list.append(pd.read_csv(f"tracks_intersections_{sat}_1.csv"))
df_all = pd.concat(df_all_list, ignore_index=True)


lon_rama, lat_rama = rama_d[SELECT_RAMA_ID]
rama_file = f"{rama_data_dir}/cur{SELECT_RAMA_ID}_dy.cdf"
if not os.path.exists(rama_file):
    raise("No rama file")
print(SELECT_RAMA_ID, lon_rama, lat_rama)

with xr.open_dataset(f"{rama_data_dir}/cur{SELECT_RAMA_ID}_dy.cdf") as ds_rama:
    # print(ds_rama)
    rama_u = 0.01 * ds_rama.U_320.isel(depth=0, drop=True)
    rama_v = 0.01 * ds_rama.V_321.isel(depth=0, drop=True)

df_selected = df_all[
    (df_all["sat"] == SELECT_SAT)
    & (df_all["track_self"].astype(str) == SELECT_TRACK_SELF)
    & (df_all["track_other"].astype(str) == SELECT_TRACK_OTHER)
].copy()
if df_selected.empty:
    raise("selected sat/track combination not found in intersections file")
lons = df_selected["lons_inter"].to_numpy(dtype=float)
lats = df_selected["lats_inter"].to_numpy(dtype=float)
distances = [distance.distance((lat, lon), (lat_rama, lon_rama)).km for lon, lat in zip(lons, lats)]
df_selected["distance_km"] = distances
row = df_selected.nsmallest(1, "distance_km").iloc[0]

sat = row["sat"]
track_number_self = str(row["track_self"])
track_number_other = str(row["track_other"])
lon1 = row["lons_inter"]
lat1 = row["lats_inter"]
distance_km = row["distance_km"]
angle_acute = row["angle_acute"]
angle_obtuse = row["angle_obtuse"]

print(
    sat,
    SELECT_RAMA_ID,
    track_number_self,
    track_number_other,
    lon1,
    lat1,
    angle_acute,
    angle_obtuse,
)

f_guv = f"{guv_dir}/guv_at_intersection_{sat}_{track_number_self}_{track_number_other}_{frac1}.nc"
if not os.path.exists(f_guv):
    print(f"missing file: {f_guv}")
    raise("missing file")

with xr.open_dataset(f_guv) as ds_guv:
    gu = ds_guv.gu
    gv = ds_guv.gv

(
    lons_track_self,
    lats_track_self,
    lon_equat_self,
    lat_equat_self,
    lon_coast_self,
    lat_coast_self,
    angle_self,
```

```
) = get_track_geometry(sat, track_number_self)
(
    lons_track_other,
    lats_track_other,
    lon_equat_other,
    lat_equat_other,
    lon_coast_other,
    lat_coast_other,
    angle_other,
) = get_track_geometry(sat, track_number_other)

gu_cut = gu
gv_cut = gv

# Collocate RAMA at track-gu timestamps; avoid sat-dependent smoothing.
rama_u_match = rama_u.reindex(
    time=gu_cut.time,
    method="nearest",
    tolerance=np.timedelta64(24, "h"),
)
valid = np.isfinite(gu_cut) & np.isfinite(rama_u_match)
gu_pair = gu_cut.where(valid)
rama_u_pair = rama_u_match.where(valid)

df_pairs = xr.Dataset({"gu": gu_pair, "rama_u": rama_u_pair}).to_dataframe().reset_index()
df_pairs = df_pairs.dropna(subset=["gu", "rama_u"])

# check valid points
fig = plt.figure(figsize=(12, 8))
ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
ax1 = fig.add_subplot(2, 1, 2)
rama_u_pair.plot(ax=ax1, color="r", linewidth=2, label="rama")
gu_pair.plot(ax=ax1, color="k", linewidth=2, label="track")

corr = xs.pearson_r(gu_pair, rama_u_pair, dim="time", skipna=True)
corr1 = corr.item()

info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
print(f"Correlation between gu and rama u: {corr1}")
plt.text(
    0.25,
    0.90,
    info,
    transform=ax1.transAxes,
    fontsize=12,
    fontweight="bold",
)
plt.legend()

rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=10)
ax2.grid()
ax2.plot(
    lon_rama,
    lat_rama,
    c="r",
    marker="o",
    markersize=4,
    markerfacecolor="red",
    markeredgecolor="red",
    markeredgewidth=2,
)

ax2.plot(
    lon1,
    lat1,
    c="k",
    marker=".",
    markersize=8,
    markerfacecolor="black",
    markeredgecolor="black",
    markeredgewidth=2,
)
plt.text(lon_rama, lat_rama, f"{SELECT_RAMA_ID}", fontsize=10, color="k")
ax2.scatter(
    lons_track_self, lats_track_self, marker=".", color="c", s=4
)
ax2.scatter(
    lons_track_other, lats_track_other, marker=".", color="c", s=4
)

lonm, latm = get_point_at_distance(
    lon_equat_self,
    lat_equat_self,
    lon_coast_self,
    lat_coast_self,
    d,
)
info = f"{sat} {track_number_self} {track_number_other} {distance_km:.2f} km"
plt.text(
    0.55,
    0.90,
    info,
    transform=ax2.transAxes,
    fontsize=10,
    fontweight="bold",
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
        latm,
        s=track_number_self,
        fontsize=14,
        rotation=angle_self,
        color="w",
    )

lonm, latm = get_point_at_distance(
    lon_equat_other,
    lat_equat_other,
    lon_coast_other,
    lat_coast_other,
    d,
)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(
        lonm,
```

```
        latm,
        s=track_number_other,
        fontsize=14,
        rotation=angle_other,
        color="w",
    )

plt.show()
plt.close("all")

dse.close()

```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
import xskillscore as xs
from geopy import distance
from tools_xtrackm import *

# codex resume 019cc947-b006-7d82-8941-d68e330c388a


def find_n_closest_points(df, lon0, lat0, n=4):
    """Return n rows closest to (lon0, lat0) using intersection lon/lat."""
    lons_inter = df["lons_inter"].tolist()
    lats_inter = df["lats_inter"].tolist()
    distances = [
        distance.distance((y, x), (lat0, lon0)).km
        for x, y in zip(lons_inter, lats_inter)
    ]
    out = df.copy()
    out["distance_km"] = distances
    return out.nsmallest(n, "distance_km")


@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(
        f_track, engine="netcdf4", decode_times=False
    ) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


cmap1 = cmo.cm.diff
d = 1.5
height, width = 9, 9
frac1 = "0.05"
tol_hours = 24
max_distance_km = 250
min_overlap_fraction = 0.30
min_matched_pairs = 50
w_d = 0.35
w_t = 0.45
w_n = 0.20


dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rama_data_dir = "/home/srinivasu/allData/rama/cur/"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"


def collocate_rama_to_gu(gu, rama_u, tol_hours=24):
    rama_u_match = rama_u.reindex(
        time=gu.time,
        method="nearest",
        tolerance=np.timedelta64(tol_hours, "h"),
    )
    valid = np.isfinite(gu) & np.isfinite(rama_u_match)
    gu_pair = gu.where(valid)
    rama_u_pair = rama_u_match.where(valid)
    matched_pairs = int(valid.sum().item())
    possible_pairs = int(np.isfinite(gu).sum().item())
    overlap_fraction = (
        matched_pairs / possible_pairs if possible_pairs > 0 else 0.0
    )
    return gu_pair, rama_u_pair, matched_pairs, overlap_fraction


def norm_01(arr):
    arr = np.asarray(arr, dtype=float)
    if arr.size == 0:
        return arr
    amin = np.nanmin(arr)
    amax = np.nanmax(arr)
    if np.isclose(amax, amin):
        return np.zeros_like(arr, dtype=float)
    return (arr - amin) / (amax - amin)


df_all_list = []
for sat in sats_new:
    df_all_list.append(pd.read_csv(f"tracks_intersections_{sat}_1.csv"))
df_all = pd.concat(df_all_list, ignore_index=True)


for rama_id, (lon_rama, lat_rama) in rama_d.items():
    if abs(lat_rama) < 2:
        continue

    rama_file = f"{rama_data_dir}/cur{rama_id}_dy.cdf"
    if not os.path.exists(rama_file):
        continue
    print(rama_id, lon_rama, lat_rama)

    with xr.open_dataset(f"{rama_data_dir}/cur{rama_id}_dy.cdf") as ds_rama:
        # print(ds_rama)
        rama_u = 0.01 * ds_rama.U_320.isel(depth=0, drop=True)
        rama_v = 0.01 * ds_rama.V_321.isel(depth=0, drop=True)

    df_4 = find_n_closest_points(df_all, lon_rama, lat_rama, n=4)
    candidates = []
    for nth in range(min(4, len(df_4))):
```

```
        row = df_4.iloc[nth]
        sat = row["sat"]
        track_number_self = str(row["track_self"])
        track_number_other = str(row["track_other"])
        lon1 = row["lons_inter"]
        lat1 = row["lats_inter"]
        distance_km = row["distance_km"]
        if abs(lat1) < 2:
            continue

        f_guv = f"{guv_dir}/guv_at_intersection_{sat}_{track_number_self}_{track_number_other}_{frac1}.nc"
        if not os.path.exists(f_guv):
            continue

        with xr.open_dataset(f_guv) as ds_guv:
            if "gu" not in ds_guv:
                continue
            gu = ds_guv.gu.load()
            gv = ds_guv.gv.load() if "gv" in ds_guv else None

        gu_pair, rama_u_pair, matched_pairs, overlap_fraction = (
            collocate_rama_to_gu(gu, rama_u, tol_hours=tol_hours)
        )
        if matched_pairs < 2:
            continue

        candidates.append(
            {
                "row": row,
                "sat": sat,
                "track_number_self": track_number_self,
                "track_number_other": track_number_other,
                "lon1": lon1,
                "lat1": lat1,
                "distance_km": float(distance_km),
                "matched_pairs": matched_pairs,
                "overlap_fraction": float(overlap_fraction),
                "gu_pair": gu_pair,
                "rama_u_pair": rama_u_pair,
                "gv": gv,
            }
        )

    if len(candidates) == 0:
        print(f"{rama_id}: no valid candidate")
        continue

    score_df = pd.DataFrame(
        {
            "distance_km": [c["distance_km"] for c in candidates],
            "overlap_fraction": [c["overlap_fraction"] for c in candidates],
            "matched_pairs": [c["matched_pairs"] for c in candidates],
        }
    )
    hard_mask = (
        (score_df["distance_km"] <= max_distance_km)
        & (score_df["overlap_fraction"] >= min_overlap_fraction)
        & (score_df["matched_pairs"] >= min_matched_pairs)
    )
    if hard_mask.any():
        idx_pool = np.where(hard_mask.values)[0]
    else:
        idx_pool = np.arange(len(candidates))

    d_norm = norm_01(score_df.loc[idx_pool, "distance_km"].values)
    o_norm = norm_01(score_df.loc[idx_pool, "overlap_fraction"].values)
    n_norm = norm_01(score_df.loc[idx_pool, "matched_pairs"].values)
    scores = w_d * d_norm + w_t * (1.0 - o_norm) + w_n * (1.0 - n_norm)
    best_idx = idx_pool[int(np.argmin(scores))]
    best = candidates[best_idx]

    sat = best["sat"]
    track_number_self = best["track_number_self"]
    track_number_other = best["track_number_other"]
    lon1 = best["lon1"]
    lat1 = best["lat1"]
    distance_km = best["distance_km"]
    gu_pair = best["gu_pair"]
    rama_u_pair = best["rama_u_pair"]
    score_best = float(np.min(scores))

    print(
        "selected:",
        rama_id,
        sat,
        track_number_self,
        track_number_other,
        f"d={distance_km:.1f}km",
        f"overlap={best['overlap_fraction']:.2f}",
        f"n={best['matched_pairs']}",
        f"score={score_best:.3f}",
    )

    (
        lons_track_self,
        lats_track_self,
        lon_equat_self,
        lat_equat_self,
        lon_coast_self,
        lat_coast_self,
        angle_self,
    ) = get_track_geometry(sat, track_number_self)
    (
        lons_track_other,
        lats_track_other,
        lon_equat_other,
        lat_equat_other,
        lon_coast_other,
        lat_coast_other,
        angle_other,
    ) = get_track_geometry(sat, track_number_other)

    fig = plt.figure(figsize=(12, 8))
    ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
    ax1 = fig.add_subplot(2, 1, 2)
    rama_u_pair.plot(ax=ax1, color="r", linewidth=2, label="rama")
    gu_pair.plot(ax=ax1, color="k", linewidth=2, label="track")
```

```
235
236     corr = xs.pearson_r(gu_pair, rama_u_pair, dim="time", skipna=True)
237     corr1 = corr.item()
238     info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
239     print(f"Correlation between gu and rama u: {corr1}")
240     plt.text(
241         0.25,
242         0.90,
243         info,
244         transform=ax1.transAxes,
245         fontsize=12,
246         fontweight="bold",
247     )
248     plt.legend()
249
250     rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
251     decorate_axis(ax2, "", *TRACKS_REG, step=10)
252     ax2.grid()
253     ax2.plot(
254         lon_rama,
255         lat_rama,
256         c="r",
257         marker="o",
258         markersize=4,
259         markerfacecolor="red",
260         markeredgecolor="red",
261         markeredgewidth=2,
262     )
263     ax2.plot(
264         lon1,
265         lat1,
266         c="k",
267         marker=".",
268         markersize=8,
269         markerfacecolor="black",
270         markeredgecolor="black",
271         markeredgewidth=2,
272     )
273     plt.text(lon_rama, lat_rama, f"{rama_id}", fontsize=10, color="k")
274     ax2.scatter(lons_track_self, lats_track_self, marker=".", color="c", s=4)
275     ax2.scatter(lons_track_other, lats_track_other, marker=".", color="c", s=4)
276
277     lonm, latm = get_point_at_distance(
278         lon_equat_self,
279         lat_equat_self,
280         lon_coast_self,
281         lat_coast_self,
282         d,
283     )
284     info = (
285         f"{sat} {track_number_self} {track_number_other} {distance_km:.2f} km"
286     )
287     plt.text(
288         0.55,
289         0.90,
290         info,
291         transform=ax2.transAxes,
292         fontsize=10,
293         fontweight="bold",
294     )
295     if is_within_region(lonm, latm, *TRACKS_REG):
296         plt.text(
297             lonm,
298             latm,
299             s=track_number_self,
300             fontsize=14,
301             rotation=angle_self,
302             color="w",
303         )
304
305     lonm, latm = get_point_at_distance(
306         lon_equat_other,
307         lat_equat_other,
308         lon_coast_other,
309         lat_coast_other,
310         d,
311     )
312     if is_within_region(lonm, latm, *TRACKS_REG):
313         plt.text(
314             lonm,
315             latm,
316             s=track_number_other,
317             fontsize=14,
318             rotation=angle_other,
319             color="w",
320         )
321
322     plt.show()
323     plt.close("all")
324
325 dse.close()
326
```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# codex resume 019cd8ab-eee9-72e0-9952-b9e4a8bf1b11

RAMA_NO_EK_CSV = "rama_guv_c_nearest_global_matches.csv"
RAMA_EK_CSV = "rama_guv_c_nearest_global_matches.ek.csv"
OMNI_NO_EK_CSV = "omni_guv_c_nearest_global_matches.csv"
OMNI_EK_CSV = "omni_guv_c_nearest_global_matches.ek.csv"

OUT_PNG = "scatter_omniVSrama_uVSgu_ww_ek.png"
MIN_DISTANCE_KM = None
MAX_DISTANCE_KM = 250.0


def load_xy(csv_path, y_col, with_ekman=False):
    df = pd.read_csv(csv_path)

    dist = df["distance_km"].to_numpy(dtype=float)
    keep = np.isfinite(dist)
    if MIN_DISTANCE_KM is not None:
        keep &= dist >= float(MIN_DISTANCE_KM)
    if MAX_DISTANCE_KM is not None:
        keep &= dist <= float(MAX_DISTANCE_KM)
    df = df.loc[keep].copy()

    x = df["gu"].to_numpy(dtype=float)
    if with_ekman:
        x = x + df["u_ek"].to_numpy(dtype=float)

    y = df[y_col].to_numpy(dtype=float)

    valid = np.isfinite(x) & np.isfinite(y)
    x = x[valid]
    y = y[valid]

    return x, y, len(df)


def add_panel(ax, x, y, title, ylabel, xlabel, extent=None):
    if x.size < 3:
        ax.text(
            0.5,
            0.98,
            f"{title} (insufficient points)",
            transform=ax.transAxes,
            va="top",
            ha="center",
            fontsize=10,
            bbox=dict(boxstyle="round,pad=0.25", facecolor="black", alpha=0.5, edgecolor="white"),
            color="white",
        )
        ax.set_xlabel(xlabel)
        ax.set_ylabel(ylabel)
        ax.grid(alpha=0.25)
        return None

    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))

    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1, extent=extent)

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=10,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="black", alpha=0.55, edgecolor="white"),
        color="white",
    )

    ax.text(
        0.6,
        0.98,
        title,
        transform=ax.transAxes,
        va="top",
        ha="center",
        fontsize=10,
        bbox=dict(boxstyle="round,pad=0.25", facecolor="black", alpha=0.5, edgecolor="white"),
        color="white",
    )
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right", fontsize=9)
    ax.set_ylim(-1, 1)
    ax.set_xlim(-1, 1)
    return hb


def main():
    min_lbl = MIN_DISTANCE_KM if MIN_DISTANCE_KM is not None else "-inf"
    max_lbl = MAX_DISTANCE_KM if MAX_DISTANCE_KM is not None else "+inf"
    x_label_no_ek = "Altimetry gu (m/s)"
    x_label_ek = "Altimetry gu + u_ek (m/s)"
    none = ""

    x_rama_no, y_rama_no, n_rama_no = load_xy(RAMA_NO_EK_CSV, "rama_u", with_ekman=False)
    x_rama_ek, y_rama_ek, n_rama_ek = load_xy(RAMA_EK_CSV, "rama_u", with_ekman=True)
    x_omni_no, y_omni_no, n_omni_no = load_xy(OMNI_NO_EK_CSV, "omni_u", with_ekman=False)
```

```
    x_omni_ek, y_omni_ek, n_omni_ek = load_xy(OMNI_EK_CSV, "omni_u", with_ekman=True)
    all_x = np.concatenate([x_rama_no, x_rama_ek, x_omni_no, x_omni_ek])
    all_y = np.concatenate([y_rama_no, y_rama_ek, y_omni_no, y_omni_ek])
    global_extent = (
        float(np.nanmin(all_x)),
        float(np.nanmax(all_x)),
        float(np.nanmin(all_y)),
        float(np.nanmax(all_y)),
    )

    fig, axs = plt.subplots(2, 2, figsize=(16, 12), sharex=True, sharey=True, constrained_layout=True)

    hb00 = add_panel(
        axs[0, 0],
        x_rama_no,
        y_rama_no,
        f"RAMA u vs gu (no Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        "RAMA u (m/s)",
        none,
        extent=global_extent,
    )
    hb01 = add_panel(
        axs[0, 1],
        x_rama_ek,
        y_rama_ek,
        f"RAMA u vs gu+u_ek (with Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        none,
        none,
        extent=global_extent,
    )
    hb10 = add_panel(
        axs[1, 0],
        x_omni_no,
        y_omni_no,
        f"OMNI u vs gu (no Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        "OMNI u (m/s)",
        x_label_no_ek,
        extent=global_extent,
    )
    hb11 = add_panel(
        axs[1, 1],
        x_omni_ek,
        y_omni_ek,
        f"OMNI u vs gu+u_ek (with Ekman) | distance_km in [{min_lbl}, {max_lbl}]",
        none,
        x_label_ek,
        extent=global_extent,
    )

    top_hbs = [hb for hb in [hb00, hb01] if hb is not None]
    if top_hbs:
        top_vmax = max(float(np.nanmax(hb.get_array())) for hb in top_hbs)
        for hb in top_hbs:
            hb.set_clim(1.0, top_vmax)
        cbar_top = fig.colorbar(top_hbs[0], ax=axs[0, :], location="right", fraction=0.03, pad=0.02)
        cbar_top.set_label("Count")

    bottom_hbs = [hb for hb in [hb10, hb11] if hb is not None]
    if bottom_hbs:
        bottom_vmax = max(float(np.nanmax(hb.get_array())) for hb in bottom_hbs)
        for hb in bottom_hbs:
            hb.set_clim(1.0, bottom_vmax)
        cbar_bottom = fig.colorbar(bottom_hbs[0], ax=axs[1, :], location="right", fraction=0.03, pad=0.02)
        cbar_bottom.set_label("Count")

    fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)

    print(f"Rows after distance filter | RAMA no-ek: {n_rama_no}, RAMA ek: {n_rama_ek}")
    print(f"Rows after distance filter | OMNI no-ek: {n_omni_no}, OMNI ek: {n_omni_ek}")
    print(f"Saved {OUT_PNG}")


if __name__ == "__main__":
    main()

```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

CSV_PATH = "omni_guv_c_nearest_global_matches.csv"
OUT_PNG = "scatter_omni_u_vs_gu.png"
MIN_DISTANCE_KM = None
MAX_DISTANCE_KM = 250.0


df = pd.read_csv(CSV_PATH)

dist = df["distance_km"].to_numpy(dtype=float)
keep = np.isfinite(dist)
if MIN_DISTANCE_KM is not None:
    keep &= dist >= float(MIN_DISTANCE_KM)
if MAX_DISTANCE_KM is not None:
    keep &= dist <= float(MAX_DISTANCE_KM)
df = df.loc[keep].copy()

x = df["gu"].to_numpy(dtype=float)
y = df["omni_u"].to_numpy(dtype=float)
valid = np.isfinite(x) & np.isfinite(y)
x = x[valid]
y = y[valid]

if x.size < 3:
    print(
        f"Only {x.size} valid points after distance filter "
        f"(min={MIN_DISTANCE_KM}, max={MAX_DISTANCE_KM})."
    )
else:
    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))

    fig, ax = plt.subplots(figsize=(9, 7))
    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1)
    cbar = fig.colorbar(hb, ax=ax)
    cbar.set_label("Count")

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=11,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="black", alpha=0.55, edgecolor="white"),
        color="white",
    )

    min_lbl = MIN_DISTANCE_KM if MIN_DISTANCE_KM is not None else "-inf"
    max_lbl = MAX_DISTANCE_KM if MAX_DISTANCE_KM is not None else "+inf"
    ax.set_title(f"OMNI u vs Altimetry gu | distance_km in [{min_lbl}, {max_lbl}]")
    ax.set_xlabel("Altimetry gu (m/s)")
    ax.set_ylabel("OMNI u (m/s)")
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right")
    fig.tight_layout()
    fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)

    print(f"Rows after distance filter: {len(df)}")
    print(f"Saved {OUT_PNG}")

```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

CSV_PATH = "omni_guv_c_nearest_global_matches.ek.csv"
OUT_PNG = "scatter_omni_u_vs_gu.png"
MIN_DISTANCE_KM = None
MAX_DISTANCE_KM = 250.0


df = pd.read_csv(CSV_PATH)

dist = df["distance_km"].to_numpy(dtype=float)
keep = np.isfinite(dist)
if MIN_DISTANCE_KM is not None:
    keep &= dist >= float(MIN_DISTANCE_KM)
if MAX_DISTANCE_KM is not None:
    keep &= dist <= float(MAX_DISTANCE_KM)
df = df.loc[keep].copy()

y = df["omni_u"].to_numpy(dtype=float)
x1 = df["gu"].to_numpy(dtype=float)
x2 = df["u_ek"].to_numpy(dtype=float)

x = x1 + x2
# x = x1
valid = np.isfinite(x) & np.isfinite(y)
x = x[valid]
y = y[valid]

if x.size < 3:
    print(
        f"Only {x.size} valid points after distance filter "
        f"(min={MIN_DISTANCE_KM}, max={MAX_DISTANCE_KM})."
    )
else:
    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))

    fig, ax = plt.subplots(figsize=(9, 7))
    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1)
    cbar = fig.colorbar(hb, ax=ax)
    cbar.set_label("Count")

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=11,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="black", alpha=0.55, edgecolor="white"),
        color="white",
    )

    min_lbl = MIN_DISTANCE_KM if MIN_DISTANCE_KM is not None else "-inf"
    max_lbl = MAX_DISTANCE_KM if MAX_DISTANCE_KM is not None else "+inf"
    ax.set_title(f"OMNI u vs Altimetry gu | distance_km in [{min_lbl}, {max_lbl}]")
    ax.set_xlabel("Altimetry gu (m/s)")
    ax.set_ylabel("OMNI u (m/s)")
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right")
    fig.tight_layout()
    # fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)

    print(f"Rows after distance filter: {len(df)}")
    print(f"Saved {OUT_PNG}")

```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

CSV_PATH = "rama_guv_c_nearest_global_matches.csv"
OUT_PNG = "scatter_rama_u_vs_gu.png"
MIN_DISTANCE_KM = None
MAX_DISTANCE_KM = 250.0


df = pd.read_csv(CSV_PATH)

dist = df["distance_km"].to_numpy(dtype=float)
keep = np.isfinite(dist)
if MIN_DISTANCE_KM is not None:
    keep &= dist >= float(MIN_DISTANCE_KM)
if MAX_DISTANCE_KM is not None:
    keep &= dist <= float(MAX_DISTANCE_KM)
df = df.loc[keep].copy()

x = df["gu"].to_numpy(dtype=float)
y = df["rama_u"].to_numpy(dtype=float)
valid = np.isfinite(x) & np.isfinite(y)
x = x[valid]
y = y[valid]

if x.size < 3:
    print(
        f"Only {x.size} valid points after distance filter "
        f"(min={MIN_DISTANCE_KM}, max={MAX_DISTANCE_KM})."
    )
else:
    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))

    fig, ax = plt.subplots(figsize=(9, 7))
    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1)
    cbar = fig.colorbar(hb, ax=ax)
    cbar.set_label("Count")

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=11,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="black", alpha=0.55, edgecolor="white"),
        color="white",
    )

    min_lbl = MIN_DISTANCE_KM if MIN_DISTANCE_KM is not None else "-inf"
    max_lbl = MAX_DISTANCE_KM if MAX_DISTANCE_KM is not None else "+inf"
    ax.set_title(f"RAMA u vs Altimetry gu | distance_km in [{min_lbl}, {max_lbl}]")
    ax.set_xlabel("Altimetry gu (m/s)")
    ax.set_ylabel("RAMA u (m/s)")
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right")
    fig.tight_layout()
    fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)

    print(f"Rows after distance filter: {len(df)}")
    print(f"Saved {OUT_PNG}")

```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

CSV_PATH = "rama_guv_c_nearest_global_matches.ek.csv"
OUT_PNG = "scatter_rama_u_vs_gu_ek.png"
MIN_DISTANCE_KM = None
MAX_DISTANCE_KM = 250.0


df = pd.read_csv(CSV_PATH)
print(df.columns)

dist = df["distance_km"].to_numpy(dtype=float)
keep = np.isfinite(dist)
if MIN_DISTANCE_KM is not None:
    keep &= dist >= float(MIN_DISTANCE_KM)
if MAX_DISTANCE_KM is not None:
    keep &= dist <= float(MAX_DISTANCE_KM)
df = df.loc[keep].copy()

x1 = df["gu"].to_numpy(dtype=float)
x2 = df["u_ek"].to_numpy(dtype=float)
y = df["rama_u"].to_numpy(dtype=float)

x = x1 + x2
valid = np.isfinite(x) & np.isfinite(y)
x = x[valid]
y = y[valid]

if x.size < 3:
    print(
        f"Only {x.size} valid points after distance filter "
        f"(min={MIN_DISTANCE_KM}, max={MAX_DISTANCE_KM})."
    )
else:
    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))

    fig, ax = plt.subplots(figsize=(9, 7))
    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1)
    cbar = fig.colorbar(hb, ax=ax)
    cbar.set_label("Count")

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=11,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="black", alpha=0.55, edgecolor="white"),
        color="white",
    )

    min_lbl = MIN_DISTANCE_KM if MIN_DISTANCE_KM is not None else "-inf"
    max_lbl = MAX_DISTANCE_KM if MAX_DISTANCE_KM is not None else "+inf"
    ax.set_title(f"RAMA u vs Altimetry gu | distance_km in [{min_lbl}, {max_lbl}]")
    ax.set_xlabel("Altimetry gu (m/s)")
    ax.set_ylabel("RAMA u (m/s)")
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right")
    fig.tight_layout()
    fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)

    print(f"Rows after distance filter: {len(df)}")
    print(f"Saved {OUT_PNG}")

```

```python
import math
import os

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
# codex resume 019c8e05-4ac3-7611-b3f5-b58c982002e0
ds_coast = xr.open_dataset(
    f"~/allData/topo/GMT_intermediate_coast_distance_01d_track_reg.nc"
)
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(*TRACKS_REG[2:]))
cmap1 = cmo.cm.diff
COAST_DISTANCE_KM = 100
d = 1.5
out_dir = "track_guvVSoscar_at_intersection_points/coastal_u_timeseries"
os.makedirs(out_dir, exist_ok=True)


def is_close_to_coast(ds_coast, lon, lat, dist_km):
    dist1 = ds_coast.coast_distance.sel(lon=lon, lat=lat, method="nearest").item()
    return abs(dist1) <= dist_km


def format_track_id(v):
    if pd.isna(v):
        return None
    if isinstance(v, (int, np.integer)):
        return str(v)
    if isinstance(v, (float, np.floating)) and float(v).is_integer():
        return str(int(v))
    return str(v).strip()


frac1 = "0.05"
track_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections/"
oscar_dir = "/home/srinivasu/allData/oscar_0.25/oscar_at_intersection_points"
std_rows = []
print(sats_new)
for sat in sats_new[:]:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    for i, r in df.iterrows():
        sat1 = r["sat"]
        track_number_self = format_track_id(r["track_self"])
        track_number_other = format_track_id(r["track_other"])
        if track_number_self is None or track_number_other is None:
            continue
        lons_inter1 = r["lons_inter"]
        lats_inter1 = r["lats_inter"]
        x_from_coast_self1 = r["x_from_coast_self"]
        x_from_coast_other1 = r["x_from_coast_other"]
        angle_acute = r["angle_acute"]
        angle_obtuse = r["angle_obtuse"]
        lon1 = lons_inter1
        lat1 = lats_inter1
        if not is_close_to_coast(ds_coast, lon1, lat1, COAST_DISTANCE_KM):
            continue
        if abs(lat1) < 2:
            continue
        track_uv = f"{track_dir}/guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{frac1}.nc"
        oscar_uv = f"{oscar_dir}/oscar_uv_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc"
        f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
        f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
        ds_track = xr.open_dataset(track_uv)
        ds_oscar = xr.open_dataset(oscar_uv)
        ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
        ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)

        gu = ds_track.gu
        oscar_u = ds_oscar.u
        tfreq = sats_tfreq.get(sat1, None)
        # if tfreq is not None:
        #     oscar_u = oscar_u.resample(time=f"{tfreq}D").mean()
        # oscar_u = oscar_u.interp(time=gu.time)
        oscar_u = oscar_u.sel(time=gu.time, method="nearest")
        std_rows.append(
            {
                "sat": sat1,
                "track_self": track_number_self,
                "track_other": track_number_other,
                "std_oscar": oscar_u.std(dim="time", skipna=True).item(),
                "std_track": gu.std(dim="time", skipna=True).item(),
            }
        )

        lons_track_self = ds_self.lon.values
        lats_track_self = ds_self.lat.values
        lons_track_other = ds_other.lon.values
        lats_track_other = ds_other.lat.values

        lon_equat_self = lons_track_self[0]
        lat_equat_self = lats_track_self[0]
        lon_coast_self = lons_track_self[-1]
        lat_coast_self = lats_track_self[-1]
        slope_self = (lat_coast_self - lat_equat_self) / (lon_coast_self - lon_equat_self)
        angle_self = np.rad2deg(np.arctan(slope_self))

        lon_equat_other = lons_track_other[0]
        lat_equat_other = lats_track_other[0]
        lon_coast_other = lons_track_other[-1]
        lat_coast_other = lats_track_other[-1]
        slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other - lon_equat_other)
        angle_other = np.rad2deg(np.arctan(slope_other))

        fig = plt.figure(figsize=(12, 10))
        ax_map = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
        ax_ts = fig.add_subplot(2, 1, 2)
        rose_cut.plot(
            ax=ax_map, add_colorbar=False, add_labels=False, cmap=cmap1
        )
        decorate_axis(ax_map, "", *TRACKS_REG, step=5)
        ax_map.grid()
        ax_map.plot(
            lon1,
```

```
            lat1,
            c="r",
            marker="o",
            markersize=6,
            markerfacecolor="red",
            markeredgecolor="black",
            markeredgewidth=1.5,
        )
        ax_map.scatter(lons_track_self, lats_track_self, marker=".", color="c", s=4)
        ax_map.scatter(lons_track_other, lats_track_other, marker=".", color="c", s=4)

        lonm, latm = get_point_at_distance(
            lon_equat_self, lat_equat_self, lon_coast_self, lat_coast_self, d
        )
        if is_within_region(lonm, latm, *TRACKS_REG):
            ax_map.text(lonm, latm, s=track_number_self, fontsize=12, rotation=angle_self, color="k")

        lonm, latm = get_point_at_distance(
            lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d
        )
        if is_within_region(lonm, latm, *TRACKS_REG):
            ax_map.text(lonm, latm, s=track_number_other, fontsize=12, rotation=angle_other, color="k")

        oscar_u.plot(ax=ax_ts, color="b", linewidth=2, label="oscar u")
        gu.plot(ax=ax_ts, color="r", linewidth=2, label="track u")
        ax_ts.legend()
        ax_ts.set_ylabel("u (m/s)")
        ax_ts.set_xlabel("Time")
        ax_ts.set_title(
            f"{sat1}: ({lon1:.2f}, {lat1:.2f}), coast <= {COAST_DISTANCE_KM} km"
        )

        fig.tight_layout()
        plt.savefig(
            f"{out_dir}/u_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
            dpi=180,
        )
        # plt.show()
        plt.close(fig)

        ds_track.close()
        ds_oscar.close()
        ds_self.close()
        ds_other.close()

ds_coast.close()
dse.close()
df_std = pd.DataFrame(
    std_rows, columns=["sat", "track_self", "track_other", "std_oscar", "std_track"]
)
df_std.to_csv(f"{out_dir}/std_oscar_track_u_coastal.csv", index=False)

```

```
1
2  gc variable which applies both on x and time dimensions.
3  This is to be used in c.gc_at_intersection_points_loess_0.05.py
4  There is 1d version of function in
5  ../lib/tools_xtrackm.py
6
7  Club following script in to p.scatter_omniVSrama_uVSgu_ww_ek.py to plot a 2x2
8  subplots with first row containing scatter plots (with and without ekman) of
9  rama_uVSgu and second two containing scatter plots (with and without ekman) of
10 omni_uVSgu
11 p.scatter_rama_u_vs_gu_ek.py
12 p.scatter_rama_u_vs_gu.py
13 p.scatter_omni_u_vs_gu_ek.py
14 p.scatter_omni_u_vs_gu.py
15
16 Refactor the code c.guv_c.sat_wise.stats.py to use pre computed guv fields
17 at /home/srinivasu/xtrackm/guv/guv_at_intersections
18
19 Refactor the code p.rama_uvVStrack_guv_c.nearest.all.py to use pre computed guv fields
20 at /home/srinivasu/xtrackm/guv/guv_at_intersections
21 use functions in p.aviso_only_uvVStrack_guv_c.at.intersections.py to find
22 track angle etc.
23
24 Write a python script to plot the scatter of rama_guv_c_nearest_global_matches.csv very much similar to p.scatter_drifter_ve_vs_gu.py
25 We should able to filter rows based on the distance_km
26 Time based filtering is not needed.
27
28 In the original code p.rama_uvVStrack_guv_c.nearest.all.py
29 add a global csv file that stores, sat, rama_id, distance_km, track_self, track_other, matched pairs of rama_u and gu along with time stamps
30 This will be used for future scatter plot.
31
32 Any major issues you found? Better ways of doing it. Please state. Don't edit any code yet.
33 rama data is usually daily. Satellite periodicity varies sat to sat which is given by tfreq key. Bringing both to same time axis properly is where I think I have issues.
34
35 Refactor the code p.aviso_only_uvVStrack_guv_c.at.intersections.py to use pre computed guv fields at /home/srinivasu/xtrackm/guv/guv_at_intersections
36
37 make use of the script c.guv_at_intersection_points_0.05.py and make a new
38 script that compares local_angle and angle from end points are reasonably
39 equal.
40
41 If g_1=u*sin(α)-v*cos(α) and g_2=u*sin(β)-v*cos(β), solve for u and v.
42 Here α+β=π, α is acute and β is obtuse
43 in g_1=u*sin(α)-v*cos(α), substitude α=π-β and simplify.
44
45 I am computing zonal and meridional components of velocities from cross track
46 geostrophic velocities at the intersection point of ascending and descending
47 satellite altimetry tracks. Do you find any major issues with the above script.
48
49 Write a python script to read oscar_onlyVStrack_corrs_rmses_biass_TP+J1+J2+J3+S6A_u.csv, oscar_ekmnVStrack_corrs_rmses_biass_TP+J1+J2+J3+S6A_u.csv to make comparison p
       lot of the respective correlations, biases, rmses before and after applying ekman correction.
50 Need to do the following steps.
51 Make a unique string column combining sat (zero padded to 20 width), track_self, track_other for both the dataframes. We need to take only those rows where this combin
       ation matches in both dataframes so that we can make correct comparison.
52 While comparing a metric, we also need to drop nan values from the dataframes. We only take those rows whose values are valid in both the metric columns.
53 Plot 2x2 grid of 3 bar charts comparing bias (first subplot), corr (next), rmse (next) of oscar only and oscar ekman from each file.
54 Keep the code simple and dirty. No need of putting a lot of checks. Nobody is going to use this piece of code expect me. I am fully of aware of the context.
55
56 When p value > 0.05, plot corr bar in black color
57
58 Make following adjustments to the attached code:
59 We need to add ekman u,v components to gu, gv before comparing them to oscar.
60 Ekman currents extracted at the intersection of tracks are in the folder /home/srinivasu/xtrackm/ekman/ekman_at_intersections/
61 in the format ekman_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc.
62 We need to read these file under the loop and make necessary adjustments to add to gu, gv.
63 meta data of one the extracted file is as follows
64 netcdf ekman_at_intersection_TP+J1+J2+J3+S6A_81_220 {
65 dimensions:
66     time = 10578 ;
67 variables:
68     double time(time) ;
69         time:_FillValue = NaN ;
70         time:standard_name = "time" ;
71         time:axis = "T" ;
72         time:units = "days since 1993-01-02" ;
73         time:calendar = "proleptic_gregorian" ;
74     float lon ;
75         lon:_FillValue = NaNf ;
76         lon:axis = "X" ;
77     float lat ;
78         lat:_FillValue = NaNf ;
79         lat:axis = "Y" ;
80     float u(time) ;
81         u:_FillValue = NaNf ;
82         u:coordinates = "lat lon" ;
83         u:missing_value = NaNf ;
84     float v(time) ;
85         v:_FillValue = NaNf ;
86         v:coordinates = "lat lon" ;
87         v:missing_value = NaNf ;
88 }
89
90
91 I am trying to compare zonal component of current computed from two cross track geostrophic currents at the intersection point of the tracks with loess smoothed sla with oscar ocean currents. The zonal
92 component of oscar compare well with computed geostrophic zonal component but meridional component does not compare well at all. What could be fundamental reason? Some
       studies claim that two tracks' cross track geostrophic currents are needed to compute components, but studies are doing with one track alone.
93
94 I expected correlations of above 0.9 with aviso guv, because they are purely geostrophic, but it was not the case. correlations are similar to
95 that of with oscar.
96 track meridional velocities are usually poorly correlated with other products. What could be the reason?
97 lots of difference between track gv and aviso gv at points very close to
98 coast.
99
```

```python
import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
from cartopy.mpl.ticker import LatitudeFormatter, LongitudeFormatter

TRACKS_REG = (43.0, 99, 0, 25.4)
fe = "~/allData/topo/etopo5.cdf"
dse = xr.open_dataset(fe)
chunk = "1_5000"
chunk = "5001_10000"
chunk = "10001_15000"
chunk = "15001_current"
chunks = ["1_5000", "5001_10000", "10001_15000", "15001_current"]


col_names = [
    "id",
    "mm",
    "dd",
    "yy",
    "lat",
    "lon",
    "temp",
    "ve",
    "vn",
    "spd",
    "var_lat",
    "var_lon",
    "var_tmp",
]


def drifter_time_asn(ds_d, var_str="ve", daily=False):
    time1 = ds_d.time.isel(traj=0).values
    var1 = ds_d[var_str].isel(traj=0).values
    var_asn = xr.DataArray(var1,
                           coords=[time1],
                           dims=["time"],
                           attrs=ds_d[var_str].attrs)
    if daily:
        var_asn = var_asn.resample(time="1D").mean()
    var_asn.name = var_str
    return var_asn


def var_asn(ds, var_str="ve", daily=False):
    lon = ds.longitude.values
    lat = ds.latitude.values
    time1= ds.time.isel(traj=0).values
    var1 = ds[var_str].values
    var_asn = xr.DataArray(var1,coords=[[1],time1],dims=["traj" ,"time"])
    if daily:
        var_asn = var_asn.resample(time="1D").mean()
    return var_asn


def _parse_date(mm, dd, yy):
    dd = float(dd)
    mm = int(mm)
    yy = int(yy)
    day = int(dd)
    hour = int(24 * (dd - day))
    return datetime(yy, mm, day, hour)


def read_buoy_data(fname):
    """Read NOAA Global Drifter Program text data into pandas format.

    Parameters
    ----------
    fname : str

    Returns
    -------
    df : datafram
    """
     return pd.read_csv(
         fname,
         names=col_names,
         sep="\s+",
         header=None,
         na_values=999.999,
         parse_dates={"time": [1, 2, 3]},
         date_parser=_parse_date,
     )


def read_buoy_data(file_path):
    # Define column names
    col_names = [
        "id",
        "mm",
        "dd",
        "yy",
        "lat",
        "lon",
        "temp",
        "ve",
        "vn",
        "spd",
        "var_lat",
        "var_lon",
        "var_tmp",
    ]
    # Read the file into a pandas dataframe
    df = pd.read_csv(
        file_path, delim_whitespace=True, header=None, names=col_names, na_values=999.999
    )
    # Separate the integer and fractional parts of the day
    df["day_fraction"], df["dd"] = np.modf(df["dd"])
    # Convert the integer part of the day to integer type
    df["dd"] = df["dd"].astype(int)
    # Calculate the hours from the fractional part of the day
    df["hours"] = (df["day_fraction"] * 24).round().astype(int)
    # Combine the date and time components into a single datetime column
```

```
    df["datetime"] = pd.to_datetime(
        df[["yy", "mm", "dd"]].astype(str).agg("-".join, axis=1)
    ) + pd.to_timedelta(df["hours"], unit="h")
    # Drop the intermediary columns used for calculations
    df = df.drop(columns=["mm", "dd", "yy", "day_fraction", "hours"])
    return df


def is_within_region(lons, lats, lon_min, lon_max, lat_min, lat_max):
    lons_within_range = np.logical_and(lons >= lon_min, lons <= lon_max)
    # Check if all latitudes are within the range
    lats_within_range = np.logical_and(lats >= lat_min, lats <= lat_max)
    # Combine both conditions
    all_within_region = np.logical_and(lons_within_range, lats_within_range)
    # Check if all points are within the region
    if np.all(all_within_region):
        print("All points are within the region.")
        return True
    else:
        print("Not all points are within the region.")
        return False

```

guv/workflow.txt

```
1
2  guv_c usually uses pre-computed and filtered guv
3
```

```
./guv/archive/
```

```
8 file(s)
```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *


frac1 = "0.05"
for sat in sats_new:
    gu_all = []
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    for i, row in df.iterrows():
        track_number_self = str(row["track_self"])
        track_number_other = str(row["track_other"])
        lons_inter1 = row["lons_inter"]
        lats_inter1 = row["lats_inter"]
        lon1 = lons_inter1
        lat1 = lats_inter1
        x_from_coast_self1 = row["x_from_coast_self"]
        x_from_coast_other1 = row["x_from_coast_other"]
        angle_acute = row["angle_acute"]
        angle_obtuse = row["angle_obtuse"]
        if abs(lat1) < 2:
            continue
        print(
            sat,
            track_number_self,
            track_number_other,
            lons_inter1,
            lats_inter1,
            x_from_coast_self1,
            x_from_coast_other1,
            angle_acute,
            angle_obtuse,
        )
        f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
        f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
        f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_self}_loess_{frac1}.nc"
        f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_other}_loess_{frac1}.nc"
        ds_self = xr.open_dataset(f_self, decode_times=False)
        ds_self_smooth = xr.open_dataset(f_self_smooth)
        sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
        sla_self_smooth = ds_self_smooth.sla_smooth
        ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
        ds_other_smooth = xr.open_dataset(f_other_smooth)
        sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
        sla_other_smooth = ds_other_smooth.sla_smooth
        lons_track_self = ds_self.lon.values
        lats_track_self = ds_self.lat.values
        lon_coast_self = lons_track_self[-1]  # this on coast
        lat_coast_self = lats_track_self[-1]  # this on coast
        lon_equat_self = lons_track_self[0]  # this on equator
        lat_equat_self = lats_track_self[0]  # this on equator
        slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
            lons_track_self[-1] - lons_track_self[0])
        angle_self = np.rad2deg(np.arctan(slope_self))
        track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
        lons_track_other = ds_other.lon.values
        lats_track_other = ds_other.lat.values
        lon_equat_other = lons_track_other[0]
        lat_equat_other = lats_track_other[0]
        lon_coast_other = lons_track_other[-1]
        lat_coast_other = lats_track_other[-1]
        slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                             lon_equat_other)
        angle_other = np.rad2deg(np.arctan(slope_other))
        track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
        gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
        gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
        # if track_path_self.intersects(track_path_other):
        gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
        gc_other_at = gc_other.sel(x=x_from_coast_other1,
                                   method="nearest",
                                   drop=True)
        gc_other_at = gc_other_at.interp(time=gc_self_at.time)
        # Better approach:
        a2 = math.atan2(lat_coast_other - lat_equat_other,
                        lon_coast_other - lon_equat_other)
        a1 = math.atan2(lat_coast_self - lat_equat_self,
                        lon_coast_self - lon_equat_self)

        gu, gv = geostrophic_components_from_a(gc_self_at, gc_other_at, a1, a2)
        # gu.plot()
        gu_all.extend(gu.values)
        # save as npz file
        # break
    # plot histogram of gu_all
    plt.hist(gu_all, bins=100, label=f"{sat}")
    np.savez(f"guv_at_intersections_{sat}_{frac1}.npz", gu_all=gu_all)
    # plt.show()

```

```python
import math

import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
from matplotlib import pyplot as plt
from scipy.stats import median_abs_deviation
import math


ioff = 3
out_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections/"
print(sats_new)
# We are computing guv for only on track
sat = "GFO"
df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
for i, r in df.iterrows():
    sat1 = r["sat"]
    track_number_self = str(r["track_self"])
    track_number_other = str(r["track_other"])
    lons_inter1 = r["lons_inter"]
    lats_inter1 = r["lats_inter"]
    x_from_coast_self1 = r["x_from_coast_self"]
    x_from_coast_other1 = r["x_from_coast_other"]
    angle_acute = r["angle_acute"]
    angle_obtuse = r["angle_obtuse"]
    lon1 = lons_inter1
    lat1 = lats_inter1
    if abs(lat1) < 2:
        continue
    print(
        sat,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_0.05a/track_sla_along_{sat}_{track_number_self}_loess_0.05.nc"
    f_other_smooth = f"../computed/sla_loess_0.05a/track_sla_along_{sat}_{track_number_other}_loess_0.05.nc"
    ds_self = xr.open_dataset(f_self,
                              engine="netcdf4",
                              decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other,
                               engine="netcdf4",
                               decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    ii_self = index_at_lat(ds_self, lats_inter1)
    a1 = local_angle(lons_track_self, lats_track_self, ii_self, ioff=ioff)
    a1_d = math.degrees(a1)
    ii_other = index_at_lat(ds_other, lats_inter1)
    a2 = local_angle(lons_track_other, lats_track_other, ii_other, ioff=ioff)
    a2_d = math.degrees(a2)
    print(f"heading angle {sat}: a1 = {a1_d:.2f}, a2 = {a2_d:.2f}")
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.sel(x=x_from_coast_self1,
                             method="nearest",
                             drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at_interp = gc_other_at.interp(time=gc_self_at.time)
    gu, gv = geostrophic_components(gc_self_at, gc_other_at_interp, a1, a2)
    break
    # filter xarray dataarray gu if greater than 3.5 or less than -3.5

```

```python
import math

import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
from matplotlib import pyplot as plt
from scipy.stats import median_abs_deviation


out_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections/"
for sat in sats_new:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    for i, r in df.iterrows():
        sat1 = r["sat"]
        track_number_self = str(r["track_self"])
        track_number_other = str(r["track_other"])
        lons_inter1 = r["lons_inter"]
        lats_inter1 = r["lats_inter"]
        x_from_coast_self1 = r["x_from_coast_self"]
        x_from_coast_other1 = r["x_from_coast_other"]
        angle_acute = r["angle_acute"]
        angle_obtuse = r["angle_obtuse"]
        lon1 = lons_inter1
        lat1 = lats_inter1
        if abs(lat1) < 2:
            continue
        print(
            sat,
            track_number_self,
            track_number_other,
            lons_inter1,
            lats_inter1,
            x_from_coast_self1,
            x_from_coast_other1,
            angle_acute,
            angle_obtuse,
        )
        f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
        f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
        f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_self}_loess_0.2.nc"
        f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_other}_loess_0.2.nc"
        ds_self = xr.open_dataset(f_self,
                                  engine="netcdf4",
                                  decode_times=False)
        ds_self_smooth = xr.open_dataset(f_self_smooth)
        sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
        sla_self_smooth = ds_self_smooth.sla_smooth
        ds_other = xr.open_dataset(f_other,
                                   engine="netcdf4",
                                   decode_times=False)
        ds_other_smooth = xr.open_dataset(f_other_smooth)
        sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
        sla_other_smooth = ds_other_smooth.sla_smooth
        lons_track_self = ds_self.lon.values
        lats_track_self = ds_self.lat.values
        lon_coast_self = lons_track_self[-1]  # this on coast
        lat_coast_self = lats_track_self[-1]  # this on coast
        lon_equat_self = lons_track_self[0]  # this on equator
        lat_equat_self = lats_track_self[0]  # this on equator
        slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
            lons_track_self[-1] - lons_track_self[0])
        lons_track_other = ds_other.lon.values
        lats_track_other = ds_other.lat.values
        lon_equat_other = lons_track_other[0]
        lat_equat_other = lats_track_other[0]
        lon_coast_other = lons_track_other[-1]
        lat_coast_other = lats_track_other[-1]
        slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                             lon_equat_other)
        gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
        gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
        gc_self_at = gc_self.sel(x=x_from_coast_self1,
                                 method="nearest",
                                 drop=True)
        gc_other_at = gc_other.sel(x=x_from_coast_other1,
                                   method="nearest",
                                   drop=True)
        gc_other_at = gc_other_at.interp(time=gc_self_at.time)
        a2 = math.atan2(
            lat_coast_other - lat_equat_other,
            lon_coast_other - lon_equat_other,
        )
        a1 = math.atan2(lat_coast_self - lat_equat_self,
                        lon_coast_self - lon_equat_self)
        print(f"azimuths {a1:.2f} {a2:.2f}")
        gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
        # filter xarray dataarray gu if greater than 3.5 or less than -3.5
        gu_filt = gu.where((gu <= 3.5) & (gu >= -3.5))
        gu_filtered = mad_filter_1d_xr(gu_filt, 3.5)
        gv_filt = gv.where((gv <= 3.5) & (gv >= -3.5))
        gv_filtered = mad_filter_1d_xr(gv_filt, 3.5)
        # gu.plot(label="gu", marker=".")
        # gu_filtered.plot(label="gu_filtered", marker="x")
        # plt.legend()
        # plt.show()
        # create a ds_out with gu and gv
        ds_out = xr.Dataset({"gu": gu_filtered, "gv": gv_filtered})
        filename = f"{out_dir}/guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_0.2.nc"
        print(filename)
        ds_out.to_netcdf(filename)
    # close df

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *


def geostrophic_components(gc1, gc2, a1_r, a2_r):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities measured by two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    a1_r : float
        Heading angle of track 1 in radians (measured counter-clockwise from east)
    a2_r : float
        Heading angle of track 2 in radians (measured counter-clockwise from east)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (intersection angle ≈ 0°)
    """
    # Convert azimuths from degrees to radians
    theta = a2_r - a1_r
    assert a2_r < 3, "check if heading angles are in radians or not"
    assert a1_r < 3, "check if heading angles are in radians or not"
    sin_theta = math.sin(theta)
    # Check for parallel tracks
    if abs(sin_theta) < 1e-6:
        raise ValueError(f"Tracks are nearly parallel"
                         "(angle = {math.degrees(theta):.2f}°). "
                        "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1_r)
    cos_a2 = math.cos(a2_r)
    sin_a1 = math.sin(a1_r)
    sin_a2 = math.sin(a2_r)
    # Solve for velocity components using the derived formulas:
    u = (gc1 * cos_a2 - gc2 * cos_a1) / math.sin(a1_r - a2_r)
    v = (gc1 * sin_a2 - gc2 * sin_a1) / math.sin(a1_r - a2_r)
    return u, v


def haversine_distance(lon1, lat1, lon2, lat2):
    """Calculate great-circle distance between (lon1, lat1)
    and arrays (lon2, lat2) in kilometers."""
    R = 6371  # Radius of Earth in km
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlon = lon2 - lon1
    dlat = lat2 - lat1
    a = (np.sin(dlat / 2)**2 +
         np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2)**2)
    return 2 * R * np.arcsin(np.sqrt(a))


def find_n_closest_points(df, lon0, lat0, n=4):
    """Return the n rows in df closest to (lon0, lat0) based on
    lons_inter/lats_inter."""
    distances = haversine_distance(lon0, lat0, df["lons_inter"].values,
                                   df["lats_inter"].values)
    df = df.copy()
    df["distance_km"] = distances
    return df.nsmallest(n, "distance_km")


cmap1 = cmo.cm.diff
d = 1.5
dpth = 15.0
height, width = 9, 9
oscar_dir = "/home/srinivasu/allData/oscar_0.25/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
omni_data_dir = "/home/srinivasu/allData/MB_cleaned/"
fracl = "0.05"

# ds_oscar_u = xr.open_dataset(f"{oscar_dir}/oscar_0.25_u_full_xr_concat.nc")
# ds_oscar_v = xr.open_dataset(f"{oscar_dir}/oscar_0.25_v_full_xr_concat.nc")
# print(ds_oscar_u)
# print(ds_oscar_v)
# u_oscar = ds_oscar_u.u
# v_oscar = ds_oscar_v.v

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

for omni_id in omni_d.keys():
    lon_omni, lat_omni = omni_d[omni_id]
    ds_omni = xr.open_dataset(f"{omni_data_dir}/{omni_id}_uvs_z_xr_daily.nc",
                              decode_times=False)
    print(ds_omni)
    ds_omni = change_time(ds_omni, "time")
    omni_u = ds_omni.ud.interp(depth=dpth)
    omni_u = omni_u.rolling(time=10, center=True).mean()
    omni_u = 0.01 * omni_u
    omni_v = ds_omni.vd.interp(depth=dpth)
    omni_v = omni_v.rolling(time=10, center=True).mean()
    omni_v = 0.01 * omni_v
    N = omni_v.count().item()
    print(N)
    df_4 = find_n_closest_points(df_all, lon_omni, lat_omni, n=4)
    # print(omni_id, lon_omni, lat_omni, "\n", df_4.head(1))
    for nth in range(4):
        sat = df_4["sat"].values[nth]
```

```
        track_number_self = df_4["track_self"].apply(lambda x: str(x)).values[nth]
        track_number_other = df_4["track_other"].apply(lambda x: str(x)).values[nth]
        lons_inter1 = df_4["lons_inter"].values[nth]
        lats_inter1 = df_4["lats_inter"].values[nth]
        lon1 = lons_inter1
        lat1 = lats_inter1
        x_from_coast_self1 = df_4["x_from_coast_self"].values[nth]
        x_from_coast_other1 = df_4["x_from_coast_other"].values[nth]
        angle_acute = df_4["angle_acute"].values[nth]
        angle_obtuse = df_4["angle_obtuse"].values[nth]
        distance_km = df_4["distance_km"].values[nth]
        if abs(lat1) < 2:
            continue
        print(
            sat,
            omni_id,
            track_number_self,
            track_number_other,
            lons_inter1,
            lats_inter1,
            x_from_coast_self1,
            x_from_coast_other1,
            angle_acute,
            angle_obtuse,
        )
        f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
        f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
        f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_self}_loess_{frac1}.nc"
        f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_other}_loess_{frac1}.nc"
        ds_self = xr.open_dataset(f_self, decode_times=False)
        ds_self_smooth = xr.open_dataset(f_self_smooth)
        sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
        sla_self_smooth = ds_self_smooth.sla_smooth
        ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
        ds_other_smooth = xr.open_dataset(f_other_smooth)
        sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
        sla_other_smooth = ds_other_smooth.sla_smooth
        lons_track_self = ds_self.lon.values
        lats_track_self = ds_self.lat.values
        lon_coast_self = lons_track_self[-1]  # this on coast
        lat_coast_self = lats_track_self[-1]  # this on coast
        lon_equat_self = lons_track_self[0]  # this on equator
        lat_equat_self = lats_track_self[0]  # this on equator
        slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
            lons_track_self[-1] - lons_track_self[0])
        angle_self = np.rad2deg(np.arctan(slope_self))
        track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
        lons_track_other = ds_other.lon.values
        lats_track_other = ds_other.lat.values
        lon_equat_other = lons_track_other[0]
        lat_equat_other = lats_track_other[0]
        lon_coast_other = lons_track_other[-1]
        lat_coast_other = lats_track_other[-1]
        slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                             lon_equat_other)
        angle_other = np.rad2deg(np.arctan(slope_other))
        track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
        # u_oscar_at = u_oscar.sel(lon=lon1, lat=lat1, method="nearest")
        # v_oscar_at = v_oscar.sel(lon=lon1, lat=lat1, method="nearest")
        gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
        gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
        # if track_path_self.intersects(track_path_other):
        gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
        gc_other_at = gc_other.sel(x=x_from_coast_other1,
                                   method="nearest",
                                   drop=True)
        gc_other_at = gc_other_at.interp(time=gc_self_at.time)
        # Better approach:
        a2 = math.atan2(lat_coast_other - lat_equat_other,
                        lon_coast_other - lon_equat_other)
        a1 = math.atan2(lat_coast_self - lat_equat_self,
                        lon_coast_self - lon_equat_self)

        gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
        # other_times.append((track_number_other, abs(dist_time)))
        # gu_cut = -1 * gu.interp(time=omni_u.time)
        # gv_cut = gv.interp(time=omni_v.time)
        gu_cut = gu
        gv_cut = gv
        # gu_cut.plot(ax=ax1, label=f"track zonal", color="r", linewidth=2)
        # omni_u.plot(ax=ax1, color="b", linewidth=2, label="omni")
        # u_oscar_at.plot(ax=ax1, color="b", linewidth=2, label="oscar u")
        # corr = xs.pearson_r(gu_cut, omni_u, dim="time", skipna=True)
        if distance_km > 30:
            continue
        fig = plt.figure(figsize=(12, 8), layout="constrained")
        ax1 = fig.add_subplot(2, 1, 1)
        gu_cut.plot(ax=ax1, label=f"track zonal", color="r", linewidth=2)
        tfreq = sats_tfreq[sat]
        omni_v = omni_v.resample(time=f"{tfreq}D").mean()
        omni_v = omni_v.interp(time=gv_cut.time)
        omni_u.plot(ax=ax1, color="b", linewidth=2, label="omni")
        # v_oscar_at.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
        corr = xs.pearson_r(gv_cut, omni_v, dim="time", skipna=True)
        corr1 = corr.item()
        # print(f"Correlation between gu and u_oscar_at: {corr1}")
        print(f"Correlation between gv and v_oscar_at: {corr1}")
        info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km}"
        plt.text(
            0.15,
            0.95,
        info,
        transform=ax1.transAxes,
        fontsize=14,
        fontweight="bold",
        )
        plt.legend()
        ax2 = fig.add_subplot(2, 1, 2, projection=ccrs.PlateCarree())
        dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
            *TRACKS_REG[2:])).plot(ax=ax2,
                                   add_colorbar=False,
                                   add_labels=False,
                                   cmap=cmap1)
        decorate_axis(ax2, "", *TRACKS_REG)
        ax2.grid()
        ax2.plot(
            lon_omni,
```

```
            lat_omni,
            c="r",
            marker="o",
            markersize=6,
            markerfacecolor="red",
            markeredgecolor="black",
            markeredgewidth=2,
        )
        # plot plat form code of omni at lon_omni, lat_omni
        plt.text(lon_omni, lat_omni, f"{omni_id}", fontsize=10, color="k")
        ax2.scatter(
            lons_track_self,
            lats_track_self,
            marker=".",
            color="c",
            s=4,
        )
        ax2.scatter(
            lons_track_other,
            lats_track_other,
            marker=".",
            color="c",
            s=4,
        )
        lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                           lon_coast_self, lat_coast_self, d)
        if is_within_region(lonm, latm, *TRACKS_REG):
            plt.text(
                lonm,
                latm,
                s=track_number_self,
                fontsize=14,
                rotation=angle_self,
                color="w",
            )
        lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                           lon_coast_other, lat_coast_other, d)
        if is_within_region(lonm, latm, *TRACKS_REG):
            plt.text(
                lonm,
                latm,
                s=track_number_other,
                fontsize=14,
                rotation=angle_other,
                color="w",
            )
        plt.savefig(
            f"omniVStrack_guv_pngs/{omni_id}_{sat}_{track_number_self}_{track_number_other}.png"
        )
        # plt.show()
        plt.close("all")

    dse.close()

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *


def normalize_track_id(series):
    return series.astype(str).str.replace(r"\.0$", "", regex=True).str.strip()


def filter_intersections(df, sat, track_self, track_other):
    sat = str(sat).strip()
    track_self = str(track_self).strip()
    track_other = str(track_other).strip()
    dff = df.copy()
    dff["track_self_norm"] = normalize_track_id(dff["track_self"])
    dff["track_other_norm"] = normalize_track_id(dff["track_other"])
    mask_sat = dff["sat"].astype(str).str.strip() == sat
    pair_forward = ((dff["track_self_norm"] == track_self) &
                    (dff["track_other_norm"] == track_other))
    pair_reverse = ((dff["track_self_norm"] == track_other) &
                    (dff["track_other_norm"] == track_self))
    return dff[mask_sat & (pair_forward | pair_reverse)].copy()


cmap1 = cmo.cm.diff
d = 1.5
dpth = 15.0
height, width = 9, 9
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
omni_data_dir = "/home/srinivasu/allData/MB_cleaned/"
frac1 = "0.05"

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

omni_id = "AD09"
sat = "S3B"
track_self = "580"
track_other = "651"

df_sel = filter_intersections(df_all, sat, track_self, track_other)
if df_sel.empty:
    raise ValueError(
        f"No intersections for sat={sat}, tracks=({track_self}, {track_other})"
    )

lon_omni, lat_omni = omni_d[omni_id]
ds_omni = xr.open_dataset(f"{omni_data_dir}/{omni_id}_uvs_z_xr_daily.nc",
                          decode_times=False)
print(ds_omni)
ds_omni = change_time(ds_omni, "time")
omni_u = ds_omni.ud.interp(depth=dpth)
omni_u = omni_u.rolling(time=10, center=True).mean()
omni_u = 0.01 * omni_u
omni_v = ds_omni.vd.interp(depth=dpth)
omni_v = omni_v.rolling(time=10, center=True).mean()
omni_v = 0.01 * omni_v
N = omni_v.count().item()
print(N)

df_1 = find_n_closest_points(df_sel, lon_omni, lat_omni, n=1)
row = df_1.iloc[0]
sat = row["sat"]
track_number_self = str(row["track_self_norm"])
track_number_other = str(row["track_other_norm"])
lons_inter1 = row["lons_inter"]
lats_inter1 = row["lats_inter"]
lon1 = lons_inter1
lat1 = lats_inter1
x_from_coast_self1 = row["x_from_coast_self"]
x_from_coast_other1 = row["x_from_coast_other"]
angle_acute = row["angle_acute"]
angle_obtuse = row["angle_obtuse"]
distance_km = row["distance_km"]
if abs(lat1) < 2:
    raise ValueError(f"Selected intersection near equator (lat={lat1:.2f}); skipped.")
print(
    sat,
    omni_id,
    track_number_self,
    track_number_other,
    lons_inter1,
    lats_inter1,
    x_from_coast_self1,
    x_from_coast_other1,
    angle_acute,
    angle_obtuse,
)
f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_self}_loess_{frac1}.nc"
f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_other}_loess_{frac1}.nc"
ds_self = xr.open_dataset(f_self, decode_times=False)
ds_self_smooth = xr.open_dataset(f_self_smooth)
sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
sla_self_smooth = ds_self_smooth.sla_smooth
ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
ds_other_smooth = xr.open_dataset(f_other_smooth)
sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
sla_other_smooth = ds_other_smooth.sla_smooth
lons_track_self = ds_self.lon.values
lats_track_self = ds_self.lat.values
lon_coast_self = lons_track_self[-1]
lat_coast_self = lats_track_self[-1]
lon_equat_self = lons_track_self[0]
lat_equat_self = lats_track_self[0]
slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
    lons_track_self[-1] - lons_track_self[0])
angle_self = np.rad2deg(np.arctan(slope_self))
```

```
lons_track_other = ds_other.lon.values
lats_track_other = ds_other.lat.values
lon_equat_other = lons_track_other[0]
lat_equat_other = lats_track_other[0]
lon_coast_other = lons_track_other[-1]
lat_coast_other = lats_track_other[-1]
slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                     lon_equat_other)
angle_other = np.rad2deg(np.arctan(slope_other))
gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
gc_other_at = gc_other.sel(x=x_from_coast_other1, method="nearest", drop=True)
gc_other_at = gc_other_at.interp(time=gc_self_at.time)
a2 = math.atan2(lat_coast_other - lat_equat_other, lon_coast_other - lon_equat_other)
a1 = math.atan2(lat_coast_self - lat_equat_self, lon_coast_self - lon_equat_self)

gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
gu_cut = gu
gv_cut = gv

if distance_km > 30:
    raise ValueError(f"Nearest selected intersection too far from omni: {distance_km:.2f} km")

fig = plt.figure(figsize=(12, 8), layout="constrained")
ax1 = fig.add_subplot(2, 1, 1)
gu_cut.plot(ax=ax1, label="track zonal", color="r", linewidth=2)
tfreq = sats_tfreq[sat]
omni_v_rs = omni_v.resample(time=f"{tfreq}D").mean().interp(time=gv_cut.time)
omni_u.plot(ax=ax1, color="b", linewidth=2, label="omni u")
ax1.set_xlim(datetime(2019, 1, 1), datetime(2021, 3, 1))
corr = xs.pearson_r(gv_cut, omni_v_rs, dim="time", skipna=True)
corr1 = corr.item()
print(f"Correlation between gv and v_oscar_at: {corr1}")
info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
plt.text(0.15, 0.95, info, transform=ax1.transAxes, fontsize=14, fontweight="bold")
plt.legend()

ax2 = fig.add_subplot(2, 1, 2, projection=ccrs.PlateCarree())
dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
    *TRACKS_REG[2:])).plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=5)
ax2.grid()
ax2.plot(
    lon_omni,
    lat_omni,
    c="r",
    marker="o",
    markersize=6,
    markerfacecolor="red",
    markeredgecolor="black",
    markeredgewidth=2,
)
plt.text(lon_omni, lat_omni, f"{omni_id}", fontsize=10, color="k")
ax2.scatter(lons_track_self, lats_track_self, marker=".", color="c", s=4)
ax2.scatter(lons_track_other, lats_track_other, marker=".", color="c", s=4)
lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self, lon_coast_self,
                                   lat_coast_self, d)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(lonm, latm, s=track_number_self, fontsize=14, rotation=angle_self, color="w")
lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other, lon_coast_other,
                                   lat_coast_other, d)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(lonm, latm, s=track_number_other, fontsize=14, rotation=angle_other, color="w")

plt.show()
plt.savefig(
    f"omniVStrack_guv_pngs/{omni_id}_{sat}_{track_number_self}_{track_number_other}_trim.png"
)
plt.close("all")
ds_omni.close()
dse.close()
# plt.close("all")
#
# dse.close()
omni_id = "AD10"
sat = "S3B"
track_self = "466"
track_other = "651"

df_sel = filter_intersections(df_all, sat, track_self, track_other)
if df_sel.empty:
    raise ValueError(
        f"No intersections for sat={sat}, tracks=({track_self}, {track_other})"
    )

lon_omni, lat_omni = omni_d[omni_id]
ds_omni = xr.open_dataset(f"{omni_data_dir}/{omni_id}_uvs_z_xr_daily.nc",
                          decode_times=False)
print(ds_omni)
ds_omni = change_time(ds_omni, "time")
omni_u = ds_omni.ud.interp(depth=dpth)
omni_u = omni_u.rolling(time=10, center=True).mean()
omni_u = 0.01 * omni_u
omni_v = ds_omni.vd.interp(depth=dpth)
omni_v = omni_v.rolling(time=10, center=True).mean()
omni_v = 0.01 * omni_v
N = omni_v.count().item()
print(N)

df_1 = find_n_closest_points(df_sel, lon_omni, lat_omni, n=1)
row = df_1.iloc[0]
sat = row["sat"]
track_number_self = str(row["track_self_norm"])
track_number_other = str(row["track_other_norm"])
lons_inter1 = row["lons_inter"]
lats_inter1 = row["lats_inter"]
lon1 = lons_inter1
lat1 = lats_inter1
x_from_coast_self1 = row["x_from_coast_self"]
x_from_coast_other1 = row["x_from_coast_other"]
angle_acute = row["angle_acute"]
angle_obtuse = row["angle_obtuse"]
distance_km = row["distance_km"]
if abs(lat1) < 2:
    raise ValueError(f"Selected intersection near equator (lat={lat1:.2f}); skipped.")
print(
```

```
    sat,
    omni_id,
    track_number_self,
    track_number_other,
    lons_inter1,
    lats_inter1,
    x_from_coast_self1,
    x_from_coast_other1,
    angle_acute,
    angle_obtuse,
)
f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_self}_loess_{frac1}.nc"
f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_other}_loess_{frac1}.nc"
ds_self = xr.open_dataset(f_self, decode_times=False)
ds_self_smooth = xr.open_dataset(f_self_smooth)
sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
sla_self_smooth = ds_self_smooth.sla_smooth
ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
ds_other_smooth = xr.open_dataset(f_other_smooth)
sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
sla_other_smooth = ds_other_smooth.sla_smooth
lons_track_self = ds_self.lon.values
lats_track_self = ds_self.lat.values
lon_coast_self = lons_track_self[-1]
lat_coast_self = lats_track_self[-1]
lon_equat_self = lons_track_self[0]
lat_equat_self = lats_track_self[0]
slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
    lons_track_self[-1] - lons_track_self[0])
angle_self = np.rad2deg(np.arctan(slope_self))
lons_track_other = ds_other.lon.values
lats_track_other = ds_other.lat.values
lon_equat_other = lons_track_other[0]
lat_equat_other = lats_track_other[0]
lon_coast_other = lons_track_other[-1]
lat_coast_other = lats_track_other[-1]
slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                     lon_equat_other)
angle_other = np.rad2deg(np.arctan(slope_other))
gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
gc_other_at = gc_other.sel(x=x_from_coast_other1, method="nearest", drop=True)
gc_other_at = gc_other_at.interp(time=gc_self_at.time)
a2 = math.atan2(lat_coast_other - lat_equat_other, lon_coast_other - lon_equat_other)
a1 = math.atan2(lat_coast_self - lat_equat_self, lon_coast_self - lon_equat_self)

gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
gu_cut = gu
gv_cut = gv

if distance_km > 30:
    raise ValueError(f"Nearest selected intersection too far from omni: {distance_km:.2f} km")

fig = plt.figure(figsize=(12, 8), layout="constrained")
ax1 = fig.add_subplot(2, 1, 1)
gu_cut.plot(ax=ax1, label="track zonal", color="r", linewidth=2)
tfreq = sats_tfreq[sat]
omni_v_rs = omni_v.resample(time=f"{tfreq}D").mean().interp(time=gv_cut.time)
omni_u.plot(ax=ax1, color="b", linewidth=2, label="omni u")
ax1.set_xlim(datetime(2019, 2, 1), datetime(2020, 6, 1))
corr = xs.pearson_r(gv_cut, omni_v_rs, dim="time", skipna=True)
corr1 = corr.item()
print(f"Correlation between gv and v_oscar_at: {corr1}")
info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
plt.text(0.15, 0.95, info, transform=ax1.transAxes, fontsize=14, fontweight="bold")
plt.legend()

ax2 = fig.add_subplot(2, 1, 2, projection=ccrs.PlateCarree())
dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
    *TRACKS_REG[2:])).plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=5)
ax2.grid()
ax2.plot(
    lon_omni,
    lat_omni,
    c="r",
    marker="o",
    markersize=6,
    markerfacecolor="red",
    markeredgecolor="black",
    markeredgewidth=2,
)
plt.text(lon_omni, lat_omni, f"{omni_id}", fontsize=10, color="k")
ax2.scatter(lons_track_self, lats_track_self, marker=".", color="c", s=4)
ax2.scatter(lons_track_other, lats_track_other, marker=".", color="c", s=4)
lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self, lon_coast_self,
                                   lat_coast_self, d)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(lonm, latm, s=track_number_self, fontsize=14, rotation=angle_self, color="w")
lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other, lon_coast_other,
                                   lat_coast_other, d)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(lonm, latm, s=track_number_other, fontsize=14, rotation=angle_other, color="w")

plt.show()
plt.savefig(
    f"omniVStrack_guv_pngs/{omni_id}_{sat}_{track_number_self}_{track_number_other}_trim.png"
)
plt.close("all")
ds_omni.close()
dse.close()
# plt.close("all")
#
# dse.close()
omni_id = "BD10"
sat = "S3B"
track_self = "636"
track_other = "393"

df_sel = filter_intersections(df_all, sat, track_self, track_other)
if df_sel.empty:
    raise ValueError(
        f"No intersections for sat={sat}, tracks=({track_self}, {track_other})"
    )
```

```
352
353 lon_omni, lat_omni = omni_d[omni_id]
354 ds_omni = xr.open_dataset(f"{omni_data_dir}/{omni_id}_uvs_z_xr_daily.nc",
355                           decode_times=False)
356 print(ds_omni)
357 ds_omni = change_time(ds_omni, "time")
358 omni_u = ds_omni.ud.interp(depth=dpth)
359 omni_u = omni_u.rolling(time=10, center=True).mean()
360 omni_u = 0.01 * omni_u
361 omni_v = ds_omni.vd.interp(depth=dpth)
362 omni_v = omni_v.rolling(time=10, center=True).mean()
363 omni_v = 0.01 * omni_v
364 N = omni_v.count().item()
365 print(N)
366
367 df_1 = find_n_closest_points(df_sel, lon_omni, lat_omni, n=1)
368 row = df_1.iloc[0]
369 sat = row["sat"]
370 track_number_self = str(row["track_self_norm"])
371 track_number_other = str(row["track_other_norm"])
372 lons_inter1 = row["lons_inter"]
373 lats_inter1 = row["lats_inter"]
374 lon1 = lons_inter1
375 lat1 = lats_inter1
376 x_from_coast_self1 = row["x_from_coast_self"]
377 x_from_coast_other1 = row["x_from_coast_other"]
378 angle_acute = row["angle_acute"]
379 angle_obtuse = row["angle_obtuse"]
380 distance_km = row["distance_km"]
381 if abs(lat1) < 2:
382     raise ValueError(f"Selected intersection near equator (lat={lat1:.2f}); skipped.")
383 print(
384     sat,
385     omni_id,
386     track_number_self,
387     track_number_other,
388     lons_inter1,
389     lats_inter1,
390     x_from_coast_self1,
391     x_from_coast_other1,
392     angle_acute,
393     angle_obtuse,
394 )
395 f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
396 f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
397 f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_self}_loess_{frac1}.nc"
398 f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_other}_loess_{frac1}.nc"
399 ds_self = xr.open_dataset(f_self, decode_times=False)
400 ds_self_smooth = xr.open_dataset(f_self_smooth)
401 sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
402 sla_self_smooth = ds_self_smooth.sla_smooth
403 ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
404 ds_other_smooth = xr.open_dataset(f_other_smooth)
405 sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
406 sla_other_smooth = ds_other_smooth.sla_smooth
407 lons_track_self = ds_self.lon.values
408 lats_track_self = ds_self.lat.values
409 lon_coast_self = lons_track_self[-1]
410 lat_coast_self = lats_track_self[-1]
411 lon_equat_self = lons_track_self[0]
412 lat_equat_self = lats_track_self[0]
413 slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
414     lons_track_self[-1] - lons_track_self[0])
415 angle_self = np.rad2deg(np.arctan(slope_self))
416 lons_track_other = ds_other.lon.values
417 lats_track_other = ds_other.lat.values
418 lon_equat_other = lons_track_other[0]
419 lat_equat_other = lats_track_other[0]
420 lon_coast_other = lons_track_other[-1]
421 lat_coast_other = lats_track_other[-1]
422 slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
423                                                      lon_equat_other)
424 angle_other = np.rad2deg(np.arctan(slope_other))
425 gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
426 gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
427 gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
428 gc_other_at = gc_other.sel(x=x_from_coast_other1, method="nearest", drop=True)
429 gc_other_at = gc_other_at.interp(time=gc_self_at.time)
430 a2 = math.atan2(lat_coast_other - lat_equat_other, lon_coast_other - lon_equat_other)
431 a1 = math.atan2(lat_coast_self - lat_equat_self, lon_coast_self - lon_equat_self)
432
433 gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
434 gu_cut = gu
435 gv_cut = gv
436
437 if distance_km > 30:
438     raise ValueError(f"Nearest selected intersection too far from omni: {distance_km:.2f} km")
439
440 fig = plt.figure(figsize=(12, 8), layout="constrained")
441 ax1 = fig.add_subplot(2, 1, 1)
442 gu_cut.plot(ax=ax1, label="track zonal", color="r", linewidth=2)
443 tfreq = sats_tfreq[sat]
444 omni_v_rs = omni_v.resample(time=f"{tfreq}D").mean().interp(time=gv_cut.time)
445 omni_u.plot(ax=ax1, color="b", linewidth=2, label="omni u")
446 ax1.set_xlim(datetime(2019, 1, 1), datetime(2023, 1, 1))
447 corr = xs.pearson_r(gv_cut, omni_v_rs, dim="time", skipna=True)
448 corr1 = corr.item()
449 print(f"Correlation between gv and v_oscar_at: {corr1}")
450 info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
451 plt.text(0.15, 0.95, info, transform=ax1.transAxes, fontsize=14, fontweight="bold")
452 plt.legend()
453
454 ax2 = fig.add_subplot(2, 1, 2, projection=ccrs.PlateCarree())
455 dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
456     *TRACKS_REG[2:])).plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
457 decorate_axis(ax2, "", *TRACKS_REG, step=5)
458 ax2.grid()
459 ax2.plot(
460     lon_omni,
461     lat_omni,
462     c="r",
463     marker="o",
464     markersize=6,
465     markerfacecolor="red",
466     markeredgecolor="black",
467     markeredgewidth=2,
468 )
```

```
plt.text(lon_omni, lat_omni, f"{omni_id}", fontsize=10, color="k")
ax2.scatter(lons_track_self, lats_track_self, marker=".", color="c", s=4)
ax2.scatter(lons_track_other, lats_track_other, marker=".", color="c", s=4)
lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self, lon_coast_self,
                                   lat_coast_self, d)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(lonm, latm, s=track_number_self, fontsize=14, rotation=angle_self, color="w")
lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other, lon_coast_other,
                                   lat_coast_other, d)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(lonm, latm, s=track_number_other, fontsize=14, rotation=angle_other, color="w")

plt.show()
plt.savefig(
    f"omniVStrack_guv_pngs/{omni_id}_{sat}_{track_number_self}_{track_number_other}_trim.png"
)
plt.close("all")
ds_omni.close()
dse.close()
# plt.close("all")
#
# dse.close()
omni_id = "AD06"
sat = "S3A"
track_self = "324"
track_other = "195"

df_sel = filter_intersections(df_all, sat, track_self, track_other)
if df_sel.empty:
    raise ValueError(
        f"No intersections for sat={sat}, tracks=({track_self}, {track_other})"
    )

lon_omni, lat_omni = omni_d[omni_id]
ds_omni = xr.open_dataset(f"{omni_data_dir}/{omni_id}_uvs_z_xr_daily.nc",
                          decode_times=False)
print(ds_omni)
ds_omni = change_time(ds_omni, "time")
omni_u = ds_omni.ud.interp(depth=dpth)
omni_u = omni_u.rolling(time=10, center=True).mean()
omni_u = 0.01 * omni_u
omni_v = ds_omni.vd.interp(depth=dpth)
omni_v = omni_v.rolling(time=10, center=True).mean()
omni_v = 0.01 * omni_v
N = omni_v.count().item()
print(N)

df_1 = find_n_closest_points(df_sel, lon_omni, lat_omni, n=1)
row = df_1.iloc[0]
sat = row["sat"]
track_number_self = str(row["track_self_norm"])
track_number_other = str(row["track_other_norm"])
lons_inter1 = row["lons_inter"]
lats_inter1 = row["lats_inter"]
lon1 = lons_inter1
lat1 = lats_inter1
x_from_coast_self1 = row["x_from_coast_self"]
x_from_coast_other1 = row["x_from_coast_other"]
angle_acute = row["angle_acute"]
angle_obtuse = row["angle_obtuse"]
distance_km = row["distance_km"]
if abs(lat1) < 2:
    raise ValueError(f"Selected intersection near equator (lat={lat1:.2f}); skipped.")
print(
    sat,
    omni_id,
    track_number_self,
    track_number_other,
    lons_inter1,
    lats_inter1,
    x_from_coast_self1,
    x_from_coast_other1,
    angle_acute,
    angle_obtuse,
)
f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_self}_loess_{frac1}.nc"
f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_other}_loess_{frac1}.nc"
ds_self = xr.open_dataset(f_self, decode_times=False)
ds_self_smooth = xr.open_dataset(f_self_smooth)
sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
sla_self_smooth = ds_self_smooth.sla_smooth
ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
ds_other_smooth = xr.open_dataset(f_other_smooth)
sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
sla_other_smooth = ds_other_smooth.sla_smooth
lons_track_self = ds_self.lon.values
lats_track_self = ds_self.lat.values
lon_coast_self = lons_track_self[-1]
lat_coast_self = lats_track_self[-1]
lon_equat_self = lons_track_self[0]
lat_equat_self = lats_track_self[0]
slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
    lons_track_self[-1] - lons_track_self[0])
angle_self = np.rad2deg(np.arctan(slope_self))
lons_track_other = ds_other.lon.values
lats_track_other = ds_other.lat.values
lon_equat_other = lons_track_other[0]
lat_equat_other = lats_track_other[0]
lon_coast_other = lons_track_other[-1]
lat_coast_other = lats_track_other[-1]
slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                     lon_equat_other)
angle_other = np.rad2deg(np.arctan(slope_other))
gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
gc_other_at = gc_other.sel(x=x_from_coast_other1, method="nearest", drop=True)
gc_other_at = gc_other_at.interp(time=gc_self_at.time)
a2 = math.atan2(lat_coast_other - lat_equat_other, lon_coast_other - lon_equat_other)
a1 = math.atan2(lat_coast_self - lat_equat_self, lon_coast_self - lon_equat_self)

gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
gu_cut = gu
gv_cut = gv

```

```python
if distance_km > 30:
    raise ValueError(f"Nearest selected intersection too far from omni: {distance_km:.2f} km")

fig = plt.figure(figsize=(12, 8), layout="constrained")
ax1 = fig.add_subplot(2, 1, 1)
gu_cut.plot(ax=ax1, label="track zonal", color="r", linewidth=2)
tfreq = sats_tfreq[sat]
omni_v_rs = omni_v.resample(time=f"{tfreq}D").mean().interp(time=gv_cut.time)
omni_u.plot(ax=ax1, color="b", linewidth=2, label="omni u")
ax1.set_xlim(datetime(2016, 3, 1), datetime(2020, 9, 1))
corr = xs.pearson_r(gv_cut, omni_v_rs, dim="time", skipna=True)
corr1 = corr.item()
print(f"Correlation between gv and v_oscar_at: {corr1}")
info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
plt.text(0.15, 0.95, info, transform=ax1.transAxes, fontsize=14, fontweight="bold")
plt.legend()

ax2 = fig.add_subplot(2, 1, 2, projection=ccrs.PlateCarree())
dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
    *TRACKS_REG[2:])).plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=5)
ax2.grid()
ax2.plot(
    lon_omni,
    lat_omni,
    c="r",
    marker="o",
    markersize=6,
    markerfacecolor="red",
    markeredgecolor="black",
    markeredgewidth=2,
)
plt.text(lon_omni, lat_omni, f"{omni_id}", fontsize=10, color="k")
ax2.scatter(lons_track_self, lats_track_self, marker=".", color="c", s=4)
ax2.scatter(lons_track_other, lats_track_other, marker=".", color="c", s=4)
lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self, lon_coast_self,
                                   lat_coast_self, d)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(lonm, latm, s=track_number_self, fontsize=14, rotation=angle_self, color="w")
lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other, lon_coast_other,
                                   lat_coast_other, d)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(lonm, latm, s=track_number_other, fontsize=14, rotation=angle_other, color="w")

plt.show()
plt.savefig(
    f"omniVStrack_guv_pngs/{omni_id}_{sat}_{track_number_self}_{track_number_other}_trim.png"
)
plt.close("all")
ds_omni.close()
dse.close()
# plt.close("all")
#
# dse.close()
omni_id = "BD14"
sat = "S3B"
track_self = "94"
track_other = "51"

df_sel = filter_intersections(df_all, sat, track_self, track_other)
if df_sel.empty:
    raise ValueError(
        f"No intersections for sat={sat}, tracks=({track_self}, {track_other})"
    )

lon_omni, lat_omni = omni_d[omni_id]
ds_omni = xr.open_dataset(f"{omni_data_dir}/{omni_id}_uvs_z_xr_daily.nc",
                          decode_times=False)
print(ds_omni)
ds_omni = change_time(ds_omni, "time")
omni_u = ds_omni.ud.interp(depth=dpth)
omni_u = omni_u.rolling(time=10, center=True).mean()
omni_u = 0.01 * omni_u
omni_v = ds_omni.vd.interp(depth=dpth)
omni_v = omni_v.rolling(time=10, center=True).mean()
omni_v = 0.01 * omni_v
N = omni_v.count().item()
print(N)

df_1 = find_n_closest_points(df_sel, lon_omni, lat_omni, n=1)
row = df_1.iloc[0]
sat = row["sat"]
track_number_self = str(row["track_self_norm"])
track_number_other = str(row["track_other_norm"])
lons_inter1 = row["lons_inter"]
lats_inter1 = row["lats_inter"]
lon1 = lons_inter1
lat1 = lats_inter1
x_from_coast_self1 = row["x_from_coast_self"]
x_from_coast_other1 = row["x_from_coast_other"]
angle_acute = row["angle_acute"]
angle_obtuse = row["angle_obtuse"]
distance_km = row["distance_km"]
if abs(lat1) < 2:
    raise ValueError(f"Selected intersection near equator (lat={lat1:.2f}); skipped.")
print(
    sat,
    omni_id,
    track_number_self,
    track_number_other,
    lons_inter1,
    lats_inter1,
    x_from_coast_self1,
    x_from_coast_other1,
    angle_acute,
    angle_obtuse,
)
f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
f_self_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_self}_loess_{frac1}.nc"
f_other_smooth = f"../computed/sla_loess_{frac1}a/track_sla_along_{sat}_{track_number_other}_loess_{frac1}.nc"
ds_self = xr.open_dataset(f_self, decode_times=False)
ds_self_smooth = xr.open_dataset(f_self_smooth)
sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
sla_self_smooth = ds_self_smooth.sla_smooth
ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
ds_other_smooth = xr.open_dataset(f_other_smooth)
```

```
sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
sla_other_smooth = ds_other_smooth.sla_smooth
lons_track_self = ds_self.lon.values
lats_track_self = ds_self.lat.values
lon_coast_self = lons_track_self[-1]
lat_coast_self = lats_track_self[-1]
lon_equat_self = lons_track_self[0]
lat_equat_self = lats_track_self[0]
slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
    lons_track_self[-1] - lons_track_self[0])
angle_self = np.rad2deg(np.arctan(slope_self))
lons_track_other = ds_other.lon.values
lats_track_other = ds_other.lat.values
lon_equat_other = lons_track_other[0]
lat_equat_other = lats_track_other[0]
lon_coast_other = lons_track_other[-1]
lat_coast_other = lats_track_other[-1]
slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                     lon_equat_other)
angle_other = np.rad2deg(np.arctan(slope_other))
gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
gc_other_at = gc_other.sel(x=x_from_coast_other1, method="nearest", drop=True)
gc_other_at = gc_other_at.interp(time=gc_self_at.time)
a2 = math.atan2(lat_coast_other - lat_equat_other, lon_coast_other - lon_equat_other)
a1 = math.atan2(lat_coast_self - lat_equat_self, lon_coast_self - lon_equat_self)

gu, gv = geostrophic_components(gc_self_at, gc_other_at, a1, a2)
gu_cut = gu
gv_cut = gv

if distance_km > 30:
    raise ValueError(f"Nearest selected intersection too far from omni: {distance_km:.2f} km")

fig = plt.figure(figsize=(12, 8), layout="constrained")
ax1 = fig.add_subplot(2, 1, 1)
gu_cut.plot(ax=ax1, label="track zonal", color="r", linewidth=2)
tfreq = sats_tfreq[sat]
omni_v_rs = omni_v.resample(time=f"{tfreq}D").mean().interp(time=gv_cut.time)
omni_u.plot(ax=ax1, color="b", linewidth=2, label="omni u")
ax1.set_xlim(datetime(2016, 3, 1), datetime(2022, 4, 1))
corr = xs.pearson_r(gv_cut, omni_v_rs, dim="time", skipna=True)
corr1 = corr.item()
print(f"Correlation between gv and v_oscar_at: {corr1}")
info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
plt.text(0.15, 0.95, info, transform=ax1.transAxes, fontsize=14, fontweight="bold")
plt.legend()

ax2 = fig.add_subplot(2, 1, 2, projection=ccrs.PlateCarree())
dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
    *TRACKS_REG[2:])).plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=5)
ax2.grid()
ax2.plot(
    lon_omni,
    lat_omni,
    c="r",
    marker="o",
    markersize=6,
    markerfacecolor="red",
    markeredgecolor="black",
    markeredgewidth=2,
)
plt.text(lon_omni, lat_omni, f"{omni_id}", fontsize=10, color="k")
ax2.scatter(lons_track_self, lats_track_self, marker=".", color="c", s=4)
ax2.scatter(lons_track_other, lats_track_other, marker=".", color="c", s=4)
lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self, lon_coast_self,
                                   lat_coast_self, d)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(lonm, latm, s=track_number_self, fontsize=14, rotation=angle_self, color="w")
lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other, lon_coast_other,
                                   lat_coast_other, d)
if is_within_region(lonm, latm, *TRACKS_REG):
    plt.text(lonm, latm, s=track_number_other, fontsize=14, rotation=angle_other, color="w")

plt.show()
plt.savefig(
    f"omniVStrack_guv_pngs/{omni_id}_{sat}_{track_number_self}_{track_number_other}_trim.png"
)
plt.close("all")
ds_omni.close()
dse.close()
# plt.close("all")
#
# dse.close()

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import sys

import math
import numpy as np


def geostrophic_components_from_slopes(gc1, gc2, slope1, slope2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Convert slopes to azimuths (angles from east axis)
    a1 = math.atan(slope1)  # azimuth of track 1 in radians
    a2 = math.atan(slope2)  # azimuth of track 2 in radians
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Check for parallel tracks (same slope)
    if abs(slope1 - slope2) < 1e-10:
        raise ValueError(f"Tracks are parallel (slopes: {slope1:.6f}, {slope2:.6f}). "
                         "Cannot resolve both velocity components.")
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
                         "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    return u, v


def haversine_distance(lon1, lat1, lon2, lat2):
    """Calculate great-circle distance between (lon1, lat1)
    and arrays (lon2, lat2) in kilometers."""
    R = 6371  # Radius of Earth in km
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlon = lon2 - lon1
    dlat = lat2 - lat1
    a = (np.sin(dlat / 2)**2 +
         np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2)**2)
    return 2 * R * np.arcsin(np.sqrt(a))


def find_n_closest_points(df, lon0, lat0, n=4):
    """Return the n rows in df closest to (lon0, lat0) based on
    lons_inter/lats_inter."""
    distances = haversine_distance(lon0, lat0, df["lons_inter"].values,
                                   df["lats_inter"].values)
    df = df.copy()
    df["distance_km"] = distances
    return df.nsmallest(n, "distance_km")


cmap1 = cmo.cm.diff

sat = "TP+J1+J2+J3+S6A"
d = 1.5
height, width = 9, 9
oscar_dir = "/home/srinivasu/allData/oscar_0.25/"
ds_oscar = xr.open_dataset(f"{oscar_dir}/oscar_0.25_u_full_xr_concat.nc")
print(ds_oscar)
u_oscar = ds_oscar.u
# v_oscar = ds_oscar.v

dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
omni_id = "BD09"
omni_data_dir = "/home/srinivasu/allData/MB_cleaned"
lon_omni, lat_omni = omni_d[omni_id]
dpth = 15.

ds_omni = xr.open_dataset(f"{omni_data_dir}/{omni_id}_uvs_z_xr_daily.nc", decode_times=False)
print(ds_omni)
ds_omni = change_time(ds_omni, "time")
omni_v = ds_omni.vd.interp(depth=dpth)
# make 10 average of omni_v
omni_v = omni_v.rolling(time=10, center=True).mean()
omni_v = 0.01 * omni_v
omni_u = ds_omni.ud.interp(depth=dpth)
# make 10 average of omni_v
omni_u = omni_u.rolling(time=10, center=True).mean()
omni_u = 0.01 * omni_u
N = omni_v.count().item()
print(N)

df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
df_4 = find_n_closest_points(df, lon_omni, lat_omni, n=4)
```

```
print(omni_id)
print(df_4)

# sys.exit()
track_self = df_4["track_self"].apply(lambda x: str(x)).values
track_other = df_4["track_other"].apply(lambda x: str(x)).values
lons_inter = df_4["lons_inter"].values
lats_inter = df_4["lats_inter"].values
x_from_coast_self = df_4["x_from_coast_self"].values
x_from_coast_other = df_4["x_from_coast_other"].values
angle_acute = df_4["angle_acute"].values
angle_obtuse = df_4["angle_obtuse"].values

for i in range(len(df_4)):
    track_number_self = track_self[i]
    track_number_other = track_other[i]
    f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_self}_loess_0.2.nc"
    f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_other}_loess_0.2.nc"
    lon1 = lons_inter[i]
    lat1 = lats_inter[i]
    x_from_coast_self1 = x_from_coast_self[i]
    x_from_coast_other1 = x_from_coast_other[i]
    angle_acute1 = angle_acute[i]
    angle_obtuse1 = angle_obtuse[i]
    ds_self = xr.open_dataset(f_self, decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other - lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    # u_oscar_at = u_oscar.sel(lon=lon1, lat=lat1, method="nearest")
    u_oscar_at = u_oscar.sel(lon=lon1, lat=lat1, method="nearest")
    # v_oscar_at = v_oscar.sel(lon=lon1, lat=lat1, method="nearest")
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    # if track_path_self.intersects(track_path_other):
    gc_self_at = gc_self.sel(x=x_from_coast_self1,
                             method="nearest",
                             drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    gu, gv = geostrophic_components_from_slopes(gc_self_at, gc_other_at, slope_self, slope_other)
    # other_times.append((track_number_other, abs(dist_time)))
    fig1, ax1 = plt.subplots(1,
                             1,
                             figsize=(width, height),
                             layout="constrained")
    # gc_self_at.plot(ax=ax1, label=f"pass {track_number_self}")
    # gc_other_at.plot(ax=ax1, label=f"pass {track_number_other}")
    # gv.plot(ax=ax1, label=f"pass {track_number_other}", color="r", linewidth=2)
    # omni_v.plot(ax=ax1, color="b", linewidth=2, label="omni")
    # v_oscar_at.plot(ax=ax1, color="k", linewidth=2, label="oscar")
    gu.plot(ax=ax1, label=f"pass {track_number_other}", color="r", linewidth=2)
    omni_u.plot(ax=ax1, color="b", linewidth=2, label="omni")
    u_oscar_at.plot(ax=ax1, color="k", linewidth=2, label="oscar")
    # text coordinate of intersection point
    plt.text(
        0.15,
        0.95,
        f"intersection point ({lon1:.2f}, {lat1:.2f})",
        transform=ax1.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    plt.legend()
    fig2, ax2 = plt.subplots(
        1,
        1,
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(width, height),
        layout="constrained",
    )
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG)
    ax2.grid()
    ax2.plot(
        lon_omni,
        lat_omni,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor='red',
        markeredgecolor='black',
        markeredgewidth=2
    )
    # plot plat form code of omni at lon_omni, lat_omni
    plt.text(lon_omni, lat_omni, f"{omni_id}", fontsize=10, color="k")
    ax2.scatter(
```

```
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="w",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="w",
        )
    plt.show()
    plt.close("all")
    break

dse.close()

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *


def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    return u, v


def haversine_distance(lon1, lat1, lon2, lat2):
    """Calculate great-circle distance between (lon1, lat1)
    and arrays (lon2, lat2) in kilometers."""
    R = 6371  # Radius of Earth in km
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlon = lon2 - lon1
    dlat = lat2 - lat1
    a = (np.sin(dlat / 2)**2 +
         np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2)**2)
    return 2 * R * np.arcsin(np.sqrt(a))


def find_n_closest_points(df, lon0, lat0, n=4):
    """Return the n rows in df closest to (lon0, lat0) based on
    lons_inter/lats_inter."""
    distances = haversine_distance(lon0, lat0, df["lons_inter"].values,
                                   df["lats_inter"].values)
    df = df.copy()
    df["distance_km"] = distances
    return df.nsmallest(n, "distance_km")


cmap1 = cmo.cm.diff
d = 1.5
dpth = 15.0
height, width = 9, 9
oscar_dir = "/home/srinivasu/allData/oscar_0.25/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
rama_data_dir = "/home/srinivasu/allData/rama/cur/"
nth = 0

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

for rama_id in rama_d.keys():
    lon_rama, lat_rama = rama_d[rama_id]
    if abs(lat_rama) < 2:
        continue
    ds_rama = xr.open_dataset(f"{rama_data_dir}/cur{rama_id}_dy.cdf")
    print(ds_rama)
    rama_u = 0.01 * ds_rama.U_320.isel(depth=0, drop=True)
    rama_v = 0.01 * ds_rama.V_321.isel(depth=0, drop=True)
    N = rama_v.count().item()
    print(N)
    df_4 = find_n_closest_points(df_all, lon_rama, lat_rama, n=4)
    # print(rama_id, lon_rama, lat_rama, "\n", df_4.head(1))
    sat = df_4["sat"].values[nth]
    track_number_self = df_4["track_self"].apply(lambda x: str(x)).values[nth]
    track_number_other = df_4["track_other"].apply(lambda x: str(x)).values[nth]
    lons_inter1 = df_4["lons_inter"].values[nth]
    lats_inter1 = df_4["lats_inter"].values[nth]
    distance_km = df_4["distance_km"].values[nth]
    lon1 = lons_inter1
    lat1 = lats_inter1
    if abs(lat1) < 2:
        continue
    x_from_coast_self1 = df_4["x_from_coast_self"].values[nth]
    x_from_coast_other1 = df_4["x_from_coast_other"].values[nth]
    angle_acute = df_4["angle_acute"].values[nth]
```

```
    angle_obtuse = df_4["angle_obtuse"].values[nth]
    print(
        sat,
        rama_id,
        track_number_self,
        track_number_other,
        lons_inter1,
        lats_inter1,
        x_from_coast_self1,
        x_from_coast_other1,
        angle_acute,
        angle_obtuse,
    )
    f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_self}_loess_0.2.nc"
    f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat}_{track_number_other}_loess_0.2.nc"
    ds_self = xr.open_dataset(f_self, decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    # u_oscar_at = u_oscar.sel(lon=lon1, lat=lat1, method="nearest")
    # v_oscar_at = v_oscar.sel(lon=lon1, lat=lat1, method="nearest")
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    # if track_path_self.intersects(track_path_other):
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    # Better approach:
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)

    gu, gv = geostrophic_components_from_a(gc_self_at, gc_other_at, a1, a2)
    # other_times.append((track_number_other, abs(dist_time)))
    fig1, ax1 = plt.subplots(1, 1, figsize=(10, 6), layout="constrained")
    gu_cut = -1 * gu.interp(time=rama_u.time)
    gv_cut = gv.interp(time=rama_v.time)
    # gu_cut.plot(ax=ax1, label=f"track zonal", color="r", linewidth=2)
    # rama_u.plot(ax=ax1, color="b", linewidth=2, label="rama")
    # u_oscar_at.plot(ax=ax1, color="b", linewidth=2, label="oscar u")
    # corr = xs.pearson_r(gu_cut, rama_u, dim="time", skipna=True)
    gv_cut.plot(ax=ax1, label=f"track zonal", color="r", linewidth=2)
    tfreq = sats_tfreq[sat]
    rama_v = rama_v.resample(time=f"{tfreq}D").mean()
    rama_v = rama_v.interp(time=gv_cut.time)
    rama_v.plot(ax=ax1, color="b", linewidth=2, label="rama")
    # v_oscar_at.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
    corr = xs.pearson_r(gv_cut, rama_v, dim="time", skipna=True)
    corr1 = corr.item()
    # print(f"Correlation between gu and u_oscar_at: {corr1}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km}",
    print(f"Correlation between gv and v_oscar_at: {corr1}")
    plt.text(
        0.15,
        0.95,
        f"{info}",
        transform=ax1.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    plt.legend()
    fig2, ax2 = plt.subplots(
        1,
        1,
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(width, height),
        layout="constrained",
    )
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG)
    ax2.grid()
    ax2.plot(
        lon_rama,
        lat_rama,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor="red",
        markeredgecolor="black",
        markeredgewidth=2,
    )
    # plot plat form code of rama at lon_rama, lat_rama
    plt.text(lon_rama, lat_rama, f"{rama_id}", fontsize=10, color="k")
    ax2.scatter(
```

```
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="w",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="w",
        )
    plt.show()
    plt.close("all")

dse.close()

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import sys

cmap1 = cmo.cm.diff
sat = "S3A"
sat = "ERS1+ERS2+ENV+SRL"
sat = "TP+J1+J2+J3+S6A"
rama_id = "0n90e"
rama_id = "4n90e"
rama_id = "15n90e"
lon_rama, lat_rama = rama_d[rama_id]
d = 1.5
height, width = 9, 9


def haversine_distance(lon1, lat1, lon2, lat2):
    """Calculate great-circle distance between (lon1, lat1)
    and arrays (lon2, lat2) in kilometers."""
    R = 6371  # Radius of Earth in km
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlon = lon2 - lon1
    dlat = lat2 - lat1
    a = (np.sin(dlat / 2)**2 +
         np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2)**2)
    return 2 * R * np.arcsin(np.sqrt(a))


def find_n_closest_points(df, lon0, lat0, n=4):
    """Return the n rows in df closest to (lon0, lat0) based on
    lons_inter/lats_inter."""
    distances = haversine_distance(lon0, lat0, df["lons_inter"].values,
                                   df["lats_inter"].values)
    df = df.copy()
    df["distance_km"] = distances
    return df.nsmallest(n, "distance_km")


df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
df_4 = find_n_closest_points(df, lon_rama, lat_rama, n=4)
print(df_4)

# sys.exit()
track_self = df_4["track_self"].apply(lambda x: str(x)).values
track_other = df_4["track_other"].apply(lambda x: str(x)).values
lons_inter = df_4["lons_inter"].values
lats_inter = df_4["lats_inter"].values
x_from_coast_self = df_4["x_from_coast_self"].values
x_from_coast_other = df_4["x_from_coast_other"].values
angle_acute = df_4["angle_acute"].values
angle_obtuse = df_4["angle_obtuse"].values
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

for i in range(len(df_4)):
    track_number_self = track_self[i]
    track_number_other = track_other[i]
    f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
    lon1 = lons_inter[i]
    lat1 = lats_inter[i]
    x_from_coast_self1 = x_from_coast_self[i]
    x_from_coast_other1 = x_from_coast_other[i]
    angle_acute1 = angle_acute[i]
    angle_obtuse1 = angle_obtuse[i]
    ds_self = xr.open_dataset(f_self, decode_times=False)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    ds_other = xr.open_dataset(f_other, engine="netcdf4", decode_times=False)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other - lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    # if track_path_self.intersects(track_path_other):
    sla_self_at = sla_self.sel(x=1000*x_from_coast_self1,
                               method="nearest",
                               drop=True)
    sla_other_at = sla_other.sel(x=1000*x_from_coast_other1,
                                 method="nearest",
                                 drop=True)
    sla_other_at = sla_other_at.interp(time=sla_self_at.time)
    # other_times.append((track_number_other, abs(dist_time)))
    fig, ax1 = plt.subplots(1,
                            1,
                            figsize=(width, height),
                            layout="constrained")
    sla_self_at.plot(ax=ax1, label=f"pass {track_number_self}")
    sla_other_at.plot(ax=ax1, label=f"pass {track_number_other}")
    # text coordinate of intersection point
    plt.text(
        0.15,
        0.95,
        f"intersection point ({lon1:.2f}, {lat1:.2f})",
        transform=ax1.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    plt.legend()
    fig2, ax2 = plt.subplots(
        1,
```

```
        1,
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(width, height),
        layout="constrained",
    )
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap1)
    decorate_axis(ax2, "", *TRACKS_REG)
    ax2.grid()
    ax2.plot(
        lon_rama,
        lat_rama,
        c="r",
        marker="o",
        markersize=6,
        markerfacecolor='red',
        markeredgecolor='black',
        markeredgewidth=2
    )
    # plot plat form code of rama at lon_rama, lat_rama
    plt.text(lon_rama, lat_rama, f"{rama_id}", fontsize=10, color="k")
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="w",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other, lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="w",
        )
    plt.show()
    plt.close("all")
    break

dse.close()

```

```
./lib/
```

```
4 file(s)
```

```python
def compute_geostrophy_v(ds, sla_smooth, current="meridional"):
    lons_track_rev = ds.lon.values[::-1]
    lats_track_rev = ds.lat.values[::-1]
    if len(lons_track_rev) == 0:
        return
    # Compute the angle using arctan2 for correct sign and quadrant
    delta_lat = lats_track_rev[-1] - lats_track_rev[0]
    delta_lon = lons_track_rev[-1] - lons_track_rev[0]
    angle_r = np.arctan2(delta_lat, delta_lon)  # Angle in radians

    # Compute Coriolis parameter and gravity
    f = cor_p(lats_track_rev).values
    f = np.where(np.abs(lats_track_rev) > 2, f, np.nan)  # Mask equatorial band
    g = grav(lats_track_rev).values
    ratio = (g / f)[:, np.newaxis]

    # Gradient of SLA along the track (s-direction)
    dhbyds = sla_smooth.differentiate("x")  # "x" is the distance along the track

    # Geostrophic current magnitude along the track
    gc = ratio * dhbyds

    # Compute zonal and meridional components using correct trigonometry
    if current == "zonal":
        u = -gc * np.sin(angle_r)
        return u
    elif current == "meridional":
        # v = gc * np.cos(angle_r)
        u = -gc * np.sin(angle_r)
        v = (gc + u*np.sin(angle_r))/np.cos(angle_r)
        return v
    elif current == "speed":
        u = gc * np.sin(angle_r)
        v = -gc * np.cos(angle_r)
        speed = np.sqrt(u**2 + v**2)
        return speed


def compute_geostrophy_atan2(ds, sla_smooth, current="zonal"):
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = ds.lon.values[::-1]
    lats_track_rev = ds.lat.values[::-1]
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    lon_equator = lons_track[0]  # this on equator
    lat_equator = lats_track[0]  # this on equator
    if len(lons_track) == 0:
        return
    angle_r2 = np.arctan2(lat_coast - lat_equator, lon_coast - lon_equator)
    angle2 = np.rad2deg(angle_r2)
    f1 = cor_p(lats_track_rev)
    f1 = f1.where(lats_track_rev > 2, np.nan)
    # 2s-2n band equatorial band is masked according to Sudre 2013
    g1 = grav(lats_track_rev)  # ms-1
    ratio = g1 / f1
    ratio1 = ratio.values
    ratio2 = ratio1[:, np.newaxis]
    dhbydx = sla_smooth.differentiate("x")
    gc = -1 * ratio2 * dhbydx
    if current == "zonal":
        gu = -1 * gc * np.sin(angle_r2)
        return gu
    elif current == "meridional":
        gv = gc * np.cos(angle_r2)
        return gv
    elif current == "speed":
        gu = -1 * gc * np.sin(angle_r2)
        gv = gc * np.cos(angle_r2)
        gs = (gu**2 + gv**2) ** 0.5
        return gs
    elif current == "gc":
        gc = ratio2 * dhbydx
        return gc


def compute_geostrophy(ds, sla_smooth, current="zonal"):
    lons_track_rev = ds.lon.values[::-1]
    lats_track_rev = ds.lat.values[::-1]
    if len(lons_track_rev) == 0:
        return
    m = (lats_track_rev[-1] - lats_track_rev[0]) / (
        lons_track_rev[-1] - lons_track_rev[0]
    )
    angle_r = np.arctan(m)
    angle = np.rad2deg(angle_r)
    f1 = cor_p(lats_track_rev)
    f2 = f1.values
    f3 = np.where(lats_track_rev > 2, f2, np.nan)
    # 2s-2n band equatorial band is masked according to Sudre 2013
    g = grav(lats_track_rev)  # ms-1
    g1 = g.values
    ratio = g1 / f3
    ratio1 = ratio[:, np.newaxis]
    dhbydx = sla_smooth.differentiate("x")
    gc = -1 * ratio1 * dhbydx
    if current == "zonal":
        gu = -1 * gc * np.cos(np.deg2rad(90.0 - abs(angle)))
        return gu
    elif current == "meridional":
        gu = gc * np.sin(np.deg2rad(90.0 - abs(angle)))
        return gu
    elif current == "speed":
        guu = -1 * gc * np.cos(np.deg2rad(90.0 - abs(angle)))
        guv = gc * np.sin(np.deg2rad(90.0 - abs(angle)))
        gu = (guu**2 + guv**2) ** 0.5
        return gu


def geostrophic_current_components_slopes(g1, g2, m1, m2):
    """
    Calculates the zonal (u) and meridional (v) components and the speed of
    the geostrophic current.
    Parameters:
    - g1: Cross-track geostrophic current along track T1 (float)
    - g2: Cross-track geostrophic current along track T2 (float)
    - m1: Slope of track T1 (float)
```

```
    - m2: Slope of track T2 (float)
    Returns:
    - u: Zonal component of the geostrophic current (float)
    - v: Meridional component of the geostrophic current (float)
    - speed: Speed (magnitude) of the geostrophic current (float)
    """
    # Convert slopes to angles in radians
    theta1 = math.atan(m1)
    theta2 = math.atan(m2)
    print(theta1, theta2)
    sin_theta1 = math.sin(theta1)
    cos_theta1 = math.cos(theta1)
    sin_theta2 = math.sin(theta2)
    cos_theta2 = math.cos(theta2)
    sin_theta_diff = math.sin(theta2 - theta1)
    if sin_theta_diff == 0:
        raise ValueError("The tracks are parallel or the angle difference is \
                         zero, cannot compute components.")
    u = (cos_theta2 * g1 - cos_theta1 * g2.interp(time=g1.time)) / sin_theta_diff
    v = (sin_theta2 * g1 - sin_theta1 * g2.interp(time=g1.time)) / sin_theta_diff
    speed = (u**2 + v**2)**0.5
    u1 = u
    v1 = v
    speed1 = speed
    return u1, v1, speed1


def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    u = -1 * u
    return u, v

```

lib/namesm.py

```
from datetime import datetime

zone = "NINDIAN"
version = "v2.2"
NIO = (40.0, 110.0, -5.0, 30.0)
TRACKS_REG = (43.0, 99, 0, 25.4)
WBOB = (78.0, 87.0, 8.0, 21.0)
alldata_dir = "/home/srinivasu/allData/"
topo_dir = "/home/srinivasu/allData/topo/"
data_dir = "/home/srinivasu/xtrackm/data/"

chunk1 = "1_5000"
chunk2 = "5001_10000"
chunk3 = "10001_15000"
chunk4 = "15001_current"
chunks = ["1_5000", "5001_10000", "10001_15000", "15001_current"]

months = {
    1: "Jan",
    2: "Feb",
    3: "Mar",
    4: "Apr",
    5: "May",
    6: "Jun",
    7: "Jul",
    8: "Aug",
    9: "Sep",
    10: "Oct",
    11: "Nov",
    12: "Dec",
}

coastal_ids = [
    "BNPD_0",
    "BNPD_A",
    "BNPD_B",
    "BNPD_C",
    "BODD_0",
    "BODD_A",
    "BODD_B",
    "BOGD_0",
    "BOGD_A",
    "BOGD_B",
    "BOGD_C",
    "BOGS_0",
    "BOKD_0",
    "BOKD_A",
    "BOKD_B",
    "BOKD_C",
    "BOKS_0",
    "BOND_A",
    "BOND_B",
    "BOPD_0",
    "BOPD_A",
    "BOPD_B",
    "BOPD_C",
    "BOPS_0",
    "BORD_0",
    "BORS_0",
    "BOVD_0",
    "BOVD_A",
    "BOVD_B",
    "BOVD_C",
    "BOVS_0",
    "BOVS_A",
    "BOVS_B",
    "BSVD_0",
    "BSVS_0",
    "Eq7700_A",
    "Eq7700_B",
    "Eq8300_A",
    "Eq8300_B",
    "Eq8300_D",
    "Eq8300_M",
    "Eq8300_U",
    "Eq8301_D",
    "Eq8301_M",
    "Eq8301_U",
    "KKD_0",
    "KKD_A",
    "KKD_B",
    "KKD_C",
    "KKS_0",
    "OBD_A",
    "OBD_B",
    "OBD_C",
    "OBS_0",
    "OBV_0",
    "OGD_0",
    "OGD_A",
    "OGD_B",
    "OGD_C",
    "OGD_D",
    "OGS_0",
    "OGV_0",
    "OGV_A",
    "OJD_A",
    "OJD_B",
    "OJD_C",
    "OKD_0",
    "OKD_A",
    "OKD_B",
    "OKD_C",
    "OKS_0",
    "OMD_0",
    "OMD_A",
    "OMD_B",
    "OMD_C",
    "OMD_D",
    "OMS_0",
    "OOD_A",
    "OOD_B",
    "OOD_C",
    "OUD_A",
    "OUD_B",
    "OUD_C",
    "SSMD_0",
```

```
]

coastal_lons = [
    88.92157936096191,
    88.94196014404297,
    88.94196014404297,
    88.94196014404297,
    88.32640075683594,
    88.32622528076172,
    88.32622528076172,
    85.6750005086263,
    85.78978983561198,
    85.78978983561198,
    85.78963851928711,
    85.4050838470459,
    82.4026596069336,
    82.2062161763509,
    82.20604269845145,
    82.2071164449056,
    82.31258114901456,
    80.41385173797607,
    80.4137020111084,
    80.20458221435547,
    80.20787896050348,
    80.20787896050348,
    80.2096176147461,
    80.13074048360188,
    80.55229949951172,
    80.33589935302734,
    84.04644966125488,
    84.04294259207589,
    84.04294259207589,
    84.03393300374348,
    83.98318767547607,
    83.98479843139648,
    83.98479843139648,
    82.98200225830078,
    82.86948267618816,
    77.36289978027344,
    77.36289978027344,
    83.00309753417969,
    83.00309753417969,
    83.00350189208984,
    83.00350189208984,
    83.00350189208984,
    83.0,
    83.0,
    83.0,
    77.39992995489212,
    77.39408547537667,
    77.393798828125,
    77.39197998046875,
    77.44730377197266,
    73.24749755859375,
    73.24749755859375,
    73.24749755859375,
    73.40830103556316,
    72.94619750976562,
    72.7425298055013,
    73.92990944602273,
    73.92989141290838,
    73.92984563654119,
    72.74420166015625,
    73.0131103515625,
    72.76448604038784,
    72.7157974243164,
    71.12096659342448,
    71.12096659342448,
    71.12096659342448,
    75.38617515563965,
    75.41748725043402,
    75.4151611328125,
    75.41189793178013,
    75.96725273132324,
    69.23359870910645,
    69.23651218414307,
    69.23651218414307,
    69.23651218414307,
    69.23789978027344,
    69.37949752807617,
    67.47264862060547,
    67.47264862060547,
    67.47264862060547,
    74.03930286296362,
    74.03943125406902,
    74.03943125406902,
    79.18406677246094,
]

coastal_lats = [
    19.056999921798706,
    19.03734016418457,
    19.03734016418457,
    19.03734016418457,
    19.97599983215332,
    19.975825309753418,
    19.975825309753418,
    19.206000010172527,
    19.394988801744248,
    19.394988801744248,
    19.3946750164032,
    19.159247970581056,
    16.28001136779785,
    16.118550300598145,
    16.116471699305944,
    16.11661688486735,
    16.260869112881746,
    14.496724605560303,
    14.496549606323242,
    12.015586535135904,
    12.012822151184082,
    12.012822151184082,
    12.01484982172648,
    12.021641731262207,
    14.996999740600586,
    15.009300231933594,
    17.75165033340454,
```

```
    17.752486092703684,
    17.752486092703684,
    17.748467127482098,
    17.80720019340515,
    17.804474353790283,
    17.804474353790283,
    16.836999893188477,
    17.085318247477215,
    0.0001999999994757503,
    0.0001999999994757503,
    0.0001999999994757503,
    0.0001999999994757503,
    0.0003999999989515007,
    0.0003999999989515007,
    0.0003999999989515007,
    1.0,
    1.0,
    1.0,
    6.961243084498814,
    6.960771424429757,
    6.961149990558624,
    6.95989990234375,
    6.997300148010254,
    13.608624935150146,
    13.608624935150146,
    13.608624935150146,
    13.873200098673502,
    15.21500015258789,
    15.161111768086752,
    15.54947246204723,
    15.54947246204723,
    15.54947246204723,
    15.166199684143066,
    15.200445938110352,
    15.168671335492816,
    15.206199645996094,
    17.525333404541016,
    17.525333404541016,
    17.525333404541016,
    9.280614614486694,
    9.049299875895182,
    9.04782497882843,
    9.04809992653983,
    9.002162456512451,
    19.998074531555176,
    20.007662057876587,
    20.007662057876587,
    20.007662057876587,
    20.010199546813965,
    19.956000328063965,
    22.262399673461914,
    22.262399673461914,
    22.262399673461914,
    12.500880852486324,
    12.50023365020752,
    12.50023365020752,
    8.804766654968262,
]

coastal_d = {key: (val2, val3) for key, val2, val3 in
             zip(coastal_ids, coastal_lons, coastal_lats)}

drifter_col_names = [
    "id",
    "mm",
    "dd",
    "yy",
    "lat",
    "lon",
    "temp",
    "ve",
    "vn",
    "spd",
    "var_lat",
    "var_lon",
    "var_tmp",
]

radar_extents = {
    "OR_RADAR" : (84.5, 87.0, 17.5, 20.0),
    "OR_BLOCK" : (82.0, 89.0, 15.0, 22.5),
    "AP_RADAR" : (80.7, 83.2, 14.3, 16.5),
    "AP_BLOCK" : (78.7, 84.2, 13.3, 17.5),
    "TN_RADAR" : (79.6, 81.8, 10.5, 12.9),
    "TN_BLOCK" : (78.0, 83.0, 8.0, 14.0)
}

# sats = ("GFO", "TP+J1+J2+J3", "S3A", "TPN+J1N+J2N", "ERS1+ERS2+ENV+SRL", "HY2")
sats_new = ("GFO", "TP+J1+J2+J3+S6A", "S3A", "S3B", "TPN+J1N+J2N+J3N", "ERS1+ERS2+ENV+SRL", "HY2A", "HY2B")

# radar_times = {TN: ("2008-03-14T00:00:00.000000000","2021-12-12T00:00:00.000000000"),
#     AP: ("2010-05-23T00:00:00.000000000","2018-12-31T23:00:00.000000000"),
#     OR: ("2009-12-04T00:00:00.000000000","2018-12-31T23:00:00.000000000")
# }

radar_times = {
    "TN": (
        datetime.strptime("2008-03-14T00:00:00.000000", "%Y-%m-%dT%H:%M:%S.%f"),
        datetime.strptime("2021-12-12T00:00:00.000000", "%Y-%m-%dT%H:%M:%S.%f"),
    ),
    "AP": (
        datetime.strptime("2010-05-23T00:00:00.000000", "%Y-%m-%dT%H:%M:%S.%f"),
        datetime.strptime("2018-12-31T23:00:00.000000", "%Y-%m-%dT%H:%M:%S.%f"),
    ),
    "OR": (
        datetime.strptime("2009-12-04T00:00:00.000000", "%Y-%m-%dT%H:%M:%S.%f"),
        datetime.strptime("2018-12-31T23:00:00.000000", "%Y-%m-%dT%H:%M:%S.%f"),
    ),
}

sats_tfreq = {
    "GFO": 17,
    "TP+J1+J2+J3+S6A": 10,
    "S3A": 27,
    "S3B": 27,
    "TPN+J1N+J2N+J3N": 10,
    "ERS1+ERS2+ENV+SRL": 35,
```

```
    "HY2A": 14,
    "HY2B": 14,
}

omni_d_str = {
    "AD06": "18.5n67.5e",
    "AD07": "14.9n69e",
    "AD08": "12.1n68.6e",
    "AD09": "8.2n73.3e",
    "AD10": "10.3n72.6e",
    "BD08": "17.8n89.2e",
    "BD09": "17.5n89.1e",
    "BD10": "16.3n87.9e",
    "BD11": "13.5n84.2e",
    "BD12": "10.5n94.1e",
    "BD13": "14n87e",
    "BD14": "6.3n88e",
}

omni_d = {
    "AD06": (67.4510948980849, 18.49199304280638),
    "AD07": (68.98244705046227, 14.923452796575168),
    "AD08": (68.63840165461586, 12.056781707039407),
    "AD09": (73.27738246580012, 8.15979102792709),
    "AD10": (72.57509770842432, 10.299223431375093),
    "BD08": (89.18693587407336, 17.762038953423957),
    "BD09": (89.10978333261187, 17.47161247600986),
    "BD10": (87.94381031303271, 16.33518105176612),
    "BD11": (84.17075906265661, 13.525793001733966),
    "BD12": (94.0689868393517, 10.513312130448263),
    "BD13": (86.99098270846096, 13.988154407967864),
    "BD14": (88.04982448269273, 6.260750665306668),
}

cb_d = {
    "CB01": (92.59, 11.59),
    "CB06": (80.07, 13.07),
}

omni_buoys = (
    "AD06",
    "AD07",
    "AD08",
    "AD09",
    "AD10",
    "BD08",
    "BD09",
    "BD10",
    "BD11",
    "BD12",
    "BD13",
    "BD14",
)

lons_omni = (
    67.4510948980849,
    68.98244705046227,
    68.63840165461586,
    73.27738246580012,
    72.57509770842432,
    89.18693587407336,
    89.10978333261187,
    87.94381031303271,
    84.17075906265661,
    94.0689868393517,
    86.99098270846096,
    88.04982448269273,
)

lats_omni = (
    18.49199304280638,
    14.923452796575168,
    12.056781707039407,
    8.15979102792709,
    10.299223431375093,
    17.762038953423957,
    17.47161247600986,
    16.33518105176612,
    13.525793001733966,
    10.513312130448263,
    13.988154407967864,
    6.260750665306668,
)

rama_d = {
    "0n67e": (67.0, 0.0),
    "0n80.5e": (80.5, 0.0),
    "0n90e": (90.0, 0.0),
    "1.5n67e": (67.0, 1.5),
    "1.5s67e": (67.0, -1.5),
    "1.5s80.5e": (80.5, -1.5),
    "12n90e": (90.0, 12.0),
    "12s55e": (55.0, -12.0),
    "12s65e": (65.0, -12.0),
    "12s67e": (67.0, -12.0),
    "12s80.5e": (80.5, -12.0),
    "12s93e": (93.0, -12.0),
    "15n65e": (65.0, 15.0),
    "15n90e": (90.0, 15.0),
    "4n67e": (67.0, 3.5),
    "4n90e": (90.0, 3.5),
    "4s57e": (57.0, -4.0),
    "4s65e": (65.0, -4.0),
    "4s67e": (67.0, -4.0),
    "4s80.5e": (80.5, -4.0),
    "5s95e": (95.0, -5.0),
    "8n67e": (67.0, 8.0),
    "8n90e": (90.0, 8.0),
    "8s55e": (55.0, -8.0),
    "8s65e": (65.0, -8.0),
    "8s67e": (67.0, -8.0),
    "8s80.5e": (80.5, -8.0),
    "8s95e": (95.0, -8.0),
}

rama_buoys = (
    "0n67e",
```

```
    "0n80.5e",
    "0n90e",
    "1.5n67e",
    "1.5s67e",
    "1.5s80.5e",
    "12n90e",
    "12s55e",
    "12s65e",
    "12s67e",
    "12s80.5e",
    "12s93e",
    "15n65e",
    "15n90e",
    "4n67e",
    "4n90e",
    "4s57e",
    "4s65e",
    "4s67e",
    "4s80.5e",
    "5s95e",
    "8n67e",
    "8n90e",
    "8s55e",
    "8s65e",
    "8s67e",
    "8s80.5e",
    "8s95e",
)


class Constants:
    rho_air = 1.204  # kg/m3
    rho_sea = 1029  # kg/m3
    cd = 0.0015
    # Citation: Talley L.D., Pickard G.L., Emery W.J., Swift J.H., 2011. Descriptive Physical Oceanography: An Introduction (Sixth Edition), Elsevier, Boston, 560 pp.
    Av = 0.05  # m2/sec eddy viscosity see page  198
    D = 30  # typical ekman layer depth

```

```python
import geopandas as gpd
import cartopy.io.shapereader as shpreader
import numpy as np
from shapely.geometry import LineString, MultiLineString, Point, box
from scipy.ndimage import gaussian_filter1d
from scipy.signal import savgol_filter

_COAST_CACHE = {}


def apply_to_geom(geom, func):
    if isinstance(geom, LineString):
        return func(geom)
    elif isinstance(geom, MultiLineString):
        return MultiLineString([func(g) for g in geom.geoms])
    else:
        return geom


def gaussian_smooth(line, sigma=10):
    coords = np.array(line.coords)
    if len(coords) < 7:
        return line
    x = gaussian_filter1d(coords[:, 0], sigma=sigma, mode="nearest")
    y = gaussian_filter1d(coords[:, 1], sigma=sigma, mode="nearest")
    return LineString(np.column_stack([x, y]))


def moving_average_smooth(line, window=15):
    coords = np.array(line.coords)
    if len(coords) < window:
        return line
    kernel = np.ones(window) / window
    x = np.convolve(coords[:, 0], kernel, mode="same")
    y = np.convolve(coords[:, 1], kernel, mode="same")
    return LineString(np.column_stack([x, y]))


def savgol_smooth(line, window=21, poly=2):
    coords = np.array(line.coords)
    if len(coords) < window:
        return line
    x = savgol_filter(coords[:, 0], window, poly)
    y = savgol_filter(coords[:, 1], window, poly)
    return LineString(np.column_stack([x, y]))


def polyfit_smooth(line, window_km=200, degree=2):
    coords = np.array(line.coords)
    if len(coords) < 10:
        return line
    x = coords[:, 0]
    y = coords[:, 1]
    # cumulative arclength (meters)
    dx = np.diff(x)
    dy = np.diff(y)
    ds = np.sqrt(dx**2 + dy**2)
    s = np.concatenate([[0], np.cumsum(ds)])
    x_new = np.zeros_like(x)
    y_new = np.zeros_like(y)
    half_window = window_km * 1000 / 2
    for i in range(len(s)):
        mask = np.abs(s - s[i]) <= half_window
        if np.sum(mask) < degree + 2:
            x_new[i] = x[i]
            y_new[i] = y[i]
            continue
        # Fit polynomial y(s), x(s)
        px = np.polyfit(s[mask], x[mask], degree)
        py = np.polyfit(s[mask], y[mask], degree)
        x_new[i] = np.polyval(px, s[i])
        y_new[i] = np.polyval(py, s[i])
    return LineString(np.column_stack([x_new, y_new]))


def _get_coastline(resolution="50m"):
    if resolution not in _COAST_CACHE:
        shpfilename = shpreader.natural_earth(
            resolution=resolution,
            category="physical",
            name="coastline",
        )
        _COAST_CACHE[resolution] = gpd.read_file(shpfilename)
    return _COAST_CACHE[resolution]


def _tangent_on_line_near_point(line, point_xy, neighborhood=2):
    coords = np.asarray(line.coords, dtype=float)
    if coords.shape[0] < 2:
        raise ValueError("LineString must have at least 2 coordinates.")

    p = np.array(point_xy, dtype=float)
    best = {"dist2": np.inf, "seg_idx": 0, "proj": coords[0].copy(), "t": 0.0}

    for i in range(len(coords) - 1):
        a = coords[i]
        b = coords[i + 1]
        ab = b - a
        denom = np.dot(ab, ab)
        if denom == 0:
            t = 0.0
            q = a
        else:
            t = np.clip(np.dot(p - a, ab) / denom, 0.0, 1.0)
            q = a + t * ab

        d2 = np.dot(p - q, p - q)
        if d2 < best["dist2"]:
            best = {"dist2": d2, "seg_idx": i, "proj": q, "t": float(t)}

    i_center = best["seg_idx"] + (1 if best["t"] >= 0.5 else 0)
    i0 = max(0, i_center - neighborhood)
    i1 = min(len(coords) - 1, i_center + neighborhood)
    tangent = coords[i1] - coords[i0]

    if np.allclose(tangent, 0.0):
        tangent = coords[best["seg_idx"] + 1] - coords[best["seg_idx"]]
```

```
    return {
        "tangent_vector": (float(tangent[0]), float(tangent[1])),
        "nearest_point": (float(best["proj"][0]), float(best["proj"][1])),
        "distance": float(np.sqrt(best["dist2"])),
    }


def coastline_gaussian_tangent(
    query_point,
    sigma=2,
    region_bbox_tuple=(70, 5, 95, 30),
):
    """
    One-call coastline tangent helper for lon/lat inputs.

    Parameters
    ----------
    query_point : tuple(float, float)
        Longitude/latitude point as (lon, lat).
    sigma : float, optional
        Gaussian smoothing sigma. Default is 2.
    region_bbox_tuple : tuple(float, float, float, float), optional
        Clipping box as (lon_min, lat_min, lon_max, lat_max).
        Default is (70, 5, 95, 30).

    Returns
    -------
    dict
        {
            "heading_angle_deg": float,   # 0-360 deg, measured CCW from +lon (east)
            "tangent_vector": (tx, ty),
            "nearest_point": (lon, lat),
            "distance": float
        }
    """
    if len(region_bbox_tuple) != 4:
        raise ValueError(
            "region_bbox_tuple must be (lon_min, lat_min, lon_max, lat_max)."
        )

    region_bbox = box(*region_bbox_tuple)
    coast = _get_coastline("50m")
    coast_clip = gpd.clip(coast, region_bbox)

    if coast_clip.empty:
        raise ValueError("No coastline found in the provided bounding box.")

    coast_gauss = coast_clip.copy()
    coast_gauss["geometry"] = coast_clip.geometry.apply(
        lambda g: apply_to_geom(g, lambda line: gaussian_smooth(line, sigma=sigma))
    )

    pt = Point(query_point)
    best_geom = min(coast_gauss.geometry, key=lambda g: g.distance(pt))

    if isinstance(best_geom, MultiLineString):
        best_geom = min(best_geom.geoms, key=lambda g: g.distance(pt))
    if not isinstance(best_geom, LineString):
        raise TypeError("Nearest coastline geometry is not line-like.")

    tangent_info = _tangent_on_line_near_point(best_geom, query_point, neighborhood=2)

    tx, ty = tangent_info["tangent_vector"]
    if tx < 0:
        tx, ty = -tx, -ty
    heading_angle_deg = float((np.degrees(np.arctan2(ty, tx)) + 360) % 360)

    return {
        "heading_angle_deg": heading_angle_deg,
        "tangent_vector": tangent_info["tangent_vector"],
        "nearest_point": tangent_info["nearest_point"],
        "distance": tangent_info["distance"],
    }
```

```python
import functools
from joblib import Parallel, delayed
import os
import glob
import time
from datetime import datetime, timedelta

import cartopy.crs as ccrs
import cartopy.feature as cfeature
import geopandas as gpd
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import shapely.geometry as sg
import statsmodels.api as sm
import xarray as xr
import xskillscore as xs
from cartopy.mpl.ticker import LatitudeFormatter, LongitudeFormatter
from dateutil.relativedelta import relativedelta
from geopy import distance
from loess.loess_1d import loess_1d
from scipy.ndimage import gaussian_filter1d
from scipy.signal import butter, filtfilt
from scipy.stats import median_abs_deviation
from shapely.geometry import Point, Polygon
from namesm import *
import math
from tools_shapely import *


def dist_asn_gshhg(ds, var_str="dist_to_coast_gshhg", units_in="m"):
    dists = compute_dist(ds, units=units_in)
    dists_rev = dists[::-1]
    # print(dists_rev)
    var1 = ds[var_str]
    var1_rev = var1.load().isel(points_numbers=slice(None, None, -1)).values
    var2 = xr.DataArray(
        var1_rev,
        coords=[dists_rev],
        dims=["x"],
    )
    if units_in == "m":
        var2.coords["x"].attrs["units"] = "m"
    elif units_in == "km":
        var2.coords["x"].attrs["units"] = "km"
    return var2


def mad_filter_1d_xr(da, thresh, dim=None):
    if dim is None:
        if "time" in da.dims:
            dim = "time"
        elif "x" in da.dims:
            dim = "x"
        elif len(da.dims) == 1:
            dim = da.dims[0]
        else:
            raise ValueError("Could not infer dimension. Pass dim='time' or dim='x'.")
    elif dim not in da.dims:
        raise ValueError(f"Dimension '{dim}' not found in input DataArray dims: {da.dims}")
    da_vals = da.values
    dim_vals = da[dim]
    if len(da_vals) < 3 or np.all(np.isnan(da_vals)):
        return xr.DataArray(
            np.full_like(da_vals, np.nan, dtype=float),
            coords={dim: dim_vals},
            dims=[dim],
        )
    valid_mask = ~np.isnan(da_vals)
    if np.sum(valid_mask) < 3:
        return xr.DataArray(
            np.full_like(da_vals, np.nan, dtype=float),
            coords={dim: dim_vals},
            dims=[dim],
        )
    # size of valid data might be less than input depending on NaNs
    valid_data = da_vals[valid_mask]
    median = np.median(valid_data)
    mad = median_abs_deviation(valid_data, scale='normal')
    if mad == 0:
        mad = np.median(np.abs(valid_data - median)) * 1.4826
        if mad == 0:
            return xr.DataArray(
                np.full_like(da_vals, np.nan, dtype=float),
                coords={dim: dim_vals},
                dims=[dim],
            )
    modified_z_scores = 0.6745 * (da_vals - median) / mad
    # size of outliers is same as input size
    outliers = np.abs(modified_z_scores) > thresh
    da_filtered = np.full_like(da_vals, np.nan, dtype=float)
    da_filtered[valid_mask] = da_vals[valid_mask]
    da_filtered[outliers] = np.nan
    da_out = xr.DataArray(da_filtered, coords={dim: dim_vals}, dims=[dim])
    return da_out


def mad_filter_2d_xr(da, thresh, dims=("time", "x")):
    """
    Apply MAD outlier filtering on a 2D DataArray across both dimensions.
    The filter is applied sequentially for each dimension listed in `dims`.
    """
    if len(da.dims) != 2:
        raise ValueError(
            f"mad_filter_2d_xr expects a 2D DataArray, got dims: {da.dims}"
        )
    for dim in dims:
        if dim not in da.dims:
            raise ValueError(
                f"Dimension '{dim}' not found in input DataArray dims: {da.dims}"
            )

    def mad_filter_1d_np(x, thresh_local):
        if len(x) < 3 or np.all(np.isnan(x)):
            return np.full_like(x, np.nan, dtype=float)
        valid_mask = ~np.isnan(x)
        if np.sum(valid_mask) < 3:
```

```
            return np.full_like(x, np.nan, dtype=float)
        valid_data = x[valid_mask]
        median = np.median(valid_data)
        mad = median_abs_deviation(valid_data, scale="normal")
        if mad == 0:
            mad = np.median(np.abs(valid_data - median)) * 1.4826
            if mad == 0:
                return np.full_like(x, np.nan, dtype=float)
        modified_z_scores = 0.6745 * (x - median) / mad
        outliers = np.abs(modified_z_scores) > thresh_local
        x_filtered = x.astype(float).copy()
        x_filtered[outliers] = np.nan
        return x_filtered

    orig_dims = da.dims
    da_filtered = da.astype(float).copy()
    for dim in dims:
        da_filtered = xr.apply_ufunc(
            lambda x: mad_filter_1d_np(x, thresh),
            da_filtered,
            input_core_dims=[[dim]],
            output_core_dims=[[dim]],
            vectorize=True,
            dask="allowed",
            output_dtypes=[float],
        )
        # apply_ufunc may reorder dimensions (e.g., to ('time', 'x'));
        # force the original order after each pass.
        da_filtered = da_filtered.transpose(*orig_dims)

    da_filtered.attrs = da.attrs
    da_filtered.name = da.name
    return da_filtered


def find_n_closest_points(df, lon0, lat0, n=4):
    """Return n rows closest to (lon0, lat0) using intersection lon/lat."""
    lons_inter = df["lons_inter"].tolist()
    lats_inter = df["lats_inter"].tolist()
    distances = [
        distance.distance((y, x), (lat0, lon0)).km
        for x, y in zip(lons_inter, lats_inter)
    ]
    out = df.copy()
    out["distance_km"] = distances
    return out.nsmallest(n, "distance_km")


def local_angle(lons, lats, ii, ioff=3, degrees=False):
    ln = len(lons)
    if ii - ioff < 0: # at the beginning
        # forward difference in to the increasing side
        idx_start = ii
        idx_end = min(ii + ioff, ln - 1) # Ensure we don't go over
    # If we are too close to the end, look backward only
    elif ii + ioff >= ln: # at the end
        # backward difference in to the decreasing side
        idx_start = max(ii - ioff, 0)
        idx_end = ii
    # Normal case: Centered difference
    else:
        idx_start = ii - ioff
        idx_end = ii + ioff
    dlat1 = lats[idx_end] - lats[idx_start]
    dlon1 = lons[idx_end] - lons[idx_start]
    a1 = math.atan2(dlat1, dlon1)
    if degrees:
        a1 = math.degrees(a1)
    return a1


def mad_outlier_detection_fixed(data, threshold=3.5, dim="cycles_numbers"):
    """
    MAD outlier detection using your actual dimension names.
    """
    def mad_filter_1d(x, thresh):
        if len(x) < 3 or np.all(np.isnan(x)):
            return np.zeros_like(x, dtype=bool)
        valid_mask = ~np.isnan(x)
        if np.sum(valid_mask) < 3:
            return np.zeros_like(x, dtype=bool)
        # size of valid data might be less than x depending on the
        # number of NaNs
        valid_data = x[valid_mask]
        median = np.median(valid_data)
        mad = median_abs_deviation(valid_data, scale='normal')
        if mad == 0:
            mad = np.median(np.abs(valid_data - median)) * 1.4826
            if mad == 0:
                return np.zeros_like(x, dtype=bool)
        modified_z_scores = 0.6745 * (x - median) / mad
        outliers = np.abs(modified_z_scores) > thresh
        # size of outliers is same as the size of x
        return outliers

    outlier_mask = xr.apply_ufunc(
        lambda x: mad_filter_1d(x, threshold),
        data,
        input_core_dims=[[dim]],
        output_core_dims=[[dim]],
        vectorize=True,
        dask="allowed",
        output_dtypes=[bool]
    )
    return outlier_mask


def ln_sf(src, dst):
    try:
        os.remove(dst)   # works for files and symlinks
    except FileNotFoundError:
        pass
    os.symlink(src, dst)


def read_drifter_data(file_path):
    # Define column names
```

```
    col_names = [
        "id",
        "mm",
        "dd",
        "yy",
        "lat",
        "lon",
        "temp",
        "ve",
        "vn",
        "spd",
        "var_lat",
        "var_lon",
        "var_tmp",
    ]
    # Read the file into a pandas dataframe
    df = pd.read_csv(
        file_path,
        delim_whitespace=True,
        header=None,
        names=col_names,
        na_values=999.999,
    )
    # Separate the integer and fractional parts of the day
    df["day_fraction"], df["dd"] = np.modf(df["dd"])
    # Convert the integer part of the day to integer type
    df["dd"] = df["dd"].astype(int)
    # Calculate the hours from the fractional part of the day
    df["hours"] = (df["day_fraction"] * 24).round().astype(int)
    # Combine the date and time components into a single datetime column
    df["datetime1"] = pd.to_datetime(df[["yy", "mm", "dd"]].astype(str).agg(
        "-".join, axis=1)) + pd.to_timedelta(df["hours"], unit="h")
    # Drop the intermediary columns used for calculations
    df = df.drop(columns=["mm", "dd", "yy", "day_fraction", "hours"])
    df.set_index("datetime1", inplace=True)
    return df


def read_drifter_data_simple(fname):
    """Read NOAA Global Drifter Program text data into pandas format.
    Parameters
    ----------
    fname : str
    Returns
    -------
    df : datafram
    """
    def _parse_date(mm, dd, yy):
        dd = float(dd)
        mm = int(mm)
        yy = int(yy)
        day = int(dd)
        hour = int(24 * (dd - day))
        return datetime(yy, mm, day, hour)
    return pd.read_csv(
        fname,
        names=col_names,
        sep=r"\s+",
        header=None,
        na_values=999.999,
        parse_dates={"time": [1, 2, 3]},
        date_parser=_parse_date,
    )


def var_asn(ds, var_str="ve", daily=False):
    lon = ds.longitude.values
    lat = ds.latitude.values
    time1= ds.time.isel(traj=0).values
    var1 = ds[var_str].values
    var_asn = xr.DataArray(var1,coords=[[1],time1],dims=["traj" ,"time"])
    if daily:
        var_asn = var_asn.resample(time="1D").mean()
    return var_asn


def drifter_time_asn(ds_d, var_str="ve", daily=False):
    time1 = ds_d.time.isel(traj=0).values
    var1 = ds_d[var_str].isel(traj=0).values
    var_asn = xr.DataArray(var1,
                           coords=[time1],
                           dims=["time"],
                           attrs=ds_d[var_str].attrs)
    if daily:
        var_asn = var_asn.resample(time="1D").mean()
    var_asn.name = var_str
    return var_asn


def angle_between_lines(m1, m2):
    angle_r = np.arctan(np.abs((m2 - m1) / (1 + m1*m2)))
    # angle_d = np.rad2deg(angle_r)
    return angle_r


def geostrophic_current_components_1(ga, gd, meridian_angle_r):
    """
    Calculates the zonal (u) and meridional (v) components and the speed
    of the geostrophic current.
    Parameters:
    - g1: Cross-track geostrophic current along track T1 (float)
    - g2: Cross-track geostrophic current along track T2 (float)
    - angle_inter: Acute angle of intersection between track T1 and T2
    Returns:
    - u: Zonal component of the geostrophic current (float)
    - v: Meridional component of the geostrophic current (float)
    - speed: Speed (magnitude) of the geostrophic current (float)
    """
    # Convert slopes to angles in radians
    sin_theta = math.sin(meridian_angle_r)
    cos_theta = math.cos(meridian_angle_r)
    u = (gd + ga.interp(time=gd.time)) / 2*cos_theta
    v = (gd - ga.interp(time=gd.time)) / 2*sin_theta
    speed = (u**2 + v**2)**0.5
    u1 = u
    v1 = v
    speed1 = speed
```

```
352     return u1, v1, speed1
353
354
355 def butter_lowpass_filter(data, cutoff, fs, order):
356     """
357     Designs and applies a Butterworth low-pass filter.
358     Parameters:
359     - data: 1D numpy array, the time series data.
360     - cutoff: float, the cutoff frequency in cycles per day.
361     - fs: float, the sampling frequency in samples per day.
362     - order: int, the order of the filter.
363     Returns:
364     - y: 1D numpy array, the filtered data.
365     """
366     nyq = 0.5 * fs  # Nyquist Frequency
367     normal_cutoff = cutoff / nyq  # Normalized cutoff frequency
368     b, a = butter(order, normal_cutoff, btype='low', analog=False)
369     y = filtfilt(b, a, data)
370     return y
371
372
373 def butter_lowpass_2d(da, cutoff_days=3, order=4):
374     """
375     Applies a Butterworth low-pass filter along the 'time' dimension.
376     Parameters:
377     - da: xarray DataArray, input data with dimensions ('x', 'time').
378     - cutoff_days: float, the cutoff period in days.
379     - order: int, the order of the Butterworth filter.
380     Returns:
381     - filtered_da: xarray DataArray, the filtered data.
382     """
383     fs = 1  # Sampling frequency: 1 sample per day
384     cutoff = 1 / cutoff_days  # Cutoff frequency in cycles per day
385     def filter_func(ts):
386         """
387         Filters a 1D time series, handling missing values.
388         Parameters:
389         - ts: 1D numpy array, the time series data.
390         Returns:
391         - y: 1D numpy array, the filtered time series.
392         """
393         # Find indices where data is not NaN
394         not_nan = ~np.isnan(ts)
395         if np.sum(not_nan) < 2:
396             # Not enough data to interpolate
397             return ts
398         else:
399             # Interpolate missing values
400             x = np.arange(len(ts))
401             ts_interp = np.interp(x, x[not_nan], ts[not_nan])
402             # Apply the Butterworth filter
403             y = butter_lowpass_filter(ts_interp, cutoff, fs, order)
404             # Restore NaN values in their original positions
405             y[~not_nan] = np.nan
406             return y
407     # Apply the filter function along the 'time' dimension
408     filtered_da = xr.apply_ufunc(
409         filter_func,
410         da,
411         input_core_dims=[['time']],
412         output_core_dims=[['time']],
413         vectorize=True,
414         dask='parallelized',  # If using Dask for parallel computation
415         output_dtypes=[da.dtype]
416     )
417     return filtered_da
418
419
420 def butter_lowpass_1d(da, cutoff_days=3, order=4):
421     """
422     Applies a Butterworth low-pass filter to a 1D xarray DataArray along the
423     'time' dimension,
424     smoothing out signals with periods shorter than the specified cutoff.
425     Parameters:
426     - da: xarray DataArray with dimension 'time'.
427     - cutoff_days: float, the cutoff period in days (default is 3 days).
428     - order: int, the order of the Butterworth filter (default is 4).
429     Returns:
430     - filtered_da: xarray DataArray, the filtered data.
431     """
432     fs = 1  # Sampling frequency: 1 sample per day
433     cutoff = 1 / cutoff_days  # Cutoff frequency in cycles per day
434     # Extract the data as a numpy array
435     ts = da.values
436     # Identify indices with valid (non-NaN) data
437     not_nan = ~np.isnan(ts)
438     if np.sum(not_nan) < 2:
439         # Not enough data to perform filtering
440         filtered_values = ts.copy()
441     else:
442         # Interpolate missing values for filtering
443         x = np.arange(len(ts))
444         ts_interp = np.interp(x, x[not_nan], ts[not_nan])
445         # Design the Butterworth filter
446         nyq = 0.5 * fs  # Nyquist Frequency
447         normal_cutoff = cutoff / nyq  # Normalized cutoff frequency
448         b, a = butter(order, normal_cutoff, btype='low', analog=False)
449         # Apply the filter using filtfilt for zero-phase filtering
450         # y = filtfilt(b, a, ts_interp)
451         if len(ts_interp) <= 15:
452             y = ts_interp.copy()  # Skip filtering or handle appropriately
453         else:
454             y = filtfilt(b, a, ts_interp)
455         # Restore NaN values at their original positions
456         y[~not_nan] = np.nan
457         filtered_values = y
458     # Create a new DataArray with the filtered values, preserving the original coordinates and attributes
459     filtered_da = xr.DataArray(filtered_values, coords=da.coords, dims=da.dims, attrs=da.attrs)
460     return filtered_da
461
462
463 def outer_labels_k(m, n, k, ax, xticks1=[], yticks1=[]):
464     """
465     In multi m x n subplots puts y axis labels and ticks only along left side of
466     leftmost vertical columns and puts x axis labels
467     and ticks only along bottom side of bottommost horizontal rows
468     """
```

```
    i1 = k // n # row number
    j1 = k % n # column number
    # all other top right subplots
    if i1 < m - 1 and j1 > 0:
        print(k, i1,j1,"some top right subplot")
        ax.set_xticks([])
        ax.set_yticks([])
        ax.set_xlabel("")
        ax.set_ylabel("")
        return False, False
    # left most column
    elif j1 == 0 and i1 != m - 1:
        ax.set_xticks([])
        ax.set_xlabel("")
        ax.set_yticks(yticks1)
        ax.yaxis.set_major_formatter(LatitudeFormatter())
        print(k, i1,j1,"leftmost but not corner bottom left subplot")
        return False, True
    # bottom most row
    elif i1 == m - 1 and j1 != 0:
        ax.set_yticks([])
        ax.set_ylabel("")
        ax.set_xticks(xticks1)
        ax.xaxis.set_major_formatter(LongitudeFormatter())
        print(k, i1,j1,"bottommost but not corner bottom left subplot")
        return True, False
        # p,"hor")
    # single bottom left subplots
    # only subplot to have all ticks and labels
    else:
        print(k, i1,j1,"corner bottom left subplot")
        ax.set_xticks(xticks1)
        ax.set_yticks(yticks1)
        ax.xaxis.set_major_formatter(LongitudeFormatter())
        ax.yaxis.set_major_formatter(LatitudeFormatter())
        return True, True


def outer_labels_k_bool(m, n, k):
    """
    In multi m x n subplots puts y axis labels and ticks only along left side of
    leftmost vertical columns and puts x axis labels
    and ticks only along bottom side of bottommost horizontal rows
    """
    i1 = k // n # row number
    j1 = k % n # column number
    print(f"{m}x{n}  {k}th suplot")
    # all other top right subplots
    if i1 < m - 1 and j1 > 0:
        print(k, i1, j1, "some top right subplot")
        return False, False
    # left most column
    elif j1 == 0 and i1 != m - 1:
        print(k, i1, j1, "leftmost but not corner bottom left subplot")
        return False, True
    # bottom most row
    elif i1 == m - 1 and j1 != 0:
        print(k, i1, j1, "bottommost but not corner bottom left subplot")
        return True, False
    # single bottom left subplots
    # only subplot to have all ticks and labels
    else:
        print(k, i1, j1, "corner bottom left subplot")
        return True, True


def print_stats(da):
    """
    Prints comprehensive statistics for a given xarray.DataArray.
    Args:
        da (xarray.DataArray): The DataArray to analyze.
    Returns:
        None
    """
    # Handle missing values
    valid_data = da  # .dropna(dim='time')
    # Calculate statistics
    min_val = valid_data.min().item()
    max_val = valid_data.max().item()
    mean_val = valid_data.mean().item()
    std_val = valid_data.std().item()
    num_valid_points = valid_data.count().item()
    # Print formatted results
    print(f"DataArray statistics:")
    print(f"  * Minimum: {min_val}")
    print(f"  * Maximum: {max_val}")
    print(f"  * Mean: {mean_val}")
    print(f"  * Standard deviation: {std_val}")
    print(f"  * Number of valid points: {num_valid_points}")


def is_monotonic(arr):
    return np.all(np.diff(arr) >= 0)


def is_ocean(lon, lat):
    land = gpd.read_file("~/allData/topo/ne_10m_land.shp")
    point = Point(lon, lat)
    return not any(land.contains(point))


def is_land(lon, lat):
    land = gpd.read_file("~/allData/topo/ne_10m_land.shp")
    point = Point(lon, lat)
    return any(land.contains(point))


def is_within_region(lon, lat, lon_min, lon_max, lat_min, lat_max):
    return (lon_min <= lon <= lon_max) and (lat_min <= lat <= lat_max)


# write a function to check if all points described by lons and lat
# are within the region defined by lon_min, lon_max, lat_min, lat_max
def is_within_region_arr(lons, lats, lon_min, lon_max, lat_min, lat_max):
    lons_within_range = np.logical_and(lons >= lon_min, lons <= lon_max)
    # Check if all latitudes are within the range
    lats_within_range = np.logical_and(lats >= lat_min, lats <= lat_max)
```

```python
    # Combine both conditions
    all_within_region = np.logical_and(lons_within_range, lats_within_range)
    # Check if all points are within the region
    if np.all(all_within_region):
        print("All points are within the region.")
        return True
    else:
        print("Not all points are within the region.")
        return False


# def is_within_region_arr(lons, lats, lon_min, lon_max, lat_min, lat_max):
#     lons_within_range = np.logical_and(lons >= lon_min, lons <= lon_max)
#     # Check if all latitudes are within the range
#     lats_within_range = np.logical_and(lats >= lat_min, lats <= lat_max)
#     # Combine both conditions
#     all_within_region = np.logical_and(lons_within_range, lats_within_range)
#     # Check if all points are within the region
#     if np.all(all_within_region):
#         print("All points are within the region.")
#         return True
#     else:
#         print("Not all points are within the region.")
#         return False
#

def timeit_decorator(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        print(
            f"Execution time for {func.__name__}: {end_time - start_time} seconds"
        )
        return result

    return wrapper


# CARTESIAN GEOMETRY FUNCTIONS
def get_point_at_distance(x1, y1, x2, y2, d):
    # point on line (x1, y1) to (x2, y2) at distance d
    dist = np.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
    x = x1 + (d / dist) * (x2 - x1)
    y = y1 + (d / dist) * (y2 - y1)
    return x, y


def filter_between(valc, vale, buoy, val1, val2):
    """
    valc is control
    vale is experimental for comparison
    """
    cond = valc.between(val1, val2) & vale.between(val1, val2)
    valc1 = valc[cond]
    vale1 = vale[cond]
    buoy1 = buoy[cond]
    return valc1, vale1, buoy1


def index_at_x(x, x1):
    i1 = np.abs(x - x1).argmin()
    return i1


def lonlat_at_x(ds, x, x1):
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    i1 = index_at_x(x, x1)
    lon1 = lons_track_rev[i1]
    lat1 = lats_track_rev[i1]
    return lon1, lat1


def index_at_lon(ds, lon1):
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    i1 = np.abs(lons_track_rev - lon1).argmin()
    return i1


def index_at_lat(ds, lat1):
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    i1 = np.abs(lats_track_rev - lat1).argmin()
    return i1


def sel_along_lon(ds, da, lon1, lon2):
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    i1 = np.abs(lons_track_rev - lon1).argmin()
    i2 = np.abs(lons_track_rev - lon2).argmin()
    da_out = da.isel(x=slice(i1, i2 + 1))
    return da_out


def sel_along_lat(ds, da, lat1, lat2):
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    i1 = np.abs(lats_track_rev - lat1).argmin()
    i2 = np.abs(lats_track_rev - lat2).argmin()
    da_out = da.isel(x=slice(i1, i2 + 1))
    return da_out


```

```python
def rename_xax(ds):
    for d in ds.dims:
        if "XAX" in d:
            xdim = d
    ds = ds.rename({xdim: "x"})
    return ds


def flip_n_asn(ds, da):
    x = ds.x.values
    times = da.time.values
    da_rev = da.isel(x=slice(None, None, -1)).values
    da_asn = xr.DataArray(da_rev, coords={"time": times, "x": x})
    return da_asn


def to_datetime(date):
    timestamp = (date - np.datetime64("1970-01-01T00:00:00")) / np.timedelta64(
        1, "s"
    )
    return datetime.utcfromtimestamp(timestamp)


def n64todatetime(d):
    if np.isnat(d):
        return None
    return datetime.strptime(
        np.datetime_as_string(d, unit="s"), "%Y-%m-%dT%H:%M:%S"
    )

def n64todatetime1(d):
    if np.isnat(d):
        return None
    return d.astype('datetime64[ms]').astype(datetime)


def append_metrics(ts1, ts2, df, sat, psmsl_id, track_no, lon1, lat1, jj, mindist):
    # print(df)
    std1 = ts1.std(dim="time").item()
    std2 = ts2.std(dim="time").item()
    bias = ts1 - ts2
    ln = len(bias)
    bias1 = bias.mean(dim="time").item()
    corr = xs.pearson_r(ts1, ts2, dim="time", skipna=True)
    corr1 = corr.item()
    p = xs.pearson_r_p_value(ts1, ts2, dim="time", skipna=True)
    p1 = p.item()
    rmse = xs.rmse(ts1, ts2, dim="time", skipna=True)
    rmse1 = rmse.item()
    vc_p = 100 * (bias.count(dim="time").item()) / ln
    row = [sat, psmsl_id, track_no, lon1, lat1, bias1, corr1, rmse1, p1, vc_p, mindist, std1, std2]
    df.loc[jj] = row
    # print(sat, psmsl_id, track_no, lon1, lat1, bias1, corr1, rmse1, p1, vc_p)
    return df


def append_metrics_s(ts1, ts2, df, sat, psmsl_id, track_no, lon1, lat1, jj):
    # print(df)
    bias = ts1 - ts2
    ln = len(bias)
    bias1 = bias.mean(dim="time").item()
    std1 = ts1.std(dim="time").item()
    std2 = ts2.std(dim="time").item()
    corr = xs.pearson_r(ts1, ts2, dim="time", skipna=True)
    corr1 = corr.item()
    p = xs.pearson_r_p_value(ts1, ts2, dim="time", skipna=True)
    p1 = p.item()
    rmse = xs.rmse(ts1, ts2, dim="time", skipna=True)
    rmse1 = rmse.item()
    vc_p = 100 * (bias.count(dim="time").item()) / ln
    row = [
        sat,
        psmsl_id,
        track_no,
        lon1,
        lat1,
        bias1,
        corr1,
        rmse1,
        p1,
        vc_p,
        std1,
        std2,
    ]
    df.loc[jj] = row
    return df


def overlap_dates(t1_start, t1_end, t2_start, t2_end):
    # Check for overlap
    if t1_start is None or t1_end is None or t2_start is None or t2_end is None:
        return None, None  # Handle the case where None is found
    if t1_end < t2_start or t1_start > t2_end:
        # No overlap, the periods are disjoint
        overlap_start = None
        overlap_end = None
    else:
        # Overlap exists, calculate the overlap
        overlap_start = max(t1_start, t2_start)
        overlap_end = min(t1_end, t2_end)
    # Now, the overlap_start and overlap_end represent the overlapping time period, if any.
    return overlap_start, overlap_end


def overlap_dates_np64(t1_start, t1_end, t2_start, t2_end):
    """
    Return overlapping interval for numpy datetime64 endpoints.
    Returns (np.datetime64("NaT"), np.datetime64("NaT")) if no overlap.
    """
    if (
        np.isnat(t1_start)
        or np.isnat(t1_end)
        or np.isnat(t2_start)
        or np.isnat(t2_end)
    ):
        return np.datetime64("NaT"), np.datetime64("NaT")
    if t1_end < t2_start or t1_start > t2_end:
```

```
820         return np.datetime64("NaT"), np.datetime64("NaT")
821     overlap_start = max(t1_start, t2_start)
822     overlap_end = min(t1_end, t2_end)
823     return overlap_start, overlap_end
824
825
826 # GEOSTROPHIC FUNCTIONS
827 def earth_radius(lat):
828     # https://en.wikipedia.org/wiki/Earth_radius
829     """
830     Calculate the Earth's radius at a given latitude based on the WGS-84 ellipsoid model
831     Parameters:
832     lat (float): Latitude in degrees.
833     Returns:
834     float: Earth's radius at the given latitude in kilometers.
835     """
836     # Convert latitude from degrees to radians
837     lat_rad = np.radians(lat)
838     # WGS-84 ellipsoid parameters
839     R_e = 6378.137  # Equatorial radius in kilometers
840     R_p = 6356.752  # Polar radius in kilometers
841     # Calculate Earth's radius at the given latitude
842     numerator = (R_e**2 * np.cos(lat_rad)) ** 2 + (
843         R_p**2 * np.sin(lat_rad)
844     ) ** 2
845     denominator = (R_e * np.cos(lat_rad)) ** 2 + (R_p * np.sin(lat_rad)) ** 2
846     return np.sqrt(numerator / denominator)
847
848
849 def degrees2kilometers(degrees, lat):
850     """
851     :type degrees: float
852     :param degrees: Distance in (great circle) degrees
853     :type radius: float, optional
854     :param radius: Radius of the Earth used for the calculation.
855     :rtype: float
856     :return: Distance in kilometers as a floating point number.
857     """
858     radius = earth_radius(lat)
859     return degrees * (2.0 * radius * np.pi / 360.0)
860
861
862 def cor_p(lat):
863     # write a test case for this function
864     """
865     Coriolis parameter in 1/s for latitude in degrees.
866     """
867     omega = 7.292115e-5  # (1/s)   (Groten, 2004).
868     f = 2 * omega * np.sin(np.radians(lat))
869     da = xr.DataArray(f, coords={"lat": lat}, dims="lat")
870     return da
871
872
873 def grav(lat):
874     X = np.sin(lat * (np.pi) / 180.0)
875     sin2 = X**2
876     grav = 9.780327 * (1.0 + (5.2792e-3 + (2.32e-5 * sin2)) * sin2)
877     # international gravity formula
878     # https://en.wikipedia.org/wiki/Gravity_of_Earth#Latitude
879     da = xr.DataArray(grav, coords={"lat": lat}, dims="lat")
880     return da
881
882
883 def compute_dist(ds, units="m"):
884     lons_track = ds.lon.values
885     lats_track = ds.lat.values
886     lon_coast = lons_track[-1]  # this on coast
887     lat_coast = lats_track[-1]  # this on coast
888     if units == "km":
889         dists = [
890             distance.distance((y, x), (lat_coast, lon_coast)).km
891             for x, y in zip(lons_track, lats_track)
892         ]
893     elif units == "m":
894         dists = [
895             distance.distance((y, x), (lat_coast, lon_coast)).m
896             for x, y in zip(lons_track, lats_track)
897         ]
898     return dists
899
900
901 def compute_geostrophy_gc(ds, sla_smooth):
902     lons_track_rev = ds.lon.values[::-1]
903     lats_track_rev = ds.lat.values[::-1]
904     if len(lons_track_rev) == 0:
905         return None
906     # Add minimum length check
907     if len(lons_track_rev) < 3:
908         print("Warning: Insufficient data points for gradient calculation")
909         return None
910     f1 = cor_p(lats_track_rev)
911     f2 = f1.values
912     f3 = np.where(lats_track_rev > 2, f2, np.nan)
913     # 2s-2n band equatorial band is masked according to Sudre 2013
914     g = grav(lats_track_rev)  # ms-1
915     g1 = g.values
916     ratio = g1 / f3
917     ratio1 = ratio[:, np.newaxis]
918     dhbydx = sla_smooth.differentiate("x")
919     gc = ratio1 * dhbydx
920     # Add metadata
921     gc.attrs["units"] = "m/s"
922     gc.attrs["long_name"] = "Cross-track geostrophic current"
923     gc.attrs["description"] = "Geostrophic current perpendicular to satellite track"
924     return gc
925
926
927 def geostrophic_components(gc1, gc2, a1_r, a2_r):
928     """
929     Compute zonal and meridional geostrophic current components from
930     cross-track velocities measured by two intersecting satellite tracks.
931     Parameters:
932     -----------
933     gc1 : float or array-like
934         Cross-track geostrophic velocity from track 1 (m/s)
935     gc2 : float or array-like
936         Cross-track geostrophic velocity from track 2 (m/s)
```

```
    a1_r : float
        Heading angle of track 1 in radians (measured counter-clockwise from east)
    a2_r : float
        Heading angle of track 2 in radians (measured counter-clockwise from east)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (intersection angle ≈ 0°)
    """
    # Convert azimuths from degrees to radians
    theta = a2_r - a1_r
    assert a2_r < 3, "check if heading angles are in radians or not"
    assert a1_r < 3, "check if heading angles are in radians or not"
    sin_theta = math.sin(theta)
    # Check for parallel tracks
    if abs(sin_theta) < 1e-6:
        raise ValueError(f"Tracks are nearly parallel"
                         "(angle = {math.degrees(theta):.2f}°). "
                        "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1_r)
    cos_a2 = math.cos(a2_r)
    sin_a1 = math.sin(a1_r)
    sin_a2 = math.sin(a2_r)
    # Solve for velocity components using the derived formulas:
    u = (gc1 * cos_a2 - gc2 * cos_a1) / math.sin(a1_r - a2_r)
    v = (gc1 * sin_a2 - gc2 * sin_a1) / math.sin(a1_r - a2_r)
    return u, v


def compute_geostrophy_from_slope(ds, slope_smooth, current="zonal"):
    lons_track_rev = ds.lon.values[::-1]
    lats_track_rev = ds.lat.values[::-1]
    if len(lons_track_rev) == 0:
        return
    m = (lats_track_rev[-1] - lats_track_rev[0]) / (
        lons_track_rev[-1] - lons_track_rev[0]
    )
    angle_r = np.arctan(m)
    angle = np.rad2deg(angle_r)
    f1 = cor_p(lats_track_rev)
    f2 = f1.values
    f3 = np.where(lats_track_rev > 2, f2, np.nan)
    g = grav(lats_track_rev)  # ms-1
    g1 = g.values
    ratio = g1 / f3
    ratio1 = ratio[:, np.newaxis]
    gc = -1 * ratio1 * slope_smooth
    if current == "zonal":
        gu = -1 * gc * np.cos(np.deg2rad(90.0 - abs(angle)))
        return gu
    elif current == "meridional":
        gu = gc * np.sin(np.deg2rad(90.0 - abs(angle)))
        return gu
    elif current == "speed":
        guu = -1 * gc * np.cos(np.deg2rad(90.0 - abs(angle)))
        guv = gc * np.sin(np.deg2rad(90.0 - abs(angle)))
        gu = (guu**2 + guv**2) ** 0.5
        return gu


def compute_ekman(ds_wind):
    uwnd1 = ds_wind.uwnd
    vwnd1 = ds_wind.vwnd
    speed1 = (uwnd1**2.0 + vwnd1**2.0) ** 0.5
    rho_air = Constants.rho_air
    rho_sea = Constants.rho_sea  # kg/m3
    cd = Constants.cd
    Av = Constants.Av
    D = Constants.D
    wstress = cd * rho_air * speed1 * speed1
    wstress_u = cd * rho_air * speed1 * uwnd1
    wstress_v = cd * rho_air * speed1 * vwnd1
    wstress_tm = wstress.mean(dim="time")
    wstress_v_tm = wstress_v.mean(dim="time")
    wstress_u_tm = wstress_u.mean(dim="time")
    lat_w = ds_wind.latitude.values
    g = grav(lat_w)  # ms-1
    f = cor_p(lat_w)
    f1 = f.where((f.lat <= -2) | (f.lat >= 2))
    f2 = f1.values[np.newaxis, :, np.newaxis]
    D1 = np.sqrt(np.abs((2 * Av) / f1)) # # see equation 7.17
    D2 = D1.values[np.newaxis, :, np.newaxis]
    # Citation: Talley L.D., Pickard G.L., Emery W.J., Swift J.H., 2011. Descriptive Physical Oceanography: An Introduction (Sixth Edition), Elsevier, Boston, 560 pp.
    ekman_u = wstress_v / (f2 * rho_sea * D2)  # see equation 7.19a and 7.19b
    ekman_v = -1 * wstress_u / (f2 * rho_sea * D2)
    return ekman_u, ekman_v


# Function to calculate u_e and v_e
def compute_ekman_currents_wenbo_wang(ds_wind, lat):
    rho = 1.02 * 10**3  # kg/m^3 (water density)
    r = 2.15 * 10**-4   # m/s (friction coefficient)
    h_md = 32.5         # m (mixed layer depth)
# Function to calculate B and theta
    def compute_B_theta(lat):
        """Calculate B and theta for a given latitude."""
        f = cor_p(lat)
        B = 1 / (rho * np.sqrt(r**2 + (f * h_md)**2))
        theta = np.arctan(f * h_md / r)
        return B, theta
    """Calculate east (u_e) and north (v_e) components of Ekman currents."""
    uwnd1 = ds_wind.uwnd
    vwnd1 = ds_wind.vwnd
    speed1 = (uwnd1**2.0 + vwnd1**2.0) ** 0.5
    rho_air = Constants.rho_air
    rho_sea = Constants.rho_sea  # kg/m3
    cd = Constants.cd
    Av = Constants.Av
    D = Constants.D
    wstress = cd * rho_air * speed1 * speed1
    wstress_u = cd * rho_air * speed1 * uwnd1
```

```
    wstress_v = cd * rho_air * speed1 * vwnd1
    B, theta = compute_B_theta(lat)
    tau_complex = tau_x + 1j * tau_y  # tau_x + i*tau_y
    ue_ve = B * np.exp(1j * theta) * tau_complex
    return np.real(ue_ve), np.imag(ue_ve)


# SMOOTHING FUNCTIONS
def genweights(p, q, dt=1):
    """
    Given p and q, return the vector of cpn's: the optimal weighting
    coefficients and the noise reduction factor of h.
    p: Number of points before the point of interest (can be negative)
    q: Number of points after the point of interest (can be negative)
    dt: Sampling period (defaults to 1 s)
    Returns:
        tuple: (cn, noise_reduction_factor)
    """
    # Do some verification (if needed, allow negative p and q)
    if p > 0 or q < 0:
        raise ValueError(
            "p is supposed to be negative and q is supposed to be positive"
        )
    # Build the matrices
    N = abs(p) + abs(q)
    # T is the window length
    T = N + 1
    A = np.zeros((T, T))
    ones = np.ones(N)  # Assign ones to the last row
    # last row is made ones except for the last element
    A[T - 1, :] = np.append(ones, 0)  # and last elements is 0
    n = np.arange(p, q + 1)
    n = n[n != 0]  # Remove the zero
    # make a numpy array from list n
    n = np.array(n, dtype=float)
    for i in range(len(n)):
        # from the equation (11) in the paper Powell and Leben (2004)
        # right side term
        first_term = (-n[i] / 2) * np.reciprocal(n)
        # left side term
        second_term = (n[i] ** 2) * (dt**2) / 4
        A[i, :] = np.append(first_term, second_term)
        A[i, i] = -1
    B = np.zeros(T)
    B[-1] = 1  # Set the last element of B to 1
    # Compute the coefficients
    cn = np.linalg.solve(A, B)
    cn = cn[:-1]  # Remove the last element
    # Compute the error (noise reduction factor)
    # from the equation (8) in the paper Powell and Leben (2004)
    summ = np.sum(cn / (n * dt))
    squared = (cn / (n * dt)) ** 2
    noise_reduction_factor = np.sqrt(np.sum(squared) + summ**2)
    return cn, noise_reduction_factor


@timeit_decorator
def track_smooth_box_x(da, smooth=1, min_periods=1):
    var1 = da.rolling(
        x=smooth,
        center=True,
        min_periods=min_periods,
    ).mean()
    return var1


@timeit_decorator
def track_smooth_gaussian_x(da, sigma=3):
    # vals = np.empty((len(dists_rev), len(times)))
    dists_rev = da.x.values
    x_size = da.x.size
    t_size = da.time.size
    vals = np.empty((x_size, t_size))
    vals[:] = np.nan
    if t_size == 1:
        times = [da.time.item()]
    else:
        times = da.time.values
    for l in range(t_size):
        if t_size == 1:
            da1 = da.values
        else:
            da1 = da.isel(time=l).values
        da1_f = gaussian_filter1d(da1, sigma=sigma)
        vals[:, l] = da1_f
    var1 = xr.DataArray(
        vals,
        coords=[dists_rev, times],
        dims=["x", "time"],
    )
    return var1


@timeit_decorator
def slope_smooth_optimal_filter_x(da, p=-5, q=6):
    window_len = abs(p) + abs(q)
    dists_rev = da.x.values
    x_size = da.x.size
    if x_size < window_len:
        print(f"returning raw data ----------------------")
        return da
    t_size = da.time.size
    vals = np.empty((x_size, t_size))
    of_weights, error = genweights(p, q, dt=1)
    vals[:] = np.nan
    if t_size == 1:
        times = [da.time.item()]
    else:
        times = da.time.values
    for l in range(t_size):
        if t_size == 1:
            da1 = da.values
        else:
            da1 = da.isel(time=l).values
        da1_f = np.convolve(da1, of_weights, mode="same")
        vals[:, l] = da1_f
    var1 = xr.DataArray(
```

```
        vals,
        coords=[dists_rev, times],
        dims=["x", "time"],
    )
    return var1


@timeit_decorator
def track_smooth_loess_x(da, frac=0.2):
    dists_rev = da.x.values
    x_size = da.x.size
    t_size = da.time.size
    if t_size == 1:
        times = [da.time.item()]
    else:
        times = da.time.values
    da_f = da.interpolate_na(
        dim="x", method="linear", fill_value="extrapolate"
    )
    vals = np.empty((x_size, t_size))
    # vals = np.empty((len(dists_rev), len(times)))
    vals[:] = np.nan
    for l in range(t_size):
        if t_size == 1:
            da1 = da_f.values
        else:
            da1 = da_f.isel(time=l).values
        xout, yout, wout = loess_1d(
            dists_rev,
            da1,
            xnew=dists_rev,
            degree=1,
            frac=frac,
            npoints=None,
            rotate=False,
            sigy=None,
        )
        vals[:, l] = yout
    # print(vals.shape)
    var1 = xr.DataArray(
        vals,
        coords=[dists_rev, times],
        dims=["x", "time"],
    )
    return var1


@timeit_decorator
# @functools.lru_cache(maxsize=128)
# @njit
# @jit(nopython=True)
def track_smooth_loess_statsmodels_x(da, frac=0.2):
    """
    Smooths along-track sea level anomaly data using LOESS.

    Parameters:
    da (xr.DataArray): Input data array with dimensions 'x' and 'time'.
    frac (float, optional): Fraction of data points to use for smoothing. Defaults to 0.02.

    Returns:
    xr.DataArray: Smoothed data array.
    """
    dists_rev = da.x.values
    x_size = da.x.size
    t_size = da.time.size
    if t_size == 1:
        times = [da.time.item()]
    else:
        times = da.time.values
    da_f = da.interpolate_na(
        dim="x", method="linear", fill_value="extrapolate"
    )
    vals = np.empty((x_size, t_size))
    vals[:] = np.nan
    for l in range(t_size):
        if t_size == 1:
            da1 = da_f.values
        else:
            da1 = da_f.isel(time=l).values
        yout = sm.nonparametric.lowess(
            exog=dists_rev, endog=da1, frac=frac, return_sorted=False
        )
        vals[:, l] = yout
    var1 = xr.DataArray(
        vals,
        coords=[dists_rev, times],
        dims=["x", "time"],
        attrs=da.attrs
    )
    return var1


@timeit_decorator
def track_smooth_loess_statsmodels_x_parallel(da, frac=0.2, n_jobs=-1):
    """
    Smooths along-track sea level anomaly data using LOESS.

    Parameters:
    da (xr.DataArray): Input data array with dimensions 'x' and 'time'.
    frac (float, optional): Fraction of data points to use for smoothing. Defaults to 0.02.

    Returns:
    xr.DataArray: Smoothed data array.
    """
    def loess_smoothing(dists_rev, da1, frac):
        """Apply LOESS smoothing to a 1D array."""
        return sm.nonparametric.lowess(
            exog=dists_rev, endog=da1, frac=frac, return_sorted=False
        )
    dists_rev = da.x.values
    x_size = da.x.size
    t_size = da.time.size
    if t_size == 1:
        times = [da.time.item()]
    else:
        times = da.time.values
    da_f = da.interpolate_na(
```

```
        dim="x", method="linear", fill_value="extrapolate"
    )
    vals = np.empty((x_size, t_size))
    vals[:] = np.nan

    def process_time_step(l):
        if t_size == 1:
            da1 = da_f.values
        else:
            da1 = da_f.isel(time=l).values
        return loess_smoothing(dists_rev, da1, frac)

    # Parallel processing using joblib
    smoothed_results = Parallel(n_jobs=n_jobs)(
        delayed(process_time_step)(l) for l in range(t_size)
    )

    # Combine the results back into the vals array
    for l, yout in enumerate(smoothed_results):
        vals[:, l] = yout

    var1 = xr.DataArray(
        vals,
        coords=[dists_rev, times],
        dims=["x", "time"],
        attrs=da.attrs
    )
    return var1


def track_dist_time_asn(ds, var_str="sla", units_in="m"):
    # dists = compute_dist(ds, units="km")  # to convert to meters
    dists = compute_dist(ds, units=units_in)  # to convert to meters
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    dt0 = datetime.strptime("1950-01-01", "%Y-%m-%d")
    # these dists start from equator
    dists_rev = dists[::-1]
    # these dists start from coast
    ln = ds[var_str].sizes["points_numbers"]
    ln2 = ln // 2
    dvals = ds.time.isel(points_numbers=ln2).values
    try:
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    except ValueError:
        # dvals = pd.Series(dvals).ffill().bfill()
        dvals = pd.Series(dvals).ffill()
        dvals = pd.Series(dvals).bfill()
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    var1 = ds[var_str]
    # now sla values oriented from coast
    # var1_rev = var1.isel(points_numbers=slice(None, None, -1)).values
    var1_rev = var1.load().isel(points_numbers=slice(None, None, -1)).values
    var2 = xr.DataArray(
        var1_rev,
        coords=[dists_rev, dates],
        dims=["x", "time"],
    )
    if units_in == "m":
        var2.coords["x"].attrs["units"] = "m"
    elif units_in == "km":
        var2.coords["x"].attrs["units"] = "km"
    return var2


def virtual_dist_time_asn(ds):
    # dists = compute_dist(ds, units="km")  # to convert to meters
    dists = ds.distance_to_coast
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    dt0 = datetime.strptime("1950-01-01", "%Y-%m-%d")
    # these dists start from equator
    dists_rev = dists[::-1]
    # these dists start from coast
    ln = ds[var_str].sizes["points_numbers"]
    ln2 = ln // 2
    dvals = ds.time.isel(points_numbers=ln2).values
    try:
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    except ValueError:
        # dvals = pd.Series(dvals).ffill().bfill()
        dvals = pd.Series(dvals).ffill()
        dvals = pd.Series(dvals).bfill()
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    var1 = ds[var_str]
    # now sla values oriented from coast
    var1_rev = var1.isel(points_numbers=slice(None, None, -1)).values
    var2 = xr.DataArray(
        var1_rev,
        coords=[dists_rev, dates],
        dims=["x", "time"],
    )
    if units_in == "m":
        var2.coords["x"].attrs["units"] = "m"
    elif units_in == "km":
        var2.coords["x"].attrs["units"] = "km"
    return var2


def track_dist_time_asn_gshhg(ds, var_str="sla", units_in="m"):
    dt0 = datetime.strptime("1950-01-01", "%Y-%m-%d")
    # these dists start from equator
    if units_in == "m":
        dists = ds.dist_to_coast_gshhg.values
    elif units_in == "km":
        dists = 0.001 * ds.dist_to_coast_gshhg.values
    dists_rev = dists[::-1]
    # these dists start from coast
    ln = ds[var_str].sizes["points_numbers"]
    ln2 = ln // 2
    dvals = ds.time.isel(points_numbers=ln2).values
    try:
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
```

```
    except ValueError:
        # dvals = pd.Series(dvals).ffill().bfill()
        dvals = pd.Series(dvals).ffill()
        dvals = pd.Series(dvals).bfill()
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    var1 = ds[var_str]
    # now sla values oriented from coast
    var1_rev = var1.load().isel(points_numbers=slice(None, None, -1)).values
    var2 = xr.DataArray(
        var1_rev,
        coords=[dists_rev, dates],
        dims=["x", "time"],
    )
    if units_in == "m":
        var2.coords["x"].attrs["units"] = "m"
    elif units_in == "km":
        var2.coords["x"].attrs["units"] = "km"
    return var2


def track_dist_time_asn_midx(ds, var_str="sla", units_in="m"):
    # dists = compute_dist(ds, units="km")  # to convert to meters
    dists = compute_dist(ds, units=units_in)  # to convert to meters
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lons_track_rev = lons_track[::-1]
    lats_track_rev = lats_track[::-1]
    dt0 = datetime.strptime("1950-01-01", "%Y-%m-%d")
    # these dists start from equator
    dists_rev = dists[::-1]
    # these dists start from coast
    ln = ds[var_str].sizes["points_numbers"]
    ln2 = ln // 2
    dvals = ds.time.isel(points_numbers=ln2).values
    try:
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    except ValueError:
        dvals = pd.Series(dvals).ffill().bfill()
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    var1 = ds[var_str]
    # now sla values oriented from coast
    var1_rev = var1.load().isel(points_numbers=slice(None, None, -1)).values
    var2 = xr.DataArray(
        var1_rev,
        dims=["x", "time"],
        coords={"x":dists_rev, "time":dates, "lon":("x", lons_track_rev), "lat":("x", lats_track_rev)}
    )
    if units_in == "m":
        var2.coords["x"].attrs["units"] = "m"
    elif units_in == "km":
        var2.coords["x"].attrs["units"] = "km"
    return var2


def track_dist_asn_mssh(ds, var_str="mssh", units_in="m"):
    dists = compute_dist(ds, units=units_in)  # to convert to meters
    dists_rev = dists[::-1]
    var1 = ds[var_str]  # .count(dim="cycles_numbers")
    var1_rev = var1.isel(points_numbers=slice(None, None, -1)).values
    var2 = xr.DataArray(
        var1_rev,
        coords=[dists_rev],
        dims=["x"],
    )
    if units_in == "m":
        var2.coords["x"].attrs["units"] = "m"
    elif units_in == "km":
        var2.coords["x"].attrs["units"] = "km"
    return var2


def track_dist_asn_mdt(ds, var_str="mdt_cnes_cls22", units_in="m"):
    dists = compute_dist(ds, units=units_in)  # to convert to meters
    dists_rev = dists[::-1]
    var1 = ds[var_str]  # .count(dim="cycles_numbers")
    var1_rev = var1.isel(points_numbers=slice(None, None, -1)).values
    var2 = xr.DataArray(
        var1_rev,
        coords=[dists_rev],
        dims=["x"],
    )
    if units_in == "m":
        var2.coords["x"].attrs["units"] = "m"
    elif units_in == "km":
        var2.coords["x"].attrs["units"] = "km"
    return var2


# META INFO FUNCTIONS
def get_first_file(sat):
    return sorted(
        glob.glob(
            f"/home/srinivasu/xtrackm/data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    )[0]


def get_first_file_a(sat):
    return sorted(
        glob.glob(
            f"/home/srinivasu/xtrackm/extracted_aviso/aviso.sla.ref.{sat}.nindian.*.nc"
        )
    )[0]


def get_total_tracks(sat):
    return len(
        glob.glob(
            f"/home/srinivasu/xtrackm/data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    )


def get_time_limits(sat):
    f = get_first_file(sat)
    ds = xr.open_dataset(f, engine="netcdf4")
    # ds = xr.open_dataset(f)
```

```
    time = ds.time.isel(points_numbers=0).values
    first_time = time[0]
    last_time = time[-1]
    # Convert the first and last time stamps to string format
    first_time_str = first_time.strftime("%Y-%m-%d")
    last_time_str = last_time.strftime("%Y-%m-%d")
    return first_time_str, last_time_str


def get_time_limits_o(sat):
    f = get_first_file(sat)
    ds = xr.open_dataset(f, engine="netcdf4")
    # ds = xr.open_dataset(f)
    time = ds.time.isel(points_numbers=0).values
    # these are cftime objects which are not very useful
    first_time = time[0]
    last_time = time[-1]
    # print(type(first_time), type(last_time))
    # Convert the first and last time stamps to string format
    first_time_str = first_time.strftime("%Y-%m-%d")
    last_time_str = last_time.strftime("%Y-%m-%d")
    # first_time_str = datetime.strptime(first_time, "%Y-%m-%d")
    # last_time_str = datetime.strptime(last_time, "%Y-%m-%d")
    # print(type(first_time_str), type(last_time_str))
    track_tsta_o = datetime.strptime(first_time_str, "%Y-%m-%d")
    track_tend_o = datetime.strptime(last_time_str, "%Y-%m-%d")
    return track_tsta_o, track_tend_o


def get_time_limits_np64(sat):
    f = get_first_file(sat)
    ds = xr.open_dataset(f, engine="netcdf4")
    time = ds.time.isel(points_numbers=0).values
    first_time = time[0]
    last_time = time[-1]
    first_time_str = first_time.strftime("%Y-%m-%d")
    last_time_str = last_time.strftime("%Y-%m-%d")
    track_tsta_o = np.datetime64(first_time_str, "s")
    track_tend_o = np.datetime64(last_time_str, "s")
    return track_tsta_o, track_tend_o


def get_drifter_time_limits_np64(fd):
    ds_d = xr.open_dataset(fd, drop_variables=["WMO"])
    drift_tsta_o1, drift_tend_o1 = (
        ds_d.start_date.values[0],
        ds_d.end_date.values[0],
    )
    drifter_tsta_o = np.datetime64(drift_tsta_o1, "s")
    drifter_tend_o = np.datetime64(drift_tend_o1, "s")
    return drifter_tsta_o, drifter_tend_o


def change_time(ds1, time_str):
    dvals = ds1[time_str].values
    splitted = ds1[time_str].units.split(" ")
    if len(splitted) == 3:
        frmt = "%Y-%m-%d"
        # rftime is reference time like since 2004-01-01
        rftime = splitted[2]
    elif len(splitted) == 4:
        frmt = "%Y-%m-%d %H:%M:%S"
        rftime = splitted[2] + " " + splitted[3]
    # dt0 is refence date object
    dt0 = datetime.strptime(rftime, frmt)
    units1 = splitted[0]
    if units1 == "months":
        dates = [dt0 + relativedelta(months=int(d)) for d in dvals]
    elif units1 == "days":
        dates = [dt0 + timedelta(days=int(d)) for d in dvals]
    elif units1 == "hours":
        dates = [dt0 + timedelta(hours=int(d)) for d in dvals]
    elif units1 == "minutes":
        dates = [dt0 + timedelta(minutes=int(d)) for d in dvals]
    elif units1 == "seconds":
        dates = [dt0 + timedelta(seconds=int(d)) for d in dvals]
    ds1[time_str] = dates
    ds1 = ds1.rename({time_str: "time"})
    return ds1


# TIDE GAUGE FUNCTIONS
# @timeit_decorator
def read_tide_meta(data_loc="/home/srinivasu/slnew/psmsl/rlr_monthly"):
    df = pd.read_csv(
        f"{data_loc}/filelist.txt",
        sep=";",
        names=[
            "id",
            "latitude",
            "longitude",
            "name",
            "coastline",
            "stationcode",
            "stationflag",
        ],
        header=None,
    )
    df = df.set_index("id")
    return df


# Filter DataFrame based on bounds
# @timeit_decorator
def region_selected(df, lon_min, lon_max, lat_min, lat_max):
    filtered_df = df[
        (df["latitude"] >= lat_min)
        & (df["latitude"] <= lat_max)
        & (df["longitude"] >= lon_min)
        & (df["longitude"] <= lon_max)
    ]
    return filtered_df


def get_lonlat_psmsl(PSMSL_ID, df_nio):
    row = df_nio.loc[PSMSL_ID]
    lon_psmsl = row["longitude"]
```

```
    lat_psmsl = row["latitude"]
    name_psmsl = row["name"]
    return lon_psmsl, lat_psmsl, name_psmsl


def read_id(id1, data_loc="/home/srinivasu/slnew/psmsl/rlr_monthly/data/"):
    def convert_partial_year(number):
        # https://stackoverflow.com/questions/19305991/convert-fractional-years-to-a-real-date-in-python
        # TODO this function need a revisit
        year = int(float(number))
        d = timedelta(days=(float(number) - year) * 365.24)
        day_one = datetime(year, 1, 1)
        date = d + day_one
        return date

    df = pd.read_csv(
        f"{data_loc}/{id1}.rlrdata",
        delimiter=";",
        names=["time", "height", "a1", "a2"],
        na_values=-99999,
        parse_dates=[0],
        date_parser=convert_partial_year,
    )
    # df.set_index("time", inplace=True)
    df = df.set_index("time")
    df = df.to_xarray()
    return df


def read_id1(id1):
    # def convert_to_datetime(year_dec):
    #     # https://github.com/astg606/py_materials/blob/master/pandas/introduction_pandas.ipynb
    #     year_int = int(float(year_dec))
    #     base = datetime(year_int, 1, 1)
    #     rem = float(year_dec) - year_int
    #     result = base + timedelta(
    #         seconds=(base.replace(year=base.year + 1) - base).total_seconds() * rem
    #     )
    #     # print(result.strftime("%y-%m-%d"))
    #     return result
    def convert_to_datetime(year_dec):
        # https://github.com/astg606/py_materials/blob/master/pandas/introduction_pandas.ipynb
        year_int = int(float(year_dec))
        base = pd.Timestamp(year_int, 1, 1)
        days = (float(year_dec) - year_int) * 365.24
        result = base + pd.Timedelta(days=days)
        return result

    df = pd.read_csv(
        f"/home/srinivasu/slnew/psmsl/rlr_monthly/data/{id1}.rlrdata",
        delimiter=";",
        names=["time", "height", "a1", "a2"],
        na_values=-99999,
    )
    df["time"] = df["time"].apply(convert_to_datetime)
    df.set_index("time", inplace=True)
    df = df.to_xarray()
    return df


def read_id_ai(id1, data_loc="/home/srinivasu/slnew/psmsl/rlr_monthly/data/"):
    def convert_partial_year(number):
        try:
            number = float(number)
            year = int(number)
            frac = number - year
            # Derive month (1-12) from fraction
            month = int(frac * 12) + 1
            # Clamp month to 1-12 in case of floating-point edge cases
            month = max(1, min(12, month))
            # Get days in that month (handles leap years)
            _, days_in_month = calendar.monthrange(year, month)
            # Calculate mid-month day (e.g., 15 for Feb, 16 for Jan/Mar)
            mid_day = (days_in_month // 2) + 1
            # Create datetime (will raise ValueError if invalid, e.g., Feb 30)
            return datetime(year, month, mid_day)
        except (ValueError, TypeError) as e:
            raise ValueError(f"Invalid fractional year '{number}': {e}")

    df = pd.read_csv(
        f"{data_loc}/{id1}.rlrdata",
        delimiter=";",
        names=["time", "height", "a1", "a2"],
        na_values=-99999,
        parse_dates=[0],
        date_parser=convert_partial_year,
    )
    df = df.set_index("time")  # Avoid inplace=True
    df = df.to_xarray()
    return df


def create_topo_map(dse,
                    xsta=30,
                    xend=120,
                    ysta=-30,
                    yend=30,
                    title1="",
                    step=5):
    lon_range = xend - xsta
    lat_range = yend - ysta
    aspect_ratio = lon_range / lat_range if lat_range != 0 else 1
    base_height = 8
    fig_width = round(aspect_ratio * base_height)
    fig_height = round(base_height)
    fig, ax1 = plt.subplots(
        1,
        1,
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(fig_width, fig_height),
    )
    fig.tight_layout(pad=1.7, h_pad=1.7, w_pad=1.2)
    dse.rose.sel(lon=slice(xsta, xend)).sel(lat=slice(ysta, yend)).plot(
        ax=ax1, add_colorbar=False, add_labels=False)
    ax1.set_extent([xsta, xend, ysta, yend], crs=ccrs.PlateCarree())
    ax1.add_feature(cfeature.COASTLINE)
    xticks = list(range(int(xsta), int(xend), step))
```

```
    yticks = list(range(int(ysta), int(yend), step))
    ax1.axhline(y=0.0, linestyle="--", color="k")
    ax1.set_xticks(xticks, crs=ccrs.PlateCarree())
    ax1.set_yticks(yticks, crs=ccrs.PlateCarree())
    ax1.set_title(title1)
    ax1.xaxis.set_major_formatter(LongitudeFormatter())
    ax1.yaxis.set_major_formatter(LatitudeFormatter())
    return fig, ax1


def create_topo_map_etopo(dse,
                          xsta=30,
                          xend=120,
                          ysta=-30,
                          yend=30,
                          title1="",
                          step=5):
    lon_range = xend - xsta
    lat_range = yend - ysta
    aspect_ratio = lon_range / lat_range if lat_range != 0 else 1
    base_height = 8
    fig_width = round(aspect_ratio * base_height)
    fig_height = round(base_height)
    fig, ax1 = plt.subplots(
        1,
        1,
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(fig_width, fig_height),
    )
    fig.tight_layout(pad=1.7, h_pad=1.7, w_pad=1.2)
    dse.ROSE.sel(ETOPO05_X=slice(xsta, xend)).sel(ETOPO05_Y=slice(ysta, yend)).plot(
        ax=ax1, add_colorbar=False, add_labels=False)
    ax1.set_extent([xsta, xend, ysta, yend], crs=ccrs.PlateCarree())
    ax1.add_feature(cfeature.COASTLINE)
    xticks = list(range(int(xsta), int(xend), step))
    yticks = list(range(int(ysta), int(yend), step))
    ax1.axhline(y=0.0, linestyle="--", color="k")
    ax1.set_xticks(xticks, crs=ccrs.PlateCarree())
    ax1.set_yticks(yticks, crs=ccrs.PlateCarree())
    ax1.set_title(title1)
    ax1.xaxis.set_major_formatter(LongitudeFormatter())
    ax1.yaxis.set_major_formatter(LatitudeFormatter())
    return fig, ax1


def plot_etopo_subplot(fig, position, tracks_reg, cmap, title="", step=2,
                       data_path="~/allData/topo/etopo5.cdf"):
    """
    Create a Cartopy subplot with ETOPO bathymetry data.
    Parameters:
        fig       : matplotlib Figure object
        position  : subplot position index (e.g., 311, 312, 313)
        tracks_reg : (xsta, xend, ysta, yend) region tuple
        cmap      : matplotlib colormap
        title     : subplot title
        step      : tick step interval
        data_path  : path to the ETOPO NetCDF file
    """
    # Load ETOPO dataset
    dse = xr.open_dataset(data_path)
    da = dse.ROSE
    # Extract region
    xsta, xend, ysta, yend = tracks_reg
    da_reg = da.sel(ETOPO05_X=slice(xsta, xend), ETOPO05_Y=slice(ysta, yend))
    # Create subplot with Cartopy
    ax = fig.add_subplot(position, projection=ccrs.PlateCarree())
    # Plot data
    da_reg.plot(ax=ax, cmap=cmap, add_colorbar=False, add_labels=False)
    # Decorate map
    ax.add_feature(cfeature.COASTLINE, linewidth=2.0)
    xticks = list(range(int(xsta), int(xend), step))
    yticks = list(range(int(ysta), int(yend), step))
    ax.axhline(y=0.0, linestyle="--", color="k")
    ax.set_xticks(xticks, crs=ccrs.PlateCarree())
    ax.set_yticks(yticks, crs=ccrs.PlateCarree())
    ax.set_title(title)
    ax.xaxis.set_major_formatter(LongitudeFormatter())
    ax.yaxis.set_major_formatter(LatitudeFormatter())
    ax.set_extent([xsta, xend, ysta, yend], crs=ccrs.PlateCarree())
    return ax

def decorate_axis(ax, title1="", xsta=60, xend=100, ysta=-5, yend=25, step=2):
    ax.add_feature(cfeature.COASTLINE, linewidth=2.0)
    xticks = list(range(int(xsta), int(xend), step))
    yticks = list(range(int(ysta), int(yend), step))
    ax.axhline(y=0.0, linestyle="--", color="k")
    ax.set_xticks(xticks, crs=ccrs.PlateCarree())
    ax.set_yticks(yticks, crs=ccrs.PlateCarree())
    ax.set_title(title1)
    ax.xaxis.set_major_formatter(LongitudeFormatter())
    ax.yaxis.set_major_formatter(LatitudeFormatter())
    ax.set_extent([xsta, xend, ysta, yend], crs=ccrs.PlateCarree())


@timeit_decorator
def omni_nearby_track(omni_id, sat):
    # https://en.wikipedia.org/wiki/Distance_from_a_point_to_a_line
    dists = []
    index_of_omni = omni_buoys.index(omni_id)
    x0, y0 = lons_omni[index_of_omni], lats_omni[index_of_omni]
    for f in sorted(
        glob.glob(
            f"/home/srinivasu/xtrackm/data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False, engine="netcdf4")
        # ds = xr.open_dataset(f, decode_times=False)
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            continue
        m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
        c = (lats_track[0]) - m * (lons_track[0])
        denom = np.sqrt(m**2 + 1)
        val = m * x0 - y0 + c
        # denom = (
```

```
        #     (lon_track_0 - lon_track_m1) ** 2
        #     + (lat_track_0 - lat_track_m1) ** 2
        # ) ** 0.5
        # val = (lon_track_0 - lon_track_m1) * (lat_track_m1 - y0) - (
        #     lat_track_0 - lat_track_m1
        # ) * (lon_track_m1 - x0)
        dist_deg = abs(val) / denom
        dist = degrees2kilometers(dist_deg, y0)
        if dist > 200:
            continue
        x1 = (x0 + m * y0 - m * c) / (1 + m**2)
        y1 = m * x1 + c
        dists.append((track_number, x1, y1, dist))
    trackn1, x2, y2, mindist = min(dists, key=lambda x: x[3])
    return trackn1, x2, y2, mindist


@timeit_decorator
def rama_nearby_track(rama_id, sat):
    # https://en.wikipedia.org/wiki/Distance_from_a_point_to_a_line
    dists = []
    x0, y0 = rama_d[rama_id]
    for f in sorted(
        glob.glob(
            f"/home/srinivasu/xtrackm/data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False, engine="netcdf4")
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            continue
        m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
        c = (lats_track[0]) - m * (lons_track[0])
        denom = np.sqrt(m**2 + 1)
        val = m * x0 - y0 + c
        dist_deg = abs(val) / denom
        dist = degrees2kilometers(dist_deg, y0)
        if dist > 200:
            continue
        x1 = (x0 + m * y0 - m * c) / (1 + m**2)
        y1 = m * x1 + c
        dists.append((track_number, x1, y1, dist))
    if len(dists) == 0:
        return None, None, None, None
    trackn1, x2, y2, mindist = min(dists, key=lambda x: x[3])
    return trackn1, x2, y2, mindist


@timeit_decorator
def rama_nearby_track_box(rama_id, sat, box_size=2.0):
    x0, y0 = rama_d[rama_id]
    polygon = create_box_patch(x0, y0, box_size)
    list_tuples = []
    for f in sorted(
        glob.glob(
            f"/home/srinivasu/xtrackm/data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False, engine="netcdf4")
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            continue
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        line_start = (lons_track[0], lats_track[0])
        line_end = (lons_track[-1], lats_track[-1])
        track_path = sg.LineString([
            (lon, lat) for lon, lat in zip(lons_track, lats_track)
        ])
        if polygon.intersects(track_path):
            # print(track_number)
            distances = [
                distance.distance((y0, x0), (y, x)).km
                for y, x in zip(lats_track, lons_track)
            ]
            # print(distances)
            minindex = np.nanargmin(distances)
            x1 = lons_track[minindex]
            y1 = lats_track[minindex]
            mindist = distances[minindex]
            if mindist > 200:
                continue
            # print(rama_id, sat, track_number, x1, y1, mindist)
            list_tuples.append((sat, track_number, x1, y1, mindist, rama_id))
    if len(list_tuples) == 0:
        return [(None, None, None, None, None, None)]
    sorted_list = sorted(list_tuples, key=lambda x: x[4])
    return sorted_list


@timeit_decorator
def coastal_nearby_track_box(coastal_id, sat, box_size=2.0):
    x0, y0 = coastal_d[coastal_id]
    polygon = create_box_patch(x0, y0, box_size)
    list_tuples = []
    for f in sorted(
        glob.glob(
            f"/home/srinivasu/xtrackm/data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False, engine="netcdf4")
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            continue
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        line_start = (lons_track[0], lats_track[0])
        line_end = (lons_track[-1], lats_track[-1])
        track_path = sg.LineString([
            (lon, lat) for lon, lat in zip(lons_track, lats_track)
        ])
```

```
        if polygon.intersects(track_path):
            # print(track_number)
            distances = [
                distance.distance((y0, x0), (y, x)).km
                for y, x in zip(lats_track, lons_track)
            ]
            # print(distances)
            minindex = np.nanargmin(distances)
            x1 = lons_track[minindex]
            y1 = lats_track[minindex]
            mindist = distances[minindex]
            if mindist > 100:
                continue
            # print(rama_id, sat, track_number, x1, y1, mindist)
            list_tuples.append((sat, track_number, x1, y1, mindist, coastal_id))
    if len(list_tuples) == 0:
        return [(None, None, None, None, None, None)]
    sorted_list = sorted(list_tuples, key=lambda x: x[4])
    return sorted_list

@timeit_decorator
def omni_nearby_track_box(omni_id, sat, box_size=2.0):
    index_of_omni = omni_buoys.index(omni_id)
    x0, y0 = lons_omni[index_of_omni], lats_omni[index_of_omni]
    polygon = create_box_patch(x0, y0, box_size)
    list_tuples = []
    for f in sorted(
        glob.glob(
            f"/home/srinivasu/xtrackm/data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False, engine="netcdf4")
        # ds = xr.open_dataset(f, decode_times=False)
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            continue
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        line_start = (lons_track[0], lats_track[0])
        line_end = (lons_track[-1], lats_track[-1])
        track_path = sg.LineString([line_start, line_end])
        track_path = sg.LineString([
            (lon, lat) for lon, lat in zip(lons_track, lats_track)
        ])
        if polygon.intersects(track_path):
            # print(track_number)
            distances = [
                distance.distance((y0, x0), (y, x)).km
                for y, x in zip(lats_track, lons_track)
            ]
            # print(distances)
            minindex = np.nanargmin(distances)
            x1 = lons_track[minindex]
            y1 = lats_track[minindex]
            mindist = distances[minindex]
            if mindist > 200:
                continue
            # print(omni_id, sat, track_number, x1, y1, mindist)
            list_tuples.append((sat, track_number, x1, y1, mindist, omni_id))
    if len(list_tuples) == 0:
        return [(None, None, None, None, None, None)]
    sorted_list = sorted(list_tuples, key=lambda x: x[4])
    return sorted_list


def create_box_patch(lon0, lat0, half_width):
    # Define the coordinates of the corners of the box
    lon_min = lon0 - half_width
    lon_max = lon0 + half_width
    lat_min = lat0 - half_width
    lat_max = lat0 + half_width
    # Create a Polygon using these coordinates
    box = Polygon(
        [
            (lon_min, lat_min),
            (lon_min, lat_max),
            (lon_max, lat_max),
            (lon_max, lat_min),
            (lon_min, lat_min),
        ]
    )
    return box


@timeit_decorator
def cb_nearby_track_box(cb_id, sat, box_size=2.0):
    x0, y0 = cb_d[cb_id]
    polygon = create_box_patch(x0, y0, box_size)
    list_tuples = []
    for f in sorted(
        glob.glob(
            f"/home/srinivasu/xtrackm/data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False, engine="netcdf4")
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            continue
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        line_start = (lons_track[0], lats_track[0])
        line_end = (lons_track[-1], lats_track[-1])
        track_path = sg.LineString([
            (lon, lat) for lon, lat in zip(lons_track, lats_track)
        ])
        if polygon.intersects(track_path):
            # print(track_number)
            distances = [
                distance.distance((y0, x0), (y, x)).km
                for y, x in zip(lats_track, lons_track)
            ]
            # print(distances)
            minindex = np.nanargmin(distances)
```

```
            x1 = lons_track[minindex]
            y1 = lats_track[minindex]
            # print(minindex, x1, y1)
            # break
            mindist = distances[minindex]
            if mindist < 200:
                list_tuples.append((sat, track_number, x1, y1, mindist, cb_id))
    sorted_list = sorted(list_tuples, key=lambda x: x[3])
    return sorted_list


@timeit_decorator
def psmsl_nearby_track_box(psmsl_id, df_nio, sat, box_size=2.0):
    lon_psmsl = df_nio.loc[psmsl_id]["longitude"]
    lat_psmsl = df_nio.loc[psmsl_id]["latitude"]
    polygon = create_box_patch(lon_psmsl, lat_psmsl, box_size)
    list_tuples = []
    for f in sorted(
        glob.glob(
            f"/home/srinivasu/xtrackm/data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False, engine="netcdf4")
        # ds = xr.open_dataset(f, decode_times=False)
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            continue
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        line_start = (lons_track[0], lats_track[0])
        line_end = (lons_track[-1], lats_track[-1])
        track_path = sg.LineString([line_start, line_end])
        if polygon.intersects(track_path):
            # print(track_number)
            distances = [
                distance.distance((lat_psmsl, lon_psmsl), (y, x)).km
                for y, x in zip(lats_track, lons_track)
            ]
            # print(distances)
            minindex = np.nanargmin(distances)
            x1 = lons_track[minindex]
            y1 = lats_track[minindex]
            mindist = distances[minindex]
            # print(minindex, x1, y1)
            # break
            if mindist > 100:
                continue
            # print(psmsl_id, sat, track_number, x1, y1, mindist)
            list_tuples.append((sat, track_number, x1, y1, mindist, psmsl_id))
    if len(list_tuples) == 0:
        return [(None, None, None, None, None, None)]
    sorted_list = sorted(list_tuples, key=lambda x: x[4])
    return sorted_list


@timeit_decorator
def psmsl_nearby_track(psmsl_id, df_nio, sat):
    lon_psmsl = df_nio.loc[psmsl_id]["longitude"]
    lat_psmsl = df_nio.loc[psmsl_id]["latitude"]
    dists = []
    for f in sorted(
        glob.glob(
            f"/home/srinivasu/xtrackm/data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False, engine="netcdf4")
        # ds = xr.open_dataset(f, decode_times=False)
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            continue
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        # distance between
        dist = distance.distance(
            (lat_psmsl, lon_psmsl), (lat_coast, lon_coast)
        ).km
        if dist > 100.0:
            continue
        dists.append((track_number, lon_coast, lat_coast, dist))
    # return None if len of dists is zero
    if len(dists) == 0:
        return None, None, None, None
    trackn1, lon_coast1, lat_coast1, mindist = min(dists, key=lambda x: x[3])
    return trackn1, lon_coast1, lat_coast1, mindist


@timeit_decorator
def cb_nearby_track_new(cb_id, sat):
    # https://en.wikipedia.org/wiki/Distance_from_a_point_to_a_line
    dists = []
    x0, y0 = cb_d[cb_id]
    for f in sorted(
        glob.glob(
            f"/home/srinivasu/xtrackm/data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False, engine="netcdf4")
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        if len(lons_track) == 0:
            continue
        m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
        c = (lats_track[0]) - m * (lons_track[0])
        # distp = distance.distance((y0, x0), (lat_coast, lon_coast)).km
        # distn = degrees2kilometers(dist_deg)
        # dist1 = min(distn, distp)
        x1 = (x0 + m * y0 - m * c) / (1 + m**2)
        y1 = m * x1 + c
        if is_land(x1, y1):
            x1 = lon_coast
```

```
            y1 = lat_coast
            dist = distance.distance((y0, x0), (lat_coast, lon_coast)).km
        else:
            denom = np.sqrt(m**2 + 1)
            val = m * x0 - y0 + c
            dist_deg = abs(val) / denom
            dist = degrees2kilometers(dist_deg, y0)
        if dist > 200.0:
            continue
        dists.append((track_number, x1, y1, dist))
    if len(dists) == 0:
        return None, None, None, None, None, None, None, None
    if len(dists) == 1:
        return *dists[0], None, None, None, None
    sorted_dists = sorted(dists, key=lambda x: x[3])
    (trackn1, x2, y2, mindist1), (trackn2, x3, y3, mindist2) = sorted_dists[:2]
    return (trackn1, x2, y2, mindist1), (trackn2, x3, y3, mindist2)


@timeit_decorator
def nearby_tide_guage_normal(ds, df_nio):
    lons_psmsl = df_nio["longitude"].to_list()
    lats_psmsl = df_nio["latitude"].to_list()
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
    c = (lats_track[0]) - m * (lons_track[0])
    denom = np.sqrt(m**2 + 1)
    dists = []
    ln = len(df_nio)
    for i in range(ln):
        x0, y0 = lons_psmsl[i], lats_psmsl[i]
        id1 = df_nio["id"].iloc[i]
        val = m * x0 - y0 + c
        dist_deg = abs(val) / denom
        dist = degrees2kilometers(dist_deg, y0)
        if dist > 200:
            continue
        x1 = (x0 + m * y0 - m * c) / (1 + m**2)
        y1 = m * x1 + c
        dists.append((id1, x1, y1, dist))
    # id2, x2, y2, mindist = min(dists, key=lambda x: x[3])
    sorted_list = sorted(dists, key=lambda x: x[3])
    return sorted_list


@timeit_decorator
def cb_nearby_track_normal(cb_id, sat):
    # https://en.wikipedia.org/wiki/Distance_from_a_point_to_a_line
    dists = []
    x0, y0 = cb_d[cb_id]
    for f in sorted(
        glob.glob(
            f"/home/srinivasu/xtrackm/data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc"
        )
    ):
        ds = xr.open_dataset(f, decode_times=False, engine="netcdf4")
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            continue
        m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
        c = (lats_track[0]) - m * (lons_track[0])
        denom = np.sqrt(m**2 + 1)
        val = m * x0 - y0 + c
        dist = abs(val) / denom
        if dist > 2.0:
            continue
        x1 = (x0 + m * y0 - m * c) / (1 + m**2)
        y1 = m * x1 + c
        dists.append((track_number, x1, y1, dist))
    if len(dists) == 0:
        return None, None, None, None
    trackn1, x2, y2, mindist = min(dists, key=lambda x: x[3])
    return trackn1, x2, y2, mindist


@timeit_decorator
def nearby_tide_guage(ds, df_nio):
    """
    To find the nearest tide gauge id given the ds of a single track
    """
    lons_psmsl = df_nio["longitude"].to_list()
    lats_psmsl = df_nio["latitude"].to_list()
    ids = df_nio["id"].to_list()
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    dists = []
    ln = len(df_nio)
    for i in range(ln):
        id1 = ids[i]
        lon_psmsl, lat_psmsl = lons_psmsl[i], lats_psmsl[i]
        dist = distance.distance(
            (lat_psmsl, lon_psmsl), (lat_coast, lon_coast)
        ).km
        if dist > 200.0:
            continue
        dists.append((id1, lon_psmsl, lat_psmsl, dist))
    id2, x1, y1, mindist = min(dists, key=lambda x: x[3])
    return id2, x1, y1, mindist

```

./meta/

61 file(s)

```
1
2  that corresponds to a fixed distnce of 40km
3  l.fraction_of_distance_loess.py
4  if we take rossby radius of deformation as 75km, the fraction of points should be around 0.03 to 0.05 for most tracks and sats
5
6  Are the time units along points_numbers and cycles_numbers different?
7
8  How to say whether a track is descending or ascending?
9  Its relationship with the track number?
10
11 There are huge gaps in the data for TPN+J1N+J2N+J3N in time
12
13 Satellite: GFO ----------------
14 Processed 82 files across for GFO.
15 Average NaN Percentage: 21.71088%
16 Average Outlier Percentage (relative to valid data): 0.00078%
17 Satellite: TP+J1+J2+J3+S6A ----------------
18 Processed 42 files across for TP+J1+J2+J3+S6A.
19 Average NaN Percentage: 7.43071%
20 Average Outlier Percentage (relative to valid data): 0.00000%
21 Satellite: S3A ----------------
22 Processed 125 files across for S3A.
23 Average NaN Percentage: 4.58146%
24 Average Outlier Percentage (relative to valid data): 0.00062%
25 Satellite: S3B ----------------
26 Processed 126 files across for S3B.
27 Average NaN Percentage: 4.18374%
28 Average Outlier Percentage (relative to valid data): 0.00115%
29 Satellite: TPN+J1N+J2N+J3N ----------------
30 Processed 44 files across for TPN+J1N+J2N+J3N.
31 Average NaN Percentage: 66.34210%
32 Average Outlier Percentage (relative to valid data): 0.00000%
33 Satellite: ERS1+ERS2+ENV+SRL ----------------
34 Processed 163 files across for ERS1+ERS2+ENV+SRL.
35 Average NaN Percentage: 23.85020%
36 Average Outlier Percentage (relative to valid data): 0.00043%
37 Satellite: HY2A ----------------
38 Processed 64 files across for HY2A.
39 Average NaN Percentage: 13.49256%
40 Average Outlier Percentage (relative to valid data): 0.01353%
41 Satellite: HY2B ----------------
42 Processed 64 files across for HY2B.
43 Average NaN Percentage: 30.35783%
44 Average Outlier Percentage (relative to valid data): 0.00960%
45
46 Outliers as percentage of valid data is close to zero for almost all satellites.
47 NaN percentage for TPN+J1N+J2N+J3N is very huge. Why?
48
49 The number of outlies is close to zero, no of NaNs is around 4% for all
50 the tracks
51
52 saving lon,lats as .npz files can improve the speeds of all
53 program in this folder
54
55 one major gotcha is xarray opening or decoding files in loop is one of the reasons for the slow performance of many scripts
56
57 for a given satellite mission, for all tracks the starting and
58 ending dates are same.
59
60 l.closest_descending_track_in_time.py don't know what this script does
61
62 sorting of lons, lats should not be done in l.inter_track.dists_eq_sort.py
63
64 GFO 77, 78.5 166km
65 TP+J1+J2+J3+S6A 77.3, 80 300km
66 S3A 67, 67.9 100km
67
68 l.closest_descending_track_in_time.py not clear on what this script does
69
70 there is some problem with l.inter_track.dists_eq_sort.py
71 the inter track distance between negative slope and positive slope tracks should be same. But they are differing alot.
72
73 ds.cycles_numbers and ds.cycle are differing only in initial values,
74 both are just integer values
75
76 previous .pass_number is changed to .Pass attribute
77 .Mission attribute is added to ds which can be useful
78
```

meta/TODO.txt

```
1
2  [] the loess fit of number of good points near coast looks so flat?
3
4  [-] to understand dates intersection business of c.intersection_points.angles_map.py
5
```

meta/c.east_coast_track_intersections_xtrack.py

```python
import math
import pickle

import cartopy.crs as ccrs
import geopandas as gpd
import matplotlib.pyplot as plt
import xarray as xr
from tools_xtrackm import *

TOPO_DIR = "/home/srinivasu/allData/topo/"
TRACKS_REG = (43.0, 99, 0, 25.4)
EAST_COAST_REG = (75.0, 99, 0, 25.4)
dse = xr.open_dataset(f"{TOPO_DIR}/etopo_60s_io.nc")
fig, ax1 = create_topo_map(dse, *EAST_COAST_REG, step=5)

SAT = "S3A"
files = sorted(glob.glob(f"/home/srinivasu/xtrackm/data/{SAT}_lon_ordered/ctoh.sla.ref.{SAT}.nindian.*.nc"))
east_coast_tracks = []
for f in files:
    ds = xr.open_dataset(f, decode_times=False)
    track_number = ds.Pass
    sla = ds.sla
    sla1 = sla.isel(cycles_numbers=0)
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lon_eq = lons_track[0]
    ln = len(lons_track)
    middle_lon = lons_track[int(0.3 * ln)]
    middle_lat = lats_track[int(0.3 * ln)]
    mid_lon_i = np.argmin(np.abs(lons_track - middle_lon))
    mid_lat_i = np.argmin(np.abs(lats_track - middle_lat))
    slope = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
    a1 = local_angle(lons_track, lats_track, mid_lat_i, ioff=3, degrees=False)
    a1_d = math.degrees(a1)
    if slope < 0:
        ax1.scatter(lons_track, lats_track, color="k", s=1, alpha=0.5)
        ax1.text(middle_lon, middle_lat, track_number, fontsize=12, ha='center', va='center',
                 transform=ccrs.PlateCarree(), color='w', rotation=a1_d)
        if lon_eq > EAST_COAST_REG[0] and lon_eq < EAST_COAST_REG[1]:
            east_coast_tracks.append(track_number)

plt.savefig(f"east_coast_intersections_a_{SAT}_xtrack.png", dpi=300)
plt.show()
plt.close("all")

print(east_coast_tracks)
with open(f"east_coast_tracks_xtrack.pkl", "wb") as f:
    pickle.dump(east_coast_tracks, f)

```

```python
from glob import glob

import xarray as xr
from tools_xtrackm import *


def intersection_angles_rad(m1, m2):
    """Return both angles of intersection (acute and obtuse) in radians between lines of slopes m1 and m2."""
    if 1 + m1 * m2 == 0:
        return (math.pi / 2, math.pi / 2)  # Perpendicular lines
    tan_theta = abs((m1 - m2) / (1 + m1 * m2))
    theta = math.atan(tan_theta)  # Always gives acute angle
    # Second angle is the obtuse supplementary angle
    other_theta = math.pi - theta
    return (theta, other_theta)


for sat in sats_new:
    sats_out = []
    lons_inter = []
    lats_inter = []
    track_self = []
    track_other = []
    angle_acute = []
    angle_obtuse = []
    x_from_coast_self = []
    x_from_coast_other = []
    for f_self in sorted(
            glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")):
        ds_self = xr.open_dataset(f_self, decode_times=False)
        track_number_self = ds_self.Pass
        lons_track_self = ds_self.lon.values
        lats_track_self = ds_self.lat.values
        if len(lons_track_self) == 0:
            continue
        lon_equat_self = lons_track_self[0]
        lat_equat_self = lats_track_self[0]
        lon_coast_self = lons_track_self[-1]  # this on coast
        lat_coast_self = lats_track_self[-1]  # this on coast
        slope_self = (lat_coast_self - lat_equat_self) / (lon_coast_self -
                                                          lon_equat_self)
        if slope_self < 0:
            continue
        track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
        for f_other in sorted(
                glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")):
            # print(f)
            ds_other = xr.open_dataset(f_other,
                                       engine="h5netcdf",
                                       decode_times=False)
            track_number_other = ds_other.Pass
            lons_track_other = ds_other.lon.values
            lats_track_other = ds_other.lat.values
            if len(lons_track_other) == 0:
                continue
            lon_equat_other = lons_track_other[0]
            lat_equat_other = lats_track_other[0]
            lon_coast_other = lons_track_other[-1]
            lat_coast_other = lats_track_other[-1]
            slope_other = (lat_coast_other - lat_equat_other) / (
                lon_coast_other - lon_equat_other)
            if slope_other > 0:
                continue
            # define a linestring for the track
            track_path_other = sg.LineString(
                zip(lons_track_other, lats_track_other))
            if track_path_self.intersects(track_path_other):
                intersection_point = track_path_self.intersection(
                    track_path_other)
                acute, obtuse = intersection_angles_rad(
                    slope_self, slope_other)
                x_from_coast_self1 = distance.distance(
                    (lat_coast_self, lon_coast_self), (intersection_point.y,
                                                       intersection_point.x)).m
                x_from_coast_other1 = distance.distance(
                    (lat_coast_other, lon_coast_other), (intersection_point.y,
                                                         intersection_point.x)).m
                print(
                    sat,
                    track_number_self,
                    track_number_other,
                    intersection_point.x,
                    intersection_point.y,
                    x_from_coast_self1,
                    x_from_coast_other1,
                )
                if is_land(intersection_point.x, intersection_point.y):
                    continue
                sats_out.append(sat)
                track_other.append(track_number_other)
                track_self.append(track_number_self)
                lons_inter.append(intersection_point.x)
                lats_inter.append(intersection_point.y)
                x_from_coast_self.append(x_from_coast_self1)
                x_from_coast_other.append(x_from_coast_other1)
                angle_acute.append(acute)
                angle_obtuse.append(obtuse)

    df = pd.DataFrame({
        "sat": sats_out,
        "track_self": track_self,
        "track_other": track_other,
        "lons_inter": lons_inter,
        "lats_inter": lats_inter,
        "x_from_coast_self": x_from_coast_self,
        "x_from_coast_other": x_from_coast_other,
        "angle_acute": angle_acute,
        "angle_obtuse": angle_obtuse,
    })
    df.to_csv(f"tracks_intersections_{sat}_1.csv")

```

```python
from glob import glob

import xarray as xr
from tools_xtrackm import *

# Checks:
# 1. Whether lat[0] is the equator-end and lat[-1] is the coast-end
# 2. MultiPoint / non-Point intersection geometries

multipoint_cases = []  # (sat, f_self, f_other, geom_type, wkt_snippet)
order_issues = []      # (sat, file, lat[0], lat[-1], note)

for sat in sats_new:
    for f_self in sorted(
            glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")):
        print(f_self)
        ds_self = xr.open_dataset(f_self, decode_times=False, engine="netcdf4")
        lons_track_self = ds_self.lon.values
        lats_track_self = ds_self.lat.values

        if len(lons_track_self) == 0:
            continue

        # Check 1: equator/coast ordering
        # Assumption: lat[0] is closer to equator (smaller |lat|)
        #             lat[-1] is closer to coast   (larger  |lat|)
        lat_first = float(lats_track_self[0])
        lat_last  = float(lats_track_self[-1])
        if abs(lat_first) >= abs(lat_last):
            order_issues.append((
                sat, f_self,
                round(lat_first, 4), round(lat_last, 4),
                "lat[0] is NOT closer to equator than lat[-1]"
            ))

        slope_self = (lat_last - lat_first) / (
            float(lons_track_self[-1]) - float(lons_track_self[0]))
        if slope_self < 0:
            continue

        track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))

        for f_other in sorted(
                glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")):
            ds_other = xr.open_dataset(f_other, engine="netcdf4",
                                       decode_times=False)
            lons_track_other = ds_other.lon.values
            lats_track_other = ds_other.lat.values

            if len(lons_track_other) == 0:
                continue

            slope_other = (
                (float(lats_track_other[-1]) - float(lats_track_other[0])) /
                (float(lons_track_other[-1]) - float(lons_track_other[0])))
            if slope_other > 0:
                continue

            track_path_other = sg.LineString(
                zip(lons_track_other, lats_track_other))

            if track_path_self.intersects(track_path_other):
                geom = track_path_self.intersection(track_path_other)

                # Check 2: expect a single Point; flag anything else
                if geom.geom_type != "Point":
                    multipoint_cases.append((
                        sat,
                        f_self,
                        f_other,
                        geom.geom_type,
                        str(geom)[:150],
                    ))

# ── Report ──────────────────────────────────────────────────────────
print("=" * 70)
print(f"CHECK 1 — EQUATOR/COAST ORDER ISSUES  ({len(order_issues)} found)")
print("=" * 70)
for sat, f, la0, la1, note in order_issues:
    print(f"  sat={sat}  lat[0]={la0:8.4f}  lat[-1]={la1:8.4f}")
    print(f"  note : {note}")
    print(f"  file : {f}")
    print()

print("=" * 70)
print(f"CHECK 2 — NON-POINT INTERSECTIONS  ({len(multipoint_cases)} found)")
print("=" * 70)
for sat, fs, fo, gtype, wkt in multipoint_cases:
    print(f"  sat={sat}  geom_type={gtype}")
    print(f"  self : {fs}")
    print(f"  other: {fo}")
    print(f"  geom : {wkt}")
    print()

print("Done.")

```

```python
# rama location plotted only for reference
from glob import glob

import cartopy.crs as ccrs
import cmocean as cmo
import matplotlib.pyplot as plt
import xarray as xr
from tools_xtrackm import *

cmap = "Spectral"
cmap = "PuBuGn"
cmap = "YlGnBu"
cmap = "YlGn"
cmap_r = plt.cm.get_cmap(cmap)
cmap1 = cmo.cm.topo
cmap1 = cmo.cm.topo
cmap1 = cmo.cm.diff

height, width = 9, 14
d = 1.5

sats = ["ERS1+ERS2+ENV+SRL"]
sats = ["GFO"]
sat = sats[0]

lons_inter = []
lats_inter = []
track_self = []
track_other = []

rama_loc = "/home/srinivasu/allData/rama/"
var_str = "cur"
var_str1 = "cur"
dt0 = datetime.strptime("1950-01-01", "%Y-%m-%d")
rama_id = "15n90e"
lon_rama, lat_rama = rama_d[rama_id]

dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
fig, ax1 = plt.subplots(
    1,
    1,
    subplot_kw={"projection": ccrs.PlateCarree()},
    figsize=(width, height),
    layout="constrained",
)
dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
    *TRACKS_REG[2:])).plot(ax=ax1,
                           add_colorbar=False,
                           add_labels=False,
                           cmap=cmap1)
decorate_axis(ax1, "", *TRACKS_REG)
ax1.grid()
ax1.plot(
    lon_rama,
    lat_rama,
    c="y",
    marker="D",
    markersize=15,
)
# 15n90e 129.0 14.0

for f_self in sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
    ds_self = xr.open_dataset(f_self, decode_times=False)
    track_number_self = ds_self.Pass
    print(ds_self)
    sla = ds_self.sla
    lns = ds_self["sla"].sizes["points_numbers"]
    lns2 = lns // 2
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_eqaut_self = lons_track_self[0]  # this on equator
    lat_eqaut_self = lats_track_self[0]  # this on equator
    if len(lons_track_self) == 0:
        continue
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    ax1.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    # for plotting the pass number at a slight distance from the track
    # from the middle
    lonm, latm = get_point_at_distance(lon_eqaut_self, lat_eqaut_self,
                                       lon_coast_self,
                                       lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=10,
            rotation=angle_self,
            color="w",
        )
    if slope_self < 0:
        continue
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    for f_other in sorted(
            glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
        # print(f)
        ds_other = xr.open_dataset(f_other,
                                   engine="netcdf4",
                                   decode_times=False)
        if len(ds_other.points_numbers) == 0:
            continue
        track_number_other = ds_other.Pass
        lno = ds_other["sla"].sizes["points_numbers"]
        lno2 = lno // 2
        lons_track_other = ds_other.lon.values
        lats_track_other = ds_other.lat.values
        lon_equat_other = lons_track_other[0]
        lat_equat_other = lats_track_other[0]
```

```
        lon_coast_other = lons_track_other[-1]
        lat_coast_other = lats_track_other[-1]
        slope_other = (lat_coast_other - lat_equat_other) / (
            lon_coast_other - lon_equat_other)
        if slope_other > 0:
            continue
        # angle_r_other = math.atan(slope_other)
        # angle_d_other = angle_r_other * (180 / math.pi)
        # define a linestring for the track
        track_path_other = sg.LineString(
            zip(lons_track_other, lats_track_other))
        other_times = []
        if track_path_self.intersects(track_path_other):
            point = track_path_self.intersection(track_path_other)
            print(sat, track_number_self, track_number_other, point)
            x_from_coast_self = distance.distance((lat_coast_self,
                                                   lon_coast_self),
                                                  (point.y, point.x)).km
            x_from_coast_other = distance.distance((lat_coast_other,
                                                    lon_coast_other),
                                                   (point.y, point.x)).km
            # x_idx = index_at_x(x, x_from_coast)
            lat_idx_self = index_at_lat(ds_self, point.y)
            lat_idx_other = index_at_lat(ds_other, point.y)
            # dvals_other = ds_other.time.isel(points_numbers=ln2).values
            # dvals_self = ds_self.time.isel(points_numbers=lat_idx).values
            dvals_self = ds_self.time.isel(points_numbers=lns2).values
            # these dates change by weeks or months
            dates_self = [dt0 + timedelta(days=int(d)) for d in dvals_self]
            time0_self = dates_self[0]
            # dvals_other = ds_other.time.isel(points_numbers=lat_idx).values
            dvals_other = ds_other.time.isel(points_numbers=lno2).values
            dates_other = [dt0 + timedelta(days=int(d)) for d in dvals_other]
            time0_other = dates_other[0]
            dist_time = (time0_other - time0_self).days

            ax1.plot(
                point.x,
                point.y,
                c="k",
                marker="x",
                markersize=15,
            )
            other_times.append((track_number_other, abs(dist_time)))
            track_other.append(track_number_other)
            track_self.append(track_number_self)
            lons_inter.append(point.x)
            lats_inter.append(point.y)
        # plt.pause(0.1)
    break
    # if len(other_times) == 0:
        # continue
    # other_times.sort(key=lambda x: x[1], reverse=True)
plt.savefig(
    f"tracks_intersections_{sat}.png",
    dpi=300,
    bbox_inches="tight",
)
plt.show()

df = pd.DataFrame(
    {
        "track_self": track_self,
        "track_other": track_other,
        "lons_inter": lons_inter,
        "lats_inter": lats_inter,
    }
)
df.to_csv(f"tracks_intersections_{sat}1.csv")

plt.show()
plt.close("all")

```

```python
from glob import glob

import xarray as xr
from tools_xtrackm import *


def intersection_angles_rad(m1, m2):
    """Return both angles of intersection (acute and obtuse) in radians between lines of slopes m1 and m2."""
    if 1 + m1 * m2 == 0:
        return (math.pi / 2, math.pi / 2)  # Perpendicular lines
    tan_theta = abs((m1 - m2) / (1 + m1 * m2))
    theta = math.atan(tan_theta)  # Always gives acute angle
    # Second angle is the obtuse supplementary angle
    other_theta = math.pi - theta
    return (theta, other_theta)


lons_inter = []
lats_inter = []
track_self = []
track_other = []
angle_acute = []
angle_obtuse = []
x_from_coast_self = []
x_from_coast_other = []

sat = "S3A"
sat = "ERS1+ERS2+ENV+SRL"
sat = "TP+J1+J2+J3+S6A"

for f_self in sorted(
        glob.glob(
            f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
    ds_self = xr.open_dataset(f_self, decode_times=False)
    track_number_self = ds_self.Pass
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    if len(lons_track_self) == 0:
        continue
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    if slope_self < 0:
        continue
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    for f_other in sorted(
            glob.glob(
                f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
        # print(f)
        ds_other = xr.open_dataset(f_other,
                                   engine="h5netcdf",
                                   decode_times=False)
        track_number_other = ds_other.Pass
        lons_track_other = ds_other.lon.values
        lats_track_other = ds_other.lat.values
        lon_equat_other = lons_track_other[0]
        lat_equat_other = lats_track_other[0]
        lon_coast_other = lons_track_other[-1]
        lat_coast_other = lats_track_other[-1]
        slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                             lon_equat_other)
        if slope_other > 0:
            continue
        # define a linestring for the track
        track_path_other = sg.LineString(
            zip(lons_track_other, lats_track_other))
        if track_path_self.intersects(track_path_other):
            point = track_path_self.intersection(track_path_other)
            acute, obtuse = intersection_angles_rad(slope_self, slope_other)
            x_from_coast_self1 = distance.distance(
                (lat_coast_self, lon_coast_self), (point.y, point.x)).m
            x_from_coast_other1 = distance.distance(
                (lat_coast_other, lon_coast_other), (point.y, point.x)).m
            print(sat, track_number_self, track_number_other, point.x, point.y, x_from_coast_self1, x_from_coast_other1)
            if is_land(point.x, point.y):
                continue
            track_other.append(track_number_other)
            track_self.append(track_number_self)
            lons_inter.append(point.x)
            lats_inter.append(point.y)
            x_from_coast_self.append(x_from_coast_self1)
            x_from_coast_other.append(x_from_coast_other1)
            angle_acute.append(acute)
            angle_obtuse.append(obtuse)

df = pd.DataFrame({
    "track_self": track_self,
    "track_other": track_other,
    "lons_inter": lons_inter,
    "lats_inter": lats_inter,
    "x_from_coast_self": x_from_coast_self,
    "x_from_coast_other": x_from_coast_other,
    "angle_acute": angle_acute,
    "angle_obtuse": angle_obtuse
})
df.to_csv(f"tracks_intersections_{sat}_1.csv")

```

```python
import numpy as np
import xarray as xr
from scipy.stats import median_abs_deviation
import glob

def mad_outlier_detection_fixed(data, threshold=3.5, dim="cycles_numbers"):
    """
    MAD outlier detection using your actual dimension names.
    Parameters:
    -----------
    data : xarray.DataArray
        Input SLA data
    threshold : float
        MAD threshold (3.5 is commonly used in X-TRACK)
    dim : str
        Dimension along which to detect outliers ('cycles_numbers' or 'points_numbers')
    Returns:
    --------
    xarray.DataArray : Boolean mask where True indicates outliers
    """
    def mad_filter_1d(x, thresh):
        """Apply MAD filter to 1D array"""
        if len(x) < 3 or np.all(np.isnan(x)):
            return np.zeros_like(x, dtype=bool)
        # Remove NaNs for calculation
        valid_mask = ~np.isnan(x)
        if np.sum(valid_mask) < 3:
            return np.zeros_like(x, dtype=bool)
        valid_data = x[valid_mask]
        median = np.median(valid_data)
        mad = median_abs_deviation(valid_data, scale='normal')
        if mad == 0:
            # If MAD is 0, use modified approach
            mad = np.median(np.abs(valid_data - median)) * 1.4826
            if mad == 0:
                return np.zeros_like(x, dtype=bool)
        # Calculate modified Z-score
        modified_z_scores = 0.6745 * (x - median) / mad
        outliers = np.abs(modified_z_scores) > thresh
        return outliers
    # Apply MAD filter along specified dimension
    outlier_mask = xr.apply_ufunc(
        lambda x: mad_filter_1d(x, threshold),
        data,
        input_core_dims=[[dim]],
        output_core_dims=[[dim]],
        vectorize=True,
        dask="allowed",
        output_dtypes=[bool]
    )
    return outlier_mask

sat = "GFO"
passn = "008"
file1 = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{passn}.nc"
ds = xr.open_dataset(file1,decode_times=False)
# print(ds)
sla = ds["sla"]  # Assuming 'sla' is the variable name
# print(f"SLA data shape: {sla.shape}")
# print(f"SLA dimensions: {sla.dims}")
# Detect outliers along time dimension (cycles_numbers)
outlier_mask = mad_outlier_detection_fixed(sla, threshold=3.5, dim='cycles_numbers')
# Remove outliers
sla_filtered = sla.where(~outlier_mask)
# Print summary statistics
total_points = sla.size
valid_original = sla.count().values
outliers_detected = outlier_mask.sum().values
valid_filtered = sla_filtered.count().values
# if outliers_detected != 0:
if True:
    print(f"\n=== Outlier Detection Summary ===")
    print(f"Total data points: {total_points}")
    print(f"Originally valid: {valid_original} ({valid_original/total_points*100:.1f}%)")
    print(f"Outliers detected: {outliers_detected} ({outliers_detected/total_points*100:.1f}%)")
    print(f"Final valid points: {valid_filtered} ({valid_filtered/total_points*100:.1f}%)")

```

```python
import numpy as np
import xarray as xr
from scipy.stats import median_abs_deviation
import glob
from namesm import sats_new


def mad_outlier_detection_fixed(data, threshold=3.5, dim="cycles_numbers"):
    """
    MAD outlier detection using your actual dimension names.
    """
    def mad_filter_1d(x, thresh):
        if len(x) < 3 or np.all(np.isnan(x)):
            return np.zeros_like(x, dtype=bool)
        valid_mask = ~np.isnan(x)
        if np.sum(valid_mask) < 3:
            return np.zeros_like(x, dtype=bool)
        # size of valid data might be less than x depending on the
        # number of NaNs
        valid_data = x[valid_mask]
        median = np.median(valid_data)
        mad = median_abs_deviation(valid_data, scale='normal')
        if mad == 0:
            mad = np.median(np.abs(valid_data - median)) * 1.4826
            if mad == 0:
                return np.zeros_like(x, dtype=bool)
        modified_z_scores = 0.6745 * (x - median) / mad
        outliers = np.abs(modified_z_scores) > thresh
        # size of outliers is same as the size of x
        return outliers

    outlier_mask = xr.apply_ufunc(
        lambda x: mad_filter_1d(x, threshold),
        data,
        input_core_dims=[[dim]],
        output_core_dims=[[dim]],
        vectorize=True,
        dask="allowed",
        output_dtypes=[bool]
    )
    return outlier_mask


for sat in sats_new:
    total_size = 0      # Total size (n_orig1)
    total_initial_nans = 0
    total_initial_valid = 0
    # Outliers are numerical values and counted over valid values
    total_outliers = 0
    file_count = 0
    file_list = sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc"))
    for f in file_list:
        ds = xr.open_dataset(f)
        sla = ds["sla"]
        outlier_mask = mad_outlier_detection_fixed(sla, threshold=3.5, dim='cycles_numbers')
        n_orig1 = sla.size                          # Total size
        n_valid = int(sla.count().values)           # Non-NaNs before filtering
        n_nans = n_orig1 - n_valid                  # Initial NaNs
        n_outliers = int(outlier_mask.sum().values) # Outliers found in n_valid
        # 3. Accumulate
        total_size += n_orig1
        total_initial_nans += n_nans
        total_initial_valid += n_valid
        total_outliers += n_outliers
        file_count += 1
        ds.close() # Good practice for large batches
    # --- Final Statistics ---
    print(f"Satellite: {sat} ----------------")
    if total_size > 0:
        avg_nan_perc = (total_initial_nans / total_size) * 100
        # Outlier percentage is typically calculated relative to the valid data points
        avg_outlier_perc = (total_outliers / total_initial_valid) * 100 if total_initial_valid > 0 else 0
        print(f"Processed {file_count} files across for {sat}.")
        print(f"Average NaN Percentage: {avg_nan_perc:.5f}%")
        print(f"Average Outlier Percentage (relative to valid data): {avg_outlier_perc:.5f}%")
    else:
        print("No files found.")

```

```python
import numpy as np
import xarray as xr
from scipy.stats import median_abs_deviation
import glob
from namesm import sats_new

def mad_outlier_detection_fixed(data, threshold=3.5, dim="cycles_numbers"):
    """
    MAD outlier detection using your actual dimension names.
    Parameters:
    -----------
    data : xarray.DataArray
        Input SLA data
    threshold : float
        MAD threshold (3.5 is commonly used in X-TRACK)
    dim : str
        Dimension along which to detect outliers ('cycles_numbers' or 'points_numbers')
    Returns:
    --------
    xarray.DataArray : Boolean mask where True indicates outliers
    """
    def mad_filter_1d(x, thresh):
        """Apply MAD filter to 1D array"""
        if len(x) < 3 or np.all(np.isnan(x)):
            return np.zeros_like(x, dtype=bool)
        # Remove NaNs for calculation
        valid_mask = ~np.isnan(x)
        if np.sum(valid_mask) < 3:
            return np.zeros_like(x, dtype=bool)
        valid_data = x[valid_mask]
        median = np.median(valid_data)
        mad = median_abs_deviation(valid_data, scale='normal')
        if mad == 0:
            # If MAD is 0, use modified approach
            mad = np.median(np.abs(valid_data - median)) * 1.4826
            if mad == 0:
                return np.zeros_like(x, dtype=bool)
        # Calculate modified Z-score
        modified_z_scores = 0.6745 * (x - median) / mad
        outliers = np.abs(modified_z_scores) > thresh
        return outliers
    # Apply MAD filter along specified dimension
    outlier_mask = xr.apply_ufunc(
        lambda x: mad_filter_1d(x, threshold),
        data,
        input_core_dims=[[dim]],
        output_core_dims=[[dim]],
        vectorize=True,
        dask="allowed",
        output_dtypes=[bool]
    )
    return outlier_mask


def load_and_filter_sla_data_corrected(file_path):
    """
    Load and filter SLA data using correct dimension names.
    """
    # Load your netCDF file
    ds = xr.open_dataset(file_path)
    sla = ds["sla"]  # Assuming 'sla' is the variable name
    # print(f"SLA data shape: {sla.shape}")
    # print(f"SLA dimensions: {sla.dims}")
    # Detect outliers along time dimension (cycles_numbers)
    outlier_mask = mad_outlier_detection_fixed(sla, threshold=3.5, dim='cycles_numbers')
    # Remove outliers
    sla_filtered = sla.where(~outlier_mask)
    # Print summary statistics
    total_points = sla.size
    valid_original = sla.count().values
    outliers_detected = outlier_mask.sum().values
    valid_filtered = sla_filtered.count().values
    if outliers_detected != 0:
        print(f"\n=== Outlier Detection Summary ===")
        print(f"Total data points: {total_points}")
        print(f"Originally valid: {valid_original} ({valid_original/total_points*100:.1f}%)")
        print(f"Outliers detected: {outliers_detected} ({outliers_detected/total_points*100:.1f}%)")
        print(f"Final valid points: {valid_filtered} ({valid_filtered/total_points*100:.1f}%)")
    return sla_filtered, outlier_mask

max_perc = []
max_valid= []

found = False
for sat in sats_new:
    for f in sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
        ds = xr.open_dataset(f)
        sla = ds["sla"]  # Assuming 'sla' is the variable name
        # Detect outliers along time dimension (cycles_numbers)
        outlier_mask = mad_outlier_detection_fixed(sla, threshold=3.5, dim='cycles_numbers')
        # Remove outliers
        sla_filtered = sla.where(~outlier_mask)
        # Print summary statistics
        n_orig1 = sla.size
        n_valid = sla.count().values # Without nans
        n_nans1 = n_orig1 - n_valid

        valid_filtered = sla_filtered.count().values
        n_outliers = outlier_mask.sum().values

        perc_outliers = n_outliers/valid_filtered*100
        if n_outliers != 0:
            found = True
            break
    if found:
        break

#         if outliers_detected != 0:
#             print(f"\n=== Outlier Detection Summary ===")
#             print(f"Total data points: {total_points}")
#             print(f"Originally valid: {valid_original} ({valid_original/total_points*100:.1f}%)")
#             print(f"Outliers detected: {outliers_detected} ({outliers_detected/total_points*100:.1f}%)")
#             print(f"Final valid points: {valid_filtered} ({valid_filtered/total_points*100:.1f}%)")
#             sla_filtered, outlier_mask = load_and_filter_sla_data_corrected(f)
#             outliers_detected = outlier_mask.sum().values
#             valid_filtered = sla_filtered.count().values
#             perc = outliers_detected/valid_filtered*100
```

```
118 #             valid_perc = valid_
119 #             max_perc.append(perc)
120 #             max_valid.append(valid_filtered)
121 #
122 # print(max_perc)
123 # print(max_valid)
124
```

```python
import glob
import sys

import numpy as np
import xarray as xr
from loess.loess_1d import loess_1d
from scipy.signal import savgol_filter
from tools_xtrackm import sats_new, get_first_file, get_total_tracks
import matplotlib.pyplot as plt

# make a function to write even numbers

frac1 = 0.05
# check if argument is supplied
if len(sys.argv) > 1:
    frac1 = float(sys.argv[1])
else:
    frac1 = 0.05

print(f"loess frac = {frac1}")

j = 0
for sat in sats_new[:]:
    i = 0
    # we are making a huge pile of gshhg distances across all tracks
    gshhg_np = np.empty(0)
    counts_np = np.empty(0)
    no_tracks = get_total_tracks(sat)
    for f in sorted(glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")):
        ds = xr.open_dataset(f, decode_times=False)
        gshhg = 0.001 * ds.dist_to_coast_gshhg.values
        counts = ds.sla.count(dim="cycles_numbers").values
        count_perc = 100 * counts / len(ds.cycles_numbers)
        gshhg_np = np.append(gshhg_np, gshhg)
        counts_np = np.append(counts_np, count_perc)
        i = i + 1
    print(len(gshhg_np), i)
    # plot the histogram of gshhg_np
    # plot the diff of gshhg_np
    gshhg_np_diff = np.diff(gshhg_np)
    # gshhg_np_diff peaks to hundreds of km near where two tracks are appended
    masked = np.where(gshhg_np_diff < 20, gshhg_np_diff, np.nan)
    # becuase some tracks are ascending and some are descending, the diff is negative
    # half of the times.
    masked_abs = np.abs(masked)
    # fig, ax = plt.subplots()
    # ax.hist(gshhg_np_diff, bins=100)
    # plt.show()
    # To get whole pile of distances to be sampled at equal intervals
    # We sort and smooth all distance values
    # these intervals are then used as output x-axis of loess filter
    gshhg_np_sorted = np.sort(gshhg_np)
    smoothed_array = savgol_filter(
        gshhg_np_sorted, window_length=51, polyorder=3
    )
    # since the gshhg_np is the result of no_tracks of appends
    # we get the average number of points (along x) for typical tracks.
    xnew = smoothed_array[::no_tracks]
    xout, yout, wout = loess_1d(
        gshhg_np,
        counts_np,
        xnew=xnew,
        degree=1,
        frac=frac1,
        npoints=None,
        rotate=False,
        sigy=None,
    )
    da_out = xr.DataArray(yout, coords={"gshhg_resampled": xout})
    ds_out = xr.Dataset({"valid_count": da_out})
    ds_out.to_netcdf(f"xtrack_sla_valid_count_{sat}_loess_fit.nc")
    j = j + 1

```

```python
import glob
import sys

import numpy as np
import xarray as xr
from loess.loess_1d import loess_1d
from scipy.signal import savgol_filter
from tools_xtrackm import sats_new, get_first_file, get_total_tracks
import matplotlib.pyplot as plt

frac1 = 0.05
# check if argument is supplied
if len(sys.argv) > 1:
    frac1 = float(sys.argv[1])
else:
    frac1 = 0.05

print(f"loess frac = {frac1}")

part1 = np.arange(0, 20, 1)

j = 0
for sat in sats_new[:]:
    i = 0
    # we are making a huge pile of gshhg distances across all tracks
    gshhg_np = np.empty(0)
    counts_np = np.empty(0)
    for f in sorted(glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")):
        ds = xr.open_dataset(f, decode_times=False)
        gshhg = 0.001 * ds.dist_to_coast_gshhg.values
        counts = ds.sla.count(dim="cycles_numbers").values
        count_perc = 100 * counts / len(ds.cycles_numbers)
        gshhg_np = np.append(gshhg_np, gshhg)
        counts_np = np.append(counts_np, count_perc)
        i = i + 1
    print(len(gshhg_np), i)
    max_dist = max(gshhg_np)
    # Next values with step 6, starting from 20
    # Example: generate 10 more values
    part2 = np.arange(20, max_dist, 6)
    # Combine both parts
    xnew = np.concatenate([part1, part2])
    xout, yout, wout = loess_1d(
        gshhg_np,
        counts_np,
        xnew=xnew,
        degree=1,
        frac=frac1,
        npoints=None,
        rotate=False,
        sigy=None,
    )
    da_out = xr.DataArray(yout, coords={"standard_6km": xout})
    ds_out = xr.Dataset({"valid_count": da_out})
    ds_out.to_netcdf(f"xtrack_sla_valid_count_{sat}_loess_fit_6km.nc")
    j = j + 1

```

```
1 htop command is a interactive process viewer
2
```

```
1
2  l.inter_track.dists_eq_sort.py
3
```

meta/data_coverage_basic.py

```python
import datetime

import pandas as pd
import xarray as xr
from tools_xtrackm import *
from geopy import distance
from namesm import *

sats = sats_new

df = pd.DataFrame(
    columns=["sat", "trackn", "first_date", "last_date", "tfreq"]
)

for sat in sats_new:
    f = get_first_file(sat)
    trackn = get_total_tracks(sat)
    ds = xr.open_dataset(f)
    track_number = ds.Pass
    time = ds.time.isel(points_numbers=0)
    time1 = time.values
    first_time = time1[0]
    last_time = time1[-1]
    first_time_str = first_time.strftime("%Y-%m-%d")
    last_time_str = last_time.strftime("%Y-%m-%d")
    time_datetime = [
        datetime(year=t.year, month=t.month, day=t.day) for t in time1
    ]
    tfreq = xr.infer_freq(time_datetime)
    print(f"{sat} {trackn} {first_time_str} {last_time_str} {tfreq}")
    df = df._append(
        {
            "sat": sat,
            "trackn": trackn,
            "first_date": first_time_str,
            "last_date": last_time_str,
            "tfreq": tfreq,
        },
        ignore_index=True,
    )

print(df)

```

meta/data_coverage_df.py

```python
import datetime
from collections import Counter

import xarray as xr
from tools_xtrackm import *

df = pd.DataFrame(
    columns=["sat", "trackn", "first_date", "last_date", "tfreq", "xfreq"]
)

for sat in sats_new:
    tracks_n = get_total_tracks(sat)
    print(f"number of tracks of {sat}: {tracks_n}")
    for f in sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
        ds = xr.open_dataset(f)
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        time_s = ds.time.isel(cycles_numbers=0).isel(points_numbers=0).item()
        time_e = ds.time.isel(cycles_numbers=0).isel(points_numbers=-1).item()
        scnds = (time_e - time_s).seconds
        time = ds.time.isel(points_numbers=0)
        time1 = time.values
        first_time = time1[0]
        last_time = time1[-1]
        first_time_str = first_time.strftime("%Y-%m-%d")
        last_time_str = last_time.strftime("%Y-%m-%d")
        time_datetime = [
            datetime(year=t.year, month=t.month, day=t.day) for t in time1
        ]
        lon_s = lons_track[0]
        lon_e = lons_track[-1]
        lat_s = lats_track[0]
        lat_e = lats_track[-1]
        intervals = []
        for i in range(1, len(time_datetime)):
            interval = time_datetime[i] - time_datetime[i - 1]
            intervals.append(interval.days)
        delds = []
        if len(lons_track) > 100:
            for i in range(len(lons_track) - 1):
                lat1 = lats_track[i]
                lon1 = lons_track[i]
                lat2 = lats_track[i + 1]
                lon2 = lons_track[i + 1]
                deld = distance.distance((lat1, lon1), (lat2, lon2)).km
                delds.append(deld)
    delds1 = delds[:]  # [x for x in delds if x!=0]
    try:
        xfreq = sum(delds1) / len(delds1)
    except ZeroDivisionError:
        xfreq = 0
    data_counts = Counter(intervals)
    most_common = data_counts.most_common(1)
    tfreq = most_common[0][0]
    df = df._append(
        {
            "sat": sat,
            "trackn": tracks_n,
            "first_date": first_time_str,
            "last_date": last_time_str,
            "tfreq": tfreq,
            "xfreq": xfreq,
        },
        ignore_index=True,
    )

print(df)
df.to_csv("data_coverage_df.csv")

```

meta/f.sla.TPN+J1N+J2N+J3N_all.py

```python
import numpy as np
import xarray as xr
import glob
from matplotlib import pyplot as plt

sat = "TPN+J1N+J2N+J3N"

file_list = sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc"))
for f in file_list:
    ds = xr.open_dataset(f)
    sla = ds["sla"]
    sla.plot()
    plt.show()

```

```
GFO: 164.25974242775692
TP+J1+J2+J3+S6A: 315.521902489642
S3A: 104.2013733809004
S3B: 104.16565596350374
TPN+J1N+J2N+J3N: 315.5729509610255
ERS1+ERS2+ENV+SRL: 80.16604996684946
HY2A: 207.59693638474025
HY2B: 207.69825051270158

total area coverage of NINDIAN region xtrack tracks

lon min is: 42.978628550724636
lon max is: 99.00159020833333
lat min is: 0.0012709791666666666
lat min is: 25.459718391304346

GFO 6.989861489601352
TP+J1+J2+J3+S6A 6.2052372955783355
S3A 7.032925570741816
S3B 7.031838805556914
TPN+J1N+J2N+J3N 6.280559936743142
ERS1+ERS2+ENV+SRL 7.762731600065951
HY2A 6.796315226383516
HY2B 6.809776899428818
{'GFO': 6.989861489601352, 'TP+J1+J2+J3+S6A': 6.2052372955783355, 'S3A': 7.032925570741816, 'S3B': 7.031838805556914, 'TPN+J1N+J2N+J3N': 6.280559936743142, 'ERS1+ERS2+
     ENV+SRL': 7.762731600065951, 'HY2A': 6.796315226383516, 'HY2B': 6.809776899428818}

zero length passes
ERS1+ERS2+ENV+SRL 451

```

```python
import glob
from collections import Counter

import xarray as xr
from geopy import distance
from namesm import *

frac1 = 0.05
frac1 = 0.2


for sat in sats_new:
    lens = []
    frac_lens = []
    for f in sorted(glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")):
        # print(f)
        ds = xr.open_dataset(f, decode_times=False)
        if len(ds.points_numbers) == 0:
            continue
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        # time_s = ds.time.isel(cycles_numbers=0).isel(points_numbers=0)
        # time_e = ds.time.isel(cycles_numbers=0).isel(points_numbers=-1)
        # scnds = (time_e - time_s).seconds
        lon_equat = lons_track[0]
        lon_coast = lons_track[-1]
        lat_equat = lats_track[0]
        lat_coast = lats_track[-1]
        total_len = distance.distance((lat_equat, lon_equat),
                                      (lat_coast, lon_coast)).km
        lens.append(total_len)
        frac_lens.append(frac1 * total_len)
        print(f"frac of {sat} {track_number} is {frac1 * sum(lens)/len(lens)}")
    print(f"mean len of {sat} is {sum(lens)/len(lens)} km -----------")
    print(f"mean frac len of {sat} is {sum(frac_lens)/len(frac_lens)} km -----------")

```

```python
import glob
import math

import shapely.geometry as sg
import xarray as xr
import numpy as np
import pandas as pd
from tools_xtrackm import *

dt0 = datetime.strptime("1950-01-01", "%Y-%m-%d")
var_str = "sla"

for sat in sats_new[:]:
    for f_self in sorted(
            glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
        ds_self = xr.open_dataset(f_self,
                                  engine="h5netcdf",
                                  decode_times=False)
        if len(ds_self.points_numbers) < 12:
            continue
        # sla_self = track_dist_time_asn(ds_self, var_str="sla")
        ln = ds_self[var_str].sizes["points_numbers"]
        ln2 = ln // 2
        dvals_self = ds_self.time.isel(points_numbers=ln2).values
        dates_self = [dt0 + timedelta(days=int(d)) for d in dvals_self]
        time0_self = dates_self[0]
        track_number_self = ds_self.pass_number
        lons_track_self = ds_self.lon.values
        lats_track_self = ds_self.lat.values
        lon_equat_self = lons_track_self[0]
        lat_equat_self = lats_track_self[0]
        lon_coast_self = lons_track_self[-1]
        lat_coast_self = lats_track_self[-1]
        slope_self = (lat_coast_self - lat_equat_self) / (lon_coast_self -
                                                          lon_equat_self)
        if slope_self < 0:
            continue
        track_path_self = sg.LineString(zip(lons_track_self,
                                            lats_track_self))
        # track_path_self = sg.LineString([(lons_track_self[0], lats_track_self[0]),
        #                                  (lons_track_self[-1], lats_track_self[-1])])
        other_times = []
        for f_other in sorted(
                glob.glob(f"../data/ctoh.sla.ref.{sat}.nindian.*.nc")):
            # print(f)
            ds_other = xr.open_dataset(f_other,
                                       engine="h5netcdf",
                                       decode_times=False)
            if len(ds_other.points_numbers) < 12:
                continue
            # sla_other = track_dist_time_asn(ds_other, var_str="sla")
            times_other = ds_other.time
            track_number_other = ds_other.pass_number
            lons_track_other = ds_other.lon.values
            lats_track_other = ds_other.lat.values
            lon_equat_other = lons_track_other[0]
            lat_equat_other = lats_track_other[0]
            lon_coast_other = lons_track_other[-1]
            lat_coast_other = lats_track_other[-1]
            slope_other = (lat_coast_other - lat_equat_other) / (
                lon_coast_other - lon_equat_other)
            if slope_other > 0:
                continue
            # define a linestring for the track
            track_path_other = sg.LineString(
                    zip(lons_track_other, lats_track_other))
            # track_path_other = sg.LineString([(lons_track_other[0], lats_track_other[0]),
            #                                   (lons_track_other[-1], lats_track_other[-1])])
            if track_path_self.intersects(track_path_other):
                point = track_path_self.intersection(track_path_other)
                x_from_coast = distance.distance((lat_coast_self,
                                                  lon_coast_self),
                                                 (point.y, point.x)).km
                # x_idx = index_at_x(x, x_from_coast)
                lat_idx = index_at_lat(ds_other, point.y)
                ln = ds_other[var_str].sizes["points_numbers"]
                ln2 = ln // 2
                # dvals_other = ds_other.time.isel(points_numbers=ln2).values
                dvals_other = ds_other.time.isel(points_numbers=lat_idx).values
                dates_other = [dt0 + timedelta(days=int(d)) for d in dvals_other]
                time0_other = dates_other[0]
                dist_time = (time0_other - time0_self).days
                print(sat, track_number_self,
                      track_number_other,
                      time0_self, time0_other,
                      dist_time, lat_idx, x_from_coast)
                other_times.append((track_number_other, abs(dist_time)))
        if len(other_times) == 0:
            continue
        other_times.sort(key=lambda x: x[1], reverse=True)
        print(track_number_self, other_times[0], "-----------------------")
        break

```

meta/l.fraction_of_distance_loess.py

```python
import glob
import xarray as xr
from namesm import *
from geopy import distance

fixed_dist = 75_000 # m

for sat in sats_new:
    for f in sorted(glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")):
        # print(f)
        ds = xr.open_dataset(f, decode_times=False)
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        track_number = ds.Pass
        if len(lats_track) == 0:
            continue
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        lon_equat = lons_track[0]  # this on equator
        lat_equat = lats_track[0]  # this on equator
        dist = distance.distance((lat_coast, lon_coast), (lat_equat, lon_equat)).m
        frac = fixed_dist/dist
        info = f"{dist} {frac} {sat} {track_number}"
        print(info)

```

```python
# picking up lons, lats at equator of tracks with negative slope
# finding successive distances between them
import glob
import math
import statistics

import xarray as xr
from geopy import distance
import numpy as np
from namesm import *

for sat in sats_new:
    lons_equat_n = []
    lats_equat_n = []
    lons_equat_p = []
    lats_equat_p = []
    for f in sorted(
        glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")
    ):
        # print(f)
        ds = xr.open_dataset(f, engine="netcdf4", decode_times=False)
        if len(ds.points_numbers) == 0:
            continue
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        lon_equat = lons_track[0]
        lat_equat = lats_track[0]
        lon_coast = lons_track[-1]
        lat_coast = lats_track[-1]
        slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
        angle_r = math.atan(slope)
        angle_d = angle_r * (180 / math.pi)
        if abs(lat_equat) >= 0 and abs(lat_equat) < 0.1:
            if slope > 0:
                lons_equat_p.append(lon_equat)
                lats_equat_p.append(lat_equat)
            elif slope < 0:
                lons_equat_n.append(lon_equat)
                lats_equat_n.append(lat_equat)
    # print(f"{sat}", lons_equat_n)
    dists_p = []
    ln = len(lons_equat_p)
    for i in range(ln - 1):
        lons_equat1 = lons_equat_p[i]
        lats_equat1 = lats_equat_p[i]
        lons_equat2 = lons_equat_p[i + 1]
        lats_equat2 = lats_equat_p[i + 1]
        dist = distance.distance((lats_equat1, lons_equat1), (lats_equat2, lons_equat2)).km
        dists_p.append(dist)
    dists_n = []
    ln = len(lons_equat_n)
    for i in range(ln - 1):
        lons_equat1 = lons_equat_n[i]
        lats_equat1 = lats_equat_n[i]
        lons_equat2 = lons_equat_n[i + 1]
        lats_equat2 = lats_equat_n[i + 1]
        dist = distance.distance((lats_equat1, lons_equat1), (lats_equat2, lons_equat2)).km
        dists_n.append(dist)
    # print(dists)
    dist_mode_n = statistics.mode(dists_n)
    dist_mode_p = statistics.mode(dists_p)
    dn = dist_mode_n * np.cos(np.pi/2 - abs(angle_r))
    dp = dist_mode_p * np.cos(np.pi/2 - abs(angle_r))
    print(f"{sat}: {dist_mode_p}")
    # print(f"{sat}: {dist_mode_n}")
    # print(f"{sat}: {dp}")
    # x = input()
    # if x == "q":
    #     break
    # else:
    #     continue

```

meta/l.lat_index_at_equator.py

```python
import glob
import xarray as xr
from namesm import *

for sat in sats_new:
    for f in sorted(glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")):
        # print(f)
        ds = xr.open_dataset(f, decode_times=False)
        lons = ds.lon
        lats = ds.lat
        if len(lats.values) == 0:
            continue
        lat_equat = lats.values[0]
        print(lat_equat)

```

```python
import glob
from collections import Counter

import xarray as xr
from geopy import distance
from namesm import *
import time
import os
from multiprocessing import Pool, cpu_count
from itertools import chain

# Get the process ID
process_id = os.getpid()
# Print the process ID
print(f"The process ID of the current Python script is: {process_id}")

def split_list(lst, n):
    size = len(lst) // n
    return [lst[i*size:(i+1)*size] for i in range(n-1)] + [lst[(n-1)*size:]]


def process_chunk(chunk):
    delds = []
    for f in chunk:
        try:
            ds = xr.open_dataset(f)
        except:
            continue
        if len(ds.points_numbers) == 0:
            continue
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        time_s = ds.time.isel(cycles_numbers=0).isel(points_numbers=0).item()
        time_e = ds.time.isel(cycles_numbers=0).isel(points_numbers=-1).item()
        scnds = (time_e - time_s).seconds
        lon_s = lons_track[0]
        lon_e = lons_track[-1]
        lat_s = lats_track[0]
        lat_e = lats_track[-1]
        if len(lons_track) > 100:
            for i in range(len(lons_track)-1):
                lat1 = lats_track[i]
                lon1 = lons_track[i]
                lat2 = lats_track[i+1]
                lon2 = lons_track[i+1]
                deld = distance.distance((lat1, lon1), (lat2, lon2)).km
                delds.append(deld)
        ds.close()
    return delds

sats_ints = {}
start_time0 = time.time()
files = sorted(glob.glob(f"../data/*_lon_ordered/ctoh.sla.ref.*.nindian.*.nc"))

number_of_workers = 4
chunks = split_list(files, number_of_workers)
with Pool(number_of_workers) as pool:
    list_of_lists = pool.map(process_chunk, chunks)
    delds_all = list(chain.from_iterable(list_of_lists))

delds1 = delds_all[:] # [x for x in delds if x!=0]
mode_value = sum(delds1)/len(delds1)
print(mode_value)
print(f"total time {(time.time() - start_time0)} seconds ---")

```

```python
import glob
from collections import Counter

import xarray as xr
from geopy import distance
from namesm import *
import time
import os

# cmapl="viridis"

# Get the process ID
process_id = os.getpid()
# Print the process ID
print(f"The process ID of the current Python script is: {process_id}")

start_time0 = time.time()
delds = []
for f in sorted(glob.glob(f"../data/*_lon_ordered/ctoh.sla.ref.*.nindian.*.nc")):
    # print(f)
    try:
        ds = xr.open_dataset(f)
    except:
        continue
    if len(ds.points_numbers) == 0:
        continue
    track_number = ds.Pass
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    time_s = ds.time.isel(cycles_numbers=0).isel(points_numbers=0).item()
    time_e = ds.time.isel(cycles_numbers=0).isel(points_numbers=-1).item()
    scnds = (time_e - time_s).seconds
    lon_s = lons_track[0]
    lon_e = lons_track[-1]
    lat_s = lats_track[0]
    lat_e = lats_track[-1]
    if len(lons_track) > 100:
        for i in range(len(lons_track)-1):
            lat1 = lats_track[i]
            lon1 = lons_track[i]
            lat2 = lats_track[i+1]
            lon2 = lons_track[i+1]
            deld = distance.distance((lat1, lon1), (lat2, lon2)).km
            delds.append(deld)
    ds.close()
delds1 = delds[:] # [x for x in delds if x!=0]
mode_value = sum(delds1)/len(delds1)
print(mode_value)
print(f"total time {(time.time() - start_time0)} seconds ---")

```

```python
import glob
from collections import Counter

import xarray as xr
from geopy import distance
from namesm import *
import time
import os

# cmapl="viridis"

# Get the process ID
process_id = os.getpid()
# Print the process ID
print(f"The process ID of the current Python script is: {process_id}")

sats_ints = {}

start_time0 = time.time()
for sat in sats_new:
    start_time = time.time()
    delds = []
    for f in sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
        # print(f)
        try:
            ds = xr.open_dataset(f)
        except:
            continue
        if len(ds.points_numbers) == 0:
            continue
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        time_s = ds.time.isel(cycles_numbers=0).isel(points_numbers=0).item()
        time_e = ds.time.isel(cycles_numbers=0).isel(points_numbers=-1).item()
        scnds = (time_e - time_s).seconds
        lon_s = lons_track[0]
        lon_e = lons_track[-1]
        lat_s = lats_track[0]
        lat_e = lats_track[-1]
        if len(lons_track) > 100:
            for i in range(len(lons_track)-1):
                lat1 = lats_track[i]
                lon1 = lons_track[i]
                lat2 = lats_track[i+1]
                lon2 = lons_track[i+1]
                deld = distance.distance((lat1, lon1), (lat2, lon2)).km
                delds.append(deld)
        ds.close()
    delds1 = delds[:] # [x for x in delds if x!=0]
    mode_value = sum(delds1)/len(delds1)
    print(f"{sat} {mode_value}")
    sats_ints[sat] = mode_value
    print(f"{sat} took {(time.time() - start_time)} seconds ---")

print(sats_ints)
print(f"total time {(time.time() - start_time0)} seconds ---")

```

```python
# fast_netCDF4_worker.py
import glob, os, time
from multiprocessing import Pool, cpu_count
import numpy as np
from netCDF4 import Dataset
from namesm import *   # sats_new

DATA_DIR = "../data"
PATTERN = "ctoh.sla.ref.{sat}.nindian.*.nc"

R = 6371.0088
def haversine_sum_and_count(lats, lons):
    if lats.size < 2:
        return 0.0, 0
    lat1 = np.deg2rad(lats[:-1]); lat2 = np.deg2rad(lats[1:])
    lon1 = np.deg2rad(lons[:-1]); lon2 = np.deg2rad(lons[1:])
    dlat = lat2 - lat1; dlon = lon2 - lon1
    a = np.sin(dlat/2.0)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2.0)**2
    a = np.minimum(1.0, np.maximum(0.0, a))
    c = 2.0 * np.arctan2(np.sqrt(a), np.sqrt(1.0 - a))
    d = R * c
    return float(d.sum()), int(d.size)

def worker(args):
    fpath, sat = args
    # NO try/except here by user request
    ds = Dataset(fpath, mode="r")
    # adapt variable names to your files if different
    lons = ds.variables['lon'][:].astype('float64')
    lats = ds.variables['lat'][:].astype('float64')
    ds.close()
    if lons.size <= 100:
        return (sat, 0.0, 0)
    return (sat, ) + haversine_sum_and_count(lats, lons)

def gather():
    tasks=[]
    for sat in list(sats_new):
        pattern = os.path.join(DATA_DIR, f"{sat}_lon_ordered", PATTERN.format(sat=sat))
        for f in sorted(glob.glob(pattern)):
            tasks.append((f, sat))
    return tasks

if __name__ == "__main__":
    t0=time.time()
    tasks=gather()
    n= max(1, cpu_count()-1)
    with Pool(n) as p:
        res = p.map(worker, tasks)
    sums, counts = {}, {}
    for sat, s, c in res:
        if c==0: continue
        sums[sat]=sums.get(sat,0.0)+s
        counts[sat]=counts.get(sat,0)+c
    sats_ints = {sat: (sums.get(sat,0.0)/counts.get(sat,1)) if counts.get(sat,0)>0 else 0.0 for sat in list(sats_new)}
    print(sats_ints)
    print("TOTAL", time.time()-t0)

```

```python
# fast_netCDF4_worker.py
import glob, os, time
from multiprocessing import Pool, cpu_count
import numpy as np
from netCDF4 import Dataset
from namesm import *   # sats_new

DATA_DIR = "../data"
PATTERN = "ctoh.sla.ref.{sat}.nindian.*.nc"

R = 6371.0088
def haversine_sum_and_count(lats, lons):
    if lats.size < 2:
        return 0.0, 0
    lat1 = np.deg2rad(lats[:-1]); lat2 = np.deg2rad(lats[1:])
    lon1 = np.deg2rad(lons[:-1]); lon2 = np.deg2rad(lons[1:])
    dlat = lat2 - lat1; dlon = lon2 - lon1
    a = np.sin(dlat/2.0)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2.0)**2
    a = np.minimum(1.0, np.maximum(0.0, a))
    c = 2.0 * np.arctan2(np.sqrt(a), np.sqrt(1.0 - a))
    d = R * c
    return float(d.sum()), int(d.size)

def worker(args):
    fpath, sat = args
    # NO try/except here by user request
    ds = Dataset(fpath, mode="r")
    # adapt variable names to your files if different
    lons = ds.variables['lon'][:].astype('float64')
    lats = ds.variables['lat'][:].astype('float64')
    ds.close()
    if lons.size <= 100:
        return (sat, 0.0, 0)
    return (sat, ) + haversine_sum_and_count(lats, lons)

def gather():
    tasks=[]
    for sat in list(sats_new):
        pattern = os.path.join(DATA_DIR, f"{sat}_lon_ordered", PATTERN.format(sat=sat))
        for f in sorted(glob.glob(pattern)):
            tasks.append((f, sat))
    return tasks

if __name__ == "__main__":
    t0=time.time()
    tasks=gather()
    n= max(1, cpu_count()-1)
    with Pool(n) as p:
        res = p.map(worker, tasks)
    sums, counts = {}, {}
    for sat, s, c in res:
        if c==0: continue
        sums[sat]=sums.get(sat,0.0)+s
        counts[sat]=counts.get(sat,0)+c
    sats_ints = {sat: (sums.get(sat,0.0)/counts.get(sat,1)) if counts.get(sat,0)>0 else 0.0 for sat in list(sats_new)}
    print(sats_ints)
    print("TOTAL", time.time()-t0)

```

```python
import glob
from collections import Counter

import xarray as xr
from geopy import distance
from namesm import *
import time
import os
from multiprocessing import Pool, cpu_count
from itertools import chain

# Get the process ID
process_id = os.getpid()
# Print the process ID
print(f"The process ID of the current Python script is: {process_id}")

def split_list(lst, n):
    size = len(lst) // n
    return [lst[i*size:(i+1)*size] for i in range(n-1)] + [lst[(n-1)*size:]]


def process_chunk(chunk):
    delds = []
    for f in chunk:
        try:
            ds = xr.open_dataset(f)
        except:
            continue
        if len(ds.points_numbers) == 0:
            continue
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        time_s = ds.time.isel(cycles_numbers=0).isel(points_numbers=0).item()
        time_e = ds.time.isel(cycles_numbers=0).isel(points_numbers=-1).item()
        scnds = (time_e - time_s).seconds
        lon_s = lons_track[0]
        lon_e = lons_track[-1]
        lat_s = lats_track[0]
        lat_e = lats_track[-1]
        if len(lons_track) > 100:
            for i in range(len(lons_track)-1):
                lat1 = lats_track[i]
                lon1 = lons_track[i]
                lat2 = lats_track[i+1]
                lon2 = lons_track[i+1]
                deld = distance.distance((lat1, lon1), (lat2, lon2)).km
                delds.append(deld)
        ds.close()
    return delds

sats_ints = {}
start_time0 = time.time()
sat = "ERS1+ERS2+ENV+SRL"
files = sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc"))

number_of_workers = 4
chunks = split_list(files, number_of_workers)
with Pool(number_of_workers) as pool:
    list_of_lists = pool.map(process_chunk, chunks)
    delds_all = list(chain.from_iterable(list_of_lists))

delds1 = delds_all[:] # [x for x in delds if x!=0]
mode_value = sum(delds1)/len(delds1)
print(f"{sat} {mode_value}")
sats_ints[sat] = mode_value
print(sats_ints)
print(f"total time {(time.time() - start_time0)} seconds ---")

```

```python
import glob
from collections import Counter

import xarray as xr
from geopy import distance
from namesm import *
import time
import os

def split_files(list_files, n):
    chunks = []
    for i in range(0, len(list_files), n):
        chunk = list_files[i*n:(i+1)*n]
        chunks.append(chunk)
    return chunks

# Get the process ID
process_id = os.getpid()
# Print the process ID
print(f"The process ID of the current Python script is: {process_id}")

sats_ints = {}

sat = "ERS1+ERS2+ENV+SRL"
start_time0 = time.time()
delds = []
for f in sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
    # print(f)
    try:
        ds = xr.open_dataset(f)
    except:
        continue
    if len(ds.points_numbers) == 0:
        continue
    track_number = ds.Pass
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    time_s = ds.time.isel(cycles_numbers=0).isel(points_numbers=0).item()
    time_e = ds.time.isel(cycles_numbers=0).isel(points_numbers=-1).item()
    scnds = (time_e - time_s).seconds
    lon_s = lons_track[0]
    lon_e = lons_track[-1]
    lat_s = lats_track[0]
    lat_e = lats_track[-1]
    if len(lons_track) > 100:
        for i in range(len(lons_track)-1):
            lat1 = lats_track[i]
            lon1 = lons_track[i]
            lat2 = lats_track[i+1]
            lon2 = lons_track[i+1]
            deld = distance.distance((lat1, lon1), (lat2, lon2)).km
            delds.append(deld)
    ds.close()
delds1 = delds[:] # [x for x in delds if x!=0]
mode_value = sum(delds1)/len(delds1)
print(f"{sat} {mode_value}")
sats_ints[sat] = mode_value
print(sats_ints)
print(f"total time {(time.time() - start_time0)} seconds ---")

```

```python
# l.lat_lon.dists2.parallel.notry.py
# Parallelized: threading for I/O, multiprocessing across satellites.
# process_file contains NO try/except/finally blocks (uses context manager instead).

import glob
import time
from collections import Counter

import xarray as xr
from geopy import distance
from namesm import *   # provides sats_new etc.

from concurrent.futures import ThreadPoolExecutor, as_completed
from multiprocessing import Pool, cpu_count

sats_ints = {}

def process_file(path):
    delds = []
    # No try/except here — let errors propagate. Using context manager to ensure close.
    with xr.open_dataset(path) as ds:
        if len(ds.points_numbers) == 0:
            return delds
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        time_s = ds.time.isel(cycles_numbers=0).isel(points_numbers=0).item()
        time_e = ds.time.isel(cycles_numbers=0).isel(points_numbers=-1).item()
        scnds = (time_e - time_s).seconds
        lon_s = lons_track[0]
        lon_e = lons_track[-1]
        lat_s = lats_track[0]
        lat_e = lats_track[-1]
        if len(lons_track) > 100:
            for i in range(len(lons_track)-1):
                lat1 = lats_track[i]
                lon1 = lons_track[i]
                lat2 = lats_track[i+1]
                lon2 = lons_track[i+1]
                deld = distance.distance((lat1, lon1), (lat2, lon2)).km
                delds.append(deld)
    return delds

def process_sat(sat):
    start_time = time.time()
    delds = []
    pattern = f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc"
    files = sorted(glob.glob(pattern))
    with ThreadPoolExecutor(max_workers=8) as executor:
        futures = {executor.submit(process_file, fpath): fpath for fpath in files}
        for fut in as_completed(futures):
            try:
                out = fut.result()
                if out:
                    delds.extend(out)
            except Exception:
                # keep outer-level minimal error skipping (per your dirty request)
                pass
    if len(delds) == 0:
        mode_value = 0.0
    else:
        delds1 = delds[:]
        mode_value = sum(delds1) / len(delds1)
    elapsed = time.time() - start_time
    print(f"{sat} {mode_value}")
    print(f"{sat} took {elapsed} seconds ---")
    return (sat, mode_value)

if __name__ == "__main__":
    ncores = max(1, cpu_count() - 1)
    sats_list = list(sats_new)
    with Pool(processes=ncores) as pool:
        results = pool.map(process_sat, sats_list)
    sats_ints = dict(results)
    print(sats_ints)

```

```python
# l.lat_lon.dists2.prll_files.py
# Parallel (faster): multiprocessing over files + vectorized haversine per file.
# Dirty style on purpose. Worker has no try/except around dataset reading.

import glob
import os
import time
from multiprocessing import Pool, cpu_count

import numpy as np
import xarray as xr
from namesm import *   # provides sats_new etc.

# Get the process ID
process_id = os.getpid()
# Print the process ID
print(f"The process ID of the current Python script is: {process_id}")

# adjust these paths/patterns to match your layout
DATA_DIR = "../data"
PATTERN = "ctoh.sla.ref.{sat}.nindian.*.nc"

def haversine_sum_and_count(lats, lons):
    """
    Vectorized haversine distances between successive points.
    Returns (sum_of_distances_km, number_of_pairs)
    lats, lons are 1D numpy arrays (degrees).
    """
    # require at least 2 points
    if lats.size < 2:
        return 0.0, 0
    # convert to radians
    lat1 = np.deg2rad(lats[:-1])
    lat2 = np.deg2rad(lats[1:])
    lon1 = np.deg2rad(lons[:-1])
    lon2 = np.deg2rad(lons[1:])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2.0) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2.0) ** 2
    # numerical safety
    a = np.minimum(1.0, np.maximum(0.0, a))
    c = 2.0 * np.arctan2(np.sqrt(a), np.sqrt(1.0 - a))
    R = 6371.0088  # mean Earth radius in km
    dists = R * c
    return float(dists.sum()), int(dists.size)

def worker_process_file(args):
    """
    Worker that reads one file and returns:
      (sat_name, sum_delds_km, n_pairs)
    No try/except (user requested).
    """
    fpath, sat = args
    with xr.open_dataset(fpath) as ds:
        # follow original logic: skip tiny tracks / use only ones with >100 points
        if len(ds.points_numbers) == 0:
            return (sat, 0.0, 0)
        lons = ds.lon.values
        lats = ds.lat.values
        if lons.size <= 100:
            return (sat, 0.0, 0)
        s, n = haversine_sum_and_count(lats, lons)
        return (sat, s, n)

def gather_file_list_for_all_sats(sats):
    tasks = []
    for sat in sats:
        pattern = os.path.join(DATA_DIR, f"{sat}_lon_ordered", PATTERN.format(sat=sat))
        files = sorted(glob.glob(pattern))
        for f in files:
            tasks.append((f, sat))
    return tasks

if __name__ == "__main__":
    start_all = time.time()
    sats = list(sats_new)
    tasks = gather_file_list_for_all_sats(sats)
    ncores = max(1, cpu_count() - 1)   # leave one core free
    with Pool(processes=ncores) as pool:
        # map over files; this returns a list of (sat, sum, count)
        results = pool.map(worker_process_file, tasks)
    # aggregate per-satellite sums and counts
    sums = {}
    counts = {}
    for sat, s, n in results:
        if n == 0:
            continue
        sums[sat] = sums.get(sat, 0.0) + s
        counts[sat] = counts.get(sat, 0) + n

    sats_ints = {}
    for sat in sats:
        if counts.get(sat, 0) == 0:
            sats_ints[sat] = 0.0
        else:
            sats_ints[sat] = sums[sat] / counts[sat]
    # print in same style as your scripts
    for sat in sats:
        print(f"{sat} {sats_ints[sat]}")
        # We don't print per-sat timings here; measure whole run
    print(sats_ints)
    print("TOTAL TIME:", time.time() - start_all)

```

```python
import glob
from collections import Counter

import xarray as xr
from geopy import distance
from namesm import *
import time
import os

# cmapl="viridis"

# Get the process ID
process_id = os.getpid()
# Print the process ID
print(f"The process ID of the current Python script is: {process_id}")

sats_ints = {}

start_time0 = time.time()
for sat in sats_new:
    start_time = time.time()
    delds = []
    for f in sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
        # print(f)
        try:
            ds = xr.open_dataset(f)
        except:
            continue
        if len(ds.points_numbers) == 0:
            continue
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        time_s = ds.time.isel(cycles_numbers=0).isel(points_numbers=0).item()
        time_e = ds.time.isel(cycles_numbers=0).isel(points_numbers=-1).item()
        scnds = (time_e - time_s).seconds
        lon_s = lons_track[0]
        lon_e = lons_track[-1]
        lat_s = lats_track[0]
        lat_e = lats_track[-1]
        if len(lons_track) > 100:
            for i in range(len(lons_track)-1):
                lat1 = lats_track[i]
                lon1 = lons_track[i]
                lat2 = lats_track[i+1]
                lon2 = lons_track[i+1]
                deld = distance.distance((lat1, lon1), (lat2, lon2)).km
                delds.append(deld)
        ds.close()
    delds1 = delds[:] # [x for x in delds if x!=0]
    mode_value = sum(delds1)/len(delds1)
    print(f"{sat} {mode_value}")
    sats_ints[sat] = mode_value
    print(f"{sat} took {(time.time() - start_time)} seconds ---")

print(sats_ints)
print(f"total time {(time.time() - start_time0)} seconds ---")

```

meta/l.lat_lon.max_min.netCDF4.py

```python
import glob
import xarray as xr
from namesm import *
import time
from netCDF4 import Dataset
import numpy as np

lon_mins=[]
lon_maxs=[]
lat_mins=[]
lat_maxs=[]

start_time = time.time()
for sat in sats_new:
    for f in sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
        # print(f)
        ds = Dataset(f, mode="r")
        # adapt variable names to your files if different
        lons = ds.variables['lon'][:].astype('float64')
        lats = ds.variables['lat'][:].astype('float64')
        ds.close()
        lon_min = np.min(lons)
        lon_max = np.max(lons)
        lat_min = np.min(lats)
        lat_max = np.max(lats)
        lon_mins.append(lon_min)
        lon_maxs.append(lon_max)
        lat_mins.append(lat_min)
        lat_maxs.append(lat_max)

print(f"lon min is: {min(lon_mins)}")
print(f"lon max is: {max(lon_maxs)}")
print(f"lat min is: {min(lat_mins)}")
print(f"lat min is: {max(lat_maxs)}")

print("--- %s seconds ---" % (time.time() - start_time))

```

meta/l.lat_lon.max_min.py

```python
import glob
import xarray as xr
from namesm import *
import time

lon_mins=[]
lon_maxs=[]
lat_mins=[]
lat_maxs=[]

start_time = time.time()
for sat in sats_new:
    for f in sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
        # print(f)
        ds = xr.open_dataset(f,decode_times=False)
        lon_min = ds.lon.min().item()
        lon_max = ds.lon.max().item()
        lat_min = ds.lat.min().item()
        lat_max = ds.lat.max().item()
        lon_mins.append(lon_min)
        lon_maxs.append(lon_max)
        lat_mins.append(lat_min)
        lat_maxs.append(lat_max)

print(f"lon min is: {min(lon_mins)}")
print(f"lon max is: {max(lon_maxs)}")
print(f"lat min is: {min(lat_mins)}")
print(f"lat min is: {max(lat_maxs)}")

print("--- %s seconds ---" % (time.time() - start_time))

```

meta/l.lat_time_diff.py

```python
import glob
from collections import Counter

import xarray as xr
from geopy import distance
from namesm import *
import time
import os

# cmapl="viridis"

# Get the process ID
process_id = os.getpid()
# Print the process ID
print(f"The process ID of the current Python script is: {process_id}")

sats_ints = {}

start_time0 = time.time()
for sat in sats_new:
    start_time = time.time()
    delds = []
    for f in sorted(glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")):
        # print(f)
        try:
            ds = xr.open_dataset(f)
        except:
            continue
        if len(ds.points_numbers) == 0:
            continue
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        time1 = ds.time.isel(cycles_numbers=0).values
        if len(lons_track) > 20:
            lat_diff = lats_track[20] - lats_track[10]
            # print(sat, track_number, lat_diff)
            time_s = time1[20]
            time_e = time1[10]
            scnds = (time_e - time_s).seconds
            print(sat, track_number, scnds)

```

```python
import xarray as xr
from namesm import *

sat = "GFO"
passn = "008"
file1 = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{passn}.nc"
ds = xr.open_dataset(file1,decode_times=False)
print(ds)

# Search for "QC" in all attributes
search_term = "Pass"

print("-------------------------------------------------")
# Global attributes
for k, v in ds.attrs.items():
    if search_term.lower() in str(v).lower() or search_term.lower() in k.lower():
        print(f"Global: {k} = {v}")

# Variable attributes and names
for var_name, var in ds.data_vars.items():
    if search_term.lower() in var_name.lower():
        print(f"Variable name: {var_name}")
    for k, v in var.attrs.items():
        if search_term.lower() in str(v).lower() or search_term.lower() in k.lower():
            print(f"Variable {var_name}.{k} = {v}")

# Coordinate attributes and names
for coord_name, coord in ds.coords.items():
    if search_term.lower() in coord_name.lower():
        print(f"Coordinate name: {coord_name}")
    for k, v in coord.attrs.items():
        if search_term.lower() in str(v).lower() or search_term.lower() in k.lower():
            print(f"Coordinate {coord_name}.{k} = {v}")

```

meta/l.one_file.test_asn.py

```python
import xarray as xr
from tools_xtrackm import track_dist_time_asn_midx

sat = "GFO"
passn = "008"
file1 = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{passn}.nc"
ds = xr.open_dataset(file1,decode_times=False)
print(ds)
sla_asn = track_dist_time_asn_midx(ds, var_str="sla", units_in="m")

# `track_dist_time_asn_midx` returns dims ("x", "time") with lat as a coordinate on "x".
# Swap "x" with "lat" so we can select directly along latitude.
sla_asn_lat = sla_asn.swap_dims({"x": "lat"})#.sortby("lat")

# Example latitude-band selection.
lat_min, lat_max = 11.0, 5.0
sla_asn_lat_band = sla_asn_lat.sel(lat=slice(lat_min, lat_max))
print(sla_asn_lat_band)

```

```python
# bench_read_xr_vs_netcdf4.py
# Dirty benchmark: read lon/lat from all files using xarray and netCDF4 and compare times.
# Only reading — no further processing.

import glob, os, time
import numpy as np
import xarray as xr
from netCDF4 import Dataset
from namesm import *   # sats_new

DATA_DIR = "../data"
PATTERN = "ctoh.sla.ref.{sat}.nindian.*.nc"

# Toggle this to test xarray decode_times True/False
DECODE_TIMES = True
DECODE_TIMES = False

def gather_files(sats):
    files = []
    for sat in sats:
        p = os.path.join(DATA_DIR, f"{sat}_lon_ordered", PATTERN.format(sat=sat))
        files.extend(sorted(glob.glob(p)))
    return files

def read_with_xarray(files, decode_times=True):
    t0 = time.perf_counter()
    sizes = []
    for f in files:
        # open, read lon/lat into memory, then close
        with xr.open_dataset(f, decode_times=decode_times) as ds:
            # force load to memory
            lons = ds.lon.values
            lats = ds.lat.values
            sizes.append((lats.size, lons.size))
    t1 = time.perf_counter()
    total = t1 - t0
    return total, sizes

def read_with_netcdf4(files):
    t0 = time.perf_counter()
    sizes = []
    for f in files:
        ds = Dataset(f, mode="r")
        lons = ds.variables['lon'][:].astype('float64')
        lats = ds.variables['lat'][:].astype('float64')
        ds.close()
        sizes.append((lats.size, lons.size))
    t1 = time.perf_counter()
    total = t1 - t0
    return total, sizes

def summarize(name, total_time, sizes):
    nfiles = len(sizes)
    counts = [s[0] for s in sizes]  # lat counts (should equal lon)
    npoints = sum(counts)
    avg_file = total_time / nfiles if nfiles else float('nan')
    avg_point = total_time / npoints if npoints else float('nan')
    print(f"--- {name} ---")
    print(f"files: {nfiles}, total points read: {npoints}")
    print(f"total time: {total_time:.4f} s")
    print(f"avg time / file: {avg_file:.6f} s")
    print(f"avg time / point: {avg_point:.9f} s")
    print()

if __name__ == "__main__":
    sats = list(sats_new)
    files = gather_files(sats)
    if not files:
        print("No files found. Check DATA_DIR / pattern.")
        raise SystemExit(1)

    print(f"Found {len(files)} files. Running benchmarks...")
    print("Xarray decode_times =", DECODE_TIMES)
    xr_time, xr_sizes = read_with_xarray(files, decode_times=DECODE_TIMES)
    summarize(f"xarray (decode_times={DECODE_TIMES})", xr_time, xr_sizes)

    nc_time, nc_sizes = read_with_netcdf4(files)
    summarize("netCDF4.Dataset", nc_time, nc_sizes)

    diff = xr_time - nc_time
    rel = diff / nc_time if nc_time else float('nan')
    print("Summary comparison:")
    print(f"xarray time: {xr_time:.4f} s")
    print(f"netCDF4 time: {nc_time:.4f} s")
    print(f"absolute difference (xarray - netCDF4): {diff:.4f} s")
    print(f"relative difference: {rel*100:.2f}% (positive => xarray slower)")

```

meta/l.rossby_radius_of_deformation.py

```python
import math

def estimate_rossby_radius(lat_deg, c1=2.64):
    """
    Roughly estimate the first-baroclinic Rossby radius of deformation.
    Parameters:
    -----------
    lat_deg : float
        Latitude in degrees.
    c1 : float
        Phase speed of the first baroclinic gravity wave in m/s.
        (Default is 2.64 m/s, typical for the Northern Indian Ocean).
    Returns:
    --------
    rossby_radius_km : float
        Estimated Rossby radius of deformation in kilometers.
    """
    # Constants
    omega = 7.292e-5      # Earth's rotation rate in rad/s
    R_earth = 6378137.0   # Earth's equatorial radius in meters
    # Convert latitude to radians
    lat_rad = math.radians(lat_deg)
    # Calculate Coriolis parameter (f) and its meridional derivative (beta)
    # f = 2 * omega * sin(lat)
    # beta = df/dy = 2 * omega * cos(lat) / R_earth
    f = 2 * omega * math.sin(lat_rad)
    beta = (2 * omega * math.cos(lat_rad)) / R_earth
    # Calculate Rossby radius based on latitude distance from the equator
    if abs(lat_deg) >= 5.0:
        # Mid-to-high latitudes: R = c / |f|
        radius_m = c1 / abs(f)
    else:
        # Equatorial region: R = sqrt(c / beta)
        radius_m = math.sqrt(c1 / beta)
    return radius_m / 1000.0  # Convert meters to kilometers

# --- Examples based on literature values ---
print(f"Equator (0°): {estimate_rossby_radius(0):.1f} km")
print(f"(15°N): {estimate_rossby_radius(15):.1f} km")
print(f"Mid-latitudes (45°N): {estimate_rossby_radius(45):.1f} km")

```

```python
import glob

import numpy as np
import xarray as xr
from namesm import *


def is_odd(track_no):
    if int(track_no) % 2 == 1:
        return True
    else:
        return False


def orbit_type(track_no):
    if int(track_no) % 2 == 1:
        return "ascending"
    else:
        return "descending"


lst = []
# sats = ["GFO"]
for sat in sats_new:
    ascending_angles = []
    descending_angles = []
    for f in sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
        ds = xr.open_dataset(f, engine="h5netcdf", decode_times=False)
        if len(ds.points_numbers) < 12:
            continue
        track_number = ds.Pass
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        lon_equat = lons_track[0]
        lat_equat = lats_track[0]
        lon_coast = lons_track[-1]
        lat_coast = lats_track[-1]
        orbit_str = orbit_type(track_number)

        slope_whole = (lat_coast - lat_equat) / (lon_coast - lon_equat)
        angle_whole_r = np.arctan(slope_whole)
        # angle_whole_d = angle_whole_r * (180 / math.pi)
        angle_whole_d = np.rad2deg(angle_whole_r)
        angle_whole_r2 = np.arctan2(lat_coast - lat_equat,
                                    lon_coast - lon_equat)
        # angle_whole_d2 = angle_whole_r2 * (180 / math.pi)
        angle_whole_d2 = np.rad2deg(angle_whole_r2)

        ll = len(lons_track)
        ll2 = ll // 2

        delta_lon = lons_track[ll2 + 5] - lons_track[ll2 - 5]
        delta_lat = lats_track[ll2 + 5] - lats_track[ll2 - 5]

        slope_point = delta_lat / delta_lon
        angle_point_r = np.arctan(slope_point)
        # angle_point_d = angle_point_r * (180 / math.pi)
        angle_point_d = np.rad2deg(angle_point_r)
        angle_point_r2 = np.arctan2(delta_lat, delta_lon)
        # angle_point_d2 = angle_point_r2 * (180 / math.pi)
        angle_point_d2 = np.rad2deg(angle_point_r2)
        # diff = angle_point_d - angle_whole_d
        # lst.append(diff)

        # print(f"whole angle: {angle_whole_d}, point angle: {angle_point_d}")
        # print(diff)
        # if not is_odd(track_number):
        #     print(f"{angle_point_d} {angle_point_d2} {orbit_str} {sat}")
        # if not is_odd(track_number):
        if True:
            print(f"{sat} {angle_whole_d} {angle_whole_d2} {orbit_str}")
        if is_odd(track_number):
            # print(f"{sat} {angle_whole_d2} {orbit_str}")
            ascending_angles.append(angle_whole_d2)
        if not is_odd(track_number):
            # print(f"{sat} {angle_whole_d2} {orbit_str}")
            descending_angles.append(angle_whole_d2)
    print(f"{sat} mean ascending: {np.mean(ascending_angles)}")
    print(f"{sat} mean descending: {np.mean(descending_angles)}")
    print(f"{sat} sum of mean: {np.mean(ascending_angles) + np.mean(descending_angles)}")
    break

```

```python
import math
import os
from itertools import zip_longest
from numbers import Number

import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *

try:
    import nc_time_axis  # noqa: F401
except ImportError:
    nc_time_axis = None

# codex resume 019cf1aa-6bd9-7d61-8b47-51d2d4635320


def track_time_asn(ds, var_str="sla"):
    if "time" not in ds:
        return ds

    # time_var is also a two d variable, its dimension are
    # (points_numbers, cycles_numbers)
    time_var = ds["time"]
    cycle_dim = (
        "cycles_numbers"
        if "cycles_numbers" in time_var.dims
        else time_var.dims[-1]
    )

    # time is stored as (points_numbers, cycles_numbers); collapse to per-cycle.
    if "points_numbers" in time_var.dims:
        p_mid = time_var.sizes["points_numbers"] // 2
        time_1d = time_var.isel(points_numbers=p_mid)
    else:
        time_1d = time_var

    vals = pd.Series(time_1d.values).ffill().bfill().to_numpy()
    units = time_1d.attrs.get("units")
    calendar = time_1d.attrs.get("calendar", "standard")

    if np.issubdtype(np.asarray(vals).dtype, np.number) and units:
        decoded = xr.coding.times.decode_cf_datetime(
            np.asarray(vals), units, calendar=calendar
        )
    else:
        decoded = pd.to_datetime(vals, errors="coerce").to_numpy()

    ds = ds.assign_coords(time=(cycle_dim, decoded))
    if cycle_dim != "time":
        ds = ds.swap_dims({cycle_dim: "time"})
    return ds


def _to_list(values):
    if hasattr(values, "values"):
        values = values.values
    if hasattr(values, "ravel"):
        values = values.ravel()
    if hasattr(values, "tolist"):
        values = values.tolist()
    if isinstance(values, list):
        return values
    return [values]


def _format_timestamp(value, units=None, calendar="standard"):
    if value == "":
        return ""

    if isinstance(value, np.datetime64):
        return pd.to_datetime(value).strftime("%Y-%m-%dT%H:%M:%S")

    if hasattr(value, "strftime"):
        try:
            return value.strftime("%Y-%m-%dT%H:%M:%S")
        except (TypeError, ValueError):
            pass

    if isinstance(value, Number) and units:
        try:
            decoded = xr.coding.times.decode_cf_datetime(
                np.array([value]), units, calendar=calendar
            )[0]
            if hasattr(decoded, "strftime"):
                return decoded.strftime("%Y-%m-%dT%H:%M:%S")
            return pd.to_datetime(decoded).strftime("%Y-%m-%dT%H:%M:%S")
        except Exception:
            return str(value)

    if isinstance(value, str):
        parsed = pd.to_datetime(value, errors="coerce")
        if pd.notna(parsed):
            return parsed.strftime("%Y-%m-%dT%H:%M:%S")
        return value

    return str(value)


def print_timestamps_side_by_side(
    time_self1,
    time_other1,
    required_length=None,
    col_width=24,
    diff_col_width=12,
):
    attrs_self = getattr(time_self1, "attrs", {})
    attrs_other = getattr(time_other1, "attrs", {})
    units_self = attrs_self.get("units")
    units_other = attrs_other.get("units")
    cal_self = attrs_self.get("calendar", "standard")
    cal_other = attrs_other.get("calendar", "standard")

    left = [
        _format_timestamp(v, units=units_self, calendar=cal_self)
        for v in _to_list(time_self1)
```

```
    ]
    right = [
        _format_timestamp(v, units=units_other, calendar=cal_other)
        for v in _to_list(time_other1)
    ]

    if required_length is None:
        required_length = max(len(left), len(right))
    else:
        required_length = int(required_length)

    left = left[:required_length]
    right = right[:required_length]
    if len(left) < required_length:
        left.extend([""] * (required_length - len(left)))
    if len(right) < required_length:
        right.extend([""] * (required_length - len(right)))

    diffs = []
    for t_self, t_other in zip_longest(left, right, fillvalue=""):
        if not t_self or not t_other:
            diffs.append("")
            continue
        dt_self = pd.to_datetime(t_self, errors="coerce")
        dt_other = pd.to_datetime(t_other, errors="coerce")
        if pd.isna(dt_self) or pd.isna(dt_other):
            diffs.append("")
            continue
        diff_days = (dt_other - dt_self).total_seconds() / 86400.0
        diffs.append(f"{diff_days:.2f}")

    print(
        f"{'time_self1':<{col_width}} | "
        f"{'time_other1':<{col_width}} | "
        f"{'diff_days':>{diff_col_width}}"
    )
    print(
        f"{'-' * col_width}-+-{'-' * col_width}-+-{'-' * diff_col_width}"
    )
    for t_self, t_other, diff in zip_longest(
        left, right, diffs, fillvalue=""
    ):
        print(
            f"{t_self:<{col_width}} | {t_other:<{col_width}} | "
            f"{diff:>{diff_col_width}}"
        )


ioff = 3
frac1 = "0.05"
method = "loess"
out_dir = f"/home/srinivasu/xtrackm/guv/guv_at_intersections_{method}/"
os.makedirs(out_dir, exist_ok=True)

sat = "TP+J1+J2+J3+S6A"
df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
# sort by track_self (convert it to int if it's not already)
if df["track_self"].dtype == object:
    df["track_self"] = df["track_self"].astype(int)
df = df.sort_values("track_self")
for i, r in df.iterrows():
    sat1 = r["sat"]
    track_number_self = str(r["track_self"])
    track_number_other = str(r["track_other"])
    lons_inter1 = r["lons_inter"]
    lats_inter1 = r["lats_inter"]
    x_from_coast_self1 = r["x_from_coast_self"]
    x_from_coast_other1 = r["x_from_coast_other"]
    angle_acute = r["angle_acute"]
    angle_obtuse = r["angle_obtuse"]
    lon1 = lons_inter1
    lat1 = lats_inter1
    if abs(lat1) < 2:
        continue
    print(
        sat,
        track_number_self,
        track_number_other,
        "----------------------------------------",
    )
    f_self = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{track_number_other.zfill(3)}.nc"
    ds_self = xr.open_dataset(f_self, engine="netcdf4", decode_times=False)
    ds_other = xr.open_dataset(
        f_other, engine="netcdf4", decode_times=False
    )
    ds_self = track_time_asn(ds_self)
    ds_other = track_time_asn(ds_other)
    sla_self = ds_self.sla
    sla_other = ds_other.sla
    ii_self = index_at_lat(ds_self, lats_inter1)
    ii_other = index_at_lat(ds_other, lats_inter1)
    # times_self = ds_self.time
    # times_other = ds_other.time
    sla_self1 = sla_self.isel(points_numbers=ii_self, drop=True)
    sla_other1 = sla_other.isel(points_numbers=ii_other, drop=True)
    time_target = sla_self.isel(points_numbers=ii_self, drop=True).time.values
    time_self1 = sla_self.isel(points_numbers=ii_self).time
    time_other1 = sla_other.isel(points_numbers=ii_other).time
    sla_other_re = sla_other.reindex(
        time=time_target,
        method="nearest",
        tolerance=pd.Timedelta(days=10),
    )
    sla_other_sel = sla_other.sel(
        time=time_target,
        method="nearest",
        tolerance=pd.Timedelta(days=10),
    )
    sla_other_re1 = sla_other_re.isel(points_numbers=ii_other, drop=True)
    sla_other_sel1 = sla_other_sel.isel(points_numbers=ii_other, drop=True)
    time_other_re = sla_other_re.isel(points_numbers=ii_other).time
    time_other_sel = sla_other_sel.isel(points_numbers=ii_other).time
    print("original:")
    print_timestamps_side_by_side(time_self1, time_other1, required_length=2)
    print("reindex (relabeled to self time):")
    print_timestamps_side_by_side(time_self1, time_other_re, required_length=2)
```

```
    print("sel nearest (actual matched other time):")
    print_timestamps_side_by_side(time_self1, time_other_sel, required_length=2)
    sla_self1.plot(label="self", color="blue")
    sla_other_re1.plot(label="other (reindexed)", color="orange")
    sla_other_sel1.plot(label="other (sel nearest)", color="green")
    plt.legend()
    plt.show()
    if i > 10:
        break

```

meta/l.zero_len.tracks.py

```python
import glob
import xarray as xr
from namesm import *

for sat in sats_new:
    for f in sorted(
        glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")
    ):
        ds = xr.open_dataset(f, decode_times=False)
        trackn = ds.Pass
        if len(ds.lon.values) == 0:
            print(f"{sat} {trackn}")

```

```
dimensions:
    points_numbers = 210 ;
    cycles_numbers = 186 ;
variables:
    double bias(points_numbers, cycles_numbers) ;
        bias:_FillValue = NaN ;
        bias:units = "m" ;
        bias:long_name = "Global Mean Sea level Bias" ;
        bias:short_name = "bias" ;
    int64 cycle(cycles_numbers) ;
        cycle:long_name = "cycle" ;
        cycle:units = "count" ;
    int cycles_numbers(cycles_numbers) ;
        cycles_numbers:long_name = "cycle number" ;
        cycles_numbers:units = "count" ;
    double dac(points_numbers, cycles_numbers) ;
        dac:_FillValue = NaN ;
        dac:units = "m" ;
        dac:long_name = "Global Dynamic Atmospheric Corrections" ;
        dac:short_name = "DAC" ;
        dac:coordinates = "lon lat" ;
        dac:source = "TUGO high frequencies forced with analysed ERA5 pressur and wind field + inverse barometer low frequencies" ;
    double dist_to_coast_gshhg(points_numbers) ;
        dist_to_coast_gshhg:_FillValue = NaN ;
        dist_to_coast_gshhg:units = "m" ;
        dist_to_coast_gshhg:long_name = "Distance to nearest coastline" ;
        dist_to_coast_gshhg:comment = "Distance to nearest GSHHG 2.3.7 coastline in m" ;
    double dry(points_numbers, cycles_numbers) ;
        dry:_FillValue = NaN ;
        dry:units = "m" ;
        dry:short_name = "dry" ;
        dry:long_name = "Dry Tropospheric Correction" ;
    double iono(points_numbers, cycles_numbers) ;
        iono:_FillValue = NaN ;
        iono:units = "m" ;
        iono:short_name = "iono" ;
        iono:long_name = "Ionospheric Correction" ;
    double lat(points_numbers) ;
        lat:_FillValue = NaN ;
        lat:units = "degrees_north" ;
        lat:long_name = "Latitude" ;
        lat:short_name = "Lat" ;
    double lon(points_numbers) ;
        lon:_FillValue = NaN ;
        lon:units = "degrees_east" ;
        lon:long_name = "Longitude" ;
        lon:short_name = "Lon" ;
    float mdt(points_numbers) ;
        mdt:units = "m" ;
        mdt:long_name = "Mean Dynamic Topography CNES CLS 22" ;
        mdt:summary = "Global Mean Dynamic Topography calculated from the combination of altimetry, gravimetry (including GOCE and GRACE) and in-situ data. The reference time-period is 1993-2012." ;
        mdt:source = "Mean Dynamic Topography HYBRID_MDT_CNES_CLS22_CMEMS2020; https://www.aviso.altimetry.fr" ;
    double mssh(points_numbers) ;
        mssh:_FillValue = NaN ;
        mssh:units = "m" ;
        mssh:long_name = "Mean Sea Surface" ;
        mssh:short_name = "mssh" ;
        mssh:mssh_period = "Cycles from 38 to 209 are used for the mssh computation" ;
        mssh:comment = "X-TRACK Mean Sea Surface calculated by inversion of the corrected Sea Surface Heights" ;
    double ocean_tide(points_numbers, cycles_numbers) ;
        ocean_tide:_FillValue = NaN ;
        ocean_tide:units = "m" ;
        ocean_tide:short_name = "ocean_tide" ;
        ocean_tide:coordinates = "lon lat" ;
        ocean_tide:source = "FES 2022" ;
        ocean_tide:long_name = "Global FES22 tide correction" ;
        ocean_tide:comment = "Geocentric ocean tide, includes the corresponding loading tide\nand equilibrium long-period ocean tide height" ;
    int points_numbers(points_numbers) ;
        points_numbers:long_name = "along-track point numbers" ;
        points_numbers:units = "count" ;
    double pole_tide(points_numbers, cycles_numbers) ;
        pole_tide:_FillValue = NaN ;
        pole_tide:units = "m" ;
        pole_tide:long_name = "Pole tide" ;
        pole_tide:short_name = "pole_tide" ;
        pole_tide:coordinates = "lon lat" ;
        pole_tide:source = "Desai et al. 2015, Mean Pole Location 2017" ;
    double sigma0_ku(points_numbers, cycles_numbers) ;
        sigma0_ku:_FillValue = NaN ;
        sigma0_ku:long_name = "Backscatter coefficeient" ;
        sigma0_ku:short_name = "sigma0_ku" ;
        sigma0_ku:units = "dB" ;
    double sla(points_numbers, cycles_numbers) ;
        sla:_FillValue = NaN ;
        sla:units = "m" ;
        sla:long_name = "X-TRACK Mean Sea Level Anomaly" ;
        sla:short_name = "SLA" ;
        sla:coordinates = "lon lat" ;
    double solid_tide(points_numbers, cycles_numbers) ;
        solid_tide:_FillValue = NaN ;
        solid_tide:units = "m" ;
        solid_tide:long_name = "solid earth tide" ;
        solid_tide:short_name = "solid_tide" ;
        solid_tide:coordinates = "lon lat" ;
        solid_tide:source = "Elastic response to tidal potential (Cartwright and Tyler, 1971 ; Cartwright and Edden, 1973)" ;
    double ssb(points_numbers, cycles_numbers) ;
        ssb:_FillValue = NaN ;
        ssb:units = "m" ;
        ssb:short_name = "ssb" ;
        ssb:long_name = "Sea State Bias" ;
    double swh(points_numbers, cycles_numbers) ;
        swh:_FillValue = NaN ;
        swh:long_name = "Significant Wave Height" ;
        swh:short_name = "swh" ;
        swh:units = "m" ;
    double time(points_numbers, cycles_numbers) ;
        time:_FillValue = NaN ;
        time:units = "days since 1950-1-1" ;
        time:calendar = "julian" ;
        time:long_name = "Time of measurement" ;
        time:short_name = "Time" ;
    double wet(points_numbers, cycles_numbers) ;
        wet:_FillValue = NaN ;
        wet:units = "m" ;
        wet:long_name = "radiometer wet tropospheric correction" ;
        wet:short_name = "wet" ;
```

```
118
119 // global attributes:
120         :Institution = "CTOH/LEGOS, Toulouse Univ., CNRS, IRD, CNES, UPS, France" ;
121         :Date = "2024-12-09" ;
122         :Pass = "8" ;
123         :Mission = "GFO" ;
124         :Title = "1 Hz X-TRACK-L2P Sea Level Anomalies" ;
125         :Version = "Version v2.2" ;
126         :Conventions = "CF Version CF-1.6" ;
127         :Summary = "This dataset contains alongtrack sea level anomaly time series at 1Hz\ncomputed from the L2P products with the X-TRACK post-processing algorithm" ;
128         :doi = "10.24400/527896/A01-2022.020" ;
129 }
130
```

```
 1
 2  Total data points: 63426
 3  Originally valid: 49889 (78.7%)
 4  Outliers detected: 1 (0.0%)
 5  Final valid points: 49888 (78.7%)
 6
 7  === Outlier Detection Summary ===
 8  Total data points: 84816
 9  Originally valid: 67282 (79.3%)
10  Outliers detected: 2 (0.0%)
11  Final valid points: 67280 (79.3%)
12
13  === Outlier Detection Summary ===
14  Total data points: 33108
15  Originally valid: 24961 (75.4%)
16  Outliers detected: 2 (0.0%)
17  Final valid points: 24959 (75.4%)
18
19  === Outlier Detection Summary ===
20  Total data points: 85002
21  Originally valid: 68996 (81.2%)
22  Outliers detected: 1 (0.0%)
23  Final valid points: 68995 (81.2%)
24
25  === Outlier Detection Summary ===
26  Total data points: 51150
27  Originally valid: 40424 (79.0%)
28  Outliers detected: 3 (0.0%)
29  Final valid points: 40421 (79.0%)
30
31  === Outlier Detection Summary ===
32  Total data points: 56358
33  Originally valid: 44317 (78.6%)
34  Outliers detected: 1 (0.0%)
35  Final valid points: 44316 (78.6%)
36
37  === Outlier Detection Summary ===
38  Total data points: 57474
39  Originally valid: 45907 (79.9%)
40  Outliers detected: 2 (0.0%)
41  Final valid points: 45905 (79.9%)
42
43  === Outlier Detection Summary ===
44  Total data points: 69378
45  Originally valid: 56003 (80.7%)
46  Outliers detected: 2 (0.0%)
47  Final valid points: 56001 (80.7%)
48
49  === Outlier Detection Summary ===
50  Total data points: 60078
51  Originally valid: 48519 (80.8%)
52  Outliers detected: 1 (0.0%)
53  Final valid points: 48518 (80.8%)
54
55  === Outlier Detection Summary ===
56  Total data points: 63054
57  Originally valid: 49891 (79.1%)
58  Outliers detected: 2 (0.0%)
59  Final valid points: 49889 (79.1%)
60
61  === Outlier Detection Summary ===
62  Total data points: 66960
63  Originally valid: 52461 (78.3%)
64  Outliers detected: 2 (0.0%)
65  Final valid points: 52459 (78.3%)
66
67  === Outlier Detection Summary ===
68  Total data points: 37944
69  Originally valid: 28966 (76.3%)
70  Outliers detected: 1 (0.0%)
71  Final valid points: 28965 (76.3%)
72
73  === Outlier Detection Summary ===
74  Total data points: 84816
75  Originally valid: 66621 (78.5%)
76  Outliers detected: 1 (0.0%)
77  Final valid points: 66620 (78.5%)
78
79  === Outlier Detection Summary ===
80  Total data points: 56544
81  Originally valid: 44225 (78.2%)
82  Outliers detected: 2 (0.0%)
83  Final valid points: 44223 (78.2%)
84
85  === Outlier Detection Summary ===
86  Total data points: 23808
87  Originally valid: 17534 (73.6%)
88  Outliers detected: 1 (0.0%)
89  Final valid points: 17533 (73.6%)
90
91  === Outlier Detection Summary ===
92  Total data points: 58032
93  Originally valid: 46092 (79.4%)
94  Outliers detected: 1 (0.0%)
95  Final valid points: 46091 (79.4%)
96
97  === Outlier Detection Summary ===
98  Total data points: 63984
99  Originally valid: 50952 (79.6%)
100 Outliers detected: 1 (0.0%)
101 Final valid points: 50951 (79.6%)
102
103 === Outlier Detection Summary ===
104 Total data points: 32890
105 Originally valid: 31285 (95.1%)
106 Outliers detected: 1 (0.0%)
107 Final valid points: 31284 (95.1%)
108
109 === Outlier Detection Summary ===
110 Total data points: 49105
111 Originally valid: 47215 (96.2%)
112 Outliers detected: 1 (0.0%)
113 Final valid points: 47214 (96.1%)
114
115 === Outlier Detection Summary ===
116 Total data points: 47955
117 Originally valid: 46124 (96.2%)
```

```
Outliers detected: 1 (0.0%)
Final valid points: 46123 (96.2%)

=== Outlier Detection Summary ===
Total data points: 48875
Originally valid: 46830 (95.8%)
Outliers detected: 14 (0.0%)
Final valid points: 46816 (95.8%)

=== Outlier Detection Summary ===
Total data points: 49220
Originally valid: 47112 (95.7%)
Outliers detected: 1 (0.0%)
Final valid points: 47111 (95.7%)

=== Outlier Detection Summary ===
Total data points: 49335
Originally valid: 47836 (97.0%)
Outliers detected: 2 (0.0%)
Final valid points: 47834 (97.0%)

=== Outlier Detection Summary ===
Total data points: 17710
Originally valid: 17094 (96.5%)
Outliers detected: 1 (0.0%)
Final valid points: 17093 (96.5%)

=== Outlier Detection Summary ===
Total data points: 41860
Originally valid: 39710 (94.9%)
Outliers detected: 1 (0.0%)
Final valid points: 39709 (94.9%)

=== Outlier Detection Summary ===
Total data points: 24035
Originally valid: 21764 (90.6%)
Outliers detected: 1 (0.0%)
Final valid points: 21763 (90.5%)

=== Outlier Detection Summary ===
Total data points: 3680
Originally valid: 3414 (92.8%)
Outliers detected: 1 (0.0%)
Final valid points: 3413 (92.7%)

=== Outlier Detection Summary ===
Total data points: 7600
Originally valid: 7352 (96.7%)
Outliers detected: 1 (0.0%)
Final valid points: 7351 (96.7%)

=== Outlier Detection Summary ===
Total data points: 21920
Originally valid: 21191 (96.7%)
Outliers detected: 1 (0.0%)
Final valid points: 21190 (96.7%)

=== Outlier Detection Summary ===
Total data points: 25520
Originally valid: 24594 (96.4%)
Outliers detected: 1 (0.0%)
Final valid points: 24593 (96.4%)

=== Outlier Detection Summary ===
Total data points: 20480
Originally valid: 19738 (96.4%)
Outliers detected: 2 (0.0%)
Final valid points: 19736 (96.4%)

=== Outlier Detection Summary ===
Total data points: 20880
Originally valid: 20265 (97.1%)
Outliers detected: 1 (0.0%)
Final valid points: 20264 (97.0%)

=== Outlier Detection Summary ===
Total data points: 34000
Originally valid: 32995 (97.0%)
Outliers detected: 1 (0.0%)
Final valid points: 32994 (97.0%)

=== Outlier Detection Summary ===
Total data points: 22480
Originally valid: 21613 (96.1%)
Outliers detected: 1 (0.0%)
Final valid points: 21612 (96.1%)

=== Outlier Detection Summary ===
Total data points: 34160
Originally valid: 32935 (96.4%)
Outliers detected: 2 (0.0%)
Final valid points: 32933 (96.4%)

=== Outlier Detection Summary ===
Total data points: 34400
Originally valid: 33244 (96.6%)
Outliers detected: 2 (0.0%)
Final valid points: 33242 (96.6%)

=== Outlier Detection Summary ===
Total data points: 25600
Originally valid: 24788 (96.8%)
Outliers detected: 1 (0.0%)
Final valid points: 24787 (96.8%)

=== Outlier Detection Summary ===
Total data points: 32320
Originally valid: 30582 (94.6%)
Outliers detected: 1 (0.0%)
Final valid points: 30581 (94.6%)

=== Outlier Detection Summary ===
Total data points: 34160
Originally valid: 33100 (96.9%)
Outliers detected: 1 (0.0%)
Final valid points: 33099 (96.9%)

```

```
=== Outlier Detection Summary ===
Total data points: 34080
Originally valid: 32991 (96.8%)
Outliers detected: 1 (0.0%)
Final valid points: 32990 (96.8%)

=== Outlier Detection Summary ===
Total data points: 34160
Originally valid: 33101 (96.9%)
Outliers detected: 1 (0.0%)
Final valid points: 33100 (96.9%)

=== Outlier Detection Summary ===
Total data points: 34080
Originally valid: 32544 (95.5%)
Outliers detected: 2 (0.0%)
Final valid points: 32542 (95.5%)

=== Outlier Detection Summary ===
Total data points: 34160
Originally valid: 33097 (96.9%)
Outliers detected: 2 (0.0%)
Final valid points: 33095 (96.9%)

=== Outlier Detection Summary ===
Total data points: 31120
Originally valid: 29616 (95.2%)
Outliers detected: 1 (0.0%)
Final valid points: 29615 (95.2%)

=== Outlier Detection Summary ===
Total data points: 14240
Originally valid: 13601 (95.5%)
Outliers detected: 1 (0.0%)
Final valid points: 13600 (95.5%)

=== Outlier Detection Summary ===
Total data points: 29040
Originally valid: 27955 (96.3%)
Outliers detected: 1 (0.0%)
Final valid points: 27954 (96.3%)

=== Outlier Detection Summary ===
Total data points: 17680
Originally valid: 16859 (95.4%)
Outliers detected: 1 (0.0%)
Final valid points: 16858 (95.4%)

=== Outlier Detection Summary ===
Total data points: 29440
Originally valid: 28092 (95.4%)
Outliers detected: 3 (0.0%)
Final valid points: 28089 (95.4%)

=== Outlier Detection Summary ===
Total data points: 3920
Originally valid: 2491 (63.5%)
Outliers detected: 1 (0.0%)
Final valid points: 2490 (63.5%)

=== Outlier Detection Summary ===
Total data points: 492
Originally valid: 151 (30.7%)
Outliers detected: 1 (0.2%)
Final valid points: 150 (30.5%)

=== Outlier Detection Summary ===
Total data points: 1230
Originally valid: 822 (66.8%)
Outliers detected: 2 (0.2%)
Final valid points: 820 (66.7%)

=== Outlier Detection Summary ===
Total data points: 43296
Originally valid: 33245 (76.8%)
Outliers detected: 1 (0.0%)
Final valid points: 33244 (76.8%)

=== Outlier Detection Summary ===
Total data points: 61500
Originally valid: 46493 (75.6%)
Outliers detected: 2 (0.0%)
Final valid points: 46491 (75.6%)

=== Outlier Detection Summary ===
Total data points: 77736
Originally valid: 57627 (74.1%)
Outliers detected: 2 (0.0%)
Final valid points: 57625 (74.1%)

=== Outlier Detection Summary ===
Total data points: 58056
Originally valid: 45813 (78.9%)
Outliers detected: 1 (0.0%)
Final valid points: 45812 (78.9%)

=== Outlier Detection Summary ===
Total data points: 63714
Originally valid: 49355 (77.5%)
Outliers detected: 1 (0.0%)
Final valid points: 49354 (77.5%)

=== Outlier Detection Summary ===
Total data points: 76260
Originally valid: 60128 (78.8%)
Outliers detected: 2 (0.0%)
Final valid points: 60126 (78.8%)

=== Outlier Detection Summary ===
Total data points: 79458
Originally valid: 60864 (76.6%)
Outliers detected: 1 (0.0%)
Final valid points: 60863 (76.6%)

=== Outlier Detection Summary ===
Total data points: 65928
Originally valid: 46764 (70.9%)
```

```
Outliers detected: 1 (0.0%)
Final valid points: 46763 (70.9%)

=== Outlier Detection Summary ===
Total data points: 88314
Originally valid: 67234 (76.1%)
Outliers detected: 4 (0.0%)
Final valid points: 67230 (76.1%)

=== Outlier Detection Summary ===
Total data points: 40590
Originally valid: 25497 (62.8%)
Outliers detected: 3 (0.0%)
Final valid points: 25494 (62.8%)

=== Outlier Detection Summary ===
Total data points: 77982
Originally valid: 58834 (75.4%)
Outliers detected: 2 (0.0%)
Final valid points: 58832 (75.4%)

=== Outlier Detection Summary ===
Total data points: 61254
Originally valid: 43606 (71.2%)
Outliers detected: 8 (0.0%)
Final valid points: 43598 (71.2%)

=== Outlier Detection Summary ===
Total data points: 21648
Originally valid: 13702 (63.3%)
Outliers detected: 1 (0.0%)
Final valid points: 13701 (63.3%)

=== Outlier Detection Summary ===
Total data points: 5610
Originally valid: 4730 (84.3%)
Outliers detected: 1 (0.0%)
Final valid points: 4729 (84.3%)

=== Outlier Detection Summary ===
Total data points: 15810
Originally valid: 13522 (85.5%)
Outliers detected: 1 (0.0%)
Final valid points: 13521 (85.5%)

=== Outlier Detection Summary ===
Total data points: 19992
Originally valid: 17578 (87.9%)
Outliers detected: 5 (0.0%)
Final valid points: 17573 (87.9%)

=== Outlier Detection Summary ===
Total data points: 22440
Originally valid: 18289 (81.5%)
Outliers detected: 1 (0.0%)
Final valid points: 18288 (81.5%)

=== Outlier Detection Summary ===
Total data points: 22593
Originally valid: 19268 (85.3%)
Outliers detected: 3 (0.0%)
Final valid points: 19265 (85.3%)

=== Outlier Detection Summary ===
Total data points: 15147
Originally valid: 12600 (83.2%)
Outliers detected: 9 (0.1%)
Final valid points: 12591 (83.1%)

=== Outlier Detection Summary ===
Total data points: 22695
Originally valid: 20636 (90.9%)
Outliers detected: 7 (0.0%)
Final valid points: 20629 (90.9%)

=== Outlier Detection Summary ===
Total data points: 16014
Originally valid: 13717 (85.7%)
Outliers detected: 1 (0.0%)
Final valid points: 13716 (85.7%)

=== Outlier Detection Summary ===
Total data points: 22134
Originally valid: 19396 (87.6%)
Outliers detected: 1 (0.0%)
Final valid points: 19395 (87.6%)

=== Outlier Detection Summary ===
Total data points: 16881
Originally valid: 14103 (83.5%)
Outliers detected: 3 (0.0%)
Final valid points: 14100 (83.5%)

=== Outlier Detection Summary ===
Total data points: 20910
Originally valid: 19603 (93.7%)
Outliers detected: 5 (0.0%)
Final valid points: 19598 (93.7%)

=== Outlier Detection Summary ===
Total data points: 18054
Originally valid: 16804 (93.1%)
Outliers detected: 1 (0.0%)
Final valid points: 16803 (93.1%)

=== Outlier Detection Summary ===
Total data points: 19278
Originally valid: 16888 (87.6%)
Outliers detected: 3 (0.0%)
Final valid points: 16885 (87.6%)

=== Outlier Detection Summary ===
Total data points: 18717
Originally valid: 17276 (92.3%)
Outliers detected: 5 (0.0%)
Final valid points: 17271 (92.3%)

```

```
469  === Outlier Detection Summary ===
470  Total data points: 22542
471  Originally valid: 20196 (89.6%)
472  Outliers detected: 3 (0.0%)
473  Final valid points: 20193 (89.6%)
474
475  === Outlier Detection Summary ===
476  Total data points: 16677
477  Originally valid: 13668 (82.0%)
478  Outliers detected: 3 (0.0%)
479  Final valid points: 13665 (81.9%)
480
481  === Outlier Detection Summary ===
482  Total data points: 22593
483  Originally valid: 20612 (91.2%)
484  Outliers detected: 1 (0.0%)
485  Final valid points: 20611 (91.2%)
486
487  === Outlier Detection Summary ===
488  Total data points: 22644
489  Originally valid: 19705 (87.0%)
490  Outliers detected: 3 (0.0%)
491  Final valid points: 19702 (87.0%)
492
493  === Outlier Detection Summary ===
494  Total data points: 10914
495  Originally valid: 8426 (77.2%)
496  Outliers detected: 2 (0.0%)
497  Final valid points: 8424 (77.2%)
498
499  === Outlier Detection Summary ===
500  Total data points: 22287
501  Originally valid: 19197 (86.1%)
502  Outliers detected: 5 (0.0%)
503  Final valid points: 19192 (86.1%)
504
505  === Outlier Detection Summary ===
506  Total data points: 8823
507  Originally valid: 7638 (86.6%)
508  Outliers detected: 1 (0.0%)
509  Final valid points: 7637 (86.6%)
510
511  === Outlier Detection Summary ===
512  Total data points: 19635
513  Originally valid: 16975 (86.5%)
514  Outliers detected: 1 (0.0%)
515  Final valid points: 16974 (86.4%)
516
517  === Outlier Detection Summary ===
518  Total data points: 14229
519  Originally valid: 11947 (84.0%)
520  Outliers detected: 1 (0.0%)
521  Final valid points: 11946 (84.0%)
522
523  === Outlier Detection Summary ===
524  Total data points: 14280
525  Originally valid: 12796 (89.6%)
526  Outliers detected: 2 (0.0%)
527  Final valid points: 12794 (89.6%)
528
529  === Outlier Detection Summary ===
530  Total data points: 17799
531  Originally valid: 14606 (82.1%)
532  Outliers detected: 3 (0.0%)
533  Final valid points: 14603 (82.0%)
534
535  === Outlier Detection Summary ===
536  Total data points: 5916
537  Originally valid: 5025 (84.9%)
538  Outliers detected: 1 (0.0%)
539  Final valid points: 5024 (84.9%)
540
541  === Outlier Detection Summary ===
542  Total data points: 19278
543  Originally valid: 15869 (82.3%)
544  Outliers detected: 1 (0.0%)
545  Final valid points: 15868 (82.3%)
546
547  === Outlier Detection Summary ===
548  Total data points: 14076
549  Originally valid: 11467 (81.5%)
550  Outliers detected: 13 (0.1%)
551  Final valid points: 11454 (81.4%)
552
553  === Outlier Detection Summary ===
554  Total data points: 19431
555  Originally valid: 17656 (90.9%)
556  Outliers detected: 2 (0.0%)
557  Final valid points: 17654 (90.9%)
558
559  === Outlier Detection Summary ===
560  Total data points: 18870
561  Originally valid: 15858 (84.0%)
562  Outliers detected: 1 (0.0%)
563  Final valid points: 15857 (84.0%)
564
565  === Outlier Detection Summary ===
566  Total data points: 16065
567  Originally valid: 13175 (82.0%)
568  Outliers detected: 2 (0.0%)
569  Final valid points: 13173 (82.0%)
570
571  === Outlier Detection Summary ===
572  Total data points: 17238
573  Originally valid: 15879 (92.1%)
574  Outliers detected: 1 (0.0%)
575  Final valid points: 15878 (92.1%)
576
577  === Outlier Detection Summary ===
578  Total data points: 17289
579  Originally valid: 16142 (93.4%)
580  Outliers detected: 4 (0.0%)
581  Final valid points: 16138 (93.3%)
582
583  === Outlier Detection Summary ===
584  Total data points: 14229
585  Originally valid: 11577 (81.4%)
```

```
Outliers detected: 1 (0.0%)
Final valid points: 11576 (81.4%)

=== Outlier Detection Summary ===
Total data points: 14688
Originally valid: 12696 (86.4%)
Outliers detected: 3 (0.0%)
Final valid points: 12693 (86.4%)

=== Outlier Detection Summary ===
Total data points: 19278
Originally valid: 16322 (84.7%)
Outliers detected: 2 (0.0%)
Final valid points: 16320 (84.7%)

=== Outlier Detection Summary ===
Total data points: 12036
Originally valid: 9282 (77.1%)
Outliers detected: 1 (0.0%)
Final valid points: 9281 (77.1%)

=== Outlier Detection Summary ===
Total data points: 19125
Originally valid: 17305 (90.5%)
Outliers detected: 2 (0.0%)
Final valid points: 17303 (90.5%)

=== Outlier Detection Summary ===
Total data points: 11214
Originally valid: 7782 (69.4%)
Outliers detected: 1 (0.0%)
Final valid points: 7781 (69.4%)

=== Outlier Detection Summary ===
Total data points: 18585
Originally valid: 13773 (74.1%)
Outliers detected: 1 (0.0%)
Final valid points: 13772 (74.1%)

=== Outlier Detection Summary ===
Total data points: 19719
Originally valid: 13126 (66.6%)
Outliers detected: 4 (0.0%)
Final valid points: 13122 (66.5%)

=== Outlier Detection Summary ===
Total data points: 12348
Originally valid: 9400 (76.1%)
Outliers detected: 1 (0.0%)
Final valid points: 9399 (76.1%)

=== Outlier Detection Summary ===
Total data points: 24633
Originally valid: 17046 (69.2%)
Outliers detected: 10 (0.0%)
Final valid points: 17036 (69.2%)

=== Outlier Detection Summary ===
Total data points: 16632
Originally valid: 12064 (72.5%)
Outliers detected: 2 (0.0%)
Final valid points: 12062 (72.5%)

=== Outlier Detection Summary ===
Total data points: 27909
Originally valid: 18540 (66.4%)
Outliers detected: 1 (0.0%)
Final valid points: 18539 (66.4%)

=== Outlier Detection Summary ===
Total data points: 18711
Originally valid: 12510 (66.9%)
Outliers detected: 2 (0.0%)
Final valid points: 12508 (66.8%)

=== Outlier Detection Summary ===
Total data points: 28098
Originally valid: 17743 (63.1%)
Outliers detected: 4 (0.0%)
Final valid points: 17739 (63.1%)

=== Outlier Detection Summary ===
Total data points: 19719
Originally valid: 14573 (73.9%)
Outliers detected: 3 (0.0%)
Final valid points: 14570 (73.9%)

=== Outlier Detection Summary ===
Total data points: 27405
Originally valid: 19038 (69.5%)
Outliers detected: 2 (0.0%)
Final valid points: 19036 (69.5%)

=== Outlier Detection Summary ===
Total data points: 22302
Originally valid: 16217 (72.7%)
Outliers detected: 1 (0.0%)
Final valid points: 16216 (72.7%)

=== Outlier Detection Summary ===
Total data points: 24003
Originally valid: 16970 (70.7%)
Outliers detected: 1 (0.0%)
Final valid points: 16969 (70.7%)

=== Outlier Detection Summary ===
Total data points: 27783
Originally valid: 18257 (65.7%)
Outliers detected: 2 (0.0%)
Final valid points: 18255 (65.7%)

=== Outlier Detection Summary ===
Total data points: 16947
Originally valid: 11875 (70.1%)
Outliers detected: 2 (0.0%)
Final valid points: 11873 (70.1%)

```

```
=== Outlier Detection Summary ===
Total data points: 27972
Originally valid: 20462 (73.2%)
Outliers detected: 1 (0.0%)
Final valid points: 20461 (73.1%)

=== Outlier Detection Summary ===
Total data points: 13545
Originally valid: 9023 (66.6%)
Outliers detected: 2 (0.0%)
Final valid points: 9021 (66.6%)

=== Outlier Detection Summary ===
Total data points: 24066
Originally valid: 16775 (69.7%)
Outliers detected: 2 (0.0%)
Final valid points: 16773 (69.7%)

=== Outlier Detection Summary ===
Total data points: 9009
Originally valid: 6027 (66.9%)
Outliers detected: 2 (0.0%)
Final valid points: 6025 (66.9%)

=== Outlier Detection Summary ===
Total data points: 23058
Originally valid: 15682 (68.0%)
Outliers detected: 2 (0.0%)
Final valid points: 15680 (68.0%)

=== Outlier Detection Summary ===
Total data points: 17703
Originally valid: 11731 (66.3%)
Outliers detected: 5 (0.0%)
Final valid points: 11726 (66.2%)

=== Outlier Detection Summary ===
Total data points: 9891
Originally valid: 7136 (72.1%)
Outliers detected: 1 (0.0%)
Final valid points: 7135 (72.1%)

=== Outlier Detection Summary ===
Total data points: 22050
Originally valid: 17073 (77.4%)
Outliers detected: 1 (0.0%)
Final valid points: 17072 (77.4%)

=== Outlier Detection Summary ===
Total data points: 23877
Originally valid: 17128 (71.7%)
Outliers detected: 1 (0.0%)
Final valid points: 17127 (71.7%)

=== Outlier Detection Summary ===
Total data points: 24129
Originally valid: 16462 (68.2%)
Outliers detected: 1 (0.0%)
Final valid points: 16461 (68.2%)

=== Outlier Detection Summary ===
Total data points: 23310
Originally valid: 15826 (67.9%)
Outliers detected: 1 (0.0%)
Final valid points: 15825 (67.9%)

=== Outlier Detection Summary ===
Total data points: 19782
Originally valid: 12691 (64.2%)
Outliers detected: 2 (0.0%)
Final valid points: 12689 (64.1%)

=== Outlier Detection Summary ===
Total data points: 21420
Originally valid: 14393 (67.2%)
Outliers detected: 1 (0.0%)
Final valid points: 14392 (67.2%)

=== Outlier Detection Summary ===
Total data points: 21357
Originally valid: 14254 (66.7%)
Outliers detected: 3 (0.0%)
Final valid points: 14251 (66.7%)

=== Outlier Detection Summary ===
Total data points: 17514
Originally valid: 12021 (68.6%)
Outliers detected: 1 (0.0%)
Final valid points: 12020 (68.6%)

=== Outlier Detection Summary ===
Total data points: 18207
Originally valid: 12165 (66.8%)
Outliers detected: 2 (0.0%)
Final valid points: 12163 (66.8%)

=== Outlier Detection Summary ===
Total data points: 23877
Originally valid: 16142 (67.6%)
Outliers detected: 1 (0.0%)
Final valid points: 16141 (67.6%)

=== Outlier Detection Summary ===
Total data points: 14490
Originally valid: 9314 (64.3%)
Outliers detected: 1 (0.0%)
Final valid points: 9313 (64.3%)

=== Outlier Detection Summary ===
Total data points: 23625
Originally valid: 15121 (64.0%)
Outliers detected: 2 (0.0%)
Final valid points: 15119 (64.0%)

=== Outlier Detection Summary ===
Total data points: 6552
Originally valid: 4071 (62.1%)
```

```
Outliers detected: 1 (0.0%)
Final valid points: 4070 (62.1%)

=== Outlier Detection Summary ===
Total data points: 22302
Originally valid: 15229 (68.3%)
Outliers detected: 4 (0.0%)
Final valid points: 15225 (68.3%)

=== Outlier Detection Summary ===
Total data points: 63426
Originally valid: 49889 (78.7%)
Outliers detected: 1 (0.0%)
Final valid points: 49888 (78.7%)

=== Outlier Detection Summary ===
Total data points: 84816
Originally valid: 67282 (79.3%)
Outliers detected: 2 (0.0%)
Final valid points: 67280 (79.3%)

=== Outlier Detection Summary ===
Total data points: 33108
Originally valid: 24961 (75.4%)
Outliers detected: 2 (0.0%)
Final valid points: 24959 (75.4%)

=== Outlier Detection Summary ===
Total data points: 85002
Originally valid: 68996 (81.2%)
Outliers detected: 1 (0.0%)
Final valid points: 68995 (81.2%)

=== Outlier Detection Summary ===
Total data points: 51150
Originally valid: 40424 (79.0%)
Outliers detected: 3 (0.0%)
Final valid points: 40421 (79.0%)

=== Outlier Detection Summary ===
Total data points: 56358
Originally valid: 44317 (78.6%)
Outliers detected: 1 (0.0%)
Final valid points: 44316 (78.6%)

=== Outlier Detection Summary ===
Total data points: 57474
Originally valid: 45907 (79.9%)
Outliers detected: 2 (0.0%)
Final valid points: 45905 (79.9%)

=== Outlier Detection Summary ===
Total data points: 69378
Originally valid: 56003 (80.7%)
Outliers detected: 2 (0.0%)
Final valid points: 56001 (80.7%)

=== Outlier Detection Summary ===
Total data points: 60078
Originally valid: 48519 (80.8%)
Outliers detected: 1 (0.0%)
Final valid points: 48518 (80.8%)

=== Outlier Detection Summary ===
Total data points: 63054
Originally valid: 49891 (79.1%)
Outliers detected: 2 (0.0%)
Final valid points: 49889 (79.1%)

=== Outlier Detection Summary ===
Total data points: 66960
Originally valid: 52461 (78.3%)
Outliers detected: 2 (0.0%)
Final valid points: 52459 (78.3%)

=== Outlier Detection Summary ===
Total data points: 37944
Originally valid: 28966 (76.3%)
Outliers detected: 1 (0.0%)
Final valid points: 28965 (76.3%)

=== Outlier Detection Summary ===
Total data points: 84816
Originally valid: 66621 (78.5%)
Outliers detected: 1 (0.0%)
Final valid points: 66620 (78.5%)

=== Outlier Detection Summary ===
Total data points: 56544
Originally valid: 44225 (78.2%)
Outliers detected: 2 (0.0%)
Final valid points: 44223 (78.2%)

=== Outlier Detection Summary ===
Total data points: 23808
Originally valid: 17534 (73.6%)
Outliers detected: 1 (0.0%)
Final valid points: 17533 (73.6%)

=== Outlier Detection Summary ===
Total data points: 58032
Originally valid: 46092 (79.4%)
Outliers detected: 1 (0.0%)
Final valid points: 46091 (79.4%)

=== Outlier Detection Summary ===
Total data points: 63984
Originally valid: 50952 (79.6%)
Outliers detected: 1 (0.0%)
Final valid points: 50951 (79.6%)

=== Outlier Detection Summary ===
Total data points: 32890
Originally valid: 31285 (95.1%)
Outliers detected: 1 (0.0%)
Final valid points: 31284 (95.1%)

```

```
=== Outlier Detection Summary ===
Total data points: 49105
Originally valid: 47215 (96.2%)
Outliers detected: 1 (0.0%)
Final valid points: 47214 (96.1%)

=== Outlier Detection Summary ===
Total data points: 47955
Originally valid: 46124 (96.2%)
Outliers detected: 1 (0.0%)
Final valid points: 46123 (96.2%)

=== Outlier Detection Summary ===
Total data points: 48875
Originally valid: 46830 (95.8%)
Outliers detected: 14 (0.0%)
Final valid points: 46816 (95.8%)

=== Outlier Detection Summary ===
Total data points: 49220
Originally valid: 47112 (95.7%)
Outliers detected: 1 (0.0%)
Final valid points: 47111 (95.7%)

=== Outlier Detection Summary ===
Total data points: 49335
Originally valid: 47836 (97.0%)
Outliers detected: 2 (0.0%)
Final valid points: 47834 (97.0%)

=== Outlier Detection Summary ===
Total data points: 17710
Originally valid: 17094 (96.5%)
Outliers detected: 1 (0.0%)
Final valid points: 17093 (96.5%)

=== Outlier Detection Summary ===
Total data points: 41860
Originally valid: 39710 (94.9%)
Outliers detected: 1 (0.0%)
Final valid points: 39709 (94.9%)

=== Outlier Detection Summary ===
Total data points: 24035
Originally valid: 21764 (90.6%)
Outliers detected: 1 (0.0%)
Final valid points: 21763 (90.5%)

=== Outlier Detection Summary ===
Total data points: 3680
Originally valid: 3414 (92.8%)
Outliers detected: 1 (0.0%)
Final valid points: 3413 (92.7%)

=== Outlier Detection Summary ===
Total data points: 7600
Originally valid: 7352 (96.7%)
Outliers detected: 1 (0.0%)
Final valid points: 7351 (96.7%)

=== Outlier Detection Summary ===
Total data points: 21920
Originally valid: 21191 (96.7%)
Outliers detected: 1 (0.0%)
Final valid points: 21190 (96.7%)

=== Outlier Detection Summary ===
Total data points: 25520
Originally valid: 24594 (96.4%)
Outliers detected: 1 (0.0%)
Final valid points: 24593 (96.4%)

=== Outlier Detection Summary ===
Total data points: 20480
Originally valid: 19738 (96.4%)
Outliers detected: 2 (0.0%)
Final valid points: 19736 (96.4%)

=== Outlier Detection Summary ===
Total data points: 20880
Originally valid: 20265 (97.1%)
Outliers detected: 1 (0.0%)
Final valid points: 20264 (97.0%)

=== Outlier Detection Summary ===
Total data points: 34000
Originally valid: 32995 (97.0%)
Outliers detected: 1 (0.0%)
Final valid points: 32994 (97.0%)

=== Outlier Detection Summary ===
Total data points: 22480
Originally valid: 21613 (96.1%)
Outliers detected: 1 (0.0%)
Final valid points: 21612 (96.1%)

=== Outlier Detection Summary ===
Total data points: 34160
Originally valid: 32935 (96.4%)
Outliers detected: 2 (0.0%)
Final valid points: 32933 (96.4%)

=== Outlier Detection Summary ===
Total data points: 34400
Originally valid: 33244 (96.6%)
Outliers detected: 2 (0.0%)
Final valid points: 33242 (96.6%)

=== Outlier Detection Summary ===
Total data points: 25600
Originally valid: 24788 (96.8%)
Outliers detected: 1 (0.0%)
Final valid points: 24787 (96.8%)

=== Outlier Detection Summary ===
Total data points: 32320
Originally valid: 30582 (94.6%)
```

```
Outliers detected: 1 (0.0%)
Final valid points: 30581 (94.6%)

=== Outlier Detection Summary ===
Total data points: 34160
Originally valid: 33100 (96.9%)
Outliers detected: 1 (0.0%)
Final valid points: 33099 (96.9%)

=== Outlier Detection Summary ===
Total data points: 34080
Originally valid: 32991 (96.8%)
Outliers detected: 1 (0.0%)
Final valid points: 32990 (96.8%)

=== Outlier Detection Summary ===
Total data points: 34160
Originally valid: 33101 (96.9%)
Outliers detected: 1 (0.0%)
Final valid points: 33100 (96.9%)

=== Outlier Detection Summary ===
Total data points: 34080
Originally valid: 32544 (95.5%)
Outliers detected: 2 (0.0%)
Final valid points: 32542 (95.5%)

=== Outlier Detection Summary ===
Total data points: 34160
Originally valid: 33097 (96.9%)
Outliers detected: 2 (0.0%)
Final valid points: 33095 (96.9%)

=== Outlier Detection Summary ===
Total data points: 31120
Originally valid: 29616 (95.2%)
Outliers detected: 1 (0.0%)
Final valid points: 29615 (95.2%)

=== Outlier Detection Summary ===
Total data points: 14240
Originally valid: 13601 (95.5%)
Outliers detected: 1 (0.0%)
Final valid points: 13600 (95.5%)

=== Outlier Detection Summary ===
Total data points: 29040
Originally valid: 27955 (96.3%)
Outliers detected: 1 (0.0%)
Final valid points: 27954 (96.3%)

=== Outlier Detection Summary ===
Total data points: 17680
Originally valid: 16859 (95.4%)
Outliers detected: 1 (0.0%)
Final valid points: 16858 (95.4%)

=== Outlier Detection Summary ===
Total data points: 29440
Originally valid: 28092 (95.4%)
Outliers detected: 3 (0.0%)
Final valid points: 28089 (95.4%)

=== Outlier Detection Summary ===
Total data points: 3920
Originally valid: 2491 (63.5%)
Outliers detected: 1 (0.0%)
Final valid points: 2490 (63.5%)

=== Outlier Detection Summary ===
Total data points: 492
Originally valid: 151 (30.7%)
Outliers detected: 1 (0.2%)
Final valid points: 150 (30.5%)

=== Outlier Detection Summary ===
Total data points: 1230
Originally valid: 822 (66.8%)
Outliers detected: 2 (0.2%)
Final valid points: 820 (66.7%)

=== Outlier Detection Summary ===
Total data points: 43296
Originally valid: 33245 (76.8%)
Outliers detected: 1 (0.0%)
Final valid points: 33244 (76.8%)

=== Outlier Detection Summary ===
Total data points: 61500
Originally valid: 46493 (75.6%)
Outliers detected: 2 (0.0%)
Final valid points: 46491 (75.6%)

=== Outlier Detection Summary ===
Total data points: 77736
Originally valid: 57627 (74.1%)
Outliers detected: 2 (0.0%)
Final valid points: 57625 (74.1%)

=== Outlier Detection Summary ===
Total data points: 58056
Originally valid: 45813 (78.9%)
Outliers detected: 1 (0.0%)
Final valid points: 45812 (78.9%)

=== Outlier Detection Summary ===
Total data points: 63714
Originally valid: 49355 (77.5%)
Outliers detected: 1 (0.0%)
Final valid points: 49354 (77.5%)

=== Outlier Detection Summary ===
Total data points: 76260
Originally valid: 60128 (78.8%)
Outliers detected: 2 (0.0%)
Final valid points: 60126 (78.8%)

```

```
=== Outlier Detection Summary ===
Total data points: 79458
Originally valid: 60864 (76.6%)
Outliers detected: 1 (0.0%)
Final valid points: 60863 (76.6%)

=== Outlier Detection Summary ===
Total data points: 65928
Originally valid: 46764 (70.9%)
Outliers detected: 1 (0.0%)
Final valid points: 46763 (70.9%)

=== Outlier Detection Summary ===
Total data points: 88314
Originally valid: 67234 (76.1%)
Outliers detected: 4 (0.0%)
Final valid points: 67230 (76.1%)

=== Outlier Detection Summary ===
Total data points: 40590
Originally valid: 25497 (62.8%)
Outliers detected: 3 (0.0%)
Final valid points: 25494 (62.8%)

=== Outlier Detection Summary ===
Total data points: 77982
Originally valid: 58834 (75.4%)
Outliers detected: 2 (0.0%)
Final valid points: 58832 (75.4%)

=== Outlier Detection Summary ===
Total data points: 61254
Originally valid: 43606 (71.2%)
Outliers detected: 8 (0.0%)
Final valid points: 43598 (71.2%)

=== Outlier Detection Summary ===
Total data points: 21648
Originally valid: 13702 (63.3%)
Outliers detected: 1 (0.0%)
Final valid points: 13701 (63.3%)

=== Outlier Detection Summary ===
Total data points: 5610
Originally valid: 4730 (84.3%)
Outliers detected: 1 (0.0%)
Final valid points: 4729 (84.3%)

=== Outlier Detection Summary ===
Total data points: 15810
Originally valid: 13522 (85.5%)
Outliers detected: 1 (0.0%)
Final valid points: 13521 (85.5%)

=== Outlier Detection Summary ===
Total data points: 19992
Originally valid: 17578 (87.9%)
Outliers detected: 5 (0.0%)
Final valid points: 17573 (87.9%)

=== Outlier Detection Summary ===
Total data points: 22440
Originally valid: 18289 (81.5%)
Outliers detected: 1 (0.0%)
Final valid points: 18288 (81.5%)

=== Outlier Detection Summary ===
Total data points: 22593
Originally valid: 19268 (85.3%)
Outliers detected: 3 (0.0%)
Final valid points: 19265 (85.3%)

=== Outlier Detection Summary ===
Total data points: 15147
Originally valid: 12600 (83.2%)
Outliers detected: 9 (0.1%)
Final valid points: 12591 (83.1%)

=== Outlier Detection Summary ===
Total data points: 22695
Originally valid: 20636 (90.9%)
Outliers detected: 7 (0.0%)
Final valid points: 20629 (90.9%)

=== Outlier Detection Summary ===
Total data points: 16014
Originally valid: 13717 (85.7%)
Outliers detected: 1 (0.0%)
Final valid points: 13716 (85.7%)

=== Outlier Detection Summary ===
Total data points: 22134
Originally valid: 19396 (87.6%)
Outliers detected: 1 (0.0%)
Final valid points: 19395 (87.6%)

=== Outlier Detection Summary ===
Total data points: 16881
Originally valid: 14103 (83.5%)
Outliers detected: 3 (0.0%)
Final valid points: 14100 (83.5%)

=== Outlier Detection Summary ===
Total data points: 20910
Originally valid: 19603 (93.7%)
Outliers detected: 5 (0.0%)
Final valid points: 19598 (93.7%)

=== Outlier Detection Summary ===
Total data points: 18054
Originally valid: 16804 (93.1%)
Outliers detected: 1 (0.0%)
Final valid points: 16803 (93.1%)

=== Outlier Detection Summary ===
Total data points: 19278
Originally valid: 16888 (87.6%)
```

```
Outliers detected: 3 (0.0%)
Final valid points: 16885 (87.6%)

=== Outlier Detection Summary ===
Total data points: 18717
Originally valid: 17276 (92.3%)
Outliers detected: 5 (0.0%)
Final valid points: 17271 (92.3%)

=== Outlier Detection Summary ===
Total data points: 22542
Originally valid: 20196 (89.6%)
Outliers detected: 3 (0.0%)
Final valid points: 20193 (89.6%)

=== Outlier Detection Summary ===
Total data points: 16677
Originally valid: 13668 (82.0%)
Outliers detected: 3 (0.0%)
Final valid points: 13665 (81.9%)

=== Outlier Detection Summary ===
Total data points: 22593
Originally valid: 20612 (91.2%)
Outliers detected: 1 (0.0%)
Final valid points: 20611 (91.2%)

=== Outlier Detection Summary ===
Total data points: 22644
Originally valid: 19705 (87.0%)
Outliers detected: 3 (0.0%)
Final valid points: 19702 (87.0%)

=== Outlier Detection Summary ===
Total data points: 10914
Originally valid: 8426 (77.2%)
Outliers detected: 2 (0.0%)
Final valid points: 8424 (77.2%)

=== Outlier Detection Summary ===
Total data points: 22287
Originally valid: 19197 (86.1%)
Outliers detected: 5 (0.0%)
Final valid points: 19192 (86.1%)

=== Outlier Detection Summary ===
Total data points: 8823
Originally valid: 7638 (86.6%)
Outliers detected: 1 (0.0%)
Final valid points: 7637 (86.6%)

=== Outlier Detection Summary ===
Total data points: 19635
Originally valid: 16975 (86.5%)
Outliers detected: 1 (0.0%)
Final valid points: 16974 (86.4%)

=== Outlier Detection Summary ===
Total data points: 14229
Originally valid: 11947 (84.0%)
Outliers detected: 1 (0.0%)
Final valid points: 11946 (84.0%)

=== Outlier Detection Summary ===
Total data points: 14280
Originally valid: 12796 (89.6%)
Outliers detected: 2 (0.0%)
Final valid points: 12794 (89.6%)

=== Outlier Detection Summary ===
Total data points: 17799
Originally valid: 14606 (82.1%)
Outliers detected: 3 (0.0%)
Final valid points: 14603 (82.0%)

=== Outlier Detection Summary ===
Total data points: 5916
Originally valid: 5025 (84.9%)
Outliers detected: 1 (0.0%)
Final valid points: 5024 (84.9%)

=== Outlier Detection Summary ===
Total data points: 19278
Originally valid: 15869 (82.3%)
Outliers detected: 1 (0.0%)
Final valid points: 15868 (82.3%)

=== Outlier Detection Summary ===
Total data points: 14076
Originally valid: 11467 (81.5%)
Outliers detected: 13 (0.1%)
Final valid points: 11454 (81.4%)

=== Outlier Detection Summary ===
Total data points: 19431
Originally valid: 17656 (90.9%)
Outliers detected: 2 (0.0%)
Final valid points: 17654 (90.9%)

=== Outlier Detection Summary ===
Total data points: 18870
Originally valid: 15858 (84.0%)
Outliers detected: 1 (0.0%)
Final valid points: 15857 (84.0%)

=== Outlier Detection Summary ===
Total data points: 16065
Originally valid: 13175 (82.0%)
Outliers detected: 2 (0.0%)
Final valid points: 13173 (82.0%)

=== Outlier Detection Summary ===
Total data points: 17238
Originally valid: 15879 (92.1%)
Outliers detected: 1 (0.0%)
Final valid points: 15878 (92.1%)

```

```
=== Outlier Detection Summary ===
Total data points: 17289
Originally valid: 16142 (93.4%)
Outliers detected: 4 (0.0%)
Final valid points: 16138 (93.3%)

=== Outlier Detection Summary ===
Total data points: 14229
Originally valid: 11577 (81.4%)
Outliers detected: 1 (0.0%)
Final valid points: 11576 (81.4%)

=== Outlier Detection Summary ===
Total data points: 14688
Originally valid: 12696 (86.4%)
Outliers detected: 3 (0.0%)
Final valid points: 12693 (86.4%)

=== Outlier Detection Summary ===
Total data points: 19278
Originally valid: 16322 (84.7%)
Outliers detected: 2 (0.0%)
Final valid points: 16320 (84.7%)

=== Outlier Detection Summary ===
Total data points: 12036
Originally valid: 9282 (77.1%)
Outliers detected: 1 (0.0%)
Final valid points: 9281 (77.1%)

=== Outlier Detection Summary ===
Total data points: 19125
Originally valid: 17305 (90.5%)
Outliers detected: 2 (0.0%)
Final valid points: 17303 (90.5%)

=== Outlier Detection Summary ===
Total data points: 11214
Originally valid: 7782 (69.4%)
Outliers detected: 1 (0.0%)
Final valid points: 7781 (69.4%)

=== Outlier Detection Summary ===
Total data points: 18585
Originally valid: 13773 (74.1%)
Outliers detected: 1 (0.0%)
Final valid points: 13772 (74.1%)

=== Outlier Detection Summary ===
Total data points: 19719
Originally valid: 13126 (66.6%)
Outliers detected: 4 (0.0%)
Final valid points: 13122 (66.5%)

=== Outlier Detection Summary ===
Total data points: 12348
Originally valid: 9400 (76.1%)
Outliers detected: 1 (0.0%)
Final valid points: 9399 (76.1%)

=== Outlier Detection Summary ===
Total data points: 24633
Originally valid: 17046 (69.2%)
Outliers detected: 10 (0.0%)
Final valid points: 17036 (69.2%)

=== Outlier Detection Summary ===
Total data points: 16632
Originally valid: 12064 (72.5%)
Outliers detected: 2 (0.0%)
Final valid points: 12062 (72.5%)

=== Outlier Detection Summary ===
Total data points: 27909
Originally valid: 18540 (66.4%)
Outliers detected: 1 (0.0%)
Final valid points: 18539 (66.4%)

=== Outlier Detection Summary ===
Total data points: 18711
Originally valid: 12510 (66.9%)
Outliers detected: 2 (0.0%)
Final valid points: 12508 (66.8%)

=== Outlier Detection Summary ===
Total data points: 28098
Originally valid: 17743 (63.1%)
Outliers detected: 4 (0.0%)
Final valid points: 17739 (63.1%)

=== Outlier Detection Summary ===
Total data points: 19719
Originally valid: 14573 (73.9%)
Outliers detected: 3 (0.0%)
Final valid points: 14570 (73.9%)

=== Outlier Detection Summary ===
Total data points: 27405
Originally valid: 19038 (69.5%)
Outliers detected: 2 (0.0%)
Final valid points: 19036 (69.5%)

=== Outlier Detection Summary ===
Total data points: 22302
Originally valid: 16217 (72.7%)
Outliers detected: 1 (0.0%)
Final valid points: 16216 (72.7%)

=== Outlier Detection Summary ===
Total data points: 24003
Originally valid: 16970 (70.7%)
Outliers detected: 1 (0.0%)
Final valid points: 16969 (70.7%)

=== Outlier Detection Summary ===
Total data points: 27783
Originally valid: 18257 (65.7%)
```

```
Outliers detected: 2 (0.0%)
Final valid points: 18255 (65.7%)

=== Outlier Detection Summary ===
Total data points: 16947
Originally valid: 11875 (70.1%)
Outliers detected: 2 (0.0%)
Final valid points: 11873 (70.1%)

=== Outlier Detection Summary ===
Total data points: 27972
Originally valid: 20462 (73.2%)
Outliers detected: 1 (0.0%)
Final valid points: 20461 (73.1%)

=== Outlier Detection Summary ===
Total data points: 13545
Originally valid: 9023 (66.6%)
Outliers detected: 2 (0.0%)
Final valid points: 9021 (66.6%)

=== Outlier Detection Summary ===
Total data points: 24066
Originally valid: 16775 (69.7%)
Outliers detected: 2 (0.0%)
Final valid points: 16773 (69.7%)

=== Outlier Detection Summary ===
Total data points: 9009
Originally valid: 6027 (66.9%)
Outliers detected: 2 (0.0%)
Final valid points: 6025 (66.9%)

=== Outlier Detection Summary ===
Total data points: 23058
Originally valid: 15682 (68.0%)
Outliers detected: 2 (0.0%)
Final valid points: 15680 (68.0%)

=== Outlier Detection Summary ===
Total data points: 17703
Originally valid: 11731 (66.3%)
Outliers detected: 5 (0.0%)
Final valid points: 11726 (66.2%)

=== Outlier Detection Summary ===
Total data points: 9891
Originally valid: 7136 (72.1%)
Outliers detected: 1 (0.0%)
Final valid points: 7135 (72.1%)

=== Outlier Detection Summary ===
Total data points: 22050
Originally valid: 17073 (77.4%)
Outliers detected: 1 (0.0%)
Final valid points: 17072 (77.4%)

=== Outlier Detection Summary ===
Total data points: 23877
Originally valid: 17128 (71.7%)
Outliers detected: 1 (0.0%)
Final valid points: 17127 (71.7%)

=== Outlier Detection Summary ===
Total data points: 24129
Originally valid: 16462 (68.2%)
Outliers detected: 1 (0.0%)
Final valid points: 16461 (68.2%)

=== Outlier Detection Summary ===
Total data points: 23310
Originally valid: 15826 (67.9%)
Outliers detected: 1 (0.0%)
Final valid points: 15825 (67.9%)

=== Outlier Detection Summary ===
Total data points: 19782
Originally valid: 12691 (64.2%)
Outliers detected: 2 (0.0%)
Final valid points: 12689 (64.1%)

=== Outlier Detection Summary ===
Total data points: 21420
Originally valid: 14393 (67.2%)
Outliers detected: 1 (0.0%)
Final valid points: 14392 (67.2%)

=== Outlier Detection Summary ===
Total data points: 21357
Originally valid: 14254 (66.7%)
Outliers detected: 3 (0.0%)
Final valid points: 14251 (66.7%)

=== Outlier Detection Summary ===
Total data points: 17514
Originally valid: 12021 (68.6%)
Outliers detected: 1 (0.0%)
Final valid points: 12020 (68.6%)

=== Outlier Detection Summary ===
Total data points: 18207
Originally valid: 12165 (66.8%)
Outliers detected: 2 (0.0%)
Final valid points: 12163 (66.8%)

=== Outlier Detection Summary ===
Total data points: 23877
Originally valid: 16142 (67.6%)
Outliers detected: 1 (0.0%)
Final valid points: 16141 (67.6%)

=== Outlier Detection Summary ===
Total data points: 14490
Originally valid: 9314 (64.3%)
Outliers detected: 1 (0.0%)
Final valid points: 9313 (64.3%)

```

```
=== Outlier Detection Summary ===
Total data points: 23625
Originally valid: 15121 (64.0%)
Outliers detected: 2 (0.0%)
Final valid points: 15119 (64.0%)

=== Outlier Detection Summary ===
Total data points: 6552
Originally valid: 4071 (62.1%)
Outliers detected: 1 (0.0%)
Final valid points: 4070 (62.1%)

=== Outlier Detection Summary ===
Total data points: 22302
Originally valid: 15229 (68.3%)
Outliers detected: 4 (0.0%)
Final valid points: 15225 (68.3%)

=== Outlier Detection Summary ===
Total data points: 63426
Originally valid: 49889 (78.7%)
Outliers detected: 1 (0.0%)
Final valid points: 49888 (78.7%)

=== Outlier Detection Summary ===
Total data points: 84816
Originally valid: 67282 (79.3%)
Outliers detected: 2 (0.0%)
Final valid points: 67280 (79.3%)

=== Outlier Detection Summary ===
Total data points: 33108
Originally valid: 24961 (75.4%)
Outliers detected: 2 (0.0%)
Final valid points: 24959 (75.4%)

=== Outlier Detection Summary ===
Total data points: 85002
Originally valid: 68996 (81.2%)
Outliers detected: 1 (0.0%)
Final valid points: 68995 (81.2%)

=== Outlier Detection Summary ===
Total data points: 51150
Originally valid: 40424 (79.0%)
Outliers detected: 3 (0.0%)
Final valid points: 40421 (79.0%)

=== Outlier Detection Summary ===
Total data points: 56358
Originally valid: 44317 (78.6%)
Outliers detected: 1 (0.0%)
Final valid points: 44316 (78.6%)

=== Outlier Detection Summary ===
Total data points: 57474
Originally valid: 45907 (79.9%)
Outliers detected: 2 (0.0%)
Final valid points: 45905 (79.9%)

=== Outlier Detection Summary ===
Total data points: 69378
Originally valid: 56003 (80.7%)
Outliers detected: 2 (0.0%)
Final valid points: 56001 (80.7%)

=== Outlier Detection Summary ===
Total data points: 60078
Originally valid: 48519 (80.8%)
Outliers detected: 1 (0.0%)
Final valid points: 48518 (80.8%)

=== Outlier Detection Summary ===
Total data points: 63054
Originally valid: 49891 (79.1%)
Outliers detected: 2 (0.0%)
Final valid points: 49889 (79.1%)

=== Outlier Detection Summary ===
Total data points: 66960
Originally valid: 52461 (78.3%)
Outliers detected: 2 (0.0%)
Final valid points: 52459 (78.3%)

=== Outlier Detection Summary ===
Total data points: 37944
Originally valid: 28966 (76.3%)
Outliers detected: 1 (0.0%)
Final valid points: 28965 (76.3%)

=== Outlier Detection Summary ===
Total data points: 84816
Originally valid: 66621 (78.5%)
Outliers detected: 1 (0.0%)
Final valid points: 66620 (78.5%)

=== Outlier Detection Summary ===
Total data points: 56544
Originally valid: 44225 (78.2%)
Outliers detected: 2 (0.0%)
Final valid points: 44223 (78.2%)

=== Outlier Detection Summary ===
Total data points: 23808
Originally valid: 17534 (73.6%)
Outliers detected: 1 (0.0%)
Final valid points: 17533 (73.6%)

=== Outlier Detection Summary ===
Total data points: 58032
Originally valid: 46092 (79.4%)
Outliers detected: 1 (0.0%)
Final valid points: 46091 (79.4%)

=== Outlier Detection Summary ===
Total data points: 63984
Originally valid: 50952 (79.6%)
```

```
Outliers detected: 1 (0.0%)
Final valid points: 50951 (79.6%)

=== Outlier Detection Summary ===
Total data points: 32890
Originally valid: 31285 (95.1%)
Outliers detected: 1 (0.0%)
Final valid points: 31284 (95.1%)

=== Outlier Detection Summary ===
Total data points: 49105
Originally valid: 47215 (96.2%)
Outliers detected: 1 (0.0%)
Final valid points: 47214 (96.1%)

=== Outlier Detection Summary ===
Total data points: 47955
Originally valid: 46124 (96.2%)
Outliers detected: 1 (0.0%)
Final valid points: 46123 (96.2%)

=== Outlier Detection Summary ===
Total data points: 48875
Originally valid: 46830 (95.8%)
Outliers detected: 14 (0.0%)
Final valid points: 46816 (95.8%)

=== Outlier Detection Summary ===
Total data points: 49220
Originally valid: 47112 (95.7%)
Outliers detected: 1 (0.0%)
Final valid points: 47111 (95.7%)

=== Outlier Detection Summary ===
Total data points: 49335
Originally valid: 47836 (97.0%)
Outliers detected: 2 (0.0%)
Final valid points: 47834 (97.0%)

=== Outlier Detection Summary ===
Total data points: 17710
Originally valid: 17094 (96.5%)
Outliers detected: 1 (0.0%)
Final valid points: 17093 (96.5%)

=== Outlier Detection Summary ===
Total data points: 41860
Originally valid: 39710 (94.9%)
Outliers detected: 1 (0.0%)
Final valid points: 39709 (94.9%)

=== Outlier Detection Summary ===
Total data points: 24035
Originally valid: 21764 (90.6%)
Outliers detected: 1 (0.0%)
Final valid points: 21763 (90.5%)

=== Outlier Detection Summary ===
Total data points: 3680
Originally valid: 3414 (92.8%)
Outliers detected: 1 (0.0%)
Final valid points: 3413 (92.7%)

=== Outlier Detection Summary ===
Total data points: 7600
Originally valid: 7352 (96.7%)
Outliers detected: 1 (0.0%)
Final valid points: 7351 (96.7%)

=== Outlier Detection Summary ===
Total data points: 21920
Originally valid: 21191 (96.7%)
Outliers detected: 1 (0.0%)
Final valid points: 21190 (96.7%)

=== Outlier Detection Summary ===
Total data points: 25520
Originally valid: 24594 (96.4%)
Outliers detected: 1 (0.0%)
Final valid points: 24593 (96.4%)

=== Outlier Detection Summary ===
Total data points: 20480
Originally valid: 19738 (96.4%)
Outliers detected: 2 (0.0%)
Final valid points: 19736 (96.4%)

=== Outlier Detection Summary ===
Total data points: 20880
Originally valid: 20265 (97.1%)
Outliers detected: 1 (0.0%)
Final valid points: 20264 (97.0%)

=== Outlier Detection Summary ===
Total data points: 34000
Originally valid: 32995 (97.0%)
Outliers detected: 1 (0.0%)
Final valid points: 32994 (97.0%)

=== Outlier Detection Summary ===
Total data points: 22480
Originally valid: 21613 (96.1%)
Outliers detected: 1 (0.0%)
Final valid points: 21612 (96.1%)

=== Outlier Detection Summary ===
Total data points: 34160
Originally valid: 32935 (96.4%)
Outliers detected: 2 (0.0%)
Final valid points: 32933 (96.4%)

=== Outlier Detection Summary ===
Total data points: 34400
Originally valid: 33244 (96.6%)
Outliers detected: 2 (0.0%)
Final valid points: 33242 (96.6%)

```

```
=== Outlier Detection Summary ===
Total data points: 25600
Originally valid: 24788 (96.8%)
Outliers detected: 1 (0.0%)
Final valid points: 24787 (96.8%)

=== Outlier Detection Summary ===
Total data points: 32320
Originally valid: 30582 (94.6%)
Outliers detected: 1 (0.0%)
Final valid points: 30581 (94.6%)

=== Outlier Detection Summary ===
Total data points: 34160
Originally valid: 33100 (96.9%)
Outliers detected: 1 (0.0%)
Final valid points: 33099 (96.9%)

=== Outlier Detection Summary ===
Total data points: 34080
Originally valid: 32991 (96.8%)
Outliers detected: 1 (0.0%)
Final valid points: 32990 (96.8%)

=== Outlier Detection Summary ===
Total data points: 34160
Originally valid: 33101 (96.9%)
Outliers detected: 1 (0.0%)
Final valid points: 33100 (96.9%)

=== Outlier Detection Summary ===
Total data points: 34080
Originally valid: 32544 (95.5%)
Outliers detected: 2 (0.0%)
Final valid points: 32542 (95.5%)

=== Outlier Detection Summary ===
Total data points: 34160
Originally valid: 33097 (96.9%)
Outliers detected: 2 (0.0%)
Final valid points: 33095 (96.9%)

=== Outlier Detection Summary ===
Total data points: 31120
Originally valid: 29616 (95.2%)
Outliers detected: 1 (0.0%)
Final valid points: 29615 (95.2%)

=== Outlier Detection Summary ===
Total data points: 14240
Originally valid: 13601 (95.5%)
Outliers detected: 1 (0.0%)
Final valid points: 13600 (95.5%)

=== Outlier Detection Summary ===
Total data points: 29040
Originally valid: 27955 (96.3%)
Outliers detected: 1 (0.0%)
Final valid points: 27954 (96.3%)

=== Outlier Detection Summary ===
Total data points: 17680
Originally valid: 16859 (95.4%)
Outliers detected: 1 (0.0%)
Final valid points: 16858 (95.4%)

=== Outlier Detection Summary ===
Total data points: 29440
Originally valid: 28092 (95.4%)
Outliers detected: 3 (0.0%)
Final valid points: 28089 (95.4%)

=== Outlier Detection Summary ===
Total data points: 3920
Originally valid: 2491 (63.5%)
Outliers detected: 1 (0.0%)
Final valid points: 2490 (63.5%)

=== Outlier Detection Summary ===
Total data points: 492
Originally valid: 151 (30.7%)
Outliers detected: 1 (0.2%)
Final valid points: 150 (30.5%)

=== Outlier Detection Summary ===
Total data points: 1230
Originally valid: 822 (66.8%)
Outliers detected: 2 (0.2%)
Final valid points: 820 (66.7%)

=== Outlier Detection Summary ===
Total data points: 43296
Originally valid: 33245 (76.8%)
Outliers detected: 1 (0.0%)
Final valid points: 33244 (76.8%)

=== Outlier Detection Summary ===
Total data points: 61500
Originally valid: 46493 (75.6%)
Outliers detected: 2 (0.0%)
Final valid points: 46491 (75.6%)

=== Outlier Detection Summary ===
Total data points: 77736
Originally valid: 57627 (74.1%)
Outliers detected: 2 (0.0%)
Final valid points: 57625 (74.1%)

=== Outlier Detection Summary ===
Total data points: 58056
Originally valid: 45813 (78.9%)
Outliers detected: 1 (0.0%)
Final valid points: 45812 (78.9%)

=== Outlier Detection Summary ===
Total data points: 63714
Originally valid: 49355 (77.5%)
```

```
Outliers detected: 1 (0.0%)
Final valid points: 49354 (77.5%)

=== Outlier Detection Summary ===
Total data points: 76260
Originally valid: 60128 (78.8%)
Outliers detected: 2 (0.0%)
Final valid points: 60126 (78.8%)

=== Outlier Detection Summary ===
Total data points: 79458
Originally valid: 60864 (76.6%)
Outliers detected: 1 (0.0%)
Final valid points: 60863 (76.6%)

=== Outlier Detection Summary ===
Total data points: 65928
Originally valid: 46764 (70.9%)
Outliers detected: 1 (0.0%)
Final valid points: 46763 (70.9%)

=== Outlier Detection Summary ===
Total data points: 88314
Originally valid: 67234 (76.1%)
Outliers detected: 4 (0.0%)
Final valid points: 67230 (76.1%)

=== Outlier Detection Summary ===
Total data points: 40590
Originally valid: 25497 (62.8%)
Outliers detected: 3 (0.0%)
Final valid points: 25494 (62.8%)

=== Outlier Detection Summary ===
Total data points: 77982
Originally valid: 58834 (75.4%)
Outliers detected: 2 (0.0%)
Final valid points: 58832 (75.4%)

=== Outlier Detection Summary ===
Total data points: 61254
Originally valid: 43606 (71.2%)
Outliers detected: 8 (0.0%)
Final valid points: 43598 (71.2%)

=== Outlier Detection Summary ===
Total data points: 21648
Originally valid: 13702 (63.3%)
Outliers detected: 1 (0.0%)
Final valid points: 13701 (63.3%)

=== Outlier Detection Summary ===
Total data points: 5610
Originally valid: 4730 (84.3%)
Outliers detected: 1 (0.0%)
Final valid points: 4729 (84.3%)

=== Outlier Detection Summary ===
Total data points: 15810
Originally valid: 13522 (85.5%)
Outliers detected: 1 (0.0%)
Final valid points: 13521 (85.5%)

=== Outlier Detection Summary ===
Total data points: 19992
Originally valid: 17578 (87.9%)
Outliers detected: 5 (0.0%)
Final valid points: 17573 (87.9%)

=== Outlier Detection Summary ===
Total data points: 22440
Originally valid: 18289 (81.5%)
Outliers detected: 1 (0.0%)
Final valid points: 18288 (81.5%)

=== Outlier Detection Summary ===
Total data points: 22593
Originally valid: 19268 (85.3%)
Outliers detected: 3 (0.0%)
Final valid points: 19265 (85.3%)

=== Outlier Detection Summary ===
Total data points: 15147
Originally valid: 12600 (83.2%)
Outliers detected: 9 (0.1%)
Final valid points: 12591 (83.1%)

=== Outlier Detection Summary ===
Total data points: 22695
Originally valid: 20636 (90.9%)
Outliers detected: 7 (0.0%)
Final valid points: 20629 (90.9%)

=== Outlier Detection Summary ===
Total data points: 16014
Originally valid: 13717 (85.7%)
Outliers detected: 1 (0.0%)
Final valid points: 13716 (85.7%)

=== Outlier Detection Summary ===
Total data points: 22134
Originally valid: 19396 (87.6%)
Outliers detected: 1 (0.0%)
Final valid points: 19395 (87.6%)

=== Outlier Detection Summary ===
Total data points: 16881
Originally valid: 14103 (83.5%)
Outliers detected: 3 (0.0%)
Final valid points: 14100 (83.5%)

=== Outlier Detection Summary ===
Total data points: 20910
Originally valid: 19603 (93.7%)
Outliers detected: 5 (0.0%)
Final valid points: 19598 (93.7%)

```

```
=== Outlier Detection Summary ===
Total data points: 18054
Originally valid: 16804 (93.1%)
Outliers detected: 1 (0.0%)
Final valid points: 16803 (93.1%)

=== Outlier Detection Summary ===
Total data points: 19278
Originally valid: 16888 (87.6%)
Outliers detected: 3 (0.0%)
Final valid points: 16885 (87.6%)

=== Outlier Detection Summary ===
Total data points: 18717
Originally valid: 17276 (92.3%)
Outliers detected: 5 (0.0%)
Final valid points: 17271 (92.3%)

=== Outlier Detection Summary ===
Total data points: 22542
Originally valid: 20196 (89.6%)
Outliers detected: 3 (0.0%)
Final valid points: 20193 (89.6%)

=== Outlier Detection Summary ===
Total data points: 16677
Originally valid: 13668 (82.0%)
Outliers detected: 3 (0.0%)
Final valid points: 13665 (81.9%)

=== Outlier Detection Summary ===
Total data points: 22593
Originally valid: 20612 (91.2%)
Outliers detected: 1 (0.0%)
Final valid points: 20611 (91.2%)

=== Outlier Detection Summary ===
Total data points: 22644
Originally valid: 19705 (87.0%)
Outliers detected: 3 (0.0%)
Final valid points: 19702 (87.0%)

=== Outlier Detection Summary ===
Total data points: 10914
Originally valid: 8426 (77.2%)
Outliers detected: 2 (0.0%)
Final valid points: 8424 (77.2%)

=== Outlier Detection Summary ===
Total data points: 22287
Originally valid: 19197 (86.1%)
Outliers detected: 5 (0.0%)
Final valid points: 19192 (86.1%)

=== Outlier Detection Summary ===
Total data points: 8823
Originally valid: 7638 (86.6%)
Outliers detected: 1 (0.0%)
Final valid points: 7637 (86.6%)

=== Outlier Detection Summary ===
Total data points: 19635
Originally valid: 16975 (86.5%)
Outliers detected: 1 (0.0%)
Final valid points: 16974 (86.4%)

=== Outlier Detection Summary ===
Total data points: 14229
Originally valid: 11947 (84.0%)
Outliers detected: 1 (0.0%)
Final valid points: 11946 (84.0%)

=== Outlier Detection Summary ===
Total data points: 14280
Originally valid: 12796 (89.6%)
Outliers detected: 2 (0.0%)
Final valid points: 12794 (89.6%)

=== Outlier Detection Summary ===
Total data points: 17799
Originally valid: 14606 (82.1%)
Outliers detected: 3 (0.0%)
Final valid points: 14603 (82.0%)

=== Outlier Detection Summary ===
Total data points: 5916
Originally valid: 5025 (84.9%)
Outliers detected: 1 (0.0%)
Final valid points: 5024 (84.9%)

=== Outlier Detection Summary ===
Total data points: 19278
Originally valid: 15869 (82.3%)
Outliers detected: 1 (0.0%)
Final valid points: 15868 (82.3%)

=== Outlier Detection Summary ===
Total data points: 14076
Originally valid: 11467 (81.5%)
Outliers detected: 13 (0.1%)
Final valid points: 11454 (81.4%)

=== Outlier Detection Summary ===
Total data points: 19431
Originally valid: 17656 (90.9%)
Outliers detected: 2 (0.0%)
Final valid points: 17654 (90.9%)

=== Outlier Detection Summary ===
Total data points: 18870
Originally valid: 15858 (84.0%)
Outliers detected: 1 (0.0%)
Final valid points: 15857 (84.0%)

=== Outlier Detection Summary ===
Total data points: 16065
Originally valid: 13175 (82.0%)
```

```
Outliers detected: 2 (0.0%)
Final valid points: 13173 (82.0%)

=== Outlier Detection Summary ===
Total data points: 17238
Originally valid: 15879 (92.1%)
Outliers detected: 1 (0.0%)
Final valid points: 15878 (92.1%)

=== Outlier Detection Summary ===
Total data points: 17289
Originally valid: 16142 (93.4%)
Outliers detected: 4 (0.0%)
Final valid points: 16138 (93.3%)

=== Outlier Detection Summary ===
Total data points: 14229
Originally valid: 11577 (81.4%)
Outliers detected: 1 (0.0%)
Final valid points: 11576 (81.4%)

=== Outlier Detection Summary ===
Total data points: 14688
Originally valid: 12696 (86.4%)
Outliers detected: 3 (0.0%)
Final valid points: 12693 (86.4%)

=== Outlier Detection Summary ===
Total data points: 19278
Originally valid: 16322 (84.7%)
Outliers detected: 2 (0.0%)
Final valid points: 16320 (84.7%)

=== Outlier Detection Summary ===
Total data points: 12036
Originally valid: 9282 (77.1%)
Outliers detected: 1 (0.0%)
Final valid points: 9281 (77.1%)

=== Outlier Detection Summary ===
Total data points: 19125
Originally valid: 17305 (90.5%)
Outliers detected: 2 (0.0%)
Final valid points: 17303 (90.5%)

=== Outlier Detection Summary ===
Total data points: 11214
Originally valid: 7782 (69.4%)
Outliers detected: 1 (0.0%)
Final valid points: 7781 (69.4%)

=== Outlier Detection Summary ===
Total data points: 18585
Originally valid: 13773 (74.1%)
Outliers detected: 1 (0.0%)
Final valid points: 13772 (74.1%)

=== Outlier Detection Summary ===
Total data points: 19719
Originally valid: 13126 (66.6%)
Outliers detected: 4 (0.0%)
Final valid points: 13122 (66.5%)

=== Outlier Detection Summary ===
Total data points: 12348
Originally valid: 9400 (76.1%)
Outliers detected: 1 (0.0%)
Final valid points: 9399 (76.1%)

=== Outlier Detection Summary ===
Total data points: 24633
Originally valid: 17046 (69.2%)
Outliers detected: 10 (0.0%)
Final valid points: 17036 (69.2%)

=== Outlier Detection Summary ===
Total data points: 16632
Originally valid: 12064 (72.5%)
Outliers detected: 2 (0.0%)
Final valid points: 12062 (72.5%)

=== Outlier Detection Summary ===
Total data points: 27909
Originally valid: 18540 (66.4%)
Outliers detected: 1 (0.0%)
Final valid points: 18539 (66.4%)

=== Outlier Detection Summary ===
Total data points: 18711
Originally valid: 12510 (66.9%)
Outliers detected: 2 (0.0%)
Final valid points: 12508 (66.8%)

=== Outlier Detection Summary ===
Total data points: 28098
Originally valid: 17743 (63.1%)
Outliers detected: 4 (0.0%)
Final valid points: 17739 (63.1%)

=== Outlier Detection Summary ===
Total data points: 19719
Originally valid: 14573 (73.9%)
Outliers detected: 3 (0.0%)
Final valid points: 14570 (73.9%)

=== Outlier Detection Summary ===
Total data points: 27405
Originally valid: 19038 (69.5%)
Outliers detected: 2 (0.0%)
Final valid points: 19036 (69.5%)

=== Outlier Detection Summary ===
Total data points: 22302
Originally valid: 16217 (72.7%)
Outliers detected: 1 (0.0%)
Final valid points: 16216 (72.7%)

```

```
=== Outlier Detection Summary ===
Total data points: 24003
Originally valid: 16970 (70.7%)
Outliers detected: 1 (0.0%)
Final valid points: 16969 (70.7%)

=== Outlier Detection Summary ===
Total data points: 27783
Originally valid: 18257 (65.7%)
Outliers detected: 2 (0.0%)
Final valid points: 18255 (65.7%)

=== Outlier Detection Summary ===
Total data points: 16947
Originally valid: 11875 (70.1%)
Outliers detected: 2 (0.0%)
Final valid points: 11873 (70.1%)

=== Outlier Detection Summary ===
Total data points: 27972
Originally valid: 20462 (73.2%)
Outliers detected: 1 (0.0%)
Final valid points: 20461 (73.1%)

=== Outlier Detection Summary ===
Total data points: 13545
Originally valid: 9023 (66.6%)
Outliers detected: 2 (0.0%)
Final valid points: 9021 (66.6%)

=== Outlier Detection Summary ===
Total data points: 24066
Originally valid: 16775 (69.7%)
Outliers detected: 2 (0.0%)
Final valid points: 16773 (69.7%)

=== Outlier Detection Summary ===
Total data points: 9009
Originally valid: 6027 (66.9%)
Outliers detected: 2 (0.0%)
Final valid points: 6025 (66.9%)

=== Outlier Detection Summary ===
Total data points: 23058
Originally valid: 15682 (68.0%)
Outliers detected: 2 (0.0%)
Final valid points: 15680 (68.0%)

=== Outlier Detection Summary ===
Total data points: 17703
Originally valid: 11731 (66.3%)
Outliers detected: 5 (0.0%)
Final valid points: 11726 (66.2%)

=== Outlier Detection Summary ===
Total data points: 9891
Originally valid: 7136 (72.1%)
Outliers detected: 1 (0.0%)
Final valid points: 7135 (72.1%)

=== Outlier Detection Summary ===
Total data points: 22050
Originally valid: 17073 (77.4%)
Outliers detected: 1 (0.0%)
Final valid points: 17072 (77.4%)

=== Outlier Detection Summary ===
Total data points: 23877
Originally valid: 17128 (71.7%)
Outliers detected: 1 (0.0%)
Final valid points: 17127 (71.7%)

=== Outlier Detection Summary ===
Total data points: 24129
Originally valid: 16462 (68.2%)
Outliers detected: 1 (0.0%)
Final valid points: 16461 (68.2%)

=== Outlier Detection Summary ===
Total data points: 23310
Originally valid: 15826 (67.9%)
Outliers detected: 1 (0.0%)
Final valid points: 15825 (67.9%)

=== Outlier Detection Summary ===
Total data points: 19782
Originally valid: 12691 (64.2%)
Outliers detected: 2 (0.0%)
Final valid points: 12689 (64.1%)

=== Outlier Detection Summary ===
Total data points: 21420
Originally valid: 14393 (67.2%)
Outliers detected: 1 (0.0%)
Final valid points: 14392 (67.2%)

=== Outlier Detection Summary ===
Total data points: 21357
Originally valid: 14254 (66.7%)
Outliers detected: 3 (0.0%)
Final valid points: 14251 (66.7%)

=== Outlier Detection Summary ===
Total data points: 17514
Originally valid: 12021 (68.6%)
Outliers detected: 1 (0.0%)
Final valid points: 12020 (68.6%)

=== Outlier Detection Summary ===
Total data points: 18207
Originally valid: 12165 (66.8%)
Outliers detected: 2 (0.0%)
Final valid points: 12163 (66.8%)

=== Outlier Detection Summary ===
Total data points: 23877
Originally valid: 16142 (67.6%)
```

```
Outliers detected: 1 (0.0%)
Final valid points: 16141 (67.6%)

=== Outlier Detection Summary ===
Total data points: 14490
Originally valid: 9314 (64.3%)
Outliers detected: 1 (0.0%)
Final valid points: 9313 (64.3%)

=== Outlier Detection Summary ===
Total data points: 23625
Originally valid: 15121 (64.0%)
Outliers detected: 2 (0.0%)
Final valid points: 15119 (64.0%)

=== Outlier Detection Summary ===
Total data points: 6552
Originally valid: 4071 (62.1%)
Outliers detected: 1 (0.0%)
Final valid points: 4070 (62.1%)

=== Outlier Detection Summary ===
Total data points: 22302
Originally valid: 15229 (68.3%)
Outliers detected: 4 (0.0%)
Final valid points: 15225 (68.3%)
[np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.00200449005772931 33), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0), np.float64(0.0
```

array(40421), array(63113), array(44316), array(57259), array(45905), array(56001), array(48518), array(54349), array(49889), array(49522), array(52459), array(432
83), array(54740), array(39198), array(62729), array(33627), array(70162), array(28965), array(65677), array(24411), array(66620), array(22882), array(65530), arra
y(40086), array(69600), array(44223), array(62907), array(51004), array(56212), array(58413), array(24438), array(56400), array(24612), array(55761), array(53829),
 array(17533), array(33403), array(49508), array(41561), array(46773), array(44504), array(40455), array(46091), array(41710), array(47055), array(38018), array(50
951), array(28695), array(51435), array(16103), array(55246), array(11998), array(52168), array(3650), array(42567), array(9165), array(47016), array(29743), array
(71115), array(46341), array(26275), array(165138), array(66475), array(384868), array(167724), array(554250), array(237336), array(555800), array(326082), array(5
45779), array(350466), array(505949), array(375307), array(455313), array(412796), array(408072), array(471405), array(318567), array(561326), array(240921), array
(553772), array(176114), array(558091), array(348200), array(455144), array(469330), array(196869), array(483477), array(220479), array(429373), array(353164), arr
ay(319250), array(387839), array(340108), array(424348), array(208106), array(462773), array(57385), array(403077), array(175262), array(2282), array(1189), array(
6767), array(4231), array(2513), array(8868), array(5355), array(3599), array(10103), array(8396), array(13606), array(10097), array(16836), array(11553), array(21
989), array(13796), array(31263), array(15579), array(31700), array(18578), array(31306), array(21014), array(33588), array(23965), array(36298), array(27974), arr
ay(42337), array(28358), array(47047), array(29477), array(47028), array(30365), array(47169), array(31284), array(47564), array(31560), array(47214), array(33125)
, array(47091), array(33482), array(47826), array(35126), array(46123), array(36769), array(44916), array(37483), array(43578), array(41264), array(41623), array(4
6395), array(39894), array(47207), array(38899), array(47041), array(39512), array(47014), array(39251), array(46816), array(33547), array(47111), array(30017), ar
ray(47706), array(27331), array(47834), array(24350), array(46458), array(21529), array(43690), array(19868), array(40030), array(16776), array(38771), array(15359
), array(39355), array(17093), array(30526), array(24974), array(20989), array(25624), array(15207), array(27659), array(16169), array(32392), array(17232), array(
36919), array(13065), array(37536), array(17693), array(40638), array(29627), array(40241), array(29725), array(40082), array(30817), array(40662), array(32512), a
rray(41396), array(33541), array(38621), array(34016), array(36309), array(36095), array(34877), array(36960), array(32660), array(37337), array(28700), array(4000
5), array(28966), array(39709), array(30024), array(39689), array(27099), array(39896), array(21763), array(39597), array(16405), array(35428), array(10213), array
(28737), array(5523), array(21686), array(1165), array(22593), array(9911), array(230), array(308), array(3965), array(2243), array(1305), array(5624), array(3328)
, array(2177), array(6354), array(3413), array(8221), array(6542), array(10351), array(7351), array(13445), array(9229), array(21190), array(10132), array(22006),
array(12135), array(21925), array(13822), array(23285), array(15804), array(24593), array(17135), array(28259), array(19736), array(31541), array(20264), array(329
94), array(21033), array(32762), array(21612), array(32933), array(22176), array(32712), array(22389), array(33046), array(23357), array(33242), array(24167), arra
y(32692), array(24787), array(31642), array(26737), array(30545), array(27969), array(30014), array(30581), array(28420), array(32631), array(27507), array(33099),
 array(27139), array(32990), array(28228), array(33100), array(24879), array(32542), array(22143), array(33095), array(19721), array(33131), array(18351), array(33
244), array(15955), array(30896), array(14369), array(29615), array(12815), array(27709), array(11089), array(27175), array(10743), array(27262), array(13600), arr
ay(16113), array(21048), array(11004), array(18452), array(10707), array(21032), array(12013), array(25284), array(10226), array(26012), array(10431), array(27954)
, array(16858), array(28041), array(20739), array(28197), array(21369), array(28407), array(21954), array(28890), array(23043), array(27724), array(23826), array(2
5963), array(24631), array(25572), array(24845), array(21383), array(25971), array(20374), array(27380), array(19970), array(28023), array(21158), array(27465), ar
ray(20883), array(28089), array(16680), array(26707), array(12623), array(26692), array(7311), array(23431), array(5910), array(18097), array(14608), array(1998),
array(2490), array(11927), array(3143), array(1267), array(30960), array(14424), array(10637), array(93523), array(33237), array(118248), array(50991), array(14553
8), array(71312), array(144323), array(86965), array(137411), array(95093), array(121537), array(103796), array(114259), array(114120), array(84683), array(141332)
, array(63959), array(138709), array(46937), array(133344), array(84129), array(124749), array(99227), array(51961), array(121614), array(42122), array(116186), ar
ray(86938), array(102214), array(94763), array(85904), array(104250), array(74371), array(110560), array(30792), array(106791), array(98350), array(4153), array(21
214), array(150), array(820), array(8645), array(3851), array(2576), array(10947), array(6253), array(4103), array(13543), array(7679), array(5566), array(14797),
array(9570), array(18671), array(14131), array(22390), array(15596), array(26218), array(17647), array(33244), array(20932), array(46491), array(22159), array(4871
4), array(25270), array(49375), array(29240), array(49004), array(32119), array(50597), array(34549), array(55459), array(38716), array(57625), array(44083), array
(64548), array(45516), array(73371), array(45812), array(72702), array(48299), array(74156), array(47195), array(73212), array(48834), array(73055), array(49354),
array(72567), array(50786), array(68242), array(52802), array(69138), array(51651), array(72470), array(55628), array(68207), array(55458), array(70972), array(601
26), array(68938), array(59631), array(66643), array(63955), array(60934), array(70413), array(61793), array(74564), array(60403), array(75355), array(59716), arra
y(74711), array(60040), array(71235), array(60863), array(71915), array(53672), array(74560), array(46763), array(71361), array(39813), array(73564), array(35555),
 array(73099), array(33504), array(74472), array(30064), array(72341), array(26985), array(67230), array(25494), array(64400), array(22166), array(59675), array(24
700), array(59945), array(23506), array(59563), array(26032), array(59216), array(32653), array(34952), array(46649), array(26340), array(38027), array(23281), arr
ay(41488), array(24652), array(47885), array(27096), array(55154), array(28439), array(56845), array(19051), array(58153), array(26032), array(61868), array(43166)
, array(60858), array(46976), array(62833), array(47675), array(60288), array(46572), array(62660), array(51307), array(59733), array(51600), array(62769), array(5
2940), array(57707), array(54125), array(56097), array(55457), array(53418), array(56449), array(46150), array(58251), array(43669), array(59759), array(43889), ar
ray(62740), array(43602), array(62196), array(46373), array(60197), array(44965), array(62448), array(39039), array(59549), array(34506), array(58832), array(24940
), array(59716), array(17124), array(53806), array(14110), array(43598), array(8383), array(33154), array(13701), array(34434), array(2630), array(32163), array(10
9), array(174), array(3398), array(1931), array(1220), array(4729), array(3878), array(8151), array(5302), array(11648), array(7053), array(13521), array(9051), ar
ray(17573), array(12491), array(18288), array(12609), array(19265), array(12591), array(20629), array(13716), array(19395), array(14100), array(19598), array(16803
), array(16885), array(18702), array(17271), array(20193), array(13665), array(20611), array(11983), array(19702), array(8424), array(19192), array(7637), array(16
974), array(5603), array(17280), array(11946), array(8343), array(12794), array(7177), array(14603), array(5024), array(15868), array(11454), array(17654), array(1
2885), array(15857), array(13173), array(15878), array(16138), array(11576), array(14765), array(12693), array(16320), array(9281), array(17303), array(4372), arra
y(15402), array(1014), array(3441), array(9418), array(125), array(150), array(3375), array(2114), array(1249), array(4602), array(3747), array(7781), array(4806),
 array(13772), array(6718), array(13122), array(9399), array(17036), array(12062), array(20424), array(12215), array(18539), array(12508), array(17739), array(1457
0), array(19036), array(15599), array(19154), array(16216), array(16969), array(20962), array(16008), array(18255), array(14982), array(18513), array(11873), array
(20461), array(9021), array(20413), array(7606), array(16773), array(6025), array(15680), array(11726), array(8213), array(11708), array(7135), array(17072), array
(5327), array(17127), array(12392), array(16461), array(12563), array(15825), array(12689), array(14392), array(14251), array(12020), array(17014), array(12163), a
rray(16141), array(9313), array(15119), array(4070), array(15225), array(1040), array(3335), array(8926)]

2486

```python
import cartopy.crs as ccrs
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *

cmap1 = "twilight_shifted"
cmap1 = ListedColormap(["white"])
TRACKS_REG = (76.0, 99, 0, 25.4)

# A map that is 100% white for every single value
d = 1.5
height, width = 9, 9
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

satellites = df_all["sat"].unique()
colors = plt.cm.get_cmap("tab10", len(satellites))
# Unique satellites for coloring

rose = dse.ROSE
fig2, ax2 = plt.subplots(
    1,
    1,
    subplot_kw={"projection": ccrs.PlateCarree()},
    figsize=(width, height),
    layout="constrained",
)
rose_cut = rose.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG)
ax2.set_extent([*TRACKS_REG], crs=ccrs.PlateCarree())
ax2.coastlines()
# list of 8 different of markers
sat_marker_dict = {
    "GFO": "*",
    "TP+J1+J2+J3+S6A": "o",
    "S3A": "^",
    "S3B": "^",
    "TPN+J1N+J2N+J3N": "o",
    "ERS1+ERS2+ENV+SRL": "x",
    "HY2A": "s",
    "HY2B": "s",
}

for i, sat in enumerate(satellites):
    sat_data = df_all[df_all["sat"] == sat]
    ax2.scatter(
        sat_data["lons_inter"],
        sat_data["lats_inter"],
        color=colors(i),
        label=f"Satellite {sat}",
        s=18,
        marker=sat_marker_dict[sat],
        transform=ccrs.PlateCarree(),
    )

# edgecolors='black',
ax2.legend(title="Satellite", loc="upper right")
plt.title("Intersection Points of Tracks in North Indian Ocean")
plt.show()
dse.close()

```

```python
import cartopy.crs as ccrs
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *

cmap1 = "twilight_shifted"
cmap1 = ListedColormap(["white"])

# A map that is 100% white for every single value
d = 1.5
height, width = 9, 9
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

satellites = df_all["sat"].unique()
colors = plt.cm.get_cmap("tab10", len(satellites))
# Unique satellites for coloring
ds_coast = xr.open_dataset(
    f"~/allData/topo/GMT_intermediate_coast_distance_01d_track_reg.nc"
)
COAST_DISTANCE_KM = 100


def is_close_to_coast(ds_coast, lon, lat, dist_km):
    dist1 = ds_coast.coast_distance.sel(lon=lon, lat=lat, method="nearest").item()
    return abs(dist1) <= dist_km

rose = dse.ROSE
fig2, ax2 = plt.subplots(
    1,
    1,
    subplot_kw={"projection": ccrs.PlateCarree()},
    figsize=(width, height),
    layout="constrained",
)
rose_cut = rose.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG, step=15)
ax2.set_extent([*TRACKS_REG], crs=ccrs.PlateCarree())
ax2.coastlines()
# list of 8 different of markers
sat_marker_dict = {
    "GFO": "*",
    "TP+J1+J2+J3+S6A": "o",
    "S3A": "^",
    "S3B": "^",
    "TPN+J1N+J2N+J3N": "o",
    "ERS1+ERS2+ENV+SRL": "x",
    "HY2A": "s",
    "HY2B": "s",
}

for i, sat in enumerate(satellites):
    sat_data = df_all[df_all["sat"] == sat]
    close_mask = sat_data.apply(
        lambda row: is_close_to_coast(
            ds_coast, row["lons_inter"], row["lats_inter"], COAST_DISTANCE_KM
        ),
        axis=1,
    )
    sat_data_close = sat_data[close_mask]
    ax2.scatter(
        sat_data_close["lons_inter"],
        sat_data_close["lats_inter"],
        color=colors(i),
        label=f"Satellite {sat}",
        s=18,
        marker=sat_marker_dict[sat],
        transform=ccrs.PlateCarree(),
    )

# edgecolors='black',
ax2.legend(title="Satellite", loc="upper right")
plt.title("Intersection Points of Tracks in North Indian Ocean")
plt.show()
dse.close()
ds_coast.close()

```

```python
import cartopy.crs as ccrs
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *

cmap1 = "twilight_shifted"
cmap1 = ListedColormap(["white"])

# A map that is 100% white for every single value
d = 1.5
height, width = 9, 9
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

satellites = df_all["sat"].unique()
colors = plt.cm.get_cmap("tab10", len(satellites))
# Unique satellites for coloring

rose = dse.ROSE
fig2, ax2 = plt.subplots(
    1,
    1,
    subplot_kw={"projection": ccrs.PlateCarree()},
    figsize=(width, height),
    layout="constrained",
)
rose_cut = rose.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
decorate_axis(ax2, "", *TRACKS_REG)
ax2.set_extent([*TRACKS_REG], crs=ccrs.PlateCarree())
ax2.coastlines()
# list of 8 different of markers
sat_marker_dict = {
    "GFO": "*",
    "TP+J1+J2+J3+S6A": "o",
    "S3A": "^",
    "S3B": "^",
    "TPN+J1N+J2N+J3N": "o",
    "ERS1+ERS2+ENV+SRL": "x",
    "HY2A": "s",
    "HY2B": "s",
}

for i, sat in enumerate(satellites):
    sat_data = df_all[df_all["sat"] == sat]
    ax2.scatter(
        sat_data["lons_inter"],
        sat_data["lats_inter"],
        color=colors(i),
        label=f"Satellite {sat}",
        s=18,
        marker=sat_marker_dict[sat],
        transform=ccrs.PlateCarree(),
    )

# edgecolors='black',
ax2.legend(title="Satellite", loc="upper right")
plt.title("Intersection Points of Tracks in North Indian Ocean")
plt.show()
dse.close()

```

```python
from glob import glob

import cartopy.crs as ccrs
import cmaps
import cmocean as cmo
import matplotlib.pyplot as plt
import xarray as xr
from cartopy.io.img_tiles import GoogleTiles
from tools_xtrackm import *
import matplotlib as mpl


cmap = "Spectral"
cmap = "PuBuGn"
cmap = "YlGnBu"
cmap = "YlGn"
cmap_r = plt.cm.get_cmap(cmap)
cmap2 = cmo.cm.thermal
cmap3 = cmaps.BlAqGrYeOrReVi200
cmap4 = cmo.cm.phase
cmap1 = cmo.cm.topo
cmap1 = cmo.cm.topo
cmap1 = cmo.cm.diff
cmap1 = mpl.colormaps["Greys"]
cmap1 = mpl.colormaps["binary"]

# cmap1="viridis"
zone = "nindian"

IO = (30.0, 120.0, -30.0, 30.0)
NIO = (40.0, 95.0, -5.0, 30.0)
NIO = (40.0, 110.0, -5.0, 30.0)
NIO = (65.0, 95.0, -2.0, 25.0)

height, width = 9, 14
dx = 1.5

xticks = [30, 45, 60, 75, 90, 105]
yticks = [-30, -15, -5, 0, 5, 15, 30]
xticks = [45, 60, 75, 90, 105]
yticks = [0, 5, 15, 30]
tiles = GoogleTiles(style="satellite")

dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")  # open etopo dataset
for sat in sats_new:
    tracks_n = get_total_tracks(sat)
    tsta, tend = get_time_limits(sat)
    fig, ax1 = plt.subplots(
        1,
        1,
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(width, height),
        layout="constrained",
    )
    dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(
        *NIO[2:])).plot(ax=ax1,
                        add_colorbar=False,
                        add_labels=False,
                        cmap=cmap1)
    decorate_axis(ax1, "", *NIO)
    ax1.grid()
    info = f"MISSION: {sat} from {tsta} to {tend} with {tracks_n} tracks"
    plt.text(
        0.15,
        0.95,
        info,
        transform=ax1.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    for f in sorted(
            glob.glob(
                f"/home/srinivasu/xtrackm/data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc"
            )):
        ds = xr.open_dataset(f, decode_times=False)
        track_number = ds.Pass
        print(ds)
        sla = ds.sla
        sla1 = sla.isel(cycles_numbers=0)
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            continue
        lonm = lons_track.mean()
        latm = lats_track.mean()
        m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
        dy = m * dx
        lonm3 = lonm - dx
        latm3 = latm - dy
        if m < 0:
            angle = np.rad2deg(np.arctan(m))
            ax1.scatter(
                lons_track,
                lats_track,
                color="b",
                marker=".",
                s=4,
            )
            if is_within_region(lonm, latm, *NIO):
                plt.text(
                    lonm3,
                    latm3,
                    s=track_number,
                    fontsize=10,
                    rotation=angle,
                    color="r",
                )
        else:
            angle = np.rad2deg(np.arctan(m))
            ax1.scatter(
                lons_track,
                lats_track,
                color="r",
                s=4,
                marker=".",
            )
            if is_within_region(lonm, latm, *NIO):
```

```
                plt.text(
                    lonm3,
                    latm3,
                    s=track_number,
                    fontsize=10,
                    rotation=angle,
                    color="b",
                )
    plt.savefig(
        f"pngs_tracks/tracks_{sat}.{zone}.with_track_numbers.ascending.png",
        bbox_inches="tight",
    )
    plt.show()
    plt.close("all")

dse.close()

```

```python
from glob import glob

import cartopy.crs as ccrs
import cmaps
import cmocean as cmo
import matplotlib.pyplot as plt
import xarray as xr
from cartopy.io.img_tiles import GoogleTiles
from tools_xtrackm import *

cmap = "Spectral"
cmap = "PuBuGn"
cmap = "YlGnBu"
cmap = "YlGn"
cmap_r = plt.cm.get_cmap(cmap)
cmap1 = cmo.cm.topo
cmap2 = cmo.cm.thermal
cmap1 = cmo.cm.topo
cmap1 = cmo.cm.diff
cmap3 = cmaps.BlAqGrYeOrReVi200
cmap4 = cmo.cm.phase

# cmap1="viridis"
zone = "nindian"

IO = (30.0, 120.0, -30.0, 30.0)
NIO = (40.0, 95.0, -5.0, 30.0)
NIO = (40.0, 110.0, -5.0, 30.0)
NIO = (60.0, 100.0, -5.0, 30.0)

height, width = 9, 14

xticks = [30, 45, 60, 75, 90, 105]
yticks = [-30, -15, -5, 0, 5, 15, 30]
xticks = [45, 60, 75, 90, 105]
yticks = [0, 5, 15, 30]
tiles = GoogleTiles(style="satellite")

sats_new = ["S3A"]
for sat in sats_new:
    tracks_n = get_total_tracks(sat)
    tsta, tend = get_time_limits(sat)
    dse = xr.open_dataset(
        "/home/srinivasu/allData/topo/etopo5.cdf"
    )  # open etopo dataset
    fig, ax1 = plt.subplots(
        1,
        1,
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(width, height),
        layout="constrained",
    )
    dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:])).plot(
        ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap1
    )
    decorate_axis(ax1, "", *NIO)
    ax1.grid()
    info = f"MISSION: {sat} from {tsta} to {tend} with {tracks_n} tracks"
    plt.text(
        0.15,
        0.95,
        info,
        transform=ax1.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    for f in sorted(
        glob.glob(f"/home/srinivasu/xtrackm/data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")
    ):
        ds = xr.open_dataset(f, decode_times=False)
        track_number = ds.Pass
        # print(ds)
        sla1 = ds.sla.values
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            continue
        lon_coast = lons_track[-1]  # this on coast
        lat_coast = lats_track[-1]  # this on coast
        lon_equat = lons_track[0]  # this on equator
        lat_equat = lats_track[0]  # this on equator
        a1 = math.atan2(lat_coast - lat_equat, lon_coast - lon_equat)
        a1_d = math.degrees(a1)
        m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
        lonm = lons_track.mean()
        latm = lats_track.mean()
        dx = 1.5
        dy = m * dx
        lonm3 = lonm - dx
        latm3 = latm - dy
        print(a1_d)
        if a1_d > 90:
            ax1.scatter(
                lons_track, lats_track, s=10
            )
            if is_within_region(lonm, latm, *NIO):
                plt.text(
                    lonm,
                    latm,
                    s=track_number,
                    fontsize=10,
                    rotation=a1_d,
                    color="k",
                )
    plt.show()
    plt.close("all")

```

```python
from glob import glob

import cartopy.crs as ccrs
import cmaps
import cmocean as cmo
import matplotlib.pyplot as plt
import pandas as pd
import xarray as xr
from cartopy.io.img_tiles import GoogleTiles
from tools_xtrackm import *

cmap = "Spectral"
cmap = "PuBuGn"
cmap = "YlGnBu"
cmap = "YlGn"
cmap_r = plt.cm.get_cmap(cmap)
cmap1 = cmo.cm.topo
cmap2 = cmo.cm.thermal
cmap1 = cmo.cm.topo
cmap1 = cmo.cm.diff
cmap3 = cmaps.BlAqGrYeOrReVi200
cmap4 = cmo.cm.phase

# cmap1="viridis"
zone = "nindian"

IO = (30.0, 120.0, -30.0, 30.0)
NIO = (40.0, 95.0, -5.0, 30.0)
NIO = (40.0, 110.0, -5.0, 30.0)
NIO = (60.0, 100.0, -5.0, 30.0)

height, width = 9, 14

df = read_tide_meta()
df_nio = region_selected(df, *NIO)
lons_psmsl = df_nio["longitude"]
lats_psmsl = df_nio["latitude"]
pd.set_option("display.max_rows", None)

df_coast = pd.read_excel(
    "/home/srinivasu/allData/coastal/CoastalADCPMeta_correction.xlsx",
    parse_dates=[6, 7],
)
print(df_coast.columns)
print(df_coast.info())
df1 = df_coast[["ID", "Station Name"]]
df2 = df1.drop_duplicates()
indices = df2.index.tolist()
buoys = df2["ID"].tolist()
lons_coast = df_coast["Longitude"].iloc[indices]  # .tolist()
lats_coast = df_coast["Latitude"].iloc[indices]  # .tolist()
print(df2)
dict1 = dict(zip(df2["ID"], df2["Station Name"]))

xticks = [30, 45, 60, 75, 90, 105]
yticks = [-30, -15, -5, 0, 5, 15, 30]
xticks = [45, 60, 75, 90, 105]
yticks = [0, 5, 15, 30]
tiles = GoogleTiles(style="satellite")

sats = ["TP+J1+J2+J3"]
for sat in sats_new:
    tracks_n = get_total_tracks(sat)
    tsta, tend = get_time_limits(sat)
    dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")  # open etopo dataset
    fig, ax1 = plt.subplots(
        1,
        1,
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(width, height),
        layout="constrained",
    )
    dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(
        ETOPO05_Y=slice(*NIO[2:])
    ).plot(ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap1)
    decorate_axis(ax1, "", *NIO)
    ax1.grid()
    info = f"MISSION: {sat} from {tsta} to {tend} with {tracks_n} tracks"
    plt.text(
        0.15,
        0.95,
        info,
        transform=ax1.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    # plt.show()
    # sys.exit()
    # OMNI buoy locations
    ax1.scatter(
        lons_omni,
        lats_omni,
        marker="o",
        c="cyan",
        s=82,
        edgecolor="black",
        label="OMNI buoys",
    )
    # OMNI buoy names
    ln = len(lons_omni)
    for i in range(ln):
        plt.text(
            lons_omni[i],
            lats_omni[i],
            omni_buoys[i],
            fontsize=10,
            ha="right",
            fontweight="bold",
        )
    # PSMSL tide gauge locations
    ax1.scatter(
        lons_psmsl,
        lats_psmsl,
        marker="o",
        c="magenta",
        s=42,
        edgecolor="black",
```

```
        label="PSMSL buoys",
    )
    # COASTAL buoy locations
    ax1.scatter(
        lons_coast,
        lats_coast,
        marker="*",
        c="red",
        s=122,
        edgecolor="black",
        label="COASTAL buoys",
    )
    plt.legend()
    for f in sorted(glob.glob(f"/home/srinivasu/slnew/xtrackd/data/ctoh.sla.ref.{sat}.nindian.*.nc")):
        ds = xr.open_dataset(f, decode_times=False)
        track_number = ds.pass_number
        print(ds)
        sla = ds.sla
        sla1 = sla.isel(cycles_numbers=0)
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            continue
        lonm = lons_track.mean()
        latm = lats_track.mean()
        m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
        dx = 1.5
        dy = m * dx
        lonm3 = lonm - dx
        latm3 = latm - dy
        # lonm3 =  lonm - 1.0
        # latm3 =  latm - 1.0
        # lonm3 =  lon[0]
        # latm3 =  lat[0]
        angle = np.rad2deg(np.arctan(m))
        if m < 0:
            ax1.scatter(lons_track, lats_track, c=sla1, cmap=cmap_r, linewidths=0.0, s=10)
            if is_within_region(lonm, latm, *NIO):
                plt.text(
                    lonm,
                    latm,
                    s=track_number,
                    fontsize=10,
                    rotation=angle,
                    color="k",
                )
    plt.savefig(
        f"pngs_tracks_all_obs/tracks_{sat}.{zone}.with_track_numbers.negative.png",
        bbox_inches="tight",
    )
    plt.show()
    plt.close("all")
    # sys.exit()
```

```python
import glob

import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
from tools_xtrackm import sats_new

fig, axes = plt.subplots(4, 2, figsize=(12, 16), sharex=True)

frac1 = 0.05
xlim1, xlim2 = (-8.0, 80.0)

j = 0
for sat in sats_new[:]:
    i = 0
    gshhg_np = np.empty(0)
    counts_np = np.empty(0)
    for f in sorted(glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")):
        ds = xr.open_dataset(f, decode_times=False)
        gshhg = 0.001 * ds.dist_to_coast_gshhg.values
        counts = ds.sla.count(dim="cycles_numbers").values
        count_perc = 100 * counts / len(ds.cycles_numbers)
        gshhg_np = np.append(gshhg_np, gshhg)
        counts_np = np.append(counts_np, count_perc)
        i = i + 1
    ax = axes.flat[j]
    ax.scatter(
        gshhg_np,
        counts_np,
        marker=".",
        s=1.0,
        color="c",
    )
    print(len(gshhg_np), i)
    f_lfit = f"xtrack_sla_valid_count_{sat}_loess_fit_6km.nc"
    ds_lfit = xr.open_dataset(f_lfit, decode_times=False)
    print(ds_lfit)
    # xout = ds_lfit.gshhg_resampled.values
    xout = ds_lfit.standard_6km.values
    yout = ds_lfit.valid_count.values
    ax.plot(xout, yout, label=f"lowess {frac1}")
    ax.set_xlabel("distance from coast")
    ax.set_ylabel("% valid points")
    ax.set_xlim(xlim1, xlim2)
    ax.set_ylim(0, 110)
    ax.axvline(x=0, color='gray', linestyle='--')
    ax.set_xticks(np.arange(0, xlim2, 10))
    ax.text(
        0.5,
        0.8,
        f"{sat}",
        transform=ax.transAxes,
        fontweight="bold",
        color="r",
    )
    j = j + 1
plt.savefig(
    f"p.count.alltracks_allsat.gshhg.loess.allsat.4by2.png",
    bbox_inches="tight",
)
plt.show()

```

```python
import glob

import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
# import funcformatter
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import sats_new, track_dist_time_asn_gshhg

fig, axes = plt.subplots(4, 2, figsize=(12, 16), sharex=True)

xlim1, xlim2 = (-10e3, 500e3)

x_common = xr.DataArray(np.arange(0, 2_300_001, 6_000), dims=["x"],
  attrs={"units": "m"})  # 0..1300 km, 1 km step


subplot_n = 0
for sat in sats_new[:]:
    counts_array = []
    ax1 = axes.flat[subplot_n]
    files = sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc"))
    for f in files:
        ds = xr.open_dataset(f, decode_times=False)
        sla_asn = track_dist_time_asn_gshhg(ds, var_str="sla", units_in="m")
        counts = sla_asn.count(dim="time")
        count_perc = 100 * counts / len(sla_asn.time)
        count_perc1 = count_perc.interp(x=x_common)
        counts_array.append(count_perc1)
        ax1.scatter(
            count_perc.x.values,
            count_perc.values,
            marker=".",
            s=1.0,
            color="c",
        )
    counts_mean = xr.concat(counts_array, dim="track").mean(dim="track")
    counts_mean.plot(ax = ax1)
    subplot_n = subplot_n + 1
    ax1.set_xlabel("distance from coast")
    ax1.set_ylabel("% valid points")
    ax1.set_xlim(xlim1, xlim2)
    ax1.set_ylim(0, 110)
    ax1.axvline(x=0, color='gray', linestyle='--')
    ax1.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
    ax1.text(
        0.5,
        0.8,
        f"{sat}",
        transform=ax1.transAxes,
        fontweight="bold",
        color="r",
    )
plt.show()

```

meta/p.valid_points.count.gshhg.mean.S3A.py

```python
import glob

import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
# import funcformatter
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import sats_new, track_dist_time_asn_gshhg

fig, ax1 = plt.subplots(1, 1, figsize=(10, 6), sharex=True)

xlim1, xlim2 = (-10e3, 500e3)

x_common = xr.DataArray(np.arange(0, 1_300_001, 6_000), dims=["x"],
  attrs={"units": "m"})  # 0..1300 km, 1 km step

sat = "S3A"
counts_array = []
files = sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc"))
for f in files:
    ds = xr.open_dataset(f, decode_times=False)
    sla_asn = track_dist_time_asn_gshhg(ds, var_str="sla", units_in="m")
    counts = sla_asn.count(dim="time")
    count_perc = 100 * counts / len(sla_asn.time)
    count_perc1 = count_perc.interp(x=x_common)
    counts_array.append(count_perc1)
    ax1.scatter(
        count_perc.x.values,
        count_perc.values,
        marker=".",
        s=1.0,
        color="c",
    )
counts_mean = xr.concat(counts_array, dim="track").mean(dim="track")
counts_mean.plot(ax = ax1)
ax1.set_xlabel("distance from coast")
ax1.set_ylabel("% valid points")
ax1.set_xlim(xlim1, xlim2)
ax1.set_ylim(0, 110)
ax1.axvline(x=0, color='gray', linestyle='--')
ax1.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
ax1.text(
    0.5,
    0.8,
    f"{sat}",
    transform=ax1.transAxes,
    fontweight="bold",
    color="r",
)
plt.show()

```

```python
import glob

import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
from tools_xtrackm import sats_new

fig, axes = plt.subplots(4, 2, figsize=(12, 16), sharex=True)

frac1 = 0.05
xlim1, xlim2 = (-8.0, 80.0)

j = 0
for sat in sats_new[:]:
    i = 0
    gshhg_np = np.empty(0)
    counts_np = np.empty(0)
    for f in sorted(glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")):
        ds = xr.open_dataset(f, decode_times=False)
        gshhg = 0.001 * ds.dist_to_coast_gshhg.values
        counts = ds.sla.count(dim="cycles_numbers").values
        count_perc = 100 * counts / len(ds.cycles_numbers)
        gshhg_np = np.append(gshhg_np, gshhg)
        counts_np = np.append(counts_np, count_perc)
        i = i + 1
    ax = axes.flat[j]
    ax.scatter(
        gshhg_np,
        counts_np,
        marker=".",
        s=1.0,
        color="c",
    )
    print(len(gshhg_np), i)
    f_lfit = f"xtrack_sla_valid_count_{sat}_loess_fit_6km.nc"
    ds_lfit = xr.open_dataset(f_lfit, decode_times=False)
    print(ds_lfit)
    # xout = ds_lfit.gshhg_resampled.values
    xout = ds_lfit.standard_6km.values
    yout = ds_lfit.valid_count.values
    ax.plot(xout, yout, label=f"lowess {frac1}")
    ax.set_xlabel("distance from coast")
    ax.set_ylabel("% valid points")
    ax.set_xlim(xlim1, xlim2)
    ax.set_ylim(0, 110)
    ax.axvline(x=0, color='gray', linestyle='--')
    ax.set_xticks(np.arange(0, xlim2, 10))
    ax.text(
        0.5,
        0.8,
        f"{sat}",
        transform=ax.transAxes,
        fontweight="bold",
        color="r",
    )
    j = j + 1
plt.savefig(
    f"p.count.alltracks_allsat.gshhg.loess.allsat.3by2.png",
    bbox_inches="tight",
)
plt.show()

```

```python
import glob

import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
# import funcformatter
from matplotlib.ticker import FuncFormatter
from tools_xtrackm import sats_new, track_dist_time_asn

fig, axes = plt.subplots(4, 2, figsize=(12, 16), sharex=True)

xlim1, xlim2 = (-10e3, 500e3)

x_common = xr.DataArray(np.arange(0, 2_300_001, 6_000), dims=["x"],
  attrs={"units": "m"})  # 0..1300 km, 1 km step

subplot_n = 0
for sat in sats_new[:]:
    counts_array = []
    ax1 = axes.flat[subplot_n]
    for f in sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
        ds = xr.open_dataset(f, decode_times=False)
        sla_asn = track_dist_time_asn(ds, var_str="sla", units_in="m")
        counts = sla_asn.count(dim="time")
        count_perc = 100 * counts / len(sla_asn.time)
        count_perc1 = count_perc.interp(x=x_common)
        counts_array.append(count_perc1)
        ax1.scatter(
            count_perc.x.values,
            count_perc.values,
            marker=".",
            s=1.0,
            color="c",
        )
    counts_mean = xr.concat(counts_array, dim="track").mean(dim="track")
    counts_mean.plot(ax = ax1)
    subplot_n = subplot_n + 1
    ax1.set_xlabel("distance from coast")
    ax1.set_ylabel("% valid points")
    ax1.set_xlim(xlim1, xlim2)
    ax1.set_ylim(0, 110)
    ax1.axvline(x=0, color='gray', linestyle='--')
    ax1.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{x / 1000:g}"))
    ax1.text(
        0.5,
        0.8,
        f"{sat}",
        transform=ax1.transAxes,
        fontweight="bold",
        color="r",
    )
plt.show()

```

```python
import datetime
from collections import Counter

import xarray as xr
from tools_xtrackm import *
import numpy as np
from math import sqrt, erf

# initiate empyt sla numpy array
sla_np = np.array([])
dac_np = np.array([])
ocean_tide_np = np.array([])
ssb_np = np.array([])

# print min, max, mean and standard deviation of numpy array
# of array containing nans
def stats_nan(sla_np):
    print(f"min is: {np.nanmin(sla_np)}")
    print(f"max is: {np.nanmax(sla_np)}")
    print(f"mean is: {np.nanmean(sla_np)}")
    print(f"std is: {np.nanstd(sla_np)}")


def perc_outside(std_limit = 3.5):
    """Percentage of values outside 3.5 standard deviations in a standard normal distribution"""
    phi0 = 0.5 * (1 + erf(std_limit / sqrt(2)))
    perc0 = (2 * phi0 - 1) * 100
    return 100 - perc0


std_limit = 3.5
for sat in sats_new:
    tracks_n = get_total_tracks(sat)
    print(f"number of tracks of {sat}: {tracks_n}")
    for f in sorted(glob.glob(f"../data/{sat}_lon_ordered/ctoh.sla.ref.{sat}.nindian.*.nc")):
        ds = xr.open_dataset(f, decode_times=False)
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        sla = ds.sla
        dac = ds.dac
        ocean_tide = ds.ocean_tide
        ssb = ds.ssb
        sla_np = np.append(sla_np, sla.values)
        dac_np = np.append(dac_np, dac.values)
        ocean_tide_np = np.append(ocean_tide_np, ocean_tide.values)
        ssb_np = np.append(ssb_np, ssb.values)
    stats_nan(sla_np)
    print(40 * "*")
    stats_nan(dac_np)
    print(40 * "*")
    stats_nan(ocean_tide_np)
    print(40 * "*")
    stats_nan(ssb_np)
    mean = np.nanmean(sla_np)
    # print outliers above 3.5 standard deviations
    std = np.nanstd(sla_np)
    outliers = np.abs(sla_np - mean) > std_limit * std
    # percentage of outliers
    perc = outliers.sum() / len(sla_np) * 100
    # normal percentage of outliers
    perc_out = perc_outside(std_limit)
    print(f"normal percentage of outliers: {perc_out}")
    print(f"percentage of outliers: {perc}")
    break

    # plot histogram
    fig, ax = plt.subplots()
    ax.hist(sla_np, bins=100, label=f"{sat}")
    plt.legend()
    plt.show()

```

meta/print_mission_names.py

```python
import datetime
from collections import Counter

import xarray as xr
from tools_xtrackm import *

for sat in sats_new:
    f = get_first_file(sat)
    tracks_n = get_total_tracks(sat)
    ds = xr.open_dataset(f)
    mission = ds.Mission
    passn = ds.Pass
    cycle = ds.cycle.values[0]
    print(f"mission and pass : {mission} {passn} {cycle}")

```

```
1
2  Before any processing (including gc computation) the original all data is realinged from coast to equator (have a look at track_dist_time_asn from tools_xtrackm), this
       flipped coast-equator distance variable is called x.
3  ∂η/∂x is always taken in the same physical direction (toward equator).
4
5  prompts:
6  write a python program to cycle through all files in xtrackm/data/TPN+J1N+J2N+J3N
7  1. plot 2-D sla shade plots (cycles_numbers as x-axis and points_numbers as y-axis) for each file and save them a directory. Use xarray for reading files.
8  I am trying to find the intersection points of satellite tracks for each
9  satellite mission with the script c.intersection_points.all.py
10 Is the logic correct? Don't worry about efficiency.
11 Don't edit any scripts. Just give opinions.
12
13 Write a function to neatly print timestaps of time_self1 and time_other1 in two side by side columns to the required length.
14
```

```python
from datetime import datetime

import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import pandas as pd
from dateutil.relativedelta import relativedelta

missions = pd.read_csv("data_coverage_df.csv")
print(missions)

colors = ["red", "green", "blue", "purple"]
date_format = "%Y-%m-%d"

min_start_date = min(pd.to_datetime(missions["first_date"]))
max_end_date = max(pd.to_datetime(missions["last_date"]))
min_start_date1 = min_start_date - relativedelta(years=1)
max_end_date1 = max_end_date + relativedelta(years=1)
# print(min_start_date1, max_end_date1)

fig, ax = plt.subplots(figsize=(10, 6), layout="constrained")

for index, row in missions.iterrows():
    sat, track_nos, start_date, end_date, tfreq, xfreq = row.loc[
        ["sat", "trackn", "first_date", "last_date", "tfreq", "xfreq"]
    ]
    sln = len(sat)
    words = [sat, str(track_nos), str(tfreq), str(round(xfreq, 1))]
    start_date_o = datetime.strptime(start_date, date_format)
    end_date_o = datetime.strptime(end_date, date_format)
    duration = end_date_o - start_date_o
    ax.barh(
        index,
        duration,
        left=start_date_o,
        color="skyblue",
        edgecolor="black",
    )
    # current_start_time = start_date_o + relativedelta(years=4)
    current_start_time = start_date_o + duration / 4
    for k, word in enumerate(words):
        sln = len(word)
        text_obj = ax.text(
            current_start_time,
            index,
            word,
            ha="center",
            va="center",
            fontweight="bold",
            fontsize=12,
            color=colors[k],
        )
        if k == 0:
            yrs = int(0.21 * sln) + 2
            mns = int(2.53 * sln) + 10
        else:
            yrs = int(0.21 * sln) + 1.5
            mns = int(2.53 * sln) + 8
        # current_start_time = current_start_time + relativedelta(years=yrs)
        current_start_time = current_start_time + relativedelta(months=mns)

ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
ax.xaxis.set_major_locator(mdates.YearLocator(2))
# ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
# ax.xaxis.set_major_locator(mdates.YearLocator(1))
ax.set_yticks([])
ax.set_yticklabels([])
ax.set_xlabel("years")
ax.set_xlim(min_start_date1, max_end_date1)
ax.set_title("Satellite Missions Timeline")
ax.invert_yaxis()

satellite = mpatches.Patch(color="red", label="satellite")
tracks = mpatches.Patch(color="green", label="no. of tracks")
tfreq = mpatches.Patch(color="blue", label="temporal frequency")
xfreq = mpatches.Patch(color="purple", label="spatial resolution")
plt.legend(handles=[satellite, tracks, tfreq, xfreq])

plt.savefig(f"satellite_mission_time_spans_new_fig01.png")
# plt.tight_layout()
plt.show()

```

meta/selected_topo_maps.txt

```
RdBu_r
coolwarm
rainbow
gist_ncar
jet
gist_gray_r
gist_yarg
binary
BrBG_r
GnBu
GnBu_r
PuOr_r
bwr

```

```
./radar/
```

15 file(s)

```
1
2  variable pool version for scatter plot.
3
4  These p radar scripts p.radar_uvVStrack_guv* are saving on correlation, rmse
5  etc to one timeseries(defined by an intersection and sat)
6  this is not saving extracted values of radar, gu to be latter plotted to
7  a global scatter plot
8  This is saving computed metrics per interseciton
9  We can make another version to global extracted ru and gu
10
11
12 One of the reason there is not much radar timeseries gu comparison scripts
13 in the plots is that is has huge temporal gaps in the radar timeseries
14
```

```
1
2  is done for drifter comparison instead of per point timeseries.
3  [ ] We still need to fix the time alignment of radar and gu
4
```

radar/c.shp.polygon_around_AP.convex_hull.py

```python
import cmaps
import geopandas as gpd
import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
from shapely.geometry import MultiPoint

cmap_rmse = "YlGn"
cmap3 = cmaps.BlAqGrYeOrReVi200

width, height = 10, 10

dsr = xr.open_dataset("/home/srinivasu/allData/radar/APCodar_daily.nc")
dsr = dsr.rename(
    {"XAXS": "longitude", "YAXS": "latitude", "ZAXS": "lev", "TAXIS1D": "time"}
)
print(dsr)

radar_da = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
sizes = dsr.sizes
ln = sizes["time"]
ngp_u = 100 * (radar_da.count(dim="time") / ln)
# Find the indices where the data array is non-zero
non_zero_indices = np.nonzero(ngp_u.values)

# Extract the corresponding latitude and longitude values
lat_vals = ngp_u.latitude.values[non_zero_indices[0]]
lon_vals = ngp_u.longitude.values[non_zero_indices[1]]

# Assuming 'points' is a list of (x, y) tuples of the non-zero coordinates
points = [(lon, lat) for lon, lat in zip(lon_vals, lat_vals)]

# Create a MultiPoint object and then create the smallest polygon that can enclose the points (Convex Hull)
polygon = MultiPoint(points).convex_hull
polygon = polygon.buffer(0.05)

x, y = polygon.exterior.xy

gdf = gpd.GeoDataFrame(
    index=[0],
    geometry=[polygon],
    crs="EPSG:4326",
)
#
gdf.to_file("polygon_shapefile_around_AP.shp")
#
fig, ax = plt.subplots(1, 1, figsize=(6, 4), layout="constrained")
c = ngp_u.plot(ax=ax, cmap=cmap3, add_colorbar=False)
ax.plot(x, y, color="k", label="Polygon Outline")
cb = plt.colorbar(c)
cb.set_label("% of valid data points")
# plt.savefig(f"APradar_valid_points.png")
plt.show()

```

radar/c.shp.polygon_around_OR.convex_hull.py

```python
import cmaps
import geopandas as gpd
import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
from shapely.geometry import MultiPoint

cmap_rmse = "YlGn"
cmap3 = cmaps.BlAqGrYeOrReVi200

width, height = 10, 10

dsr = xr.open_dataset("/home/srinivasu/allData/radar/ORCodar_daily.nc")
dsr = dsr.rename(
    {"XAXS": "longitude", "YAXS": "latitude", "ZAXS": "lev", "TAXIS1D": "time"}
)
print(dsr)

radar_da = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
sizes = dsr.sizes
ln = sizes["time"]
ngp_u = 100 * (radar_da.count(dim="time") / ln)
# Find the indices where the data array is non-zero
non_zero_indices = np.nonzero(ngp_u.values)

# Extract the corresponding latitude and longitude values
lat_vals = ngp_u.latitude.values[non_zero_indices[0]]
lon_vals = ngp_u.longitude.values[non_zero_indices[1]]

# Assuming 'points' is a list of (x, y) tuples of the non-zero coordinates
points = [(lon, lat) for lon, lat in zip(lon_vals, lat_vals)]

# Create a MultiPoint object and then create the smallest polygon that can enclose the points (Convex Hull)
polygon = MultiPoint(points).convex_hull
polygon = polygon.buffer(0.05)

x, y = polygon.exterior.xy

gdf = gpd.GeoDataFrame(
    index=[0],
    geometry=[polygon],
    crs="EPSG:4326",
)
#
gdf.to_file("polygon_shapefile_around_OR.shp")
#
fig, ax = plt.subplots(1, 1, figsize=(6, 4), layout="constrained")
c = ngp_u.plot(ax=ax, cmap=cmap3, add_colorbar=False)
ax.plot(x, y, color="k", label="Polygon Outline")
cb = plt.colorbar(c)
cb.set_label("% of valid data points")
# plt.savefig(f"ORradar_valid_points.png")
plt.show()

```

```python
import cmaps
import geopandas as gpd
import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
from shapely.geometry import MultiPoint

cmap_rmse = "YlGn"
cmap3 = cmaps.BlAqGrYeOrReVi200

width, height = 10, 10

dsr = xr.open_dataset("/home/srinivasu/allData/radar/TNCodar_daily.nc")
dsr = dsr.rename(
    {"XAXS": "longitude", "YAXS": "latitude", "ZAXS": "lev", "TAXIS1D": "time"}
)
print(dsr)

radar_da = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
sizes = dsr.sizes
ln = sizes["time"]
ngp_u = 100 * (radar_da.count(dim="time") / ln)
# Find the indices where the data array is non-zero
non_zero_indices = np.nonzero(ngp_u.values)

# Extract the corresponding latitude and longitude values
lat_vals = ngp_u.latitude.values[non_zero_indices[0]]
lon_vals = ngp_u.longitude.values[non_zero_indices[1]]

# Assuming 'points' is a list of (x, y) tuples of the non-zero coordinates
points = [(lon, lat) for lon, lat in zip(lon_vals, lat_vals)]

# Create a MultiPoint object and then create the smallest polygon that can enclose the points (Convex Hull)
polygon = MultiPoint(points).convex_hull
polygon = polygon.buffer(0.05)

x, y = polygon.exterior.xy

gdf = gpd.GeoDataFrame(
    index=[0],
    geometry=[polygon],
    crs="EPSG:4326",
)
#
gdf.to_file("polygon_shapefile_around_TN.shp")
#
fig, ax = plt.subplots(1, 1, figsize=(6, 4), layout="constrained")
c = ngp_u.plot(ax=ax, cmap=cmap3, add_colorbar=False)
ax.plot(x, y, color="k", label="Polygon Outline")
cb = plt.colorbar(c)
cb.set_label("% of valid data points")
# plt.savefig(f"TNradar_valid_points.png")
plt.show()

```

```python
import cmaps
import cmocean as cmo
import xarray as xr
import matplotlib.pyplot as plt

cmap1 = cmo.cm.topo
cmap2 = cmo.cm.thermal
cmap1 = cmo.cm.topo
cmap3 = cmaps.BlAqGrYeOrReVi200
cmap4 = cmo.cm.phase
cmap_rmse = "YlGn"

width, height = 10, 10
WBOB = (78.0, 87.0, 8.0, 21.0)
AP_RADAR = (80.5, 83.5, 14.2, 16.5)

dsr = xr.open_dataset("/home/srinivasu/allData/radar/APCodar_daily.nc")
print(dsr)
dsr = dsr.rename(
    {"XAXS": "longitude", "YAXS": "latitude", "ZAXS": "lev", "TAXIS1D": "time"}
)
print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
u_radar_m = u_radar.resample(time="1M").mean()
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
v_radar_m = v_radar.resample(time="1M").mean()
print(u_radar)

sizes = v_radar.sizes
ln = sizes["time"]
ngp_u = 100*(v_radar.count(dim="time")/ln)
fig, ax = plt.subplots(1,1, figsize=(6,4), layout="constrained")
c = ngp_u.plot(ax=ax, cmap=cmap3, add_colorbar=False)
cb = plt.colorbar(c)
cb.set_label("% of valid data points")
plt.savefig(f"APradar_valid_points.png")
plt.show()

```

```python
import cmaps
import cmocean as cmo
import xarray as xr
import matplotlib.pyplot as plt

cmap1 = cmo.cm.topo
cmap2 = cmo.cm.thermal
cmap1 = cmo.cm.topo
cmap3 = cmaps.BlAqGrYeOrReVi200
cmap4 = cmo.cm.phase
cmap_rmse = "YlGn"

width, height = 10, 10
WBOB = (78.0, 87.0, 8.0, 21.0)
OR_RADAR = (84.5, 87.0, 17.5, 20.0)

dsr = xr.open_dataset("/home/srinivasu/allData/radar/ORCodar_daily.nc")
print(dsr)
dsr = dsr.rename(
    {"XAXS": "longitude", "YAXS": "latitude", "ZAXS": "lev", "TAXIS1D": "time"}
)
print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
u_radar_m = u_radar.resample(time="1M").mean()
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
v_radar_m = v_radar.resample(time="1M").mean()
print(u_radar)

sizes = v_radar.sizes
ln = sizes["time"]
ngp_u = 100*(v_radar.count(dim="time")/ln)
fig, ax = plt.subplots(1,1, figsize=(6,4), layout="constrained")
c = ngp_u.plot(ax=ax, cmap=cmap3, add_colorbar=False)
cb = plt.colorbar(c)
cb.set_label("% of valid data points")
plt.savefig(f"ORradar_valid_points.png")
plt.show()

```

```python
import cmaps
import cmocean as cmo
import xarray as xr
import matplotlib.pyplot as plt

cmap1 = cmo.cm.topo
cmap2 = cmo.cm.thermal
cmap1 = cmo.cm.topo
cmap3 = cmaps.BlAqGrYeOrReVi200
cmap4 = cmo.cm.phase
cmap_rmse = "YlGn"

width, height = 10, 10
WBOB = (78.0, 87.0, 8.0, 21.0)
TN_RADAR = (79.6, 81.8, 10.5, 12.9)

dsr = xr.open_dataset("/home/srinivasu/allData/radar/TNCodar_daily.nc")
print(dsr)
dsr = dsr.rename(
    {"XAXS": "longitude", "YAXS": "latitude", "ZAXS": "lev", "TAXIS1D": "time"}
)
print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
u_radar_m = u_radar.resample(time="1M").mean()
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
v_radar_m = v_radar.resample(time="1M").mean()
print(u_radar)

sizes = v_radar.sizes
ln = sizes["time"]
ngp_u = 100*(v_radar.count(dim="time")/ln)
fig, ax = plt.subplots(1,1, figsize=(6,4), layout="constrained")
c = ngp_u.plot(ax=ax, cmap=cmap3, add_colorbar=False)
cb = plt.colorbar(c)
cb.set_label("% of valid data points")
plt.savefig(f"TNradar_valid_points.png")
plt.show()

```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs
from matplotlib.colors import ListedColormap

# codex resume 019cce35-fb6a-7e52-9604-ec4d6b2361c9

cmap1 = cmo.cm.diff
cmap2 = ListedColormap(['white'])
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
oscar_extracted_dir = f"/home/srinivasu/xtrackm/guv/oscar_at_intersections/"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
frac1 = "0.05"

radar_str = "OR"
width, height = 13, 6
REG = radar_extents[radar_str + "_BLOCK"]

gdf = gpd.read_file(f"polygon_shapefile_around_{radar_str}.shp")
polygon = gdf.geometry.iloc[0]

@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(f_track, engine="netcdf4", decode_times=False) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg

dsr = xr.open_dataset(
    f"/home/srinivasu/allData/radar/{radar_str}Codar_daily.nc")
# print(dsr)
dsr = dsr.rename({
    "XAXS": "longitude",
    "YAXS": "latitude",
    "ZAXS": "lev",
    "TAXIS1D": "time"
})
# print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
s_radar = 0.01 * dsr.S_RADAR.isel(lev=0, drop=True)
sizes = u_radar.sizes
ln = sizes["time"]

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"../guv/tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

corrs_u = []
biass_u = []
rmses_u = []
unique_strs = []

corrs_v = []
biass_v = []
rmses_v = []
df_out = pd.DataFrame()

ln = len(df)
for idx, row in df_all.iterrows():
    sat1 = row["sat"]
    track_number_self = str(row["track_self"])
    track_number_other = str(row["track_other"])
    unique_str = f"{sat1}_{track_number_self}_{track_number_other}"
    lon1 = row["lons_inter"]
    lat1 = row["lats_inter"]
    lons_inter1 = row["lons_inter"]
    lats_inter1 = row["lats_inter"]
    angle_acute = row["angle_acute"]
    angle_obtuse = row["angle_obtuse"]
    # check if radar region contains intersection
    if not polygon.contains(Point(lon1, lat1)):
        continue
    if abs(lat1) < 2:
        continue
    # print(
    #     sat1,
    #     track_number_self,
    #     track_number_other,
    #     lons_inter1,
    #     lats_inter1,
    #     x_from_coast_self1,
    #     x_from_coast_other1,
    #     angle_acute,
    #     angle_obtuse,
    # )
    ru_at = u_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    rv_at = v_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    f_oscar = f"{oscar_extracted_dir}/oscar_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc"
    if not os.path.exists(f_oscar):
        continue
    with xr.open_dataset(f_oscar) as ds_oscar:
        oscar_u = ds_oscar.u
        oscar_v = ds_oscar.v

    f_guv = f"{guv_dir}/guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{frac1}.nc"
    if not os.path.exists(f_guv):
        print(f"missing file: {f_guv}")
        continue
    with xr.open_dataset(f_guv) as ds_guv:
```

```
        gu = ds_guv.gu.load()
        gv = ds_guv.gv.load()

    (
        lons_track_self,
        lats_track_self,
        lon_equat_self,
        lat_equat_self,
        lon_coast_self,
        lat_coast_self,
        angle_self,
    ) = get_track_geometry(sat1, track_number_self)
    (
        lons_track_other,
        lats_track_other,
        lon_equat_other,
        lat_equat_other,
        lon_coast_other,
        lat_coast_other,
        angle_other,
    ) = get_track_geometry(sat1, track_number_other)
    fig = plt.figure(figsize=(12, 10))
    ax0 = fig.add_subplot(3, 1, 2)
    ax1 = fig.add_subplot(3, 1, 3)

    gu_cut = gu
    gv_cut = gv

    # Collocate radar at track-gu timestamps (same approach as RAMA workflow).
    ru_match = ru_at.reindex(
        time=gu_cut.time,
        method="nearest",
        tolerance=np.timedelta64(24, "h"),
    )
    valid_u = np.isfinite(gu_cut) & np.isfinite(ru_match)
    gu_cut = gu_cut.where(valid_u)
    ru_at = ru_match.where(valid_u)
    oscar_u = oscar_u.reindex(time=gu_cut.time)
    oscar_max = oscar_u.max().item()
    gu_cut = gu_cut.where(gu_cut < oscar_max, np.nan)
    gu_cut = gu_cut.where(gu_cut > -oscar_max, np.nan)

    rv_match = rv_at.reindex(
        time=gv_cut.time,
        method="nearest",
        tolerance=np.timedelta64(24, "h"),
    )
    valid_v = np.isfinite(gv_cut) & np.isfinite(rv_match)
    gv_cut = gv_cut.where(valid_v)
    rv_at = rv_match.where(valid_v)
    oscar_v = oscar_v.reindex(time=gv_cut.time)
    oscar_max = oscar_v.max().item()
    gv_cut = gv_cut.where(gv_cut < oscar_max, np.nan)
    gv_cut = gv_cut.where(gv_cut > -oscar_max, np.nan)

    oscar_u.plot(ax=ax0, color="b", linewidth=2, label="oscar u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="k", linewidth=2)
    ru_at.plot(ax=ax0, label=f"radar zonal", color="r", linewidth=2)
    # ax0.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax0.legend()
    ax0.set_ylabel("oscar u m/s")

    oscar_v.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="k", linewidth=2)
    rv_at.plot(ax=ax1, label=f"radar meridional", color="r", linewidth=2)

    # corr_u = xs.pearson_r(gu_cut, oscar_u, dim="time", skipna=True)
    # rmse_u = xs.rmse(gu_cut, oscar_u, dim="time", skipna=True)
    # bias_var_u = oscar_u - gu_cut
    corr_u = xs.pearson_r(gu_cut, ru_at, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, ru_at, dim="time", skipna=True)
    bias_var_u = ru_at - gu_cut
    bias_u = bias_var_u.mean(dim="time")
    # corr_v = xs.pearson_r(gv_cut, oscar_v, dim="time", skipna=True)
    # rmse_v = xs.rmse(gv_cut, oscar_v, dim="time", skipna=True)
    # bias_var_v = oscar_v - gv_cut
    corr_v = xs.pearson_r(gv_cut, rv_at, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, rv_at, dim="time", skipna=True)
    bias_var_v = rv_at - gv_cut
    bias_v = bias_var_v.mean(dim="time")

    corrs_u.append(corr_u.item())
    rmses_u.append(rmse_u.item())
    biass_u.append(bias_u.item())
    unique_strs.append(unique_str)

    corrs_v.append(corr_v.item())
    rmses_v.append(rmse_v.item())
    biass_v.append(bias_v.item())

    print(f"Correlation between gu and u_radar: {corr_u.item()}")
    print(f"Correlation between gv and v_radar: {corr_v.item()}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.05,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("oscar v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap2)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.add_geometries(
        [polygon],
        ccrs.PlateCarree(),
        facecolor="lightblue",
        edgecolor="black",
```

```
        alpha=0.2,
    )
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=4,
        markerfacecolor="red",
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"track_guvVSoscar_at_intersection_points/guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    plt.show()
    plt.close("all")

dse.close()
# add corrs, rmses, biass to dataframe df_out
df_out = pd.DataFrame()
df_out["corrs"] = corrs_u
df_out["rmses"] = rmses_u
df_out["biass"] = biass_u
df_out["unique_str"] = unique_strs
# drop nans
df_out = df_out.dropna(subset=["corrs", "rmses", "biass"])
df_out.to_csv(f"radar_metrics_{radar_str}_u.csv", index=False)

df_out = pd.DataFrame()
df_out["corrs"] = corrs_v
df_out["rmses"] = rmses_v
df_out["biass"] = biass_v
df_out["unique_str"] = unique_strs

df_out = df_out.dropna(subset=["corrs", "rmses", "biass"])
df_out.to_csv(f"radar_metrics_{radar_str}_v.csv", index=False)

```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
from matplotlib.colors import ListedColormap

# codex resume 019cce35-fb6a-7e52-9604-ec4d6b2361c9

cmap1 = cmo.cm.diff
cmap2 = ListedColormap(['white'])
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
oscar_extracted_dir = f"/home/srinivasu/xtrackm/guv/oscar_at_intersections/"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
EKMAN_DIR = "/home/srinivasu/xtrackm/ekman/ekman_at_intersections"
frac1 = "0.05"

radar_str = "OR"
width, height = 13, 6
REG = radar_extents[radar_str + "_BLOCK"]

gdf = gpd.read_file(f"polygon_shapefile_around_{radar_str}.shp")
polygon = gdf.geometry.iloc[0]

@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(f_track, engine="netcdf4", decode_times=False) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


def build_unique_str(sat, row):
    track_self = track_to_str(row["track_self"])
    track_other = track_to_str(row["track_other"])
    return f"{sat}_{track_self}_{track_other}"


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)


dsr = xr.open_dataset(
    f"/home/srinivasu/allData/radar/{radar_str}Codar_daily.nc")
# print(dsr)
dsr = dsr.rename({
    "XAXS": "longitude",
    "YAXS": "latitude",
    "ZAXS": "lev",
    "TAXIS1D": "time"
})
# print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
s_radar = 0.01 * dsr.S_RADAR.isel(lev=0, drop=True)
sizes = u_radar.sizes
ln = sizes["time"]

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"../guv/tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

ru_gu_records = []

ln = len(df)
for idx, row in df_all.iterrows():
    sat1 = row["sat"]
    track_number_self = str(row["track_self"])
    track_number_other = str(row["track_other"])
    lon1 = row["lons_inter"]
    lat1 = row["lats_inter"]
    lons_inter1 = row["lons_inter"]
    lats_inter1 = row["lats_inter"]
    unique_str = build_unique_str(sat1, row)
    angle_acute = row["angle_acute"]
    angle_obtuse = row["angle_obtuse"]
    f_ek = f"{EKMAN_DIR}/{unique_str}_ekman_at_intersections.nc"
    ds_ek = xr.open_dataset(f_ek)
    u_ek = ds_ek['u_at']
    v_ek = ds_ek['v_at']
    # check if radar region contains intersection
    if not polygon.contains(Point(lon1, lat1)):
        continue
    if abs(lat1) < 2:
        continue
    # print(
    #     sat1,
    #     track_number_self,
    #     track_number_other,
    #     lons_inter1,
    #     lats_inter1,
    #     x_from_coast_self1,
    #     x_from_coast_other1,
    #     angle_acute,
    #     angle_obtuse,
    # )
    ru_at = u_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    rv_at = v_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    f_oscar = f"{oscar_extracted_dir}/oscar_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc"
    if not os.path.exists(f_oscar):
```

```
        continue
    with xr.open_dataset(f_oscar) as ds_oscar:
        oscar_u = ds_oscar.u
        oscar_v = ds_oscar.v

    f_guv = f"{guv_dir}/guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{frac1}.nc"
    if not os.path.exists(f_guv):
        print(f"missing file: {f_guv}")
        continue
    with xr.open_dataset(f_guv) as ds_guv:
        gu = ds_guv.gu.load()
        gv = ds_guv.gv.load()

    (
        lons_track_self,
        lats_track_self,
        lon_equat_self,
        lat_equat_self,
        lon_coast_self,
        lat_coast_self,
        angle_self,
    ) = get_track_geometry(sat1, track_number_self)
    (
        lons_track_other,
        lats_track_other,
        lon_equat_other,
        lat_equat_other,
        lon_coast_other,
        lat_coast_other,
        angle_other,
    ) = get_track_geometry(sat1, track_number_other)
    fig = plt.figure(figsize=(12, 10))
    ax0 = fig.add_subplot(3, 1, 2)
    ax1 = fig.add_subplot(3, 1, 3)

    gu_cut = gu
    gv_cut = gv

    # Collocate radar at track-gu timestamps (same approach as RAMA workflow).
    ru_match = ru_at.reindex(
        time=gu_cut.time,
        method="nearest",
        tolerance=np.timedelta64(24, "h"),
    )
    ek_u_match = u_ek.reindex(
        time=gu_cut.time,
        method="nearest",
        tolerance=np.timedelta64(24, "h"),
    )
    valid_u = np.isfinite(gu_cut) & np.isfinite(ru_match)
    gu_cut = gu_cut.where(valid_u)
    ru_at = ru_match.where(valid_u)
    ek_u = ek_u_match.where(valid_u)
    gu_ek = gu_cut + ek_u
    oscar_u = oscar_u.reindex(time=gu_cut.time)
    oscar_max = oscar_u.max().item()
    gu_cut = gu_cut.where(gu_cut < oscar_max, np.nan)
    gu_cut = gu_cut.where(gu_cut > -oscar_max, np.nan)

    rv_match = rv_at.reindex(
        time=gv_cut.time,
        method="nearest",
        tolerance=np.timedelta64(24, "h"),
    )
    ek_v_match = v_ek.reindex(
        time=gv_cut.time,
        method="nearest",
        tolerance=np.timedelta64(24, "h"),
    )
    valid_v = np.isfinite(gv_cut) & np.isfinite(rv_match)
    gv_cut = gv_cut.where(valid_v)
    rv_at = rv_match.where(valid_v)
    ek_v = ek_v_match.where(valid_v)
    gv_ek = gv_cut + ek_v
    oscar_v = oscar_v.reindex(time=gv_cut.time)
    oscar_max = oscar_v.max().item()
    gv_cut = gv_cut.where(gv_cut < oscar_max, np.nan)
    gv_cut = gv_cut.where(gv_cut > -oscar_max, np.nan)

    oscar_u.plot(ax=ax0, color="b", linewidth=2, label="oscar u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="k", linewidth=2)
    ru_at.plot(ax=ax0, label=f"radar zonal", color="r", linewidth=2)
    # ax0.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax0.legend()
    ax0.set_ylabel("oscar u m/s")

    oscar_v.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="k", linewidth=2)
    rv_at.plot(ax=ax1, label=f"radar meridional", color="r", linewidth=2)

    valid_ru_gu = np.isfinite(gu_cut.values) & np.isfinite(ru_at.values)
    if np.any(valid_ru_gu):
        ru_gu_records.append(
            pd.DataFrame(
                {
                    "time": pd.to_datetime(gu_cut.time.values[valid_ru_gu]),
                    "sat": sat1,
                    "track_self": track_number_self,
                    "track_other": track_number_other,
                    "lon_intersection": lon1,
                    "lat_intersection": lat1,
                    "ru": ru_at.values[valid_ru_gu],
                    "gu": gu_cut.values[valid_ru_gu],
                    "ek_u": ek_u.values[valid_ru_gu],
                }
            )
        )
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.05,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
```

```
    ax1.legend()
    ax1.set_ylabel("oscar v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap2)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.add_geometries(
        [polygon],
        ccrs.PlateCarree(),
        facecolor="lightblue",
        edgecolor="black",
        alpha=0.2,
    )
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=4,
        markerfacecolor="red",
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"track_guvVSoscar_at_intersection_points/guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    plt.show()
    plt.close("all")

dse.close()
if ru_gu_records:
    df_out = pd.concat(ru_gu_records, ignore_index=True)
else:
    df_out = pd.DataFrame(
        columns=[
            "time",
            "sat",
            "track_self",
            "track_other",
            "lon_intersection",
            "lat_intersection",
            "ru",
            "gu",
        ]
    )
df_out.to_csv(f"radar_valid_ru_gu_{radar_str}.csv", index=False)

```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
from matplotlib.colors import ListedColormap

# codex resume 019cce35-fb6a-7e52-9604-ec4d6b2361c9

cmap1 = cmo.cm.diff
cmap2 = ListedColormap(['white'])
d = 1.5
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
oscar_extracted_dir = f"/home/srinivasu/xtrackm/guv/oscar_at_intersections/"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
EKMAN_DIR = "/home/srinivasu/xtrackm/ekman/ekman_at_intersections"
frac1 = "0.05"

radar_str = "OR"
width, height = 13, 6
REG = radar_extents[radar_str + "_BLOCK"]

gdf = gpd.read_file(f"polygon_shapefile_around_{radar_str}.shp")
polygon = gdf.geometry.iloc[0]

@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(f_track, engine="netcdf4", decode_times=False) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


def build_unique_str(sat, row):
    track_self = track_to_str(row["track_self"])
    track_other = track_to_str(row["track_other"])
    return f"{sat}_{track_self}_{track_other}"


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)


dsr = xr.open_dataset(
    f"/home/srinivasu/allData/radar/{radar_str}Codar_daily.nc")
# print(dsr)
dsr = dsr.rename({
    "XAXS": "longitude",
    "YAXS": "latitude",
    "ZAXS": "lev",
    "TAXIS1D": "time"
})
# print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
s_radar = 0.01 * dsr.S_RADAR.isel(lev=0, drop=True)
sizes = u_radar.sizes
ln = sizes["time"]

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"../guv/tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

ru_gu_records = []

ln = len(df)
for idx, row in df_all.iterrows():
    sat1 = row["sat"]
    track_number_self = str(row["track_self"])
    track_number_other = str(row["track_other"])
    lon1 = row["lons_inter"]
    lat1 = row["lats_inter"]
    lons_inter1 = row["lons_inter"]
    lats_inter1 = row["lats_inter"]
    angle_acute = row["angle_acute"]
    angle_obtuse = row["angle_obtuse"]
    f_ek = f"{EKMAN_DIR}/{unique_str}_ekman_at_intersections.nc"
    ds_ek = xr.open_dataset(f_ek)
    u_ek = ds_ek['u_at']
    v_ek = ds_ek['v_at']
    # check if radar region contains intersection
    if not polygon.contains(Point(lon1, lat1)):
        continue
    if abs(lat1) < 2:
        continue
    # print(
    #     sat1,
    #     track_number_self,
    #     track_number_other,
    #     lons_inter1,
    #     lats_inter1,
    #     x_from_coast_self1,
    #     x_from_coast_other1,
    #     angle_acute,
    #     angle_obtuse,
    # )
    ru_at = u_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    rv_at = v_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    f_oscar = f"{oscar_extracted_dir}/oscar_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc"
    if not os.path.exists(f_oscar):
        continue
```

```
    with xr.open_dataset(f_oscar) as ds_oscar:
        oscar_u = ds_oscar.u
        oscar_v = ds_oscar.v

    f_guv = f"{guv_dir}/guv_at_intersection_{sat1}_{track_number_self}_{track_number_other}_{frac1}.nc"
    if not os.path.exists(f_guv):
        print(f"missing file: {f_guv}")
        continue
    with xr.open_dataset(f_guv) as ds_guv:
        gu = ds_guv.gu.load()
        gv = ds_guv.gv.load()

    (
        lons_track_self,
        lats_track_self,
        lon_equat_self,
        lat_equat_self,
        lon_coast_self,
        lat_coast_self,
        angle_self,
    ) = get_track_geometry(sat1, track_number_self)
    (
        lons_track_other,
        lats_track_other,
        lon_equat_other,
        lat_equat_other,
        lon_coast_other,
        lat_coast_other,
        angle_other,
    ) = get_track_geometry(sat1, track_number_other)
    fig = plt.figure(figsize=(12, 10))
    ax0 = fig.add_subplot(3, 1, 2)
    ax1 = fig.add_subplot(3, 1, 3)

    gu_cut = gu
    gv_cut = gv

    # Collocate radar at track-gu timestamps (same approach as RAMA workflow).
    ru_match = ru_at.reindex(
        time=gu_cut.time,
        method="nearest",
        tolerance=np.timedelta64(24, "h"),
    )
    ek_u_match = u_ek.reindex(
        time=gu_cut.time,
        method="nearest",
        tolerance=np.timedelta64(24, "h"),
    )
    valid_u = np.isfinite(gu_cut) & np.isfinite(ru_match)
    gu_cut = gu_cut.where(valid_u)
    ru_at = ru_match.where(valid_u)
    ek_u = ek_u_match.where(valid)
    gu_ek = gu_cut + ek_u
    oscar_u = oscar_u.reindex(time=gu_cut.time)
    oscar_max = oscar_u.max().item()
    gu_cut = gu_cut.where(gu_cut < oscar_max, np.nan)
    gu_cut = gu_cut.where(gu_cut > -oscar_max, np.nan)

    rv_match = rv_at.reindex(
        time=gv_cut.time,
        method="nearest",
        tolerance=np.timedelta64(24, "h"),
    )
    ek_v_match = v_ek.reindex(
        time=gv_cut.time,
        method="nearest",
        tolerance=np.timedelta64(24, "h"),
    )
    valid_v = np.isfinite(gv_cut) & np.isfinite(rv_match)
    gv_cut = gv_cut.where(valid_v)
    rv_at = rv_match.where(valid_v)
    ek_v = ek_v_match.where(valid)
    gv_ek = gv_cut + ek_v
    oscar_v = oscar_v.reindex(time=gv_cut.time)
    oscar_max = oscar_v.max().item()
    gv_cut = gv_cut.where(gv_cut < oscar_max, np.nan)
    gv_cut = gv_cut.where(gv_cut > -oscar_max, np.nan)

    oscar_u.plot(ax=ax0, color="b", linewidth=2, label="oscar u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="k", linewidth=2)
    ru_at.plot(ax=ax0, label=f"radar zonal", color="r", linewidth=2)
    # ax0.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax0.legend()
    ax0.set_ylabel("oscar u m/s")

    oscar_v.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="k", linewidth=2)
    rv_at.plot(ax=ax1, label=f"radar meridional", color="r", linewidth=2)

    valid_ru_gu = np.isfinite(gu_cut.values) & np.isfinite(ru_at.values)
    if np.any(valid_ru_gu):
        ru_gu_records.append(
            pd.DataFrame(
                {
                    "time": pd.to_datetime(gu_cut.time.values[valid_ru_gu]),
                    "sat": sat1,
                    "track_self": track_number_self,
                    "track_other": track_number_other,
                    "lon_intersection": lon1,
                    "lat_intersection": lat1,
                    "ru": ru_at.values[valid_ru_gu],
                    "gu": gu_cut.values[valid_ru_gu],
                }
            )
        )
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.05,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("oscar v m/s")
```

```
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap2)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.add_geometries(
        [polygon],
        ccrs.PlateCarree(),
        facecolor="lightblue",
        edgecolor="black",
        alpha=0.2,
    )
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=4,
        markerfacecolor="red",
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"track_guvVSoscar_at_intersection_points/guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    plt.show()
    plt.close("all")

dse.close()
if ru_gu_records:
    df_out = pd.concat(ru_gu_records, ignore_index=True)
else:
    df_out = pd.DataFrame(
        columns=[
            "time",
            "sat",
            "track_self",
            "track_other",
            "lon_intersection",
            "lat_intersection",
            "ru",
            "gu",
        ]
    )
df_out.to_csv(f"radar_valid_ru_gu_{radar_str}.csv", index=False)

```

```python
import os
from functools import lru_cache

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
import xskillscore as xs
from geopy import distance
from tools_xtrackm import *


@lru_cache(maxsize=None)
def get_track_geometry(sat, track_number):
    f_track = f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.{str(track_number).zfill(3)}.nc"
    with xr.open_dataset(
        f_track, engine="netcdf4", decode_times=False
    ) as ds_track:
        lons = ds_track.lon.values
        lats = ds_track.lat.values
    lon_equat = lons[0]
    lat_equat = lats[0]
    lon_coast = lons[-1]
    lat_coast = lats[-1]
    slope = (lat_coast - lat_equat) / (lon_coast - lon_equat)
    angle_deg = np.rad2deg(np.arctan(slope))
    return lons, lats, lon_equat, lat_equat, lon_coast, lat_coast, angle_deg


def build_unique_str(sat, row):
    track_self = track_to_str(row["track_self"])
    track_other = track_to_str(row["track_other"])
    return f"{sat}_{track_self}_{track_other}"


def track_to_str(track):
    track_f = float(track)
    if track_f.is_integer():
        return str(int(track_f))
    return str(track)


cmap1 = cmo.cm.diff
d = 1.5
height, width = 9, 9
frac1 = "0.05"


dse = xr.open_dataset("~/allData/topo/etopo5.cdf")
rose_cut = dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(
    ETOPO05_Y=slice(*TRACKS_REG[2:])
)
rama_data_dir = "/home/srinivasu/allData/rama/cur/"
guv_dir = "/home/srinivasu/xtrackm/guv/guv_at_intersections"
out_match_csv = "rama_guv_c_nearest_global_matches.ek.csv"
EKMAN_DIR = "/home/srinivasu/xtrackm/ekman/ekman_at_intersections"
matched_rows_all = []

df_all_list = []
for sat in sats_new:
    df_all_list.append(pd.read_csv(f"tracks_intersections_{sat}_1.csv"))
df_all = pd.concat(df_all_list, ignore_index=True)

for rama_id, (lon_rama, lat_rama) in rama_d.items():
    if abs(lat_rama) < 2:
        continue
    rama_file = f"{rama_data_dir}/cur{rama_id}_dy.cdf"
    if not os.path.exists(rama_file):
        continue
    print(rama_id, lon_rama, lat_rama)
    with xr.open_dataset(f"{rama_data_dir}/cur{rama_id}_dy.cdf") as ds_rama:
        # print(ds_rama)
        rama_u = 0.01 * ds_rama.U_320.isel(depth=0, drop=True)
        rama_v = 0.01 * ds_rama.V_321.isel(depth=0, drop=True)

    # print(rama_v.count().item())
    df_4 = find_n_closest_points(df_all, lon_rama, lat_rama, n=4)
    for nth in range(4):
        row = df_4.iloc[nth]

        sat = row["sat"]
        track_number_self = str(row["track_self"])
        track_number_other = str(row["track_other"])
        lon1 = row["lons_inter"]
        lat1 = row["lats_inter"]
        distance_km = row["distance_km"]
        angle_acute = row["angle_acute"]
        angle_obtuse = row["angle_obtuse"]

        unique_str = build_unique_str(sat, row)
        f_ek = f"{EKMAN_DIR}/{unique_str}_ekman_at_intersections.nc"
        ds_ek = xr.open_dataset(f_ek)
        u_ek = ds_ek['u_at']
        v_ek = ds_ek['v_at']

        if abs(lat1) < 2:
            continue

        print(
            sat,
            rama_id,
            track_number_self,
            track_number_other,
            lon1,
            lat1,
            angle_acute,
            angle_obtuse,
        )
        print((
        f"\nSELECT_SAT = '{sat}'"
        f"\nSELECT_RAMA_ID = '{rama_id}'"
        f"\nSELECT_TRACK_SELF = '{track_number_self}'"
        f"\nSELECT_TRACK_OTHER = '{track_number_other}'"
        ))

        f_guv = f"{guv_dir}/guv_at_intersection_{sat}_{track_number_self}_{track_number_other}_{frac1}.nc"
```

```
        if not os.path.exists(f_guv):
            print(f"missing file: {f_guv}")
            continue

        with xr.open_dataset(f_guv) as ds_guv:
            gu = ds_guv.gu
            gv = ds_guv.gv

        (
            lons_track_self,
            lats_track_self,
            lon_equat_self,
            lat_equat_self,
            lon_coast_self,
            lat_coast_self,
            angle_self,
        ) = get_track_geometry(sat, track_number_self)
        (
            lons_track_other,
            lats_track_other,
            lon_equat_other,
            lat_equat_other,
            lon_coast_other,
            lat_coast_other,
            angle_other,
        ) = get_track_geometry(sat, track_number_other)

        gu_cut = gu
        gv_cut = gv

        # Collocate RAMA at track-gu timestamps; avoid sat-dependent smoothing.
        rama_u_match = rama_u.reindex(
            time=gu_cut.time,
            method="nearest",
            tolerance=np.timedelta64(24, "h"),
        )
        ek_u_match = u_ek.reindex(
            time=gu_cut.time,
            method="nearest",
            tolerance=np.timedelta64(24, "h"),
        )
        valid = np.isfinite(gu_cut) & np.isfinite(rama_u_match)
        gu_pair = gu_cut.where(valid)
        rama_u_pair = rama_u_match.where(valid)
        ek_u_pair = ek_u_match.where(valid)
        gu_ek_pair = gu_pair + ek_u_pair

        df_pairs = xr.Dataset({"gu": gu_pair, "rama_u": rama_u_pair, "u_ek": ek_u_pair}).to_dataframe().reset_index()
        df_pairs = df_pairs.dropna(subset=["gu", "rama_u", "u_ek"])
        print(df_pairs.columns)
        matched_pairs = len(df_pairs)
        if matched_pairs > 0:
            """All these will broadcasted along length of gu, rama_u"""
            df_pairs["sat"] = sat
            df_pairs["rama_id"] = rama_id
            df_pairs["distance_km"] = distance_km
            df_pairs["track_self"] = track_number_self
            df_pairs["track_other"] = track_number_other
            df_pairs["matched_pairs"] = matched_pairs
            df_pairs["timestamp"] = pd.to_datetime(df_pairs["time"]).astype(str)
            print(df_pairs.columns)
            matched_rows_all.append(
                df_pairs[
                    [
                        "sat",
                        "rama_id",
                        "distance_km",
                        "track_self",
                        "track_other",
                        "matched_pairs",
                        "timestamp",
                        "gu",
                        "rama_u",
                        "u_ek",
                    ]
                ]
            )

        # check valid points
        ngp = np.sum(valid)
        if ngp < 2:
            continue
        fig = plt.figure(figsize=(12, 8))
        ax2 = fig.add_subplot(2, 1, 1, projection=ccrs.PlateCarree())
        ax1 = fig.add_subplot(2, 1, 2)
        rama_u_pair.plot(ax=ax1, color="r", linewidth=2, label="rama")
        gu_pair.plot(ax=ax1, color="k", linewidth=2, label="track")
        gu_ek_pair.plot(ax=ax1, color="b", linewidth=2, label="track+ekman")

        corr = xs.pearson_r(gu_pair, rama_u_pair, dim="time", skipna=True)
        corr1 = corr.item()

        info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat} {distance_km:.2f} km"
        print(f"Correlation between gu and rama u: {corr1}")
        plt.text(
            0.25,
            0.90,
            info,
            transform=ax1.transAxes,
            fontsize=12,
            fontweight="bold",
        )
        plt.legend()

        rose_cut.plot(ax=ax2, add_colorbar=False, add_labels=False, cmap=cmap1)
        decorate_axis(ax2, "", *TRACKS_REG, step=10)
        ax2.grid()
        ax2.plot(
            lon_rama,
            lat_rama,
            c="r",
            marker="o",
            markersize=4,
            markerfacecolor="red",
            markeredgecolor="red",
            markeredgewidth=2,
        )
```

```
235
236         ax2.plot(
237             lon1,
238             lat1,
239             c="k",
240             marker=".",
241             markersize=8,
242             markerfacecolor="black",
243             markeredgecolor="black",
244             markeredgewidth=2,
245         )
246         plt.text(lon_rama, lat_rama, f"{rama_id}", fontsize=10, color="k")
247         ax2.scatter(
248             lons_track_self, lats_track_self, marker=".", color="c", s=4
249         )
250         ax2.scatter(
251             lons_track_other, lats_track_other, marker=".", color="c", s=4
252         )
253
254         lonm, latm = get_point_at_distance(
255             lon_equat_self,
256             lat_equat_self,
257             lon_coast_self,
258             lat_coast_self,
259             d,
260         )
261         info = f"{sat} {track_number_self} {track_number_other} {distance_km:.2f} km"
262         plt.text(
263             0.55,
264             0.90,
265             info,
266             transform=ax2.transAxes,
267             fontsize=10,
268             fontweight="bold",
269         )
270         if is_within_region(lonm, latm, *TRACKS_REG):
271             plt.text(
272                 lonm,
273                 latm,
274                 s=track_number_self,
275                 fontsize=14,
276                 rotation=angle_self,
277                 color="w",
278             )
279
280         lonm, latm = get_point_at_distance(
281             lon_equat_other,
282             lat_equat_other,
283             lon_coast_other,
284             lat_coast_other,
285             d,
286         )
287         if is_within_region(lonm, latm, *TRACKS_REG):
288             plt.text(
289                 lonm,
290                 latm,
291                 s=track_number_other,
292                 fontsize=14,
293                 rotation=angle_other,
294                 color="w",
295             )
296
297         # plt.show()
298         plt.close("all")
299
300 if len(matched_rows_all) > 0:
301     df_matched_all = pd.concat(matched_rows_all, ignore_index=True)
302     df_matched_all.to_csv(out_match_csv, index=False)
303     print(f"saved global match csv: {out_match_csv}, rows={len(df_matched_all)}")
304 #
305 dse.close()
306
```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

CSV_PATH = "radar_valid_ru_gu_OR.csv"
OUT_PNG = "scatter_radar_u_vs_gu.png"
MIN_DISTANCE_KM = None
MAX_DISTANCE_KM = 250.0


df = pd.read_csv(CSV_PATH)

x = df["gu"].to_numpy(dtype=float)
y = df["ru"].to_numpy(dtype=float)
valid = np.isfinite(x) & np.isfinite(y)
x = x[valid]
y = y[valid]

if x.size < 3:
    print(
        f"Only {x.size} valid points after distance filter "
        f"(min={MIN_DISTANCE_KM}, max={MAX_DISTANCE_KM})."
    )
else:
    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))

    fig, ax = plt.subplots(figsize=(9, 7))
    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1)
    cbar = fig.colorbar(hb, ax=ax)
    cbar.set_label("Count")

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=11,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="black", alpha=0.55, edgecolor="white"),
        color="white",
    )

    ax.set_title(f"")
    ax.set_xlabel("Altimetry gu (m/s)")
    ax.set_ylabel("radar u (m/s)")
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right")
    fig.tight_layout()
    fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)

    print(f"Rows after distance filter: {len(df)}")
    print(f"Saved {OUT_PNG}")

```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

CSV_PATH = "radar_valid_ru_gu_OR.csv"
OUT_PNG = "scatter_radar_u_vs_gu_ek.png"
MIN_DISTANCE_KM = None
MAX_DISTANCE_KM = 250.0


df = pd.read_csv(CSV_PATH)

x1 = df["gu"].to_numpy(dtype=float)
x2 = df["ek_u"].to_numpy(dtype=float)
x = x1 + x2
y = df["ru"].to_numpy(dtype=float)
valid = np.isfinite(x) & np.isfinite(y)
x = x[valid]
y = y[valid]

if x.size < 3:
    print(
        f"Only {x.size} valid points after distance filter "
        f"(min={MIN_DISTANCE_KM}, max={MAX_DISTANCE_KM})."
    )
else:
    slope, intercept = np.polyfit(x, y, 1)
    yhat = slope * x + intercept
    r = np.corrcoef(x, y)[0, 1]
    stderr = np.sqrt(np.sum((y - yhat) ** 2) / (x.size - 2))

    fig, ax = plt.subplots(figsize=(9, 7))
    hb = ax.hexbin(x, y, gridsize=55, cmap="viridis", mincnt=1)
    cbar = fig.colorbar(hb, ax=ax)
    cbar.set_label("Count")

    xs = np.linspace(np.nanmin(x), np.nanmax(x), 200)
    ax.plot(xs, slope * xs + intercept, color="crimson", linewidth=2.0, label="Linear fit")
    ax.plot(xs, xs, color="white", linestyle="--", linewidth=1.2, label="1:1 line")

    text = (
        f"N = {x.size}\n"
        f"r = {r:.3f}\n"
        f"Std error = {stderr:.3f} m/s\n"
        f"y = {slope:.3f}x + {intercept:.3f}"
    )
    ax.text(
        0.03,
        0.97,
        text,
        transform=ax.transAxes,
        va="top",
        ha="left",
        fontsize=11,
        bbox=dict(boxstyle="round,pad=0.4", facecolor="black", alpha=0.55, edgecolor="white"),
        color="white",
    )

    ax.set_title(f"")
    ax.set_xlabel("Altimetry gu (m/s)")
    ax.set_ylabel("radar u (m/s)")
    ax.grid(alpha=0.25)
    ax.legend(loc="lower right")
    fig.tight_layout()
    fig.savefig(OUT_PNG, dpi=180)
    plt.show()
    plt.close(fig)

    print(f"Rows after distance filter: {len(df)}")
    print(f"Saved {OUT_PNG}")

```

```
1
2  p.radar_uvVStrack_guv_c.at.intersections.py similar to
3  p.rama_uvVStrack_guv_c.nearest.all.py
4
5  Refactor the code p.rama_uvVStrack_guv_c.nearest.all.py to use pre computed guv fields
6  at /home/srinivasu/xtrackm/guv/guv_at_intersections
7  use functions in p.aviso_only_uvVStrack_guv_c.at.intersections.py to find
8  track angle etc.
9
```

```
./radar/archive/
```

```
1 file(s)
```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs
from matplotlib.colors import ListedColormap

def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    return u, v


cmap1 = cmo.cm.diff
cmap2 = ListedColormap(['white'])
d = 1.5
oscar_dir = "/home/srinivasu/allData/oscar_0.25/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
oscar_extracted_dir = f"/home/srinivasu/xtrackm/guv/oscar_at_intersections/"

radar_str = "OR"
width, height = 13, 6
REG = radar_extents[radar_str + "_BLOCK"]

gdf = gpd.read_file(f"polygon_shapefile_around_{radar_str}.shp")
polygon = gdf.geometry.iloc[0]

radar_tsta_o, radar_tend_o = radar_times[radar_str]

dsr = xr.open_dataset(
    f"/home/srinivasu/allData/radar/{radar_str}Codar_daily.nc")
# print(dsr)
dsr = dsr.rename({
    "XAXS": "longitude",
    "YAXS": "latitude",
    "ZAXS": "lev",
    "TAXIS1D": "time"
})
# print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
s_radar = 0.01 * dsr.S_RADAR.isel(lev=0, drop=True)
sizes = u_radar.sizes
ln = sizes["time"]

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"../guv/tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

corrs_u = []
biass_u = []
rmses_u = []

corrs_v = []
biass_v = []
rmses_v = []

ln = len(df)
for idx, row in df_all.iterrows():
    sat1 = row["sat"]
    track_number_self = str(row["track_self"])
    track_number_other = str(row["track_other"])
    lon1 = row["lons_inter"]
    lat1 = row["lats_inter"]
    lons_inter1 = row["lons_inter"]
    lats_inter1 = row["lats_inter"]
    x_from_coast_self1 = row["x_from_coast_self"]
    x_from_coast_other1 = row["x_from_coast_other"]
    angle_acute = row["angle_acute"]
    angle_obtuse = row["angle_obtuse"]
    # check if radar region contains intersection
    if not polygon.contains(Point(lon1, lat1)):
```

```
        corrs_u.append(np.nan)
        rmses_u.append(np.nan)
        biass_u.append(np.nan)
        corrs_v.append(np.nan)
        rmses_v.append(np.nan)
        biass_v.append(np.nan)
        continue
    if abs(lat1) < 2:
        corrs_u.append(np.nan)
        rmses_u.append(np.nan)
        biass_u.append(np.nan)
        corrs_v.append(np.nan)
        rmses_v.append(np.nan)
        biass_v.append(np.nan)
        continue
    # print(
    #     sat1,
    #     track_number_self,
    #     track_number_other,
    #     lons_inter1,
    #     lats_inter1,
    #     x_from_coast_self1,
    #     x_from_coast_other1,
    #     angle_acute,
    #     angle_obtuse,
    # )
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    overlap_tsta, overlap_tend = overlap_dates(track_tsta_o, track_tend_o,
                                               radar_tsta_o, radar_tend_o)
    # ru_cut = u_radar.sel(time=slice(overlap_tsta, overlap_tend))
    # ru_interp = ru_cut.interp(time=gu_cut.time)
    ru_at = u_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    rv_at = v_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    f_oscar = f"{oscar_extracted_dir}/oscar_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc"
    ds_oscar = xr.open_dataset(f_oscar)
    oscar_u = ds_oscar.u
    oscar_v = ds_oscar.v
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_self}_loess_0.2.nc"
    f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_other}_loess_0.2.nc"
    ds_self = xr.open_dataset(f_self, engine="h5netcdf", decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, engine="h5netcdf", decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)
    gu, gv = geostrophic_components_from_a(gc_self_at, gc_other_at, a1, a2)
    fig = plt.figure(figsize=(12, 10))
    ax0 = fig.add_subplot(3, 1, 2)
    ax1 = fig.add_subplot(3, 1, 3)

    gu_cut = -1 * gu
    gv_cut = gv
    tfreq = sats_tfreq[sat1]

    oscar_u = oscar_u.resample(time=f"{tfreq}D").mean()
    ru_at = ru_at.resample(time=f"{tfreq}D").mean()
    oscar_u = oscar_u.interp(time=gu_cut.time)
    ru_at = ru_at.interp(time=gu_cut.time)
    oscar_max = oscar_u.max().item()
    gu_cut = gu_cut.where(gu_cut < oscar_max, np.nan)
    gu_cut = gu_cut.where(gu_cut > -oscar_max, np.nan)

    oscar_v = oscar_v.resample(time=f"{tfreq}D").mean()
    rv_at = rv_at.resample(time=f"{tfreq}D").mean()
    oscar_v = oscar_v.interp(time=gv_cut.time)
    rv_at = rv_at.interp(time=gu_cut.time)
    oscar_max = oscar_v.max().item()
    gv_cut = gv_cut.where(gv_cut < oscar_max, np.nan)
    gv_cut = gv_cut.where(gv_cut > -oscar_max, np.nan)

    oscar_u.plot(ax=ax0, color="b", linewidth=2, label="oscar u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="k", linewidth=2)
    ru_at.plot(ax=ax0, label=f"radar zonal", color="r", linewidth=2)
    # ax0.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax0.legend()
    ax0.set_ylabel("oscar u m/s")

    oscar_v.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
    gv_cut.plot(ax=ax1, label=f"track meridional", color="k", linewidth=2)
    rv_at.plot(ax=ax1, label=f"radar meridional", color="r", linewidth=2)

    # corr_u = xs.pearson_r(gu_cut, oscar_u, dim="time", skipna=True)
```

```
    # rmse_u = xs.rmse(gu_cut, oscar_u, dim="time", skipna=True)
    # bias_var_u = oscar_u - gu_cut
    corr_u = xs.pearson_r(gu_cut, ru_at, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, ru_at, dim="time", skipna=True)
    bias_var_u = ru_at - gu_cut
    bias_u = bias_var_u.mean(dim="time")

    # corr_v = xs.pearson_r(gv_cut, oscar_v, dim="time", skipna=True)
    # rmse_v = xs.rmse(gv_cut, oscar_v, dim="time", skipna=True)
    # bias_var_v = oscar_v - gv_cut
    corr_v = xs.pearson_r(gv_cut, rv_at, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, rv_at, dim="time", skipna=True)
    bias_var_v = rv_at - gv_cut
    bias_v = bias_var_v.mean(dim="time")

    corrs_u.append(corr_u.item())
    rmses_u.append(rmse_u.item())
    biass_u.append(bias_u.item())

    corrs_v.append(corr_v.item())
    rmses_v.append(rmse_v.item())
    biass_v.append(bias_v.item())

    print(f"Correlation between gu and u_radar: {corr_u.item()}")
    print(f"Correlation between gv and v_radar: {corr_v.item()}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.05,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("oscar v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap2)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.add_geometries(
        [polygon],
        ccrs.PlateCarree(),
        facecolor="lightblue",
        edgecolor="black",
        alpha=0.2,
    )
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=4,
        markerfacecolor="red",
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"track_guvVSoscar_at_intersection_points/guv_timeseries_{sat1}_{track_number_self}_{track_number_other}.png",
    )
    plt.show()
    plt.close("all")

# add corrs, rmses, biass to dataframe df_out
# df_out["corrs"] = corrs_u
# df_out["rmses"] = rmses_u
# df_out["biass"] = biass_u

# df_out.to_csv(f"tracks_corrs_rmses_biass_{sat1}_u.csv", index=False)

# df_out["corrs"] = corrs_v
# df_out["rmses"] = rmses_v
# df_out["biass"] = biass_v

# df_out.to_csv(f"tracks_corrs_rmses_biass_{sat1}_v.csv", index=False)
dse.close()

```

```
./schema/
```

```
7 file(s)
```

```python
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import Arc
import sympy as sp

base1 = 2

x = sp.symbols('x')
a = sp.symbols('a', constant=True)

f = a**x
fa = f.subs(a, base1)
expon = sp.lambdify(x, fa)

# Find the differential
fa_dx = sp.diff(fa, x)
expon_dx = sp.lambdify(x, fa_dx)

x = np.linspace(0, 5, 500)
y = expon(x)

fig, ax = plt.subplots(figsize=(8, 8))
ax.plot(x, y, label="$y=2^x$")

# Point of tangency P
x0 = 2.5
y0 = expon(x0)

# Compute derivative at x0 (slope of tangent)
# m_tangent = np.log(2) * y0  # y' = ln(2) * 2^x
m_tangent = expon_dx(x0)

# Tangent line at P
x_tangent = np.linspace(x0 - 1, x0 + 1, 100)
y_tangent = m_tangent * (x_tangent - x0) + y0
ax.plot(x_tangent, y_tangent, label="Tangent at P", linestyle="--", color="blue")

# Mark point P
ax.plot(x0, y0, "ko")  # Plot point P as black dot
plt.text(x0 + 0.1, y0, " P", ha="left", va="bottom")

# Draw line T1 passing through P making angle 110 degrees with x-axis
theta_deg = 110
theta_rad = np.deg2rad(theta_deg)
m_T1 = np.tan(theta_rad)
x_T1 = np.linspace(x0 - 2, x0 + 2, 100)
y_T1 = m_T1 * (x_T1 - x0) + y0
ax.plot(x_T1, y_T1, label="$T_1$", color="red")

m_T1 = (y_T1[-1] - y_T1[0]) / (x_T1[-1] - x_T1[0])
m_T1_perp = -1 / m_T1
# m_T1_perp = 1.76
x_perp = x_T1
y_perp = m_T1_perp * (x_perp - x0) + y0
# plt.plot(x_perp, y_perp, "r--", linewidth=1)
# t1 = (y_perp[-1] - y_perp[0])/(x_perp[-1] - x_perp[0])
# t2 = (y_T1[-1] - y_T1[0])/(x_T1[-1] - x_T1[0])

ax.annotate(
    "",
    xy=(x_perp[0], y_perp[0]),
    xytext=(x_perp[-1], y_perp[-1]),
    arrowprops=dict(arrowstyle="<->", linestyle="--", color="r"),
)

# Mark the angle theta that T1 makes with the x-axis
arc_radius = 0.5
arc = Arc(
    (x0, y0),
    arc_radius,
    arc_radius,
    angle=0,
    theta1=0,
    theta2=theta_deg,
    edgecolor="black",
)
ax.add_patch(arc)

# Position for theta symbol
theta_label_x = x0 + arc_radius * np.cos(theta_rad / 2)
theta_label_y = y0 + arc_radius * np.sin(theta_rad / 2)
plt.text(theta_label_x, theta_label_y, r"$\theta$", fontsize=14)

# At point P, draw vertical arrow (u bar) pointing both sides
arrow_length = 1
ax.annotate(
    "",
    xy=(x0, y0 + arrow_length),
    xytext=(x0, y0 - arrow_length),
    arrowprops=dict(arrowstyle="<->", color="black"),
)
plt.text(x0 + 0.1, y0 + arrow_length + 0.1, r"$\vec{u}$", fontsize=12)

# At point P, draw horizontal arrow (v bar) pointing both sides
ax.annotate(
    "",
    xy=(x0 - arrow_length, y0),
    xytext=(x0 + arrow_length, y0),
    arrowprops=dict(arrowstyle="<->", color="black"),
)
plt.text(x0 + arrow_length + 0.1, y0 + 0.1, r"$\vec{v}$", fontsize=12)

# plt.xlim(x0 - 2, x0 + 2)
# plt.xlim(1, 4)
ax.set_ylim(y0 - 3, y0 + 3)
ax.set_aspect('equal', adjustable='box')


ax.set_xlabel("")
ax.set_ylabel("")
# plt.legend()
plt.grid(True)
# plt.title("Plot of $y=2^x$ with Tangent at P and Line $T_1$ at $110^\circ$")

plt.savefig("p.schematic.coastal_slope.png", bbox_inches="tight", dpi=300)
plt.show()

```

```python
import matplotlib.pyplot as plt
import numpy as np

# Create a figure and axis
fig, ax = plt.subplots(figsize=(8, 8))

# Parameters for the parallel line
y_parallel = 5  # Height above x-axis
x_width = 10  # Width of the parallel line

# Draw the parallel thin line to the x-axis
ax.plot(
    [-x_width / 2, x_width / 2],
    [y_parallel, y_parallel],
    "b-",
    lw=1,
    label="Parallel Line",
)

# Define the traversing line with an angle of 110 degrees
angle = 77  # Angle in degrees (counterclockwise from x-axis)
slope = np.tan(np.radians(angle))  # Slope of the line
x_traverse = np.array([-6, 6])  # X-range of the traversing line
y_traverse = slope * x_traverse  # Y-values for the traversing line

# Plot the traversing line
ax.plot(x_traverse, y_traverse, "r-", lw=1, label="Traversing Line")

# Calculate intersection point P
x_intersect = y_parallel / slope
y_intersect = y_parallel

# Plot and annotate the intersection point P
ax.plot(x_intersect, y_intersect, "ko", markersize=4)  # Mark intersection point
ax.text(x_intersect - 0.3, y_intersect - 0.3, "P", fontsize=14)

# Label the angle θ near the x-axis
ax.text(-0.2, -1.8, r"$\alpha$", fontsize=14)

# Draw vertical and horizontal arrows at the intersection point (P)
# Vertical arrow (u-bar)
ax.arrow(
    x_intersect,
    y_intersect,
    0,
    2,
    head_width=0.2,
    head_length=0.4,
    fc="g",
    ec="g",
)
ax.arrow(
    x_intersect,
    y_intersect,
    0,
    -2,
    head_width=0.2,
    head_length=0.4,
    fc="g",
    ec="g",
)
ax.text(
    x_intersect + 0.2,
    y_intersect + 2,
    r"$\bar{v}$",
    fontsize=14,
    color="green",
)
ax.text(
    x_intersect + 0.2,
    y_intersect - 2,
    r"$-\bar{v}$",
    fontsize=14,
    color="green",
)

# Horizontal arrow (v-bar)
ax.arrow(
    x_intersect,
    y_intersect,
    2,
    0,
    head_width=0.2,
    head_length=0.4,
    fc="g",
    ec="g",
)
ax.arrow(
    x_intersect,
    y_intersect,
    -2,
    0,
    head_width=0.2,
    head_length=0.4,
    fc="g",
    ec="g",
)
ax.text(
    x_intersect + 2,
    y_intersect + 0.2,
    r"$\bar{u}$",
    fontsize=14,
    color="green",
)
ax.text(
    x_intersect - 2.4,
    y_intersect + 0.2,
    r"$-\bar{u}$",
    fontsize=14,
    color="green",
)

# Compute the perpendicular slope (negative reciprocal of the traversing line's slope)
perpendicular_slope = -1 / slope

# Define the x-range for the perpendicular line (centered at P)
length = 6  # Length of the dotted perpendicular line in both directions
```

```python
x_perpendicular = np.array([x_intersect - length / 2, x_intersect + length / 2])

# Compute the corresponding y-values for the perpendicular line
y_perpendicular = perpendicular_slope * (x_perpendicular - x_intersect) + y_intersect

# Draw the thin, dotted perpendicular line with arrows
ax.plot(x_perpendicular, y_perpendicular, "k--", lw=1)  # Dotted line

# Calculate direction vectors for arrow alignment
dx = (x_perpendicular[1] - x_perpendicular[0]) / 10  # Small increment for arrow length
dy = (y_perpendicular[1] - y_perpendicular[0]) / 10  # Small increment for arrow length

# Add arrows along the perpendicular line pointing correctly in both directions
ax.arrow(
    x_perpendicular[0],
    y_perpendicular[0],
    -dx,
    -dy,
    head_width=0.2,
    head_length=0.2,
    fc="k",
    ec="k",
)

ax.arrow(
    x_perpendicular[1],
    y_perpendicular[1],
    dx,
    dy,
    head_width=0.2,
    head_length=0.2,
    fc="k",
    ec="k",
)

ax.text(
    x_perpendicular[1] + 0.5,
    y_perpendicular[1] + 0.0,
    r"$\bar{g}$",
    fontsize=14,
    color="k",
)
ax.text(
    x_perpendicular[0] - 0.5,
    y_perpendicular[0] - 0.5,
    r"$-\bar{g}$",
    fontsize=14,
    color="k",
)
# Set axis limits and aspect ratio
ax.set_xlim(-6, 7)
ax.set_ylim(-2, 10)
ax.set_aspect("equal", "box")

# Add grid, legend, and labels
ax.grid(True, linestyle="--", alpha=0.7)
ax.legend()
ax.set_xlabel("X-axis")
ax.set_ylabel("Y-axis")
plt.savefig("p.schematic.matplotlib.lines.acute.png", bbox_inches="tight", dpi=300)

# Display the plot
plt.show()

```

```python
import matplotlib.pyplot as plt
import numpy as np

# Create a figure and axis
fig, ax = plt.subplots(figsize=(8, 8))

# Define angles in degrees and calculate slopes for the two lines
angle1 = 70  # Angle of the first line with the x-axis
angle2 = 110  # Angle of the second line with the x-axis

slope1 = np.tan(np.radians(angle1))  # Slope of the first line
slope2 = np.tan(np.radians(angle2))  # Slope of the second line

# Define starting points for the two lines on the x-axis
x_start1, y_start1 = -2, 0  # Start of Line 1 on the x-axis
x_start2, y_start2 = 2, 0  # Start of Line 2 on the x-axis

# Define the x-range for both lines (extending from their respective starting points)
x_range1 = np.linspace(x_start1, x_start1 + 6, 100)
x_range2 = np.linspace(x_start2, x_start2 - 6, 100)

# Calculate the y-values for both lines using their slopes
y_line1 = slope1 * (x_range1 - x_start1) + y_start1
y_line2 = slope2 * (x_range2 - x_start2) + y_start2

# Plot the two lines
(line1, ) = ax.plot(x_range1, y_line1, "r-", lw=1, label=f"Line 1 (acute)")
(line2, ) = ax.plot(x_range2, y_line2, "b-", lw=1, label=f"Line 2 (obtuse)")

# Find the intersection point of the two lines
# Solve y_line1 = y_line2 to find the intersection (numerically)
A = np.array([[slope1, -1], [slope2, -1]])
b = np.array([slope1 * x_start1, slope2 * x_start2])
x_intersect, y_intersect = np.linalg.solve(A, b)

# Mark and label the intersection point P
ax.plot(x_intersect, y_intersect, "ko",
        markersize=4)  # Mark intersection point
ax.text(x_intersect + 0.2, y_intersect + 0.2, "P", fontsize=14)

# Draw vertical and horizontal arrows at the intersection point P
# Vertical arrow (v-bar)
ax.arrow(
    x_intersect,
    y_intersect,
    0,
    2,
    head_width=0.2,
    head_length=0.4,
    fc="g",
    ec="g",
)
ax.arrow(
    x_intersect,
    y_intersect,
    0,
    -2,
    head_width=0.2,
    head_length=0.4,
    fc="g",
    ec="g",
)
ax.text(
    x_intersect - 0.1,
    y_intersect + 2.5,
    r"$\bar{v}$",
    fontsize=14,
    color="green",
)

# Horizontal arrow (u-bar)
ax.arrow(
    x_intersect,
    y_intersect,
    2,
    0,
    head_width=0.2,
    head_length=0.4,
    fc="g",
    ec="g",
)
ax.arrow(
    x_intersect,
    y_intersect,
    -2,
    0,
    head_width=0.2,
    head_length=0.4,
    fc="g",
    ec="g",
)
ax.text(
    x_intersect + 2.5,
    y_intersect - 0.1,
    r"$\bar{u}$",
    fontsize=14,
    color="green",
)

# Draw perpendicular dotted arrows to each line at the intersection point P
# Perpendicular slope for Line 1 (negative reciprocal of slope1)
perp_slope1 = -1 / slope1
x_perp1 = np.linspace(x_intersect - 3, x_intersect + 3, 100)
y_perp1 = perp_slope1 * (x_perp1 - x_intersect) + y_intersect

# Plot the perpendicular line to Line 1 with the same color (Red)
ax.plot(x_perp1, y_perp1, "r--",
        lw=1)  # Dotted line for perpendicular to Line 1

dx = (x_perp1[1] - x_perp1[0]) / 10  # Small increment for arrow length
dy = (y_perp1[1] - y_perp1[0]) / 10  # Small increment for arrow length

# Add arrows along the perpendicular line pointing correctly in both directions
ax.arrow(x_perp1[0], y_perp1[0], -dx, -dy,
         head_width=0.2, head_length=0.2, fc='r', ec='r')

ax.arrow(x_perp1[-1], y_perp1[-1], dx, dy,
```

```
        head_width=0.2, head_length=0.2, fc='r', ec='r')
ax.text(
    x_perp1[-1] + 0.5,
    y_perp1[-1] + 0.0,
    r"$\bar{g}$",
    fontsize=14,
    color="r",
)
ax.text(
    x_perp1[0] - 0.5,
    y_perp1[0] - 0.5,
    r"$-\bar{g}$",
    fontsize=14,
    color="r",
)
# Perpendicular slope for Line 2 (negative reciprocal of slope2)
perp_slope2 = -1 / slope2
x_perp2 = np.linspace(x_intersect - 3, x_intersect + 3, 100)
y_perp2 = perp_slope2 * (x_perp2 - x_intersect) + y_intersect

# Plot the perpendicular line to Line 2 with the same color (Blue)
ax.plot(x_perp2, y_perp2, "b--",
        lw=1)  # Dotted line for perpendicular to Line 2

dx = (x_perp2[1] - x_perp2[0]) / 10  # Small increment for arrow length
dy = (y_perp2[1] - y_perp2[0]) / 10  # Small increment for arrow length

# Add arrows along the perpendicular line pointing correctly in both directions
ax.arrow(x_perp2[0], y_perp2[0], -dx, -dy,
         head_width=0.2, head_length=0.2, fc='b', ec='b')

ax.arrow(x_perp2[-1], y_perp2[-1], dx, dy,
         head_width=0.2, head_length=0.2, fc='b', ec='b')

ax.text(
    x_perp2[-1] + 0.5,
    y_perp2[-1] + 0.0,
    r"$\bar{g}$",
    fontsize=14,
    color="b",
)
ax.text(
    x_perp2[0] - 0.5,
    y_perp2[0] - 0.5,
    r"$-\bar{g}$",
    fontsize=14,
    color="b",
)
# Set axis limits and aspect ratio
ax.set_xlim(-6, 6)
ax.set_ylim(0, 12)
ax.set_aspect("equal", "box")

# Add grid, legend, and labels
ax.grid(True, linestyle="--", alpha=0.7)
ax.legend(loc="upper right")
ax.set_xlabel("X-axis")
ax.set_ylabel("Y-axis")

plt.savefig("p.schematic.matplotlib_lines_g1_g2.png", bbox_inches="tight")
# Display the plot
plt.show()

```

```python
import matplotlib.pyplot as plt
import numpy as np

# Create a figure and axis
fig, ax = plt.subplots(figsize=(8, 8))

# Parameters for the parallel line
y_parallel = 5  # Height above x-axis
x_width = 10  # Width of the parallel line

# Draw the parallel thin line to the x-axis
ax.plot(
    [-x_width / 2, x_width / 2],
    [y_parallel, y_parallel],
    "b-",
    lw=1,
    label="Parallel Line",
)

# Define the traversing line with an angle of 110 degrees
angle = 110  # Angle in degrees (counterclockwise from x-axis)
slope = np.tan(np.radians(angle))  # Slope of the line
x_traverse = np.array([-6, 6])  # X-range of the traversing line
y_traverse = slope * x_traverse  # Y-values for the traversing line

# Plot the traversing line
ax.plot(x_traverse, y_traverse, "r-", lw=1, label="Traversing Line")

# Calculate intersection point P
x_intersect = y_parallel / slope
y_intersect = y_parallel

# Plot and annotate the intersection point P
ax.plot(x_intersect, y_intersect, "ko", markersize=4)  # Mark intersection point
ax.text(x_intersect - 0.4, y_intersect - 0.4, "P", fontsize=14)

# Label the angle θ near the x-axis
# ax.text(0.0, -1.8, r"$\alpha$", fontsize=14)
ax.text(0.8, -1.9, r"$\beta$", fontsize=14)

# Draw vertical and horizontal arrows at the intersection point (P)
# Vertical arrow (u-bar)
ax.arrow(
    x_intersect,
    y_intersect,
    0,
    2,
    head_width=0.2,
    head_length=0.4,
    fc="g",
    ec="g",
)
ax.arrow(
    x_intersect,
    y_intersect,
    0,
    -2,
    head_width=0.2,
    head_length=0.4,
    fc="g",
    ec="g",
)
ax.text(
    x_intersect + 0.2,
    y_intersect + 2,
    r"$\bar{v}$",
    fontsize=14,
    color="green",
)
ax.text(
    x_intersect + 0.2,
    y_intersect - 2,
    r"$-\bar{v}$",
    fontsize=14,
    color="green",
)

# Horizontal arrow (v-bar)
ax.arrow(
    x_intersect,
    y_intersect,
    2,
    0,
    head_width=0.2,
    head_length=0.4,
    fc="g",
    ec="g",
)
ax.arrow(
    x_intersect,
    y_intersect,
    -2,
    0,
    head_width=0.2,
    head_length=0.4,
    fc="g",
    ec="g",
)
ax.text(
    x_intersect + 2,
    y_intersect + 0.2,
    r"$\bar{u}$",
    fontsize=14,
    color="green",
)
ax.text(
    x_intersect - 2.4,
    y_intersect + 0.2,
    r"$-\bar{u}$",
    fontsize=14,
    color="green",
)

# Compute the perpendicular slope (negative reciprocal of the traversing line's slope)
perpendicular_slope = -1 / slope

# Define the x-range for the perpendicular line (centered at P)
```

```python
length = 6  # Length of the dotted perpendicular line in both directions
x_perpendicular = np.array([x_intersect - length / 2, x_intersect + length / 2])

# Compute the corresponding y-values for the perpendicular line
y_perpendicular = perpendicular_slope * (x_perpendicular - x_intersect) + y_intersect

# Draw the thin, dotted perpendicular line with arrows
ax.plot(x_perpendicular, y_perpendicular, 'k--', lw=1)  # Dotted line

# Calculate direction vectors for arrow alignment
dx = (x_perpendicular[1] - x_perpendicular[0]) / 10  # Small increment for arrow length
dy = (y_perpendicular[1] - y_perpendicular[0]) / 10  # Small increment for arrow length

# Add arrows along the perpendicular line pointing correctly in both directions
ax.arrow(x_perpendicular[0], y_perpendicular[0], -dx, -dy,
         head_width=0.2, head_length=0.2, fc='k', ec='k')

ax.arrow(x_perpendicular[1], y_perpendicular[1], dx, dy,
         head_width=0.2, head_length=0.2, fc='k', ec='k')

ax.text(
    x_perpendicular[1] + 0.5,
    y_perpendicular[1] + 0.5,
    r"$\bar{g}$",
    fontsize=14,
    color="k",
)
ax.text(
    x_perpendicular[0] - 0.5,
    y_perpendicular[0] - 0.5,
    r"$-\bar{g}$",
    fontsize=14,
    color="k",
)
# Set axis limits and aspect ratio
ax.set_xlim(-8, 8)
ax.set_ylim(-2, 10)
ax.set_aspect('equal', 'box')

# Add grid, legend, and labels
ax.grid(True, linestyle='--', alpha=0.7)
ax.legend()
ax.set_xlabel('X-axis')
ax.set_ylabel('Y-axis')
plt.savefig('p.schematic.matplotlib.lines.obtuse.png', bbox_inches='tight', dpi=300)

# Display the plot
plt.show()

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
from matplotlib.lines import Line2D
from tools_xtrackm import *

cmap1 = cmo.cm.topo
cmap1 = cmo.cm.diff

NIO = (60.0, 100.0, -5.0, 27.0)

width, height = 14, 9


def is_odd(track_no):
    if int(track_no) % 2 == 1:
        return True
    else:
        return False


def is_even(track_no):
    if int(track_no) % 2 == 0:
        return True
    else:
        return False


def orbit_type(track_no):
    if int(track_no) % 2 == 1:
        return "ascending"
    else:
        return "descending"


def perpendicular_line(lonm, latm, m):
    m2 = -1 / m
    x_values = np.linspace(lonm - 5, lonm + 5, 100)
    y_values = m2 * (x_values - lonm) + latm
    return x_values, y_values


def horizontal_line(lonm, latm):
    x_values = np.linspace(lonm - 5, lonm + 5, 100)
    y_values = np.full_like(x_values, latm)
    return x_values, y_values


def vertical_line(lonm, latm):
    y_values = np.linspace(latm - 5, latm + 5, 100)
    x_values = np.full_like(y_values, lonm)
    return x_values, y_values


def point_at_distance(lonm, latm, m, d):
    denom = np.sqrt(1 + m**2)
    lonm1 = lonm - d / denom
    latm1 = latm + m * (d / denom)
    return lonm1, latm1


custom_lines = [
    Line2D([0], [0], color="blue", lw=2, linestyle="dashed",
           label="Ascending"),
    Line2D([0], [0],
           color="green",
           lw=2,
           linestyle="dashed",
           label="descending"),
]

dse = xr.open_dataset(
    "/home/srinivasu/allData/topo/etopo5.cdf")  # open etopo dataset

for sat in sats_new:
    oi = 0
    ei = 0
    fig, ax1 = plt.subplots(
        1,
        1,
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(width, height),
        layout="constrained",
    )
    dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(
        *NIO[2:])).plot(ax=ax1,
                        add_colorbar=False,
                        add_labels=False,
                        cmap=cmap1)
    decorate_axis(ax1, "", *NIO)
    for f in sorted(glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")):
        ds = xr.open_dataset(f, decode_times=False)
        track_number = ds.Pass
        orbit_str = orbit_type(track_number)
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) < 12:
            continue
        lon_equat = lons_track[0]
        lat_equat = lats_track[0]
        lon_coast = lons_track[-1]
        lat_coast = lats_track[-1]
        m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
        # angle_radian = np.arctan(m)
        # angle_radian = np.arctan2(lats_track[-1] - lats_track[0],
        #                           lons_track[-1] - lons_track[0])
        # angle_radian = np.arctan2(lat_equat - lat_coast, lon_equat - lon_coast)
        angle_radian = np.arctan2(lat_coast - lat_equat, lon_coast - lon_equat)
        angle = np.rad2deg(angle_radian)
        angle_str = str(round(angle, 2))
        lonm = lons_track.mean()
        latm = lats_track.mean()
        xp, yp = perpendicular_line(lonm, latm, m)
        xh, yh = horizontal_line(lonm, latm)
        xy, yy = vertical_line(lonm, latm)
        d = 0.5
```

```
        lonm1, latm1 = point_at_distance(lonm, latm, m, d)
        if oi > 0 and ei > 0:
            break
        if is_odd(track_number) and oi == 0:
            print(sat, angle, orbit_str, "----------------------------")
            oi += 1
            ax1.scatter(lons_track, lats_track, linewidths=0.0, s=2, color="b")
            ax1.scatter(xp, yp, linewidths=0.0, s=2, color="b")
            ax1.scatter(xh,
                        yh,
                        linewidths=0.0,
                        s=1,
                        color="w",
                        linestyle="dashed")
            ax1.scatter(xy,
                        yy,
                        linewidths=0.0,
                        s=1,
                        color="w",
                        linestyle="dashed")
            if is_within_region(lonm, latm, *NIO):
                plt.text(
                    lonm1,
                    latm1,
                    s=track_number,
                    fontsize=14,
                    rotation=angle,
                    fontweight="bold",
                    color="#800000",
                    horizontalalignment="center",
                    verticalalignment="center",
                )
                plt.text(
                    lon_equat,
                    lat_equat,
                    s="(" + angle_str + ")",
                    fontsize=14,
                    rotation=angle,
                    fontweight="bold",
                    color="#800000",
                    horizontalalignment="left",
                    verticalalignment="top",
                )
        if is_even(track_number) and ei == 0:
            ei += 1
            print(sat, angle, orbit_str, "----------------------------")
            ax1.scatter(lons_track, lats_track, linewidths=0.0, s=2, color="g")
            ax1.scatter(xp, yp, linewidths=0.0, s=2, color="g")
            ax1.scatter(xh,
                        yh,
                        linewidths=0.0,
                        s=1,
                        color="w",
                        linestyle="dashed")
            ax1.scatter(xy,
                        yy,
                        linewidths=0.0,
                        s=1,
                        color="w",
                        linestyle="dashed")
            try:
                lonm = lons_track[-15]
                latm = lats_track[-15]
            except:
                lonm = 0
                latm = 0
            if is_within_region(lonm, latm, *NIO):
                plt.text(
                    lonm1,
                    latm1,
                    s=track_number,
                    fontsize=14,
                    rotation=angle,
                    fontweight="bold",
                    color="g",
                    horizontalalignment="left",
                    verticalalignment="top",
                )
                plt.text(
                    lon_equat,
                    lat_equat,
                    s="(" + angle_str + ")",
                    fontsize=14,
                    rotation=angle,
                    fontweight="bold",
                    color="g",
                    horizontalalignment="center",
                    verticalalignment="center",
                )
    info = f"{sat}"
    ax1.text(0.4,
             0.9,
             info,
             transform=ax1.transAxes,
             fontsize=30,
             fontweight="bold")
    plt.legend(handles=custom_lines, loc="upper right")
    plt.savefig(
        f"p.track.angles.{sat}.atan2.png",
        bbox_inches="tight",
    )
    plt.show()

```

```python
import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import xarray as xr
from matplotlib.lines import Line2D
from tools_xtrackm import *

cmap1 = cmo.cm.topo
cmap1 = cmo.cm.diff

IO = (30.0, 120.0, -30.0, 30.0)
NIO = (60.0, 100.0, -5.0, 27.0)

width, height = 14, 9


def is_odd(track_no):
    if int(track_no) % 2 == 1:
        return True
    else:
        return False


def perpendicular_line(lonm, latm, m):
    m2 = -1 / m
    x_values = np.linspace(lonm - 5, lonm + 5, 100)
    y_values = m2 * (x_values - lonm) + latm
    return x_values, y_values


def horizontal_line(lonm, latm):
    x_values = np.linspace(lonm - 5, lonm + 5, 100)
    y_values = np.full_like(x_values, latm)
    return x_values, y_values


def vertical_line(lonm, latm):
    y_values = np.linspace(latm - 5, latm + 5, 100)
    x_values = np.full_like(y_values, lonm)
    return x_values, y_values


def point_at_distance(lonm, latm, m, d):
    # https://www.geeksforgeeks.org/find-points-at-a-given-distance-on-a-line-of-given-slope/
    # m is the slope of the line
    # lonm and latm are the coordinates of the point on the line
    # d is the distance from the point to a point above the line
    denom = np.sqrt(1 + m**2)
    lonm1 = lonm - d / denom
    latm1 = latm + m * (d / denom)
    return lonm1, latm1


dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")  # open etopo dataset

# sats = ["GFO"]
for sat in sats_new:
    ODD = False
    EVEN = False
    fig, ax1 = plt.subplots(
        1,
        1,
        subplot_kw={"projection": ccrs.PlateCarree()},
        figsize=(width, height),
        layout="constrained",
    )
    dse.ROSE.sel(ETOPO05_X=slice(*NIO[:2])).sel(ETOPO05_Y=slice(*NIO[2:])).plot(
        ax=ax1, add_colorbar=False, add_labels=False, cmap=cmap1
    )
    decorate_axis(ax1, "", *NIO)
    for f in sorted(glob.glob(f"../data/{sat}/ctoh.sla.ref.{sat}.nindian.*.nc")):
        ds = xr.open_dataset(f, decode_times=False)
        print(ds)
        track_number = ds.Pass
        track_number = int(track_number)
        lons_track = ds.lon.values
        lats_track = ds.lat.values
        if len(lons_track) == 0:
            continue
        lon_eq = lons_track[0]
        lat_eq = lats_track[0]
        m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
        angle_radian = np.arctan(m)
        angle_radian = np.arctan2(
            lats_track[-1] - lats_track[0], lons_track[-1] - lons_track[0]
        )
        angle = np.rad2deg(angle_radian)
        angle_str = str(round(angle, 2))
        print(angle, angle_radian, "-----------------------------")
        lonm = lons_track.mean()
        latm = lats_track.mean()
        xp, yp = perpendicular_line(lonm, latm, m)
        xh, yh = horizontal_line(lonm, latm)
        xy, yy = vertical_line(lonm, latm)
        # Horizontal and vertical arrow parameters
        x_start, x_end = lonm, lonm + 5
        y_start, y_end = latm, latm + 5
        # get the coordinates of a point at a distance d from the point (lonm, latm)
        # above the line with slope m
        d = 0.5
        lonm1, latm1 = point_at_distance(lonm, latm, m, d)
        if is_odd(track_number):
            ODD = True
            ax1.scatter(lons_track, lats_track, linewidths=0.0, s=2, color="b")
            ax1.scatter(xp, yp, linewidths=0.0, s=2, color="b")
            ax1.annotate(
                "",
                xy=(x_end, 0),
                xytext=(x_start, 0),
                arrowprops=dict(arrowstyle="->", color="blue", lw=1),
                transform=ccrs.PlateCarree(),  # Ensure projection compatibility
            )
            ax1.annotate(
                "",
                xy=(0, y_end),
                xytext=(0, y_start),
                arrowprops=dict(arrowstyle="->", color="red", lw=1),
```

```
                transform=ccrs.PlateCarree(),  # Ensure projection compatibility
            )
            if is_within_region(lonm, latm, *NIO):
                plt.text(
                    lonm1,
                    latm1,
                    s=track_number,
                    fontsize=14,
                    rotation=angle,
                    fontweight="bold",
                    color="#800000",
                    horizontalalignment="center",
                    verticalalignment="center",
                )
                plt.text(
                    lon_eq,
                    lat_eq,
                    s="(" + angle_str + ")",
                    fontsize=14,
                    rotation=angle,
                    fontweight="bold",
                    color="#800000",
                    horizontalalignment="left",
                    verticalalignment="top",
                )
        else:
            EVEN = True
            ax1.scatter(lons_track, lats_track, linewidths=0.0, s=2, color="g")
            ax1.scatter(xp, yp, linewidths=0.0, s=2, color="g")
            ax1.annotate(
                "",
                xy=(x_end, 0),
                xytext=(x_start, 0),
                arrowprops=dict(arrowstyle="->", color="blue", lw=1),
                transform=ccrs.PlateCarree(),  # Ensure projection compatibility
            )
            ax1.annotate(
                "",
                xy=(0, y_end),
                xytext=(0, y_start),
                arrowprops=dict(arrowstyle="->", color="red", lw=1),
                transform=ccrs.PlateCarree(),  # Ensure projection compatibility
            )
            try:
                lonm = lons_track[-15]
                latm = lats_track[-15]
            except:
                lonm = 0
                latm = 0
            if is_within_region(lonm, latm, *NIO):
                plt.text(
                    lonm1,
                    latm1,
                    s=track_number,
                    fontsize=14,
                    rotation=angle,
                    fontweight="bold",
                    color="g",
                    horizontalalignment="left",
                    verticalalignment="top",
                )
                plt.text(
                    lon_eq,
                    lat_eq,
                    s="(" + angle_str + ")",
                    fontsize=14,
                    rotation=angle,
                    fontweight="bold",
                    color="g",
                    horizontalalignment="center",
                    verticalalignment="center",
                )
        if ODD and EVEN:
            break
    info = f"{sat}"
    ax1.text(0.4, 0.9, info, transform=ax1.transAxes, fontsize=30, fontweight="bold")

    # Create a custom legend with specific markers and colors
    custom_lines = [
        Line2D([0], [0], color="blue", lw=2, linestyle="dashed", label="Ascending"),
        Line2D([0], [0], color="green", lw=2, linestyle="dashed", label="descending"),
    ]

    plt.legend(handles=custom_lines, loc="upper right")
    plt.savefig(
        f"p.track.angles.{sat}.png",
        bbox_inches="tight",
    )
    plt.show()
    break

```

```
1
2  ground track of a satellite. If α (an acute angle) is the angle made by the
3  satellite track with positive x-axis counter clock wise, and the dashed line
4  (gg) is perpendicular to the ground track at the intersection point P, then
5  solve for the angle between uP+g.
6
7  -uu is parallel to x-axis and -vv is parallel to y-axis. The slant line is
8  ground track of a satellite. The dashed line (gg) is perpendicular to the
9  ground track at the intersection point P. If β (an obtuse angle) is the angle
10 made by the satellite track with positive x-axis counter clock wise, then solve
11 for the angle between uP+g.
12
```

```
./selected/
```

```
11 file(s)
```

selected/f.ngp.APCodar_daily.py

```python
import cmaps
import cmocean as cmo
import matplotlib.pyplot as plt
import xarray as xr

cmap1 = cmo.cm.topo
cmap2 = cmo.cm.thermal
cmap1 = cmo.cm.topo
cmap3 = cmaps.BlAqGrYeOrReVi200
cmap4 = cmo.cm.phase
cmap_rmse = "YlGn"

width, height = 10, 10
WBOB = (78.0, 87.0, 8.0, 21.0)
AP_RADAR = (80.5, 83.5, 14.2, 16.5)

dsr = xr.open_dataset("/home/srinivasu/allData/radar/APCodar_daily.nc")
print(dsr)
dsr = dsr.rename({
    "XAXS": "longitude",
    "YAXS": "latitude",
    "ZAXS": "lev",
    "TAXIS1D": "time"
})
print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
u_radar_m = u_radar.resample(time="1M").mean()
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
v_radar_m = v_radar.resample(time="1M").mean()
print(u_radar)

sizes = v_radar.sizes
ln = sizes["time"]
ngp_u = 100 * (v_radar.count(dim="time") / ln)
fig, ax = plt.subplots(1, 1, figsize=(6, 4), layout="constrained")
c = ngp_u.plot(ax=ax, cmap=cmap3, add_colorbar=False)
ax.text(
    0.8,
    0.8,
    "AP",
    transform=ax.transAxes,
    color="white",
    fontsize=20,
    ha="center",
    va="center",
)
cb = plt.colorbar(c)
cb.set_label("% of valid data points")
plt.savefig(f"APradar_valid_points.png")
plt.show()

```

selected/f.ngp.ORCodar_daily.py

```python
import cmaps
import cmocean as cmo
import matplotlib.pyplot as plt
import xarray as xr

cmap1 = cmo.cm.topo
cmap2 = cmo.cm.thermal
cmap1 = cmo.cm.topo
cmap3 = cmaps.BlAqGrYeOrReVi200
cmap4 = cmo.cm.phase
cmap_rmse = "YlGn"

width, height = 10, 10
WBOB = (78.0, 87.0, 8.0, 21.0)
OR_RADAR = (84.5, 87.0, 17.5, 20.0)

dsr = xr.open_dataset("/home/srinivasu/allData/radar/ORCodar_daily.nc")
print(dsr)
dsr = dsr.rename({
    "XAXS": "longitude",
    "YAXS": "latitude",
    "ZAXS": "lev",
    "TAXIS1D": "time"
})
print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
u_radar_m = u_radar.resample(time="1M").mean()
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
v_radar_m = v_radar.resample(time="1M").mean()
print(u_radar)

sizes = v_radar.sizes
ln = sizes["time"]
ngp_u = 100 * (v_radar.count(dim="time") / ln)
fig, ax = plt.subplots(1, 1, figsize=(6, 4), layout="constrained")
c = ngp_u.plot(ax=ax, cmap=cmap3, add_colorbar=False)
ax.text(
    0.8,
    0.8,
    "OR",
    transform=ax.transAxes,
    color="white",
    fontsize=20,
    ha="center",
    va="center",
)
cb = plt.colorbar(c)
cb.set_label("% of valid data points")
plt.savefig(f"ORradar_valid_points.png")
plt.show()

```

```python
import cmaps
import cmocean as cmo
import matplotlib.pyplot as plt
import xarray as xr

cmap1 = cmo.cm.topo
cmap2 = cmo.cm.thermal
cmap1 = cmo.cm.topo
cmap3 = cmaps.BlAqGrYeOrReVi200
cmap4 = cmo.cm.phase
cmap_rmse = "YlGn"

width, height = 10, 10
WBOB = (78.0, 87.0, 8.0, 21.0)
TN_RADAR = (79.6, 81.8, 10.5, 12.9)

dsr = xr.open_dataset("/home/srinivasu/allData/radar/TNCodar_daily.nc")
print(dsr)
dsr = dsr.rename({
    "XAXS": "longitude",
    "YAXS": "latitude",
    "ZAXS": "lev",
    "TAXIS1D": "time"
})
print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
u_radar_m = u_radar.resample(time="1M").mean()
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
v_radar_m = v_radar.resample(time="1M").mean()
print(u_radar)

sizes = v_radar.sizes
ln = sizes["time"]
ngp_u = 100 * (v_radar.count(dim="time") / ln)
fig, ax = plt.subplots(1, 1, figsize=(6, 4), layout="constrained")
c = ngp_u.plot(ax=ax, cmap=cmap3, add_colorbar=False)
cb = plt.colorbar(c)
cb.set_label("% of valid data points")
ax.text(
    0.8,
    0.8,
    "TN",
    transform=ax.transAxes,
    color="white",
    fontsize=20,
    ha="center",
    va="center",
)
plt.savefig(f"TNradar_valid_points.png")
plt.show()

```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

only_df = pd.read_csv(
    "oscar_onlyVStrack_corrs_rmses_biass_TP+J1+J2+J3+S6A_u.csv")
ekmn_df = pd.read_csv(
    "oscar_ekmnVStrack_corrs_rmses_biass_TP+J1+J2+J3+S6A_u.csv")

# Create unique column combining zero-padded sat and track_self, track_other
only_df["unique_id"] = (only_df["sat"].apply(lambda s: s.zfill(20)) + "_" +
                        only_df["track_self"].astype(str) + "_" +
                        only_df["track_other"].astype(str))
only_df = only_df.set_index("unique_id")

ekmn_df["unique_id"] = (ekmn_df["sat"].apply(lambda s: s.zfill(20)) + "_" +
                        ekmn_df["track_self"].astype(str) + "_" +
                        ekmn_df["track_other"].astype(str))
ekmn_df = ekmn_df.set_index("unique_id")

common_ids = only_df.index.intersection(ekmn_df.index)
only_df = only_df.loc[common_ids]
ekmn_df = ekmn_df.loc[common_ids]

# Extract metrics for plotting
metrics = ["biass", "corrs", "rmses"]
titles = ["Bias", "Correlation", "Rmse"]

# Prepare figure with 2x2 grid
fig, axs = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle("comparison metrics with oscar u: track gu only VS track gu + ekman", fontsize=14)

summary_lines = []

for i, metric in enumerate(metrics):
    ax = axs[i // 2, i % 2]
    valid_mask = only_df[metric].notna() & ekmn_df[metric].notna()
    only_df1 = only_df[valid_mask]
    ekmn_df1 = ekmn_df[valid_mask]
    # Get data
    only_df_data = only_df1[metric].values
    ekmn_df_data = ekmn_df1[metric].values
    indices = np.arange(len(only_df_data))
    ax.plot(indices, only_df_data, label="track gu only")
    ax.plot(indices, ekmn_df_data, label="track gu+ekman")
    # draw zero line
    ax.axhline(0, color="gray", linestyle="--")
    ax.set_xlabel("Data Point Index")
    ax.set_ylabel(titles[i])
    ax.set_title(titles[i])
    ax.legend()
    # collect summary text for this metric
    if len(only_df_data) > 0:
        mean_o = np.nanmean(only_df_data)
        mean_e = np.nanmean(ekmn_df_data)
        diff = mean_e - mean_o
        cnt = len(only_df_data)
        summary_lines.append(f"\n{titles[i]}:")
        summary_lines.append(f" count = {cnt}")
        summary_lines.append(f" mean track u = {mean_o:.4f}")
        summary_lines.append(f" mean track u + ekman = {mean_e:.4f}")
        summary_lines.append(f" mean improvement = {diff:.4f}")
    else:
        summary_lines.append(f"\n{titles[i]}: no valid paired data")

ax_sum = axs[1, 1]
ax_sum.axis("off")
text = " ".join(summary_lines)

ax_sum.annotate(
    "\n".join(summary_lines),
    xy=(0, 1),
    xycoords="axes fraction",
    va="top",
    ha="left",
    fontsize=10,
    family="monospace",
    bbox=dict(boxstyle="round,pad=0.5", facecolor="white", alpha=0.7),
    annotation_clip=False,
)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(f"track_guvVSoscar_ww_ekman_u.png")
plt.show()

```

```python
import math

import cmocean as cmo
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import xarray as xr
from tools_xtrackm import *
import xskillscore as xs
from matplotlib.colors import ListedColormap


def geostrophic_components_from_a(gc1, gc2, a1, a2):
    """
    Compute zonal and meridional geostrophic current components from
    cross-track velocities and track slopes of two intersecting satellite tracks.
    Parameters:
    -----------
    gc1 : float or array-like
        Cross-track geostrophic velocity from track 1 (m/s)
    gc2 : float or array-like
        Cross-track geostrophic velocity from track 2 (m/s)
    slope1 : float
        Slope of track 1 (dy/dx = rise/run)
    slope2 : float
        Slope of track 2 (dy/dx = rise/run)
    Returns:
    --------
    u : float or array-like
        Zonal (eastward) velocity component (m/s)
    v : float or array-like
        Meridional (northward) velocity component (m/s)
    Raises:
    -------
    ValueError: If tracks are parallel (same slope)
    """
    # Calculate intersection angle
    theta = a2 - a1
    sin_theta = math.sin(theta)
    # Additional check using sine of intersection angle
    if abs(sin_theta) < 1e-6:
        raise ValueError(
            f"Tracks are nearly parallel (intersection angle = {math.degrees(theta):.2f}°). "
            "Cannot resolve both velocity components.")
    # Calculate trigonometric values
    cos_a1 = math.cos(a1)
    cos_a2 = math.cos(a2)
    sin_a1 = math.sin(a1)
    sin_a2 = math.sin(a2)
    # Solve for velocity components using the derived formulas:
    # u = (gc1 * cos(a2) - gc2 * cos(a1)) / sin(a2 - a1)
    # v = (gc2 * sin(a1) - gc1 * sin(a2)) / sin(a2 - a1)
    u = (gc1 * cos_a2 - gc2 * cos_a1) / sin_theta
    v = (gc2 * sin_a1 - gc1 * sin_a2) / sin_theta
    u = u * -1
    return u, v


cmap1 = cmo.cm.diff
cmap2 = ListedColormap(['white'])
d = 1.5
oscar_dir = "/home/srinivasu/allData/oscar_0.25/"
dse = xr.open_dataset("~/allData/topo/etopo5.cdf")  # open etopo dataset
oscar_extracted_dir = f"/home/srinivasu/xtrackm/guv/oscar_at_intersections/"

radar_str = "OR"
radar_str = "TN"
width, height = 13, 6
REG = radar_extents[radar_str + "_BLOCK"]
TRACKS_REG = (73, 99, 0, 26)

gdf = gpd.read_file(f"polygon_shapefile_around_{radar_str}.shp")
polygon = gdf.geometry.iloc[0]

radar_tsta_o, radar_tend_o = radar_times[radar_str]

dsr = xr.open_dataset(
    f"/home/srinivasu/allData/radar/{radar_str}Codar_daily.nc")
# print(dsr)
dsr = dsr.rename({
    "XAXS": "longitude",
    "YAXS": "latitude",
    "ZAXS": "lev",
    "TAXIS1D": "time"
})
# print(dsr)
u_radar = 0.01 * dsr.U_RADAR.isel(lev=0, drop=True)
v_radar = 0.01 * dsr.V_RADAR.isel(lev=0, drop=True)
s_radar = 0.01 * dsr.S_RADAR.isel(lev=0, drop=True)
sizes = u_radar.sizes
ln = sizes["time"]

df_all_list = []
for sat in sats_new:
    df = pd.read_csv(f"../guv/tracks_intersections_{sat}_1.csv")
    df_all_list.append(df)
df_all = pd.concat(df_all_list, ignore_index=True)

corrs_u = []
biass_u = []
rmses_u = []

corrs_v = []
biass_v = []
rmses_v = []

ln = len(df)
for idx, row in df_all.iterrows():
    sat1 = row["sat"]
    track_number_self = str(row["track_self"])
    track_number_other = str(row["track_other"])
    lon1 = row["lons_inter"]
    lat1 = row["lats_inter"]
    lons_inter1 = row["lons_inter"]
    lats_inter1 = row["lats_inter"]
    x_from_coast_self1 = row["x_from_coast_self"]
    x_from_coast_other1 = row["x_from_coast_other"]
```

```
    angle_acute = row["angle_acute"]
    angle_obtuse = row["angle_obtuse"]
    # check if radar region contains intersection
    if not polygon.contains(Point(lon1, lat1)):
        corrs_u.append(np.nan)
        rmses_u.append(np.nan)
        biass_u.append(np.nan)
        corrs_v.append(np.nan)
        rmses_v.append(np.nan)
        biass_v.append(np.nan)
        continue
    if abs(lat1) < 2:
        corrs_u.append(np.nan)
        rmses_u.append(np.nan)
        biass_u.append(np.nan)
        corrs_v.append(np.nan)
        rmses_v.append(np.nan)
        biass_v.append(np.nan)
        continue
    # print(
    #     sat1,
    #     track_number_self,
    #     track_number_other,
    #     lons_inter1,
    #     lats_inter1,
    #     x_from_coast_self1,
    #     x_from_coast_other1,
    #     angle_acute,
    #     angle_obtuse,
    # )
    track_tsta_o, track_tend_o = get_time_limits_o(sat1)
    overlap_tsta, overlap_tend = overlap_dates(track_tsta_o, track_tend_o,
                                               radar_tsta_o, radar_tend_o)
    # ru_cut = u_radar.sel(time=slice(overlap_tsta, overlap_tend))
    # ru_interp = ru_cut.interp(time=gu_cut.time)
    ru_at = u_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    rv_at = v_radar.sel(longitude=lon1, latitude=lat1, method="nearest", drop=True)
    f_oscar = f"{oscar_extracted_dir}/oscar_at_intersection_{sat1}_{track_number_self}_{track_number_other}.nc"
    ds_oscar = xr.open_dataset(f_oscar)
    oscar_u = ds_oscar.u
    oscar_v = ds_oscar.v
    f_self = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_self.zfill(3)}.nc"
    f_other = f"../data/{sat1}/ctoh.sla.ref.{sat1}.nindian.{track_number_other.zfill(3)}.nc"
    f_self_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_self}_loess_0.2.nc"
    f_other_smooth = f"../computed/sla_loess_0.2a/track_sla_along_{sat1}_{track_number_other}_loess_0.2.nc"
    ds_self = xr.open_dataset(f_self, decode_times=False)
    ds_self_smooth = xr.open_dataset(f_self_smooth)
    sla_self = track_dist_time_asn(ds_self, var_str="sla", units_in="m")
    sla_self_smooth = ds_self_smooth.sla_smooth
    ds_other = xr.open_dataset(f_other, decode_times=False)
    ds_other_smooth = xr.open_dataset(f_other_smooth)
    sla_other = track_dist_time_asn(ds_other, var_str="sla", units_in="m")
    sla_other_smooth = ds_other_smooth.sla_smooth
    lons_track_self = ds_self.lon.values
    lats_track_self = ds_self.lat.values
    lon_coast_self = lons_track_self[-1]  # this on coast
    lat_coast_self = lats_track_self[-1]  # this on coast
    lon_equat_self = lons_track_self[0]  # this on equator
    lat_equat_self = lats_track_self[0]  # this on equator
    slope_self = (lats_track_self[-1] - lats_track_self[0]) / (
        lons_track_self[-1] - lons_track_self[0])
    angle_self = np.rad2deg(np.arctan(slope_self))
    track_path_self = sg.LineString(zip(lons_track_self, lats_track_self))
    lons_track_other = ds_other.lon.values
    lats_track_other = ds_other.lat.values
    lon_equat_other = lons_track_other[0]
    lat_equat_other = lats_track_other[0]
    lon_coast_other = lons_track_other[-1]
    lat_coast_other = lats_track_other[-1]
    slope_other = (lat_coast_other - lat_equat_other) / (lon_coast_other -
                                                         lon_equat_other)
    angle_other = np.rad2deg(np.arctan(slope_other))
    track_path_other = sg.LineString(zip(lons_track_other, lats_track_other))
    gc_self = compute_geostrophy_gc(ds_self, sla_self_smooth)
    gc_other = compute_geostrophy_gc(ds_other, sla_other_smooth)
    gc_self_at = gc_self.sel(x=x_from_coast_self1, method="nearest", drop=True)
    gc_other_at = gc_other.sel(x=x_from_coast_other1,
                               method="nearest",
                               drop=True)
    gc_other_at = gc_other_at.interp(time=gc_self_at.time)
    a2 = math.atan2(lat_coast_other - lat_equat_other,
                    lon_coast_other - lon_equat_other)
    a1 = math.atan2(lat_coast_self - lat_equat_self,
                    lon_coast_self - lon_equat_self)
    gu, gv = geostrophic_components_from_a(gc_self_at, gc_other_at, a1, a2)
    fig = plt.figure(figsize=(12, 10))
    ax0 = fig.add_subplot(3, 1, 2)
    ax1 = fig.add_subplot(3, 1, 3)

    gu_cut = gu
    gv_cut = gv
    tfreq = sats_tfreq[sat1]

    oscar_u = oscar_u.resample(time=f"{tfreq}D").mean()
    ru_at = ru_at.resample(time=f"{tfreq}D").mean()
    oscar_u = oscar_u.interp(time=gu_cut.time)
    ru_at = ru_at.interp(time=gu_cut.time)
    oscar_max = oscar_u.max().item()
    gu_cut = gu_cut.where(gu_cut < oscar_max, np.nan)
    gu_cut = gu_cut.where(gu_cut > -oscar_max, np.nan)

    oscar_v = oscar_v.resample(time=f"{tfreq}D").mean()
    rv_at = rv_at.resample(time=f"{tfreq}D").mean()
    oscar_v = oscar_v.interp(time=gv_cut.time)
    rv_at = rv_at.interp(time=gu_cut.time)
    oscar_max = oscar_v.max().item()
    gv_cut = gv_cut.where(gv_cut < oscar_max, np.nan)
    gv_cut = gv_cut.where(gv_cut > -oscar_max, np.nan)

    oscar_u.plot(ax=ax0, color="b", linewidth=2, label="oscar u")
    gu_cut.plot(ax=ax0, label=f"track zonal", color="k", linewidth=2)
    ru_at.plot(ax=ax0, label=f"radar zonal", color="r", linewidth=2)
    # ax0.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    ax0.legend()
    ax0.set_ylabel("oscar u m/s")

    oscar_v.plot(ax=ax1, color="b", linewidth=2, label="oscar v")
```

```
    gv_cut.plot(ax=ax1, label=f"track meridional", color="k", linewidth=2)
    rv_at.plot(ax=ax1, label=f"radar meridional", color="r", linewidth=2)

    # corr_u = xs.pearson_r(gu_cut, oscar_u, dim="time", skipna=True)
    # rmse_u = xs.rmse(gu_cut, oscar_u, dim="time", skipna=True)
    # bias_var_u = oscar_u - gu_cut
    corr_u = xs.pearson_r(gu_cut, ru_at, dim="time", skipna=True)
    rmse_u = xs.rmse(gu_cut, ru_at, dim="time", skipna=True)
    bias_var_u = ru_at - gu_cut
    bias_u = bias_var_u.mean(dim="time")

    # corr_v = xs.pearson_r(gv_cut, oscar_v, dim="time", skipna=True)
    # rmse_v = xs.rmse(gv_cut, oscar_v, dim="time", skipna=True)
    # bias_var_v = oscar_v - gv_cut
    corr_v = xs.pearson_r(gv_cut, rv_at, dim="time", skipna=True)
    rmse_v = xs.rmse(gv_cut, rv_at, dim="time", skipna=True)
    bias_var_v = rv_at - gv_cut
    bias_v = bias_var_v.mean(dim="time")

    corrs_u.append(corr_u.item())
    rmses_u.append(rmse_u.item())
    biass_u.append(bias_u.item())

    corrs_v.append(corr_v.item())
    rmses_v.append(rmse_v.item())
    biass_v.append(bias_v.item())

    print(f"Correlation between gu and u_radar: {corr_u.item()}")
    print(f"Correlation between gv and v_radar: {corr_v.item()}")
    info = f"intersection point ({lon1:.2f}, {lat1:.2f}) {sat1} {track_number_self} {track_number_other}"
    # ax1.set_xlim(datetime(2010, 1, 1), datetime(2020, 1, 1))
    plt.text(
        0.05,
        0.90,
        info,
        transform=ax0.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    ax1.legend()
    ax1.set_ylabel("oscar v m/s")
    ax2 = fig.add_subplot(3, 1, 1, projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*TRACKS_REG[:2])).sel(ETOPO05_Y=slice(
        *TRACKS_REG[2:])).plot(ax=ax2,
                               add_colorbar=False,
                               add_labels=False,
                               cmap=cmap2)
    decorate_axis(ax2, "", *TRACKS_REG, step=5)
    ax2.grid()
    ax2.add_geometries(
        [polygon],
        ccrs.PlateCarree(),
        facecolor="blue",
        edgecolor="red",
        alpha=0.2,
    )
    ax2.plot(
        lon1,
        lat1,
        c="r",
        marker="o",
        markersize=4,
        markerfacecolor="red",
    )
    ax2.scatter(
        lons_track_self,
        lats_track_self,
        marker=".",
        color="c",
        s=4,
    )
    ax2.scatter(
        lons_track_other,
        lats_track_other,
        marker=".",
        color="c",
        s=4,
    )
    # plot.text radar_str
    plt.text(
        0.15,
        0.90,
        f"radar region: {radar_str}",
        transform=ax2.transAxes,
        fontsize=14,
        fontweight="bold",
    )
    lonm, latm = get_point_at_distance(lon_equat_self, lat_equat_self,
                                       lon_coast_self, lat_coast_self, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_self,
            fontsize=14,
            rotation=angle_self,
            color="k",
        )
    lonm, latm = get_point_at_distance(lon_equat_other, lat_equat_other,
                                       lon_coast_other, lat_coast_other, d)
    if is_within_region(lonm, latm, *TRACKS_REG):
        plt.text(
            lonm,
            latm,
            s=track_number_other,
            fontsize=14,
            rotation=angle_other,
            color="k",
        )
    plt.savefig(
        f"track_guvVSoscar_at_intersection_points/guv_timeseries_{sat1}_{track_number_self}_{track_number_other}_{radar_str}.png",
    )
    plt.show()
    plt.close("all")

# add corrs, rmses, biass to dataframe df_out
# df_out["corrs"] = corrs_u
```

```
# df_out["rmses"] = rmses_u
# df_out["biass"] = biass_u

# df_out.to_csv(f"tracks_corrs_rmses_biass_{sat1}_u.csv", index=False)

# df_out["corrs"] = corrs_v
# df_out["rmses"] = rmses_v
# df_out["biass"] = biass_v

# df_out.to_csv(f"tracks_corrs_rmses_biass_{sat1}_v.csv", index=False)
dse.close()

```

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import sys

import numpy as np
import pandas as pd


def clean_outliers(gu, ve, max_speed=3.0, mad_thresh=3.5):
    """
    Remove outliers from paired arrays (gu, ve).
    Parameters
    ----------
    gu, ve : array-like
        Arrays of satellite geostrophic u and drifter ve.
    max_speed : float
        Max absolute speed (m/s) allowed before removing.
    mad_thresh : float
        Threshold (in MAD units) for robust outlier removal.
    Returns
    -------
    gu_clean, ve_clean : ndarray
        Filtered arrays with outliers removed.
    mask : ndarray (bool)
        Boolean mask of valid points.
    """
    gu = np.asarray(gu)
    ve = np.asarray(ve)
    # Step 1: remove NaNs and infinities
    mask = np.isfinite(gu) & np.isfinite(ve)
    # Step 2: physical limit
    mask &= (np.abs(gu) <= max_speed) & (np.abs(ve) <= max_speed)
    # Step 3: robust MAD filter (applied separately to gu and ve)
    def mad_based_outlier(points, thresh=mad_thresh):
        median = np.median(points)
        mad = np.median(np.abs(points - median))
        if mad == 0:
            return np.zeros_like(points, dtype=bool)
        z = 0.6745 * (points - median) / mad
        return np.abs(z) > thresh

    mask &= ~mad_based_outlier(gu, mad_thresh)
    mask &= ~mad_based_outlier(ve, mad_thresh)

    return gu[mask], ve[mask], mask

sat_here = sys.argv[1]
df1 = pd.read_csv(f"track_guvVSnearby_drifters_at_intersection_points_{sat_here}_2.csv")

gu_clean, ve_clean, mask = clean_outliers(df1["gu"], df1["ve"])

df = pd.DataFrame({
    "gu_mask": gu_clean,
    "ve_mask": ve_clean
})
# Create a scatter plot
plt.figure(figsize=(8, 6))
plt.scatter(df["gu_mask"], df["ve_mask"], alpha=0.5)
plt.title("Scatter Plot of gu vs ve")
plt.xlabel("track gu")
plt.ylabel("drifter ve")
plt.grid(True)

# Calculate the correlation coefficient
correlation = df["gu_mask"].corr(df["ve_mask"])

# Place the correlation coefficient value on the plot
plt.text(0.05, 0.95, f'Correlation: {correlation:.2f}', transform=plt.gca().transAxes, fontsize=12,
         verticalalignment='top', bbox=dict(boxstyle='round,pad=0.5', fc='yellow', alpha=0.5))

# Calculate the trend line (line of best fit)
m, b = np.polyfit(df['gu_mask'], df['ve_mask'], 1)
x_sorted = np.sort(df['gu_mask'])

# import statsmodels.api as sm
# X = sm.add_constant(df['gu_mask'])
# model = sm.RLM(df['ve_mask'], X, M=sm.robust.norms.HuberT()).fit()
# b, m = model.params
# x_sorted = np.sort(df['gu_mask'])

# Add the trend line to the plot
plt.plot(x_sorted, m*x_sorted + b, color='red', linewidth=2, label='Trend Line')
plt.legend()
# Save the plot
plt.savefig(f'scatter_plot_{sat_here}_track_uVSnearby_drifter_u.png')
plt.show()

print(f"Correlation coefficient: {correlation}")

```

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import sys

import numpy as np
import pandas as pd


def clean_outliers(gu, ve, ek, max_speed=3.0, mad_thresh=3.5):
    """
    Remove outliers from paired arrays (gu, ve).
    Parameters
    ----------
    gu, ve : array-like
        Arrays of satellite geostrophic u and drifter ve.
    max_speed : float
        Max absolute speed (m/s) allowed before removing.
    mad_thresh : float
        Threshold (in MAD units) for robust outlier removal.
    Returns
    -------
    gu_clean, ve_clean : ndarray
        Filtered arrays with outliers removed.
    mask : ndarray (bool)
        Boolean mask of valid points.
    """
     gu = np.asarray(gu)
     ve = np.asarray(ve)
     ek = np.asarray(ek)
     # Step 1: remove NaNs and infinities
     mask = np.isfinite(gu) & np.isfinite(ve) & np.isfinite(ek)
     # Step 2: physical limit
     mask &= (np.abs(gu) <= max_speed) & (np.abs(ve) <= max_speed) & (np.abs(ek) <= max_speed)
     # Step 3: robust MAD filter (applied separately to gu and ve)
     def mad_based_outlier(points, thresh=mad_thresh):
         median = np.median(points)
         mad = np.median(np.abs(points - median))
         if mad == 0:
             return np.zeros_like(points, dtype=bool)
         z = 0.6745 * (points - median) / mad
         return np.abs(z) > thresh

     mask &= ~mad_based_outlier(gu, mad_thresh)
     mask &= ~mad_based_outlier(ve, mad_thresh)
     mask &= ~mad_based_outlier(ek, mad_thresh)

     return gu[mask], ve[mask], ek[mask], mask

sat_here = sys.argv[1]
df1 = pd.read_csv(f"track_guvVSnearby_drifters_at_intersection_points_{sat_here}_w_ekman.csv")

gu_clean, ve_clean, ek_clean, mask = clean_outliers(df1["gu"], df1["ve"], df1["ek"])

df = pd.DataFrame({
    "gu_mask": gu_clean,
    "ve_mask": ve_clean,
    "ek_mask": ek_clean,
    "gu_ek": gu_clean + ek_clean
})
# Create a scatter plot
plt.figure(figsize=(8, 6))
plt.scatter(df["gu_ek"], df["ve_mask"], alpha=0.5)
plt.title("Scatter Plot of gu+ek vs ve")
plt.xlabel("track gu+ek")
plt.ylabel("drifter ve")
plt.grid(True)

# Calculate the correlation coefficient
correlation = df["gu_ek"].corr(df["ve_mask"])

# Place the correlation coefficient value on the plot
plt.text(0.05, 0.95, f'Correlation: {correlation:.2f}', transform=plt.gca().transAxes, fontsize=12,
         verticalalignment='top', bbox=dict(boxstyle='round,pad=0.5', fc='yellow', alpha=0.5))

# Calculate the trend line (line of best fit)
m, b = np.polyfit(df['gu_mask'], df['ve_mask'], 1)
x_sorted = np.sort(df['gu_mask'])

# import statsmodels.api as sm
# X = sm.add_constant(df['gu_mask'])
# model = sm.RLM(df['ve_mask'], X, M=sm.robust.norms.HuberT()).fit()
# b, m = model.params
# x_sorted = np.sort(df['gu_mask'])

# Add the trend line to the plot
plt.plot(x_sorted, m*x_sorted + b, color='red', linewidth=2, label='Trend Line')
plt.legend()
# Save the plot
plt.savefig(f'scatter_plot_{sat_here}_track_uVSnearby_drifter_u_ek.png')
plt.show()

print(f"Correlation coefficient: {correlation}")

```

```python
import sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# ---------------------------------------------------------------------
# Outlier cleaning
# ---------------------------------------------------------------------
def clean_outliers(gu, ve, ek=None, max_speed=3.0, mad_thresh=3.5):
    gu = np.asarray(gu)
    ve = np.asarray(ve)

    mask = np.isfinite(gu) & np.isfinite(ve)

    if ek is not None:
        ek = np.asarray(ek)
        mask &= np.isfinite(ek)

    # Physical bounds
    mask &= (np.abs(gu) <= max_speed) & (np.abs(ve) <= max_speed)
    if ek is not None:
        mask &= (np.abs(ek) <= max_speed)

    def mad_outlier(x, thresh):
        med = np.median(x)
        mad = np.median(np.abs(x - med))
        if mad == 0:
            return np.zeros_like(x, dtype=bool)
        z = 0.6745 * (x - med) / mad
        return np.abs(z) > thresh

    mask &= ~mad_outlier(gu, mad_thresh)
    mask &= ~mad_outlier(ve, mad_thresh)
    if ek is not None:
        mask &= ~mad_outlier(ek, mad_thresh)

    if ek is None:
        return gu[mask], ve[mask]
    else:
        return gu[mask], ve[mask], ek[mask]


# ---------------------------------------------------------------------
# Plot helper
# ---------------------------------------------------------------------
def scatter_with_fit(ax, x, y, xlabel, title):
    ax.scatter(x, y, alpha=0.5)
    ax.grid(True)

    # Correlation
    corr = np.corrcoef(x, y)[0, 1]

    ax.text(
        0.05, 0.95, f"Correlation: {corr:.2f}",
        transform=ax.transAxes,
        va="top",
        bbox=dict(boxstyle="round,pad=0.4", fc="yellow", alpha=0.5),
    )

    # Trend line
    m, b = np.polyfit(x, y, 1)
    xs = np.sort(x)
    ax.plot(xs, m * xs + b, color="red", lw=2, label="Trend line")

    ax.set_xlabel(xlabel)
    ax.set_title(title)
    ax.legend()


# ---------------------------------------------------------------------
# Main
# ---------------------------------------------------------------------
sat = sys.argv[1]

# Read data
df_no_ek = pd.read_csv(
    f"track_guvVSnearby_drifters_at_intersection_points_{sat}_2.csv"
)

df_ek = pd.read_csv(
    f"track_guvVSnearby_drifters_at_intersection_points_{sat}_w_ekman.csv"
)

# Clean data
gu1, ve1 = clean_outliers(df_no_ek["gu"], df_no_ek["ve"])
gu2, ve2, ek2 = clean_outliers(df_ek["gu"], df_ek["ve"], df_ek["ek"])
gu_ek = gu2 + ek2

# ---------------------------------------------------------------------
# Figure: side-by-side, shared y-axis
# ---------------------------------------------------------------------
fig, axes = plt.subplots(
    ncols=2,
    nrows=1,
    figsize=(14, 6),
    sharey=True
)

# Left: gu vs ve
scatter_with_fit(
    axes[0],
    gu1,
    ve1,
    xlabel="Track gu (m/s)",
    title="gu vs drifter ve"
)
axes[0].set_ylabel("Drifter ve (m/s)")

# Right: gu+ek vs ve
scatter_with_fit(
    axes[1],
    gu_ek,
    ve2,
    xlabel="Track (gu + ek) (m/s)",
    title="gu + ek vs drifter ve"
)
```

```
118
119 plt.tight_layout()
120 plt.savefig(f"scatter_{sat}_gu_vs_ve_and_guEk_vs_ve.png", dpi=300)
121 plt.show()
122
```

```python
import math
import pickle
from datetime import datetime

import cmocean as cmo
import matplotlib.gridspec as gridspec
import matplotlib.pyplot as plt
import numpy as np
import shapely
import shapely as shp
import xarray as xr
from tools_xtrackm import *


# function to get percentage valid point in an xarray dataarray
def perc_valid_points(da):
    return 100 * (da.count().item() / len(da.time))


xr.set_options(display_expand_data=False)
gs = gridspec.GridSpec(1,
                       2,
                       width_ratios=[3, 1],
                       height_ratios=[2],
                       wspace=0.1,
                       hspace=0.05)
PSMSL_ID = 414  # id for visakhaptnam
width, height = 13, 6
data_dir = f"/home/srinivasu/xtrackm/data/"

cmap1 = cmo.cm.topo
cmap1 = cmo.cm.diff
cmap1 = cmo.cm.balance
cmap2 = cmo.cm.thermal

# tide gauge data
df = read_tide_meta()
df_nio = region_selected(df, *NIO)
lon_psmsl, lat_psmsl, name_psmsl = get_lonlat_psmsl(PSMSL_ID, df_nio)
print(lon_psmsl, lat_psmsl)
name_psmsl = name_psmsl.strip()

delx2 = 5
lonlat_box2 = (
    lon_psmsl - delx2,
    lon_psmsl + delx2,
    lat_psmsl - delx2,
    lat_psmsl + delx2,
)
poly_box = shp.geometry.box(lonlat_box2[0], lonlat_box2[2], lonlat_box2[1],
                            lonlat_box2[3])
# get the perimeter of the box
ds_id = read_id(PSMSL_ID)
sla_tide = ds_id["height"]
sla_tide = 0.001 * sla_tide
tide_tsta_o = ds_id.time[0].values
tide_tend_o = ds_id.time[-1].values
tide_tsta_o = n64todatetime(tide_tsta_o)
tide_tend_o = n64todatetime(tide_tend_o)

tlim1 = datetime.strptime("2013-01-01", "%Y-%m-%d")
tlim2 = datetime.strptime("2021-01-01", "%Y-%m-%d")


def point_on_line_at_distance(m, c, x0, y0, d):
    """
    Find a point on the line y = mx + c at a distance d from (x0, y0).
    Parameters:
        m (float): Slope of the line.
        c (float): y-intercept of the line.
        x0 (float): x-coordinate of the starting point.
        y0 (float): y-coordinate of the starting point.
        d (float): Distance from the starting point.
    Returns:
        tuple: Coordinates (x, y) of the new point.
    """
    # Normalize the direction vector of the line
    dx = 1 / math.sqrt(1 + m**2)  # x-component of the unit direction vector
    dy = m / math.sqrt(1 + m**2)  # y-component of the unit direction vector
    # Compute the new point
    if m < 0:
        d = -d
    x_new = x0 + d * dx
    y_new = y0 + d * dy
    return x_new, y_new


ds_dt1 = xr.open_dataset(
    "/home/srinivasu/allData/aviso/allyear_dt_global_allsat_msla_h.nc",
    decode_times=False,
)
# ds_dt1 = xr.open_dataset(
#     "/media/srinivasu/allData/sealevel/data/allyear_dt_global_allsat_msla_h.nc",
#     decode_times=False,
# )
print(ds_dt1)
ds_dt = change_time(ds_dt1, "time")
sla_aviso = ds_dt.sla  # .sel(time=slice(tsta_o, tend_o))
sla_aviso_at = sla_aviso.interp(longitude=lon_psmsl,
                                method="nearest").interp(latitude=lat_psmsl,
                                                         method="nearest")

sla_tide_interp = sla_tide.interp(time=sla_aviso.time)
sla_tide_interp = sla_tide_interp - sla_tide_interp.mean(dim="time")
# sla_tide_adj = sla_tide_interp  - dac_p_smooth

chosen_index = 0
dse = xr.open_dataset("/home/srinivasu/allData/topo/etopo5.cdf")

with open("closest_tracks_all_psmsl_new.pkl", "rb") as file:
    sorted_list = pickle.load(file)


def filter_list(sorted_list, sat=None, id=None):
    result = sorted_list
    if sat is not None:
        result = [tup for tup in result if tup[0] == sat]
```

```
    if id is not None:
        result = [tup for tup in result if tup[5] == id]
    return result


filtered_list = sorted_list[:]
filtered_list = filter_list(sorted_list, id=PSMSL_ID)
print(filtered_list)


# sys.exit(0)
for sat, trackn, x1, y1, mindist, id in filtered_list:
    if mindist > 50:
        continue
    track_tsta_o, track_tend_o = get_time_limits_o(sat)
    overlap_tsta, overlap_tend = overlap_dates(track_tsta_o, track_tend_o,
                                               tide_tsta_o, tide_tend_o)
    tsta_o, tend_o = get_time_limits_o(sat)
    print(sat, trackn, x1, y1, mindist)
    if trackn is None:
        continue
    # zero pad trackn with three digits
    trackn1 = trackn.zfill(3)
    f = f"{data_dir}/{sat}/ctoh.sla.ref.{sat}.nindian.{trackn1}.nc"
    ds = xr.open_dataset(f, decode_times=False)
    lons_track = ds.lon.values
    lats_track = ds.lat.values
    lon_coast = lons_track[-1]  # this on coast
    lat_coast = lats_track[-1]  # this on coast
    m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
    c = (lats_track[0]) - m * (lons_track[0])
    x_from_coast = distance.distance((lat_coast, lon_coast), (y1, x1)).km
    # shapely path of the track
    track_path = shapely.geometry.LineString(list(zip(lons_track, lats_track)))
    if len(lons_track) == 0:
        continue
    m = (lats_track[-1] - lats_track[0]) / (lons_track[-1] - lons_track[0])
    angle = np.rad2deg(np.arctan(m))
    c = (lats_track[0]) - m * (lons_track[0])
    ds_smooth = xr.open_dataset(
        f"/home/srinivasu/xtrackm/computed/sla_loess_0.2a/track_sla_along_{sat}_{trackn}_loess_0.2.nc"
    )
    sla_track_smooth = ds_smooth.sla_smooth
    dac_track_smooth = ds_smooth.dac_smooth
    sla_track_p_smooth = sla_track_smooth.sel(x=x_from_coast,
                                              method="nearest",
                                              drop=True)
    dac_track_p_smooth = dac_track_smooth.sel(x=x_from_coast,
                                              method="nearest",
                                              drop=True)
    sla_plus = sla_track_p_smooth + dac_track_p_smooth
    fig = plt.figure(figsize=(width, height))
    ax1 = fig.add_subplot(gs[0, 0])
    # sla_aviso_at.plot(ax=ax1,
    #                   label="extracted aviso grid data",
    #                   linestyle="--")
    sla_tide_interp.plot(ax=ax1, label="tide gauge data", linewidth=4)
    # sla_track_p_smooth.plot(ax=ax1, label=f"track {trackn} {sat}")
    sla_plus.plot(ax=ax1, label=f"track {trackn} {sat} + dac")
    # ax1.set_xlim(tlim1, tlim2)
    ax1.set_xlim(overlap_tsta, overlap_tend)
    ax1.set_ylim(-0.6, 0.6)
    info1 = f"{sat} {trackn} {mindist:.2f}km"
    ax1.text(
        0.5,
        0.9,
        info1,
        horizontalalignment="center",
        verticalalignment="center",
        transform=ax1.transAxes,
        fontweight="bold",
        fontsize=14,
    )
    ax1.legend(loc="lower right")

    ax2 = fig.add_subplot(gs[0, 1], projection=ccrs.PlateCarree())
    dse.ROSE.sel(ETOPO05_X=slice(*lonlat_box2[:2])).sel(ETOPO05_Y=slice(
        *lonlat_box2[2:])).plot(ax=ax2,
                                add_colorbar=False,
                                add_labels=False,
                                cmap=cmap1)
    decorate_axis(ax2, "", *lonlat_box2, step=2)
    # get intersection of track and box
    intersection = track_path.intersection(poly_box)
    # get the intersection points
    intersection_points = intersection.coords
    # sort intesections points by latitude
    intersection_points = sorted(intersection_points, key=lambda x: x[1])
    # take the first intersection points
    if len(intersection_points) == 0:
        continue
    lonl = intersection_points[0][0]
    latl = intersection_points[0][1]
    print(lonl, latl)
    info = f"{sat} {trackn}"
    lonm, latm = point_on_line_at_distance(m, c, lonl, latl, 2)
    print(lonm, latm)
    ax2.scatter(lons_track, lats_track, linewidths=0.0, s=8, color="r")
    ax2.text(
        lonm,
        latm,
        s=f"{trackn}",
        fontsize=10,
        rotation=angle,
        color="white",
        horizontalalignment="center",
        verticalalignment="center",
        fontweight="bold",
    )
    ax2.add_geometries(
        [poly_box],
        ccrs.PlateCarree(),
        facecolor="none",
        edgecolor="lightblue",
        alpha=0.5,
    )
    # PSMSL tide gauge location
```

```
    ax2.scatter(
        lon_psmsl,
        lat_psmsl,
        marker="*",
        c="yellow",
        s=150,
        edgecolor="black",
    )
    ax2.scatter(
        x1,
        y1,
        marker="x",
        c="cyan",
        s=10,
    )
    # PSMSL tide gauge name
    ax2.text(
        lon_psmsl,
        lat_psmsl,
        s=name_psmsl,
        fontsize=10,
        color="k",
        horizontalalignment="left",
    )
    ax2.set_xlim([*lonlat_box2[:2]])
    ax2.set_ylim([*lonlat_box2[2:]])
    # sla_tide_adj.plot(ax=ax1, label="tide gauge data adjusted")
    ax2.legend(loc="lower right")
    info = f"{sat}"
    ax2.text(0.1, 0.95, info, transform=ax2.transAxes, fontweight="bold")
    plt.savefig(
        f"tseries_loess_adj_allsat_map_box_separate_{sat}_{trackn}.png",
        bbox_inches="tight",
    )
    plt.show()
    plt.close("all")

```

selected/prompts.txt

```
1
2
```

selected/satellite_mission_time_spans_new.py

```python
from datetime import datetime

import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import pandas as pd
from dateutil.relativedelta import relativedelta

missions = pd.read_csv("data_coverage_df.csv")
print(missions)

colors = ["red", "green", "blue", "purple"]
date_format = "%Y-%m-%d"

min_start_date = min(pd.to_datetime(missions["first_date"]))
max_end_date = max(pd.to_datetime(missions["last_date"]))
min_start_date1 = min_start_date - relativedelta(years=1)
max_end_date1 = max_end_date + relativedelta(years=1)
# print(min_start_date1, max_end_date1)

fig, ax = plt.subplots(figsize=(10, 6), layout="constrained")

for index, row in missions.iterrows():
    sat, track_nos, start_date, end_date, tfreq, xfreq = row.loc[
        ["sat", "trackn", "first_date", "last_date", "tfreq", "xfreq"]
    ]
    sln = len(sat)
    words = [sat, str(track_nos), str(tfreq), str(round(xfreq, 1))]
    start_date_o = datetime.strptime(start_date, date_format)
    end_date_o = datetime.strptime(end_date, date_format)
    duration = end_date_o - start_date_o
    ax.barh(
        index,
        duration,
        left=start_date_o,
        color="skyblue",
        edgecolor="black",
    )
    # current_start_time = start_date_o + relativedelta(years=4)
    current_start_time = start_date_o + duration / 4
    for k, word in enumerate(words):
        sln = len(word)
        text_obj = ax.text(
            current_start_time,
            index,
            word,
            ha="center",
            va="center",
            fontweight="bold",
            fontsize=12,
            color=colors[k],
        )
        if k == 0:
            yrs = int(0.21 * sln) + 2
            mns = int(2.53 * sln) + 10
        else:
            yrs = int(0.21 * sln) + 1.5
            mns = int(2.53 * sln) + 8
        # current_start_time = current_start_time + relativedelta(years=yrs)
        current_start_time = current_start_time + relativedelta(months=mns)

ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
ax.xaxis.set_major_locator(mdates.YearLocator(2))
# ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m"))
# ax.xaxis.set_major_locator(mdates.YearLocator(1))
ax.set_yticks([])
ax.set_yticklabels([])
ax.set_xlabel("years")
ax.set_xlim(min_start_date1, max_end_date1)
ax.set_title("Satellite Missions Timeline")
ax.invert_yaxis()

satellite = mpatches.Patch(color="red", label="satellite")
tracks = mpatches.Patch(color="green", label="no. of tracks")
tfreq = mpatches.Patch(color="blue", label="temporal frequency")
xfreq = mpatches.Patch(color="purple", label="spatial resolution")
plt.legend(handles=[satellite, tracks, tfreq, xfreq])

plt.savefig(f"satellite_mission_time_spans_new_fig01.png")
# plt.tight_layout()
plt.show()

```